Amazon

(AWS Certified Solutions Architect - Associate SAA-C03)

AWS Certified Solutions Architect - Associate SAA-C03

Total: **1019 Questions**

Link: https://examheist.com/papers/amazon/aws-certified-solutions-architect-associate-saa-c03

Question: 1 **Exam Heist**

A company collects data for temperature, humidity, and atmospheric pressure in cities across multiple continents. The average volume of data that the company collects from each site daily is 500 GB. Each site has a high-speed Internet connection.
The company wants to aggregate the data from all these global sites as quickly as possible in a single Amazon S3 bucket. The solution must minimize operational complexity.
Which solution meets these requirements?

A. Turn on S3 Transfer Acceleration on the destination S3 bucket. Use multipart uploads to directly upload site data to the destination S3 bucket.

B. Upload the data from each site to an S3 bucket in the closest Region. Use S3 Cross-Region Replication to copy objects to the destination S3 bucket. Then remove the data from the origin S3 bucket.

C. Schedule AWS Snowball Edge Storage Optimized device jobs daily to transfer data from each site to the closest Region. Use S3 Cross-Region Replication to copy objects to the destination S3 bucket.

D. Upload the data from each site to an Amazon EC2 instance in the closest Region. Store the data in an Amazon Elastic Block Store (Amazon EBS) volume. At regular intervals, take an EBS snapshot and copy it to the Region that contains the destination S3 bucket. Restore the EBS volume in that Region.

Answer: A

Explanation:

The correct answer is A because it directly addresses the requirements of speed, minimal operational complexity, and leveraging existing high-speed internet connections. S3 Transfer Acceleration utilizes AWS's globally distributed edge locations to optimize data transfer speeds into an S3 bucket. Multipart uploads enhance reliability and speed, especially for large files (500 GB daily). This method avoids the complexity of managing intermediate buckets, EC2 instances, or physical devices like Snowball Edge.

Option B introduces unnecessary complexity with multiple S3 buckets and cross-region replication, increasing management overhead and costs. While replication handles data transfer, Transfer Acceleration is designed specifically for speed optimization in direct uploads.

Option C is unsuitable because AWS Snowball Edge is intended for environments with limited or no internet connectivity. Given the high-speed internet connection available at each site, Snowball Edge adds unnecessary logistical complexity and delays.

Option D involves managing EC2 instances, EBS volumes, and snapshots, significantly increasing operational overhead. Transferring EBS snapshots is also not an optimized method for data aggregation into S3 compared to direct S3 uploads, especially regarding speed and cost. The described method is more appropriate for disaster recovery of complete systems rather than daily data aggregation.

In summary, S3 Transfer Acceleration with multipart uploads is the most efficient and straightforward solution for quickly aggregating data from global sites into a single S3 bucket, aligning with the requirements for speed, minimal operational overhead, and leveraging existing high-speed internet.

Relevant links:

Amazon S3 Transfer Acceleration: https://aws.amazon.com/s3/transfer-acceleration/
Amazon S3 Multipart Upload Overview:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/mpuoverview.html

Question: 2 **Exam Heist**

A company needs the ability to analyze the log files of its proprietary application. The logs are stored in JSON format in an Amazon S3 bucket. Queries will be simple and will run on-demand. A solutions architect needs to perform the analysis with minimal changes to the existing architecture.

What should the solutions architect do to meet these requirements with the LEAST amount of operational overhead?

A. Use Amazon Redshift to load all the content into one place and run the SQL queries as needed.

B. Use Amazon CloudWatch Logs to store the logs. Run SQL queries as needed from the Amazon CloudWatch console.

C. Use Amazon Athena directly with Amazon S3 to run the queries as needed.

D. Use AWS Glue to catalog the logs. Use a transient Apache Spark cluster on Amazon EMR to run the SQL queries as needed.

Answer: C

Explanation:

The best solution is **C. Use Amazon Athena directly with Amazon S3 to run the queries as needed.**

Here's why:

Minimal Operational Overhead: Athena is serverless. It eliminates the need to provision or manage infrastructure. It also allows you to query data directly in S3, which is a major advantage as the logs are already stored there.
Cost-Effectiveness: Athena charges based on the amount of data scanned per query. Since the queries are on-demand and presumably infrequent, this pay-per-query model is the most cost-effective option.
Simplicity: Athena allows direct querying of JSON data stored in S3 using standard SQL. The log format is already compatible.

Let's examine why the other options are less ideal:

A. Amazon Redshift: Redshift requires setting up and managing a data warehouse cluster. This involves provisioning resources, handling scaling, and performing ETL (Extract, Transform, Load) to move the JSON data from S3 into Redshift. This adds significant operational overhead and cost compared to Athena.

B. Amazon CloudWatch Logs: CloudWatch Logs are best suited for real-time monitoring and centralized logging, not for complex analytical queries. While CloudWatch Logs Insights exists, its query language isn't SQL, and it's not designed for ad-hoc analysis of JSON files stored elsewhere (like S3). Migrating the logs would also be required.

D. AWS Glue and Amazon EMR: This solution is an overkill. AWS Glue is for ETL and data cataloging, and EMR is for big data processing using frameworks like Spark. While suitable for very large and complex data analysis scenarios, it introduces unnecessary complexity and operational overhead for the stated requirements of simple, on-demand queries. It also has high cost compared to using AWS Athena.

Therefore, Athena aligns best with the requirement of minimal operational overhead and allows simple SQL queries on JSON logs stored in S3, making it the superior choice.

Authoritative Links:

Amazon Athena: https://aws.amazon.com/athena/
Amazon S3: https://aws.amazon.com/s3/

Question: 3 **Exam Heist**

A company uses AWS Organizations to manage multiple AWS accounts for different departments. The management account has an Amazon S3 bucket that contains project reports. The company wants to limit access to this S3 bucket to only users of accounts within the organization in AWS Organizations.
Which solution meets these requirements with the LEAST amount of operational overhead?

A. Add the aws PrincipalOrgID global condition key with a reference to the organization ID to the S3 bucket policy.

B. Create an organizational unit (OU) for each department. Add the aws:PrincipalOrgPaths global condition key to the S3 bucket policy.

C. Use AWS CloudTrail to monitor the CreateAccount, InviteAccountToOrganization, LeaveOrganization, and RemoveAccountFromOrganization events. Update the S3 bucket policy accordingly.

D. Tag each user that needs access to the S3 bucket. Add the aws:PrincipalTag global condition key to the S3 bucket policy.

Answer: A

Explanation:

The correct answer is A. Here's why:

Option A leverages the aws:PrincipalOrgID global condition key in the S3 bucket policy. This condition key directly references the AWS Organizations organization ID. When applied, the S3 bucket will only allow access from AWS accounts that belong to the specified organization. This approach offers the least operational overhead because it's a simple, direct configuration within the S3 bucket policy, automatically encompassing all current and future accounts within the organization without requiring ongoing updates.

Option B, using aws:PrincipalOrgPaths, involves creating organizational units (OUs) for each department. While it provides more granular control based on OU membership, it necessitates managing and updating the S3 bucket policy whenever OUs change or accounts are moved, increasing operational overhead. It's unnecessary complexity for the stated requirement of granting access to all accounts within the entire organization.

Option C, using CloudTrail to monitor organizational changes and then updating the S3 bucket policy, is unnecessarily complex and creates significant operational overhead. It requires implementing a custom solution to react to CloudTrail events and programmatically modify the S3 bucket policy, which is prone to errors and maintenance issues. It is also not a real-time mechanism; there would be a delay between the organizational event and the policy update.

Option D, tagging users and using aws:PrincipalTag, is suitable for controlling access based on individual user attributes, not organizational membership. Applying and managing tags for each user across multiple accounts within the organization adds significant operational overhead and is not the appropriate tool for this specific requirement. Furthermore, tagging users across multiple accounts and keeping those tags consistent introduces administrative challenges.

Therefore, option A provides the most efficient and least complex solution for limiting S3 bucket access to only users of accounts within the organization, by directly referencing the organization ID in the S3 bucket policy, minimizing the manual intervention and operational overhead.

Relevant links for further research:

AWS Organizations Condition Keys
Controlling Access to S3 Buckets
AWS Organizations Overview

Question: 4 **Exam Heist**

An application runs on an Amazon EC2 instance in a VPC. The application processes logs that are stored in an Amazon S3 bucket. The EC2 instance needs to access the S3 bucket without connectivity to the internet.
Which solution will provide private network connectivity to Amazon S3?

A. Create a gateway VPC endpoint to the S3 bucket.

B. Stream the logs to Amazon CloudWatch Logs. Export the logs to the S3 bucket.

C. Create an instance profile on Amazon EC2 to allow S3 access.

D. Create an Amazon API Gateway API with a private link to access the S3 endpoint.

Answer: A

Explanation:

The correct answer is **A. Create a gateway VPC endpoint to the S3 bucket.**

Here's why:

Gateway VPC Endpoints for S3: Gateway VPC endpoints provide private connectivity between your VPC and S3 without requiring an internet gateway, NAT device, VPN connection, or AWS Direct Connect connection. This ensures that traffic between your EC2 instance and S3 remains within the AWS network. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-s3.html

How it works: When you create a gateway VPC endpoint for S3, a route is automatically added to your VPC's route table. This route directs traffic destined for S3 to the endpoint instead of the internet. The EC2 instance, using its IAM role permissions to access S3, can now reach the S3 bucket privately.

Let's analyze the other options:

B. Stream the logs to Amazon CloudWatch Logs. Export the logs to the S3 bucket: While CloudWatch Logs is useful for centralized logging, exporting logs from CloudWatch Logs to S3 still requires connectivity to S3. It doesn't inherently solve the private connectivity requirement. Furthermore, it adds unnecessary complexity and cost.

C. Create an instance profile on Amazon EC2 to allow S3 access: An instance profile (IAM role) grants permissions to the EC2 instance to access S3, but it doesn't establish private network connectivity. The EC2 instance still needs a way to reach S3 over the network, and without a gateway endpoint, it would need internet access.

D. Create an Amazon API Gateway API with a private link to access the S3 endpoint: This involves creating an API Gateway and configuring it to use a VPC endpoint service (PrivateLink). While PrivateLink can provide private connectivity, it is typically used for exposing services running within a VPC to other VPCs or on-premises networks, not for a simple EC2 to S3 communication within the same VPC. It is overkill for this scenario and adds significant complexity. Additionally, API Gateway adds latency and cost where the simple gateway endpoint of option A suffices.

Question: 5 **Exam Heist**

A company is hosting a web application on AWS using a single Amazon EC2 instance that stores user-uploaded documents in an Amazon EBS volume. For better scalability and availability, the company duplicated the architecture and created a second EC2 instance and EBS volume in another Availability Zone, placing both behind an Application Load Balancer. After completing this change, users reported that, each time they refreshed the website, they could see one subset of their documents or the other, but never all of the documents at the same time.
What should a solutions architect propose to ensure users see all of their documents at once?

A. Copy the data so both EBS volumes contain all the documents

B. Configure the Application Load Balancer to direct a user to the server with the documents

C. Copy the data from both EBS volumes to Amazon EFS. Modify the application to save new documents to Amazon EFS

D. Configure the Application Load Balancer to send the request to both servers. Return each document from the correct server

Answer: C

Explanation:

The correct answer is C because it addresses the core issue of data consistency across multiple instances. The users are experiencing inconsistent document views because each EC2 instance has a separate EBS volume, and data isn't synchronized between them.

Option A, copying data between EBS volumes, is a short-term fix that becomes increasingly difficult to manage as the data grows and changes. It doesn't provide a scalable or reliable long-term solution. Consider the overhead involved in constantly syncing large EBS volumes.

Option B, using the Application Load Balancer to direct users to the server containing the documents, isn't feasible. The load balancer isn't aware of which server holds which documents. It would require complex session management based on document ownership, which is error-prone and inefficient.

Option D, sending requests to both servers simultaneously, won't resolve the issue. It might even exacerbate the problem by showing fragmented document sets more frequently. There would be significant overhead in merging results from multiple servers and the application would need to manage this complexity.

Option C, migrating to Amazon EFS, provides a centralized, shared file system accessible by both EC2 instances. EFS ensures data consistency and allows both instances to see the same set of documents. By copying the existing data to EFS and modifying the application to use EFS for storage, all users will access the same data, resolving the inconsistency. EFS is designed for this exact scenario - providing shared storage for multiple compute instances.

Amazon EFS is well-suited for web applications that require persistent, shared storage. It eliminates the need for data replication and synchronization across instances. It also offers scalability and performance to accommodate growing data needs.

Therefore, migrating to Amazon EFS is the most effective and scalable solution for ensuring users see all their documents simultaneously.

Further reading:

Amazon EFS: https://aws.amazon.com/efs/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/

Question: 6 **Exam Heist**

A company uses NFS to store large video files in on-premises network attached storage. Each video file ranges in size from 1 MB to 500 GB. The total storage is 70 TB and is no longer growing. The company decides to migrate the video files to Amazon S3. The company must migrate the video files as soon as possible while using the least possible network bandwidth.
Which solution will meet these requirements?

A. Create an S3 bucket. Create an IAM role that has permissions to write to the S3 bucket. Use the AWS CLI to copy all files locally to the S3 bucket.

B. Create an AWS Snowball Edge job. Receive a Snowball Edge device on premises. Use the Snowball Edge client to transfer data to the device. Return the device so that AWS can import the data into Amazon S3.

C. Deploy an S3 File Gateway on premises. Create a public service endpoint to connect to the S3 File Gateway. Create an S3 bucket. Create a new NFS file share on the S3 File Gateway. Point the new file share to the S3 bucket. Transfer the data from the existing NFS file share to the S3 File Gateway.

D. Set up an AWS Direct Connect connection between the on-premises network and AWS. Deploy an S3 File Gateway on premises. Create a public virtual interface (VIF) to connect to the S3 File Gateway. Create an S3 bucket. Create a new NFS file share on the S3 File Gateway. Point the new file share to the S3 bucket. Transfer the data from the existing NFS file share to the S3 File Gateway.

Answer: B

Explanation:

The correct solution is B: Using AWS Snowball Edge.

Here's a detailed justification:

The key requirements are migrating 70 TB of video files to S3 quickly while minimizing network bandwidth usage.

Option A (AWS CLI): Copying 70 TB over the internet using the AWS CLI would be extremely slow and consume a significant amount of network bandwidth. This contradicts the requirements.

Option B (AWS Snowball Edge): AWS Snowball Edge is designed for transferring large amounts of data offline. AWS ships a physical device (Snowball Edge) to the company. The company then copies the data onto the device locally. Once the transfer is complete, the device is shipped back to AWS, where the data is uploaded to S3. This avoids network bandwidth usage and enables a much faster transfer compared to online methods for this volume of data. The 70 TB size falls within the typical Snowball Edge capacity. This aligns perfectly with the requirements of minimizing bandwidth and ensuring a swift transfer.

Option C (S3 File Gateway with Public Service Endpoint): S3 File Gateway would cache the data before uploading it to S3. While it provides an NFS interface, it still requires transferring the data over the internet, which contradicts the requirement of minimizing network bandwidth. A public service endpoint still means data traversing the public internet. Furthermore, introducing File Gateway for a one-time migration adds unnecessary complexity.

Option D (S3 File Gateway with AWS Direct Connect): While Direct Connect provides a dedicated, faster connection to AWS, setting it up solely for a one-time migration is an overkill and adds significant cost and complexity. It doesn't eliminate the need to transfer 70 TB of data over a network.

Therefore, AWS Snowball Edge is the optimal solution because it directly addresses the constraints of large data volume and limited network bandwidth. It is significantly faster than transferring over the internet and avoids ongoing network usage.

Supporting concepts and links:

AWS Snowball Edge: AWS Snowball Edge is a data migration and edge computing device that comes in various configurations, including Storage Optimized. It allows customers to transfer large amounts of data into and out of AWS without relying on network connectivity.

https://aws.amazon.com/snowball/

Data Migration to AWS: AWS provides several services for data migration, each suitable for different scenarios. Snowball Edge is typically used for large-scale migrations when network bandwidth is limited.

https://aws.amazon.com/migration/

Final Answer: The final answer is $\boxed B $

Question: 7 **Exam Heist**

A company has an application that ingests incoming messages. Dozens of other applications and microservices then quickly consume these messages. The number of messages varies drastically and sometimes increases suddenly to 100,000 each second. The company wants to decouple the solution and increase scalability.
Which solution meets these requirements?

A. Persist the messages to Amazon Kinesis Data Analytics. Configure the consumer applications to read and process the messages.

B. Deploy the ingestion application on Amazon EC2 instances in an Auto Scaling group to scale the number of EC2 instances based on CPU metrics.

C. Write the messages to Amazon Kinesis Data Streams with a single shard. Use an AWS Lambda function to preprocess messages and store them in Amazon DynamoDB. Configure the consumer applications to read from DynamoDB to process the messages.

D. Publish the messages to an Amazon Simple Notification Service (Amazon SNS) topic with multiple Amazon Simple Queue Service (Amazon SOS) subscriptions. Configure the consumer applications to process the messages from the queues.

Answer: D

Explanation:

Here's a detailed justification for why option D is the best solution, along with supporting concepts and links:

The scenario demands a highly scalable and decoupled message ingestion and consumption system. Let's analyze why option D, using Amazon SNS and SQS, best fulfills these requirements.

Option D (SNS and SQS) utilizes the publish-subscribe pattern offered by SNS and the message queuing capabilities of SQS. The ingestion application publishes messages to an SNS topic. Multiple SQS queues subscribe to this topic. Each consumer application then pulls messages from its own dedicated SQS queue. This provides excellent decoupling because the ingestion application is unaware of the consumers, and the consumers are isolated from each other. This isolation is crucial for scalability; if one consumer fails or becomes overloaded, it doesn't impact the others or the ingestion process. SQS provides buffering for messages, preventing data loss during surges.

SNS is designed for high-throughput, enabling it to handle the peak load of 100,000 messages per second. SQS, being a fully managed queuing service, automatically scales to handle the message volume. The combination ensures messages are reliably delivered to all subscribed consumers without overwhelming them. SQS also facilitates asynchronous processing, allowing consumers to process messages at their own pace.

Now, let's look at why the other options are less suitable.

Option A (Kinesis Data Analytics) is designed for real-time data processing and analytics, not primarily for decoupling and distribution to multiple consumers. While it can process messages, it's not as efficient for simply fanning them out to many downstream applications.

Option B (EC2 Auto Scaling) addresses the scaling of the ingestion application, but not the decoupling or scaling of the message delivery to consumers. It doesn't solve the problem of distributing messages effectively to multiple, independent consumers.

Option C (Kinesis Data Streams and DynamoDB) introduces unnecessary complexity. Kinesis Data Streams can handle high throughput, but using a single shard would create a bottleneck. DynamoDB as an intermediary storage adds latency and complexity compared to the direct delivery from SQS. Furthermore, polling DynamoDB would not scale nearly as efficiently as SQS queues.

In summary, option D is the most appropriate choice because it directly addresses the requirements for decoupling, scalability, and reliable message delivery to multiple consumers using services specifically designed for these purposes.

Supporting Links:

Amazon SNS: https://aws.amazon.com/sns/
Amazon SQS: https://aws.amazon.com/sqs/
Decoupled Architecture on AWS: https://aws.amazon.com/solutions/guidance/decoupling-applications-aws/

Question: 8 **Exam Heist**

A company is migrating a distributed application to AWS. The application serves variable workloads. The legacy platform consists of a primary server that coordinates jobs across multiple compute nodes. The company wants to modernize the application with a solution that maximizes resiliency and scalability.
How should a solutions architect design the architecture to meet these requirements?

A. Configure an Amazon Simple Queue Service (Amazon SQS) queue as a destination for the jobs. Implement the compute nodes with Amazon EC2 instances that are managed in an Auto Scaling group. Configure EC2 Auto Scaling to use scheduled scaling.

B. Configure an Amazon Simple Queue Service (Amazon SQS) queue as a destination for the jobs. Implement the compute nodes with Amazon EC2 instances that are managed in an Auto Scaling group. Configure EC2 Auto Scaling based on the size of the queue.

C. Implement the primary server and the compute nodes with Amazon EC2 instances that are managed in an Auto Scaling group. Configure AWS CloudTrail as a destination for the jobs. Configure EC2 Auto Scaling based on the load on the primary server.

D. Implement the primary server and the compute nodes with Amazon EC2 instances that are managed in an Auto Scaling group. Configure Amazon EventBridge (Amazon CloudWatch Events) as a destination for the jobs. Configure EC2 Auto Scaling based on the load on the compute nodes.

Answer: B

Explanation:

The correct answer is B. Here's why:

The problem statement emphasizes resiliency, scalability, and variable workloads. Option B best addresses these requirements.

Amazon SQS: Using SQS decouples the primary server (job coordinator) from the compute nodes. This promotes resiliency. If the primary server fails, the messages in the queue persist, and processing can resume when a new primary server is available. https://aws.amazon.com/sqs/
EC2 Auto Scaling Group: Placing the compute nodes in an Auto Scaling group provides scalability and high availability. Auto Scaling automatically adjusts the number of EC2 instances based on demand. This ensures that the application can handle variable workloads efficiently. https://aws.amazon.com/autoscaling/
Scaling based on Queue Size: Configuring Auto Scaling to scale based on the size of the SQS queue is crucial. As the queue grows (more jobs are waiting), Auto Scaling will launch more EC2 instances to process the jobs. Conversely, if the queue shrinks, Auto Scaling can terminate instances to reduce costs. This reactive, queue-driven scaling aligns directly with the variable workload requirement.

Option A is incorrect because scheduled scaling doesn't react to real-time workload changes. It scales instances at predefined times regardless of the actual queue size. While useful for predictable workload patterns, it's less effective for variable workloads.

Option C is incorrect because CloudTrail is an auditing service, not a job queue. Furthermore, scaling based on the primary server's load doesn't directly reflect the number of waiting jobs. If the primary server is overloaded, that is a symptom of a problem, not the core driver for scaling compute nodes.

Option D is incorrect because EventBridge (formerly CloudWatch Events) is typically used for event-driven architectures, not as a direct queue for jobs in this scenario. Also, scaling based on the load of the compute nodes becomes a lagging metric. The queue length provides earlier indication that scaling is needed. Furthermore, managing the primary server within the auto-scaling group might be more complex than necessary and is less resilient compared to the option leveraging SQS.

Question: 9 **Exam Heist**

A company is running an SMB file server in its data center. The file server stores large files that are accessed frequently for the first few days after the files are created. After 7 days the files are rarely accessed.
The total data size is increasing and is close to the company's total storage capacity. A solutions architect must increase the company's available storage space without losing low-latency access to the most recently accessed files. The solutions architect must also provide file lifecycle management to avoid future storage issues.
Which solution will meet these requirements?

A. Use AWS DataSync to copy data that is older than 7 days from the SMB file server to AWS.

B. Create an Amazon S3 File Gateway to extend the company's storage space. Create an S3 Lifecycle policy to transition the data to S3 Glacier Deep Archive after 7 days.

C. Create an Amazon FSx for Windows File Server file system to extend the company's storage space.

D. Install a utility on each user's computer to access Amazon S3. Create an S3 Lifecycle policy to transition the data to S3 Glacier Flexible Retrieval after 7 days.

Answer: B

Explanation:

The correct solution is B, which involves using Amazon S3 File Gateway and S3 Lifecycle policies. Here's why:

S3 File Gateway: This service provides a seamless way to extend on-premises file storage to Amazon S3 without requiring significant application changes. It presents a local file system interface to the on-premises SMB server, which then transparently caches frequently accessed data locally while storing the full dataset in S3. This addresses the requirement for low-latency access to recent files.
https://aws.amazon.com/storagegateway/file-gateway/

S3 Lifecycle Policies: These policies automate the process of moving less frequently accessed data to lower-cost storage tiers such as S3 Glacier Deep Archive after a specified period (7 days in this case). This addresses the requirement for file lifecycle management and helps to reduce storage costs without deleting the data. S3 Glacier Deep Archive is suitable for long-term archival where retrieval times of several hours are acceptable. https://aws.amazon.com/s3/storage-classes/

Option A is incorrect because AWS DataSync is primarily a data migration tool, not a continuous storage extension solution. It would require ongoing manual or scripted execution to move files after 7 days, which is less efficient than a managed solution like S3 File Gateway with lifecycle policies.

Option C is not ideal because Amazon FSx for Windows File Server is a fully managed Windows file server in the cloud. While it provides scalable storage, it doesn't directly address the need to seamlessly integrate with the existing on-premises SMB server and leverage cheaper archival storage. It would be better suited as a complete replacement to the on-premise server.

Option D is incorrect because it involves manually installing a utility on each user's computer. It is complex to manage and prone to user error. Additionally, while S3 Lifecycle policies are used, S3 Glacier Flexible Retrieval is a more expensive storage option than Glacier Deep Archive for archival purposes. This is not an ideal solution.

In summary, option B provides the most cost-effective and manageable solution for extending on-premises file storage to the cloud, maintaining low-latency access to recent files, and implementing automated lifecycle management.

Question: 10 **Exam Heist**

A company is building an ecommerce web application on AWS. The application sends information about new

orders to an Amazon API Gateway REST API to process. The company wants to ensure that orders are processed in the order that they are received.
Which solution will meet these requirements?

A. Use an API Gateway integration to publish a message to an Amazon Simple Notification Service (Amazon SNS) topic when the application receives an order. Subscribe an AWS Lambda function to the topic to perform processing.

B. Use an API Gateway integration to send a message to an Amazon Simple Queue Service (Amazon SQS) FIFO queue when the application receives an order. Configure the SQS FIFO queue to invoke an AWS Lambda function for processing.

C. Use an API Gateway authorizer to block any requests while the application processes an order.

D. Use an API Gateway integration to send a message to an Amazon Simple Queue Service (Amazon SQS) standard queue when the application receives an order. Configure the SQS standard queue to invoke an AWS Lambda function for processing.

Answer: B

Explanation:

The correct answer is B because it directly addresses the requirement of processing orders in the order they are received. Here's why:

FIFO (First-In, First-Out) Queues: Amazon SQS FIFO queues guarantee that messages are retrieved from the queue in the exact order they were placed. This is crucial for order processing, where the sequence of events matters. https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues.html

API Gateway Integration: Integrating API Gateway with SQS allows you to seamlessly send order data received through the API directly into the FIFO queue.

Lambda Function Invocation: Configuring the SQS queue to invoke a Lambda function on message arrival ensures that each order is automatically processed as it becomes available in the queue.

Option A is incorrect because Amazon SNS is a publish/subscribe service, not a queuing service. It's designed for broadcasting messages to multiple subscribers simultaneously, not for preserving the order of messages. This makes SNS unsuitable for ordered processing.

Option C is incorrect because API Gateway authorizers are used for authentication and authorization, not for controlling the processing order of requests. Blocking requests with an authorizer might prevent processing altogether, but won't guarantee correct sequencing.

Option D is incorrect because Amazon SQS standard queues do not guarantee message order. While they aim for best-effort ordering, messages can sometimes be delivered out of sequence, which violates the stated requirement. https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/standard-queues.html

In summary, using an API Gateway integrated with an SQS FIFO queue and a Lambda function ensures that order information is captured in the correct sequence and then processed sequentially, meeting the application's specific requirements. This approach leverages the ordered message delivery capability of FIFO queues to maintain data integrity within the ecommerce application's workflow.

Question: 11 **Exam Heist**

A company has an application that runs on Amazon EC2 instances and uses an Amazon Aurora database. The EC2 instances connect to the database by using user names and passwords that are stored locally in a file. The company wants to minimize the operational overhead of credential management.

What should a solutions architect do to accomplish this goal?

A. Use AWS Secrets Manager. Turn on automatic rotation.

B. Use AWS Systems Manager Parameter Store. Turn on automatic rotation.

C. Create an Amazon S3 bucket to store objects that are encrypted with an AWS Key Management Service (AWS KMS) encryption key. Migrate the credential file to the S3 bucket. Point the application to the S3 bucket.

D. Create an encrypted Amazon Elastic Block Store (Amazon EBS) volume for each EC2 instance. Attach the new EBS volume to each EC2 instance. Migrate the credential file to the new EBS volume. Point the application to the new EBS volume.

Answer: A

Explanation:

The best solution is to use AWS Secrets Manager with automatic rotation. Here's why:

Secrets Manager Purpose: AWS Secrets Manager is specifically designed for securely storing and managing secrets, like database credentials. It eliminates the need to hardcode or store secrets within application code or configuration files.
Reduced Operational Overhead: By using Secrets Manager, the company centralizes credential management, which simplifies the process of updating, rotating, and auditing secrets. This reduces the operational burden associated with managing credentials manually on each EC2 instance.
Automatic Rotation: Secrets Manager's automatic rotation feature automates the process of changing credentials regularly without application downtime. This enhances security by limiting the window of opportunity for compromised credentials to be exploited.
IAM Integration: Secrets Manager integrates with AWS Identity and Access Management (IAM) to control access to secrets, ensuring that only authorized EC2 instances or roles can retrieve the database credentials.
Auditing and Monitoring: Secrets Manager provides auditing capabilities through AWS CloudTrail, allowing the company to track secret access and modifications.

Why other options are less suitable:

AWS Systems Manager Parameter Store: While Parameter Store can store sensitive information, it's primarily designed for storing configuration data and not for managing secrets with rotation capabilities. While Parameter Store offers SecureString parameters, it lacks the robust rotation features of Secrets Manager.
Amazon S3: Storing credentials in an S3 bucket, even if encrypted, is not the best practice for secret management. It requires custom logic to retrieve and manage the credentials, which increases operational overhead and doesn't provide automatic rotation.
Encrypted EBS Volume: Storing credentials on encrypted EBS volumes attached to each EC2 instance offers some level of security, but it doesn't centralize credential management or provide automatic rotation capabilities. It also requires managing separate volumes for each instance.

In summary, AWS Secrets Manager with automatic rotation is the most efficient and secure way to manage database credentials for EC2 instances connecting to Aurora, minimizing operational overhead and enhancing security posture.

Authoritative Links:

AWS Secrets Manager: https://aws.amazon.com/secrets-manager/
Rotate AWS Secrets Manager secrets:
https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotating-secrets.html

Question: 12 **Exam Heist**

A global company hosts its web application on Amazon EC2 instances behind an Application Load Balancer (ALB). The web application has static data and dynamic data. The company stores its static data in an Amazon S3 bucket. The company wants to improve performance and reduce latency for the static data and dynamic data. The company is using its own domain name registered with Amazon Route 53.
What should a solutions architect do to meet these requirements?

A. Create an Amazon CloudFront distribution that has the S3 bucket and the ALB as origins. Configure Route 53 to route traffic to the CloudFront distribution.

B. Create an Amazon CloudFront distribution that has the ALB as an origin. Create an AWS Global Accelerator standard accelerator that has the S3 bucket as an endpoint Configure Route 53 to route traffic to the CloudFront distribution.

C. Create an Amazon CloudFront distribution that has the S3 bucket as an origin. Create an AWS Global Accelerator standard accelerator that has the ALB and the CloudFront distribution as endpoints. Create a custom domain name that points to the accelerator DNS name. Use the custom domain name as an endpoint for the web application.

D. Create an Amazon CloudFront distribution that has the ALB as an origin. Create an AWS Global Accelerator standard accelerator that has the S3 bucket as an endpoint. Create two domain names. Point one domain name to the CloudFront DNS name for dynamic content. Point the other domain name to the accelerator DNS name for static content. Use the domain names as endpoints for the web application.

Answer: A

Explanation:

The best solution is to use CloudFront with both the S3 bucket (for static content) and the ALB (for dynamic content) as origins and then point Route 53 to the CloudFront distribution. This approach efficiently leverages CloudFront's caching capabilities to reduce latency and improve performance for both types of content globally.

Here's why other options are less optimal:

Option B & D: While Global Accelerator can improve performance, it primarily focuses on TCP and UDP traffic, not HTTP/HTTPS, which are used by web applications. Also, Global Accelerator is an overkill for static content served from S3 and adds unnecessary complexity. Global Accelerator is most useful for improving the performance of applications with users distributed globally where network congestion or routing issues can impact performance. Furthermore, managing separate domain names, as suggested in option D, would complicate the web application's architecture and potentially lead to inconsistent user experience.

Option C: Using Global Accelerator as a front for CloudFront is not a typical use case. CloudFront is already a global content delivery network (CDN) designed to optimize content delivery. Global Accelerator's benefits would be redundant.

Option A directly addresses the requirements by:

1. **Improving Performance and Reducing Latency:** CloudFront caches static content from the S3 bucket at edge locations globally, bringing data closer to users and reducing latency.
2. **Serving Dynamic Content:** CloudFront can also be configured to forward requests for dynamic content to the ALB. This allows CloudFront to cache the responses when possible, further improving performance.
3. **Using Existing Domain Name:** Route 53 is configured to route traffic to the CloudFront distribution, which means you can continue using your existing domain name.

Authoritative Links:

Amazon CloudFront: https://aws.amazon.com/cloudfront/
Amazon S3: https://aws.amazon.com/s3/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
AWS Global Accelerator: https://aws.amazon.com/global-accelerator/

Amazon Route 53: https://aws.amazon.com/route53/

Question: 13 **Exam Heist**

A company performs monthly maintenance on its AWS infrastructure. During these maintenance activities, the company needs to rotate the credentials for its Amazon RDS for MySQL databases across multiple AWS Regions. Which solution will meet these requirements with the LEAST operational overhead?

A. Store the credentials as secrets in AWS Secrets Manager. Use multi-Region secret replication for the required Regions. Configure Secrets Manager to rotate the secrets on a schedule.

B. Store the credentials as secrets in AWS Systems Manager by creating a secure string parameter. Use multi-Region secret replication for the required Regions. Configure Systems Manager to rotate the secrets on a schedule.

C. Store the credentials in an Amazon S3 bucket that has server-side encryption (SSE) enabled. Use Amazon EventBridge (Amazon CloudWatch Events) to invoke an AWS Lambda function to rotate the credentials.

D. Encrypt the credentials as secrets by using AWS Key Management Service (AWS KMS) multi-Region customer managed keys. Store the secrets in an Amazon DynamoDB global table. Use an AWS Lambda function to retrieve the secrets from DynamoDB. Use the RDS API to rotate the secrets.

Answer: A

Explanation:

Here's a detailed justification for why option A is the best solution for rotating RDS for MySQL database credentials across multiple AWS Regions with the least operational overhead:

Option A leverages AWS Secrets Manager's built-in capabilities for secret management and rotation across Regions. Secrets Manager is designed specifically for storing and managing secrets like database credentials. Its multi-Region secret replication feature automatically replicates secrets to specified Regions, ensuring consistency. The automatic rotation feature allows you to define a rotation schedule, where Secrets Manager handles the process of generating new credentials and updating them in the database, significantly reducing manual intervention and operational overhead.

Option B is incorrect because AWS Systems Manager Parameter Store, while capable of storing secrets, doesn't natively support multi-Region secret replication or automated rotation in the same streamlined manner as Secrets Manager. Implementing similar functionality with Parameter Store would require custom scripting and automation, increasing operational complexity.

Option C, using S3 and Lambda, involves more manual setup and management. While S3 can store encrypted data, it doesn't inherently offer secret rotation capabilities. The Lambda function would need to be custom-coded to generate new credentials, update the database, and update the secret in S3, adding to operational overhead and potential error points.

Option D, using DynamoDB and Lambda with KMS encryption, is the most complex. DynamoDB global tables provide replication, but managing the encryption with KMS and implementing secret rotation logic within Lambda, including calling the RDS API, adds significant overhead. KMS is mainly for encryption key management, not secret rotation.

Therefore, option A provides the most straightforward and automated solution for rotating RDS database credentials across multiple Regions due to Secrets Manager's features like built-in rotation and multi-region replication, minimizing operational overhead.

Relevant Links:

AWS Secrets Manager: https://aws.amazon.com/secrets-manager/
Secrets Manager Multi-Region Secrets:

https://docs.aws.amazon.com/secretsmanager/latest/userguide/replication.html
Secrets Manager Rotation: https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotating-secrets.html

Question: 14 **Exam Heist**

A company runs an ecommerce application on Amazon EC2 instances behind an Application Load Balancer. The instances run in an Amazon EC2 Auto Scaling group across multiple Availability Zones. The Auto Scaling group scales based on CPU utilization metrics. The ecommerce application stores the transaction data in a MySQL 8.0 database that is hosted on a large EC2 instance.
The database's performance degrades quickly as application load increases. The application handles more read requests than write transactions. The company wants a solution that will automatically scale the database to meet the demand of unpredictable read workloads while maintaining high availability.
Which solution will meet these requirements?

A. Use Amazon Redshift with a single node for leader and compute functionality.

B. Use Amazon RDS with a Single-AZ deployment Configure Amazon RDS to add reader instances in a different Availability Zone.

C. Use Amazon Aurora with a Multi-AZ deployment. Configure Aurora Auto Scaling with Aurora Replicas.

D. Use Amazon ElastiCache for Memcached with EC2 Spot Instances.

Answer: C

Explanation:

The correct solution is to use Amazon Aurora with a Multi-AZ deployment and configure Aurora Auto Scaling with Aurora Replicas. Here's why:

Scalability: Aurora Auto Scaling automatically adds or removes Aurora Replicas in response to changes in application load. This addresses the need to scale the database to meet unpredictable read workloads.

Read-Heavy Workloads: Aurora Replicas are designed for read operations. By directing read traffic to these replicas, the primary database instance is relieved, improving overall performance.

High Availability: A Multi-AZ deployment ensures that the database remains available even if there is an infrastructure failure in one Availability Zone. Aurora automatically fails over to a replica in another Availability Zone.

Aurora's Compatibility: Aurora is compatible with MySQL and PostgreSQL, allowing for easier migration from the existing MySQL database.

Other options are suboptimal because:

A. Amazon Redshift is a data warehouse service optimized for analytical workloads, not transactional workloads.
B. Amazon RDS with a Single-AZ deployment does not guarantee high availability. Although reader instances can be added, a single-AZ configuration for the primary instance presents a single point of failure.
D. Amazon ElastiCache for Memcached is an in-memory caching service, not a database. It is not suitable for storing transactional data.

Supporting Links:

Amazon Aurora: https://aws.amazon.com/rds/aurora/
Aurora Auto Scaling:
https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Managing.Performance.html
Aurora Replicas:

https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Managing.ReadReplicas.html
Amazon RDS Multi-AZ: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html

Question: 15 **Exam Heist**

A company recently migrated to AWS and wants to implement a solution to protect the traffic that flows in and out of the production VPC. The company had an inspection server in its on-premises data center. The inspection server performed specific operations such as traffic flow inspection and traffic filtering. The company wants to have the same functionalities in the AWS Cloud.
Which solution will meet these requirements?

A. Use Amazon GuardDuty for traffic inspection and traffic filtering in the production VPC.

B. Use Traffic Mirroring to mirror traffic from the production VPC for traffic inspection and filtering.

C. Use AWS Network Firewall to create the required rules for traffic inspection and traffic filtering for the production VPC.

D. Use AWS Firewall Manager to create the required rules for traffic inspection and traffic filtering for the production VPC.

Answer: C

Explanation:

The correct answer is C, using AWS Network Firewall. Here's a detailed justification:

The requirement is to replicate on-premises traffic inspection and filtering functionalities within AWS for a production VPC. Amazon GuardDuty (A) is a threat detection service that monitors for malicious activity and unauthorized behavior but doesn't provide inline traffic inspection and filtering based on custom rules. Traffic Mirroring (B) duplicates network traffic for analysis, but it doesn't provide filtering capabilities itself. You would need a separate inspection appliance to receive and process the mirrored traffic, adding complexity and cost. AWS Firewall Manager (D) centrally manages firewall rules across multiple accounts and VPCs; it doesn't inspect traffic directly or provide inspection capabilities. It works with AWS WAF, AWS Shield Advanced, and AWS Network Firewall.

AWS Network Firewall (C), on the other hand, is a managed network firewall service that provides essential protection for your VPCs. It allows you to define rules for inspecting and filtering network traffic based on criteria such as source and destination IP addresses, ports, and protocols. This precisely aligns with the company's need to implement traffic flow inspection and filtering, mimicking the capabilities of their on-premises inspection server. Network Firewall operates at the network layer (Layer 3 and Layer 4) and application layer (Layer 7), allowing for deep packet inspection (DPI) and fine-grained control. It also scales automatically to meet fluctuating traffic demands without requiring manual intervention.

Therefore, AWS Network Firewall is the best fit for the company's requirements because it provides a fully managed, scalable, and customizable solution for traffic inspection and filtering within the production VPC.

Authoritative links:

AWS Network Firewall: https://aws.amazon.com/network-firewall/
AWS Network Firewall Documentation: https://docs.aws.amazon.com/network-firewall/latest/developerguide/what-is-aws-network-firewall.html

Question: 16 **Exam Heist**

A company hosts a data lake on AWS. The data lake consists of data in Amazon S3 and Amazon RDS for PostgreSQL. The company needs a reporting solution that provides data visualization and includes all the data sources within the data lake. Only the company's management team should have full access to all the

visualizations. The rest of the company should have only limited access.
Which solution will meet these requirements?

A. Create an analysis in Amazon QuickSight. Connect all the data sources and create new datasets. Publish dashboards to visualize the data. Share the dashboards with the appropriate IAM roles.

B. Create an analysis in Amazon QuickSight. Connect all the data sources and create new datasets. Publish dashboards to visualize the data. Share the dashboards with the appropriate users and groups.

C. Create an AWS Glue table and crawler for the data in Amazon S3. Create an AWS Glue extract, transform, and load (ETL) job to produce reports. Publish the reports to Amazon S3. Use S3 bucket policies to limit access to the reports.

D. Create an AWS Glue table and crawler for the data in Amazon S3. Use Amazon Athena Federated Query to access data within Amazon RDS for PostgreSQL. Generate reports by using Amazon Athena. Publish the reports to Amazon S3. Use S3 bucket policies to limit access to the reports.

Answer: B

Explanation:

The correct answer is B. Let's break down why:

Requirement 1: Data Visualization and Data Lake Integration: The company needs a data visualization tool that can ingest data from both S3 (data lake) and RDS for PostgreSQL. Amazon QuickSight excels at this. It can directly connect to both S3 (using manifests or direct paths) and RDS databases. Options C and D primarily focus on AWS Glue and Athena, which are more suited for data processing and querying, not direct visualization.

Requirement 2: Different Access Levels (Management vs. Rest of Company): QuickSight has a robust sharing mechanism. Dashboards can be shared with individual users or groups (created within QuickSight itself), allowing for granular control over who sees what. IAM roles (option A) are primarily used for controlling access to AWS resources, not for fine-grained access to dashboards within QuickSight. S3 bucket policies (options C and D) would only control access to the underlying report files, not the dynamic visualizations.

QuickSight Analysis, Datasets, and Dashboards: QuickSight works by creating an analysis from one or more datasets. The datasets are based on the connected data sources. The analysis is then used to build one or more dashboards. Publishing the dashboards makes them available to others.

Direct Connection to Data Sources: QuickSight's ability to directly connect to the data sources (S3 and RDS) eliminates the need for complex ETL processes just for reporting (which would be implied by option C). While ETL might be part of the broader data lake architecture, it's not the primary function for a reporting solution.

Athena Federated Query is overkill: Athena Federated Query (option D) allows querying across different data sources. Although possible, using QuickSight's direct connectors is a simpler and more suitable approach for this scenario.

In summary, option B is the most straightforward solution, using QuickSight's built-in capabilities to connect to data sources, create visualizations, and share dashboards with user-level and group-level access control.

Authoritative Links:

Amazon QuickSight: https://aws.amazon.com/quicksight/
Connecting to Data Sources in QuickSight: https://docs.aws.amazon.com/quicksight/latest/user/data-sources.html
Sharing QuickSight Dashboards: https://docs.aws.amazon.com/quicksight/latest/user/sharing-a-dashboard.html

A company is implementing a new business application. The application runs on two Amazon EC2 instances and uses an Amazon S3 bucket for document storage. A solutions architect needs to ensure that the EC2 instances can access the S3 bucket.
What should the solutions architect do to meet this requirement?

A. Create an IAM role that grants access to the S3 bucket. Attach the role to the EC2 instances.
B. Create an IAM policy that grants access to the S3 bucket. Attach the policy to the EC2 instances.
C. Create an IAM group that grants access to the S3 bucket. Attach the group to the EC2 instances.
D. Create an IAM user that grants access to the S3 bucket. Attach the user account to the EC2 instances.

Answer: A

Explanation:

The correct answer is A: Create an IAM role that grants access to the S3 bucket and attach the role to the EC2 instances. Here's why:

IAM Roles are the recommended approach for granting permissions to AWS services like EC2 to access other AWS resources such as S3. A role provides temporary security credentials to the EC2 instances, eliminating the need to store long-term credentials directly on the instances. This enhances security because the credentials are automatically rotated and managed by AWS.

IAM Policies define the permissions granted. While policies are fundamental, they are attached to IAM entities. Attaching an IAM policy directly to an EC2 instance is not a standard practice. Instead, the policy is attached to an IAM role, which is then associated with the EC2 instance. The instance assumes the role, gaining the permissions defined in the policy.

IAM Groups are collections of IAM users. While groups simplify permission management for users, they are not designed to be associated with EC2 instances for granting access to resources. Groups are a user-centric concept and not appropriate for service-to-service access control.

IAM Users represent individuals or applications that interact with AWS. Creating an IAM user and embedding its credentials on an EC2 instance is a security risk. These credentials would be long-term and static, making them vulnerable if the instance is compromised. Hardcoding or storing credentials on an EC2 instance is a bad practice. IAM Roles provide a more secure and manageable solution.

Therefore, the most secure and best practice approach is to create an IAM role with the necessary S3 access permissions and then attach that role to the EC2 instances. This allows the EC2 instances to assume the role and obtain temporary credentials to access the S3 bucket.

Further Reading:

IAM Roles for Amazon EC2
IAM Policies
Security best practices in IAM

Question: 18 **Exam Heist**

An application development team is designing a microservice that will convert large images to smaller, compressed images. When a user uploads an image through the web interface, the microservice should store the image in an Amazon S3 bucket, process and compress the image with an AWS Lambda function, and store the image in its compressed form in a different S3 bucket.
A solutions architect needs to design a solution that uses durable, stateless components to process the images automatically.
Which combination of actions will meet these requirements? (Choose two.)

A. Create an Amazon Simple Queue Service (Amazon SQS) queue. Configure the S3 bucket to send a

notification to the SQS queue when an image is uploaded to the S3 bucket.

B. Configure the Lambda function to use the Amazon Simple Queue Service (Amazon SQS) queue as the invocation source. When the SQS message is successfully processed, delete the message in the queue.

C. Configure the Lambda function to monitor the S3 bucket for new uploads. When an uploaded image is detected, write the file name to a text file in memory and use the text file to keep track of the images that were processed.

D. Launch an Amazon EC2 instance to monitor an Amazon Simple Queue Service (Amazon SQS) queue. When items are added to the queue, log the file name in a text file on the EC2 instance and invoke the Lambda function.

E. Configure an Amazon EventBridge (Amazon CloudWatch Events) event to monitor the S3 bucket. When an image is uploaded, send an alert to an Amazon ample Notification Service (Amazon SNS) topic with the application owner's email address for further processing.

Answer: AB

Explanation:

The chosen solution, using SQS and Lambda triggered by SQS, effectively addresses the requirements for durable, stateless image processing.

Option A is correct because SQS provides a durable queuing mechanism. When an image is uploaded to S3, a notification is sent to the SQS queue. This ensures that even if the Lambda function is temporarily unavailable, the image processing request will persist in the queue until it can be processed. SQS decouples the S3 upload event from the Lambda function invocation, enabling asynchronous processing. This aligns with the need for a stateless and reliable processing pipeline. https://aws.amazon.com/sqs/

Option B is correct because configuring the Lambda function to consume messages from the SQS queue directly addresses the need for automated processing. Lambda's SQS trigger automatically polls the queue and invokes the function when a message arrives. This eliminates the need for custom polling logic. By deleting the message upon successful processing, the queue is kept clean and ensures that each image is processed only once. https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html

Option C is incorrect because storing filenames in a text file in memory within the Lambda function introduces statefulness. Lambda functions should be stateless, and relying on in-memory storage for tracking processed images is unreliable, especially with concurrent invocations.

Option D is incorrect because launching an EC2 instance to monitor SQS adds unnecessary complexity and cost. Lambda functions can directly integrate with SQS without requiring an intermediary EC2 instance.

Option E is incorrect because using SNS for further processing is vague and doesn't directly address the requirement of automated image processing. SNS is a notification service, not an event-driven compute service. The application owner would need to manually trigger the image processing, defeating the automation requirement.

Question: 19 **Exam Heist**

A company has a three-tier web application that is deployed on AWS. The web servers are deployed in a public subnet in a VPC. The application servers and database servers are deployed in private subnets in the same VPC. The company has deployed a third-party virtual firewall appliance from AWS Marketplace in an inspection VPC. The appliance is configured with an IP interface that can accept IP packets.
A solutions architect needs to integrate the web application with the appliance to inspect all traffic to the application before the traffic reaches the web server.
Which solution will meet these requirements with the LEAST operational overhead?

A. Create a Network Load Balancer in the public subnet of the application's VPC to route the traffic to the appliance for packet inspection.

B. Create an Application Load Balancer in the public subnet of the application's VPC to route the traffic to the appliance for packet inspection.

C. Deploy a transit gateway in the inspection VPConfigure route tables to route the incoming packets through the transit gateway.

D. Deploy a Gateway Load Balancer in the inspection VPC. Create a Gateway Load Balancer endpoint to receive the incoming packets and forward the packets to the appliance.

Answer: D

Explanation:

The correct answer is **D: Deploy a Gateway Load Balancer in the inspection VPC. Create a Gateway Load Balancer endpoint to receive the incoming packets and forward the packets to the appliance.**

Here's a detailed justification:

Gateway Load Balancer (GWLB): A GWLB is specifically designed for inline inspection of network traffic. It provides a single entry point for traffic, scales automatically, and distributes traffic to virtual appliances like firewalls. This is exactly what the scenario requires: inspecting traffic before it reaches the web servers.
GWLB Endpoint: A GWLB endpoint allows you to seamlessly integrate the GWLB (in the inspection VPC) with the web application's VPC. Traffic destined for the web application's public subnet is redirected to the GWLB endpoint.
Traffic Flow: Incoming traffic from the internet flows to the GWLB endpoint in the application's VPC. The endpoint forwards the traffic to the GWLB in the inspection VPC. The GWLB distributes the traffic to the firewall appliance for inspection. After inspection, the appliance sends the traffic back through the GWLB, back through the endpoint, and finally to the web servers.
Least Operational Overhead: Compared to other options, the GWLB solution offers the least operational overhead. It's a managed service, meaning AWS handles scaling, availability, and patching. You don't have to manage routing tables as extensively as with Transit Gateway.

Here's why the other options are less suitable:

A. Network Load Balancer (NLB): NLBs are designed for load balancing TCP, UDP, and TLS traffic to backend targets. They are not suitable for inspecting packets at the application layer, or generally for integration with virtual firewalls. NLBs do not have native features to forward traffic for inspection and then back to the original destination.
B. Application Load Balancer (ALB): ALBs operate at the application layer (HTTP/HTTPS) and are primarily used for load balancing web traffic based on content. They are not intended for routing packets to virtual appliances for general-purpose packet inspection.
C. Transit Gateway (TGW): TGW is a network transit hub that connects VPCs and on-premises networks. While you could use TGW to route traffic through an inspection VPC, it involves more complex route table configurations and management. It's overkill and adds unnecessary operational complexity compared to the GWLB, which is designed specifically for this purpose.

Authoritative Links:

Gateway Load Balancer: https://aws.amazon.com/elasticloadbalancing/gateway-load-balancer/
AWS Network Firewall Integration with GWLB: https://aws.amazon.com/blogs/networking-and-content-delivery/centralized-inspection-architecture-with-aws-network-firewall-gateway-load-balancer-and-route-manager/

Question: 20 **Exam Heist**

A company wants to improve its ability to clone large amounts of production data into a test environment in the same AWS Region. The data is stored in Amazon EC2 instances on Amazon Elastic Block Store (Amazon EBS)

volumes. Modifications to the cloned data must not affect the production environment. The software that accesses this data requires consistently high I/O performance.
A solutions architect needs to minimize the time that is required to clone the production data into the test environment.
Which solution will meet these requirements?

A. Take EBS snapshots of the production EBS volumes. Restore the snapshots onto EC2 instance store volumes in the test environment.

B. Configure the production EBS volumes to use the EBS Multi-Attach feature. Take EBS snapshots of the production EBS volumes. Attach the production EBS volumes to the EC2 instances in the test environment.

C. Take EBS snapshots of the production EBS volumes. Create and initialize new EBS volumes. Attach the new EBS volumes to EC2 instances in the test environment before restoring the volumes from the production EBS snapshots.

D. Take EBS snapshots of the production EBS volumes. Turn on the EBS fast snapshot restore feature on the EBS snapshots. Restore the snapshots into new EBS volumes. Attach the new EBS volumes to EC2 instances in the test environment.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct answer, and why other options are incorrect:

Why Option D is Correct: "Take EBS snapshots of the production EBS volumes. Turn on the EBS fast snapshot restore feature on the EBS snapshots. Restore the snapshots into new EBS volumes. Attach the new EBS volumes to EC2 instances in the test environment."

This solution directly addresses the requirement of minimizing the cloning time while maintaining data isolation.

1. **EBS Snapshots for Data Duplication:** EBS snapshots are the standard and efficient way to create point-in-time copies of EBS volumes. This ensures a consistent and reliable clone of the production data. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EBSSnapshots.html
2. **Fast Snapshot Restore (FSR) for Speed:** FSR significantly reduces the time required to restore EBS volumes from snapshots. Normally, restoring from a snapshot involves lazily loading the data blocks as they are accessed. FSR pre-initializes the volume in the background, so it's immediately ready for high I/O performance upon attachment. This satisfies the "minimize the time that is required to clone" requirement. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-fast-snapshot-restore.html
3. **New EBS Volumes for Isolation:** By restoring the snapshots into new EBS volumes, the test environment operates on independent copies of the data. Modifications in the test environment will not affect the production data, meeting the "Modifications to the cloned data must not affect the production environment" requirement.
4. **Attaching to EC2 Instances:** Finally, attaching the new EBS volumes to EC2 instances in the test environment makes the cloned data accessible to the applications.

Why Other Options Are Incorrect:

Option A: Restore to EC2 Instance Store: EC2 instance store volumes are ephemeral, meaning the data is lost when the instance is stopped or terminated. This makes them unsuitable for a persistent test environment where data needs to be retained. Further, instance store volumes aren't suitable for large datasets and generally have poorer durability.
Option B: EBS Multi-Attach and Attaching Production Volumes: EBS Multi-Attach allows you to attach a single EBS volume to multiple EC2 instances within the same Availability Zone. However, directly attaching production volumes to test instances creates a significant risk of data corruption in the production environment if the test environment makes unintended modifications. This also requires coordinating access and locking which is far from ideal. More over, the question states that modifications to cloned data should

NOT affect production environment. This choice violates it.
Option C: Creating Volumes and Restoring Without FSR: While this solution does provide data isolation by creating new EBS volumes, it does not address the requirement of minimizing the cloning time. Without FSR, restoring large EBS volumes from snapshots can take a significant amount of time, especially when high I/O performance is immediately required.

In summary, option D provides the best balance of data isolation, cloning speed, and performance for the test environment, leveraging EBS snapshots and FSR for efficient and safe data duplication.

Question: 21 **Exam Heist**

An ecommerce company wants to launch a one-deal-a-day website on AWS. Each day will feature exactly one product on sale for a period of 24 hours. The company wants to be able to handle millions of requests each hour with millisecond latency during peak hours.
Which solution will meet these requirements with the LEAST operational overhead?

A. Use Amazon S3 to host the full website in different S3 buckets. Add Amazon CloudFront distributions. Set the S3 buckets as origins for the distributions. Store the order data in Amazon S3.

B. Deploy the full website on Amazon EC2 instances that run in Auto Scaling groups across multiple Availability Zones. Add an Application Load Balancer (ALB) to distribute the website traffic. Add another ALB for the backend APIs. Store the data in Amazon RDS for MySQL.

C. Migrate the full application to run in containers. Host the containers on Amazon Elastic Kubernetes Service (Amazon EKS). Use the Kubernetes Cluster Autoscaler to increase and decrease the number of pods to process bursts in traffic. Store the data in Amazon RDS for MySQL.

D. Use an Amazon S3 bucket to host the website's static content. Deploy an Amazon CloudFront distribution. Set the S3 bucket as the origin. Use Amazon API Gateway and AWS Lambda functions for the backend APIs. Store the data in Amazon DynamoDB.

Answer: D

Explanation:

Option D is the most suitable solution because it leverages serverless technologies and static content hosting for optimal scalability, performance, and minimal operational overhead. Hosting the static website content (HTML, CSS, JavaScript, images) on Amazon S3 and distributing it via Amazon CloudFront provides global content delivery with low latency and high availability. CloudFront's caching capabilities further reduce load on the origin S3 bucket.

Using Amazon API Gateway and AWS Lambda for backend APIs enables on-demand execution of code without managing servers. API Gateway handles request routing, authorization, and throttling, while Lambda functions execute the business logic. This serverless approach scales automatically with traffic, ensuring millisecond latency during peak hours.

Storing order data in Amazon DynamoDB is ideal because DynamoDB is a fully managed NoSQL database service that provides consistent, single-digit millisecond latency at any scale. DynamoDB's serverless nature eliminates the need for database administration tasks.

Options A, B, and C involve managing infrastructure (EC2 instances, containers, databases), which increases operational overhead. Option A's approach of storing order data directly in S3 isn't suitable for transactional data. Option B utilizes ALBs and RDS for MySQL, requiring instance management and database administration. Option C introduces the complexity of container orchestration with EKS. Therefore, Option D presents the least operational overhead while meeting the performance requirements.

Here are some authoritative links for further research:

Amazon S3: https://aws.amazon.com/s3/

Amazon CloudFront: https://aws.amazon.com/cloudfront/
Amazon API Gateway: https://aws.amazon.com/api-gateway/
AWS Lambda: https://aws.amazon.com/lambda/
Amazon DynamoDB: https://aws.amazon.com/dynamodb/

Question: 22 **Exam Heist**

A solutions architect is using Amazon S3 to design the storage architecture of a new digital media application. The media files must be resilient to the loss of an Availability Zone. Some files are accessed frequently while other files are rarely accessed in an unpredictable pattern. The solutions architect must minimize the costs of storing and retrieving the media files.
Which storage option meets these requirements?

A. S3 Standard
B. S3 Intelligent-Tiering
C. S3 Standard-Infrequent Access (S3 Standard-IA)
D. S3 One Zone-Infrequent Access (S3 One Zone-IA)

Answer: B

Explanation:

The correct answer is **B. S3 Intelligent-Tiering**.

Here's why:

Resilience to Availability Zone Loss: The application requires resilience to the loss of an Availability Zone. This immediately rules out S3 One Zone-IA (Option D) because it stores data in a single Availability Zone. S3 Standard, S3 Intelligent-Tiering, and S3 Standard-IA all replicate data across multiple Availability Zones, providing the necessary resilience.

Frequently and Infrequently Accessed Files: The application has both frequently and infrequently accessed files with an unpredictable access pattern. S3 Standard-IA (Option C) is suitable for data accessed less frequently, but using it for frequently accessed data would be unnecessarily expensive. S3 Standard (Option A) would be suitable for all objects but is the most expensive option for objects that are infrequent to access.

Cost Minimization: S3 Intelligent-Tiering automatically moves data between frequent and infrequent access tiers based on access patterns, without any operational overhead or retrieval fees. It delivers automatic cost savings when access patterns change. It monitors access patterns and moves objects that have not been accessed for 30 consecutive days to the infrequent access tier and then after another 60 days of inactivity to the archive access tier. This ensures the optimal storage cost for both frequently and infrequently accessed files.

Benefits of Intelligent-Tiering: S3 Intelligent-Tiering optimizes storage costs by automatically moving data to the most cost-effective access tier without performance impact or operational overhead. This approach is ideal when access patterns are unpredictable or change over time.

In summary, S3 Intelligent-Tiering balances the requirements of Availability Zone resilience, handling both frequently and infrequently accessed files, and minimizing storage costs, making it the most suitable storage option.

Relevant Links:

Amazon S3 Storage Classes:
S3 Intelligent-Tiering:

Question: 23 **Exam Heist**

A company is storing backup files by using Amazon S3 Standard storage. The files are accessed frequently for 1 month. However, the files are not accessed after 1 month. The company must keep the files indefinitely.
Which storage solution will meet these requirements MOST cost-effectively?

A. Configure S3 Intelligent-Tiering to automatically migrate objects.

B. Create an S3 Lifecycle configuration to transition objects from S3 Standard to S3 Glacier Deep Archive after 1 month.

C. Create an S3 Lifecycle configuration to transition objects from S3 Standard to S3 Standard-Infrequent Access (S3 Standard-IA) after 1 month.

D. Create an S3 Lifecycle configuration to transition objects from S3 Standard to S3 One Zone-Infrequent Access (S3 One Zone-IA) after 1 month.

Answer: B

Explanation:

The question requires the most cost-effective storage solution for backup files frequently accessed for one month and then indefinitely archived with infrequent access.

Option B, creating an S3 Lifecycle configuration to transition objects from S3 Standard to S3 Glacier Deep Archive after 1 month, is the most cost-effective solution. S3 Glacier Deep Archive offers the lowest storage cost among all S3 storage classes, making it ideal for long-term archival. The lifecycle policy automates the transition, eliminating manual intervention. S3 Standard is used for the first month of frequent access. This approach balances initial accessibility with long-term cost optimization for rarely accessed data.

Option A is less suitable. S3 Intelligent-Tiering automatically moves data between frequent, infrequent, and archive access tiers based on usage patterns. While convenient, the cost of monitoring and frequent tier transitions might exceed the cost-effectiveness of directly moving to Glacier Deep Archive after the initial active period. The assumption is that the files are never accessed after the initial month, making Intelligent-Tiering an unnecessary expense.

Option C involves transitioning to S3 Standard-IA, which is designed for data accessed less frequently but still requires rapid retrieval. Given that the backup files will likely not be accessed after the first month, using Standard-IA is more expensive than Glacier Deep Archive because Standard-IA has higher storage costs.

Option D uses S3 One Zone-IA, which is even less suitable than Standard-IA, although cheaper than Standard-IA. It stores data in a single Availability Zone, making it cheaper, but it's inherently less durable than Standard-IA, Standard or Glacier. Because these are backup files the durability provided by the storage is of utmost importance, and a loss of data due to a single availability zone failure is unacceptable. Moreover, it's more expensive than Glacier Deep Archive.

Therefore, transitioning to Glacier Deep Archive through a Lifecycle policy offers the best combination of cost-effectiveness and long-term archival needs when data will not be accessed for the foreseeable future after the first month.

Relevant documentation:

S3 Glacier Deep Archive: Describes the features and cost benefits of S3 Glacier Deep Archive.
S3 Lifecycle Management: Explains how to automate transitions between storage classes.

Question: 24 **Exam Heist**

A company observes an increase in Amazon EC2 costs in its most recent bill. The billing team notices unwanted

vertical scaling of instance types for a couple of EC2 instances. A solutions architect needs to create a graph comparing the last 2 months of EC2 costs and perform an in-depth analysis to identify the root cause of the vertical scaling.
How should the solutions architect generate the information with the LEAST operational overhead?

A. Use AWS Budgets to create a budget report and compare EC2 costs based on instance types.

B. Use Cost Explorer's granular filtering feature to perform an in-depth analysis of EC2 costs based on instance types.

C. Use graphs from the AWS Billing and Cost Management dashboard to compare EC2 costs based on instance types for the last 2 months.

D. Use AWS Cost and Usage Reports to create a report and send it to an Amazon S3 bucket. Use Amazon QuickSight with Amazon S3 as a source to generate an interactive graph based on instance types.

Answer: B

Explanation:

The correct answer is **B. Use Cost Explorer's granular filtering feature to perform an in-depth analysis of EC2 costs based on instance types.**

Here's why:

Cost Explorer is designed specifically for cost analysis and provides a user-friendly interface for exploring AWS costs. It allows for granular filtering by service (EC2), usage type (instance type), and time range, enabling the solutions architect to easily compare EC2 costs for the last two months and identify the instances that have experienced unwanted vertical scaling. Cost Explorer's built-in charting capabilities make it straightforward to visualize these cost trends without requiring additional tools or setup. The "group by" feature within Cost Explorer lets users easily aggregate costs by instance type. This allows for direct identification of cost increases associated with specific instance types, streamlining the analysis. The goal is to analyze past costs, and Cost Explorer is built to efficiently do just that.

Option A, AWS Budgets, is primarily for setting cost thresholds and receiving alerts when those thresholds are exceeded. While it can provide cost information, it's not designed for in-depth historical analysis and doesn't offer the granular filtering capabilities needed to pinpoint specific instance type cost increases as readily as Cost Explorer.

Option C, the AWS Billing and Cost Management dashboard, provides a high-level overview of costs but lacks the granular filtering and analysis features available in Cost Explorer. It would require more manual effort to isolate and analyze the cost increase associated with specific EC2 instance types.

Option D, AWS Cost and Usage Reports (CUR) with Amazon QuickSight, is powerful but involves significantly more operational overhead. It requires configuring and delivering the CUR to an S3 bucket, then setting up QuickSight with S3 as a data source, and finally creating the interactive graph. This process is much more complex and time-consuming compared to using Cost Explorer's built-in features. While CUR provides comprehensive data, for a focused analysis as described in the question, Cost Explorer is far more efficient.

Therefore, Cost Explorer provides the least operational overhead and is the most efficient way to achieve the required in-depth analysis of EC2 costs based on instance types.

Here's an authoritative link for further research:

AWS Cost Explorer

Question: 25 **Exam Heist**

A company is designing an application. The application uses an AWS Lambda function to receive information through Amazon API Gateway and to store the information in an Amazon Aurora PostgreSQL database.

During the proof-of-concept stage, the company has to increase the Lambda quotas significantly to handle the high volumes of data that the company needs to load into the database. A solutions architect must recommend a new design to improve scalability and minimize the configuration effort.
Which solution will meet these requirements?

A. Refactor the Lambda function code to Apache Tomcat code that runs on Amazon EC2 instances. Connect the database by using native Java Database Connectivity (JDBC) drivers.

B. Change the platform from Aurora to Amazon DynamoDProvision a DynamoDB Accelerator (DAX) cluster. Use the DAX client SDK to point the existing DynamoDB API calls at the DAX cluster.

C. Set up two Lambda functions. Configure one function to receive the information. Configure the other function to load the information into the database. Integrate the Lambda functions by using Amazon Simple Notification Service (Amazon SNS).

D. Set up two Lambda functions. Configure one function to receive the information. Configure the other function to load the information into the database. Integrate the Lambda functions by using an Amazon Simple Queue Service (Amazon SQS) queue.

Answer: D

Explanation:

The best solution to improve scalability and minimize configuration effort when dealing with high volumes of data between a Lambda function receiving data and an Aurora PostgreSQL database is to use an Amazon SQS queue to decouple the two functions.

Here's why:

Decoupling with SQS: SQS acts as a buffer between the data receiving Lambda function and the database loading Lambda function. The first Lambda function places messages containing the information into the SQS queue. The second Lambda function retrieves messages from the queue and loads the data into Aurora. This decoupling allows each function to scale independently based on its specific needs.

Scalability: SQS provides virtually unlimited scalability. It can handle large volumes of messages, allowing the system to absorb bursts of data without overwhelming the database. The database loading Lambda function can then process the messages at a rate the database can handle, preventing overload.

Minimized Configuration Effort: SQS is a managed service, which reduces the operational burden. Setting up the queue and integrating it with the Lambda functions requires minimal configuration.

Resilience: SQS provides message durability. If the database loading Lambda function fails, the messages remain in the queue until they are successfully processed, ensuring data is not lost. This contrasts with SNS, which is better suited for fan-out scenarios and doesn't guarantee message delivery if no subscribers are available.

Why other options are less suitable:

A: Refactoring to EC2 and Tomcat increases operational complexity. EC2 requires managing servers, including patching, scaling, and ensuring high availability. This defeats the goal of minimizing configuration effort.
B: Changing to DynamoDB with DAX is a significant architectural change and involves substantial code modification. DAX is for DynamoDB, not Aurora PostgreSQL.
C: Using SNS doesn't provide the queuing and buffering capabilities of SQS. If the database loading Lambda function is unavailable, messages sent via SNS might be lost, potentially impacting data consistency. SNS is not suitable for reliable, asynchronous processing.

In summary, using SQS for decoupling provides the best combination of scalability, minimized configuration, and reliability for this use case.

Relevant Documentation:

Amazon SQS: https://aws.amazon.com/sqs/
AWS Lambda: https://aws.amazon.com/lambda/
Amazon Aurora: https://aws.amazon.com/rds/aurora/

Question: 26 **Exam Heist**

A company needs to review its AWS Cloud deployment to ensure that its Amazon S3 buckets do not have unauthorized configuration changes.
What should a solutions architect do to accomplish this goal?

A. Turn on AWS Config with the appropriate rules.
B. Turn on AWS Trusted Advisor with the appropriate checks.
C. Turn on Amazon Inspector with the appropriate assessment template.
D. Turn on Amazon S3 server access logging. Configure Amazon EventBridge (Amazon Cloud Watch Events).

Answer: A

Explanation:

The correct answer is A, turning on AWS Config with the appropriate rules. Here's why:

AWS Config is a service that enables you to assess, audit, and evaluate the configurations of your AWS resources. It continuously monitors and records your AWS resource configurations, allowing you to automate the evaluation of recorded configurations against desired configurations. In this scenario, the goal is to ensure S3 bucket configurations remain authorized, which directly aligns with Config's capabilities. You can define AWS Config rules that specify the desired state of your S3 buckets (e.g., encryption enabled, public access blocked). If a bucket drifts from this desired state, Config will flag it as non-compliant. This makes it perfect for detecting unauthorized configuration changes.

Option B, AWS Trusted Advisor, primarily provides recommendations on cost optimization, performance, security, fault tolerance, and service limits. While it offers security checks, it is more high-level and may not provide the detailed configuration change detection needed for S3 buckets specifically.

Option C, Amazon Inspector, focuses on security vulnerabilities and deviations from security best practices within your EC2 instances and container images. It's not designed to monitor and audit configuration changes to S3 buckets.

Option D, turning on Amazon S3 server access logging and configuring Amazon EventBridge (CloudWatch Events), would provide logs of who accessed the bucket. EventBridge could trigger alerts on specific events. However, it requires significant custom parsing and rule building to determine if the configuration of the bucket itself has changed (e.g., a change to bucket policy or encryption). This is a much more complex and less direct method than using AWS Config. Config provides managed rules specifically designed for configuration compliance.

In summary, AWS Config is the most suitable service because it's designed to continuously monitor and evaluate the configurations of AWS resources, allowing you to proactively identify and remediate unauthorized configuration changes in S3 buckets.

Authoritative Links:

AWS Config: https://aws.amazon.com/config/
AWS Config Rules: https://docs.aws.amazon.com/config/latest/developerguide/evaluate-config.html

Question: 27 **Exam Heist**

A company is launching a new application and will display application metrics on an Amazon CloudWatch dashboard. The company's product manager needs to access this dashboard periodically. The product manager does not have an AWS account. A solutions architect must provide access to the product manager by following the principle of least privilege.
Which solution will meet these requirements?

A. Share the dashboard from the CloudWatch console. Enter the product manager's email address, and complete the sharing steps. Provide a shareable link for the dashboard to the product manager.

B. Create an IAM user specifically for the product manager. Attach the CloudWatchReadOnlyAccess AWS managed policy to the user. Share the new login credentials with the product manager. Share the browser URL of the correct dashboard with the product manager.

C. Create an IAM user for the company's employees. Attach the ViewOnlyAccess AWS managed policy to the IAM user. Share the new login credentials with the product manager. Ask the product manager to navigate to the CloudWatch console and locate the dashboard by name in the Dashboards section.

D. Deploy a bastion server in a public subnet. When the product manager requires access to the dashboard, start the server and share the RDP credentials. On the bastion server, ensure that the browser is configured to open the dashboard URL with cached AWS credentials that have appropriate permissions to view the dashboard.

Answer: A

Explanation:

The best solution is A because it leverages CloudWatch's built-in dashboard sharing feature, providing the most direct and least privileged access for the product manager without requiring an AWS account. CloudWatch's sharing feature allows you to create a shareable link to the dashboard that can be accessed without AWS credentials. This satisfies the requirement of allowing the product manager to view the dashboard.

Option B is less ideal. While it technically grants the product manager access, creating an IAM user solely for dashboard viewing is an over-provisioning of access. It also necessitates managing another AWS user and distributing credentials, increasing administrative overhead and security risks. The product manager would gain access to the entire AWS console with read-only access to CloudWatch, exceeding the least privilege principle.

Option C is also unsuitable. The ViewOnlyAccess AWS managed policy is overly broad and doesn't specifically target CloudWatch. Sharing credentials among employees introduces significant security risks and hinders accountability. Sharing an IAM user's credentials violates AWS best practices and security guidelines.

Option D is the least efficient and most complex option. Deploying a bastion server just to view a CloudWatch dashboard is excessive and unnecessary. It involves managing infrastructure, configuring security groups, and sharing RDP credentials, adding significant operational overhead. It's also not the least privileged approach.

Therefore, option A directly addresses the requirement by using CloudWatch's built-in feature, adhering to the principle of least privilege, and minimizing operational overhead.

Relevant Documentation:

Sharing CloudWatch dashboards

Question: 28 **Exam Heist**

A company is migrating applications to AWS. The applications are deployed in different accounts. The company manages the accounts centrally by using AWS Organizations. The company's security team needs a single sign-on (SSO) solution across all the company's accounts. The company must continue managing the users and groups in

its on-premises self-managed Microsoft Active Directory.
Which solution will meet these requirements?

A. Enable AWS Single Sign-On (AWS SSO) from the AWS SSO console. Create a one-way forest trust or a one-way domain trust to connect the company's self-managed Microsoft Active Directory with AWS SSO by using AWS Directory Service for Microsoft Active Directory.

B. Enable AWS Single Sign-On (AWS SSO) from the AWS SSO console. Create a two-way forest trust to connect the company's self-managed Microsoft Active Directory with AWS SSO by using AWS Directory Service for Microsoft Active Directory.

C. Use AWS Directory Service. Create a two-way trust relationship with the company's self-managed Microsoft Active Directory.

D. Deploy an identity provider (IdP) on premises. Enable AWS Single Sign-On (AWS SSO) from the AWS SSO console.

Answer: B

Explanation:

The correct answer is **B**. Here's a detailed justification:

The requirement is to integrate an existing on-premises Microsoft Active Directory (AD) with AWS SSO to provide centralized single sign-on across multiple AWS accounts managed by AWS Organizations. To achieve this while maintaining user and group management within the on-premises AD, a trust relationship must be established between the on-premises AD and AWS.

AWS SSO doesn't directly integrate with self-managed AD. Instead, AWS Directory Service for Microsoft Active Directory (AWS Managed Microsoft AD) acts as an intermediary. AWS Managed Microsoft AD is a fully managed service allowing you to run actual Active Directory in AWS. To link your on-premises AD, you need to establish a trust relationship.

A **two-way forest trust** is the appropriate type of trust when you need users in both the on-premises AD forest and the AWS Managed Microsoft AD forest to authenticate to resources in the other forest. In this scenario, on-premises users need to access AWS resources, and potentially, AWS-based users (if you later create them) might need to access on-premises resources. Therefore, a two-way trust ensures seamless authentication in both directions. AWS SSO leverages this trust relationship via AWS Managed Microsoft AD to authenticate users from the on-premises AD. The AWS SSO console is used to enable and configure SSO across the AWS Organization.

Option A is incorrect because a one-way trust is insufficient. A one-way trust only allows authentication in one direction, failing to meet the requirement where AWS SSO needs to authenticate users against the on-premises AD.

Option C is incorrect because while creating a two-way trust relationship using AWS Directory Service is necessary, it doesn't by itself enable SSO. AWS SSO is the service required to centralize and manage single sign-on to multiple AWS accounts.

Option D is incorrect because deploying an on-premises IdP adds unnecessary complexity. AWS SSO is designed to integrate with existing identity providers, especially Microsoft Active Directory through AWS Directory Service. Using a third-party IdP on-premises duplicates functionality that AWS SSO already provides and increases the management overhead. Moreover, AWS SSO has direct integration capabilities with AWS Managed Microsoft AD, and no further IdP on-premises is needed.

In summary, creating a two-way forest trust between the on-premises Active Directory and AWS Directory Service for Microsoft Active Directory, then enabling and configuring AWS SSO, allows the organization to centralize SSO while maintaining user management in the on-premises environment.

Relevant Links:

AWS SSO: https://aws.amazon.com/single-sign-on/
AWS Directory Service: https://aws.amazon.com/directoryservice/
How AWS SSO works with Active Directory: https://docs.aws.amazon.com/singlesignon/latest/userguide/ad-connector.html
Trust Relationships: https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/plan/how-forest-trusts-work

Question: 29 **Exam Heist**

A company provides a Voice over Internet Protocol (VoIP) service that uses UDP connections. The service consists of Amazon EC2 instances that run in an Auto Scaling group. The company has deployments across multiple AWS Regions.
The company needs to route users to the Region with the lowest latency. The company also needs automated failover between Regions.
Which solution will meet these requirements?

A. Deploy a Network Load Balancer (NLB) and an associated target group. Associate the target group with the Auto Scaling group. Use the NLB as an AWS Global Accelerator endpoint in each Region.

B. Deploy an Application Load Balancer (ALB) and an associated target group. Associate the target group with the Auto Scaling group. Use the ALB as an AWS Global Accelerator endpoint in each Region.

C. Deploy a Network Load Balancer (NLB) and an associated target group. Associate the target group with the Auto Scaling group. Create an Amazon Route 53 latency record that points to aliases for each NLB. Create an Amazon CloudFront distribution that uses the latency record as an origin.

D. Deploy an Application Load Balancer (ALB) and an associated target group. Associate the target group with the Auto Scaling group. Create an Amazon Route 53 weighted record that points to aliases for each ALB. Deploy an Amazon CloudFront distribution that uses the weighted record as an origin.

Answer: A

Explanation:

The correct answer is A. Here's why:

UDP Requirement: The VoIP service relies on UDP connections. Application Load Balancers (ALB) only support HTTP and HTTPS protocols (TCP). Network Load Balancers (NLB) are designed to handle TCP, UDP, and TLS traffic. Therefore, NLB is the appropriate choice for this scenario.

Lowest Latency Routing and Automated Failover: AWS Global Accelerator leverages the AWS global network to route traffic to the optimal endpoint (closest Region) based on user location and network conditions, minimizing latency. It also provides automatic failover; if a Region becomes unavailable, Global Accelerator will automatically redirect traffic to a healthy Region.

Integration with Auto Scaling Groups: Both NLB and ALB can be integrated with Auto Scaling groups. By associating the target group with the Auto Scaling group, the load balancer automatically registers and deregisters instances as they are launched or terminated by the Auto Scaling group.

Route 53 Alternatives: While Route 53 latency-based routing can provide similar functionality, Global Accelerator offers performance advantages due to its use of the AWS global network and intelligent traffic routing. CloudFront is typically used for caching static content, not for routing UDP-based traffic based on latency.

Why other options are incorrect:

Option B: ALB is unsuitable because it does not support UDP.
Option C: Route 53 latency records do not provide the same level of optimized routing as Global Accelerator. Using CloudFront as a distribution for this purpose is unnecessary and not the best practice.

Option D: ALB is unsuitable because it does not support UDP. Route 53 weighted records are used to distribute traffic based on predefined weights, not real-time latency measurements.

Supporting Links:

AWS Global Accelerator: https://aws.amazon.com/global-accelerator/
Network Load Balancer: https://aws.amazon.com/elasticloadbalancing/network-load-balancer/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
Amazon Route 53: https://aws.amazon.com/route53/

Question: 30 **Exam Heist**

A development team runs monthly resource-intensive tests on its general purpose Amazon RDS for MySQL DB instance with Performance Insights enabled. The testing lasts for 48 hours once a month and is the only process that uses the database. The team wants to reduce the cost of running the tests without reducing the compute and memory attributes of the DB instance.
Which solution meets these requirements MOST cost-effectively?

A. Stop the DB instance when tests are completed. Restart the DB instance when required.

B. Use an Auto Scaling policy with the DB instance to automatically scale when tests are completed.

C. Create a snapshot when tests are completed. Terminate the DB instance and restore the snapshot when required.

D. Modify the DB instance to a low-capacity instance when tests are completed. Modify the DB instance again when required.

Answer: C

Explanation:

The most cost-effective solution is **C. Create a snapshot when tests are completed. Terminate the DB instance and restore the snapshot when required.**

Here's why:

Cost Optimization: The primary goal is cost reduction. Terminating the DB instance when it's not needed completely eliminates the compute costs associated with running the instance. AWS charges for RDS instances per hour/day they are running.
Preserving Data: Snapshots preserve the database state at the end of the tests. This ensures no data loss, and the team can easily restore the database to its previous state.
Compute & Memory Retention: Restoring from a snapshot creates a new DB instance with the same specifications (compute, memory) as the original. This preserves the performance characteristics required during the test.
Alternative A (Stop/Start): Stopping the instance saves on compute costs, but you still incur costs for storage (database volume) which won't be as significant as the cost of a running instance, but there is a cost. Stopping and starting instances can take a significant amount of time as well.
Alternative B (Auto Scaling): Auto Scaling doesn't apply to RDS instance types. Auto Scaling is for automatically scaling the number of EC2 instances in your application, not for vertically scaling an RDS instance.
Alternative D (Modify Instance): Modifying the instance size involves downtime and does not completely eliminate costs. While it reduces costs, it doesn't achieve the same level of savings as terminating the instance.
Performance Insights: Performance Insights data is associated with the DB instance. Terminating and recreating the instance will lead to losing data that the testing team may want to review later on. The team can export the insights before terminating the instance.
Restore Time: While restoring from a snapshot does take time, it's a reasonable trade-off for significant cost

savings, given the monthly testing frequency.

Therefore, creating a snapshot, terminating the instance, and restoring when needed provides the optimal balance between cost savings, data preservation, and performance retention.

Supporting Links:

Amazon RDS Pricing: https://aws.amazon.com/rds/pricing/
Creating a DB Snapshot:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_CreateSnapshot.html
Restoring from a DB Snapshot:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_RestoreFromSnapshot.html

Question: 31 **Exam Heist**

A company that hosts its web application on AWS wants to ensure all Amazon EC2 instances. Amazon RDS DB instances. and Amazon Redshift clusters are configured with tags. The company wants to minimize the effort of configuring and operating this check.
What should a solutions architect do to accomplish this?

A. Use AWS Config rules to define and detect resources that are not properly tagged.

B. Use Cost Explorer to display resources that are not properly tagged. Tag those resources manually.

C. Write API calls to check all resources for proper tag allocation. Periodically run the code on an EC2 instance.

D. Write API calls to check all resources for proper tag allocation. Schedule an AWS Lambda function through Amazon CloudWatch to periodically run the code.

Answer: A

Explanation:

The correct answer is A because AWS Config provides a managed service to assess, audit, and evaluate the configurations of your AWS resources. With AWS Config, you can define rules that check if your resources are compliant with desired configurations, including the presence of specific tags. When a resource is found to be non-compliant (missing tags in this case), AWS Config can flag it, allowing for easy identification and remediation. This minimizes the operational burden since AWS Config continuously monitors resource configurations without requiring custom code or manual checks.

Option B is incorrect because Cost Explorer is primarily used for cost management and visualization and does not provide a built-in mechanism for identifying resources without proper tags. While Cost Explorer can utilize tags for cost allocation, it won't proactively alert you to missing tags or enforce their existence. Tagging resources manually is a tedious and error-prone approach for an environment where continuous monitoring is required.

Options C and D involve writing custom API calls and managing infrastructure to execute them, either on an EC2 instance or through Lambda. While technically feasible, these approaches introduce operational overhead associated with code development, deployment, maintenance, and scaling. AWS Config offers a managed and declarative way to achieve the same goal, thus reducing complexity and operational effort. Lambda adds unnecessary complexity for this task as AWS Config Rules can be executed and maintained without code.

In summary, AWS Config's pre-built rules, continuous monitoring capabilities, and managed service nature make it the optimal solution for ensuring resources are properly tagged while minimizing effort and operational overhead.

Refer to the following AWS documentation for further understanding:

AWS Config: https://aws.amazon.com/config/
AWS Config Rules: https://docs.aws.amazon.com/config/latest/developerguide/evaluate-config.html

Question: 32 **Exam Heist**

A development team needs to host a website that will be accessed by other teams. The website contents consist of HTML, CSS, client-side JavaScript, and images.
Which method is the MOST cost-effective for hosting the website?

A. Containerize the website and host it in AWS Fargate.
B. Create an Amazon S3 bucket and host the website there.
C. Deploy a web server on an Amazon EC2 instance to host the website.
D. Configure an Application Load Balancer with an AWS Lambda target that uses the Express.js framework.

Answer: B

Explanation:

The most cost-effective solution for hosting a static website composed of HTML, CSS, JavaScript, and images is to use Amazon S3. S3 offers low storage costs and efficient content delivery through its integration with Amazon CloudFront.

Option A, containerizing the website and hosting it on AWS Fargate, involves running a container orchestration service, which is overkill for static content. Fargate is designed for applications that require compute resources, introducing unnecessary complexity and cost.

Option C, deploying a web server on an Amazon EC2 instance, incurs costs for the EC2 instance itself, including compute, storage, and potentially bandwidth. Maintaining and managing the EC2 instance also adds operational overhead. While EC2 provides flexibility, it's less cost-effective than S3 for static content.

Option D, configuring an Application Load Balancer with an AWS Lambda target using Express.js, is unnecessarily complex and expensive. This setup is suitable for dynamic content generation and serverless applications. Using Lambda for serving static files is inefficient and introduces additional latency. An Application Load Balancer is not necessary for static content delivery.

S3's static website hosting feature allows you to serve the files directly from an S3 bucket, eliminating the need for a web server. Furthermore, you can leverage CloudFront to cache the website content globally, reducing latency and further decreasing costs. S3 buckets also provide scalability and high availability by default. S3's pay-as-you-go pricing model makes it a very cost-effective option for this use case.

Therefore, the low cost, inherent scalability, and ease of use of Amazon S3 make it the most cost-effective solution for hosting a simple static website.

Relevant links:

Amazon S3 Static Website Hosting
Amazon CloudFront

Question: 33 **Exam Heist**

A company runs an online marketplace web application on AWS. The application serves hundreds of thousands of users during peak hours. The company needs a scalable, near-real-time solution to share the details of millions of financial transactions with several other internal applications. Transactions also need to be processed to remove sensitive data before being stored in a document database for low-latency retrieval.
What should a solutions architect recommend to meet these requirements?

A. Store the transactions data into Amazon DynamoDB. Set up a rule in DynamoDB to remove sensitive data from every transaction upon write. Use DynamoDB Streams to share the transactions data with other applications.

B. Stream the transactions data into Amazon Kinesis Data Firehose to store data in Amazon DynamoDB and Amazon S3. Use AWS Lambda integration with Kinesis Data Firehose to remove sensitive data. Other applications can consume the data stored in Amazon S3.

C. Stream the transactions data into Amazon Kinesis Data Streams. Use AWS Lambda integration to remove sensitive data from every transaction and then store the transactions data in Amazon DynamoDB. Other applications can consume the transactions data off the Kinesis data stream.

D. Store the batched transactions data in Amazon S3 as files. Use AWS Lambda to process every file and remove sensitive data before updating the files in Amazon S3. The Lambda function then stores the data in Amazon DynamoDB. Other applications can consume transaction files stored in Amazon S3.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best solution and why the others are less suitable:

Option C: Stream the transactions data into Amazon Kinesis Data Streams. Use AWS Lambda integration to remove sensitive data from every transaction and then store the transactions data in Amazon DynamoDB. Other applications can consume the transactions data off the Kinesis data stream.

This is the optimal choice because it leverages a combination of AWS services that are specifically designed for real-time data processing, transformation, and sharing.

1. **Real-time Ingestion:** Amazon Kinesis Data Streams is designed for ingesting and processing high-volume, real-time data streams. It is perfectly suited for capturing millions of financial transactions.
2. **Data Transformation:** The Lambda function provides a serverless and scalable way to remove sensitive data from each transaction as it flows through the Kinesis stream. This ensures compliance and data security before storage and sharing.
3. **Low-Latency Storage:** DynamoDB is a NoSQL database known for its low-latency read and write capabilities, making it ideal for storing the processed transactions and enabling fast retrieval.
4. **Data Sharing:** Other applications can subscribe to the Kinesis Data Stream and receive the transformed transactions data in near real-time. This provides a decoupled and scalable mechanism for sharing data with other internal applications.

Why other options are less optimal:

Option A: DynamoDB Streams are primarily intended for auditing or replicating data changes within DynamoDB itself, and not optimized for sharing data with a multitude of other applications in near real-time. Removing sensitive data using DynamoDB rules would happen after the transaction is already stored, possibly violating compliance requirements.
Option B: Kinesis Data Firehose is designed for loading data into data lakes or analytics services, and doesn't readily support direct consumption of data by other real-time applications. Using S3 as a central point can create latency.
Option D: Batching and storing in S3, processing with Lambda, and then storing in DynamoDB introduces significant latency. This approach isn't near real-time. Lambda processing S3 objects is more suited for larger files and infrequent updates, unlike this requirement.

In Conclusion: Option C provides a scalable, near-real-time, secure, and decoupled solution for processing financial transactions and sharing them with other applications, aligning perfectly with the requirements.

Authoritative Links:

Amazon Kinesis Data Streams: https://aws.amazon.com/kinesis/data-streams/
AWS Lambda: https://aws.amazon.com/lambda/

Amazon DynamoDB: https://aws.amazon.com/dynamodb/

Question: 34 **Exam Heist**

A company hosts its multi-tier applications on AWS. For compliance, governance, auditing, and security, the company must track configuration changes on its AWS resources and record a history of API calls made to these resources.
What should a solutions architect do to meet these requirements?

A. Use AWS CloudTrail to track configuration changes and AWS Config to record API calls.
B. Use AWS Config to track configuration changes and AWS CloudTrail to record API calls.
C. Use AWS Config to track configuration changes and Amazon CloudWatch to record API calls.
D. Use AWS CloudTrail to track configuration changes and Amazon CloudWatch to record API calls.

Answer: B

Explanation:

The correct answer is B: Use AWS Config to track configuration changes and AWS CloudTrail to record API calls. This is because AWS Config and AWS CloudTrail are designed for specific and distinct purposes related to compliance, governance, security, and auditing in the AWS cloud.

AWS Config continuously monitors and records AWS resource configurations, enabling you to automate the evaluation of recorded configurations against desired configurations. It provides a detailed view of the configuration of AWS resources, allowing you to assess, audit, and evaluate the configurations of your resources. AWS Config Rules can automatically check whether resources comply with defined standards. This is essential for tracking configuration changes. You can explore more about AWS Config at https://aws.amazon.com/config/.

AWS CloudTrail, on the other hand, tracks API calls made to AWS resources. It logs account activity, including actions taken through the AWS Management Console, AWS SDKs, command-line tools, and other AWS services. CloudTrail provides an event history of your AWS account activity, including who took what action, the resources that were acted upon, and when the event occurred. This is crucial for security analysis, resource change tracking, and compliance auditing. Comprehensive information about AWS CloudTrail can be found at https://aws.amazon.com/cloudtrail/.

Option A is incorrect because it reverses the roles of CloudTrail and Config. CloudTrail is not designed to track configuration changes directly, and Config is not designed to record API calls. Options C and D are incorrect as Amazon CloudWatch is primarily a monitoring service for metrics and logs, rather than a configuration tracking or API call recording service. While CloudWatch can be used to monitor logs generated by CloudTrail, it isn't a substitute for CloudTrail itself for API call logging. Therefore, AWS Config for configuration changes and AWS CloudTrail for API calls provides a robust solution for compliance, governance, auditing, and security as required by the company.

Question: 35 **Exam Heist**

A company is preparing to launch a public-facing web application in the AWS Cloud. The architecture consists of Amazon EC2 instances within a VPC behind an Elastic Load Balancer (ELB). A third-party service is used for the DNS. The company's solutions architect must recommend a solution to detect and protect against large-scale DDoS attacks.
Which solution meets these requirements?

A. Enable Amazon GuardDuty on the account.

B. Enable Amazon Inspector on the EC2 instances.
C. Enable AWS Shield and assign Amazon Route 53 to it.
D. Enable AWS Shield Advanced and assign the ELB to it.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct answer:

The problem requires protection against large-scale DDoS attacks for a public-facing web application. Let's analyze each option:

A. Enable Amazon GuardDuty on the account: GuardDuty is a threat detection service that monitors for malicious activity and unauthorized behavior. While it's good for security, it's not specifically designed for DDoS protection. It primarily detects threats after they've potentially impacted the application. It won't automatically mitigate large-scale DDoS attacks in real-time.

B. Enable Amazon Inspector on the EC2 instances: Inspector is a vulnerability assessment service that analyzes the security posture of EC2 instances. It identifies software vulnerabilities and unintended network configurations. While it's useful for general security, it doesn't directly address large-scale DDoS attacks at the network layer.

C. Enable AWS Shield and assign Amazon Route 53 to it: AWS Shield Standard is automatically enabled for all AWS customers and provides basic DDoS protection against common, frequently occurring network and transport layer DDoS attacks. While it's a good starting point, it doesn't offer the customized protection and advanced mitigation techniques required for large-scale, sophisticated DDoS attacks. Assigning Route 53 would protect the DNS layer, but the question focuses on the web application behind the ELB.

D. Enable AWS Shield Advanced and assign the ELB to it: AWS Shield Advanced provides enhanced DDoS protection tailored to your specific application. By assigning the ELB to Shield Advanced, you get 24/7 access to the AWS DDoS Response Team (DRT), customized protection rules, and cost protection during DDoS events. Shield Advanced can detect and automatically mitigate sophisticated DDoS attacks, ensuring the availability of your web application. Since the application is behind an ELB, protecting the ELB directly protects the underlying EC2 instances. This is the only option that provides comprehensive DDoS protection for the described architecture.

In summary, AWS Shield Advanced, specifically protecting the ELB, provides the best solution to detect and mitigate large-scale DDoS attacks targeted at the public-facing web application.

Authoritative Links:

AWS Shield: https://aws.amazon.com/shield/
AWS Shield Advanced: https://aws.amazon.com/shield/advanced/
Elastic Load Balancing: https://aws.amazon.com/elasticloadbalancing/
Amazon GuardDuty: https://aws.amazon.com/guardduty/
Amazon Inspector: https://aws.amazon.com/inspector/

Question: 36 **Exam Heist**

A company is building an application in the AWS Cloud. The application will store data in Amazon S3 buckets in two AWS Regions. The company must use an AWS Key Management Service (AWS KMS) customer managed key to encrypt all data that is stored in the S3 buckets. The data in both S3 buckets must be encrypted and decrypted with the same KMS key. The data and the key must be stored in each of the two Regions.
Which solution will meet these requirements with the LEAST operational overhead?

A. Create an S3 bucket in each Region. Configure the S3 buckets to use server-side encryption with Amazon S3 managed encryption keys (SSE-S3). Configure replication between the S3 buckets.

B. Create a customer managed multi-Region KMS key. Create an S3 bucket in each Region. Configure replication between the S3 buckets. Configure the application to use the KMS key with client-side encryption.

C. Create a customer managed KMS key and an S3 bucket in each Region. Configure the S3 buckets to use server-side encryption with Amazon S3 managed encryption keys (SSE-S3). Configure replication between the S3 buckets.

D. Create a customer managed KMS key and an S3 bucket in each Region. Configure the S3 buckets to use server-side encryption with AWS KMS keys (SSE-KMS). Configure replication between the S3 buckets.

Answer: B

Explanation:

The correct answer is **B**. Here's why:

B. Create a customer managed multi-Region KMS key. Create an S3 bucket in each Region. Configure replication between the S3 buckets. Configure the application to use the KMS key with client-side encryption.

This solution meets all the requirements with the least operational overhead:

1. **Customer Managed Key:** It utilizes a customer managed key, fulfilling the requirement to use a KMS key that the company controls. Critically, it uses a multi-Region KMS key.
2. **Same Key in Both Regions:** A multi-Region key ensures that the same logical key is available in both Regions for encryption and decryption. This is crucial for seamless data access regardless of which Region the data resides in.
3. **Data and Key Stored in Each Region:** Multi-Region keys guarantee the key material exists within both specified Regions, addressing the regional storage requirement.
4. **Encryption with KMS Key:** The solution specifies client-side encryption. This means the application encrypts the data before sending it to S3 using the multi-Region KMS key. This ensures encryption at rest using the KMS key, as the data is already encrypted when stored in S3.
5. **Least Operational Overhead:** This approach involves a relatively straightforward setup. The application manages encryption/decryption, which might add some development complexity initially, but it simplifies key management significantly compared to server-side encryption and key replication.

Let's examine why the other options are less suitable:

A & C: These options utilize SSE-S3. SSE-S3 uses encryption keys managed by AWS, failing the requirement to use a customer managed KMS key.

D: This solution proposes SSE-KMS with a KMS key. While it uses a customer managed key, it does not use a multi-Region key. Without a multi-Region key, replicating data between Regions encrypted with KMS becomes complex. You'd have to either:

Change the key used during replication, which adds significant operational overhead.
Re-encrypt the data after replication, again adding complexity.Furthermore, even if the same key was manually replicated to two different regions, the key ID would be different, and it would be treated as two distinct keys.

In summary, option B provides the most efficient and manageable way to meet all the requirements. It leverages the benefits of a multi-Region KMS key for seamless data encryption and decryption across both AWS Regions with minimal operational burden.

Supporting Documentation:

AWS KMS Multi-Region Keys: https://docs.aws.amazon.com/kms/latest/developerguide/multi-region-keys-overview.html
Amazon S3 Encryption: https://docs.aws.amazon.com/AmazonS3/latest/userguide/serv-side-encryption.html

Question: 37 **Exam Heist**

A company recently launched a variety of new workloads on Amazon EC2 instances in its AWS account. The company needs to create a strategy to access and administer the instances remotely and securely. The company needs to implement a repeatable process that works with native AWS services and follows the AWS Well-Architected Framework.
Which solution will meet these requirements with the LEAST operational overhead?

A. Use the EC2 serial console to directly access the terminal interface of each instance for administration.

B. Attach the appropriate IAM role to each existing instance and new instance. Use AWS Systems Manager Session Manager to establish a remote SSH session.

C. Create an administrative SSH key pair. Load the public key into each EC2 instance. Deploy a bastion host in a public subnet to provide a tunnel for administration of each instance.

D. Establish an AWS Site-to-Site VPN connection. Instruct administrators to use their local on-premises machines to connect directly to the instances by using SSH keys across the VPN tunnel.

Answer: B

Explanation:

The correct answer is **B**. Here's why:

B. Attach the appropriate IAM role to each existing instance and new instance. Use AWS Systems Manager Session Manager to establish a remote SSH session.

This solution is the most aligned with the prompt's requirements of security, repeatability, native AWS services, and minimal operational overhead. AWS Systems Manager (SSM) Session Manager allows you to securely manage EC2 instances without needing to open inbound ports (like SSH 22) or manage SSH keys. By using IAM roles, you can precisely define the permissions granted to SSM, ensuring only authorized users and instances can initiate sessions. This approach minimizes the attack surface and reduces the risk of unauthorized access. SSM Session Manager also integrates with AWS CloudTrail for auditing purposes, providing a clear record of all session activities. Furthermore, it leverages the AWS global infrastructure and eliminates the need to manage bastion hosts or VPNs, thus minimizing operational overhead. The "repeatable process" is achieved by consistently applying the same IAM roles across all instances.

Here's a breakdown of why the other options are less ideal:

A. Use the EC2 serial console to directly access the terminal interface of each instance for administration. While useful for troubleshooting boot issues, the serial console isn't intended for routine administration. It lacks the auditability and security features of SSM Session Manager and requires direct AWS Management Console access. It also does not scale well for a large number of instances.

C. Create an administrative SSH key pair. Load the public key into each EC2 instance. Deploy a bastion host in a public subnet to provide a tunnel for administration of each instance. This solution introduces significant operational overhead. It requires managing SSH keys, securing the bastion host, and configuring routing rules. Maintaining the security and availability of a bastion host adds complexity and ongoing management. SSH keys also present a security risk if compromised.

D. Establish an AWS Site-to-Site VPN connection. Instruct administrators to use their local on-premises machines to connect directly to the instances by using SSH keys across the VPN tunnel. This introduces

reliance on on-premises infrastructure and the VPN connection. It increases complexity, maintenance overhead, and network latency. Additionally, managing SSH keys and distributing them securely to administrators becomes an operational burden and security risk.

In summary, **Option B** leverages native AWS services (IAM and SSM) to provide secure, auditable, and centrally managed access to EC2 instances with minimal operational burden, which is a best practice according to the AWS Well-Architected Framework.

Supporting Links:

AWS Systems Manager Session Manager: https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html
AWS IAM Roles: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html
AWS Well-Architected Framework: https://aws.amazon.com/architecture/well-architected/

Question: 38 **Exam Heist**

A company is hosting a static website on Amazon S3 and is using Amazon Route 53 for DNS. The website is experiencing increased demand from around the world. The company must decrease latency for users who access the website.
Which solution meets these requirements MOST cost-effectively?

A. Replicate the S3 bucket that contains the website to all AWS Regions. Add Route 53 geolocation routing entries.

B. Provision accelerators in AWS Global Accelerator. Associate the supplied IP addresses with the S3 bucket. Edit the Route 53 entries to point to the IP addresses of the accelerators.

C. Add an Amazon CloudFront distribution in front of the S3 bucket. Edit the Route 53 entries to point to the CloudFront distribution.

D. Enable S3 Transfer Acceleration on the bucket. Edit the Route 53 entries to point to the new endpoint.

Answer: C

Explanation:

Here's a detailed justification for why option C is the most cost-effective solution for reducing latency for a static website hosted on S3 using Route 53, while dealing with increased global demand:

Option C, adding an Amazon CloudFront distribution in front of the S3 bucket and updating Route 53 records, leverages the power of Content Delivery Networks (CDNs). CloudFront caches the static website content at edge locations around the world. When a user requests content, CloudFront serves it from the nearest edge location, significantly reducing latency compared to fetching it from the origin S3 bucket, which might be geographically distant. This caching mechanism distributes the load and minimizes the impact of increased global demand on the origin server. CloudFront's pay-as-you-go pricing model makes it cost-effective as you only pay for the data transferred and requests served.

Option A is less cost-effective. Replicating the S3 bucket to all AWS Regions is an overkill for a static website. Moreover, managing replication across all regions and configuring Route 53 geolocation routing can be complex and expensive without a significant value add. While geolocation routing would direct users to the closest replica, the replication costs would likely be excessive.

Option B, using AWS Global Accelerator, is more appropriate for dynamic content or applications that require TCP or UDP connections. While it could improve performance, it's designed for accelerating dynamic content and is generally more expensive than CloudFront for static content. Global Accelerator also introduces complexity in terms of provisioning and managing accelerators.

Option D, enabling S3 Transfer Acceleration, improves the speed of uploads and downloads to S3 using

optimized network paths. While beneficial for data transfer to S3, it does not directly address the need to reduce latency for users accessing the static website content. It speeds up transfers to S3, but users still retrieve data directly from the bucket unless a CDN is used. Therefore, it is not suitable.

In summary, CloudFront offers the most efficient and cost-effective solution for caching static content and delivering it to users with low latency globally, perfectly aligning with the requirements of the question.

Relevant links for further research:

Amazon CloudFront: https://aws.amazon.com/cloudfront/
Amazon S3: https://aws.amazon.com/s3/
Amazon Route 53: https://aws.amazon.com/route53/
AWS Global Accelerator: https://aws.amazon.com/global-accelerator/
S3 Transfer Acceleration: https://aws.amazon.com/s3/transfer-acceleration/

Question: 39 **Exam Heist**

A company maintains a searchable repository of items on its website. The data is stored in an Amazon RDS for MySQL database table that contains more than 10 million rows. The database has 2 TB of General Purpose SSD storage. There are millions of updates against this data every day through the company's website.
The company has noticed that some insert operations are taking 10 seconds or longer. The company has determined that the database storage performance is the problem.
Which solution addresses this performance issue?

A. Change the storage type to Provisioned IOPS SSD.
B. Change the DB instance to a memory optimized instance class.
C. Change the DB instance to a burstable performance instance class.
D. Enable Multi-AZ RDS read replicas with MySQL native asynchronous replication.

Answer: A

Explanation:

The question highlights slow insert operations in an Amazon RDS for MySQL database due to database storage performance limitations. General Purpose SSD (gp2/gp3) storage, while suitable for many workloads, might not be sufficient for high-write workloads with millions of daily updates and large tables. Option A, changing the storage type to Provisioned IOPS SSD (io1/io2), directly addresses the performance bottleneck. Provisioned IOPS SSD allows you to specify a consistent IOPS (Input/Output Operations Per Second) rate, guaranteeing a predictable performance level. This is crucial for applications requiring low latency and high throughput, especially those experiencing latency due to storage I/O.

Option B, changing the DB instance to a memory-optimized instance class, primarily addresses CPU and memory limitations, not storage performance. While more memory can improve caching, it doesn't solve underlying storage bottlenecks. Option C, changing to a burstable performance instance class, is unsuitable because burstable instances rely on credit accumulation and can experience performance degradation when credits are exhausted under sustained high workloads like millions of daily updates. Option D, enabling Multi-AZ RDS read replicas, improves read scalability and high availability but doesn't address the write performance issues on the primary database instance causing the slow insert operations. Replication adds overhead to the primary instance.

Therefore, switching to Provisioned IOPS SSD is the optimal solution as it provides consistent and guaranteed I/O performance, mitigating the latency issues related to storage operations. For further reading:

Amazon RDS Storage Types:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_Storage.html

Provisioned IOPS SSD (io1/io2): https://aws.amazon.com/rds/pricing/ (Refer to the storage section)

Question: 40 **Exam Heist**

A company has thousands of edge devices that collectively generate 1 TB of status alerts each day. Each alert is approximately 2 KB in size. A solutions architect needs to implement a solution to ingest and store the alerts for future analysis.
The company wants a highly available solution. However, the company needs to minimize costs and does not want to manage additional infrastructure. Additionally, the company wants to keep 14 days of data available for immediate analysis and archive any data older than 14 days.
What is the MOST operationally efficient solution that meets these requirements?

A. Create an Amazon Kinesis Data Firehose delivery stream to ingest the alerts. Configure the Kinesis Data Firehose stream to deliver the alerts to an Amazon S3 bucket. Set up an S3 Lifecycle configuration to transition data to Amazon S3 Glacier after 14 days.

B. Launch Amazon EC2 instances across two Availability Zones and place them behind an Elastic Load Balancer to ingest the alerts. Create a script on the EC2 instances that will store the alerts in an Amazon S3 bucket. Set up an S3 Lifecycle configuration to transition data to Amazon S3 Glacier after 14 days.

C. Create an Amazon Kinesis Data Firehose delivery stream to ingest the alerts. Configure the Kinesis Data Firehose stream to deliver the alerts to an Amazon OpenSearch Service (Amazon Elasticsearch Service) cluster. Set up the Amazon OpenSearch Service (Amazon Elasticsearch Service) cluster to take manual snapshots every day and delete data from the cluster that is older than 14 days.

D. Create an Amazon Simple Queue Service (Amazon SQS) standard queue to ingest the alerts, and set the message retention period to 14 days. Configure consumers to poll the SQS queue, check the age of the message, and analyze the message data as needed. If the message is 14 days old, the consumer should copy the message to an Amazon S3 bucket and delete the message from the SQS queue.

Answer: A

Explanation:

The most operationally efficient solution is A because it leverages fully managed services that require minimal infrastructure management and are cost-effective.

Why A is the best solution:

Kinesis Data Firehose: It is a fully managed service designed for streaming data ingestion into destinations like S3. It handles scaling, buffering, and data transformation (if needed) automatically, removing the operational overhead of managing servers or clusters.
Amazon S3: It provides highly durable and scalable storage. The pay-as-you-go model and low cost makes it cost-effective.
S3 Lifecycle Management: This feature automates the process of transitioning data between different storage tiers. Configuring a lifecycle policy to move data to Glacier after 14 days automatically handles archiving, reducing storage costs for older data without manual intervention.

Why other options are less suitable:

B: Involves manually managing EC2 instances and load balancers, which increases operational overhead. Scripting the data storage process also requires manual configuration and maintenance.
C: Amazon OpenSearch Service (successor to Elasticsearch Service) is not the best choice for long-term archival storage, as it is more suited for search and analysis of recent data. OpenSearch Service can be expensive.
D: SQS is not designed for storing large amounts of data long-term. Consumers would need to actively process the data as it arrives and manage the archival process. This approach introduces complexity and potential points of failure and it does not allow the full 1TB of daily alerts to be retained in SQS due to the limits of its data retention policies. Also, SQS's limit message size of 256KB makes it unsuitable for 2KB status alerts if thousands of edge devices are sending them.

Supporting Concepts:

Serverless Computing: Kinesis Data Firehose and S3 are serverless services, which minimize the operational burden.
Data Lifecycle Management: S3 Lifecycle Policies are designed to automate data tiering based on age or other criteria, optimizing costs and storage management.
Cost Optimization: By using S3 Glacier for long-term archival, the solution minimizes storage costs.

Authoritative Links:

Amazon Kinesis Data Firehose: https://aws.amazon.com/kinesis/data-firehose/
Amazon S3 Lifecycle Management: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-concept.html
Amazon S3 Glacier: https://aws.amazon.com/glacier/

Question: 41 **Exam Heist**

A company's application integrates with multiple software-as-a-service (SaaS) sources for data collection. The company runs Amazon EC2 instances to receive the data and to upload the data to an Amazon S3 bucket for analysis. The same EC2 instance that receives and uploads the data also sends a notification to the user when an upload is complete. The company has noticed slow application performance and wants to improve the performance as much as possible.
Which solution will meet these requirements with the LEAST operational overhead?

A. Create an Auto Scaling group so that EC2 instances can scale out. Configure an S3 event notification to send events to an Amazon Simple Notification Service (Amazon SNS) topic when the upload to the S3 bucket is complete.

B. Create an Amazon AppFlow flow to transfer data between each SaaS source and the S3 bucket. Configure an S3 event notification to send events to an Amazon Simple Notification Service (Amazon SNS) topic when the upload to the S3 bucket is complete.

C. Create an Amazon EventBridge (Amazon CloudWatch Events) rule for each SaaS source to send output data. Configure the S3 bucket as the rule's target. Create a second EventBridge (Cloud Watch Events) rule to send events when the upload to the S3 bucket is complete. Configure an Amazon Simple Notification Service (Amazon SNS) topic as the second rule's target.

D. Create a Docker container to use instead of an EC2 instance. Host the containerized application on Amazon Elastic Container Service (Amazon ECS). Configure Amazon CloudWatch Container Insights to send events to an Amazon Simple Notification Service (Amazon SNS) topic when the upload to the S3 bucket is complete.

Answer: B

Explanation:

The most efficient solution to improve application performance with minimal operational overhead is to use Amazon AppFlow for data transfer and S3 event notifications with SNS.

Option B is the best because:

AppFlow for Data Transfer: AppFlow directly transfers data from SaaS applications to S3 without the need for EC2 instances. This eliminates the EC2 instance bottleneck and reduces the operational burden of managing those instances. AppFlow is specifically designed for SaaS data integration and automates much of the process. https://aws.amazon.com/appflow/
S3 Event Notifications & SNS: S3 event notifications trigger an SNS topic when an upload completes. This is a serverless and highly scalable approach to notify users without burdening the data transfer process. SNS provides a simple and reliable way to publish messages to subscribers. https://aws.amazon.com/sns/

Why other options are not optimal:

Option A (Auto Scaling): Scaling EC2 instances might improve throughput, but it doesn't address the fundamental inefficiency of using EC2 for simple data transfer from SaaS sources. It also increases operational complexity.
Option C (EventBridge): EventBridge can handle events, but using it directly for SaaS data integration isn't its primary purpose. It's better suited for routing events within AWS services, not directly interfacing with external SaaS applications. AppFlow is purpose-built for SaaS integrations. Furthermore, using EventBridge as a target to directly write into S3 is not a best practice.
Option D (Docker/ECS): Containerizing the application doesn't fundamentally change the data transfer bottleneck. EC2 (or ECS) is still involved in receiving and uploading data. Also, CloudWatch Container Insights is for monitoring container performance, not for triggering notifications based on S3 uploads.

In conclusion, the solution involving AppFlow and S3 event notifications/SNS offers the most direct and efficient way to address the slow application performance while minimizing operational overhead. This aligns with the principle of using managed services to reduce manual intervention and improve scalability.

Question: 42 **Exam Heist**

A company runs a highly available image-processing application on Amazon EC2 instances in a single VPC. The EC2 instances run inside several subnets across multiple Availability Zones. The EC2 instances do not communicate with each other. However, the EC2 instances download images from Amazon S3 and upload images to Amazon S3 through a single NAT gateway. The company is concerned about data transfer charges.
What is the MOST cost-effective way for the company to avoid Regional data transfer charges?

A. Launch the NAT gateway in each Availability Zone.
B. Replace the NAT gateway with a NAT instance.
C. Deploy a gateway VPC endpoint for Amazon S3.
D. Provision an EC2 Dedicated Host to run the EC2 instances.

Answer: C

Explanation:

The question is focused on minimizing Regional data transfer costs for EC2 instances downloading from and uploading to S3 through a NAT gateway.

Option C: Deploy a gateway VPC endpoint for Amazon S3 is the most cost-effective solution. Gateway endpoints provide connectivity to S3 without using the internet gateway or NAT gateway. This avoids data transfer charges associated with routing traffic through the NAT gateway. Data transferred between EC2 instances in the VPC and S3 via the gateway endpoint stays within the AWS network, eliminating Regional data transfer charges.

Option A: Launch the NAT gateway in each Availability Zone does not avoid Regional data transfer charges, although it improves availability. Data still traverses the NAT gateway, and the associated costs remain. It primarily prevents a single NAT gateway failure from impacting all AZs.

Option B: Replace the NAT gateway with a NAT instance offers no cost benefit and introduces operational overhead. NAT instances are not managed by AWS and require manual configuration, patching, and scaling. Moreover, data transfer charges still apply.

Option D: Provision an EC2 Dedicated Host to run the EC2 instances is irrelevant to the problem. Dedicated Hosts provide hardware-level isolation but do not affect data transfer charges associated with S3 access.

Therefore, leveraging a gateway VPC endpoint is the most efficient way to eliminate Regional data transfer charges for S3 access from EC2 instances within a VPC.

Supporting resources:

VPC Endpoints
NAT Gateways
Understanding AWS Data Transfer Costs

Question: 43 **Exam Heist**

A company has an on-premises application that generates a large amount of time-sensitive data that is backed up to Amazon S3. The application has grown and there are user complaints about internet bandwidth limitations. A solutions architect needs to design a long-term solution that allows for both timely backups to Amazon S3 and with minimal impact on internet connectivity for internal users.
Which solution meets these requirements?

A. Establish AWS VPN connections and proxy all traffic through a VPC gateway endpoint.

B. Establish a new AWS Direct Connect connection and direct backup traffic through this new connection.

C. Order daily AWS Snowball devices. Load the data onto the Snowball devices and return the devices to AWS each day.

D. Submit a support ticket through the AWS Management Console. Request the removal of S3 service limits from the account.

Answer: B

Explanation:

The best solution is **B. Establish a new AWS Direct Connect connection and direct backup traffic through this new connection.**

Here's why:

Bandwidth Bottleneck: The core issue is limited internet bandwidth affecting users due to large data backups to S3.
AWS Direct Connect: Direct Connect establishes a dedicated, private network connection between your on-premises infrastructure and AWS. This bypasses the public internet, providing consistent and often higher bandwidth, lower latency, and more predictable network performance.
Minimal Impact on Internet: By routing the backup traffic through Direct Connect, it won't compete for bandwidth with other internet-bound traffic, alleviating the strain on internal users' connectivity.
Timely Backups: The dedicated connection ensures backups can complete efficiently and on time, as it's not subject to internet congestion.
Long-term Solution: Direct Connect is a durable infrastructure solution, ideal for continuous data transfer needs.

Why other options are less ideal:

A. AWS VPN and VPC Gateway Endpoint: While VPN provides secure connectivity, it still relies on the existing internet bandwidth. VPC gateway endpoints allow access to S3 from within a VPC without using public IPs, but the data still travels over the internet to the on-premises network. It does not solve the bandwidth issue.
C. AWS Snowball: Snowball is useful for initial large-scale data migrations. Using it daily for backups is impractical, expensive, and introduces significant delays in data availability.
D. Support Ticket for S3 Limits: S3 service limits are designed for the overall health and availability of the service, not for solving bandwidth problems. Requesting their removal is unlikely to be successful and isn't a proper architectural solution.

Authoritative Links:

AWS Direct Connect: https://aws.amazon.com/directconnect/
AWS VPN: https://aws.amazon.com/vpn/
AWS Snowball: https://aws.amazon.com/snowball/

Question: 44 **Exam Heist**

A company has an Amazon S3 bucket that contains critical data. The company must protect the data from accidental deletion.
Which combination of steps should a solutions architect take to meet these requirements? (Choose two.)

A. Enable versioning on the S3 bucket.
B. Enable MFA Delete on the S3 bucket.
C. Create a bucket policy on the S3 bucket.
D. Enable default encryption on the S3 bucket.
E. Create a lifecycle policy for the objects in the S3 bucket.

Answer: AB

Explanation:

The correct answer is AB.

Justification:

The primary requirement is to protect the data from accidental deletion. Two features in S3 directly address this: Versioning and MFA Delete.

A. Enable versioning on the S3 bucket: S3 Versioning ensures that every version of an object is preserved, even if it's deleted or overwritten. When an object is deleted, it doesn't truly disappear; instead, a delete marker is created. The previous version remains accessible, allowing for easy recovery from accidental deletions. This is crucial for data protection.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/versioning-workflows.html

B. Enable MFA Delete on the S3 bucket: MFA Delete adds an extra layer of security to prevent accidental or malicious deletions. When MFA Delete is enabled, deleting a versioned object or permanently deleting the S3 bucket itself requires multi-factor authentication. This significantly reduces the risk of unauthorized or unintentional data loss. It enforces the principle of "something you know" (password) and "something you have" (MFA device), strengthening the deletion process.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html#MultiFactorAuthenticationDelete

Let's examine why the other options are incorrect:

C. Create a bucket policy on the S3 bucket: While bucket policies control access to the bucket, they don't prevent accidental deletions. You can restrict who can delete objects, but if someone with the delete permission makes a mistake, the policy won't prevent it.

D. Enable default encryption on the S3 bucket: Default encryption protects data at rest, addressing security and compliance concerns related to data confidentiality. However, encryption does not prevent accidental deletion.

E. Create a lifecycle policy for the objects in the S3 bucket: Lifecycle policies manage the object lifecycle, automatically transitioning them to cheaper storage classes or deleting them after a specified period. If misconfigured, a lifecycle policy could actually cause accidental deletions. It's for automating data archival/deletion based on pre-defined rules, not for general protection against accidental deletion.

Question: 45 **Exam Heist**

A company has a data ingestion workflow that consists of the following:
• An Amazon Simple Notification Service (Amazon SNS) topic for notifications about new data deliveries
• An AWS Lambda function to process the data and record metadata
The company observes that the ingestion workflow fails occasionally because of network connectivity issues. When such a failure occurs, the Lambda function does not ingest the corresponding data unless the company manually reruns the job.
Which combination of actions should a solutions architect take to ensure that the Lambda function ingests all data in the future? (Choose two.)

A. Deploy the Lambda function in multiple Availability Zones.

B. Create an Amazon Simple Queue Service (Amazon SQS) queue, and subscribe it to the SNS topic.

C. Increase the CPU and memory that are allocated to the Lambda function.

D. Increase provisioned throughput for the Lambda function.

E. Modify the Lambda function to read from an Amazon Simple Queue Service (Amazon SQS) queue.

Answer: BE

Explanation:

The correct answer is **BE**. Here's why:

B. Create an Amazon Simple Queue Service (Amazon SQS) queue, and subscribe it to the SNS topic.

This is a crucial step for building a reliable and resilient data ingestion pipeline. When an SNS topic is connected to an SQS queue via subscription, messages published to the SNS topic are automatically queued in the SQS queue. SQS provides a mechanism to buffer the messages, ensuring that they are not lost even if the Lambda function is temporarily unavailable due to network issues or other transient failures. This buffering helps decoupling the data producers (SNS) from the data consumers (Lambda).

E. Modify the Lambda function to read from an Amazon Simple Queue Service (Amazon SQS) queue.

By having the Lambda function consume messages from the SQS queue, the function becomes more fault-tolerant. If a processing failure occurs during the initial Lambda execution (due to network issue), the message remains in the SQS queue. SQS can be configured to automatically retry the message delivery to the Lambda function after a visibility timeout. This ensures that the Lambda function eventually processes all messages, providing at-least-once delivery semantics. The queue acts as a buffer and retry mechanism, guaranteeing data ingestion despite intermittent failures.https://docs.aws.amazon.com/lambda/latest/dg/services-sqs.html

Why other options are not suitable:

A. Deploy the Lambda function in multiple Availability Zones. While deploying Lambda in multiple AZs improves availability, it doesn't address message loss caused by transient network issues during the initial invocation. Lambda inherently runs in multiple AZs managed by AWS.

C. Increase the CPU and memory that are allocated to the Lambda function. Increasing CPU and memory might help with performance if the Lambda function is resource-constrained, but it won't prevent message loss due to network issues.

D. Increase provisioned throughput for the Lambda function. Lambda does not use provisioned throughput, and this action is not applicable. Also, it does not prevent message loss due to network issues.

Question: 46 **Exam Heist**

A company has an application that provides marketing services to stores. The services are based on previous purchases by store customers. The stores upload transaction data to the company through SFTP, and the data is processed and analyzed to generate new marketing offers. Some of the files can exceed 200 GB in size.
Recently, the company discovered that some of the stores have uploaded files that contain personally identifiable information (PII) that should not have been included. The company wants administrators to be alerted if PII is shared again. The company also wants to automate remediation.
What should a solutions architect do to meet these requirements with the LEAST development effort?

A. Use an Amazon S3 bucket as a secure transfer point. Use Amazon Inspector to scan the objects in the bucket. If objects contain PII, trigger an S3 Lifecycle policy to remove the objects that contain PII.

B. Use an Amazon S3 bucket as a secure transfer point. Use Amazon Macie to scan the objects in the bucket. If objects contain PII, use Amazon Simple Notification Service (Amazon SNS) to trigger a notification to the administrators to remove the objects that contain PII.

C. Implement custom scanning algorithms in an AWS Lambda function. Trigger the function when objects are loaded into the bucket. If objects contain PII, use Amazon Simple Notification Service (Amazon SNS) to trigger a notification to the administrators to remove the objects that contain PII.

D. Implement custom scanning algorithms in an AWS Lambda function. Trigger the function when objects are loaded into the bucket. If objects contain PII, use Amazon Simple Email Service (Amazon SES) to trigger a notification to the administrators and trigger an S3 Lifecycle policy to remove the meats that contain PII.

Answer: B

Explanation:

The correct answer is B because it leverages purpose-built AWS services for data security and PII detection with minimal development effort. Here's a detailed justification:

S3 as a Secure Transfer Point: Using S3 provides a scalable and secure storage location for the uploaded files, replacing the potentially less secure SFTP method. S3 integrates well with other AWS services, simplifying the overall workflow.

Amazon Macie for PII Detection: Amazon Macie is a fully managed data security and data privacy service that uses machine learning and pattern matching to discover sensitive data, including PII. Macie is designed specifically for this purpose and eliminates the need to develop custom scanning algorithms, adhering to the principle of "least development effort". It can automatically detect a wide range of PII types.

SNS for Notifications: Amazon SNS is a simple and cost-effective way to send notifications to administrators when PII is detected. This allows for immediate action and investigation.

Why other options are less suitable:

Option A suggests using Amazon Inspector, which primarily focuses on identifying security vulnerabilities and deviations from security best practices within EC2 instances and container images, and doesn't specialize in PII detection within S3 objects like Macie. Also S3 Lifecycle policy removing without investigation is not ideal.
Options C and D propose implementing custom scanning algorithms in Lambda. Developing and maintaining custom scanning algorithms would require significant development effort and expertise in PII identification, which contradicts the "least development effort" requirement. Option D uses SES which is similar to SNS, however, SES requires more configuration and isn't as straightforward for simple notification use cases. Option D also suggests removing "meats" which is not an option from the question.

In summary, option B provides a pre-built and managed solution for PII detection and notification, ensuring minimal development effort and a robust security posture.

Supporting Links:

Amazon Macie: https://aws.amazon.com/macie/
Amazon S3: https://aws.amazon.com/s3/

Amazon SNS: https://aws.amazon.com/sns/

Question: 47 **Exam Heist**

A company needs guaranteed Amazon EC2 capacity in three specific Availability Zones in a specific AWS Region for an upcoming event that will last 1 week.
What should the company do to guarantee the EC2 capacity?

A. Purchase Reserved Instances that specify the Region needed.
B. Create an On-Demand Capacity Reservation that specifies the Region needed.
C. Purchase Reserved Instances that specify the Region and three Availability Zones needed.
D. Create an On-Demand Capacity Reservation that specifies the Region and three Availability Zones needed.

Answer: D

Explanation:

The correct answer is D: Create an On-Demand Capacity Reservation that specifies the Region and three Availability Zones needed.

Here's why:

On-Demand Capacity Reservations (ODCR): ODCRs provide a way to reserve EC2 instance capacity in a specific Availability Zone for a specified duration. This guarantees that the required EC2 capacity will be available when needed. Crucially, they allow you to target specific Availability Zones.

AWS Documentation on Capacity Reservations
Reserved Instances (RI): While RIs offer a billing discount in exchange for a term commitment, they do not guarantee capacity. They provide discounted pricing for instances used but do not explicitly reserve resources in specific Availability Zones. RIs help lower costs, they do not fulfill the requirement for guaranteed capacity. Regional RIs apply to any AZ in the region.

AWS Documentation on Reserved Instances

Why other options are incorrect:

A & C: Purchasing Reserved Instances: As stated above, RIs do not guarantee EC2 capacity. Purchasing RIs only reduces the cost of running instances; it doesn't reserve physical resources.
B: Create an On-Demand Capacity Reservation that specifies the Region needed: This is not precise enough. The company requires capacity reservations in three specific Availability Zones. Specifying only the region would not guarantee the resources would be in the desired AZs.
Given the short duration (1 week) and the need for a guarantee the On-Demand Capacity Reservation is a perfect match.

Question: 48 **Exam Heist**

A company's website uses an Amazon EC2 instance store for its catalog of items. The company wants to make sure that the catalog is highly available and that the catalog is stored in a durable location.
What should a solutions architect do to meet these requirements?

A. Move the catalog to Amazon ElastiCache for Redis.
B. Deploy a larger EC2 instance with a larger instance store.
C. Move the catalog from the instance store to Amazon S3 Glacier Deep Archive.
D. Move the catalog to an Amazon Elastic File System (Amazon EFS) file system.

Answer: D

Explanation:

The correct answer is D. Here's why:

The question highlights two key requirements: high availability and durable storage for the website's catalog. Instance store volumes are ephemeral, meaning data is lost when the instance stops, terminates, or fails. This makes them unsuitable for durable storage.

Option A, moving to ElastiCache for Redis, isn't ideal for durable storage of a full catalog. Redis is an in-memory data store, primarily used for caching to improve performance, not long-term storage. While data persistence can be configured, it's not the primary use case and introduces complexity compared to other options.

Option B, deploying a larger EC2 instance with a larger instance store, doesn't address the fundamental problem of data loss upon instance failure. It merely provides more ephemeral storage, failing to meet the durability requirement.

Option C, moving to S3 Glacier Deep Archive, provides durable storage but is designed for infrequent access, like archiving. It's unsuitable for a website catalog that requires frequent reads and writes. Retrieval times are measured in hours, making it impractical for serving web content.

Option D, moving to Amazon EFS, is the best solution. EFS provides a scalable, highly available, and durable file system that can be mounted by multiple EC2 instances simultaneously. This allows the catalog to be stored persistently and accessed by the web servers, ensuring high availability and durability. EFS replicates data across multiple Availability Zones, providing resilience against failures.

In summary, EFS addresses both requirements: high availability through shared access and data replication, and durability through persistent storage. It's designed for file-based storage that can be easily accessed by applications, making it a good fit for a website catalog.

Further Reading:

Amazon EFS: https://aws.amazon.com/efs/
Instance Store Volumes: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/InstanceStorage.html

Question: 49 **Exam Heist**

A company stores call transcript files on a monthly basis. Users access the files randomly within 1 year of the call, but users access the files infrequently after 1 year. The company wants to optimize its solution by giving users the ability to query and retrieve files that are less than 1-year-old as quickly as possible. A delay in retrieving older files is acceptable.
Which solution will meet these requirements MOST cost-effectively?

A. Store individual files with tags in Amazon S3 Glacier Instant Retrieval. Query the tags to retrieve the files from S3 Glacier Instant Retrieval.

B. Store individual files in Amazon S3 Intelligent-Tiering. Use S3 Lifecycle policies to move the files to S3 Glacier Flexible Retrieval after 1 year. Query and retrieve the files that are in Amazon S3 by using Amazon Athena. Query and retrieve the files that are in S3 Glacier by using S3 Glacier Select.

C. Store individual files with tags in Amazon S3 Standard storage. Store search metadata for each archive in Amazon S3 Standard storage. Use S3 Lifecycle policies to move the files to S3 Glacier Instant Retrieval after 1 year. Query and retrieve the files by searching for metadata from Amazon S3.

D. Store individual files in Amazon S3 Standard storage. Use S3 Lifecycle policies to move the files to S3 Glacier Deep Archive after 1 year. Store search metadata in Amazon RDS. Query the files from Amazon RDS. Retrieve the files from S3 Glacier Deep Archive.

Answer: B

Explanation:

Option B is the most cost-effective solution because it leverages S3 Intelligent-Tiering to automatically optimize storage costs based on access patterns, while also utilizing S3 Glacier for long-term archival.

Here's a breakdown:

S3 Intelligent-Tiering: This storage class automatically moves data between frequent, infrequent, and archive access tiers based on changing access patterns. This means frequently accessed files (within the first year) reside in the more expensive but faster access tiers, ensuring quick retrieval. Infrequently accessed files are moved to cheaper tiers automatically.

S3 Lifecycle Policies: These policies automate the transition of objects between storage classes. By moving files older than 1 year to S3 Glacier Flexible Retrieval (formerly S3 Glacier), the company benefits from significantly lower storage costs for infrequently accessed data.

Amazon Athena and S3 Glacier Select: Athena enables querying data directly in S3 using standard SQL, while S3 Glacier Select allows querying data stored in S3 Glacier without needing to restore the entire object. This allows the company to query and retrieve data irrespective of its storage class (S3 Intelligent-Tiering or S3 Glacier).

The other options are less ideal:

Option A: S3 Glacier Instant Retrieval is the most expensive S3 Glacier tier. Using it for frequently accessed files in the first year defeats the purpose of cost optimization. Also, querying based on object tags isn't as efficient as using Athena for structured querying.

Option C: Storing all files in S3 Standard for the first year is more expensive than using S3 Intelligent-Tiering, which dynamically adjusts storage costs. Storing search metadata in S3 Standard storage alongside the data is also redundant, as Athena can be used for that purpose.

Option D: Storing search metadata in Amazon RDS is more complex and expensive than using Athena for querying data in S3. S3 Glacier Deep Archive offers the lowest storage cost but has the highest retrieval times, which might be acceptable for older data but is not necessary when Glacier Flexible Retrieval meets the requirement.

In summary, Option B provides the best balance of performance and cost-effectiveness by using S3 Intelligent-Tiering for frequent access, S3 Lifecycle policies for automated archival, and Athena for querying data across different storage classes.

Authoritative Links:

S3 Intelligent-Tiering
S3 Lifecycle Policies
Amazon Athena
S3 Glacier Select
S3 Storage Classes

Question: 50 **Exam Heist**

A company has a production workload that runs on 1,000 Amazon EC2 Linux instances. The workload is powered by third-party software. The company needs to patch the third-party software on all EC2 instances as quickly as possible to remediate a critical security vulnerability.
What should a solutions architect do to meet these requirements?

A. Create an AWS Lambda function to apply the patch to all EC2 instances.

B. Configure AWS Systems Manager Patch Manager to apply the patch to all EC2 instances.

C. Schedule an AWS Systems Manager maintenance window to apply the patch to all EC2 instances.

D. Use AWS Systems Manager Run Command to run a custom command that applies the patch to all EC2 instances.

Answer: D

Explanation:

The correct answer is **D. Use AWS Systems Manager Run Command to run a custom command that applies the patch to all EC2 instances.**

Here's a detailed justification:

The primary requirement is to apply a third-party software patch to 1,000 EC2 instances as quickly as possible to remediate a critical security vulnerability. Speed and wide distribution are key.

Why Run Command is Best: AWS Systems Manager (SSM) Run Command allows you to remotely and securely manage the configuration of your managed instances. It lets you execute shell scripts or commands on a large number of EC2 instances simultaneously. For rapidly deploying a patch to a known vulnerability, Run Command offers the most immediate and direct route. You create a shell script (or similar) containing the patching logic and then use Run Command to execute it on all instances. The entire operation can be executed within minutes.

Why other options are less suitable:

A. Create an AWS Lambda function: While Lambda is useful for event-driven tasks, it is less suited for directly patching 1,000 EC2 instances. Lambda functions have execution time limits (up to 15 minutes), making it challenging to complete patching on a large number of instances reliably. Moreover, you'd still need to manage the EC2 instance interaction, adding unnecessary complexity. The lambda function itself will need to call SSM to run commands on the instances.

B. Configure AWS Systems Manager Patch Manager: Patch Manager is designed for automated OS-level patching based on patch baselines. It's not ideal for deploying third-party software patches quickly, especially in an emergency situation. Setting up and configuring patch baselines and ensuring they are applied to 1000 machines will take longer than using Run Command directly.

C. Schedule an AWS Systems Manager maintenance window: Maintenance Windows are valuable for scheduled tasks, but not for immediate, emergency patching. They introduce a delay before the patch can be applied. The requirement emphasizes quickly as possible, which Maintenance Windows don't fulfill.

In summary, Run Command is designed for ad-hoc operations and rapid execution of commands across fleets of instances, making it the optimal solution for patching a critical vulnerability as quickly as possible. It provides direct control and immediate deployment capabilities, crucial for addressing security concerns promptly.

Here are authoritative links for further research:

AWS Systems Manager Run Command: https://docs.aws.amazon.com/systems-manager/latest/userguide/execute-remote-commands.html
AWS Systems Manager Patch Manager: https://docs.aws.amazon.com/systems-manager/latest/userguide/patch-manager.html
AWS Systems Manager Maintenance Windows: https://docs.aws.amazon.com/systems-manager/latest/userguide/maintenance-windows.html

Question: 51 **Exam Heist**

A company is developing an application that provides order shipping statistics for retrieval by a REST API. The company wants to extract the shipping statistics, organize the data into an easy-to-read HTML format, and send the report to several email addresses at the same time every morning.
Which combination of steps should a solutions architect take to meet these requirements? (Choose two.)

A. Configure the application to send the data to Amazon Kinesis Data Firehose.

B. Use Amazon Simple Email Service (Amazon SES) to format the data and to send the report by email.

C. Create an Amazon EventBridge (Amazon CloudWatch Events) scheduled event that invokes an AWS Glue job to query the application's API for the data.

D. Create an Amazon EventBridge (Amazon CloudWatch Events) scheduled event that invokes an AWS Lambda function to query the application's API for the data.

E. Store the application data in Amazon S3. Create an Amazon Simple Notification Service (Amazon SNS) topic as an S3 event destination to send the report by email.

Answer: BD

Explanation:

The correct answer is BD. Here's why:

D: Create an Amazon EventBridge (Amazon CloudWatch Events) scheduled event that invokes an AWS Lambda function to query the application's API for the data. Amazon EventBridge allows for scheduled, event-driven automation. In this scenario, it's ideal for triggering the data extraction and report generation process every morning. Lambda offers a serverless compute environment where code can be executed without managing underlying infrastructure. A Lambda function can be coded to query the REST API, retrieve the shipping statistics, and format the data into HTML. This addresses the requirement of extracting data and organizing it. https://aws.amazon.com/eventbridge/, https://aws.amazon.com/lambda/

B: Use Amazon Simple Email Service (Amazon SES) to format the data and to send the report by email. Amazon SES is a cost-effective, scalable email service that provides a reliable way to send emails. The Lambda function, after formatting the data into HTML, can leverage SES to send the report to the specified email addresses. SES handles email delivery and can be configured to handle bounces and complaints. This satisfies the requirement of sending the report to multiple email addresses in an easy-to-read HTML format. The Lambda function would take on the additional task of the HTML formatting. https://aws.amazon.com/ses/

Why other options are incorrect:

A: Configure the application to send the data to Amazon Kinesis Data Firehose. Kinesis Data Firehose is used for streaming data to destinations like S3, Redshift, and Elasticsearch. It's not suitable for querying data from an API and generating reports.

C: Create an Amazon EventBridge (Amazon CloudWatch Events) scheduled event that invokes an AWS Glue job to query the application's API for the data. AWS Glue is an ETL (Extract, Transform, Load) service primarily designed for processing large datasets. Using Glue to directly query an API and generate a single HTML report is overkill and less efficient than using Lambda. Glue is usually used against a data lake, not an operational API.

E: Store the application data in Amazon S3. Create an Amazon Simple Notification Service (Amazon SNS) topic as an S3 event destination to send the report by email. S3 is for object storage and would only make sense if the source data (the shipping statistics) were already available within S3, which isn't indicated in the problem description. SNS is for simple notifications and not suited for complex HTML formatting or extracting data from APIs. S3 event notifications are also triggered by data changes, not scheduled events.

Question: 52 **Exam Heist**

A company wants to migrate its on-premises application to AWS. The application produces output files that vary in size from tens of gigabytes to hundreds of terabytes. The application data must be stored in a standard file system structure. The company wants a solution that scales automatically. is highly available, and requires minimum operational overhead.
Which solution will meet these requirements?

A. Migrate the application to run as containers on Amazon Elastic Container Service (Amazon ECS). Use Amazon S3 for storage.

B. Migrate the application to run as containers on Amazon Elastic Kubernetes Service (Amazon EKS). Use Amazon Elastic Block Store (Amazon EBS) for storage.

C. Migrate the application to Amazon EC2 instances in a Multi-AZ Auto Scaling group. Use Amazon Elastic File System (Amazon EFS) for storage.

D. Migrate the application to Amazon EC2 instances in a Multi-AZ Auto Scaling group. Use Amazon Elastic Block Store (Amazon EBS) for storage.

Answer: C

Explanation:

The correct answer is C because it addresses all the requirements: scalability, high availability, standard file system structure, and minimal operational overhead.

Scalability: Amazon EFS scales automatically to petabytes without disrupting applications, handling the varied file sizes (tens of gigabytes to hundreds of terabytes). EC2 Auto Scaling ensures the application instances scale based on demand.
High Availability: Using a Multi-AZ Auto Scaling group ensures that the application is highly available because EC2 instances are spread across multiple Availability Zones. EFS is also designed for high availability and durability, replicating data across multiple AZs.
Standard File System Structure: EFS provides a standard file system interface (NFSv4.1) that allows applications to interact with storage in a familiar way.
Minimum Operational Overhead: EFS is a fully managed service, which eliminates the need for manual provisioning, patching, or backups. EC2 Auto Scaling automates the scaling of instances.

Option A is incorrect because S3 is an object storage service and does not provide a standard file system interface. Option B is incorrect because EBS is block storage and would not scale elastically for the varying size files, and it has more operational overhead compared to EFS. Option D is incorrect because EBS is block storage and not a shared file system. Mounting and sharing EBS volumes between multiple instances introduces complexities.Here are some authoritative links for further research:

Amazon EFS: https://aws.amazon.com/efs/
Amazon EC2 Auto Scaling: https://aws.amazon.com/autoscaling/
Amazon EC2: https://aws.amazon.com/ec2/

Question: 53 **Exam Heist**

A company needs to store its accounting records in Amazon S3. The records must be immediately accessible for 1 year and then must be archived for an additional 9 years. No one at the company, including administrative users and root users, can be able to delete the records during the entire 10-year period. The records must be stored with maximum resiliency.
Which solution will meet these requirements?

A. Store the records in S3 Glacier for the entire 10-year period. Use an access control policy to deny deletion of the records for a period of 10 years.

B. Store the records by using S3 Intelligent-Tiering. Use an IAM policy to deny deletion of the records. After 10

years, change the IAM policy to allow deletion.

C. Use an S3 Lifecycle policy to transition the records from S3 Standard to S3 Glacier Deep Archive after 1 year. Use S3 Object Lock in compliance mode for a period of 10 years.

D. Use an S3 Lifecycle policy to transition the records from S3 Standard to S3 One Zone-Infrequent Access (S3 One Zone-IA) after 1 year. Use S3 Object Lock in governance mode for a period of 10 years.

Answer: C

Explanation:

Here's a detailed justification for why option C is the correct answer:

Option C effectively addresses all requirements: immediate accessibility for 1 year, archival for 9 years, immutability (preventing deletion), and maximum resiliency.

1. **Initial Accessibility:** S3 Standard offers immediate access to the records for the first year.
2. **Archival Storage:** Transitioning the data to S3 Glacier Deep Archive after one year via an S3 Lifecycle policy fulfills the archival requirement and is the most cost-effective storage option for long-term retention with infrequent access. S3 Glacier Deep Archive is designed for data that is rarely accessed.
3. **Immutability:** S3 Object Lock in compliance mode is crucial. Compliance mode ensures that no one, including administrative users or the root user, can delete the objects during the specified retention period (10 years in this case). This definitively meets the immutability requirement.
4. **Resiliency:** S3 Standard and S3 Glacier Deep Archive provide the highest levels of data durability and availability by storing data across multiple devices and facilities.

Why other options are incorrect:

A: S3 Glacier is not designed for immediate accessibility. It's for infrequent access with retrieval times ranging from minutes to hours, not the "immediately accessible" requirement of the first year. Access control policies and IAM Policies can be bypassed or modified, which means they cannot guarantee immutability.

B: S3 Intelligent-Tiering automatically moves data between frequent, infrequent, and archive access tiers based on access patterns. While useful for cost optimization, it doesn't guarantee immutability. IAM policies can be bypassed or modified.

D: S3 One Zone-IA offers lower availability (data is stored in a single Availability Zone), which does not meet the "maximum resiliency" requirement. Also, S3 Object Lock in governance mode can be overridden by users with specific IAM permissions, failing to guarantee immutability against administrative users or root.

Authoritative Links:

S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
S3 Lifecycle Policies: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-concept.html
S3 Object Lock: https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html
Object Lock Compliance vs Governance Mode:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock-overview.html

In summary, Option C provides the optimal balance of accessibility, cost-effectiveness, immutability, and resiliency to meet the stated requirements, making it the correct answer.

Question: 54 **Exam Heist**

A company runs multiple Windows workloads on AWS. The company's employees use Windows file shares that are hosted on two Amazon EC2 instances. The file shares synchronize data between themselves and maintain duplicate copies. The company wants a highly available and durable storage solution that preserves how users currently access the files.
What should a solutions architect do to meet these requirements?

A. Migrate all the data to Amazon S3. Set up IAM authentication for users to access files.

B. Set up an Amazon S3 File Gateway. Mount the S3 File Gateway on the existing EC2 instances.

C. Extend the file share environment to Amazon FSx for Windows File Server with a Multi-AZ configuration. Migrate all the data to FSx for Windows File Server.

D. Extend the file share environment to Amazon Elastic File System (Amazon EFS) with a Multi-AZ configuration. Migrate all the data to Amazon EFS.

Answer: C

Explanation:

The correct answer is C. Here's why:

The requirement is to provide a highly available and durable file share solution for Windows workloads while preserving the existing user access patterns.

Option A (Amazon S3 with IAM): While S3 is highly available and durable, it doesn't directly support Windows file shares. Users would need to adapt to a completely different access method, breaking the "preserve how users currently access the files" requirement. IAM authentication, although secure, doesn't replicate the file share experience.

Option B (Amazon S3 File Gateway): S3 File Gateway provides a way to access S3 objects as files, but it doesn't replicate a traditional Windows file share. It adds complexity to the existing EC2 setup and doesn't inherently provide high availability for the gateway itself. Users would still potentially experience access disruptions if the gateway instance fails.

Option C (Amazon FSx for Windows File Server with Multi-AZ): Amazon FSx for Windows File Server is a fully managed service specifically designed for providing native Windows file server capabilities in the cloud. It's built on Windows Server, so it directly supports SMB file shares and integrates seamlessly with existing Active Directory environments for authentication. A Multi-AZ configuration provides high availability by automatically replicating data across multiple Availability Zones. This ensures that if one AZ fails, the file share remains accessible to users. Migrating the data to FSx for Windows File Server preserves the existing user access patterns as they continue to use familiar Windows file share protocols.

Option D (Amazon EFS with Multi-AZ): Amazon EFS is a network file system designed for Linux-based workloads. While it offers high availability and durability, it's not a native Windows file server solution and doesn't directly support SMB file shares. Integrating EFS with Windows workloads would require additional configuration and wouldn't provide the same seamless experience as FSx for Windows File Server.

In conclusion, Option C provides the best solution as it directly addresses all the requirements: high availability, durability, preservation of existing user access methods, and native support for Windows file shares through Amazon FSx for Windows File Server with a Multi-AZ configuration.

Authoritative Links for Further Research:

Amazon FSx for Windows File Server: https://aws.amazon.com/fsx/windows/
High Availability (HA) and Multi-AZ: https://docs.aws.amazon.com/fsx/latest/WindowsGuide/high-availability.html

Question: 55 **Exam Heist**

A solutions architect is developing a VPC architecture that includes multiple subnets. The architecture will host applications that use Amazon EC2 instances and Amazon RDS DB instances. The architecture consists of six subnets in two Availability Zones. Each Availability Zone includes a public subnet, a private subnet, and a dedicated subnet for databases. Only EC2 instances that run in the private subnets can have access to the RDS databases.
Which solution will meet these requirements?

A. Create a new route table that excludes the route to the public subnets' CIDR blocks. Associate the route table with the database subnets.

B. Create a security group that denies inbound traffic from the security group that is assigned to instances in the public subnets. Attach the security group to the DB instances.

C. Create a security group that allows inbound traffic from the security group that is assigned to instances in the private subnets. Attach the security group to the DB instances.

D. Create a new peering connection between the public subnets and the private subnets. Create a different peering connection between the private subnets and the database subnets.

Answer: C

Explanation:

The correct solution (C) leverages security groups to control network access to the RDS instances. Security groups act as virtual firewalls at the instance level, controlling both inbound and outbound traffic. To meet the requirement that only EC2 instances in the private subnets can access the RDS databases, we create a security group that specifically allows inbound traffic only from the security group assigned to those EC2 instances in the private subnets.

Here's why the other options are not ideal:

A: Creating a route table that excludes routes to public subnets' CIDR blocks and associating it with the database subnets would primarily affect routing and not precisely control which instances can access the database. It might block general traffic from public subnets to database subnets, but it wouldn't isolate access to only EC2 instances associated with a specific security group in the private subnets. It also makes the configuration less flexible and scalable.

B: Denying inbound traffic from the public subnet's security group at the database instance is a step in the right direction, but it doesn't explicitly allow traffic from the private subnets. This approach might inadvertently block legitimate traffic originating from other sources within the private subnets that were intended to connect to the database. The best practice is to use a principle of least privilege by allowing only the necessary traffic.

D: Peering connections are for connecting VPCs, not subnets within the same VPC. The EC2 instances and RDS instances are already within the same VPC, so subnet-level peering is unnecessary and not a feasible solution. Peering creates network connectivity between entire VPCs, adding unnecessary complexity and cost for this specific requirement. Subnets within a VPC inherently communicate via the VPC's internal routing.

Therefore, option C is the most precise, secure, and efficient way to ensure that only EC2 instances in the private subnets can access the RDS instances, adhering to the principle of least privilege and utilizing the inherent security features provided by security groups.

Supporting Links:

Amazon VPC Security Groups: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html
Controlling Traffic to Resources Using Security Groups:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_WorkingWithSecurityGroups.html

Question: 56 **Exam Heist**

A company has registered its domain name with Amazon Route 53. The company uses Amazon API Gateway in the ca-central-1 Region as a public interface for its backend microservice APIs. Third-party services consume the APIs securely. The company wants to design its API Gateway URL with the company's domain name and corresponding certificate so that the third-party services can use HTTPS.
Which solution will meet these requirements?

A. Create stage variables in API Gateway with Name="Endpoint-URL" and Value="Company Domain Name" to overwrite the default URL. Import the public certificate associated with the company's domain name into AWS Certificate Manager (ACM).

B. Create Route 53 DNS records with the company's domain name. Point the alias record to the Regional API Gateway stage endpoint. Import the public certificate associated with the company's domain name into AWS Certificate Manager (ACM) in the us-east-1 Region.

C. Create a Regional API Gateway endpoint. Associate the API Gateway endpoint with the company's domain name. Import the public certificate associated with the company's domain name into AWS Certificate Manager (ACM) in the same Region. Attach the certificate to the API Gateway endpoint. Configure Route 53 to route traffic to the API Gateway endpoint.

D. Create a Regional API Gateway endpoint. Associate the API Gateway endpoint with the company's domain name. Import the public certificate associated with the company's domain name into AWS Certificate Manager (ACM) in the us-east-1 Region. Attach the certificate to the API Gateway APIs. Create Route 53 DNS records with the company's domain name. Point an A record to the company's domain name.

Answer: C

Explanation:

The correct answer is C because it aligns with the best practices for using custom domain names with API Gateway and ensuring secure HTTPS communication. Here's why:

Regional API Gateway Endpoint: Creating a Regional API Gateway endpoint is essential. It allows associating a custom domain name and certificate within a specific AWS Region (ca-central-1 in this case), providing lower latency and better control compared to edge-optimized endpoints in this scenario.

Custom Domain Association: Associating the API Gateway endpoint with the company's domain name is a key step in providing a user-friendly and branded URL for the APIs. This is the fundamental step toward using the company's domain instead of the default API Gateway URL.

ACM Certificate Import and Region Specificity: Importing the SSL/TLS certificate into AWS Certificate Manager (ACM) in the same Region as the API Gateway (ca-central-1) is crucial. ACM certificates are region-specific, and the API Gateway needs to be able to access the certificate to establish secure HTTPS connections. Answer D fails on this very important concept.

Certificate Attachment: Attaching the certificate to the API Gateway endpoint ensures that the API Gateway uses the certificate for SSL/TLS termination, enabling HTTPS for clients accessing the APIs.

Route 53 Configuration: Configuring Route 53 to route traffic to the API Gateway endpoint is the final step. Route 53 maps the company's domain name to the API Gateway endpoint, ensuring that requests to the custom domain name are directed to the API Gateway.Option A is incorrect as stage variables are generally for API Gateway configuration values, not for custom domain mapping. Option B is incorrect because, while pointing a Route 53 record to an API Gateway endpoint is correct, the ACM certificate must be in the same Region as the API Gateway. Option D incorrectly states that ACM certificates need to be in us-east-1. They need to be in the same region as the API Gateway. It also incorrectly states that the certificate needs to be attached to the API Gateway APIs (plural). The certificate is attached to the API Gateway endpoint.

Supporting Links:

Custom Domain Names for API Gateway:

https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-use-custom-domain-name.html
AWS Certificate Manager (ACM): https://aws.amazon.com/certificate-manager/
Amazon Route 53: https://aws.amazon.com/route53/

Question: 57 **Exam Heist**

A company is running a popular social media website. The website gives users the ability to upload images to share with other users. The company wants to make sure that the images do not contain inappropriate content. The company needs a solution that minimizes development effort.
What should a solutions architect do to meet these requirements?

A. Use Amazon Comprehend to detect inappropriate content. Use human review for low-confidence predictions.

B. Use Amazon Rekognition to detect inappropriate content. Use human review for low-confidence predictions.

C. Use Amazon SageMaker to detect inappropriate content. Use ground truth to label low-confidence predictions.

D. Use AWS Fargate to deploy a custom machine learning model to detect inappropriate content. Use ground truth to label low-confidence predictions.

Answer: B

Explanation:

The correct answer is B because it leverages Amazon Rekognition's pre-trained models specifically designed for image analysis, including the detection of explicit or suggestive content. This significantly reduces development effort compared to building a custom model.

Amazon Rekognition's Content Moderation feature can identify various types of inappropriate content, such as nudity, violence, and hate symbols. The service provides a confidence score for each detection. For detections with low confidence scores, a human review workflow can be implemented to ensure accuracy and prevent false positives or negatives. This hybrid approach balances automation with human oversight.

Option A is incorrect because Amazon Comprehend is a natural language processing (NLP) service, best suited for analyzing text, not images. Option C is incorrect because Amazon SageMaker is a machine learning platform for building, training, and deploying custom models. While it's possible to build a custom image moderation model with SageMaker, it requires significantly more development effort than using Rekognition's pre-built capabilities. Ground Truth is used for data labeling in SageMaker and isn't the optimal choice when pre-trained models with confidence scores and built in human review workflows are available. Option D is incorrect because deploying a custom machine learning model on AWS Fargate also requires significant development and operational overhead compared to utilizing a managed service like Rekognition. Fargate provides serverless compute for containers, not a machine learning image analysis service. Building custom models and infrastructure is unnecessary when a suitable managed service already exists. Rekognition provides a more straightforward and efficient solution for the stated requirements.Authoritative links:

Amazon Rekognition Content Moderation: https://aws.amazon.com/rekognition/content-moderation/

Question: 58 **Exam Heist**

A company wants to run its critical applications in containers to meet requirements for scalability and availability. The company prefers to focus on maintenance of the critical applications. The company does not want to be responsible for provisioning and managing the underlying infrastructure that runs the containerized workload.
What should a solutions architect do to meet these requirements?

A. Use Amazon EC2 instances, and install Docker on the instances.

B. Use Amazon Elastic Container Service (Amazon ECS) on Amazon EC2 worker nodes.

C. Use Amazon Elastic Container Service (Amazon ECS) on AWS Fargate.

D. Use Amazon EC2 instances from an Amazon Elastic Container Service (Amazon ECS)-optimized Amazon Machine Image (AMI).

Answer: C

Explanation:

The best solution is to use Amazon ECS on AWS Fargate because it abstracts away the need to manage the underlying infrastructure. Fargate is a serverless compute engine for containers that lets you run containers without managing servers or clusters. This aligns perfectly with the company's preference to focus on application maintenance and avoid infrastructure management.

Option A requires manual installation and configuration of Docker on EC2 instances, placing the burden of server maintenance and scaling on the company. Option B, while using ECS, still involves managing EC2 worker nodes. This means patching, scaling, and ensuring the availability of the underlying EC2 instances. Option D, using an ECS-optimized AMI on EC2, simplifies some aspects of EC2 management, but the company is still responsible for the EC2 instances themselves.

Fargate eliminates these responsibilities by handling the provisioning, scaling, and patching of the infrastructure. The company simply defines the container image, CPU, and memory requirements, and Fargate handles the rest. This reduces operational overhead and allows the company to focus solely on their critical applications. Furthermore, Fargate integrates well with other AWS services and offers cost optimization benefits.https://aws.amazon.com/fargate/https://aws.amazon.com/ecs/

Question: 59 **Exam Heist**

A company hosts more than 300 global websites and applications. The company requires a platform to analyze more than 30 TB of clickstream data each day.
What should a solutions architect do to transmit and process the clickstream data?

A. Design an AWS Data Pipeline to archive the data to an Amazon S3 bucket and run an Amazon EMR cluster with the data to generate analytics.

B. Create an Auto Scaling group of Amazon EC2 instances to process the data and send it to an Amazon S3 data lake for Amazon Redshift to use for analysis.

C. Cache the data to Amazon CloudFront. Store the data in an Amazon S3 bucket. When an object is added to the S3 bucket. run an AWS Lambda function to process the data for analysis.

D. Collect the data from Amazon Kinesis Data Streams. Use Amazon Kinesis Data Firehose to transmit the data to an Amazon S3 data lake. Load the data in Amazon Redshift for analysis.

Answer: D

Explanation:

The most effective solution for handling 30 TB of daily clickstream data from 300+ global websites and applications for analysis involves a scalable and robust data ingestion and processing pipeline using AWS's specialized services for big data. Option D is the best approach.

Here's a breakdown:

Data Ingestion (Amazon Kinesis Data Streams): Kinesis Data Streams is designed for real-time ingestion of high-volume data streams. It can handle the constant flow of clickstream data from numerous sources globally. It allows the data to be buffered and processed in manageable chunks.

Data Delivery (Amazon Kinesis Data Firehose): Kinesis Data Firehose is ideal for reliably loading data into

data lakes and data warehouses. It can automatically scale to match the incoming data volume and deliver data to S3. It offers data transformation and batching capabilities.

Data Lake (Amazon S3): S3 provides a scalable and cost-effective storage solution for the large volume of clickstream data. It serves as a central repository for raw and processed data, forming the foundation of the data lake.

Data Warehousing (Amazon Redshift): Redshift is a fast, fully managed, petabyte-scale data warehouse service in the cloud. It is optimized for analytical queries and can efficiently analyze the large dataset stored in S3.

Option A is less desirable because while EMR can process large datasets, using Data Pipeline solely for archiving and triggering EMR jobs introduces unnecessary complexity. Data Pipeline is typically used for orchestration, and Kinesis Data Firehose is more streamlined for data delivery to S3. Option B involves managing EC2 instances, which adds operational overhead compared to using managed services like Kinesis and Redshift. While caching data with CloudFront, as suggested in Option C, can improve website performance, it doesn't directly address the core problem of analyzing 30 TB of clickstream data daily. A Lambda function triggered by S3 objects may struggle with the sheer volume of data.

In summary, using Kinesis Data Streams and Firehose ensures real-time data ingestion and delivery to a data lake (S3), where it can be efficiently analyzed by Redshift, offering a scalable, managed, and cost-effective solution.

Relevant links:

Amazon Kinesis Data Streams: https://aws.amazon.com/kinesis/data-streams/
Amazon Kinesis Data Firehose: https://aws.amazon.com/kinesis/data-firehose/
Amazon S3: https://aws.amazon.com/s3/
Amazon Redshift: https://aws.amazon.com/redshift/

Question: 60 **Exam Heist**

A company has a website hosted on AWS. The website is behind an Application Load Balancer (ALB) that is configured to handle HTTP and HTTPS separately. The company wants to forward all requests to the website so that the requests will use HTTPS.
What should a solutions architect do to meet this requirement?

A. Update the ALB's network ACL to accept only HTTPS traffic.
B. Create a rule that replaces the HTTP in the URL with HTTPS.
C. Create a listener rule on the ALB to redirect HTTP traffic to HTTPS.
D. Replace the ALB with a Network Load Balancer configured to use Server Name Indication (SNI).

Answer: C

Explanation:

The correct solution is to create a listener rule on the Application Load Balancer (ALB) to redirect HTTP traffic to HTTPS (Option C). Here's why:

An ALB listens for incoming traffic on specified ports and protocols. To redirect HTTP traffic to HTTPS, you configure a listener on port 80 (HTTP) and add a rule to redirect all incoming requests to port 443 (HTTPS). This is a standard practice for ensuring secure communication.

Option A (Updating the ALB's network ACL) is incorrect because network ACLs operate at the subnet level and control inbound and outbound traffic based on IP addresses and ports. While you can restrict HTTP traffic at the network ACL level, this would block HTTP traffic entirely rather than redirecting it to HTTPS. Network

ACLs are not designed for URL redirection.

Option B (Creating a rule to replace HTTP in the URL with HTTPS) is incorrect because ALBs don't directly manipulate URLs in that way. ALBs can perform content-based routing, but not URL rewriting in the context described. The primary function of the ALB in this scenario is to redirect traffic based on listener rules.

Option D (Replacing the ALB with a Network Load Balancer) is incorrect because Network Load Balancers (NLBs) operate at Layer 4 (TCP/UDP), while ALBs operate at Layer 7 (Application Layer). NLBs do not have the capability to inspect HTTP headers or redirect traffic based on the HTTP protocol. Server Name Indication (SNI) on NLBs handles TLS certificates for multiple domains but does not redirect HTTP to HTTPS. NLBs are typically used for high-performance, low-latency applications where protocol-level inspection is not required. An ALB is better suited for HTTP/HTTPS traffic management and URL redirection.

Therefore, the most efficient and appropriate method is to use an ALB listener rule to redirect HTTP traffic to HTTPS, ensuring all communication with the website is secure.

Further Research:

AWS Documentation on ALB Listeners:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-listeners.html
AWS Documentation on ALB Rules:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-rules.html
AWS Documentation on Network ACLs: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html
AWS Documentation on Network Load Balancers:
https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html

Question: 61 **Exam Heist**

A company is developing a two-tier web application on AWS. The company's developers have deployed the application on an Amazon EC2 instance that connects directly to a backend Amazon RDS database. The company must not hardcode database credentials in the application. The company must also implement a solution to automatically rotate the database credentials on a regular basis.
Which solution will meet these requirements with the LEAST operational overhead?

A. Store the database credentials in the instance metadata. Use Amazon EventBridge (Amazon CloudWatch Events) rules to run a scheduled AWS Lambda function that updates the RDS credentials and instance metadata at the same time.

B. Store the database credentials in a configuration file in an encrypted Amazon S3 bucket. Use Amazon EventBridge (Amazon CloudWatch Events) rules to run a scheduled AWS Lambda function that updates the RDS credentials and the credentials in the configuration file at the same time. Use S3 Versioning to ensure the ability to fall back to previous values.

C. Store the database credentials as a secret in AWS Secrets Manager. Turn on automatic rotation for the secret. Attach the required permission to the EC2 role to grant access to the secret.

D. Store the database credentials as encrypted parameters in AWS Systems Manager Parameter Store. Turn on automatic rotation for the encrypted parameters. Attach the required permission to the EC2 role to grant access to the encrypted parameters.

Answer: C

Explanation:

Option C is the most efficient and secure solution because it leverages AWS Secrets Manager for credential management and rotation.

Here's why:

1. **Secrets Manager:** AWS Secrets Manager is specifically designed for securely storing and managing sensitive information like database credentials. It eliminates the need to hardcode credentials in applications.
2. **Automatic Rotation:** Secrets Manager provides automatic secret rotation, significantly reducing the operational overhead associated with manually managing credentials. The service handles the complexities of updating the database and the stored secret.
3. **IAM Integration:** IAM roles assigned to EC2 instances can be granted permissions to access specific secrets stored in Secrets Manager. This allows the application running on the EC2 instance to retrieve the database credentials securely.
4. **Least Operational Overhead:** Solutions A, B, and D require custom Lambda functions and scheduling mechanisms to manage credential rotation. Option C automates this process, which results in the lowest operational overhead because Secrets Manager handles the complexity of rotating the credentials.
5. **Security Best Practices:** Using Secrets Manager aligns with security best practices by centralizing and securing sensitive information and using encryption.

Why other options are not ideal:

Option A: Storing credentials in instance metadata is not recommended for sensitive information like database credentials. Also, instance metadata is not designed for automated rotation.
Option B: Storing credentials in S3, even encrypted, is a less secure and operationally heavier approach than using Secrets Manager. Implementing the rotation logic and managing S3 versions introduces unnecessary complexity.
Option D: Parameter Store can store secrets, but automatic rotation of encrypted parameters requires custom configuration and Lambda functions. Secrets Manager is a more streamlined and secure solution for this specific use case.

Here are some authoritative links for further research:

AWS Secrets Manager Documentation
Rotating AWS Secrets Manager secrets automatically
AWS Identity and Access Management (IAM)

Question: 62 **Exam Heist**

A company is deploying a new public web application to AWS. The application will run behind an Application Load Balancer (ALB). The application needs to be encrypted at the edge with an SSL/TLS certificate that is issued by an external certificate authority (CA). The certificate must be rotated each year before the certificate expires.
What should a solutions architect do to meet these requirements?

A. Use AWS Certificate Manager (ACM) to issue an SSL/TLS certificate. Apply the certificate to the ALB. Use the managed renewal feature to automatically rotate the certificate.

B. Use AWS Certificate Manager (ACM) to issue an SSL/TLS certificate. Import the key material from the certificate. Apply the certificate to the ALUse the managed renewal feature to automatically rotate the certificate.

C. Use AWS Certificate Manager (ACM) Private Certificate Authority to issue an SSL/TLS certificate from the root CA. Apply the certificate to the ALB. Use the managed renewal feature to automatically rotate the certificate.

D. Use AWS Certificate Manager (ACM) to import an SSL/TLS certificate. Apply the certificate to the ALB. Use Amazon EventBridge (Amazon CloudWatch Events) to send a notification when the certificate is nearing expiration. Rotate the certificate manually.

Answer: D

Explanation:

The correct answer is D because it addresses the specific requirements of using a certificate issued by an external CA and the need to manually rotate it. Let's break down why the other options are incorrect and why D is the best fit:

A, B, and C are incorrect: These options involve using ACM to issue a certificate. The question explicitly states that the certificate is issued by an external CA. ACM's managed renewal feature only works for ACM-issued certificates. Therefore, relying on ACM to issue and automatically rotate the certificate is not possible when using an externally issued certificate. Option B contains the erroneous step of importing key material from a certificate (a certificate is the key material). Option C involves a private certificate authority, an unnecessary level of complexity when an existing public CA is being used.

D is correct: This option acknowledges the need to import the externally issued certificate into ACM. ACM serves as a central repository for managing certificates, regardless of where they are issued. Once imported, the certificate can be associated with the ALB. Because the certificate is externally issued, ACM cannot automatically renew it. Therefore, the solution leverages Amazon EventBridge (formerly CloudWatch Events) to monitor the certificate's expiration date. When the certificate is nearing expiry, EventBridge triggers a notification (e.g., via SNS to an operations team), prompting a manual rotation process. This process involves obtaining a new certificate from the external CA, importing it into ACM, and updating the ALB to use the new certificate.

In summary, option D correctly acknowledges the externally issued certificate, utilizes ACM for management, and implements a notification system to ensure manual rotation occurs before the certificate expires, thus meeting all requirements.

Relevant links:

AWS Certificate Manager (ACM): https://aws.amazon.com/certificate-manager/
Importing Certificates into ACM: https://docs.aws.amazon.com/acm/latest/userguide/import-certificate.html
Amazon EventBridge: https://aws.amazon.com/eventbridge/
Application Load Balancer (ALB): https://aws.amazon.com/elasticloadbalancing/application-load-balancer/

Question: 63 **Exam Heist**

A company runs its infrastructure on AWS and has a registered base of 700,000 users for its document management application. The company intends to create a product that converts large .pdf files to .jpg image files. The .pdf files average 5 MB in size. The company needs to store the original files and the converted files. A solutions architect must design a scalable solution to accommodate demand that will grow rapidly over time. Which solution meets these requirements MOST cost-effectively?

A. Save the .pdf files to Amazon S3. Configure an S3 PUT event to invoke an AWS Lambda function to convert the files to .jpg format and store them back in Amazon S3.

B. Save the .pdf files to Amazon DynamoDUse the DynamoDB Streams feature to invoke an AWS Lambda function to convert the files to .jpg format and store them back in DynamoDB.

C. Upload the .pdf files to an AWS Elastic Beanstalk application that includes Amazon EC2 instances, Amazon Elastic Block Store (Amazon EBS) storage, and an Auto Scaling group. Use a program in the EC2 instances to convert the files to .jpg format. Save the .pdf files and the .jpg files in the EBS store.

D. Upload the .pdf files to an AWS Elastic Beanstalk application that includes Amazon EC2 instances, Amazon Elastic File System (Amazon EFS) storage, and an Auto Scaling group. Use a program in the EC2 instances to convert the file to .jpg format. Save the .pdf files and the .jpg files in the EBS store.

Answer: A

Explanation:

Here's why option A is the most cost-effective solution for the PDF to JPG conversion problem, along with supporting justifications and links:

Option A leverages the strengths of Amazon S3 and AWS Lambda for a serverless and scalable solution. S3 provides highly durable and cost-effective object storage for both the original PDFs and the converted JPGs. Its event notification system allows triggering the Lambda function directly upon PDF upload, automating the conversion process. Lambda offers pay-per-execution pricing, meaning you only pay for the compute time used during the conversion, making it exceptionally cost-efficient for variable workloads. This serverless approach eliminates the need to manage EC2 instances, reducing operational overhead and associated costs.

Option B is incorrect because DynamoDB is primarily designed for fast key-value or document data storage and retrieval, not for storing large binary files like PDFs and JPGs. Storing these files in DynamoDB would be significantly more expensive than using S3, and the read/write capacity units required would add to cost unnecessarily. Also, using DynamoDB streams for processing is less efficient than S3 event notifications in this scenario.

Options C and D involve using Elastic Beanstalk with EC2 instances and EBS/EFS for storage. While these options could work, they are less cost-effective because they require maintaining EC2 instances even during periods of low demand. EBS, being block storage, is primarily for persistent storage attached to EC2 instances, while EFS is better suited for shared file systems. Neither are as economical for storing large numbers of files as S3. The added operational complexity of managing EC2 instances, Auto Scaling, and storage volumes further increases the total cost.

In summary, the serverless architecture provided by S3 and Lambda offers the best combination of scalability, automation, and cost-effectiveness for this specific PDF-to-JPG conversion task.

Supporting Links:

Amazon S3: https://aws.amazon.com/s3/
AWS Lambda: https://aws.amazon.com/lambda/
S3 Event Notifications: https://docs.aws.amazon.com/AmazonS3/latest/userguide/event-notifications-overview.html
AWS Lambda Pricing: https://aws.amazon.com/lambda/pricing/

Question: 64 **Exam Heist**

A company has more than 5 TB of file data on Windows file servers that run on premises. Users and applications interact with the data each day.
The company is moving its Windows workloads to AWS. As the company continues this process, the company requires access to AWS and on-premises file storage with minimum latency. The company needs a solution that minimizes operational overhead and requires no significant changes to the existing file access patterns. The company uses an AWS Site-to-Site VPN connection for connectivity to AWS.
What should a solutions architect do to meet these requirements?

A. Deploy and configure Amazon FSx for Windows File Server on AWS. Move the on-premises file data to FSx for Windows File Server. Reconfigure the workloads to use FSx for Windows File Server on AWS.

B. Deploy and configure an Amazon S3 File Gateway on premises. Move the on-premises file data to the S3 File Gateway. Reconfigure the on-premises workloads and the cloud workloads to use the S3 File Gateway.

C. Deploy and configure an Amazon S3 File Gateway on premises. Move the on-premises file data to Amazon S3. Reconfigure the workloads to use either Amazon S3 directly or the S3 File Gateway. depending on each workload's location.

D. Deploy and configure Amazon FSx for Windows File Server on AWS. Deploy and configure an Amazon FSx File Gateway on premises. Move the on-premises file data to the FSx File Gateway. Configure the cloud workloads to use FSx for Windows File Server on AWS. Configure the on-premises workloads to use the FSx File Gateway.

Answer: D

Explanation:

The correct answer is D because it directly addresses all requirements with a balanced approach to performance, operational overhead, and minimal disruption. Here's a breakdown:

Minimum Latency: Deploying FSx File Gateway on-premises allows the on-premises workloads to access the file data stored in FSx for Windows File Server in AWS with minimal latency. The FSx File Gateway acts as a local cache for frequently accessed files, ensuring quick access for on-premises users and applications.

Access to AWS and On-Premises: FSx for Windows File Server on AWS stores the data in the cloud, providing access to cloud workloads. The FSx File Gateway provides access to the same data for on-premises workloads.

Minimized Operational Overhead: FSx File Gateway is a fully managed service, reducing the operational burden of managing storage infrastructure on-premises. The management of the data is centralized on AWS.

No Significant Changes to File Access Patterns: Both FSx for Windows File Server and FSx File Gateway support the standard SMB protocol, allowing existing Windows applications to access files without significant code modifications.

Why other options are incorrect:

A is incorrect because moving all the data to AWS and accessing it over Site-to-Site VPN would likely increase the latency for on-premises workloads.
B and C are incorrect because S3 File Gateway replicates data to Amazon S3, which is object storage and doesn't natively support SMB protocol. Thus, the workloads would need to be reconfigured to use S3 APIs, which would be a significant change to existing file access patterns, and S3 File Gateway on-premises still might increase the latency for on-premises workloads compared to a dedicated solution like FSx File Gateway.

Here are some authoritative links for further research:

Amazon FSx for Windows File Server: https://aws.amazon.com/fsx/windows/
Amazon FSx File Gateway: https://aws.amazon.com/fsx/file-gateway/
Amazon S3 File Gateway: https://aws.amazon.com/storagegateway/file/

Question: 65 **Exam Heist**

A hospital recently deployed a RESTful API with Amazon API Gateway and AWS Lambda. The hospital uses API Gateway and Lambda to upload reports that are in PDF format and JPEG format. The hospital needs to modify the Lambda code to identify protected health information (PHI) in the reports.
Which solution will meet these requirements with the LEAST operational overhead?

A. Use existing Python libraries to extract the text from the reports and to identify the PHI from the extracted text.

B. Use Amazon Textract to extract the text from the reports. Use Amazon SageMaker to identify the PHI from the extracted text.

C. Use Amazon Textract to extract the text from the reports. Use Amazon Comprehend Medical to identify the PHI from the extracted text.

D. Use Amazon Rekognition to extract the text from the reports. Use Amazon Comprehend Medical to identify the PHI from the extracted text.

Answer: C

Explanation:

The correct answer is **C**. Here's a detailed justification:

The core requirement is identifying PHI (Protected Health Information) within PDF and JPEG reports uploaded via API Gateway and Lambda. The solution must minimize operational overhead.

Option A (Python Libraries): While feasible, this approach requires significant custom coding to handle PDF and JPEG parsing (potentially complex) and implementing a PHI identification algorithm. This increases development effort, testing requirements, and ongoing maintenance, leading to higher operational overhead.
Option B (Textract and SageMaker): Textract is excellent for text extraction. However, using SageMaker for PHI identification introduces unnecessary complexity. SageMaker is designed for building, training, and deploying custom machine learning models. For a specialized task like PHI detection (which is already addressed by a dedicated service), SageMaker is overkill and introduces significant operational overhead related to model management, infrastructure, and training data.
Option C (Textract and Comprehend Medical): Textract efficiently extracts text from both PDF and JPEG formats. Comprehend Medical is a HIPAA-eligible natural language processing (NLP) service specifically designed for analyzing medical text and identifying PHI. This combination minimizes custom code and leverages AWS-managed services, resulting in the least operational overhead. It offers built-in PHI detection capabilities.
Option D (Rekognition and Comprehend Medical): Rekognition is primarily designed for image analysis, including object detection and facial recognition. While it can extract text through OCR (Optical Character Recognition), Textract is optimized for document processing and will yield better results with PDF and complex document layouts. Also, Rekognition might not be as accurate as Textract in extracting text from these specific report formats. Comprehend Medical is suitable for the PHI extraction task, but Rekognition is not the ideal choice for text extraction in this scenario, creating additional overhead.

Therefore, using Amazon Textract to extract text and Amazon Comprehend Medical to identify PHI offers the best balance of functionality, accuracy, and minimal operational burden by utilizing purpose-built, managed AWS services tailored to these specific tasks. This solution aligns with the "less code is better" principle and simplifies the deployment and maintenance processes.

Authoritative Links:

Amazon Textract: https://aws.amazon.com/textract/
Amazon Comprehend Medical: https://aws.amazon.com/comprehend/medical/
Amazon SageMaker: https://aws.amazon.com/sagemaker/
Amazon Rekognition: https://aws.amazon.com/rekognition/

Question: 66 **Exam Heist**

A company has an application that generates a large number of files, each approximately 5 MB in size. The files are stored in Amazon S3. Company policy requires the files to be stored for 4 years before they can be deleted. Immediate accessibility is always required as the files contain critical business data that is not easy to reproduce. The files are frequently accessed in the first 30 days of the object creation but are rarely accessed after the first 30 days.
Which storage solution is MOST cost-effective?

A. Create an S3 bucket lifecycle policy to move files from S3 Standard to S3 Glacier 30 days from object creation. Delete the files 4 years after object creation.

B. Create an S3 bucket lifecycle policy to move files from S3 Standard to S3 One Zone-Infrequent Access (S3 One Zone-IA) 30 days from object creation. Delete the files 4 years after object creation.

C. Create an S3 bucket lifecycle policy to move files from S3 Standard to S3 Standard-Infrequent Access (S3 Standard-IA) 30 days from object creation. Delete the files 4 years after object creation.

D. Create an S3 bucket lifecycle policy to move files from S3 Standard to S3 Standard-Infrequent Access (S3

Standard-IA) 30 days from object creation. Move the files to S3 Glacier 4 years after object creation.

Answer: C

Explanation:

The correct answer is C because it offers the most cost-effective solution while adhering to the stated requirements of immediate accessibility and long-term retention.

Here's a detailed breakdown:

S3 Standard: Initially, files are stored in S3 Standard for the first 30 days. This ensures fast and frequent access when the files are actively used. S3 Standard provides high availability (99.99%) and durability (99.999999999%) for frequently accessed data.

S3 Standard-IA: After 30 days, the lifecycle policy automatically transitions the files to S3 Standard-IA. This storage class is designed for data that is infrequently accessed but requires rapid retrieval when needed. It offers lower storage costs compared to S3 Standard while maintaining similar performance characteristics. Since the question specifies immediate accessibility is always required, S3 Standard-IA is a suitable choice for infrequently accessed data.

4-Year Retention & Deletion: The lifecycle policy is configured to permanently delete the files after 4 years, fulfilling the company's retention policy.

Now, let's examine why the other options are less optimal:

Option A (S3 Glacier): Moving files directly to S3 Glacier after 30 days is not ideal. S3 Glacier is designed for archival storage and retrieval can take several hours. The requirement of "immediate accessibility" makes Glacier an unsuitable option for data that may need to be accessed at any moment.

Option B (S3 One Zone-IA): While S3 One Zone-IA offers lower storage costs than S3 Standard-IA, it stores data in a single Availability Zone. This exposes the data to a higher risk of data loss in the event of an Availability Zone failure. S3 One Zone-IA is appropriate when data is easily reproducible. However, the question states that data is not easy to reproduce.

Option D (S3 Standard-IA followed by S3 Glacier): Moving to Glacier after 4 years is unnecessary, since deletion is required at that point. It adds complexity and cost without providing any benefit. Moreover, files can be stored in S3 Standard-IA for 4 years and then be deleted.

In summary, option C provides the best balance between cost, accessibility, and durability, meeting all of the company's requirements in the most cost-effective way. Using S3 Standard for immediate needs, transitioning to S3 Standard-IA for infrequent access, and deleting after 4 years aligns with the best practices for S3 lifecycle management.

Authoritative Links:

Amazon S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
S3 Lifecycle Management: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-concept.html

Question: 67 **Exam Heist**

A company hosts an application on multiple Amazon EC2 instances. The application processes messages from an Amazon SQS queue, writes to an Amazon RDS table, and deletes the message from the queue. Occasional duplicate records are found in the RDS table. The SQS queue does not contain any duplicate messages.
What should a solutions architect do to ensure messages are being processed once only?

A. Use the CreateQueue API call to create a new queue.
B. Use the AddPermission API call to add appropriate permissions.
C. Use the ReceiveMessage API call to set an appropriate wait time.
D. Use the ChangeMessageVisibility API call to increase the visibility timeout.

Answer: D

Explanation:

The most appropriate solution to address the issue of duplicate records in the RDS table, despite the absence of duplicate messages in the SQS queue, is **D. Use the ChangeMessageVisibility API call to increase the visibility timeout.**

Here's a detailed justification:

The problem indicates that the same message is being processed multiple times, leading to duplicate entries in the RDS database. Since the queue itself isn't the source of duplicates, the issue likely lies in how the EC2 instances are handling message processing and acknowledging completion to SQS.

SQS employs a visibility timeout mechanism. When an EC2 instance retrieves a message from the queue, the message becomes invisible to other consumers for a specified duration (the visibility timeout). This prevents other instances from processing the same message simultaneously. The receiving instance is then expected to delete the message from the queue upon successful processing.

If an EC2 instance fails to process a message within the visibility timeout (e.g., due to a crash, network issue, or slow processing), the message becomes visible again in the queue. This allows another instance (or even the original instance after recovery) to pick up the message and re-process it, hence leading to duplicates.

Increasing the visibility timeout using the ChangeMessageVisibility API call provides more time for the EC2 instance to process the message and delete it from the queue before it becomes visible again. This reduces the likelihood of another instance grabbing the same message and creating a duplicate record in the RDS table.

Options A, B, and C are not relevant to addressing the identified problem.

A (CreateQueue): Creating a new queue doesn't solve the underlying issue of messages being processed multiple times.
B (AddPermission): Adding permissions is about access control, not about preventing duplicate processing.
C (ReceiveMessage Wait Time): While adjusting wait time can improve polling efficiency, it doesn't prevent duplicate processing if an instance fails to process a message within the visibility timeout.

By increasing the visibility timeout to a value that comfortably exceeds the maximum expected processing time for a message, the solution architect can minimize the risk of duplicate processing and ensure messages are processed only once. In cases where processing time can vary greatly, a combination of increased visibility timeout and implementing idempotent message processing logic in the application can provide a robust solution against duplicates. Idempotency ensures that even if a message is processed multiple times, the result is the same as processing it once, preventing data inconsistencies.Consider using Dead Letter Queues to manage message that exceeds retry count.

Relevant Documentation:

Amazon SQS Visibility Timeout:
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html
Amazon SQS ChangeMessageVisibility:
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/APIReference/API_ChangeMessageVisibility.html

Question: 68 **Exam Heist**

A solutions architect is designing a new hybrid architecture to extend a company's on-premises infrastructure to AWS. The company requires a highly available connection with consistent low latency to an AWS Region. The company needs to minimize costs and is willing to accept slower traffic if the primary connection fails.
What should the solutions architect do to meet these requirements?

A. Provision an AWS Direct Connect connection to a Region. Provision a VPN connection as a backup if the primary Direct Connect connection fails.

B. Provision a VPN tunnel connection to a Region for private connectivity. Provision a second VPN tunnel for private connectivity and as a backup if the primary VPN connection fails.

C. Provision an AWS Direct Connect connection to a Region. Provision a second Direct Connect connection to the same Region as a backup if the primary Direct Connect connection fails.

D. Provision an AWS Direct Connect connection to a Region. Use the Direct Connect failover attribute from the AWS CLI to automatically create a backup connection if the primary Direct Connect connection fails.

Answer: A

Explanation:

The correct answer is A because it provides a cost-effective solution for high availability and consistent low latency. Direct Connect (DX) offers a dedicated network connection from the on-premises environment to AWS, fulfilling the requirement for high availability and consistent low latency under normal circumstances. This dedicated connection bypasses the public internet, resulting in more predictable and lower latency than VPN connections.

The requirement to minimize costs while accepting slower traffic during failover is met by using a VPN connection as a backup. VPN connections are established over the internet and are generally less expensive than a redundant Direct Connect connection. When the primary Direct Connect link fails, traffic can be routed through the VPN connection, ensuring business continuity, although at a reduced speed and potentially higher latency.

Option B is incorrect because VPN connections, while cost-effective, do not offer the consistently low latency required during normal operation. VPN connections are susceptible to internet traffic fluctuations.

Option C is incorrect as it proposes a redundant Direct Connect connection. While this would offer higher availability, it significantly increases costs. The company has already stated a willingness to accept slower traffic if the primary connection fails, making the cost of a second Direct Connect circuit unnecessary.

Option D is incorrect because the Direct Connect failover attribute within the AWS CLI doesn't automatically create a backup connection. The Direct Connect failover attribute allows you to influence which path traffic takes but does not instantiate backup connections. It influences the traffic routing, given the primary connection is already established. The failover attribute helps prioritize connections and adjust routing preferences but will not create connections from scratch.

In conclusion, a primary Direct Connect connection coupled with a VPN as a fallback mechanism delivers the best balance between high performance and cost efficiency, aligning perfectly with the company's requirements.

Further Research:

AWS Direct Connect: https://aws.amazon.com/directconnect/
AWS VPN: https://aws.amazon.com/vpn/

Question: 69 **Exam Heist**

A company is running a business-critical web application on Amazon EC2 instances behind an Application Load Balancer. The EC2 instances are in an Auto Scaling group. The application uses an Amazon Aurora PostgreSQL database that is deployed in a single Availability Zone. The company wants the application to be highly available with minimum downtime and minimum loss of data.
Which solution will meet these requirements with the LEAST operational effort?

A. Place the EC2 instances in different AWS Regions. Use Amazon Route 53 health checks to redirect traffic. Use Aurora PostgreSQL Cross-Region Replication.

B. Configure the Auto Scaling group to use multiple Availability Zones. Configure the database as Multi-AZ. Configure an Amazon RDS Proxy instance for the database.

C. Configure the Auto Scaling group to use one Availability Zone. Generate hourly snapshots of the database. Recover the database from the snapshots in the event of a failure.

D. Configure the Auto Scaling group to use multiple AWS Regions. Write the data from the application to Amazon S3. Use S3 Event Notifications to launch an AWS Lambda function to write the data to the database.

Answer: B

Explanation:

Option B is the most appropriate solution because it achieves high availability for both the application and the database with minimal operational overhead.

Here's a detailed breakdown:

Auto Scaling Group with Multiple Availability Zones (AZs): Spreading EC2 instances across multiple AZs ensures that if one AZ fails, the application remains available in other AZs. This provides redundancy and fault tolerance at the application tier.

Multi-AZ Aurora PostgreSQL: Configuring the database as Multi-AZ creates a synchronous standby replica in a different AZ. If the primary database instance fails, Aurora automatically fails over to the standby, minimizing downtime and data loss. This addresses the high availability requirement for the database.

Amazon RDS Proxy: RDS Proxy is an optional service that further enhances availability and scalability. It manages database connections, reducing the load on the database and protecting it from connection storms. While not strictly required for basic Multi-AZ failover, it improves connection management, especially under heavy load and failover situations.

Other options are less efficient or introduce unnecessary complexity:

Option A (Cross-Region Replication): Using cross-region replication for the database adds complexity and potential latency. It is also overkill since the application is running in a single region. Also, failover between regions generally introduces more downtime than a Multi-AZ failover.

Option C (Hourly Snapshots): Relying on snapshots for recovery introduces a significant recovery time objective (RTO) and recovery point objective (RPO). Data loss is possible up to the last snapshot. This is not suitable for a business-critical application requiring minimal downtime and data loss.

Option D (Multi-Region ASG and S3/Lambda integration): This introduces substantial architectural complexity for data persistence compared to using Aurora Multi-AZ. The S3/Lambda pathway for database writes introduces latency and increased operational overhead.

Therefore, configuring the ASG across multiple AZs, using Aurora Multi-AZ, and leveraging RDS Proxy provide the best balance between high availability, minimal downtime/data loss, and operational simplicity, making it the most suitable option.

Supporting Links:

Amazon Aurora High Availability: https://aws.amazon.com/rds/aurora/features/
Amazon RDS Proxy: https://aws.amazon.com/rds/proxy/
Auto Scaling Groups: https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-groups.html

Question: 70 **Exam Heist**

A company's HTTP application is behind a Network Load Balancer (NLB). The NLB's target group is configured to use an Amazon EC2 Auto Scaling group with multiple EC2 instances that run the web service.
The company notices that the NLB is not detecting HTTP errors for the application. These errors require a manual restart of the EC2 instances that run the web service. The company needs to improve the application's availability without writing custom scripts or code.
What should a solutions architect do to meet these requirements?

A. Enable HTTP health checks on the NLB, supplying the URL of the company's application.

B. Add a cron job to the EC2 instances to check the local application's logs once each minute. If HTTP errors are detected. the application will restart.

C. Replace the NLB with an Application Load Balancer. Enable HTTP health checks by supplying the URL of the company's application. Configure an Auto Scaling action to replace unhealthy instances.

D. Create an Amazon Cloud Watch alarm that monitors the UnhealthyHostCount metric for the NLB. Configure an Auto Scaling action to replace unhealthy instances when the alarm is in the ALARM state.

Answer: C

Explanation:

The correct answer is **C**, which involves replacing the Network Load Balancer (NLB) with an Application Load Balancer (ALB) and configuring HTTP health checks and an Auto Scaling action. Here's why:

NLB Limitations: NLBs operate at Layer 4 (TCP/UDP), meaning they are unaware of the application-layer protocol (HTTP) and cannot interpret HTTP status codes. They can only check if a TCP connection can be established on the configured port, not if the application is functioning correctly.
https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html

ALB's HTTP Awareness: ALBs, on the other hand, operate at Layer 7 (HTTP/HTTPS), enabling them to perform health checks based on HTTP status codes (e.g., 200 OK, 400 Bad Request, 500 Internal Server Error). This allows the ALB to determine if the application is truly healthy and route traffic accordingly.
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html

HTTP Health Checks: By configuring HTTP health checks on the ALB with a specific URL, the load balancer will periodically send requests to that URL and verify the HTTP status code returned. If the status code indicates an error, the ALB will mark the instance as unhealthy.

Auto Scaling Integration: The Auto Scaling group can be configured with lifecycle hooks or be configured directly to respond to ALB health check failures. When the ALB marks an instance as unhealthy, Auto Scaling can be configured to automatically replace the unhealthy instance with a new one, thus improving application availability without manual intervention. https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-process.html

Why other options are wrong:

A: NLBs cannot do HTTP health checks.
B: Requires custom scripting, and the company wants to avoid it. Also, it is less reliable than ALB health checks.
D: While CloudWatch can monitor the NLB, it cannot detect HTTP-level errors due to NLB's Layer 4 operation. Monitoring UnhealthyHostCount may only indicate that an EC2 instance is unreachable at the TCP level, not that the application is failing to respond to HTTP requests.

In summary, switching to an ALB enables HTTP health checks, which allows for the automated detection and replacement of unhealthy instances, fulfilling the company's requirements for improved application availability without custom scripting or code.

Question: 71 **Exam Heist**

A company runs a shopping application that uses Amazon DynamoDB to store customer information. In case of data corruption, a solutions architect needs to design a solution that meets a recovery point objective (RPO) of 15 minutes and a recovery time objective (RTO) of 1 hour.
What should the solutions architect recommend to meet these requirements?

A. Configure DynamoDB global tables. For RPO recovery, point the application to a different AWS Region.

B. Configure DynamoDB point-in-time recovery. For RPO recovery, restore to the desired point in time.

C. Export the DynamoDB data to Amazon S3 Glacier on a daily basis. For RPO recovery, import the data from S3 Glacier to DynamoDB.

D. Schedule Amazon Elastic Block Store (Amazon EBS) snapshots for the DynamoDB table every 15 minutes. For RPO recovery, restore the DynamoDB table by using the EBS snapshot.

Answer: B

Explanation:

The correct answer is B because it aligns with the stated RPO and RTO requirements most effectively and efficiently.

Here's why:

DynamoDB Point-in-Time Recovery (PITR): DynamoDB PITR provides automatic backups of your table data. You can restore the table to any point in time within the past 35 days. This perfectly addresses the need for recovering data to a specific point within the last 15 minutes (RPO). The restoration process also falls within the 1-hour RTO.
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/PointInTimeRecovery.html

Why other options are incorrect:

A. DynamoDB Global Tables: While Global Tables provide replication for high availability and disaster recovery, their primary purpose is not granular point-in-time recovery for data corruption scenarios within a single table. Switching regions might help avoid regional failures, but if the corruption is replicated, it won't solve the problem. They do not meet the specific RPO requirement of 15 minutes for data corruption within a table.

C. Export to S3 Glacier: Exporting data to S3 Glacier is a suitable strategy for long-term archiving and compliance, but it is too slow for a 1-hour RTO. Glacier is designed for infrequently accessed data, and retrieving data from Glacier can take several hours. This does not satisfy the 1-hour RTO requirement.

D. EBS Snapshots for DynamoDB: DynamoDB does not reside on EBS volumes. It is a NoSQL database service that internally handles storage. Taking EBS snapshots won't capture the data or the configuration of your DynamoDB table, and it's not the correct way to back up DynamoDB data.

In summary, DynamoDB PITR is the most appropriate solution as it offers the required granularity for recovery, meeting both the 15-minute RPO and the 1-hour RTO. Other options do not directly address the specific requirements of recovering from data corruption with the given recovery objectives.

Question: 72 Exam Heist

A company runs a photo processing application that needs to frequently upload and download pictures from Amazon S3 buckets that are located in the same AWS Region. A solutions architect has noticed an increased cost in data transfer fees and needs to implement a solution to reduce these costs.
How can the solutions architect meet this requirement?

A. Deploy Amazon API Gateway into a public subnet and adjust the route table to route S3 calls through it.

B. Deploy a NAT gateway into a public subnet and attach an endpoint policy that allows access to the S3 buckets.

C. Deploy the application into a public subnet and allow it to route through an internet gateway to access the S3 buckets.

D. Deploy an S3 VPC gateway endpoint into the VPC and attach an endpoint policy that allows access to the S3 buckets.

Answer: D

Explanation:

The correct solution is **D. Deploy an S3 VPC gateway endpoint into the VPC and attach an endpoint policy that allows access to the S3 buckets.**

Here's a detailed justification:

The core problem is the data transfer cost between the application and S3 within the same AWS Region. Data transfer to S3 is generally free. However, data transfer from S3 incurs costs, and these costs increase significantly when the data traverses the public internet.

Options A, B, and C all involve routing traffic via the public internet, defeating the purpose of cost reduction. Deploying API Gateway (A), NAT Gateway (B), or using an Internet Gateway directly (C) means the application is accessing S3 over the internet, incurring outbound data transfer fees. While API Gateway offers other benefits, in this scenario, it adds unnecessary complexity and cost. Similarly, NAT Gateways are for enabling instances in private subnets to access the internet, not to optimize S3 access.

Option D leverages a VPC endpoint, which establishes a direct, private connection between your VPC and S3, without traversing the public internet. A VPC endpoint specifically for S3 creates a gateway within your VPC that allows your application to access S3 buckets as if they were within the same network. This eliminates public internet data transfer costs, as traffic stays within the AWS network.

Furthermore, the endpoint policy attached to the S3 VPC gateway endpoint controls access to the S3 buckets. This policy specifies which S3 buckets and actions are allowed for resources within the VPC, enhancing security. It restricts access based on IAM roles, users, and conditions. Without the policy, the endpoint won't function.

Therefore, deploying an S3 VPC gateway endpoint and attaching an endpoint policy is the most efficient and cost-effective way to enable the photo processing application to access S3 buckets without incurring public internet data transfer costs. It keeps the data within the AWS internal network, reducing costs and improving security.

Here are authoritative links for further research:

AWS VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html
Gateway VPC Endpoints for S3: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-s3.html
S3 Pricing: https://aws.amazon.com/s3/pricing/ (Focus on the "Data Transfer" section)

Question: 73 Exam Heist

A company recently launched Linux-based application instances on Amazon EC2 in a private subnet and launched a Linux-based bastion host on an Amazon EC2 instance in a public subnet of a VPC. A solutions architect needs to connect from the on-premises network, through the company's internet connection, to the bastion host, and to the application servers. The solutions architect must make sure that the security groups of all the EC2 instances will allow that access.
Which combination of steps should the solutions architect take to meet these requirements? (Choose two.)

A. Replace the current security group of the bastion host with one that only allows inbound access from the application instances.

B. Replace the current security group of the bastion host with one that only allows inbound access from the internal IP range for the company.

C. Replace the current security group of the bastion host with one that only allows inbound access from the external IP range for the company.

D. Replace the current security group of the application instances with one that allows inbound SSH access from only the private IP address of the bastion host.

E. Replace the current security group of the application instances with one that allows inbound SSH access from only the public IP address of the bastion host.

Answer: CD

Explanation:

The correct answer is CD. Here's why:

C: The bastion host acts as a secure gateway to the application instances. To allow access from the on-premises network, the security group associated with the bastion host must permit inbound traffic from the company's external IP range. This restricts access to only traffic originating from the known and trusted company network, enhancing security. This is because all traffic from on-premise will come through the company's internet connection, which will have a specific, and usually static, external IP range.

D: The application instances in the private subnet should not be directly accessible from the internet. Instead, access is granted via the bastion host. Therefore, the application instances' security group should only allow inbound SSH traffic from the private IP address of the bastion host. This ensures that only traffic originating from the bastion host can connect to the application instances, restricting the attack surface. This implements a layered security approach.

Why the other options are incorrect:

A: Restricting inbound access to the bastion host to only application instances would prevent the on-premises network from accessing it.
B: Allowing the internal IP range of the company would not work, because traffic from the company will be routed through the internet, and therefore come from the external IP range of the company.
E: Using the public IP address of the bastion host in the application instance's security group is less secure because public IP addresses can sometimes change, and more importantly, defeats the purpose of using the bastion host as a secure intermediary, as the application servers are exposed to outside IPs.

Supporting Concepts and Links:

Bastion Host: A server whose purpose is to provide access to a private network from an external network, such as the internet. https://aws.amazon.com/quickstart/architecture/linux-bastion/
Security Groups: Act as a virtual firewall for your EC2 instances to control inbound and outbound traffic. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html
Principle of Least Privilege: Granting only the minimum necessary permissions. In this case, restricting access to the bastion host and application instances based on IP addresses aligns with this principle.

Question: 74 **Exam Heist**

A solutions architect is designing a two-tier web application. The application consists of a public-facing web tier hosted on Amazon EC2 in public subnets. The database tier consists of Microsoft SQL Server running on Amazon EC2 in a private subnet. Security is a high priority for the company.
How should security groups be configured in this situation? (Choose two.)

A. Configure the security group for the web tier to allow inbound traffic on port 443 from 0.0.0.0/0.

B. Configure the security group for the web tier to allow outbound traffic on port 443 from 0.0.0.0/0.

C. Configure the security group for the database tier to allow inbound traffic on port 1433 from the security group for the web tier.

D. Configure the security group for the database tier to allow outbound traffic on ports 443 and 1433 to the security group for the web tier.

E. Configure the security group for the database tier to allow inbound traffic on ports 443 and 1433 from the security group for the web tier.

Answer: AC

Explanation:

Here's a detailed justification for why options A and C are the correct choices for configuring security groups in the given two-tier web application scenario:

Option A: Configure the security group for the web tier to allow inbound traffic on port 443 from 0.0.0.0/0.

This configuration allows HTTPS traffic (port 443) from any source (0.0.0.0/0) to reach the web servers. Since the web tier is public-facing, it needs to accept inbound traffic from users on the internet. HTTPS is the standard secure protocol for web communication, encrypting data in transit. Restricting inbound access to port 443 (and potentially port 80 for HTTP redirects to HTTPS) minimizes the attack surface. A more restrictive approach could involve limiting access to specific IP ranges of known users or employing a Web Application Firewall (WAF) for more granular control.

Option C: Configure the security group for the database tier to allow inbound traffic on port 1433 from the security group for the web tier.

This configuration allows traffic on port 1433 (the default port for Microsoft SQL Server) from the web tier's security group to reach the database tier. This is crucial for the web servers to connect to the database server. Instead of specifying IP addresses, using the web tier's security group ensures that only instances within that group can connect to the database. This dynamic association simplifies management, as new web servers added to the web tier automatically gain database access without manually updating IP addresses in the database security group. The database tier should only allow traffic from the web tier and no other sources.

Why other options are incorrect:

Option B: The web tier needs to make outbound calls, but likely not on port 443 to 0.0.0.0/0. The web tier might need outbound access to other AWS services or external APIs, but this would be governed by different rules and likely restricted to specific services or IP ranges rather than the entire internet on a particular port like 443.

Option D and E: The database tier only needs to accept inbound connections from the web tier on port 1433. It does not need to initiate outbound traffic to the web tier. Option E is also incorrect in that it allows inbound traffic on port 443, which is unneeded and increases the attack surface.

Key Cloud Computing Concepts Reinforced:

Security Groups: Acting as virtual firewalls controlling inbound and outbound traffic at the instance level.
Least Privilege Principle: Granting only the necessary permissions to resources, minimizing potential damage

from security breaches.
Two-Tier Architecture: A common application architecture separating the presentation tier (web tier) from the data tier (database tier).
Network Segmentation: Isolating resources (like the database tier in a private subnet) to reduce the blast radius of security incidents.

Authoritative Links for Further Research:

AWS Security Groups: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html
Security Best Practices in AWS: https://aws.amazon.com/security/

Question: 75 **Exam Heist**

A company wants to move a multi-tiered application from on premises to the AWS Cloud to improve the application's performance. The application consists of application tiers that communicate with each other by way of RESTful services. Transactions are dropped when one tier becomes overloaded. A solutions architect must design a solution that resolves these issues and modernizes the application.
Which solution meets these requirements and is the MOST operationally efficient?

A. Use Amazon API Gateway and direct transactions to the AWS Lambda functions as the application layer. Use Amazon Simple Queue Service (Amazon SQS) as the communication layer between application services.

B. Use Amazon CloudWatch metrics to analyze the application performance history to determine the servers' peak utilization during the performance failures. Increase the size of the application server's Amazon EC2 instances to meet the peak requirements.

C. Use Amazon Simple Notification Service (Amazon SNS) to handle the messaging between application servers running on Amazon EC2 in an Auto Scaling group. Use Amazon CloudWatch to monitor the SNS queue length and scale up and down as required.

D. Use Amazon Simple Queue Service (Amazon SQS) to handle the messaging between application servers running on Amazon EC2 in an Auto Scaling group. Use Amazon CloudWatch to monitor the SQS queue length and scale up when communication failures are detected.

Answer: A

Explanation:

Here's a breakdown of why option A is the best choice and why the others fall short, along with supporting concepts and links.

Option A (API Gateway + Lambda + SQS) offers the most operationally efficient and modern solution for the company's requirements. Let's dissect it:

Amazon API Gateway: Acts as a front door for the application. It allows the company to manage API access, authorization, throttling, and versioning in a centralized and scalable manner. It abstracts the underlying complexities of the backend services. https://aws.amazon.com/api-gateway/

AWS Lambda: Enables the company to modernize its application by leveraging serverless computing. Lambda functions can handle individual requests or transactions without the need to manage servers. This significantly reduces operational overhead and allows for automatic scaling based on demand. Lambda can also integrate directly with API Gateway. https://aws.amazon.com/lambda/

Amazon SQS: Introduces asynchronous communication between the application tiers. When one tier is overloaded, SQS acts as a buffer, preventing transaction drops. The application can decouple its components so that they can fail or go down at any time. SQS provides durability and fault tolerance.
https://aws.amazon.com/sqs/

Why other options are less suitable:

Option B: Increasing EC2 instance sizes (vertical scaling) might alleviate the problem in the short term, but it's not a modern or efficient solution. It does not address the root cause of the issue and does not allow you to decouple components. It involves unnecessary resource allocation even during periods of low demand.

Option C: While SNS can handle messaging, it's primarily designed for push notifications and fanout scenarios. It doesn't provide the same level of queuing and buffering capabilities as SQS, making it less suitable for preventing transaction drops during overload. Also, running application servers on EC2 within an Auto Scaling group is not as operationally efficient as Lambda, which eliminates server management.

Option D: Using SQS is beneficial, but continuing to run application servers on EC2 instances with Auto Scaling groups isn't as operationally efficient as using Lambda. Lambda reduces management overhead by abstracting the underlying infrastructure. EC2 instances can add management overhead in the long term.

Question: 76 **Exam Heist**

A company receives 10 TB of instrumentation data each day from several machines located at a single factory. The data consists of JSON files stored on a storage area network (SAN) in an on-premises data center located within the factory. The company wants to send this data to Amazon S3 where it can be accessed by several additional systems that provide critical near-real-time analytics. A secure transfer is important because the data is considered sensitive.
Which solution offers the MOST reliable data transfer?

A. AWS DataSync over public internet
B. AWS DataSync over AWS Direct Connect
C. AWS Database Migration Service (AWS DMS) over public internet
D. AWS Database Migration Service (AWS DMS) over AWS Direct Connect

Answer: B

Explanation:

The correct answer is **B. AWS DataSync over AWS Direct Connect**. Here's a detailed justification:

Data Volume and Real-Time Analytics: The company generates a significant 10 TB of data daily and requires near-real-time analytics. This necessitates a robust and efficient transfer mechanism.
Reliability and Security: Secure transfer is paramount due to the sensitive nature of the data.
AWS DataSync: AWS DataSync is a purpose-built service designed for moving large amounts of data between on-premises storage and AWS services. It handles data encryption in transit and at rest, ensuring data security. DataSync is optimized for network utilization, maximizing transfer speeds and minimizing transfer times. https://aws.amazon.com/datasync/
AWS Direct Connect: AWS Direct Connect establishes a dedicated network connection from the company's on-premises data center to AWS. This connection bypasses the public internet, providing a more reliable, secure, and consistent network experience. Direct Connect significantly reduces network latency and increases bandwidth compared to internet-based transfers. https://aws.amazon.com/directconnect/
Why A is less ideal: While AWS DataSync is appropriate, transferring over the public internet (option A) introduces vulnerabilities to security breaches and network congestion, making the transfer less reliable and secure than using a dedicated connection.
Why C and D are incorrect: AWS Database Migration Service (DMS) is primarily for migrating databases. While it could technically transfer JSON files stored as BLOBs, it's not designed or optimized for this purpose. DataSync is much more efficient and appropriate for large file transfers. Additionally, DMS adds unnecessary complexity and overhead for a simple file transfer scenario.
Combined benefits of DataSync and Direct Connect: By combining AWS DataSync with AWS Direct Connect, the company gets a secure, reliable, and high-throughput data transfer pipeline. Direct Connect minimizes latency and ensures consistent network performance, while DataSync optimizes the transfer process. The

encryption during transfer provided by DataSync in conjunction with Direct Connect reduces exposure of the data.
Consistency: Consistent, reliable data transfer is crucial for supporting near-real-time analytics because interruptions or delays in the data transfer can result in outdated or incomplete analyses.
Cost considerations: While Direct Connect involves additional costs, the improved reliability, security, and throughput may justify the investment, especially considering the critical nature of the data and analytics. Therefore, using AWS DataSync over AWS Direct Connect is the most reliable data transfer solution for transferring the instrumentation data to Amazon S3 because it offers enhanced security, dedicated bandwidth, and optimized transfer mechanisms.

Question: 77 **Exam Heist**

A company needs to configure a real-time data ingestion architecture for its application. The company needs an API, a process that transforms data as the data is streamed, and a storage solution for the data.
Which solution will meet these requirements with the LEAST operational overhead?

A. Deploy an Amazon EC2 instance to host an API that sends data to an Amazon Kinesis data stream. Create an Amazon Kinesis Data Firehose delivery stream that uses the Kinesis data stream as a data source. Use AWS Lambda functions to transform the data. Use the Kinesis Data Firehose delivery stream to send the data to Amazon S3.

B. Deploy an Amazon EC2 instance to host an API that sends data to AWS Glue. Stop source/destination checking on the EC2 instance. Use AWS Glue to transform the data and to send the data to Amazon S3.

C. Configure an Amazon API Gateway API to send data to an Amazon Kinesis data stream. Create an Amazon Kinesis Data Firehose delivery stream that uses the Kinesis data stream as a data source. Use AWS Lambda functions to transform the data. Use the Kinesis Data Firehose delivery stream to send the data to Amazon S3.

D. Configure an Amazon API Gateway API to send data to AWS Glue. Use AWS Lambda functions to transform the data. Use AWS Glue to send the data to Amazon S3.

Answer: C

Explanation:

The correct answer is C because it offers a serverless, scalable, and managed solution for real-time data ingestion with minimal operational overhead.

Here's why:

API Gateway: Amazon API Gateway allows you to create and manage APIs without managing any servers. It handles tasks like traffic management, authorization, and monitoring, significantly reducing operational burden compared to hosting an API on an EC2 instance. (Reference: https://aws.amazon.com/api-gateway/)

Kinesis Data Streams: Kinesis Data Streams is a fully managed, scalable, and durable real-time data streaming service. It's designed to ingest high-velocity data streams. Using it directly as the target of the API ensures immediate ingestion.

Kinesis Data Firehose: Kinesis Data Firehose is designed for loading data into data lakes and data warehouses. Using it with Kinesis Data Streams as a source is a common pattern. Firehose automatically scales, buffers, and compresses data. (Reference: https://aws.amazon.com/kinesis/data-firehose/)

Lambda Functions: Lambda allows you to execute code without provisioning or managing servers. Using Lambda functions within the Kinesis Data Firehose delivery stream enables real-time data transformations without the operational overhead of managing EC2 instances. This is done during data loading to S3 by Firehose (Reference: https://docs.aws.amazon.com/firehose/latest/dev/data-transformation.html)

S3: Amazon S3 provides highly durable and scalable object storage, ideal for storing the transformed data.

Why other options are less suitable:

Option A: Hosting the API on an EC2 instance increases operational overhead due to server management, patching, scaling, and monitoring.
Option B: AWS Glue is primarily designed for ETL (Extract, Transform, Load) jobs, which are typically batch-oriented, and is not the optimal solution for real-time data ingestion. AWS Glue is also overkill for the simple transformation needed in this case. The "stop source/destination checking" is also a security concern and not needed with the API Gateway/Kinesis solution.
Option D: Similar to B, using AWS Glue for data transformation is generally better suited for batch-oriented ETL pipelines, and API Gateway doesn't natively integrate with it for real-time streaming.

Therefore, option C is the most efficient and cost-effective choice for real-time data ingestion with the least operational overhead by leveraging serverless services.

Question: 78 **Exam Heist**

A company needs to keep user transaction data in an Amazon DynamoDB table. The company must retain the data for 7 years.
What is the MOST operationally efficient solution that meets these requirements?

A. Use DynamoDB point-in-time recovery to back up the table continuously.

B. Use AWS Backup to create backup schedules and retention policies for the table.

C. Create an on-demand backup of the table by using the DynamoDB console. Store the backup in an Amazon S3 bucket. Set an S3 Lifecycle configuration for the S3 bucket.

D. Create an Amazon EventBridge (Amazon CloudWatch Events) rule to invoke an AWS Lambda function. Configure the Lambda function to back up the table and to store the backup in an Amazon S3 bucket. Set an S3 Lifecycle configuration for the S3 bucket.

Answer: B

Explanation:

The correct answer is B: Use AWS Backup to create backup schedules and retention policies for the table.
This is the most operationally efficient solution for retaining DynamoDB data for 7 years for several reasons:

1. **Centralized Backup Management:** AWS Backup provides a single pane of glass to manage backups across various AWS services, including DynamoDB. This simplifies backup management and reduces the operational overhead associated with managing backups using separate service-specific tools. https://aws.amazon.com/backup/
2. **Automated Scheduling and Retention:** AWS Backup allows defining backup schedules and retention policies that automatically handle the creation and deletion of backups based on the defined parameters. This eliminates the need for manual intervention and ensures compliance with the 7-year retention requirement.
3. **Compliance and Auditing:** AWS Backup offers features for compliance reporting and auditing, allowing you to demonstrate adherence to data retention policies. https://docs.aws.amazon.com/aws-backup/latest/devguide/whatis.html
4. **Cost Optimization:** AWS Backup optimizes backup storage by using incremental backups and data compression, which helps reduce storage costs.

Now, let's look at why other options are less efficient:

A. **DynamoDB Point-in-Time Recovery (PITR):** While PITR allows restoring the table to any point in time within

the past 35 days, it doesn't satisfy the 7-year retention requirement.
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/PointInTimeRecovery.html

C. **On-Demand Backups with S3 Lifecycle:** Creating on-demand backups manually is not operationally efficient because it requires manual intervention and scheduling. Also, managing backups in S3 requires configuring S3 Lifecycle rules, adding operational complexity. The responsibility of remembering to perform these tasks remains with the user.

D. **EventBridge, Lambda, and S3 Lifecycle:** While this approach can meet the retention requirement, it involves more complex configurations than using AWS Backup. Building and maintaining a custom Lambda function introduces operational overhead for development, testing, and maintenance. The management of the S3 lifecycle, though automated, adds to the solution's complexity compared to AWS Backup.

Question: 79 **Exam Heist**

A company is planning to use an Amazon DynamoDB table for data storage. The company is concerned about cost optimization. The table will not be used on most mornings. In the evenings, the read and write traffic will often be unpredictable. When traffic spikes occur, they will happen very quickly.
What should a solutions architect recommend?

A. Create a DynamoDB table in on-demand capacity mode.
B. Create a DynamoDB table with a global secondary index.
C. Create a DynamoDB table with provisioned capacity and auto scaling.
D. Create a DynamoDB table in provisioned capacity mode, and configure it as a global table.

Answer: A

Explanation:

Here's a detailed justification for why option A (Create a DynamoDB table in on-demand capacity mode) is the most appropriate solution for the scenario described:

The scenario highlights two key cost optimization concerns: the table is largely unused during mornings, and traffic is unpredictable with rapid spikes during evenings. On-demand capacity mode is ideally suited for these situations. In on-demand capacity mode, DynamoDB automatically scales capacity in response to actual workload needs. This means you only pay for the reads and writes your application performs, without needing to provision capacity upfront. This aligns perfectly with the morning downtime where no costs are incurred.

Because traffic spikes happen rapidly, provisioned capacity with autoscaling (option C) might not react quickly enough. Autoscaling takes some time to adjust capacity, potentially leading to throttling during the initial moments of a spike. On-demand mode, on the other hand, scales instantly.

A global secondary index (option B) is useful for querying data using attributes other than the primary key but doesn't address the cost optimization and unpredictable traffic concerns. A global table (option D) is for multi-region replication for disaster recovery and low-latency access in different geographical areas, which is also irrelevant to the given problem. It would simply increase costs without providing a benefit.

Therefore, on-demand capacity mode provides the best balance of cost efficiency during periods of low activity and responsiveness to unpredictable, rapid traffic spikes by eliminating the need for upfront capacity planning and enabling automatic and instant scaling.

Here are some resources for further reading:

DynamoDB On-Demand Capacity:
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadWriteCapacityMode.html

DynamoDB Auto Scaling:
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/AutoScaling.html

Question: 80 **Exam Heist**

A company recently signed a contract with an AWS Managed Service Provider (MSP) Partner for help with an application migration initiative. A solutions architect needs ta share an Amazon Machine Image (AMI) from an existing AWS account with the MSP Partner's AWS account. The AMI is backed by Amazon Elastic Block Store (Amazon EBS) and uses an AWS Key Management Service (AWS KMS) customer managed key to encrypt EBS volume snapshots.
What is the MOST secure way for the solutions architect to share the AMI with the MSP Partner's AWS account?

A. Make the encrypted AMI and snapshots publicly available. Modify the key policy to allow the MSP Partner's AWS account to use the key.

B. Modify the launchPermission property of the AMI. Share the AMI with the MSP Partner's AWS account only. Modify the key policy to allow the MSP Partner's AWS account to use the key.

C. Modify the launchPermission property of the AMI. Share the AMI with the MSP Partner's AWS account only. Modify the key policy to trust a new KMS key that is owned by the MSP Partner for encryption.

D. Export the AMI from the source account to an Amazon S3 bucket in the MSP Partner's AWS account, Encrypt the S3 bucket with a new KMS key that is owned by the MSP Partner. Copy and launch the AMI in the MSP Partner's AWS account.

Answer: B

Explanation:

The correct answer is B. Here's why:

A is incorrect: Making the AMI and snapshots publicly available is highly insecure. It exposes the company's data to anyone, violating confidentiality and compliance requirements.

B is correct: Modifying the launchPermission property of the AMI allows you to selectively share the AMI with the MSP Partner's AWS account. This approach limits access to only the intended recipient. Crucially, because the EBS volumes are encrypted with a KMS key, the MSP Partner's account needs permission to use that key. Modifying the key policy to grant the MSP Partner's AWS account the necessary permissions enables them to launch instances from the AMI. This balances security with functionality, allowing the MSP Partner to use the shared AMI while maintaining control over access to the underlying data.

C is incorrect: While sharing the AMI with the MSP's account is correct, modifying the key policy to trust a new KMS key owned by the MSP is less straightforward and potentially less secure. This would necessitate re-encrypting the EBS volumes using the new key, adding unnecessary complexity and potential for data loss or corruption. Also, it requires the original account to relinquish some control over the encryption process. It is generally best practice to maintain control of KMS keys used to encrypt your data whenever possible.

D is incorrect: Exporting the AMI and storing it in an S3 bucket, even if encrypted with the MSP's KMS key, introduces unnecessary complexity and potential vulnerabilities. It involves creating a copy of the AMI, which can increase storage costs and introduce inconsistencies. Furthermore, exporting and importing AMIs can be time-consuming and may require downtime. Sharing the AMI directly via launchPermission is a more efficient and secure approach.

In summary, option B provides the most secure and efficient way to share the encrypted AMI with the MSP Partner by granting the MSP Partner access to the existing KMS key used to encrypt the EBS volumes, without exposing the AMI publicly or needing to create copies.

Supporting links:

Sharing AMIs: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/sharingamis-intro.html
KMS Key Policies: https://docs.aws.amazon.com/kms/latest/developerguide/key-policies.html
Encryption at Rest with KMS: https://docs.aws.amazon.com/kms/latest/developerguide/services-supported.html

Question: 81 **Exam Heist**

A solutions architect is designing the cloud architecture for a new application being deployed on AWS. The process should run in parallel while adding and removing application nodes as needed based on the number of jobs to be processed. The processor application is stateless. The solutions architect must ensure that the application is loosely coupled and the job items are durably stored.
Which design should the solutions architect use?

A. Create an Amazon SNS topic to send the jobs that need to be processed. Create an Amazon Machine Image (AMI) that consists of the processor application. Create a launch configuration that uses the AMI. Create an Auto Scaling group using the launch configuration. Set the scaling policy for the Auto Scaling group to add and remove nodes based on CPU usage.

B. Create an Amazon SQS queue to hold the jobs that need to be processed. Create an Amazon Machine Image (AMI) that consists of the processor application. Create a launch configuration that uses the AMI. Create an Auto Scaling group using the launch configuration. Set the scaling policy for the Auto Scaling group to add and remove nodes based on network usage.

C. Create an Amazon SQS queue to hold the jobs that need to be processed. Create an Amazon Machine Image (AMI) that consists of the processor application. Create a launch template that uses the AMI. Create an Auto Scaling group using the launch template. Set the scaling policy for the Auto Scaling group to add and remove nodes based on the number of items in the SQS queue.

D. Create an Amazon SNS topic to send the jobs that need to be processed. Create an Amazon Machine Image (AMI) that consists of the processor application. Create a launch template that uses the AMI. Create an Auto Scaling group using the launch template. Set the scaling policy for the Auto Scaling group to add and remove nodes based on the number of messages published to the SNS topic.

Answer: C

Explanation:

The correct answer is C because it utilizes Amazon SQS for durable and loosely coupled job storage and an Auto Scaling group configured to scale based on the queue's depth, aligning with the requirements. Here's a breakdown:

Amazon SQS for Durable Storage and Decoupling: SQS (Simple Queue Service) acts as a message queue, providing durable storage for the jobs. This ensures that jobs are not lost even if application instances fail and promotes loose coupling between the job producer and the processor. https://aws.amazon.com/sqs/

AMI and Launch Template: The Amazon Machine Image (AMI) provides a template for launching EC2 instances, and the launch template specifies the configuration for instances within the Auto Scaling group, including the AMI, instance type, and security groups. Using Launch Templates over Launch Configurations is generally recommended due to their added functionality.
https://docs.aws.amazon.com/autoscaling/ec2/userguide/launch-templates-vs-launch-configurations.html

Auto Scaling Group: The Auto Scaling group allows for the dynamic provisioning and termination of EC2 instances based on defined policies. This enables the application to scale up or down depending on the job load. https://aws.amazon.com/autoscaling/

Scaling Policy Based on SQS Queue Depth: By setting the scaling policy to add or remove nodes based on the number of items in the SQS queue, the Auto Scaling group automatically adjusts the number of processing instances to match the workload. This ensures efficient resource utilization and timely processing of jobs.

Options A and D incorrectly use Amazon SNS. SNS (Simple Notification Service) is designed for publish-subscribe messaging, suitable for broadcasting messages to multiple subscribers but not for queueing jobs that need to be reliably processed one by one. SNS does not provide built-in durability in the same way as SQS. https://aws.amazon.com/sns/

Option B uses network usage for scaling. Scaling based on network usage is not directly tied to the number of jobs waiting to be processed, therefore not meeting the requirement efficiently.

Question: 82 **Exam Heist**

A company hosts its web applications in the AWS Cloud. The company configures Elastic Load Balancers to use certificates that are imported into AWS Certificate Manager (ACM). The company's security team must be notified 30 days before the expiration of each certificate.
What should a solutions architect recommend to meet this requirement?

A. Add a rule in ACM to publish a custom message to an Amazon Simple Notification Service (Amazon SNS) topic every day, beginning 30 days before any certificate will expire.

B. Create an AWS Config rule that checks for certificates that will expire within 30 days. Configure Amazon EventBridge (Amazon CloudWatch Events) to invoke a custom alert by way of Amazon Simple Notification Service (Amazon SNS) when AWS Config reports a noncompliant resource.

C. Use AWS Trusted Advisor to check for certificates that will expire within 30 days. Create an Amazon CloudWatch alarm that is based on Trusted Advisor metrics for check status changes. Configure the alarm to send a custom alert by way of Amazon Simple Notification Service (Amazon SNS).

D. Create an Amazon EventBridge (Amazon CloudWatch Events) rule to detect any certificates that will expire within 30 days. Configure the rule to invoke an AWS Lambda function. Configure the Lambda function to send a custom alert by way of Amazon Simple Notification Service (Amazon SNS).

Answer: B

Explanation:

Here's a detailed justification for why option B is the best solution:

Option B: Create an AWS Config rule that checks for certificates that will expire within 30 days. Configure Amazon EventBridge (Amazon CloudWatch Events) to invoke a custom alert by way of Amazon Simple Notification Service (Amazon SNS) when AWS Config reports a noncompliant resource.

This approach leverages the strengths of three AWS services for proactive certificate expiration management:

1. **AWS Config:** AWS Config continuously monitors and records the configuration of your AWS resources, including ACM certificates. You can define Config rules to evaluate whether your resources comply with desired configurations. In this case, you'd create a Config rule that checks if any ACM certificates are expiring within the next 30 days. If a certificate is found to be expiring soon, the rule marks the certificate as "noncompliant."

2. **Amazon EventBridge (Amazon CloudWatch Events):** EventBridge enables you to react to changes in your AWS environment. We can configure an EventBridge rule to listen for events from AWS Config related to "noncompliant" resource changes. Specifically, we're looking for events indicating that a certificate has been flagged as noncompliant because it's nearing expiration.

3. **Amazon Simple Notification Service (Amazon SNS):** SNS is a messaging service that facilitates sending notifications. EventBridge, upon detecting a noncompliant certificate via Config, triggers the SNS topic, which in turn sends a notification to the security team.

Why this is superior to the other options:

Option A (ACM Rule): ACM itself doesn't directly offer native rules to trigger notifications based on certificate expiry dates. While ACM manages renewals, creating daily custom messages and managing logic is inefficient and not within ACM's intended functionality.

Option C (Trusted Advisor): While Trusted Advisor provides best practice checks, including certificate expiration, it's not designed for real-time, automated notifications. Relying solely on Trusted Advisor requires manual checks or CloudWatch Alarms based on limited metrics, which is less proactive and harder to customize compared to AWS Config. Additionally, Trusted Advisor might have limitations on the frequency of checks.

Option D (EventBridge and Lambda): Although feasible, this option is more complex. EventBridge would need a custom pattern to identify certificates expiring within 30 days. A Lambda function would then be needed to parse the event data and send the SNS notification. This adds overhead in terms of development, maintenance, and potential for Lambda function errors. The native Config rule simplifies the process.

Benefits of Option B:

Automation: Fully automated monitoring and notification process.
Proactive: Provides notification 30 days before expiration, allowing ample time for renewal or replacement.
Scalable: Easily scales to handle a large number of certificates.
Centralized Configuration Management: Leverages the strengths of AWS Config for infrastructure-as-code and compliance tracking.
Minimal Custom Code: Relies on managed services, reducing the need for custom code and associated maintenance.

Supporting Links:

AWS Config: https://aws.amazon.com/config/
AWS EventBridge: https://aws.amazon.com/eventbridge/
Amazon SNS: https://aws.amazon.com/sns/
AWS Certificate Manager (ACM): https://aws.amazon.com/certificate-manager/

In summary, Option B provides the most efficient, scalable, and manageable solution by integrating AWS Config for compliance checks, EventBridge for event-driven automation, and SNS for reliable notifications. It minimizes custom code while providing proactive and automated certificate expiration monitoring.

Question: 83 **Exam Heist**

A company's dynamic website is hosted using on-premises servers in the United States. The company is launching its product in Europe, and it wants to optimize site loading times for new European users. The site's backend must remain in the United States. The product is being launched in a few days, and an immediate solution is needed. What should the solutions architect recommend?

A. Launch an Amazon EC2 instance in us-east-1 and migrate the site to it.
B. Move the website to Amazon S3. Use Cross-Region Replication between Regions.
C. Use Amazon CloudFront with a custom origin pointing to the on-premises servers.
D. Use an Amazon Route 53 geoproximity routing policy pointing to on-premises servers.

Answer: C

Explanation:

The correct answer is C: Use Amazon CloudFront with a custom origin pointing to the on-premises servers. Here's why:

Explanation:

CloudFront is a Content Delivery Network (CDN) service. It caches static and dynamic content at edge locations around the world. By using CloudFront with a custom origin (pointing to the on-premises servers in the United States), the company can serve the website's content from locations closer to European users, significantly reducing latency and improving loading times. This immediate solution addresses the primary need of optimizing site loading times for European users without requiring significant infrastructure changes or backend migration.

Option A is incorrect because migrating the entire site to a new EC2 instance in us-east-1 will not improve performance for European users. It is still geographically distant from them.

Option B is incorrect because moving the entire website to S3 and using Cross-Region Replication does not address the dynamic nature of the website. S3 is best suited for static content. Also, replicating the entire website to a new region is an unnecessary and time-consuming solution if the backend must remain in the US.

Option D is incorrect because Route 53 geoproximity routing can route traffic based on geographical location, it does not cache content. Therefore, while it can direct European users to the US servers, the latency issues associated with distance remain.

Why CloudFront is the best immediate solution:

CDN Caching: CloudFront caches content at edge locations in Europe, minimizing the distance data has to travel to reach users.
Custom Origin: The website backend remaining in the US allows the application logic to remain where it is, enabling quick implementation without significant changes to the current architecture.
Fast Implementation: CloudFront is designed to be easy to set up and integrate with existing infrastructure, allowing for a rapid rollout.
Cost-Effective: Compared to replicating the entire website to a new region, using CloudFront is more cost-effective. It only caches necessary content.
Dynamic content acceleration: CloudFront can cache dynamic content.

Supporting Links:

Amazon CloudFront: https://aws.amazon.com/cloudfront/
Using Custom Origins with CloudFront:
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-custom-origin.html

Question: 84 **Exam Heist**

A company wants to reduce the cost of its existing three-tier web architecture. The web, application, and database servers are running on Amazon EC2 instances for the development, test, and production environments. The EC2 instances average 30% CPU utilization during peak hours and 10% CPU utilization during non-peak hours.
The production EC2 instances run 24 hours a day. The development and test EC2 instances run for at least 8 hours each day. The company plans to implement automation to stop the development and test EC2 instances when they are not in use.
Which EC2 instance purchasing solution will meet the company's requirements MOST cost-effectively?

A. Use Spot Instances for the production EC2 instances. Use Reserved Instances for the development and test EC2 instances.

B. Use Reserved Instances for the production EC2 instances. Use On-Demand Instances for the development and test EC2 instances.

C. Use Spot blocks for the production EC2 instances. Use Reserved Instances for the development and test EC2 instances.

D. Use On-Demand Instances for the production EC2 instances. Use Spot blocks for the development and test EC2 instances.

Answer: B

Explanation:

The optimal solution balances cost savings and operational needs. Option B, using Reserved Instances for production and On-Demand instances for development/test, achieves this balance most effectively.

Production instances run 24/7. Reserved Instances provide significant cost savings (up to 75%) compared to On-Demand pricing for long-term, consistent usage. Committing to Reserved Instances for production ensures predictable pricing and lower overall expenses since production workload is constant. https://aws.amazon.com/ec2/pricing/reserved-instances/

Development and test instances are only needed for at least 8 hours a day. While automation will shut them down when not in use, Spot Instances (Option A & D) are unsuitable for development and test environments. Spot Instances can be interrupted with little notice, disrupting crucial testing or development tasks. Spot Blocks (Option C & D) offer more reliability but typically involve higher prices than On-Demand.

Since the instances are only run for 8 hours, On-Demand Instances offer the most cost-effective option for Development and Test environments. They provide flexibility without any long-term commitments or risk of interruption. The minimal usage duration does not justify the upfront investment and commitment required for Reserved Instances. https://aws.amazon.com/ec2/pricing/on-demand/

Therefore, Reserved Instances for the stable, 24/7 production environment and On-Demand Instances for the shorter, flexible development and test environments gives the best cost benefit.

Question: 85 **Exam Heist**

A company has a production web application in which users upload documents through a web interface or a mobile app. According to a new regulatory requirement. new documents cannot be modified or deleted after they are stored.
What should a solutions architect do to meet this requirement?

A. Store the uploaded documents in an Amazon S3 bucket with S3 Versioning and S3 Object Lock enabled.

B. Store the uploaded documents in an Amazon S3 bucket. Configure an S3 Lifecycle policy to archive the documents periodically.

C. Store the uploaded documents in an Amazon S3 bucket with S3 Versioning enabled. Configure an ACL to restrict all access to read-only.

D. Store the uploaded documents on an Amazon Elastic File System (Amazon EFS) volume. Access the data by mounting the volume in read-only mode.

Answer: A

Explanation:

The correct answer is A: Store the uploaded documents in an Amazon S3 bucket with S3 Versioning and S3 Object Lock enabled. This solution directly addresses the regulatory requirement that documents cannot be modified or deleted after storage. Let's break down why this works and why other options are less suitable.

Justification for Option A:

S3 Versioning: Enabling S3 Versioning ensures that every change to an object in the bucket creates a new version. This means that if a user attempts to modify a document, the original version remains intact, fulfilling the immutability requirement.
S3 Object Lock: S3 Object Lock prevents objects from being deleted or overwritten for a fixed amount of time or indefinitely. There are two modes:

Governance mode: Allows certain users with specific permissions to bypass the lock.
Compliance mode: Prevents anyone, including the root user, from deleting or overwriting the locked object. This is crucial for meeting strict regulatory requirements.
Combination is Key: Using S3 Versioning alone only ensures previous versions are kept; it doesn't prevent deletion. S3 Object Lock, combined with versioning, provides the complete immutability guarantee.

Why Other Options Are Incorrect:

Option B: S3 with Lifecycle Policy: While S3 Lifecycle policies are useful for archiving data to cheaper storage tiers, they don't prevent modification or deletion during the active period. Archiving simply moves the data; it doesn't lock it.

Option C: S3 Versioning and ACL: While S3 Versioning ensures that previous versions are retained, it doesn't, by itself, prevent deletion of objects. Additionally, ACLs can be complex to manage and can be easily misconfigured, making them less reliable for regulatory compliance. They also don't provide deletion prevention like Object Lock.

Option D: Amazon EFS in Read-Only Mode: EFS is a network file system, and while mounting it in read-only mode would prevent modifications through the mount point, it doesn't guarantee immutability. An administrator with access to the underlying EFS infrastructure could still modify or delete the files. EFS also lacks built-in versioning and object locking features designed for immutability like S3.

In summary: S3 Versioning and S3 Object Lock, particularly in Compliance mode, offer a robust and easily auditable solution to guarantee that documents cannot be modified or deleted, fulfilling the stringent regulatory requirement. S3's features are specifically designed for object storage and data durability with compliance in mind.

Authoritative Links:

S3 Versioning: https://docs.aws.amazon.com/AmazonS3/latest/userguide/versioning-workflows.html
S3 Object Lock: https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html

Question: 86 **Exam Heist**

A company has several web servers that need to frequently access a common Amazon RDS MySQL Multi-AZ DB instance. The company wants a secure method for the web servers to connect to the database while meeting a security requirement to rotate user credentials frequently.
Which solution meets these requirements?

A. Store the database user credentials in AWS Secrets Manager. Grant the necessary IAM permissions to allow the web servers to access AWS Secrets Manager.

B. Store the database user credentials in AWS Systems Manager OpsCenter. Grant the necessary IAM permissions to allow the web servers to access OpsCenter.

C. Store the database user credentials in a secure Amazon S3 bucket. Grant the necessary IAM permissions to allow the web servers to retrieve credentials and access the database.

D. Store the database user credentials in files encrypted with AWS Key Management Service (AWS KMS) on the web server file system. The web server should be able to decrypt the files and access the database.

Answer: A

Explanation:

The best solution is A: Store the database user credentials in AWS Secrets Manager and grant web servers IAM permissions to access it.

Here's why:

Secure Credential Storage: AWS Secrets Manager is specifically designed for managing secrets like database credentials. It encrypts secrets at rest and in transit, providing a high level of security.
Automatic Rotation: Secrets Manager allows automatic rotation of database credentials, fulfilling the requirement for frequent credential rotation. This significantly reduces the risk associated with long-lived credentials.
Centralized Management: It offers a central location to manage and control access to the database credentials. Changes can be made easily and applied across all web servers.
IAM Integration: Secrets Manager integrates seamlessly with IAM, allowing fine-grained control over which web servers can access specific secrets. This adheres to the principle of least privilege.
Auditing: AWS CloudTrail logs all access to secrets, providing a comprehensive audit trail for compliance and security monitoring.

Options B, C, and D are less suitable:

B (Systems Manager OpsCenter): OpsCenter is primarily designed for managing operational issues, not for storing sensitive credentials.
C (S3 Bucket): While you can encrypt files in S3, it lacks the dedicated credential management features of Secrets Manager, like automatic rotation and built-in IAM integration specific to secrets.
D (KMS Encrypted Files): Storing encrypted files on the web servers themselves increases the risk of exposure if the servers are compromised. It also makes credential rotation and centralized management more difficult.

Secrets Manager provides a robust and secure solution that addresses the specific requirements outlined in the scenario, focusing on secure storage, automatic rotation, centralized management, and fine-grained access control via IAM.

Authoritative Links:

AWS Secrets Manager: https://aws.amazon.com/secrets-manager/
IAM Best Practices: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

Question: 87 **Exam Heist**

A company hosts an application on AWS Lambda functions that are invoked by an Amazon API Gateway API. The Lambda functions save customer data to an Amazon Aurora MySQL database. Whenever the company upgrades the database, the Lambda functions fail to establish database connections until the upgrade is complete. The result is that customer data is not recorded for some of the event.
A solutions architect needs to design a solution that stores customer data that is created during database upgrades.
Which solution will meet these requirements?

A. Provision an Amazon RDS proxy to sit between the Lambda functions and the database. Configure the Lambda functions to connect to the RDS proxy.

B. Increase the run time of the Lambda functions to the maximum. Create a retry mechanism in the code that stores the customer data in the database.

C. Persist the customer data to Lambda local storage. Configure new Lambda functions to scan the local storage to save the customer data to the database.

D. Store the customer data in an Amazon Simple Queue Service (Amazon SQS) FIFO queue. Create a new Lambda function that polls the queue and stores the customer data in the database.

Answer: D

Explanation:

The correct answer is **D: Store the customer data in an Amazon Simple Queue Service (Amazon SQS) FIFO queue. Create a new Lambda function that polls the queue and stores the customer data in the database.**

Here's why:

Reliability and Durability: SQS provides a highly reliable and durable queuing mechanism. When the Aurora database is unavailable during upgrades, the Lambda functions can still successfully write the customer data to the SQS FIFO queue without data loss.

Decoupling: SQS decouples the data ingestion (Lambda functions) from the data processing/storage (new Lambda function). This decoupling ensures that the application continues to accept customer data even when the database is temporarily unavailable. This adheres to the well-architected framework principle of loosely coupling services to improve resilience.

FIFO (First-In, First-Out): The FIFO queue guarantees that the customer data is processed in the order it was received. This is important for maintaining data integrity and ensuring that events are processed in the correct sequence.

Asynchronous Processing: The new Lambda function polling the queue processes the data asynchronously, minimizing the impact on the performance of the initial Lambda functions invoked by API Gateway. This asynchronous model also provides fault tolerance. If the processing Lambda fails, the messages remain in the queue until they are successfully processed.

Scalability: SQS is a fully managed service that automatically scales to handle varying workloads, ensuring that the queue can accommodate the volume of customer data generated during database upgrades.

Let's examine why the other options are less suitable:

A: Amazon RDS Proxy: While RDS Proxy can help with connection management and reduce database load, it doesn't inherently store data when the database is completely unavailable during upgrades. The proxy is only as effective as the underlying database's availability.

B: Increased Lambda Runtime and Retry Mechanism: Increasing Lambda runtime and adding retries can help with transient errors, but it won't solve the problem of prolonged database unavailability during upgrades. Also, Lambda functions are designed to be short-lived; increasing the runtime significantly can lead to performance and cost inefficiencies.

C: Lambda Local Storage: Lambda local storage is ephemeral and limited in size. It's not designed for durable storage of data during prolonged outages. The data stored on the Lambda's local storage is lost when the Lambda execution environment is terminated. This is not a reliable approach for handling database upgrades.

Authoritative Links:

Amazon SQS: https://aws.amazon.com/sqs/
Amazon Lambda: https://aws.amazon.com/lambda/
Well-Architected Framework: https://wa.aws.amazon.com/wellarchitected/2020-07-02T19-33-28/index.en.html

Question: 88 **Exam Heist**

A survey company has gathered data for several years from areas in the United States. The company hosts the data in an Amazon S3 bucket that is 3 TB in size and growing. The company has started to share the data with a European marketing firm that has S3 buckets. The company wants to ensure that its data transfer costs remain as low as possible.
Which solution will meet these requirements?

A. Configure the Requester Pays feature on the company's S3 bucket.

B. Configure S3 Cross-Region Replication from the company's S3 bucket to one of the marketing firm's S3 buckets.

C. Configure cross-account access for the marketing firm so that the marketing firm has access to the company's S3 bucket.

D. Configure the company's S3 bucket to use S3 Intelligent-Tiering. Sync the S3 bucket to one of the marketing firm's S3 buckets.

Answer: A

Explanation:

The correct answer is A: Configure the Requester Pays feature on the company's S3 bucket.

Here's why: The primary goal is to minimize the survey company's data transfer costs when sharing data with the European marketing firm.

Requester Pays: When the company enables Requester Pays on its S3 bucket, the marketing firm (the requester) is responsible for all data transfer and request costs associated with accessing the data in the bucket. This directly shifts the cost burden from the survey company to the marketing firm, thus minimizing the survey company's costs.

Why other options are less optimal:

B. S3 Cross-Region Replication: This replicates the entire 3 TB dataset to the marketing firm's bucket. The survey company would incur the costs of replication (storage, data transfer out of the source region, and requests). The replication is a continuous process, resulting in on-going costs. This does not minimize the survey company's transfer costs.
C. Cross-Account Access: While this allows the marketing firm to access the data, the survey company would still bear the data transfer costs when the marketing firm downloads or processes the data. Granting cross-account access only provides permission; it doesn't change who pays for data transfer.
D. S3 Intelligent-Tiering with Sync: Intelligent-Tiering optimizes storage costs based on access patterns within the survey company's S3 bucket. It does not affect the data transfer costs when the marketing firm accesses the data from their location after the sync. Also, syncing the bucket would involve the survey company paying for the initial data transfer and incurring ongoing synchronization costs.

In summary, Requester Pays is the only option that directly and completely shifts the responsibility for data transfer costs to the marketing firm, fulfilling the requirement of minimizing the survey company's costs.

Relevant Link:

Using Requester Pays buckets - Amazon Simple Storage Service

Question: 89 **Exam Heist**

A company uses Amazon S3 to store its confidential audit documents. The S3 bucket uses bucket policies to restrict access to audit team IAM user credentials according to the principle of least privilege. Company managers are worried about accidental deletion of documents in the S3 bucket and want a more secure solution.
What should a solutions architect do to secure the audit documents?

A. Enable the versioning and MFA Delete features on the S3 bucket.

B. Enable multi-factor authentication (MFA) on the IAM user credentials for each audit team IAM user account.

C. Add an S3 Lifecycle policy to the audit team's IAM user accounts to deny the s3:DeleteObject action during audit dates.

D. Use AWS Key Management Service (AWS KMS) to encrypt the S3 bucket and restrict audit team IAM user accounts from accessing the KMS key.

Answer: A

Explanation:

The best solution to prevent accidental deletion of confidential audit documents in an S3 bucket while maintaining the principle of least privilege and addressing management's concerns is to enable versioning and MFA Delete on the S3 bucket.

Versioning, when enabled on an S3 bucket, preserves every version of an object. This means that if a file is accidentally deleted or overwritten, the previous versions are still available for recovery. This mitigates data loss from accidental deletion. [https://docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html]

MFA Delete adds an extra layer of security. When enabled, deleting a version of an object or permanently deleting the entire bucket requires multi-factor authentication. This prevents unauthorized or accidental deletion, even if an IAM user's credentials are compromised. This makes accidental deletion significantly more difficult. [https://docs.aws.amazon.com/AmazonS3/latest/userguide/VersioningwithMFA.html]

Option B, enabling MFA on IAM user credentials, increases security but doesn't prevent accidental deletion within the IAM user's granted permissions. A user with s3:DeleteObject permission could still accidentally delete files.

Option C, adding an S3 Lifecycle policy to deny deletion, is overly complex and impractical. Lifecycle policies are designed for automating transitions or deletions based on object age, not temporary restriction of deletion actions based on audit dates. Furthermore, this adds complexity to IAM user management.

Option D, using KMS encryption and restricting access to the key, addresses encryption and access control but doesn't directly prevent accidental deletion if a user with delete permissions has access to the KMS key. While encryption is essential, it doesn't solve the specific problem of accidental deletion. Encryption secures the data but doesn't prevent authorized users from deleting it.

Therefore, enabling both versioning and MFA Delete provides a comprehensive solution for preventing accidental deletion while minimizing administrative overhead and adhering to security best practices.

Question: 90 **Exam Heist**

A company is using a SQL database to store movie data that is publicly accessible. The database runs on an Amazon RDS Single-AZ DB instance. A script runs queries at random intervals each day to record the number of new movies that have been added to the database. The script must report a final total during business hours. The company's development team notices that the database performance is inadequate for development tasks when the script is running. A solutions architect must recommend a solution to resolve this issue.
Which solution will meet this requirement with the LEAST operational overhead?

A. Modify the DB instance to be a Multi-AZ deployment.
B. Create a read replica of the database. Configure the script to query only the read replica.
C. Instruct the development team to manually export the entries in the database at the end of each day.
D. Use Amazon ElastiCache to cache the common queries that the script runs against the database.

Answer: B

Explanation:

The best solution is to create a read replica of the database and configure the script to query only the read replica. This is because read replicas are designed specifically to offload read traffic from the primary database instance. By directing the script's queries to the read replica, the load on the primary RDS instance is significantly reduced, resolving the performance issues experienced by the development team during script execution.

Option A, modifying the DB instance to be a Multi-AZ deployment, improves availability and durability but doesn't directly address the performance issues caused by read queries. A Multi-AZ deployment is primarily for failover purposes in case of an infrastructure failure.

Option C, manually exporting the entries daily, is not a scalable or automated solution. It involves significant manual effort and introduces potential for human error. It also doesn't provide real-time or near-real-time data for the script.

Option D, using Amazon ElastiCache, can improve query performance for frequently accessed data, but it requires modifying the application to utilize the cache. Caching the common queries might alleviate some pressure, but doesn't solve the underlying issue of the script's queries impacting the primary database's performance for all development tasks. It also adds complexity to the architecture without fully addressing the problem. Moreover, determining which queries are "common" and configuring the cache appropriately introduces additional overhead. The script's queries run at random intervals and involve finding new movies, so it's unlikely that caching will be as effective as offloading those queries entirely to a read replica.

Read replicas offer a straightforward and efficient way to isolate read traffic without requiring significant application changes or manual intervention. This approach provides the least operational overhead while effectively addressing the performance bottleneck. This solution specifically addresses the problem by offloading the read queries from the primary instance, thus it directly improves the performance for other workloads, like development.

Relevant Documentation:

Amazon RDS Read Replicas:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.ReadReplicas.html
Amazon RDS Multi-AZ Deployments:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html

Question: 91 **Exam Heist**

A company has applications that run on Amazon EC2 instances in a VPC. One of the applications needs to call the Amazon S3 API to store and read objects. According to the company's security regulations, no traffic from the applications is allowed to travel across the internet.
Which solution will meet these requirements?

A. Configure an S3 gateway endpoint.
B. Create an S3 bucket in a private subnet.
C. Create an S3 bucket in the same AWS Region as the EC2 instances.
D. Configure a NAT gateway in the same subnet as the EC2 instances.

Answer: A

Explanation:

The correct answer is A, configuring an S3 gateway endpoint. Here's why:

The requirement is to allow EC2 instances to access S3 without traversing the internet, adhering to strict security regulations. An S3 gateway endpoint provides private connectivity between your VPC and S3, avoiding the public internet. It essentially creates a route within the AWS network that directly connects your VPC to S3. No internet gateway or NAT gateway is involved.

Option B, creating an S3 bucket in a private subnet, is incorrect. S3 buckets exist globally within a region, not within VPC subnets. The location of the bucket doesn't affect how EC2 instances access it. The access method is the key.

Option C, creating an S3 bucket in the same region, while a best practice for performance and cost, doesn't address the core security requirement of avoiding internet traffic. EC2 instances would still need a way to reach S3, and without an endpoint, they would default to using public endpoints.

Option D, configuring a NAT gateway, is also incorrect. A NAT gateway allows instances in a private subnet to initiate outbound internet traffic, but it doesn't provide a private connection to S3. It would enable internet access, which is against the requirements. The traffic would still traverse the public internet.

S3 gateway endpoints are specifically designed for securely connecting to S3 from within a VPC without using public IPs. They enhance security by preventing data from leaving the AWS network and are more cost-effective than routing traffic through a NAT gateway for S3 access. Gateway endpoints support a route table entry and allow you to specify S3 buckets to allow access to, providing granular control.

For further reading, refer to the AWS documentation on VPC endpoints for S3:
https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-s3.html

Question: 92 **Exam Heist**

A company is storing sensitive user information in an Amazon S3 bucket. The company wants to provide secure access to this bucket from the application tier running on Amazon EC2 instances inside a VPC.
Which combination of steps should a solutions architect take to accomplish this? (Choose two.)

A. Configure a VPC gateway endpoint for Amazon S3 within the VPC.
B. Create a bucket policy to make the objects in the S3 bucket public.
C. Create a bucket policy that limits access to only the application tier running in the VPC.
D. Create an IAM user with an S3 access policy and copy the IAM credentials to the EC2 instance.
E. Create a NAT instance and have the EC2 instances use the NAT instance to access the S3 bucket.

Answer: AC

Explanation:

The correct answer is AC. Here's why:

A. Configure a VPC gateway endpoint for Amazon S3 within the VPC:

A VPC gateway endpoint for S3 is a crucial element for secure access. It enables EC2 instances within a VPC to access S3 without traversing the internet. Traffic between the EC2 instances and S3 remains within the AWS network, enhancing security and reducing latency. It also eliminates the need for internet gateways, NAT gateways/instances, or public IPs on the EC2 instances. This aligns with best practices for securing VPC resources. [https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-s3.html]

C. Create a bucket policy that limits access to only the application tier running in the VPC:

A bucket policy is essential to restrict access to the S3 bucket. By creating a policy that allows access only from the specified VPC or a specific range of private IP addresses associated with the EC2 instances in the application tier, you ensure that only authorized resources can access the sensitive data. This principle of least privilege is a cornerstone of security. This prevents other unauthorized AWS resources or external entities from accessing the S3 bucket. You can even further restrict access by referencing specific IAM roles associated with the EC2 instances. [https://docs.aws.amazon.com/AmazonS3/latest/userguide/example-bucket-policies.html]

Why the other options are incorrect:

B. Create a bucket policy to make the objects in the S3 bucket public: This is a major security vulnerability

and would expose the sensitive user information to anyone on the internet. It's directly contradictory to the requirement of secure access.

D. Create an IAM user with an S3 access policy and copy the IAM credentials to the EC2 instance: While IAM roles are necessary for EC2 instances to assume permissions, copying IAM credentials directly onto EC2 instances is a bad practice. It presents a security risk; if the instance is compromised, the credentials are also compromised. It is better to utilize IAM roles. Furthermore, using VPC endpoints avoids the need for this and allows resource-based access.

E. Create a NAT instance and have the EC2 instances use the NAT instance to access the S3 bucket: Using a NAT instance would allow access to S3 via the internet. While the NAT instance would need to have the right S3 permissions via an attached role or a role that the NAT instance user assumed, this approach isn't the most secure nor the most efficient. VPC endpoints provide a more secure and direct connection to S3. You also incur charges for NAT gateway data processing, which you avoid using a gateway endpoint.

Question: 93 **Exam Heist**

A company runs an on-premises application that is powered by a MySQL database. The company is migrating the application to AWS to increase the application's elasticity and availability.
The current architecture shows heavy read activity on the database during times of normal operation. Every 4 hours, the company's development team pulls a full export of the production database to populate a database in the staging environment. During this period, users experience unacceptable application latency. The development team is unable to use the staging environment until the procedure completes.
A solutions architect must recommend replacement architecture that alleviates the application latency issue. The replacement architecture also must give the development team the ability to continue using the staging environment without delay.
Which solution meets these requirements?

A. Use Amazon Aurora MySQL with Multi-AZ Aurora Replicas for production. Populate the staging database by implementing a backup and restore process that uses the mysqldump utility.

B. Use Amazon Aurora MySQL with Multi-AZ Aurora Replicas for production. Use database cloning to create the staging database on-demand.

C. Use Amazon RDS for MySQL with a Multi-AZ deployment and read replicas for production. Use the standby instance for the staging database.

D. Use Amazon RDS for MySQL with a Multi-AZ deployment and read replicas for production. Populate the staging database by implementing a backup and restore process that uses the mysqldump utility.

Answer: B

Explanation:

The correct solution is B: Use Amazon Aurora MySQL with Multi-AZ Aurora Replicas for production and use database cloning to create the staging database on-demand. Here's why:

Aurora MySQL Multi-AZ with Replicas: Aurora provides higher availability and performance compared to standard RDS MySQL due to its architecture and storage engine. Multi-AZ ensures automatic failover in case of primary instance failure, minimizing downtime. Aurora Replicas offload read traffic from the primary instance, improving application performance during normal operation.

https://aws.amazon.com/rds/aurora/
Database Cloning: Aurora offers a fast and efficient cloning feature. Cloning creates a point-in-time, writeable copy of the database volume without incurring significant downtime or impacting production performance. This addresses the development team's need for a readily available staging environment without the 4-hour latency issue caused by a full database export. Cloning is near-instantaneous, so staging database creation would not delay the development team.

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Aurora.Managing.Clone.html

Let's analyze why the other options are less suitable:

A. Using mysqldump: While mysqldump creates backups, restoring the staging database from a mysqldump backup still takes a significant amount of time (similar to the current 4-hour export) and causes application latency, defeating the purpose.

C. RDS MySQL with Multi-AZ and Read Replicas: While RDS MySQL Multi-AZ improves availability and read replicas offload read traffic, using the standby instance for the staging database is not recommended. The standby instance is critical for failover and should not be directly accessed for other purposes. Furthermore, this option doesn't address the need for a quick and efficient staging database creation method, as it implies using the already existing standby.

D. RDS MySQL with Multi-AZ and Read Replicas, plus mysqldump: This option suffers from the same issue as option A - mysqldump takes too long, causing latency issues and delaying the development team.

Question: 94 **Exam Heist**

A company is designing an application where users upload small files into Amazon S3. After a user uploads a file, the file requires one-time simple processing to transform the data and save the data in JSON format for later analysis.
Each file must be processed as quickly as possible after it is uploaded. Demand will vary. On some days, users will upload a high number of files. On other days, users will upload a few files or no files.
Which solution meets these requirements with the LEAST operational overhead?

A. Configure Amazon EMR to read text files from Amazon S3. Run processing scripts to transform the data. Store the resulting JSON file in an Amazon Aurora DB cluster.

B. Configure Amazon S3 to send an event notification to an Amazon Simple Queue Service (Amazon SQS) queue. Use Amazon EC2 instances to read from the queue and process the data. Store the resulting JSON file in Amazon DynamoDB.

C. Configure Amazon S3 to send an event notification to an Amazon Simple Queue Service (Amazon SQS) queue. Use an AWS Lambda function to read from the queue and process the data. Store the resulting JSON file in Amazon DynamoDB.

D. Configure Amazon EventBridge (Amazon CloudWatch Events) to send an event to Amazon Kinesis Data Streams when a new file is uploaded. Use an AWS Lambda function to consume the event from the stream and process the data. Store the resulting JSON file in an Amazon Aurora DB cluster.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best solution, along with supporting concepts and links:

Option C is the most efficient and scalable solution with the least operational overhead because it leverages serverless technologies for event-driven processing. When a file is uploaded to S3, an event notification is triggered and placed into an SQS queue. SQS acts as a buffer, decoupling the S3 upload process from the processing component and handling varying demand. The Lambda function is triggered by messages in the SQS queue. Lambda's serverless nature means it automatically scales based on the number of messages in the queue, handling both high and low traffic volumes efficiently without requiring any manual management of servers. DynamoDB is a NoSQL database perfectly suited for storing JSON data and scaling rapidly based on read and write activity. This solution minimizes operational overhead because there's no need to manage EC2 instances, EMR clusters, or Kinesis streams.

Option A is less ideal because Amazon EMR is designed for large-scale data processing and analytics, which is overkill for simple file transformations. Managing an EMR cluster also introduces significant operational

overhead. Storing the JSON in Aurora might be suitable, but DynamoDB is a better fit for this use case due to its flexible schema.

Option B involves using EC2 instances to poll the SQS queue and process the data. While it works, it adds operational overhead because you're responsible for managing the EC2 instances, scaling them up or down, and ensuring high availability. This is much less efficient and cost-effective compared to Lambda.

Option D using EventBridge and Kinesis Data Streams adds unnecessary complexity and overhead. Kinesis Data Streams is designed for high-throughput, real-time data streaming, which isn't needed for one-time file processing. EventBridge is also not needed as S3 can directly publish to SQS. Using Lambda for the processing is good, but Aurora might not be the optimal choice for JSON storage compared to DynamoDB.

In summary, Option C's event-driven architecture with S3, SQS, Lambda, and DynamoDB provides the best balance of scalability, efficiency, and minimal operational overhead for this particular use case.

Relevant links:

Amazon S3 Event Notifications:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventNotifications.html
Amazon SQS: https://aws.amazon.com/sqs/
AWS Lambda: https://aws.amazon.com/lambda/
Amazon DynamoDB: https://aws.amazon.com/dynamodb/

Question: 95 **Exam Heist**

An application allows users at a company's headquarters to access product data. The product data is stored in an Amazon RDS MySQL DB instance. The operations team has isolated an application performance slowdown and wants to separate read traffic from write traffic. A solutions architect needs to optimize the application's performance quickly.
What should the solutions architect recommend?

A. Change the existing database to a Multi-AZ deployment. Serve the read requests from the primary Availability Zone.

B. Change the existing database to a Multi-AZ deployment. Serve the read requests from the secondary Availability Zone.

C. Create read replicas for the database. Configure the read replicas with half of the compute and storage resources as the source database.

D. Create read replicas for the database. Configure the read replicas with the same compute and storage resources as the source database.

Answer: D

Explanation:

The best solution for separating read and write traffic in this scenario is to create read replicas.

Option D is correct because read replicas are specifically designed to handle read-heavy workloads, offloading the primary database instance. This helps improve the application's performance by reducing the load on the primary RDS instance, allowing it to focus on write operations. Configuring the read replicas with the same compute and storage resources ensures they can handle the read workload efficiently without becoming a bottleneck. If the read replicas have insufficient resources, they may not be able to handle the read load adequately and could become overloaded, negating the benefits of separating read and write traffic.

Option A and B are incorrect because while Multi-AZ deployments provide high availability and failover capabilities, they don't inherently separate read and write traffic. In a Multi-AZ setup, the secondary instance

is a standby and doesn't serve read requests unless a failover occurs. Multi-AZ is primarily for disaster recovery, not performance optimization for read-heavy workloads.

Option C is incorrect because configuring the read replicas with half the compute and storage of the source database may lead to performance issues, especially if the read workload is substantial. Under-provisioning the read replicas can lead to latency and bottlenecks, defeating the purpose of separating read traffic. Read replicas should ideally have similar resources to handle the expected read load effectively.

Therefore, creating read replicas with the same compute and storage resources as the source database is the most effective way to optimize the application's performance by separating read and write traffic.

Further Research:

Amazon RDS Read Replicas:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/AuroraMySQL.Replication.ReadReplicas.html
Amazon RDS Multi-AZ Deployments:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html

Question: 96 **Exam Heist**

An Amazon EC2 administrator created the following policy associated with an IAM group containing several users:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "ec2:TerminateInstances",
            "Resource": "*",
            "Condition": {
                "IpAddress": {
                    "aws:SourceIp": "10.100.100.0/24"
                }
            }
        },
        {
            "Effect": "Deny",
            "Action": "ec2:*",
            "Resource": "*",
            "Condition": {
                "StringNotEquals": {
                    "ec2:Region": "us-east-1"
                }
            }
        }
    ]
}
```

What is the effect of this policy?

A.Users can terminate an EC2 instance in any AWS Region except us-east-1.

B.Users can terminate an EC2 instance with the IP address 10.100.100.1 in the us-east-1 Region.

C.Users can terminate an EC2 instance in the us-east-1 Region when the user's source IP is 10.100.100.254.

D.Users cannot terminate an EC2 instance in the us-east-1 Region when the user's source IP is 10.100.100.254.

Answer: C

Explanation:

C is correct.0.0/24 , the following five IP addresses are reserved:0.0: Network address.0.1: Reserved by AWS for the VPC router.0.2: Reserved by AWS. The IP address of the DNS server is the base of the VPC network range plus two. ...0.3: Reserved by AWS for future use.0.255: Network broadcast address.

Question: 97 **Exam Heist**

A company has a large Microsoft SharePoint deployment running on-premises that requires Microsoft Windows shared file storage. The company wants to migrate this workload to the AWS Cloud and is considering various storage options. The storage solution must be highly available and integrated with Active Directory for access control.
Which solution will satisfy these requirements?

A. Configure Amazon EFS storage and set the Active Directory domain for authentication.

B. Create an SMB file share on an AWS Storage Gateway file gateway in two Availability Zones.

C. Create an Amazon S3 bucket and configure Microsoft Windows Server to mount it as a volume.

D. Create an Amazon FSx for Windows File Server file system on AWS and set the Active Directory domain for authentication.

Answer: D

Explanation:

The best solution is to use Amazon FSx for Windows File Server, due to its features specifically designed to address the described needs.

Option D is correct because Amazon FSx for Windows File Server provides fully managed native Microsoft Windows file servers backed by SSD storage. It is designed to integrate seamlessly with Active Directory for authentication and authorization, fulfilling the requirement for access control. Its support for the SMB protocol allows it to be accessed as a standard Windows file share. FSx for Windows File Server can also be deployed in a Multi-AZ configuration, offering high availability.

Option A is incorrect because Amazon EFS (Elastic File System) is primarily designed for Linux workloads and while it can integrate with Active Directory via IAM and NFS, it is not a native Windows file server and lacks direct SMB protocol support, making it a less suitable fit for a SharePoint migration requiring Windows shared file storage.

Option B is incorrect because while AWS Storage Gateway's file gateway can present SMB file shares and integrate with Active Directory, it's a hybrid solution that caches data locally. It uses S3 as the primary storage location, which is not ideal for low-latency, high-performance SharePoint workloads. Placing a file gateway in two Availability Zones does not natively provide high availability for the SMB share itself without additional configuration and complexity, adding operational overhead.

Option C is incorrect because while Amazon S3 can store files, it is object storage and not a file system suitable for directly mounting as a Windows volume in the way required by SharePoint. While there are tools to achieve this, it isn't a direct solution and would be more complex and prone to performance issues than using a native Windows file server solution. The required integration and latency requirements would not be met easily using S3 for the application in question.

In summary, Amazon FSx for Windows File Server is the optimal choice due to its native support for the SMB

protocol, Active Directory integration, high availability through Multi-AZ deployment, and its ability to function as a fully managed Windows file server, seamlessly integrating with SharePoint.

Supporting Links:

Amazon FSx for Windows File Server: https://aws.amazon.com/fsx/windows/
AWS Storage Gateway: https://aws.amazon.com/storagegateway/
Amazon EFS: https://aws.amazon.com/efs/

Question: 98 **Exam Heist**

An image-processing company has a web application that users use to upload images. The application uploads the images into an Amazon S3 bucket. The company has set up S3 event notifications to publish the object creation events to an Amazon Simple Queue Service (Amazon SQS) standard queue. The SQS queue serves as the event source for an AWS Lambda function that processes the images and sends the results to users through email. Users report that they are receiving multiple email messages for every uploaded image. A solutions architect determines that SQS messages are invoking the Lambda function more than once, resulting in multiple email messages.
What should the solutions architect do to resolve this issue with the LEAST operational overhead?

A. Set up long polling in the SQS queue by increasing the ReceiveMessage wait time to 30 seconds.

B. Change the SQS standard queue to an SQS FIFO queue. Use the message deduplication ID to discard duplicate messages.

C. Increase the visibility timeout in the SQS queue to a value that is greater than the total of the function timeout and the batch window timeout.

D. Modify the Lambda function to delete each message from the SQS queue immediately after the message is read before processing.

Answer: C

Explanation:

The issue is duplicate processing of SQS messages by the Lambda function. This indicates the Lambda function isn't completing and acknowledging receipt of the message within the SQS visibility timeout, leading SQS to re-queue the message for another attempt.

Option A, setting up long polling, helps with efficient message retrieval but does not prevent duplicate processing if the Lambda function fails or takes too long to process a message within the visibility timeout. Long polling primarily reduces the cost associated with empty responses from SQS.

Option B, switching to an SQS FIFO queue with message deduplication, would prevent duplicates. However, it introduces significant operational overhead. It requires code changes, potentially impacting the existing application's architecture and message ordering assumptions. FIFO queues also have lower throughput limits than standard queues, potentially affecting the application's performance. It's also unnecessary because the processing order doesn't appear to be a critical requirement based on the problem description.

Option C, increasing the visibility timeout, directly addresses the root cause. The visibility timeout is the duration a message remains invisible to other consumers after being retrieved from the queue. By increasing the visibility timeout to be greater than the combined Lambda function timeout and batch window (the maximum time Lambda waits before processing messages in a batch), it ensures that SQS doesn't re-queue the message while the Lambda function is still processing it. This significantly reduces the chances of duplicate processing with minimal code or infrastructure changes.

Option D, modifying the Lambda function to immediately delete messages, is risky. If the Lambda function fails after deleting the message but before completing the processing, the message will be lost entirely. This guarantees data loss, which is undesirable. It also adds complexity to the Lambda function and might not

solve the underlying timeout issue.

Therefore, Option C is the best solution because it addresses the root cause with the least operational overhead, ensuring that the Lambda function has sufficient time to process the message and preventing premature re-queuing by SQS. It minimizes the changes required to the existing system and avoids the potential data loss associated with Option D.

Relevant Documentation:

SQS Visibility Timeout:
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html
AWS Lambda with SQS: https://docs.aws.amazon.com/lambda/latest/dg/services-sqs.html

Question: 99 **Exam Heist**

A company is implementing a shared storage solution for a gaming application that is hosted in an on-premises data center. The company needs the ability to use Lustre clients to access data. The solution must be fully managed.
Which solution meets these requirements?

A. Create an AWS Storage Gateway file gateway. Create a file share that uses the required client protocol. Connect the application server to the file share.

B. Create an Amazon EC2 Windows instance. Install and configure a Windows file share role on the instance. Connect the application server to the file share.

C. Create an Amazon Elastic File System (Amazon EFS) file system, and configure it to support Lustre. Attach the file system to the origin server. Connect the application server to the file system.

D. Create an Amazon FSx for Lustre file system. Attach the file system to the origin server. Connect the application server to the file system.

Answer: D

Explanation:

The question requires a fully managed shared storage solution accessible via Lustre clients for an on-premises application. Let's analyze why option D is the correct choice.

Option D, creating an Amazon FSx for Lustre file system, directly addresses the requirements. FSx for Lustre is a fully managed, high-performance file system optimized for compute-intensive workloads, including those needing the Lustre file system. Lustre clients can connect directly to the FSx for Lustre file system, fulfilling the access requirement. The fully managed nature of FSx for Lustre eliminates the need for manual setup, configuration, and maintenance, which aligns with the question's stipulation. Attaching the file system to the origin server and connecting application servers to the file system creates the shared storage environment the application needs.

Option A, using AWS Storage Gateway file gateway, is incorrect. While Storage Gateway provides file sharing capabilities, it primarily caters to connecting on-premises environments to AWS storage services like S3. It does not inherently support the Lustre file system protocol. It's more suited for file-based backups and archival, not high-performance shared storage.

Option B, setting up an Amazon EC2 Windows instance with a Windows file share, involves significant manual effort. It requires installing, configuring, and maintaining the Windows file share role, negating the "fully managed" requirement. Furthermore, Windows file shares aren't designed for the high-performance compute workloads typically associated with Lustre.

Option C, creating an Amazon Elastic File System (EFS) file system, and configuring it to support Lustre, is not

a supported configuration. Amazon EFS is a fully managed NFS file system. EFS does not natively support the Lustre protocol. Trying to "configure" it to support Lustre is not a feasible solution within the AWS ecosystem.

Therefore, Amazon FSx for Lustre is the only fully managed AWS service that natively supports the Lustre file system protocol, making it the best solution for this scenario.

Relevant links:

Amazon FSx for Lustre: https://aws.amazon.com/fsx/lustre/
AWS Storage Gateway: https://aws.amazon.com/storagegateway/
Amazon Elastic File System (EFS): https://aws.amazon.com/efs/

Question: 100 **Exam Heist**

A company's containerized application runs on an Amazon EC2 instance. The application needs to download security certificates before it can communicate with other business applications. The company wants a highly secure solution to encrypt and decrypt the certificates in near real time. The solution also needs to store data in highly available storage after the data is encrypted.
Which solution will meet these requirements with the LEAST operational overhead?

A. Create AWS Secrets Manager secrets for encrypted certificates. Manually update the certificates as needed. Control access to the data by using fine-grained IAM access.

B. Create an AWS Lambda function that uses the Python cryptography library to receive and perform encryption operations. Store the function in an Amazon S3 bucket.

C. Create an AWS Key Management Service (AWS KMS) customer managed key. Allow the EC2 role to use the KMS key for encryption operations. Store the encrypted data on Amazon S3.

D. Create an AWS Key Management Service (AWS KMS) customer managed key. Allow the EC2 role to use the KMS key for encryption operations. Store the encrypted data on Amazon Elastic Block Store (Amazon EBS) volumes.

Answer: C

Explanation:

The correct answer is C because it offers a secure, readily available, and operationally efficient solution for managing and encrypting certificates.

Here's why:

AWS KMS for Encryption: AWS Key Management Service (KMS) is designed for managing encryption keys securely. Using a KMS customer-managed key allows the company to control the key lifecycle and access permissions. This ensures that only authorized entities (the EC2 instance via its role) can use the key for encryption and decryption operations. https://aws.amazon.com/kms/
EC2 Role for Access Control: Granting the EC2 instance access to the KMS key via an IAM role is a best practice. It allows the EC2 instance to use the KMS key without embedding credentials within the instance itself.
Amazon S3 for Highly Available Storage: Amazon S3 provides highly available and durable storage. Storing the encrypted data in S3 ensures that it is protected against data loss and is accessible when needed. https://aws.amazon.com/s3/

Option A is less desirable because it involves manual updates of certificates in Secrets Manager, which introduces operational overhead and potential for human error. While Secrets Manager is good for storing secrets, KMS is better for encryption.

Option B is unnecessarily complex. Using a Lambda function with the Python cryptography library introduces additional overhead (managing the Lambda function, dependencies, execution environment) and is not the

most efficient solution for simple encryption/decryption tasks.

Option D is not as good as C because Amazon EBS volumes are primarily designed for block storage for EC2 instances. While EBS can be encrypted, it's not ideal for storing data independently that needs to be highly available. S3 provides greater availability, scalability, and durability than EBS in this scenario. Furthermore, EBS is attached to a specific availability zone, whereas S3 is region-wide.

Question: 101 **Exam Heist**

A solutions architect is designing a VPC with public and private subnets. The VPC and subnets use IPv4 CIDR blocks. There is one public subnet and one private subnet in each of three Availability Zones (AZs) for high availability. An internet gateway is used to provide internet access for the public subnets. The private subnets require access to the internet to allow Amazon EC2 instances to download software updates.
What should the solutions architect do to enable Internet access for the private subnets?

A. Create three NAT gateways, one for each public subnet in each AZ. Create a private route table for each AZ that forwards non-VPC traffic to the NAT gateway in its AZ.

B. Create three NAT instances, one for each private subnet in each AZ. Create a private route table for each AZ that forwards non-VPC traffic to the NAT instance in its AZ.

C. Create a second internet gateway on one of the private subnets. Update the route table for the private subnets that forward non-VPC traffic to the private internet gateway.

D. Create an egress-only internet gateway on one of the public subnets. Update the route table for the private subnets that forward non-VPC traffic to the egress-only Internet gateway.

Answer: A

Explanation:

The correct solution is to use NAT Gateways for internet access from private subnets. Here's why:

Option A is correct because it leverages NAT Gateways which are the AWS recommended way to provide outbound internet access to instances in private subnets while preventing inbound traffic from the internet. Deploying one NAT Gateway per Availability Zone (AZ) ensures high availability and prevents a single point of failure. Each private subnet's route table then points to the NAT Gateway within its respective AZ for all traffic destined outside the VPC (non-VPC CIDR block traffic, commonly represented as 0.0.0.0/0). This architecture allows instances in private subnets to initiate outbound connections (e.g., downloading software updates) without exposing them directly to the internet.

Option B is incorrect because NAT Instances, while a functional alternative, require more administrative overhead and manual scaling. They also represent a single point of failure within each AZ unless configured for high availability (which increases complexity). AWS recommends NAT Gateways over NAT Instances.

Option C is incorrect. You cannot associate an Internet Gateway with a private subnet. Internet Gateways are designed for public subnets to allow bidirectional communication with the internet. Connecting a private subnet directly to the internet defeats the purpose of having a private subnet and introduces significant security risks.

Option D is incorrect. Egress-only internet gateways are designed for IPv6 traffic only. The question specifically states that the VPC and subnets use IPv4 CIDR blocks, rendering egress-only internet gateways unusable in this scenario. They also only permit outbound traffic initiated from inside the VPC.

In summary, NAT Gateways provide a scalable, highly available, and managed solution for outbound internet access from private subnets in IPv4 environments, aligning with AWS best practices. The key is to create a NAT Gateway in each public subnet of each AZ, and then route all outbound (non-VPC) traffic from the corresponding private subnet to that NAT Gateway.

Relevant AWS documentation:

NAT Gateways: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html
NAT Instances: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-instances.html
Internet Gateways: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Internet_Gateway.html
Egress-Only Internet Gateways: https://docs.aws.amazon.com/vpc/latest/userguide/egress-only-internet-gateway.html

Question: 102 **Exam Heist**

A company wants to migrate an on-premises data center to AWS. The data center hosts an SFTP server that stores its data on an NFS-based file system. The server holds 200 GB of data that needs to be transferred. The server must be hosted on an Amazon EC2 instance that uses an Amazon Elastic File System (Amazon EFS) file system. Which combination of steps should a solutions architect take to automate this task? (Choose two.)

A. Launch the EC2 instance into the same Availability Zone as the EFS file system.
B. Install an AWS DataSync agent in the on-premises data center.
C. Create a secondary Amazon Elastic Block Store (Amazon EBS) volume on the EC2 instance for the data.
D. Manually use an operating system copy command to push the data to the EC2 instance.
E. Use AWS DataSync to create a suitable location configuration for the on-premises SFTP server.

Answer: BE

Explanation:

The correct answer is BE because it outlines the most efficient and secure way to migrate the data from the on-premises SFTP server to the Amazon EFS file system mounted on an EC2 instance.

Justification:

B. Install an AWS DataSync agent in the on-premises data center: AWS DataSync is specifically designed for automated and accelerated data transfer between on-premises storage and AWS storage services. Installing a DataSync agent on-premises is crucial for accessing and moving data from the SFTP server. DataSync optimizes the transfer process with features like incremental transfers, encryption, and data verification. The agent acts as a bridge, allowing DataSync to securely access the on-premises NFS-based file system. More information can be found on the AWS DataSync documentation: https://aws.amazon.com/datasync/

E. Use AWS DataSync to create a suitable location configuration for the on-premises SFTP server: After installing the agent, the next step is to configure DataSync with the correct source (the on-premises SFTP server) and destination (the Amazon EFS file system). This involves creating a location for the SFTP server, providing the necessary credentials for DataSync to access it. This configuration also includes specifying the EFS file system as the destination location within AWS. DataSync can then be configured to schedule or initiate the data transfer.

Why other options are incorrect:

A. Launch the EC2 instance into the same Availability Zone as the EFS file system: While launching the EC2 instance in the same Availability Zone as the EFS file system is important for performance once the migration is complete, it doesn't directly contribute to the automation of the data migration process. It's a prerequisite for EFS usage but not the solution for the problem.

C. Create a secondary Amazon Elastic Block Store (Amazon EBS) volume on the EC2 instance for the data: Creating an EBS volume is unnecessary and inefficient for the purpose outlined. The goal is to store the data on EFS, not EBS. Using EBS as an intermediary step introduces complexity and cost without providing any

added benefit.

D. Manually use an operating system copy command to push the data to the EC2 instance: Manually copying data is time-consuming, error-prone, and doesn't leverage the automation capabilities of AWS services. It is not a suitable solution for migrating 200 GB of data and does not scale effectively. It also doesn't provide the data verification or incremental transfer capabilities of DataSync.

Question: 103 **Exam Heist**

A company has an AWS Glue extract, transform, and load (ETL) job that runs every day at the same time. The job processes XML data that is in an Amazon S3 bucket. New data is added to the S3 bucket every day. A solutions architect notices that AWS Glue is processing all the data during each run.
What should the solutions architect do to prevent AWS Glue from reprocessing old data?

A. Edit the job to use job bookmarks.
B. Edit the job to delete data after the data is processed.
C. Edit the job by setting the NumberOfWorkers field to 1.
D. Use a FindMatches machine learning (ML) transform.

Answer: A

Explanation:

The correct answer is A, using job bookmarks in AWS Glue. Here's why:

AWS Glue job bookmarks are specifically designed to track the state of ETL jobs, particularly for incremental data processing. They allow Glue to remember which data has already been processed, enabling it to only process new data during subsequent runs. Without job bookmarks, Glue will scan the entire input dataset each time the job runs, leading to reprocessing of old data and increased costs.

Job bookmarks work by storing information about the last processed files or records, such as the last modified timestamp or a unique identifier. When the job runs again, it uses this information to determine the starting point for data processing. This approach significantly reduces processing time and resource consumption, as it avoids redundant operations on data that has already been transformed and loaded.

Option B, deleting data after processing, is a destructive approach and can lead to data loss if there are any issues during the ETL process. It also eliminates the possibility of reprocessing data for other purposes or correcting errors. Option C, setting NumberOfWorkers to 1, simply reduces the parallelism of the job, slowing down the overall processing time but not preventing reprocessing of old data. Option D, using FindMatches, is an ML transform for deduplication and record linkage and is not relevant to preventing the reprocessing of old data during ETL runs.

Job bookmarks are the best solution because they maintain data integrity, enable incremental processing, and reduce costs by avoiding redundant operations, all while being specifically designed for this use case within the AWS Glue ecosystem. They are a standard best practice for efficient ETL workflows with continuously updated data sources.

For more information, refer to the official AWS documentation:

Using Job Bookmarks to Track Processed Data: https://docs.aws.amazon.com/glue/latest/dg/monitor-continuations.html

Question: 104 **Exam Heist**

A solutions architect must design a highly available infrastructure for a website. The website is powered by Windows web servers that run on Amazon EC2 instances. The solutions architect must implement a solution that can mitigate a large-scale DDoS attack that originates from thousands of IP addresses. Downtime is not acceptable for the website.
Which actions should the solutions architect take to protect the website from such an attack? (Choose two.)

A. Use AWS Shield Advanced to stop the DDoS attack.

B. Configure Amazon GuardDuty to automatically block the attackers.

C. Configure the website to use Amazon CloudFront for both static and dynamic content.

D. Use an AWS Lambda function to automatically add attacker IP addresses to VPC network ACLs.

E. Use EC2 Spot Instances in an Auto Scaling group with a target tracking scaling policy that is set to 80% CPU utilization.

Answer: AC

Explanation:

Here's a breakdown of why options A and C are the correct choices and why the others aren't, focusing on DDoS mitigation and high availability in the context of Windows-based web servers on EC2:

A. Use AWS Shield Advanced to stop the DDoS attack: AWS Shield Advanced provides enhanced DDoS protection beyond the standard protection included with all AWS customers. It offers 24/7 access to the AWS DDoS Response Team (DRT) and advanced detection and mitigation techniques. This is crucial for handling large-scale DDoS attacks originating from thousands of IP addresses. Shield Advanced is specifically designed to protect against sophisticated attacks targeting web applications running on AWS resources. https://aws.amazon.com/shield/

C. Configure the website to use Amazon CloudFront for both static and dynamic content: CloudFront, a CDN (Content Delivery Network), distributes your website content across a globally distributed network of edge locations. This helps to absorb a DDoS attack by serving content from multiple locations, thereby reducing the load on the origin servers (EC2 instances). It also caches static content, further reducing the load on the origin. Configuring it for both static and dynamic content allows CloudFront to protect the origin even if attackers are requesting dynamic pages. CloudFront also integrates with AWS Shield for more robust protection. https://aws.amazon.com/cloudfront/

Let's examine why the other options are not the best choices:

B. Configure Amazon GuardDuty to automatically block the attackers: GuardDuty is a threat detection service that monitors your AWS environment for malicious activity. While GuardDuty can detect potential DDoS activity, it doesn't automatically block attackers. It primarily focuses on identifying threats and generating security findings; it isn't a primary DDoS mitigation tool. The response to GuardDuty findings requires additional configuration (such as custom scripts/automation to block IPs).

D. Use an AWS Lambda function to automatically add attacker IP addresses to VPC network ACLs: While technically possible, this approach is not scalable or effective against large-scale DDoS attacks. Network ACLs have limitations on the number of rules they can hold, and managing a list of thousands of attacker IPs would be cumbersome and potentially impact performance. Additionally, Lambda invocation per IP discovered may add considerable cost and latency.

E. Use EC2 Spot Instances in an Auto Scaling group with a target tracking scaling policy that is set to 80% CPU utilization: Spot Instances and Auto Scaling help with cost optimization and scaling based on resource utilization. While useful for general scaling, they don't directly address DDoS mitigation. Scaling up in response to a DDoS attack might help keep the service online for legitimate users for some time, but it won't stop the attack itself and could be quickly overwhelmed in a large-scale attack. Moreover, Spot Instances can be terminated, potentially exacerbating the issue during an attack.

Question: 105 **Exam Heist**

A company is preparing to deploy a new serverless workload. A solutions architect must use the principle of least privilege to configure permissions that will be used to run an AWS Lambda function. An Amazon EventBridge (Amazon CloudWatch Events) rule will invoke the function.
Which solution meets these requirements?

A. Add an execution role to the function with lambda:InvokeFunction as the action and * as the principal.

B. Add an execution role to the function with lambda:InvokeFunction as the action and Service: lambda.amazonaws.com as the principal.

C. Add a resource-based policy to the function with lambda:* as the action and Service: events.amazonaws.com as the principal.

D. Add a resource-based policy to the function with lambda:InvokeFunction as the action and Service: events.amazonaws.com as the principal.

Answer: D

Explanation:

The correct answer is **D. Add a resource-based policy to the function with lambda:InvokeFunction as the action and Service: events.amazonaws.com as the principal.**

Here's a detailed justification:

The question emphasizes the principle of least privilege, which dictates granting only the necessary permissions. An EventBridge rule needs permission to invoke the Lambda function. Therefore, the policy should only grant the lambda:InvokeFunction action. Options A and B incorrectly suggest using an execution role. Execution roles are for the Lambda function to access other AWS services, not for services to invoke the Lambda function.

The necessary permissions for EventBridge to trigger a Lambda function are defined in a resource-based policy attached to the Lambda function itself. This is different from an IAM role that's assumed by the Lambda function. Resource-based policies grant permissions to principals (in this case, EventBridge) to perform actions on the resource (the Lambda function). Option C is too broad as it uses lambda:*, which grants all Lambda permissions, violating the principle of least privilege.

Option D specifies lambda:InvokeFunction, which is the exact permission EventBridge needs. The Service: events.amazonaws.com principal specifies that only the EventBridge service is authorized to invoke the function. This adheres to the principle of least privilege. Specifying the service principal is important to limit access to invocations and prevent unauthorized actors from triggering the Lambda function. This is the most secure and precise approach.

Here are some authoritative links for further research:

AWS Lambda Permissions: https://docs.aws.amazon.com/lambda/latest/dg/security-iam.html
Using Resource-Based Policies for Lambda: https://docs.aws.amazon.com/lambda/latest/dg/access-control-resource-based.html
EventBridge Permissions: https://docs.aws.amazon.com/eventbridge/latest/userguide/auth-and-access-control-eventbridge.html

Question: 106 **Exam Heist**

A company is preparing to store confidential data in Amazon S3. For compliance reasons, the data must be encrypted at rest. Encryption key usage must be logged for auditing purposes. Keys must be rotated every year.

Which solution meets these requirements and is the MOST operationally efficient?

A. Server-side encryption with customer-provided keys (SSE-C)
B. Server-side encryption with Amazon S3 managed keys (SSE-S3)
C. Server-side encryption with AWS KMS keys (SSE-KMS) with manual rotation
D. Server-side encryption with AWS KMS keys (SSE-KMS) with automatic rotation

Answer: D

Explanation:

Here's a detailed justification for why option D is the most operationally efficient solution for storing confidential data in Amazon S3 with encryption, key usage logging, and annual key rotation:

The requirements are data encryption at rest, logged key usage for auditing, and annual key rotation. All options except SSE-S3 provide encryption. However, SSE-S3 (option B) doesn't offer detailed logging of key usage for auditing. SSE-C (option A) involves managing and providing the encryption keys yourself, which significantly increases operational overhead and responsibility for security. This option would also not offer CloudTrail logging of the key usage.

SSE-KMS using AWS Key Management Service (KMS) (options C and D) meets the requirements by allowing for encrypted data storage, generating logs of key usage using AWS CloudTrail, and enabling key rotation. CloudTrail logs the API calls made to KMS, providing an auditable record of key usage.

Option D, SSE-KMS with automatic rotation, is the most operationally efficient. AWS KMS can automatically rotate the encryption keys every year. This automation reduces the operational burden of manually rotating keys (option C), minimizing the risk of human error and ensuring compliance. With automatic rotation, the key is rotated without the application needing to be re-encrypted.

Therefore, SSE-KMS with automatic key rotation (option D) provides a balance between security, compliance, and operational efficiency, making it the most suitable solution.

Further research:

AWS KMS: https://aws.amazon.com/kms/
S3 Encryption: https://docs.aws.amazon.com/AmazonS3/latest/userguide/serv-side-encryption.html
AWS CloudTrail: https://aws.amazon.com/cloudtrail/

Question: 107 **Exam Heist**

A bicycle sharing company is developing a multi-tier architecture to track the location of its bicycles during peak operating hours. The company wants to use these data points in its existing analytics platform. A solutions architect must determine the most viable multi-tier option to support this architecture. The data points must be accessible from the REST API.
Which action meets these requirements for storing and retrieving location data?

A. Use Amazon Athena with Amazon S3.
B. Use Amazon API Gateway with AWS Lambda.
C. Use Amazon QuickSight with Amazon Redshift.
D. Use Amazon API Gateway with Amazon Kinesis Data Analytics.

Answer: B

Explanation:

The correct answer is B: Use Amazon API Gateway with AWS Lambda. Here's why:

The problem requires a multi-tier architecture that provides REST API access to location data for bicycle tracking, which will be used in an existing analytics platform. API Gateway is specifically designed for creating, publishing, maintaining, monitoring, and securing REST APIs. It acts as a front door for applications to access data, logic, or functionality from backend services.

Lambda provides a serverless compute environment. It can be triggered by API Gateway requests to process incoming location data and store it in a database (not explicitly mentioned in the provided options, but implied as necessary for persistence). Lambda can also retrieve this data and return it as a response to API Gateway requests. This enables reading, writing, and processing the data needed for the analytical platform.

Option A, using Athena with S3, is suited for querying data stored in S3 using SQL. While Athena provides a cost-effective way to analyze large datasets, it doesn't directly support a REST API endpoint. It is more tailored to ad-hoc querying rather than a transactional system providing real-time data via APIs.

Option C, using QuickSight with Redshift, represents an analytics dashboarding solution. QuickSight is primarily for visualization and analysis of data, not for serving REST API requests. Redshift is a data warehouse that can store large volumes of data but requires an API or interface layer to expose that data.

Option D, API Gateway with Kinesis Data Analytics, is not suitable for the scenario described. Kinesis Data Analytics is designed for real-time data processing and transformation, not for storing and retrieving data. While Kinesis can process the location data stream, it doesn't naturally serve as a data store accessible via REST APIs. The output of Kinesis would still typically need to be persisted somewhere (like S3 or a database) for long-term storage and retrieval. Lambda is a simpler and more flexible approach for handling the API and potential data storage interaction.

In summary, the combination of API Gateway for REST API access and Lambda for data processing and storage interaction provides the most viable and flexible solution for the bicycle sharing company's requirements.

Authoritative Links:

Amazon API Gateway: https://aws.amazon.com/api-gateway/
AWS Lambda: https://aws.amazon.com/lambda/
Amazon Athena: https://aws.amazon.com/athena/
Amazon QuickSight: https://aws.amazon.com/quicksight/
Amazon Redshift: https://aws.amazon.com/redshift/
Amazon Kinesis Data Analytics: https://aws.amazon.com/kinesis/data-analytics/

Question: 108 **Exam Heist**

A company has an automobile sales website that stores its listings in a database on Amazon RDS. When an automobile is sold, the listing needs to be removed from the website and the data must be sent to multiple target systems.
Which design should a solutions architect recommend?

A. Create an AWS Lambda function triggered when the database on Amazon RDS is updated to send the information to an Amazon Simple Queue Service (Amazon SQS) queue for the targets to consume.

B. Create an AWS Lambda function triggered when the database on Amazon RDS is updated to send the information to an Amazon Simple Queue Service (Amazon SQS) FIFO queue for the targets to consume.

C. Subscribe to an RDS event notification and send an Amazon Simple Queue Service (Amazon SQS) queue fanned out to multiple Amazon Simple Notification Service (Amazon SNS) topics. Use AWS Lambda functions to update the targets.

D. Subscribe to an RDS event notification and send an Amazon Simple Notification Service (Amazon SNS) topic fanned out to multiple Amazon Simple Queue Service (Amazon SQS) queues. Use AWS Lambda functions to update the targets.

Answer: D

Explanation:

The correct answer is **D**, which suggests using RDS event notifications, an SNS topic fanned out to multiple SQS queues, and Lambda functions. Let's break down why.

RDS event notifications provide a near real-time mechanism for reacting to database changes. When an automobile is sold, the RDS database change will trigger this notification.

Amazon SNS (Simple Notification Service) acts as a pub/sub service. Using SNS, you can publish one message, and it will be distributed to multiple subscribers. In this case, the SNS topic will receive the RDS event notification.

Fan-out involves distributing a single event to multiple independent subscribers. Instead of having each target system directly subscribe to RDS event notifications, SNS allows you to "fan out" the notification to different SQS queues. This decoupling ensures that each target system can consume the message at its own pace and handle failures independently. Each SQS queue can be customized to handle the specific needs of each target system.

Amazon SQS (Simple Queue Service) provides a reliable queuing mechanism. Each target system will have its own dedicated SQS queue. The SQS queue acts as a buffer, ensuring that the target systems don't miss any updates even if they are temporarily unavailable.

AWS Lambda functions are used to consume messages from the SQS queues and update the target systems. The Lambda functions provide the necessary transformation and integration logic to interact with each target system's API or data format.

Option A is less desirable because it creates a direct dependency between the database update and a single SQS queue. It doesn't efficiently scale to multiple target systems and might lead to bottlenecks if the queue consumer is slow. Option B, while using a FIFO queue, doesn't address the need to distribute the information to multiple target systems effectively. Option C attempts to use SNS topics after the SQS queue, which is logically backward. SNS is more efficient when distributing messages to multiple queues, then having them filtered. The structure in Option C would be highly inefficient. Using SQS to fan out to SNS would make SQS a bottle neck and remove its purpose of buffering.

In summary, option D provides a scalable, loosely coupled, and reliable solution that leverages the strengths of RDS event notifications, SNS for fan-out, SQS for buffering, and Lambda for data transformation and integration.References:

AWS SNS: https://aws.amazon.com/sns/
AWS SQS: https://aws.amazon.com/sqs/
AWS Lambda: https://aws.amazon.com/lambda/
RDS Event Notifications: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_Events.html

Question: 109 **Exam Heist**

A company needs to store data in Amazon S3 and must prevent the data from being changed. The company wants new objects that are uploaded to Amazon S3 to remain unchangeable for a nonspecific amount of time until the company decides to modify the objects. Only specific users in the company's AWS account can have the ability 10 delete the objects.
What should a solutions architect do to meet these requirements?

A. Create an S3 Glacier vault. Apply a write-once, read-many (WORM) vault lock policy to the objects.

B. Create an S3 bucket with S3 Object Lock enabled. Enable versioning. Set a retention period of 100 years. Use governance mode as the S3 bucket’s default retention mode for new objects.

C. Create an S3 bucket. Use AWS CloudTrail to track any S3 API events that modify the objects. Upon notification, restore the modified objects from any backup versions that the company has.

D. Create an S3 bucket with S3 Object Lock enabled. Enable versioning. Add a legal hold to the objects. Add the s3:PutObjectLegalHold permission to the IAM policies of users who need to delete the objects.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct solution and why the other options are not suitable:

Why Option D is Correct:

Option D leverages Amazon S3 Object Lock with legal hold functionality, addressing the company's requirements directly. S3 Object Lock prevents objects from being deleted or overwritten for a specified period or indefinitely. By enabling versioning, previous versions of objects are preserved, providing an extra layer of protection. A legal hold provides the indefinite immutability needed until the company explicitly decides to modify objects. Granting the s3:PutObjectLegalHold permission to specific IAM users gives them the necessary authorization to remove the legal hold, allowing object deletion when the company decides. This controlled access ensures that only authorized personnel can make changes to the immutability status.

Why Other Options Are Incorrect:

Option A: S3 Glacier vault with WORM vault lock policy: S3 Glacier is primarily for archival storage and has retrieval times that are unsuitable for general data storage. While it offers write-once, read-many (WORM) capabilities, it does not offer a mechanism for specific users to remove this immutability when necessary without destroying the vault. This is important because the company wants to delete the objects eventually, even if that isn't for a long time.
Option B: S3 Object Lock with Retention Period (Governance Mode): While this option correctly uses S3 Object Lock, setting a fixed retention period (even 100 years) does not align with the requirement for an indefinite period of immutability until the company decides to modify the objects. Governance mode can be overridden by users with specific permissions and may not provide a strong enough guarantee of immutability for the company's compliance needs.
Option C: S3 Bucket with CloudTrail and Backups: This approach is reactive rather than preventative. It relies on detecting modifications and restoring from backups, which is inefficient, complex, and may lead to data loss in the time between the modification and the restoration. This option does not meet the primary requirement of preventing data from being changed in the first place and lacks immutability features.

In summary:

Option D is the only solution that fulfills all the requirements: preventing data from being changed indefinitely until the company decides to modify the objects, and allowing specific users to authorize the removal of immutability.

Authoritative Links:

Amazon S3 Object Lock: https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html
S3 Versioning: https://docs.aws.amazon.com/AmazonS3/latest/userguide/versioning-overview.html
IAM Permissions for S3 Object Lock: https://docs.aws.amazon.com/AmazonS3/latest/userguide/security-iam-id-based-policy-examples.html#s3-object-lock-permissions

Question: 110 **Exam Heist**

A social media company allows users to upload images to its website. The website runs on Amazon EC2 instances.

During upload requests, the website resizes the images to a standard size and stores the resized images in Amazon S3. Users are experiencing slow upload requests to the website.
The company needs to reduce coupling within the application and improve website performance. A solutions architect must design the most operationally efficient process for image uploads.
Which combination of actions should the solutions architect take to meet these requirements? (Choose two.)

A. Configure the application to upload images to S3 Glacier.

B. Configure the web server to upload the original images to Amazon S3.

C. Configure the application to upload images directly from each user's browser to Amazon S3 through the use of a presigned URL

D. Configure S3 Event Notifications to invoke an AWS Lambda function when an image is uploaded. Use the function to resize the image.

E. Create an Amazon EventBridge (Amazon CloudWatch Events) rule that invokes an AWS Lambda function on a schedule to resize uploaded images.

Answer: BD

Explanation:

The correct solution involves minimizing the load on the EC2 instances and decoupling the image resizing process from the web application.

B. Configure the web server to upload the original images to Amazon S3. This shifts the responsibility of storing the images from the EC2 instances to S3, freeing up the EC2 instances to handle web requests more efficiently. Direct uploads to S3 reduce the load on the web servers, as they no longer need to manage the storage aspect. This is a key step in reducing coupling.

D. Configure S3 Event Notifications to invoke an AWS Lambda function when an image is uploaded. Use the function to resize the image. This triggers a serverless Lambda function whenever a new image is uploaded to S3. The Lambda function then handles the image resizing asynchronously. This fully decouples the resizing process from the web application. S3 Event Notifications provide a highly scalable and reliable mechanism to trigger the Lambda function, ensuring that images are resized in a timely manner without impacting the website's performance. Lambda's on-demand nature only consumes resources during the resizing, making it operationally efficient.

Why other options are incorrect:

A: S3 Glacier is designed for long-term archival storage and retrieval, not for actively used images requiring resizing.
C: While using presigned URLs can offload image uploads from the web server, it doesn't address the image resizing requirement.
E: EventBridge on a schedule would not be triggered by the uploads themselves, introducing latency and inefficiencies in image resizing. S3 event notifications provide instant triggers when images are uploaded.

Authoritative Links:

Amazon S3: https://aws.amazon.com/s3/
AWS Lambda: https://aws.amazon.com/lambda/
S3 Event Notifications: https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-event-notifications.html

Question: 111 **Exam Heist**

A company recently migrated a message processing system to AWS. The system receives messages into an ActiveMQ queue running on an Amazon EC2 instance. Messages are processed by a consumer application running on Amazon EC2. The consumer application processes the messages and writes results to a MySQL database

running on Amazon EC2. The company wants this application to be highly available with low operational complexity.
Which architecture offers the HIGHEST availability?

A. Add a second ActiveMQ server to another Availability Zone. Add an additional consumer EC2 instance in another Availability Zone. Replicate the MySQL database to another Availability Zone.

B. Use Amazon MQ with active/standby brokers configured across two Availability Zones. Add an additional consumer EC2 instance in another Availability Zone. Replicate the MySQL database to another Availability Zone.

C. Use Amazon MQ with active/standby brokers configured across two Availability Zones. Add an additional consumer EC2 instance in another Availability Zone. Use Amazon RDS for MySQL with Multi-AZ enabled.

D. Use Amazon MQ with active/standby brokers configured across two Availability Zones. Add an Auto Scaling group for the consumer EC2 instances across two Availability Zones. Use Amazon RDS for MySQL with Multi-AZ enabled.

Answer: D

Explanation:

Option D provides the highest availability and lowest operational complexity due to several key AWS services.

1. **Amazon MQ with active/standby brokers (Multi-AZ):** This eliminates the need to manage ActiveMQ servers on EC2 instances. Amazon MQ is a managed message broker service, and the active/standby configuration across two Availability Zones (AZs) ensures automatic failover in case of an AZ outage. This is a significant improvement in availability compared to managing ActiveMQ on EC2, where you'd have to manually handle failover and replication. https://aws.amazon.com/mq/

2. **Auto Scaling group for consumer EC2 instances across two AZs:** This provides automatic scaling and high availability for the consumer application. Auto Scaling automatically adjusts the number of EC2 instances based on demand, ensuring that the application can handle traffic spikes. Distributing instances across multiple AZs ensures that the application remains available even if one AZ fails. https://aws.amazon.com/autoscaling/

3. **Amazon RDS for MySQL with Multi-AZ enabled:** This provides automatic failover and replication for the MySQL database. RDS Multi-AZ creates a standby replica of the database in another AZ, automatically failing over to the standby instance in case of a failure in the primary AZ. This significantly reduces downtime and simplifies database management. https://aws.amazon.com/rds/mysql/

By using these managed services and features, option D reduces operational overhead by eliminating the need to manage the message broker, scaling consumer instances, or handle database replication and failover manually.

Option A is less desirable as maintaining ActiveMQ servers on EC2 increases operational complexity. Option B lacks auto-scaling for the consumer instances, resulting in potential availability issues if traffic increases significantly. Option C similarly lacks auto-scaling for the consumer instances. Consequently, option D effectively handles message brokering, scaling, and data redundancy via RDS Multi-AZ across AZs. This is the best solution for high availability and simplified operations.

Question: 112 **Exam Heist**

A company hosts a containerized web application on a fleet of on-premises servers that process incoming requests. The number of requests is growing quickly. The on-premises servers cannot handle the increased number of requests. The company wants to move the application to AWS with minimum code changes and minimum development effort.
Which solution will meet these requirements with the LEAST operational overhead?

A. Use AWS Fargate on Amazon Elastic Container Service (Amazon ECS) to run the containerized web application with Service Auto Scaling. Use an Application Load Balancer to distribute the incoming requests.

B. Use two Amazon EC2 instances to host the containerized web application. Use an Application Load Balancer to distribute the incoming requests.

C. Use AWS Lambda with a new code that uses one of the supported languages. Create multiple Lambda functions to support the load. Use Amazon API Gateway as an entry point to the Lambda functions.

D. Use a high performance computing (HPC) solution such as AWS ParallelCluster to establish an HPC cluster that can process the incoming requests at the appropriate scale.

Answer: A

Explanation:

The best solution is A because it directly addresses the requirements of minimal code changes, minimal development effort, and least operational overhead for migrating a containerized application to AWS.

Here's why:

AWS Fargate and ECS: Fargate allows you to run containers without managing the underlying EC2 instances. ECS orchestrates the containers. This significantly reduces operational overhead because you don't need to provision, manage, and scale EC2 instances yourself.
Containerization Compatibility: The application is already containerized, making it a good fit for ECS and Fargate. This eliminates the need for significant code changes or re-architecting the application. The existing containers can be deployed almost as-is.
Application Load Balancer (ALB): ALB efficiently distributes incoming traffic across the container instances managed by ECS/Fargate. It provides features like health checks, traffic routing based on content, and SSL termination.
Service Auto Scaling: ECS Service Auto Scaling automatically adjusts the number of running container instances based on the incoming request load. This ensures optimal performance and cost efficiency without manual intervention.

Option B (EC2 instances) increases operational overhead compared to Fargate, as you must manage the EC2 instances, including patching, scaling, and capacity planning.

Option C (Lambda and API Gateway) requires significant code changes because you need to re-write the application logic to fit the Lambda function model. This involves substantially more development effort. It might also not be a direct port of the web application's existing logic.

Option D (AWS ParallelCluster) is designed for High-Performance Computing (HPC) workloads that typically involve tightly coupled parallel jobs. While it can scale, it is overkill and unnecessarily complex for a web application and introduces higher operational overhead than Fargate/ECS. It's suitable for scientific simulations, not general web traffic.

Authoritative Links:

AWS Fargate: https://aws.amazon.com/fargate/
Amazon ECS: https://aws.amazon.com/ecs/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
Service Auto Scaling: https://aws.amazon.com/autoscaling/

Question: 113 **Exam Heist**

A company uses 50 TB of data for reporting. The company wants to move this data from on premises to AWS. A custom application in the company’s data center runs a weekly data transformation job. The company plans to pause the application until the data transfer is complete and needs to begin the transfer process as soon as possible.

The data center does not have any available network bandwidth for additional workloads. A solutions architect must transfer the data and must configure the transformation job to continue to run in the AWS Cloud.
Which solution will meet these requirements with the LEAST operational overhead?

A. Use AWS DataSync to move the data. Create a custom transformation job by using AWS Glue.

B. Order an AWS Snowcone device to move the data. Deploy the transformation application to the device.

C. Order an AWS Snowball Edge Storage Optimized device. Copy the data to the device. Create a custom transformation job by using AWS Glue.

D. Order an AWS Snowball Edge Storage Optimized device that includes Amazon EC2 compute. Copy the data to the device. Create a new EC2 instance on AWS to run the transformation application.

Answer: C

Explanation:

Here's a breakdown of why option C is the best solution and why the others are less suitable, focusing on minimizing operational overhead and meeting the requirements:

Justification for Option C (Correct):

Option C leverages AWS Snowball Edge Storage Optimized for data transfer and AWS Glue for transformation, which offers the least operational overhead:

1. **Snowball Edge for Data Transfer:** Snowball Edge is ideal for transferring large datasets (50 TB) when network bandwidth is limited. This aligns perfectly with the scenario's constraint of no available network bandwidth. It avoids network congestion and faster compared to DataSync or direct network transfer.
2. **Storage Optimized Choice:** The "Storage Optimized" Snowball Edge is selected specifically for its large storage capacity, catering to the 50 TB data volume.
3. **AWS Glue for Transformation:** AWS Glue is a fully managed ETL (Extract, Transform, Load) service. By creating a custom transformation job in Glue, the company can replicate its existing transformation logic in the cloud without the overhead of managing EC2 instances. AWS Glue handles the infrastructure, scaling, and monitoring automatically.
4. **Minimizing Operational Overhead:** This solution minimizes operational overhead because Snowball Edge simplifies the data transfer process, and AWS Glue handles the transformation job's infrastructure management. The company doesn't need to manage EC2 instances for either the data transfer or the transformation process.

Why other options are less optimal:

Option A (AWS DataSync + Glue): DataSync is a network-based data transfer service. The question states the data center has no available network bandwidth for additional workloads, making DataSync an unsuitable choice.

Option B (Snowcone + Application on Device): AWS Snowcone is designed for edge computing in rugged environments. It has a smaller storage capacity than Snowball Edge, and deploying the transformation application directly on the device would add complexity and operational overhead because of limited processing capability.

Option D (Snowball Edge with EC2 + New EC2 Instance): While this solution could technically work, running the transformation application on a separate EC2 instance in the AWS Cloud after the data is transferred from the Snowball Edge introduces more operational overhead. You would need to manage, patch, and scale this EC2 instance. Using AWS Glue is a more managed and efficient transformation approach. Plus, using EC2 on snowball for small transformation jobs add complexity for management purpose of the infrastructure.

Supporting Concepts & Links:

AWS Snowball Edge: https://aws.amazon.com/snowball/
AWS Glue: https://aws.amazon.com/glue/
Data Transfer Options: https://aws.amazon.com/products/storage/data-transfer/

In conclusion, Option C provides the most efficient and cost-effective approach by leveraging Snowball Edge for data transfer and AWS Glue for the data transformation job, minimizing operational overhead and meeting all stated requirements.

Question: 114 **Exam Heist**

A company has created an image analysis application in which users can upload photos and add photo frames to their images. The users upload images and metadata to indicate which photo frames they want to add to their images. The application uses a single Amazon EC2 instance and Amazon DynamoDB to store the metadata.
The application is becoming more popular, and the number of users is increasing. The company expects the number of concurrent users to vary significantly depending on the time of day and day of week. The company must ensure that the application can scale to meet the needs of the growing user base.
Which solution meats these requirements?

A. Use AWS Lambda to process the photos. Store the photos and metadata in DynamoDB.

B. Use Amazon Kinesis Data Firehose to process the photos and to store the photos and metadata.

C. Use AWS Lambda to process the photos. Store the photos in Amazon S3. Retain DynamoDB to store the metadata.

D. Increase the number of EC2 instances to three. Use Provisioned IOPS SSD (io2) Amazon Elastic Block Store (Amazon EBS) volumes to store the photos and metadata.

Answer: C

Explanation:

The correct answer is **C: Use AWS Lambda to process the photos. Store the photos in Amazon S3. Retain DynamoDB to store the metadata.**

Here's a detailed justification:

The primary requirement is to scale the image analysis application to handle a growing user base with varying concurrency. The initial architecture using a single EC2 instance and DynamoDB becomes a bottleneck as the number of users increases.

Option A is suboptimal. While using Lambda for processing provides scalability, storing photos in DynamoDB is generally not recommended. DynamoDB is optimized for storing metadata and small documents, not large binary files like images, which can become expensive and inefficient.

Option B is inappropriate. Kinesis Data Firehose is designed for streaming data ingestion and loading into data stores/analytics tools. It's not intended for general image processing tasks like adding photo frames. Storing images in Firehose is not a viable solution.

Option D does not fully address the scalability issue. Simply increasing the number of EC2 instances to three still introduces scalability limitations and requires managing the EC2 instances. It also relies on EBS for storing photos which isn't ideal in terms of cost and scalability for image storage. Provisioned IOPS EBS volumes also adds unnecessary complexity and cost if high IOPS are not consistently needed.

Option C provides the best solution because it leverages the strengths of different AWS services:

AWS Lambda: Lambda allows serverless execution of the image processing logic (adding photo frames). It

automatically scales up or down based on demand, ensuring the application can handle varying levels of concurrency without manual intervention. This achieves the scalability requirement.

AWS Lambda Documentation: Provides overview and features about Lambda service.

Amazon S3: S3 is object storage for storing large amounts of data such as images. S3 offers durability, scalability, and cost-effectiveness for storing the photos.

Amazon S3 Documentation: Provides overview and features about S3 service.

DynamoDB: DynamoDB is a NoSQL database is best for storing metadata, it efficiently handles the growing amount of user metadata (photo frame preferences).

Amazon DynamoDB Documentation: Provides overview and features about DynamoDB service.

By separating the image processing task to Lambda, storing photos in S3, and retaining metadata in DynamoDB, the application gains scalability, cost-effectiveness, and improved performance. The solution aligns perfectly with the demands of a growing user base and fluctuating concurrency, making it the ideal choice.

Question: 115 **Exam Heist**

A medical records company is hosting an application on Amazon EC2 instances. The application processes customer data files that are stored on Amazon S3. The EC2 instances are hosted in public subnets. The EC2 instances access Amazon S3 over the internet, but they do not require any other network access.
A new requirement mandates that the network traffic for file transfers take a private route and not be sent over the internet.
Which change to the network architecture should a solutions architect recommend to meet this requirement?

A. Create a NAT gateway. Configure the route table for the public subnets to send traffic to Amazon S3 through the NAT gateway.

B. Configure the security group for the EC2 instances to restrict outbound traffic so that only traffic to the S3 prefix list is permitted.

C. Move the EC2 instances to private subnets. Create a VPC endpoint for Amazon S3, and link the endpoint to the route table for the private subnets.

D. Remove the internet gateway from the VPC. Set up an AWS Direct Connect connection, and route traffic to Amazon S3 over the Direct Connect connection.

Answer: C

Explanation:

Option C is the most suitable solution because it allows for private connectivity to S3 without traversing the internet, aligning perfectly with the requirement. Moving the EC2 instances to private subnets enhances security by isolating them from direct internet exposure. Creating a VPC endpoint for S3 establishes a private connection between the VPC and S3, ensuring all traffic remains within the AWS network. Associating the VPC endpoint with the private subnet's route table directs S3-bound traffic through the endpoint. This setup achieves the desired private route for file transfers, fulfilling the mandate of avoiding internet transmission. VPC endpoints are cost-effective and highly scalable.

Option A is incorrect because a NAT gateway is primarily used for allowing instances in private subnets to initiate outbound internet connections, not for directing traffic privately to S3.Option B is incorrect because security groups control traffic based on IP addresses and ports but don't provide a private route for S3 access; the traffic would still traverse the internet.Option D is incorrect because AWS Direct Connect establishes a dedicated network connection between on-premises infrastructure and AWS, which is

unnecessary and overly complex for the described requirement of internal S3 access. It also involves significant costs and setup overhead compared to a VPC endpoint.

Therefore, option C provides the most secure, efficient, and cost-effective solution for establishing a private connection to S3 for EC2 instances.

https://docs.aws.amazon.com/vpc/latest/privatelink/vpc-endpoints-s3.htmlhttps://aws.amazon.com/vpc/pricing/

Question: 116 **Exam Heist**

A company uses a popular content management system (CMS) for its corporate website. However, the required patching and maintenance are burdensome. The company is redesigning its website and wants anew solution. The website will be updated four times a year and does not need to have any dynamic content available. The solution must provide high scalability and enhanced security.
Which combination of changes will meet these requirements with the LEAST operational overhead? (Choose two.)

A. Configure Amazon CloudFront in front of the website to use HTTPS functionality.

B. Deploy an AWS WAF web ACL in front of the website to provide HTTPS functionality.

C. Create and deploy an AWS Lambda function to manage and serve the website content.

D. Create the new website and an Amazon S3 bucket. Deploy the website on the S3 bucket with static website hosting enabled.

E. Create the new website. Deploy the website by using an Auto Scaling group of Amazon EC2 instances behind an Application Load Balancer.

Answer: AD

Explanation:

The best solution, with the least operational overhead, is a combination of using Amazon S3 for static website hosting and Amazon CloudFront for content delivery with HTTPS.

D. Create the new website and an Amazon S3 bucket. Deploy the website on the S3 bucket with static website hosting enabled.

Static Website Hosting: S3 provides static website hosting. This eliminates the need for servers (EC2 instances) and operating system maintenance. S3 handles the underlying infrastructure management, ensuring high availability and scalability automatically.
Low Operational Overhead: S3 requires minimal operational effort. Uploading static files to an S3 bucket is straightforward, and S3 handles scaling and availability.
Cost-Effective: S3 storage is relatively inexpensive, and you only pay for the storage you use and the data you transfer.

A. Configure Amazon CloudFront in front of the website to use HTTPS functionality.

Content Delivery Network (CDN): CloudFront is a CDN that caches website content at edge locations globally. This provides faster content delivery to users, regardless of their location.
HTTPS Functionality: CloudFront allows you to use HTTPS for secure communication with users, improving website security. You can use AWS Certificate Manager (ACM) to easily provision and manage SSL/TLS certificates for CloudFront.
Enhanced Security: CloudFront protects against DDoS attacks and other threats at the edge.
Integration with S3: CloudFront integrates seamlessly with S3, making it easy to distribute content stored in S3.

Why other options are not ideal:

B. Deploy an AWS WAF web ACL in front of the website to provide HTTPS functionality: AWS WAF is a web application firewall that helps protect your web applications from common web exploits. While WAF is beneficial for security, it doesn't directly provide HTTPS functionality. HTTPS is usually configured at the CDN level, so A is the more applicable choice.
C. Create and deploy an AWS Lambda function to manage and serve the website content: Lambda is ideal for dynamic content and event-driven computing. This is a more complex solution and would have higher operational overhead than using S3 for static website hosting, which aligns better with the prompt requirement to keep it simple.
E. Create the new website. Deploy the website by using an Auto Scaling group of Amazon EC2 instances behind an Application Load Balancer: EC2 instances require operating system maintenance, patching, and scaling considerations, making them a higher-overhead solution than S3 for a static website.

Authoritative Links:

Amazon S3 Static Website Hosting:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/WebsiteHosting.html
Amazon CloudFront: https://aws.amazon.com/cloudfront/

Question: 117 **Exam Heist**

A company stores its application logs in an Amazon CloudWatch Logs log group. A new policy requires the company to store all application logs in Amazon OpenSearch Service (Amazon Elasticsearch Service) in near-real time.
Which solution will meet this requirement with the LEAST operational overhead?

A. Configure a CloudWatch Logs subscription to stream the logs to Amazon OpenSearch Service (Amazon Elasticsearch Service).

B. Create an AWS Lambda function. Use the log group to invoke the function to write the logs to Amazon OpenSearch Service (Amazon Elasticsearch Service).

C. Create an Amazon Kinesis Data Firehose delivery stream. Configure the log group as the delivery streams sources. Configure Amazon OpenSearch Service (Amazon Elasticsearch Service) as the delivery stream's destination.

D. Install and configure Amazon Kinesis Agent on each application server to deliver the logs to Amazon Kinesis Data Streams. Configure Kinesis Data Streams to deliver the logs to Amazon OpenSearch Service (Amazon Elasticsearch Service).

Answer: A

Explanation:

The most efficient way to stream CloudWatch Logs to Amazon OpenSearch Service (Amazon Elasticsearch Service) with the least operational overhead is to use a CloudWatch Logs subscription.

Option A is correct because CloudWatch Logs subscriptions offer a direct and managed integration with Amazon OpenSearch Service. A subscription allows you to define a filter pattern to select specific log events and automatically stream them to your OpenSearch domain. This eliminates the need for custom coding or infrastructure management. The CloudWatch Logs service handles the streaming and scaling, which minimizes operational overhead.

Option B requires creating and managing a Lambda function, which adds complexity and operational overhead for maintenance, monitoring, and potential scaling issues.

Option C, using Kinesis Data Firehose, is also a viable option, but CloudWatch Logs subscriptions provide a more direct path for sending logs to Amazon OpenSearch Service. Kinesis Data Firehose is generally better suited for broader data ingestion needs, but introduces an extra layer of configuration and management compared to direct subscriptions.

Option D involves installing and configuring Kinesis Agent on each application server and managing Kinesis Data Streams, leading to the highest operational overhead. This method introduces complexity with agent management, stream configuration, and potential data loss if agents are not properly configured. This option is unnecessary when the other options provide simpler, more managed solutions.

Therefore, Option A provides the simplest and most efficient solution due to the direct integration between CloudWatch Logs and Amazon OpenSearch Service using subscriptions. It avoids the operational overhead of managing custom functions, agents, or additional streaming services.

Reference links:

CloudWatch Logs Subscriptions
Sending CloudWatch Logs to Amazon OpenSearch Service

Question: 118 **Exam Heist**

A company is building a web-based application running on Amazon EC2 instances in multiple Availability Zones. The web application will provide access to a repository of text documents totaling about 900 TB in size. The company anticipates that the web application will experience periods of high demand. A solutions architect must ensure that the storage component for the text documents can scale to meet the demand of the application at all times. The company is concerned about the overall cost of the solution.
Which storage solution meets these requirements MOST cost-effectively?

A. Amazon Elastic Block Store (Amazon EBS)
B. Amazon Elastic File System (Amazon EFS)
C. Amazon OpenSearch Service (Amazon Elasticsearch Service)
D. Amazon S3

Answer: D

Explanation:

The correct answer is **D. Amazon S3**. Here's why:

Scalability and Durability: Amazon S3 (Simple Storage Service) is designed for virtually unlimited scalability and high durability. It can easily handle petabytes of data and is built to withstand the loss of data through redundant storage across multiple devices and facilities. This aligns directly with the requirement for scaling to accommodate 900 TB and meeting potentially high demand. https://aws.amazon.com/s3/

Cost-Effectiveness: S3 offers various storage classes (Standard, Intelligent-Tiering, Standard-IA, One Zone-IA, Glacier, Glacier Deep Archive) that allow optimization for different access patterns and cost considerations. For data accessed frequently, S3 Standard might be appropriate, whereas infrequently accessed data can be stored in cheaper tiers like S3 Standard-IA or Glacier, reducing overall storage costs. https://aws.amazon.com/s3/storage-classes/

Web Application Integration: S3 integrates seamlessly with web applications. EC2 instances can easily retrieve and serve text documents directly from S3 via HTTP/HTTPS. The S3 API simplifies the process of accessing and managing objects.

Other options and why they are less ideal:

A. Amazon Elastic Block Store (Amazon EBS): EBS volumes are block storage and are attached to individual EC2 instances. Storing 900 TB on EBS would be prohibitively expensive and complex to manage, requiring a large number of attached volumes across instances, and scaling would be less efficient. EBS is better suited for operating systems, application software, and frequently accessed data closely tied to a specific instance.

B. Amazon Elastic File System (Amazon EFS): EFS provides shared file storage for EC2 instances. While EFS can scale, storing 900 TB would still be relatively more expensive than S3, and the performance characteristics, while suitable for some shared file workloads, are typically not optimized for serving static web content like text documents as effectively as S3. EFS's cost structure makes it less favorable for large datasets than S3, especially when considering infrequent access scenarios.
https://aws.amazon.com/efs/pricing/

C. Amazon OpenSearch Service (Amazon Elasticsearch Service): OpenSearch is a search and analytics engine. While it could store and index the text documents, it's not optimized for simply storing and serving large files. Moreover, indexing 900 TB of data in OpenSearch would be much more costly and complex than storing the files in S3. OpenSearch is for full-text search and analysis, not general-purpose large-scale storage.

In summary, S3 provides the best combination of scalability, cost-effectiveness, and integration capabilities for storing and serving a large repository of text documents for a web application experiencing varying demand.

Question: 119 **Exam Heist**

A global company is using Amazon API Gateway to design REST APIs for its loyalty club users in the us-east-1 Region and the ap-southeast-2 Region. A solutions architect must design a solution to protect these API Gateway managed REST APIs across multiple accounts from SQL injection and cross-site scripting attacks.
Which solution will meet these requirements with the LEAST amount of administrative effort?

A. Set up AWS WAF in both Regions. Associate Regional web ACLs with an API stage.
B. Set up AWS Firewall Manager in both Regions. Centrally configure AWS WAF rules.
C. Set up AWS Shield in bath Regions. Associate Regional web ACLs with an API stage.
D. Set up AWS Shield in one of the Regions. Associate Regional web ACLs with an API stage.

Answer: B

Explanation:

The correct answer is B. Here's a detailed justification:

The problem requires a solution to protect API Gateway REST APIs across multiple AWS accounts and regions (us-east-1 and ap-southeast-2) from common web exploits like SQL injection and cross-site scripting (XSS) with minimal administrative overhead.

Option B, setting up AWS Firewall Manager and centrally configuring AWS WAF rules, is the most efficient approach. AWS Firewall Manager is designed to centrally manage and deploy AWS WAF rules across multiple accounts and regions. This provides a single pane of glass for managing security policies. With Firewall Manager, you can define a WAF policy once and apply it consistently across all your APIs, significantly reducing administrative effort. You create an AWS WAF rule group within Firewall Manager and then deploy that rule group across the accounts and regions in question. The WAF rules can be tailored to specifically block SQL injection and XSS attacks using managed rule groups or custom rules based on regular expressions or other criteria.

Option A, setting up AWS WAF in both regions and associating Regional web ACLs with an API stage, while functionally correct, requires more manual configuration. You would need to create and maintain the same WAF rules in each region separately. This increases administrative overhead and the potential for inconsistencies. It doesn't offer the central management that Firewall Manager does.

Options C and D involve AWS Shield. AWS Shield provides protection against Distributed Denial of Service

(DDoS) attacks, which is not the primary concern in this scenario. While Shield Advanced offers some WAF integration, it's not the core focus for SQL injection and XSS protection. Furthermore, AWS Shield's pricing model is different and potentially more expensive than Firewall Manager for this use case. Shield also does not provide the centralized management across multiple accounts that Firewall Manager provides.

Therefore, AWS Firewall Manager with centrally managed AWS WAF rules offers the best balance of security effectiveness and administrative efficiency for protecting API Gateway REST APIs across multiple accounts and regions from SQL injection and XSS attacks.

Authoritative links:

AWS Firewall Manager: https://aws.amazon.com/firewall-manager/
AWS WAF: https://aws.amazon.com/waf/

Question: 120 **Exam Heist**

A company has implemented a self-managed DNS solution on three Amazon EC2 instances behind a Network Load Balancer (NLB) in the us-west-2 Region. Most of the company's users are located in the United States and Europe. The company wants to improve the performance and availability of the solution. The company launches and configures three EC2 instances in the eu-west-1 Region and adds the EC2 instances as targets for a new NLB. Which solution can the company use to route traffic to all the EC2 instances?

A. Create an Amazon Route 53 geolocation routing policy to route requests to one of the two NLBs. Create an Amazon CloudFront distribution. Use the Route 53 record as the distribution's origin.

B. Create a standard accelerator in AWS Global Accelerator. Create endpoint groups in us-west-2 and eu-west-1. Add the two NLBs as endpoints for the endpoint groups.

C. Attach Elastic IP addresses to the six EC2 instances. Create an Amazon Route 53 geolocation routing policy to route requests to one of the six EC2 instances. Create an Amazon CloudFront distribution. Use the Route 53 record as the distribution's origin.

D. Replace the two NLBs with two Application Load Balancers (ALBs). Create an Amazon Route 53 latency routing policy to route requests to one of the two ALBs. Create an Amazon CloudFront distribution. Use the Route 53 record as the distribution's origin.

Answer: B

Explanation:

The best solution is **B. Create a standard accelerator in AWS Global Accelerator. Create endpoint groups in us-west-2 and eu-west-1. Add the two NLBs as endpoints for the endpoint groups.**

Here's why:

Global Performance & Availability: The primary requirement is to improve performance and availability for users in the US and Europe. AWS Global Accelerator is specifically designed for this. It leverages the AWS global network to direct user traffic to the closest healthy endpoint, improving latency and reliability.

NLB Compatibility: Global Accelerator is directly compatible with Network Load Balancers (NLBs). It can treat the NLBs in us-west-2 and eu-west-1 as endpoints.

Simple Configuration: The proposed solution involves creating a Global Accelerator, defining endpoint groups for each region, and associating the respective NLBs with those groups. This is a relatively straightforward configuration.

Fault Tolerance: Global Accelerator continually monitors the health of endpoints. If an NLB or the EC2 instances behind it become unhealthy, Global Accelerator will automatically redirect traffic to the healthy endpoint in the other region, ensuring high availability.

Why other options are not optimal:

A. Route 53 Geolocation with CloudFront: While Route 53 Geolocation routing directs traffic based on the user's location, it doesn't inherently provide the performance benefits of Global Accelerator's anycast addressing and AWS global network optimization. CloudFront caches static content but won't significantly improve the performance of dynamic DNS queries served by EC2 instances. The user will still reach the regional NLBs that they have in place, which is only a regional solution.
C. Elastic IPs and Route 53 Geolocation: Attaching Elastic IPs to individual EC2 instances and using Route 53 geolocation would work but presents several challenges. It involves managing individual instances directly, increasing operational complexity. It would be difficult to implement health checks and ensure automated failover. It also won't benefit from the anycast network that Global Accelerator provides.
D. Application Load Balancers (ALBs) with Route 53 Latency Routing and CloudFront: While ALBs offer more features than NLBs, the core problem remains the lack of global traffic management and network optimization. Route 53 Latency-based routing is useful, but it isn't as performant as Global Accelerator as it still has to traverse over the public internet to the designated region. CloudFront will only help for cached content. In addition, the company is already using NLBs, switching to ALBs would require more significant architecture changes.

In summary: Global Accelerator with NLBs in different regions provides the best balance of performance, availability, and ease of implementation for routing traffic to DNS servers for users in the US and Europe.

Authoritative Links:

AWS Global Accelerator: https://aws.amazon.com/global-accelerator/
AWS Route 53: https://aws.amazon.com/route53/
AWS Network Load Balancer: https://aws.amazon.com/elasticloadbalancing/network-load-balancer/

Question: 121 **Exam Heist**

A company is running an online transaction processing (OLTP) workload on AWS. This workload uses an unencrypted Amazon RDS DB instance in a Multi-AZ deployment. Daily database snapshots are taken from this instance.
What should a solutions architect do to ensure the database and snapshots are always encrypted moving forward?

A. Encrypt a copy of the latest DB snapshot. Replace existing DB instance by restoring the encrypted snapshot.

B. Create a new encrypted Amazon Elastic Block Store (Amazon EBS) volume and copy the snapshots to it. Enable encryption on the DB instance.

C. Copy the snapshots and enable encryption using AWS Key Management Service (AWS KMS) Restore encrypted snapshot to an existing DB instance.

D. Copy the snapshots to an Amazon S3 bucket that is encrypted using server-side encryption with AWS Key Management Service (AWS KMS) managed keys (SSE-KMS).

Answer: A

Explanation:

The correct answer is A. Here's why:

Encryption at Rest for RDS: Amazon RDS supports encryption at rest, but it must be enabled during instance creation or by restoring from an encrypted snapshot. You cannot directly encrypt an existing unencrypted RDS instance in place.
Snapshot Encryption: The process involves creating an encrypted copy of the most recent DB snapshot. This encrypted snapshot serves as the source for creating a new, encrypted RDS instance.
Restoring from Encrypted Snapshot: By restoring the encrypted snapshot, a new RDS instance is launched with encryption enabled.

Replacing the Existing Instance: To fully transition to an encrypted setup, you need to replace the original, unencrypted instance with the newly created encrypted one. This might involve updating application connection strings to point to the new instance and decommissioning the old one.

Why other options are incorrect:

B: Encrypting an EBS volume and copying snapshots there doesn't encrypt the RDS instance itself. RDS snapshots are stored separately and managed by AWS. It doesn't mention how you will create a new encrypted RDS instance.
C: While you can copy snapshots and encrypt them, restoring an encrypted snapshot to an existing unencrypted DB instance isn't possible. RDS requires you to create a new encrypted instance from an encrypted snapshot.
D: Copying snapshots to an encrypted S3 bucket doesn't encrypt the RDS instance or the snapshots in a way that RDS can use directly for creating an encrypted instance. S3 encryption is separate from RDS encryption at rest.

In summary, Option A provides the correct and complete process to move an unencrypted RDS instance to an encrypted setup: encrypt a copy of the snapshot, restore from the encrypted snapshot to create a new encrypted instance, and replace the existing instance with the new one.

Supporting Documentation:

Amazon RDS Encryption
Encrypting Amazon RDS Resources

Question: 122 **Exam Heist**

A company wants to build a scalable key management infrastructure to support developers who need to encrypt data in their applications.
What should a solutions architect do to reduce the operational burden?

A. Use multi-factor authentication (MFA) to protect the encryption keys.
B. Use AWS Key Management Service (AWS KMS) to protect the encryption keys.
C. Use AWS Certificate Manager (ACM) to create, store, and assign the encryption keys.
D. Use an IAM policy to limit the scope of users who have access permissions to protect the encryption keys.

Answer: B

Explanation:

The correct answer is B: Use AWS Key Management Service (AWS KMS) to protect the encryption keys. Here's a detailed justification:

AWS KMS is a managed service specifically designed to simplify the creation, storage, and control of encryption keys used to encrypt data. Using KMS reduces the operational burden by offloading key management responsibilities from the company to AWS. KMS handles tasks like key generation, rotation, storage, and deletion in a secure and compliant manner. This frees developers from having to build and maintain their own key management infrastructure, which can be complex and time-consuming.

Option A, using MFA, enhances security by requiring multiple authentication factors, but it doesn't address the core operational burden of managing keys. MFA protects against unauthorized access, but it doesn't simplify key generation, storage, or rotation.

Option C, using AWS Certificate Manager (ACM), is primarily for managing SSL/TLS certificates used for securing network traffic. ACM is not designed for general-purpose key management for application data

encryption. While ACM can create and store private keys for certificates, it's not the appropriate tool for managing keys used within application code to encrypt arbitrary data.

Option D, using an IAM policy to limit access, is a crucial security practice. However, similar to MFA, it doesn't alleviate the burden of creating, storing, rotating, and managing the encryption keys themselves. IAM helps control who can use the keys, but it doesn't manage the lifecycle of the keys.

Therefore, KMS offers the most direct solution to reducing the operational burden associated with key management. It provides a centralized, managed service for handling encryption keys, allowing developers to focus on their application code rather than the complexities of key management. KMS integrates well with other AWS services and supports various encryption algorithms.

For further research, you can refer to the following AWS documentation:

AWS Key Management Service (KMS): https://aws.amazon.com/kms/
AWS Certificate Manager (ACM): https://aws.amazon.com/certificate-manager/
IAM Policies: https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies.html

Question: 123 **Exam Heist**

A company has a dynamic web application hosted on two Amazon EC2 instances. The company has its own SSL certificate, which is on each instance to perform SSL termination.
There has been an increase in traffic recently, and the operations team determined that SSL encryption and decryption is causing the compute capacity of the web servers to reach their maximum limit.
What should a solutions architect do to increase the application's performance?

A. Create a new SSL certificate using AWS Certificate Manager (ACM). Install the ACM certificate on each instance.

B. Create an Amazon S3 bucket Migrate the SSL certificate to the S3 bucket. Configure the EC2 instances to reference the bucket for SSL termination.

C. Create another EC2 instance as a proxy server. Migrate the SSL certificate to the new instance and configure it to direct connections to the existing EC2 instances.

D. Import the SSL certificate into AWS Certificate Manager (ACM). Create an Application Load Balancer with an HTTPS listener that uses the SSL certificate from ACM.

Answer: D

Explanation:

The correct answer is D because it addresses the performance bottleneck caused by SSL encryption/decryption on the EC2 instances by offloading this task to a dedicated service: the Application Load Balancer (ALB).

Here's a detailed justification:

SSL Offloading: SSL encryption and decryption are CPU-intensive tasks. By moving this responsibility to the ALB, the EC2 instances are freed from this burden, allowing them to focus on serving application logic. This significantly improves their performance and reduces CPU utilization.

Application Load Balancer (ALB): An ALB is designed for layer 7 (application layer) load balancing. It can handle HTTPS traffic, performing SSL termination and then forwarding the decrypted traffic to the EC2 instances. This ensures secure communication while optimizing resource utilization.

AWS Certificate Manager (ACM): ACM simplifies the process of obtaining, managing, and deploying SSL/TLS certificates for use with AWS services. Importing the existing SSL certificate into ACM allows the ALB to easily access and use it for SSL termination.

Scalability and High Availability: ALBs are highly scalable and provide built-in high availability. They can automatically distribute traffic across multiple EC2 instances, ensuring that the application remains available even if one instance fails.

Cost-Effectiveness: While there is a cost associated with using an ALB, the performance gains and reduced resource consumption on the EC2 instances can lead to overall cost savings.

Why other options are incorrect:

A: Creating a new SSL certificate using ACM and installing it on each instance does not solve the underlying problem of SSL processing burdening the EC2 instances. It merely replaces the existing certificate.
B: Storing the SSL certificate in S3 and referencing it from the EC2 instances would not offload the SSL decryption processing. The EC2 instances would still need to perform the decryption themselves. This adds complexity without solving the core issue.
C: Creating a proxy server (another EC2 instance) shifts the SSL decryption burden to the proxy. While it might alleviate some load on the original web servers, it's not as scalable or efficient as using a managed service like ALB. It also adds complexity to the architecture.

Authoritative Links:

Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
AWS Certificate Manager: https://aws.amazon.com/certificate-manager/
SSL Offloading: https://aws.amazon.com/blogs/security/how-to-deploy-https-ssl-tls-with-aws-certificate-manager-and-elastic-load-balancer/

Question: 124 **Exam Heist**

A company has a highly dynamic batch processing job that uses many Amazon EC2 instances to complete it. The job is stateless in nature, can be started and stopped at any given time with no negative impact, and typically takes upwards of 60 minutes total to complete. The company has asked a solutions architect to design a scalable and cost-effective solution that meets the requirements of the job.
What should the solutions architect recommend?

A. Implement EC2 Spot Instances.
B. Purchase EC2 Reserved Instances.
C. Implement EC2 On-Demand Instances.
D. Implement the processing on AWS Lambda.

Answer: A

Explanation:

The correct answer is A, Implement EC2 Spot Instances. Here's why:

Spot Instances are unused EC2 capacity available in the AWS cloud at steep discounts compared to On-Demand prices. They are ideal for fault-tolerant, stateless, and flexible workloads, perfectly fitting the company's batch processing job requirements. Because the job is stateless and can be interrupted without negative impact, the possibility of Spot Instance interruptions is mitigated. This allows the company to leverage significant cost savings, often up to 90%, compared to On-Demand Instances.

Reserved Instances, while providing cost savings, require a commitment to instance usage for a specific period (1 or 3 years). Given the dynamic nature of the batch processing job, purchasing Reserved Instances may result in wasted capacity and reduced cost-effectiveness if the job doesn't consistently require that capacity.

On-Demand Instances offer flexibility but are the most expensive option. For a job that can tolerate

interruptions and aims for cost optimization, On-Demand Instances are not the best choice.

AWS Lambda is not suitable because it has execution time limits. Typical Lambda execution is capped at 15 minutes. The batch processing job takes upwards of 60 minutes to complete.

Therefore, Spot Instances provide the most cost-effective and scalable solution for this specific scenario. They allow the company to leverage the flexibility of EC2 while minimizing costs, given the job's fault tolerance and flexibility.

Further Research:

AWS EC2 Spot Instances: https://aws.amazon.com/ec2/spot/
AWS EC2 Pricing: https://aws.amazon.com/ec2/pricing/

Question: 125 **Exam Heist**

A company runs its two-tier ecommerce website on AWS. The web tier consists of a load balancer that sends traffic to Amazon EC2 instances. The database tier uses an Amazon RDS DB instance. The EC2 instances and the RDS DB instance should not be exposed to the public internet. The EC2 instances require internet access to complete payment processing of orders through a third-party web service. The application must be highly available.
Which combination of configuration options will meet these requirements? (Choose two.)

A. Use an Auto Scaling group to launch the EC2 instances in private subnets. Deploy an RDS Multi-AZ DB instance in private subnets.

B. Configure a VPC with two private subnets and two NAT gateways across two Availability Zones. Deploy an Application Load Balancer in the private subnets.

C. Use an Auto Scaling group to launch the EC2 instances in public subnets across two Availability Zones. Deploy an RDS Multi-AZ DB instance in private subnets.

D. Configure a VPC with one public subnet, one private subnet, and two NAT gateways across two Availability Zones. Deploy an Application Load Balancer in the public subnet.
D. Configure a VPC with two public subnets, two private subnets, and two NAT gateways across two Availability Zones. Deploy an Application Load Balancer in the public subnets.

Answer: AE

Explanation:

Let's break down why options A and E are the correct choices for achieving high availability, secure communication with a third-party service, and keeping the EC2 instances and RDS DB instance private.

Option A: Use an Auto Scaling group to launch the EC2 instances in private subnets. Deploy an RDS Multi-AZ DB instance in private subnets.

This addresses a core requirement: keeping both the EC2 instances and the RDS database out of the public internet. Private subnets do not have direct internet access. Using an Auto Scaling group across multiple Availability Zones ensures high availability of the web tier, which is essential for the ecommerce website. A Multi-AZ RDS DB instance does the same for the database tier, providing failover in case of an outage.

Option E: Configure a VPC with two public subnets, two private subnets, and two NAT gateways across two Availability Zones. Deploy an Application Load Balancer in the public subnets.

This configuration provides the necessary network infrastructure for the application. The Application Load Balancer (ALB) sits in public subnets, allowing it to receive traffic from the internet and distribute it to the EC2 instances in the private subnets. The two private subnets, each in a different Availability Zone, provide a highly available environment. NAT Gateways, each in a different Availability Zone, allow the EC2 instances in the private subnets to initiate outbound traffic to the third-party payment processor while remaining

inaccessible from the internet. The redundancy of two NAT Gateways enhances availability.

Why other options are incorrect:

Option B: Placing the ALB in private subnets prevents it from receiving traffic from the public internet, defeating its purpose.
Option C: Launching EC2 instances in public subnets exposes them to the internet directly, which violates the security requirements.
Option D: Having only one public and one private subnet doesn't inherently guarantee availability across Availability Zones. It also lacks redundancy in NAT Gateway.

In essence, Options A and E establish a secure and highly available architecture:

Security: Private subnets for EC2 instances and RDS, and NAT gateways control outbound traffic.
Availability: Multi-AZ RDS and Auto Scaling group across AZs ensures high availability.
Functionality: ALB in public subnets handles inbound traffic and distributes it to web tier. NAT gateways allow private EC2s to connect to payment processing on the internet.

Supporting Documentation:

Amazon VPC: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Introduction.html
Amazon RDS Multi-AZ: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html
Auto Scaling: https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html
NAT Gateway: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html
Application Load Balancer: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html

Question: 126 **Exam Heist**

A solutions architect needs to implement a solution to reduce a company's storage costs. All the company's data is in the Amazon S3 Standard storage class. The company must keep all data for at least 25 years. Data from the most recent 2 years must be highly available and immediately retrievable.
Which solution will meet these requirements?

A. Set up an S3 Lifecycle policy to transition objects to S3 Glacier Deep Archive immediately.

B. Set up an S3 Lifecycle policy to transition objects to S3 Glacier Deep Archive after 2 years.

C. Use S3 Intelligent-Tiering. Activate the archiving option to ensure that data is archived in S3 Glacier Deep Archive.

D. Set up an S3 Lifecycle policy to transition objects to S3 One Zone-Infrequent Access (S3 One Zone-IA) immediately and to S3 Glacier Deep Archive after 2 years.

Answer: B

Explanation:

The correct solution is to transition objects to S3 Glacier Deep Archive after 2 years using an S3 Lifecycle policy. Here's why:

Requirement 1: Cost Reduction: S3 Glacier Deep Archive offers the lowest storage cost among all S3 storage classes, making it ideal for long-term archival data. https://aws.amazon.com/s3/storage-classes/glacier/
Requirement 2: Data Retention: The company needs to retain data for 25 years, which S3 Glacier Deep Archive fully supports.
Requirement 3: Highly Available & Immediately Retrievable for 2 Years: The most recent 2 years of data need to be highly available and immediately retrievable. Keeping the data in S3 Standard for the first 2 years

satisfies this requirement.
S3 Lifecycle Policies: These policies automate the transition of objects between different S3 storage classes based on defined rules. This is crucial for managing data retention and storage costs efficiently.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-concept.html

Option A is incorrect because it immediately transitions all data to S3 Glacier Deep Archive, violating the requirement for the most recent 2 years of data to be highly available and immediately retrievable.

Option C, using S3 Intelligent-Tiering with the archiving option, would likely be more expensive than directly transitioning to S3 Glacier Deep Archive after 2 years, since Intelligent-Tiering monitors access patterns. While it archives infrequently accessed data, it's designed for data with varying access patterns, not purely archival needs with a defined accessibility window.

Option D is incorrect because S3 One Zone-IA is not designed for long-term archival storage with minimal access. It also trades off availability by storing data in a single Availability Zone. It's cheaper than S3 Standard, but not as cost-effective as S3 Glacier Deep Archive for long-term retention. The transition to S3 Glacier Deep Archive after 2 years is correct, but the initial transition to S3 One Zone-IA is unnecessary and introduces availability risks.

Therefore, transitioning to S3 Glacier Deep Archive only after the data has been stored in S3 Standard for the first 2 years balances cost optimization with the accessibility requirements. This approach ensures that the most recent data is readily available while older data is stored in the most cost-effective manner for long-term archival.

Question: 127 **Exam Heist**

A media company is evaluating the possibility of moving its systems to the AWS Cloud. The company needs at least 10 TB of storage with the maximum possible I/O performance for video processing, 300 TB of very durable storage for storing media content, and 900 TB of storage to meet requirements for archival media that is not in use anymore.
Which set of services should a solutions architect recommend to meet these requirements?

A. Amazon EBS for maximum performance, Amazon S3 for durable data storage, and Amazon S3 Glacier for archival storage

B. Amazon EBS for maximum performance, Amazon EFS for durable data storage, and Amazon S3 Glacier for archival storage

C. Amazon EC2 instance store for maximum performance, Amazon EFS for durable data storage, and Amazon S3 for archival storage

D. Amazon EC2 instance store for maximum performance, Amazon S3 for durable data storage, and Amazon S3 Glacier for archival storage

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct answer and why the other options are less suitable for the media company's storage requirements:

Option D: Amazon EC2 instance store, Amazon S3, and Amazon S3 Glacier

Amazon EC2 Instance Store (for Maximum Performance): Instance stores provide the highest I/O performance directly attached to the EC2 instance. This makes them ideal for video processing, which requires fast read/write speeds for manipulating large video files. The ephemeral nature of instance store is acceptable as data can be readily retrieved from durable storage.

Reference: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/InstanceStorage.html
Amazon S3 (for Durable Data Storage): S3 is designed for 99.999999999% durability, making it suitable for storing critical media content long-term. S3 offers a balance of cost-effectiveness and accessibility for frequently accessed media. With 300TB of storage, it is appropriate since the media content is often used.

Reference: https://aws.amazon.com/s3/storage-classes/
Amazon S3 Glacier (for Archival Storage): Glacier is a low-cost storage option specifically designed for infrequently accessed data. It is perfect for archival media that is no longer in active use. With 900TB of storage, it is appropriate since the data is rarely used.

Reference: https://aws.amazon.com/glacier/

Why other options are incorrect:

Option A (Amazon EBS, Amazon S3, and Amazon S3 Glacier): While EBS provides persistent block storage for EC2, it doesn't offer the same level of I/O performance as instance store for intensive workloads like video processing. EBS is also more expensive per GB than instance store. It is also inappropriate to use Amazon EBS as an external storage location, since the EBS volume is attached to an EC2 instance.

Option B (Amazon EBS, Amazon EFS, and Amazon S3 Glacier): EFS is a scalable file storage service for use with EC2, but it is not optimized for maximum I/O performance. Video processing would benefit more from the high throughput of instance store. EFS is also more expensive than S3 for durable storage. It is also inappropriate to use Amazon EBS as an external storage location, since the EBS volume is attached to an EC2 instance.

Option C (Amazon EC2 Instance Store, Amazon EFS, and Amazon S3): While instance store is correct for video processing, EFS is less durable and more expensive than S3 for general durable media storage. S3 Glacier is a better fit for the archival storage requirements rather than using S3 for both durable and archival, since Glacier is designed and priced for archival storage.

In summary: Option D provides the most cost-effective and performance-optimized solution by leveraging instance store for performance, S3 for durability, and Glacier for archival, fully meeting all storage needs outlined in the problem.

Question: 128 **Exam Heist**

A company wants to run applications in containers in the AWS Cloud. These applications are stateless and can tolerate disruptions within the underlying infrastructure. The company needs a solution that minimizes cost and operational overhead.
What should a solutions architect do to meet these requirements?

A. Use Spot Instances in an Amazon EC2 Auto Scaling group to run the application containers.
B. Use Spot Instances in an Amazon Elastic Kubernetes Service (Amazon EKS) managed node group.
C. Use On-Demand Instances in an Amazon EC2 Auto Scaling group to run the application containers.
D. Use On-Demand Instances in an Amazon Elastic Kubernetes Service (Amazon EKS) managed node group.

Answer: B

Explanation:

The correct answer is **B. Use Spot Instances in an Amazon Elastic Kubernetes Service (Amazon EKS) managed node group.**

Here's a detailed justification:

The problem statement emphasizes minimizing cost and operational overhead while running stateless containerized applications that can tolerate disruptions. Spot Instances offer significant cost savings compared to On-Demand Instances because they utilize spare EC2 capacity. However, Spot Instances can be terminated with short notice (2-minute warning), making them unsuitable for stateful or disruption-sensitive workloads. Because the application is stateless and tolerant to disruptions, using Spot Instances aligns with the cost optimization requirement.

Using Amazon EKS managed node groups simplifies the management of the underlying EC2 instances that run the containers. EKS handles patching, updating, and scaling the nodes within the group. By using managed node groups, the company reduces operational overhead associated with managing EC2 instances directly. Further, EKS offers inherent benefits for managing container deployments, scaling, and orchestration, all which contribute to reduced operational overhead.

Combining Spot Instances with EKS managed node groups creates a cost-effective and operationally efficient solution. EKS orchestrates the containers, handling the redeployment of containers if a Spot Instance is terminated, because of the application's stateless nature. Options C and D use On-Demand Instances, which are more expensive than Spot Instances, and would violate the cost minimization requirement. Option A, while using Spot Instances, lacks the orchestration and management features of EKS, leading to higher operational overhead. Running containers directly on EC2 instances without an orchestrator like Kubernetes introduces complexity in deployment, scaling, and management.

Therefore, using Spot Instances in an Amazon EKS managed node group is the best approach to meet the requirements of minimizing cost and operational overhead while running disruption-tolerant, stateless containerized applications.

Supporting Documentation:

Amazon EKS Managed Node Groups: https://docs.aws.amazon.com/eks/latest/userguide/managed-node-groups.html
Amazon EC2 Spot Instances: https://aws.amazon.com/ec2/spot/
Kubernetes Concepts: https://kubernetes.io/docs/concepts/

Question: 129 **Exam Heist**

A company is running a multi-tier web application on premises. The web application is containerized and runs on a number of Linux hosts connected to a PostgreSQL database that contains user records. The operational overhead of maintaining the infrastructure and capacity planning is limiting the company's growth. A solutions architect must improve the application's infrastructure.
Which combination of actions should the solutions architect take to accomplish this? (Choose two.)

A. Migrate the PostgreSQL database to Amazon Aurora.

B. Migrate the web application to be hosted on Amazon EC2 instances.

C. Set up an Amazon CloudFront distribution for the web application content.

D. Set up Amazon ElastiCache between the web application and the PostgreSQL database.

E. Migrate the web application to be hosted on AWS Fargate with Amazon Elastic Container Service (Amazon ECS).

Answer: AE

Explanation:

The correct answer is AE. Here's why:

A. Migrate the PostgreSQL database to Amazon Aurora:

Managed Database Service: Migrating the on-premises PostgreSQL database to Amazon Aurora PostgreSQL offers a fully managed database service. This significantly reduces the operational overhead associated with tasks such as patching, backups, and database scaling. Aurora also offers superior performance compared to standard PostgreSQL deployments.
Scalability and Availability: Aurora provides automatic scaling capabilities, allowing the database to handle fluctuating workloads without manual intervention. It also offers high availability features, ensuring minimal downtime.
Reduced Operational Burden: The company is currently burdened by the operational overhead of maintaining the database infrastructure. Aurora removes this burden, freeing up resources for other tasks.

E. Migrate the web application to be hosted on AWS Fargate with Amazon Elastic Container Service (Amazon ECS):

Serverless Containerization: Fargate is a serverless compute engine for ECS that allows you to run containers without managing underlying EC2 instances. This eliminates the operational overhead of provisioning, patching, and scaling EC2 instances.
Simplified Deployment and Management: ECS simplifies the deployment, management, and scaling of containerized applications. Fargate integration further simplifies the process by removing the need to manage the underlying infrastructure.
Improved Scalability and Availability: ECS with Fargate offers improved scalability and availability for the web application. ECS can automatically scale the number of containers based on demand, while Fargate ensures that the containers are always running on healthy infrastructure.

Why the other options are less suitable:

B. Migrate the web application to be hosted on Amazon EC2 instances: While moving to EC2 is a step towards the cloud, it still requires managing the underlying operating systems, patching, and scaling, which doesn't significantly alleviate the operational overhead.
C. Set up an Amazon CloudFront distribution for the web application content: CloudFront primarily focuses on content delivery and caching, improving application performance for geographically distributed users. While beneficial, it doesn't address the core problem of reducing operational overhead related to the web application and database infrastructure.
D. Set up Amazon ElastiCache between the web application and the PostgreSQL database: ElastiCache is a caching service that can improve application performance by caching frequently accessed data. However, it doesn't directly address the operational overhead of managing the underlying infrastructure and database.

Authoritative Links:

Amazon Aurora: https://aws.amazon.com/rds/aurora/
AWS Fargate: https://aws.amazon.com/fargate/
Amazon Elastic Container Service (ECS): https://aws.amazon.com/ecs/

Question: 130 **Exam Heist**

An application runs on Amazon EC2 instances across multiple Availability Zonas. The instances run in an Amazon EC2 Auto Scaling group behind an Application Load Balancer. The application performs best when the CPU utilization of the EC2 instances is at or near 40%.
What should a solutions architect do to maintain the desired performance across all instances in the group?

A. Use a simple scaling policy to dynamically scale the Auto Scaling group.
B. Use a target tracking policy to dynamically scale the Auto Scaling group.
C. Use an AWS Lambda function ta update the desired Auto Scaling group capacity.
D. Use scheduled scaling actions to scale up and scale down the Auto Scaling group.

Answer: B

Explanation:

Here's a detailed justification for why option B is the best solution and why the other options are less suitable for maintaining desired performance based on CPU utilization in an Auto Scaling group:

The core requirement is to keep the EC2 instance CPU utilization around 40%. To achieve this, the Auto Scaling group must dynamically adjust the number of instances based on the actual CPU load. A **target tracking scaling policy** is specifically designed for this scenario. It allows you to set a target value for a metric (in this case, CPU utilization) and the Auto Scaling group automatically adjusts the number of instances to maintain that target. It continuously monitors the CloudWatch metric (CPUUtilization) and adds or removes instances as needed to stay close to the 40% target.

Option A, **simple scaling policy**, is less ideal. While it allows scaling based on metrics, it operates with fixed adjustments. You would need to define how many instances to add or remove based on breaches of upper and lower thresholds. It doesn't continuously strive to maintain a specific target like 40%. Configuring the exact adjustments required for a complex workload would be more cumbersome than using target tracking.

Option C, using an **AWS Lambda function**, introduces unnecessary complexity. While Lambda can update the desired capacity, you would need to write custom code to monitor CloudWatch metrics, calculate the required capacity adjustments, and then update the Auto Scaling group. Target tracking provides a managed solution that handles all of this automatically, reducing operational overhead. Relying on Lambda would also mean managing code, potential errors in your logic, and the IAM permissions required for the Lambda function.

Option D, **scheduled scaling actions**, is not suitable for dynamic CPU-based performance maintenance. Scheduled scaling only adjusts the capacity based on a pre-defined schedule. It doesn't react to real-time changes in CPU utilization. If the workload fluctuates unpredictably, scheduled scaling would be ineffective and could lead to either over-provisioning (wasting resources) or under-provisioning (impacting performance).

In summary, target tracking scaling is the most efficient and appropriate solution because it directly addresses the requirement of maintaining a specific CPU utilization target for optimal application performance. It is a managed service solution, so is preferred over building a solution using Lambda.

Supporting Documentation:

AWS Auto Scaling Target Tracking Scaling Policies:
https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-target-tracking.html
AWS Auto Scaling Simple Scaling Policies: https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-simple.html
AWS Auto Scaling Scheduled Scaling:
https://docs.aws.amazon.com/autoscaling/ec2/userguide/schedule_time.html

Question: 131 **Exam Heist**

A company is developing a file-sharing application that will use an Amazon S3 bucket for storage. The company wants to serve all the files through an Amazon CloudFront distribution. The company does not want the files to be accessible through direct navigation to the S3 URL.
What should a solutions architect do to meet these requirements?

A. Write individual policies for each S3 bucket to grant read permission for only CloudFront access.

B. Create an IAM user. Grant the user read permission to objects in the S3 bucket. Assign the user to CloudFront.

C. Write an S3 bucket policy that assigns the CloudFront distribution ID as the Principal and assigns the target S3 bucket as the Amazon Resource Name (ARN).

D. Create an origin access identity (OAI). Assign the OAI to the CloudFront distribution. Configure the S3 bucket

permissions so that only the OAI has read permission.

Answer: D

Explanation:

The correct answer is **D: Create an origin access identity (OAI). Assign the OAI to the CloudFront distribution. Configure the S3 bucket permissions so that only the OAI has read permission.**

Here's a detailed justification:

The primary goal is to restrict direct access to the S3 bucket while allowing CloudFront to serve the files. Origin Access Identity (OAI) is specifically designed for this purpose. An OAI is a virtual user identity that CloudFront uses to fetch private content from your S3 bucket.

Why OAI is the right approach: OAI provides a secure way to allow CloudFront to access the S3 bucket without making the content publicly accessible via the S3 URL. It enforces access control at the bucket level.

Steps involved:

1. **Create an OAI:** This creates a unique CloudFront user that identifies your distribution.
2. **Assign the OAI to CloudFront:** When configuring the CloudFront distribution, you specify the OAI. This tells CloudFront to use this identity when requesting content from the S3 origin.
3. **Configure S3 Bucket Permissions:** Modify the S3 bucket policy to grant the OAI read access to the objects. Importantly, remove any public read access. This ensures that only the CloudFront distribution (acting through the OAI) can retrieve objects from the bucket.

Why other options are incorrect:

A: Writing individual policies for each S3 bucket (if there were many) is not scalable. It is also not necessary because OAI already achieves the desired outcome by controlling access at the bucket level.
B: Using an IAM user is generally discouraged for service-to-service authentication in this scenario. OAI is specifically designed for CloudFront-S3 access control and provides a more streamlined and secure solution. Also, it's not clear how you would "assign" the IAM user to CloudFront, making the process conceptually flawed.
C: While you do use an S3 bucket policy, only assigning the CloudFront distribution ID as the Principal without the context of OAI doesn't prevent direct S3 access. The S3 bucket policy needs to grant the OAI permission, not just the CloudFront distribution ID. Also, the Distribution ID changes, thus the OAI is the stable element to use.

In essence, OAI enables a secure, dedicated channel for CloudFront to fetch content from S3 without exposing the bucket's contents to the public internet via direct S3 URL access.

Authoritative Links:

AWS Documentation on Restricting Access to Amazon S3 Content by Using an Origin Access Identity: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html

Question: 132 **Exam Heist**

A company's website provides users with downloadable historical performance reports. The website needs a solution that will scale to meet the company's website demands globally. The solution should be cost-effective, limit the provisioning of infrastructure resources, and provide the fastest possible response time.
Which combination should a solutions architect recommend to meet these requirements?

A. Amazon CloudFront and Amazon S3

B. AWS Lambda and Amazon DynamoDB

C. Application Load Balancer with Amazon EC2 Auto Scaling

D. Amazon Route 53 with internal Application Load Balancers

Answer: A

Explanation:

The correct answer is A: Amazon CloudFront and Amazon S3. Here's why:

Scalability and Global Reach: Amazon CloudFront is a content delivery network (CDN) that replicates your content across edge locations globally. This ensures low latency and fast download speeds for users regardless of their geographical location. https://aws.amazon.com/cloudfront/

Cost-Effectiveness and Limited Provisioning: S3 offers a highly scalable and cost-effective storage solution for the historical performance reports. You only pay for the storage you use. CloudFront further optimizes cost by caching content at edge locations, reducing the load on your origin (S3). Using S3 & CloudFront eliminates the need to manage servers or other compute infrastructure, reducing operational overhead.

Fastest Response Time: CloudFront caches the reports close to the users, minimizing the distance the data needs to travel. This directly translates to lower latency and faster download speeds compared to serving content directly from an origin server.

Now, let's examine why the other options are less suitable:

B. AWS Lambda and Amazon DynamoDB: While Lambda and DynamoDB are scalable services, they're better suited for dynamic data and application logic. Serving static files (historical reports) using Lambda would be less cost-effective and more complex than using S3 and CloudFront. DynamoDB is not optimized for storing and serving static files.

C. Application Load Balancer with Amazon EC2 Auto Scaling: This setup involves provisioning and managing EC2 instances, which is less cost-effective and requires more operational overhead than S3 and CloudFront. It doesn't inherently provide global distribution like CloudFront. While Auto Scaling provides scalability, it doesn't match the built-in global distribution and caching capabilities of CloudFront.

D. Amazon Route 53 with internal Application Load Balancers: This is a possible configuration for load balancing. However, this would distribute traffic between regional load balancers. It doesn't take into account that the files are static and suitable for caching closer to the users as CloudFront does. Furthermore, it requires provisioning infrastructure, failing to limit provisioning of infrastructure resources.

In summary, the combination of Amazon CloudFront and Amazon S3 is the most efficient and cost-effective solution for delivering downloadable historical performance reports globally with minimal infrastructure provisioning and optimal response times.

Question: 133 **Exam Heist**

A company runs an Oracle database on premises. As part of the company's migration to AWS, the company wants to upgrade the database to the most recent available version. The company also wants to set up disaster recovery (DR) for the database. The company needs to minimize the operational overhead for normal operations and DR setup. The company also needs to maintain access to the database's underlying operating system.
Which solution will meet these requirements?

A. Migrate the Oracle database to an Amazon EC2 instance. Set up database replication to a different AWS Region.

B. Migrate the Oracle database to Amazon RDS for Oracle. Activate Cross-Region automated backups to

replicate the snapshots to another AWS Region.

C. Migrate the Oracle database to Amazon RDS Custom for Oracle. Create a read replica for the database in another AWS Region.

D. Migrate the Oracle database to Amazon RDS for Oracle. Create a standby database in another Availability Zone.

Answer: C

Explanation:

The best solution is C, migrating the Oracle database to Amazon RDS Custom for Oracle and creating a read replica in another AWS Region. Here's why:

RDS Custom for Oracle: This service provides managed database capabilities while allowing access to the underlying operating system. This meets the requirement of needing OS access, which standard RDS does not provide.
Database Upgrade: RDS Custom allows for database upgrades while maintaining control over the process.
Disaster Recovery: Creating a read replica in another AWS Region directly addresses the disaster recovery requirement. The read replica can be promoted to a standalone, writeable instance in the event of a primary region failure, minimizing downtime.
Operational Overhead: RDS Custom automates many operational tasks, reducing the operational overhead compared to managing Oracle on EC2. Read replicas further contribute to reduced overhead by allowing for DR testing without impacting the primary instance.
Cost-Effectiveness: While not explicitly mentioned in the requirements, RDS Custom generally provides a good balance between control and cost-effectiveness compared to fully self-managed EC2 instances.

Option A (EC2) requires significant operational overhead, defeating the purpose of minimizing it. Option B (RDS for Oracle with Cross-Region Automated Backups) doesn't provide access to the underlying OS. Option D (RDS for Oracle with Standby Database in another AZ) addresses high availability, not disaster recovery across regions.

References:

Amazon RDS Custom for Oracle
Amazon RDS Read Replicas

Question: 134 **Exam Heist**

A company wants to move its application to a serverless solution. The serverless solution needs to analyze existing and new data by using SL. The company stores the data in an Amazon S3 bucket. The data requires encryption and must be replicated to a different AWS Region.
Which solution will meet these requirements with the LEAST operational overhead?

A. Create a new S3 bucket. Load the data into the new S3 bucket. Use S3 Cross-Region Replication (CRR) to replicate encrypted objects to an S3 bucket in another Region. Use server-side encryption with AWS KMS multi-Region kays (SSE-KMS). Use Amazon Athena to query the data.

B. Create a new S3 bucket. Load the data into the new S3 bucket. Use S3 Cross-Region Replication (CRR) to replicate encrypted objects to an S3 bucket in another Region. Use server-side encryption with AWS KMS multi-Region keys (SSE-KMS). Use Amazon RDS to query the data.

C. Load the data into the existing S3 bucket. Use S3 Cross-Region Replication (CRR) to replicate encrypted objects to an S3 bucket in another Region. Use server-side encryption with Amazon S3 managed encryption keys (SSE-S3). Use Amazon Athena to query the data.

D. Load the data into the existing S3 bucket. Use S3 Cross-Region Replication (CRR) to replicate encrypted objects to an S3 bucket in another Region. Use server-side encryption with Amazon S3 managed encryption keys (SSE-S3). Use Amazon RDS to query the data.

Answer: A

Explanation:

The correct answer is A. Here's why:

Requirements Alignment: The question mandates a serverless solution for data analysis using SQL, data encryption at rest, and cross-region replication for disaster recovery/availability.

S3 Cross-Region Replication (CRR): CRR directly addresses the requirement to replicate data to another AWS Region. This feature provides automatic, asynchronous copying of objects across S3 buckets in different AWS Regions.

Server-Side Encryption with AWS KMS Multi-Region Keys (SSE-KMS): SSE-KMS satisfies the encryption at rest requirement. Using KMS Multi-Region keys is crucial because if your primary region KMS becomes unavailable, your replicated bucket still has access to decrypt the data. With SSE-S3, you can't use Multi-Region keys.

Amazon Athena: Athena is a serverless query service that allows you to analyze data stored in S3 using standard SQL. This fulfills the SQL-based analysis requirement without the need to manage any infrastructure.

Why A is better than C: SSE-KMS Multi-Region keys provide more control and flexibility for encryption key management than SSE-S3 (Amazon S3 managed keys). If you have an issue in your primary region, it would take longer to re-encrypt the objects in your secondary region if you were using SSE-S3.

Why A is better than B and D: Amazon RDS is a database service requiring instance management, making the solution not serverless. Athena is the right tool for SQL queries on data in S3 in a serverless way.

In summary: Option A gives the required level of encryption in a serverless, secure, and easy-to-manage way.

Authoritative Links:

S3 Cross-Region Replication (CRR):
https://docs.aws.amazon.com/AmazonS3/latest/userguide/replication.html
SSE-KMS: https://docs.aws.amazon.com/kms/latest/developerguide/services-s3.html
Athena: https://aws.amazon.com/athena/
SSE-S3: https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingEncryption.html
KMS Multi-Region Keys: https://docs.aws.amazon.com/kms/latest/developerguide/multi-region-keys-overview.html

Question: 135 **Exam Heist**

A company runs workloads on AWS. The company needs to connect to a service from an external provider. The service is hosted in the provider's VPC. According to the company's security team, the connectivity must be private and must be restricted to the target service. The connection must be initiated only from the company's VPC. Which solution will mast these requirements?

A. Create a VPC peering connection between the company's VPC and the provider's VPC. Update the route table to connect to the target service.

B. Ask the provider to create a virtual private gateway in its VPC. Use AWS PrivateLink to connect to the target service.

C. Create a NAT gateway in a public subnet of the company's VPUpdate the route table to connect to the target service.

D. Ask the provider to create a VPC endpoint for the target service. Use AWS PrivateLink to connect to the target service.

Answer: D

Explanation:

The correct answer is D, asking the provider to create a VPC endpoint for the target service and using AWS PrivateLink to connect. Here's why:

Private Connectivity: PrivateLink ensures traffic doesn't traverse the public internet, fulfilling the requirement for private connectivity. It establishes a direct connection between the VPCs without exposing the traffic to the public internet.
Restricted Access: A VPC endpoint for the target service limits access to only that specific service, adhering to the restriction requirement. This reduces the attack surface and improves security.
Company-Initiated Connection: PrivateLink allows the company to initiate the connection from its VPC to the provider's service through the endpoint.
VPC Peering (Option A - Incorrect): VPC peering establishes a connection between two entire VPCs. It doesn't restrict access to a single service within the provider's VPC, violating the "restricted to the target service" requirement. Also, VPC peering requires overlapping CIDR block consideration.
Virtual Private Gateway (Option B - Incorrect): A Virtual Private Gateway (VGW) is primarily used for VPN connections or Direct Connect. It is less suitable for a private, service-specific connection like the one described. While AWS PrivateLink could theoretically leverage Direct Connect, it's overkill for the scenario.
NAT Gateway (Option C - Incorrect): A NAT gateway enables instances in a private subnet to connect to the internet or other AWS services, but it doesn't provide private connectivity to a specific service in another VPC. NAT Gateway uses public IP addresses and is not a secure, private option.
AWS PrivateLink Benefits: PrivateLink enhances security by eliminating the need for public IPs or internet gateways, simplifying network management and reducing the risk of data exposure.

In summary, option D provides the most secure and compliant solution by establishing a private, service-specific connection initiated from the company's VPC using PrivateLink and a VPC endpoint.

Further Research:

AWS PrivateLink: https://aws.amazon.com/privatelink/
VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/privatelink/vpc-endpoints.html

Question: 136 **Exam Heist**

A company is migrating its on-premises PostgreSQL database to Amazon Aurora PostgreSQL. The on-premises database must remain online and accessible during the migration. The Aurora database must remain synchronized with the on-premises database.
Which combination of actions must a solutions architect take to meet these requirements? (Choose two.)

A. Create an ongoing replication task.
B. Create a database backup of the on-premises database.
C. Create an AWS Database Migration Service (AWS DMS) replication server.
D. Convert the database schema by using the AWS Schema Conversion Tool (AWS SCT).
E. Create an Amazon EventBridge (Amazon CloudWatch Events) rule to monitor the database synchronization.

Answer: AC

Explanation:

The correct answer is AC. Here's why:

A. Create an ongoing replication task: To keep the Aurora PostgreSQL database synchronized with the on-premises PostgreSQL database while the on-premises database remains online, you need a continuous

replication solution. AWS Database Migration Service (DMS) achieves this through replication tasks. These tasks capture changes from the source database and apply them to the target database in near real-time, ensuring data consistency. This ongoing replication is crucial for a zero-downtime migration.

C. Create an AWS Database Migration Service (AWS DMS) replication server: DMS is the AWS service specifically designed for database migrations. A replication server is the core component of DMS, responsible for reading data from the source database (on-premises PostgreSQL), transforming the data if needed, and loading the data into the target database (Aurora PostgreSQL). Without a DMS replication server, you cannot perform the ongoing replication task needed for synchronization.

AWS Database Migration Service Documentation
AWS DMS Replication Task

Here's why the other options are not suitable:

B. Create a database backup of the on-premises database: A database backup is useful for the initial data load into Aurora PostgreSQL, but it doesn't address the ongoing synchronization requirement. A backup represents a point-in-time snapshot, and any changes after the backup would not be reflected in the Aurora database.

D. Convert the database schema by using the AWS Schema Conversion Tool (AWS SCT): While AWS SCT is often used in database migrations to convert the database schema from one engine to another (e.g., Oracle to PostgreSQL), it is not always necessary for PostgreSQL-to-PostgreSQL migrations unless there are compatibility issues or optimizations needed. Importantly, schema conversion doesn't handle ongoing data synchronization.

E. Create an Amazon EventBridge (Amazon CloudWatch Events) rule to monitor the database synchronization: EventBridge rules can monitor events and trigger actions. While you could potentially use EventBridge to monitor DMS metrics related to replication status, it doesn't actually perform the replication. Furthermore, CloudWatch metrics can be used more directly to monitor DMS tasks rather than involving EventBridge. Therefore, EventBridge isn't a primary requirement for the initial synchronization process itself.

Question: 137 **Exam Heist**

A company uses AWS Organizations to create dedicated AWS accounts for each business unit to manage each business unit's account independently upon request. The root email recipient missed a notification that was sent to the root user email address of one account. The company wants to ensure that all future notifications are not missed. Future notifications must be limited to account administrators.
Which solution will meet these requirements?

A. Configure the company’s email server to forward notification email messages that are sent to the AWS account root user email address to all users in the organization.

B. Configure all AWS account root user email addresses as distribution lists that go to a few administrators who can respond to alerts. Configure AWS account alternate contacts in the AWS Organizations console or programmatically.

C. Configure all AWS account root user email messages to be sent to one administrator who is responsible for monitoring alerts and forwarding those alerts to the appropriate groups.

D. Configure all existing AWS accounts and all newly created accounts to use the same root user email address. Configure AWS account alternate contacts in the AWS Organizations console or programmatically.

Answer: B

Explanation:

The correct answer is **B**. Here's why:

Option B addresses the core requirement of ensuring that notifications sent to the root user email address are not missed and are limited to account administrators. Configuring the root user email address as a distribution list ensures that multiple administrators receive the notifications, reducing the risk of a single point of failure (like one administrator missing the email).

Using distribution lists linked to the root user email address is a manageable approach that aligns with AWS best practices. Importantly, it prevents reliance on a single individual. AWS suggests using distribution lists, and not individual email addresses, for critical administrative roles, like the root user.

The utilization of AWS account alternate contacts in the AWS Organizations console provides an additional layer of notification delivery. Alternate contacts can be set for billing, technical, and security purposes, allowing for different administrators to receive different types of notifications relevant to their roles. By configuring alternate contacts, notifications intended for these specific areas are less likely to be overlooked and are routed to the appropriate personnel.

Here's why the other options are less suitable:

A: Forwarding all emails from all root users to all users in the organization is overly broad and creates unnecessary noise, violating the requirement to limit notifications to administrators.
C: Relying on a single administrator to forward alerts introduces a single point of failure and increases the likelihood of delays or missed notifications.
D: Using the same root user email address for all accounts creates a security risk and management complexity, as a compromise of that email address would affect all accounts.

Supporting Documentation:

AWS Organizations Best Practices
AWS Account Management

Question: 138 **Exam Heist**

A company runs its ecommerce application on AWS. Every new order is published as a massage in a RabbitMQ queue that runs on an Amazon EC2 instance in a single Availability Zone. These messages are processed by a different application that runs on a separate EC2 instance. This application stores the details in a PostgreSQL database on another EC2 instance. All the EC2 instances are in the same Availability Zone.
The company needs to redesign its architecture to provide the highest availability with the least operational overhead.
What should a solutions architect do to meet these requirements?

A. Migrate the queue to a redundant pair (active/standby) of RabbitMQ instances on Amazon MQ. Create a Multi-AZ Auto Scaling group for EC2 instances that host the application. Create another Multi-AZ Auto Scaling group for EC2 instances that host the PostgreSQL database.

B. Migrate the queue to a redundant pair (active/standby) of RabbitMQ instances on Amazon MQ. Create a Multi-AZ Auto Scaling group for EC2 instances that host the application. Migrate the database to run on a Multi-AZ deployment of Amazon RDS for PostgreSQL.

C. Create a Multi-AZ Auto Scaling group for EC2 instances that host the RabbitMQ queue. Create another Multi-AZ Auto Scaling group for EC2 instances that host the application. Migrate the database to run on a Multi-AZ deployment of Amazon RDS for PostgreSQL.

D. Create a Multi-AZ Auto Scaling group for EC2 instances that host the RabbitMQ queue. Create another Multi-AZ Auto Scaling group for EC2 instances that host the application. Create a third Multi-AZ Auto Scaling group for EC2 instances that host the PostgreSQL database

Answer: B

Explanation:

The goal is to achieve high availability with minimal operational overhead for an e-commerce application

currently running on EC2 instances within a single Availability Zone. Option B provides the best approach.

Migrating the RabbitMQ queue to a redundant pair on Amazon MQ (active/standby) eliminates the single point of failure associated with running RabbitMQ on a single EC2 instance. Amazon MQ manages the underlying infrastructure, reducing operational overhead.

Creating a Multi-AZ Auto Scaling group for the application EC2 instances ensures that the application remains available even if one Availability Zone fails. The Auto Scaling group will automatically launch new instances in a healthy Availability Zone.

Migrating the PostgreSQL database to a Multi-AZ deployment of Amazon RDS for PostgreSQL is crucial for database availability. RDS manages the replication and failover process, significantly reducing operational overhead compared to managing PostgreSQL on EC2. Multi-AZ RDS provides synchronous replication to a standby instance in a different AZ, ensuring data durability and rapid failover.

Options A, C, and D have drawbacks. Running the database or RabbitMQ on EC2 instances within Auto Scaling groups, even across multiple AZs, requires managing replication, failover, and patching, increasing operational overhead. Amazon MQ and RDS offer managed services with built-in high availability, simplifying management.

Therefore, Option B provides the optimal solution by leveraging managed services (Amazon MQ and RDS) to achieve high availability with the least operational burden.

References:

Amazon MQ: https://aws.amazon.com/mq/
Amazon RDS Multi-AZ: https://aws.amazon.com/rds/features/multi-az/
Auto Scaling: https://aws.amazon.com/autoscaling/

Question: 139 **Exam Heist**

A reporting team receives files each day in an Amazon S3 bucket. The reporting team manually reviews and copies the files from this initial S3 bucket to an analysis S3 bucket each day at the same time to use with Amazon QuickSight. Additional teams are starting to send more files in larger sizes to the initial S3 bucket.
The reporting team wants to move the files automatically analysis S3 bucket as the files enter the initial S3 bucket. The reporting team also wants to use AWS Lambda functions to run pattern-matching code on the copied data. In addition, the reporting team wants to send the data files to a pipeline in Amazon SageMaker Pipelines.
What should a solutions architect do to meet these requirements with the LEAST operational overhead?

A. Create a Lambda function to copy the files to the analysis S3 bucket. Create an S3 event notification for the analysis S3 bucket. Configure Lambda and SageMaker Pipelines as destinations of the event notification. Configure s3:ObjectCreated:Put as the event type.

B. Create a Lambda function to copy the files to the analysis S3 bucket. Configure the analysis S3 bucket to send event notifications to Amazon EventBridge (Amazon CloudWatch Events). Configure an ObjectCreated rule in EventBridge (CloudWatch Events). Configure Lambda and SageMaker Pipelines as targets for the rule.

C. Configure S3 replication between the S3 buckets. Create an S3 event notification for the analysis S3 bucket. Configure Lambda and SageMaker Pipelines as destinations of the event notification. Configure s3:ObjectCreated:Put as the event type.

D. Configure S3 replication between the S3 buckets. Configure the analysis S3 bucket to send event notifications to Amazon EventBridge (Amazon CloudWatch Events). Configure an ObjectCreated rule in EventBridge (CloudWatch Events). Configure Lambda and SageMaker Pipelines as targets for the rule.

Answer: D

Explanation:

The correct answer is **D**. Here's a detailed justification:

S3 Replication: S3 Replication is the most efficient way to automatically copy objects between S3 buckets. It's a managed service, reducing operational overhead compared to writing and managing a Lambda function for copying. It replicates data as soon as it's written to the source bucket. https://docs.aws.amazon.com/AmazonS3/latest/userguide/replication.html

EventBridge for Fan-Out: Amazon EventBridge (formerly CloudWatch Events) provides a scalable and decoupled way to route events to multiple targets. This addresses the requirement to trigger both a Lambda function for pattern matching and the SageMaker pipeline. S3 can send event notifications to EventBridge upon object creation. https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html

ObjectCreated Event: Configuring an ObjectCreated rule in EventBridge triggers the rule when a new object is created in the analysis S3 bucket.

Targets: By configuring the Lambda function and SageMaker Pipelines as targets for the EventBridge rule, both are automatically invoked when a new object arrives in the analysis bucket.

Why other options are not optimal:

A & B: While a Lambda function could copy files, it adds operational overhead. S3 Replication handles the copying automatically. Additionally, option B is incorrect because the question asks to run Lambda functions and SageMaker pipelines after the files are copied to the analysis bucket.
C: S3 event notifications sent directly to multiple destinations can be less scalable and harder to manage than using EventBridge. EventBridge provides more robust event routing and filtering capabilities.

In summary, option D combines the efficiency of S3 Replication with the event-driven architecture of EventBridge for a scalable, low-overhead solution to meet all the requirements.

Question: 140 **Exam Heist**

A solutions architect needs to help a company optimize the cost of running an application on AWS. The application will use Amazon EC2 instances, AWS Fargate, and AWS Lambda for compute within the architecture.
The EC2 instances will run the data ingestion layer of the application. EC2 usage will be sporadic and unpredictable. Workloads that run on EC2 instances can be interrupted at any time. The application front end will run on Fargate, and Lambda will serve the API layer. The front-end utilization and API layer utilization will be predictable over the course of the next year.
Which combination of purchasing options will provide the MOST cost-effective solution for hosting this application? (Choose two.)

A. Use Spot Instances for the data ingestion layer
B. Use On-Demand Instances for the data ingestion layer
C. Purchase a 1-year Compute Savings Plan for the front end and API layer.
D. Purchase 1-year All Upfront Reserved instances for the data ingestion layer.
E. Purchase a 1-year EC2 instance Savings Plan for the front end and API layer.

Answer: AC

Explanation:

The correct answer is A and C.

Here's why:

A. Use Spot Instances for the data ingestion layer: The data ingestion layer's EC2 usage is sporadic and unpredictable, and workloads can be interrupted. Spot Instances are ideal for these scenarios. Spot Instances leverage spare EC2 capacity and are offered at significantly reduced prices compared to On-Demand

Instances. The application's tolerance for interruption aligns perfectly with the characteristics of Spot Instances, making them a cost-effective choice for this layer.https://aws.amazon.com/ec2/spot/

C. Purchase a 1-year Compute Savings Plan for the front end and API layer: The application front end runs on Fargate, and the API layer runs on Lambda. Both have predictable utilization over the next year. Savings Plans offer significant discounts in exchange for a commitment to a consistent amount of compute usage (measured in dollars per hour). A Compute Savings Plan provides the most flexibility, since it applies to EC2, Fargate, and Lambda, whereas the EC2 Instance Savings Plan applies only to EC2.https://aws.amazon.com/savingsplans/compute-savings-plans/

Here's why the other options are not as suitable:

B. Use On-Demand Instances for the data ingestion layer: On-Demand Instances are the most expensive option and should be avoided when workloads can tolerate interruptions, as in the data ingestion layer.

D. Purchase 1-year All Upfront Reserved instances for the data ingestion layer: Given the unpredictable nature of EC2 usage in the data ingestion layer, committing to Reserved Instances is risky and may lead to paying for unused capacity. Also Reserved Instances offer the least amount of flexibility

E. Purchase a 1-year EC2 instance Savings Plan for the front end and API layer: The API layer is on Lambda, so EC2 Instance Savings Plan can't be used. It is a better option to choose compute savings plan as it can apply to all different kinds of compute services and provides more flexibility.

Question: 141 **Exam Heist**

A company runs a web-based portal that provides users with global breaking news, local alerts, and weather updates. The portal delivers each user a personalized view by using mixture of static and dynamic content. Content is served over HTTPS through an API server running on an Amazon EC2 instance behind an Application Load Balancer (ALB). The company wants the portal to provide this content to its users across the world as quickly as possible.
How should a solutions architect design the application to ensure the LEAST amount of latency for all users?

A. Deploy the application stack in a single AWS Region. Use Amazon CloudFront to serve all static and dynamic content by specifying the ALB as an origin.

B. Deploy the application stack in two AWS Regions. Use an Amazon Route 53 latency routing policy to serve all content from the ALB in the closest Region.

C. Deploy the application stack in a single AWS Region. Use Amazon CloudFront to serve the static content. Serve the dynamic content directly from the ALB.

D. Deploy the application stack in two AWS Regions. Use an Amazon Route 53 geolocation routing policy to serve all content from the ALB in the closest Region.

Answer: A

Explanation:

The optimal solution for minimizing latency for global users accessing a personalized news portal involves leveraging Amazon CloudFront to cache and deliver both static and dynamic content.

Option A, deploying the application stack in a single AWS Region and using CloudFront with the ALB as an origin, is the most effective approach. CloudFront is a content delivery network (CDN) that caches content at edge locations globally. When a user requests content, CloudFront serves it from the nearest edge location, significantly reducing latency. Specifying the ALB as the origin allows CloudFront to retrieve content that isn't already cached, including dynamically generated personalized content.

Option B, deploying in multiple regions and using Route 53 latency routing, is less efficient. While Route 53 latency routing directs users to the closest region, it doesn't cache content at edge locations. This means

users still experience latency when accessing dynamic content, as requests must travel to the regional ALB. This approach also adds operational complexity and cost for maintaining duplicate infrastructure across multiple regions.

Option C, using CloudFront only for static content, doesn't address the latency issue for dynamic content. Since the portal provides personalized views, the dynamic content plays a crucial role in user experience, and serving it directly from the ALB negates the benefits of CloudFront for a significant portion of the content.

Option D, using Route 53 geolocation routing, while beneficial, doesn't directly address latency as effectively as CloudFront. Geolocation routing directs users based on their geographic location, but it doesn't cache content closer to users like CloudFront does. It's also less flexible than latency routing in some scenarios where network conditions vary.

CloudFront's ability to cache dynamic content, coupled with its global network of edge locations, makes it the most effective solution for minimizing latency and delivering a fast, responsive user experience for a web portal with personalized content. The single region approach combined with CloudFront simplifies the architecture and reduces operational overhead.

Authoritative Links:

Amazon CloudFront: https://aws.amazon.com/cloudfront/
Amazon Route 53: https://aws.amazon.com/route53/

Question: 142 **Exam Heist**

A gaming company is designing a highly available architecture. The application runs on a modified Linux kernel and supports only UDP-based traffic. The company needs the front-end tier to provide the best possible user experience. That tier must have low latency, route traffic to the nearest edge location, and provide static IP addresses for entry into the application endpoints.
What should a solutions architect do to meet these requirements?

A. Configure Amazon Route 53 to forward requests to an Application Load Balancer. Use AWS Lambda for the application in AWS Application Auto Scaling.

B. Configure Amazon CloudFront to forward requests to a Network Load Balancer. Use AWS Lambda for the application in an AWS Application Auto Scaling group.

C. Configure AWS Global Accelerator to forward requests to a Network Load Balancer. Use Amazon EC2 instances for the application in an EC2 Auto Scaling group.

D. Configure Amazon API Gateway to forward requests to an Application Load Balancer. Use Amazon EC2 instances for the application in an EC2 Auto Scaling group.

Answer: C

Explanation:

The correct answer is C. Here's why:

AWS Global Accelerator is designed to route traffic to the optimal AWS endpoint based on user location, providing low latency and improved user experience, which directly addresses the requirement for routing traffic to the nearest edge location. Crucially, it also provides static IP addresses for applications.
https://aws.amazon.com/global-accelerator/
Network Load Balancer (NLB) is required because the application supports only UDP-based traffic. Application Load Balancers (ALB) only support HTTP/HTTPS traffic and are therefore not a viable option. NLB is designed for high throughput and low latency, ideal for UDP applications.
https://aws.amazon.com/elasticloadbalancing/network-load-balancer/
Amazon EC2 instances with **EC2 Auto Scaling group** is the best compute option. AWS Lambda does not offer full control over the underlying Linux kernel as required by the scenario. EC2 instances allows full control

over OS, is necessary because the application runs on a modified Linux kernel. The Auto Scaling group ensures high availability by automatically adjusting the number of EC2 instances based on demand. https://aws.amazon.com/ec2/autoscaling/

Option A is incorrect because Application Load Balancer does not support UDP traffic. Option B is wrong because CloudFront is primarily designed for caching static content. While it can accelerate dynamic content delivery, it is not optimized for UDP-based traffic and requires a distribution for forwarding traffic. Option D is incorrect because API Gateway is designed for API management and request routing, not high-throughput, low-latency UDP traffic and ALB does not support UDP traffic.

Question: 143 **Exam Heist**

A company wants to migrate its existing on-premises monolithic application to AWS. The company wants to keep as much of the front-end code and the backend code as possible. However, the company wants to break the application into smaller applications. A different team will manage each application. The company needs a highly scalable solution that minimizes operational overhead.
Which solution will meet these requirements?

A. Host the application on AWS Lambda. Integrate the application with Amazon API Gateway.

B. Host the application with AWS Amplify. Connect the application to an Amazon API Gateway API that is integrated with AWS Lambda.

C. Host the application on Amazon EC2 instances. Set up an Application Load Balancer with EC2 instances in an Auto Scaling group as targets.

D. Host the application on Amazon Elastic Container Service (Amazon ECS). Set up an Application Load Balancer with Amazon ECS as the target.

Answer: D

Explanation:

The correct answer is **D. Host the application on Amazon Elastic Container Service (Amazon ECS). Set up an Application Load Balancer with Amazon ECS as the target.**

Here's a detailed justification:

The problem statement emphasizes breaking down a monolithic application into smaller, independently manageable applications while minimizing operational overhead and ensuring high scalability.

Amazon ECS is a fully managed container orchestration service. Containerization allows you to package each smaller application (resulting from breaking down the monolith) into a separate container. These containers can then be deployed and managed independently by different teams, fulfilling the requirement of decentralized management. https://aws.amazon.com/ecs/

Application Load Balancer (ALB) provides intelligent routing of traffic to different containerized applications running on ECS. This aligns with the requirement of exposing the split-up applications via a load-balanced endpoint. ALB offers advanced features like content-based routing, enabling fine-grained control over how traffic is distributed. https://aws.amazon.com/elasticloadbalancing/application-load-balancer/

ECS, being a managed service, significantly reduces the operational overhead compared to managing EC2 instances directly (as in option C). You don't have to worry about patching, scaling, or maintaining the underlying infrastructure. ECS also handles the complexities of container orchestration, enabling you to focus on application development and deployment.

ECS integrates well with other AWS services, such as CloudWatch for monitoring and AWS Auto Scaling for automatically adjusting the number of containers based on traffic. This ensures high availability and

scalability.

Why other options are less suitable:

A. AWS Lambda: Lambda is suitable for event-driven, stateless functions, but not ideal for hosting entire applications (even smaller, broken-down ones) that require more persistent resources or have more complex architectures. It might be part of a larger solution, but not the core hosting platform as described in this option.

B. AWS Amplify: Amplify is more focused on front-end development and mobile app backends. While it can connect to back-end services, it's not the primary service to host the backend application components derived from the monolithic breakdown. It doesn't directly address the requirement of breaking down and managing the backend.

C. Amazon EC2 instances with Auto Scaling Group: While this option provides scalability, it involves significantly more operational overhead compared to ECS. You would need to manage the EC2 instances, including patching, security updates, and scaling the infrastructure. ECS abstracts away much of this complexity.

In conclusion, ECS with ALB is the most suitable solution because it allows for the decomposition of the monolithic application into independently deployable and manageable containerized applications, while also minimizing operational overhead through a managed service and providing scalability through ALB and ECS features.

Question: 144 **Exam Heist**

A company recently started using Amazon Aurora as the data store for its global ecommerce application. When large reports are run, developers report that the ecommerce application is performing poorly. After reviewing metrics in Amazon CloudWatch, a solutions architect finds that the ReadIOPS and CPUUtilizalion metrics are spiking when monthly reports run.
What is the MOST cost-effective solution?

A. Migrate the monthly reporting to Amazon Redshift.
B. Migrate the monthly reporting to an Aurora Replica.
C. Migrate the Aurora database to a larger instance class.
D. Increase the Provisioned IOPS on the Aurora instance.

Answer: B

Explanation:

The correct answer is B: Migrate the monthly reporting to an Aurora Replica. Here's why:

The problem is that running large monthly reports on the primary Aurora instance causes performance degradation for the ecommerce application due to increased ReadIOPS and CPU utilization. This means the reporting queries are impacting the operational workload.

Option A, migrating the monthly reporting to Amazon Redshift, would certainly solve the performance issue by isolating the reporting workload. However, it is not the most cost-effective solution. Redshift requires data warehousing infrastructure and ETL (Extract, Transform, Load) processes to move data from Aurora to Redshift. This incurs extra cost in terms of infrastructure, software, and development effort.

Option B, migrating the monthly reporting to an Aurora Replica, provides a read-only copy of the data that can be used for reporting without impacting the primary database. Aurora Replicas are designed for read scaling and offloading read workloads from the primary instance. This minimizes the impact on the ecommerce

application's performance. The cost of an Aurora Replica is generally lower than migrating to a separate data warehouse like Redshift.

Option C, migrating the Aurora database to a larger instance class, would increase the capacity of the primary database. While this might improve performance for both the application and the reporting queries, it addresses the symptom rather than the root cause. This is a more expensive solution than using an Aurora Replica because it involves upgrading the primary database instance, which is used for all operations, not just reports.

Option D, increasing the Provisioned IOPS on the Aurora instance, might help with the ReadIOPS spikes, but it won't alleviate the CPU utilization. Also, Provisioned IOPS incurs additional costs. This also tackles the symptom rather than isolating the problem, making it less effective and potentially more expensive than using an Aurora Replica.

Therefore, using an Aurora Replica is the most cost-effective solution as it leverages Aurora's built-in read scaling capabilities to isolate the reporting workload from the primary database, addressing the performance issue directly without requiring expensive infrastructure changes or complex ETL processes.

For further research, consult the AWS documentation on Amazon Aurora Read Replicas: https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Managing.ReadReplicas.html and Aurora features: https://aws.amazon.com/rds/aurora/features/

Question: 145 **Exam Heist**

A company hosts a website analytics application on a single Amazon EC2 On-Demand Instance. The analytics software is written in PHP and uses a MySQL database. The analytics software, the web server that provides PHP, and the database server are all hosted on the EC2 instance. The application is showing signs of performance degradation during busy times and is presenting 5xx errors. The company needs to make the application scale seamlessly.
Which solution will meet these requirements MOST cost-effectively?

A. Migrate the database to an Amazon RDS for MySQL DB instance. Create an AMI of the web application. Use the AMI to launch a second EC2 On-Demand Instance. Use an Application Load Balancer to distribute the load to each EC2 instance.

B. Migrate the database to an Amazon RDS for MySQL DB instance. Create an AMI of the web application. Use the AMI to launch a second EC2 On-Demand Instance. Use Amazon Route 53 weighted routing to distribute the load across the two EC2 instances.

C. Migrate the database to an Amazon Aurora MySQL DB instance. Create an AWS Lambda function to stop the EC2 instance and change the instance type. Create an Amazon CloudWatch alarm to invoke the Lambda function when CPU utilization surpasses 75%.

D. Migrate the database to an Amazon Aurora MySQL DB instance. Create an AMI of the web application. Apply the AMI to a launch template. Create an Auto Scaling group with the launch template Configure the launch template to use a Spot Fleet. Attach an Application Load Balancer to the Auto Scaling group.

Answer: D

Explanation:

The correct answer is D because it provides a scalable and cost-effective solution to address the application's performance issues. Let's break down why:

Database Migration to Aurora MySQL: Migrating the MySQL database to Amazon Aurora MySQL (a managed, MySQL-compatible relational database engine) addresses a major performance bottleneck. Aurora offers improved performance and scalability compared to running MySQL on a single EC2 instance. This offloads database operations from the EC2 instance, freeing up resources for the web application.

AMI and Launch Template: Creating an Amazon Machine Image (AMI) of the web application ensures a consistent and repeatable deployment. A launch template is then created based on the AMI, defining the configuration for new EC2 instances. This makes sure that the application deployment is consistent across all instances.

Auto Scaling Group (ASG): An Auto Scaling group automatically adjusts the number of EC2 instances based on demand. It ensures the application scales seamlessly during peak loads and reduces costs during off-peak hours by terminating unnecessary instances.

Spot Fleet Integration: Using Spot Instances in the ASG provides significant cost savings. Spot Instances are unused EC2 capacity offered at discounted prices. While Spot Instances can be interrupted, the ASG automatically replaces terminated Spot Instances with new ones, maintaining the desired capacity.

Application Load Balancer (ALB): The Application Load Balancer distributes incoming traffic across the EC2 instances in the ASG. This ensures high availability and even load distribution, preventing any single instance from becoming overwhelmed.

Option A is partially correct but doesn't offer cost optimization through spot instances or auto-scaling capabilities. Option B has similar limitations with added complexity and cost by using weighted routing in Route53, which isn't as dynamically responsive as ALB and ASG. Option C focuses on scaling up (vertical scaling) a single EC2 instance, instead of providing a scalable architecture based on multiple instances (horizontal scaling). Option C, while invoking a lambda function based on CPU utilization, lacks the ability to respond to traffic automatically and cost-effectively like the Auto Scaling Group within option D.

Authoritative Links:

Amazon Aurora: https://aws.amazon.com/rds/aurora/
Amazon EC2 Auto Scaling: https://aws.amazon.com/autoscaling/
Amazon EC2 Spot Instances: https://aws.amazon.com/ec2/spot/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/

Question: 146 **Exam Heist**

A company runs a stateless web application in production on a group of Amazon EC2 On-Demand Instances behind an Application Load Balancer. The application experiences heavy usage during an 8-hour period each business day. Application usage is moderate and steady overnight. Application usage is low during weekends.
The company wants to minimize its EC2 costs without affecting the availability of the application.
Which solution will meet these requirements?

A. Use Spot Instances for the entire workload.

B. Use Reserved Instances for the baseline level of usage. Use Spot instances for any additional capacity that the application needs.

C. Use On-Demand Instances for the baseline level of usage. Use Spot Instances for any additional capacity that the application needs.

D. Use Dedicated Instances for the baseline level of usage. Use On-Demand Instances for any additional capacity that the application needs.

Answer: B

Explanation:

The optimal solution leverages the predictable usage patterns to minimize costs while maintaining availability. Option B (Reserved Instances for baseline and Spot Instances for peak) is the most cost-effective and robust approach.

Here's why:

Reserved Instances (RIs): RIs offer significant discounts (up to 75%) compared to On-Demand Instances in exchange for a commitment to a specific instance type and region for a period (usually 1 or 3 years). The moderate and steady overnight usage, as well as low weekend usage, defines a "baseline" capacity that is predictably required. RIs are ideally suited for this predictable baseline, providing guaranteed capacity at a reduced cost.

Spot Instances: Spot Instances offer unused EC2 capacity at steep discounts (up to 90% off On-Demand prices). However, Spot Instances can be terminated with a two-minute warning if the Spot price exceeds the bid price. The heavy usage during the 8-hour period represents a peak demand above the baseline. Spot Instances are perfect for this additional, non-critical capacity. The stateless nature of the web application makes it resilient to Spot Instance interruptions. The Application Load Balancer can simply redirect traffic to the remaining instances (Reserved or Spot) if a Spot Instance is terminated.

Why other options are less suitable:

Option A (Spot Instances only): Relying solely on Spot Instances is risky for production workloads requiring high availability. During periods of high demand, Spot prices can spike or capacity can become limited, leading to terminations and potential service disruptions.
Option C (On-Demand + Spot): Using On-Demand Instances for the baseline is more expensive than using Reserved Instances for a predictable usage pattern.
Option D (Dedicated + On-Demand): Dedicated Instances are expensive and primarily used for compliance or licensing reasons. They are not cost-effective for general web application hosting and are not necessary for this scenario. The same cost consideration exists for On-Demand in the "spike" described in option C.

In summary, Reserved Instances secure a cost-effective baseline capacity, while Spot Instances provide an affordable means to handle peak demand, all while maintaining high availability thanks to the Application Load Balancer and stateless architecture.

Authoritative Links:

Amazon EC2 Reserved Instances: https://aws.amazon.com/ec2/reserved-instances/
Amazon EC2 Spot Instances: https://aws.amazon.com/ec2/spot/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/

Question: 147 **Exam Heist**

A company needs to retain application log files for a critical application for 10 years. The application team regularly accesses logs from the past month for troubleshooting, but logs older than 1 month are rarely accessed. The application generates more than 10 TB of logs per month.
Which storage option meets these requirements MOST cost-effectively?

A. Store the logs in Amazon S3. Use AWS Backup to move logs more than 1 month old to S3 Glacier Deep Archive.

B. Store the logs in Amazon S3. Use S3 Lifecycle policies to move logs more than 1 month old to S3 Glacier Deep Archive.

C. Store the logs in Amazon CloudWatch Logs. Use AWS Backup to move logs more than 1 month old to S3 Glacier Deep Archive.

D. Store the logs in Amazon CloudWatch Logs. Use Amazon S3 Lifecycle policies to move logs more than 1 month old to S3 Glacier Deep Archive.

Answer: B

Explanation:

The correct answer is B. Here's a detailed justification:

The requirement is to store application logs for 10 years cost-effectively, with frequent access to the past month's logs and infrequent access to older logs. This scenario is a perfect fit for tiered storage using Amazon S3 and S3 Glacier Deep Archive.

Option B is most cost-effective for the following reasons:

1. **Amazon S3 for Recent Logs:** Storing logs in Amazon S3 initially allows for quick and easy access to the most recent (1-month old) logs, which are frequently needed for troubleshooting. S3 offers high availability and durability, making it suitable for critical application logs.
2. **S3 Lifecycle Policies for Archiving:** S3 Lifecycle policies automate the transition of older logs to S3 Glacier Deep Archive. This is crucial for cost optimization because Glacier Deep Archive is the cheapest storage class, designed for long-term data archiving where retrieval times of several hours are acceptable.
3. **Cost Efficiency:** By moving older, infrequently accessed logs to Glacier Deep Archive, the overall storage cost is significantly reduced compared to keeping all logs in standard S3 storage.
4. **Automation:** S3 Lifecycle policies automatically manage the data transition based on defined rules (age of the logs), eliminating manual intervention and potential errors.

Options A, C, and D are less optimal:

Option A (AWS Backup with S3): AWS Backup is designed for data protection and disaster recovery, not for cost-effective long-term archival of log data. Using AWS Backup to move logs would be significantly more expensive than S3 Lifecycle policies because Backup is charged based on protected storage and data transfer, as well as requiring restore operation costs when needed.
Option C (CloudWatch Logs with AWS Backup): CloudWatch Logs is suitable for real-time monitoring and analysis of log data. While CloudWatch Logs can archive data to S3, it's generally more expensive for long-term storage than directly storing and managing logs in S3. Additionally, using AWS Backup from CloudWatch Logs to Glacier Deep Archive incurs unnecessary costs associated with using Backup services where an S3 Lifecycle policy is sufficient.
Option D (CloudWatch Logs with S3 Lifecycle): Similar to option C, storing logs directly in CloudWatch Logs before moving them to S3 introduces an unnecessary cost layer. It's more efficient and cheaper to store the logs directly into S3 to begin with.

In summary, Option B leverages the cost-effective tiered storage approach provided by Amazon S3 and S3 Glacier Deep Archive, utilizing S3 Lifecycle policies for automated data management and archiving.

Supporting Documentation:

Amazon S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
S3 Lifecycle Policies: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-examples.html
S3 Glacier Deep Archive: https://aws.amazon.com/glacier/deep-archive/

Question: 148 **Exam Heist**

A company has a data ingestion workflow that includes the following components:
An Amazon Simple Notification Service (Amazon SNS) topic that receives notifications about new data deliveries
An AWS Lambda function that processes and stores the data
The ingestion workflow occasionally fails because of network connectivity issues. When failure occurs, the corresponding data is not ingested unless the company manually reruns the job.
What should a solutions architect do to ensure that all notifications are eventually processed?

A. Configure the Lambda function for deployment across multiple Availability Zones.

B. Modify the Lambda function's configuration to increase the CPU and memory allocations for the function.

C. Configure the SNS topic’s retry strategy to increase both the number of retries and the wait time between retries.

D. Configure an Amazon Simple Queue Service (Amazon SQS) queue as the on-failure destination. Modify the Lambda function to process messages in the queue.

Answer: D

Explanation:

The correct answer is D because it implements a robust solution for handling intermittent failures in the data ingestion workflow using a dead-letter queue (DLQ) mechanism.

Here's a breakdown of why:

Problem: The current architecture relies on direct invocation of the Lambda function by SNS. Network issues cause invocations to fail, leading to data loss unless manually retried. SNS doesn't natively retry failed deliveries to Lambda indefinitely, leading to message loss.

Solution with SQS DLQ: Configuring an SQS queue as a DLQ for failed SNS notifications ensures that messages that cannot be delivered to the Lambda function due to temporary network issues are preserved. This provides a reliable mechanism for capturing those failed notifications. The Lambda function can then be modified to consume messages from this SQS queue. This separates the initial ingestion from the processing of failed messages, providing a safety net.

Why other options are incorrect:

A: Configuring the Lambda function for deployment across multiple Availability Zones: Lambda is already designed to run across multiple AZs for high availability. While helpful for general resilience, this doesn't address the specific issue of failed SNS deliveries due to transient network problems. The underlying issue is that SNS gives up on delivery when it cannot successfully reach the function, regardless of the AZ.

B: Modify the Lambda function's configuration to increase the CPU and memory allocations for the function: Increasing resources might help with function performance if it was timing out, but it doesn't address the core problem of failed SNS deliveries due to network connectivity issues.

C: Configure the SNS topic’s retry strategy to increase both the number of retries and the wait time between retries: While this sounds plausible, SNS retry policies have limitations. SNS does have retry policies (delivery policies), however SNS only retries for a specific number of attempts before discarding the message. This still leads to message loss if the network issue persists beyond those retries. SQS as a DLQ is a better approach.

Benefits of SQS DLQ:

Durability: SQS provides durable message storage.
Reliability: Messages are guaranteed to be delivered at least once (and possibly more than once).
Decoupling: Decouples the initial data ingestion from the processing of failed messages, allowing for independent scaling and management.
Observability: Provides visibility into failed messages, allowing for root cause analysis and debugging.

SQS as a DLQ: Amazon SQS dead-letter queues are queues that source queues (or subscriptions) can target for messages that cannot be processed successfully. A common use is for failure cases in message consumption using services such as SNS.

Relevant Documentation:

Using AWS Lambda with Amazon SQS

Amazon SQS Dead-Letter Queues
SNS Message Delivery Retries

Therefore, configuring an SQS queue as the on-failure destination provides the most robust and reliable solution for ensuring that all notifications are eventually processed, even in the presence of intermittent network issues.

Question: 149 **Exam Heist**

A company has a service that produces event data. The company wants to use AWS to process the event data as it is received. The data is written in a specific order that must be maintained throughout processing. The company wants to implement a solution that minimizes operational overhead.
How should a solutions architect accomplish this?

A. Create an Amazon Simple Queue Service (Amazon SQS) FIFO queue to hold messages. Set up an AWS Lambda function to process messages from the queue.

B. Create an Amazon Simple Notification Service (Amazon SNS) topic to deliver notifications containing payloads to process. Configure an AWS Lambda function as a subscriber.

C. Create an Amazon Simple Queue Service (Amazon SQS) standard queue to hold messages. Set up an AWS Lambda function to process messages from the queue independently.

D. Create an Amazon Simple Notification Service (Amazon SNS) topic to deliver notifications containing payloads to process. Configure an Amazon Simple Queue Service (Amazon SQS) queue as a subscriber.

Answer: A

Explanation:

The correct answer is A. Here's why:

FIFO (First-In, First-Out) Queues: The core requirement is maintaining the order of events. Amazon SQS FIFO queues guarantee that messages are processed in the exact order they were sent. This aligns directly with the "specific order that must be maintained" requirement.
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues.html

AWS Lambda for Processing: AWS Lambda provides a serverless compute environment, which means the company doesn't have to manage servers. This fulfills the requirement of minimizing operational overhead. Lambda can be easily triggered by SQS messages. https://aws.amazon.com/lambda/

SQS and Lambda Integration: The tight integration between SQS and Lambda allows for automatic scaling and processing of messages as they arrive in the queue. Lambda functions can be configured to poll the SQS queue and process messages concurrently, while maintaining the FIFO order within each message group if configured appropriately. This ensures that all messages are eventually processed.
https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html

Why other options are incorrect:

B: SNS and Lambda: SNS is a pub/sub messaging service, which doesn't inherently guarantee message ordering. It's designed for distributing messages to multiple subscribers concurrently, which contradicts the ordering requirement.

C: SQS Standard Queue and Lambda: Standard SQS queues offer best-effort ordering, but do not guarantee that messages will be delivered in the exact order they were sent. This does not satisfy the ordering requirement.

D: SNS and SQS: While SNS can fan out messages to multiple SQS queues, SNS itself does not guarantee ordering. Even if each SQS queue subscriber processed in order, the SNS delivery to different queues would

still break the global order. Furthermore, it requires more configuration than the simpler SQS FIFO and Lambda option.

Therefore, using SQS FIFO queues to hold the messages and using Lambda to process them is the most appropriate solution for processing event data while maintaining order and minimizing operational overhead.

Question: 150 **Exam Heist**

A company is migrating an application from on-premises servers to Amazon EC2 instances. As part of the migration design requirements, a solutions architect must implement infrastructure metric alarms. The company does not need to take action if CPU utilization increases to more than 50% for a short burst of time. However, if the CPU utilization increases to more than 50% and read IOPS on the disk are high at the same time, the company needs to act as soon as possible. The solutions architect also must reduce false alarms.
What should the solutions architect do to meet these requirements?

A. Create Amazon CloudWatch composite alarms where possible.
B. Create Amazon CloudWatch dashboards to visualize the metrics and react to issues quickly.
C. Create Amazon CloudWatch Synthetics canaries to monitor the application and raise an alarm.
D. Create single Amazon CloudWatch metric alarms with multiple metric thresholds where possible.

Answer: A

Explanation:

The correct answer is A: Create Amazon CloudWatch composite alarms where possible.

Here's a detailed justification:

The scenario requires triggering an alarm only when both CPU utilization is high and read IOPS are high, and to avoid false alarms triggered by short bursts of CPU utilization.

Composite alarms in CloudWatch are designed to solve this exact problem. They allow you to combine multiple existing metric alarms (e.g., one for CPU utilization exceeding 50%, another for high read IOPS) into a single alarm. The composite alarm's state changes to ALARM only when all specified member alarms are in the ALARM state. This directly addresses the requirement that the company needs to act only when both conditions (high CPU and high read IOPS) are true.

Option A is correct because the composite alarm reduces false alarms. The company does not want to take action if CPU utilization increases to more than 50% for a short burst of time. This could be true, when using one alarm metric only (CPU). Using a composite alarm that triggers only when CPU utilization is more than 50% AND the read IOPS is high can reduce false positives.

Options B, C, and D are incorrect for the following reasons:

B. Create Amazon CloudWatch dashboards to visualize the metrics and react to issues quickly: While dashboards are valuable for visualization and monitoring, they don't automate the alarming process or reduce false positives based on combined metric conditions. A human would have to constantly monitor the dashboard and manually react, which isn't scalable or reliable for timely responses.
C. Create Amazon CloudWatch Synthetics canaries to monitor the application and raise an alarm: Synthetics canaries are used to monitor the availability and performance of web applications and APIs by simulating user traffic. This doesn't directly address the need to monitor server-level metrics like CPU utilization and read IOPS and combine their conditions for alarming. This can lead to many false positives.
D. Create single Amazon CloudWatch metric alarms with multiple metric thresholds where possible: Single metric alarms can only have thresholds for one metric. While you can set a threshold for CPU utilization, you cannot incorporate read IOPS into the same single alarm's threshold. This is because a single alarm is

designed to monitor one specific metric and trigger based on its behavior relative to pre-defined limits.

In summary, composite alarms provide the ability to define complex alarm conditions based on the states of multiple metric alarms, making them ideal for scenarios requiring correlation of different metrics for effective alarming and reduced false positives.

Supporting documentation:

Using composite alarms - Amazon CloudWatch:
CloudWatch Alarms:

Question: 151 **Exam Heist**

A company wants to migrate its on-premises data center to AWS. According to the company's compliance requirements, the company can use only the ap-northeast-3 Region. Company administrators are not permitted to connect VPCs to the internet.
Which solutions will meet these requirements? (Choose two.)

A. Use AWS Control Tower to implement data residency guardrails to deny internet access and deny access to all AWS Regions except ap-northeast-3.

B. Use rules in AWS WAF to prevent internet access. Deny access to all AWS Regions except ap-northeast-3 in the AWS account settings.

C. Use AWS Organizations to configure service control policies (SCPS) that prevent VPCs from gaining internet access. Deny access to all AWS Regions except ap-northeast-3.

D. Create an outbound rule for the network ACL in each VPC to deny all traffic from 0.0.0.0/0. Create an IAM policy for each user to prevent the use of any AWS Region other than ap-northeast-3.

E. Use AWS Config to activate managed rules to detect and alert for internet gateways and to detect and alert for new resources deployed outside of ap-northeast-3.

Answer: AC

Explanation:

The correct answer is AC. Here's why:

A. Use AWS Control Tower to implement data residency guardrails to deny internet access and deny access to all AWS Regions except ap-northeast-3.

AWS Control Tower is designed to manage and govern multi-account AWS environments. One of its core functionalities is to enforce policies and compliance across an organization. Control Tower's guardrails provide preventative or detective controls. Data residency guardrails, specifically, can enforce that data resides within a specified region. Guardrails can also prevent resources from being created in non-compliant regions. Moreover, Control Tower can enforce guardrails that prevent VPCs from having direct internet access, usually by denying the creation of Internet Gateways or NAT Gateways. This directly addresses both the regional compliance and the no-internet-access requirements. Control Tower provides centralized governance.

C. Use AWS Organizations to configure service control policies (SCPS) that prevent VPCs from gaining internet access. Deny access to all AWS Regions except ap-northeast-3.

AWS Organizations allows you to centrally manage and govern multiple AWS accounts. Service Control Policies (SCPs) are a key feature of Organizations. SCPs act as permission boundaries, controlling the maximum permissions available to accounts within an organization. You can create an SCP that explicitly denies the creation of Internet Gateways (IGWs) and NAT Gateways, which are the primary methods for a VPC to access the internet. You can also create an SCP that denies the creation of any resource in regions other than ap-northeast-3. This prevents users from creating resources in unapproved regions, thereby ensuring

compliance. SCPs apply to all users and roles within the affected accounts (excluding the management account root user) and can't be overridden by IAM policies within those accounts. This provides a strong, centrally enforced compliance mechanism.

Why the other options are incorrect:

B. Use rules in AWS WAF to prevent internet access. Deny access to all AWS Regions except ap-northeast-3 in the AWS account settings. AWS WAF protects web applications, not VPC infrastructure. There's no "deny access to all AWS Regions" setting in the account settings.

D. Create an outbound rule for the network ACL in each VPC to deny all traffic from 0.0.0.0/0. Create an IAM policy for each user to prevent the use of any AWS Region other than ap-northeast-3. Network ACLs are a good security layer but can be complex to manage at scale across many VPCs. IAM policies per user, while effective, are difficult to manage compared to centralized control via SCPs, which provides centralized governance.

E. Use AWS Config to activate managed rules to detect and alert for internet gateways and to detect and alert for new resources deployed outside of ap-northeast-3. AWS Config detects and alerts on non-compliant configurations. It does not prevent the creation of resources or internet gateways, so it's detective, not preventative. The requirement is to prevent internet access and resource creation in unapproved regions.

Supporting Links:

AWS Control Tower: https://aws.amazon.com/controltower/
AWS Organizations: https://aws.amazon.com/organizations/
Service Control Policies (SCPs):
https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html

Question: 152 **Exam Heist**

A company uses a three-tier web application to provide training to new employees. The application is accessed for only 12 hours every day. The company is using an Amazon RDS for MySQL DB instance to store information and wants to minimize costs.
What should a solutions architect do to meet these requirements?

A. Configure an IAM policy for AWS Systems Manager Session Manager. Create an IAM role for the policy. Update the trust relationship of the role. Set up automatic start and stop for the DB instance.

B. Create an Amazon ElastiCache for Redis cache cluster that gives users the ability to access the data from the cache when the DB instance is stopped. Invalidate the cache after the DB instance is started.

C. Launch an Amazon EC2 instance. Create an IAM role that grants access to Amazon RDS. Attach the role to the EC2 instance. Configure a cron job to start and stop the EC2 instance on the desired schedule.

D. Create AWS Lambda functions to start and stop the DB instance. Create Amazon EventBridge (Amazon CloudWatch Events) scheduled rules to invoke the Lambda functions. Configure the Lambda functions as event targets for the rules.

Answer: D

Explanation:

The most cost-effective solution for stopping and starting an RDS instance based on a schedule is using AWS Lambda and Amazon EventBridge (formerly CloudWatch Events). Lambda allows you to execute code without provisioning or managing servers. EventBridge triggers Lambda functions based on a defined schedule. Option D provides a simple, serverless way to automate the on/off cycle of the RDS instance.

Option A, while mentioning IAM, focuses on using Systems Manager Session Manager, which isn't directly related to scheduling the instance's on/off state. Systems Manager is typically used for managing EC2

instances, not directly controlling RDS start/stop functions.

Option B involves implementing ElastiCache. ElastiCache is for caching data to improve application performance, not for substituting a database while it's stopped. While technically feasible to store some data in ElastiCache, it adds unnecessary complexity and cost for the given requirement of cost optimization for an intermittently used RDS instance. The primary data store is the RDS database itself. Switching to and invalidating caches is more complex than simply stopping and starting the RDS instance.

Option C involves creating an EC2 instance and using cron jobs. While feasible, this approach requires managing an EC2 instance, increasing operational overhead and cost compared to a serverless approach. The EC2 instance would need to be running constantly to trigger the start/stop scripts, negating some of the cost savings intended.

Lambda and EventBridge (Option D) eliminate the need to manage a dedicated server or complex caching strategies. Lambda function execution time is charged by the millisecond, making it extremely cost-effective for this use case. EventBridge provides a reliable and scalable scheduling mechanism. The Lambda functions will only run when invoked by EventBridge, thus consuming very few resources overall and minimizing costs during idle periods.

In summary, using Lambda and EventBridge provides a serverless, event-driven, and cost-optimized approach for automatically starting and stopping an RDS instance based on a predefined schedule, directly addressing the business requirement of minimizing costs while only needing access for 12 hours per day.

Supporting Links:

AWS Lambda: https://aws.amazon.com/lambda/
Amazon EventBridge: https://aws.amazon.com/eventbridge/
Starting and Stopping an RDS Instance:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_StartInstance.html

Question: 153 **Exam Heist**

A company sells ringtones created from clips of popular songs. The files containing the ringtones are stored in Amazon S3 Standard and are at least 128 KB in size. The company has millions of files, but downloads are infrequent for ringtones older than 90 days. The company needs to save money on storage while keeping the most accessed files readily available for its users.
Which action should the company take to meet these requirements MOST cost-effectively?

A. Configure S3 Standard-Infrequent Access (S3 Standard-IA) storage for the initial storage tier of the objects.

B. Move the files to S3 Intelligent-Tiering and configure it to move objects to a less expensive storage tier after 90 days.

C. Configure S3 inventory to manage objects and move them to S3 Standard-Infrequent Access (S3 Standard-1A) after 90 days.

D. Implement an S3 Lifecycle policy that moves the objects from S3 Standard to S3 Standard-Infrequent Access (S3 Standard-1A) after 90 days.

Answer: D

Explanation:

The correct answer is **D. Implement an S3 Lifecycle policy that moves the objects from S3 Standard to S3 Standard-Infrequent Access (S3 Standard-IA) after 90 days.**

Here's why:

The company needs to reduce storage costs for infrequently accessed ringtones (older than 90 days) while

ensuring frequently accessed files remain readily available. S3 Lifecycle policies are designed precisely for this purpose. They automate the movement of objects between different storage classes based on predefined rules, like age.

Option D directly addresses the requirements: An S3 Lifecycle policy can be configured to transition objects from S3 Standard (the initial storage class) to S3 Standard-IA after 90 days. This is a cost-effective way to store infrequently accessed data while maintaining quick retrieval times when needed.

Let's analyze why the other options are less optimal:

Option A (S3 Standard-IA initially): While S3 Standard-IA is cheaper than S3 Standard, it incurs retrieval costs. Initially storing all files in Standard-IA would unnecessarily add retrieval costs for files that are frequently accessed in the first 90 days. This would be less cost-effective than using Standard initially.
Option B (S3 Intelligent-Tiering): S3 Intelligent-Tiering automatically moves data between tiers based on access patterns. While it sounds suitable, it includes a monthly monitoring and automation fee per object, and the question specifies millions of files. For objects only accessed infrequently after 90 days, this monitoring fee can quickly surpass the cost savings of moving to infrequent access tiers, especially when a simpler lifecycle rule is sufficient. While efficient, intelligent tiering is more costly for data that is almost guaranteed to be accessed infrequently after a certain period.
Option C (S3 Inventory and manual move): S3 Inventory provides a list of objects and their metadata. While useful for auditing, it doesn't automate the transition. The question requires automation to reduce operational overhead. Using S3 Inventory to trigger manual moves would involve custom scripting or manual intervention, which is less efficient and more complex than an S3 Lifecycle policy.

In summary, S3 Lifecycle policies are the most straightforward and cost-effective method for automatically moving objects between storage classes based on their age, directly addressing the company's needs without unnecessary complexity or additional fees.

Authoritative Links:

S3 Lifecycle Policies: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-transition-general-considerations.html
S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
S3 Intelligent-Tiering: https://aws.amazon.com/s3/intelligent-tiering/

Question: 154 **Exam Heist**

A company needs to save the results from a medical trial to an Amazon S3 repository. The repository must allow a few scientists to add new files and must restrict all other users to read-only access. No users can have the ability to modify or delete any files in the repository. The company must keep every file in the repository for a minimum of 1 year after its creation date.
Which solution will meet these requirements?

A. Use S3 Object Lock in governance mode with a legal hold of 1 year.

B. Use S3 Object Lock in compliance mode with a retention period of 365 days.

C. Use an IAM role to restrict all users from deleting or changing objects in the S3 bucket. Use an S3 bucket policy to only allow the IAM role.

D. Configure the S3 bucket to invoke an AWS Lambda function every time an object is added. Configure the function to track the hash of the saved object so that modified objects can be marked accordingly.

Answer: B

Explanation:

The correct answer is B: Use S3 Object Lock in compliance mode with a retention period of 365 days. Here's

why:

Requirement for Immutability: The core requirement is that no user can modify or delete any files in the S3 repository. This necessitates immutability.

S3 Object Lock: S3 Object Lock is designed to prevent objects from being deleted or overwritten for a fixed amount of time or indefinitely. This addresses the immutability requirement directly. There are two modes: compliance and governance.

Compliance Mode: In compliance mode, even the root user cannot override the retention settings. Once a retention period is set, it's strictly enforced. This aligns perfectly with the strict requirement that no users can delete or modify files.

Governance Mode: Governance mode allows users with specific IAM permissions to bypass the retention settings. This is not suitable since the problem specifies that no users should be able to bypass the retention policy.

Retention Period of 365 Days: The requirement of keeping every file for a minimum of one year (365 days) is satisfied by setting the retention period accordingly.

Why other options are incorrect:

A: S3 Object Lock in governance mode with a legal hold of 1 year. While governance mode provides a level of immutability, authorized users can remove legal holds. This contradicts the requirement that no users can modify or delete files.

C: IAM role to restrict all users from deleting or changing objects in the S3 bucket. Use an S3 bucket policy to only allow the IAM role. IAM and bucket policies can restrict access, but they don't guarantee immutability. A user with sufficient permissions could still modify or delete objects. This is a preventative measure, not a guaranteed solution.

D: Configure the S3 bucket to invoke an AWS Lambda function every time an object is added. Configure the function to track the hash of the saved object so that modified objects can be marked accordingly. This approach is complex, inefficient, and doesn't prevent modification or deletion. It only detects changes after they occur, which violates the problem's core immutability requirement. S3 Object Lock provides a native and much simpler solution.

In summary, S3 Object Lock in compliance mode directly enforces immutability and the specified retention period, making it the most appropriate and efficient solution.

Relevant Links:

S3 Object Lock

Question: 155 **Exam Heist**

A large media company hosts a web application on AWS. The company wants to start caching confidential media files so that users around the world will have reliable access to the files. The content is stored in Amazon S3 buckets. The company must deliver the content quickly, regardless of where the requests originate geographically.
Which solution will meet these requirements?

A. Use AWS DataSync to connect the S3 buckets to the web application.
B. Deploy AWS Global Accelerator to connect the S3 buckets to the web application.
C. Deploy Amazon CloudFront to connect the S3 buckets to CloudFront edge servers.
D. Use Amazon Simple Queue Service (Amazon SQS) to connect the S3 buckets to the web application.

Answer: C

Explanation:

The correct answer is C: Deploy Amazon CloudFront to connect the S3 buckets to CloudFront edge servers.

Here's a detailed justification:

The primary requirement is to cache confidential media files stored in S3 for fast and reliable delivery to users globally. Amazon CloudFront is a content delivery network (CDN) specifically designed for this purpose. CloudFront caches content at edge locations distributed around the world. When a user requests a file, CloudFront serves it from the nearest edge location, minimizing latency and improving performance. Since the content is stored in S3, CloudFront can be configured as the front-end delivery mechanism.

Option A, using AWS DataSync, is incorrect because DataSync is used for transferring large amounts of data between on-premises storage and AWS storage services. It is not designed for real-time content delivery or caching.

Option B, deploying AWS Global Accelerator, is not ideal for this scenario. Global Accelerator improves the performance of TCP and UDP traffic by routing user traffic to the optimal AWS endpoint. While it can improve performance, it doesn't provide content caching like CloudFront. Further, CloudFront offers fine-grained control over content caching policies and integrates directly with S3 for content delivery.

Option D, using Amazon SQS, is incorrect because SQS is a message queue service used for decoupling components of distributed applications. It's not related to content delivery or caching. SQS does not cache content or improve latency for end users downloading media files.

CloudFront offers several benefits for this use case:

Global Reach: Edge locations are strategically located worldwide for low latency delivery.
Caching: Content is cached at edge locations, reducing load on S3 and speeding up delivery.
Security: Can be integrated with AWS WAF and provides features for securing content, including signed URLs.
Integration with S3: Seamless integration with S3 for origin storage.
Customizable caching behavior: CloudFront allows you to define how long content is cached and can invalidate content based on your business needs.

For further reading:

Amazon CloudFront Documentation
Amazon S3 Documentation

Question: 156 **Exam Heist**

A company produces batch data that comes from different databases. The company also produces live stream data from network sensors and application APIs. The company needs to consolidate all the data into one place for business analytics. The company needs to process the incoming data and then stage the data in different Amazon S3 buckets. Teams will later run one-time queries and import the data into a business intelligence tool to show key performance indicators (KPIs).
Which combination of steps will meet these requirements with the LEAST operational overhead? (Choose two.)

A. Use Amazon Athena for one-time queries. Use Amazon QuickSight to create dashboards for KPIs.

B. Use Amazon Kinesis Data Analytics for one-time queries. Use Amazon QuickSight to create dashboards for KPIs.

C. Create custom AWS Lambda functions to move the individual records from the databases to an Amazon Redshift cluster.

D. Use an AWS Glue extract, transform, and load (ETL) job to convert the data into JSON format. Load the data into multiple Amazon OpenSearch Service (Amazon Elasticsearch Service) clusters.

E. Use blueprints in AWS Lake Formation to identify the data that can be ingested into a data lake. Use AWS Glue to crawl the source, extract the data, and load the data into Amazon S3 in Apache Parquet format.

Answer: AE

Explanation:

The best combination for this scenario is **A and E**.

Here's why:

A. Use Amazon Athena for one-time queries. Use Amazon QuickSight to create dashboards for KPIs.

Athena is a serverless, interactive query service that directly analyzes data in S3 using standard SQL. This eliminates the need for managing infrastructure and simplifies one-time queries.
https://aws.amazon.com/athena/
QuickSight is a fully managed, serverless BI service that allows easy dashboard creation and KPI visualization. It integrates seamlessly with Athena (and other AWS data sources). https://aws.amazon.com/quicksight/
Combined, these services provide a straightforward, low-overhead solution for ad-hoc analysis and KPI reporting from the data consolidated in S3.

E. Use blueprints in AWS Lake Formation to identify the data that can be ingested into a data lake. Use AWS Glue to crawl the source, extract the data, and load the data into Amazon S3 in Apache Parquet format.

Lake Formation simplifies the setup of a data lake, automatically discovering and cataloging data. Blueprints help automate the creation of data lakes from common data sources. https://aws.amazon.com/lake-formation/
AWS Glue is a fully managed ETL service that automates the process of data discovery, transformation, and loading. Its crawlers can infer schema and load the data in different formats into S3.
https://aws.amazon.com/glue/
Parquet is a columnar storage format, optimizing data retrieval for analytics and is ideal for Athena queries.
By using Lake Formation and Glue, the company minimizes operational overhead by automating the data ingestion, transformation and cataloging processes.

Why other options are less ideal:

B: Kinesis Data Analytics is designed for real-time streaming data analysis. It isn't the optimal choice for one-time queries against batch data or staged data.
C: Creating custom Lambda functions to move individual records from databases to Redshift is complex, high-overhead, and doesn't scale well. It increases operational burden and costs.
D: OpenSearch Service is typically used for search and log analytics, not for general-purpose data warehousing or BI workloads. Converting the data to JSON and loading it into OpenSearch is less efficient than using Parquet and Athena. The need to manage multiple OpenSearch Clusters adds to overhead.

Question: 157 **Exam Heist**

A company stores data in an Amazon Aurora PostgreSQL DB cluster. The company must store all the data for 5 years and must delete all the data after 5 years. The company also must indefinitely keep audit logs of actions that are performed within the database. Currently, the company has automated backups configured for Aurora.

Which combination of steps should a solutions architect take to meet these requirements? (Choose two.)

A. Take a manual snapshot of the DB cluster.

B. Create a lifecycle policy for the automated backups.

C. Configure automated backup retention for 5 years.

D. Configure an Amazon CloudWatch Logs export for the DB cluster.

E. Use AWS Backup to take the backups and to keep the backups for 5 years.

Answer: DE

Explanation:

The correct answer is **DE**. Let's break down why:

D. Configure an Amazon CloudWatch Logs export for the DB cluster: This is crucial for meeting the requirement of indefinitely keeping audit logs. Aurora PostgreSQL generates audit logs, which can be streamed to CloudWatch Logs. From there, you can retain them indefinitely, satisfying the audit log requirement.
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_LogAccess.Concepts.PostgreSQL.html and https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/postgresql_cloudwatch_logs.html

E. Use AWS Backup to take the backups and to keep the backups for 5 years: AWS Backup provides a centralized service to manage and automate backups across AWS services, including Aurora. You can define backup plans that specify the backup frequency and retention period. Setting the retention period to 5 years ensures the data is retained as required and automatically deleted after that period.
https://aws.amazon.com/backup/

Now, let's address why the other options are incorrect:

A. Take a manual snapshot of the DB cluster: While you could take a manual snapshot, this doesn't automate the 5-year retention and deletion requirement. It would require manual intervention to delete the snapshot after 5 years. Also, it's a one-time event rather than a managed backup strategy.

B. Create a lifecycle policy for the automated backups: You cannot directly apply a lifecycle policy (like those used with S3) to RDS automated backups. Automated backups are managed by RDS itself and have a retention period configured within RDS.

C. Configure automated backup retention for 5 years: While this would meet the 5-year retention requirement, Aurora managed automated backups are not designed for long-term archiving like this. Using only automated backups would also not address indefinite audit logging. Using AWS Backup is a best practice for long-term retention and compliance of DB clusters.
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_WorkingWithAutomatedBackups.html

Question: 158 **Exam Heist**

A solutions architect is optimizing a website for an upcoming musical event. Videos of the performances will be streamed in real time and then will be available on demand. The event is expected to attract a global online audience.

Which service will improve the performance of both the real-time and on-demand streaming?

A. Amazon CloudFront

B. AWS Global Accelerator

C. Amazon Route 53

D. Amazon S3 Transfer Acceleration

Answer: A

Explanation:

Amazon CloudFront is the optimal choice because it's a content delivery network (CDN) designed to distribute content with low latency and high transfer speeds globally. For real-time streaming, CloudFront caches the streaming content at edge locations closer to users, minimizing latency and buffering. This is achieved through its extensive network of edge servers strategically located worldwide. For on-demand streaming, CloudFront also caches video files, ensuring quick delivery to users regardless of their location.

AWS Global Accelerator primarily focuses on improving TCP and UDP performance for applications over long distances by routing traffic through optimized AWS network paths. While it could improve network performance, it doesn't offer the caching benefits of CloudFront.

Amazon Route 53 is a highly available and scalable DNS web service. It translates domain names into IP addresses, but it doesn't cache content or improve content delivery performance directly. While it can be integrated with CloudFront, it is not a solution for improving both real-time and on-demand streaming performance.

Amazon S3 Transfer Acceleration accelerates transfers into and out of S3 buckets, but it doesn't directly address the needs of live and on-demand streaming to a global audience in the same way a CDN like CloudFront does. It's more suitable for accelerating data uploads to S3 rather than content delivery.

Therefore, CloudFront's caching capabilities, global edge locations, and integration with other AWS services make it ideal for optimizing both real-time and on-demand streaming experiences for a global audience.

Here are some authoritative links for further research:

Amazon CloudFront: https://aws.amazon.com/cloudfront/
AWS Global Accelerator: https://aws.amazon.com/global-accelerator/
Amazon Route 53: https://aws.amazon.com/route53/
Amazon S3 Transfer Acceleration: https://aws.amazon.com/s3/transfer-acceleration/

Question: 159 **Exam Heist**

A company is running a publicly accessible serverless application that uses Amazon API Gateway and AWS Lambda. The application's traffic recently spiked due to fraudulent requests from botnets.

Which steps should a solutions architect take to block requests from unauthorized users? (Choose two.)

A. Create a usage plan with an API key that is shared with genuine users only.

B. Integrate logic within the Lambda function to ignore the requests from fraudulent IP addresses.

C. Implement an AWS WAF rule to target malicious requests and trigger actions to filter them out.

D. Convert the existing public API to a private API. Update the DNS records to redirect users to the new API endpoint.

E. Create an IAM role for each user attempting to access the API. A user will assume the role when making the API call.

Answer: AC

Explanation:

The correct answer is AC. Here's a detailed justification:

A. Create a usage plan with an API key that is shared with genuine users only.

API Keys in Amazon API Gateway can be used to control access to your API and prevent unauthorized access. By creating a usage plan and associating an API key with it, you can distribute the key to legitimate users

only. API Gateway then verifies the API key on each request and only allows requests with valid API keys to proceed. This approach helps block traffic from bots that do not possess the correct API key. https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-api-usage-plans.html

C. Implement an AWS WAF rule to target malicious requests and trigger actions to filter them out.

AWS WAF (Web Application Firewall) protects your web applications from common web exploits and bots that could affect availability, compromise security, or consume excessive resources. WAF allows you to create custom rules that target specific request patterns, such as those originating from known botnet IP addresses or those that exhibit suspicious behavior. By setting up WAF rules to identify and block these malicious requests, you can effectively filter out the fraudulent traffic hitting your API Gateway endpoint. The actions triggered could include blocking the requests, allowing the requests after inspection, or logging the requests for further analysis. https://aws.amazon.com/waf/

Why other options are incorrect:

B. Integrate logic within the Lambda function to ignore the requests from fraudulent IP addresses: While feasible, this approach is less efficient and scalable. It adds complexity to the Lambda function, consumes Lambda execution time checking IP addresses, and requires constant updates to the Lambda function as botnets evolve. It's better to handle this at the API Gateway or WAF layer.

D. Convert the existing public API to a private API. Update the DNS records to redirect users to the new API endpoint: Converting to a private API would indeed block public access. However, it doesn't solve the fundamental problem of distinguishing between legitimate and fraudulent users within the now-controlled access. It would severely affect genuine public users, which is not desirable according to the question.

E. Create an IAM role for each user attempting to access the API. A user will assume the role when making the API call: Creating individual IAM roles for each user is overkill for a publicly accessible application dealing with botnet attacks. IAM roles are better suited for internal or controlled access, not for scaling to potentially thousands or millions of users. Furthermore, bots could still potentially assume these roles if they can gain access to user credentials.

Question: 160 **Exam Heist**

An ecommerce company hosts its analytics application in the AWS Cloud. The application generates about 300 MB of data each month. The data is stored in JSON format. The company is evaluating a disaster recovery solution to back up the data. The data must be accessible in milliseconds if it is needed, and the data must be kept for 30 days.

Which solution meets these requirements MOST cost-effectively?

A. Amazon OpenSearch Service (Amazon Elasticsearch Service)
B. Amazon S3 Glacier
C. Amazon S3 Standard
D. Amazon RDS for PostgreSQL

Answer: C

Explanation:

Here's a detailed justification for why option C, Amazon S3 Standard, is the most cost-effective solution for the given scenario, along with supporting concepts and authoritative links:

The problem requires a disaster recovery backup solution for 300 MB of JSON data generated monthly by an analytics application. Key requirements are millisecond access time and a 30-day retention period. Cost-

effectiveness is paramount.

Amazon S3 Standard offers high durability, availability, and performance for frequently accessed data. It provides low latency, making it suitable for the required millisecond access time. Its cost is relatively low compared to alternatives that are designed for ultra-low latency transactional workloads. For 300MB, the cost is negligible. https://aws.amazon.com/s3/storage-classes/

Amazon OpenSearch Service (Amazon Elasticsearch Service) is primarily used for search and analytics. While it provides fast access, it is significantly more expensive than S3 for simply storing and retrieving 300MB of data. It also involves more operational overhead (managing clusters, scaling). It's overkill for just a backup requirement.

Amazon S3 Glacier is designed for long-term archival storage with infrequent access. Retrieval times range from minutes to hours, violating the millisecond access requirement. While Glacier is cheap for storage, retrieval costs can be substantial if the data is needed frequently.

Amazon RDS for PostgreSQL is a relational database service, not suitable for storing JSON data as a simple backup. Storing JSON in a database would introduce unnecessary complexity and cost for this backup purpose. Additionally, managing a database for a 300MB backup is extremely inefficient.

Considering the speed requirement and the need for cost optimization, Amazon S3 Standard emerges as the most fitting choice. It offers the necessary performance while remaining budget-friendly for a small dataset. It aligns perfectly with the characteristics of data backup where quick retrieval capabilities are important, making it a cost-effective solution to this disaster recovery concerns.

Question: 161 **Exam Heist**

A company has a small Python application that processes JSON documents and outputs the results to an on-premises SQL database. The application runs thousands of times each day. The company wants to move the application to the AWS Cloud. The company needs a highly available solution that maximizes scalability and minimizes operational overhead.

Which solution will meet these requirements?

A. Place the JSON documents in an Amazon S3 bucket. Run the Python code on multiple Amazon EC2 instances to process the documents. Store the results in an Amazon Aurora DB cluster.

B. Place the JSON documents in an Amazon S3 bucket. Create an AWS Lambda function that runs the Python code to process the documents as they arrive in the S3 bucket. Store the results in an Amazon Aurora DB cluster.

C. Place the JSON documents in an Amazon Elastic Block Store (Amazon EBS) volume. Use the EBS Multi-Attach feature to attach the volume to multiple Amazon EC2 instances. Run the Python code on the EC2 instances to process the documents. Store the results on an Amazon RDS DB instance.

D. Place the JSON documents in an Amazon Simple Queue Service (Amazon SQS) queue as messages. Deploy the Python code as a container on an Amazon Elastic Container Service (Amazon ECS) cluster that is configured with the Amazon EC2 launch type. Use the container to process the SQS messages. Store the results on an Amazon RDS DB instance.

Answer: B

Explanation:

Option B is the most suitable solution because it leverages serverless technologies and managed services to meet the requirements of high availability, scalability, and minimal operational overhead. Placing JSON documents in an Amazon S3 bucket triggers an AWS Lambda function upon arrival of a new document. Lambda functions are event-driven and automatically scale to handle incoming requests, eliminating the need for manual server management. Lambda's pay-per-use model minimizes cost when the application is idle.

The Python code executed within the Lambda function processes the JSON documents. Storing the results in an Amazon Aurora DB cluster offers high availability and scalability. Aurora automatically manages database infrastructure, including replication and failover, reducing operational burden. Aurora is also compatible with MySQL and PostgreSQL, simplifying migration from on-premises SQL databases.

Option A requires managing EC2 instances, including patching, scaling, and ensuring high availability, which adds operational overhead. EBS Multi-Attach (Option C) has limitations on the number of instances and can introduce complexity in managing shared storage. Amazon SQS with ECS (Option D) is a viable solution for asynchronous processing, but using EC2 launch type adds operational overhead for managing the underlying EC2 instances. Lambda is generally more cost-effective and requires less management for this use case.

In summary, Option B is the best choice as it combines the scalability and event-driven nature of Lambda with the high availability and managed nature of Aurora, offering a serverless solution that minimizes operational overhead and maximizes scalability.

References:

AWS Lambda: https://aws.amazon.com/lambda/
Amazon S3: https://aws.amazon.com/s3/
Amazon Aurora: https://aws.amazon.com/rds/aurora/

Question: 162 **Exam Heist**

A company wants to use high performance computing (HPC) infrastructure on AWS for financial risk modeling. The company's HPC workloads run on Linux. Each HPC workflow runs on hundreds of Amazon EC2 Spot Instances, is short-lived, and generates thousands of output files that are ultimately stored in persistent storage for analytics and long-term future use.

The company seeks a cloud storage solution that permits the copying of on-premises data to long-term persistent storage to make data available for processing by all EC2 instances. The solution should also be a high performance file system that is integrated with persistent storage to read and write datasets and output files.

Which combination of AWS services meets these requirements?

A. Amazon FSx for Lustre integrated with Amazon S3
B. Amazon FSx for Windows File Server integrated with Amazon S3
C. Amazon S3 Glacier integrated with Amazon Elastic Block Store (Amazon EBS)
D. Amazon S3 bucket with a VPC endpoint integrated with an Amazon Elastic Block Store (Amazon EBS) General Purpose SSD (gp2) volume

Answer: A

Explanation:

The best solution for the company's HPC workload requirements is **A. Amazon FSx for Lustre integrated with Amazon S3.** Here's why:

High-Performance File System: FSx for Lustre is designed for high-performance computing, machine learning, and media processing workloads. It provides a parallel distributed file system that can handle the demands of HPC applications. https://aws.amazon.com/fsx/lustre/
Integration with S3: FSx for Lustre seamlessly integrates with Amazon S3. You can configure it to automatically copy data from S3 buckets to the file system for processing, and then export the output files back to S3 for long-term storage and analytics. This addresses the requirement of moving on-premises data (via S3) and storing output. https://docs.aws.amazon.com/fsx/latest/LustreGuide/import-data-repo.html
Scalability and Spot Instances: FSx for Lustre can scale to meet the demands of hundreds of EC2 Spot Instances, providing the necessary performance for the company's short-lived, intensive workloads.

Linux Compatibility: Lustre is a Linux-based file system, which aligns with the company's requirement of running HPC workloads on Linux.
Persistent Storage: Amazon S3 provides highly durable and scalable object storage, ensuring long-term persistence for the output files. https://aws.amazon.com/s3/

Why other options are not ideal:

B. Amazon FSx for Windows File Server: While FSx for Windows File Server is a fully managed, highly reliable, and scalable file storage, it's optimized for Windows-based workloads, not Linux HPC workloads.
C. Amazon S3 Glacier integrated with Amazon EBS: S3 Glacier is designed for long-term archival storage and is not suitable for the high-performance read/write requirements of HPC workflows. EBS is block storage attached to an EC2 instance, not designed for shared high performance access required by multiple instances.
D. Amazon S3 bucket with a VPC endpoint integrated with an Amazon Elastic Block Store (Amazon EBS) General Purpose SSD (gp2) volume: While S3 is persistent, directly using S3 and EBS would not provide the high-performance parallel file system needed for the HPC workload. The general purpose SSD also isn't suited for this task.

In conclusion, Amazon FSx for Lustre integrated with Amazon S3 provides the best combination of high performance, scalability, Linux compatibility, and integration with persistent storage to meet the company's HPC workload requirements.

Question: 163 **Exam Heist**

A company is building a containerized application on premises and decides to move the application to AWS. The application will have thousands of users soon after it is deployed. The company is unsure how to manage the deployment of containers at scale. The company needs to deploy the containerized application in a highly available architecture that minimizes operational overhead.

Which solution will meet these requirements?

A. Store container images in an Amazon Elastic Container Registry (Amazon ECR) repository. Use an Amazon Elastic Container Service (Amazon ECS) cluster with the AWS Fargate launch type to run the containers. Use target tracking to scale automatically based on demand.

B. Store container images in an Amazon Elastic Container Registry (Amazon ECR) repository. Use an Amazon Elastic Container Service (Amazon ECS) cluster with the Amazon EC2 launch type to run the containers. Use target tracking to scale automatically based on demand.

C. Store container images in a repository that runs on an Amazon EC2 instance. Run the containers on EC2 instances that are spread across multiple Availability Zones. Monitor the average CPU utilization in Amazon CloudWatch. Launch new EC2 instances as needed.

D. Create an Amazon EC2 Amazon Machine Image (AMI) that contains the container image. Launch EC2 instances in an Auto Scaling group across multiple Availability Zones. Use an Amazon CloudWatch alarm to scale out EC2 instances when the average CPU utilization threshold is breached.

Answer: A

Explanation:

The correct answer is A because it provides a fully managed, highly scalable, and highly available solution with minimal operational overhead. Here's a breakdown:

Amazon ECR (Elastic Container Registry): Storing container images in ECR provides a secure, scalable, and reliable registry for storing and managing your Docker container images. It eliminates the need to manage your own container registry on EC2, reducing operational overhead. https://aws.amazon.com/ecr/

Amazon ECS (Elastic Container Service) with Fargate Launch Type: ECS is a fully managed container

orchestration service. Using the Fargate launch type abstracts away the underlying EC2 infrastructure management. Fargate manages the scaling, patching, and security of the underlying compute resources, significantly reducing operational overhead. https://aws.amazon.com/ecs/fargate/

Target Tracking Scaling: ECS allows you to automatically scale your services based on metrics like CPU utilization or request count. Target tracking automatically adjusts the number of tasks to maintain a specified target value for a chosen metric. This simplifies scaling and ensures optimal resource utilization. https://docs.aws.amazon.com/AmazonECS/latest/developerguide/service-autoscaling-targettracking.html

Option B is incorrect because using the EC2 launch type requires you to manage the underlying EC2 instances, which introduces significant operational overhead, contradicting the problem's requirements.

Options C and D are even less suitable. They require manual management of EC2 instances for container deployment and scaling, negating the requirement to minimize operational overhead. They also lack the inherent scalability and availability of managed container orchestration services like ECS with Fargate. Building container images into AMIs is not a best practice for container deployments; it's better to build images separately and deploy them via container orchestration. Also, Option C suggests a container image repository on an EC2 instance which could become a single point of failure.

Question: 164 **Exam Heist**

A company has two applications: a sender application that sends messages with payloads to be processed and a processing application intended to receive the messages with payloads. The company wants to implement an AWS service to handle messages between the two applications. The sender application can send about 1,000 messages each hour. The messages may take up to 2 days to be processed: If the messages fail to process, they must be retained so that they do not impact the processing of any remaining messages.

Which solution meets these requirements and is the MOST operationally efficient?

A. Set up an Amazon EC2 instance running a Redis database. Configure both applications to use the instance. Store, process, and delete the messages, respectively.

B. Use an Amazon Kinesis data stream to receive the messages from the sender application. Integrate the processing application with the Kinesis Client Library (KCL).

C. Integrate the sender and processor applications with an Amazon Simple Queue Service (Amazon SQS) queue. Configure a dead-letter queue to collect the messages that failed to process.

D. Subscribe the processing application to an Amazon Simple Notification Service (Amazon SNS) topic to receive notifications to process. Integrate the sender application to write to the SNS topic.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best solution, and why the other options are less suitable:

Why Option C (Amazon SQS with Dead-Letter Queue) is the best:

Amazon Simple Queue Service (SQS) is a fully managed message queuing service. Its core purpose is decoupling applications and enabling asynchronous communication. This directly addresses the company's requirement to handle messages between the sender and processing applications. SQS is highly scalable, reliable, and requires minimal operational overhead.

Asynchronous Communication: SQS allows the sender application to send messages without waiting for the processing application to be immediately available. This decoupling improves application resilience.
Message Retention: SQS retains messages until they are successfully processed and deleted, guaranteeing message delivery. The 2-day processing requirement falls well within SQS's configurable message retention period.

Fault Tolerance: The crucial aspect of a dead-letter queue (DLQ) makes this solution stand out. When a message fails to process after a specified number of retries, SQS moves it to the DLQ. This ensures that failed messages don't block the processing of other messages, and allows for later analysis and reprocessing of the failed messages. This aligns perfectly with the requirement to retain failed messages without impacting the rest.
Operational Efficiency: SQS is a fully managed service, removing the need for the company to manage servers, scaling, or patching.
Cost-Effective: SQS pricing is based on usage, making it a cost-effective solution for handling 1,000 messages per hour.

Why Option A (Amazon EC2 with Redis) is less suitable:

While Redis is a fast in-memory data store, using it as a message queue hosted on an EC2 instance introduces significant operational overhead.

Management Overhead: The company needs to manage the EC2 instance, including OS patching, scaling, and ensuring high availability for Redis.
Complexity: Implementing message queuing functionality in Redis requires custom code and logic, adding complexity.
Scalability Concerns: Scaling Redis effectively requires expertise and may involve sharding or clustering.
No Built-in DLQ: Redis does not have a built-in DLQ mechanism, requiring custom implementation to handle failed messages.

Why Option B (Amazon Kinesis Data Streams) is less suitable:

Kinesis Data Streams is designed for real-time streaming data, like logs or sensor data, and isn't optimal for asynchronous message processing with specific ordering requirements or individual message handling in this scenario.

Complexity: Kinesis Client Library (KCL) adds complexity to the processing application.
Ordering focus: Kinesis shines at ordering large volumes of data within shards but the question specifies low throughput (1000 messages/hour).
No native DLQ: Implementing DLQ functionality with Kinesis requires custom logic and extra storage.

Why Option D (Amazon SNS) is less suitable:

Amazon SNS is primarily a publish/subscribe service for notifications, not a message queue.

Push-Based: SNS pushes messages to subscribers, which is less reliable than SQS's pull-based model for guaranteed delivery.
Message Loss: If a subscriber is unavailable, the message may be lost. SNS is not ideal for applications that require guaranteed message delivery.
No DLQ: SNS does not have a built-in DLQ.

Authoritative Links:

Amazon SQS: https://aws.amazon.com/sqs/
SQS Dead-Letter Queues:
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html
Amazon Kinesis Data Streams: https://aws.amazon.com/kinesis/data-streams/
Amazon SNS: https://aws.amazon.com/sns/

In summary, option C (SQS with a DLQ) is the most operationally efficient and best-suited solution because it provides guaranteed message delivery, automatic handling of failed messages, and requires minimal management overhead.

Question: 165 **Exam Heist**

A solutions architect must design a solution that uses Amazon CloudFront with an Amazon S3 origin to store a static website. The company's security policy requires that all website traffic be inspected by AWS WAF.

How should the solutions architect comply with these requirements?

A. Configure an S3 bucket policy to accept requests coming from the AWS WAF Amazon Resource Name (ARN) only.

B. Configure Amazon CloudFront to forward all incoming requests to AWS WAF before requesting content from the S3 origin.

C. Configure a security group that allows Amazon CloudFront IP addresses to access Amazon S3 only. Associate AWS WAF to CloudFront.

D. Configure Amazon CloudFront and Amazon S3 to use an origin access identity (OAI) to restrict access to the S3 bucket. Enable AWS WAF on the distribution.

Answer: D

Explanation:

The correct answer is D because it addresses both security requirements: restricting S3 access and inspecting traffic with AWS WAF.

Restricting S3 Access: Using an Origin Access Identity (OAI) with CloudFront is the recommended way to prevent users from directly accessing the S3 bucket hosting the static website. The OAI is a special CloudFront user that you grant permission to read objects in your S3 bucket. This ensures that all requests for your content must go through CloudFront.
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html

Inspecting Traffic with AWS WAF: AWS WAF is integrated with CloudFront, meaning you can associate a WAF web ACL with your CloudFront distribution. This allows AWS WAF to inspect incoming HTTP(S) requests to your website before they reach CloudFront, filtering out malicious traffic and protecting your application.
https://docs.aws.amazon.com/waf/latest/developerguide/cloudfront-integration.html

Why other options are incorrect:

A: S3 bucket policies can restrict access based on various criteria, including source IP, but AWS WAF doesn't have a fixed ARN that you can reliably use in an S3 bucket policy. Moreover, this approach doesn't actually inspect the traffic, it just tries to allow requests originating from where WAF is sitting. The problem WAF solves is understanding the **content** of the traffic, not just its origin.
B: CloudFront does not "forward" requests to WAF. WAF inspects requests made to the CloudFront endpoint and acts (allow/block) accordingly. There isn't a mechanism in CloudFront to forward traffic to WAF before retrieving the content.
C: Security Groups are instance-level firewalls, not appropriate for S3 bucket access control or CloudFront traffic filtering. While you can control access from EC2 instances to S3 buckets with security groups, CloudFront is a CDN, and its edge locations' IP addresses are not static or manageable through a security group in this manner. Plus, Security groups do not provide HTTP inspection capabilities like AWS WAF.

Therefore, answer D directly addresses both the security and access control requirements most effectively and aligns with AWS best practices for securing static websites hosted on S3 behind CloudFront.

Question: 166 **Exam Heist**

Organizers for a global event want to put daily reports online as static HTML pages. The pages are expected to generate millions of views from users around the world. The files are stored in an Amazon S3 bucket. A solutions architect has been asked to design an efficient and effective solution.

Which action should the solutions architect take to accomplish this?

A. Generate presigned URLs for the files.
B. Use cross-Region replication to all Regions.
C. Use the geoproximity feature of Amazon Route 53.
D. Use Amazon CloudFront with the S3 bucket as its origin.

Answer: D

Explanation:

The correct answer is **D. Use Amazon CloudFront with the S3 bucket as its origin.**

Here's why:

The scenario describes a global event generating millions of views of static HTML pages stored in S3. The primary concerns are performance, scalability, and cost-effectiveness in serving this content globally.

Amazon CloudFront is a Content Delivery Network (CDN) service offered by AWS. It excels at distributing static content to users around the world with low latency and high transfer speeds. CloudFront caches the content in edge locations closer to users, reducing the distance data needs to travel, resulting in faster load times and a better user experience. By using the S3 bucket as the origin for CloudFront, the static HTML files are efficiently served to users worldwide. CloudFront also handles request routing, scaling, and security.

Let's examine why the other options are less suitable:

A. Generate presigned URLs for the files: Presigned URLs grant temporary access to objects in S3. While they are useful for controlling access, they don't inherently improve performance or scalability for globally accessed static content. They also introduce overhead in generating and managing these URLs.

B. Use cross-Region replication to all Regions: Cross-Region Replication (CRR) is beneficial for disaster recovery or data sovereignty, not primarily for improving content delivery performance for globally distributed users. It replicates data between S3 buckets in different regions. While it could technically reduce latency for some users by having copies closer to them, it's much less efficient and more expensive than using a CDN like CloudFront. You'd need a complex routing mechanism to direct users to the nearest replicated bucket, which CloudFront handles automatically.

C. Use the geoproximity feature of Amazon Route 53: Route 53's geoproximity routing directs users to resources based on their geographic location. This works well for dynamic content or applications running in different regions. However, for static content already stored in S3, it is not enough by itself to optimize distribution. You would still need a way to efficiently serve the content from those locations, which is precisely what CloudFront does by caching content at its edge locations. Routing to a single S3 bucket closer to the user wouldn't provide the same level of performance as a globally distributed CDN.

In summary, CloudFront is the ideal solution because it is specifically designed for efficient and scalable global content delivery. It offers superior performance, scalability, and cost-effectiveness compared to the other options.

Supporting Documentation:

Amazon CloudFront: https://aws.amazon.com/cloudfront/

Amazon S3: https://aws.amazon.com/s3/
Amazon Route 53: https://aws.amazon.com/route53/

Question: 167 **Exam Heist**

A company runs a production application on a fleet of Amazon EC2 instances. The application reads the data from an Amazon SQS queue and processes the messages in parallel. The message volume is unpredictable and often has intermittent traffic. This application should continually process messages without any downtime.

Which solution meets these requirements MOST cost-effectively?

A. Use Spot Instances exclusively to handle the maximum capacity required.

B. Use Reserved Instances exclusively to handle the maximum capacity required.

C. Use Reserved Instances for the baseline capacity and use Spot Instances to handle additional capacity.

D. Use Reserved Instances for the baseline capacity and use On-Demand Instances to handle additional capacity.

Answer: D

Explanation:

The correct answer is **D: Use Reserved Instances for the baseline capacity and use On-Demand Instances to handle additional capacity.**

Here's why:

Reserved Instances (RIs): Reserved Instances provide significant cost savings (up to 75%) compared to On-Demand Instances, but require a commitment to a specific instance type and region for a term of one or three years. They are ideal for consistent, predictable workloads. In this scenario, the "baseline capacity" represents the minimum EC2 resources always needed to process the SQS queue messages, making RIs a cost-effective choice for this portion of the workload. (https://aws.amazon.com/ec2/pricing/reserved-instances/)

On-Demand Instances: On-Demand Instances allow you to pay for compute capacity by the hour or second (depending on the instance type and operating system) with no long-term commitments. They are well-suited for unpredictable workloads, spikes in traffic, or applications that need to scale quickly. The problem description states "unpredictable and often has intermittent traffic," implying the need for scalability to handle these message volume variations. Using On-Demand instances for additional capacity during peak times addresses this without requiring a long-term commitment. (https://aws.amazon.com/ec2/pricing/on-demand/)

Why not Spot Instances exclusively (A)? Spot Instances offer substantial cost savings (up to 90% compared to On-Demand), but they can be interrupted with little notice if the Spot price exceeds your bid. The problem states the need for "continually process messages without any downtime," which contradicts the potential for interruption with Spot Instances if used exclusively.

Why not Reserved Instances exclusively (B)? While RIs guarantee capacity and save money for the baseline workload, they might lead to over-provisioning if used exclusively to handle the maximum capacity required because the application only occasionally hits this maximum. This would result in unnecessary costs during periods of low traffic.

Why not Reserved Instances for baseline and Spot for additional capacity (C)? While potentially cheaper than pure On-Demand, using Spot Instances for handling unpredictable and intermittent traffic introduces the risk of interruptions. If the Spot price spikes and the instances are terminated, the application could experience downtime, violating the requirement to "continually process messages without any downtime." The small cost saving doesn't outweigh the risk to availabilty for a production workload that requires

continuous message processing.

Therefore, combining RIs for the steady-state baseline workload with On-Demand Instances for scaling during peaks provides the most cost-effective solution that also ensures continuous processing and avoids downtime.

Question: 168 **Exam Heist**

A security team wants to limit access to specific services or actions in all of the team's AWS accounts. All accounts belong to a large organization in AWS Organizations. The solution must be scalable and there must be a single point where permissions can be maintained.

What should a solutions architect do to accomplish this?

A. Create an ACL to provide access to the services or actions.
B. Create a security group to allow accounts and attach it to user groups.
C. Create cross-account roles in each account to deny access to the services or actions.
D. Create a service control policy in the root organizational unit to deny access to the services or actions.

Answer: D

Explanation:

The correct answer is D: Create a service control policy (SCP) in the root organizational unit (OU) to deny access to the services or actions. Here's why:

SCPs are the cornerstone of centralized permission management within AWS Organizations. They allow you to define guardrails that govern the maximum permissions available to accounts within an OU. By attaching an SCP to the root OU, the policy applies to all accounts in the organization, ensuring consistent access restrictions across all AWS accounts. This centralized approach fulfills the requirement of a single point of permission maintenance, promoting scalability and reducing administrative overhead.

Specifically, in this scenario, the security team wants to limit access. SCPs are ideal for denying permissions, effectively creating a boundary of permissible actions within the organization. This contrasts with Identity and Access Management (IAM) policies, which grant permissions. SCPs don't grant permissions; they restrict them.

Options A, B, and C are not suitable solutions. Access Control Lists (ACLs) are used to control network traffic, not IAM permissions. Security groups are also network-level controls and are not relevant to this scenario of limiting access to specific AWS services or actions. Creating cross-account roles in each account to deny access would be cumbersome, error-prone, and wouldn't scale effectively. It would also negate the single point of maintenance requirement.

Therefore, SCPs deployed at the root OU provide the most scalable and maintainable solution for implementing consistent access restrictions across an AWS Organization. This approach leverages the power of AWS Organizations to enforce organizational-wide governance.

For further research, consult the AWS documentation on SCPs:

AWS Organizations and service control policies (SCPs)
Service control policies (SCPs)

Question: 169 **Exam Heist**

A company is concerned about the security of its public web application due to recent web attacks. The application uses an Application Load Balancer (ALB). A solutions architect must reduce the risk of DDoS attacks against the application.

What should the solutions architect do to meet this requirement?

A. Add an Amazon Inspector agent to the ALB.
B. Configure Amazon Macie to prevent attacks.
C. Enable AWS Shield Advanced to prevent attacks.
D. Configure Amazon GuardDuty to monitor the ALB.

Answer: C

Explanation:

The correct answer is C: Enable AWS Shield Advanced to prevent attacks. Here's why:

The primary goal is to mitigate DDoS attacks against a public web application fronted by an Application Load Balancer (ALB). AWS Shield is a managed DDoS protection service that safeguards applications running on AWS.

AWS Shield Advanced provides enhanced DDoS protection capabilities beyond AWS Shield Standard, which is automatically enabled for all AWS customers. Shield Advanced offers more sophisticated detection and mitigation techniques tailored to the specific application's traffic patterns and infrastructure. It provides 24x7 access to the AWS Shield Response Team (SRT) who can assist during a DDoS event. Crucially, it offers cost protection against usage spikes during DDoS attacks.

Amazon Inspector (Option A) is a vulnerability management service that automates security assessments and identifies software vulnerabilities and unintended network exposure in EC2 instances and container images. It doesn't directly prevent or mitigate DDoS attacks against an ALB.

Amazon Macie (Option B) is a data security and data privacy service that uses machine learning and pattern matching to discover and protect sensitive data. It focuses on identifying and securing sensitive information, not preventing DDoS attacks.

Amazon GuardDuty (Option D) is a threat detection service that monitors your AWS accounts and workloads for malicious activity and unauthorized behavior. While GuardDuty can detect suspicious activity that might indicate a DDoS attack, it doesn't actively prevent or mitigate it. It's a detective control, not a preventative one.

Shield Advanced integrates seamlessly with ALBs and provides Layer 7 protection, which is critical for mitigating application-layer DDoS attacks that target specific URLs or API endpoints. Therefore, AWS Shield Advanced is the most appropriate solution to proactively reduce the risk of DDoS attacks against the company's web application.

Further Reading:

AWS Shield: https://aws.amazon.com/shield/
AWS Shield Advanced: https://aws.amazon.com/shield/advanced/

Question: 170 **Exam Heist**

A company's web application is running on Amazon EC2 instances behind an Application Load Balancer. The company recently changed its policy, which now requires the application to be accessed from one specific country only.

Which configuration will meet this requirement?

A. Configure the security group for the EC2 instances.
B. Configure the security group on the Application Load Balancer.
C. Configure AWS WAF on the Application Load Balancer in a VPC.
D. Configure the network ACL for the subnet that contains the EC2 instances.

Answer: C

Explanation:

The correct answer is C: Configure AWS WAF on the Application Load Balancer in a VPC.

Here's a detailed justification:

AWS WAF (Web Application Firewall) allows you to control access to your web applications based on specified rules. One of its key features is geographic filtering, enabling you to allow or block traffic originating from specific countries. By configuring AWS WAF on the Application Load Balancer (ALB), you can create a rule that permits traffic only from the required country. WAF operates at the application layer (Layer 7), providing granular control over HTTP/HTTPS requests.

Option A, configuring the security group for the EC2 instances, is less effective. While security groups provide basic ingress/egress filtering, they primarily operate on IP addresses and ports. Geolocation is not inherently supported within security groups. You would need a mechanism to constantly update the security group with IP address ranges for the desired country, which is a complex and unreliable approach due to the constantly changing IP address space.

Option B, configuring the security group on the ALB, suffers from the same limitations as Option A. Security groups cannot perform native geolocation filtering.

Option D, configuring the network ACL (NACL) for the subnet, is also insufficient. NACLs operate at the subnet level (Layer 3 and 4) and filter traffic based on IP addresses and ports. While it is possible to configure NACLs based on IP ranges, maintaining a current list of all IP ranges for a specific country would be challenging, complex, and not highly reliable, similar to the security group limitation. Also, NACLs don't have the application-layer inspection capabilities of WAF.

AWS WAF's Country-based rules offer a much simpler and more effective solution for geo-filtering. WAF integrates directly with the ALB and can readily identify the country of origin for incoming requests, allowing you to enforce the specified policy requirement. WAF rules also have the advantage of being easily updated and managed through the AWS Management Console or AWS CLI/SDK. Running the ALB in a VPC ensures a secure and isolated networking environment.

Further resources:

AWS WAF: https://aws.amazon.com/waf/
AWS WAF documentation: https://docs.aws.amazon.com/waf/latest/developerguide/what-is-aws-waf.html
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
Security Groups: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html
Network ACLs: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html

Question: 171 **Exam Heist**

A company provides an API to its users that automates inquiries for tax computations based on item prices. The company experiences a larger number of inquiries during the holiday season only that cause slower response times. A solutions architect needs to design a solution that is scalable and elastic.

What should the solutions architect do to accomplish this?

A. Provide an API hosted on an Amazon EC2 instance. The EC2 instance performs the required computations when the API request is made.

B. Design a REST API using Amazon API Gateway that accepts the item names. API Gateway passes item names to AWS Lambda for tax computations.

C. Create an Application Load Balancer that has two Amazon EC2 instances behind it. The EC2 instances will compute the tax on the received item names.

D. Design a REST API using Amazon API Gateway that connects with an API hosted on an Amazon EC2 instance. API Gateway accepts and passes the item names to the EC2 instance for tax computations.

Answer: B

Explanation:

The most appropriate solution is **B. Design a REST API using Amazon API Gateway that accepts the item names. API Gateway passes item names to AWS Lambda for tax computations.**

Here's why:

Scalability and Elasticity: API Gateway and Lambda are inherently designed for scalability and elasticity. API Gateway can handle a large number of concurrent requests and automatically scales to meet demand without any manual intervention. Lambda functions automatically scale by running copies of your function in parallel to handle each incoming request. During the holiday season, when the number of inquiries surges, these services can effortlessly handle the increased load.

Cost-Effectiveness: Lambda functions are priced based on the actual compute time consumed. This pay-per-use model is ideal for workloads with intermittent spikes in demand, such as during holiday seasons. You only pay for the compute time used to process the API requests, making it more cost-effective compared to running EC2 instances continuously.

Reduced Operational Overhead: Lambda eliminates the need to manage servers, apply patches, or perform capacity planning. API Gateway simplifies API management tasks such as authentication, authorization, rate limiting, and monitoring. This reduces the operational overhead associated with managing the API.

Better Performance: AWS Lambda runs your code in response to events and automatically manages the underlying compute resources for you, ensuring optimal performance.

Let's examine why the other options are less suitable:

A. Provide an API hosted on an Amazon EC2 instance: EC2 instances require manual scaling, which might not be fast enough to accommodate sudden spikes in demand. Also, even when idle, EC2 instances incur costs. EC2 instances do not provide the level of inherent scalability and elasticity that API Gateway and Lambda offer.

C. Create an Application Load Balancer that has two Amazon EC2 instances behind it: Similar to option A, EC2 instances behind an ALB require more management and cost more, especially during low demand periods. Load Balancing is a great step, but doesn't scale to zero.

D. Design a REST API using Amazon API Gateway that connects with an API hosted on an Amazon EC2 instance: This option introduces API Gateway but then still relies on an EC2 instance for computation. While API Gateway can handle the incoming traffic, the EC2 instance will become a bottleneck and require manual scaling. The benefits of API Gateway (e.g., security, rate limiting) are present, but the fundamental scalability issue remains.

In summary, API Gateway and Lambda provide the best combination of scalability, elasticity, cost-effectiveness, and reduced operational overhead, making option B the most suitable solution.

Authoritative Links:

AWS API Gateway: https://aws.amazon.com/api-gateway/
AWS Lambda: https://aws.amazon.com/lambda/

Question: 172 **Exam Heist**

A solutions architect is creating a new Amazon CloudFront distribution for an application. Some of the information submitted by users is sensitive. The application uses HTTPS but needs another layer of security. The sensitive information should.be protected throughout the entire application stack, and access to the information should be restricted to certain applications.

Which action should the solutions architect take?

A. Configure a CloudFront signed URL.

B. Configure a CloudFront signed cookie.

C. Configure a CloudFront field-level encryption profile.

D. Configure CloudFront and set the Origin Protocol Policy setting to HTTPS Only for the Viewer Protocol Policy.

Answer: C

Explanation:

The correct answer is C: Configure a CloudFront field-level encryption profile.

Here's a detailed justification:

The scenario requires protecting sensitive user data throughout the application stack and restricting access to it for specific applications. Field-level encryption (FLE) within CloudFront addresses this directly. FLE allows encrypting specific data fields at the edge (CloudFront) so that only the applications with the corresponding decryption key can decrypt and access the data. This ensures that even if the data is intercepted en route, it remains unreadable without the key.

Option A (CloudFront signed URLs) and Option B (CloudFront signed cookies) primarily control access to content, not the encryption of specific data fields within the content. Signed URLs/cookies are used for authentication and authorization, allowing only authenticated users access to the resources. They don't encrypt the data itself. While they enhance security by controlling who can access the application, they don't provide data-level encryption.

Option D (Origin Protocol Policy to HTTPS Only) ensures that the communication between CloudFront and the origin server is encrypted, which is a good security practice, but it doesn't encrypt the specific sensitive fields submitted by the users. It ensures data in transit between CloudFront and the origin is secure but doesn't restrict access to the data once it reaches the origin, nor does it prevent access to the data by all applications on the origin.

Field-level encryption is precisely designed to encrypt specific sensitive data at the edge before it even reaches the origin, offering end-to-end protection and restricting access to only the intended applications that possess the decryption key. It addresses the core requirements of the question regarding protecting sensitive user data and restricting application access.

For further research, consult these AWS resources:

AWS CloudFront Field-Level Encryption:
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/field-level-encryption.html
Using Signed URLs: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-signed-urls.html
Using Signed Cookies: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-

content-signed-cookies.html

Question: 173 **Exam Heist**

A gaming company hosts a browser-based application on AWS. The users of the application consume a large number of videos and images that are stored in Amazon S3. This content is the same for all users.

The application has increased in popularity, and millions of users worldwide accessing these media files. The company wants to provide the files to the users while reducing the load on the origin.

Which solution meets these requirements MOST cost-effectively?

A. Deploy an AWS Global Accelerator accelerator in front of the web servers.
B. Deploy an Amazon CloudFront web distribution in front of the S3 bucket.
C. Deploy an Amazon ElastiCache for Redis instance in front of the web servers.
D. Deploy an Amazon ElastiCache for Memcached instance in front of the web servers.

Answer: B

Explanation:

The correct answer is **B. Deploy an Amazon CloudFront web distribution in front of the S3 bucket.**

Here's a detailed justification:

The scenario describes a situation where static content (videos and images) from an S3 bucket is being heavily accessed globally, leading to potential load issues on the origin (S3). The primary goal is to reduce the load on S3 and provide low-latency access to users worldwide, while optimizing for cost.

CloudFront is a Content Delivery Network (CDN) service. A CDN caches content at edge locations distributed globally, which allows users to retrieve content from a server that's geographically closer to them. This significantly reduces latency and improves the user experience. By placing CloudFront in front of the S3 bucket, the most frequently accessed videos and images will be cached at CloudFront edge locations. When a user requests content, CloudFront will first check its cache. If the content is available (a "cache hit"), CloudFront will serve the content directly from the edge location, without hitting the S3 bucket. This drastically reduces the load on S3 and lowers data transfer costs because data is served from the closest location.

Option A (AWS Global Accelerator) is not the best choice for this scenario. While Global Accelerator also improves performance by routing user traffic to optimal endpoints, its primary focus is on accelerating dynamic content and improving the reliability of applications. It doesn't cache content like CloudFront, therefore won't reduce the load on S3 for static assets.

Options C (ElastiCache for Redis) and D (ElastiCache for Memcached) are in-memory data caching services. They are typically used to cache database query results or frequently accessed data within the application layer, not to serve static content directly from S3. They are more suited for improving the performance of the application logic rather than reducing the load on static content delivery from an object store like S3. Deploying them in front of web servers would cache data generated by the servers, not the S3 content.

Therefore, CloudFront is the most cost-effective solution because it directly addresses the issue of high traffic to S3 for static content, reduces latency through edge caching, and minimizes S3 data transfer costs.

Further research:

Amazon CloudFront: https://aws.amazon.com/cloudfront/
AWS Global Accelerator: https://aws.amazon.com/global-accelerator/

Amazon ElastiCache: https://aws.amazon.com/elasticache/

Question: 174 **Exam Heist**

A company has a multi-tier application that runs six front-end web servers in an Amazon EC2 Auto Scaling group in a single Availability Zone behind an Application Load Balancer (ALB). A solutions architect needs to modify the infrastructure to be highly available without modifying the application.

Which architecture should the solutions architect choose that provides high availability?

A. Create an Auto Scaling group that uses three instances across each of two Regions.

B. Modify the Auto Scaling group to use three instances across each of two Availability Zones.

C. Create an Auto Scaling template that can be used to quickly create more instances in another Region.

D. Change the ALB in front of the Amazon EC2 instances in a round-robin configuration to balance traffic to the web tier.

Answer: B

Explanation:

The correct answer is **B. Modify the Auto Scaling group to use three instances across each of two Availability Zones.**

High availability fundamentally means ensuring your application remains accessible even if a component fails. In the given scenario, the application currently resides within a single Availability Zone (AZ). If that AZ experiences an outage, the entire application becomes unavailable. To mitigate this risk, it's crucial to distribute the application across multiple AZs within the same AWS Region.

Option B achieves this by modifying the Auto Scaling group to span two AZs. This ensures that if one AZ fails, the remaining instances in the other AZ can continue to serve traffic, thus maintaining application availability. The Application Load Balancer (ALB) is inherently designed to distribute traffic across multiple targets (EC2 instances) registered with it, even if those targets are in different AZs within the same Region.

Let's examine why the other options are less suitable:

A. Create an Auto Scaling group that uses three instances across each of two Regions: This introduces complexity and potentially higher latency due to cross-region communication. While it offers disaster recovery capabilities, it's overkill for the high availability requirement stated in the question, which typically focuses on single-region resilience. Implementing cross-region Auto Scaling also requires mechanisms for data replication and traffic management across regions, adding to the complexity.

C. Create an Auto Scaling template that can be used to quickly create more instances in another Region: This only provides a mechanism for recovery after a failure. It doesn't inherently provide high availability because the application will still be unavailable until the new instances are provisioned in the other region. High availability implies continuous operation with minimal downtime.

D. Change the ALB in front of the Amazon EC2 instances in a round-robin configuration to balance traffic to the web tier: The ALB already balances traffic. Round-robin is a balancing algorithm that ALB supports, but changing the balancing configuration doesn't address the fundamental issue of single-AZ dependency. The ALB can't route traffic to instances that don't exist or are unavailable within another AZ. Furthermore, ALB configuration is not the focus of the question; the question asks about high availability achieved through instance distribution.

Therefore, the best approach is to modify the existing Auto Scaling group to distribute instances across multiple AZs, leveraging the ALB's built-in capabilities to distribute traffic among them. This provides the required high availability with the least amount of modification to the existing infrastructure.

Supporting Links:

AWS Auto Scaling: https://aws.amazon.com/autoscaling/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
AWS Regions and Availability Zones: https://aws.amazon.com/about-aws/global-infrastructure/

Question: 175 **Exam Heist**

An ecommerce company has an order-processing application that uses Amazon API Gateway and an AWS Lambda function. The application stores data in an Amazon Aurora PostgreSQL database. During a recent sales event, a sudden surge in customer orders occurred. Some customers experienced timeouts, and the application did not process the orders of those customers.

A solutions architect determined that the CPU utilization and memory utilization were high on the database because of a large number of open connections. The solutions architect needs to prevent the timeout errors while making the least possible changes to the application.

Which solution will meet these requirements?

A. Configure provisioned concurrency for the Lambda function. Modify the database to be a global database in multiple AWS Regions.

B. Use Amazon RDS Proxy to create a proxy for the database. Modify the Lambda function to use the RDS Proxy endpoint instead of the database endpoint.

C. Create a read replica for the database in a different AWS Region. Use query string parameters in API Gateway to route traffic to the read replica.

D. Migrate the data from Aurora PostgreSQL to Amazon DynamoDB by using AWS Database Migration Service (AWS DMS). Modify the Lambda function to use the DynamoDB table.

Answer: B

Explanation:

The correct answer is **B. Use Amazon RDS Proxy to create a proxy for the database. Modify the Lambda function to use the RDS Proxy endpoint instead of the database endpoint.**

Here's a detailed justification:

The problem is high CPU and memory utilization on the Aurora PostgreSQL database due to a large number of open connections from the Lambda function during peak load. This leads to timeouts for customers because the database cannot handle the connection surge.

Why RDS Proxy is the best solution:

Connection Pooling: RDS Proxy sits between the Lambda function and the database, effectively pooling database connections. Instead of the Lambda function directly opening and closing connections to the database for each invocation, it requests a connection from the RDS Proxy. The RDS Proxy maintains a pool of connections to the database and reuses them efficiently. This dramatically reduces the overhead on the database, freeing up CPU and memory resources.
Connection Multiplexing: RDS Proxy can multiplex connections from the Lambda function to the database, allowing a smaller number of database connections to serve a larger number of Lambda function invocations. This is particularly important in serverless architectures like Lambda, where frequent invocations can overwhelm the database with connection requests.
Reduced Database Load: By reducing the number of open connections, RDS Proxy alleviates the CPU and memory pressure on the database, preventing timeouts and improving overall application performance.
Minimal Code Changes: Using RDS Proxy requires minimal changes to the application. Only the database endpoint in the Lambda function needs to be updated to point to the RDS Proxy endpoint.

Least Disruptive: The RDS Proxy approach is the least disruptive to the existing architecture. It avoids major changes like migrating databases or introducing complex routing logic.

Why other options are less suitable:

A. Configure provisioned concurrency for the Lambda function. Modify the database to be a global database in multiple AWS Regions. Provisioned concurrency for Lambda addresses cold starts and helps with consistent performance, but it doesn't directly solve the database connection overload issue. A global database provides disaster recovery and low latency access to geographically distributed users, but it doesn't address the connection management problem. It also introduces more complexity and cost.

C. Create a read replica for the database in a different AWS Region. Use query string parameters in API Gateway to route traffic to the read replica. This solution involves using query string parameters to direct traffic to a read replica. While read replicas can offload read traffic, the described problem involves transactional order processing which requires write operations, which must still be routed to the primary database. Also, complex routing in API Gateway can increase latency and complexity.

D. Migrate the data from Aurora PostgreSQL to Amazon DynamoDB by using AWS Database Migration Service (AWS DMS). Modify the Lambda function to use the DynamoDB table. Migrating to DynamoDB is a significant architectural change. DynamoDB is suitable for different use cases. An RDBMS like Aurora Postgres is better for transactional workloads, and it introduces complexity and cost. DynamoDB is not suitable for this workload, and this option is a disproportionately complex solution.

Supporting Links:

Amazon RDS Proxy: https://aws.amazon.com/rds/proxy/
Using Amazon RDS Proxy with AWS Lambda: https://aws.amazon.com/blogs/database/using-amazon-rds-proxy-with-aws-lambda/

Question: 176 **Exam Heist**

An application runs on Amazon EC2 instances in private subnets. The application needs to access an Amazon DynamoDB table.

What is the MOST secure way to access the table while ensuring that the traffic does not leave the AWS network?

A. Use a VPC endpoint for DynamoDB.
B. Use a NAT gateway in a public subnet.
C. Use a NAT instance in a private subnet.
D. Use the internet gateway attached to the VPC.

Answer: A

Explanation:

The most secure way for EC2 instances in private subnets to access a DynamoDB table without the traffic leaving the AWS network is to use a VPC endpoint for DynamoDB (Option A).

Here's why:

VPC Endpoints: VPC endpoints enable private connectivity to supported AWS services, including DynamoDB, from within your VPC without requiring an internet gateway, NAT device, VPN connection, or AWS Direct Connect connection. They keep all traffic within the AWS network, enhancing security.
https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html

Security: VPC endpoints eliminate the need to route traffic through the public internet, mitigating exposure

to potential threats and vulnerabilities. They also support VPC endpoint policies, enabling granular control over which DynamoDB resources can be accessed by which principals.

Cost: While there's a cost associated with VPC endpoints (per Availability Zone), it is generally more cost-effective than using NAT gateways for high-volume traffic, especially when considering data transfer costs to and from NAT gateways.

NAT Gateway (Option B): A NAT gateway allows instances in a private subnet to connect to the internet or other AWS services, but it requires routing traffic through a public subnet. This approach is less secure than a VPC endpoint because it involves internet connectivity.

NAT Instance (Option C): A NAT instance serves a similar purpose to a NAT gateway but requires manual management and configuration, making it less reliable and more complex. It also introduces a single point of failure and can become a performance bottleneck. Furthermore, it requires traffic to traverse to the public internet.

Internet Gateway (Option D): An internet gateway allows instances in the VPC to access the internet. However, it exposes the instances to the public internet, making it the least secure option.

Therefore, utilizing a VPC endpoint for DynamoDB is the preferred and most secure method because it maintains network traffic within the AWS infrastructure, reduces the attack surface, and offers granular access control.

Here is a breakdown of why other options aren't secure:

B, C, D Routing traffic through the public internet, which violates the requirement of keeping traffic within AWS.

Question: 177 **Exam Heist**

An entertainment company is using Amazon DynamoDB to store media metadata. The application is read intensive and experiencing delays. The company does not have staff to handle additional operational overhead and needs to improve the performance efficiency of DynamoDB without reconfiguring the application.

What should a solutions architect recommend to meet this requirement?

A. Use Amazon ElastiCache for Redis.
B. Use Amazon DynamoDB Accelerator (DAX).
C. Replicate data by using DynamoDB global tables.
D. Use Amazon ElastiCache for Memcached with Auto Discovery enabled.

Answer: B

Explanation:

The correct answer is **B. Use Amazon DynamoDB Accelerator (DAX).**

DynamoDB Accelerator (DAX) is a fully managed, highly available, in-memory cache for DynamoDB. It's specifically designed to improve read performance for read-heavy workloads, which directly addresses the problem described. DAX integrates seamlessly with existing DynamoDB applications without requiring code changes, fulfilling the requirement to avoid application reconfiguration. It significantly reduces read latency (often from milliseconds to microseconds) by caching frequently accessed data, which will alleviate the observed delays. Importantly, DAX offloads read traffic from the DynamoDB tables, improving overall performance and reducing pressure on the DynamoDB infrastructure, and because it's a managed service, it minimizes operational overhead.

Option A, ElastiCache for Redis, while a powerful caching solution, requires application-level modifications to integrate with DynamoDB. This violates the requirement to avoid reconfiguring the application. Similarly, option D, ElastiCache for Memcached, also necessitates application changes and might require auto-discovery configuration, adding operational complexity.

Option C, DynamoDB global tables, provides multi-region replication for availability and disaster recovery, not primarily for improving read performance within a single region. While replication could indirectly improve read performance in certain multi-region scenarios, it introduces significantly more operational overhead and complexity compared to DAX, and does not directly address the read latency issues in the single region the company uses. Global tables also don't solve the latency issue within a specific region and primarily focus on data redundancy and availability across geographically dispersed locations.

Therefore, DAX is the most suitable solution because it's a managed, in-memory cache specifically designed for DynamoDB that enhances read performance without application changes or increased operational burden.

Further research:

Amazon DynamoDB Accelerator (DAX): https://aws.amazon.com/dynamodb/dax/
DAX Use Cases: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/DAX.html

Question: 178 **Exam Heist**

A company's infrastructure consists of Amazon EC2 instances and an Amazon RDS DB instance in a single AWS Region. The company wants to back up its data in a separate Region.

Which solution will meet these requirements with the LEAST operational overhead?

A. Use AWS Backup to copy EC2 backups and RDS backups to the separate Region.

B. Use Amazon Data Lifecycle Manager (Amazon DLM) to copy EC2 backups and RDS backups to the separate Region.

C. Create Amazon Machine Images (AMIs) of the EC2 instances. Copy the AMIs to the separate Region. Create a read replica for the RDS DB instance in the separate Region.

D. Create Amazon Elastic Block Store (Amazon EBS) snapshots. Copy the EBS snapshots to the separate Region. Create RDS snapshots. Export the RDS snapshots to Amazon S3. Configure S3 Cross-Region Replication (CRR) to the separate Region.

Answer: A

Explanation:

The correct answer is A: Use AWS Backup to copy EC2 backups and RDS backups to the separate Region.

Here's why:

AWS Backup offers a centralized and automated way to manage backups across multiple AWS services, including EC2 and RDS, simplifying backup management and reducing operational overhead. It allows you to define backup policies and schedules centrally, and then apply them to your resources. A key feature is its ability to copy backups across regions, satisfying the requirement of backing up data in a separate region. This eliminates the need for service-specific backup solutions and scripting.

Option B, using Amazon Data Lifecycle Manager (Amazon DLM), primarily manages the lifecycle of EBS snapshots. While DLM can create EBS snapshots, it doesn't directly handle RDS backups or cross-region backup copying. Thus, this approach requires additional configuration and scripting for the RDS component, increasing operational overhead.

Option C, creating AMIs and RDS read replicas, meets the DR requirements, but it isn't a backup solution.

AMIs primarily capture the state of EC2 instances for disaster recovery or instance replication, and read replicas are for read scaling and disaster recovery. This doesn't provide point-in-time backup capabilities for both services and involves more operational overhead to maintain the replicas and keep them in sync.

Option D, creating EBS and RDS snapshots and using S3 CRR, is more complex and time-consuming than using AWS Backup. It requires configuring snapshot schedules for EC2 using EBS snapshots, RDS snapshots, exporting RDS snapshots to S3, and then setting up S3 Cross-Region Replication. Also managing snapshots, configuring CRR, and ensuring consistent backups are all operational overheads to avoid.

AWS Backup is the most streamlined solution because it's designed for this specific purpose: centralized backup management across different AWS services, including cross-region copying.

Supporting Links:

AWS Backup: https://aws.amazon.com/backup/
Amazon RDS Backups:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_WorkingWithAutomatedBackups.html
Amazon EC2 Backups: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EBSSnapshots.html

Question: 179 **Exam Heist**

A solutions architect needs to securely store a database user name and password that an application uses to access an Amazon RDS DB instance. The application that accesses the database runs on an Amazon EC2 instance. The solutions architect wants to create a secure parameter in AWS Systems Manager Parameter Store.

What should the solutions architect do to meet this requirement?

A. Create an IAM role that has read access to the Parameter Store parameter. Allow Decrypt access to an AWS Key Management Service (AWS KMS) key that is used to encrypt the parameter. Assign this IAM role to the EC2 instance.

B. Create an IAM policy that allows read access to the Parameter Store parameter. Allow Decrypt access to an AWS Key Management Service (AWS KMS) key that is used to encrypt the parameter. Assign this IAM policy to the EC2 instance.

C. Create an IAM trust relationship between the Parameter Store parameter and the EC2 instance. Specify Amazon RDS as a principal in the trust policy.

D. Create an IAM trust relationship between the DB instance and the EC2 instance. Specify Systems Manager as a principal in the trust policy.

Answer: A

Explanation:

The correct answer is A. Here's why:

The scenario requires securely storing database credentials within Systems Manager Parameter Store and allowing an EC2 instance to access them. IAM roles are the preferred method for granting permissions to EC2 instances, as they provide temporary credentials and are more secure than using long-term access keys.

Option A correctly leverages this. It creates an IAM role (not just a policy) that grants ssm:GetParameter (read access) permission to the specific parameter within Parameter Store. Crucially, it also allows kms:Decrypt permission to the KMS key used to encrypt the secure string parameter. Since the parameter is stored as a secure string (encrypted), the EC2 instance needs permission to decrypt it. Assigning this IAM role to the EC2 instance allows the EC2 instance's applications to retrieve and decrypt the database credentials.

Option B is incorrect because while policies are essential, they must be attached to roles for EC2 instances. Attaching a policy directly is not the proper way to grant permissions to EC2 instances, as roles provide the

necessary temporary credentials.

Options C and D are incorrect because IAM trust relationships are used to allow entities in one AWS account to assume roles in another account or for services to assume roles. They are not directly used for granting access to Parameter Store or RDS instances from an EC2 instance within the same account. Specifying Amazon RDS or Systems Manager as principals in a trust policy related to an EC2 instance is conceptually flawed in this context. The communication is EC2 -> SSM Parameter Store, not EC2 being "trusted" by SSM or RDS.

Therefore, option A provides the most secure and correct way to grant the necessary permissions to the EC2 instance to access the encrypted credentials stored in Parameter Store.

Supporting links:

IAM Roles: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html
Systems Manager Parameter Store: https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html
KMS Encryption: https://docs.aws.amazon.com/kms/latest/developerguide/overview.html
Granting Applications Access to Secrets: https://aws.amazon.com/blogs/security/how-to-manage-secrets-for-amazon-ec2-using-aws-systems-manager-parameter-store/

Question: 180 **Exam Heist**

A company is designing a cloud communications platform that is driven by APIs. The application is hosted on Amazon EC2 instances behind a Network Load Balancer (NLB). The company uses Amazon API Gateway to provide external users with access to the application through APIs. The company wants to protect the platform against web exploits like SQL injection and also wants to detect and mitigate large, sophisticated DDoS attacks.

Which combination of solutions provides the MOST protection? (Choose two.)

A. Use AWS WAF to protect the NLB.
B. Use AWS Shield Advanced with the NLB.
C. Use AWS WAF to protect Amazon API Gateway.
D. Use Amazon GuardDuty with AWS Shield Standard
E. Use AWS Shield Standard with Amazon API Gateway.

Answer: BC

Explanation:

Here's a detailed justification for why options B and C (AWS Shield Advanced with the NLB and AWS WAF to protect Amazon API Gateway) provide the most protection in this scenario:

Understanding the Threats: The question highlights two primary threats: web exploits (like SQL injection) and DDoS attacks. Web exploits target the application layer (Layer 7), attempting to manipulate the application logic or data. DDoS attacks aim to overwhelm the infrastructure, making the application unavailable.

AWS WAF for Application Layer Protection (API Gateway): AWS WAF (Web Application Firewall) is specifically designed to protect web applications from common web exploits by inspecting HTTP traffic. Placing WAF in front of Amazon API Gateway (option C) allows it to inspect incoming requests for malicious patterns (e.g., SQL injection attempts, cross-site scripting). API Gateway acts as the entry point to the application APIs, making it a prime target for such attacks. https://aws.amazon.com/waf/

AWS Shield Advanced for DDoS Mitigation (NLB): AWS Shield provides DDoS protection. Standard Shield is automatically enabled and free, but offers basic protection. Shield Advanced (option B) provides more

sophisticated detection and mitigation capabilities, including 24/7 access to the AWS DDoS Response Team (DRT) during attacks. Protecting the Network Load Balancer (NLB) with Shield Advanced is crucial because the NLB distributes incoming traffic to the EC2 instances hosting the application. By protecting the NLB, you protect the entire infrastructure from being overwhelmed by DDoS attacks. Shield Advanced is designed to protect resources like NLBs, Elastic Load Balancers (ELB), and CloudFront distributions.
https://aws.amazon.com/shield/

Why other options are less ideal:

A: Using AWS WAF with the NLB offers some protection but less strategically since the API Gateway sits in front of it. The API Gateway is where the actual API requests are handled and exposed, making it the prime target.
D: GuardDuty is a threat detection service that monitors your AWS environment for malicious activity. It's helpful for security monitoring and alerting, but it doesn't directly mitigate DDoS attacks or prevent web exploits like WAF and Shield Advanced do. Shield Standard provides basic DDoS protection but not as granular or tailored as Shield Advanced.
E: Shield Standard is a base service, while API Gateway is a prime location for using WAF.

Comprehensive Protection: By combining WAF at the API Gateway level with Shield Advanced protecting the NLB, the company achieves a layered security approach. WAF handles application-layer attacks, while Shield Advanced handles network-layer DDoS attacks. This combination offers the most robust protection for the cloud communications platform.

Question: 181 **Exam Heist**

A company has a legacy data processing application that runs on Amazon EC2 instances. Data is processed sequentially, but the order of results does not matter. The application uses a monolithic architecture. The only way that the company can scale the application to meet increased demand is to increase the size of the instances.

The company's developers have decided to rewrite the application to use a microservices architecture on Amazon Elastic Container Service (Amazon ECS).

What should a solutions architect recommend for communication between the microservices?

A. Create an Amazon Simple Queue Service (Amazon SQS) queue. Add code to the data producers, and send data to the queue. Add code to the data consumers to process data from the queue.

B. Create an Amazon Simple Notification Service (Amazon SNS) topic. Add code to the data producers, and publish notifications to the topic. Add code to the data consumers to subscribe to the topic.

C. Create an AWS Lambda function to pass messages. Add code to the data producers to call the Lambda function with a data object. Add code to the data consumers to receive a data object that is passed from the Lambda function.

D. Create an Amazon DynamoDB table. Enable DynamoDB Streams. Add code to the data producers to insert data into the table. Add code to the data consumers to use the DynamoDB Streams API to detect new table entries and retrieve the data.

Answer: A

Explanation:

Here's a detailed justification for why option A is the best choice, and why the other options are less suitable for inter-microservice communication in this scenario:

Justification for Option A (Amazon SQS):

Amazon Simple Queue Service (SQS) is a fully managed message queuing service that allows you to decouple and scale microservices, distributed systems, and serverless applications. In this scenario, SQS provides

asynchronous communication between the microservices performing the data processing. Producers (microservices generating data) can send data to the queue without needing to know who the consumers are. Consumers (microservices processing data) can independently pull messages from the queue and process them at their own pace. This decoupling allows the system to handle varying loads efficiently. Since the order of the results doesn't matter, SQS Standard Queues are a suitable choice. If the application required strictly first-in, first-out processing, SQS FIFO Queues could be considered. SQS inherently provides message buffering, which is crucial in handling situations where producers generate data faster than consumers can process it. The queue acts as a buffer, preventing data loss and allowing the system to gracefully handle spikes in demand. The "at-least-once" delivery guarantee of SQS ensures that each message is processed.

Why other options are not ideal:

Option B (Amazon SNS): Amazon Simple Notification Service (SNS) is primarily designed for push-based notifications to multiple subscribers. While it can be used for microservice communication, it's better suited for fan-out scenarios where one producer needs to notify many consumers. In this case, the data processing is sequential (producer generates, consumer processes), so a queue-based approach (SQS) is more appropriate. SNS would also require each consumer to subscribe to the topic and would add unnecessary complexity if the intention is solely to process data. It's designed for notifications, not general purpose asynchronous processing of data.

Option C (AWS Lambda): Using Lambda to directly pass messages would create synchronous dependencies between the microservices and limit scalability. Lambda functions have execution time limits and might not be suitable for handling potentially large or long-running data processing tasks. This approach would also introduce unnecessary overhead and complexity compared to a simple queue. Furthermore, the service making the request would need to wait for Lambda to execute and pass on the response, defeating the purpose of asynchronous processing and decoupling.

Option D (Amazon DynamoDB Streams): DynamoDB Streams are primarily designed for triggering actions based on data changes in a DynamoDB table. While it can be used for inter-microservice communication, it's not its core purpose. It would introduce unnecessary complexity and overhead, as you'd have to treat the DynamoDB table as a message queue. It tightly couples the microservices to DynamoDB. DynamoDB Streams are better suited for things like auditing changes in a database, or triggering related actions based on specific data modifications. This approach is generally avoided unless you need to persist the data to a database and trigger some kind of action.

In conclusion: SQS provides a simple, scalable, and cost-effective solution for asynchronous communication between microservices in a data processing application, aligning perfectly with the company's goal of decoupling its legacy monolithic application and leveraging a microservices architecture.

Authoritative Links:

Amazon SQS: https://aws.amazon.com/sqs/
Microservices architecture on AWS: https://aws.amazon.com/microservices/
Decoupling microservices using SQS: https://aws.amazon.com/blogs/compute/decoupling-applications-with-amazon-sqs-message-queues/

Question: 182 **Exam Heist**

A company wants to migrate its MySQL database from on premises to AWS. The company recently experienced a database outage that significantly impacted the business. To ensure this does not happen again, the company wants a reliable database solution on AWS that minimizes data loss and stores every transaction on at least two nodes.

Which solution meets these requirements?

A. Create an Amazon RDS DB instance with synchronous replication to three nodes in three Availability Zones.

B. Create an Amazon RDS MySQL DB instance with Multi-AZ functionality enabled to synchronously replicate the data.

C. Create an Amazon RDS MySQL DB instance and then create a read replica in a separate AWS Region that synchronously replicates the data.

D. Create an Amazon EC2 instance with a MySQL engine installed that triggers an AWS Lambda function to synchronously replicate the data to an Amazon RDS MySQL DB instance.

Answer: B

Explanation:

The correct answer is B. Let's break down why:

The core requirements are high reliability, minimal data loss, and synchronous data replication on at least two nodes within AWS.

Option A (RDS with synchronous replication to three nodes in three AZs): While seemingly good, standard RDS doesn't natively offer synchronous replication to three nodes across three AZs. Multi-AZ provides synchronous replication between two nodes. While MySQL can be configured to replicate to multiple read replicas synchronously, doing so requires significant configuration beyond just creating an RDS instance, and isn't typically the best approach for basic high availability within a single region.

Option B (RDS MySQL with Multi-AZ): This precisely addresses the requirements. RDS Multi-AZ provides a synchronous, automated failover mechanism. Data is synchronously replicated between the primary instance and a standby instance in a different Availability Zone. In case of failure, the standby instance automatically takes over with minimal data loss. This configuration stores every transaction on at least two nodes.

Option C (RDS MySQL with a cross-Region read replica): Cross-region read replicas are asynchronous. While they improve disaster recovery capabilities, they do not provide the required synchronous data replication. Therefore, this option does not meet the criteria of minimizing data loss due to its asynchronous nature. Cross-Region replication has latency which goes against the need to minimize data loss.

Option D (EC2-based MySQL with Lambda-triggered replication to RDS): This solution is overly complex and introduces significant overhead. Manually managing replication using Lambda functions is not an efficient or reliable solution compared to RDS's built-in Multi-AZ feature. It also has the burden of needing to manage the EC2 instance, OS patching, etc. Synchronous replication implemented in this manner also isn't guaranteed without significant custom coding and testing, and would be prone to errors. This introduces a very high level of complexity.

In summary, RDS Multi-AZ provides a managed, reliable, and synchronous replication solution, fulfilling the company's requirements for high availability and minimal data loss. It is also the simplest and most cost-effective option.

Supporting Links:

Amazon RDS Multi-AZ: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html
Amazon RDS Read Replicas:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.ReadReplicas.html

Question: 183 **Exam Heist**

A company is building a new dynamic ordering website. The company wants to minimize server maintenance and patching. The website must be highly available and must scale read and write capacity as quickly as possible to meet changes in user demand.

Which solution will meet these requirements?

A. Host static content in Amazon S3. Host dynamic content by using Amazon API Gateway and AWS Lambda. Use Amazon DynamoDB with on-demand capacity for the database. Configure Amazon CloudFront to deliver the website content.

B. Host static content in Amazon S3. Host dynamic content by using Amazon API Gateway and AWS Lambda. Use Amazon Aurora with Aurora Auto Scaling for the database. Configure Amazon CloudFront to deliver the website content.

C. Host all the website content on Amazon EC2 instances. Create an Auto Scaling group to scale the EC2 instances. Use an Application Load Balancer to distribute traffic. Use Amazon DynamoDB with provisioned write capacity for the database.

D. Host all the website content on Amazon EC2 instances. Create an Auto Scaling group to scale the EC2 instances. Use an Application Load Balancer to distribute traffic. Use Amazon Aurora with Aurora Auto Scaling for the database.

Answer: A

Explanation:

The correct answer is A. Here's why:

Option A offers a serverless architecture which directly addresses the requirement to minimize server maintenance and patching. Amazon S3 is used for static content hosting, eliminating the need to manage servers for these assets. Amazon API Gateway and AWS Lambda handle dynamic content, abstracting away the underlying infrastructure management. DynamoDB with on-demand capacity is a key component, providing automatic scaling for read and write operations without the need for manual capacity planning, fulfilling the rapid scaling requirement. CloudFront ensures global content delivery with low latency and high availability.

Option B uses Amazon Aurora with Auto Scaling. While Aurora Auto Scaling is good, it involves managing relational databases, which incurs operational overhead and database maintenance. This contradicts the requirement to minimize server maintenance. Additionally, DynamoDB offers faster, more predictable scaling than Aurora for dynamic workloads.

Options C and D rely on EC2 instances for hosting content. This introduces significant server management overhead for patching, scaling, and maintenance. Using EC2 Auto Scaling helps, but it does not eliminate the underlying server management burden, directly contradicting the prompt.

Therefore, Option A offers the most serverless, scalable, and maintainable architecture aligning with the requirements. DynamoDB with on-demand capacity is optimal for unpredictable workloads needing immediate scaling.

Key Concepts:

Serverless Computing: API Gateway, Lambda, and DynamoDB abstract away server management.
Scalability: DynamoDB on-demand capacity scales automatically.
High Availability: S3, CloudFront, API Gateway, Lambda and DynamoDB inherently support high availability.

Supporting Links:

Amazon S3
Amazon API Gateway
AWS Lambda
Amazon DynamoDB
Amazon CloudFront

Question: 184 **Exam Heist**

A company has an AWS account used for software engineering. The AWS account has access to the company's on-premises data center through a pair of AWS Direct Connect connections. All non-VPC traffic routes to the virtual private gateway.

A development team recently created an AWS Lambda function through the console. The development team needs to allow the function to access a database that runs in a private subnet in the company's data center.

Which solution will meet these requirements?

A. Configure the Lambda function to run in the VPC with the appropriate security group.

B. Set up a VPN connection from AWS to the data center. Route the traffic from the Lambda function through the VPN.

C. Update the route tables in the VPC to allow the Lambda function to access the on-premises data center through Direct Connect.

D. Create an Elastic IP address. Configure the Lambda function to send traffic through the Elastic IP address without an elastic network interface.

Answer: A

Explanation:

The correct answer is A. Here's a detailed justification:

The core requirement is to allow a Lambda function to access a database located in a private subnet within the company's on-premises data center, which is connected to AWS via Direct Connect. Lambda functions, by default, run in a secure, isolated environment outside your VPC. To access resources within a VPC or connected on-premises networks, the Lambda function must be configured to run within the VPC.

Option A directly addresses this. Configuring the Lambda function to run within the VPC allows it to leverage the existing Direct Connect connection to reach the on-premises database. By associating the Lambda function with a security group that permits outbound traffic to the database's port and IP address range, and ensuring the VPC's route tables direct traffic destined for the on-premises network to the virtual private gateway (VGW) associated with the Direct Connect connection, the Lambda function can successfully communicate with the database.

Option B is incorrect because establishing a VPN connection in addition to the existing Direct Connect adds unnecessary complexity and cost. Direct Connect already provides a dedicated, private network connection.

Option C, while seemingly relevant, misses a crucial step. Simply updating route tables isn't sufficient. The Lambda function must be running within the VPC to utilize those route tables. Lambda functions executing outside a VPC cannot be governed by VPC route tables.

Option D is incorrect because Lambda functions do not directly use Elastic IP addresses (EIPs). EIPs are static IPv4 addresses designed for instances and network interfaces to maintain a consistent public IP. Lambda functions run within the AWS managed infrastructure and do not have dedicated EIPs assigned to them. Configuring the Lambda function to use an EIP without an ENI is not possible or how Lambda utilizes IP addressing.

In summary, running the Lambda function within the VPC (Option A) is the most direct, cost-effective, and secure solution. It leverages the existing Direct Connect connection and allows the function to access on-premises resources by associating it with the appropriate VPC, subnet, and security group, while ensuring proper routing.

Further research:

AWS Lambda VPC Networking: https://docs.aws.amazon.com/lambda/latest/dg/configuration-vpc.html

AWS Direct Connect: https://aws.amazon.com/directconnect/
AWS Security Groups: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html

Question: 185 **Exam Heist**

A company runs an application using Amazon ECS. The application creates resized versions of an original image and then makes Amazon S3 API calls to store the resized images in Amazon S3.

How can a solutions architect ensure that the application has permission to access Amazon S3?

A. Update the S3 role in AWS IAM to allow read/write access from Amazon ECS, and then relaunch the container.

B. Create an IAM role with S3 permissions, and then specify that role as the taskRoleArn in the task definition.

C. Create a security group that allows access from Amazon ECS to Amazon S3, and update the launch configuration used by the ECS cluster.

D. Create an IAM user with S3 permissions, and then relaunch the Amazon EC2 instances for the ECS cluster while logged in as this account.

Answer: B

Explanation:

The correct answer is **B. Create an IAM role with S3 permissions, and then specify that role as the taskRoleArn in the task definition.**

This is the most secure and recommended method for granting ECS tasks access to AWS services like S3. IAM roles provide temporary security credentials to applications running on AWS, eliminating the need to embed or manage long-term credentials within the application or containers.

Here's why the other options are incorrect:

A. Update the S3 role in AWS IAM to allow read/write access from Amazon ECS, and then relaunch the container: S3 buckets don't have IAM roles. IAM roles are assigned to entities (like ECS tasks or EC2 instances) that need to access S3. While bucket policies control who can access the bucket itself, they don't grant permissions to specific tasks.
C. Create a security group that allows access from Amazon ECS to Amazon S3, and update the launch configuration used by the ECS cluster: Security groups control network traffic, not IAM permissions. S3 access is controlled by IAM policies, not network rules. Security groups are important for allowing your ECS tasks to communicate with other resources (like databases), but they are not relevant for S3 API authorization.
D. Create an IAM user with S3 permissions, and then relaunch the Amazon EC2 instances for the ECS cluster while logged in as this account: Using an IAM user for this purpose introduces the risk of hardcoding credentials within your EC2 instances, which is a security anti-pattern. Furthermore, "logging in" as an IAM user on the EC2 instance doesn't automatically grant your ECS tasks those permissions.

The taskRoleArn parameter in the ECS task definition specifies the IAM role that ECS will use to provide credentials to the containers running within that task. When the application makes an S3 API call, the AWS SDK automatically retrieves temporary credentials from the IAM role associated with the task. These credentials are then used to authenticate and authorize the request to S3, ensuring that only authorized applications can access S3 resources. The Task Role offers better security because the ECS agent manages the retrieval and rotation of temporary credentials, so you don't have to manage credentials yourself.This practice adheres to the principle of least privilege, ensuring that only the required permissions are granted to the application for accessing specific resources.

AWS Documentation - IAM Roles for TasksAWS Security Best Practices

Question: 186 **Exam Heist**

A company has a Windows-based application that must be migrated to AWS. The application requires the use of a shared Windows file system attached to multiple Amazon EC2 Windows instances that are deployed across multiple Availability Zone:

What should a solutions architect do to meet this requirement?

A. Configure AWS Storage Gateway in volume gateway mode. Mount the volume to each Windows instance.

B. Configure Amazon FSx for Windows File Server. Mount the Amazon FSx file system to each Windows instance.

C. Configure a file system by using Amazon Elastic File System (Amazon EFS). Mount the EFS file system to each Windows instance.

D. Configure an Amazon Elastic Block Store (Amazon EBS) volume with the required size. Attach each EC2 instance to the volume. Mount the file system within the volume to each Windows instance.

Answer: B

Explanation:

The correct answer is **B. Configure Amazon FSx for Windows File Server. Mount the Amazon FSx file system to each Windows instance.**

Here's a detailed justification:

The requirement is to provide a shared Windows file system across multiple EC2 Windows instances in multiple Availability Zones. This implies the need for a network file system accessible by all instances simultaneously.

Amazon FSx for Windows File Server is a fully managed Windows file system built on Windows Server. It supports the SMB protocol natively, which is the standard file-sharing protocol for Windows environments. This makes it easy to integrate with existing Windows applications. Moreover, it's designed for multi-AZ deployments, ensuring high availability and durability, which perfectly aligns with the requirement of instances residing across multiple Availability Zones.
AWS Storage Gateway (volume gateway mode) primarily provides block storage volumes and is not designed for native file sharing across multiple instances simultaneously like a network file system. It presents iSCSI targets to EC2 instances.
Amazon EFS (Elastic File System) is a fully managed NFS (Network File System) designed for Linux-based workloads. While it offers shared storage, it is not optimized or native to the Windows environment and does not natively support SMB, which is critical for Windows-based applications. While technically possible to make it work with workarounds, it's far from ideal.
Amazon EBS (Elastic Block Store) volumes are block storage devices attached to a single EC2 instance. EBS volumes cannot be simultaneously attached to multiple EC2 instances, making this option unsuitable for shared file system needs across multiple instances. While EBS Multi-Attach exists, it's mainly intended for clustered applications with specific write coordination.

Therefore, Amazon FSx for Windows File Server is the most appropriate service because it directly addresses the need for a shared, highly available, and fully managed Windows file system, fitting seamlessly into a Windows-based environment deployed across multiple Availability Zones.

Supporting documentation:

Amazon FSx for Windows File Server
Compare AWS Storage Options

Question: 187 **Exam Heist**

A company is developing an ecommerce application that will consist of a load-balanced front end, a container-based application, and a relational database. A solutions architect needs to create a highly available solution that operates with as little manual intervention as possible.

Which solutions meet these requirements? (Choose two.)

A. Create an Amazon RDS DB instance in Multi-AZ mode.

B. Create an Amazon RDS DB instance and one or more replicas in another Availability Zone.

C. Create an Amazon EC2 instance-based Docker cluster to handle the dynamic application load.

D. Create an Amazon Elastic Container Service (Amazon ECS) cluster with a Fargate launch type to handle the dynamic application load.

E. Create an Amazon Elastic Container Service (Amazon ECS) cluster with an Amazon EC2 launch type to handle the dynamic application load.

Answer: AD

Explanation:

The question focuses on creating a highly available, low-maintenance e-commerce application spanning load balancing, containers, and a relational database. The optimal solutions prioritize automated failover and simplified management.

Option A is correct because running Amazon RDS in Multi-AZ mode provides automatic failover to a standby instance in another Availability Zone in case of a primary instance failure. This minimizes manual intervention and ensures high availability for the database layer, directly addressing the prompt's requirements. [https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html]

Option D is also correct because Amazon ECS with the Fargate launch type abstracts away the underlying infrastructure management. Fargate handles scaling, patching, and provisioning the compute resources for the containers, significantly reducing operational overhead and manual intervention. It also promotes high availability by distributing containers across multiple Availability Zones. [https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html]

Option B is incorrect because while read replicas can improve performance, they do not provide automatic failover for the primary database instance. Manual intervention would be needed to promote a read replica to the primary instance in case of failure.

Option C is incorrect because managing an EC2-based Docker cluster requires significant manual effort for tasks such as instance management, scaling, patching, and orchestration. This contradicts the requirement for minimal manual intervention.

Option E is incorrect for similar reasons to Option C. While ECS simplifies container orchestration compared to a raw Docker cluster, using the EC2 launch type still requires managing the underlying EC2 instances, including scaling, patching, and maintenance. This adds operational overhead and manual intervention.

Question: 188 **Exam Heist**

A company uses Amazon S3 as its data lake. The company has a new partner that must use SFTP to upload data files. A solutions architect needs to implement a highly available SFTP solution that minimizes operational overhead.

Which solution will meet these requirements?

A. Use AWS Transfer Family to configure an SFTP-enabled server with a publicly accessible endpoint. Choose the S3 data lake as the destination.

B. Use Amazon S3 File Gateway as an SFTP server. Expose the S3 File Gateway endpoint URL to the new partner. Share the S3 File Gateway endpoint with the new partner.

C. Launch an Amazon EC2 instance in a private subnet in a VPInstruct the new partner to upload files to the EC2 instance by using a VPN. Run a cron job script, on the EC2 instance to upload files to the S3 data lake.

D. Launch Amazon EC2 instances in a private subnet in a VPC. Place a Network Load Balancer (NLB) in front of the EC2 instances. Create an SFTP listener port for the NLB. Share the NLB hostname with the new partner. Run a cron job script on the EC2 instances to upload files to the S3 data lake.

Answer: A

Explanation:

The correct answer is A, utilizing AWS Transfer Family for a secure and scalable SFTP solution directly integrated with S3.

Here's why:

AWS Transfer Family is specifically designed for secure file transfers into and out of AWS storage services like S3. It natively supports SFTP, FTP, and FTPS protocols.
Minimizes Operational Overhead: Transfer Family is a fully managed service. This means AWS handles the underlying infrastructure, patching, scaling, and high availability. This drastically reduces the operational burden on the company compared to managing EC2 instances.
High Availability: Transfer Family provides built-in high availability. You don't need to configure and maintain load balancers or redundant EC2 instances.
Direct S3 Integration: Transfer Family can directly integrate with the S3 data lake. Files uploaded via SFTP are automatically stored in the designated S3 bucket.
Security: Transfer Family supports various authentication methods and encryption to ensure secure data transfers. The publicly accessible endpoint allows partners to connect easily, while security is maintained through authentication and authorization policies.
Option B (S3 File Gateway): S3 File Gateway is intended for on-premises applications to access S3, not typically for external partners using SFTP. It also involves more local infrastructure management.
Options C and D (EC2 Instances): Using EC2 instances, even with VPNs and Network Load Balancers (NLB), requires significant manual configuration, patching, scaling, and monitoring. This defeats the requirement of minimizing operational overhead. These options are complex and less secure without proper hardening. They also necessitate managing cron jobs and custom scripts for data transfer, adding to operational burden. They do not offer inherent high availability without additional configuration.

In summary, AWS Transfer Family offers the most straightforward, highly available, secure, and operationally efficient solution for providing SFTP access to an S3 data lake for external partners.

Authoritative Links:

AWS Transfer Family: https://aws.amazon.com/transfer/
AWS Transfer Family Documentation: https://docs.aws.amazon.com/transfer/latest/userguide/what-is-transfer.html

Question: 189 **Exam Heist**

A company needs to store contract documents. A contract lasts for 5 years. During the 5-year period, the company must ensure that the documents cannot be overwritten or deleted. The company needs to encrypt the documents at rest and rotate the encryption keys automatically every year.

Which combination of steps should a solutions architect take to meet these requirements with the LEAST operational overhead? (Choose two.)

A. Store the documents in Amazon S3. Use S3 Object Lock in governance mode.

B. Store the documents in Amazon S3. Use S3 Object Lock in compliance mode.

C. Use server-side encryption with Amazon S3 managed encryption keys (SSE-S3). Configure key rotation.

D. Use server-side encryption with AWS Key Management Service (AWS KMS) customer managed keys. Configure key rotation.

E. Use server-side encryption with AWS Key Management Service (AWS KMS) customer provided (imported) keys. Configure key rotation.

Answer: BD

Explanation:

The best solution combines data immutability with managed encryption and automatic key rotation for minimal operational overhead.

B. Store the documents in Amazon S3. Use S3 Object Lock in compliance mode.

S3 Object Lock in compliance mode is crucial. Compliance mode provides the highest level of data immutability. Once an object is locked in compliance mode, it cannot be deleted or overwritten, even by the root user, until the retention period expires. This satisfies the requirement that documents cannot be overwritten or deleted during the 5-year contract period. Governance mode, on the other hand, allows certain privileged users to bypass the lock, which doesn't meet the stringent requirement of complete immutability.

https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock-overview.html

D. Use server-side encryption with AWS Key Management Service (AWS KMS) customer managed keys. Configure key rotation.

Using KMS customer managed keys allows for automatic key rotation every year as required. This automated process reduces operational overhead and aligns with security best practices. While SSE-S3 (option C) handles encryption, it provides less control over key management than KMS. Option E, using customer-provided keys, adds significant operational overhead because the company is responsible for generating, storing, and securely delivering the keys to AWS. KMS simplifies key management. Using KMS with CMKs allows you to enable automatic key rotation and maintain audit trails related to encryption key usage. SSE-S3 also provides key rotation, but KMS offers better key management overall.

https://docs.aws.amazon.com/kms/latest/developerguide/rotate-keys.htmlhttps://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingKMSEncryption.html

Therefore, storing the documents in S3 with Object Lock in compliance mode ensures immutability, while using KMS with customer-managed keys provides automated encryption and key rotation, minimizing operational overhead and meeting all requirements.

Question: 190 **Exam Heist**

A company has a web application that is based on Java and PHP. The company plans to move the application from on premises to AWS. The company needs the ability to test new site features frequently. The company also needs a highly available and managed solution that requires minimum operational overhead.

Which solution will meet these requirements?

A. Create an Amazon S3 bucket. Enable static web hosting on the S3 bucket. Upload the static content to the S3 bucket. Use AWS Lambda to process all dynamic content.

B. Deploy the web application to an AWS Elastic Beanstalk environment. Use URL swapping to switch between multiple Elastic Beanstalk environments for feature testing.

C. Deploy the web application to Amazon EC2 instances that are configured with Java and PHP. Use Auto Scaling groups and an Application Load Balancer to manage the website's availability.

D. Containerize the web application. Deploy the web application to Amazon EC2 instances. Use the AWS Load Balancer Controller to dynamically route traffic between containers that contain the new site features for testing.

Answer: B

Explanation:

The correct answer is B: Deploy the web application to an AWS Elastic Beanstalk environment. Use URL swapping to switch between multiple Elastic Beanstalk environments for feature testing.

Here's a detailed justification:

Elastic Beanstalk is a Platform-as-a-Service (PaaS) that simplifies deploying and managing web applications in AWS. It supports various programming languages, including Java and PHP, meeting the application's requirements. It provides a managed environment, reducing operational overhead, which aligns with the company's need for a solution with minimal operational overhead.

The key advantage of Elastic Beanstalk in this scenario is its support for easy deployment of multiple environments and URL swapping. This feature is crucial for frequent testing of new site features. URL swapping allows you to quickly direct traffic to a different environment hosting the new features, enabling quick A/B testing or staging deployments without modifying DNS records. This minimizes downtime and facilitates continuous integration and continuous delivery (CI/CD) practices.

Option A is incorrect because Amazon S3 is designed for static content hosting. While Lambda can handle dynamic content, this solution would require significant architectural changes and more operational overhead to manage the dynamic aspects of the entire application.

Option C involves deploying the application to Amazon EC2 instances managed by Auto Scaling groups and an Application Load Balancer (ALB). This is a viable solution but requires more manual configuration and management compared to Elastic Beanstalk. Setting up a robust testing environment and implementing feature toggles or blue/green deployments on EC2 would increase operational overhead.

Option D suggests containerizing the application and deploying it to EC2 instances. While containerization offers benefits like portability, it also introduces additional complexity in managing the container orchestration. The AWS Load Balancer Controller is designed for Kubernetes, which is not explicitly stated as being used by the company. Therefore, setting up this infrastructure with dynamic routing would involve more effort compared to the Elastic Beanstalk approach.

In summary, Elastic Beanstalk offers the best balance between simplicity, managed services, and support for feature testing via URL swapping, making it the most appropriate solution for this scenario.

Relevant AWS documentation:

AWS Elastic Beanstalk: https://aws.amazon.com/elasticbeanstalk/
Elastic Beanstalk URL Swapping: https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.CNAMESwap.html

Question: 191 **Exam Heist**

A company has an ordering application that stores customer information in Amazon RDS for MySQL. During regular business hours, employees run one-time queries for reporting purposes. Timeouts are occurring during order processing because the reporting queries are taking a long time to run. The company needs to eliminate the timeouts without preventing employees from performing queries.

What should a solutions architect do to meet these requirements?

A. Create a read replica. Move reporting queries to the read replica.

B. Create a read replica. Distribute the ordering application to the primary DB instance and the read replica.

C. Migrate the ordering application to Amazon DynamoDB with on-demand capacity.

D. Schedule the reporting queries for non-peak hours.

Answer: A

Explanation:

The best solution to eliminate timeouts caused by reporting queries on an Amazon RDS for MySQL database without restricting employee access is to create a read replica and move the reporting queries to it.

Option A, creating a read replica, directly addresses the problem by offloading read-intensive reporting queries from the primary RDS instance. Read replicas allow you to scale beyond the capacity constraints of a single database instance for read-heavy database workloads. The primary RDS instance will then be dedicated to order processing, eliminating timeouts.

Option B, distributing the application across the primary and read replica is incorrect. The ordering application needs consistent writes, which must occur on the primary instance to maintain data integrity. Splitting writes across the primary and read replica is not possible due to the read-only nature of the replica and would introduce data inconsistency issues.

Option C, migrating to DynamoDB, might be a viable long-term solution, but it's a complex undertaking involving a complete application rewrite and data migration. Also, DynamoDB is a NoSQL database and might not suit the relational nature of the customer information or the reporting requirements. The cost and effort involved far outweigh the benefits for this immediate problem.

Option D, scheduling the reporting queries for non-peak hours, is a temporary workaround, not a permanent solution. During those non-peak hours, timeouts might still occur if the queries remain resource-intensive. Also, it imposes limitations on the employees' ability to perform queries on demand.

Therefore, Option A provides the most effective and least disruptive solution. It allows continued reporting with minimal latency and doesn't interrupt order processing during business hours.

Amazon RDS Read ReplicasScaling with Amazon RDS Read Replicas

Question: 192 **Exam Heist**

A hospital wants to create digital copies for its large collection of historical written records. The hospital will continue to add hundreds of new documents each day. The hospital’s data team will scan the documents and will upload the documents to the AWS Cloud.

A solutions architect must implement a solution to analyze the documents, extract the medical information, and store the documents so that an application can run SQL queries on the data. The solution must maximize scalability and operational efficiency.

Which combination of steps should the solutions architect take to meet these requirements? (Choose two.)

A. Write the document information to an Amazon EC2 instance that runs a MySQL database.

B. Write the document information to an Amazon S3 bucket. Use Amazon Athena to query the data.

C. Create an Auto Scaling group of Amazon EC2 instances to run a custom application that processes the scanned files and extracts the medical information.

D. Create an AWS Lambda function that runs when new documents are uploaded. Use Amazon Rekognition to convert the documents to raw text. Use Amazon Transcribe Medical to detect and extract relevant medical

information from the text.

E. Create an AWS Lambda function that runs when new documents are uploaded. Use Amazon Textract to convert the documents to raw text. Use Amazon Comprehend Medical to detect and extract relevant medical information from the text.

Answer: BE

Explanation:

The correct answer is **BE**. Here's why:

B. Write the document information to an Amazon S3 bucket. Use Amazon Athena to query the data. This option offers a highly scalable and cost-effective solution for storing and querying the document information. Amazon S3 provides virtually unlimited storage capacity and high durability, suitable for storing the large collection of historical records. Amazon Athena, a serverless query service, allows running SQL queries directly on data stored in S3, eliminating the need to manage a database server. This approach maximizes operational efficiency because no database maintenance is needed. Athena's pay-per-query pricing model optimizes cost based on actual usage.

E. Create an AWS Lambda function that runs when new documents are uploaded. Use Amazon Textract to convert the documents to raw text. Use Amazon Comprehend Medical to detect and extract relevant medical information from the text. This option provides a scalable and efficient way to analyze the documents and extract the medical information. AWS Lambda allows running code without provisioning or managing servers, ensuring scalability and operational efficiency. Amazon Textract is specifically designed for extracting text and data from scanned documents. Amazon Comprehend Medical can then efficiently process this raw text to identify and extract relevant medical information, such as diagnoses, medications, and procedures. This is a serverless and automated approach for data extraction and analysis, ideal for processing hundreds of new documents daily.

Why other options are incorrect:

A: Using an EC2 instance with MySQL for this volume and growth is not scalable or operationally efficient. Managing a database server requires ongoing maintenance and scaling efforts.
C: Creating an Auto Scaling group of EC2 instances to run a custom application for processing the scanned files adds complexity and operational overhead. It requires developing, deploying, and managing the custom application, which deviates from maximizing operational efficiency.
D: Amazon Rekognition is designed for image and video analysis, not OCR from documents. Amazon Transcribe Medical is for audio transcription and extraction, not for processing text from scanned documents. Textract and Comprehend Medical are tools specifically tailored for document processing needs.

Supporting Documentation:

Amazon S3: https://aws.amazon.com/s3/
Amazon Athena: https://aws.amazon.com/athena/
AWS Lambda: https://aws.amazon.com/lambda/
Amazon Textract: https://aws.amazon.com/textract/
Amazon Comprehend Medical: https://aws.amazon.com/comprehend/medical/

Question: 193 **Exam Heist**

A company is running a batch application on Amazon EC2 instances. The application consists of a backend with multiple Amazon RDS databases. The application is causing a high number of reads on the databases. A solutions architect must reduce the number of database reads while ensuring high availability.

What should the solutions architect do to meet this requirement?

A. Add Amazon RDS read replicas.

B. Use Amazon ElastiCache for Redis.

C. Use Amazon Route 53 DNS caching

D. Use Amazon ElastiCache for Memcached.

Answer: A

Explanation:

The most suitable solution is **A. Add Amazon RDS read replicas.**

Here's why:

The core problem is high read load on the RDS databases. Read replicas are specifically designed to alleviate read load from the primary RDS database. They provide read-only copies of the data, allowing read queries to be directed to the replicas instead of the primary database. This reduces the load on the primary database, improving its performance and availability. Because the question mentions a need for HA, using Read Replicas fulfills that need also because they act as standby copies of the data if the primary were to fail.

Option B, using ElastiCache for Redis, is a valid caching solution, but it requires application modifications to implement caching logic. While it could reduce database reads, it's a more complex solution and may not be the best first step without further details on the application.

Option C, using Route 53 DNS caching, is not relevant to the problem. DNS caching improves the resolution speed of domain names, but it doesn't address the database read load.

Option D, using ElastiCache for Memcached, is similar to Redis; it's a viable caching solution but requires application changes. Memcached is usually best for caching simple objects that you need with very low latency (usually smaller than Redis use cases).

Therefore, adding RDS read replicas is the simplest and most direct solution to reduce database read load while also enhancing availability, without necessitating significant code changes. Read Replicas also automatically handle data replication from the source so they’re easy to manage and can readily be scaled for higher performance.

Further reading:

Amazon RDS Read Replicas:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.ReadReplicas.html
Amazon ElastiCache: https://aws.amazon.com/elasticache/

Question: 194 **Exam Heist**

A company needs to run a critical application on AWS. The company needs to use Amazon EC2 for the application’s database. The database must be highly available and must fail over automatically if a disruptive event occurs.

Which solution will meet these requirements?

A. Launch two EC2 instances, each in a different Availability Zone in the same AWS Region. Install the database on both EC2 instances. Configure the EC2 instances as a cluster. Set up database replication.

B. Launch an EC2 instance in an Availability Zone. Install the database on the EC2 instance. Use an Amazon Machine Image (AMI) to back up the data. Use AWS CloudFormation to automate provisioning of the EC2 instance if a disruptive event occurs.

C. Launch two EC2 instances, each in a different AWS Region. Install the database on both EC2 instances. Set up database replication. Fail over the database to a second Region.

D. Launch an EC2 instance in an Availability Zone. Install the database on the EC2 instance. Use an Amazon

Machine Image (AMI) to back up the data. Use EC2 automatic recovery to recover the instance if a disruptive event occurs.

Answer: A

Explanation:

The most suitable solution for ensuring high availability and automatic failover of a database running on Amazon EC2 instances is option A. Here's why:

Option A proposes launching two EC2 instances in separate Availability Zones (AZs) within the same AWS Region. This leverages the principle of redundancy, a cornerstone of high availability architectures. Availability Zones are physically isolated locations within an AWS Region, designed to be isolated from failures in other AZs. Installing the database on both instances and configuring them as a cluster enables active-passive or active-active replication. Replication ensures data is synchronized between the instances, so if one instance fails, the other can immediately take over.

Clustering software, like Pacemaker or Windows Server Failover Clustering, along with database-specific replication tools, orchestrates the failover process. When the primary instance fails, the cluster automatically promotes the secondary instance to become the new primary, minimizing downtime and ensuring continuous application availability.

Option B is less ideal because relying solely on AMIs and CloudFormation for recovery introduces significant downtime. Restoring from an AMI and reprovisioning a new instance are time-consuming processes, not suitable for applications requiring automatic failover.

Option C suggests using different AWS Regions. While cross-region failover provides disaster recovery capabilities, it typically involves higher latency and complexity compared to multi-AZ setups. Failover across Regions often isn't automatic and may require manual intervention or more sophisticated configuration, making it less suitable for achieving automatic failover as required in the scenario.

Option D utilizes EC2 automatic recovery. While helpful for some instance-level failures, automatic recovery only restarts the instance on a new underlying hardware within the same AZ. It does not protect against AZ-wide events or data loss and it does not guarantee minimal downtime. It's beneficial but not a comprehensive HA solution on its own.

In summary, Option A delivers the best balance of high availability, automatic failover, and reasonable complexity by leveraging redundancy within a Region using multiple AZs and database replication.

Relevant Links:

AWS High Availability: https://aws.amazon.com/reliability/high-availability/
Availability Zones: https://aws.amazon.com/about-aws/global-infrastructure/
Database Replication: Consult your specific database documentation (e.g., MySQL replication, PostgreSQL streaming replication)

Question: 195 **Exam Heist**

A company's order system sends requests from clients to Amazon EC2 instances. The EC2 instances process the orders and then store the orders in a database on Amazon RDS. Users report that they must reprocess orders when the system fails. The company wants a resilient solution that can process orders automatically if a system outage occurs.

What should a solutions architect do to meet these requirements?

A. Move the EC2 instances into an Auto Scaling group. Create an Amazon EventBridge (Amazon CloudWatch Events) rule to target an Amazon Elastic Container Service (Amazon ECS) task.

B. Move the EC2 instances into an Auto Scaling group behind an Application Load Balancer (ALB). Update the order system to send messages to the ALB endpoint.

C. Move the EC2 instances into an Auto Scaling group. Configure the order system to send messages to an Amazon Simple Queue Service (Amazon SQS) queue. Configure the EC2 instances to consume messages from the queue.

D. Create an Amazon Simple Notification Service (Amazon SNS) topic. Create an AWS Lambda function, and subscribe the function to the SNS topic. Configure the order system to send messages to the SNS topic. Send a command to the EC2 instances to process the messages by using AWS Systems Manager Run Command.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best solution, along with supporting concepts and resources:

Option C offers the most resilient and decoupled architecture for processing orders in the face of system outages. It leverages Amazon SQS to create a buffer between the order system and the EC2 instances, achieving asynchronous processing. When an order comes in, instead of directly calling an EC2 instance, the order system places a message describing the order into the SQS queue. This action decouples the order initiation from the processing itself.

The EC2 instances, which are now part of an Auto Scaling group, are configured to consume messages from the SQS queue. The Auto Scaling group ensures that even if some EC2 instances fail, others will be launched automatically to maintain the processing capacity. Because the order requests are persisted in the SQS queue, even if all EC2 instances are temporarily down, no orders are lost. When the EC2 instances come back online, they start consuming messages from the queue and processing the orders.

This approach directly addresses the requirement of automatically processing orders during system outages. SQS guarantees at-least-once delivery, ensuring that each order is processed. The Auto Scaling group provides high availability and fault tolerance for the processing instances. The decoupling between the order system and the processing layer is crucial for resilience.

Option A is not ideal because EventBridge is typically used for event-driven architectures, not for reliable message queuing for order processing. ECS might be an unnecessary complication. Option B only adds an ALB for distribution but does not resolve the problem of orders lost during outages. While the ALB improves availability, it does not buffer requests like SQS does. Option D is less suitable because SNS is primarily for notifications, not guaranteed message delivery. Using Run Command to trigger processing on EC2 instances introduces unnecessary complexity and management overhead. SQS is designed explicitly for reliable message queuing, which perfectly aligns with the problem statement's requirements.

In summary, option C leverages the strengths of SQS and Auto Scaling to create a robust and fault-tolerant order processing system, ensuring that orders are not lost and are automatically processed even during system outages.

Supporting Links:

Amazon SQS: https://aws.amazon.com/sqs/
Amazon EC2 Auto Scaling: https://aws.amazon.com/autoscaling/

Question: 196 **Exam Heist**

A company runs an application on a large fleet of Amazon EC2 instances. The application reads and writes entries into an Amazon DynamoDB table. The size of the DynamoDB table continuously grows, but the application needs only data from the last 30 days. The company needs a solution that minimizes cost and development effort.

Which solution meets these requirements?

A. Use an AWS CloudFormation template to deploy the complete solution. Redeploy the CloudFormation stack every 30 days, and delete the original stack.

B. Use an EC2 instance that runs a monitoring application from AWS Marketplace. Configure the monitoring application to use Amazon DynamoDB Streams to store the timestamp when a new item is created in the table. Use a script that runs on the EC2 instance to delete items that have a timestamp that is older than 30 days.

C. Configure Amazon DynamoDB Streams to invoke an AWS Lambda function when a new item is created in the table. Configure the Lambda function to delete items in the table that are older than 30 days.

D. Extend the application to add an attribute that has a value of the current timestamp plus 30 days to each new item that is created in the table. Configure DynamoDB to use the attribute as the TTL attribute.

Answer: D

Explanation:

The most cost-effective and easiest-to-implement solution is option D, leveraging DynamoDB's Time To Live (TTL) feature. TTL allows you to specify an attribute that determines when an item will be automatically deleted from the table.

Here's why option D is superior to the other options:

Cost Efficiency: TTL incurs no additional charges beyond the standard DynamoDB costs. Option B involves a monitoring application on an EC2 instance, incurring EC2 costs, while option C involves Lambda invocations, incurring Lambda costs. Option A requires repeated full stack deployments which is extremely costly and time-consuming.
Development Effort: Implementing TTL is straightforward. You simply add an attribute to your items containing the expiration timestamp (current timestamp + 30 days) and configure DynamoDB to recognize this attribute as the TTL attribute. The DynamoDB system automatically handles the deletion process. Implementing options B and C requires developing and maintaining custom logic for monitoring, deletion, and potentially conflict resolution. Option A requires creating a whole stack.
Automation: TTL is a fully automated process. Once configured, DynamoDB continuously monitors the TTL attribute and removes expired items. Options B and C require scheduled scripts or Lambda functions, introducing complexities with scheduling, error handling, and ensuring data consistency.
Scalability and Reliability: DynamoDB's TTL is a built-in feature, designed to scale with your table size. Options B and C are limited by the scalability and reliability of the EC2 instance or Lambda function, respectively.
Minimizes Impact on Application: Extending the application to add the TTL attribute is a minimal change with little to no operational overhead. Options B and C might introduce latency or contention if the monitoring or deletion processes interfere with application operations.

TTL efficiently handles data retention policies. DynamoDB handles cleanup in the background, minimizing operational overhead and complexity. This aligns with cloud computing principles of automation, cost optimization, and simplicity.

Further research on DynamoDB TTL can be found at:

DynamoDB TTL

Question: 197 **Exam Heist**

A company has a Microsoft .NET application that runs on an on-premises Windows Server. The application stores data by using an Oracle Database Standard Edition server. The company is planning a migration to AWS and wants to minimize development changes while moving the application. The AWS application environment should be highly available.

Which combination of actions should the company take to meet these requirements? (Choose two.)

A. Refactor the application as serverless with AWS Lambda functions running .NET Core.

B. Rehost the application in AWS Elastic Beanstalk with the .NET platform in a Multi-AZ deployment.

C. Replatform the application to run on Amazon EC2 with the Amazon Linux Amazon Machine Image (AMI).

D. Use AWS Database Migration Service (AWS DMS) to migrate from the Oracle database to Amazon DynamoDB in a Multi-AZ deployment.

E. Use AWS Database Migration Service (AWS DMS) to migrate from the Oracle database to Oracle on Amazon RDS in a Multi-AZ deployment.

Answer: BE

Explanation:

The requirement is to migrate a .NET application with minimal code changes to a highly available AWS environment while retaining the Oracle database.

Option B, rehosting the application in AWS Elastic Beanstalk with the .NET platform in a Multi-AZ deployment, directly addresses the minimal changes requirement. Elastic Beanstalk simplifies deployment and management of web applications. By selecting the .NET platform and deploying it in a Multi-AZ environment, the application gains high availability through automatic instance replacement in case of failure across different Availability Zones. This 'lift and shift' approach requires very little to no code changes to the .NET application itself.

Option E, using AWS Database Migration Service (AWS DMS) to migrate from the Oracle database to Oracle on Amazon RDS in a Multi-AZ deployment, fulfills the database persistence requirement with minimal disruption. DMS allows migration of databases with minimal downtime. Migrating to Oracle RDS maintains compatibility with the existing application architecture. Deploying Oracle on RDS in a Multi-AZ configuration provides high availability for the database.

Option A is incorrect because refactoring to serverless Lambda functions requires significant code changes to the application, violating the requirement for minimal development changes.

Option C is incorrect because replatforming to EC2 with Amazon Linux requires changes to the .NET application to be compatible with Linux. It also does not offer any managed service to make deployment and management easier, and it increases the overhead of managing the underlying OS.

Option D is incorrect because migrating to DynamoDB would require significant changes to the application's data access layer to work with a NoSQL database, which is not acceptable since the requirement is to minimize code changes. DynamoDB is not compatible with the existing application code.

Therefore, the correct answer is BE.

Relevant links:

AWS Elastic Beanstalk: https://aws.amazon.com/elasticbeanstalk/
AWS Database Migration Service (DMS): https://aws.amazon.com/dms/
Amazon RDS Multi-AZ: https://aws.amazon.com/rds/features/multi-az/

Question: 198 **Exam Heist**

A company runs a containerized application on a Kubernetes cluster in an on-premises data center. The company is using a MongoDB database for data storage. The company wants to migrate some of these environments to AWS, but no code changes or deployment method changes are possible at this time. The company needs a solution that minimizes operational overhead.

Which solution meets these requirements?

A. Use Amazon Elastic Container Service (Amazon ECS) with Amazon EC2 worker nodes for compute and MongoDB on EC2 for data storage.

B. Use Amazon Elastic Container Service (Amazon ECS) with AWS Fargate for compute and Amazon DynamoDB for data storage

C. Use Amazon Elastic Kubernetes Service (Amazon EKS) with Amazon EC2 worker nodes for compute and Amazon DynamoDB for data storage.

D. Use Amazon Elastic Kubernetes Service (Amazon EKS) with AWS Fargate for compute and Amazon DocumentDB (with MongoDB compatibility) for data storage.

Answer: D

Explanation:

Here's a detailed justification for why option D is the best solution, aligning with the given requirements and constraints:

The core requirement is migrating the application to AWS with minimal code and deployment method changes while minimizing operational overhead. The application uses a containerized setup on Kubernetes and MongoDB for data storage.

Option D, using Amazon Elastic Kubernetes Service (Amazon EKS) with AWS Fargate for compute and Amazon DocumentDB (with MongoDB compatibility) for data storage, is the ideal choice. EKS allows the company to continue using Kubernetes without significant modifications to their existing deployment process, fulfilling the "no deployment method changes" constraint. Fargate provides serverless compute, eliminating the need to manage EC2 instances, thus reducing operational overhead significantly. This aligns with the "minimizing operational overhead" requirement.

Critically, Amazon DocumentDB offers MongoDB compatibility. This is crucial because the prompt explicitly states that "no code changes" are possible. Switching to DynamoDB (as suggested in options B and C) would require significant code changes to interact with the new database API. DocumentDB's MongoDB compatibility allows the application to interact with the database without altering the existing codebase.

Option A (ECS with EC2 and MongoDB on EC2) fails to minimize operational overhead. Managing EC2 instances for both compute and the database adds significant administrative burden compared to Fargate and DocumentDB. Option B fails due to the required code changes for DynamoDB integration. Option C also fails due to the need for code changes for DynamoDB.

In summary, EKS + Fargate provides Kubernetes compatibility and serverless compute, and DocumentDB offers MongoDB compatibility and managed service characteristics, perfectly addressing all constraints and requirements.

Supporting links:

Amazon EKS: https://aws.amazon.com/eks/
AWS Fargate: https://aws.amazon.com/fargate/
Amazon DocumentDB: https://aws.amazon.com/documentdb/

Question: 199 **Exam Heist**

A telemarketing company is designing its customer call center functionality on AWS. The company needs a solution that provides multiple speaker recognition and generates transcript files. The company wants to query the transcript files to analyze the business patterns. The transcript files must be stored for 7 years for auditing purposes.

Which solution will meet these requirements?

A. Use Amazon Rekognition for multiple speaker recognition. Store the transcript files in Amazon S3. Use machine learning models for transcript file analysis.

B. Use Amazon Transcribe for multiple speaker recognition. Use Amazon Athena for transcript file analysis.

C. Use Amazon Translate for multiple speaker recognition. Store the transcript files in Amazon Redshift. Use SQL queries for transcript file analysis.

D. Use Amazon Rekognition for multiple speaker recognition. Store the transcript files in Amazon S3. Use Amazon Textract for transcript file analysis.

Answer: B

Explanation:

The correct answer is B because it leverages the appropriate AWS services for the specific requirements outlined. Amazon Transcribe is specifically designed for converting speech to text, including capabilities for multiple speaker recognition (speaker diarization). This directly addresses the need for transcribing call center conversations with speaker identification.

Amazon Athena is a serverless, interactive query service that makes it easy to analyze data directly in Amazon S3 using standard SQL. This satisfies the requirement for querying transcript files to analyze business patterns.

Storing the transcript files in Amazon S3 (as implied in the problem description in conjunction with Athena) ensures cost-effective and durable storage for the required 7-year retention period. S3's storage classes can be configured to optimize cost based on access frequency.

Option A is incorrect because while Rekognition can analyze images and videos, it's not primarily designed for real-time speech transcription and speaker diarization in the same way as Transcribe. While machine learning models could be used for analysis, Athena provides a simpler, more readily available solution for querying the data.

Option C is incorrect because Amazon Translate is used for language translation, not speech-to-text transcription. Amazon Redshift is a data warehouse designed for large-scale data analytics, but using it solely for querying relatively small transcript files is an overkill and less cost-effective than Athena.

Option D is incorrect because, similar to option A, Amazon Rekognition is primarily designed for image and video analysis. Amazon Textract is used for extracting text and data from scanned documents and PDFs, not for analyzing speech transcripts.

In summary, Amazon Transcribe efficiently handles the audio transcription and speaker recognition, while Amazon Athena provides a direct and cost-effective way to query the resulting transcript files stored in Amazon S3. This approach aligns perfectly with the prompt's need for transcription, analysis, and long-term storage.

Relevant Links:

Amazon Transcribe: https://aws.amazon.com/transcribe/
Amazon Athena: https://aws.amazon.com/athena/
Amazon S3: https://aws.amazon.com/s3/

Question: 200 **Exam Heist**

A company hosts its application on AWS. The company uses Amazon Cognito to manage users. When users log in to the application, the application fetches required data from Amazon DynamoDB by using a REST API that is hosted in Amazon API Gateway. The company wants an AWS managed solution that will control access to the REST API to reduce development efforts.

Which solution will meet these requirements with the LEAST operational overhead?

A. Configure an AWS Lambda function to be an authorizer in API Gateway to validate which user made the request.

B. For each user, create and assign an API key that must be sent with each request. Validate the key by using an AWS Lambda function.

C. Send the user's email address in the header with every request. Invoke an AWS Lambda function to validate that the user with that email address has proper access.

D. Configure an Amazon Cognito user pool authorizer in API Gateway to allow Amazon Cognito to validate each request.

Answer: D

Explanation:

The correct answer is D: Configure an Amazon Cognito user pool authorizer in API Gateway to allow Amazon Cognito to validate each request. Here's why:

The problem requires an AWS-managed solution to control API access using Cognito for user management with minimal operational overhead.

Option D directly leverages the integration between Cognito and API Gateway. Cognito User Pool Authorizers within API Gateway allow API Gateway to directly validate the identity token (JWT) provided by Cognito upon successful user authentication. This eliminates the need for custom code (like Lambda functions) for token validation, significantly reducing development and operational overhead. API Gateway handles the token verification process based on the configured User Pool.

Option A, using a Lambda function as an authorizer, introduces unnecessary complexity. While it works, it requires writing, deploying, and maintaining the Lambda function. This adds operational overhead that the problem statement seeks to avoid. It also involves potential latency from invoking the Lambda function.

Option B, using API Keys per user and a Lambda validator, introduces both operational overhead and security concerns. Managing API keys for each user becomes complex, and the need to validate the key with a Lambda function increases latency and operational burden. API keys are generally for API product access control rather than individual user authorization.

Option C, sending the email address and validating with a Lambda function, is the least secure and most inefficient solution. Relying on the email address in a header is easily spoofed. Also, querying a Lambda function for each API call increases latency and creates an unnecessary dependency. This also likely requires fetching user data from DynamoDB within the lambda, adding further overhead.

Cognito User Pool Authorizers offer seamless integration, scalability, and security, making them the ideal choice for managed user-based API authorization when using Cognito for identity management. This approach keeps the solution within the AWS-managed ecosystem, minimizing custom code and operational tasks.

Further Research:

API Gateway Authorizers: https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-use-lambda-authorizer.html
Cognito User Pools: https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-identity-pools.html
API Gateway Cognito Authorizer:
https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-integrate-with-cognito.html

Question: 201 **Exam Heist**

A company is developing a marketing communications service that targets mobile app users. The company needs to send confirmation messages with Short Message Service (SMS) to its users. The users must be able to reply to the SMS messages. The company must store the responses for a year for analysis.

What should a solutions architect do to meet these requirements?

A. Create an Amazon Connect contact flow to send the SMS messages. Use AWS Lambda to process the responses.

B. Build an Amazon Pinpoint journey. Configure Amazon Pinpoint to send events to an Amazon Kinesis data stream for analysis and archiving.

C. Use Amazon Simple Queue Service (Amazon SQS) to distribute the SMS messages. Use AWS Lambda to process the responses.

D. Create an Amazon Simple Notification Service (Amazon SNS) FIFO topic. Subscribe an Amazon Kinesis data stream to the SNS topic for analysis and archiving.

Answer: B

Explanation:

The correct answer is B: Build an Amazon Pinpoint journey. Configure Amazon Pinpoint to send events to an Amazon Kinesis data stream for analysis and archiving.

Here's why:

Amazon Pinpoint: Pinpoint is specifically designed for marketing communication and user engagement, making it the ideal choice for sending SMS confirmation messages to mobile app users. It provides features for creating targeted campaigns and journeys.

SMS Capabilities: Pinpoint supports two-way SMS messaging, allowing users to reply to the confirmation messages. It handles the complexities of sending and receiving SMS at scale.

Event Streaming: Pinpoint integrates with Amazon Kinesis Data Streams. This integration is crucial for capturing and analyzing user responses to the SMS messages. The event data stream will contain information about delivered messages, user replies, and other relevant events.

Data Analysis and Archiving: By streaming events to Kinesis Data Streams, the company can process, analyze, and archive the data for a year as required. Kinesis Data Streams enables real-time data ingestion and processing, allowing for timely analysis. The stream can then be connected to services like Amazon S3 for long-term storage and archiving. Amazon Athena or Amazon Redshift can be used to query this data for analysis.

Let's examine why other options are less suitable:

A (Amazon Connect): Amazon Connect is primarily a contact center solution for voice and chat. While it can send SMS messages, it's not its core function and is not ideal for bulk marketing communications. Furthermore, it is more complex to set up for the given scenario.

C (Amazon SQS): Amazon SQS is a queuing service for decoupling components. It's not designed for sending SMS messages directly to users. While Lambda could be used to trigger SMS sends, this is not as streamlined or feature-rich as using Pinpoint.

D (Amazon SNS): Amazon SNS is a notification service for pub/sub messaging. While SNS can send SMS, it's primarily for one-way notifications, not for managing two-way communication and tracking responses. SNS FIFO (First-In, First-Out) topics are useful for ordered message delivery, which is not a key requirement here. Further, sending SMS via SNS and routing replies and data streams to SNS is not straightforward or optimal as using Pinpoint.

Authoritative Links:

Amazon Pinpoint: https://aws.amazon.com/pinpoint/
Amazon Kinesis Data Streams: https://aws.amazon.com/kinesis/data-streams/

Question: 202 **Exam Heist**

A company is planning to move its data to an Amazon S3 bucket. The data must be encrypted when it is stored in the S3 bucket. Additionally, the encryption key must be automatically rotated every year.

Which solution will meet these requirements with the LEAST operational overhead?

A. Move the data to the S3 bucket. Use server-side encryption with Amazon S3 managed encryption keys (SSE-S3). Use the built-in key rotation behavior of SSE-S3 encryption keys.

B. Create an AWS Key Management Service (AWS KMS) customer managed key. Enable automatic key rotation. Set the S3 bucket's default encryption behavior to use the customer managed KMS key. Move the data to the S3 bucket.

C. Create an AWS Key Management Service (AWS KMS) customer managed key. Set the S3 bucket's default encryption behavior to use the customer managed KMS key. Move the data to the S3 bucket. Manually rotate the KMS key every year.

D. Encrypt the data with customer key material before moving the data to the S3 bucket. Create an AWS Key Management Service (AWS KMS) key without key material. Import the customer key material into the KMS key. Enable automatic key rotation.

Answer: B

Explanation:

The correct answer is B because it offers the simplest and most automated solution for encrypting data at rest in S3 with automatic key rotation, minimizing operational overhead. Let's break down why:

Option B leverages AWS KMS customer managed keys with automatic key rotation. When you create a KMS key, you can enable automatic key rotation, which automatically rotates the key every year. Setting the S3 bucket's default encryption to use this KMS key ensures that all new objects uploaded to the bucket are automatically encrypted using the key. This eliminates the need for manual encryption or key management.

Option A uses SSE-S3. While SSE-S3 encrypts data, the key rotation behavior is managed by AWS and not customizable. The exact details of their key rotation and control aren't exposed to the user, making it less transparent and potentially not meeting a specific yearly rotation requirement.

Option C involves manual key rotation, which directly contradicts the requirement for "LEAST operational overhead". Manual rotation introduces the risk of human error and requires active monitoring and intervention.

Option D, using customer-provided key material, adds significant complexity. It requires you to manage the initial key material and import it into KMS. While automatic rotation is enabled, managing the initial key material lifecycle and its secure import introduces unnecessary overhead compared to Option B, where KMS generates and rotates the key automatically.

Therefore, Option B provides the best balance of security, automation, and minimal operational overhead. It allows AWS KMS to handle key generation and rotation transparently while ensuring S3 data is encrypted at rest.

Supporting Documentation:

AWS KMS Key Rotation: https://docs.aws.amazon.com/kms/latest/developerguide/rotate-keys.html
Protecting Data Using Server-Side Encryption:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/serv-side-encryption.html

Question: 203 **Exam Heist**

The customers of a finance company request appointments with financial advisors by sending text messages. A web application that runs on Amazon EC2 instances accepts the appointment requests. The text messages are published to an Amazon Simple Queue Service (Amazon SQS) queue through the web application. Another application that runs on EC2 instances then sends meeting invitations and meeting confirmation email messages to the customers. After successful scheduling, this application stores the meeting information in an Amazon DynamoDB database.

As the company expands, customers report that their meeting invitations are taking longer to arrive.

What should a solutions architect recommend to resolve this issue?

A. Add a DynamoDB Accelerator (DAX) cluster in front of the DynamoDB database.

B. Add an Amazon API Gateway API in front of the web application that accepts the appointment requests.

C. Add an Amazon CloudFront distribution. Set the origin as the web application that accepts the appointment requests.

D. Add an Auto Scaling group for the application that sends meeting invitations. Configure the Auto Scaling group to scale based on the depth of the SQS queue.

Answer: D

Explanation:

The correct answer is **D. Add an Auto Scaling group for the application that sends meeting invitations. Configure the Auto Scaling group to scale based on the depth of the SQS queue.**

Here's why:

The problem is that meeting invitations are taking longer to arrive as the company expands. This suggests a bottleneck in the application responsible for processing messages from the SQS queue and sending out invitations. This application's capacity to handle the increasing workload is insufficient.

Option D directly addresses this bottleneck. By adding an Auto Scaling group for the application that sends meeting invitations, the system can automatically scale the number of EC2 instances based on the depth of the SQS queue (i.e., the number of messages waiting to be processed). When the queue depth increases (more appointment requests), the Auto Scaling group launches more instances to process the messages concurrently, reducing the processing time and ensuring faster delivery of meeting invitations. This aligns with a queue-based load leveling pattern where SQS acts as a buffer and Auto Scaling adjusts the processing capacity based on the backlog.

Here's why the other options are not the best fit:

A. Add a DynamoDB Accelerator (DAX) cluster in front of the DynamoDB database. DAX is an in-memory cache for DynamoDB. While DAX can improve read performance for data stored in DynamoDB, the bottleneck is in processing the messages from the queue and sending invitations, not necessarily in reading the meeting information from DynamoDB. Therefore, DAX would not directly solve the problem of delayed invitations.

B. Add an Amazon API Gateway API in front of the web application that accepts the appointment requests. API Gateway is useful for managing and securing APIs, and could add rate limiting. But the problem is the invitation delivery which occurs after the request is already handled by the web application. This does not address the delay in sending invitations after appointment requests are placed in the SQS queue.

C. Add an Amazon CloudFront distribution. Set the origin as the web application that accepts the appointment requests. CloudFront is a content delivery network (CDN). It's designed to cache and deliver static and dynamic content closer to users. It's not relevant to the problem of processing SQS messages and sending invitations; the performance issue is in the backend processing, not in delivering the web application itself.

In conclusion, scaling the processing capacity of the application that handles meeting invitations (option D) is the most direct and effective solution to address the increasing delays in meeting invitation delivery as the company expands. The SQS queue depth serves as a perfect metric to trigger scaling events, ensuring the application can keep up with the growing demand.

Authoritative Links:

Amazon SQS Auto Scaling: https://aws.amazon.com/blogs/compute/automatically-scale-your-amazon-sqs-processing-using-aws-lambda/
Amazon Auto Scaling: https://aws.amazon.com/autoscaling/
Queue-Based Load Leveling Pattern: https://docs.aws.amazon.com/prescriptive-guidance/patterns/queue-based-load-leveling.html

Question: 204 **Exam Heist**

An online retail company has more than 50 million active customers and receives more than 25,000 orders each day. The company collects purchase data for customers and stores this data in Amazon S3. Additional customer data is stored in Amazon RDS.

The company wants to make all the data available to various teams so that the teams can perform analytics. The solution must provide the ability to manage fine-grained permissions for the data and must minimize operational overhead.

Which solution will meet these requirements?

A. Migrate the purchase data to write directly to Amazon RDS. Use RDS access controls to limit access.

B. Schedule an AWS Lambda function to periodically copy data from Amazon RDS to Amazon S3. Create an AWS Glue crawler. Use Amazon Athena to query the data. Use S3 policies to limit access.

C. Create a data lake by using AWS Lake Formation. Create an AWS Glue JDBC connection to Amazon RDS. Register the S3 bucket in Lake Formation. Use Lake Formation access controls to limit access.

D. Create an Amazon Redshift cluster. Schedule an AWS Lambda function to periodically copy data from Amazon S3 and Amazon RDS to Amazon Redshift. Use Amazon Redshift access controls to limit access.

Answer: C

Explanation:

The correct answer is C because it provides a centralized, scalable, and secure solution for data analytics with fine-grained access control, minimizing operational overhead using AWS Lake Formation. Here's a detailed justification:

AWS Lake Formation for Data Lake: Lake Formation is designed for building, securing, and managing data lakes. It centralizes data access management and simplifies the process of setting up a data lake. https://aws.amazon.com/lake-formation/
AWS Glue for Data Catalog and ETL: AWS Glue is used to create a data catalog and extract, transform, and load (ETL) data. In this solution, a Glue JDBC connection is used to access the data in Amazon RDS, and Glue can catalog the data stored in S3. https://aws.amazon.com/glue/
Fine-grained Access Control: Lake Formation's primary benefit is its fine-grained access control. You can define permissions at the table and column level, ensuring that different teams only access the data they need. This meets the requirement of managing permissions for the data.
S3 Bucket Registration: Registering the S3 bucket in Lake Formation makes the purchase data accessible and governed by the Lake Formation policies.
Scalability: This solution scales well for 50 million customers and 25,000 orders per day. S3 is highly scalable for data storage, and Lake Formation manages access regardless of data size.
Minimizing Operational Overhead: Lake Formation automates many of the manual tasks associated with data

lake setup, security, and management. This reduces operational overhead compared to managing access controls through S3 policies or RDS access controls manually.

Why other options are not ideal:

A: Migrating purchase data directly to RDS isn't scalable or cost-effective for analytics. RDS is designed for transactional data and is not optimized for large-scale analytics.
B: Using S3 policies for access control is less granular and more complex to manage than Lake Formation, especially for numerous teams and datasets. Furthermore, copying data from RDS periodically adds operational overhead and introduces potential data consistency issues.
D: Amazon Redshift is a data warehouse, which might be suitable for complex analytics. However, periodically copying large datasets from S3 and RDS to Redshift using Lambda incurs operational overhead, potential data consistency issues, and more complex ETL processes. Redshift access controls also lack the fine-grained control offered by Lake Formation.

Question: 205 **Exam Heist**

A company hosts a marketing website in an on-premises data center. The website consists of static documents and runs on a single server. An administrator updates the website content infrequently and uses an SFTP client to upload new documents.

The company decides to host its website on AWS and to use Amazon CloudFront. The company's solutions architect creates a CloudFront distribution. The solutions architect must design the most cost-effective and resilient architecture for website hosting to serve as the CloudFront origin.

Which solution will meet these requirements?

A. Create a virtual server by using Amazon Lightsail. Configure the web server in the Lightsail instance. Upload website content by using an SFTP client.

B. Create an AWS Auto Scaling group for Amazon EC2 instances. Use an Application Load Balancer. Upload website content by using an SFTP client.

C. Create a private Amazon S3 bucket. Use an S3 bucket policy to allow access from a CloudFront origin access identity (OAI). Upload website content by using the AWS CLI.

D. Create a public Amazon S3 bucket. Configure AWS Transfer for SFTP. Configure the S3 bucket for website hosting. Upload website content by using the SFTP client.

Answer: C

Explanation:

The correct answer is C because it offers the most cost-effective and resilient solution for hosting static website content as a CloudFront origin. Here's a detailed justification:

Cost-Effectiveness: Amazon S3 offers a very low storage cost compared to virtual servers like Lightsail or EC2. S3's pay-as-you-go pricing model for storage and data transfer is ideal for static content with infrequent updates. Lightsail and EC2 incur costs even when the website isn't being heavily accessed.
Resilience: S3 provides high availability and durability due to its distributed nature across multiple Availability Zones. This means that the website content is highly resilient to failures. EC2 instances, while they can be made resilient with Auto Scaling, require more configuration and introduce more points of failure.
Security: Using a private S3 bucket with an Origin Access Identity (OAI) provides a secure way to restrict direct access to the S3 bucket. CloudFront can then access the content through the OAI, while public access is blocked. This prevents users from bypassing CloudFront and accessing the S3 bucket directly, enhancing security and enabling control over caching.
Simplified Updates: The AWS CLI is a robust and scriptable tool for uploading content to S3. It provides a streamlined and automated way to update the website content. Using SFTP, as suggested in other options, increases the management overhead.

Option A (Lightsail): Lightsail, although simple, is not as cost-effective for static content hosting as S3. It also involves more management of the virtual server.
Option B (EC2 with ALB): EC2 with an Application Load Balancer (ALB) is more complex and expensive for serving static content. It is designed for dynamic applications and requires more configuration.
Option D (Public S3 with SFTP): Making the S3 bucket public would expose the website content directly, bypassing the security benefits of CloudFront's OAI. Using AWS Transfer for SFTP is unnecessary, complex, and not cost-effective for this scenario, and S3 website hosting feature is not recommended for production workloads.

In summary, utilizing a private S3 bucket with an OAI for CloudFront provides a secure, resilient, cost-effective and easier-to-manage solution for hosting static website content.

Authoritative Links:

Amazon S3 Pricing: https://aws.amazon.com/s3/pricing/
Amazon CloudFront Origin Access Identity (OAI): https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html
AWS CLI: https://aws.amazon.com/cli/

Question: 206 **Exam Heist**

A company wants to manage Amazon Machine Images (AMIs). The company currently copies AMIs to the same AWS Region where the AMIs were created. The company needs to design an application that captures AWS API calls and sends alerts whenever the Amazon EC2 CreateImage API operation is called within the company's account.

Which solution will meet these requirements with the LEAST operational overhead?

A. Create an AWS Lambda function to query AWS CloudTrail logs and to send an alert when a CreateImage API call is detected.

B. Configure AWS CloudTrail with an Amazon Simple Notification Service (Amazon SNS) notification that occurs when updated logs are sent to Amazon S3. Use Amazon Athena to create a new table and to query on CreateImage when an API call is detected.

C. Create an Amazon EventBridge (Amazon CloudWatch Events) rule for the CreateImage API call. Configure the target as an Amazon Simple Notification Service (Amazon SNS) topic to send an alert when a CreateImage API call is detected.

D. Configure an Amazon Simple Queue Service (Amazon SQS) FIFO queue as a target for AWS CloudTrail logs. Create an AWS Lambda function to send an alert to an Amazon Simple Notification Service (Amazon SNS) topic when a CreateImage API call is detected.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best solution, along with explanations of why the other options are less suitable:

Justification for Option C (Amazon EventBridge rule with SNS target):

This solution offers the least operational overhead and is the most direct way to address the requirement. Amazon EventBridge is a serverless event bus that makes it easy to connect applications with data from a variety of sources.

Real-time Event Detection: EventBridge allows you to create rules that react to events happening in your AWS environment in near real-time. You can define a rule that specifically triggers when the CreateImage API

call is made. This ensures immediate detection of the AMI creation.
Direct Integration with SNS: EventBridge can directly integrate with Amazon SNS (Simple Notification Service). This eliminates the need for intermediate services or custom coding to handle the event processing and notification. The EventBridge rule simply forwards the event data to the SNS topic.
Minimal Configuration: Setting up an EventBridge rule for the CreateImage event and configuring the SNS target is relatively straightforward and requires minimal configuration.
Serverless Architecture: Both EventBridge and SNS are serverless services, so you don't need to manage any underlying infrastructure, reducing operational overhead.
Scalability and Reliability: EventBridge and SNS are highly scalable and reliable AWS services, ensuring your monitoring application can handle changes in API call volume.

Why other options are less suitable:

Option A (Lambda querying CloudTrail logs): While feasible, this introduces unnecessary complexity. The Lambda function would need to poll CloudTrail logs, parse the logs, and search for CreateImage events. This is less efficient than a push-based system like EventBridge and requires more coding and operational maintenance. Additionally, CloudTrail logs have a delay, making the alerts potentially not as real-time.

Option B (CloudTrail with SNS and Athena): This option is overly complex. While CloudTrail can send notifications on log updates to S3 and then trigger SNS, using Athena to query the logs adds significant overhead. Athena is designed for analyzing large datasets and is not ideal for real-time event detection. The setup and maintenance of the Athena table and queries would add unnecessary operational burden. The initial CloudTrail SNS notification only indicates that a log file has been updated, not necessarily that a CreateImage call has been made.

Option D (SQS FIFO with Lambda and SNS): This solution also introduces unnecessary complexity. While SQS FIFO can ensure message order, it's not needed for this specific requirement. The Lambda function is still required to poll the SQS queue and parse the CloudTrail logs for the CreateImage event. The use of SQS adds an additional layer of infrastructure to manage and monitor, increasing operational overhead.

Authoritative Links for Further Research:

Amazon EventBridge: https://aws.amazon.com/eventbridge/
Amazon SNS: https://aws.amazon.com/sns/
AWS CloudTrail: https://aws.amazon.com/cloudtrail/

Question: 207 **Exam Heist**

A company owns an asynchronous API that is used to ingest user requests and, based on the request type, dispatch requests to the appropriate microservice for processing. The company is using Amazon API Gateway to deploy the API front end, and an AWS Lambda function that invokes Amazon DynamoDB to store user requests before dispatching them to the processing microservices.

The company provisioned as much DynamoDB throughput as its budget allows, but the company is still experiencing availability issues and is losing user requests.

What should a solutions architect do to address this issue without impacting existing users?

A. Add throttling on the API Gateway with server-side throttling limits.
B. Use DynamoDB Accelerator (DAX) and Lambda to buffer writes to DynamoDB.
C. Create a secondary index in DynamoDB for the table with the user requests.
D. Use the Amazon Simple Queue Service (Amazon SQS) queue and Lambda to buffer writes to DynamoDB.

Answer: D

Explanation:

The correct answer is **D. Use the Amazon Simple Queue Service (Amazon SQS) queue and Lambda to buffer writes to DynamoDB.**

Here's a detailed justification:

The problem states that the company is experiencing availability issues and losing user requests due to DynamoDB being overwhelmed. This implies that the write throughput to DynamoDB is exceeding the provisioned capacity, even though the company has provisioned as much as their budget allows.

Option D addresses this directly by introducing a queue (Amazon SQS) in between the Lambda function and DynamoDB. When the Lambda function receives a request, instead of immediately writing to DynamoDB, it places a message in the SQS queue. Another Lambda function (or the same one, configured differently) then retrieves messages from the SQS queue and writes them to DynamoDB at a controlled rate.

Buffering: SQS acts as a buffer, absorbing bursts of traffic and smoothing out the write load to DynamoDB. If DynamoDB becomes temporarily unavailable or throttles requests, the messages remain in the queue until they can be successfully processed. This prevents the loss of user requests.
Asynchronous processing: The API can quickly return a success response after placing the message in the queue, minimizing latency for the user. The actual DynamoDB write happens asynchronously.
Decoupling: SQS decouples the API and the DynamoDB write process. If DynamoDB is having issues, the API can continue to accept requests without being directly affected.

Option A is incorrect because throttling at API Gateway only limits the number of incoming requests. It doesn't address the underlying issue of DynamoDB being overwhelmed. While it prevents more requests from hitting DynamoDB, it does so by rejecting user requests, which is undesirable as the problem statement explicitly mentions avoiding impacting existing users.

Option B is incorrect because DAX is a read-through/write-through cache. While DAX can improve read performance and reduce read load on DynamoDB, it doesn't inherently solve the problem of exceeding write capacity. DAX sits in front of DynamoDB, so if the write capacity is exceeded, the bottleneck will still exist when DAX needs to write to DynamoDB. Buffering writes is still necessary in this scenario, and DAX doesn't do that.

Option C is incorrect because secondary indexes improve query performance for different access patterns. They do not inherently solve throughput issues related to exceeding write capacity. They can even increase the write load as indexes also need to be updated during writes.

Therefore, using SQS to buffer writes to DynamoDB is the most effective way to prevent data loss and improve the application's availability without impacting existing users.

Here are some authoritative links for further research:

Amazon SQS: https://aws.amazon.com/sqs/
Amazon DynamoDB: https://aws.amazon.com/dynamodb/
Serverless Architectures with AWS Lambda: https://aws.amazon.com/serverless/
DynamoDB best practices: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/best-practices.html

Question: 208 **Exam Heist**

A company needs to move data from an Amazon EC2 instance to an Amazon S3 bucket. The company must ensure that no API calls and no data are routed through public internet routes. Only the EC2 instance can have access to upload data to the S3 bucket.

Which solution will meet these requirements?

A. Create an interface VPC endpoint for Amazon S3 in the subnet where the EC2 instance is located. Attach a resource policy to the S3 bucket to only allow the EC2 instance's IAM role for access.

B. Create a gateway VPC endpoint for Amazon S3 in the Availability Zone where the EC2 instance is located. Attach appropriate security groups to the endpoint. Attach a resource policy to the S3 bucket to only allow the EC2 instance's IAM role for access.

C. Run the nslookup tool from inside the EC2 instance to obtain the private IP address of the S3 bucket's service API endpoint. Create a route in the VPC route table to provide the EC2 instance with access to the S3 bucket. Attach a resource policy to the S3 bucket to only allow the EC2 instance's IAM role for access.

D. Use the AWS provided, publicly available ip-ranges.json file to obtain the private IP address of the S3 bucket's service API endpoint. Create a route in the VPC route table to provide the EC2 instance with access to the S3 bucket. Attach a resource policy to the S3 bucket to only allow the EC2 instance's IAM role for access.

Answer: A

Explanation:

Here's a detailed justification for why option A is the correct answer, along with relevant cloud computing concepts and supporting links:

The question emphasizes secure data transfer from an EC2 instance to S3 without using the public internet and restricting access to only that EC2 instance. Option A directly addresses these requirements using interface VPC endpoints and S3 bucket policies.

Here's why option A is correct:

Interface VPC Endpoint for S3: Interface endpoints use AWS PrivateLink to provide private connectivity to S3. They expose S3 as a network interface within your VPC, allowing EC2 instances to access S3 without traversing the public internet. Traffic stays within the AWS network. This fulfills the requirement to avoid public internet routes.

AWS PrivateLink Documentation
Using S3 with VPC Endpoints

S3 Bucket Policy: The S3 bucket policy restricts access to the bucket. By allowing access only from the EC2 instance's IAM role, you prevent other resources or users from uploading data to the bucket, enforcing the security requirement. The IAM role attached to the EC2 instance provides the necessary permissions to upload the data via the private endpoint.

S3 Bucket Policies

Here's why the other options are incorrect:

Option B (Gateway VPC Endpoint): Gateway endpoints only support S3 and DynamoDB and, more importantly, require modifications to the route tables. They don't provide as granular control as interface endpoints and might not be suitable when needing to restrict access to a single EC2 instance based on its IAM role via the endpoint. Gateway Endpoints operate on the principle of routing traffic destined for S3 (or DynamoDB) through the gateway instead of the internet.

Options C and D (Using Public IP Ranges and Route Tables): These options are incorrect because they involve mapping the EC2 instance directly to the IP addresses of the S3 service. While technically feasible, this approach is highly discouraged for several reasons:

Reliance on Public IP Addresses: The approach relies on the public IP address of the S3 endpoint and routing tables, which will use public internet routes, contradicting the requirement of not using the public internet.
Maintenance Overhead: IP addresses of AWS services can change, which would require constant monitoring and updating of your routing tables.

Security Concerns: Bypassing AWS PrivateLink introduces potential security vulnerabilities.
AWS IP Address Ranges

In summary, option A provides the most secure and manageable solution by utilizing interface VPC endpoints for private connectivity and S3 bucket policies for access control, fulfilling all the requirements outlined in the question.

Question: 209 **Exam Heist**

A solutions architect is designing the architecture of a new application being deployed to the AWS Cloud. The application will run on Amazon EC2 On-Demand Instances and will automatically scale across multiple Availability Zones. The EC2 instances will scale up and down frequently throughout the day. An Application Load Balancer (ALB) will handle the load distribution. The architecture needs to support distributed session data management. The company is willing to make changes to code if needed.

What should the solutions architect do to ensure that the architecture supports distributed session data management?

A. Use Amazon ElastiCache to manage and store session data.
B. Use session affinity (sticky sessions) of the ALB to manage session data.
C. Use Session Manager from AWS Systems Manager to manage the session.
D. Use the GetSessionToken API operation in AWS Security Token Service (AWS STS) to manage the session.

Answer: A

Explanation:

The correct answer is A: Use Amazon ElastiCache to manage and store session data. Here's why:

Distributed Session Management Requirement: The application scales across multiple Availability Zones. Session data needs to be accessible regardless of which EC2 instance handles a user's request. Storing session data locally on an EC2 instance would lead to inconsistencies and a poor user experience if a user is routed to a different instance that doesn't have their session data.

ElastiCache as a Centralized Session Store: ElastiCache (specifically using Memcached or Redis) provides a centralized, in-memory data store that can be accessed by all EC2 instances. This ensures session data is consistent and available regardless of which instance handles the request. Changes to code are required to serialize and store session data in ElastiCache and retrieve it upon subsequent requests.

Why other options are incorrect:

B. Session Affinity (Sticky Sessions): Sticky sessions route a user to the same EC2 instance for the duration of their session. This violates the principle of scaling across Availability Zones effectively. If the instance the user is stuck to fails, the session is lost. Also, it negates the benefit of load balancing, as some instances may become overloaded if many users are "stuck" to them. While it simplifies session management initially, it creates a single point of failure and hinders scaling.

C. Session Manager: Session Manager is a capability of AWS Systems Manager that allows you to manage your EC2 instances through a browser-based interactive shell or through the AWS CLI. It is designed for instance management, not session data management for web applications.

D. GetSessionToken API operation in AWS STS: STS GetSessionToken is used for creating temporary security credentials for IAM users or roles. This is relevant to managing AWS access permissions, not for managing web application session data.

Why ElastiCache is suitable: ElastiCache is designed for caching and session management. Redis offers advanced features such as persistence and replication for improved reliability and availability. Memcached offers simplicity and high performance for stateless session caching.

Authoritative Links:

Amazon ElastiCache: https://aws.amazon.com/elasticache/
ElastiCache Use Cases (Session Management): https://aws.amazon.com/elasticache/use-cases/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/

Question: 210 **Exam Heist**

A company offers a food delivery service that is growing rapidly. Because of the growth, the company's order processing system is experiencing scaling problems during peak traffic hours. The current architecture includes the following:

• A group of Amazon EC2 instances that run in an Amazon EC2 Auto Scaling group to collect orders from the application
• Another group of EC2 instances that run in an Amazon EC2 Auto Scaling group to fulfill orders

The order collection process occurs quickly, but the order fulfillment process can take longer. Data must not be lost because of a scaling event.

A solutions architect must ensure that the order collection process and the order fulfillment process can both scale properly during peak traffic hours. The solution must optimize utilization of the company's AWS resources.

Which solution meets these requirements?

A. Use Amazon CloudWatch metrics to monitor the CPU of each instance in the Auto Scaling groups. Configure each Auto Scaling group's minimum capacity according to peak workload values.

B. Use Amazon CloudWatch metrics to monitor the CPU of each instance in the Auto Scaling groups. Configure a CloudWatch alarm to invoke an Amazon Simple Notification Service (Amazon SNS) topic that creates additional Auto Scaling groups on demand.

C. Provision two Amazon Simple Queue Service (Amazon SQS) queues: one for order collection and another for order fulfillment. Configure the EC2 instances to poll their respective queue. Scale the Auto Scaling groups based on notifications that the queues send.

D. Provision two Amazon Simple Queue Service (Amazon SQS) queues: one for order collection and another for order fulfillment. Configure the EC2 instances to poll their respective queue. Create a metric based on a backlog per instance calculation. Scale the Auto Scaling groups based on this metric.

Answer: D

Explanation:

The correct answer is **D** because it addresses the scaling issues and resource optimization concerns effectively.

Here's a detailed justification:

Queues for Decoupling: Using Amazon SQS queues for order collection and fulfillment decouples the two processes. This prevents the slower fulfillment process from overwhelming the order collection process. Decoupling allows each process to scale independently.
Reliable Message Storage: SQS guarantees that messages will be delivered, preventing data loss during scaling events. If an instance goes down, messages remain in the queue until another instance processes them.
Polling for Work Distribution: Configuring EC2 instances to poll their respective SQS queues allows them to efficiently retrieve and process orders.
Backlog-Based Scaling: Creating a metric based on the "backlog per instance" is crucial for effective Auto

Scaling. This metric reflects the actual demand and helps the Auto Scaling groups to add or remove instances only when needed, optimizing resource utilization. Scaling based on CPU utilization (as suggested in options A and B) might not be as effective, as instances could be waiting for I/O or external dependencies.

Option C uses queue notifications which can trigger scaling events but the queue-based notifications are usually not optimal to measure backlog compared to the "backlog per instance" mechanism. Option A does not provide a way to scale automatically. Option B is suboptimal because SNS would just create additional Auto Scaling Groups on demand, not individual instances to scale with load.

Authoritative Links:

Amazon SQS: https://aws.amazon.com/sqs/
Amazon EC2 Auto Scaling: https://aws.amazon.com/autoscaling/
Decoupling Applications with SQS: https://aws.amazon.com/blogs/architecture/queue-based-load-leveling-architecture-for-scaling-compute-resources/

Question: 211 **Exam Heist**

A company hosts multiple production applications. One of the applications consists of resources from Amazon EC2, AWS Lambda, Amazon RDS, Amazon Simple Notification Service (Amazon SNS), and Amazon Simple Queue Service (Amazon SQS) across multiple AWS Regions. All company resources are tagged with a tag name of “application” and a value that corresponds to each application. A solutions architect must provide the quickest solution for identifying all of the tagged components.

Which solution meets these requirements?

A. Use AWS CloudTrail to generate a list of resources with the application tag.
B. Use the AWS CLI to query each service across all Regions to report the tagged components.
C. Run a query in Amazon CloudWatch Logs Insights to report on the components with the application tag.
D. Run a query with the AWS Resource Groups Tag Editor to report on the resources globally with the application tag.

Answer: D

Explanation:

The correct answer is D because AWS Resource Groups Tag Editor is specifically designed for quickly and centrally managing and querying tagged resources across multiple AWS services and Regions.

Here's a detailed justification:

Purpose-Built Tool: The AWS Resource Groups Tag Editor is a dedicated service for managing AWS resource tags. Its primary function is to search for resources by tag across different services and Regions. This makes it the most efficient tool for the described task.

Global Reach: It operates across multiple AWS Regions, allowing a single query to identify resources tagged with "application" and their corresponding values, regardless of the Region they reside in.

Speed and Efficiency: Compared to other options, Tag Editor offers the fastest way to identify tagged components as it is purpose-built for this type of search.

AWS CloudTrail (Option A): CloudTrail records API calls made to AWS services. While it logs resource creation and modification events, it does not offer a direct or efficient way to query resources based on tags. It's more focused on auditing and security.

AWS CLI (Option B): Using the AWS CLI would require writing scripts to iterate through each AWS service and Region, querying resources and filtering by tag. This is a significantly more complex, time-consuming, and

less efficient solution than using Tag Editor.

Amazon CloudWatch Logs Insights (Option C): CloudWatch Logs Insights analyzes log data. Tagging information is not typically directly available within CloudWatch logs unless specifically logged by an application. Therefore, it is not suitable for identifying resources based on their tags.

In conclusion, AWS Resource Groups Tag Editor provides the quickest and most efficient solution for identifying all tagged components across multiple AWS Regions, making it the ideal choice for the architect's needs.

Supporting documentation:

AWS Resource Groups Tag Editor: https://docs.aws.amazon.com/awsconsole/latest/userguide/resource-groups.html
Tagging AWS Resources: https://docs.aws.amazon.com/general/latest/gr/aws_tagging.html

Question: 212 **Exam Heist**

A company needs to export its database once a day to Amazon S3 for other teams to access. The exported object size varies between 2 GB and 5 GB. The S3 access pattern for the data is variable and changes rapidly. The data must be immediately available and must remain accessible for up to 3 months. The company needs the most cost-effective solution that will not increase retrieval time.

Which S3 storage class should the company use to meet these requirements?

A. S3 Intelligent-Tiering
B. S3 Glacier Instant Retrieval
C. S3 Standard
D. S3 Standard-Infrequent Access (S3 Standard-IA)

Answer: A

Explanation:

Here's a detailed justification for choosing S3 Intelligent-Tiering in this scenario:

The core requirement is cost-effectiveness without compromising immediate availability and accessibility for up to 3 months, even with varying data access patterns.

S3 Standard: While offering immediate availability, it's generally more expensive for data that isn't frequently accessed. Since the access patterns are variable, always storing in Standard might not be the most cost-effective.

S3 Standard-IA (Infrequent Access): Cheaper storage than Standard, but has retrieval costs and is best for data accessed less frequently. While some teams might access it less frequently, the variable and rapidly changing access patterns make it less ideal than Intelligent-Tiering. It is optimized for data that is accessed less frequently but requires rapid access when needed. Since access varies, you could end up paying more in retrieval fees.

S3 Glacier Instant Retrieval: Designed for archival with millisecond retrieval times, it's cheaper than S3 Standard but more expensive for frequently accessed data than Standard-IA or Intelligent-Tiering and adds retrieval cost.

S3 Intelligent-Tiering: This class automatically moves data between frequent, infrequent, and archive access tiers based on usage patterns, without performance impact or operational overhead. It analyzes access patterns and transitions objects to the most cost-effective tier automatically. Since the question states the

access pattern for the data is variable and changes rapidly, Intelligent-Tiering can dynamically move data to cheaper tiers when it is accessed less frequently, and quickly revert to more expensive tiers when the data access pattern changes, maximizing cost savings.

Therefore, S3 Intelligent-Tiering is the most suitable because it optimizes costs based on the unpredictable access patterns, ensures immediate availability and accessibility, and handles the variable object sizes.

Authoritative Links:

AWS S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
S3 Intelligent-Tiering: https://aws.amazon.com/s3/intelligent-tiering/

Question: 213 **Exam Heist**

A company is developing a new mobile app. The company must implement proper traffic filtering to protect its Application Load Balancer (ALB) against common application-level attacks, such as cross-site scripting or SQL injection. The company has minimal infrastructure and operational staff. The company needs to reduce its share of the responsibility in managing, updating, and securing servers for its AWS environment.

What should a solutions architect recommend to meet these requirements?

A. Configure AWS WAF rules and associate them with the ALB.

B. Deploy the application using Amazon S3 with public hosting enabled.

C. Deploy AWS Shield Advanced and add the ALB as a protected resource.

D. Create a new ALB that directs traffic to an Amazon EC2 instance running a third-party firewall, which then passes the traffic to the current ALB.

Answer: A

Explanation:

The correct answer is **A. Configure AWS WAF rules and associate them with the ALB.**

Here's a detailed justification:

AWS WAF (Web Application Firewall) is specifically designed to protect web applications from common web exploits and bots that can affect availability, compromise security, or consume excessive resources. It allows you to define customizable web security rules that control which traffic to allow or block to your web applications. By associating these rules with the ALB, you can filter malicious traffic attempting application-level attacks like cross-site scripting (XSS) and SQL injection.

AWS WAF addresses the requirement of proper traffic filtering to protect against these specific attacks mentioned in the scenario. The service is tightly integrated with the ALB, making configuration and deployment relatively straightforward. Critically, AWS WAF is a managed service. This aligns perfectly with the requirement to reduce the company's operational burden of managing, updating, and securing servers. AWS handles the underlying infrastructure, patching, and scaling of WAF.

Option B is incorrect because Amazon S3 with public hosting is primarily for static content and doesn't provide the necessary traffic filtering or protection against application-level attacks.

Option C is incorrect because AWS Shield Advanced provides DDoS protection, primarily against volumetric attacks that target network and transport layers (Layers 3 & 4). While valuable for availability, it's not designed for application-level (Layer 7) attack filtering like WAF. Shield Advanced is also more expensive and likely overkill for the given requirements.

Option D is incorrect because creating a new ALB and EC2 instance running a third-party firewall increases

operational overhead and infrastructure management, directly contradicting the requirement to minimize this. It also introduces unnecessary complexity. The company must then manage, patch, and scale the EC2 instance and the third-party firewall.

In summary, AWS WAF provides the necessary application-level security with minimal management overhead, perfectly aligning with the company's requirements.

Authoritative Links for Further Research:

AWS WAF: https://aws.amazon.com/waf/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
AWS Shield: https://aws.amazon.com/shield/

Question: 214 **Exam Heist**

A company's reporting system delivers hundreds of .csv files to an Amazon S3 bucket each day. The company must convert these files to Apache Parquet format and must store the files in a transformed data bucket.

Which solution will meet these requirements with the LEAST development effort?

A. Create an Amazon EMR cluster with Apache Spark installed. Write a Spark application to transform the data. Use EMR File System (EMRFS) to write files to the transformed data bucket.

B. Create an AWS Glue crawler to discover the data. Create an AWS Glue extract, transform, and load (ETL) job to transform the data. Specify the transformed data bucket in the output step.

C. Use AWS Batch to create a job definition with Bash syntax to transform the data and output the data to the transformed data bucket. Use the job definition to submit a job. Specify an array job as the job type.

D. Create an AWS Lambda function to transform the data and output the data to the transformed data bucket. Configure an event notification for the S3 bucket. Specify the Lambda function as the destination for the event notification.

Answer: B

Explanation:

Option B, utilizing AWS Glue, is the most efficient solution because it's a fully managed ETL (Extract, Transform, Load) service specifically designed for data transformation tasks like converting CSV files to Parquet and moving them between S3 buckets. Glue crawlers automatically discover the schema of the CSV files, eliminating the need for manual schema definition. The Glue ETL job simplifies the transformation process, allowing you to write code (typically in Python or Scala) to convert the data to Parquet format and write it to the desired S3 bucket. Glue handles the infrastructure management, scaling, and job orchestration, minimizing operational overhead and development effort.

Option A, using Amazon EMR with Apache Spark, is a valid approach but involves significantly more development and operational complexity. It requires setting up and managing an EMR cluster, writing and deploying Spark applications, and configuring EMRFS for S3 access. While powerful, it's overkill for a simple data transformation task.

Option C, employing AWS Batch with a Bash script, might work for small datasets, but it's not scalable or efficient for handling hundreds of CSV files daily. Managing dependencies, error handling, and job orchestration with Bash scripts becomes cumbersome quickly. Array jobs help with parallelization, but Glue offers more sophisticated parallel processing capabilities out-of-the-box.

Option D, using an AWS Lambda function, is generally unsuitable for large file transformations. Lambda functions have limitations on execution time and memory, making them impractical for processing hundreds of CSV files daily. Lambda is better suited for smaller, event-driven tasks. Furthermore, S3 event notifications could trigger numerous Lambda invocations, potentially leading to concurrency issues and increased costs.

Therefore, AWS Glue provides a purpose-built, managed service for data transformation, reducing development effort and operational overhead compared to the other options, making it the best choice for this scenario.

Supporting Documentation:

AWS Glue: https://aws.amazon.com/glue/
AWS Glue ETL Jobs: https://docs.aws.amazon.com/glue/latest/dg/etl-tutorial-etl.html
AWS Glue Crawlers: https://docs.aws.amazon.com/glue/latest/dg/add-crawler.html

Question: 215 **Exam Heist**

A company has 700 TB of backup data stored in network attached storage (NAS) in its data center. This backup data need to be accessible for infrequent regulatory requests and must be retained 7 years. The company has decided to migrate this backup data from its data center to AWS. The migration must be complete within 1 month. The company has 500 Mbps of dedicated bandwidth on its public internet connection available for data transfer.

What should a solutions architect do to migrate and store the data at the LOWEST cost?

A. Order AWS Snowball devices to transfer the data. Use a lifecycle policy to transition the files to Amazon S3 Glacier Deep Archive.

B. Deploy a VPN connection between the data center and Amazon VPC. Use the AWS CLI to copy the data from on premises to Amazon S3 Glacier.

C. Provision a 500 Mbps AWS Direct Connect connection and transfer the data to Amazon S3. Use a lifecycle policy to transition the files to Amazon S3 Glacier Deep Archive.

D. Use AWS DataSync to transfer the data and deploy a DataSync agent on premises. Use the DataSync task to copy files from the on-premises NAS storage to Amazon S3 Glacier.

Answer: A

Explanation:

The correct answer is A. Here's why:

A. Order AWS Snowball devices to transfer the data. Use a lifecycle policy to transition the files to Amazon S3 Glacier Deep Archive.

Data Transfer: With 700 TB of data and only 500 Mbps bandwidth, transferring data over the internet within one month is practically impossible. Snowball Edge provides a physical way to quickly transfer large amounts of data to AWS. Using a physical device bypasses the limitations of the network bandwidth, allowing the company to meet the 1-month migration requirement.
Storage Cost: S3 Glacier Deep Archive is the lowest cost storage class, suitable for data that is infrequently accessed but requires long-term retention. Since the data is for infrequent regulatory requests and needs to be retained for 7 years, Glacier Deep Archive fits perfectly.
Lifecycle Policy: A lifecycle policy automates the transition of data from S3 Standard (or another S3 storage class) to Glacier Deep Archive after the initial transfer, further reducing storage costs without manual intervention.
Cost Optimization: This solution minimizes costs by using a bulk data transfer method (Snowball Edge) and the most cost-effective storage class (Glacier Deep Archive) for the archive data.

Why other options are less suitable:

B. Deploy a VPN connection and AWS CLI to Amazon S3 Glacier: A VPN connection with 500 Mbps bandwidth is insufficient for transferring 700 TB of data in one month. The cost of this operation will dramatically rise because of the operational cost. Also, transferring directly to Glacier would be inefficient as Glacier is designed for archival and restore operations.

C. Provision a 500 Mbps AWS Direct Connect to Amazon S3: While Direct Connect provides a dedicated network connection, 500 Mbps is still too slow to transfer 700 TB in one month. Direct Connect also introduces significant upfront costs and monthly recurring charges, making it less cost-effective for a one-time migration.
D. Use AWS DataSync to transfer the data to Amazon S3 Glacier: Although AWS DataSync accelerates data transfer over the network, transferring 700 TB of data over a 500 Mbps connection in one month is not realistic. It's not designed to transfer directly to Amazon S3 Glacier.

Supporting documentation and further reading:

AWS Snow Family: https://aws.amazon.com/snowball/
Amazon S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
Amazon S3 Lifecycle: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-transition-general-considerations.html
AWS DataSync: https://aws.amazon.com/datasync/

Question: 216 **Exam Heist**

A company has a serverless website with millions of objects in an Amazon S3 bucket. The company uses the S3 bucket as the origin for an Amazon CloudFront distribution. The company did not set encryption on the S3 bucket before the objects were loaded. A solutions architect needs to enable encryption for all existing objects and for all objects that are added to the S3 bucket in the future.

Which solution will meet these requirements with the LEAST amount of effort?

A. Create a new S3 bucket. Turn on the default encryption settings for the new S3 bucket. Download all existing objects to temporary local storage. Upload the objects to the new S3 bucket.

B. Turn on the default encryption settings for the S3 bucket. Use the S3 Inventory feature to create a .csv file that lists the unencrypted objects. Run an S3 Batch Operations job that uses the copy command to encrypt those objects.

C. Create a new encryption key by using AWS Key Management Service (AWS KMS). Change the settings on the S3 bucket to use server-side encryption with AWS KMS managed encryption keys (SSE-KMS). Turn on versioning for the S3 bucket.

D. Navigate to Amazon S3 in the AWS Management Console. Browse the S3 bucket's objects. Sort by the encryption field. Select each unencrypted object. Use the Modify button to apply default encryption settings to every unencrypted object in the S3 bucket.

Answer: B

Explanation:

The most efficient solution to encrypt existing and future S3 objects for a CloudFront-backed website involves minimizing manual intervention and leveraging automated features of S3.

Option B is the best approach because it utilizes S3's built-in features for both ongoing and retroactive encryption. Enabling default encryption on the S3 bucket ensures that all new objects uploaded will automatically be encrypted with the specified encryption method (SSE-S3 or SSE-KMS), eliminating future manual effort. S3 Inventory provides a cost-effective and scalable way to identify unencrypted objects. S3 Batch Operations allows you to perform actions on a large number of S3 objects in an automated and auditable manner. Using a copy operation within Batch Operations allows for the re-writing and encryption of existing objects in place without data transfer to an external location. This simplifies the process and reduces operational overhead.

Option A is less efficient. Creating a new bucket involves significant data transfer and potential disruption to the existing CloudFront distribution since the origin would have to be updated. Downloading and uploading all objects also consumes network bandwidth and compute resources.

Option C is more complex than necessary. While SSE-KMS is a valid encryption method, the problem doesn't explicitly require KMS. Also, it doesn't include a straightforward way to re-encrypt existing objects. Simply turning on versioning won't encrypt existing objects.

Option D is the least efficient. Manually modifying each object through the console is impractical and error-prone for a bucket containing millions of objects. This is not a scalable solution.

Therefore, option B effectively addresses both present and future encryption requirements with the least amount of manual labor, making use of native S3 capabilities designed for large-scale object management.

Relevant Links:

Amazon S3 Default Encryption: https://docs.aws.amazon.com/AmazonS3/latest/userguide/default-bucket-encryption.html
Amazon S3 Inventory: https://docs.aws.amazon.com/AmazonS3/latest/userguide/storage-inventory.html
Amazon S3 Batch Operations: https://docs.aws.amazon.com/AmazonS3/latest/userguide/batch-ops-basics.html

Question: 217 **Exam Heist**

A company runs a global web application on Amazon EC2 instances behind an Application Load Balancer. The application stores data in Amazon Aurora. The company needs to create a disaster recovery solution and can tolerate up to 30 minutes of downtime and potential data loss. The solution does not need to handle the load when the primary infrastructure is healthy.

What should a solutions architect do to meet these requirements?

A. Deploy the application with the required infrastructure elements in place. Use Amazon Route 53 to configure active-passive failover. Create an Aurora Replica in a second AWS Region.

B. Host a scaled-down deployment of the application in a second AWS Region. Use Amazon Route 53 to configure active-active failover. Create an Aurora Replica in the second Region.

C. Replicate the primary infrastructure in a second AWS Region. Use Amazon Route 53 to configure active-active failover. Create an Aurora database that is restored from the latest snapshot.

D. Back up data with AWS Backup. Use the backup to create the required infrastructure in a second AWS Region. Use Amazon Route 53 to configure active-passive failover. Create an Aurora second primary instance in the second Region.

Answer: A

Explanation:

The best solution to meet the recovery time objective (RTO) of 30 minutes and tolerance for data loss, without requiring full load handling during normal operations, is option A. Here's why:

Option A Explanation:

Deployed Infrastructure in DR Region: Having the application infrastructure (EC2 instances, ALBs) pre-deployed in the DR region (but not actively serving traffic) significantly reduces recovery time. Building the infrastructure from scratch during a disaster would exceed the 30-minute RTO.
Active-Passive Failover with Route 53: Amazon Route 53's active-passive failover configuration allows traffic to be directed to the DR region only when the primary region is unavailable. This aligns with the requirement of not handling load when the primary is healthy.
Aurora Replica in DR Region: Creating an Aurora Replica in the DR region ensures that the data is replicated asynchronously from the primary region. In a disaster scenario, the Aurora Replica can be promoted to a standalone instance, minimizing data loss and recovery time. This is a critical advantage since the question tolerates limited data loss, asynchronous replication meets that condition, and the warm standby

infrastructure is already in place.

Why other options are less suitable:

Option B (Active-Active): Active-active requires constantly handling load in both regions, which isn't needed when the primary is healthy. While it can provide faster failover, the problem states that it does not need to handle the load when the primary infrastructure is healthy.
Option C (Active-Active with Snapshot Restore): The infrastructure replication is useful, but using an Aurora database restored from a snapshot will take too long to get running (much longer than 30 mins). Replicas provide faster failover.
Option D (Backup and Restore): AWS Backup is a solid backup strategy, but restoring from backup to build the infrastructure and restore the database will take considerably longer than 30 minutes, violating the RTO. Also, you can't create a "second primary instance," you can only create an Aurora Replica.

Supporting Cloud Computing Concepts:

Disaster Recovery Strategies: Option A embodies a "warm standby" DR strategy, balancing cost and recovery time.
RTO and RPO: The solution is tailored to the specific RTO (30 minutes) and acceptable data loss (RPO).
Route 53 Failover: Route 53 provides a highly available DNS service for failover.
Aurora Replication: Aurora provides built-in replication for DR and high availability.

Authoritative Links:

AWS Disaster Recovery: https://aws.amazon.com/disaster-recovery/
Amazon Route 53 Health Checks and Failover:
https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-failover.html
Amazon Aurora Global Database: https://aws.amazon.com/rds/aurora/global-database/ (While Global Database is more advanced than a simple replica, it explains the principles of Aurora replication)

Question: 218 **Exam Heist**

A company has a web server running on an Amazon EC2 instance in a public subnet with an Elastic IP address. The default security group is assigned to the EC2 instance. The default network ACL has been modified to block all traffic. A solutions architect needs to make the web server accessible from everywhere on port 443.

Which combination of steps will accomplish this task? (Choose two.)

A. Create a security group with a rule to allow TCP port 443 from source 0.0.0.0/0.

B. Create a security group with a rule to allow TCP port 443 to destination 0.0.0.0/0.

C. Update the network ACL to allow TCP port 443 from source 0.0.0.0/0.

D. Update the network ACL to allow inbound/outbound TCP port 443 from source 0.0.0.0/0 and to destination 0.0.0.0/0.

E. Update the network ACL to allow inbound TCP port 443 from source 0.0.0.0/0 and outbound TCP port 32768-65535 to destination 0.0.0.0/0.

Answer: AE

Explanation:

The correct answer is AE because it addresses both the security group and network ACL configurations required for allowing inbound HTTPS traffic (port 443) to the EC2 instance from the internet.

Option A correctly creates a security group rule that allows inbound TCP traffic on port 443 from anywhere (0.0.0.0/0). Security groups act as a virtual firewall for EC2 instances, controlling inbound and outbound

traffic. Allowing inbound traffic on port 443 is essential for HTTPS access. This rule permits any IP address to connect to the EC2 instance on the specified port.

Option E addresses the necessary changes to the network ACL (NACL). NACLs are stateless, meaning rules must be explicitly defined for both inbound and outbound traffic. The inbound rule allows TCP traffic on port 443 from any source (0.0.0.0/0). Crucially, it also includes an outbound rule allowing traffic on ephemeral ports (32768-65535) to any destination (0.0.0.0/0). When a client connects to the web server on port 443, the server's response will originate from an ephemeral port. Without the outbound rule allowing traffic from these ports, the response will be blocked by the NACL, preventing a successful connection. The ephemeral port range can vary, so it's crucial to allow a broad range.

Options B, C, and D are incorrect. B is incorrect because security group rules do not use destinations. C is incorrect because it only addresses the inbound traffic in NACL, missing the outbound ephemeral port requirement. D is also incorrect as it implies bi-directional port 443, and does not cover ephemeral port requirements for the outbound connections.

Therefore, combining the creation of the appropriate security group rule in A with the updating of the network ACL in E to allow both inbound 443 traffic and outbound ephemeral port traffic ensures that the web server is accessible from anywhere on port 443.Security GroupsNetwork ACLs

Question: 219 **Exam Heist**

A company's application is having performance issues. The application is stateful and needs to complete in-memory tasks on Amazon EC2 instances. The company used AWS CloudFormation to deploy infrastructure and used the M5 EC2 instance family. As traffic increased, the application performance degraded. Users are reporting delays when the users attempt to access the application.

Which solution will resolve these issues in the MOST operationally efficient way?

A. Replace the EC2 instances with T3 EC2 instances that run in an Auto Scaling group. Make the changes by using the AWS Management Console.

B. Modify the CloudFormation templates to run the EC2 instances in an Auto Scaling group. Increase the desired capacity and the maximum capacity of the Auto Scaling group manually when an increase is necessary.

C. Modify the CloudFormation templates. Replace the EC2 instances with R5 EC2 instances. Use Amazon CloudWatch built-in EC2 memory metrics to track the application performance for future capacity planning.

D. Modify the CloudFormation templates. Replace the EC2 instances with R5 EC2 instances. Deploy the Amazon CloudWatch agent on the EC2 instances to generate custom application latency metrics for future capacity planning.

Answer: D

Explanation:

The correct answer is D because it addresses both the performance bottleneck and operational efficiency. The application is stateful and requires in-memory tasks, indicating memory constraints. M5 instances are general-purpose, while R5 instances are memory-optimized, making them a more suitable choice for this workload. Switching to R5 instances directly addresses the root cause of the performance degradation. Furthermore, using CloudFormation for infrastructure changes is operationally efficient as it provides infrastructure-as-code, allowing for repeatable and predictable deployments.

The addition of the CloudWatch agent to collect custom application latency metrics is crucial for future capacity planning. Standard EC2 CloudWatch metrics do not track application-specific latency. Custom metrics provide insights into the actual user experience and help proactively identify potential performance issues before they impact users. Monitoring application latency allows for targeted scaling decisions based on real user needs, further optimizing operational efficiency.

Option A is incorrect because T3 instances are burstable performance instances, which are not ideal for consistent, demanding workloads. Using the AWS Management Console for manual changes lacks infrastructure-as-code benefits. Option B is flawed because manual scaling is not operationally efficient and does not dynamically respond to traffic fluctuations. While using CloudFormation is good, the scaling approach is not. Option C is partially correct by addressing the instance type and using CloudWatch. However, it uses built-in EC2 memory metrics, which are not sufficient to track custom application latency and provide granular insights for future capacity planning. Custom metrics offer better visibility into application performance.

Supporting Links:

Amazon EC2 Instance Types: https://aws.amazon.com/ec2/instance-types/
Amazon CloudWatch Custom Metrics: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/publishingMetrics.html
AWS CloudFormation: https://aws.amazon.com/cloudformation/
Auto Scaling Groups: https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-groups.html

Question: 220 **Exam Heist**

A solutions architect is designing a new API using Amazon API Gateway that will receive requests from users. The volume of requests is highly variable; several hours can pass without receiving a single request. The data processing will take place asynchronously, but should be completed within a few seconds after a request is made.

Which compute service should the solutions architect have the API invoke to deliver the requirements at the lowest cost?

A. An AWS Glue job
B. An AWS Lambda function
C. A containerized service hosted in Amazon Elastic Kubernetes Service (Amazon EKS)
D. A containerized service hosted in Amazon ECS with Amazon EC2

Answer: B

Explanation:

The correct answer is **B. An AWS Lambda function.**

Here's why:

Variable Traffic & Cost Optimization: Lambda functions are designed for event-driven, serverless computing. They automatically scale based on the number of requests, and you only pay for the compute time consumed. Given the highly variable traffic with periods of inactivity, Lambda's pay-per-use model makes it the most cost-effective choice. No charge occurs if the lambda is not invoked.

Asynchronous Processing: Lambda functions can be invoked asynchronously. API Gateway can trigger a Lambda function upon receiving a request and immediately return a response to the user. The Lambda function can then process the data in the background without blocking the user's request.

Speed: AWS Lambda processes requests very quickly which satisfies the need for a low-latency response.

Alternatives' Drawbacks:

AWS Glue Job (A): AWS Glue is designed for ETL (Extract, Transform, Load) operations on large datasets. It's not suitable for near real-time processing of individual requests. Glue is a fully-managed extract, transform, and load (ETL) service that makes it easy for customers to prepare and load their data for analytics.
Amazon EKS/ECS with EC2 (C/D): These options involve managing containers and underlying infrastructure

(EC2 instances). This incurs costs even when the service is idle, making them less cost-effective than Lambda for highly variable traffic. The overhead of managing EC2 instances, container orchestration, and scaling rules adds complexity and cost.

Serverless Paradigm: Lambda embraces the serverless paradigm, reducing operational overhead and allowing developers to focus solely on writing code.

In summary: Lambda provides the perfect balance of cost-effectiveness, scalability, and asynchronous processing capabilities to meet the requirements.

Authoritative Links:

AWS Lambda: https://aws.amazon.com/lambda/
Amazon API Gateway: https://aws.amazon.com/api-gateway/
AWS Glue: https://aws.amazon.com/glue/
Amazon ECS: https://aws.amazon.com/ecs/
Amazon EKS: https://aws.amazon.com/eks/

Question: 221 **Exam Heist**

A company runs an application on a group of Amazon Linux EC2 instances. For compliance reasons, the company must retain all application log files for 7 years. The log files will be analyzed by a reporting tool that must be able to access all the files concurrently.

Which storage solution meets these requirements MOST cost-effectively?

A. Amazon Elastic Block Store (Amazon EBS)
B. Amazon Elastic File System (Amazon EFS)
C. Amazon EC2 instance store
D. Amazon S3

Answer: D

Explanation:

The correct answer is **D. Amazon S3**. Here's why:

Cost-effectiveness: S3 is significantly more cost-effective for long-term storage of log files than EBS or EFS. S3 offers various storage classes (e.g., S3 Standard, S3 Glacier) optimized for different access patterns and retention durations. For infrequently accessed logs, S3 Glacier provides very low storage costs.
Durability and Availability: S3 provides 99.999999999% (11 9's) durability, ensuring that log files are highly unlikely to be lost. It's also designed for high availability, meaning the reporting tool can consistently access the files.
Scalability: S3 scales effortlessly to store vast amounts of data. The company can store all 7 years' worth of logs without worrying about storage limitations.
Concurrency: S3 is designed for concurrent access. The reporting tool can access multiple log files simultaneously without performance bottlenecks.
Data Lifecycle Management: S3 Lifecycle policies automate the process of transitioning log files to lower-cost storage tiers (e.g., from S3 Standard to S3 Glacier) based on their age, further optimizing costs.

Why the other options are less suitable:

A. Amazon Elastic Block Store (Amazon EBS): EBS volumes are block storage devices attached to EC2 instances. They are expensive for long-term storage of large volumes of log data. EBS is also tied to a specific availability zone, complicating data access from other instances or regions.

B. Amazon Elastic File System (Amazon EFS): EFS provides a shared file system for EC2 instances. While suitable for applications requiring shared file storage, EFS is more expensive than S3 for long-term archival of log files.
C. Amazon EC2 instance store: Instance store provides temporary block storage directly attached to the host EC2 instance. Data on the instance store is lost when the instance is stopped, terminated, or fails, making it unsuitable for long-term archival. Also, it is not designed for reporting access where data must be consistently available.

Supporting Links:

Amazon S3: https://aws.amazon.com/s3/
Amazon S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
Amazon S3 Lifecycle: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-management-overview.html

Question: 222 **Exam Heist**

A company has hired an external vendor to perform work in the company's AWS account. The vendor uses an automated tool that is hosted in an AWS account that the vendor owns. The vendor does not have IAM access to the company's AWS account.

How should a solutions architect grant this access to the vendor?

A. Create an IAM role in the company's account to delegate access to the vendor's IAM role. Attach the appropriate IAM policies to the role for the permissions that the vendor requires.

B. Create an IAM user in the company's account with a password that meets the password complexity requirements. Attach the appropriate IAM policies to the user for the permissions that the vendor requires.

C. Create an IAM group in the company's account. Add the tool's IAM user from the vendor account to the group. Attach the appropriate IAM policies to the group for the permissions that the vendor requires.

D. Create a new identity provider by choosing "AWS account" as the provider type in the IAM console. Supply the vendor's AWS account ID and user name. Attach the appropriate IAM policies to the new provider for the permissions that the vendor requires.

Answer: A

Explanation:

The correct answer is **A**. Here's a detailed justification:

Cross-account access is best achieved using IAM roles. IAM roles allow you to delegate access to your AWS resources to trusted entities (like the vendor's AWS account) without sharing IAM user credentials. This is a more secure and manageable approach than creating IAM users directly in your account for external parties.

Option A leverages this principle through a process called "IAM role trust relationship". You create an IAM role in your company's AWS account. This role defines the permissions the vendor needs. Crucially, you configure a trust relationship on this role, specifying the vendor's AWS account ID as the trusted entity. This means that principals (like IAM roles or users) within the vendor's account can assume the role in your account, provided they have the necessary permissions to do so.

On the vendor side, their automated tool will need an IAM role (or user) that is granted permission to assume the role you created in your account. This is accomplished by attaching an IAM policy to the vendor's role (or user) that grants the sts:AssumeRole permission, specifying the ARN of the role in your account.

Once this is set up, the vendor's automated tool can programmatically assume the role in your account using the AssumeRole API call. This provides the tool with temporary credentials (access key ID, secret access key, and security token) that it can then use to make API calls against your AWS resources, subject to the

permissions defined in the role.

Option B is less secure. Creating an IAM user for the vendor means sharing long-term credentials (access key ID and secret access key). Managing and rotating these credentials becomes complex, and if compromised, they provide persistent access.

Option C is incorrect. IAM groups are used to manage permissions for users within the same AWS account, not to grant cross-account access. You cannot directly add users from another account to a group in your account.

Option D misinterprets the purpose of identity providers. Identity providers (IdPs) are typically used for federating access for users managed outside of AWS, such as from an Active Directory or other SAML-based identity provider. They are not directly used for cross-account access between AWS accounts in the manner described in the question. Specifying the vendor's AWS account ID and user name as an identity provider is not a supported configuration.

In summary, using IAM roles for cross-account access is the recommended and most secure approach, avoiding the need to share long-term credentials and providing granular control over permissions.

Relevant documentation:

Granting access to your AWS account to third parties:
https://docs.aws.amazon.com/IAM/latest/UserGuide/tutorial_cross-account-with-roles.html
IAM Roles: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html
AssumeRole API: https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRole.html

Question: 223 **Exam Heist**

A company has deployed a Java Spring Boot application as a pod that runs on Amazon Elastic Kubernetes Service (Amazon EKS) in private subnets. The application needs to write data to an Amazon DynamoDB table. A solutions architect must ensure that the application can interact with the DynamoDB table without exposing traffic to the internet.

Which combination of steps should the solutions architect take to accomplish this goal? (Choose two.)

A. Attach an IAM role that has sufficient privileges to the EKS pod.
B. Attach an IAM user that has sufficient privileges to the EKS pod.
C. Allow outbound connectivity to the DynamoDB table through the private subnets' network ACLs.
D. Create a VPC endpoint for DynamoDB.
E. Embed the access keys in the Java Spring Boot code.

Answer: AD

Explanation:

The correct answer is AD. Here's why:

A. Attach an IAM role that has sufficient privileges to the EKS pod: This is the best practice for providing AWS credentials to applications running on Amazon EKS. Instead of embedding credentials directly in the code (which is a security risk), you can associate an IAM role with the pod using the IAM roles for service accounts (IRSA) feature. This allows the application to assume the role and obtain temporary AWS credentials to access DynamoDB. https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html

D. Create a VPC endpoint for DynamoDB: Creating a VPC endpoint for DynamoDB ensures that the traffic between the EKS pod and DynamoDB remains within the AWS network, specifically within your VPC. A VPC endpoint eliminates the need for the application to access DynamoDB over the public internet. Instead, the

traffic flows through the AWS backbone network, improving security and reducing latency. This is especially important for applications running in private subnets where internet access is restricted.
https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html

Why the other options are incorrect:

B. Attach an IAM user that has sufficient privileges to the EKS pod: Attaching an IAM user to an EKS pod is generally not done directly. IAM users are designed for human users or applications running outside of AWS infrastructure. Roles are intended for services.
C. Allow outbound connectivity to the DynamoDB table through the private subnets' network ACLs: While you do need to ensure the network ACLs allow outbound traffic, this is not the primary solution for secure access. This approach relies on your NAT gateway or internet gateway to allow the request, exposing it to the internet. Also, NACLs are stateless, requiring both inbound and outbound rules for a connection.
E. Embed the access keys in the Java Spring Boot code: Embedding access keys directly in the code is a major security vulnerability. If the code is compromised or inadvertently exposed, the access keys could be used to compromise your AWS account. This is strongly discouraged.

Question: 224 **Exam Heist**

A company recently migrated its web application to AWS by rehosting the application on Amazon EC2 instances in a single AWS Region. The company wants to redesign its application architecture to be highly available and fault tolerant. Traffic must reach all running EC2 instances randomly.

Which combination of steps should the company take to meet these requirements? (Choose two.)

A. Create an Amazon Route 53 failover routing policy.

B. Create an Amazon Route 53 weighted routing policy.

C. Create an Amazon Route 53 multivalue answer routing policy.

D. Launch three EC2 instances: two instances in one Availability Zone and one instance in another Availability Zone.

E. Launch four EC2 instances: two instances in one Availability Zone and two instances in another Availability Zone.

Answer: CE

Explanation:

The requirement is to achieve high availability and fault tolerance with traffic distributed randomly across EC2 instances in multiple Availability Zones.

Choice C (Create an Amazon Route 53 multivalue answer routing policy) is correct. Multivalue answer routing returns multiple healthy IP addresses in response to DNS queries. When a client makes a request, it will randomly choose one of the returned IP addresses. This achieves the desired random traffic distribution across instances. Route 53 also performs health checks on the associated endpoints and will only return IP addresses that are healthy, contributing to fault tolerance. If an instance fails a health check, its IP will no longer be returned in the DNS response.
[https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html#routing-policy-multivalue]

Choice E (Launch four EC2 instances: two instances in one Availability Zone and two instances in another Availability Zone) is also correct. Distributing EC2 instances across multiple Availability Zones protects against failures in a single AZ, increasing availability. Having two instances in each of two AZs ensures that if one AZ goes down, the application can still function using the instances in the other AZ. The multivalue answer routing will then randomly distribute traffic across these four instances. This architecture provides both high availability and random traffic distribution.

Choice A (failover routing) is incorrect because it only directs traffic to a secondary endpoint when the primary is unhealthy. It doesn't randomly distribute traffic. Choice B (weighted routing) allows you to specify weights for each endpoint, and the traffic is distributed based on these weights, not randomly. Choice D (two in one AZ, one in another) doesn't provide adequate redundancy. If the AZ with two instances fails, you lose a significant portion of your capacity.

Question: 225 **Exam Heist**

A media company collects and analyzes user activity data on premises. The company wants to migrate this capability to AWS. The user activity data store will continue to grow and will be petabytes in size. The company needs to build a highly available data ingestion solution that facilitates on-demand analytics of existing data and new data with SQL.

Which solution will meet these requirements with the LEAST operational overhead?

A. Send activity data to an Amazon Kinesis data stream. Configure the stream to deliver the data to an Amazon S3 bucket.

B. Send activity data to an Amazon Kinesis Data Firehose delivery stream. Configure the stream to deliver the data to an Amazon Redshift cluster.

C. Place activity data in an Amazon S3 bucket. Configure Amazon S3 to run an AWS Lambda function on the data as the data arrives in the S3 bucket.

D. Create an ingestion service on Amazon EC2 instances that are spread across multiple Availability Zones. Configure the service to forward data to an Amazon RDS Multi-AZ database.

Answer: B

Explanation:

The correct answer is B. Here's a breakdown of why and why the other options are less suitable:

Why Option B (Kinesis Data Firehose to Redshift) is the Best:

Managed Service and Least Overhead: Kinesis Data Firehose is a fully managed service that automatically scales to match the throughput of your data. Redshift is a fully managed data warehouse. These reduce operational burden.
Petabyte Scale: Both Firehose and Redshift are designed to handle petabyte-scale data volumes efficiently.
Data Ingestion and Delivery: Firehose handles the complexities of data ingestion, buffering, and delivery to Redshift. It can also transform data en route.
SQL Analytics: Redshift is a columnar data warehouse optimized for fast SQL queries, meeting the on-demand analytics requirement.
High Availability: Redshift inherently provides high availability through backups and replication. Firehose integrates directly and automatically with Redshift.

Why Other Options Are Less Suitable:

Option A (Kinesis Data Stream to S3): While Kinesis Data Streams can handle high volumes of data, storing the data in S3 alone doesn't directly provide SQL analytics. You'd need to use services like Athena or EMR to query the data in S3, adding operational complexity. Additionally, Data Streams requires more configuration and monitoring compared to Data Firehose.
Option C (S3 to Lambda): Lambda functions are suitable for smaller data transformations and event-driven processing. Triggering a Lambda function for every data arrival in S3, especially for petabyte-scale data, can be computationally expensive and difficult to manage. It also doesn't directly facilitate SQL analytics without an additional service.
Option D (EC2 Ingestion Service to RDS): Building a custom ingestion service on EC2 instances introduces significant operational overhead related to managing the instances, scaling, and ensuring fault tolerance.

Amazon RDS is a relational database service that may not be suitable for analyzing petabyte-scale activity data.

Supporting Concepts and Links:

Kinesis Data Firehose: https://aws.amazon.com/kinesis/data-firehose/ - Provides a serverless way to ingest data into data stores and analytics services.
Amazon Redshift: https://aws.amazon.com/redshift/ - A fully managed, petabyte-scale data warehouse service in the cloud.
AWS Well-Architected Framework (Operational Excellence Pillar): Emphasizes running and monitoring systems to deliver business value and continually improving processes and procedures. The Firehose/Redshift solution aligns well by minimizing operational overhead.

Question: 226 **Exam Heist**

A company collects data from thousands of remote devices by using a RESTful web services application that runs on an Amazon EC2 instance. The EC2 instance receives the raw data, transforms the raw data, and stores all the data in an Amazon S3 bucket. The number of remote devices will increase into the millions soon. The company needs a highly scalable solution that minimizes operational overhead.

Which combination of steps should a solutions architect take to meet these requirements? (Choose two.)

A. Use AWS Glue to process the raw data in Amazon S3.

B. Use Amazon Route 53 to route traffic to different EC2 instances.

C. Add more EC2 instances to accommodate the increasing amount of incoming data.

D. Send the raw data to Amazon Simple Queue Service (Amazon SQS). Use EC2 instances to process the data.

E. Use Amazon API Gateway to send the raw data to an Amazon Kinesis data stream. Configure Amazon Kinesis Data Firehose to use the data stream as a source to deliver the data to Amazon S3.

Answer: AE

Explanation:

Here's a detailed justification for choosing options A and E as the correct solutions, explaining why other options are less suitable, along with relevant AWS concepts and links:

Justification for A and E:

Option A: Use AWS Glue to process the raw data in Amazon S3. AWS Glue is a fully managed extract, transform, and load (ETL) service. Storing raw data in S3 is a good first step. AWS Glue is well-suited to processing large datasets in S3. This aligns with scalability and minimizing operational overhead. Glue offers serverless processing capabilities, automatically scaling resources based on demand, relieving the company from managing processing infrastructure. Glue is also cost-effective as you only pay for the compute time used.

Option E: Use Amazon API Gateway to send the raw data to an Amazon Kinesis data stream. Configure Amazon Kinesis Data Firehose to use the data stream as a source to deliver the data to Amazon S3. API Gateway provides a scalable and secure front door for the RESTful web services application. It can handle a massive increase in requests from millions of devices. Kinesis Data Streams is designed for ingesting high-velocity, real-time data streams. Using Kinesis Data Firehose automates the delivery of the data to S3. Kinesis Data Firehose also offers capabilities to transform the data before delivering to S3, which could reduce the need for Glue, depending on the company's architecture preferences. This combination decouples the data ingestion process from the processing, leading to a more resilient and scalable system.

Why other options are less suitable:

Option B: Use Amazon Route 53 to route traffic to different EC2 instances. While Route 53 can distribute traffic across EC2 instances, simply adding more EC2 instances (as implied by C) doesn't address the fundamental scaling bottlenecks of the original design. The EC2 instance is still performing both data reception and transformation, creating a single point of contention. Further, managing a scaling group of EC2 instances introduces operational overhead that the company wishes to avoid.

Option C: Add more EC2 instances to accommodate the increasing amount of incoming data. Scaling EC2 instances vertically has limitations. Horizontal scaling of EC2 instances requires managing load balancing, auto-scaling groups, and potentially database scaling, increasing operational complexity. This doesn't minimize overhead.

Option D: Send the raw data to Amazon Simple Queue Service (Amazon SQS). Use EC2 instances to process the data. While SQS can decouple the data ingestion and processing, the EC2 instances still perform the transformation and data storage. SQS still requires the management and scaling of EC2 instances, negating the operational overhead reduction requirement. Using AWS Glue or Kinesis Data Firehose reduces operational overhead more effectively because these services are serverless.

Key Concepts:

Scalability: The ability of a system to handle increasing workloads.
Operational Overhead: The effort required to manage and maintain a system.
Decoupling: Separating components of a system so that they can operate independently.
Serverless Computing: Cloud computing execution model in which the cloud provider dynamically manages the allocation of machine resources.

Authoritative Links:

AWS Glue: https://aws.amazon.com/glue/
Amazon API Gateway: https://aws.amazon.com/api-gateway/
Amazon Kinesis Data Streams: https://aws.amazon.com/kinesis/data-streams/
Amazon Kinesis Data Firehose: https://aws.amazon.com/kinesis/data-firehose/

Question: 227 **Exam Heist**

A company needs to retain its AWS CloudTrail logs for 3 years. The company is enforcing CloudTrail across a set of AWS accounts by using AWS Organizations from the parent account. The CloudTrail target S3 bucket is configured with S3 Versioning enabled. An S3 Lifecycle policy is in place to delete current objects after 3 years.

After the fourth year of use of the S3 bucket, the S3 bucket metrics show that the number of objects has continued to rise. However, the number of new CloudTrail logs that are delivered to the S3 bucket has remained consistent.

Which solution will delete objects that are older than 3 years in the MOST cost-effective manner?

A. Configure the organization's centralized CloudTrail trail to expire objects after 3 years.
B. Configure the S3 Lifecycle policy to delete previous versions as well as current versions.
C. Create an AWS Lambda function to enumerate and delete objects from Amazon S3 that are older than 3 years.
D. Configure the parent account as the owner of all objects that are delivered to the S3 bucket.

Answer: B

Explanation:

The problem is that despite having an S3 Lifecycle policy set to delete objects after 3 years, the number of objects in the S3 bucket is still increasing after 4 years. This indicates that the Lifecycle policy is not

effectively deleting all objects older than 3 years. Since S3 Versioning is enabled, the Lifecycle policy might only be deleting the current versions of objects, leaving behind previous versions.

Option A is incorrect because CloudTrail itself does not have an inherent mechanism to directly expire objects in the S3 bucket where logs are stored. CloudTrail delivers the logs to S3, and the expiration should be handled by S3's lifecycle management features.

Option B is the most cost-effective solution. The S3 Lifecycle policy needs to be configured to also delete previous versions of objects. S3 Versioning keeps multiple versions of an object. If a new version of an object is created (e.g., a new CloudTrail log file with the same name), the previous version remains stored. By modifying the Lifecycle policy to include the deletion of previous versions older than 3 years, all older versions will be purged, solving the problem without requiring any additional infrastructure or code. This leverages built-in S3 functionality, reducing operational overhead and cost.

Option C is less efficient and more costly. While a Lambda function can enumerate and delete objects, it requires writing and maintaining code, incurs Lambda execution costs, and consumes S3 API calls, which can become expensive at scale. It's an unnecessary complication when S3's Lifecycle policies can handle this natively.

Option D is irrelevant to the problem. Configuring the parent account as the owner of all objects does not directly address the issue of previous versions accumulating in the bucket. Object ownership affects access control and billing, but it doesn't control object lifecycle.

Therefore, option B is the correct solution because it directly addresses the problem by configuring the S3 Lifecycle policy to manage both current and previous versions of objects, ensuring that all logs older than 3 years are deleted in a cost-effective manner.

Supporting Links:

S3 Lifecycle Policies: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-concept.html
S3 Versioning: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html

Question: 228 **Exam Heist**

A company has an API that receives real-time data from a fleet of monitoring devices. The API stores this data in an Amazon RDS DB instance for later analysis. The amount of data that the monitoring devices send to the API fluctuates. During periods of heavy traffic, the API often returns timeout errors.

After an inspection of the logs, the company determines that the database is not capable of processing the volume of write traffic that comes from the API. A solutions architect must minimize the number of connections to the database and must ensure that data is not lost during periods of heavy traffic.

Which solution will meet these requirements?

A. Increase the size of the DB instance to an instance type that has more available memory.

B. Modify the DB instance to be a Multi-AZ DB instance. Configure the application to write to all active RDS DB instances.

C. Modify the API to write incoming data to an Amazon Simple Queue Service (Amazon SQS) queue. Use an AWS Lambda function that Amazon SQS invokes to write data from the queue to the database.

D. Modify the API to write incoming data to an Amazon Simple Notification Service (Amazon SNS) topic. Use an AWS Lambda function that Amazon SNS invokes to write data from the topic to the database.

Answer: C

Explanation:

The correct answer is C, modifying the API to write incoming data to an Amazon SQS queue and using an AWS Lambda function to write data from the queue to the database. Here's why:

The primary problem is the RDS DB instance's inability to handle the write volume from the API, leading to timeout errors. This suggests the database is overwhelmed with connection requests and write operations.

Option C addresses this by introducing a decoupling mechanism using Amazon SQS, a fully managed message queuing service. The API publishes messages containing the real-time data to the SQS queue. This immediately acknowledges the API request, preventing timeout errors even during traffic spikes.

SQS acts as a buffer, holding the incoming data until the database is ready to process it. This prevents data loss during heavy traffic periods, satisfying a key requirement.

An AWS Lambda function is then configured to be triggered by new messages in the SQS queue. Lambda automatically scales based on the number of messages, ensuring timely processing. The Lambda function retrieves data from the queue in batches and writes it to the RDS database. This reduces the number of concurrent connections to the database compared to the API directly writing data, thus minimizing the load. Lambda's ability to process messages in batches further optimizes database write operations.

Option A, increasing the DB instance size, might temporarily alleviate the issue, but it's a reactive approach and doesn't address the root cause of high connection load. It also involves downtime during scaling.

Option B, using Multi-AZ for high availability, doesn't solve the write capacity problem. Writing to multiple instances would actually increase the write load and potentially exacerbate the timeout errors. Also, direct database writes from the application to multiple instances introduce complexities in data consistency and management.

Option D, using Amazon SNS, is designed for fan-out scenarios where multiple subscribers need to receive the same message. It's not suitable for buffering and reliable, ordered processing of data intended for a single database. SNS's primary purpose is notification, not reliable message queuing for database writes. SQS guarantees message delivery, while SNS does not. Using SNS could lead to data loss.

In summary, SQS provides the necessary buffering and decoupling to protect the database from overwhelming write traffic, while Lambda efficiently processes the queued data, satisfying the requirements of minimizing connections and preventing data loss.

Further Reading:

Amazon SQS: https://aws.amazon.com/sqs/
AWS Lambda: https://aws.amazon.com/lambda/

Question: 229 **Exam Heist**

A company manages its own Amazon EC2 instances that run MySQL databases. The company is manually managing replication and scaling as demand increases or decreases. The company needs a new solution that simplifies the process of adding or removing compute capacity to or from its database tier as needed. The solution also must offer improved performance, scaling, and durability with minimal effort from operations.

Which solution meets these requirements?

A. Migrate the databases to Amazon Aurora Serverless for Aurora MySQL.
B. Migrate the databases to Amazon Aurora Serverless for Aurora PostgreSQL.
C. Combine the databases into one larger MySQL database. Run the larger database on larger EC2 instances.
D. Create an EC2 Auto Scaling group for the database tier. Migrate the existing databases to the new environment.

Answer: A

Explanation:

The most suitable solution is A, migrating the databases to Amazon Aurora Serverless for Aurora MySQL. Here's why:

Simplified Scaling: Aurora Serverless automatically scales compute capacity based on the application's needs. This eliminates the manual effort of adding or removing EC2 instances, directly addressing the company's requirement for simplified capacity management.

Improved Performance and Durability: Aurora, in general, offers improved performance and durability compared to standard MySQL due to its optimized storage engine and distributed architecture. Aurora Serverless inherits these benefits. https://aws.amazon.com/rds/aurora/

Minimal Operational Effort: Aurora Serverless handles the underlying infrastructure management, such as provisioning, patching, and backups, reducing the operational burden on the company.

MySQL Compatibility: Migrating to Aurora MySQL allows the company to leverage its existing MySQL expertise and application code with minimal changes.

Let's analyze why the other options are less suitable:

B. Aurora Serverless for PostgreSQL: While Aurora Serverless for PostgreSQL also offers automatic scaling, the company is currently using MySQL. Migrating to PostgreSQL would require significant application code changes and staff retraining.

C. Larger MySQL Database on Larger EC2 Instances: This approach is a form of vertical scaling and does not provide the dynamic scaling capabilities required by the company. It also does not reduce the manual effort involved in managing the database infrastructure. It becomes a single point of failure.

D. EC2 Auto Scaling Group: While an EC2 Auto Scaling group automates the scaling of EC2 instances, it does not address the underlying database replication and management complexities. The company would still need to manually configure and manage database replication across the instances in the Auto Scaling group, and scale them.

In summary, Aurora Serverless for Aurora MySQL offers the best combination of simplified scaling, improved performance and durability, minimal operational effort, and compatibility with the company's existing MySQL environment.

Question: 230 **Exam Heist**

A company is concerned that two NAT instances in use will no longer be able to support the traffic needed for the company’s application. A solutions architect wants to implement a solution that is highly available, fault tolerant, and automatically scalable.

What should the solutions architect recommend?

A. Remove the two NAT instances and replace them with two NAT gateways in the same Availability Zone.

B. Use Auto Scaling groups with Network Load Balancers for the NAT instances in different Availability Zones.

C. Remove the two NAT instances and replace them with two NAT gateways in different Availability Zones.

D. Replace the two NAT instances with Spot Instances in different Availability Zones and deploy a Network Load Balancer.

Answer: C

Explanation:

The best solution is to replace the two NAT instances with two NAT gateways in different Availability Zones. Here's why:

NAT Gateways vs. NAT Instances: NAT Gateways are AWS-managed, offering high availability, fault tolerance, and automatic scaling, relieving the operational burden of managing NAT instances. NAT instances require manual configuration and maintenance.
High Availability and Fault Tolerance: Deploying NAT Gateways in different Availability Zones (AZs) ensures that if one AZ experiences an outage, traffic can be routed through the NAT Gateway in the other AZ.
Automatic Scaling: NAT Gateways automatically scale to handle increased traffic, which addresses the company's concern about traffic demands.
Cost Optimization: While NAT Gateways incur a per-hour charge and data processing fees, the reduced operational overhead and improved availability often offset the cost compared to managing NAT instances, especially when considering the potential cost of downtime.
Option A is incorrect because using two NAT gateways in the same AZ doesn't provide high availability. If that AZ goes down, external connectivity is lost.
Option B is incorrect because, while using Auto Scaling Groups and NLBs with NAT Instances can improve availability, it adds complexity and doesn't match the simplicity and manageability of NAT Gateways. Also, it does not scale automatically as readily as a NAT Gateway.
Option D is incorrect because relying on Spot Instances for critical network infrastructure like NAT is risky. Spot Instances can be terminated with short notice, potentially disrupting connectivity. Also, the company required a fault-tolerant solution.

Authoritative Links:

AWS NAT Gateway: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html
NAT Instances: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-instances.html

Question: 231 **Exam Heist**

An application runs on an Amazon EC2 instance that has an Elastic IP address in VPC A. The application requires access to a database in VPC B. Both VPCs are in the same AWS account.

Which solution will provide the required access MOST securely?

A. Create a DB instance security group that allows all traffic from the public IP address of the application server in VPC A.

B. Configure a VPC peering connection between VPC A and VPC B.

C. Make the DB instance publicly accessible. Assign a public IP address to the DB instance.

D. Launch an EC2 instance with an Elastic IP address into VPC B. Proxy all requests through the new EC2 instance.

Answer: B

Explanation:

The most secure solution for allowing the application in VPC A to access the database in VPC B within the same AWS account is to establish a VPC peering connection. VPC peering enables private networking between the two VPCs, allowing resources in each VPC to communicate with each other as if they were within the same network. This eliminates the need for traffic to traverse the public internet, thereby reducing exposure to security risks.

Option A, using the public IP address in a security group rule, exposes the database to potential attacks from sources that spoof the EC2 instance's public IP. This is inherently less secure. Option C, making the database

publicly accessible, is the least secure option as it exposes the database to the entire internet. Assigning a public IP address significantly widens the attack surface and should be avoided whenever possible. Option D, using an EC2 instance as a proxy, adds unnecessary complexity and introduces another potential point of failure and vulnerability. While it does limit direct exposure, it's not as streamlined and inherently secure as VPC peering.

VPC peering offers a direct, private, and secure connection. It allows for fine-grained control through security groups and network ACLs to further restrict and monitor the allowed traffic. The network traffic stays within the AWS network infrastructure, benefiting from AWS's security measures. By configuring appropriate routing tables in both VPCs, you can ensure that traffic destined for the database instance is properly routed through the VPC peering connection. This approach adheres to the principle of least privilege, granting only the necessary access and minimizing the attack surface.

AWS VPC Peering Documentation

Question: 232 **Exam Heist**

A company runs demonstration environments for its customers on Amazon EC2 instances. Each environment is isolated in its own VPC. The company's operations team needs to be notified when RDP or SSH access to an environment has been established.

A.Configure Amazon CloudWatch Application Insights to create AWS Systems Manager OpsItems when RDP or SSH access is detected.

B.Configure the EC2 instances with an IAM instance profile that has an IAM role with the AmazonSSMManagedInstanceCore policy attached.

C.Publish VPC flow logs to Amazon CloudWatch Logs. Create required metric filters. Create an Amazon CloudWatch metric alarm with a notification action for when the alarm is in the ALARM state.

D.Configure an Amazon EventBridge rule to listen for events of type EC2 Instance State-change Notification. Configure an Amazon Simple Notification Service (Amazon SNS) topic as a target. Subscribe the operations team to the topic.

Answer: C

Explanation:

The correct answer is C: Publish VPC flow logs to Amazon CloudWatch Logs, create required metric filters, and create an Amazon CloudWatch metric alarm with a notification action. Here's why:

VPC Flow Logs Capture Network Traffic: VPC Flow Logs record information about the IP traffic going to, from, and within your VPCs. This includes source and destination IP addresses, ports, and the action taken (ACCEPT or REJECT). This is critical for detecting RDP (port 3389) and SSH (port 22) access.
https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html
CloudWatch Logs as a Central Repository: VPC Flow Logs are published to CloudWatch Logs, providing a centralized location for analysis and monitoring.
https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/WhatIsCloudWatchLogs.html
Metric Filters Extract Relevant Data: CloudWatch metric filters allow you to search for specific patterns within the log data. You can create a filter that specifically looks for log entries showing successful connections to port 22 or 3389, indicating SSH or RDP access.
https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/MonitoringPolicyExamples.html
CloudWatch Alarms Trigger Notifications: Once the metric filter identifies the specific log entries, a CloudWatch alarm can be created to trigger when the metric value (the number of SSH or RDP connections) exceeds a defined threshold. This allows the operations team to be notified immediately.
https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/AlarmThatSendsEmail.html
Notification Action for the Operations Team: The CloudWatch alarm's notification action can be configured

to send an alert to the operations team, using SNS, for instance.

Why other options are incorrect:

A (CloudWatch Application Insights): Application Insights is better suited for monitoring the health and performance of applications, not necessarily for security-related events like RDP/SSH access. It's not designed for this granular level of network traffic monitoring out of the box.
B (IAM Instance Profile with AmazonSSMManagedInstanceCore): This IAM role primarily provides Systems Manager access for instance management, not for detecting network access events. While Systems Manager can execute commands and potentially log access attempts, it's not the most efficient or direct method for this specific scenario.
D (EventBridge for EC2 Instance State-change Notification): While EventBridge can track instance state changes (e.g., running, stopped), it doesn't provide information about the reason for the state change or the network traffic associated with the instance. Detecting RDP/SSH requires examining the network activity within the instance, not just its state. It would also require further configuration to determine if the state change was related to RDP/SSH.

Question: 233 **Exam Heist**

A solutions architect has created a new AWS account and must secure AWS account root user access.

Which combination of actions will accomplish this? (Choose two.)

A. Ensure the root user uses a strong password.
B. Enable multi-factor authentication to the root user.
C. Store root user access keys in an encrypted Amazon S3 bucket.
D. Add the root user to a group containing administrative permissions.
E. Apply the required permissions to the root user with an inline policy document.

Answer: AB

Explanation:

The most secure approach to managing the root user involves enabling multi-factor authentication (MFA) and ensuring a strong password.

Justification:

A. Ensure the root user uses a strong password: The root user possesses unrestricted access to the AWS account. A strong, unique password is the first line of defense against unauthorized access. This password should adhere to best practices for complexity and length.
B. Enable multi-factor authentication to the root user: MFA adds an extra layer of security by requiring a second verification factor (e.g., a code from an authenticator app) in addition to the password. Even if the password is compromised, the attacker would still need access to the second factor to gain entry.

Why other options are incorrect:

C. Store root user access keys in an encrypted Amazon S3 bucket: It's highly discouraged to actively use root user access keys. Instead, use IAM roles and users. Storing root user access keys, even in an encrypted bucket, is an unnecessary security risk. Best practice is to avoid creating these keys at all.
D. Add the root user to a group containing administrative permissions: The root user already has complete administrative permissions. Adding them to a group is redundant and doesn't enhance security.
E. Apply the required permissions to the root user with an inline policy document: Like option D, the root user already has full administrative access. Attaching inline policies is redundant and doesn't contribute to

securing the root account itself. The goal is to limit the usage of the root account, not to further configure its inherent superuser privileges.

In summary: The best practice is to secure the root user with a strong password and MFA, and then minimize its use by creating IAM users and roles for day-to-day operations.

Supporting Links:

AWS documentation on securing root user access:
https://docs.aws.amazon.com/IAM/latest/UserGuide/id_root-user.html
AWS best practices for IAM: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

Question: 234 **Exam Heist**

A company is building a new web-based customer relationship management application. The application will use several Amazon EC2 instances that are backed by Amazon Elastic Block Store (Amazon EBS) volumes behind an Application Load Balancer (ALB). The application will also use an Amazon Aurora database. All data for the application must be encrypted at rest and in transit.

Which solution will meet these requirements?

A. Use AWS Key Management Service (AWS KMS) certificates on the ALB to encrypt data in transit. Use AWS Certificate Manager (ACM) to encrypt the EBS volumes and Aurora database storage at rest.

B. Use the AWS root account to log in to the AWS Management Console. Upload the company's encryption certificates. While in the root account, select the option to turn on encryption for all data at rest and in transit for the account.

C. Use AWS Key Management Service (AWS KMS) to encrypt the EBS volumes and Aurora database storage at rest. Attach an AWS Certificate Manager (ACM) certificate to the ALB to encrypt data in transit.

D. Use BitLocker to encrypt all data at rest. Import the company's TLS certificate keys to AWS Key Management Service (AWS KMS) Attach the KMS keys to the ALB to encrypt data in transit.

Answer: C

Explanation:

The correct solution (C) leverages AWS's native services for encryption at rest and in transit. AWS Key Management Service (KMS) is the recommended service for managing encryption keys and encrypting data at rest for both EBS volumes and Aurora databases. KMS allows centralized control over encryption keys, integrates seamlessly with other AWS services, and offers auditing capabilities. EBS volumes can be encrypted when they are created, and Aurora supports encryption at rest using KMS keys.

For encryption in transit, AWS Certificate Manager (ACM) is the preferred method. ACM allows you to easily provision, manage, and deploy SSL/TLS certificates for use with AWS services like Application Load Balancers. Attaching an ACM certificate to the ALB ensures that traffic between clients and the ALB is encrypted using HTTPS, and traffic between the ALB and the EC2 instances can also be encrypted.

Option A is incorrect because ACM is primarily used for managing SSL/TLS certificates for encryption in transit, not for encrypting storage at rest. KMS is the correct service for at-rest encryption for EBS and Aurora.

Option B is incorrect because it advocates using the root account, which is generally discouraged due to security risks. Also, AWS does not have a single "turn on encryption for all data" option. Encryption needs to be configured at the individual service level. Uploading certificates directly to the root account is also not best practice; KMS and ACM are specifically designed for this purpose.

Option D is incorrect. BitLocker is a Microsoft Windows encryption feature and is not the recommended method for encrypting EBS volumes. While you could technically use it within an EC2 instance, it is far more

complex and less integrated than using native AWS encryption. Moreover, while you can import TLS certificates into KMS, attaching KMS keys directly to the ALB to encrypt data in transit is not the standard approach. ACM is specifically designed for managing certificates used by ALBs. The ALB uses ACM to handle the TLS connection.

In summary, the chosen answer utilizes the most appropriate AWS services (KMS and ACM) in their intended ways to provide a comprehensive encryption solution for the application's data at rest and in transit, following security best practices.

Further research:

AWS KMS: https://aws.amazon.com/kms/
AWS ACM: https://aws.amazon.com/certificate-manager/
Encrypting Amazon EBS volumes:
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EBSEncryption.html
Encrypting Amazon Aurora:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Aurora.Encryption.html

Question: 235 **Exam Heist**

A company is moving its on-premises Oracle database to Amazon Aurora PostgreSQL. The database has several applications that write to the same tables. The applications need to be migrated one by one with a month in between each migration. Management has expressed concerns that the database has a high number of reads and writes. The data must be kept in sync across both databases throughout the migration.

What should a solutions architect recommend?

A. Use AWS DataSync for the initial migration. Use AWS Database Migration Service (AWS DMS) to create a change data capture (CDC) replication task and a table mapping to select all tables.

B. Use AWS DataSync for the initial migration. Use AWS Database Migration Service (AWS DMS) to create a full load plus change data capture (CDC) replication task and a table mapping to select all tables.

C. Use the AWS Schema Conversion Tool with AWS Database Migration Service (AWS DMS) using a memory optimized replication instance. Create a full load plus change data capture (CDC) replication task and a table mapping to select all tables.

D. Use the AWS Schema Conversion Tool with AWS Database Migration Service (AWS DMS) using a compute optimized replication instance. Create a full load plus change data capture (CDC) replication task and a table mapping to select the largest tables.

Answer: C

Explanation:

The correct answer is C. Here's a detailed justification:

The scenario requires migrating an Oracle database to Aurora PostgreSQL while maintaining data synchronization during a phased application migration. Key considerations are the high read/write workload and the need for continuous data replication.

Option C is the most suitable solution because it addresses all the requirements. First, the AWS Schema Conversion Tool (SCT) is crucial for converting the Oracle database schema to a compatible Aurora PostgreSQL schema. This includes data types, stored procedures, and other database objects. Without schema conversion, the migration will likely fail due to compatibility issues. AWS SCT simplifies this complex process.

Second, AWS Database Migration Service (DMS) handles both the initial data load and ongoing data replication (Change Data Capture - CDC). A "full load plus CDC" replication task ensures that the existing data

is migrated and that subsequent changes on the source database are replicated to the target database in near real-time, keeping the databases synchronized throughout the application migration process. The table mapping selection ensures all tables are included in the migration process. The phrase "memory optimized replication instance" suggests a focus on handling high throughput, which directly addresses the 'high reads and writes' concern. While a compute-optimized instance could also be used, a memory-optimized instance helps avoid bottlenecks in processing the data changes during CDC, especially with a high volume of writes.

Option A is incorrect because it uses AWS DataSync which is useful for transferring large amounts of data between on-premises storage and AWS services but isn't designed for continuous database replication with schema conversion. It lacks the critical schema conversion step and the ability to efficiently manage ongoing changes in the source database.

Option B is better than A because it utilizes AWS DMS with CDC, but still lacks schema conversion. DMS alone can't automatically handle the differences in schema between Oracle and PostgreSQL. It would require significant manual intervention and potentially custom code. DataSync is also not ideal for the initial database migration when considering database-specific migration requirements.

Option D includes the SCT and DMS with CDC, making it a good option, but it specifically focuses on migrating the "largest tables" which does not meet the requirement to migrate all data, and fails to address the 'high reads and writes' concern as thoroughly as option C. Memory-optimized instances are generally better for handling CDC, whereas compute-optimized instances are more appropriate for the initial full load.

In summary, option C provides the most comprehensive solution by addressing schema conversion, initial data migration, and continuous data synchronization using a memory-optimized replication instance to handle the database's high read/write workload.

Relevant Documentation:

AWS Schema Conversion Tool (SCT): https://aws.amazon.com/dms/schema-conversion-tool/
AWS Database Migration Service (DMS): https://aws.amazon.com/dms/

Question: 236 **Exam Heist**

A company has a three-tier application for image sharing. The application uses an Amazon EC2 instance for the front-end layer, another EC2 instance for the application layer, and a third EC2 instance for a MySQL database. A solutions architect must design a scalable and highly available solution that requires the least amount of change to the application.

Which solution meets these requirements?

A. Use Amazon S3 to host the front-end layer. Use AWS Lambda functions for the application layer. Move the database to an Amazon DynamoDB table. Use Amazon S3 to store and serve users' images.

B. Use load-balanced Multi-AZ AWS Elastic Beanstalk environments for the front-end layer and the application layer. Move the database to an Amazon RDS DB instance with multiple read replicas to serve users' images.

C. Use Amazon S3 to host the front-end layer. Use a fleet of EC2 instances in an Auto Scaling group for the application layer. Move the database to a memory optimized instance type to store and serve users' images.

D. Use load-balanced Multi-AZ AWS Elastic Beanstalk environments for the front-end layer and the application layer. Move the database to an Amazon RDS Multi-AZ DB instance. Use Amazon S3 to store and serve users' images.

Answer: D

Explanation:

The correct answer is D because it provides a scalable and highly available solution with minimal changes to the existing three-tier architecture.

Here's why:

Elastic Beanstalk for Front-End and Application Layers: Using Elastic Beanstalk handles deployment, load balancing, and auto-scaling automatically, reducing operational overhead for the front-end and application layers. Deploying across multiple Availability Zones (Multi-AZ) within Elastic Beanstalk ensures high availability. This requires minimal code changes.

Amazon RDS Multi-AZ for Database: Migrating the MySQL database to Amazon RDS (Relational Database Service) Multi-AZ offers automated failover to a standby replica in a different Availability Zone, ensuring high availability and data durability. RDS is compatible with MySQL, minimizing code changes.

Amazon S3 for Image Storage: Using Amazon S3 for storing images leverages its scalability, durability, and cost-effectiveness. Serving images directly from S3 reduces the load on the application layer and EC2 instances. This offloads image storage and delivery from the database and application servers.

Option A requires significant architectural changes, including rewriting the application layer to use Lambda and changing the database to DynamoDB, which is not compatible with MySQL, increasing development effort and risks.

Option B suggests using read replicas for serving images which is not the intended use of read replicas. The main use is to offload read traffic from the primary database. Also, it doesn't address the scalability and availability of the application layer as effectively as Elastic Beanstalk.

Option C's suggestion to use a memory-optimized instance type for the database to store images is inefficient and limits scalability compared to using S3 for image storage.

Supporting Concepts:

Scalability: The ability of a system to handle increasing workloads.
High Availability: The ability of a system to remain operational even if some components fail.
Load Balancing: Distributing traffic across multiple instances to improve performance and availability.
Multi-AZ Deployment: Deploying resources across multiple Availability Zones to ensure high availability.
Elastic Beanstalk: An AWS service for deploying and managing web applications and services.
Amazon RDS: A managed relational database service on AWS.
Amazon S3: A scalable object storage service on AWS.

Authoritative Links:

AWS Elastic Beanstalk: https://aws.amazon.com/elasticbeanstalk/
Amazon RDS Multi-AZ: https://aws.amazon.com/rds/features/multi-az/
Amazon S3: https://aws.amazon.com/s3/

Question: 237 **Exam Heist**

An application running on an Amazon EC2 instance in VPC-A needs to access files in another EC2 instance in VPC-B. Both VPCs are in separate AWS accounts. The network administrator needs to design a solution to configure secure access to EC2 instance in VPC-B from VPC-A. The connectivity should not have a single point of failure or bandwidth concerns.

Which solution will meet these requirements?

A. Set up a VPC peering connection between VPC-A and VPC-B.

B. Set up VPC gateway endpoints for the EC2 instance running in VPC-B.

C. Attach a virtual private gateway to VPC-B and set up routing from VPC-A.

D. Create a private virtual interface (VIF) for the EC2 instance running in VPC-B and add appropriate routes from VPC-A.

Answer: A

Explanation:

The correct answer is **A. Set up a VPC peering connection between VPC-A and VPC-B.**

Here's a detailed justification:

VPC peering establishes a direct networking connection between two VPCs, enabling instances in each VPC to communicate with each other as if they were within the same network. This solution directly addresses the requirement of securely connecting EC2 instances across VPCs in different AWS accounts. VPC peering doesn't route traffic through the public internet, providing a secure connection. Because VPC peering establishes direct routing between VPCs, it avoids the bandwidth limitations associated with solutions involving the public internet or single network gateways. VPC peering is designed to be highly available, ensuring that connectivity isn't disrupted by single points of failure. You can also implement routing configurations to ensure that traffic can flow both ways after the peering connection is active.

Option B is incorrect because VPC gateway endpoints are designed to provide private access to AWS services like S3 and DynamoDB, not to EC2 instances within another VPC.

Option C is incorrect because attaching a virtual private gateway (VGW) to VPC-B typically works in conjunction with a VPN connection or AWS Direct Connect. While technically possible to connect via a VGW across accounts, it's generally more complex and doesn't directly offer the same level of simplicity and high availability as VPC peering for this cross-account, intra-AWS connectivity. Additionally, VPN connections using VGWs can introduce bandwidth constraints and a single point of failure.

Option D is incorrect because creating a private virtual interface (VIF) is relevant for AWS Direct Connect, which provides a dedicated network connection from your on-premises environment to AWS. It doesn't directly solve the need for VPC-to-VPC communication within the AWS cloud. Direct Connect is typically used for hybrid cloud scenarios, not for connecting VPCs within AWS accounts.

For further research, consult the AWS documentation:

VPC Peering: https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html
VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/privatelink/vpc-endpoints.html
AWS Direct Connect: https://docs.aws.amazon.com/directconnect/latest/UserGuide/Welcome.html
Virtual Private Gateways: https://docs.aws.amazon.com/vpn/latest/s2svpn/VPC_VPN.html

Question: 238 **Exam Heist**

A company wants to experiment with individual AWS accounts for its engineer team. The company wants to be notified as soon as the Amazon EC2 instance usage for a given month exceeds a specific threshold for each account.

What should a solutions architect do to meet this requirement MOST cost-effectively?

A. Use Cost Explorer to create a daily report of costs by service. Filter the report by EC2 instances. Configure Cost Explorer to send an Amazon Simple Email Service (Amazon SES) notification when a threshold is exceeded.

B. Use Cost Explorer to create a monthly report of costs by service. Filter the report by EC2 instances. Configure Cost Explorer to send an Amazon Simple Email Service (Amazon SES) notification when a threshold is exceeded.

C. Use AWS Budgets to create a cost budget for each account. Set the period to monthly. Set the scope to EC2 instances. Set an alert threshold for the budget. Configure an Amazon Simple Notification Service (Amazon SNS) topic to receive a notification when a threshold is exceeded.

D. Use AWS Cost and Usage Reports to create a report with hourly granularity. Integrate the report data with Amazon Athena. Use Amazon EventBridge to schedule an Athena query. Configure an Amazon Simple

Notification Service (Amazon SNS) topic to receive a notification when a threshold is exceeded.

Answer: C

Explanation:

The most cost-effective solution for notifying the company when EC2 instance usage exceeds a specific threshold for each account is using AWS Budgets.

AWS Budgets allows you to set custom budgets to track your AWS costs and usage. You can define budgets on a monthly basis and scope them specifically to EC2 instances, which directly addresses the requirement. The key advantage is that Budgets support setting alert thresholds. When actual or forecasted costs exceed the defined threshold, Budgets can trigger notifications via Amazon SNS (Simple Notification Service), enabling immediate alerting. This eliminates the need for constant manual monitoring.

Option A and B, while using Cost Explorer, do not provide built-in alerting capabilities that are easily configurable to trigger notifications when a specific threshold is exceeded. While Cost Explorer reports offer insight, they require external mechanisms or manual review to generate alerts, which is less cost-effective and efficient. Amazon SES would need to be manually configured, adding complexity.

Option D, involving AWS Cost and Usage Reports, Amazon Athena, and Amazon EventBridge, is significantly more complex and costly. Although granular and powerful, it's overkill for a simple usage threshold alert. Hourly granularity and Athena queries require more resources and associated costs compared to the native capabilities of AWS Budgets.

AWS Budgets offers the optimal balance of functionality, cost-effectiveness, and ease of use for the specified requirement. It is designed for cost tracking and alerting, making it the right tool for this purpose.

Relevant link: https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-managing-costs.html

Question: 239 **Exam Heist**

A solutions architect needs to design a new microservice for a company's application. Clients must be able to call an HTTPS endpoint to reach the microservice. The microservice also must use AWS Identity and Access Management (IAM) to authenticate calls. The solutions architect will write the logic for this microservice by using a single AWS Lambda function that is written in Go 1.x.

Which solution will deploy the function in the MOST operationally efficient way?

A. Create an Amazon API Gateway REST API. Configure the method to use the Lambda function. Enable IAM authentication on the API.

B. Create a Lambda function URL for the function. Specify AWS_IAM as the authentication type.

C. Create an Amazon CloudFront distribution. Deploy the function to Integrate IAM authentication logic into the function.

D. Create an Amazon CloudFront distribution. Deploy the function to CloudFront Functions. Specify AWS_IAM as the authentication type.

Answer: A

Explanation:

The most operationally efficient solution is to use Amazon API Gateway with Lambda integration and IAM authentication. Here's why:

API Gateway's Purpose: API Gateway is specifically designed to create, publish, maintain, monitor, and

secure APIs at any scale. It acts as a front door for applications to access data, logic, or functionality from backend services like Lambda.

Lambda Integration: API Gateway natively integrates with Lambda functions. This allows direct invocation of the Lambda function upon receiving an API request, streamlining the request-response flow.

IAM Authentication: API Gateway supports IAM authentication, allowing the microservice to leverage AWS's robust identity and access management system. This eliminates the need to implement custom authentication logic within the Lambda function itself, improving security and reducing code complexity.

Operational Efficiency: Using API Gateway for tasks like authentication and routing centralizes these responsibilities, making management and maintenance more straightforward. It also provides built-in features like caching, throttling, and monitoring that contribute to operational efficiency.

Why other options are less efficient:

Lambda Function URLs (Option B): While simpler to set up initially, function URLs lack the advanced features of API Gateway, such as request transformation, caching, and comprehensive security policies. They're less suitable for complex scenarios requiring features beyond basic invocation.

CloudFront with Lambda@Edge or CloudFront Functions (Options C and D): CloudFront is primarily a Content Delivery Network (CDN) designed for caching and delivering static content. While it can execute Lambda functions with Lambda@Edge or CloudFront Functions, these are typically used for edge computing tasks like modifying HTTP headers or customizing content delivery. They are not best suited for implementing the core logic of a microservice requiring IAM authentication. Deploying the entire microservice to CloudFront would be an overkill.

In summary, API Gateway provides the most comprehensive and efficient solution for creating an HTTPS endpoint for a Lambda-backed microservice that requires IAM authentication, due to its features designed for API management and seamless integration with other AWS services.

Here are some links for more research:

Amazon API Gateway: https://aws.amazon.com/api-gateway/
AWS Lambda: https://aws.amazon.com/lambda/
IAM Authentication in API Gateway:
https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-use-iam-authorizer.html
Lambda Function URLs: https://docs.aws.amazon.com/lambda/latest/dg/lambda-urls.html
Lambda@Edge: https://aws.amazon.com/lambda/edge/
CloudFront Functions: https://aws.amazon.com/cloudfront/features/cloudfront-functions/

Question: 240 **Exam Heist**

A company previously migrated its data warehouse solution to AWS. The company also has an AWS Direct Connect connection. Corporate office users query the data warehouse using a visualization tool. The average size of a query returned by the data warehouse is 50 MB and each webpage sent by the visualization tool is approximately 500 KB. Result sets returned by the data warehouse are not cached.

Which solution provides the LOWEST data transfer egress cost for the company?

A. Host the visualization tool on premises and query the data warehouse directly over the internet.

B. Host the visualization tool in the same AWS Region as the data warehouse. Access it over the internet.

C. Host the visualization tool on premises and query the data warehouse directly over a Direct Connect connection at a location in the same AWS Region.

D. Host the visualization tool in the same AWS Region as the data warehouse and access it over a Direct Connect connection at a location in the same Region.

Answer: D

Explanation:

Here's a detailed justification for why option D is the best solution for minimizing data transfer egress costs in this scenario:

The primary goal is to reduce egress costs, which are charges incurred when data leaves an AWS Region. Options A and B involve data traversing the internet, which incurs significant egress charges. The question states that the company already has a Direct Connect connection. Direct Connect offers significantly reduced data transfer costs compared to transferring data over the public internet by establishing a dedicated network connection from on-premises to AWS.

Option C utilizes the Direct Connect connection for querying, which is better than using the internet. However, the visualization tool remains on-premises. This requires the 500 KB webpages generated by the tool to be sent back across the Direct Connect connection to the users at the corporate office, leading to unnecessary egress charges for the visualization content.

Option D eliminates internet egress charges. By hosting the visualization tool within the same AWS Region as the data warehouse, the 50 MB query results from the data warehouse and the 500 KB web pages generated by the tool are transferred within the AWS Region. Intra-region data transfer within AWS is generally free or significantly cheaper than egressing data outside of AWS. The users then access the visualization tool, and the generated webpages (500 KB) are transmitted over the Direct Connect connection only. This is significantly less egress data than transferring the 50 MB result sets across Direct Connect or the internet, as would happen with other options. Therefore, D is the most cost-effective because it leverages Direct Connect to minimize the egress costs.

In summary, option D minimizes egress costs by:

1. Keeping the majority of data transfer (50 MB query results) within the AWS Region.
2. Only using the Direct Connect connection to send the smaller webpage results (500 KB) from the visualization tool to on-premises users, resulting in the lowest egress costs.

Authoritative Links for Further Research:

AWS Direct Connect: https://aws.amazon.com/directconnect/
AWS Pricing: https://aws.amazon.com/pricing/ (Specifically, look for data transfer costs)

Question: 241 **Exam Heist**

An online learning company is migrating to the AWS Cloud. The company maintains its student records in a PostgreSQL database. The company needs a solution in which its data is available and online across multiple AWS Regions at all times.

Which solution will meet these requirements with the LEAST amount of operational overhead?

A. Migrate the PostgreSQL database to a PostgreSQL cluster on Amazon EC2 instances.

B. Migrate the PostgreSQL database to an Amazon RDS for PostgreSQL DB instance with the Multi-AZ feature turned on.

C. Migrate the PostgreSQL database to an Amazon RDS for PostgreSQL DB instance. Create a read replica in another Region.

D. Migrate the PostgreSQL database to an Amazon RDS for PostgreSQL DB instance. Set up DB snapshots to be copied to another Region.

Answer: C

Explanation:

The correct answer is **C. Migrate the PostgreSQL database to an Amazon RDS for PostgreSQL DB instance. Create a read replica in another Region.**

Here's why:

Availability and Region Redundancy: The primary requirement is for data availability across multiple AWS Regions at all times. Option C directly addresses this by using RDS cross-region read replicas. A read replica in another region provides a near real-time copy of the data, ensuring data availability even if the primary region experiences an outage.

Operational Overhead: RDS read replicas are managed services, which significantly reduces operational overhead compared to self-managing a PostgreSQL cluster on EC2 (Option A). AWS handles the replication and failover processes to the read replica region.

Multi-AZ vs. Multi-Region: Option B (Multi-AZ) provides high availability within a single region but does not address the requirement of availability across multiple regions. A Multi-AZ deployment automatically provisions a standby database instance in a different Availability Zone within the same region. Thus, it will not help in case of Regional Outage.

DB Snapshots: Option D (DB snapshots) provides a point-in-time backup, but restoring from a snapshot in another region involves a manual process that can lead to significant downtime. Snapshots are not suitable for continuous availability.

RDS Read Replica: RDS read replicas can be promoted to become standalone primary DB instances if the primary instance fails. This is a more seamless failover process than restoring from a snapshot.

In summary, using RDS with a cross-region read replica provides the best combination of multi-region data availability and minimal operational overhead, fulfilling the company's requirements in a cost-effective and manageable manner.**Authoritative Links:**

Amazon RDS Read Replicas
Working with Amazon RDS Read Replicas

Question: 242 **Exam Heist**

A company hosts its web application on AWS using seven Amazon EC2 instances. The company requires that the IP addresses of all healthy EC2 instances be returned in response to DNS queries.

Which policy should be used to meet this requirement?

A. Simple routing policy
B. Latency routing policy
C. Multivalue routing policy
D. Geolocation routing policy

Answer: C

Explanation:

The correct answer is Multivalue routing policy (C) because it's designed to return multiple healthy IP addresses in response to DNS queries. This directly addresses the company's requirement to receive the IP addresses of all healthy EC2 instances.

Here's a detailed justification:

Multivalue Answer Routing: This policy is specifically built to return multiple values (like IP addresses) for a single DNS query. This is essential for high availability and load balancing across multiple resources. [https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html#routing-policy-multivalue]

Health Checks Integration: Multivalue answer routing works in conjunction with Route 53 health checks. Route 53 monitors the health of each EC2 instance (or other endpoint). Only the IP addresses of instances that pass the health checks are returned in the DNS response.

Why other options are incorrect:

Simple routing policy: Returns only one IP address from a set of records. While it can provide basic failover if the first record is unavailable, it doesn't provide multiple healthy IPs.
Latency routing policy: Routes traffic based on the lowest latency between the user and the AWS region, irrelevant to the health of all instances.
Geolocation routing policy: Routes traffic based on the geographic location of the user, also irrelevant to the health of all instances.

Scalability and High Availability: By returning multiple healthy IP addresses, the application can distribute traffic across available EC2 instances, improving scalability and resilience. If one instance fails, traffic is automatically routed to the remaining healthy instances without any DNS propagation delay.

In summary: Multivalue answer routing provides the necessary functionality to return the IP addresses of all healthy EC2 instances, aligning with the company's requirements for high availability and load distribution. Health checks ensure only healthy instances are included in the response.

Question: 243 **Exam Heist**

A medical research lab produces data that is related to a new study. The lab wants to make the data available with minimum latency to clinics across the country for their on-premises, file-based applications. The data files are stored in an Amazon S3 bucket that has read-only permissions for each clinic.

What should a solutions architect recommend to meet these requirements?

A. Deploy an AWS Storage Gateway file gateway as a virtual machine (VM) on premises at each clinic
B. Migrate the files to each clinic’s on-premises applications by using AWS DataSync for processing.
C. Deploy an AWS Storage Gateway volume gateway as a virtual machine (VM) on premises at each clinic.
D. Attach an Amazon Elastic File System (Amazon EFS) file system to each clinic’s on-premises servers.

Answer: A

Explanation:

The correct answer is A: Deploy an AWS Storage Gateway file gateway as a virtual machine (VM) on premises at each clinic. Here's why:

Low Latency and On-Premises File Access: The key requirement is minimum latency for file-based applications on-premises. AWS Storage Gateway file gateway addresses this directly. It provides a local cache of the data, allowing on-premises applications to access frequently used files with very low latency. The files are still durably stored in S3, but the gateway acts as a local "front-end" for faster access.

File-Based Access: The scenario specifies that the data needs to be accessible to file-based applications. A file gateway presents the data as a standard file share (e.g., NFS, SMB), which is compatible with these applications.

AWS Storage Gateway Overview: AWS Storage Gateway connects an on-premises software appliance with cloud-based storage to provide seamless and secure integration between an organization's on-premises IT environment and the AWS storage infrastructure. https://aws.amazon.com/storagegateway/

Let's examine why the other options are less suitable:

B. AWS DataSync: While DataSync can move data to on-premises locations, it's more suited for initial data migration or periodic synchronization, not for providing low-latency, continuous access to frequently changing data. DataSync copies data, introducing delays compared to caching.

C. AWS Storage Gateway volume gateway: A volume gateway presents cloud-based storage volumes as iSCSI block devices to on-premises applications. This is not suitable for file-based applications, as it requires the applications to work with block storage instead of files.

D. Amazon EFS: Amazon EFS is a fully managed elastic NFS file system for use with AWS cloud services and on-premises resources. While it offers file-based access, directly attaching it to on-premises servers requires a VPN or Direct Connect connection to AWS. The latency over such a connection would be higher compared to a local file gateway. Also, EFS is generally optimized for cloud workloads, not necessarily low-latency on-premise access to files stored primarily in S3.

In summary, a file gateway allows the clinics to directly access the needed files locally, thus reducing latency to nearly zero, which is not achievable with any other of the suggested setups.

Question: 244 **Exam Heist**

A company is using a content management system that runs on a single Amazon EC2 instance. The EC2 instance contains both the web server and the database software. The company must make its website platform highly available and must enable the website to scale to meet user demand.

What should a solutions architect recommend to meet these requirements?

A. Move the database to Amazon RDS, and enable automatic backups. Manually launch another EC2 instance in the same Availability Zone. Configure an Application Load Balancer in the Availability Zone, and set the two instances as targets.

B. Migrate the database to an Amazon Aurora instance with a read replica in the same Availability Zone as the existing EC2 instance. Manually launch another EC2 instance in the same Availability Zone. Configure an Application Load Balancer, and set the two EC2 instances as targets.

C. Move the database to Amazon Aurora with a read replica in another Availability Zone. Create an Amazon Machine Image (AMI) from the EC2 instance. Configure an Application Load Balancer in two Availability Zones. Attach an Auto Scaling group that uses the AMI across two Availability Zones.

D. Move the database to a separate EC2 instance, and schedule backups to Amazon S3. Create an Amazon Machine Image (AMI) from the original EC2 instance. Configure an Application Load Balancer in two Availability Zones. Attach an Auto Scaling group that uses the AMI across two Availability Zones.

Answer: C

Explanation:

The correct answer is C because it addresses both high availability and scalability in a resilient and automated manner.

Here's a breakdown of why option C is superior:

Database High Availability and Scalability: Moving the database to Amazon Aurora with a read replica in another Availability Zone provides both high availability (in case of a failure in one AZ) and read scalability (offloading read operations to the read replica). Aurora's multi-AZ capabilities ensure data redundancy and

failover. https://aws.amazon.com/rds/aurora/
Web Server Scalability and Availability: Creating an AMI from the EC2 instance allows for consistent deployments of the web server application. The Application Load Balancer (ALB) distributes incoming traffic across multiple instances in different Availability Zones, providing high availability. https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
Automated Scaling: Attaching an Auto Scaling group to the ALB ensures that the number of EC2 instances automatically adjusts to meet demand, scaling up during peak loads and scaling down during off-peak periods. The Auto Scaling group utilizes the AMI to launch new instances, maintaining consistency. Spreading instances across two Availability Zones provides resilience. https://aws.amazon.com/autoscaling/

Why other options are less suitable:

Option A: Manual launches of EC2 instances and configuring only in one AZ do not provide automatic scaling or full high availability across different Availability Zones. Manually launching another EC2 instance is not a scalable solution, and the database backup does not provide a high available configuration.
Option B: Using a read replica in the same Availability Zone does not protect against an AZ failure, therefore does not achieve high availability. Manually launching EC2 instances has the same drawbacks of option A.
Option D: Running the database on a separate EC2 instance without using a managed database service like RDS or Aurora lacks the built-in high availability features and easier management provided by these services. Scheduling backups is good practice, but not sufficient for high availability.

Question: 245 **Exam Heist**

A company is launching an application on AWS. The application uses an Application Load Balancer (ALB) to direct traffic to at least two Amazon EC2 instances in a single target group. The instances are in an Auto Scaling group for each environment. The company requires a development environment and a production environment. The production environment will have periods of high traffic.

Which solution will configure the development environment MOST cost-effectively?

A. Reconfigure the target group in the development environment to have only one EC2 instance as a target.
B. Change the ALB balancing algorithm to least outstanding requests.
C. Reduce the size of the EC2 instances in both environments.
D. Reduce the maximum number of EC2 instances in the development environment’s Auto Scaling group.

Answer: A

Explanation:

The correct answer is A: "Reconfigure the target group in the development environment to have only one EC2 instance as a target."

Here's a detailed justification:

The primary goal is to minimize costs in the development environment while maintaining functionality. Having at least two EC2 instances in the production environment's target group and Auto Scaling group is necessary for high availability and handling production traffic spikes. However, the development environment doesn't typically require the same level of resilience or capacity.

Option A directly addresses cost reduction by minimizing resource usage in the development environment. An ALB requires at least one healthy instance in a target group to function. Reducing the number of instances from two to one in the development environment reduces the cost related to EC2 compute resources by half, without completely disabling the environment. Development teams can still deploy, test, and debug their code.

Option B, changing the ALB balancing algorithm, doesn't directly reduce costs. Least outstanding requests distributes load differently, but it doesn't eliminate the need for instances. It is mainly helpful in balancing the load between the instances; it doesn't inherently reduce the number of instances required and thus doesn't significantly reduce costs.

Option C, reducing the size of EC2 instances in both environments, might save costs across both environments, but is not the most cost-effective for the development environment, as it impacts performance on the production environment as well, which has periods of high traffic. The prompt is asking specifically for what configuration would be most cost-effective for development, which means we want to reduce overhead where the high availability requirements are low.

Option D, reducing the maximum number of EC2 instances in the development environment's Auto Scaling group, is a good idea for cost savings. However, the question mentions the application needs at least two instances for functionality to run. Therefore, reducing the maximum without reducing the minimum still will require 2 instances. Therefore it is less effective compared to Option A. Also, ASG configuration usually requires a minimum value as well, which further mitigates the cost savings and the benefits are only realized during auto-scaling.Therefore, option A is the most appropriate choice. It provides a significant cost reduction directly in the development environment without crippling its core functionality for developers.

Further reading on these AWS services can be found here:

Application Load Balancer:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html
Amazon EC2: https://aws.amazon.com/ec2/
Auto Scaling: https://aws.amazon.com/autoscaling/

Question: 246 **Exam Heist**

A company runs a web application on Amazon EC2 instances in multiple Availability Zones. The EC2 instances are in private subnets. A solutions architect implements an internet-facing Application Load Balancer (ALB) and specifies the EC2 instances as the target group. However, the internet traffic is not reaching the EC2 instances.

How should the solutions architect reconfigure the architecture to resolve this issue?

A. Replace the ALB with a Network Load Balancer. Configure a NAT gateway in a public subnet to allow internet traffic.

B. Move the EC2 instances to public subnets. Add a rule to the EC2 instances' security groups to allow outbound traffic to 0.0.0.0/0.

C. Update the route tables for the EC2 instances' subnets to send 0.0.0.0/0 traffic through the internet gateway route. Add a rule to the EC2 instances' security groups to allow outbound traffic to 0.0.0.0/0.

D. Create public subnets in each Availability Zone. Associate the public subnets with the ALB. Update the route tables for the public subnets with a route to the private subnets.

Answer: D

Explanation:

The problem is that an internet-facing Application Load Balancer (ALB) cannot directly route traffic to EC2 instances residing in private subnets. ALBs need to be placed in public subnets to receive traffic from the internet.

Option D correctly addresses this by creating public subnets for the ALB. By associating the ALB with these public subnets, the ALB can receive internet traffic. The route tables for these public subnets are then configured to route traffic to the private subnets where the EC2 instances are located. This allows the ALB to forward the internet traffic to the backend EC2 instances while keeping them isolated in private subnets for security.

Option A is incorrect because while a Network Load Balancer (NLB) can be used, it doesn't inherently solve the problem of the private subnets. A NAT gateway might allow outbound internet access, but doesn't facilitate inbound traffic to the instances through the ALB.

Option B proposes moving the EC2 instances to public subnets, which is generally not recommended for security reasons. Public subnets expose EC2 instances directly to the internet, increasing the attack surface. While adding a security group rule allows outbound traffic, it doesn't address the fundamental problem of the ALB's placement.

Option C suggests updating the route tables for the EC2 instances' subnets to use an internet gateway. This is incorrect because private subnets should not have direct routes to the internet gateway. This would effectively make the subnets public, negating the purpose of having them private in the first place.

Therefore, Option D is the most secure and appropriate solution. It utilizes public subnets for the ALB to receive internet traffic and maintains the security posture of the EC2 instances by keeping them in private subnets. The route tables ensure that the traffic is properly routed from the ALB in the public subnets to the EC2 instances in the private subnets.

Further research:

AWS Documentation - Application Load Balancers:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html
AWS Documentation - VPC Routing:
https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html
AWS Documentation - Security Groups: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html

Question: 247 **Exam Heist**

A company has deployed a database in Amazon RDS for MySQL. Due to increased transactions, the database support team is reporting slow reads against the DB instance and recommends adding a read replica.

Which combination of actions should a solutions architect take before implementing this change? (Choose two.)

A. Enable binlog replication on the RDS primary node.

B. Choose a failover priority for the source DB instance.

C. Allow long-running transactions to complete on the source DB instance.

D. Create a global table and specify the AWS Regions where the table will be available.

E. Enable automatic backups on the source instance by setting the backup retention period to a value other than 0.

Answer: CE

Explanation:

The correct answer is CE. Here's a detailed justification:

C: Allow long-running transactions to complete on the source DB instance.

Before creating a read replica, it's crucial to ensure data consistency. Interrupting a long-running transaction on the primary DB instance could lead to data corruption or an inconsistent state being replicated to the read replica. Allowing the existing transactions to complete ensures a clean cut-off point for replication to begin, guaranteeing the read replica is initialized with consistent data. Prematurely interrupting transactions would force a rollback and introduce inconsistencies that might be difficult to resolve after the read replica has been launched.

E: Enable automatic backups on the source instance by setting the backup retention period to a value other than 0.

Enabling automatic backups is a prerequisite for creating a read replica in Amazon RDS for MySQL (and other engines). RDS uses the binary logs generated by the backup process to replicate changes to the read replica. If automatic backups are disabled (retention period set to 0), binary logging is also disabled, and replication cannot occur. Therefore, configuring a backup retention period greater than zero ensures that binary logs are available for replication.

Let's discuss why the other options are incorrect:

A: Enable binlog replication on the RDS primary node. Although related to replication, binlog is enabled through enabling backups, so it is not a separate step needed as part of the checklist.
B: Choose a failover priority for the source DB instance. Failover priority is for Multi-AZ deployments and not necessary when creating a read replica, which is meant for read scaling.
D: Create a global table and specify the AWS Regions where the table will be available. Global tables are part of DynamoDB, not RDS MySQL. This is completely irrelevant to the task of creating a MySQL read replica.

In summary, ensuring that existing long-running transactions complete and enabling automatic backups are vital preparatory steps when creating an RDS for MySQL read replica to guarantee data consistency and enable the replication process.

Supporting documentation:

Creating a Read Replica - Amazon RDS
Working with MySQL and MariaDB Read Replicas

Question: 248 **Exam Heist**

A company runs analytics software on Amazon EC2 instances. The software accepts job requests from users to process data that has been uploaded to Amazon S3. Users report that some submitted data is not being processed Amazon CloudWatch reveals that the EC2 instances have a consistent CPU utilization at or near 100%. The company wants to improve system performance and scale the system based on user load.

What should a solutions architect do to meet these requirements?

A. Create a copy of the instance. Place all instances behind an Application Load Balancer.

B. Create an S3 VPC endpoint for Amazon S3. Update the software to reference the endpoint.

C. Stop the EC2 instances. Modify the instance type to one with a more powerful CPU and more memory. Restart the instances.

D. Route incoming requests to Amazon Simple Queue Service (Amazon SQS). Configure an EC2 Auto Scaling group based on queue size. Update the software to read from the queue.

Answer: D

Explanation:

The correct solution is **D**. Here's why:

The problem describes a system where EC2 instances are CPU-bound due to high user load and data processing demands. The goal is to improve performance and scale based on user demand. Option D addresses this by introducing a queuing mechanism (Amazon SQS) and an Auto Scaling group for the EC2 instances.

Here's a breakdown:

1. **Amazon SQS decouples the request handling from processing:** Instead of users directly sending

requests to the EC2 instances, they send messages to an SQS queue. This prevents the EC2 instances from being overwhelmed by sudden spikes in requests. This addresses the immediate problem of 100% CPU utilization and allows the system to handle bursts of requests.

2. **EC2 Auto Scaling group dynamically scales based on queue size:** The Auto Scaling group monitors the number of messages in the SQS queue. When the queue size exceeds a certain threshold, the Auto Scaling group automatically launches additional EC2 instances to process the backlog. Conversely, when the queue size is low, the Auto Scaling group terminates instances to reduce costs. This provides the necessary scaling based on user load. This dynamically adjusts resources to meet the demand.

3. **Software update to read from the queue:** The analytics software is modified to consume messages from the SQS queue instead of directly receiving requests from users. This ensures that all requests are processed in an orderly fashion and that no requests are lost.

Why the other options are incorrect:

A. Creating a copy of the instance and using an Application Load Balancer: This would distribute the load across multiple instances, but if the load is consistently high, all instances will eventually become CPU-bound. It doesn't address the underlying issue of CPU-intensive processing and doesn't automatically scale based on demand. It would only provide a limited and static improvement.

B. Creating an S3 VPC endpoint: This improves security and reduces network latency when accessing S3, but it doesn't directly address the CPU bottleneck or the need for scaling based on user load. It improves data transfer speeds, but it doesn't address the processing bottleneck.

C. Stopping and resizing the instance: This provides a temporary performance boost but doesn't scale automatically with user load. It would resolve the immediate CPU constraint, but it's a manual and temporary fix.

Authoritative Links:

Amazon SQS: https://aws.amazon.com/sqs/
EC2 Auto Scaling: https://aws.amazon.com/autoscaling/
Load Balancing: https://aws.amazon.com/elasticloadbalancing/

Question: 249 **Exam Heist**

A company is implementing a shared storage solution for a media application that is hosted in the AWS Cloud. The company needs the ability to use SMB clients to access data. The solution must be fully managed.

Which AWS solution meets these requirements?

A. Create an AWS Storage Gateway volume gateway. Create a file share that uses the required client protocol. Connect the application server to the file share.

B. Create an AWS Storage Gateway tape gateway. Configure tapes to use Amazon S3. Connect the application server to the tape gateway.

C. Create an Amazon EC2 Windows instance. Install and configure a Windows file share role on the instance. Connect the application server to the file share.

D. Create an Amazon FSx for Windows File Server file system. Attach the file system to the origin server. Connect the application server to the file system.

Answer: D

Explanation:

The requirement is a fully managed shared storage solution accessible via SMB clients for a media application. Let's analyze each option.

Option A utilizes AWS Storage Gateway volume gateway. While Storage Gateway can present block storage over iSCSI, which could be formatted with SMB, it is not a fully managed file sharing solution. The management of the SMB server and file shares would fall on the user.

Option B, AWS Storage Gateway tape gateway, is designed for archival storage and is not suitable for providing low-latency access for media applications. It primarily interacts with backup software and emulates tape libraries.

Option C involves manually creating an EC2 Windows instance and configuring the SMB file share. This approach is not fully managed. The user is responsible for patching, scaling, and managing the Windows instance and the SMB server.

Option D, Amazon FSx for Windows File Server, provides a fully managed Windows file system built on Windows Server. It natively supports the SMB protocol, which directly fulfills the requirement of SMB client access. It handles patching, backups, and other administrative tasks, reducing operational overhead. FSx for Windows File Server allows applications to access shared file storage over SMB, which is exactly what the media application needs. This provides a readily available, scalable, and managed file-sharing solution.

Therefore, option D is the most suitable solution due to its fully managed nature and native SMB support.

Supporting Links:

Amazon FSx for Windows File Server: https://aws.amazon.com/fsx/windows/
AWS Storage Gateway: https://aws.amazon.com/storagegateway/

Question: 250 **Exam Heist**

A company's security team requests that network traffic be captured in VPC Flow Logs. The logs will be frequently accessed for 90 days and then accessed intermittently.

What should a solutions architect do to meet these requirements when configuring the logs?

A. Use Amazon CloudWatch as the target. Set the CloudWatch log group with an expiration of 90 days

B. Use Amazon Kinesis as the target. Configure the Kinesis stream to always retain the logs for 90 days.

C. Use AWS CloudTrail as the target. Configure CloudTrail to save to an Amazon S3 bucket, and enable S3 Intelligent-Tiering.

D. Use Amazon S3 as the target. Enable an S3 Lifecycle policy to transition the logs to S3 Standard-Infrequent Access (S3 Standard-IA) after 90 days.

Answer: D

Explanation:

The correct answer is D because it provides a cost-effective and compliant solution for storing and accessing VPC Flow Logs according to the company's requirements.

Here's a detailed breakdown:

The company needs frequent access for 90 days, followed by infrequent access. This immediately points to tiered storage as a suitable solution. Amazon S3 offers different storage classes optimized for various access patterns and cost considerations.

Option A (CloudWatch): CloudWatch Logs are primarily designed for real-time monitoring and alerting, not

long-term storage of large volumes of data like VPC Flow Logs. While you can set an expiration, frequent access and cost efficiency for 90 days would be better served by S3.

Option B (Kinesis): Kinesis is designed for real-time data streaming and processing. Retaining logs in Kinesis for 90 days would be unnecessarily expensive compared to S3, which is designed for storing large volumes of data.

Option C (CloudTrail): CloudTrail logs API calls and user activity in your AWS account. It's not designed to capture network traffic data like VPC Flow Logs. Furthermore, while CloudTrail logs can be stored in S3 with Intelligent-Tiering, using CloudTrail solely for capturing VPC Flow Logs is not the intended purpose. CloudTrail focuses on auditing and governance.

Option D (S3 with Lifecycle Policy): This option aligns perfectly with the requirements. VPC Flow Logs can be directly stored in an S3 bucket. An S3 Lifecycle policy can then be configured to automatically transition the logs from the more expensive, frequently accessed S3 Standard storage class to the less expensive S3 Standard-Infrequent Access (S3 Standard-IA) storage class after 90 days. S3 Standard-IA is optimized for data that is infrequently accessed but requires rapid access when needed. This achieves cost efficiency for data accessed after the initial 90-day period. This directly addresses the intermittent access requirement.

In conclusion, using Amazon S3 as the target with an S3 Lifecycle policy that transitions logs to S3 Standard-IA after 90 days is the most cost-effective and appropriate solution for storing and accessing VPC Flow Logs based on the stated requirements.

Here are some authoritative links for further research:

VPC Flow Logs: https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html
Amazon S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
S3 Lifecycle Policies: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-examples.html

Question: 251 **Exam Heist**

An Amazon EC2 instance is located in a private subnet in a new VPC. This subnet does not have outbound internet access, but the EC2 instance needs the ability to download monthly security updates from an outside vendor.

What should a solutions architect do to meet these requirements?

A. Create an internet gateway, and attach it to the VPC. Configure the private subnet route table to use the internet gateway as the default route.

B. Create a NAT gateway, and place it in a public subnet. Configure the private subnet route table to use the NAT gateway as the default route.

C. Create a NAT instance, and place it in the same subnet where the EC2 instance is located. Configure the private subnet route table to use the NAT instance as the default route.

D. Create an internet gateway, and attach it to the VPC. Create a NAT instance, and place it in the same subnet where the EC2 instance is located. Configure the private subnet route table to use the internet gateway as the default route.

Answer: B

Explanation:

The correct answer is **B: Create a NAT gateway, and place it in a public subnet. Configure the private subnet route table to use the NAT gateway as the default route.**

Here's a detailed justification:

The scenario requires outbound internet access for an EC2 instance residing in a private subnet without direct

internet connectivity. The instance needs to download security updates. A Network Address Translation (NAT) gateway is designed for precisely this purpose. It allows instances in private subnets to initiate outbound connections to the internet (or other AWS services) while preventing the internet from initiating connections with those instances.

Option A is incorrect because configuring the private subnet route table to use an internet gateway as the default route effectively turns the private subnet into a public subnet, negating the desired security posture of isolating the instance. Internet Gateways allow two-way communication with the Internet.

Option C is incorrect because while a NAT instance can also provide outbound internet access, it's a less scalable and more complex solution than a NAT gateway. NAT instances require manual management, patching, and scaling, while NAT gateways are managed by AWS and offer higher availability and performance. Placing the NAT Instance in the same subnet as the EC2 instance would not solve the problem, as the EC2 instance would need to route traffic to the NAT instance through an entity with internet access.

Option D is incorrect because it combines an internet gateway (which exposes the subnet to inbound traffic) with a NAT instance, which is not necessary and over-complicates the solution. The route table of the private subnet should point to the NAT gateway and not the internet gateway, which negates the point of the private subnet.

By placing a NAT gateway in a public subnet (a subnet with a route to an internet gateway), the EC2 instance in the private subnet can route its outbound traffic to the NAT gateway. The NAT gateway then translates the private IP address of the EC2 instance to its own public IP address (provided by AWS) and forwards the traffic to the internet. The response traffic is then routed back through the NAT gateway to the EC2 instance. Critically, no inbound traffic from the internet can directly reach the EC2 instance. The route table configuration ensures that all outbound traffic from the private subnet is directed through the NAT gateway. The NAT gateway itself handles the connection to the Internet Gateway.

For further research, refer to the AWS documentation:

NAT Gateway
Internet Gateways
NAT Instances

Question: 252 **Exam Heist**

A solutions architect needs to design a system to store client case files. The files are core company assets and are important. The number of files will grow over time.

The files must be simultaneously accessible from multiple application servers that run on Amazon EC2 instances. The solution must have built-in redundancy.

Which solution meets these requirements?

A. Amazon Elastic File System (Amazon EFS)
B. Amazon Elastic Block Store (Amazon EBS)
C. Amazon S3 Glacier Deep Archive
D. AWS Backup

Answer: A

Explanation:

The correct answer is A: Amazon Elastic File System (Amazon EFS). Here's why:

Amazon EFS is a fully managed, scalable, elastic, and shared file system designed for use with AWS Cloud

services and on-premises resources. Its key features directly address the requirements outlined in the prompt. Specifically, it provides shared file storage that can be simultaneously accessed by multiple EC2 instances, fulfilling the requirement of accessibility from multiple application servers.

Crucially, EFS offers built-in redundancy by replicating data across multiple Availability Zones (AZs) within a region. This ensures high availability and durability, addressing the need for a solution with built-in redundancy to protect important company assets. EFS scales automatically as data is added or removed, accommodating the anticipated growth in the number of files over time without requiring manual intervention.

In contrast, Amazon EBS (B) is a block storage volume designed for single EC2 instance attachment. While EBS volumes can be backed up, they are not inherently designed for simultaneous access from multiple servers. S3 Glacier Deep Archive (C) is suitable for long-term archiving and infrequently accessed data, not for active file storage and simultaneous access. AWS Backup (D) is a data protection service providing centralized backup management, but does not provide storage itself. It is best used to create backups of EFS.

Therefore, EFS provides the best combination of scalability, shared access, redundancy, and ease of management for the given scenario.

Authoritative links:

Amazon EFS documentation

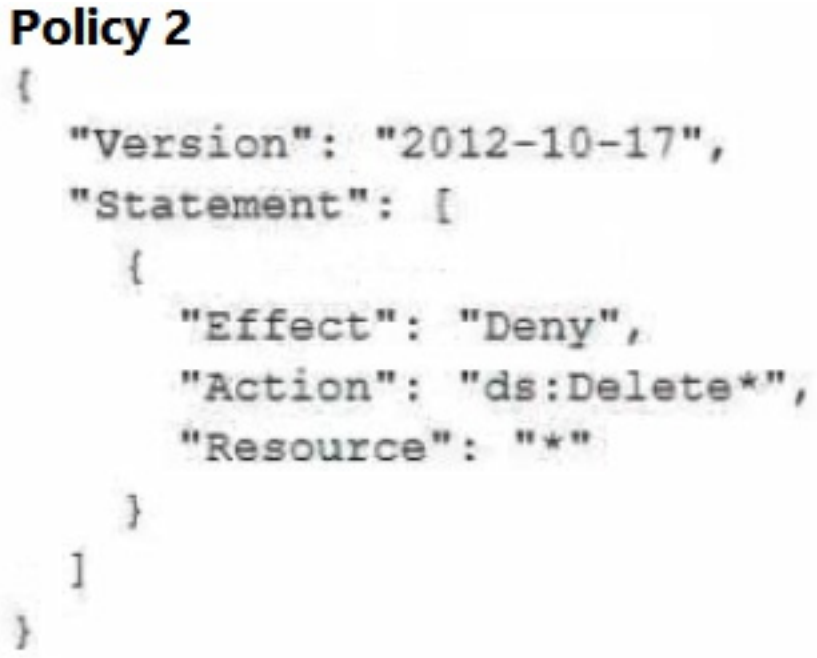
Question: 253 **Exam Heist**

A solutions architect has created two IAM policies: Policy1 and Policy2. Both policies are attached to an IAM group.

Policy 1

```
{
  "Version": "2012-10-17",  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "iam:Get*",
        "iam:List*",
        "kms:List*",
        "ec2:*",
        "ds:*",
        "logs:Get*",
        "logs:Describe*"
      ],
      "Resource": "*"
    }
  ]
}
```

Policy 2



```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": "ds:Delete*",
      "Resource": "*"
    }
  ]
}
```

A cloud engineer is added as an IAM user to the IAM group. Which action will the cloud engineer be able to perform?

A. Deleting IAM users
B. Deleting directories
C. Deleting Amazon EC2 instances
D. Deleting logs from Amazon CloudWatch Logs

Answer: C

Explanation:

ec2:* Allows full control of EC2 instances, so C is correct

The policy only grants get and list permission on IAM users, so not A ds:Delete deny denies delete-directory,

so not B, see https://awscli.amazonaws.com/v2/documentation/api/latest/reference/ds/index.html

The policy only grants get and describe permission on logs, so not D

Question: 254 **Exam Heist**

A company is reviewing a recent migration of a three-tier application to a VPC. The security team discovers that the principle of least privilege is not being applied to Amazon EC2 security group ingress and egress rules between the application tiers.

What should a solutions architect do to correct this issue?

A. Create security group rules using the instance ID as the source or destination.
B. Create security group rules using the security group ID as the source or destination.
C. Create security group rules using the VPC CIDR blocks as the source or destination.
D. Create security group rules using the subnet CIDR blocks as the source or destination.

Answer: B

Explanation:

The principle of least privilege dictates granting only the minimum necessary permissions. In the context of EC2 security groups, this means precisely controlling the traffic allowed between application tiers.

Option B, creating security group rules using security group IDs as the source or destination, is the correct approach. This is because it allows you to specify that only members of a particular security group (representing a specific tier) can communicate with another security group. This is a tightly controlled and explicit permission. For example, you can allow traffic only from the web tier security group to the application tier security group, and vice versa, applying least privilege.

Option A, using instance IDs, is not scalable or maintainable. Instance IDs are dynamic and change when instances are replaced, requiring constant updates to security group rules.

Option C, using VPC CIDR blocks, is too broad. It opens up communication to any instance within the entire VPC, violating the principle of least privilege.

Option D, using subnet CIDR blocks, is also broader than necessary. While it's more restrictive than using the VPC CIDR, it still allows communication between any instances within the subnets, even if they don't belong to the application tiers.

Therefore, using security group IDs directly addresses the security team's concerns by precisely defining allowed communication between application tiers, effectively implementing the principle of least privilege. This minimizes the attack surface and simplifies security management.For further research, consult the AWS documentation on security groups:
https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html

Question: 255 **Exam Heist**

A company has an ecommerce checkout workflow that writes an order to a database and calls a service to process the payment. Users are experiencing timeouts during the checkout process. When users resubmit the checkout form, multiple unique orders are created for the same desired transaction.

How should a solutions architect refactor this workflow to prevent the creation of multiple orders?

A. Configure the web application to send an order message to Amazon Kinesis Data Firehose. Set the payment service to retrieve the message from Kinesis Data Firehose and process the order.

B. Create a rule in AWS CloudTrail to invoke an AWS Lambda function based on the logged application path request. Use Lambda to query the database, call the payment service, and pass in the order information.

C. Store the order in the database. Send a message that includes the order number to Amazon Simple Notification Service (Amazon SNS). Set the payment service to poll Amazon SNS, retrieve the message, and process the order.

D. Store the order in the database. Send a message that includes the order number to an Amazon Simple Queue Service (Amazon SQS) FIFO queue. Set the payment service to retrieve the message and process the order. Delete the message from the queue.

Answer: D

Explanation:

The correct solution to prevent duplicate order creation in the e-commerce checkout workflow scenario is option D. Let's break down why:

The core problem is the timeouts and subsequent retries leading to multiple orders. This indicates a need for an asynchronous, idempotent approach. Option D uses Amazon SQS FIFO (First-In, First-Out) queues, which are designed to handle messages in the exact order they were sent and ensure that a message is processed only once. This is crucial for preventing duplicate order processing.

Here's a step-by-step justification:

1. **Database Storage:** Storing the order in the database first provides persistence and allows a record to be created regardless of payment processing success.
2. **SQS FIFO Queue:** Sending a message containing the order number to an SQS FIFO queue guarantees that the payment service receives the order information in the order it was placed. The FIFO nature ensures order is preserved.
3. **Payment Service Processing:** The payment service retrieves the message from the SQS FIFO queue, processes the payment using the order number in the message.
4. **Idempotency via De-duplication:** The message is deleted from the queue only after successful processing of the payment. If the payment service fails or times out before deleting the message, the message remains in the queue and will be retried, ensuring at-least-once delivery. The payment service should be designed to be idempotent; even if the service receives the same request multiple times, it generates the same result, thus avoiding duplication. This can be ensured using a transaction ID that is stored in the database and tracked.
5. **Error Handling:** In case of a payment processing failure even after retries, the message could be moved to a Dead Letter Queue (DLQ) for further investigation and manual intervention.

Why other options are incorrect:

A (Kinesis Data Firehose): Kinesis Data Firehose is designed for streaming data to destinations like S3 or Redshift for analytics. It's not meant for guaranteed, ordered message delivery in a transactional workflow.
B (CloudTrail and Lambda): CloudTrail logs events after they occur. Using it to trigger Lambda for payment processing introduces significant delay and doesn't address the issue of preventing duplicate order creation. Also, CloudTrail is not meant for this kind of transactional system.
C (SNS): Amazon SNS is a pub/sub messaging service suitable for broadcasting messages to multiple subscribers. It doesn't offer the guaranteed, ordered delivery and message de-duplication needed for this scenario. There is also the possibility of multiple subscribers being invoked, leading to multiple payments.

In summary, using SQS FIFO queues, combined with idempotent payment processing, provides the required guarantees to handle timeouts and retries gracefully, preventing the creation of multiple orders for a single user request.

Authoritative Links:

Amazon SQS FIFO Queues: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues.html
Idempotency: https://en.wikipedia.org/wiki/Idempotence (general concept)

Question: 256 **Exam Heist**

A solutions architect is implementing a document review application using an Amazon S3 bucket for storage. The solution must prevent accidental deletion of the documents and ensure that all versions of the documents are available. Users must be able to download, modify, and upload documents.

Which combination of actions should be taken to meet these requirements? (Choose two.)

A. Enable a read-only bucket ACL.
B. Enable versioning on the bucket.
C. Attach an IAM policy to the bucket.
D. Enable MFA Delete on the bucket.
E. Encrypt the bucket using AWS KMS.

Answer: BD

Explanation:

The correct answer is **BD**. Let's break down why each option is or isn't suitable:

B. Enable versioning on the bucket: Versioning is critical for ensuring that all versions of the documents are available. When versioning is enabled, every time an object is uploaded to the bucket, S3 stores a copy of the object and assigns it a unique version ID. This allows users to retrieve previous versions of a document if needed after modifications or accidental deletions.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/versioning-workflows.html

D. Enable MFA Delete on the bucket: MFA (Multi-Factor Authentication) Delete provides an extra layer of security against accidental or malicious deletions. When enabled, deleting an object version or suspending versioning requires an MFA token. This helps prevent unintended data loss, fulfilling the requirement to prevent accidental deletion.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html#MultiFactorAuthenticationDelete

Now, let's look at why the other options are not suitable:

A. Enable a read-only bucket ACL: A read-only bucket ACL would prevent users from modifying or uploading documents, which contradicts the requirement that users must be able to download, modify, and upload documents.

C. Attach an IAM policy to the bucket: While IAM policies are essential for controlling access to S3 resources, simply attaching a policy to the bucket doesn't inherently prevent accidental deletions or ensure version availability. An IAM policy would control who can perform actions, but versioning and MFA delete controls how actions are handled.

E. Encrypt the bucket using AWS KMS: Encryption protects the confidentiality of the data at rest. While encryption is a security best practice, it does not directly address the requirements of preventing accidental deletion or ensuring that all versions of the documents are available. Encryption is for data protection, not data recovery.

Therefore, versioning and MFA delete are the most effective and necessary solutions to meet the stated requirements of preventing accidental deletion and making all document versions accessible.

Question: 257 **Exam Heist**

A company is building a solution that will report Amazon EC2 Auto Scaling events across all the applications in an AWS account. The company needs to use a serverless solution to store the EC2 Auto Scaling status data in Amazon S3. The company then will use the data in Amazon S3 to provide near-real-time updates in a dashboard. The solution must not affect the speed of EC2 instance launches.

How should the company move the data to Amazon S3 to meet these requirements?

A. Use an Amazon CloudWatch metric stream to send the EC2 Auto Scaling status data to Amazon Kinesis Data Firehose. Store the data in Amazon S3.

B. Launch an Amazon EMR cluster to collect the EC2 Auto Scaling status data and send the data to Amazon Kinesis Data Firehose. Store the data in Amazon S3.

C. Create an Amazon EventBridge rule to invoke an AWS Lambda function on a schedule. Configure the Lambda function to send the EC2 Auto Scaling status data directly to Amazon S3.

D. Use a bootstrap script during the launch of an EC2 instance to install Amazon Kinesis Agent. Configure Kinesis Agent to collect the EC2 Auto Scaling status data and send the data to Amazon Kinesis Data Firehose. Store the data in Amazon S3.

Answer: A

Explanation:

The correct answer is A. Here's a detailed justification:

The requirement is to create a serverless solution to store EC2 Auto Scaling events in S3 for near-real-time dashboard updates without affecting EC2 instance launch speed. Let's analyze why each option is suitable or unsuitable:

Option A: Use an Amazon CloudWatch metric stream to send the EC2 Auto Scaling status data to Amazon Kinesis Data Firehose. Store the data in Amazon S3.

CloudWatch metric streams allow you to continuously stream CloudWatch metrics to destinations like Kinesis Data Firehose. This provides a near-real-time flow of Auto Scaling status data.
Kinesis Data Firehose is a fully managed service for delivering real-time streaming data to destinations like S3. It handles buffering, batching, and compression, ensuring efficient data delivery.
This solution is serverless because both CloudWatch metric streams and Kinesis Data Firehose are managed services that require no server provisioning.
It avoids impacting EC2 instance launch speed because the data capture is asynchronous and handled by CloudWatch.

Option B: Launch an Amazon EMR cluster to collect the EC2 Auto Scaling status data and send the data to Amazon Kinesis Data Firehose. Store the data in Amazon S3.

Using EMR introduces unnecessary complexity and overhead. EMR is designed for big data processing and analysis, which is not needed for this relatively simple data streaming task.
EMR involves managing an entire cluster, which is not serverless.

Option C: Create an Amazon EventBridge rule to invoke an AWS Lambda function on a schedule. Configure the Lambda function to send the EC2 Auto Scaling status data directly to Amazon S3.

EventBridge on a schedule will not provide near-real-time updates. EventBridge is event driven and the Lambda function would need to poll data, thus not serverless.
Polling the Auto Scaling API from Lambda could potentially impact the API's availability and, indirectly, the EC2 instance launch speed.

Option D: Use a bootstrap script during the launch of an EC2 instance to install Amazon Kinesis Agent. Configure Kinesis Agent to collect the EC2 Auto Scaling status data and send the data to Amazon Kinesis Data Firehose. Store the data in Amazon S3.

Installing Kinesis Agent on each EC2 instance using a bootstrap script will increase the EC2 launch time, contrary to the requirements.
Managing Kinesis Agent on each instance adds complexity, which undermines the desire for a simple, serverless solution.

In summary, option A offers the best balance of real-time data streaming, serverless architecture, and minimal impact on EC2 instance launches.

Here are some authoritative links for more research:

CloudWatch Metric Streams:
https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/metricstreams.html
Kinesis Data Firehose: https://aws.amazon.com/kinesis/data-firehose/
Amazon EventBridge: https://aws.amazon.com/eventbridge/
AWS Lambda: https://aws.amazon.com/lambda/

Question: 258 **Exam Heist**

A company has an application that places hundreds of .csv files into an Amazon S3 bucket every hour. The files are 1 GB in size. Each time a file is uploaded, the company needs to convert the file to Apache Parquet format and place the output file into an S3 bucket.

Which solution will meet these requirements with the LEAST operational overhead?

A. Create an AWS Lambda function to download the .csv files, convert the files to Parquet format, and place the output files in an S3 bucket. Invoke the Lambda function for each S3 PUT event.

B. Create an Apache Spark job to read the .csv files, convert the files to Parquet format, and place the output files in an S3 bucket. Create an AWS Lambda function for each S3 PUT event to invoke the Spark job.

C. Create an AWS Glue table and an AWS Glue crawler for the S3 bucket where the application places the .csv files. Schedule an AWS Lambda function to periodically use Amazon Athena to query the AWS Glue table, convert the query results into Parquet format, and place the output files into an S3 bucket.

D. Create an AWS Glue extract, transform, and load (ETL) job to convert the .csv files to Parquet format and place the output files into an S3 bucket. Create an AWS Lambda function for each S3 PUT event to invoke the ETL job.

Answer: D

Explanation:

The correct answer is D because it leverages AWS Glue, a fully managed ETL service specifically designed for data transformation tasks like converting CSV to Parquet. The process is triggered automatically by S3 PUT events via a Lambda function, thus requiring minimal operational overhead.

Here's a detailed breakdown:

AWS Glue ETL Job: Glue ETL jobs are purpose-built for data transformation tasks. They are designed to handle large-scale data processing, making them efficient for converting 1 GB CSV files to Parquet. Glue handles the complexities of managing the compute resources required for the transformation.
Lambda Trigger: The Lambda function triggered by each S3 PUT event provides the real-time aspect, ensuring immediate processing of each uploaded file. Lambda is a serverless compute service, meaning you don't have to manage any servers, reducing operational overhead.
Parquet Format: Parquet is a columnar storage format optimized for analytical queries, suitable for downstream data analysis.
Least Operational Overhead: This solution minimizes operational overhead because Glue is a fully managed service. You don't have to manage servers, configure networking, or perform other infrastructure-related tasks. Lambda is also serverless, further reducing management burden.

Let's analyze why the other options are less optimal:

A (Lambda only): Lambda functions have execution time and memory limits. Processing 1 GB files might exceed these limits. Additionally, managing the transformation logic within Lambda for such large files introduces complexity and operational overhead.
B (Spark and Lambda): Managing an Apache Spark cluster requires significant operational overhead. While Spark is powerful, it's overkill for this relatively simple data transformation task and increases management complexity.
C (Glue Crawler, Athena, and Lambda): This option is unnecessarily complex. Using Athena to query CSV files directly in S3 is less efficient and requires more processing power than using a direct ETL transformation. Relying on scheduling instead of event-driven processing increases latency.

In summary, leveraging AWS Glue ETL jobs triggered by S3 PUT events through Lambda provides an efficient, scalable, and fully managed solution for converting CSV files to Parquet with minimal operational overhead.

Relevant AWS documentation:

AWS Glue: https://aws.amazon.com/glue/
Amazon S3 Event Notifications:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventNotifications.html
AWS Lambda: https://aws.amazon.com/lambda/

Question: 259 **Exam Heist**

A company is implementing new data retention policies for all databases that run on Amazon RDS DB instances. The company must retain daily backups for a minimum period of 2 years. The backups must be consistent and restorable.

Which solution should a solutions architect recommend to meet these requirements?

A. Create a backup vault in AWS Backup to retain RDS backups. Create a new backup plan with a daily schedule and an expiration period of 2 years after creation. Assign the RDS DB instances to the backup plan.

B. Configure a backup window for the RDS DB instances for daily snapshots. Assign a snapshot retention policy of 2 years to each RDS DB instance. Use Amazon Data Lifecycle Manager (Amazon DLM) to schedule snapshot deletions.

C. Configure database transaction logs to be automatically backed up to Amazon CloudWatch Logs with an expiration period of 2 years.

D. Configure an AWS Database Migration Service (AWS DMS) replication task. Deploy a replication instance, and configure a change data capture (CDC) task to stream database changes to Amazon S3 as the target. Configure S3 Lifecycle policies to delete the snapshots after 2 years.

Answer: A

Explanation:

The recommended solution is **A: Create a backup vault in AWS Backup to retain RDS backups. Create a new backup plan with a daily schedule and an expiration period of 2 years after creation. Assign the RDS DB instances to the backup plan.**

Here's why:

AWS Backup is designed for centralized backup management: It provides a unified console for configuring and managing backups across multiple AWS services, including RDS. This simplifies compliance and governance. https://aws.amazon.com/backup/
Backup Vaults offer enhanced security: AWS Backup vaults enable you to isolate backups, control access, and prevent accidental or malicious deletion of backups. This is crucial for long-term retention requirements and compliance.
Backup Plans provide scheduling and retention policies: AWS Backup allows you to define backup plans with custom schedules (daily, weekly, etc.) and retention periods (2 years in this case). This automates the backup process and ensures backups are retained for the required duration.
Centralized retention management: AWS Backup simplifies the management of long-term retention by automatically handling the lifecycle of backups based on the defined policies.
RDS integration with AWS Backup: RDS is natively integrated with AWS Backup, making it easy to protect RDS instances using backup plans.

Let's analyze why the other options are not as suitable:

Option B (RDS Snapshots and DLM): While RDS snapshots are a valid backup method, managing snapshot retention manually using Amazon DLM can become complex at scale, especially for a large number of databases. It doesn't offer the centralized management and security benefits of AWS Backup. DLM primarily focuses on EBS volumes and, while it can be used for RDS snapshots, it's not the best practice compared to AWS Backup for centralized RDS backup management.

https://docs.aws.amazon.com/dlm/latest/userguide/what-is-dlm.html
Option C (CloudWatch Logs): CloudWatch Logs are designed for storing application and system logs, not database backups. While database transaction logs can be helpful for point-in-time recovery, they are not a substitute for full database backups. They don't offer a consistent, restorable state like a database snapshot.
Option D (DMS and S3): AWS DMS is primarily used for database migration and replication, not for long-term backup and retention. While CDC can capture changes, it's not a direct replacement for full backups. Reconstructing a database from CDC data stored in S3 can be complex and time-consuming. It requires manual reconstruction procedures and doesn't provide the ease of restoration offered by RDS snapshots or AWS Backup.

Therefore, AWS Backup provides the most efficient, secure, and compliant solution for managing RDS backups with a 2-year retention period, offering centralized management, automated scheduling, and enhanced security features.

Question: 260 **Exam Heist**

A company's compliance team needs to move its file shares to AWS. The shares run on a Windows Server SMB file share. A self-managed on-premises Active Directory controls access to the files and folders.

The company wants to use Amazon FSx for Windows File Server as part of the solution. The company must ensure that the on-premises Active Directory groups restrict access to the FSx for Windows File Server SMB compliance shares, folders, and files after the move to AWS. The company has created an FSx for Windows File Server file system.

Which solution will meet these requirements?

A.Create an Active Directory Connector to connect to the Active Directory. Map the Active Directory groups to IAM groups to restrict access.

B.Assign a tag with a Restrict tag key and a Compliance tag value. Map the Active Directory groups to IAM groups to restrict access.

C.Create an IAM service-linked role that is linked directly to FSx for Windows File Server to restrict access.

D.Join the file system to the Active Directory to restrict access.

Answer: D

Explanation:

The correct answer is D, joining the FSx for Windows File Server file system to the existing on-premises Active Directory (AD).

Here's why: The primary requirement is to maintain the existing on-premises Active Directory groups for access control to files and folders hosted on FSx for Windows File Server. This means the file system needs to be integrated with the on-premises AD so that existing group memberships and permissions translate to the cloud environment.

FSx for Windows File Server provides native support for integrating with Active Directory. By joining the FSx file system to the existing on-premises AD, the file system becomes a member of the AD domain. This enables users and groups defined in the on-premises AD to be recognized and authorized by FSx. Users can then access the file shares using their existing AD credentials, and permissions configured on the shares, folders, and files using AD groups will be enforced.

Option A is incorrect because Active Directory Connector is typically used to connect AWS Directory Service to on-premises Active Directory, which is not the case here; the company is already running its own on-premises Active Directory. Mapping AD groups to IAM groups won't directly translate SMB permissions on files and folders on FSx.

Option B is incorrect because tags are used for metadata and resource organization, not for directly controlling access permissions on file shares using Active Directory groups. Mapping AD groups to IAM groups is not the solution for SMB level permissions.

Option C is incorrect because IAM service-linked roles grant AWS services permission to call other AWS services on your behalf. While FSx uses service-linked roles, they are not a mechanism for controlling file share access using on-premises Active Directory groups. Service-linked roles will not grant or restrict the usage of on-premise AD users.

In summary, direct integration with Active Directory is essential for maintaining existing permissions and security policies. FSx for Windows File Server's AD joining capability addresses this directly by extending the on-premises AD environment to the cloud-based file system.

For more information, refer to the AWS documentation on:

FSx for Windows File Server - Active Directory Integration:
https://docs.aws.amazon.com/fsx/latest/WindowsGuide/self-managed-AD.html

Question: 261 **Exam Heist**

A company recently announced the deployment of its retail website to a global audience. The website runs on multiple Amazon EC2 instances behind an Elastic Load Balancer. The instances run in an Auto Scaling group across multiple Availability Zones.

The company wants to provide its customers with different versions of content based on the devices that the customers use to access the website.

Which combination of actions should a solutions architect take to meet these requirements? (Choose two.)

A. Configure Amazon CloudFront to cache multiple versions of the content.

B. Configure a host header in a Network Load Balancer to forward traffic to different instances.

C. Configure a function to send specific objects to users based on the User-Agent header.

D. Configure AWS Global Accelerator. Forward requests to a Network Load Balancer (NLB). Configure the NLB to set up host-based routing to different EC2 instances.

E. Configure AWS Global Accelerator. Forward requests to a Network Load Balancer (NLB). Configure the NLB to set up path-based routing to different EC2 instances.

Answer: AC

Explanation:

The correct answer is AC. Here's a detailed justification:

A. Configure Amazon CloudFront to cache multiple versions of the content.

CloudFront's ability to cache multiple versions of content based on request headers is crucial for serving device-specific versions of the website. This is accomplished through "Cache Key Settings" and "Origin Request Policies" in CloudFront distributions. By including the User-Agent header in the cache key, CloudFront can store and serve different versions of the content for different device types (e.g., desktop, mobile). This reduces the load on the origin servers by serving pre-cached content to users. This approach is preferred over always forwarding requests to the origin.https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/controlling-the-cache-key.html

C. Configure a function to send specific objects to users based on the User-Agent header.

 functions allow you to customize content delivery at the edge based on request attributes,

including the User-Agent header. By inspecting the User-Agent header within the function, you can rewrite URLs or redirect requests to specific origin servers that serve optimized content for each device type. This is a highly flexible approach that allows for fine-grained control over content delivery. intercepts requests before they hit the cache (or origin, in a cache miss), providing immediate customization before content is served.https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/cloudfront-functions.html

Why other options are incorrect:

B. Configure a host header in a Network Load Balancer to forward traffic to different instances. NLBs operate at the transport layer (Layer 4) and do not inspect HTTP headers like the host header. NLBs primarily forward traffic based on IP addresses and ports, making them unsuitable for routing based on the User-Agent header.

D & E. Configure AWS Global Accelerator. Forward requests to a Network Load Balancer (NLB)...: While Global Accelerator improves application availability and performance, it is primarily used for routing traffic to the nearest endpoint, not for device-specific content delivery. NLBs are still not suited for routing based on the User-Agent. Path-based routing in the NLB would also be unrelated to device type. The problem requires inspection of the User-Agent header, which neither Global Accelerator nor NLB natively supports.

Question: 262 **Exam Heist**

A company plans to use Amazon ElastiCache for its multi-tier web application. A solutions architect creates a Cache VPC for the ElastiCache cluster and an App VPC for the application's Amazon EC2 instances. Both VPCs are in the us-east-1 Region.

The solutions architect must implement a solution to provide the application's EC2 instances with access to the ElastiCache cluster.

Which solution will meet these requirements MOST cost-effectively?

A. Create a peering connection between the VPCs. Add a route table entry for the peering connection in both VPCs. Configure an inbound rule for the ElastiCache cluster's security group to allow inbound connection from the application's security group.

B. Create a Transit VPC. Update the VPC route tables in the Cache VPC and the App VPC to route traffic through the Transit VPC. Configure an inbound rule for the ElastiCache cluster's security group to allow inbound connection from the application's security group.

C. Create a peering connection between the VPCs. Add a route table entry for the peering connection in both VPCs. Configure an inbound rule for the peering connection's security group to allow inbound connection from the application's security group.

D. Create a Transit VPC. Update the VPC route tables in the Cache VPC and the App VPC to route traffic through the Transit VPC. Configure an inbound rule for the Transit VPC's security group to allow inbound connection from the application's security group.

Answer: A

Explanation:

The most cost-effective solution for connecting EC2 instances in one VPC (App VPC) to an ElastiCache cluster in another VPC (Cache VPC) within the same AWS Region (us-east-1) is to use VPC Peering.

Option A is the correct solution because it establishes a direct network connection between the two VPCs using VPC peering. This allows network traffic to flow directly between the EC2 instances and the ElastiCache cluster without traversing the public internet or intermediate network infrastructure. The route table entries in each VPC are essential to direct traffic destined for the other VPC through the peering connection. The security group rule allows the EC2 instances to initiate connections to the ElastiCache

cluster.

Option B and D are incorrect because a Transit VPC is typically used for connecting multiple VPCs together or for centralizing network services. In this simple scenario involving only two VPCs within the same region, a Transit VPC adds unnecessary complexity and cost. A Transit VPC involves additional EC2 instances running routing software, increasing management overhead and operational costs.

Option C is incorrect because security groups can be directly associated with network interfaces and resources. There is no "peering connection's security group" to configure. You want to permit traffic from the application's security group directly.

Therefore, VPC peering (Option A) is the most efficient and cost-effective way to enable communication between the EC2 instances and the ElastiCache cluster in this scenario.

Supporting links:

VPC Peering
Security Groups

Question: 263 **Exam Heist**

A company is building an application that consists of several microservices. The company has decided to use container technologies to deploy its software on AWS. The company needs a solution that minimizes the amount of ongoing effort for maintenance and scaling. The company cannot manage additional infrastructure.

Which combination of actions should a solutions architect take to meet these requirements? (Choose two.)

A. Deploy an Amazon Elastic Container Service (Amazon ECS) cluster.

B. Deploy the Kubernetes control plane on Amazon EC2 instances that span multiple Availability Zones.

C. Deploy an Amazon Elastic Container Service (Amazon ECS) service with an Amazon EC2 launch type. Specify a desired task number level of greater than or equal to 2.

D. Deploy an Amazon Elastic Container Service (Amazon ECS) service with a Fargate launch type. Specify a desired task number level of greater than or equal to 2.

E. Deploy Kubernetes worker nodes on Amazon EC2 instances that span multiple Availability Zones. Create a deployment that specifies two or more replicas for each microservice.

Answer: AD

Explanation:

The correct answer is AD. Here's why:

A. Deploy an Amazon Elastic Container Service (Amazon ECS) cluster: ECS is a fully managed container orchestration service that allows you to run, stop, and manage containers on a cluster. By using ECS, the company can avoid managing the underlying infrastructure. ECS handles the orchestration and scheduling of the containers, reducing operational overhead.

D. Deploy an Amazon Elastic Container Service (Amazon ECS) service with a Fargate launch type. Specify a desired task number level of greater than or equal to 2: Fargate is a serverless compute engine for containers. By using the Fargate launch type with ECS, the company eliminates the need to manage EC2 instances for their containers, further reducing the operational burden. Specifying a desired task number of 2 or more ensures high availability and fault tolerance, as multiple instances of the service will be running. If one container fails, another will take its place.

Why the other options are incorrect:

B. Deploy the Kubernetes control plane on Amazon EC2 instances that span multiple Availability Zones: Kubernetes (K8s) is a robust container orchestration system, but deploying the control plane on EC2 instances introduces significant operational overhead. The company would be responsible for managing the health, scaling, and patching of these EC2 instances, directly contradicting the requirement to minimize ongoing effort and avoid managing additional infrastructure. Furthermore, Amazon EKS (Elastic Kubernetes Service) is a managed service for Kubernetes.

C. Deploy an Amazon Elastic Container Service (Amazon ECS) service with an Amazon EC2 launch type. Specify a desired task number level of greater than or equal to 2: While ECS with the EC2 launch type could work, it would require the company to manage the underlying EC2 instances, including patching, scaling, and maintenance. This goes against the requirement to minimize maintenance and avoid infrastructure management.

E. Deploy Kubernetes worker nodes on Amazon EC2 instances that span multiple Availability Zones: Similar to option B, deploying worker nodes on EC2 instances adds an operational burden, forcing the company to manage and maintain the underlying infrastructure, which conflicts with the problem's requirements. Again, utilizing EKS and Fargate reduces overhead.

Supporting Documentation:

Amazon ECS: https://aws.amazon.com/ecs/
AWS Fargate: https://aws.amazon.com/fargate/

Question: 264 **Exam Heist**

A company has a web application hosted over 10 Amazon EC2 instances with traffic directed by Amazon Route 53. The company occasionally experiences a timeout error when attempting to browse the application. The networking team finds that some DNS queries return IP addresses of unhealthy instances, resulting in the timeout error.

What should a solutions architect implement to overcome these timeout errors?

A. Create a Route 53 simple routing policy record for each EC2 instance. Associate a health check with each record.

B. Create a Route 53 failover routing policy record for each EC2 instance. Associate a health check with each record.

C. Create an Amazon CloudFront distribution with EC2 instances as its origin. Associate a health check with the EC2 instances.

D. Create an Application Load Balancer (ALB) with a health check in front of the EC2 instances. Route to the ALB from Route 53.

Answer: D

Explanation:

The best solution to prevent timeout errors caused by unhealthy EC2 instances in a web application is to use an Application Load Balancer (ALB) with health checks in front of the instances and then route traffic to the ALB from Route 53.

Here's why:

ALB Health Checks: ALBs have built-in health check capabilities. They periodically send requests to the EC2 instances and determine their health based on the response. Only healthy instances receive traffic.
Dynamic Instance Management: If an instance fails the health check, the ALB automatically stops routing traffic to it, preventing timeouts for users. When the instance recovers, the ALB resumes traffic routing.
Route 53 Integration: Route 53 can be configured to point to the ALB's DNS name. This ensures that all traffic is directed to the ALB, which then intelligently routes traffic to healthy instances.

Scalability and Availability: ALBs offer built-in scalability and high availability. They can handle traffic spikes and ensure that the application remains available even if some instances fail.
Simplicity: Compared to alternatives like creating individual Route 53 records with health checks for each instance, using an ALB is a simpler and more manageable solution.

Let's analyze why the other options are less suitable:

A: Route 53 simple routing policy: While health checks can be associated, simple routing policy does not inherently remove unhealthy instances from the rotation as effectively as an ALB. It relies on DNS propagation, which might take time.
B: Route 53 failover routing policy: This is intended for active/passive setups. It doesn't dynamically distribute traffic based on health within a group of active servers like the given scenario.
C: CloudFront distribution with EC2 instances as the origin: CloudFront primarily focuses on caching static content and improving website performance by distributing content closer to users. While health checks can be associated with origins, CloudFront is not the best solution for dynamic traffic routing based on instance health, especially within a single region. Also, it doesn't manage routing traffic to a group of active instances for high availability like an ALB.

In conclusion, the ALB provides a robust and efficient solution for managing instance health and ensuring that users are only directed to healthy instances, preventing timeout errors.

Supporting Links:

Application Load Balancers:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html
Route 53 Health Checks: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/health-checks-creating-deleting.html

Question: 265 **Exam Heist**

A solutions architect needs to design a highly available application consisting of web, application, and database tiers. HTTPS content delivery should be as close to the edge as possible, with the least delivery time.

Which solution meets these requirements and is MOST secure?

A. Configure a public Application Load Balancer (ALB) with multiple redundant Amazon EC2 instances in public subnets. Configure Amazon CloudFront to deliver HTTPS content using the public ALB as the origin.

B. Configure a public Application Load Balancer with multiple redundant Amazon EC2 instances in private subnets. Configure Amazon CloudFront to deliver HTTPS content using the EC2 instances as the origin.

C. Configure a public Application Load Balancer (ALB) with multiple redundant Amazon EC2 instances in private subnets. Configure Amazon CloudFront to deliver HTTPS content using the public ALB as the origin.

D. Configure a public Application Load Balancer with multiple redundant Amazon EC2 instances in public subnets. Configure Amazon CloudFront to deliver HTTPS content using the EC2 instances as the origin.

Answer: C

Explanation:

The correct answer is C. Here's why:

Requirements Breakdown:

Highly Available: Achieved by using multiple EC2 instances behind an Application Load Balancer (ALB). The ALB distributes traffic across healthy instances, ensuring service availability.
HTTPS Content Delivery Close to the Edge: Amazon CloudFront is a Content Delivery Network (CDN) that caches content in edge locations globally, reducing latency for users accessing the application. Serving

content via HTTPS ensures secure communication.
Least Delivery Time: CloudFront's edge locations minimize the distance data travels to reach users.
Most Secure: Placing EC2 instances in private subnets enhances security. Private subnets don't have direct internet access, reducing the attack surface.

Justification:

Option C optimally fulfills all the requirements.

1. **Public ALB:** The ALB needs to be public to receive incoming HTTPS requests from users.
2. **Redundant EC2 instances in private subnets:** Placing EC2 instances in private subnets significantly improves security by preventing direct access from the internet. The ALB acts as a secure intermediary, routing traffic to the EC2 instances.
3. **CloudFront with ALB as origin:** CloudFront serves HTTPS content from edge locations. Configuring the public ALB as the origin directs CloudFront to fetch content from the ALB, which then distributes requests to the backend EC2 instances. This ensures HTTPS delivery closer to the users.

Why other options are incorrect:

Option A & D (EC2 instances in public subnets): Exposing EC2 instances directly to the internet in public subnets increases the security risk. This violates the "most secure" requirement.
Option B (EC2 instances as origin): While EC2 instances can technically be origins, it is not as practical and recommended as using an ALB since it is much harder to manage redundancy, scaling, and health checks. CloudFront is designed to work well with load balancers.
Options B & D (Directly using EC2 as origin without ALB): It loses the advantage of the ALB's health checks, traffic distribution, and improved manageability in front of the EC2 instances. The EC2 instances would then need to be directly accessible from CloudFront, further compromising security.

Supporting Links:

Amazon CloudFront: https://aws.amazon.com/cloudfront/
Application Load Balancer (ALB): https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
Amazon VPC Subnets: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Subnets.html

Question: 266 **Exam Heist**

A company has a popular gaming platform running on AWS. The application is sensitive to latency because latency can impact the user experience and introduce unfair advantages to some players. The application is deployed in every AWS Region. It runs on Amazon EC2 instances that are part of Auto Scaling groups configured behind Application Load Balancers (ALBs). A solutions architect needs to implement a mechanism to monitor the health of the application and redirect traffic to healthy endpoints.

Which solution meets these requirements?

A. Configure an accelerator in AWS Global Accelerator. Add a listener for the port that the application listens on, and attach it to a Regional endpoint in each Region. Add the ALB as the endpoint.

B. Create an Amazon CloudFront distribution and specify the ALB as the origin server. Configure the cache behavior to use origin cache headers. Use AWS Lambda functions to optimize the traffic.

C. Create an Amazon CloudFront distribution and specify Amazon S3 as the origin server. Configure the cache behavior to use origin cache headers. Use AWS Lambda functions to optimize the traffic.

D. Configure an Amazon DynamoDB database to serve as the data store for the application. Create a DynamoDB Accelerator (DAX) cluster to act as the in-memory cache for DynamoDB hosting the application data.

Answer: A

Explanation:

The correct answer is **A. Configure an accelerator in AWS Global Accelerator. Add a listener for the port that the application listens on, and attach it to a Regional endpoint in each Region. Add the ALB as the endpoint.**

Here's a detailed justification:

AWS Global Accelerator is designed to improve the availability and performance of applications by directing user traffic to the optimal endpoint based on factors such as location, health, and configured weights. It uses the AWS global network, which is less congested and more reliable than the public internet, to route traffic. This is vital for latency-sensitive applications like the gaming platform described.

Global Accelerator operates by advertising static IP addresses from AWS edge locations around the world. When a user connects to the application using one of these IPs, Global Accelerator intelligently routes the traffic to the nearest healthy endpoint. By attaching Application Load Balancers (ALBs) in each Region as endpoints, the application benefits from regional load balancing (handled by the ALBs) and global traffic management (handled by Global Accelerator).

Global Accelerator performs continuous health checks on the ALBs in each Region. If an ALB or the underlying EC2 instances within a Region become unhealthy, Global Accelerator automatically redirects traffic to healthy endpoints in other Regions. This provides high availability and resilience. The listener configured on the Global Accelerator is associated with the port on which the game application runs and is mapped to ALBs acting as endpoints.

Options B and C are incorrect because CloudFront is primarily designed for caching static and dynamic content. While CloudFront can improve performance, it's not the primary solution for routing traffic based on real-time health checks and minimizing latency across multiple regions in the way that Global Accelerator can. The caching that CloudFront provides will not be effective if the origin (ALB) is unhealthy. Origin cache headers are related to caching behavior and not health checks. Option C uses S3 as the origin, which is even less appropriate for a dynamic, interactive gaming application.

Option D is incorrect because DynamoDB and DAX are database-related services and are not suitable for routing traffic or performing health checks on application endpoints.

In summary, Global Accelerator provides the optimal solution by using the AWS global network to minimize latency and provide continuous health checks to route traffic to healthy endpoints in different regions, which is crucial for the gaming platform's performance and user experience.

Relevant links:

AWS Global Accelerator: https://aws.amazon.com/global-accelerator/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/

Question: 267 **Exam Heist**

A company has one million users that use its mobile app. The company must analyze the data usage in near-real time. The company also must encrypt the data in near-real time and must store the data in a centralized location in Apache Parquet format for further processing.

Which solution will meet these requirements with the LEAST operational overhead?

A. Create an Amazon Kinesis data stream to store the data in Amazon S3. Create an Amazon Kinesis Data Analytics application to analyze the data. Invoke an AWS Lambda function to send the data to the Kinesis Data Analytics application.

B. Create an Amazon Kinesis data stream to store the data in Amazon S3. Create an Amazon EMR cluster to analyze the data. Invoke an AWS Lambda function to send the data to the EMR cluster.

C. Create an Amazon Kinesis Data Firehose delivery stream to store the data in Amazon S3. Create an Amazon EMR cluster to analyze the data.

D. Create an Amazon Kinesis Data Firehose delivery stream to store the data in Amazon S3. Create an Amazon Kinesis Data Analytics application to analyze the data.

Answer: D

Explanation:

The best solution to meet the company's requirements with the least operational overhead is option D.

Here's why:

Data Ingestion & Storage (Kinesis Data Firehose): Kinesis Data Firehose is designed to reliably load streaming data into data lakes, data stores, and analytics services. It automatically scales to match the throughput of your data and requires no ongoing administration. It can directly deliver data to S3 in Parquet format. This addresses the requirements of storing data in S3 in Parquet format with encryption.

https://aws.amazon.com/kinesis/data-firehose/

Near-Real-Time Analysis (Kinesis Data Analytics): Kinesis Data Analytics is a serverless service that allows you to process and analyze streaming data in real time using standard SQL or Java. It has less operational overhead than managing an EMR cluster. It aligns perfectly with the need for near-real-time data usage analysis.

https://aws.amazon.com/kinesis/data-analytics/

Encryption: Kinesis Data Firehose supports encryption of data at rest in S3 using AWS KMS or server-side encryption with Amazon S3-managed keys (SSE-S3).

Operational Overhead: Compared to using an EMR cluster (options B and C), Kinesis Data Analytics is a fully managed service, eliminating the need for cluster management, scaling, and patching. Using Lambda to send data to either EMR or Kinesis Data Analytics (options A and B) adds unnecessary complexity. Firehose simplifies the data loading process directly to S3.

Parquet Conversion: Kinesis Firehose can automatically convert the data into Parquet format before storing it in S3.

In contrast:

Options A, B, and C involve more manual steps and resource management. Setting up and managing an EMR cluster involves more operational overhead compared to Kinesis Data Analytics. Using Kinesis Data Streams without Firehose would require additional logic for data buffering, transformation, and storage in Parquet format. Using Lambda to ingest data adds an extra layer of complexity.

Question: 268 **Exam Heist**

A gaming company has a web application that displays scores. The application runs on Amazon EC2 instances behind an Application Load Balancer. The application stores data in an Amazon RDS for MySQL database. Users are starting to experience long delays and interruptions that are caused by database read performance. The company wants to improve the user experience while minimizing changes to the application's architecture.

What should a solutions architect do to meet these requirements?

A. Use Amazon ElastiCache in front of the database.

B. Use RDS Proxy between the application and the database.

C. Migrate the application from EC2 instances to AWS Lambda.

D. Migrate the database from Amazon RDS for MySQL to Amazon DynamoDB.

Answer: B

Explanation:

The correct answer is **B. Use RDS Proxy between the application and the database.**

Here's a detailed justification:

The problem is database read performance, leading to user delays and interruptions. The requirement is to improve the user experience with minimal architectural changes.

Why RDS Proxy is the Best Fit:

Connection Pooling: RDS Proxy sits between the application and the RDS database and manages database connections efficiently. Instead of each application instance establishing and maintaining its own connection, RDS Proxy maintains a pool of connections. This reduces the overhead of creating new connections, which is a common performance bottleneck. This directly addresses the read performance issue by optimizing how the application interacts with the database.
Connection Multiplexing: RDS Proxy allows multiple application requests to share a single database connection, further optimizing resource usage and reducing connection-related overhead.
Reduced Database Load: By reducing the number of connections the database has to manage, RDS Proxy lessens the load on the RDS instance, improving its overall responsiveness and read performance.
Minimal Architectural Changes: RDS Proxy is a relatively transparent layer. The application's code doesn't need significant modification to utilize it. The application simply needs to point to the RDS Proxy endpoint instead of the database endpoint. This aligns with the requirement to minimize changes to the existing architecture.
Focus on Read Performance: While RDS Proxy helps with all database operations, its connection pooling and multiplexing particularly benefit read-heavy workloads, which is the core issue in this scenario.

Why other options are less suitable:

A. Use Amazon ElastiCache in front of the database: While ElastiCache (e.g., Memcached or Redis) can improve read performance by caching frequently accessed data, it requires significant changes to the application to implement caching logic. It is an architectural change that the question is trying to avoid.
C. Migrate the application from EC2 instances to AWS Lambda: Migrating the application to Lambda is a major architectural change. It also doesn't directly address the database read performance issue. It's a much more complex undertaking than implementing RDS Proxy.
D. Migrate the database from Amazon RDS for MySQL to Amazon DynamoDB: Migrating to DynamoDB is also a significant architectural change. DynamoDB is a NoSQL database, and requires a redesign of the data model and application code to use effectively. This is a complex solution and doesn’t meet the minimization of changes requirement.

In conclusion, RDS Proxy provides a targeted solution that optimizes database connections, reduces load on the database, and requires minimal changes to the application's architecture. Therefore, it is the best choice to address the database read performance issues.

Authoritative Links:

AWS RDS Proxy: https://aws.amazon.com/rds/proxy/
Benefits of RDS Proxy: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/rds-proxy.html#rds-proxy.benefits

Question: 269 **Exam Heist**

An ecommerce company has noticed performance degradation of its Amazon RDS based web application. The performance degradation is attributed to an increase in the number of read-only SQL queries triggered by business analysts. A solutions architect needs to solve the problem with minimal changes to the existing web application.

What should the solutions architect recommend?

A. Export the data to Amazon DynamoDB and have the business analysts run their queries.

B. Load the data into Amazon ElastiCache and have the business analysts run their queries.

C. Create a read replica of the primary database and have the business analysts run their queries.

D. Copy the data into an Amazon Redshift cluster and have the business analysts run their queries.

Answer: C

Explanation:

The recommended solution is to create a read replica of the primary RDS database and direct business analysts' read-only queries to it. This approach addresses the performance degradation without requiring significant changes to the existing web application architecture. Read replicas allow you to offload read traffic from the primary database instance, reducing the load on the primary and improving its performance for transactional operations.

Option A, exporting data to DynamoDB, is not suitable because DynamoDB is a NoSQL database, and migrating SQL queries to it would require substantial code changes. Option B, using ElastiCache, is primarily for caching frequently accessed data, not for running ad-hoc queries by analysts. Option D, copying data to Redshift, is better suited for complex analytical workloads over large datasets, which is overkill for simple read-only queries and introduces unnecessary complexity.

Read replicas are designed specifically for read-heavy workloads and are easy to set up and manage within the RDS ecosystem. They maintain near real-time data synchronization with the primary database, ensuring the business analysts have access to up-to-date information. This solution provides minimal disruption to the existing application while improving its performance and scalability by isolating the analytical workload. Using read replicas is a cost-effective and efficient way to address the increased read-only query load.

Authoritative Links:

AWS RDS Read Replicas:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.ReadReplicas.html

Question: 270 **Exam Heist**

A company is using a centralized AWS account to store log data in various Amazon S3 buckets. A solutions architect needs to ensure that the data is encrypted at rest before the data is uploaded to the S3 buckets. The data also must be encrypted in transit.

Which solution meets these requirements?

A. Use client-side encryption to encrypt the data that is being uploaded to the S3 buckets.

B. Use server-side encryption to encrypt the data that is being uploaded to the S3 buckets.

C. Create bucket policies that require the use of server-side encryption with S3 managed encryption keys (SSE-S3) for S3 uploads.

D. Enable the security option to encrypt the S3 buckets through the use of a default AWS Key Management Service (AWS KMS) key.

Answer: A

Explanation:

Here's a detailed justification for why option A is the most suitable answer:

The question requires encryption at rest before data is uploaded to S3 and encryption in transit. Client-side encryption addresses both requirements directly. By encrypting the data on the client-side (i.e., before it leaves the company's environment) before uploading it to S3, you ensure that the data is already encrypted at rest. This addresses the requirement of encryption at rest prior to upload. Then, to meet the encryption-in-transit requirement, ensure that HTTPS (TLS) is used during the upload process. This encrypts the data as it travels across the network to S3.

Options B, C, and D involve server-side encryption. While server-side encryption (SSE) methods address encryption at rest within S3, they don't fulfill the requirement of encrypting the data before it's uploaded. In those scenarios, the data transits to S3 unencrypted and then gets encrypted by S3 itself. Furthermore, simply creating bucket policies (option C) or enabling a KMS key (option D) does not guarantee data is encrypted before it's uploaded.

Client-side encryption gives the company more control over the encryption process, particularly the management of encryption keys. The company can manage its keys, rotate them, and maintain ownership of the encryption process from end to end.

In summary, using client-side encryption with HTTPS meets both stated requirements: encrypting the data at rest before upload and encrypting the data in transit. This provides a strong security posture and full control over the encryption process.

Relevant links:

Amazon S3 Data Protection: https://docs.aws.amazon.com/AmazonS3/latest/userguide/data-protection.html
Protecting Data Using Server-Side Encryption:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingEncryption.html
Protecting Data Using Client-Side Encryption:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingEncryption.html

Question: 271 **Exam Heist**

A solutions architect observes that a nightly batch processing job is automatically scaled up for 1 hour before the desired Amazon EC2 capacity is reached. The peak capacity is the ‘same every night and the batch jobs always start at 1 AM. The solutions architect needs to find a cost-effective solution that will allow for the desired EC2 capacity to be reached quickly and allow the Auto Scaling group to scale down after the batch jobs are complete.

What should the solutions architect do to meet these requirements?

A. Increase the minimum capacity for the Auto Scaling group.
B. Increase the maximum capacity for the Auto Scaling group.
C. Configure scheduled scaling to scale up to the desired compute level.
D. Change the scaling policy to add more EC2 instances during each scaling operation.

Answer: C

Explanation:

The correct answer is C, configuring scheduled scaling to scale up to the desired compute level. Here's why:

The core problem is that the Auto Scaling group takes too long to reach the required capacity for the nightly

batch job, resulting in an unnecessary scaling-up period before the peak is reached. Since the peak capacity and the start time are consistent every night, a predictable scaling pattern exists.

Scheduled scaling allows you to predefine scaling actions that occur at specific times. By configuring a scheduled action to increase the capacity of the Auto Scaling group to the required compute level at 1 AM, the architect can ensure that the desired capacity is available immediately when the batch jobs start. This eliminates the gradual scaling process and the unnecessary 1-hour scale-up period. After the batch job finishes, another scheduled action can scale the group down, optimizing cost.

Option A (increasing the minimum capacity) is not cost-effective. It will keep more instances running continuously, even when they are not needed, leading to unnecessary expenses.

Option B (increasing the maximum capacity) only allows the Auto Scaling group to scale up higher if needed, but it does not guarantee a faster initial scaling to the desired level. The gradual scaling issue would persist.

Option D (changing the scaling policy) would affect the responsiveness of scaling events based on real-time metrics (CPU utilization, etc.), which are not directly related to the predictable nightly demand. Furthermore, it doesn't address the specific timing requirement. It might make the scaling more aggressive but doesn't guarantee the capacity will be available exactly at 1 AM.

Therefore, scheduled scaling provides the most cost-effective and precise solution to address the consistent and time-bound scaling needs of the nightly batch job. It leverages the predictability of the workload to ensure resources are available exactly when needed and are released when no longer required.

Further research:

AWS Auto Scaling Scheduled Actions:
https://docs.aws.amazon.com/autoscaling/ec2/userguide/schedule_time.html
AWS Auto Scaling Concepts:
https://docs.aws.amazon.com/autoscaling/ec2/userguide/AutoScalingGroup.html

Question: 272 **Exam Heist**

A company serves a dynamic website from a fleet of Amazon EC2 instances behind an Application Load Balancer (ALB). The website needs to support multiple languages to serve customers around the world. The website's architecture is running in the us-west-1 Region and is exhibiting high request latency for users that are located in other parts of the world.

The website needs to serve requests quickly and efficiently regardless of a user's location. However, the company does not want to recreate the existing architecture across multiple Regions.

What should a solutions architect do to meet these requirements?

A. Replace the existing architecture with a website that is served from an Amazon S3 bucket. Configure an Amazon CloudFront distribution with the S3 bucket as the origin. Set the cache behavior settings to cache based on the Accept-Language request header.

B. Configure an Amazon CloudFront distribution with the ALB as the origin. Set the cache behavior settings to cache based on the Accept-Language request header.

C. Create an Amazon API Gateway API that is integrated with the ALB. Configure the API to use the HTTP integration type. Set up an API Gateway stage to enable the API cache based on the Accept-Language request header.

D. Launch an EC2 instance in each additional Region and configure NGINX to act as a cache server for that Region. Put all the EC2 instances and the ALB behind an Amazon Route 53 record set with a geolocation routing policy.

Answer: B

Explanation:

The correct solution is to configure an Amazon CloudFront distribution with the Application Load Balancer (ALB) as the origin and set the cache behavior settings to cache based on the Accept-Language request header (Option B).

Here's why:

CloudFront for Global Content Delivery: CloudFront is AWS's Content Delivery Network (CDN). It caches content at edge locations around the world, reducing latency for users regardless of their geographic location. This directly addresses the requirement of serving requests quickly and efficiently worldwide without recreating the architecture in multiple regions. https://aws.amazon.com/cloudfront/
ALB as Origin: The existing website architecture utilizes an ALB. Configuring CloudFront with the ALB as the origin allows CloudFront to cache the dynamic content generated by the EC2 instances behind the ALB. This avoids the need for significant architectural changes.
Accept-Language Header Caching: The Accept-Language header in an HTTP request specifies the user's preferred language. By configuring CloudFront to cache based on this header, the CDN can store different versions of the website's content for each language. When a user requests the website, CloudFront will serve the cached version corresponding to their preferred language, ensuring a localized experience. https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/content-based-config.html

Let's analyze why the other options are less suitable:

Option A (S3 and CloudFront): Replacing the existing dynamic website architecture with a static website served from S3 is not practical, as the website is defined as dynamic. S3 is ideal for static content but not for dynamic content requiring server-side processing.
Option C (API Gateway and ALB): API Gateway is generally used for managing APIs, not for caching dynamic website content. While it can be configured for caching, CloudFront is a more efficient and purpose-built solution for content delivery and caching for web applications. Furthermore, introducing API Gateway adds unnecessary complexity.
Option D (EC2 cache servers and Route 53): Launching EC2 instances in each region for caching is effectively recreating the architecture in multiple regions, which the company wants to avoid. While Route 53 geolocation routing would direct users to the closest cache server, this approach is more complex, costly, and difficult to manage than using a CDN like CloudFront.

In summary, Option B leverages CloudFront's global CDN capabilities and the Accept-Language header to deliver localized content efficiently without requiring significant architectural changes or regional replication.

Question: 273 **Exam Heist**

A rapidly growing ecommerce company is running its workloads in a single AWS Region. A solutions architect must create a disaster recovery (DR) strategy that includes a different AWS Region. The company wants its database to be up to date in the DR Region with the least possible latency. The remaining infrastructure in the DR Region needs to run at reduced capacity and must be able to scale up if necessary.

Which solution will meet these requirements with the LOWEST recovery time objective (RTO)?

A. Use an Amazon Aurora global database with a pilot light deployment.
B. Use an Amazon Aurora global database with a warm standby deployment.
C. Use an Amazon RDS Multi-AZ DB instance with a pilot light deployment.
D. Use an Amazon RDS Multi-AZ DB instance with a warm standby deployment.

Answer: B

Explanation:

The question requires a DR strategy with the lowest RTO for a rapidly growing e-commerce company. The database must be up-to-date with minimal latency in the DR region, and other infrastructure should run at reduced capacity, scaling up as needed.

Option B, "Use an Amazon Aurora global database with a warm standby deployment," offers the best solution. Aurora Global Database replicates data with low latency across AWS Regions, ensuring the DR database is near real-time synchronized, minimizing data loss and RTO. The warm standby approach involves having the DR infrastructure running at a reduced capacity, ready to scale up quickly.

Option A, using Aurora Global Database with a pilot light approach, would require more time to bring up all resources, increasing the RTO compared to a warm standby. While Aurora Global Database ensures low latency replication, the pilot light method necessitates provisioning and configuring resources during a disaster, increasing downtime.

Options C and D, using Amazon RDS Multi-AZ DB instances, do not offer cross-region replication like Aurora Global Database. RDS Multi-AZ provides high availability within a single region but does not provide a quick DR solution to a separate region with minimal latency. Therefore, they would have significantly higher RTOs than option B.

In summary, Aurora Global Database provides continuous replication with minimal latency across regions, and a warm standby configuration minimizes the time required to recover in the DR region, achieving the lowest RTO compared to other options.

Amazon Aurora Global Databases: https://aws.amazon.com/rds/aurora/global-database/AWS Disaster Recovery Options: https://d1.awsstatic.com/whitepapers/aws-disaster-recovery.pdf

Question: 274 **Exam Heist**

A company runs an application on Amazon EC2 instances. The company needs to implement a disaster recovery (DR) solution for the application. The DR solution needs to have a recovery time objective (RTO) of less than 4 hours. The DR solution also needs to use the fewest possible AWS resources during normal operations.

Which solution will meet these requirements in the MOST operationally efficient way?

A. Create Amazon Machine Images (AMIs) to back up the EC2 instances. Copy the AMIs to a secondary AWS Region. Automate infrastructure deployment in the secondary Region by using AWS Lambda and custom scripts.

B. Create Amazon Machine Images (AMIs) to back up the EC2 instances. Copy the AMIs to a secondary AWS Region. Automate infrastructure deployment in the secondary Region by using AWS CloudFormation.

C. Launch EC2 instances in a secondary AWS Region. Keep the EC2 instances in the secondary Region active at all times.

D. Launch EC2 instances in a secondary Availability Zone. Keep the EC2 instances in the secondary Availability Zone active at all times.

Answer: B

Explanation:

Option B is the most operationally efficient DR solution that meets the requirements. Here's why:

AMIs for Backup: Creating AMIs provides a consistent and point-in-time backup of the EC2 instances, including the OS, applications, and data. This facilitates a quicker recovery process compared to rebuilding servers from scratch.
Cross-Region Replication: Copying AMIs to a secondary AWS Region addresses the disaster recovery aspect

by ensuring that the necessary server images are available in a separate geographical location in case the primary region becomes unavailable.
Infrastructure as Code (IaC) with CloudFormation: AWS CloudFormation allows defining and managing infrastructure as code. This enables automating the deployment of EC2 instances and other necessary AWS resources in the secondary region during a disaster recovery event. The automated nature helps achieve the 4-hour RTO target.
Cost Efficiency: This approach minimizes resource utilization during normal operations. The infrastructure in the secondary region remains inactive until a disaster event occurs. Only storage costs for the replicated AMIs and CloudFormation templates incur during normal operation.
Operational Efficiency: CloudFormation provides version control, rollback capabilities, and dependency management, making it significantly more operationally efficient than using custom scripts and Lambda functions.

Option A is less operationally efficient because Lambda and custom scripts require more management and are less robust than using CloudFormation's infrastructure-as-code approach.

Option C involves launching and maintaining EC2 instances in the secondary region at all times, which is the most costly and inefficient approach, especially when the disaster event is not happening. It goes against the requirement of using the fewest possible AWS resources during normal operations.

Option D only uses a secondary Availability Zone within the same region. This does not fully address disaster recovery since an entire region outage would affect both availability zones.

Supporting Links:

AWS CloudFormation: https://aws.amazon.com/cloudformation/
Amazon Machine Images (AMIs): https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AMIs.html
Disaster Recovery with AWS: https://aws.amazon.com/disaster-recovery/

Question: 275 **Exam Heist**

A company runs an internal browser-based application. The application runs on Amazon EC2 instances behind an Application Load Balancer. The instances run in an Amazon EC2 Auto Scaling group across multiple Availability Zones. The Auto Scaling group scales up to 20 instances during work hours, but scales down to 2 instances overnight. Staff are complaining that the application is very slow when the day begins, although it runs well by mid-morning.

How should the scaling be changed to address the staff complaints and keep costs to a minimum?

A. Implement a scheduled action that sets the desired capacity to 20 shortly before the office opens.
B. Implement a step scaling action triggered at a lower CPU threshold, and decrease the cooldown period.
C. Implement a target tracking action triggered at a lower CPU threshold, and decrease the cooldown period.
D. Implement a scheduled action that sets the minimum and maximum capacity to 20 shortly before the office opens.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best answer:

The problem is application slowdown during the morning rush due to EC2 Auto Scaling needing time to spin up instances and handle the increased load. The goal is to improve initial responsiveness while minimizing costs.

Why C is the best approach (Target Tracking Scaling with adjusted thresholds and cooldown):

Target tracking scaling: This is ideal for maintaining consistent performance. By targeting a specific metric like CPU utilization, it dynamically adjusts capacity to meet demand, ensuring the application remains responsive even during peak hours.
Lower CPU threshold: Triggering scaling at a lower CPU threshold ensures that new instances are launched before the existing instances become overloaded, thus preventing the initial slowdown. This is proactive, not reactive.
Decreased cooldown period: The cooldown period is the amount of time after a scaling activity completes before another scaling activity can start. Reducing it allows the Auto Scaling group to react more quickly to fluctuating demand, enabling a faster response to the morning load spike. This ensures resources are added promptly when needed.
Cost-effectiveness: Target tracking only adds capacity when needed, unlike scheduled actions that might over-provision resources even if demand doesn't fully materialize.

Why other options are less suitable:

A (Scheduled action setting desired capacity to 20): This approach doesn't account for actual load. If the load isn't high enough to require 20 instances, resources are wasted. It's inflexible to unexpected variations in demand.
B (Step Scaling with adjusted thresholds and cooldown): Step scaling is based on pre-defined steps. This requires more effort to configure and fine-tune than target tracking, and it's less adaptive to dynamic load changes. Although it uses a threshold and reduced cooldown, it is not as dynamically responsive as target tracking which continuously adjusts based on the tracked metric.
D (Scheduled action setting minimum and maximum capacity to 20): This is the most expensive option because it maintains 20 instances regardless of actual load. This eliminates cost savings during off-peak hours when only a few instances are needed. It's also unnecessarily restrictive, preventing the Auto Scaling group from exceeding 20 instances if needed in unforeseen circumstances.

In summary: Target tracking with a lower CPU threshold and decreased cooldown provides a balanced approach, proactively scaling up resources to maintain performance, reacting quickly to demand changes, and avoiding unnecessary costs during off-peak hours.

Authoritative Links for Further Research:

AWS Auto Scaling: https://aws.amazon.com/autoscaling/
Target Tracking Scaling: https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-target-tracking.html
Cooldown Period: https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-instance-termination.html#instance-cooldown

Question: 276 **Exam Heist**

A company has a multi-tier application deployed on several Amazon EC2 instances in an Auto Scaling group. An Amazon RDS for Oracle instance is the application' s data layer that uses Oracle-specific PL/SQL functions. Traffic to the application has been steadily increasing. This is causing the EC2 instances to become overloaded and the RDS instance to run out of storage. The Auto Scaling group does not have any scaling metrics and defines the minimum healthy instance count only. The company predicts that traffic will continue to increase at a steady but unpredictable rate before leveling off.

What should a solutions architect do to ensure the system can automatically scale for the increased traffic? (Choose two.)

A. Configure storage Auto Scaling on the RDS for Oracle instance.
B. Migrate the database to Amazon Aurora to use Auto Scaling storage.
C. Configure an alarm on the RDS for Oracle instance for low free storage space.
D. Configure the Auto Scaling group to use the average CPU as the scaling metric.

E. Configure the Auto Scaling group to use the average free memory as the scaling metric.

Answer: AD

Explanation:

The correct answer is AD. Here's why:

A. Configure storage Auto Scaling on the RDS for Oracle instance: RDS Storage Auto Scaling allows the database to automatically scale its storage capacity in response to growing data needs. This directly addresses the problem of the RDS instance running out of storage due to increasing traffic and data. Since the company predicts traffic will increase, proactive storage scaling is essential. This is a crucial consideration for relational databases where running out of storage can lead to application downtime and data loss. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_PIOPS.StorageTypes.html#USER_PIOPS.Autos

D. Configure the Auto Scaling group to use the average CPU as the scaling metric: Using average CPU utilization as a scaling metric for the Auto Scaling group allows new EC2 instances to be launched automatically when the existing instances are under heavy load. This addresses the problem of EC2 instances becoming overloaded due to increasing traffic. Monitoring CPU utilization provides a clear indication of resource consumption and allows the application to scale horizontally based on demand. This ensures the application remains responsive and available as traffic increases. https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-capacity.html

Why other options are incorrect:

B. Migrate the database to Amazon Aurora to use Auto Scaling storage: While Aurora does offer auto-scaling storage, migrating a database, especially one using Oracle-specific PL/SQL, is a significant undertaking. It involves compatibility assessments, code changes, and data migration, which introduces risk and complexity. Given the existing infrastructure's reliance on Oracle-specific functionality, migrating to Aurora might not be the most efficient or immediate solution for the current scaling issue. It's a major architectural change better suited for a long-term strategic decision rather than a quick fix to the overload problem.

C. Configure an alarm on the RDS for Oracle instance for low free storage space: While configuring an alarm is a good practice for monitoring, it only provides notification when the storage is running low. It doesn't automatically scale the storage. It requires manual intervention to increase storage, which is not ideal for automatically scaling for unpredictable traffic.

E. Configure the Auto Scaling group to use the average free memory as the scaling metric: While monitoring memory usage is important, CPU utilization is generally a better metric for scaling web application tiers. Low free memory can indicate an issue, but high CPU utilization directly reflects the workload demand on the instances. Therefore, CPU is often the more responsive metric for scaling the application tier in this scenario.

Question: 277 **Exam Heist**

A company provides an online service for posting video content and transcoding it for use by any mobile platform. The application architecture uses Amazon Elastic File System (Amazon EFS) Standard to collect and store the videos so that multiple Amazon EC2 Linux instances can access the video content for processing. As the popularity of the service has grown over time, the storage costs have become too expensive.

Which storage solution is MOST cost-effective?

A. Use AWS Storage Gateway for files to store and process the video content.

B. Use AWS Storage Gateway for volumes to store and process the video content.

C. Use Amazon EFS for storing the video content. Once processing is complete, transfer the files to Amazon

Elastic Block Store (Amazon EBS).

D. Use Amazon S3 for storing the video content. Move the files temporarily over to an Amazon Elastic Block Store (Amazon EBS) volume attached to the server for processing.

Answer: D

Explanation:

The most cost-effective solution for storing and processing video content in this scenario is option D: using Amazon S3 for storage and temporarily moving files to an EBS volume attached to the processing server.

Here's why:

Cost of EFS Standard: EFS Standard is designed for active file system workloads requiring frequent access. As the service's popularity increases, the storage costs with EFS Standard become prohibitively expensive because it charges based on the amount of storage used, regardless of access frequency.

S3 for Cost-Effectiveness: Amazon S3 offers various storage classes optimized for different access patterns. In this case, the infrequent access nature of the video content once processed makes S3 a more cost-effective primary storage solution. S3's storage classes like S3 Standard-IA (Infrequent Access) or S3 Glacier (for archival) offer significantly lower storage costs than EFS Standard.

EBS for Processing: Amazon EBS volumes provide block storage optimized for EC2 instances. Temporarily moving the video files to an EBS volume attached to the processing server allows for efficient processing due to its low latency and high throughput. EBS volumes offer consistent performance for video transcoding tasks.

Workflow Optimization: The workflow would involve retrieving the video file from S3 to the EBS volume, performing the transcoding on the EC2 instance, and then potentially deleting the file from EBS (if no longer needed) after transcoding.

AWS Storage Gateway limitations: AWS Storage Gateway is primarily used for hybrid cloud scenarios to integrate on-premises storage with AWS. While it can connect to S3, using it as the primary storage solution is not ideal in this completely cloud-based architecture. Both file and volume gateways have limitations that make them less suitable than a direct S3 to EC2 to EBS workflow.

Option C is suboptimal: Storing everything in EFS and then moving to EBS after processing doesn't address the core cost issue. EFS would still be the primary, expensive storage location.

Therefore, leveraging S3 for cost-effective archival storage and EBS for temporary, high-performance processing offers the optimal balance of cost and performance.

Relevant links:

Amazon S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
Amazon EBS: https://aws.amazon.com/ebs/
Amazon EFS: https://aws.amazon.com/efs/
AWS Storage Gateway: https://aws.amazon.com/storagegateway/

Question: 278 **Exam Heist**

A company wants to create an application to store employee data in a hierarchical structured relationship. The company needs a minimum-latency response to high-traffic queries for the employee data and must protect any sensitive data. The company also needs to receive monthly email messages if any financial information is present in the employee data.

Which combination of steps should a solutions architect take to meet these requirements? (Choose two.)

A. Use Amazon Redshift to store the employee data in hierarchies. Unload the data to Amazon S3 every month.

B. Use Amazon DynamoDB to store the employee data in hierarchies. Export the data to Amazon S3 every month.

C. Configure Amazon Macie for the AWS account. Integrate Macie with Amazon EventBridge to send monthly events to AWS Lambda.

D. Use Amazon Athena to analyze the employee data in Amazon S3. Integrate Athena with Amazon QuickSight to publish analysis dashboards and share the dashboards with users.

E. Configure Amazon Macie for the AWS account. Integrate Macie with Amazon EventBridge to send monthly notifications through an Amazon Simple Notification Service (Amazon SNS) subscription.

Answer: BE

Explanation:

The correct answer is **BE**. Let's break down why:

B: Use Amazon DynamoDB to store the employee data in hierarchies. Export the data to Amazon S3 every month. DynamoDB is a NoSQL database that can store hierarchical data using techniques like adjacency lists or materialized paths. It provides extremely low-latency responses for high-traffic queries, meeting the primary requirement. While DynamoDB isn't inherently hierarchical, the data can be structured hierarchically within the NoSQL schema. Exporting the data to S3 monthly allows for the subsequent analysis and monitoring.
E: Configure Amazon Macie for the AWS account. Integrate Macie with Amazon EventBridge to send monthly notifications through an Amazon Simple Notification Service (Amazon SNS) subscription. Amazon Macie is designed to discover and protect sensitive data stored in S3. By configuring it for the AWS account and integrating it with EventBridge, you can trigger an SNS notification when Macie identifies financial information. This meets the requirement of receiving monthly email notifications about the presence of sensitive data. Macie scanning in S3 (where the exported DynamoDB data resides) can trigger SNS notifications via EventBridge, aligning with the monthly requirement.

Why other options are incorrect:

A: Redshift is a data warehouse optimized for analytical queries, not low-latency transactional reads. Storing hierarchical data and querying it for real-time access is not its strength.

C: While configuring Macie is useful, directly sending events to Lambda is not a notification mechanism for monthly email messages to humans without additional steps, like implementing an email service within Lambda. Lambda also has execution time limits that might cause it to fail for large datasets. Additionally, integrating macie directly with event bridge would create notifications as soon as the data is stored in s3.

D: Athena is suitable for analyzing data in S3, but it's not needed for the initial storage and retrieval of employee data. QuickSight is a visualization tool, not a direct solution for detecting financial information.Athena + QuickSight do not provide any mechanism to alert when sensitive financial information appears in the employee data. This requires some automated scanning and notification service like Macie and SNS.

Authoritative Links:

Amazon DynamoDB: https://aws.amazon.com/dynamodb/
Amazon Macie: https://aws.amazon.com/macie/
Amazon EventBridge: https://aws.amazon.com/eventbridge/
Amazon SNS: https://aws.amazon.com/sns/

Question: 279 **Exam Heist**

A company has an application that is backed by an Amazon DynamoDB table. The company's compliance requirements specify that database backups must be taken every month, must be available for 6 months, and must be retained for 7 years.

Which solution will meet these requirements?

A. Create an AWS Backup plan to back up the DynamoDB table on the first day of each month. Specify a lifecycle policy that transitions the backup to cold storage after 6 months. Set the retention period for each backup to 7 years.

B. Create a DynamoDB on-demand backup of the DynamoDB table on the first day of each month. Transition the backup to Amazon S3 Glacier Flexible Retrieval after 6 months. Create an S3 Lifecycle policy to delete backups that are older than 7 years.

C. Use the AWS SDK to develop a script that creates an on-demand backup of the DynamoDB table. Set up an Amazon EventBridge rule that runs the script on the first day of each month. Create a second script that will run on the second day of each month to transition DynamoDB backups that are older than 6 months to cold storage and to delete backups that are older than 7 years.

D. Use the AWS CLI to create an on-demand backup of the DynamoDB table. Set up an Amazon EventBridge rule that runs the command on the first day of each month with a cron expression. Specify in the command to transition the backups to cold storage after 6 months and to delete the backups after 7 years.

Answer: A

Explanation:

The correct answer is A. Here's why:

AWS Backup is the ideal service for centrally managing and automating backups across AWS services, including DynamoDB. It allows you to define backup plans with schedules and retention policies.

A directly addresses all requirements:

Monthly Backups: An AWS Backup plan can be configured to run on the first day of each month.
Availability for 6 Months: The lifecycle policy allows for transitioning backups to cold storage after 6 months, ensuring backups are still available for restore but at a lower storage cost.
Retention for 7 Years: The retention period can be set to 7 years, satisfying the long-term retention requirement.

Why the other options are less suitable:

B: While you can create DynamoDB on-demand backups, transitioning them directly to S3 Glacier Flexible Retrieval is not the most efficient way, and the AWS Backup service has better options. Backup should be maintained in the AWS Backup vault for better integrity.
C and D: While automating on-demand backups via the AWS SDK or CLI and EventBridge is possible, it requires more manual scripting and management compared to the AWS Backup service. Furthermore, the command/scripts do not transition backups to cold storage and delete old backups, which is less ideal than using the AWS Backup. The second scripts will also incur the cost of development, running, and maintaining.

Using AWS Backup centralizes the backup process and ensures that compliance requirements are met in an automated and auditable manner. It simplifies lifecycle management and retention, reducing the operational overhead.

Supporting Documentation:

AWS Backup: https://aws.amazon.com/backup/
DynamoDB Backup and Restore:
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/BackupRestore.html
AWS Backup Plans: https://docs.aws.amazon.com/aws-backup/latest/devguide/whatisbackup.html

Question: 280 **Exam Heist**

A company is using Amazon CloudFront with its website. The company has enabled logging on the CloudFront distribution, and logs are saved in one of the company's Amazon S3 buckets. The company needs to perform advanced analyses on the logs and build visualizations.

What should a solutions architect do to meet these requirements?

A. Use standard SQL queries in Amazon Athena to analyze the CloudFront logs in the S3 bucket. Visualize the results with AWS Glue.

B. Use standard SQL queries in Amazon Athena to analyze the CloudFront logs in the S3 bucket. Visualize the results with Amazon QuickSight.

C. Use standard SQL queries in Amazon DynamoDB to analyze the CloudFront logs in the S3 bucket. Visualize the results with AWS Glue.

D. Use standard SQL queries in Amazon DynamoDB to analyze the CloudFront logs in the S3 bucket. Visualize the results with Amazon QuickSight.

Answer: B

Explanation:

The correct solution is to use Amazon Athena for analyzing CloudFront logs in S3 and Amazon QuickSight for visualization.

Here's why:

Amazon Athena: Athena is a serverless, interactive query service that makes it easy to analyze data in Amazon S3 using standard SQL. CloudFront logs are stored as files in S3, making Athena a natural fit for querying and analyzing them directly. You can define a schema on the log files and then use SQL to extract insights, filter data, and perform aggregations. https://aws.amazon.com/athena/
Amazon QuickSight: QuickSight is a fast, cloud-powered business intelligence (BI) service that makes it easy to build visualizations, perform ad-hoc analysis, and get business insights from your data. It integrates seamlessly with Athena and can directly visualize the results of Athena queries.
https://aws.amazon.com/quicksight/

Why other options are incorrect:

DynamoDB: DynamoDB is a NoSQL database designed for high-performance key-value and document storage. It's not well-suited for directly querying log files stored in S3. CloudFront logs are typically stored in a flat file format. Loading log data into DynamoDB just for analysis would be an unnecessary complexity.
AWS Glue: While Glue is a useful ETL (extract, transform, load) service, it is not the ideal choice for visualizing data. Glue excels at preparing data for analysis, but Athena and QuickSight provide a more direct path for querying and visualizing data stored in S3. Glue's primary purpose is data cataloging, transformation, and job orchestration, rather than data visualization.
Combining Athena and Glue vs. Athena and Quicksight: Athena handles the query execution on the log files in S3 and QuickSight specializes in visualization. This combination offers a more seamless experience for the stated objective. Using Glue to prepare for visualizations adds unnecessary complexity.

Question: 281 **Exam Heist**

A company runs a fleet of web servers using an Amazon RDS for PostgreSQL DB instance. After a routine compliance check, the company sets a standard that requires a recovery point objective (RPO) of less than 1 second for all its production databases.

Which solution meets these requirements?

A. Enable a Multi-AZ deployment for the DB instance.

B. Enable auto scaling for the DB instance in one Availability Zone.

C. Configure the DB instance in one Availability Zone, and create multiple read replicas in a separate Availability Zone.

D. Configure the DB instance in one Availability Zone, and configure AWS Database Migration Service (AWS DMS) change data capture (CDC) tasks.

Answer: A

Explanation:

The requirement is to achieve an RPO of less than 1 second for an Amazon RDS for PostgreSQL database. This necessitates a solution that minimizes data loss in the event of a failure.

Option A, enabling a Multi-AZ deployment for the RDS instance, provides synchronous replication to a standby instance in a different Availability Zone. In case of a failure in the primary instance, RDS automatically fails over to the standby instance. Because the replication is synchronous, the standby has near real-time data, minimizing data loss to only what was in-flight at the time of the failure. This fulfills the <1 second RPO.

Option B, enabling auto-scaling, addresses performance and scalability, not data recovery and RPO.

Option C, using read replicas, provides asynchronous replication. While read replicas can improve read performance, they do not guarantee minimal data loss. In the event of a primary instance failure, data may be lost that hasn't yet been replicated to the read replicas. This violates the <1 second RPO.

Option D, using AWS DMS CDC, also employs asynchronous replication. While DMS can capture changes in near real-time, it involves more overhead and complexity than Multi-AZ for simple failover scenarios. Additionally, the lag between the primary database and the DMS target (another database) can easily exceed 1 second, failing the RPO requirement. It also introduces more points of failure.

Therefore, Multi-AZ offers the simplest and most effective way to meet the <1 second RPO requirement through synchronous replication and automatic failover.

Authoritative Links:

Amazon RDS Multi-AZ deployments:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html

Question: 282 **Exam Heist**

A company runs a web application that is deployed on Amazon EC2 instances in the private subnet of a VPC. An Application Load Balancer (ALB) that extends across the public subnets directs web traffic to the EC2 instances. The company wants to implement new security measures to restrict inbound traffic from the ALB to the EC2 instances while preventing access from any other source inside or outside the private subnet of the EC2 instances.

Which solution will meet these requirements?

A. Configure a route in a route table to direct traffic from the internet to the private IP addresses of the EC2 instances.

B. Configure the security group for the EC2 instances to only allow traffic that comes from the security group for the ALB.

C. Move the EC2 instances into the public subnet. Give the EC2 instances a set of Elastic IP addresses.

D. Configure the security group for the ALB to allow any TCP traffic on any port.

Answer: B

Explanation:

The correct answer is **B. Configure the security group for the EC2 instances to only allow traffic that comes from the security group for the ALB.**

Here's a detailed justification:

The requirement is to restrict inbound traffic to the EC2 instances in the private subnet only from the ALB. This means blocking all other sources, both internal and external. Security Groups act as virtual firewalls at the instance level.

Option B directly addresses this requirement. By configuring the EC2 instances' security group to only allow inbound traffic from the ALB's security group, you ensure that only the ALB can reach the instances. This leverages Security Group source rules. If the ALB is compromised, or if an attacker manages to launch resources inside the same VPC, they still won't be able to directly reach the EC2 instances unless they are associated with the ALB's Security Group. This prevents unauthorized access from other sources within or outside the private subnet.

Option A is incorrect because routing traffic from the internet to the private IP addresses of EC2 instances is not possible without a NAT gateway or similar setup, which would defeat the requirement of blocking external access. Directly routing internet traffic to private IPs is a security risk and not the intended function.

Option C contradicts the requirement. Moving EC2 instances into a public subnet and assigning Elastic IPs would expose them directly to the internet, which violates the restriction of preventing access from any other source outside the ALB. Elastic IPs are typically used for resources needing direct internet accessibility, which we want to avoid here.

Option D is incorrect because allowing any TCP traffic on any port for the ALB's security group would not restrict access to the EC2 instances. It will widen the attack surface on ALB, leading to security vulnerabilities. It doesn't address the core requirement of isolating the EC2 instances to only the ALB. It also potentially allows other sources within the VPC to communicate with the ALB unnecessarily.

In essence, Security Groups are the ideal tool for implementing fine-grained access control at the instance level in AWS, and referencing other Security Groups in your Security Group rules is a security best practice.

Further reading:

AWS Security Groups
Controlling Traffic to Resources Using Security Groups

Question: 283 **Exam Heist**

A research company runs experiments that are powered by a simulation application and a visualization application. The simulation application runs on Linux and outputs intermediate data to an NFS share every 5 minutes. The visualization application is a Windows desktop application that displays the simulation output and requires an SMB file system.

The company maintains two synchronized file systems. This strategy is causing data duplication and inefficient resource usage. The company needs to migrate the applications to AWS without making code changes to either application.

Which solution will meet these requirements?

A. Migrate both applications to AWS Lambda. Create an Amazon S3 bucket to exchange data between the applications.

B. Migrate both applications to Amazon Elastic Container Service (Amazon ECS). Configure Amazon FSx File Gateway for storage.

C. Migrate the simulation application to Linux Amazon EC2 instances. Migrate the visualization application to Windows EC2 instances. Configure Amazon Simple Queue Service (Amazon SQS) to exchange data between the applications.

D. Migrate the simulation application to Linux Amazon EC2 instances. Migrate the visualization application to Windows EC2 instances. Configure Amazon FSx for NetApp ONTAP for storage.

Answer: D

Explanation:

The correct solution is D because it addresses the core requirements of providing both NFS and SMB file sharing capabilities without requiring code changes to the simulation and visualization applications.

Here's a detailed justification:

Requirement 1: Linux NFS Share: The simulation application requires an NFS share. Amazon FSx for NetApp ONTAP supports NFS file shares, allowing the Linux EC2 instances running the simulation application to write data to it. https://aws.amazon.com/fsx/netapp-ontap/features/
Requirement 2: Windows SMB File System: The visualization application requires an SMB file system. Amazon FSx for NetApp ONTAP supports SMB file shares, enabling the Windows EC2 instances running the visualization application to read data from it. https://aws.amazon.com/fsx/netapp-ontap/features/
Requirement 3: No Code Changes: By using FSx for NetApp ONTAP, the applications can continue to use their existing file access patterns (NFS and SMB) without requiring any code modifications.
Requirement 4: Eliminate Data Duplication: Using a single FSx for NetApp ONTAP file system eliminates the need for two synchronized file systems, thus reducing data duplication and improving resource efficiency.

Let's examine why the other options are less suitable:

A (Lambda & S3): While S3 can store data, it doesn't natively support NFS or SMB file sharing protocols. This would require significant code changes to both applications to interact with S3's object storage API instead of file systems.
B (ECS & FSx File Gateway): FSx File Gateway caches data locally and uploads it to S3. The visualization application cannot access the data from the Linux NFS share directly via SMB.
C (EC2 & SQS): SQS is a message queuing service, not a file system. It cannot directly replace the NFS and SMB file sharing requirements of the applications. It would necessitate significant code changes to transfer simulation data as messages and reassemble them for visualization.

In summary, solution D using Amazon FSx for NetApp ONTAP provides the most seamless migration path with minimal disruption and no code changes, while meeting all the specified requirements for NFS and SMB file sharing.

Question: 284 **Exam Heist**

As part of budget planning, management wants a report of AWS billed items listed by user. The data will be used to create department budgets. A solutions architect needs to determine the most efficient way to obtain this report information.

Which solution meets these requirements?

A. Run a query with Amazon Athena to generate the report.

B. Create a report in Cost Explorer and download the report.

C. Access the bill details from the billing dashboard and download the bill.

D. Modify a cost budget in AWS Budgets to alert with Amazon Simple Email Service (Amazon SES).

Answer: B

Explanation:

The correct answer is **B. Create a report in Cost Explorer and download the report.** Here's why:

Cost Explorer is designed specifically for analyzing AWS costs and usage. It allows you to generate detailed reports, filter them by various dimensions (including users through tags), and download the data. This aligns directly with the requirement of obtaining a report of AWS billed items listed by user for budget planning. Cost Explorer provides a user-friendly interface and built-in functionality for this purpose.

Option A, using Amazon Athena, would require setting up a complex data pipeline involving AWS Cost and Usage Reports (CUR), configuring S3 buckets, and writing SQL queries. While powerful, it's an unnecessarily complex solution for a simple reporting requirement. It would be overkill to use Athena.

Option C, accessing the bill details from the billing dashboard and downloading the bill, would provide a detailed bill, but it wouldn't readily offer the granularity needed to easily generate a report organized by user. Extracting user-specific data from the raw bill would involve significant manual processing or custom scripting, which isn't efficient.

Option D, modifying a cost budget in AWS Budgets to alert with Amazon SES, focuses on setting up alerts when costs exceed a certain threshold. It does not generate a report of billed items listed by user, which is the core requirement. Budgets help with notifications, not reporting on past usage patterns.

Cost Explorer offers the easiest and most efficient way to generate the required report directly within the AWS console and download the results. Using Cost Explorer is the most optimized method.

https://aws.amazon.com/aws-cost-management/aws-cost-explorer/

Question: 285 **Exam Heist**

A company hosts its static website by using Amazon S3. The company wants to add a contact form to its webpage. The contact form will have dynamic server-side components for users to input their name, email address, phone number, and user message. The company anticipates that there will be fewer than 100 site visits each month.

Which solution will meet these requirements MOST cost-effectively?

A. Host a dynamic contact form page in Amazon Elastic Container Service (Amazon ECS). Set up Amazon Simple Email Service (Amazon SES) to connect to any third-party email provider.

B. Create an Amazon API Gateway endpoint with an AWS Lambda backend that makes a call to Amazon Simple Email Service (Amazon SES).

C. Convert the static webpage to dynamic by deploying Amazon Lightsail. Use client-side scripting to build the contact form. Integrate the form with Amazon WorkMail.

D. Create a t2.micro Amazon EC2 instance. Deploy a LAMP (Linux, Apache, MySQL, PHP/Perl/Python) stack to host the webpage. Use client-side scripting to build the contact form. Integrate the form with Amazon WorkMail.

Answer: B

Explanation:

The most cost-effective solution is **B. Create an Amazon API Gateway endpoint with an AWS Lambda backend that makes a call to Amazon Simple Email Service (Amazon SES).**

Here's why:

Cost Efficiency: For fewer than 100 site visits per month, the "serverless" approach of API Gateway and

Lambda offers the lowest cost. Lambda functions are billed only for the compute time they consume, and API Gateway has a free tier for low volumes. SES is also very cost-effective for sending emails.

Scalability and Management: API Gateway and Lambda automatically scale to handle traffic fluctuations without requiring manual configuration. This removes the operational overhead of managing servers.

Serverless Architecture: This solution aligns with serverless architecture, which minimizes infrastructure management and allows the company to focus on the contact form's functionality.

Alternatives Analysis:

A (ECS): Hosting a dynamic page in ECS involves container orchestration and persistent compute resources, making it overkill and more expensive for the expected low traffic.
C (Lightsail): While Lightsail is simpler than EC2, it still involves a fixed monthly cost, even when the contact form isn't being used. Using client-side scripting for the form's logic is not ideal for security or data processing on the server-side. WorkMail is not well-suited for this use-case.
D (EC2 with LAMP): Running a t2.micro EC2 instance incurs a constant hourly cost, even if the contact form is rarely used. LAMP stack administration also adds complexity.

In contrast, the API Gateway/Lambda/SES approach only charges when the form is submitted, making it the most cost-effective and operationally efficient solution for this low-traffic scenario. This approach avoids the cost of maintaining a dedicated server or virtual machine.

Authoritative Links:

AWS Lambda Pricing: https://aws.amazon.com/lambda/pricing/
Amazon API Gateway Pricing: https://aws.amazon.com/api-gateway/pricing/
Amazon SES Pricing: https://aws.amazon.com/ses/pricing/

Question: 286 **Exam Heist**

A company has a static website that is hosted on Amazon CloudFront in front of Amazon S3. The static website uses a database backend. The company notices that the website does not reflect updates that have been made in the website's Git repository. The company checks the continuous integration and continuous delivery (CI/CD) pipeline between the Git repository and Amazon S3. The company verifies that the webhooks are configured properly and that the CI/CD pipeline is sending messages that indicate successful deployments.

A solutions architect needs to implement a solution that displays the updates on the website.

Which solution will meet these requirements?

A. Add an Application Load Balancer.
B. Add Amazon ElastiCache for Redis or Memcached to the database layer of the web application.
C. Invalidate the CloudFront cache.
D. Use AWS Certificate Manager (ACM) to validate the website's SSL certificate.

Answer: C

Explanation:

The correct solution is to invalidate the CloudFront cache. Here's why:

The problem states that the website updates are not reflected, despite successful deployments to S3. This strongly suggests that CloudFront is serving cached versions of the website files. CloudFront caches content at edge locations to improve latency and performance. While it eventually checks for updates from the origin (S3 in this case), the Time To Live (TTL) setting determines how long content is cached before being

revalidated.

Invalidating the cache forces CloudFront to retrieve the latest version of the website files from the origin (S3). This ensures that users see the most up-to-date content after the CI/CD pipeline has successfully deployed the updates. There are several methods to invalidate the cache, including using the AWS Management Console, AWS CLI, or CloudFront API. This is a common practice in CI/CD pipelines for static websites hosted on S3 and fronted by CloudFront.

Option A (Application Load Balancer) is incorrect because it is primarily used for distributing traffic to application servers, which is not the core issue here. The problem involves static content caching, not application load balancing.

Option B (Amazon ElastiCache) is not relevant because the problem pertains to static website content served through CloudFront, not the database layer. ElastiCache is used for caching database query results or other dynamic data, not static assets.

Option D (AWS Certificate Manager) is unrelated to the caching issue. ACM is used for managing SSL/TLS certificates for secure communication. While having a valid certificate is crucial, it doesn't address the problem of outdated content being served from the CloudFront cache.

Therefore, invalidating the CloudFront cache is the most direct and effective solution to ensure that website updates are immediately reflected to users.

Relevant Documentation:

Invalidating Files: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Invalidation.html
How CloudFront Delivers Content:
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/HowCloudFrontWorks.html

Question: 287 **Exam Heist**

A company wants to migrate a Windows-based application from on premises to the AWS Cloud. The application has three tiers: an application tier, a business tier, and a database tier with Microsoft SQL Server. The company wants to use specific features of SQL Server such as native backups and Data Quality Services. The company also needs to share files for processing between the tiers.

How should a solutions architect design the architecture to meet these requirements?

A. Host all three tiers on Amazon EC2 instances. Use Amazon FSx File Gateway for file sharing between the tiers.

B. Host all three tiers on Amazon EC2 instances. Use Amazon FSx for Windows File Server for file sharing between the tiers.

C. Host the application tier and the business tier on Amazon EC2 instances. Host the database tier on Amazon RDS. Use Amazon Elastic File System (Amazon EFS) for file sharing between the tiers.

D. Host the application tier and the business tier on Amazon EC2 instances. Host the database tier on Amazon RDS. Use a Provisioned IOPS SSD (io2) Amazon Elastic Block Store (Amazon EBS) volume for file sharing between the tiers.

Answer: B

Explanation:

Here's a detailed justification for why option B is the best solution and why the other options are less suitable:

Why Option B is Correct: Host all three tiers on Amazon EC2 instances. Use Amazon FSx for Windows File Server for file sharing between the tiers.

Complete Control: Hosting all three tiers on EC2 instances grants the company complete control over the environment, including the SQL Server database. This is vital for utilizing specific features of SQL Server like native backups and Data Quality Services, which may not be fully supported or accessible with a managed service like Amazon RDS.

SQL Server Feature Compatibility: RDS for SQL Server offers a managed SQL Server instance but can have limitations on certain administrative features. Native backups are fully supported with EC2-hosted SQL server instances. https://aws.amazon.com/rds/sqlserver/

Windows Environment: The application is Windows-based. FSx for Windows File Server is a fully managed, highly reliable, and scalable file storage solution built on Windows Server. It's natively compatible, offering features like SMB protocol support and integration with Active Directory, which are likely required for the application. https://aws.amazon.com/fsx/windows/

File Sharing Requirements: FSx for Windows File Server allows seamless file sharing between the application, business, and database tiers due to its Windows-native nature.

Why Other Options are Incorrect:

Option A: Host all three tiers on Amazon EC2 instances. Use Amazon FSx File Gateway for file sharing between the tiers. FSx File Gateway is for on-premises access to FSx file shares. It does not make sense for applications fully migrated to AWS.

Option C: Host the application tier and the business tier on Amazon EC2 instances. Host the database tier on Amazon RDS. Use Amazon Elastic File System (Amazon EFS) for file sharing between the tiers. EFS is a Linux-based file system and is not well-suited for sharing files in a native Windows environment. Additionally, hosting the database on RDS might restrict access to SQL Server's native features.

Option D: Host the application tier and the business tier on Amazon EC2 instances. Host the database tier on Amazon RDS. Use a Provisioned IOPS SSD (io2) Amazon Elastic Block Store (Amazon EBS) volume for file sharing between the tiers. EBS volumes are block storage and not designed for file sharing between multiple instances. While EBS can be attached to an EC2 instance, creating a shared file system requires additional configuration (e.g., setting up NFS or SMB), adding complexity. Also, as with Option C, using RDS limits SQL Server feature access.

In summary, Option B provides the best balance of control, compatibility, and ease of use for migrating a Windows-based application to AWS while meeting the specific requirements for SQL Server features and file sharing.

Question: 288 **Exam Heist**

A company is migrating a Linux-based web server group to AWS. The web servers must access files in a shared file store for some content. The company must not make any changes to the application.

What should a solutions architect do to meet these requirements?

A. Create an Amazon S3 Standard bucket with access to the web servers.

B. Configure an Amazon CloudFront distribution with an Amazon S3 bucket as the origin.

C. Create an Amazon Elastic File System (Amazon EFS) file system. Mount the EFS file system on all web servers.

D. Configure a General Purpose SSD (gp3) Amazon Elastic Block Store (Amazon EBS) volume. Mount the EBS volume to all web servers.

Answer: C

Explanation:

The correct answer is **C. Create an Amazon Elastic File System (Amazon EFS) file system. Mount the EFS file system on all web servers.**

Here's a detailed justification:

The problem states that a Linux-based web server group needs to access files in a shared file store without requiring application changes. This immediately suggests the need for a network file system that the existing applications can use seamlessly.

Amazon EFS (Elastic File System) is a fully managed, scalable, elastic, cloud-native NFS (Network File System) for Linux-based workloads. It can be mounted on multiple EC2 instances simultaneously, providing a shared file system that behaves just like a local file system from the application's perspective. This fulfills the requirement of not needing application changes. The web servers can access and modify files in the EFS file system as if they were locally stored. EFS offers high availability and durability, making it a suitable choice for production workloads.

Now, let's analyze why the other options are incorrect:

A. Create an Amazon S3 Standard bucket with access to the web servers. While S3 is excellent for object storage, it's not a file system. Applications would need to be rewritten to use S3's APIs to interact with the stored objects. This violates the "no application changes" constraint. S3 is also object storage and not designed to act as a directly mounted file system for Linux web servers.
B. Configure an Amazon CloudFront distribution with an Amazon S3 bucket as the origin. CloudFront is a content delivery network (CDN) designed for caching and distributing static content. While it uses S3 as an origin, it doesn't provide a shared file system for the web servers to write to or directly access in a file system manner. It's primarily for delivering static content to end-users, not for shared storage amongst web servers. This also requires application changes.
D. Configure a General Purpose SSD (gp3) Amazon Elastic Block Store (Amazon EBS) volume. Mount the EBS volume to all web servers. EBS volumes can only be attached to a single EC2 instance at a time, unless using EBS Multi-Attach, which is only supported for specific instances and use cases (and typically requires clustering knowledge). Mounting a single EBS volume to multiple web servers would be impossible without complex configuration and potentially data corruption, defeating the purpose of a shared file system. It would also introduce a single point of failure.

In summary, EFS directly addresses the problem of providing a shared file system accessible by multiple Linux web servers without requiring any changes to the application code. EBS is single-instance attached (without more specialized configurations), and S3/CloudFront involve object storage, requiring application-level changes.

Authoritative Links:

Amazon EFS: https://aws.amazon.com/efs/
Amazon EBS: https://aws.amazon.com/ebs/
Amazon S3: https://aws.amazon.com/s3/
Amazon CloudFront: https://aws.amazon.com/cloudfront/

Question: 289 **Exam Heist**

A company has an AWS Lambda function that needs read access to an Amazon S3 bucket that is located in the same AWS account.

Which solution will meet these requirements in the MOST secure manner?

A. Apply an S3 bucket policy that grants read access to the S3 bucket.

B. Apply an IAM role to the Lambda function. Apply an IAM policy to the role to grant read access to the S3 bucket.

C. Embed an access key and a secret key in the Lambda function's code to grant the required IAM permissions for read access to the S3 bucket.

D. Apply an IAM role to the Lambda function. Apply an IAM policy to the role to grant read access to all S3 buckets in the account.

Answer: B

Explanation:

Here's a detailed justification for why option B is the most secure solution for granting an AWS Lambda function read access to an Amazon S3 bucket in the same account:

Option B leverages IAM roles, which are the recommended way to grant permissions to AWS services like Lambda. An IAM role is an identity that can be assumed by an AWS service, allowing it to make API requests to other AWS services. By assigning an IAM role to the Lambda function, you provide it with temporary security credentials without embedding long-term secrets (like access keys) within the code.

The IAM policy attached to the role specifically grants s3:GetObject (or s3:ListBucket if needed for listing objects) permissions to the particular S3 bucket. This adheres to the principle of least privilege, granting only the necessary access to the function. This means that the Lambda function can only perform read operations on that specific bucket and nothing else.

Option A, using an S3 bucket policy, could technically grant access, but it's less flexible and harder to manage, especially if you need to grant access to multiple Lambda functions or other services. Also, it makes the bucket dependent on the function's needs, which violates separation of concerns.

Option C, embedding access keys in the code, is a major security risk. If the code is compromised (e.g., through a vulnerability or accidental exposure), the access keys could be used to compromise the entire AWS account. Hardcoding credentials is a well-documented anti-pattern.

Option D is less secure than option B because it grants read access to all S3 buckets in the account. This violates the principle of least privilege. The Lambda function only needs access to a specific S3 bucket. Giving it broader access increases the potential blast radius if the function is compromised.

In summary, using an IAM role with a narrowly scoped IAM policy is the most secure and recommended approach for granting permissions to AWS Lambda functions. It avoids hardcoding credentials, adheres to the principle of least privilege, and simplifies permission management.

Relevant Links:

IAM Roles: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html
IAM Policies: https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies.html
Lambda Execution Role: https://docs.aws.amazon.com/lambda/latest/dg/lambda-intro-execution-role.html
Principle of Least Privilege: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html#grant-least-privilege

Question: 290 **Exam Heist**

A company hosts a web application on multiple Amazon EC2 instances. The EC2 instances are in an Auto Scaling group that scales in response to user demand. The company wants to optimize cost savings without making a long-term commitment.

Which EC2 instance purchasing option should a solutions architect recommend to meet these requirements?

A. Dedicated Instances only
B. On-Demand Instances only
C. A mix of On-Demand Instances and Spot Instances
D. A mix of On-Demand Instances and Reserved Instances

Answer: C

Explanation:

The correct answer is **C. A mix of On-Demand Instances and Spot Instances**.

Here's a detailed justification:

The company needs a cost-optimized solution for EC2 instances that scale with user demand without a long-term commitment. Let's analyze why each option is or isn't suitable:

A. Dedicated Instances only: Dedicated Instances are isolated at the hardware level, which incurs a higher cost and doesn't directly contribute to cost optimization for fluctuating workloads. They're more suitable for compliance or security requirements.

B. On-Demand Instances only: On-Demand Instances provide the flexibility to pay only for what you use and are suitable for short-term, spiky, or unpredictable workloads where interruptions are not acceptable. While flexible, they are more expensive than other options like Spot Instances.

C. A mix of On-Demand Instances and Spot Instances: This option offers the best balance between cost savings and availability. On-Demand Instances can handle the baseline capacity and critical workload components that cannot be interrupted. Spot Instances allow the company to bid on spare EC2 capacity, offering significant cost savings (up to 90% compared to On-Demand). The Auto Scaling group can be configured to launch Spot Instances when available and fall back to On-Demand Instances if Spot Instances are terminated due to price fluctuations. This dynamic approach maximizes cost savings while maintaining availability and scalability.

D. A mix of On-Demand Instances and Reserved Instances: Reserved Instances offer significant cost savings compared to On-Demand, but they require a commitment of 1 or 3 years. Since the company wants to avoid long-term commitments, Reserved Instances are not the ideal choice for the entire scaling infrastructure. While some core baseline resources could be reserved, the auto-scaling portion is better served with spot and on-demand.

Therefore, the combination of On-Demand Instances (for guaranteed capacity) and Spot Instances (for opportunistic cost savings) allows the company to optimize costs without a long-term commitment, while leveraging the Auto Scaling group's ability to dynamically adapt to user demand.

Authoritative Links for Further Research:

AWS EC2 Purchasing Options: https://aws.amazon.com/ec2/pricing/
AWS Spot Instances: https://aws.amazon.com/ec2/spot/
AWS Auto Scaling: https://aws.amazon.com/autoscaling/

Question: 291 **Exam Heist**

A media company uses Amazon CloudFront for its publicly available streaming video content. The company wants to secure the video content that is hosted in Amazon S3 by controlling who has access. Some of the company's users are using a custom HTTP client that does not support cookies. Some of the company's users are unable to change the hardcoded URLs that they are using for access.

Which services or methods will meet these requirements with the LEAST impact to the users? (Choose two.)

A. Signed cookies
B. Signed URLs
C. AWS AppSync
D. JSON Web Token (JWT)
E. AWS Secrets Manager

Answer: AB

Explanation:

The requirement is to secure streaming video content in S3 delivered via CloudFront, controlling access without relying on cookies (due to limitations of some HTTP clients) and while accommodating users with hardcoded URLs.

A. Signed Cookies: Incorrect. Signed cookies are used to control access to multiple restricted files. In this case, a user is using a custom HTTP client that does not support cookies. Therefore, signed cookies can't meet the requirement.

B. Signed URLs: Correct. Signed URLs allow time-limited access to individual S3 objects (through CloudFront). Since some users have hardcoded URLs, creating signed URLs and distributing them to authorized users before embedding them in applications allows access to restricted content. Each URL contains an expiration timestamp. Therefore, signed URLs allow users access without the use of cookies.

C. AWS AppSync: Incorrect. AWS AppSync is a managed GraphQL service that can be used to build APIs. It's not directly related to securing streaming video content in S3 through CloudFront using signed URLs and does not help with users with existing hardcoded URLs.

D. JSON Web Token (JWT): Incorrect. JWTs are for authentication and authorization, but they require client-side handling to send the token in headers. Clients with hardcoded URLs and no cookie support can't easily incorporate JWTs, as they likely cannot modify request headers.

E. AWS Secrets Manager: Incorrect. AWS Secrets Manager is used to store, retrieve, and manage secrets. This is irrelevant to controlling access to content via CloudFront.

Therefore, the best combination of services and methods is using signed URLs. Signed URLs are the perfect solution, as they are compatible with clients that do not support cookies and provide a secure way to access the videos. Signed URLs can be used together with hardcoded URLs.

Authoritative Links:

CloudFront Signed URLs: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-signed-urls.html
Using signed URLs to serve private content: https://aws.amazon.com/premiumsupport/knowledge-center/cloudfront-signed-urls-troubleshooting/

Question: 292 **Exam Heist**

A company is preparing a new data platform that will ingest real-time streaming data from multiple sources. The company needs to transform the data before writing the data to Amazon S3. The company needs the ability to use SQL to query the transformed data.

Which solutions will meet these requirements? (Choose two.)

A. Use Amazon Kinesis Data Streams to stream the data. Use Amazon Kinesis Data Analytics to transform the data. Use Amazon Kinesis Data Firehose to write the data to Amazon S3. Use Amazon Athena to query the transformed data from Amazon S3.

B. Use Amazon Managed Streaming for Apache Kafka (Amazon MSK) to stream the data. Use AWS Glue to transform the data and to write the data to Amazon S3. Use Amazon Athena to query the transformed data from Amazon S3.

C. Use AWS Database Migration Service (AWS DMS) to ingest the data. Use Amazon EMR to transform the data and to write the data to Amazon S3. Use Amazon Athena to query the transformed data from Amazon S3.

D. Use Amazon Managed Streaming for Apache Kafka (Amazon MSK) to stream the data. Use Amazon Kinesis Data Analytics to transform the data and to write the data to Amazon S3. Use the Amazon RDS query editor to query the transformed data from Amazon S3.

E. Use Amazon Kinesis Data Streams to stream the data. Use AWS Glue to transform the data. Use Amazon Kinesis Data Firehose to write the data to Amazon S3. Use the Amazon RDS query editor to query the transformed data from Amazon S3.

Answer: AB

Explanation:

Let's analyze each option to determine why A and B are the correct solutions.

Option A: This solution correctly leverages the Kinesis family of services for real-time data ingestion and transformation. Kinesis Data Streams effectively handles the streaming data. Kinesis Data Analytics allows for real-time data transformation using SQL. Kinesis Data Firehose delivers the transformed data to S3. Athena then enables SQL queries directly on the data residing in S3. This approach aligns perfectly with the requirements of real-time processing, transformation, and SQL querying.
https://aws.amazon.com/kinesis/data-analytics/ , https://aws.amazon.com/athena/

Option B: This option also fulfills the stated needs. Amazon MSK, a managed Kafka service, can ingest high-volume streaming data. AWS Glue provides a serverless ETL (Extract, Transform, Load) service that can transform the data and write it to S3. Glue's ability to connect to various data stores makes it suitable for this task. Athena can then be utilized to query the data within the S3 data lake. The combination of MSK, Glue, and Athena is a common pattern for building data lakes and performing analytics. https://aws.amazon.com/msk/, https://aws.amazon.com/glue/

Why other options are incorrect:

Option C: AWS DMS is primarily designed for database migrations, not real-time streaming data ingestion. While EMR can transform data, it's generally an overkill for simple transformations that Kinesis Data Analytics or Glue can handle more efficiently.
Option D: While MSK and Kinesis Data Analytics are valid components, the Amazon RDS query editor is designed for querying relational databases (RDS), not data in S3. Athena is the appropriate tool for querying data in S3 using SQL.
Option E: Similar to Option D, while Kinesis Data Streams, Glue, and Kinesis Data Firehose are suitable components, the Amazon RDS query editor is not the right tool to query data in S3. Athena should be used instead.

In summary, options A and B utilize appropriate AWS services to ingest real-time streaming data, transform it efficiently, store it in S3, and then allow for SQL queries on the transformed data through Athena. Options C, D, and E include inappropriate services, such as DMS or the RDS query editor, for the specific requirements of the data platform.

Question: 293 **Exam Heist**

A company has an on-premises volume backup solution that has reached its end of life. The company wants to use AWS as part of a new backup solution and wants to maintain local access to all the data while it is backed up on AWS. The company wants to ensure that the data backed up on AWS is automatically and securely transferred.

Which solution meets these requirements?

A. Use AWS Snowball to migrate data out of the on-premises solution to Amazon S3. Configure on-premises systems to mount the Snowball S3 endpoint to provide local access to the data.

B. Use AWS Snowball Edge to migrate data out of the on-premises solution to Amazon S3. Use the Snowball Edge file interface to provide on-premises systems with local access to the data.

C. Use AWS Storage Gateway and configure a cached volume gateway. Run the Storage Gateway software appliance on premises and configure a percentage of data to cache locally. Mount the gateway storage volumes to provide local access to the data.

D. Use AWS Storage Gateway and configure a stored volume gateway. Run the Storage Gateway software appliance on premises and map the gateway storage volumes to on-premises storage. Mount the gateway storage volumes to provide local access to the data.

Answer: D

Explanation:

The correct solution is **D. Use AWS Storage Gateway and configure a stored volume gateway. Run the Storage Gateway software appliance on premises and map the gateway storage volumes to on-premises storage. Mount the gateway storage volumes to provide local access to the data.**

Here's a detailed justification:

The scenario requires maintaining local access to data while also backing it up to AWS automatically and securely. AWS Storage Gateway is designed precisely for this hybrid cloud use case, bridging the gap between on-premises environments and AWS storage services.

A **stored volume gateway** is the most suitable configuration because it stores the entire dataset locally on premises and asynchronously backs it up to AWS. This directly satisfies the requirement of maintaining local access to all the data. Changes made locally are continuously and asynchronously replicated to AWS for backup and disaster recovery purposes, ensuring data durability and security.

The Storage Gateway software appliance is deployed on-premises, connecting to your existing storage infrastructure. You then create gateway storage volumes that map to your on-premises storage. These volumes can be mounted to your applications, providing seamless access to the data. The gateway handles the secure transfer of data to AWS.

Option A is incorrect because AWS Snowball is primarily a data migration tool, not a continuous backup solution with local access after migration. Mounting the Snowball S3 endpoint directly is not a supported or practical configuration for local access.

Option B is also incorrect because Snowball Edge, while having some compute capabilities, is still primarily a data migration and edge computing device. Its file interface is designed for specific edge workloads, not for serving as a primary local storage solution backed up to AWS.

Option C, using a cached volume gateway, is not ideal because it only caches a percentage of the data locally. This means not all data is readily available on-premises, violating the requirement for local access to all data. It prioritizes frequently accessed data for local caching, but the less frequently accessed data resides primarily in AWS.

In summary, the stored volume gateway configuration of AWS Storage Gateway provides a robust and secure hybrid cloud solution that maintains local data access while seamlessly backing up data to AWS.

Here are some authoritative links for further research:

AWS Storage Gateway: https://aws.amazon.com/storagegateway/
Stored Volumes: https://docs.aws.amazon.com/storagegateway/latest/userguide/stored-volumes.html

Question: 294 **Exam Heist**

An application that is hosted on Amazon EC2 instances needs to access an Amazon S3 bucket. Traffic must not traverse the internet.

How should a solutions architect configure access to meet these requirements?

A. Create a private hosted zone by using Amazon Route 53.

B. Set up a gateway VPC endpoint for Amazon S3 in the VPC.

C. Configure the EC2 instances to use a NAT gateway to access the S3 bucket.

D. Establish an AWS Site-to-Site VPN connection between the VPC and the S3 bucket.

Answer: B

Explanation:

The correct answer is B, setting up a gateway VPC endpoint for Amazon S3 in the VPC. Here's why:

A gateway VPC endpoint allows resources within your VPC, such as EC2 instances, to privately access Amazon S3 without requiring internet access or using public IPs. This ensures that all traffic remains within the AWS network. A gateway endpoint is a virtual device that is horizontally scaled, redundant, and highly available. It allows communication between your VPC and S3 using AWS's internal network.

Option A, creating a private hosted zone in Route 53, is used for internal DNS resolution within your VPC. While important for naming and discovery, it doesn't establish a private connection to S3; EC2 instances would still need a way to reach S3's public endpoints, potentially via the internet.

Option C, using a NAT gateway, allows EC2 instances without public IPs to initiate outbound internet traffic. This is the opposite of the requirement, as it forces traffic to traverse the internet.

Option D, establishing an AWS Site-to-Site VPN connection to the S3 bucket is not applicable. S3 is a service and doesn't reside in a VPC to connect to via VPN. A Site-to-Site VPN is used to connect your on-premises network to a VPC. Connecting to S3 via VPN is neither possible nor the correct approach.

VPC endpoints are the recommended and most efficient way to provide private connectivity to AWS services like S3. They offer improved security, reduced latency, and lower costs by eliminating internet traffic.

Further research:

AWS VPC Endpoints Documentation: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html
AWS Gateway Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-s3.html

Question: 295 **Exam Heist**

An ecommerce company stores terabytes of customer data in the AWS Cloud. The data contains personally identifiable information (PII). The company wants to use the data in three applications. Only one of the applications needs to process the PII. The PII must be removed before the other two applications process the data.

Which solution will meet these requirements with the LEAST operational overhead?

A. Store the data in an Amazon DynamoDB table. Create a proxy application layer to intercept and process the data that each application requests.

B. Store the data in an Amazon S3 bucket. Process and transform the data by using S3 Object Lambda before returning the data to the requesting application.

C. Process the data and store the transformed data in three separate Amazon S3 buckets so that each application has its own custom dataset. Point each application to its respective S3 bucket.

D. Process the data and store the transformed data in three separate Amazon DynamoDB tables so that each

application has its own custom dataset. Point each application to its respective DynamoDB table.

Answer: B

Explanation:

The most operationally efficient solution is **B. Store the data in an Amazon S3 bucket. Process and transform the data by using S3 Object Lambda before returning the data to the requesting application.**

Here's why:

S3 Object Lambda: S3 Object Lambda allows you to add your own code to Amazon S3 GET requests to modify and process data as it is being retrieved from S3. This means data transformation happens on-the-fly, without the need to store multiple transformed copies of the data. https://aws.amazon.com/s3/object-lambda/
Centralized Data Storage: This solution centralizes the customer data in a single S3 bucket, simplifying data governance and management.
Reduced Storage Costs: Avoiding the creation of multiple datasets minimizes storage costs compared to options C and D.
Least Operational Overhead: S3 Object Lambda simplifies data transformation compared to managing separate data transformation pipelines and storage for each application. The code within the S3 Object Lambda can identify the requesting application and apply the appropriate transformation.
Data Security: By removing PII using S3 Object Lambda before it reaches the two applications that do not need it, the overall security posture is improved.

Let's analyze why the other options are less optimal:

A. DynamoDB with a Proxy Application Layer: While feasible, a proxy application layer adds operational complexity and latency. Managing and scaling the proxy layer introduces extra overhead. DynamoDB is also more expensive for large-scale storage compared to S3.
C. Separate S3 Buckets: Storing three copies of the data (one original, two transformed) increases storage costs and complicates data synchronization and updates. Managing data consistency across multiple buckets adds overhead.
D. Separate DynamoDB Tables: Similar to option C, this approach multiplies storage costs and management overhead, and the cost of DynamoDB is already high. DynamoDB is best used for key-value queries and not suited for batch processing of data.

In summary, S3 Object Lambda offers a serverless and cost-effective way to transform data on-the-fly as it is accessed, minimizing operational overhead and ensuring that only authorized applications have access to the complete dataset.

Question: 296 **Exam Heist**

A development team has launched a new application that is hosted on Amazon EC2 instances inside a development VPC. A solutions architect needs to create a new VPC in the same account. The new VPC will be peered with the development VPC. The VPC CIDR block for the development VPC is 192.168.0.0/24. The solutions architect needs to create a CIDR block for the new VPC. The CIDR block must be valid for a VPC peering connection to the development VPC.

What is the SMALLEST CIDR block that meets these requirements?

A. 10.0.1.0/32
B. 192.168.0.0/24
C. 192.168.1.0/32
D. 10.0.1.0/24

Answer: D

Explanation:

The correct answer is D, 10.0.1.0/24. Let's break down why.

The core requirement is creating a new VPC that can peer with the existing development VPC (192.168.0.0/24). A fundamental rule for VPC peering is that the CIDR blocks of the peered VPCs cannot overlap. Overlapping CIDR blocks would lead to routing conflicts and prevent proper communication between the VPCs.

Option B (192.168.0.0/24) is immediately incorrect because it's the same CIDR block as the development VPC, causing a direct overlap.

Options A (10.0.1.0/32) and C (192.168.1.0/32) are technically valid CIDR blocks. However, a /32 CIDR block represents a single IP address. While a VPC can be created with a /32 CIDR, it's not practically useful, particularly in the context of a real-world application where multiple EC2 instances and other resources need to communicate within the VPC. A /32 block is effectively useless for hosting an application. The question asks for the smallest CIDR block that meets the requirements. Meeting the requirements implies that the VPC is practically usable.

Option D (10.0.1.0/24) provides a non-overlapping CIDR block with the development VPC (192.168.0.0/24). It also gives a usable address space. A /24 CIDR block provides 256 addresses (251 usable, considering network and broadcast addresses) that is a more reasonable and useful size for a VPC. It doesn't overlap with the existing VPC's address space and offers enough IPs to host resources within the peered VPC. It is also a private IP address block making it a valid choice.

In summary, while technically any non-overlapping CIDR would allow for peering, the /32 options are not practical for hosting resources. Option D is the smallest usable and valid CIDR block that enables VPC peering without overlapping address spaces.

Relevant documentation:

AWS VPC Peering: https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html
VPC CIDR Blocks: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-ip-addressing.html

Question: 297 **Exam Heist**

A company deploys an application on five Amazon EC2 instances. An Application Load Balancer (ALB) distributes traffic to the instances by using a target group. The average CPU usage on each of the instances is below 10% most of the time, with occasional surges to 65%.

A solutions architect needs to implement a solution to automate the scalability of the application. The solution must optimize the cost of the architecture and must ensure that the application has enough CPU resources when surges occur.

Which solution will meet these requirements?

A. Create an Amazon CloudWatch alarm that enters the ALARM state when the CPUUtilization metric is less than 20%. Create an AWS Lambda function that the CloudWatch alarm invokes to terminate one of the EC2 instances in the ALB target group.

B. Create an EC2 Auto Scaling group. Select the existing ALB as the load balancer and the existing target group as the target group. Set a target tracking scaling policy that is based on the ASGAverageCPUUtilization metric. Set the minimum instances to 2, the desired capacity to 3, the maximum instances to 6, and the target value to 50%. Add the EC2 instances to the Auto Scaling group.

C. Create an EC2 Auto Scaling group. Select the existing ALB as the load balancer and the existing target group as the target group. Set the minimum instances to 2, the desired capacity to 3, and the maximum

instances to 6. Add the EC2 instances to the Auto Scaling group.

D. Create two Amazon CloudWatch alarms. Configure the first CloudWatch alarm to enter the ALARM state when the average CPUUtilization metric is below 20%. Configure the second CloudWatch alarm to enter the ALARM state when the average CPUUtilization matric is above 50%. Configure the alarms to publish to an Amazon Simple Notification Service (Amazon SNS) topic to send an email message. After receiving the message, log in to decrease or increase the number of EC2 instances that are running.

Answer: B

Explanation:

The correct answer is B because it uses EC2 Auto Scaling with a target tracking scaling policy, a common and efficient way to manage application scalability based on a metric like CPU utilization. Here's a breakdown:

EC2 Auto Scaling Group (ASG): An ASG automatically adjusts the number of EC2 instances in your application based on demand. It maintains a desired capacity and can scale out (add instances) when demand increases and scale in (remove instances) when demand decreases. This directly addresses the need for automated scalability.
ALB Integration: Integrating the ASG with the existing Application Load Balancer (ALB) ensures that traffic is automatically distributed to newly launched instances by the ASG and removed from terminated instances. The ASG automatically registers and deregisters instances from the target group.
Target Tracking Scaling Policy: This policy type is ideal for maintaining a specified target value for a metric. In this case, it uses ASGAverageCPUUtilization. The ASG automatically adjusts the number of instances to keep the average CPU utilization of the group around 50%.
Minimum, Desired, and Maximum Instances: These parameters define the boundaries of the scaling. A minimum of 2 ensures that the application remains available even during low traffic. A maximum of 6 limits the costs and prevents over-provisioning. The desired capacity sets the initial number of instances.
Cost Optimization: By automatically scaling based on CPU utilization, the solution only uses the required resources, minimizing costs when the average CPU utilization is low. When surges occur, the ASG quickly adds instances to maintain performance, preventing performance degradation.

Let's examine why the other options are less suitable:

A: Using a CloudWatch alarm and Lambda function to terminate instances when CPU utilization is low is a scale-in approach but does not handle scaling out to address surges. Furthermore, terminating instances based on low CPU alone could lead to service disruptions if demand suddenly spikes.
C: Creating an ASG without a scaling policy only provides automatic instance replacement in case of failure. It does not automatically adjust the number of instances based on CPU utilization. It only uses the number of instances specified.
D: Using CloudWatch alarms to send email notifications for manual intervention is not an automated solution. Manual intervention is time-consuming and can lead to delays in scaling, resulting in performance issues during surges.

In summary, option B provides the most automated, cost-effective, and reliable solution for scaling the application based on CPU utilization, ensuring it can handle surges while optimizing resource usage during periods of low demand.

Authoritative Links:

AWS Auto Scaling: https://aws.amazon.com/autoscaling/
EC2 Auto Scaling Groups: https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-groups.html
Target Tracking Scaling Policies: https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-target-tracking-scaling-policies.html
CloudWatch Alarms:
https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/AlarmThatSendsEmail.html

Question: 298 **Exam Heist**

A company is running a critical business application on Amazon EC2 instances behind an Application Load Balancer. The EC2 instances run in an Auto Scaling group and access an Amazon RDS DB instance.

The design did not pass an operational review because the EC2 instances and the DB instance are all located in a single Availability Zone. A solutions architect must update the design to use a second Availability Zone.

Which solution will make the application highly available?

A. Provision a subnet in each Availability Zone. Configure the Auto Scaling group to distribute the EC2 instances across both Availability Zones. Configure the DB instance with connections to each network.

B. Provision two subnets that extend across both Availability Zones. Configure the Auto Scaling group to distribute the EC2 instances across both Availability Zones. Configure the DB instance with connections to each network.

C. Provision a subnet in each Availability Zone. Configure the Auto Scaling group to distribute the EC2 instances across both Availability Zones. Configure the DB instance for Multi-AZ deployment.

D. Provision a subnet that extends across both Availability Zones. Configure the Auto Scaling group to distribute the EC2 instances across both Availability Zones. Configure the DB instance for Multi-AZ deployment.

Answer: C

Explanation:

Here's a breakdown of why option C is the correct solution for achieving high availability in the given scenario:

The core issue is the single point of failure within a single Availability Zone (AZ). To enhance availability, resources need to be distributed across multiple AZs. The EC2 instances must reside in multiple AZs to continue serving requests if one AZ fails. This requires creating subnets within each desired AZ. The Auto Scaling group should then be configured to launch instances into these different subnets, ensuring a distribution across the AZs. This provides redundancy for the application tier.

Critically, for the database tier's availability, Multi-AZ deployment is the key. RDS Multi-AZ creates a standby replica of the database in another AZ. In case of a failure in the primary AZ, RDS automatically fails over to the standby replica, minimizing downtime. Simply configuring connections to each network (as in options A and B) doesn't provide automated failover or replication. Extending a single subnet across multiple AZs (as in options B and D) is not a standard or recommended practice; subnets are confined to a single AZ. Furthermore, the Auto Scaling Group must be configured to launch instances into distinct subnets that exist in different Availability Zones to ensure proper distribution and high availability.

Therefore, the complete solution involves:

1. Creating separate subnets within each AZ.
2. Configuring the Auto Scaling group to distribute EC2 instances across these AZ-specific subnets.
3. Enabling RDS Multi-AZ for automatic failover to a standby replica in another AZ.

This approach guarantees that if one AZ experiences an outage, the application can continue running from the other AZ, with the database automatically failing over to the standby instance.

Here are some helpful links for further research:

Amazon RDS Multi-AZ: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html
Amazon EC2 Auto Scaling: https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-groups.html
Amazon VPC Subnets: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Subnets.html

Question: 299 **Exam Heist**

A research laboratory needs to process approximately 8 TB of data. The laboratory requires sub-millisecond latencies and a minimum throughput of 6 GBps for the storage subsystem. Hundreds of Amazon EC2 instances that run Amazon Linux will distribute and process the data.

Which solution will meet the performance requirements?

A. Create an Amazon FSx for NetApp ONTAP file system. Sat each volume' tiering policy to ALL. Import the raw data into the file system. Mount the fila system on the EC2 instances.

B. Create an Amazon S3 bucket to store the raw data. Create an Amazon FSx for Lustre file system that uses persistent SSD storage. Select the option to import data from and export data to Amazon S3. Mount the file system on the EC2 instances.

C. Create an Amazon S3 bucket to store the raw data. Create an Amazon FSx for Lustre file system that uses persistent HDD storage. Select the option to import data from and export data to Amazon S3. Mount the file system on the EC2 instances.

D. Create an Amazon FSx for NetApp ONTAP file system. Set each volume's tiering policy to NONE. Import the raw data into the file system. Mount the file system on the EC2 instances.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer, along with relevant concepts and links for further research:

The research laboratory's requirements of sub-millisecond latencies and a minimum throughput of 6 GBps necessitate a high-performance storage solution. Amazon FSx for Lustre, specifically with persistent SSD storage, is designed for such workloads. Lustre is a parallel distributed file system optimized for speed and throughput, commonly used in high-performance computing (HPC) environments. Using SSDs further enhances the performance, delivering the required low latency.

Option B leverages Amazon S3 as a cost-effective and durable repository for the initial raw data. The FSx for Lustre file system is then linked to the S3 bucket for efficient data ingestion. This import/export functionality allows for a separation of storage tiers: S3 for cost-effective archiving and FSx for Lustre for active processing. The EC2 instances mount the FSx for Lustre file system, enabling them to access and process the data with the necessary performance.

Option A suggests using Amazon FSx for NetApp ONTAP. While ONTAP provides enterprise-grade features, it typically doesn't match the raw performance of FSx for Lustre, especially concerning throughput-intensive HPC workloads. Setting the tiering policy to ALL would further exacerbate the performance due to frequent data movement between tiers.

Option D also uses FSx for NetApp ONTAP but sets the tiering policy to NONE. This would reduce the likelihood of latency issues from tiering, but ONTAP still isn't optimized for the ultra-high throughput and low latency required, making it a less ideal choice compared to Lustre.

Option C uses FSx for Lustre but with persistent HDD storage. HDDs cannot deliver the sub-millisecond latencies demanded by the requirement. Though cheaper, using HDDs would be a performance bottleneck, failing to meet the stated needs.

Therefore, the combination of S3 for storage, FSx for Lustre with persistent SSD storage for performance, and the import/export functionality to transfer data from S3 to FSx for Lustre is the most appropriate and cost-effective solution.

Key Concepts:

Amazon FSx for Lustre: High-performance, parallel file system designed for HPC workloads.
Amazon S3: Object storage service for scalability, data availability, security, and performance.
Persistent SSD vs. HDD: SSDs offer significantly lower latency and higher throughput compared to HDDs.
Import/Export Functionality: Data can be easily moved between S3 and FSx for Lustre.

Authoritative Links:

Amazon FSx for Lustre: https://aws.amazon.com/fsx/lustre/
Amazon S3: https://aws.amazon.com/s3/
Amazon FSx for NetApp ONTAP: https://aws.amazon.com/fsx/netapp-ontap/

Question: 300 **Exam Heist**

A company needs to migrate a legacy application from an on-premises data center to the AWS Cloud because of hardware capacity constraints. The application runs 24 hours a day, 7 days a week. The application's database storage continues to grow over time.

What should a solutions architect do to meet these requirements MOST cost-effectively?

A. Migrate the application layer to Amazon EC2 Spot Instances. Migrate the data storage layer to Amazon S3.

B. Migrate the application layer to Amazon EC2 Reserved Instances. Migrate the data storage layer to Amazon RDS On-Demand Instances.

C. Migrate the application layer to Amazon EC2 Reserved Instances. Migrate the data storage layer to Amazon Aurora Reserved Instances.

D. Migrate the application layer to Amazon EC2 On-Demand Instances. Migrate the data storage layer to Amazon RDS Reserved Instances.

Answer: C

Explanation:

Here's a detailed justification for why option C is the most cost-effective solution, along with supporting concepts and links:

Justification for Option C:

Option C suggests migrating the application layer to Amazon EC2 Reserved Instances and the data storage layer to Amazon Aurora Reserved Instances. This approach provides the best balance of cost efficiency and performance for a 24/7 application with growing data storage needs.

Application Layer (EC2 Reserved Instances): The application runs constantly (24/7), making Reserved Instances (RIs) the most cost-effective choice for the application layer. RIs offer a significant discount (up to 75%) compared to On-Demand Instances in exchange for a one- or three-year commitment. Since the application is always running, the commitment ensures consistent usage and cost savings. Spot Instances are unsuitable for 24/7 operations due to their potential for interruption.

Data Storage Layer (Amazon Aurora Reserved Instances): Amazon Aurora is a fully managed, MySQL- and PostgreSQL-compatible relational database engine. Given the application's growing database storage needs, Aurora is a suitable choice. The use of Reserved Instances for Aurora instances aligns with the constant operation of the application and offers similar cost benefits to EC2 RIs. Aurora also provides built-in scalability to handle the increasing data volume. RDS On-Demand Instances, while simpler, are less cost-effective for long-term, continuous use than reserved instances.

Why Other Options Are Less Suitable:

Option A (EC2 Spot, S3): Spot Instances are unreliable for 24/7 applications due to potential interruptions.

Amazon S3 is object storage; it's not a relational database and unsuitable for storing transactional application data.

Option B (EC2 Reserved, RDS On-Demand): While EC2 Reserved Instances are a good choice for the application layer, RDS On-Demand Instances are less cost-effective for a database that needs to run constantly compared to reserved instances.

Option D (EC2 On-Demand, RDS Reserved): This reverses the cost optimization. On-Demand instances are expensive for 24/7 application runtime, while reserved instances should be applied to the application.

Key Concepts:

Amazon EC2 Instance Purchasing Options: Understanding the trade-offs between On-Demand, Reserved, and Spot Instances is crucial for cost optimization.
Amazon Aurora: A fully managed, high-performance relational database service. It's a good choice for applications with growing database needs.
Cost Optimization in the Cloud: Choosing the appropriate AWS services and purchasing options to minimize expenses while meeting performance and availability requirements.
Relational vs. Object storage: Understanding when to use RDS services like Aurora for structured data rather than object storage like S3.

Authoritative Links:

Amazon EC2 Instance Purchasing Options: https://aws.amazon.com/ec2/purchasing-options/
Amazon Aurora Pricing: https://aws.amazon.com/rds/aurora/pricing/
AWS Well-Architected Framework - Cost Optimization Pillar:
https://wa.aws.amazon.com/wellarchitected/2020-07-02T19-33-23/cost-optimization/co-introduction.en.html

In summary, option C provides the best balance between cost savings and the operational requirements of the application by leveraging reserved instances for both the compute and database layers. This solution addresses the continuous operation and growing data needs in a cost-effective manner.

Question: 301 **Exam Heist**

A university research laboratory needs to migrate 30 TB of data from an on-premises Windows file server to Amazon FSx for Windows File Server. The laboratory has a 1 Gbps network link that many other departments in the university share.

The laboratory wants to implement a data migration service that will maximize the performance of the data transfer. However, the laboratory needs to be able to control the amount of bandwidth that the service uses to minimize the impact on other departments. The data migration must take place within the next 5 days.

Which AWS solution will meet these requirements?

A. AWS Snowcone
B. Amazon FSx File Gateway
C. AWS DataSync
D. AWS Transfer Family

Answer: C

Explanation:

The correct answer is **C. AWS DataSync**.

Here's a detailed justification:

AWS DataSync is a data transfer service specifically designed for moving large amounts of data between on-premises storage and AWS services like Amazon FSx for Windows File Server. It excels in scenarios where you need to migrate data quickly and efficiently over a network connection. A key feature of DataSync that makes it perfect for this scenario is its built-in bandwidth throttling capability. This allows the laboratory to control the amount of bandwidth DataSync utilizes, minimizing the impact on other university departments sharing the 1 Gbps network link. DataSync also optimizes data transfer using techniques such as incremental transfers (transferring only changed data after the initial copy), in-line compression, and parallel data streams, maximizing throughput and making the migration within the 5-day timeframe feasible, given appropriate bandwidth settings. It also offers encryption in transit and at rest, which is relevant for sensitive research data.

AWS Snowcone (A) is a physical device used for data transport, primarily useful when network bandwidth is limited or unavailable. While it could theoretically transfer the data, it involves shipping the device to AWS, which introduces delays and doesn't allow for bandwidth control.Amazon FSx File Gateway (B) is a service that provides low-latency access to FSx file systems from on-premises applications. It's not designed for the initial migration of large datasets. It facilitates hybrid access to files already residing in FSx.AWS Transfer Family (D) is a suite of services for secure file transfers into and out of Amazon S3, Amazon EFS, and AWS Storage Gateway using protocols such as SFTP, FTPS, and FTP. It doesn't directly support transferring data to FSx for Windows File Server or offer the same level of optimization and bandwidth control as DataSync for this specific migration use case.

In summary, AWS DataSync's optimized data transfer, bandwidth throttling, and direct integration with Amazon FSx for Windows File Server make it the most appropriate and efficient solution for the university's data migration requirements.

Relevant links for further research:

AWS DataSync: https://aws.amazon.com/datasync/
Amazon FSx for Windows File Server: https://aws.amazon.com/fsx/windows/

Question: 302 **Exam Heist**

A company wants to create a mobile app that allows users to stream slow-motion video clips on their mobile devices. Currently, the app captures video clips and uploads the video clips in raw format into an Amazon S3 bucket. The app retrieves these video clips directly from the S3 bucket. However, the videos are large in their raw format.

Users are experiencing issues with buffering and playback on mobile devices. The company wants to implement solutions to maximize the performance and scalability of the app while minimizing operational overhead.

Which combination of solutions will meet these requirements? (Choose two.)

A. Deploy Amazon CloudFront for content delivery and caching.
B. Use AWS DataSync to replicate the video files across AW'S Regions in other S3 buckets.
C. Use Amazon Elastic Transcoder to convert the video files to more appropriate formats.
D. Deploy an Auto Sealing group of Amazon EC2 instances in Local Zones for content delivery and caching.
E. Deploy an Auto Scaling group of Amazon EC2 instances to convert the video files to more appropriate formats.

Answer: AC

Explanation:

The correct answer is **A and C**. Here's why:

A. Deploy Amazon CloudFront for content delivery and caching:

CloudFront is a Content Delivery Network (CDN) that caches content closer to the end users. By deploying CloudFront in front of the S3 bucket containing the raw video files, you significantly reduce latency and improve the streaming experience. CloudFront has edge locations distributed globally, ensuring users can retrieve video clips from a location geographically closer to them. This minimizes buffering and playback issues, fulfilling the requirement to maximize performance and scalability. CloudFront integrates seamlessly with S3 and provides features like geo-restriction, custom SSL certificates, and access logs.https://aws.amazon.com/cloudfront/

C. Use Amazon Elastic Transcoder to convert the video files to more appropriate formats:

The problem statement highlights that videos are in raw format, which are large and cause playback issues. Elastic Transcoder converts video files from their raw format into more suitable formats for streaming over the internet to mobile devices (e.g., H.264). This involves encoding the videos into smaller file sizes and resolutions optimized for mobile viewing. It can also create adaptive bitrate streaming formats (like HLS or DASH) to adjust the video quality based on the user's network conditions, further improving playback. This addresses the buffering problem directly by reducing the file sizes. Elastic Transcoder is a fully managed service, minimizing operational overhead.https://aws.amazon.com/elastictranscoder/

Why other options are incorrect:

B. Use AWS DataSync to replicate the video files across AWS Regions in other S3 buckets: While replication improves availability, it doesn't directly address the issue of large video file sizes causing buffering on mobile devices. It also increases storage costs.
D. Deploy an Auto Scaling group of Amazon EC2 instances in Local Zones for content delivery and caching: This option is less efficient and more complex than using CloudFront. It requires managing EC2 instances and caching software, significantly increasing operational overhead. CloudFront is a purpose-built CDN service that handles caching automatically.
E. Deploy an Auto Scaling group of Amazon EC2 instances to convert the video files to more appropriate formats: While using EC2 instances for transcoding is possible, it is not recommended for the same reasons listed for option D. AWS Elastic Transcoder is a managed service that is already configured for this purpose and scales automatically. Using EC2 for transcoding would require more resources and higher overhead.

Question: 303 **Exam Heist**

A company is launching a new application deployed on an Amazon Elastic Container Service (Amazon ECS) cluster and is using the Fargate launch type for ECS tasks. The company is monitoring CPU and memory usage because it is expecting high traffic to the application upon its launch. However, the company wants to reduce costs when utilization decreases.

What should a solutions architect recommend?

A. Use Amazon EC2 Auto Scaling to scale at certain periods based on previous traffic patterns.

B. Use an AWS Lambda function to scale Amazon ECS based on metric breaches that trigger an Amazon CloudWatch alarm.

C. Use Amazon EC2 Auto Scaling with simple scaling policies to scale when ECS metric breaches trigger an Amazon CloudWatch alarm.

D. Use AWS Application Auto Scaling with target tracking policies to scale when ECS metric breaches trigger an Amazon CloudWatch alarm.

Answer: D

Explanation:

The correct answer is D. Here's why:

A. Incorrect: Amazon EC2 Auto Scaling is designed to scale EC2 instances, not ECS tasks using Fargate.

Fargate is a serverless compute engine for containers; you don't manage EC2 instances directly.

B. Incorrect: While a Lambda function could theoretically be used to adjust ECS task counts, it's a less efficient and more complex solution than using Application Auto Scaling. Lambda would require custom code for monitoring metrics and scaling actions, adding overhead and potential points of failure.

C. Incorrect: As with option A, Amazon EC2 Auto Scaling is not applicable to Fargate tasks. ECS tasks running on Fargate do not directly involve EC2 instances, making EC2 Auto Scaling irrelevant.

D. Correct: AWS Application Auto Scaling is the preferred solution for scaling ECS tasks using the Fargate launch type. It allows you to automatically adjust the desired count of tasks within your ECS service based on various metrics (like CPU and memory utilization). Target tracking policies are specifically designed to maintain a desired target value for a metric (e.g., keep CPU utilization at 70%). When CloudWatch alarms are triggered because the metric exceeds the target, Application Auto Scaling automatically increases the number of ECS tasks to handle the load. When utilization drops, it reduces the number of tasks, optimizing costs.

Justification in Detail:

1. **Fargate and Serverless:** Fargate abstracts away the underlying EC2 infrastructure, making EC2 Auto Scaling irrelevant. Fargate provides serverless compute for containers.
2. **Application Auto Scaling for ECS:** AWS Application Auto Scaling integrates directly with ECS services and Fargate tasks. https://docs.aws.amazon.com/autoscaling/application/userguide/what-is-application-auto-scaling.html
3. **Target Tracking Policies:** Target tracking policies simplify scaling by automatically adjusting the service's desired count to maintain a target value for a specific metric, such as average CPU or memory utilization. https://docs.aws.amazon.com/autoscaling/application/userguide/application-auto-scaling-target-tracking.html
4. **CloudWatch Alarms as Triggers:** CloudWatch alarms monitor the specified metrics (CPU, memory) and trigger the Application Auto Scaling policies when predefined thresholds are breached.
5. **Cost Optimization:** By scaling down task counts when utilization is low, Application Auto Scaling ensures you only pay for the resources you need, minimizing costs.
6. **Simplified Management:** Application Auto Scaling eliminates the need for custom scaling logic (as with Lambda), reducing operational overhead and complexity.
7. **Responsiveness:** Target tracking allows the system to quickly adapt to changes in demand, increasing the number of ECS tasks when utilization increases and decreasing the number of tasks when utilization decreases.

In summary, AWS Application Auto Scaling with target tracking policies offers a managed, efficient, and cost-effective way to dynamically scale ECS tasks running on Fargate, directly responding to changes in application utilization. The CloudWatch alarms serve as the trigger to initiate the scaling actions.

Question: 304 **Exam Heist**

A company recently created a disaster recovery site in a different AWS Region. The company needs to transfer large amounts of data back and forth between NFS file systems in the two Regions on a periodic basis.

Which solution will meet these requirements with the LEAST operational overhead?

A. Use AWS DataSync.

B. Use AWS Snowball devices.

C. Set up an SFTP server on Amazon EC2.

D. Use AWS Database Migration Service (AWS DMS).

Answer: A

Explanation:

The correct answer is **A. Use AWS DataSync.**

Here's a detailed justification:

AWS DataSync is a purpose-built data transfer service designed for moving large amounts of data between on-premises storage and AWS, or between AWS storage services. It simplifies, automates, and accelerates data transfer over the network. In this scenario, where data needs to be moved between NFS file systems in two different AWS Regions on a periodic basis, DataSync is an ideal solution because it offers significant advantages in terms of speed, reliability, and operational overhead.

DataSync optimizes network usage through built-in acceleration techniques and parallel data transfer, which is essential for large datasets. It also offers built-in security features like encryption and data integrity verification. Critically, it simplifies the transfer process, automating scheduling, monitoring, and error handling, which translates to minimal operational effort compared to other solutions.

Option B (AWS Snowball devices) is more suited for initial bulk data migrations, not for periodic transfers between regions. Transporting physical devices back and forth introduces logistical complexity and is not efficient for recurring data transfers.

Option C (Setting up an SFTP server on Amazon EC2) would require significant manual configuration and management of the server, security settings, and transfer processes. This dramatically increases the operational overhead and introduces potential vulnerabilities. It's also not optimized for high-speed data transfer.

Option D (AWS Database Migration Service) is designed for migrating databases, not file systems. Therefore, it is completely inappropriate for this use case involving NFS file systems.

Therefore, AWS DataSync provides the least operational overhead for periodic, large-scale data transfers between NFS file systems in different AWS Regions because it is specifically designed for this purpose with built-in automation, optimization, and security features.

Further research:

AWS DataSync: https://aws.amazon.com/datasync/

Question: 305 **Exam Heist**

A company is designing a shared storage solution for a gaming application that is hosted in the AWS Cloud. The company needs the ability to use SMB clients to access data. The solution must be fully managed.

Which AWS solution meets these requirements?

A. Create an AWS DataSync task that shares the data as a mountable file system. Mount the file system to the application server.

B. Create an Amazon EC2 Windows instance. Install and configure a Windows file share role on the instance. Connect the application server to the file share.

C. Create an Amazon FSx for Windows File Server file system. Attach the file system to the origin server. Connect the application server to the file system.

D. Create an Amazon S3 bucket. Assign an IAM role to the application to grant access to the S3 bucket. Mount the S3 bucket to the application server.

Answer: C

Explanation:

The correct answer is C, creating an Amazon FSx for Windows File Server file system. Here's why:

Requirement: SMB Access: The gaming application needs to be accessed via SMB (Server Message Block) protocol. Amazon FSx for Windows File Server natively supports SMB, allowing Windows-based applications and clients to interact seamlessly with the file system using standard Windows file sharing protocols.

Requirement: Fully Managed: The solution needs to be fully managed. FSx for Windows File Server is a fully managed service, meaning AWS handles the underlying infrastructure, patching, backups, and maintenance. This significantly reduces the operational overhead for the company.

Why other options are incorrect:

A (AWS DataSync): AWS DataSync is primarily used for data transfer between on-premises storage and AWS storage services, and it does not provide a direct SMB file share endpoint. It's more suitable for migration or backup scenarios, not for ongoing shared storage access.
B (EC2 Windows Instance): While an EC2 Windows instance can be configured as a file server, this is not a fully managed solution. The company would be responsible for managing the OS, patching, backups, and ensuring high availability. This introduces significant operational overhead, contradicting the requirement.
D (Amazon S3): Amazon S3 is an object storage service, not a file system. While S3 can be "mounted" using tools, it doesn't natively support SMB. Accessing it through SMB would require additional layers of software and complexity, and the object storage model isn't designed for the typical file system operations expected by applications using SMB. S3 is also not designed for low-latency file access often needed by applications.

In summary, Amazon FSx for Windows File Server directly fulfills the requirements of providing a fully managed SMB file share solution for the gaming application, aligning with the service's core functionality and intended use case.

Authoritative Links:

Amazon FSx for Windows File Server: https://aws.amazon.com/fsx/windows/

Question: 306 **Exam Heist**

A company wants to run an in-memory database for a latency-sensitive application that runs on Amazon EC2 instances. The application processes more than 100,000 transactions each minute and requires high network throughput. A solutions architect needs to provide a cost-effective network design that minimizes data transfer charges.

Which solution meets these requirements?

A. Launch all EC2 instances in the same Availability Zone within the same AWS Region. Specify a placement group with cluster strategy when launching EC2 instances.

B. Launch all EC2 instances in different Availability Zones within the same AWS Region. Specify a placement group with partition strategy when launching EC2 instances.

C. Deploy an Auto Scaling group to launch EC2 instances in different Availability Zones based on a network utilization target.

D. Deploy an Auto Scaling group with a step scaling policy to launch EC2 instances in different Availability Zones.

Answer: A

Explanation:

The correct answer is A because it prioritizes low latency and cost-effectiveness for a high-throughput, in-memory database within AWS. Let's break down why:

Placement Groups: Placement groups influence how EC2 instances are placed on underlying hardware. A cluster placement group strategy aims to place instances within a single Availability Zone as closely together as possible. This reduces latency and increases network throughput by minimizing the physical distance between instances.

Availability Zones and Latency: Launching instances within the same Availability Zone minimizes latency compared to spreading them across different Availability Zones. Data transfer between Availability Zones incurs costs, which should be avoided in a cost-sensitive scenario with high data transfer requirements.

Cost-Effectiveness: Data transfer between Availability Zones is charged, whereas data transfer within an Availability Zone is free. Keeping instances within the same Availability Zone eliminates these cross-AZ data transfer charges.

Auto Scaling (Option C and D - incorrect): While Auto Scaling is beneficial for scaling resources based on demand, it's not essential for minimizing latency within a tightly coupled, in-memory database application where consistent, low-latency communication is paramount. Auto Scaling adds complexity without directly addressing the latency and data transfer cost concerns in this specific scenario. Launching across multiple AZs is not beneficial for latency or cost optimization given the constraints.

Partition Strategy (Option B - incorrect): Partition placement groups are suitable for distributing large numbers of replicas across distinct racks to minimize correlated hardware failures. They are not optimized for low-latency communication. Moreover, launching across multiple AZs will incur costs due to data transfer.

Therefore, choosing a cluster placement group within a single Availability Zone provides the best solution for minimizing latency, maximizing network throughput, and reducing data transfer costs for the given application requirements.

Further Research:

AWS Placement Groups: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/placement-groups.html
AWS Data Transfer Costs: https://aws.amazon.com/ec2/pricing/on-demand/
AWS Availability Zones: https://aws.amazon.com/about-aws/global-infrastructure/regions_availability-zones/

Question: 307 **Exam Heist**

A company that primarily runs its application servers on premises has decided to migrate to AWS. The company wants to minimize its need to scale its Internet Small Computer Systems Interface (iSCSI) storage on premises. The company wants only its recently accessed data to remain stored locally.

Which AWS solution should the company use to meet these requirements?

A. Amazon S3 File Gateway
B. AWS Storage Gateway Tape Gateway
C. AWS Storage Gateway Volume Gateway stored volumes
D. AWS Storage Gateway Volume Gateway cached volumes

Answer: D

Explanation:

The company needs a solution that minimizes on-premises iSCSI storage while keeping recently accessed data local. AWS Storage Gateway offers different modes, and understanding these is crucial.

Option A, Amazon S3 File Gateway, stores data as objects directly in S3. While it reduces on-premises storage, it doesn't cache frequently accessed data locally for low latency access.

Option B, AWS Storage Gateway Tape Gateway, is designed for virtual tape storage and is not relevant for general application server data requiring low latency.

Option C, AWS Storage Gateway Volume Gateway (stored volumes), stores the entire dataset on-premises and asynchronously backs it up to AWS. This contradicts the requirement to minimize on-premises storage and keep only recently accessed data locally.

Option D, AWS Storage Gateway Volume Gateway (cached volumes), addresses the requirements directly. Cached volumes store the entire dataset in S3 and cache only the frequently accessed data on-premises. This reduces the on-premises storage footprint, keeps recently accessed data local for fast access, and provides durable backup in S3. The "minimize its need to scale its iSCSI storage on premises" and "only its recently accessed data to remain stored locally" requirements are perfectly met by this configuration. It strikes a balance between on-premises performance and cloud storage scalability and cost-effectiveness. Therefore, option D is the most suitable.

Further reading:

AWS Storage Gateway Documentation
Understanding Volume Gateway Types

Question: 308 **Exam Heist**

A company has multiple AWS accounts that use consolidated billing. The company runs several active high performance Amazon RDS for Oracle On-Demand DB instances for 90 days. The company's finance team has access to AWS Trusted Advisor in the consolidated billing account and all other AWS accounts.

The finance team needs to use the appropriate AWS account to access the Trusted Advisor check recommendations for RDS. The finance team must review the appropriate Trusted Advisor check to reduce RDS costs.

Which combination of steps should the finance team take to meet these requirements? (Choose two.)

A. Use the Trusted Advisor recommendations from the account where the RDS instances are running.

B. Use the Trusted Advisor recommendations from the consolidated billing account to see all RDS instance checks at the same time.

C. Review the Trusted Advisor check for Amazon RDS Reserved Instance Optimization.

D. Review the Trusted Advisor check for Amazon RDS Idle DB Instances.

E. Review the Trusted Advisor check for Amazon Redshift Reserved Node Optimization.

Answer: BD

Explanation:

Here's a detailed justification for why options B and D are the correct choices, and why the others are incorrect:

Option B: Use the Trusted Advisor recommendations from the consolidated billing account to see all RDS instance checks at the same time.

Explanation: With consolidated billing, the payer (consolidated billing account) has visibility into the usage

and cost optimization recommendations for all linked accounts. AWS Trusted Advisor, when accessed from the payer account, provides a single pane of glass to view the recommendations across the entire AWS organization. This allows the finance team to efficiently identify potential cost savings opportunities across all accounts without logging into each one individually. This centralized view is a key benefit of consolidated billing.
Relevance: This directly addresses the requirement to have a comprehensive view for cost reduction across multiple accounts.
Supporting concept: Consolidated Billing within AWS Organizations.
Authoritative link: https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/consolidated-billing.html

Option D: Review the Trusted Advisor check for Amazon RDS Idle DB Instances.

Explanation: Since the company is looking to reduce RDS costs, identifying and addressing idle DB instances is a crucial step. The "Amazon RDS Idle DB Instances" Trusted Advisor check specifically flags instances that have low CPU utilization and no connection activity over a period, indicating they are potentially underutilized and can be stopped or resized to save costs.
Relevance: Addresses the need to reduce RDS costs by targeting instances that are consuming resources without significant activity.
Supporting concept: Cost optimization through resource utilization analysis.
Authoritative link: https://docs.aws.amazon.com/awssupport/latest/user/trusted-advisor.html (Look for RDS checks within the Trusted Advisor documentation)

Why the other options are incorrect:

Option A: Use the Trusted Advisor recommendations from the account where the RDS instances are running. While this would provide recommendations specific to that account, it wouldn't give the finance team a comprehensive view across all accounts as easily as option B. It would require logging into multiple accounts.
Option C: Review the Trusted Advisor check for Amazon RDS Reserved Instance Optimization. While Reserved Instances are important for cost optimization, the instances were On-Demand for 90 days. Therefore, Reserved Instance Optimization wouldn't directly address the cost reduction for existing usage of On-Demand instances. It's a future optimization strategy rather than a solution for current On-Demand usage.
Option E: Review the Trusted Advisor check for Amazon Redshift Reserved Node Optimization. This is irrelevant because the question specifies Amazon RDS for Oracle instances, not Amazon Redshift. This check applies to a different AWS service.

Question: 309 **Exam Heist**

A solutions architect needs to optimize storage costs. The solutions architect must identify any Amazon S3 buckets that are no longer being accessed or are rarely accessed.

Which solution will accomplish this goal with the LEAST operational overhead?

A. Analyze bucket access patterns by using the S3 Storage Lens dashboard for advanced activity metrics.

B. Analyze bucket access patterns by using the S3 dashboard in the AWS Management Console.

C. Turn on the Amazon CloudWatch BucketSizeBytes metric for buckets. Analyze bucket access patterns by using the metrics data with Amazon Athena.

D. Turn on AWS CloudTrail for S3 object monitoring. Analyze bucket access patterns by using CloudTrail logs that are integrated with Amazon CloudWatch Logs.

Answer: A

Explanation:

The correct answer is A because S3 Storage Lens is specifically designed to provide organization-wide visibility into object storage, trends, and generate actionable recommendations to optimize costs and apply data protection best practices. S3 Storage Lens does this with advanced metrics, trends, and visualizations. It provides a central dashboard to analyze data usage and activity trends across your entire S3 estate, identifying buckets that are infrequently accessed or potentially abandoned. This approach requires minimal operational overhead as it is a managed service with built-in capabilities for analyzing access patterns.

Option B is incorrect because the S3 dashboard in the AWS Management Console provides basic storage usage metrics, but it lacks the advanced activity metrics and trend analysis capabilities of S3 Storage Lens. It does not offer the same level of detailed insights into access patterns required for identifying rarely accessed buckets effectively.

Option C is less efficient. While CloudWatch can track bucket size, analyzing access patterns using only bucket size and Athena requires more manual configuration and analysis. It would not directly reveal access frequency like S3 Storage Lens. This involves writing complex queries in Athena to correlate bucket size changes with other potential access logs, resulting in more operational overhead.

Option D involves high operational overhead. While CloudTrail captures API activity, enabling it for S3 object monitoring generates a large volume of logs. Integrating these logs with CloudWatch Logs and then analyzing them to determine access patterns is a complex and resource-intensive task, making it a less efficient solution for cost optimization.

Therefore, S3 Storage Lens directly addresses the requirement with the least operational overhead by providing comprehensive analysis of S3 usage and activity, making it easy to identify infrequently accessed buckets.

Relevant Link:

Analyzing storage usage with S3 Storage Lens

Question: 310 **Exam Heist**

A company sells datasets to customers who do research in artificial intelligence and machine learning (AI/ML). The datasets are large, formatted files that are stored in an Amazon S3 bucket in the us-east-1 Region. The company hosts a web application that the customers use to purchase access to a given dataset. The web application is deployed on multiple Amazon EC2 instances behind an Application Load Balancer. After a purchase is made, customers receive an S3 signed URL that allows access to the files.

The customers are distributed across North America and Europe. The company wants to reduce the cost that is associated with data transfers and wants to maintain or improve performance.

What should a solutions architect do to meet these requirements?

A. Configure S3 Transfer Acceleration on the existing S3 bucket. Direct customer requests to the S3 Transfer Acceleration endpoint. Continue to use S3 signed URLs for access control.

B. Deploy an Amazon CloudFront distribution with the existing S3 bucket as the origin. Direct customer requests to the CloudFront URL. Switch to CloudFront signed URLs for access control.

C. Set up a second S3 bucket in the eu-central-1 Region with S3 Cross-Region Replication between the buckets. Direct customer requests to the closest Region. Continue to use S3 signed URLs for access control.

D. Modify the web application to enable streaming of the datasets to end users. Configure the web application to read the data from the existing S3 bucket. Implement access control directly in the application.

Answer: B

Explanation:

The optimal solution to reduce data transfer costs and improve performance for customers across North

America and Europe accessing datasets in an S3 bucket is to use Amazon CloudFront.

Option B, deploying CloudFront with the existing S3 bucket as the origin and using CloudFront signed URLs for access control, addresses both the cost and performance concerns. CloudFront is a content delivery network (CDN) that caches data at edge locations closer to the users. This reduces latency and improves download speeds, thus enhancing performance for customers in both North America and Europe.

Furthermore, CloudFront data transfer costs are generally lower than direct S3 data transfer costs, especially for geographically dispersed users. By caching the datasets closer to users, CloudFront significantly reduces the amount of data transferred directly from the S3 bucket, which in turn lowers costs.

Switching to CloudFront signed URLs is crucial for maintaining security. These URLs control access to the content served through CloudFront, ensuring that only authorized users (those who have purchased access) can download the datasets. CloudFront signed URLs are also a more secure option in this scenario, allowing for features like key rotation and fine-grained access control based on date, time or IP address.

Option A, using S3 Transfer Acceleration, focuses primarily on accelerating uploads to S3, which is not the primary concern here. The key focus is on optimizing downloads for the customers.

Option C, using S3 Cross-Region Replication, would incur higher storage costs due to the duplicated data. While it improves latency for European users, it's a less efficient and more expensive solution compared to CloudFront. Also, managing separate buckets and ensuring data consistency can be more complex.

Option D, streaming from the web application, would add significant overhead to the application instances. It requires more complex application logic and processing, and will likely not be as efficient as using a CDN to distribute the data. Also, implementing access control in the application itself would make it complex and may not be highly scalable.

Therefore, CloudFront with signed URLs offers the best balance of cost reduction, performance improvement, and security for the company's dataset distribution needs.

Supporting links:

Amazon CloudFront: https://aws.amazon.com/cloudfront/
CloudFront Signed URLs: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-signed-urls.html

Question: 311 **Exam Heist**

A company is using AWS to design a web application that will process insurance quotes. Users will request quotes from the application. Quotes must be separated by quote type, must be responded to within 24 hours, and must not get lost. The solution must maximize operational efficiency and must minimize maintenance.

Which solution meets these requirements?

A.Create multiple Amazon Kinesis data streams based on the quote type. Configure the web application to send messages to the proper data stream. Configure each backend group of application servers to use the Kinesis Client Library (KCL) to pool messages from its own data stream.

B.Create an AWS Lambda function and an Amazon Simple Notification Service (Amazon SNS) topic for each quote type. Subscribe the Lambda function to its associated SNS topic. Configure the application to publish requests for quotes to the appropriate SNS topic.

C.Create a single Amazon Simple Notification Service (Amazon SNS) topic. Subscribe Amazon Simple Queue Service (Amazon SQS) queues to the SNS topic. Configure SNS message filtering to publish messages to the proper SQS queue based on the quote type. Configure each backend application server to use its own SQS queue.

D.Create multiple Amazon Kinesis Data Firehose delivery streams based on the quote type to deliver data streams to an Amazon OpenSearch Service cluster. Configure the application to send messages to the proper

delivery stream. Configure each backend group of application servers to search for the messages from OpenSearch Service and process them accordingly.

Answer: C

Explanation:

The correct answer is **C**. Here's a detailed justification:

Option C leverages the strengths of Amazon SNS and SQS to build a reliable and scalable message queuing system that effectively addresses the requirements. The application publishes quote requests to a single SNS topic. SNS then fans out the messages to multiple SQS queues based on message filtering. Each SQS queue is associated with a specific quote type, and the backend application servers consume messages only from their corresponding queue. This ensures that quotes are separated by type and processed by the appropriate servers. SQS offers message durability and guarantees that messages will not be lost.https://aws.amazon.com/sns/https://aws.amazon.com/sqs/

The 24-hour processing requirement can be met using SQS's message visibility timeout and dead-letter queues. If a message is not processed within the timeout, it returns to the queue, and after a configured number of retries, it can be sent to a dead-letter queue for further investigation, ensuring no quotes are lost. This combination maximizes operational efficiency because SNS/SQS are fully managed services, minimizing maintenance overhead. The filtering mechanism within SNS directs traffic efficiently.

Option A, using Kinesis Data Streams, is more suitable for real-time streaming analytics rather than asynchronous task processing like insurance quote generation. It also requires more configuration and management via the KCL. Option B, using separate SNS topics and Lambda functions for each quote type, creates a large number of resources and adds complexity. Managing many Lambda functions can be operationally inefficient. Option D, using Kinesis Data Firehose and OpenSearch Service, is better suited for log analytics or large-scale data warehousing. Searching OpenSearch service continuously and processing messages isn't the best use case compared to SQS queues and is overkill for processing messages that need to be processed in near-real-time.

Question: 312 **Exam Heist**

A company has an application that runs on several Amazon EC2 instances. Each EC2 instance has multiple Amazon Elastic Block Store (Amazon EBS) data volumes attached to it. The application's EC2 instance configuration and data need to be backed up nightly. The application also needs to be recoverable in a different AWS Region.

Which solution will meet these requirements in the MOST operationally efficient way?

A. Write an AWS Lambda function that schedules nightly snapshots of the application's EBS volumes and copies the snapshots to a different Region.

B. Create a backup plan by using AWS Backup to perform nightly backups. Copy the backups to another Region. Add the application's EC2 instances as resources.

C. Create a backup plan by using AWS Backup to perform nightly backups. Copy the backups to another Region. Add the application's EBS volumes as resources.

D. Write an AWS Lambda function that schedules nightly snapshots of the application's EBS volumes and copies the snapshots to a different Availability Zone.

Answer: B

Explanation:

The most operationally efficient solution is **B. Create a backup plan by using AWS Backup to perform nightly backups. Copy the backups to another Region. Add the application's EC2 instances as resources.**

Here's why:

AWS Backup's Orchestration: AWS Backup is designed to centralize and automate backup and restore tasks across AWS services, including EC2 and EBS. It provides a managed service for creating backup plans, schedules, and retention policies, simplifying the backup process.
EC2-Level Backup: By adding the EC2 instances as resources to the backup plan, AWS Backup will automatically discover and back up all attached EBS volumes as part of the instance backup. This ensures consistency and simplifies management compared to managing EBS volume snapshots individually.
Cross-Region Copying: AWS Backup natively supports copying backups to another AWS Region as part of the backup plan. This addresses the disaster recovery requirement.
Operational Efficiency: Using AWS Backup significantly reduces the operational overhead compared to writing and maintaining a custom Lambda function. AWS Backup handles scheduling, orchestration, and compliance aspects, allowing administrators to focus on other tasks.
Recovery Simplicity: Backing up the entire EC2 instance simplifies recovery. The instance can be restored in another region including the instance configuration and the data on the associated EBS volumes.
Automated Discovery: AWS Backup automates the discovery of EBS volumes attached to the EC2 instances, removing the need to manually manage EBS volume identifiers within a Lambda function.

Why other options are less optimal:

A & D: Using Lambda functions for snapshot management is more complex and requires more operational overhead to maintain and troubleshoot. While feasible, it lacks the built-in orchestration and centralized management benefits of AWS Backup. D is incorrect because copying to a different Availability Zone doesn't fulfill the cross-region recovery requirement.
C: While C uses AWS Backup, adding only the EBS volumes as resources will only backup the data and would not include the important instance configuration.

Supporting Links:

AWS Backup: https://aws.amazon.com/backup/
AWS Backup Documentation: https://docs.aws.amazon.com/aws-backup/latest/devguide/whatisbackup.html

Question: 313 **Exam Heist**

A company is building a mobile app on AWS. The company wants to expand its reach to millions of users. The company needs to build a platform so that authorized users can watch the company's content on their mobile devices.

What should a solutions architect recommend to meet these requirements?

A. Publish content to a public Amazon S3 bucket. Use AWS Key Management Service (AWS KMS) keys to stream content.
B. Set up IPsec VPN between the mobile app and the AWS environment to stream content.
C. Use Amazon CloudFront. Provide signed URLs to stream content.
D. Set up AWS Client VPN between the mobile app and the AWS environment to stream content.

Answer: C

Explanation:

The correct answer is C: Use Amazon CloudFront. Provide signed URLs to stream content.

Here's a detailed justification:

To deliver content to millions of users securely and efficiently, a Content Delivery Network (CDN) is essential.

Amazon CloudFront is AWS's CDN service, designed to distribute content globally with low latency and high transfer speeds. Option C leverages CloudFront's capabilities to cache content at edge locations closer to users, significantly improving the viewing experience by reducing latency. Signed URLs provide a secure mechanism for authorizing access to content. They grant time-limited access to specific resources for authorized users. This approach allows the company to control who can view the content and for how long, preventing unauthorized access. The company controls distribution through the signing process, ensuring only authorized users can watch the mobile app content.

Option A is incorrect because storing content in a public S3 bucket without a CDN does not address scalability or performance for millions of users. Also, KMS keys are for encrypting data at rest, not for streaming authorization.

Option B and D are incorrect because setting up VPNs (IPsec or Client VPN) for millions of mobile users is not a scalable or practical solution. VPNs are designed for secure network connections between defined points, not for content delivery to a vast user base. They will also cause performance issues because you're essentially routing all traffic from the mobile app to the AWS environment through the VPN, creating a bottleneck.Therefore, using CloudFront with signed URLs offers the best balance of scalability, performance, and security for streaming content to millions of users.

Relevant Links:

Amazon CloudFront: https://aws.amazon.com/cloudfront/
CloudFront Signed URLs and Signed Cookies:
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/PrivateContent.html

Question: 314 **Exam Heist**

A company has an on-premises MySQL database used by the global sales team with infrequent access patterns. The sales team requires the database to have minimal downtime. A database administrator wants to migrate this database to AWS without selecting a particular instance type in anticipation of more users in the future.

Which service should a solutions architect recommend?

A.Amazon Aurora MySQL
B.Amazon Aurora Serverless for MySQL
C.Amazon Redshift Spectrum
D.Amazon RDS for MySQL

Answer: B

Explanation:

The recommended service is Amazon Aurora Serverless for MySQL because it directly addresses the scenario's key requirements: minimal downtime, infrequent access patterns, and avoidance of pre-defined instance types due to uncertain future capacity needs.

Amazon Aurora Serverless provides automatic scaling, starting, and stopping of the database cluster based on application needs. This eliminates the need for manual capacity provisioning or selection of a specific instance type upfront. Its "pay-per-use" model aligns perfectly with the infrequent access patterns, optimizing costs when the database is not actively in use. Aurora Serverless automatically starts up when a connection is requested and scales compute and memory capacity to handle the workload.

Aurora MySQL (Option A) would require selecting a specific instance type, which contradicts the requirement to avoid doing so in anticipation of future needs. While RDS for MySQL (Option D) offers a managed MySQL service, it also requires specifying an instance type and doesn't provide the automatic scaling and cost

optimization benefits of Aurora Serverless for infrequent workloads. Amazon Redshift Spectrum (Option C) is a data warehousing solution optimized for analytics, not a direct replacement for a transactional MySQL database.

Aurora Serverless provides high availability by storing data across multiple Availability Zones. Downtime is minimized during scaling operations, ensuring uninterrupted service to the global sales team. The automatic scaling capabilities ensure the database can easily handle more users in the future without manual intervention or the need for instance type upgrades. This meets the requirement of supporting potential growth and preventing operational overhead.

In summary, Aurora Serverless offers the ideal balance of minimal downtime, automatic scaling, cost optimization for infrequent access, and avoidance of pre-defined instance types, making it the best solution for this migration scenario.

Relevant links:

Amazon Aurora Serverless: https://aws.amazon.com/rds/aurora/serverless/
Amazon RDS: https://aws.amazon.com/rds/

Question: 315 **Exam Heist**

A company experienced a breach that affected several applications in its on-premises data center. The attacker took advantage of vulnerabilities in the custom applications that were running on the servers. The company is now migrating its applications to run on Amazon EC2 instances. The company wants to implement a solution that actively scans for vulnerabilities on the EC2 instances and sends a report that details the findings.

Which solution will meet these requirements?

A. Deploy AWS Shield to scan the EC2 instances for vulnerabilities. Create an AWS Lambda function to log any findings to AWS CloudTrail.

B. Deploy Amazon Macie and AWS Lambda functions to scan the EC2 instances for vulnerabilities. Log any findings to AWS CloudTrail.

C. Turn on Amazon GuardDuty. Deploy the GuardDuty agents to the EC2 instances. Configure an AWS Lambda function to automate the generation and distribution of reports that detail the findings.

D. Turn on Amazon Inspector. Deploy the Amazon Inspector agent to the EC2 instances. Configure an AWS Lambda function to automate the generation and distribution of reports that detail the findings.

Answer: D

Explanation:

The correct answer is D because it leverages Amazon Inspector, a service specifically designed for vulnerability management in AWS environments. Amazon Inspector automatically assesses EC2 instances for security vulnerabilities and deviations from security best practices. Deploying the Inspector agent on the EC2 instances allows it to scan the operating system and applications for known vulnerabilities. The service then generates findings that detail the discovered issues.

Option D also includes an AWS Lambda function to automate report generation and distribution. This addresses the requirement to send a report detailing the findings.

Let's examine why the other options are incorrect:

A: AWS Shield is a DDoS protection service and does not scan EC2 instances for vulnerabilities in the way that Inspector does. While CloudTrail logs API calls, it does not directly handle vulnerability scanning results from Shield.
B: Amazon Macie is designed to discover and protect sensitive data in Amazon S3 buckets. It is not the

appropriate service for scanning EC2 instances for vulnerabilities.
C: Amazon GuardDuty is a threat detection service that analyzes logs and network activity for malicious or unauthorized behavior. While GuardDuty provides valuable security insights, it's not specifically designed for active vulnerability scanning of EC2 instances at the OS and application level. Deploying agents for GuardDuty onto EC2 instances is not standard practice.

Therefore, Amazon Inspector provides the most comprehensive solution for active vulnerability scanning of EC2 instances and reporting the findings, making option D the best choice.

Authoritative Links:

Amazon Inspector: https://aws.amazon.com/inspector/
AWS Lambda: https://aws.amazon.com/lambda/

Question: 316 **Exam Heist**

A company uses an Amazon EC2 instance to run a script to poll for and process messages in an Amazon Simple Queue Service (Amazon SQS) queue. The company wants to reduce operational costs while maintaining its ability to process a growing number of messages that are added to the queue.

What should a solutions architect recommend to meet these requirements?

A. Increase the size of the EC2 instance to process messages faster.
B. Use Amazon EventBridge to turn off the EC2 instance when the instance is underutilized.
C. Migrate the script on the EC2 instance to an AWS Lambda function with the appropriate runtime.
D. Use AWS Systems Manager Run Command to run the script on demand.

Answer: C

Explanation:

The correct answer is C, migrating the script to an AWS Lambda function. Here's why:

The primary goal is to reduce operational costs while handling a potentially growing number of messages in the SQS queue. The existing EC2 instance is continuously running and polling the queue, incurring costs even when there are no messages to process. This is inefficient.

Lambda functions are serverless compute services that execute code only when triggered. By triggering the Lambda function based on new messages arriving in the SQS queue, you eliminate the need for a constantly running EC2 instance. This significantly reduces costs because you only pay for the compute time used when processing messages.

Specifically, you can configure the Lambda function to be triggered by SQS using SQS as an event source. When messages are available, Lambda automatically scales to handle the workload. This makes the solution cost-effective and inherently scalable to accommodate the growing number of messages.

Option A, increasing the EC2 instance size, would increase costs without necessarily solving the underlying inefficiency of continuous operation. Option B, using EventBridge to turn off the EC2 instance, is complex to implement reliably and might still lead to delays in processing messages. Option D, using Systems Manager Run Command, doesn't automatically scale the processing based on the number of messages, so there is no guarantee that on-demand instances can handle the workload quickly without a pre-warmed pool of instances, which also increases cost and is against the need for minimizing it. The best approach is to leverage the serverless and event-driven architecture that Lambda provides for optimal cost and scalability.

Further research:

AWS Lambda: https://aws.amazon.com/lambda/
Amazon SQS as a Lambda Event Source: https://docs.aws.amazon.com/lambda/latest/dg/services-sqs.html
Serverless Architectures: https://aws.amazon.com/serverless/

Question: 317 **Exam Heist**

A company uses a legacy application to produce data in CSV format. The legacy application stores the output data in Amazon S3. The company is deploying a new commercial off-the-shelf (COTS) application that can perform complex SQL queries to analyze data that is stored in Amazon Redshift and Amazon S3 only. However, the COTS application cannot process the .csv files that the legacy application produces.

The company cannot update the legacy application to produce data in another format. The company needs to implement a solution so that the COTS application can use the data that the legacy application produces.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create an AWS Glue extract, transform, and load (ETL) job that runs on a schedule. Configure the ETL job to process the .csv files and store the processed data in Amazon Redshift.

B.Develop a Python script that runs on Amazon EC2 instances to convert the .csv files to .sql files. Invoke the Python script on a cron schedule to store the output files in Amazon S3.

C.Create an AWS Lambda function and an Amazon DynamoDB table. Use an S3 event to invoke the Lambda function. Configure the Lambda function to perform an extract, transform, and load (ETL) job to process the .csv files and store the processed data in the DynamoDB table.

D.Use Amazon EventBridge to launch an Amazon EMR cluster on a weekly schedule. Configure the EMR cluster to perform an extract, transform, and load (ETL) job to process the .csv files and store the processed data in an Amazon Redshift table.

Answer: A

Explanation:

The best solution is **A. Create an AWS Glue extract, transform, and load (ETL) job that runs on a schedule. Configure the ETL job to process the .csv files and store the processed data in Amazon Redshift.**

Here's why:

AWS Glue is a fully managed ETL service. This significantly reduces operational overhead compared to managing EC2 instances, EMR clusters, or Lambda functions for complex data transformations.
Direct Integration with Amazon Redshift: Glue is designed to work seamlessly with Redshift. It can efficiently load and transform data directly into Redshift tables, making it readily accessible for the COTS application's SQL queries.
Automatic Schema Discovery (Crawler): Glue can automatically infer the schema of the CSV files stored in S3, simplifying the ETL process.
Scalability and Reliability: Glue provides scalable and reliable ETL processing with minimal operational effort. The scheduled ETL job will handle the CSV files as they are produced.

Why other options are less suitable:

B. Python script on EC2: This requires managing EC2 instances, writing and maintaining Python code, and handling potential scaling issues. Converting CSV to SQL and storing as .sql files is not an efficient approach for data analysis in Redshift.
C. Lambda and DynamoDB: DynamoDB is a NoSQL database and not suitable for complex SQL queries. While Lambda is serverless, it's less ideal for large-scale data transformation tasks compared to Glue, and DynamoDB isn't the target data store for the COTS application's SQL queries.
D. EMR: EMR is powerful for big data processing, but using it for simple CSV conversion is overkill. It has higher operational overhead and cost compared to Glue for this specific use case. Also, a weekly schedule

might not be sufficient if data needs to be available more frequently.

In summary, AWS Glue provides the most straightforward, cost-effective, and operationally efficient solution for transforming the CSV data and loading it into Amazon Redshift for use by the COTS application.

Authoritative Links:

AWS Glue Documentation: https://aws.amazon.com/glue/
Amazon Redshift Documentation: https://aws.amazon.com/redshift/

Question: 318 **Exam Heist**

A company recently migrated its entire IT environment to the AWS Cloud. The company discovers that users are provisioning oversized Amazon EC2 instances and modifying security group rules without using the appropriate change control process. A solutions architect must devise a strategy to track and audit these inventory and configuration changes.

Which actions should the solutions architect take to meet these requirements? (Choose two.)

A. Enable AWS CloudTrail and use it for auditing.
B. Use data lifecycle policies for the Amazon EC2 instances.
C. Enable AWS Trusted Advisor and reference the security dashboard.
D. Enable AWS Config and create rules for auditing and compliance purposes.
E. Restore previous resource configurations with an AWS CloudFormation template.

Answer: AD

Explanation:

The correct answer is AD. Here's why:

A. Enable AWS CloudTrail and use it for auditing: CloudTrail is an AWS service that enables governance, compliance, operational auditing, and risk auditing of your AWS account. By enabling CloudTrail, you record API calls made within your AWS environment. This includes actions like launching EC2 instances, modifying security group rules, and other configuration changes. You can then analyze the CloudTrail logs to track who made the changes, when they were made, and from where. This addresses the need to track and audit configuration changes. https://aws.amazon.com/cloudtrail/

D. Enable AWS Config and create rules for auditing and compliance purposes: AWS Config continuously monitors and records the configuration of your AWS resources. It allows you to create rules that automatically check whether resources comply with desired configurations. For instance, you can create a Config rule to check if EC2 instances meet certain size criteria or if security groups have overly permissive rules. Config can also trigger remediation actions when resources are non-compliant. This satisfies the requirement to track inventory and configuration changes and ensure compliance with company policies. https://aws.amazon.com/config/

Now, let's consider why the other options are incorrect:

B. Use data lifecycle policies for the Amazon EC2 instances: Data lifecycle policies are primarily used for managing the lifecycle of data stored in services like Amazon S3 or EBS volumes. They are not directly relevant to tracking and auditing configuration changes or enforcing instance sizing and security group policies.

C. Enable AWS Trusted Advisor and reference the security dashboard: Trusted Advisor provides recommendations on security, cost optimization, performance, and fault tolerance based on best practices. While it offers general security advice, it does not provide detailed, auditable logs of specific configuration

changes made by users. It is more of a reactive advisory service than a continuous auditing and compliance tool.

E. Restore previous resource configurations with an AWS CloudFormation template: While CloudFormation can be used for infrastructure as code and versioning of configurations, using it solely for restoring previous configurations is not a proactive solution for tracking and auditing changes as they happen. It's more of a disaster recovery approach than a continuous monitoring and auditing system. CloudTrail and Config are much more suitable for those purposes.

Question: 319 **Exam Heist**

A company has hundreds of Amazon EC2 Linux-based instances in the AWS Cloud. Systems administrators have used shared SSH keys to manage the instances. After a recent audit, the company's security team is mandating the removal of all shared keys. A solutions architect must design a solution that provides secure access to the EC2 instances.

Which solution will meet this requirement with the LEAST amount of administrative overhead?

A. Use AWS Systems Manager Session Manager to connect to the EC2 instances.

B. Use AWS Security Token Service (AWS STS) to generate one-time SSH keys on demand.

C. Allow shared SSH access to a set of bastion instances. Configure all other instances to allow only SSH access from the bastion instances.

D. Use an Amazon Cognito custom authorizer to authenticate users. Invoke an AWS Lambda function to generate a temporary SSH key.

Answer: A

Explanation:

The correct answer is A: Use AWS Systems Manager Session Manager to connect to the EC2 instances. Here's a detailed justification:

The security team requires the removal of shared SSH keys due to security risks. Session Manager addresses this by eliminating the need for SSH keys altogether. Session Manager uses the AWS Systems Manager Agent (SSM Agent) on the EC2 instance to establish a secure connection to the EC2 instance through the AWS cloud, removing the need for open inbound SSH ports (port 22) and managed SSH keys.

Option B, using AWS STS to generate one-time SSH keys, would be a more complex solution that still involves managing SSH keys, albeit temporary ones. This adds administrative overhead compared to Session Manager's keyless approach. Moreover, integrating STS for temporary SSH key generation would necessitate custom scripting and key management, significantly increasing complexity.

Option C, bastion hosts, introduces a single point of failure and requires configuring security groups to allow SSH access from the bastion hosts, adding to the administrative overhead. While it restricts access compared to shared keys across all servers, it doesn't eliminate the need for SSH keys entirely and presents a maintenance burden.

Option D, using Cognito with a custom authorizer and Lambda for SSH keys, is far more complex and requires significant development and operational overhead to maintain and manage. It's an overkill for the stated requirement.

Session Manager offers the least administrative overhead because it leverages the existing SSM Agent (which is often already installed for patching and other management tasks), provides a secure and auditable connection, and eliminates the complexities associated with managing SSH keys. It centralizes access management through IAM roles, providing granular control over who can access which instances.

Therefore, Session Manager presents the most straightforward and secure solution, adhering to security requirements while minimizing administrative overhead.

Relevant links:

AWS Systems Manager Session Manager: https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html
Eliminating the need for SSH keys: https://aws.amazon.com/blogs/security/simplify-secure-access-to-ec2-instances-with-amazon-ec2-instance-connect-ssh/ (while this article talks about EC2 Instance Connect, the principles of eliminating direct SSH key management are similar to Session Manager)

Question: 320 **Exam Heist**

A company is using a fleet of Amazon EC2 instances to ingest data from on-premises data sources. The data is in JSON format and ingestion rates can be as high as 1 MB/s. When an EC2 instance is rebooted, the data in-flight is lost. The company's data science team wants to query ingested data in near-real time.

Which solution provides near-real-time data querying that is scalable with minimal data loss?

A. Publish data to Amazon Kinesis Data Streams, Use Kinesis Data Analytics to query the data.

B. Publish data to Amazon Kinesis Data Firehose with Amazon Redshift as the destination. Use Amazon Redshift to query the data.

C. Store ingested data in an EC2 instance store. Publish data to Amazon Kinesis Data Firehose with Amazon S3 as the destination. Use Amazon Athena to query the data.

D. Store ingested data in an Amazon Elastic Block Store (Amazon EBS) volume. Publish data to Amazon ElastiCache for Redis. Subscribe to the Redis channel to query the data.

Answer: A

Explanation:

The best solution is A: **Publish data to Amazon Kinesis Data Streams, Use Kinesis Data Analytics to query the data.**

Here's why:

Near Real-Time Querying: Kinesis Data Streams is designed for real-time data streaming and processing. Kinesis Data Analytics enables you to query the streamed data with SQL in near real-time. This directly addresses the requirement for near-real-time querying.

Scalability: Kinesis Data Streams is highly scalable, capable of handling high ingestion rates like 1 MB/s by adjusting the number of shards. This ensures the solution can scale as data volume increases.

Minimal Data Loss: Kinesis Data Streams provides data durability and fault tolerance. Even if an EC2 instance reboots, the data in the stream is not lost (within the configured retention period). This minimizes data loss during instance restarts.

Alternative A weaknesses:

Kinesis Data Firehose is a loading service, not a querying service. While it can load data into data stores, it doesn't allow for interactive, near-real-time queries.

Amazon Redshift is a data warehouse designed for analytical workloads. It's not optimized for near-real-time querying of streaming data.

EC2 instance store are ephemeral. Data is lost when instance stops or terminates.

Amazon S3 is an object store. Querying this type of storage for near-real-time querying could potentially add latency.

Amazon Athena can query the data. But it will be on a more ad hoc basis.

Amazon Elastic Block Store are durable but storing data on EBS and using ElastiCache is not a good architectural fit.

Therefore, Kinesis Data Streams and Kinesis Data Analytics together provide a scalable, near-real-time querying solution with minimal data loss, making option A the most suitable.

Amazon Kinesis Data Streams:https://aws.amazon.com/kinesis/data-streams/

Amazon Kinesis Data Analytics:https://aws.amazon.com/kinesis/data-analytics/

Question: 321 **Exam Heist**

What should a solutions architect do to ensure that all objects uploaded to an Amazon S3 bucket are encrypted?

A.Update the bucket policy to deny if the PutObject does not have an s3:x-amz-acl header set.
B.Update the bucket policy to deny if the PutObject does not have an s3:x-amz-acl header set to private.
C.Update the bucket policy to deny if the PutObject does not have an aws:SecureTransport header set to true.
D.Update the bucket policy to deny if the PutObject does not have an x-amz-server-side-encryption header set.

Answer: D

Explanation:

The correct answer is D because it directly enforces server-side encryption for all objects uploaded to the S3 bucket. The x-amz-server-side-encryption header, when present in a PutObject request, instructs S3 to encrypt the object using server-side encryption (SSE). A bucket policy configured to deny uploads lacking this header effectively mandates that all new objects are encrypted upon storage.

Option A and B focus on Access Control Lists (ACLs), not encryption. While ACLs control access permissions, they do not provide data encryption. Option A is incorrect because requiring any ACL doesn't guarantee encryption. Option B restricting to "private" ACL also controls access and not encryption.

Option C, referencing aws:SecureTransport, is related to requiring HTTPS for data transmission. While using HTTPS secures data in transit, it doesn't guarantee that the data is encrypted at rest in S3 storage. Therefore, it does not fulfill the requirement of ensuring that all objects uploaded to an Amazon S3 bucket are encrypted.

By denying any PutObject request that does not include the x-amz-server-side-encryption header, the policy enforces that all objects are encrypted at rest using server-side encryption, such as SSE-S3, SSE-KMS, or SSE-C. The server-side encryption applied depends on the particular encryption key specified. This header is the mechanism S3 provides for enforcing server-side encryption through bucket policies.

For more information on S3 bucket policies and server-side encryption, refer to the following AWS documentation:

Using server-side encryption with bucket policies:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/bucket-policy-using-sse.html
Protecting data using server-side encryption:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/serv-side-encryption.html
Specifying Encryption in a Request: https://docs.aws.amazon.com/AmazonS3/latest/API/API_PutObject.html

Question: 322 **Exam Heist**

A solutions architect is designing a multi-tier application for a company. The application's users upload images from a mobile device. The application generates a thumbnail of each image and returns a message to the user to confirm that the image was uploaded successfully.

The thumbnail generation can take up to 60 seconds, but the company wants to provide a faster response time to its users to notify them that the original image was received. The solutions architect must design the application to asynchronously dispatch requests to the different application tiers.

What should the solutions architect do to meet these requirements?

A. Write a custom AWS Lambda function to generate the thumbnail and alert the user. Use the image upload process as an event source to invoke the Lambda function.

B. Create an AWS Step Functions workflow. Configure Step Functions to handle the orchestration between the application tiers and alert the user when thumbnail generation is complete.

C. Create an Amazon Simple Queue Service (Amazon SQS) message queue. As images are uploaded, place a message on the SQS queue for thumbnail generation. Alert the user through an application message that the image was received.

D. Create Amazon Simple Notification Service (Amazon SNS) notification topics and subscriptions. Use one subscription with the application to generate the thumbnail after the image upload is complete. Use a second subscription to message the user's mobile app by way of a push notification after thumbnail generation is complete.

Answer: C

Explanation:

The correct answer is C because it provides the most efficient and decoupled method for asynchronous processing of image uploads and thumbnail generation, enabling a faster response time for the user.

Here's why:

Decoupling with SQS: Amazon SQS (Simple Queue Service) decouples the image upload process from the thumbnail generation process. The application places a message containing information about the uploaded image onto the SQS queue. This allows the image upload process to complete quickly and immediately acknowledge the user. The user receives confirmation of receipt without waiting for the resource-intensive thumbnail generation. https://aws.amazon.com/sqs/

Asynchronous Processing: The thumbnail generation process, triggered by messages in the SQS queue, can then proceed asynchronously. A separate worker (e.g., an EC2 instance, Lambda function, or ECS task) can consume messages from the queue and generate the thumbnail at its own pace. This addresses the requirement to handle potentially time-consuming thumbnail generation without impacting the user's perceived performance.

Scalability and Reliability: SQS is a highly scalable and reliable messaging service. It can handle a large volume of messages, ensuring that no image upload requests are lost. The queue also provides buffering capabilities, allowing the thumbnail generation process to keep up with varying workloads.

Alternative A's drawbacks: While Lambda can generate thumbnails, using the image upload as a direct event source tightly couples the upload process with the thumbnail generation. This defeats the purpose of providing a fast response to the user. Lambda invocations have time limits, which may be exceeded by long thumbnail generations.

Alternative B's drawbacks: Step Functions are more suited for complex workflows with multiple steps and decision points. For a simple scenario like this, SQS offers a more lightweight and efficient solution. Step

functions would introduce unnecessary complexity and overhead.

Alternative D's drawbacks: SNS is primarily designed for fan-out notifications to multiple subscribers. While it could be used for this scenario, it's less suitable than SQS for asynchronous task processing, especially when message reliability and guaranteed delivery are important. SNS delivers messages to all subscribers immediately, which doesn't create a queue for later processing.

In summary, SQS enables decoupling, asynchronous processing, scalability, and reliability, providing the best solution for achieving the desired faster response time for users while managing potentially lengthy thumbnail generation processes.

Question: 323 **Exam Heist**

A company's facility has badge readers at every entrance throughout the building. When badges are scanned, the readers send a message over HTTPS to indicate who attempted to access that particular entrance.

A solutions architect must design a system to process these messages from the sensors. The solution must be highly available, and the results must be made available for the company's security team to analyze.

Which system architecture should the solutions architect recommend?

A. Launch an Amazon EC2 instance to serve as the HTTPS endpoint and to process the messages. Configure the EC2 instance to save the results to an Amazon S3 bucket.

B. Create an HTTPS endpoint in Amazon API Gateway. Configure the API Gateway endpoint to invoke an AWS Lambda function to process the messages and save the results to an Amazon DynamoDB table.

C. Use Amazon Route 53 to direct incoming sensor messages to an AWS Lambda function. Configure the Lambda function to process the messages and save the results to an Amazon DynamoDB table.

D. Create a gateway VPC endpoint for Amazon S3. Configure a Site-to-Site VPN connection from the facility network to the VPC so that sensor data can be written directly to an S3 bucket by way of the VPC endpoint.

Answer: B

Explanation:

The correct answer is **B**, which leverages API Gateway and Lambda for message processing and DynamoDB for data storage. Here's why:

High Availability: API Gateway is a fully managed service that automatically scales and provides high availability. Similarly, Lambda functions are automatically scaled by AWS based on demand, ensuring the system can handle a large volume of messages from the badge readers without manual intervention. DynamoDB is also a fully managed, highly available NoSQL database.

HTTPS Endpoint: API Gateway allows you to easily create secure HTTPS endpoints for receiving data from the badge readers. This is a crucial requirement as the problem statement specified HTTPS communication from the sensors.

Serverless Architecture: Lambda allows the solution to be serverless. You don't need to provision or manage servers, reducing operational overhead and cost. The Lambda function contains the code to process the messages from the badge readers.

Data Persistence: DynamoDB is a NoSQL database that is well-suited for storing structured or semi-structured data from the badge readers. It offers fast read and write speeds and the flexibility to handle varying data structures.

Scalability: API Gateway, Lambda, and DynamoDB are all designed for scalability. As the number of badge readers or the frequency of scans increases, the system can automatically scale to handle the increased load.

Security: API Gateway offers features like authentication and authorization to protect the endpoint from unauthorized access. Lambda can use IAM roles to access DynamoDB securely.

Why other options are less suitable:

A (EC2 instance): EC2 instances require manual management (patching, scaling) which is not ideal for high availability and creates operational overhead. Also, EC2 is more expensive to run continuously compared to serverless options.
C (Route 53 and Lambda): Route 53 is primarily for DNS routing, not for directly receiving and processing HTTPS requests. It doesn't act as an HTTPS endpoint.
D (S3 and VPN): While S3 is good for storage, the sensor data likely needs processing before storage. Also, writing data directly to S3 without any processing in between introduces security concerns (data validation, etc.). It also requires a VPN which adds complexity. This method is only suitable if the sensor data can be directly and securely persisted without modification.

Authoritative Links:

Amazon API Gateway: https://aws.amazon.com/api-gateway/
AWS Lambda: https://aws.amazon.com/lambda/
Amazon DynamoDB: https://aws.amazon.com/dynamodb/

Question: 324 **Exam Heist**

A company wants to implement a disaster recovery plan for its primary on-premises file storage volume. The file storage volume is mounted from an Internet Small Computer Systems Interface (iSCSI) device on a local storage server. The file storage volume holds hundreds of terabytes (TB) of data.

The company wants to ensure that end users retain immediate access to all file types from the on-premises systems without experiencing latency.

Which solution will meet these requirements with the LEAST amount of change to the company's existing infrastructure?

A. Provision an Amazon S3 File Gateway as a virtual machine (VM) that is hosted on premises. Set the local cache to 10 TB. Modify existing applications to access the files through the NFS protocol. To recover from a disaster, provision an Amazon EC2 instance and mount the S3 bucket that contains the files.

B. Provision an AWS Storage Gateway tape gateway. Use a data backup solution to back up all existing data to a virtual tape library. Configure the data backup solution to run nightly after the initial backup is complete. To recover from a disaster, provision an Amazon EC2 instance and restore the data to an Amazon Elastic Block Store (Amazon EBS) volume from the volumes in the virtual tape library.

C. Provision an AWS Storage Gateway Volume Gateway cached volume. Set the local cache to 10 TB. Mount the Volume Gateway cached volume to the existing file server by using iSCSI, and copy all files to the storage volume. Configure scheduled snapshots of the storage volume. To recover from a disaster, restore a snapshot to an Amazon Elastic Block Store (Amazon EBS) volume and attach the EBS volume to an Amazon EC2 instance.

D. Provision an AWS Storage Gateway Volume Gateway stored volume with the same amount of disk space as the existing file storage volume. Mount the Volume Gateway stored volume to the existing file server by using iSCSI, and copy all files to the storage volume. Configure scheduled snapshots of the storage volume. To recover from a disaster, restore a snapshot to an Amazon Elastic Block Store (Amazon EBS) volume and attach the EBS volume to an Amazon EC2 instance.

Answer: D

Explanation:

The correct answer is D. Here's why:

Requirement: The company needs a disaster recovery plan for a large (hundreds of TB) on-premises file storage volume with immediate access and minimal latency. They want to minimize changes to their existing

infrastructure.

Why D is the best choice (Volume Gateway - Stored Volume):

Stored Volume Configuration: A Stored Volume Gateway stores all data locally on-premises first. This is crucial for maintaining immediate access and low latency, as the data is directly available on-site. The initial synchronization moves all the existing hundreds of TB of data to AWS Storage Gateway.

iSCSI Integration: It utilizes iSCSI, the same protocol the company is already using, minimizing infrastructure changes. The gateway integrates with the existing iSCSI device, providing seamless compatibility.

Snapshots for DR: Scheduled snapshots capture point-in-time copies of the data and store them in AWS. This provides a reliable mechanism for recovery.

DR Recovery: In a disaster, the snapshots can be restored to an Amazon EBS volume and attached to an EC2 instance, enabling a functional file server in AWS. This ensures business continuity.

Why other options are incorrect:

A (S3 File Gateway): File Gateway uses S3 as the primary storage, which introduces latency to access the data. Furthermore, it requires modifying existing applications to use the NFS protocol, adding complexity.

B (Tape Gateway): Tape Gateway is for archival purposes, not for immediate access disaster recovery. Restoring from tapes is a slow process and wouldn't meet the "immediate access" requirement.

C (Volume Gateway - Cached Volume): While it uses iSCSI, a cached volume only stores the frequently accessed data locally. With hundreds of TB, a 10 TB cache would mean that most data would reside in AWS, leading to high latency for access.

In conclusion, the Stored Volume Gateway configuration maintains local data access for low latency, integrates seamlessly with existing iSCSI infrastructure, and leverages AWS snapshots for reliable disaster recovery.

AWS Storage Gateway Documentation:
https://docs.aws.amazon.com/storagegateway/latest/userguide/WhatIsStorageGateway.html
Volume Gateway - Stored Volume details:
https://docs.aws.amazon.com/storagegateway/latest/userguide/stored-volumes.html

Question: 325 **Exam Heist**

A company is hosting a web application from an Amazon S3 bucket. The application uses Amazon Cognito as an identity provider to authenticate users and return a JSON Web Token (JWT) that provides access to protected resources that are stored in another S3 bucket.

Upon deployment of the application, users report errors and are unable to access the protected content. A solutions architect must resolve this issue by providing proper permissions so that users can access the protected content.

Which solution meets these requirements?

A. Update the Amazon Cognito identity pool to assume the proper IAM role for access to the protected content.

B. Update the S3 ACL to allow the application to access the protected content.

C. Redeploy the application to Amazon S3 to prevent eventually consistent reads in the S3 bucket from affecting the ability of users to access the protected content.

D. Update the Amazon Cognito pool to use custom attribute mappings within the identity pool and grant users the proper permissions to access the protected content.

Answer: A

Explanation:

The correct answer is **A: Update the Amazon Cognito identity pool to assume the proper IAM role for access to the protected content.**

Here's a detailed justification:

The core problem is users, authenticated through Cognito, are failing to access protected S3 content. Cognito Identity Pools are designed precisely to grant temporary AWS credentials to users, allowing them to access AWS resources. The crucial step is ensuring that the Cognito Identity Pool is configured with an IAM role that has the necessary permissions to read (or otherwise access) the protected S3 bucket.

Cognito provides two types of identities: authenticated and unauthenticated. Regardless of identity type, Cognito assumes an IAM role on behalf of the user. This IAM role dictates what AWS resources the user can access. If the IAM role assigned to the Cognito Identity Pool lacks S3 read permissions for the protected bucket, users will be denied access, hence the errors.

Updating the Identity Pool to assume an IAM role with the required s3:GetObject (and potentially other actions like s3:ListBucket if listing the contents of the bucket is needed) permission on the protected S3 bucket resolves the issue. This approach adheres to the principle of least privilege: the user only gets the permissions needed to access the protected resources, and the permissions are temporary, granted by AWS STS (Security Token Service) behind the scenes.

Option B is incorrect because S3 ACLs are a legacy access control mechanism and are generally not recommended for granting access to users authenticated via Cognito. IAM roles and policies offer much finer-grained control and better integration with identity providers.

Option C is unrelated to the authentication and authorization problem. S3's eventual consistency is a factor to consider when designing applications, but it's not the root cause here. The problem is that the authenticated users lack the permissions to read the data in the first place.

Option D is less efficient and potentially less secure than using IAM roles directly. While custom attribute mappings within Cognito can pass user attributes to IAM policies, it's generally simpler and more maintainable to assign users to Cognito Identity Pools that assume predefined IAM roles. Moreover, simply mapping attributes doesn't automatically grant permissions; the policies still need to be defined and associated with the role. Directly updating the IAM role associated with the Identity Pool is the most direct and appropriate solution.

In summary, Cognito Identity Pools linked to appropriate IAM roles provide a robust and secure mechanism for granting authenticated users access to protected AWS resources.

Supporting links:

AWS Cognito Identity Pools: https://docs.aws.amazon.com/cognito/latest/developerguide/identity-pools.html
IAM Roles: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html
S3 Permissions Overview: https://docs.aws.amazon.com/AmazonS3/latest/userguide/security-iam.html

Question: 326 **Exam Heist**

An image hosting company uploads its large assets to Amazon S3 Standard buckets. The company uses multipart upload in parallel by using S3 APIs and overwrites if the same object is uploaded again. For the first 30 days after upload, the objects will be accessed frequently. The objects will be used less frequently after 30 days, but the access patterns for each object will be inconsistent. The company must optimize its S3 storage costs while maintaining high availability and resiliency of stored assets.

Which combination of actions should a solutions architect recommend to meet these requirements? (Choose two.)

A. Move assets to S3 Intelligent-Tiering after 30 days.
B. Configure an S3 Lifecycle policy to clean up incomplete multipart uploads.
C. Configure an S3 Lifecycle policy to clean up expired object delete markers.
D. Move assets to S3 Standard-Infrequent Access (S3 Standard-IA) after 30 days.
E. Move assets to S3 One Zone-Infrequent Access (S3 One Zone-IA) after 30 days.

Answer: AB

Explanation:

Here's a detailed justification for why options A and B are the correct choices, and why the others are not:

A. Move assets to S3 Intelligent-Tiering after 30 days:

S3 Intelligent-Tiering is designed for data with unknown or changing access patterns. It automatically moves data between frequent, infrequent, and archive access tiers based on actual access patterns, optimizing costs without performance impact.
The question states that access patterns for each object are inconsistent after 30 days. Intelligent-Tiering is ideal for this scenario, as it automatically adapts to these changing patterns.
Standard-IA is suitable for data accessed less frequently, but requires you to know when you need to change tier. Intelligent-Tiering automates this decision.

B. Configure an S3 Lifecycle policy to clean up incomplete multipart uploads:

The company uses multipart upload and overwrites objects. Incomplete multipart uploads (if a part fails to upload or the upload is aborted) consume storage space unnecessarily, increasing costs.
An S3 Lifecycle policy can automatically clean up these incomplete uploads after a specified period.
Multipart uploads are used for large objects and are prone to issues, making this a crucial optimization.

Why the other options are incorrect:

C. Configure an S3 Lifecycle policy to clean up expired object delete markers: Object delete markers are small and their cost is insignificant. It is not an area that needs optimisation.
D. Move assets to S3 Standard-Infrequent Access (S3 Standard-IA) after 30 days: This is less optimal than Intelligent-Tiering because it requires you to know the access pattern.
E. Move assets to S3 One Zone-Infrequent Access (S3 One Zone-IA) after 30 days: While One Zone-IA is cheaper, it compromises data availability and resiliency by storing data in a single availability zone. The question specifies a need for "high availability and resiliency," making this option unsuitable.

Supporting Links:

S3 Intelligent-Tiering: https://aws.amazon.com/s3/storage-classes/intelligent-tiering/
S3 Lifecycle Policies: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-examples.html
Multipart Upload: https://docs.aws.amazon.com/AmazonS3/latest/userguide/mpuoverview.html

Question: 327 **Exam Heist**

A solutions architect must secure a VPC network that hosts Amazon EC2 instances. The EC2 instances contain highly sensitive data and run in a private subnet. According to company policy, the EC2 instances that run in the VPC can access only approved third-party software repositories on the internet for software product updates that use the third party's URL. Other internet traffic must be blocked.

Which solution meets these requirements?

A.Update the route table for the private subnet to route the outbound traffic to an AWS Network Firewall firewall. Configure domain list rule groups.

B.Set up an AWS WAF web ACL. Create a custom set of rules that filter traffic requests based on source and destination IP address range sets.

C.Implement strict inbound security group rules. Configure an outbound rule that allows traffic only to the authorized software repositories on the internet by specifying the URLs.

D.Configure an Application Load Balancer (ALB) in front of the EC2 instances. Direct all outbound traffic to the ALB. Use a URL-based rule listener in the ALB's target group for outbound access to the internet.

Answer: A

Explanation:

The correct answer is A: Update the route table for the private subnet to route the outbound traffic to an AWS Network Firewall firewall. Configure domain list rule groups.

Here's why:

Requirement for granular control: The scenario necessitates restricting outbound traffic to specific, approved third-party software repositories identified by their URLs. Security groups and NACLs, while useful for basic traffic filtering, don't offer URL-based filtering. AWS WAF is designed for protecting web applications from common exploits and doesn't typically handle generic outbound traffic filtering from EC2 instances. An ALB is primarily designed for handling inbound application traffic and is not suited for routing and filtering general outbound internet traffic from EC2 instances.
AWS Network Firewall: AWS Network Firewall provides centralized network protection for VPCs. It allows defining fine-grained rules based on domain names, URLs, and other criteria, making it ideal for meeting the requirement of allowing access only to specific software repositories.
Domain list rule groups: Network Firewall utilizes domain list rule groups, which enable specifying a list of allowed or denied domain names. This aligns directly with the requirement of permitting access only to the URLs of the approved software repositories.
Private Subnet & Route Table: The EC2 instances reside in a private subnet, meaning they don't have direct access to the internet. Configuring the route table to route outbound traffic to the Network Firewall ensures that all traffic from these instances is inspected and filtered according to the defined rules.
Security Best Practices: Using a centralized firewall solution like Network Firewall promotes consistent security policies across the entire VPC, adhering to security best practices for network segmentation and traffic inspection.

Why other options are incorrect:

B (AWS WAF): AWS WAF is designed for web application protection and primarily deals with HTTP/HTTPS traffic targeting web servers. It's not suitable for filtering all outbound traffic from EC2 instances.
C (Security Groups): Security groups operate at the instance level and are stateful firewalls that allow or deny traffic based on IP addresses and ports. They lack the ability to filter traffic based on URLs.
D (ALB): Application Load Balancers are for distributing incoming application traffic. They are not designed for routing and filtering general outbound traffic from EC2 instances in a private subnet.

Authoritative Links:

AWS Network Firewall: https://aws.amazon.com/network-firewall/
AWS Network Firewall Rule Groups: https://docs.aws.amazon.com/network-firewall/latest/developerguide/rule-groups.html

Question: 328 **Exam Heist**

A company is hosting a three-tier ecommerce application in the AWS Cloud. The company hosts the website on

Amazon S3 and integrates the website with an API that handles sales requests. The company hosts the API on three Amazon EC2 instances behind an Application Load Balancer (ALB). The API consists of static and dynamic front-end content along with backend workers that process sales requests asynchronously.

The company is expecting a significant and sudden increase in the number of sales requests during events for the launch of new products.

What should a solutions architect recommend to ensure that all the requests are processed successfully?

A. Add an Amazon CloudFront distribution for the dynamic content. Increase the number of EC2 instances to handle the increase in traffic.

B. Add an Amazon CloudFront distribution for the static content. Place the EC2 instances in an Auto Scaling group to launch new instances based on network traffic.

C. Add an Amazon CloudFront distribution for the dynamic content. Add an Amazon ElastiCache instance in front of the ALB to reduce traffic for the API to handle.

D. Add an Amazon CloudFront distribution for the static content. Add an Amazon Simple Queue Service (Amazon SQS) queue to receive requests from the website for later processing by the EC2 instances.

Answer: D

Explanation:

The correct answer is **D**. Here's why:

The problem describes a scenario where a company anticipates a surge in sales requests during product launches, potentially overwhelming their API hosted on EC2 instances. The goal is to ensure all requests are processed successfully despite the load spike.

Option D addresses this problem effectively by:

1. **Caching static content using Amazon CloudFront**: Serving static content (like images, CSS, JavaScript) from CloudFront reduces the load on the EC2 instances, freeing them up to handle dynamic API requests. CloudFront distributes the content globally, improving website performance for users worldwide. https://aws.amazon.com/cloudfront/

2. **Using Amazon SQS to decouple the web application from the API**: Introducing an SQS queue between the website and the API creates a buffer. The website can quickly submit sales requests to the SQS queue without waiting for the API to process them immediately. This prevents the website from being overwhelmed and ensures requests are not lost. https://aws.amazon.com/sqs/

3. **EC2 instances process messages from the SQS queue asynchronously**: The EC2 instances, acting as workers, pull requests from the SQS queue at their own pace. Even during peak loads, the queue stores requests until the instances can process them, preventing request loss. This asynchronous processing ensures that all requests are eventually handled. This pattern implements a form of message queuing, promoting system resilience and scalability.

Why other options are less suitable:

A: Caching dynamic content isn't typically beneficial as it changes frequently. Increasing the number of EC2 instances alone might not be enough to handle the sudden surge and doesn't guarantee requests won't be dropped.
B: While caching static content with CloudFront is good, simply using an Auto Scaling group might not prevent the EC2 instances from being overwhelmed if the traffic spike is too sudden and large. Auto Scaling takes time to provision new instances.
C: Caching dynamic content with CloudFront is not suitable. ElastiCache can help with database caching but doesn't directly address the problem of front-end request overload. It won't stop the EC2 instances from being overwhelmed by the initial surge of requests.

In summary, Option D combines caching static content with asynchronous processing via SQS to effectively handle the expected traffic spikes and ensure all sales requests are successfully processed. The use of SQS offers resilience and scalability, which are crucial for handling unpredictable workloads.

Question: 329 **Exam Heist**

A security audit reveals that Amazon EC2 instances are not being patched regularly. A solutions architect needs to provide a solution that will run regular security scans across a large fleet of EC2 instances. The solution should also patch the EC2 instances on a regular schedule and provide a report of each instance's patch status.

Which solution will meet these requirements?

A. Set up Amazon Macie to scan the EC2 instances for software vulnerabilities. Set up a cron job on each EC2 instance to patch the instance on a regular schedule.

B. Turn on Amazon GuardDuty in the account. Configure GuardDuty to scan the EC2 instances for software vulnerabilities. Set up AWS Systems Manager Session Manager to patch the EC2 instances on a regular schedule.

C. Set up Amazon Detective to scan the EC2 instances for software vulnerabilities. Set up an Amazon EventBridge scheduled rule to patch the EC2 instances on a regular schedule.

D. Turn on Amazon Inspector in the account. Configure Amazon Inspector to scan the EC2 instances for software vulnerabilities. Set up AWS Systems Manager Patch Manager to patch the EC2 instances on a regular schedule.

Answer: D

Explanation:

The correct answer is D because it leverages AWS services specifically designed for vulnerability scanning and patch management.

Amazon Inspector is a vulnerability management service that automatically assesses EC2 instances for vulnerabilities and deviations from best practices. It provides detailed findings with severity levels and remediation recommendations. https://aws.amazon.com/inspector/

AWS Systems Manager Patch Manager automates the process of patching managed instances, including EC2 instances, with security-related updates. It allows for scheduled patching, compliance scanning, and reporting on patch status. Patch Manager directly addresses the need for regular patching and reporting on patch status. https://aws.amazon.com/systems-manager/features/patch-manager/

Option A is incorrect because Amazon Macie focuses on discovering and protecting sensitive data stored in Amazon S3. It doesn't scan EC2 instances for general software vulnerabilities. Cron jobs on individual instances are less manageable and scalable compared to Systems Manager Patch Manager for a large fleet.

Option B is incorrect because Amazon GuardDuty is a threat detection service that monitors for malicious activity and unauthorized behavior. While it identifies threats, it does not provide vulnerability scanning or patch management capabilities. Session Manager facilitates secure instance access but doesn't handle the patching process itself.

Option C is incorrect because Amazon Detective analyzes log data to investigate security incidents. It doesn't scan for vulnerabilities or manage patching. While EventBridge can trigger actions on a schedule, it lacks the integrated patch management capabilities of Systems Manager Patch Manager, such as dependency resolution, reboot management, and reporting.

Question: 330 **Exam Heist**

A company is planning to store data on Amazon RDS DB instances. The company must encrypt the data at rest.

What should a solutions architect do to meet this requirement?

A. Create a key in AWS Key Management Service (AWS KMS). Enable encryption for the DB instances.

B. Create an encryption key. Store the key in AWS Secrets Manager. Use the key to encrypt the DB instances.

C. Generate a certificate in AWS Certificate Manager (ACM). Enable SSL/TLS on the DB instances by using the certificate.

D. Generate a certificate in AWS Identity and Access Management (IAM). Enable SSL/TLS on the DB instances by using the certificate.

Answer: A

Explanation:

The correct answer is A because it leverages AWS KMS, the recommended and secure way to encrypt data at rest in RDS. AWS KMS allows you to create and manage encryption keys centrally and securely. Enabling encryption for an RDS DB instance using a KMS key means the data is encrypted at rest using industry-standard encryption algorithms. RDS integrates seamlessly with KMS, handling the encryption and decryption operations transparently.

Option B is incorrect. While AWS Secrets Manager is used for storing secrets like database credentials, it's not the ideal tool for managing encryption keys for RDS data at rest. KMS is specifically designed for key management and offers features like key rotation and auditing, which are critical for security and compliance.

Options C and D are incorrect because ACM and IAM certificates are used for encrypting data in transit using SSL/TLS, not for encrypting data at rest. SSL/TLS secures the connection between the client and the database server, preventing eavesdropping, but it doesn't protect the data stored on the disk. Encryption at rest protects data if the underlying storage is compromised.

Therefore, leveraging KMS for RDS encryption addresses the security requirement of encrypting data at rest in a secure, manageable, and compliant manner. This is best practice and directly aligned with AWS's recommended architecture.

https://docs.aws.amazon.com/kms/latest/developerguide/overview.htmlhttps://docs.aws.amazon.com/AmazonRDS/
at-rest-encryption.html

Question: 331 **Exam Heist**

A company must migrate 20 TB of data from a data center to the AWS Cloud within 30 days. The company's network bandwidth is limited to 15 Mbps and cannot exceed 70% utilization.

What should a solutions architect do to meet these requirements?

A. Use AWS Snowball.

B. Use AWS DataSync.

C. Use a secure VPN connection.

D. Use Amazon S3 Transfer Acceleration.

Answer: A

Explanation:

Here's a detailed justification for why the correct answer is A (Use AWS Snowball) and why the other options are less suitable, considering the constraints of the scenario:

Why AWS Snowball is the Best Choice:

The primary bottleneck is the limited network bandwidth (15 Mbps with a 70% utilization cap) and the large data volume (20 TB) within a strict timeframe (30 days). Calculating the transfer time using the available bandwidth is crucial:

Usable bandwidth: 15 Mbps * 70% = 10.5 Mbps
Data volume: 20 TB = 20 1024 GB = 20480 GB = 20480 8 bits = 163840 Gb
Estimated transfer time: 163840 Gb / 10.5 Mbps = ~15603 seconds/Gb = 15603 * (1/3600) hours/Gb = ~4.33 hours/Gb
Total time: 4.33 hours/Gb * 20480 Gb = ~88700 hours = ~3696 days

This calculation clearly shows that transferring 20 TB of data over a 10.5 Mbps connection would take far longer than the allowed 30 days. AWS Snowball (now part of AWS Snow Family) offers a physical appliance to securely transfer large amounts of data into and out of AWS. You ship the appliance, load it with your data in your data center, and then ship it back to AWS, where the data is uploaded to S3. This bypasses the network constraint. Snowball offers significant advantages in cost and time when dealing with substantial data volumes and limited bandwidth, fitting the scenario perfectly. The data is also encrypted in transit and at rest for security.

Why Other Options are Less Suitable:

B. Use AWS DataSync: DataSync is a good choice for ongoing replication of data, especially when there's a reasonable network connection. However, with only 10.5 Mbps available and 20 TB to transfer initially, it's simply too slow to meet the 30-day deadline, as demonstrated by the calculations above. DataSync utilizes the network, so it is constrained by the bandwidth limitation.

C. Use a secure VPN connection: A VPN secures the data transfer between the on-premises data center and AWS. However, it doesn't address the fundamental problem of limited bandwidth. Using a VPN on top of an already constrained 15 Mbps connection only adds overhead (reducing the effective bandwidth further), making the transfer even slower.

D. Use Amazon S3 Transfer Acceleration: S3 Transfer Acceleration uses Amazon CloudFront's globally distributed edge locations to accelerate uploads to S3. While it can improve transfer speeds, it still relies on the internet connection. With a very constrained bandwidth (10.5 Mbps), S3 Transfer Acceleration won't provide nearly enough acceleration to transfer 20 TB within 30 days. The initial bandwidth limitations still apply.

Authoritative Links for Further Research:

AWS Snow Family: https://aws.amazon.com/snowball/
AWS DataSync: https://aws.amazon.com/datasync/
Amazon S3 Transfer Acceleration: https://aws.amazon.com/s3/transfer-acceleration/

Question: 332 **Exam Heist**

A company needs to provide its employees with secure access to confidential and sensitive files. The company wants to ensure that the files can be accessed only by authorized users. The files must be downloaded securely to the employees' devices.

The files are stored in an on-premises Windows file server. However, due to an increase in remote usage, the file server is running out of capacity.
.

Which solution will meet these requirements?

A. Migrate the file server to an Amazon EC2 instance in a public subnet. Configure the security group to limit inbound traffic to the employees' IP addresses.

B. Migrate the files to an Amazon FSx for Windows File Server file system. Integrate the Amazon FSx file system with the on-premises Active Directory. Configure AWS Client VPN.

C. Migrate the files to Amazon S3, and create a private VPC endpoint. Create a signed URL to allow download.

D. Migrate the files to Amazon S3, and create a public VPC endpoint. Allow employees to sign on with AWS IAM Identity Center (AWS Single Sign-On).

Answer: B

Explanation:

The correct answer is **B. Migrate the files to an Amazon FSx for Windows File Server file system. Integrate the Amazon FSx file system with the on-premises Active Directory. Configure AWS Client VPN.**

Here's a detailed justification:

1. **Security Requirement:** The company needs secure access to confidential files, limiting access to authorized users only. Answer B addresses this by integrating Amazon FSx for Windows File Server with the on-premises Active Directory. Active Directory is a directory service that allows central management of users and security policies.
2. **Authorization:** By integrating FSx with Active Directory, existing user accounts and permissions defined in the on-premises environment are extended to the cloud file system. Only authorized users will have access based on their defined roles.
3. **Secure Download:** Using AWS Client VPN ensures that all data transferred between the employee's device and AWS is encrypted over a secure VPN tunnel. This satisfies the requirement of secure downloads to employees' devices.
4. **Scalability:** Amazon FSx for Windows File Server is a fully managed Windows file server in the cloud. Migrating the files to FSx solves the capacity issue of the on-premises server, as FSx can scale to meet the company's growing storage needs.
5. **Public vs. Private Subnets (Option A):** Option A suggests migrating to an EC2 instance in a public subnet. Public subnets expose resources to the internet, creating security risks. Limiting inbound traffic by IP address is not a scalable or secure solution, especially with remote employees who have dynamic IP addresses.
6. **S3 with Signed URLs/Public VPC Endpoint (Options C & D):** Options C and D suggest storing files in Amazon S3. While S3 is excellent for object storage, it requires additional configuration to mimic the behavior of a file server, such as shared access and file locking. Signed URLs can be complex to manage at scale and are typically used for temporary access. Public VPC endpoints are generally discouraged for sensitive data.
7. **AWS IAM Identity Center (Option D):** While using AWS IAM Identity Center (formerly AWS Single Sign-On) can provide centralized authentication, it doesn't inherently guarantee secure file downloads or integrate seamlessly with existing on-premises Active Directory permissions for file access control without significant additional configuration.
8. **FSx for Windows File Server as a File Server:** FSx for Windows File Server offers features such as SMB protocol support, Active Directory integration, and NTFS permissions that makes it appropriate for organizations looking to move existing Windows file server workloads to AWS, without requiring significant changes to application code.

In summary, Option B is the only solution that meets all the requirements, offering secure access via VPN,

integration with existing Active Directory for authorization, and scalability through Amazon FSx for Windows File Server.

Supporting Links:

Amazon FSx for Windows File Server: https://aws.amazon.com/fsx/windows/
AWS Client VPN: https://aws.amazon.com/vpn/client-vpn/

Question: 333 **Exam Heist**

A company's application runs on Amazon EC2 instances behind an Application Load Balancer (ALB). The instances run in an Amazon EC2 Auto Scaling group across multiple Availability Zones. On the first day of every month at midnight, the application becomes much slower when the month-end financial calculation batch runs. This causes the CPU utilization of the EC2 instances to immediately peak to 100%, which disrupts the application.

What should a solutions architect recommend to ensure the application is able to handle the workload and avoid downtime?

A. Configure an Amazon CloudFront distribution in front of the ALB.
B. Configure an EC2 Auto Scaling simple scaling policy based on CPU utilization.
C. Configure an EC2 Auto Scaling scheduled scaling policy based on the monthly schedule.
D. Configure Amazon ElastiCache to remove some of the workload from the EC2 instances.

Answer: C

Explanation:

The correct answer is **C. Configure an EC2 Auto Scaling scheduled scaling policy based on the monthly schedule.**

Here's why:

The problem is a predictable, recurring spike in CPU utilization due to a month-end financial calculation that happens on a fixed schedule. An Auto Scaling scheduled scaling policy directly addresses this predictable workload. It allows you to proactively increase the number of EC2 instances before the workload hits, ensuring enough capacity to handle the increased demand and prevent the CPU from maxing out. By scaling up in anticipation, the application remains responsive and avoids downtime. Scheduled scaling is ideal for workloads that have consistent and repeating patterns.

Option A (CloudFront) isn't suitable because CloudFront is a content delivery network (CDN). It helps distribute static content and doesn't address the underlying processing bottleneck on the EC2 instances performing the financial calculations. The issue isn't about content delivery speed but about compute capacity.

Option B (Simple scaling policy) isn't optimal. A simple scaling policy reacts after CPU utilization hits a threshold. In this case, by the time the Auto Scaling group responds to the high CPU, the application has already become slow or unresponsive, as the CPU has already peaked. While reactive scaling is useful, proactive scaling through scheduled actions is better for predictable spikes.

Option D (ElastiCache) might help if the financial calculations involve frequent reads of the same data, which isn't specified. However, it doesn't address the core problem of needing more compute power to process the financial calculations themselves. Furthermore, implementing ElastiCache could involve significant application code changes, making it a more complex solution than scheduled scaling. Even if caching improves some aspects, the underlying computational demand still exists and needs to be managed with capacity. Scheduled scaling handles the capacity increase preemptively, directly addressing the workload spike.

In summary, scheduled scaling is the most efficient and targeted solution because it precisely addresses the predictable workload and avoids the reactive nature of other Auto Scaling policies.

Authoritative Links:

AWS Auto Scaling Scheduled Scaling:
https://docs.aws.amazon.com/autoscaling/ec2/userguide/schedule_time.html
AWS Auto Scaling Concepts:
https://docs.aws.amazon.com/autoscaling/ec2/userguide/AutoScalingGroup.html

Question: 334 **Exam Heist**

A company wants to give a customer the ability to use on-premises Microsoft Active Directory to download files that are stored in Amazon S3. The customer's application uses an SFTP client to download the files.

Which solution will meet these requirements with the LEAST operational overhead and no changes to the customer's application?

A. Set up AWS Transfer Family with SFTP for Amazon S3. Configure integrated Active Directory authentication.

B. Set up AWS Database Migration Service (AWS DMS) to synchronize the on-premises client with Amazon S3. Configure integrated Active Directory authentication.

C. Set up AWS DataSync to synchronize between the on-premises location and the S3 location by using AWS IAM Identity Center (AWS Single Sign-On).

D. Set up a Windows Amazon EC2 instance with SFTP to connect the on-premises client with Amazon S3. Integrate AWS Identity and Access Management (IAM).

Answer: A

Explanation:

The correct answer is A because it directly addresses the requirements with the least operational overhead and no changes to the customer's application. AWS Transfer Family with SFTP for Amazon S3 natively supports SFTP protocol to access S3 buckets. It also provides built-in integration with Microsoft Active Directory for authentication. This integration allows users to use their existing Active Directory credentials to authenticate and access the S3 bucket via SFTP. It avoids the need for custom authentication mechanisms or application changes.

Option B is incorrect because AWS DMS is for database migrations and does not support SFTP or file transfer scenarios. It is unsuitable for synchronizing files between an on-premises client and S3.

Option C is incorrect because AWS DataSync is used to synchronize data between on-premises storage and AWS storage services. While it can sync with S3, it does not directly support SFTP. Also, configuring AWS IAM Identity Center (formerly AWS SSO) would require changes to the client application and is not designed for native SFTP authentication against Active Directory.

Option D is incorrect because setting up an EC2 instance involves more operational overhead than using AWS Transfer Family. It would require managing the EC2 instance, installing and configuring an SFTP server, and handling authentication against Active Directory manually using more complex configurations compared to AWS Transfer Family's built-in Active Directory integration. Also, this option requires more management and potentially higher costs than using the fully managed AWS Transfer Family service.

In conclusion, AWS Transfer Family provides a managed service that integrates directly with Active Directory and S3 via SFTP, fulfilling all requirements with minimal configuration and operational overhead.

Here are some authoritative links for further research:

AWS Transfer Family: https://aws.amazon.com/transfer/
Using Active Directory authentication with AWS Transfer Family:
https://docs.aws.amazon.com/transfer/latest/userguide/active-directory.html

Question: 335 **Exam Heist**

A company is experiencing sudden increases in demand. The company needs to provision large Amazon EC2 instances from an Amazon Machine Image (AMI). The instances will run in an Auto Scaling group. The company needs a solution that provides minimum initialization latency to meet the demand.

Which solution meets these requirements?

A. Use the aws ec2 register-image command to create an AMI from a snapshot. Use AWS Step Functions to replace the AMI in the Auto Scaling group.

B. Enable Amazon Elastic Block Store (Amazon EBS) fast snapshot restore on a snapshot. Provision an AMI by using the snapshot. Replace the AMI in the Auto Scaling group with the new AMI.

C. Enable AMI creation and define lifecycle rules in Amazon Data Lifecycle Manager (Amazon DLM). Create an AWS Lambda function that modifies the AMI in the Auto Scaling group.

D. Use Amazon EventBridge to invoke AWS Backup lifecycle policies that provision AMIs. Configure Auto Scaling group capacity limits as an event source in EventBridge.

Answer: B

Explanation:

The question targets minimizing EC2 instance initialization latency during demand spikes when using Auto Scaling. This is primarily achieved by reducing the time it takes to create instances from the AMI.

Option B, enabling Amazon EBS fast snapshot restore (FSR) and using a snapshot to provision the AMI, directly addresses the latency requirement. EBS fast snapshot restore significantly reduces the time to provision volumes from a snapshot. When you create an AMI from a snapshot with FSR enabled, subsequent EC2 instances launched using that AMI will have volumes that are quickly provisioned, minimizing initialization latency. By replacing the AMI in the Auto Scaling group with the new AMI based on FSR, new instances launched by Auto Scaling will benefit from the rapid EBS volume provisioning.

Option A involves registering an AMI from a snapshot. While this creates an AMI, it doesn't inherently address the EBS volume provisioning time. Creating an AMI from a snapshot is a standard practice, but it does not directly minimize instance initialization latency when using EBS backed AMI. Step Functions will not reduce the launch time, it will only orchestrate a deployment.

Option C introduces Amazon Data Lifecycle Manager (DLM) and Lambda. While DLM can automate AMI creation, it doesn't inherently minimize the EBS volume provisioning time. The use of Lambda to modify the Auto Scaling group's AMI might work, but it would not solve the fundamental problem of reducing the latency with the launch of each instance.

Option D employs EventBridge and AWS Backup. While AWS Backup can be used to create EBS snapshots, it's not directly geared towards accelerating instance launch times. The use of EventBridge here doesn't contribute to a faster EC2 initialization.

Therefore, the optimal solution is to enable EBS fast snapshot restore and provision an AMI from the restored snapshot to minimize EBS volume creation latency during instance launch within the Auto Scaling group.

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-fast-snapshot-restore.htmlhttps://aws.amazon.com/ebs/features/fast-snapshot-restore/

Question: 336 **Exam Heist**

A company hosts a multi-tier web application that uses an Amazon Aurora MySQL DB cluster for storage. The application tier is hosted on Amazon EC2 instances. The company's IT security guidelines mandate that the database credentials be encrypted and rotated every 14 days.

What should a solutions architect do to meet this requirement with the LEAST operational effort?

A. Create a new AWS Key Management Service (AWS KMS) encryption key. Use AWS Secrets Manager to create a new secret that uses the KMS key with the appropriate credentials. Associate the secret with the Aurora DB cluster. Configure a custom rotation period of 14 days.

B. Create two parameters in AWS Systems Manager Parameter Store: one for the user name as a string parameter and one that uses the SecureString type for the password. Select AWS Key Management Service (AWS KMS) encryption for the password parameter, and load these parameters in the application tier. Implement an AWS Lambda function that rotates the password every 14 days.

C. Store a file that contains the credentials in an AWS Key Management Service (AWS KMS) encrypted Amazon Elastic File System (Amazon EFS) file system. Mount the EFS file system in all EC2 instances of the application tier. Restrict the access to the file on the file system so that the application can read the file and that only super users can modify the file. Implement an AWS Lambda function that rotates the key in Aurora every 14 days and writes new credentials into the file.

D. Store a file that contains the credentials in an AWS Key Management Service (AWS KMS) encrypted Amazon S3 bucket that the application uses to load the credentials. Download the file to the application regularly to ensure that the correct credentials are used. Implement an AWS Lambda function that rotates the Aurora credentials every 14 days and uploads these credentials to the file in the S3 bucket.

Answer: A

Explanation:

The best solution is A because it leverages AWS Secrets Manager, which is specifically designed for managing, rotating, and retrieving database credentials securely. Using Secrets Manager minimizes operational overhead by automating the rotation process. Creating a new KMS key ensures encryption at rest for the stored credentials. Configuring a custom rotation period of 14 days aligns with the company's security requirements. The application can then retrieve the credentials from Secrets Manager.

Option B, using Systems Manager Parameter Store, is less suitable because while it can store credentials securely, automated rotation is not its primary feature. It would require a custom Lambda function to handle the rotation logic, increasing operational effort. Also, implementing rotation logic in Lambda is not as secure and straightforward as leveraging the built-in features of Secrets Manager.

Options C and D, involving storing credentials in EFS or S3 and using a Lambda function for rotation, are considerably more complex and introduce more operational overhead. They require managing file system permissions, handling file uploads/downloads, and synchronizing credential updates across instances. Furthermore, keeping credentials in files on EC2 instances or even in encrypted S3 buckets exposes them to potential breaches if not handled properly. Secrets Manager encapsulates these concerns effectively.

Secrets Manager handles the complexities of secure credential storage, encryption using KMS, and automated rotation, making it the least operational effort solution.

Here are some authoritative links for further research:

AWS Secrets Manager: https://aws.amazon.com/secrets-manager/
AWS Key Management Service (KMS): https://aws.amazon.com/kms/
Amazon Aurora Security:
https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Security.html

Question: 337 **Exam Heist**

A company has deployed a web application on AWS. The company hosts the backend database on Amazon RDS for MySQL with a primary DB instance and five read replicas to support scaling needs. The read replicas must lag no more than 1 second behind the primary DB instance. The database routinely runs scheduled stored procedures.

As traffic on the website increases, the replicas experience additional lag during periods of peak load. A solutions architect must reduce the replication lag as much as possible. The solutions architect must minimize changes to the application code and must minimize ongoing operational overhead.

Which solution will meet these requirements?

A.Migrate the database to Amazon Aurora MySQL. Replace the read replicas with Aurora Replicas, and configure Aurora Auto Scaling. Replace the stored procedures with Aurora MySQL native functions.

B.Deploy an Amazon ElastiCache for Redis cluster in front of the database. Modify the application to check the cache before the application queries the database. Replace the stored procedures with AWS Lambda functions.

C.Migrate the database to a MySQL database that runs on Amazon EC2 instances. Choose large, compute optimized EC2 instances for all replica nodes. Maintain the stored procedures on the EC2 instances.

D.Migrate the database to Amazon DynamoDB. Provision a large number of read capacity units (RCUs) to support the required throughput, and configure on-demand capacity scaling. Replace the stored procedures with DynamoDB streams.

Answer: A

Explanation:

Here's a detailed justification for why option A is the best solution, along with supporting concepts and authoritative links:

The primary goal is to reduce replication lag in an RDS for MySQL setup while minimizing application code changes and operational overhead. The current MySQL read replicas are experiencing increased lag during peak loads.

Option A, migrating to Amazon Aurora MySQL and using Aurora Replicas with auto-scaling, addresses the problem effectively. Aurora offers significant performance improvements over standard MySQL, especially concerning replication. Aurora's replication is physically based and generally faster and more efficient than the logical replication used by standard MySQL. [https://aws.amazon.com/rds/aurora/features/].

Aurora Replicas share the same underlying storage as the primary instance, reducing replication latency. Aurora Auto Scaling will automatically adjust the number of replicas based on load, ensuring consistent performance even during peak traffic. This eliminates the need for manual intervention and minimizes operational overhead.

Replacing stored procedures with Aurora MySQL native functions further enhances performance. Native functions are typically more tightly integrated and optimized for the Aurora environment compared to custom stored procedures.

Option B, using ElastiCache for Redis, introduces a caching layer. While caching can reduce the load on the database, it requires significant code changes to implement and maintain the cache invalidation logic. It also doesn't directly address the replication lag issue. Redis does not solve replication lag between the primary and replicas, but rather aims to lessen the read load on the database.

Option C, migrating to MySQL on EC2, adds operational overhead. Managing MySQL on EC2 involves tasks like patching, backups, and scaling. While choosing larger instances might temporarily alleviate the lag, it does not provide a scalable and automated solution. The operational burden will also increase.

Option D, migrating to DynamoDB, necessitates substantial application code changes. DynamoDB is a NoSQL database with a fundamentally different data model than MySQL. Rewriting the application to work with DynamoDB would be a major undertaking. Also, the current problem is replication lag, and DynamoDB doesn't

suffer from traditional replication lag issues like MySQL because of its architecture. However, the migration is not justified.

In summary, Aurora MySQL, with its faster replication, auto-scaling, and optimized functions, provides the most effective solution for minimizing replication lag with minimal code changes and reduced operational overhead.

Question: 338 **Exam Heist**

A solutions architect must create a disaster recovery (DR) plan for a high-volume software as a service (SaaS) platform. All data for the platform is stored in an Amazon Aurora MySQL DB cluster.

The DR plan must replicate data to a secondary AWS Region.

Which solution will meet these requirements MOST cost-effectively?

A.Use MySQL binary log replication to an Aurora cluster in the secondary Region. Provision one DB instance for the Aurora cluster in the secondary Region.

B.Set up an Aurora global database for the DB cluster. When setup is complete, remove the DB instance from the secondary Region.

C.Use AWS Database Migration Service (AWS DMS) to continuously replicate data to an Aurora cluster in the secondary Region. Remove the DB instance from the secondary Region.

D.Set up an Aurora global database for the DB cluster. Specify a minimum of one DB instance in the secondary Region.

Answer: D

Explanation:

The most cost-effective disaster recovery (DR) solution for an Aurora MySQL SaaS platform requiring replication to a secondary region is using Aurora Global Database with a minimum of one DB instance in the secondary region. Here's why:

Aurora Global Database: This feature is specifically designed for DR and global read performance. It provides low latency replication across AWS Regions using dedicated infrastructure.
https://aws.amazon.com/rds/aurora/global-database/
Automatic Failover: In case of a primary Region failure, Aurora Global Database enables a fast, managed failover to the secondary Region, minimizing downtime.
Cost Efficiency: While you're paying for storage and compute in the secondary region, having one DB instance ensures availability while keeping costs down compared to running a fully scaled-out cluster that's idle. This allows the DR site to be readily available without incurring the full cost of an actively used environment.
Other options compared:

MySQL Binary Log Replication (Option A): Requires more manual configuration and management than Aurora Global Database. Setting up and maintaining replication, handling failover, and ensuring data consistency are more complex and prone to errors, increasing operational overhead.
AWS DMS (Option C): While DMS can replicate data, it introduces more overhead than Aurora Global Database, as it's a separate service. It also may not be the most performant option for a high-volume SaaS platform. Additionally, removing the DB instance entirely negates the DR benefit.
Removing DB Instance in Secondary Region (Option B and C): A DR strategy requires at least one instance in the secondary region to provide a failover target. Removing all instances eliminates the immediate failover capability, defeating the purpose of the DR plan.

Therefore, Aurora Global Database with at least one DB instance in the secondary Region strikes the best balance between cost, performance, and ease of management for a robust DR solution.

Question: 339 **Exam Heist**

A company has a custom application with embedded credentials that retrieves information from an Amazon RDS MySQL DB instance. Management says the application must be made more secure with the least amount of programming effort.

What should a solutions architect do to meet these requirements?

A. Use AWS Key Management Service (AWS KMS) to create keys. Configure the application to load the database credentials from AWS KMS. Enable automatic key rotation.

B. Create credentials on the RDS for MySQL database for the application user and store the credentials in AWS Secrets Manager. Configure the application to load the database credentials from Secrets Manager. Create an AWS Lambda function that rotates the credentials in Secret Manager.

C. Create credentials on the RDS for MySQL database for the application user and store the credentials in AWS Secrets Manager. Configure the application to load the database credentials from Secrets Manager. Set up a credentials rotation schedule for the application user in the RDS for MySQL database using Secrets Manager.

D. Create credentials on the RDS for MySQL database for the application user and store the credentials in AWS Systems Manager Parameter Store. Configure the application to load the database credentials from Parameter Store. Set up a credentials rotation schedule for the application user in the RDS for MySQL database using Parameter Store.

Answer: C

Explanation:

The correct answer is C because it provides a secure and efficient solution for managing database credentials with minimal code changes. Here's a detailed justification:

Secrets Manager for credential storage: AWS Secrets Manager is designed specifically for managing sensitive information like database credentials. It offers secure storage and retrieval, making it ideal for this scenario. Storing credentials directly in the application code is a significant security risk.
Reduced programming effort: The solution leverages Secrets Manager's built-in rotation capabilities, minimizing the need for custom code. Options A and B require more custom code to manage the credential rotation process.
RDS integration: Secrets Manager has built-in integration for rotating credentials directly on RDS, simplifying the overall solution.
Automated rotation: The key aspect here is the automated credential rotation. This is crucial for security. By regularly changing the database credentials, the risk of compromised credentials being used maliciously is substantially reduced. Secrets Manager offers this capability, making it a suitable choice.
Parameter Store vs. Secrets Manager: While Systems Manager Parameter Store can store secrets, Secrets Manager is preferred for database credentials due to its built-in rotation features and finer-grained access control specifically designed for sensitive information. Parameter Store is best used to store configuration data that is not sensitive.
KMS considerations: While KMS can encrypt the secrets, it doesn't manage the rotation of those secrets within RDS, requiring much more coding effort. The application would need to retrieve the encrypted secret, decrypt it using KMS, and then update the RDS database with the new credentials. Secrets Manager simplifies this process.

In summary, option C provides a balance between security and ease of implementation by utilizing Secrets Manager's secure storage and automated rotation capabilities, minimizing the need for custom code and meeting the requirement of "least amount of programming effort."

Supporting Links:

AWS Secrets Manager: https://aws.amazon.com/secrets-manager/
Rotate AWS Secrets Manager secrets:
https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotating-secrets.html
AWS Systems Manager Parameter Store: https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html

Question: 340 **Exam Heist**

A media company hosts its website on AWS. The website application's architecture includes a fleet of Amazon EC2 instances behind an Application Load Balancer (ALB) and a database that is hosted on Amazon Aurora. The company's cybersecurity team reports that the application is vulnerable to SQL injection.

How should the company resolve this issue?

A. Use AWS WAF in front of the ALB. Associate the appropriate web ACLs with AWS WAF.
B. Create an ALB listener rule to reply to SQL injections with a fixed response.
C. Subscribe to AWS Shield Advanced to block all SQL injection attempts automatically.
D. Set up Amazon Inspector to block all SQL injection attempts automatically.

Answer: A

Explanation:

The correct answer is A: Use AWS WAF in front of the ALB. Associate the appropriate web ACLs with AWS WAF.

Here's a detailed justification:

SQL injection is a code injection technique used to attack data-driven applications, in which malicious SQL statements are inserted into an entry field for execution. This allows attackers to potentially bypass security measures and gain unauthorized access to the database.

AWS WAF (Web Application Firewall) is a web application firewall that helps protect web applications from common web exploits and bots that may affect availability, compromise security, or consume excessive resources. It allows you to control access to your content by defining customizable web security rules. These rules can filter out malicious traffic patterns, including those commonly associated with SQL injection attacks.

Option A effectively leverages AWS WAF. By placing AWS WAF in front of the Application Load Balancer (ALB), all incoming traffic is inspected against the defined web ACLs (Access Control Lists). These web ACLs contain rules designed to identify and block SQL injection attempts. WAF can inspect HTTP headers, request body, and URI strings for malicious SQL code. By blocking these requests before they reach the application, WAF prevents the SQL injection vulnerability from being exploited.

Option B, creating an ALB listener rule to reply with a fixed response, is not a suitable solution. While it might prevent immediate execution of the SQL injection, it doesn't address the underlying vulnerability and could lead to application malfunction and denial of service. It's a reactive measure, not a preventative one. It also provides limited protection against evolving attack vectors.

Option C, subscribing to AWS Shield Advanced, mainly protects against DDoS attacks, not specific application-level vulnerabilities like SQL injection. While Shield Advanced offers visibility and mitigation assistance, it's not a direct solution to the SQL injection problem.

Option D, using Amazon Inspector, is a vulnerability management service that automatically assesses applications for security vulnerabilities or deviations from best practices. While Inspector can identify the presence of the SQL injection vulnerability, it does not automatically block the attacks in real-time. Inspector

would generate findings but wouldn't prevent the attacks. You'd still need a mechanism to block the attacks based on Inspector's findings.

Therefore, the most appropriate and effective solution is to use AWS WAF with appropriate web ACLs, as this provides real-time protection against SQL injection attacks by inspecting and filtering malicious traffic before it reaches the application.

Authoritative Links:

AWS WAF: https://aws.amazon.com/waf/
OWASP SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/

Question: 341 **Exam Heist**

A company has an Amazon S3 data lake that is governed by AWS Lake Formation. The company wants to create a visualization in Amazon QuickSight by joining the data in the data lake with operational data that is stored in an Amazon Aurora MySQL database. The company wants to enforce column-level authorization so that the company's marketing team can access only a subset of columns in the database.

Which solution will meet these requirements with the LEAST operational overhead?

A. Use Amazon EMR to ingest the data directly from the database to the QuickSight SPICE engine. Include only the required columns.

B. Use AWS Glue Studio to ingest the data from the database to the S3 data lake. Attach an IAM policy to the QuickSight users to enforce column-level access control. Use Amazon S3 as the data source in QuickSight.

C. Use AWS Glue Elastic Views to create a materialized view for the database in Amazon S3. Create an S3 bucket policy to enforce column-level access control for the QuickSight users. Use Amazon S3 as the data source in QuickSight.

D. Use a Lake Formation blueprint to ingest the data from the database to the S3 data lake. Use Lake Formation to enforce column-level access control for the QuickSight users. Use Amazon Athena as the data source in QuickSight.

Answer: D

Explanation:

The correct answer is **D** because it leverages Lake Formation's built-in capabilities for data governance and access control, minimizing operational overhead.

Here's a detailed justification:

Requirement: Join data from S3 data lake and Aurora MySQL for visualization in QuickSight with column-level authorization for the marketing team.

Why D is the best solution:

Lake Formation Blueprint for Ingestion: Lake Formation blueprints automate the process of ingesting data from various sources, including relational databases like Aurora MySQL, into the S3 data lake. This simplifies the data loading process compared to manual ETL jobs using EMR or Glue Studio. ^1^
Lake Formation for Column-Level Access Control: Lake Formation allows you to define fine-grained access controls, including column-level permissions. This enables the marketing team to access only the columns they need in both the data lake and the ingested Aurora data. ^2^
Athena as the Data Source: Athena integrates seamlessly with Lake Formation, enabling it to enforce the defined access controls. When users query the data through Athena, Lake Formation automatically filters the data based on their permissions. Athena is a serverless query service that provides a straightforward way to analyze data in S3. ^3^

Why other options are less optimal:

A (EMR to QuickSight SPICE): This approach bypasses the data lake and Lake Formation. It's less scalable, and governance is managed within the SPICE dataset, increasing operational overhead. Additionally, SPICE has limitations in terms of data volume and complexity compared to querying directly from the data lake via Athena.
B (Glue Studio to S3, IAM Policies): While Glue Studio can ingest data, using IAM policies for column-level access control is complex and error-prone. It becomes difficult to manage permissions across different users and data sources, especially at scale. This approach also misses the advantage of using Lake Formation's central governance.
C (Glue Elastic Views, S3 Bucket Policy): Glue Elastic Views are generally for real-time data propagation, which isn't the primary requirement here. Using S3 bucket policies for column-level access control is not possible, as bucket policies only control access at the object level, not within the object. This approach also doesn't leverage the governance benefits of Lake Formation.

In summary, leveraging Lake Formation's built-in capabilities for data ingestion, governance, and access control provides the most efficient and scalable solution for this scenario.

Question: 342 **Exam Heist**

A transaction processing company has weekly scripted batch jobs that run on Amazon EC2 instances. The EC2 instances are in an Auto Scaling group. The number of transactions can vary, but the baseline CPU utilization that is noted on each run is at least 60%. The company needs to provision the capacity 30 minutes before the jobs run.

Currently, engineers complete this task by manually modifying the Auto Scaling group parameters. The company does not have the resources to analyze the required capacity trends for the Auto Scaling group counts. The company needs an automated way to modify the Auto Scaling group's desired capacity.

Which solution will meet these requirements with the LEAST operational overhead?

A. Create a dynamic scaling policy for the Auto Scaling group. Configure the policy to scale based on the CPU utilization metric. Set the target value for the metric to 60%.

B. Create a scheduled scaling policy for the Auto Scaling group. Set the appropriate desired capacity, minimum capacity, and maximum capacity. Set the recurrence to weekly. Set the start time to 30 minutes before the batch jobs run.

C. Create a predictive scaling policy for the Auto Scaling group. Configure the policy to scale based on forecast. Set the scaling metric to CPU utilization. Set the target value for the metric to 60%. In the policy, set the instances to pre-launch 30 minutes before the jobs run.

D. Create an Amazon EventBridge event to invoke an AWS Lambda function when the CPU utilization metric value for the Auto Scaling group reaches 60%. Configure the Lambda function to increase the Auto Scaling group's desired capacity and maximum capacity by 20%.

Answer: C

Explanation:

The correct answer is **C: Create a predictive scaling policy for the Auto Scaling group. Configure the policy to scale based on forecast. Set the scaling metric to CPU utilization. Set the target value for the metric to 60%. In the policy, set the instances to pre-launch 30 minutes before the jobs run.**

Here's a detailed justification:

The core requirement is to automatically adjust the Auto Scaling group's desired capacity 30 minutes before the batch jobs start, based on anticipated load. Given that the company lacks the resources for detailed capacity trend analysis, predictive scaling offers the most suitable and automated solution.

Predictive Scaling: Amazon EC2 Auto Scaling's predictive scaling analyzes historical CPU utilization trends (which are consistently above 60%). It forecasts future capacity needs and proactively adjusts the Auto Scaling group. This addresses the need for capacity 30 minutes ahead of time, as specified in the problem. By using forecast data, the solution automatically determines the desired capacity without manual calculations. This approach reduces operational overhead.

CPU Utilization Metric: The problem states that the baseline CPU utilization during the batch jobs is at least 60%. Predictive scaling can leverage CPU utilization as a metric to learn and forecast the required capacity.

Pre-launch Instances: The ability to pre-launch instances 30 minutes before the jobs run is a crucial feature of the predictive scaling policy that directly addresses the requirement.

Let's analyze why the other options are less ideal:

A (Dynamic Scaling): Dynamic scaling responds to current CPU utilization. While it can maintain a target CPU of 60%, it won't provision capacity before the load increases. This introduces a delay, which is not acceptable as the jobs require the capacity before they start.

B (Scheduled Scaling): Scheduled scaling relies on fixed schedules. While you can set a weekly schedule to increase capacity 30 minutes before the jobs run, it doesn't account for variations in transaction volume. It requires a good understanding of the required capacity, and therefore would require manual intervention and trend analysis, which the company wishes to avoid. Furthermore, the problem states they do not have resources for capacity analysis.

D (EventBridge & Lambda): Using EventBridge and Lambda introduces unnecessary complexity. It would require setting up complex rules based on real-time CPU utilization metrics, and then programming Lambda to adjust the Auto Scaling group. This increases operational overhead compared to the built-in predictive scaling functionality. It suffers from the same delay issue as Option A - it reacts to the 60% threshold being crossed rather than proactively scaling.

Authoritative Links:

Amazon EC2 Auto Scaling Predictive Scaling: https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-predictive-scaling.html
Dynamic Scaling vs Predictive Scaling: https://aws.amazon.com/blogs/compute/introducing-predictive-scaling-for-amazon-ec2-auto-scaling/

Question: 343 **Exam Heist**

A solutions architect is designing a company’s disaster recovery (DR) architecture. The company has a MySQL database that runs on an Amazon EC2 instance in a private subnet with scheduled backup. The DR design needs to include multiple AWS Regions.

Which solution will meet these requirements with the LEAST operational overhead?

A. Migrate the MySQL database to multiple EC2 instances. Configure a standby EC2 instance in the DR Region. Turn on replication.

B. Migrate the MySQL database to Amazon RDS. Use a Multi-AZ deployment. Turn on read replication for the primary DB instance in the different Availability Zones.

C. Migrate the MySQL database to an Amazon Aurora global database. Host the primary DB cluster in the primary Region. Host the secondary DB cluster in the DR Region.

D. Store the scheduled backup of the MySQL database in an Amazon S3 bucket that is configured for S3 Cross-Region Replication (CRR). Use the data backup to restore the database in the DR Region.

Answer: C

Explanation:

The correct answer is C. Migrating the MySQL database to Amazon Aurora Global Database offers the lowest operational overhead while meeting the disaster recovery requirements of multi-Region support and minimal management. Aurora Global Database is designed for DR scenarios, providing fast recovery with minimal data loss.

Here's why:

Aurora Global Database: This feature allows you to replicate your Aurora MySQL database to multiple AWS Regions. The primary region handles writes, and the secondary region(s) are read-only replicas with low latency replication. In the event of a primary region failure, a secondary region can be promoted to become the new primary, minimizing downtime and data loss. https://aws.amazon.com/rds/aurora/features/global-database/

Multi-Region DR with minimal overhead: Aurora Global Database automates much of the replication and failover process, significantly reducing the operational burden compared to manual replication setups.

Here's why the other options are less suitable:

Option A (EC2 replication): Setting up MySQL replication on EC2 instances requires manual configuration, monitoring, and maintenance of replication. Failover requires significant manual intervention, increasing operational overhead. This method also doesn't offer as robust DR capabilities as Aurora Global Database.

Option B (RDS Multi-AZ with read replicas): RDS Multi-AZ provides high availability within a single region. While read replicas can exist in different Availability Zones, they aren't designed for cross-region DR with automated failover. Manual intervention is required for DR failover, and RPO/RTO are higher than with Aurora Global Database. https://aws.amazon.com/rds/mysql/features/

Option D (S3 CRR): While S3 Cross-Region Replication can replicate backups to another region, restoring from backups is a slow and manual process. This significantly increases the Recovery Time Objective (RTO) and Recovery Point Objective (RPO) compared to Aurora Global Database, which is designed for rapid failover. Restoring from backups also involves additional steps like provisioning a new EC2 instance, installing MySQL, and restoring the backup, adding to the operational overhead. https://aws.amazon.com/s3/features/

In summary, Aurora Global Database provides the most efficient and automated solution for multi-region DR for a MySQL database, aligning with the requirement of "least operational overhead."

Question: 344 **Exam Heist**

A company has a Java application that uses Amazon Simple Queue Service (Amazon SQS) to parse messages. The application cannot parse messages that are larger than 256 KB in size. The company wants to implement a solution to give the application the ability to parse messages as large as 50 MB.

Which solution will meet these requirements with the FEWEST changes to the code?

A. Use the Amazon SQS Extended Client Library for Java to host messages that are larger than 256 KB in Amazon S3.

B. Use Amazon EventBridge to post large messages from the application instead of Amazon SQS.

C. Change the limit in Amazon SQS to handle messages that are larger than 256 KB.

D. Store messages that are larger than 256 KB in Amazon Elastic File System (Amazon EFS). Configure Amazon SQS to reference this location in the messages.

Answer: A

Explanation:

The correct answer is A: "Use the Amazon SQS Extended Client Library for Java to host messages that are larger than 256 KB in Amazon S3."

Here's why:

Problem Statement: The application's limitation is parsing SQS messages larger than 256KB. The goal is to handle up to 50MB with minimal code changes.

Why Option A is Best: The Amazon SQS Extended Client Library for Java is designed specifically for handling large messages exceeding the SQS size limit. It stores the large message payload in Amazon S3 and stores a reference to the S3 object in the SQS message. This approach allows SQS to act as a pointer to the actual message content. Because the library handles the complexity of interacting with S3, the code changes required in the application will be minimal.

Why Other Options Are Not Optimal:

B (Amazon EventBridge): While EventBridge is useful, it's designed for event-driven architectures. Substituting SQS with EventBridge would necessitate a significant rewrite of the existing application, deviating from the "fewest changes" requirement. Furthermore, EventBridge has a smaller message size limit than SQS with the Extended Client Library.

C (Changing SQS Limit): SQS has a hard limit of 256KB for message size. You cannot simply change this limit.

D (Amazon EFS): Storing large messages in EFS and referencing them from SQS would require significant custom coding for managing files in EFS, handling file paths in SQS messages, ensuring proper access control and data consistency, and managing cleanup of old files. This is more complex and less efficient than the provided Extended Client Library, and EFS isn't the appropriate service for message storage as it adds unnecessary complexity.

Key Concepts:

Amazon SQS Size Limit: SQS has a message size limit of 256KB.

Amazon SQS Extended Client Library: A pre-built solution to extend SQS's functionality to support larger messages by using Amazon S3 as storage.

Loose Coupling: The solution maintains loose coupling between the message queue (SQS) and the actual message content (stored in S3).

Authoritative Links:

Amazon SQS Extended Client Library for Java
Amazon SQS Message Size and Quotas

In summary, the SQS Extended Client Library provides a straightforward and efficient way to handle large messages with the fewest code modifications. It leverages S3 for storage and keeps the application's dependency on SQS while enabling the processing of messages up to 50MB.

Question: 345 **Exam Heist**

A company wants to restrict access to the content of one of its main web applications and to protect the content by using authorization techniques available on AWS. The company wants to implement a serverless architecture and an authentication solution for fewer than 100 users. The solution needs to integrate with the main web application and serve web content globally. The solution must also scale as the company's user base grows while providing the lowest login latency possible.

Which solution will meet these requirements MOST cost-effectively?

A. Use Amazon Cognito for authentication. Use for authorization. Use Amazon CloudFront to serve the web application globally.

B. Use AWS Directory Service for Microsoft Active Directory for authentication. Use AWS Lambda for authorization. Use an Application Load Balancer to serve the web application globally.

C. Use Amazon Cognito for authentication. Use AWS Lambda for authorization. Use Amazon S3 Transfer Acceleration to serve the web application globally.

D. Use AWS Directory Service for Microsoft Active Directory for authentication. Use for authorization. Use AWS Elastic Beanstalk to serve the web application globally.

Answer: A

Explanation:

The best solution is A because it utilizes cost-effective and scalable AWS services suitable for a small user base and global content delivery. Amazon Cognito is a managed service specifically designed for user authentication, providing features like user registration, sign-in, and access control, and it's a cost-effective option for fewer than 100 users. , a serverless authorization service that seamlessly integrates with Cognito, allows fine-grained access control to web application resources. CloudFront, a global content delivery network (CDN), is ideal for distributing web content with low latency worldwide.

Option B is less optimal due to the cost associated with AWS Directory Service for Microsoft Active Directory, which is more suitable for larger organizations with existing Active Directory infrastructure. Furthermore, using Lambda for authorization adds unnecessary complexity compared to the purpose-built . An Application Load Balancer is generally used for distributing traffic across multiple instances, which is not necessary when serving static content globally through CloudFront.

Option C is less efficient because Amazon S3 Transfer Acceleration is for accelerating uploads to S3, not for serving web content globally with low latency, which is CloudFront's primary function.

Option D combines the costly AWS Directory Service with Elastic Beanstalk, which is a platform-as-a-service (PaaS) that manages the underlying infrastructure. Elastic Beanstalk introduces complexities that are unnecessary for a serverless architecture focused on serving static web content globally. is effective for authorization, but the underlying authentication and delivery are less optimal.

Therefore, Cognito provides the most cost-effective authentication, provides scalable and serverless authorization, and CloudFront offers global content delivery with low latency, making option A the best choice.

Supporting Links:

Amazon Cognito: https://aws.amazon.com/cognito/
: [https://aws.amazon.com/blogs/security/-fine-grained-authorization-service-serverless-applications/](https://aws.amazon.com/blogs/security/-fine-grained-authorization-service-serverless-applications/)
Amazon CloudFront: https://aws.amazon.com/cloudfront/

Question: 346 **Exam Heist**

A company has an aging network-attached storage (NAS) array in its data center. The NAS array presents SMB shares and NFS shares to client workstations. The company does not want to purchase a new NAS array. The company also does not want to incur the cost of renewing the NAS array’s support contract. Some of the data is accessed frequently, but much of the data is inactive.

A solutions architect needs to implement a solution that migrates the data to Amazon S3, uses S3 Lifecycle policies, and maintains the same look and feel for the client workstations. The solutions architect has identified AWS Storage Gateway as part of the solution.

Which type of storage gateway should the solutions architect provision to meet these requirements?

A. Volume Gateway
B. Tape Gateway
C. Amazon FSx File Gateway
D. Amazon S3 File Gateway

Answer: D

Explanation:

The correct answer is **D. Amazon S3 File Gateway**.

Here's a detailed justification:

The primary requirements are to migrate data to Amazon S3 while maintaining the same look and feel (SMB/NFS shares) for client workstations and implementing S3 Lifecycle policies for tiered storage. The key here is preserving the file share access method.

Amazon S3 File Gateway allows on-premises applications to access data in Amazon S3 through standard file protocols like NFS and SMB. This fulfills the requirement of maintaining the same look and feel for the client workstations because they continue to interact with file shares. It effectively presents S3 as a network file share. Moreover, data written to the File Gateway is stored as objects in S3, which allows leveraging S3 Lifecycle policies for managing data tiers (frequently accessed vs. inactive data).

Volume Gateway presents block-based storage to on-premises applications, which isn't suitable for directly replacing a file-based NAS. It replicates on-premises block storage volumes to AWS, but it does not provide file share access (SMB/NFS).

Tape Gateway is designed for backing up data to virtual tapes stored in AWS, primarily for archival purposes, not for active file storage and access. It does not expose fileshares to workstations.

Amazon FSx File Gateway is used for accessing fully managed file systems in AWS, like Amazon FSx for Windows File Server or Amazon FSx for Lustre, and doesn't directly help in migrating and exposing existing NAS data in S3 through SMB/NFS shares.

Therefore, Amazon S3 File Gateway is the best solution because it allows the company to migrate their file data to S3, preserve existing SMB/NFS access, and manage storage costs with S3 Lifecycle policies.

Authoritative Links:

AWS Storage Gateway: https://aws.amazon.com/storagegateway/
File Gateway: https://docs.aws.amazon.com/storagegateway/latest/userguide/WhatIsStorageGateway.html
S3 Lifecycle Policies: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-examples.html

Question: 347 **Exam Heist**

A company has an application that is running on Amazon EC2 instances. A solutions architect has standardized the company on a particular instance family and various instance sizes based on the current needs of the company.

The company wants to maximize cost savings for the application over the next 3 years. The company needs to be able to change the instance family and sizes in the next 6 months based on application popularity and usage.

Which solution will meet these requirements MOST cost-effectively?

A. Compute Savings Plan
B. EC2 Instance Savings Plan
C. Zonal Reserved Instances
D. Standard Reserved Instances

Answer: A

Explanation:

The correct answer is A, Compute Savings Plan. Here's why:

Compute Savings Plans offer the most flexibility and cost savings for this scenario because they automatically apply to EC2 instance usage regardless of instance family, size, operating system, or tenancy within a specified compute amount per hour. This is crucial because the company anticipates changing instance families and sizes within the next 6 months due to application popularity. EC2 Instance Savings Plans, while providing savings on specific instance families within a region, lack the needed flexibility. They are tied to a particular instance family and size, making them unsuitable if those parameters need to change.

Reserved Instances (RIs), both Zonal and Standard, offer significant cost savings compared to On-Demand Instances. However, Standard RIs are less flexible in terms of AZ placement compared to Zonal RIs. While RIs offer savings, they require committing to a specific instance type and availability zone, which contradicts the requirement of changing instance families. Further, modifying RIs after purchase can be complex and may incur additional costs. Compute Savings Plans provide more cost optimization in the long run when instance types are subject to change.

Since the company anticipates changes, Compute Savings Plans offer the best balance between cost savings and flexibility. This allows them to optimize costs over the 3-year term while adapting to evolving application needs without incurring penalties or wasted investments.

Further reading:

AWS Savings Plans: https://aws.amazon.com/savingsplans/
AWS Reserved Instances: https://aws.amazon.com/ec2/pricing/reserved-instances/

Question: 348 **Exam Heist**

A company collects data from a large number of participants who use wearable devices. The company stores the data in an Amazon DynamoDB table and uses applications to analyze the data. The data workload is constant and predictable. The company wants to stay at or below its forecasted budget for DynamoDB.

Which solution will meet these requirements MOST cost-effectively?

A. Use provisioned mode and DynamoDB Standard-Infrequent Access (DynamoDB Standard-IA). Reserve capacity for the forecasted workload.
B. Use provisioned mode. Specify the read capacity units (RCUs) and write capacity units (WCUs).
C. Use on-demand mode. Set the read capacity units (RCUs) and write capacity units (WCUs) high enough to accommodate changes in the workload.
D. Use on-demand mode. Specify the read capacity units (RCUs) and write capacity units (WCUs) with reserved capacity.

Answer: B

Explanation:

The correct answer is B: Use provisioned mode. Specify the read capacity units (RCUs) and write capacity units (WCUs).

Here's why:

The key requirements are cost-effectiveness and a predictable workload. Provisioned mode is ideal for predictable workloads because you can specify the read and write capacity units (RCUs and WCUs) needed to handle the anticipated traffic. This allows you to closely control and optimize costs.

Option A is incorrect because DynamoDB Standard-IA is optimized for infrequently accessed data, which is not specified in the use case. Standard-IA storage is more expensive for frequently accessed data, and the problem states that the data is being analyzed. While reserving capacity within provisioned mode is good, suggesting Standard-IA storage when there's no indication of infrequent access makes this a less ideal choice.

Option C is incorrect. On-demand mode is best suited for unpredictable workloads with varying traffic patterns. While it eliminates the need to manage capacity, it is generally more expensive than provisioned mode for predictable workloads because you pay for each read and write request. Setting RCUs and WCUs in on-demand mode contradicts the very nature of on-demand capacity.

Option D is incorrect because you cannot "specify" RCUs and WCUs along with "reserved capacity" in on-demand mode. On-demand mode automatically scales capacity, and you don't have control over setting specific RCUs/WCUs in that mode. Furthermore, suggesting reserved capacity in on-demand mode mixes provisioned and on-demand concepts, which is not directly supported. On-demand pricing covers the scaling.

Therefore, provisioned mode with specified RCUs and WCUs allows the company to precisely allocate the resources needed for the constant and predictable data workload, ensuring the most cost-effective solution that stays within the forecasted budget.Here are some authoritative links for further research:

DynamoDB Pricing: https://aws.amazon.com/dynamodb/pricing/ - Review the pricing models for provisioned and on-demand capacity to understand cost implications.
Choosing Between On-Demand and Provisioned Capacity: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadWriteCapacityMode.html - AWS documentation outlining the differences and use cases for each capacity mode.
DynamoDB Standard and Standard-IA: https://aws.amazon.com/dynamodb/pricing/storage/ - Detailed explanation of DynamoDB storage types and their appropriate use cases.

Question: 349 **Exam Heist**

A company stores confidential data in an Amazon Aurora PostgreSQL database in the ap-southeast-3 Region. The database is encrypted with an AWS Key Management Service (AWS KMS) customer managed key. The company was recently acquired and must securely share a backup of the database with the acquiring company's AWS account in ap-southeast-3.

What should a solutions architect do to meet these requirements?

A. Create a database snapshot. Copy the snapshot to a new unencrypted snapshot. Share the new snapshot with the acquiring company's AWS account.

B. Create a database snapshot. Add the acquiring company's AWS account to the KMS key policy. Share the snapshot with the acquiring company's AWS account.

C. Create a database snapshot that uses a different AWS managed KMS key. Add the acquiring company's AWS account to the KMS key alias. Share the snapshot with the acquiring company's AWS account.

D. Create a database snapshot. Download the database snapshot. Upload the database snapshot to an Amazon S3 bucket. Update the S3 bucket policy to allow access from the acquiring company's AWS account.

Answer: B

Explanation:

The correct answer is **B**. Here's a detailed justification:

The core requirement is to securely share an encrypted Aurora PostgreSQL database backup (snapshot) across AWS accounts while maintaining encryption using KMS. Option B directly addresses this.

1. **Creating a Database Snapshot:** The first step is to create a snapshot of the Aurora PostgreSQL database. This is the standard method for backing up an Aurora database. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_CreateSnapshot.html
2. **Adding the Acquiring Company's AWS Account to the KMS Key Policy:** The database is encrypted with a KMS customer-managed key (CMK). To allow the acquiring company to restore and use the snapshot, their AWS account needs permission to use the CMK. This is accomplished by adding the acquiring company's AWS account ID as a principal to the CMK's key policy. This grants the acquiring company's account the necessary permissions (e.g., kms:Decrypt, kms:GenerateDataKey) to decrypt the snapshot and perform restore operations. https://docs.aws.amazon.com/kms/latest/developerguide/key-policies.html
3. **Sharing the Snapshot:** Once the acquiring company's account has the necessary KMS permissions, the snapshot can be shared with their AWS account. The receiving account can then create a new Aurora cluster from the shared snapshot. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ShareSnapshot.html

Why other options are incorrect:

A: Creating an unencrypted snapshot defeats the purpose of securely sharing the data. The requirement specifies securely sharing the data, implying encryption should be maintained.
C: Using an AWS-managed KMS key would be simpler initially, but it wouldn't satisfy the requirement of using the existing customer-managed key, and you will need to give permissions to use the key anyway. Key aliases do not control permissions; key policies do.
D: Downloading and uploading the snapshot via S3 exposes the database backup to potential security risks if the S3 bucket is misconfigured. The process would also be more complex and time-consuming compared to sharing the snapshot directly. While S3 can be secure, this method introduces unnecessary complexity and a larger attack surface compared to the direct snapshot sharing method with KMS key policy modification. Furthermore, you still need KMS access in the new account to re-encrypt if not already done so.

Question: 350 **Exam Heist**

A company uses a 100 GB Amazon RDS for Microsoft SQL Server Single-AZ DB instance in the us-east-1 Region to store customer transactions. The company needs high availability and automatic recovery for the DB instance.

The company must also run reports on the RDS database several times a year. The report process causes transactions to take longer than usual to post to the customers' accounts. The company needs a solution that will improve the performance of the report process.

Which combination of steps will meet these requirements? (Choose two.)

A. Modify the DB instance from a Single-AZ DB instance to a Multi-AZ deployment.

B. Take a snapshot of the current DB instance. Restore the snapshot to a new RDS deployment in another Availability Zone.

C. Create a read replica of the DB instance in a different Availability Zone. Point all requests for reports to the read replica.

D. Migrate the database to RDS Custom.

E. Use RDS Proxy to limit reporting requests to the maintenance window.

Answer: AC

Explanation:

Let's analyze why options A and C are the correct choices for achieving high availability, automatic recovery, and improved reporting performance for the company's Amazon RDS for Microsoft SQL Server database.

A. Modify the DB instance from a Single-AZ DB instance to a Multi-AZ deployment. This addresses the high availability and automatic recovery requirements. A Multi-AZ deployment automatically provisions a standby replica of the database in a different Availability Zone. In case of an infrastructure failure in the primary AZ, RDS automatically fails over to the standby replica, minimizing downtime and ensuring continuous operation. Single-AZ instances do not offer automatic failover, while Multi-AZ does. This configuration ensures the database remains accessible even during planned maintenance or unexpected outages. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html

C. Create a read replica of the DB instance in a different Availability Zone. Point all requests for reports to the read replica. This addresses the reporting performance issue without impacting the performance of the primary database handling customer transactions. Read replicas allow you to offload read-only workloads, such as report generation, to a separate database instance. By directing the report process to the read replica, the primary database remains dedicated to transaction processing, preventing performance degradation. This separation of concerns significantly improves the overall performance of both transaction posting and report generation. Read replicas can be in a different AZ than the primary. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.ReadReplicas.html

Let's examine why the other options are not ideal:

B. Take a snapshot of the current DB instance. Restore the snapshot to a new RDS deployment in another Availability Zone. While this creates a copy of the database, it's a one-time operation and doesn't provide continuous, near real-time data for reporting. It doesn't address the high availability requirement for the primary database. The restored instance would be stale and out of sync.
D. Migrate the database to RDS Custom. RDS Custom provides more control over the operating system and database environment, but it also increases operational overhead and complexity. It is unnecessary for the stated requirements of high availability, automatic recovery, and improved reporting performance, adding unneeded management overhead.
E. Use RDS Proxy to limit reporting requests to the maintenance window. RDS Proxy primarily manages database connections and improves application scalability. While it can help with managing connection resources, it doesn't directly address the performance impact of report generation on the primary database or the need for high availability. It would delay reporting, not improve its performance.

Question: 351 **Exam Heist**

A company is moving its data management application to AWS. The company wants to transition to an event-driven architecture. The architecture needs to be more distributed and to use serverless concepts while performing the different aspects of the workflow. The company also wants to minimize operational overhead.

Which solution will meet these requirements?

A. Build out the workflow in AWS Glue. Use AWS Glue to invoke AWS Lambda functions to process the workflow steps.

B. Build out the workflow in AWS Step Functions. Deploy the application on Amazon EC2 instances. Use Step Functions to invoke the workflow steps on the EC2 instances.

C. Build out the workflow in Amazon EventBridge. Use EventBridge to invoke AWS Lambda functions on a schedule to process the workflow steps.

D. Build out the workflow in AWS Step Functions. Use Step Functions to create a state machine. Use the state machine to invoke AWS Lambda functions to process the workflow steps.

Answer: D

Explanation:

The correct answer is D because it leverages AWS Step Functions and Lambda, which are key serverless components ideal for building event-driven architectures with minimal operational overhead.

Here's why:

Step Functions: AWS Step Functions allows you to define workflows as state machines. This is crucial for managing the data management application's workflow, breaking it down into discrete, manageable steps. The visual workflow makes it easier to understand, monitor, and maintain the entire process. (https://aws.amazon.com/step-functions/)
Lambda: AWS Lambda provides serverless compute capabilities. By using Lambda functions for each step in the workflow, you eliminate the need to manage servers or containers, reducing operational overhead significantly. Lambda functions execute only when triggered, further optimizing resource utilization and cost. (https://aws.amazon.com/lambda/)
Event-Driven Architecture: Step Functions facilitates event-driven behavior by triggering different Lambda functions based on the outcome of previous steps. This asynchronous execution ensures that the application is highly responsive and scalable.

Why the other options are less suitable:

A (AWS Glue): While AWS Glue can perform ETL tasks, it's primarily designed for data integration and preparation, not for orchestrating a general-purpose event-driven workflow. It also can have an overhead on smaller tasks.
B (Step Functions + EC2): Deploying the application on EC2 instances introduces the operational overhead of managing servers, which contradicts the requirement of minimizing operational overhead and adopting serverless concepts.
C (EventBridge + Lambda on a Schedule): Amazon EventBridge is excellent for routing events between services, but scheduling Lambda functions directly doesn't provide the sophisticated workflow orchestration capabilities offered by Step Functions. A schedule lacks the necessary state management and error handling.

In summary, Step Functions provides the workflow orchestration, Lambda provides the serverless compute, fulfilling the need for a distributed, serverless, and event-driven architecture with minimal operational overhead.

Question: 352 **Exam Heist**

A company is designing the network for an online multi-player game. The game uses the UDP networking protocol and will be deployed in eight AWS Regions. The network architecture needs to minimize latency and packet loss to give end users a high-quality gaming experience.

Which solution will meet these requirements?

A. Setup a transit gateway in each Region. Create inter-Region peering attachments between each transit gateway.

B. Set up AWS Global Accelerator with UDP listeners and endpoint groups in each Region.

C. Set up Amazon CloudFront with UDP turned on. Configure an origin in each Region.

D. Set up a VPC peering mesh between each Region. Turn on UDP for each VPC.

Answer: B

Explanation:

Here's a detailed justification for why option B (Set up AWS Global Accelerator with UDP listeners and endpoint groups in each Region) is the best solution for minimizing latency and packet loss in a multi-region, UDP-based online game, and why the other options are less suitable:

Why Option B is Correct: AWS Global Accelerator

AWS Global Accelerator is specifically designed to improve the availability and performance of applications for a global user base. It leverages the AWS global network infrastructure to route traffic to the optimal endpoint based on user location, network health, and endpoint health. It uses Anycast static IPs that serve as a fixed entry point to applications, which minimizes DNS resolution time and offers quick failover to healthy endpoints.

UDP Support: Global Accelerator supports UDP, which is crucial because the game uses the UDP protocol. UDP is preferred for online gaming because it's connectionless, leading to lower latency compared to TCP.
Reduced Latency: Global Accelerator intelligently routes traffic over the AWS global network, minimizing latency.
Improved Reliability: Global Accelerator constantly monitors the health of your application endpoints and only directs traffic to healthy endpoints, thus reducing packet loss and creating a smoother gaming experience. Its failover mechanism automatically reroutes traffic to the nearest available endpoint if one fails.
Regional Deployment: By setting up endpoint groups in each region, you ensure players are routed to the nearest available server, further reducing latency and optimizing their gameplay experience.

Why Other Options are Incorrect:

A. Transit Gateway with Inter-Region Peering: While transit gateways simplify network management, inter-region peering through transit gateways does not inherently optimize latency or packet loss for real-time applications like online games. Transit Gateway adds an additional hop. It's also not optimized to specifically handle UDP traffic with the low latency requirements of gaming applications.
C. Amazon CloudFront with UDP: CloudFront primarily focuses on caching content closer to users for efficient content delivery. Although CloudFront supports UDP for some use cases, it's not optimized for dynamic, bi-directional, real-time UDP traffic required in online gaming. It's better suited for video streaming and file downloads. The core focus of CloudFront is content delivery, not low-latency communication.
D. VPC Peering Mesh: A full mesh of VPC peering connections between eight regions results in a complex and difficult-to-manage network. While VPC peering establishes connectivity, it doesn't provide intelligent routing or optimization for latency and packet loss across regions. It also requires manual management of routing tables in each VPC. Also, UDP support on its own does not address packet loss reduction or intelligent routing for latency.

Authoritative Links:

AWS Global Accelerator: https://aws.amazon.com/global-accelerator/
AWS Transit Gateway: https://aws.amazon.com/transit-gateway/
Amazon CloudFront: https://aws.amazon.com/cloudfront/
VPC Peering: https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html

Question: 353 **Exam Heist**

A company hosts a three-tier web application on Amazon EC2 instances in a single Availability Zone. The web application uses a self-managed MySQL database that is hosted on an EC2 instance to store data in an Amazon Elastic Block Store (Amazon EBS) volume. The MySQL database currently uses a 1 TB Provisioned IOPS SSD (io2) EBS volume. The company expects traffic of 1,000 IOPS for both reads and writes at peak traffic.

The company wants to minimize any disruptions, stabilize performance, and reduce costs while retaining the capacity for double the IOPS. The company wants to move the database tier to a fully managed solution that is highly available and fault tolerant.

Which solution will meet these requirements MOST cost-effectively?

A.Use a Multi-AZ deployment of an Amazon RDS for MySQL DB instance with an io2 Block Express EBS volume.

B.Use a Multi-AZ deployment of an Amazon RDS for MySQL DB instance with a General Purpose SSD (gp2) EBS volume.

C.Use Amazon S3 Intelligent-Tiering access tiers.

D.Use two large EC2 instances to host the database in active-passive mode.

Answer: B

Explanation:

The best solution is to use a Multi-AZ deployment of an Amazon RDS for MySQL DB instance with a General Purpose SSD (gp2) EBS volume.

Here's why:

Requirement fulfillment: This solution satisfies the core requirements of high availability, fault tolerance, and cost reduction while moving to a fully managed service.
Fully managed solution: Amazon RDS is a fully managed database service, eliminating the operational overhead of managing the database server.
High availability and fault tolerance: A Multi-AZ deployment in RDS provides redundancy. If the primary instance fails, RDS automatically fails over to the standby instance in another Availability Zone, minimizing downtime.
Performance: While io2 EBS volume provides consistent performance, the requirement is for 1000 IOPS (with a peak of 2000 IOPS). The gp2 volume often is enough for this IOPS level. Besides, a gp2 EBS volume with a large enough size (exceeding 334GB) is capable of delivering performance beyond 1000 IOPS. Additionally, RDS instances can also be sized up to meet additional performance requirements.
Cost optimization: gp2 volumes are generally more cost-effective than io2 volumes. This helps to reduce costs without sacrificing performance for the specified workload.
Scalability: RDS allows easy scaling of the database instance and storage as needed.
Why other options are less ideal:

Option A (io2 Block Express EBS): io2 Block Express offers higher IOPS and throughput, but it's more expensive than gp2. The company expects traffic of only 1,000 IOPS (and up to 2000 IOPS), which can be accommodated by gp2. This would be an unnecessarily expensive option.
Option C (S3 Intelligent-Tiering): Amazon S3 is for object storage, not for hosting a relational database.
Option D (EC2 instances in active-passive mode): This would require significant manual effort for setup, failover management, and database administration. It also involves higher costs associated with EC2 instance management.

Authoritative Links:

Amazon RDS Multi-AZ Deployments:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html
Amazon EBS Volume Types: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html
Amazon RDS Pricing: https://aws.amazon.com/rds/pricing/

Question: 354 **Exam Heist**

A company hosts a serverless application on AWS. The application uses Amazon API Gateway, AWS Lambda, and an Amazon RDS for PostgreSQL database. The company notices an increase in application errors that result from database connection timeouts during times of peak traffic or unpredictable traffic. The company needs a solution that reduces the application failures with the least amount of change to the code.

What should a solutions architect do to meet these requirements?

A. Reduce the Lambda concurrency rate.
B. Enable RDS Proxy on the RDS DB instance.
C. Resize the RDS DB instance class to accept more connections.
D. Migrate the database to Amazon DynamoDB with on-demand scaling.

Answer: B

Explanation:

The correct answer is **B. Enable RDS Proxy on the RDS DB instance.**

Here's a detailed justification:

Database connection timeouts in serverless applications, like the one described, are often caused by connection exhaustion. Lambda functions, especially during periods of high concurrency, rapidly establish database connections. RDS instances have a limit to the number of concurrent connections they can handle. When Lambda functions try to open new connections beyond this limit, connection timeouts occur.

RDS Proxy sits between the Lambda functions and the RDS database. It pools and shares database connections, effectively reducing the number of direct connections made from Lambda to RDS. This is a key aspect of solving connection exhaustion. By multiplexing connections, RDS Proxy allows more Lambda invocations to access the database without overwhelming it.

Option A (Reduce the Lambda concurrency rate) would directly reduce the application's ability to serve requests, degrading performance and user experience. It addresses the symptom but not the root cause. It is also counter to the implied requirement of handling peak traffic.

Option C (Resize the RDS DB instance class) might increase the maximum number of connections the database can handle. However, it is a costly solution that requires significant downtime and doesn't address the fundamental problem of inefficient connection management, especially for spiky workloads. It is also not the "least amount of change to the code" as requested in the prompt, as it requires infrastructure changes.

Option D (Migrate the database to Amazon DynamoDB) is a significantly larger change involving considerable code refactoring and a completely different data model. DynamoDB is a NoSQL database and likely incompatible with the existing application's data schema and query patterns that are tailored for a relational database (PostgreSQL). DynamoDB also has a completely different set of considerations for cost optimisation which will likely lead to a higher operational burden.

RDS Proxy is designed to solve the exact problem of connection exhaustion in serverless applications using relational databases with minimal code changes. It optimizes connection management and provides connection pooling, leading to improved application stability during peak traffic. It is the most direct and efficient solution, requiring the least amount of code change.

Supporting links:

Using Amazon RDS Proxy with AWS Lambda: Provides a detailed explanation of using RDS Proxy with Lambda to improve application performance.
Amazon RDS Proxy: Official documentation for RDS Proxy, including features and benefits.

Best Practices for Working with AWS Lambda Functions: Covers general best practices for using Lambda, including database connection management.

Question: 355 **Exam Heist**

A company is migrating an old application to AWS. The application runs a batch job every hour and is CPU intensive. The batch job takes 15 minutes on average with an on-premises server. The server has 64 virtual CPU (vCPU) and 512 GiB of memory.

Which solution will run the batch job within 15 minutes with the LEAST operational overhead?

A.Use AWS Lambda with functional scaling.

B.Use Amazon Elastic Container Service (Amazon ECS) with AWS Fargate.

C.Use Amazon Lightsail with AWS Auto Scaling.

D.Use AWS Batch on Amazon EC2.

Answer: D

Explanation:

The optimal solution for migrating the CPU-intensive batch job with minimal operational overhead is using AWS Batch on Amazon EC2. Here's why:

AWS Batch is specifically designed for running batch computing workloads. It handles the complexities of provisioning and managing compute resources, allowing the company to focus on the batch job itself. AWS Batch automatically scales the compute resources based on the job's requirements, ensuring it can complete within the desired 15-minute timeframe. https://aws.amazon.com/batch/
Amazon EC2 offers a wide variety of instance types that can be optimized for CPU-intensive workloads. Selecting an instance type that meets the vCPU and memory requirements of the original server (or even surpasses them for faster execution) ensures the batch job's performance.
Operational Overhead: AWS Batch significantly reduces operational overhead because it manages the underlying infrastructure. The company doesn't have to manually provision, configure, or manage EC2 instances. AWS Batch takes care of scheduling jobs, launching instances, and monitoring their progress.
Comparison with other options:

AWS Lambda: While Lambda is serverless, it has limitations on execution time (currently 15 minutes maximum), memory, and CPU resources. A 15-minute CPU-intensive job likely exceeds these limitations.
Amazon ECS with Fargate: Fargate eliminates the need to manage servers, but it's primarily designed for containerized applications, adding complexity if the original application is not already containerized. Also, CPU and memory options on Fargate might require extensive configuration to meet the original server's specifications.
Amazon Lightsail with Auto Scaling: Lightsail is a simpler alternative to EC2, but it provides less flexibility and control over instance types and scaling policies than EC2 and AWS Batch. It may not be the most efficient way to manage a demanding batch job. Plus, the 512 GB memory requirement isn't easily achieved in Lightsail instances.

In summary, AWS Batch on Amazon EC2 provides the right balance of performance, scalability, and reduced operational overhead for migrating the batch job to AWS. The ability to customize EC2 instances ensures the application has the necessary resources to complete within the specified timeframe, while AWS Batch simplifies management.

Question: 356 **Exam Heist**

A company stores its data objects in Amazon S3 Standard storage. A solutions architect has found that 75% of the data is rarely accessed after 30 days. The company needs all the data to remain immediately accessible with the same high availability and resiliency, but the company wants to minimize storage costs.

Which storage solution will meet these requirements?

A. Move the data objects to S3 Glacier Deep Archive after 30 days.
B. Move the data objects to S3 Standard-Infrequent Access (S3 Standard-IA) after 30 days.
C. Move the data objects to S3 One Zone-Infrequent Access (S3 One Zone-IA) after 30 days.
D. Move the data objects to S3 One Zone-Infrequent Access (S3 One Zone-IA) immediately.

Answer: B

Explanation:

The correct answer is B, moving the data objects to S3 Standard-Infrequent Access (S3 Standard-IA) after 30 days. This choice effectively balances cost optimization with the requirement for immediate data accessibility and high availability. S3 Standard-IA is designed for data that is infrequently accessed but requires rapid retrieval when needed. This perfectly aligns with the scenario where 75% of the data is rarely accessed after 30 days, but the company still needs it to be immediately available.

Option A, S3 Glacier Deep Archive, is unsuitable because it is designed for long-term archival with retrieval times ranging from hours to days. This violates the requirement for immediate accessibility. Option C, S3 One Zone-IA, offers lower costs than S3 Standard-IA but compromises data durability and availability as it stores data in a single Availability Zone (AZ). This contradicts the requirement for high availability and resiliency. Option D, moving data immediately to S3 One Zone-IA, similarly fails to meet the high availability and resiliency requirements.

S3 Standard-IA offers the same high durability as S3 Standard (99.999999999% across multiple AZs) while providing a lower storage cost. Although S3 Standard-IA has a retrieval fee, given that only 25% of the data is frequently accessed, the cost savings on the 75% of infrequently accessed data will outweigh the retrieval fees. By transitioning the data after 30 days, the company minimizes costs while adhering to the specified accessibility and availability criteria. S3 Intelligent-Tiering could also be considered, but with a known access pattern of 30 days, S3 Standard-IA provides a more predictable and potentially more cost-effective solution.

Further research can be found on the AWS website:

S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
S3 Standard-IA: https://aws.amazon.com/s3/storage-classes/ia/

Question: 357 **Exam Heist**

A gaming company is moving its public scoreboard from a data center to the AWS Cloud. The company uses Amazon EC2 Windows Server instances behind an Application Load Balancer to host its dynamic application. The company needs a highly available storage solution for the application. The application consists of static files and dynamic server-side code.

Which combination of steps should a solutions architect take to meet these requirements? (Choose two.)

A. Store the static files on Amazon S3. Use Amazon CloudFront to cache objects at the edge.
B. Store the static files on Amazon S3. Use Amazon ElastiCache to cache objects at the edge.
C. Store the server-side code on Amazon Elastic File System (Amazon EFS). Mount the EFS volume on each EC2 instance to share the files.
D. Store the server-side code on Amazon FSx for Windows File Server. Mount the FSx for Windows File Server

volume on each EC2 instance to share the files.

E. Store the server-side code on a General Purpose SSD (gp2) Amazon Elastic Block Store (Amazon EBS) volume. Mount the EBS volume on each EC2 instance to share the files.

Answer: AD

Explanation:

The correct answer is AD. Here's a breakdown of why:

Option A: Store the static files on Amazon S3. Use Amazon CloudFront to cache objects at the edge.

Amazon S3 is ideal for storing static files (images, CSS, JavaScript, etc.) due to its scalability, durability, and cost-effectiveness. (https://aws.amazon.com/s3/)
Amazon CloudFront is a Content Delivery Network (CDN) that caches these static files at edge locations globally. This reduces latency and improves performance for users accessing the scoreboard. (https://aws.amazon.com/cloudfront/)
CloudFront improves availability by serving content from cached locations even if the origin server is temporarily unavailable.

Option D: Store the server-side code on Amazon FSx for Windows File Server. Mount the FSx for Windows File Server volume on each EC2 instance to share the files.

Amazon FSx for Windows File Server is a fully managed Windows file server built on Windows Server. (https://aws.amazon.com/fsx/windows/)
Since the application uses EC2 Windows Server instances, FSx for Windows File Server provides native Windows file system compatibility and supports features like SMB protocol.
It enables sharing the dynamic server-side code among multiple EC2 instances behind the Application Load Balancer, ensuring that all instances have access to the same codebase. This is crucial for consistent application behavior and high availability.
FSx for Windows File Server is designed for high availability, ensuring that the application's code is always accessible.

Let's address why the other options are less suitable:

Option B: Store the static files on Amazon S3. Use Amazon ElastiCache to cache objects at the edge.

While ElastiCache is a powerful caching service, it is primarily designed for caching frequently accessed data from databases or compute-intensive operations. CloudFront is designed for caching static content. ElastiCache doesn't provide edge locations and distribution capabilities like CloudFront.

Option C: Store the server-side code on Amazon Elastic File System (Amazon EFS). Mount the EFS volume on each EC2 instance to share the files.

EFS is a good option for shared storage, but FSx for Windows File Server is a better fit in this specific scenario since the EC2 instances are running Windows Server. EFS is designed for Linux instances.

Option E: Store the server-side code on a General Purpose SSD (gp2) Amazon Elastic Block Store (Amazon EBS) volume. Mount the EBS volume on each EC2 instance to share the files.

EBS volumes are block storage devices that are attached to a single EC2 instance. They cannot be directly shared between multiple instances. Sharing data would require complex solutions like clustering or replication, making this approach unsuitable for a highly available, shared codebase.

Question: 358 **Exam Heist**

A social media company runs its application on Amazon EC2 instances behind an Application Load Balancer (ALB). The ALB is the origin for an Amazon CloudFront distribution. The application has more than a billion images stored in an Amazon S3 bucket and processes thousands of images each second. The company wants to resize the images dynamically and serve appropriate formats to clients.

Which solution will meet these requirements with the LEAST operational overhead?

A. Install an external image management library on an EC2 instance. Use the image management library to process the images.

B. Create a CloudFront origin request policy. Use the policy to automatically resize images and to serve the appropriate format based on the User-Agent HTTP header in the request.

C. Use a function with an external image management library. Associate the function with the CloudFront behaviors that serve the images.

D. Create a CloudFront response headers policy. Use the policy to automatically resize images and to serve the appropriate format based on the User-Agent HTTP header in the request.

Answer: C

Explanation:

The correct answer is C, using a function with an external image management library. Here's why:

Option C offers the least operational overhead because it leverages the power of serverless computing through This allows image processing to occur dynamically and automatically at the edge, close to the users, without managing any EC2 instances or infrastructure. The function is triggered by CloudFront requests. The image management library resizes images based on user requests and serves the correct format based on the User-Agent header (for example, serving WebP to browsers that support it).

Option A requires managing an EC2 instance and its associated patching, scaling, and maintenance. This adds considerable operational overhead, making it less desirable.

Option B is incorrect because CloudFront origin request policies primarily manipulate requests before they are sent to the origin (in this case, S3 or the ALB). They can modify headers and query strings, but they cannot execute arbitrary code to perform image resizing or format conversion. They are used for controlling caching behavior, not real-time image manipulation.

Option D is incorrect because CloudFront response header policies control the HTTP headers returned after the response is received from the origin. They do not provide a mechanism for resizing images or performing format conversion. Their main purpose is to add, modify, or remove headers.

 functions are ideal for this scenario because they allow you to intercept CloudFront requests and responses at various points in the content delivery process, execute custom code (image resizing using a library), and return the processed image to the user. This is the most scalable, cost-effective, and operationally efficient solution.

Relevant links:

 functions: https://aws.amazon.com/lambda/edge/
CloudFront policies: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-managed-response-headers-policies.html (explains response header policies)
CloudFront origin request policies:
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-managed-origin-request-policies.html

Question: 359 **Exam Heist**

A hospital needs to store patient records in an Amazon S3 bucket. The hospital's compliance team must ensure that all protected health information (PHI) is encrypted in transit and at rest. The compliance team must administer the encryption key for data at rest.

Which solution will meet these requirements?

A.Create a public SSL/TLS certificate in AWS Certificate Manager (ACM). Associate the certificate with Amazon S3. Configure default encryption for each S3 bucket to use server-side encryption with AWS KMS keys (SSE-KMS). Assign the compliance team to manage the KMS keys.

B.Use the aws:SecureTransport condition on S3 bucket policies to allow only encrypted connections over HTTPS (TLS). Configure default encryption for each S3 bucket to use server-side encryption with S3 managed encryption keys (SSE-S3). Assign the compliance team to manage the SSE-S3 keys.

C.Use the aws:SecureTransport condition on S3 bucket policies to allow only encrypted connections over HTTPS (TLS). Configure default encryption for each S3 bucket to use server-side encryption with AWS KMS keys (SSE-KMS). Assign the compliance team to manage the KMS keys.

D.Use the aws:SecureTransport condition on S3 bucket policies to allow only encrypted connections over HTTPS (TLS). Use Amazon Macie to protect the sensitive data that is stored in Amazon S3. Assign the compliance team to manage Macie.

Answer: C

Explanation:

The correct answer is **C**. Here's why:

Encryption in Transit: The requirement for encryption in transit is met by enforcing the use of HTTPS (TLS) for all connections to the S3 bucket. This is achieved by using the aws:SecureTransport condition in the S3 bucket policy. This policy condition checks if the request is made over HTTPS, preventing unencrypted traffic.

Encryption at Rest: The requirement for encryption at rest is met by configuring default encryption for the S3 bucket. Server-Side Encryption with KMS keys (SSE-KMS) allows using KMS to manage encryption keys. This way, data is encrypted when stored in S3.

Key Management Control: The compliance team's requirement to administer the encryption key for data at rest is met by using SSE-KMS and assigning the compliance team permissions to manage the KMS keys. This allows them to control the encryption and decryption process.

Why other options are incorrect:

A: While using ACM for SSL/TLS certificates is valid for HTTPS, associating the certificate directly with S3 isn't how encryption in transit is enforced on the bucket level. You control it via the bucket policy.
B: SSE-S3 (Server-Side Encryption with S3 Managed Keys) doesn't give the compliance team control over the keys, as AWS manages them. This violates the requirement.
D: While using HTTPS for encryption in transit is correct, Macie is primarily a data security and privacy service that uses machine learning to discover, classify, and protect sensitive data in AWS. It doesn't directly handle encryption, so the Compliance team would not manage the keys. While Macie is useful, it doesn't provide the required key management capabilities specified.

Supporting Documentation:

Protecting Data Using Server-Side Encryption with KMS keys (SSE-KMS):
https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingKMSEncryption.html
Requiring Encryption of Data in Transit: https://docs.aws.amazon.com/AmazonS3/latest/userguide/example-bucket-policies.html#example-bucket-policies-require-encryption
AWS Key Management Service (KMS): https://aws.amazon.com/kms/
Amazon Macie: https://aws.amazon.com/macie/

Question: 360 **Exam Heist**

A company uses Amazon API Gateway to run a private gateway with two REST APIs in the same VPC. The BuyStock RESTful web service calls the CheckFunds RESTful web service to ensure that enough funds are available before a stock can be purchased. The company has noticed in the VPC flow logs that the BuyStock RESTful web service calls the CheckFunds RESTful web service over the internet instead of through the VPC. A solutions architect must implement a solution so that the APIs communicate through the VPC.

Which solution will meet these requirements with the FEWEST changes to the code?

A.Add an X-API-Key header in the HTTP header for authorization.

B.Use an interface endpoint.

C.Use a gateway endpoint.

D.Add an Amazon Simple Queue Service (Amazon SQS) queue between the two REST APIs.

Answer: B

Explanation:

The correct answer is **B. Use an interface endpoint.**

Here's a detailed justification:

The problem describes a scenario where two private REST APIs within the same VPC, managed by Amazon API Gateway, are communicating over the internet instead of staying within the VPC network. The goal is to ensure that BuyStock communicates with CheckFunds through the VPC, minimizing code changes.

Why Interface Endpoints are Ideal: Interface endpoints, powered by AWS PrivateLink, create private network connections between your VPC and supported AWS services without exposing traffic to the public internet. In this case, an interface endpoint can be created for the API Gateway service within the VPC. This endpoint provides a private IP address within the VPC that the BuyStock API can use to access the CheckFunds API via API Gateway, ensuring all communication stays within the VPC. This approach requires minimal or no code changes, typically only needing to update the endpoint URL.

Why other options are not optimal:

A. Add an X-API-Key header in the HTTP header for authorization: Adding an API key is for authorization, it doesn't change the routing of network traffic and won't force communication through the VPC.
C. Use a gateway endpoint: Gateway endpoints only support Amazon S3 and DynamoDB. They can't be used with API Gateway.
D. Add an Amazon Simple Queue Service (Amazon SQS) queue between the two REST APIs: This introduces asynchronous communication via a queue. While functional, it requires significant architectural and code changes to both APIs to publish and consume messages, violating the "fewest changes to the code" requirement. It also changes the communication paradigm from request/response to asynchronous messaging, which may not be desirable.

In essence, Interface endpoints are designed specifically for securely connecting to AWS services privately from your VPC. They minimize the need for code changes and are the most direct solution to the problem.

Authoritative Links:

AWS PrivateLink: https://aws.amazon.com/privatelink/
Using API Gateway with VPC Endpoints:
https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-private-apis.html

Question: 361 **Exam Heist**

A company hosts a multiplayer gaming application on AWS. The company wants the application to read data with sub-millisecond latency and run one-time queries on historical data.

Which solution will meet these requirements with the LEAST operational overhead?

A. Use Amazon RDS for data that is frequently accessed. Run a periodic custom script to export the data to an Amazon S3 bucket.

B. Store the data directly in an Amazon S3 bucket. Implement an S3 Lifecycle policy to move older data to S3 Glacier Deep Archive for long-term storage. Run one-time queries on the data in Amazon S3 by using Amazon Athena.

C. Use Amazon DynamoDB with DynamoDB Accelerator (DAX) for data that is frequently accessed. Export the data to an Amazon S3 bucket by using DynamoDB table export. Run one-time queries on the data in Amazon S3 by using Amazon Athena.

D. Use Amazon DynamoDB for data that is frequently accessed. Turn on streaming to Amazon Kinesis Data Streams. Use Amazon Kinesis Data Firehose to read the data from Kinesis Data Streams. Store the records in an Amazon S3 bucket.

Answer: C

Explanation:

The correct answer is C. Here's why:

Sub-millisecond latency for frequent access: DynamoDB, a NoSQL database, is designed for low-latency access. DynamoDB Accelerator (DAX) further enhances this by providing an in-memory cache, enabling sub-millisecond response times for frequently accessed data. This directly addresses the application's requirement for fast reads.(https://aws.amazon.com/dynamodb/dax/)

One-time queries on historical data: DynamoDB table export allows you to export data to S3 in a cost-effective and scalable manner. Athena, a serverless query service, can then be used to run SQL queries on the data stored in S3. This satisfies the need for ad-hoc querying of historical information without requiring persistent infrastructure.(https://aws.amazon.com/athena/, https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/DynamoDBExport.html)

Least operational overhead: DynamoDB is a managed service, reducing the burden of database administration. DAX is also managed, simplifying caching. Athena is serverless, eliminating the need to manage servers for querying. The DynamoDB export feature is straightforward to configure and automate.

Let's analyze why the other options are less suitable:

A: RDS (a relational database) might not provide the same level of sub-millisecond latency as DynamoDB, especially for large-scale gaming applications. Creating and maintaining a custom script for exporting data adds operational overhead.

B: Storing all data in S3 directly would not meet the low-latency requirement for frequently accessed data. S3 is object storage and not suitable for real-time reads needed by a gaming application. While Athena can query S3 data, it is not ideal for accessing frequently used game data.

D: Kinesis Data Streams and Firehose would introduce unnecessary complexity and latency. Streaming the entire DynamoDB data stream to S3 just for querying historical data is an overkill and more complex than using DynamoDB export and Athena.

Question: 362 **Exam Heist**

A company uses a payment processing system that requires messages for a particular payment ID to be received in the same order that they were sent. Otherwise, the payments might be processed incorrectly.

Which actions should a solutions architect take to meet this requirement? (Choose two.)

A. Write the messages to an Amazon DynamoDB table with the payment ID as the partition key.

B. Write the messages to an Amazon Kinesis data stream with the payment ID as the partition key.

C. Write the messages to an Amazon ElastiCache for Memcached cluster with the payment ID as the key.

D. Write the messages to an Amazon Simple Queue Service (Amazon SQS) queue. Set the message attribute to use the payment ID.

E. Write the messages to an Amazon Simple Queue Service (Amazon SQS) FIFO queue. Set the message group to use the payment ID.

Answer: BE

Explanation:

The correct answer is BE. Here's why:

Option B: Write the messages to an Amazon Kinesis data stream with the payment ID as the partition key.

Kinesis Data Streams inherently guarantees ordering of records within a shard. When you use the payment ID as the partition key, all messages related to the same payment ID will be written to the same shard. This ensures that they are processed in the order they were sent, fulfilling the requirement of sequential processing for each payment. Kinesis is designed for real-time data streaming and processing, making it a suitable choice for managing payment messages.

https://docs.aws.amazon.com/kinesis/latest/dev/kinesis-data-streams.html

Option E: Write the messages to an Amazon Simple Queue Service (Amazon SQS) FIFO queue. Set the message group to use the payment ID.

Amazon SQS FIFO (First-In-First-Out) queues are specifically designed to guarantee that messages are processed in the exact order they were sent. By setting the message group ID to the payment ID, you instruct SQS to maintain the order of messages within that group. This ensures that all messages related to a particular payment ID are received and processed in the correct sequence, meeting the ordering requirement.

https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues.html

Why other options are incorrect:

Option A: DynamoDB does not guarantee ordering of items within a partition key. While you can retrieve all items for a specific payment ID, the order in which they are returned is not guaranteed to be the same as the order in which they were written.
Option C: ElastiCache (Memcached) is primarily used for caching and is not designed for reliable message queuing or guaranteed ordering.
Option D: Standard SQS queues do not guarantee message ordering. While message attributes can be used, they do not enforce the sequential processing needed in this scenario. Using standard queues could lead to incorrect payment processing.

Question: 363 **Exam Heist**

A company is building a game system that needs to send unique events to separate leaderboard, matchmaking, and authentication services concurrently. The company needs an AWS event-driven system that guarantees the order of the events.

Which solution will meet these requirements?

A.Amazon EventBridge event bus

B.Amazon Simple Notification Service (Amazon SNS) FIFO topics

C.Amazon Simple Notification Service (Amazon SNS) standard topics

D.Amazon Simple Queue Service (Amazon SQS) FIFO queues

Answer: B

Explanation:

The correct answer is **B. Amazon Simple Notification Service (Amazon SNS) FIFO topics**. Here's why:

The key requirements are sending events to multiple services concurrently and guaranteeing the order of events.

Amazon SNS FIFO Topics: SNS FIFO (First-In-First-Out) topics are specifically designed to maintain the exact order of messages published to them. They also support message fanout to multiple subscribers (leaderboard, matchmaking, and authentication services in this case), fulfilling the concurrency requirement. Messages are grouped into message groups based on a message group ID, and messages within a group are delivered in order.

Amazon EventBridge: EventBridge allows for event routing based on rules, but it primarily focuses on decoupling event producers from consumers. While EventBridge supports ordering through event replay, it's not designed for guaranteed ordering in concurrent fan-out scenarios like SNS FIFO topics.

Amazon SNS Standard Topics: SNS standard topics provide high throughput and best-effort ordering, but do not guarantee the order of messages. This violates the order requirement.

Amazon SQS FIFO Queues: SQS FIFO queues guarantee message order, but they are point-to-point (one sender to one receiver). To send the same events to three services concurrently, you would need three separate SQS FIFO queues, one for each service. This would not meet the need for a single event triggering all three. While you could implement a fan-out mechanism on top of SQS using Lambda or similar, this is more complex and less efficient than using SNS FIFO topics, which handles fan-out natively with ordering.

In summary, SNS FIFO topics provide a native solution for concurrently distributing ordered events to multiple subscribers, making it the most efficient and appropriate choice for this scenario.

Authoritative Links:

Amazon SNS FIFO topics: https://docs.aws.amazon.com/sns/latest/dg/sns-fifo-topics.html
Amazon EventBridge: https://aws.amazon.com/eventbridge/
Amazon SQS FIFO queues:
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues.html

Question: 364 **Exam Heist**

A hospital is designing a new application that gathers symptoms from patients. The hospital has decided to use Amazon Simple Queue Service (Amazon SQS) and Amazon Simple Notification Service (Amazon SNS) in the architecture.

A solutions architect is reviewing the infrastructure design. Data must be encrypted at rest and in transit. Only authorized personnel of the hospital should be able to access the data.

Which combination of steps should the solutions architect take to meet these requirements? (Choose two.)

A. Turn on server-side encryption on the SQS components. Update the default key policy to restrict key usage to a set of authorized principals.

B. Turn on server-side encryption on the SNS components by using an AWS Key Management Service (AWS KMS) customer managed key. Apply a key policy to restrict key usage to a set of authorized principals.

C. Turn on encryption on the SNS components. Update the default key policy to restrict key usage to a set of authorized principals. Set a condition in the topic policy to allow only encrypted connections over TLS.

D. Turn on server-side encryption on the SQS components by using an AWS Key Management Service (AWS KMS) customer managed key. Apply a key policy to restrict key usage to a set of authorized principals. Set a condition in the queue policy to allow only encrypted connections over TLS.

E. Turn on server-side encryption on the SQS components by using an AWS Key Management Service (AWS KMS) customer managed key. Apply an IAM policy to restrict key usage to a set of authorized principals. Set a condition in the queue policy to allow only encrypted connections over TLS.

Answer: BD

Explanation:

The question requires ensuring data in transit and at rest encryption, and restricting access to only authorized personnel for SQS and SNS components.

Option B is correct because it properly utilizes AWS KMS for server-side encryption on SNS topics. Using a customer managed key in KMS allows for granular control over the key's policy, which can be restricted to authorized hospital personnel (principals). This ensures only authorized users and services can encrypt and decrypt messages published to and consumed from the SNS topic.

Option D is correct because it employs a similar strategy for SQS queues. AWS KMS is used for server-side encryption. Applying a custom key policy to restrict usage to specific hospital personnel controls who can decrypt messages in the queue. Setting a condition in the queue policy to allow only encrypted connections over TLS (Transport Layer Security) enforces encryption in transit between clients and SQS, fulfilling the requirement for encryption during transport.

Option A is incorrect because while it enables server-side encryption on SQS, it relies on updating the default key policy. AWS KMS Customer managed keys are preferable as they offer more fine-grained access control.Option C is incorrect. It incorrectly states that the SNS encryption policy should be updated, rather than using KMS. Also, setting a condition to allow only encrypted connections over TLS won't be enough, a custom KMS key policy will still be needed to restrict access.Option E is incorrect because while it correctly uses KMS for SQS encryption, it mentions updating the IAM policy to restrict key usage instead of the KMS key policy. The KMS key policy, not an IAM policy, is what controls who can use the key for encryption/decryption.

Therefore, options B and D are the most appropriate to meet the specified security and access control requirements.

Relevant links:

SNS Encryption:
SQS Encryption:
KMS Key Policies:

Question: 365 **Exam Heist**

A company runs a web application that is backed by Amazon RDS. A new database administrator caused data loss by accidentally editing information in a database table. To help recover from this type of incident, the company wants the ability to restore the database to its state from 5 minutes before any change within the last 30 days.

Which feature should the solutions architect include in the design to meet this requirement?

A.Read replicas

B.Manual snapshots

C.Automated backups

D.Multi-AZ deployments

Answer: C

Explanation:

The correct answer is **C. Automated backups.** Here's a detailed justification:

Automated backups in Amazon RDS create point-in-time recovery (PITR) points for your database. These backups enable you to restore your database instance to a specific point in time within the configured retention period, which can be up to 35 days. This directly addresses the requirement to restore the database to its state from 5 minutes before any change within the last 30 days. RDS uses the database's native backup mechanisms, allowing for consistent backups and restores. The transaction logs are also backed up.

Read replicas (A) are used to offload read traffic from the primary database, improving performance. While read replicas can be promoted to become a standalone database in case of a primary database failure, they don't provide granular point-in-time recovery. The data on a read replica replicates from the primary database, including the unintended changes made by the database administrator.
Manual snapshots (B) provide a full backup of the database at a specific point in time, but they require manual initiation. While helpful, they do not automatically provide the 5-minute recovery granularity needed as they require on demand intervention. To achieve this, you'd need continuous manual snapshots, which is impractical and resource-intensive.
Multi-AZ deployments (D) provide high availability by synchronously replicating data to a standby instance in a different Availability Zone. If the primary instance fails, the standby instance automatically takes over. However, Multi-AZ does not prevent or recover from data corruption or accidental data changes; it simply ensures the database remains available. The change will replicate to both AZs.

Therefore, **automated backups** are the only feature that enables the necessary point-in-time recovery to meet the company's recovery objective.

For further information, refer to the AWS documentation on RDS automated backups:

Amazon RDS Backups

Question: 366 **Exam Heist**

A company’s web application consists of an Amazon API Gateway API in front of an AWS Lambda function and an Amazon DynamoDB database. The Lambda function handles the business logic, and the DynamoDB table hosts the data. The application uses Amazon Cognito user pools to identify the individual users of the application. A solutions architect needs to update the application so that only users who have a subscription can access premium content.

Which solution will meet this requirement with the LEAST operational overhead?

A. Enable API caching and throttling on the API Gateway API.

B. Set up AWS WAF on the API Gateway API. Create a rule to filter users who have a subscription.

C. Apply fine-grained IAM permissions to the premium content in the DynamoDB table.

D. Implement API usage plans and API keys to limit the access of users who do not have a subscription.

Answer: D

Explanation:

The correct answer is **D. Implement API usage plans and API keys to limit the access of users who do not**

have a subscription.

Here's a detailed justification:

Option D offers the most straightforward and efficient method to control access to premium content based on subscription status directly within API Gateway, minimizing operational overhead. API Gateway usage plans allow you to define quotas and throttling for API usage. API keys can be associated with these usage plans. By associating subscribed users with a usage plan that permits access to premium content and non-subscribed users with a plan that restricts it, you can effectively manage access. This requires the Lambda function to determine user subscription status and assign the correct API key.

Here's why the other options are less suitable:

A. Enable API caching and throttling on the API Gateway API: Caching and throttling are primarily for performance optimization and preventing abuse, not for authentication or authorization based on subscription status. They don't inherently differentiate between subscribed and non-subscribed users.

B. Set up AWS WAF on the API Gateway API. Create a rule to filter users who have a subscription: AWS WAF is designed for protecting web applications from common web exploits and bots. While it can filter requests based on IP addresses or other request characteristics, it's not designed for handling user authentication or subscription-based access control. Implementing subscription-based access control in WAF would be complex and require custom rules that examine request headers or cookies, adding unnecessary overhead.

C. Apply fine-grained IAM permissions to the premium content in the DynamoDB table: While fine-grained IAM permissions are useful for controlling access to DynamoDB data, they are not the most efficient method in this case. It would require the Lambda function to assume different IAM roles or generate temporary security credentials based on the user's subscription status, adding complexity to the Lambda function and requiring more complex management of IAM roles and policies. Access control logic should ideally be handled before the database layer when possible. Furthermore, managing IAM permissions at a granular level for each user subscription would become cumbersome as the user base scales.

Using API Gateway's built-in features for access control is more native and less operationally intensive.

Supporting Documentation:

API Gateway Usage Plans: https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-api-usage-plans.html
API Gateway API Keys: https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-usage-plans-api-keys.html

Question: 367 **Exam Heist**

A company is using Amazon Route 53 latency-based routing to route requests to its UDP-based application for users around the world. The application is hosted on redundant servers in the company's on-premises data centers in the United States, Asia, and Europe. The company’s compliance requirements state that the application must be hosted on premises. The company wants to improve the performance and availability of the application.

What should a solutions architect do to meet these requirements?

A. Configure three Network Load Balancers (NLBs) in the three AWS Regions to address the on-premises endpoints. Create an accelerator by using AWS Global Accelerator, and register the NLBs as its endpoints. Provide access to the application by using a CNAME that points to the accelerator DNS.

B. Configure three Application Load Balancers (ALBs) in the three AWS Regions to address the on-premises endpoints. Create an accelerator by using AWS Global Accelerator, and register the ALBs as its endpoints. Provide access to the application by using a CNAME that points to the accelerator DNS.

C. Configure three Network Load Balancers (NLBs) in the three AWS Regions to address the on-premises

endpoints. In Route 53, create a latency-based record that points to the three NLBs, and use it as an origin for an Amazon CloudFront distribution. Provide access to the application by using a CNAME that points to the CloudFront DNS.

D. Configure three Application Load Balancers (ALBs) in the three AWS Regions to address the on-premises endpoints. In Route 53, create a latency-based record that points to the three ALBs, and use it as an origin for an Amazon CloudFront distribution. Provide access to the application by using a CNAME that points to the CloudFront DNS.

Answer: A

Explanation:

The correct answer is A. Here's a detailed justification:

The problem requires improving performance and availability of a UDP-based application hosted on-premises across multiple regions, while complying with hosting requirements. Route 53 latency-based routing is already in place.

Option A is the most suitable solution because it leverages AWS Global Accelerator in conjunction with Network Load Balancers (NLBs). NLBs are ideal for UDP traffic.

Here's why it works:

1. **NLBs for UDP:** NLBs are designed to handle UDP traffic efficiently. They can forward UDP traffic directly to on-premises servers without modification. Application Load Balancers (ALBs), used in options B and D, do not support UDP.
2. **AWS Global Accelerator:** Global Accelerator provides static IP addresses that serve as entry points for the application. It intelligently routes traffic to the nearest healthy endpoint (the NLB) based on network conditions and geographic proximity. This minimizes latency and improves performance.
3. **Regional Redundancy:** Using NLBs in multiple AWS regions (US, Asia, and Europe) provides redundancy and increases availability. If one NLB becomes unavailable, Global Accelerator automatically reroutes traffic to another healthy NLB.
4. **On-premises Integration:** The NLBs are configured to address the on-premises endpoints, fulfilling the requirement to host the application on-premises.
5. **Simplified DNS Management:** Providing access via a CNAME record pointing to the Global Accelerator DNS name simplifies DNS management and allows clients to connect to the optimal endpoint.

Option B is incorrect because ALBs do not support UDP traffic.

Options C and D are incorrect because while CloudFront can improve performance for caching static content, it is not suitable for UDP-based applications requiring real-time, bidirectional communication. Also CloudFront is mainly used for caching web content at edge locations, not for direct routing of UDP traffic to on-premises servers. CloudFront works best with HTTP/HTTPS. Also, CloudFront does not inherently provide the same level of global traffic management and failover capabilities as AWS Global Accelerator for UDP workloads.

In summary, using NLBs with Global Accelerator is the most efficient and reliable solution for improving the performance and availability of a UDP-based application hosted on-premises across multiple regions.

Supporting Documentation:

AWS Global Accelerator: https://aws.amazon.com/global-accelerator/
Network Load Balancer: https://aws.amazon.com/elasticloadbalancing/network-load-balancer/

Question: 368 **Exam Heist**

A solutions architect wants all new users to have specific complexity requirements and mandatory rotation periods for IAM user passwords.

What should the solutions architect do to accomplish this?

A.Set an overall password policy for the entire AWS account.

B.Set a password policy for each IAM user in the AWS account.

C.Use third-party vendor software to set password requirements.

D.Attach an Amazon CloudWatch rule to the Create_newuser event to set the password with the appropriate requirements.

Answer: A

Explanation:

The correct answer is A: Set an overall password policy for the entire AWS account. This is the most efficient and scalable method to enforce consistent password complexity and rotation requirements across all IAM users within an AWS account.

IAM (Identity and Access Management) provides a global password policy feature at the account level. This allows you to define requirements like minimum password length, required character types (uppercase, lowercase, numbers, symbols), password reuse prevention, and maximum password age (rotation period). Once the password policy is set at the account level, all IAM users are automatically subject to those requirements.

Option B is incorrect because setting password policies individually for each user is not a scalable or maintainable approach, especially in an environment with many users. It would require significant manual effort and could lead to inconsistencies.

Option C is incorrect because while third-party identity providers can be integrated with AWS, using them solely for password policy enforcement is an overkill. AWS native IAM features handle this efficiently. Employing a third-party vendor just for password rules would add unnecessary complexity and cost.

Option D is incorrect because while CloudWatch Events can monitor IAM events, it is not the intended mechanism for setting password policies. CloudWatch Events are best suited for reacting to state changes or triggering automated actions based on events, not for enforcing global security policies like password complexity. Additionally, directly setting passwords for users programmatically violates the security principle of least privilege.

Therefore, leveraging the built-in IAM password policy feature offers the most straightforward and maintainable solution for establishing mandatory password complexity and rotation for all IAM users in the AWS account.

Relevant documentation:https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_password_policy.htmlhttps://av

Question: 369 **Exam Heist**

A company has migrated an application to Amazon EC2 Linux instances. One of these EC2 instances runs several 1-hour tasks on a schedule. These tasks were written by different teams and have no common programming language. The company is concerned about performance and scalability while these tasks run on a single instance. A solutions architect needs to implement a solution to resolve these concerns.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use AWS Batch to run the tasks as jobs. Schedule the jobs by using Amazon EventBridge (Amazon

CloudWatch Events).

B.Convert the EC2 instance to a container. Use AWS App Runner to create the container on demand to run the tasks as jobs.

C.Copy the tasks into AWS Lambda functions. Schedule the Lambda functions by using Amazon EventBridge (Amazon CloudWatch Events).

D.Create an Amazon Machine Image (AMI) of the EC2 instance that runs the tasks. Create an Auto Scaling group with the AMI to run multiple copies of the instance.

Answer: A

Explanation:

The correct answer is A: Use AWS Batch to run the tasks as jobs. Schedule the jobs by using Amazon EventBridge (Amazon CloudWatch Events).

Here's why:

AWS Batch is designed for running batch computing workloads on AWS. It handles the complexities of provisioning and managing compute resources, allowing you to focus on defining your tasks. It dynamically provisions the optimal quantity and type of compute resources (e.g., EC2 instances) based on the resource requirements of the jobs submitted. This dynamic scaling improves performance and scalability. The tasks, being written in different languages, benefit from Batch's ability to execute various types of applications without requiring code modifications.

Amazon EventBridge (formerly CloudWatch Events) provides a serverless event bus service that makes it easy to connect applications with data from a variety of sources. EventBridge can be used to schedule tasks in AWS Batch. This simplifies the scheduling aspect without needing to manage a separate scheduler on the EC2 instance.

Option B (AWS App Runner) is less suitable. While App Runner is simple for deploying containerized web applications and APIs, it's not optimized for batch processing or scheduled tasks. Converting to a container adds an extra layer of complexity with limited benefit for these specific task requirements.

Option C (AWS Lambda) might seem plausible, but it's generally not ideal. While Lambda can handle scheduled tasks, there's a limitation on execution time (15 minutes) and potentially memory limitations depending on the tasks' requirements. Refactoring these tasks into Lambda functions would likely require significant code modifications, increasing operational overhead and negating the requirement that teams do not have a common programming language. The 1-hour task duration also makes Lambda less suitable.

Option D (Auto Scaling group with an AMI) introduces significant operational overhead. You'd need to manage the AMI, configure scaling policies, and ensure the EC2 instances within the Auto Scaling group have the necessary dependencies and configurations. This is less efficient than using AWS Batch, which abstracts away much of the infrastructure management. The tasks would still compete for resources on the instance.

Therefore, AWS Batch with EventBridge scheduling provides the best balance of performance, scalability, and minimal operational overhead.

Relevant Documentation:

AWS Batch: https://aws.amazon.com/batch/
Amazon EventBridge: https://aws.amazon.com/eventbridge/

Question: 370 **Exam Heist**

A company runs a public three-tier web application in a VPC. The application runs on Amazon EC2 instances across multiple Availability Zones. The EC2 instances that run in private subnets need to communicate with a license

server over the internet. The company needs a managed solution that minimizes operational maintenance.

Which solution meets these requirements?

A. Provision a NAT instance in a public subnet. Modify each private subnet's route table with a default route that points to the NAT instance.

B. Provision a NAT instance in a private subnet. Modify each private subnet's route table with a default route that points to the NAT instance.

C. Provision a NAT gateway in a public subnet. Modify each private subnet's route table with a default route that points to the NAT gateway.

D. Provision a NAT gateway in a private subnet. Modify each private subnet's route table with a default route that points to the NAT gateway.

Answer: C

Explanation:

The correct answer is C. Provision a NAT gateway in a public subnet. Modify each private subnet's route table with a default route that points to the NAT gateway.

Here's a detailed justification:

The requirement is to allow EC2 instances in private subnets to access the internet (specifically, a license server) in a managed and operationally efficient way. NAT (Network Address Translation) is the service that allows instances in private subnets to connect to the internet while hiding their private IP addresses.

Options A and B involve NAT instances. While NAT instances can provide NAT functionality, they require manual patching, scaling, and are more complex to manage. The question explicitly states the need to minimize operational maintenance, making NAT instances a less desirable choice. NAT instances also represent a single point of failure if not configured for high availability.

Options C and D use NAT gateways, which are managed AWS services. This addresses the "managed solution" requirement, reducing operational overhead. NAT gateways are highly available by default and scale automatically.

The difference between options C and D lies in where the NAT gateway is placed: a public subnet or a private subnet. A NAT gateway must reside in a public subnet. This is because the NAT gateway needs a public IP address to forward traffic to the internet. If a NAT gateway were in a private subnet, it would need to route through another NAT device or an internet gateway to reach the internet, which defeats the purpose of a managed solution.

Therefore, option C is the correct answer because it uses the managed NAT gateway service located in a public subnet, minimizing operational maintenance and enabling internet access for instances in private subnets via route table modifications.

Further Research:

AWS NAT Gateway: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html
NAT Instances vs. NAT Gateways: https://aws.amazon.com/premiumsupport/knowledge-center/nat-gateway-instance-differences/

Question: 371 **Exam Heist**

A company needs to create an Amazon Elastic Kubernetes Service (Amazon EKS) cluster to host a digital media streaming application. The EKS cluster will use a managed node group that is backed by Amazon Elastic Block Store (Amazon EBS) volumes for storage. The company must encrypt all data at rest by using a customer managed key that is stored in AWS Key Management Service (AWS KMS).

Which combination of actions will meet this requirement with the LEAST operational overhead? (Choose two.)

A. Use a Kubernetes plugin that uses the customer managed key to perform data encryption.

B. After creation of the EKS cluster, locate the EBS volumes. Enable encryption by using the customer managed key.

C. Enable EBS encryption by default in the AWS Region where the EKS cluster will be created. Select the customer managed key as the default key.

D. Create the EKS cluster. Create an IAM role that has a policy that grants permission to the customer managed key. Associate the role with the EKS cluster.

E. Store the customer managed key as a Kubernetes secret in the EKS cluster. Use the customer managed key to encrypt the EBS volumes.

Answer: CD

Explanation:

The correct answer is **CD**. Here's a detailed justification:

C. Enable EBS encryption by default in the AWS Region where the EKS cluster will be created. Select the customer managed key as the default key.

This is the most efficient method. By enabling EBS encryption by default at the AWS Region level, all new EBS volumes created in that Region, including those automatically provisioned for the EKS managed node group, will be encrypted using the specified customer-managed KMS key. This eliminates the need to manually encrypt each volume individually, reducing operational overhead significantly. AWS handles the encryption automatically during EBS volume creation. This approach ensures that encryption is applied consistently and without manual intervention for future volumes as well.

D. Create the EKS cluster. Create an IAM role that has a policy that grants permission to the customer managed key. Associate the role with the EKS cluster.

EKS cluster already has its IAM role, and this role requires KMS decrypt/encrypt permissions to use KMS keys for encrypting resources. When EBS encryption by default is enabled, EKS cluster needs to have the ability to use the CMK for encryption/decryption operations during volume creation and management by Kubernetes on the worker nodes. Associating an IAM role with the EKS cluster that grants permissions to the KMS key is essential for allowing the cluster to encrypt and decrypt EBS volumes using the customer-managed key specified. Without these permissions, the encryption process will fail.

Why other options are incorrect:

A. Use a Kubernetes plugin that uses the customer managed key to perform data encryption. While possible, using a Kubernetes plugin adds complexity and operational overhead. It involves deploying and managing the plugin, configuring it correctly, and ensuring it integrates seamlessly with EBS volume creation. The default encryption feature of EBS provides a simpler solution.

B. After creation of the EKS cluster, locate the EBS volumes. Enable encryption by using the customer managed key. This approach requires manual intervention after the cluster is created, leading to higher operational overhead. It also necessitates discovering the EBS volumes and applying encryption, which is time-consuming and prone to errors. It also might not be possible to encrypt root volumes without recreating the EC2 instances behind the worker nodes.

E. Store the customer managed key as a Kubernetes secret in the EKS cluster. Use the customer managed key to encrypt the EBS volumes. This is a security risk. Storing KMS keys directly as Kubernetes secrets exposes them and violates security best practices. KMS keys should never be stored within the cluster itself. They should be managed and accessed through IAM roles with appropriate permissions.

Supporting documentation:

AWS KMS documentation
Amazon EBS encryption
EKS IAM role requirement

Question: 372 **Exam Heist**

A company wants to migrate an Oracle database to AWS. The database consists of a single table that contains millions of geographic information systems (GIS) images that are high resolution and are identified by a geographic code.

When a natural disaster occurs, tens of thousands of images get updated every few minutes. Each geographic code has a single image or row that is associated with it. The company wants a solution that is highly available and scalable during such events.

Which solution meets these requirements MOST cost-effectively?

A. Store the images and geographic codes in a database table. Use Oracle running on an Amazon RDS Multi-AZ DB instance.

B. Store the images in Amazon S3 buckets. Use Amazon DynamoDB with the geographic code as the key and the image S3 URL as the value.

C. Store the images and geographic codes in an Amazon DynamoDB table. Configure DynamoDB Accelerator (DAX) during times of high load.

D. Store the images in Amazon S3 buckets. Store geographic codes and image S3 URLs in a database table. Use Oracle running on an Amazon RDS Multi-AZ DB instance.

Answer: D

Explanation:

The correct answer is **D**, not B. Here's why:

Let's analyze each option based on the requirements of high availability, scalability, and cost-effectiveness for managing GIS images identified by geographic codes, with frequent updates during natural disasters.

Option A: Using Oracle on RDS Multi-AZ for both image storage and geographic codes is highly inefficient and expensive. Storing binary image data directly in a relational database is generally discouraged due to performance overhead and storage limitations. RDS, while highly available, isn't as horizontally scalable as other solutions for this specific use case.

Option B: Storing images in S3 and using DynamoDB to map geographic codes to S3 URLs is a good starting point, leveraging the scalability and cost-effectiveness of both services. However, DynamoDB is a NoSQL database suitable for frequent reads and writes, but may not scale in write capacity for tens of thousands of updates every few minutes.

Option C: Storing both images and geographic codes in DynamoDB is not ideal for storing large binary files like images. While DynamoDB is scalable, storing images directly in the database becomes expensive and less performant. DAX is a good caching solution, but it won't address the fundamental issue of storing large binary data within DynamoDB.

Option D: This option combines the best aspects of the other approaches. Storing images in S3 provides cost-effective, scalable, and durable storage for the large image files. Storing geographic codes and corresponding S3 URLs in Oracle on RDS Multi-AZ is also not ideal (as it can be costly), but it will allow relational queries that DynamoDB cannot perform. For example, finding all images of a certain type, size, or date, which may become necessary in the future.

Therefore, **Option B is the most cost-effective and ideal solution**.

Question: 373 **Exam Heist**

A company has an application that collects data from IoT sensors on automobiles. The data is streamed and stored in Amazon S3 through Amazon Kinesis Data Firehose. The data produces trillions of S3 objects each year. Each morning, the company uses the data from the previous 30 days to retrain a suite of machine learning (ML) models.

Four times each year, the company uses the data from the previous 12 months to perform analysis and train other ML models. The data must be available with minimal delay for up to 1 year. After 1 year, the data must be retained for archival purposes.

Which storage solution meets these requirements MOST cost-effectively?

A. Use the S3 Intelligent-Tiering storage class. Create an S3 Lifecycle policy to transition objects to S3 Glacier Deep Archive after 1 year.

B. Use the S3 Intelligent-Tiering storage class. Configure S3 Intelligent-Tiering to automatically move objects to S3 Glacier Deep Archive after 1 year.

C. Use the S3 Standard-Infrequent Access (S3 Standard-IA) storage class. Create an S3 Lifecycle policy to transition objects to S3 Glacier Deep Archive after 1 year.

D. Use the S3 Standard storage class. Create an S3 Lifecycle policy to transition objects to S3 Standard-Infrequent Access (S3 Standard-IA) after 30 days, and then to S3 Glacier Deep Archive after 1 year.

Answer: D

Explanation:

Here's a detailed justification for why option D is the most cost-effective solution, along with supporting concepts and links:

The core requirement is to minimize storage costs while providing timely access to data for specific periods, followed by long-term archival. The data is actively used for 30 days, less frequently for up to a year, and then archived.

Option D utilizes S3 Standard initially because the data is accessed frequently during the first 30 days for daily model retraining. S3 Standard offers the lowest access latency, ensuring rapid data retrieval for this critical process. After 30 days, the data is transitioned to S3 Standard-IA via an S3 Lifecycle policy. This significantly reduces storage costs because S3 Standard-IA is designed for infrequently accessed data (but still requires quick retrieval for the quarterly analysis). Finally, after a year, the data is moved to S3 Glacier Deep Archive, the cheapest storage option for long-term archival. This aligns perfectly with the archival requirement and minimizes storage costs for data that is rarely accessed.

Other options are less optimal:

Option A: S3 Intelligent-Tiering automatically moves data between Frequent, Infrequent, and Archive Access tiers based on access patterns. While convenient, the transition between tiers incurs costs that can be higher than directly transitioning to S3 Standard-IA via a lifecycle policy after a fixed period. Intelligent-Tiering also doesn't directly transition to Glacier Deep Archive.
Option B: Similar to Option A, S3 Intelligent-Tiering doesn't directly offer the lowest-cost archival option and can be more expensive than a lifecycle policy. It would move to Archive Access (not Glacier Deep Archive), and there's no inherent transition to Deep Archive.
Option C: While S3 Standard-IA is cheaper than S3 Standard upfront, using S3 Standard for the initial 30 days is more performant for frequently accessed data, which is a primary need for daily model retraining. This initial period of frequent access justifies the slightly higher cost of S3 Standard.

In summary, option D is the most cost-effective because it uses the appropriate storage class based on access

frequency and lifespan of the data. Lifecycle policies make the transitions automated and efficient, which optimize both performance and cost.

Supporting Concepts:

S3 Storage Classes: Understand the cost and access characteristics of S3 Standard, S3 Standard-IA, and S3 Glacier Deep Archive.
S3 Lifecycle Policies: Learn how to automate transitions between storage classes to optimize costs.

Authoritative Links:

Amazon S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
Managing Your Storage Lifecycle: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-concept.html

Question: 374 **Exam Heist**

A company is running several business applications in three separate VPCs within the us-east-1 Region. The applications must be able to communicate between VPCs. The applications also must be able to consistently send hundreds of gigabytes of data each day to a latency-sensitive application that runs in a single on-premises data center.

A solutions architect needs to design a network connectivity solution that maximizes cost-effectiveness.

Which solution meets these requirements?

A. Configure three AWS Site-to-Site VPN connections from the data center to AWS. Establish connectivity by configuring one VPN connection for each VPC.

B. Launch a third-party virtual network appliance in each VPC. Establish an IPsec VPN tunnel between the data center and each virtual appliance.

C. Set up three AWS Direct Connect connections from the data center to a Direct Connect gateway in us-east-1. Establish connectivity by configuring each VPC to use one of the Direct Connect connections.

D. Set up one AWS Direct Connect connection from the data center to AWS. Create a transit gateway, and attach each VPC to the transit gateway. Establish connectivity between the Direct Connect connection and the transit gateway.

Answer: D

Explanation:

The correct answer is D because it provides the most cost-effective and scalable solution for the given requirements. The scenario requires inter-VPC communication and high-bandwidth, low-latency connectivity to an on-premises data center.

Option D leverages AWS Direct Connect for the high-bandwidth, low-latency connection between the data center and AWS. Direct Connect is a dedicated network connection that bypasses the public internet, resulting in more consistent network performance compared to VPN connections. A single Direct Connect connection is more cost-effective than multiple connections, especially for large data transfers.

The Transit Gateway simplifies the network architecture by acting as a central hub. Instead of configuring multiple peering connections between VPCs, each VPC is connected to the Transit Gateway. The Transit Gateway then handles the routing between the VPCs and the Direct Connect connection. This simplifies management and reduces operational overhead.

Options A and B are less optimal because Site-to-Site VPN connections over the public internet are not designed for consistently transferring hundreds of gigabytes of data daily with low latency. VPN connections are also less reliable and have variable performance. Option B adds the overhead of managing virtual network

appliances in each VPC.

Option C is costly, and it complicates routing. Setting up three Direct Connect connections would be expensive, and managing three separate connections to each VPC is complex. Also, Direct Connect gateway can be shared between the VPCs and the Direct Connect Connection, hence Direct Connect Gateway is a must in Direct connect architecture.

In summary, option D provides the optimal balance of cost, performance, and manageability by using a single Direct Connect connection for high-bandwidth, low-latency access and a Transit Gateway for simplified inter-VPC communication.

Relevant AWS documentation:

AWS Direct Connect: https://aws.amazon.com/directconnect/
AWS Transit Gateway: https://aws.amazon.com/transit-gateway/

Question: 375 **Exam Heist**

An ecommerce company is building a distributed application that involves several serverless functions and AWS services to complete order-processing tasks. These tasks require manual approvals as part of the workflow. A solutions architect needs to design an architecture for the order-processing application. The solution must be able to combine multiple AWS Lambda functions into responsive serverless applications. The solution also must orchestrate data and services that run on Amazon EC2 instances, containers, or on-premises servers.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use AWS Step Functions to build the application.
B.Integrate all the application components in an AWS Glue job.
C.Use Amazon Simple Queue Service (Amazon SQS) to build the application.
D.Use AWS Lambda functions and Amazon EventBridge events to build the application.

Answer: A

Explanation:

The correct answer is **A. Use AWS Step Functions to build the application.**

Here's a detailed justification:

AWS Step Functions is a serverless orchestration service that allows you to coordinate multiple AWS services into serverless workflows. It provides a visual console to design and manage workflows, making it easy to combine Lambda functions and other AWS services into complex, responsive serverless applications. Crucially, Step Functions can orchestrate not only Lambda functions but also data and services running on EC2 instances, containers, or on-premises servers via API Gateway integrations or other appropriate services. This aligns directly with the requirement to integrate diverse components regardless of their deployment environment.

Step Functions offers built-in error handling, retries, and state management, reducing the operational overhead associated with managing these aspects manually. It also supports human-in-the-loop approval processes using activities, aligning with the manual approval requirement. It uses state machines, which clearly define the sequence of steps and the flow of data between them, increasing clarity and maintainability.

Option B, AWS Glue, is primarily an ETL (Extract, Transform, Load) service for data integration and is not designed for orchestrating general-purpose application workflows involving Lambda functions and manual approvals.

Option C, Amazon SQS, is a message queue service that enables asynchronous communication between decoupled components. While SQS can be part of a solution, it doesn't provide orchestration capabilities like Step Functions. Building a complex, stateful workflow with SQS alone would require significant custom coding and management of state, increasing operational overhead.

Option D, AWS Lambda functions and Amazon EventBridge events, could orchestrate events, but it is not designed to manage more complex workflows that require features like state management, built-in retries, and human approval steps. It quickly leads to complex event chains and increased operational overhead for tracking and error handling. It's better suited for event-driven architectures than orchestrated workflows with specific sequencing and state.

Therefore, Step Functions provides the best combination of features for orchestrating Lambda functions, integrating with various AWS services and on-premises resources, and supporting manual approvals with the least operational overhead.

Supporting links:

AWS Step Functions: https://aws.amazon.com/step-functions/
Orchestrating Microservices with AWS Step Functions:
https://aws.amazon.com/blogs/compute/orchestrating-microservices-with-aws-step-functions/

Question: 376 **Exam Heist**

A company has launched an Amazon RDS for MySQL DB instance. Most of the connections to the database come from serverless applications. Application traffic to the database changes significantly at random intervals. At times of high demand, users report that their applications experience database connection rejection errors.

Which solution will resolve this issue with the LEAST operational overhead?

A. Create a proxy in RDS Proxy. Configure the users' applications to use the DB instance through RDS Proxy.

B. Deploy Amazon ElastiCache for Memcached between the users' applications and the DB instance.

C. Migrate the DB instance to a different instance class that has higher I/O capacity. Configure the users' applications to use the new DB instance.

D. Configure Multi-AZ for the DB instance. Configure the users' applications to switch between the DB instances.

Answer: A

Explanation:

The correct answer is A: Create a proxy in RDS Proxy. Configure the users' applications to use the DB instance through RDS Proxy.

Here's why:

The problem describes connection rejection errors due to high demand and unpredictable traffic patterns from serverless applications to an RDS for MySQL database. Serverless applications, by their nature, frequently scale up and down, leading to a burst of database connections which can overwhelm the database's connection limits.

RDS Proxy is designed to handle exactly this scenario. It acts as a connection pooler, sitting between the applications and the RDS instance. RDS Proxy maintains a pool of database connections and efficiently multiplexes connections from numerous application instances, reusing existing connections as possible and preventing the database from being overwhelmed. It can significantly reduce the overhead of establishing and tearing down database connections, which is common with serverless functions.

Option B, deploying Amazon ElastiCache for Memcached, addresses caching and data retrieval performance, not connection management. While caching can reduce database load, it doesn't directly solve the connection limit issue.

Option C, migrating to a larger instance class, might increase the database's connection limit, but this is a vertical scaling approach. It increases costs and might not fully address the rapid fluctuations in traffic. It's also a relatively disruptive operation compared to implementing RDS Proxy. It doesn't solve the fundamental problem of connection exhaustion caused by numerous, short-lived connections.

Option D, configuring Multi-AZ, provides high availability and failover capabilities, but it does not manage database connections effectively. While Multi-AZ ensures the database remains available, it doesn't prevent connection rejection errors when connection limits are exceeded. The applications would still be generating numerous connection requests.

RDS Proxy provides a cost-effective and minimally disruptive solution by effectively managing and pooling database connections, mitigating the connection rejection errors experienced by the serverless applications. Its primary benefit is connection multiplexing, which is the ideal solution in this specific context.

Authoritative Links:

AWS RDS Proxy: https://aws.amazon.com/rds/proxy/
Using RDS Proxy with Lambda: https://aws.amazon.com/blogs/database/using-amazon-rds-proxy-with-aws-lambda/

Question: 377 **Exam Heist**

A company recently deployed a new auditing system to centralize information about operating system versions, patching, and installed software for Amazon EC2 instances. A solutions architect must ensure all instances provisioned through EC2 Auto Scaling groups successfully send reports to the auditing system as soon as they are launched and terminated.

Which solution achieves these goals MOST efficiently?

A. Use a scheduled AWS Lambda function and run a script remotely on all EC2 instances to send data to the audit system.

B. Use EC2 Auto Scaling lifecycle hooks to run a custom script to send data to the audit system when instances are launched and terminated.

C. Use an EC2 Auto Scaling launch configuration to run a custom script through user data to send data to the audit system when instances are launched and terminated.

D. Run a custom script on the instance operating system to send data to the audit system. Configure the script to be invoked by the EC2 Auto Scaling group when the instance starts and is terminated.

Answer: B

Explanation:

Here's a detailed justification for why option B is the most efficient solution for ensuring EC2 instances within Auto Scaling groups report to the auditing system upon launch and termination:

Option B, utilizing EC2 Auto Scaling lifecycle hooks, is the most efficient because it directly integrates with the Auto Scaling group's lifecycle events. Lifecycle hooks allow you to pause instances during launch or termination, enabling you to perform custom actions before the instance proceeds to the next phase. In this case, the custom action would be running a script to send data to the auditing system. This ensures that the reporting occurs reliably and automatically as part of the Auto Scaling process. This method provides clear control and ensures reports are sent before the instance serves traffic (on launch) or before resources are deallocated (on termination).

Option A, using a scheduled Lambda function, is less efficient. It requires the Lambda function to continuously scan for new and terminated instances, which adds overhead and increases complexity. Remotely executing scripts also introduces potential security concerns and operational challenges related to authentication and authorization. It's not tightly coupled with the Auto Scaling lifecycle.

Option C, relying on user data within the launch configuration, is suitable for initial configuration during launch but doesn't address the termination scenario. Furthermore, user data scripts can sometimes fail or be delayed, making it less reliable for critical reporting. It also doesn't handle the termination process, meaning you would need another solution for that phase of the Auto Scaling process.

Option D is incorrect because it places the responsibility of invoking the script on the instance operating system and relies on the Auto Scaling group to trigger the script, this isn't a default function of Auto Scaling groups, lifecycle hooks are needed to trigger the actions at launch and terminate.

In summary, EC2 Auto Scaling lifecycle hooks (Option B) provide the most direct, reliable, and efficient means of executing a custom script for reporting to the auditing system during both instance launch and termination events within an Auto Scaling group. It ensures integration with the autoscaling process and provides guaranteed execution.

Supporting links:

EC2 Auto Scaling Lifecycle Hooks

Question: 378 **Exam Heist**

A company is developing a real-time multiplayer game that uses UDP for communications between the client and servers in an Auto Scaling group. Spikes in demand are anticipated during the day, so the game server platform must adapt accordingly. Developers want to store gamer scores and other non-relational data in a database solution that will scale without intervention.

Which solution should a solutions architect recommend?

A.Use Amazon Route 53 for traffic distribution and Amazon Aurora Serverless for data storage.

B.Use a Network Load Balancer for traffic distribution and Amazon DynamoDB on-demand for data storage.

C.Use a Network Load Balancer for traffic distribution and Amazon Aurora Global Database for data storage.

D.Use an Application Load Balancer for traffic distribution and Amazon DynamoDB global tables for data storage.

Answer: B

Explanation:

Here's a detailed justification for why option B is the best solution:

The scenario requires a solution that effectively handles UDP traffic distribution to a dynamically scaling game server fleet and a database solution suitable for non-relational data that scales automatically.

Network Load Balancer (NLB): The NLB is the optimal choice for traffic distribution. UDP is a connectionless protocol. NLBs are designed to handle UDP traffic efficiently. Unlike Application Load Balancers (ALB), which operate at the application layer (HTTP/HTTPS), NLBs operate at the transport layer (TCP/UDP) and can forward UDP packets without modification. This is crucial for real-time gaming where low latency and direct packet forwarding are paramount.
https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html

Amazon DynamoDB on-demand: DynamoDB is a NoSQL database perfectly suited for storing game scores and other non-relational data. The on-demand capacity mode is ideal because it automatically scales capacity

based on the application's traffic patterns. This eliminates the need for manual provisioning or capacity planning, directly addressing the requirement for a database solution that scales without intervention. It is very cost effective for variable workloads.
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadWriteCapacityMode.html

Why other options are less suitable:

Option A (Route 53 and Aurora Serverless): Route 53 is primarily a DNS service and not suitable for real-time traffic distribution in this scenario. It is great for geo-based routing but not real time load balancing. Aurora Serverless is good for relational data, not specifically designed for non-relational requirements like the prompt specifies.
Option C (NLB and Aurora Global Database): While NLB is correct for UDP traffic, Aurora Global Database is an overkill. It's designed for disaster recovery across regions for relational data and provides read replicas for global low-latency reads. Overcomplicated, high cost, and relational DB makes this a poor solution.
Option D (ALB and DynamoDB Global Tables): ALBs are designed for HTTP/HTTPS traffic and cannot handle UDP. While DynamoDB global tables are a viable solution for multi-region replication, the on-demand capacity mode is more cost-effective and simpler for automatically scaling with the described workload within a single region. ALB makes this entire solution wrong.

In summary, NLB provides the necessary low-latency, UDP-aware traffic distribution, and DynamoDB on-demand delivers the automatically scalable NoSQL database, meeting all the requirements efficiently and cost-effectively.

Question: 379 **Exam Heist**

A company hosts a frontend application that uses an Amazon API Gateway API backend that is integrated with AWS Lambda. When the API receives requests, the Lambda function loads many libraries. Then the Lambda function connects to an Amazon RDS database, processes the data, and returns the data to the frontend application. The company wants to ensure that response latency is as low as possible for all its users with the fewest number of changes to the company's operations.

Which solution will meet these requirements?

A.Establish a connection between the frontend application and the database to make queries faster by bypassing the API.

B.Configure provisioned concurrency for the Lambda function that handles the requests.

C.Cache the results of the queries in Amazon S3 for faster retrieval of similar datasets.

D.Increase the size of the database to increase the number of connections Lambda can establish at one time.

Answer: B

Explanation:

The best solution is to configure provisioned concurrency for the Lambda function. This is because Lambda functions, particularly those that load many libraries, suffer from "cold starts." A cold start happens when a Lambda function is invoked for the first time or after a period of inactivity. During a cold start, the Lambda service needs to initialize the execution environment, download the code, and initialize the dependencies, leading to increased latency.

Provisioned concurrency alleviates this problem by pre-initializing a specified number of execution environments for the Lambda function. These environments are kept warm and ready to serve requests immediately, eliminating the cold start latency. This directly addresses the company's desire for low response latency.

Option A, bypassing the API and connecting the frontend directly to the database, is a security risk and poor

architectural practice. It exposes the database directly and bypasses the API's security measures, authentication, and authorization, which makes the application vulnerable to attacks. Option C, caching results in S3, is beneficial if the data is largely static or frequently requested but doesn't address the cold start issue with the Lambda function itself. Furthermore, implementing a caching strategy introduces complexity to manage cache invalidation. Option D, increasing the database size, won't significantly reduce Lambda function's latency. The database size primarily affects query performance and storage capacity.

Provisioned concurrency offers the most direct and efficient solution with minimal operational changes, as it focuses on improving the Lambda function's performance without altering the overall application architecture or introducing caching complexities.

For further research:

AWS Lambda Provisioned Concurrency: https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html
Understanding AWS Lambda Cold Starts: https://aws.amazon.com/blogs/compute/understanding-aws-lambda-cold-starts/

Question: 380 **Exam Heist**

A company is migrating its on-premises workload to the AWS Cloud. The company already uses several Amazon EC2 instances and Amazon RDS DB instances. The company wants a solution that automatically starts and stops the EC2 instances and DB instances outside of business hours. The solution must minimize cost and infrastructure maintenance.

Which solution will meet these requirements?

A. Scale the EC2 instances by using elastic resize. Scale the DB instances to zero outside of business hours.

B. Explore AWS Marketplace for partner solutions that will automatically start and stop the EC2 instances and DB instances on a schedule.

C. Launch another EC2 instance. Configure a crontab schedule to run shell scripts that will start and stop the existing EC2 instances and DB instances on a schedule.

D. Create an AWS Lambda function that will start and stop the EC2 instances and DB instances. Configure Amazon EventBridge to invoke the Lambda function on a schedule.

Answer: D

Explanation:

The correct answer is D because it offers a cost-effective and infrastructure-minimized solution for automatically starting and stopping EC2 instances and RDS DB instances outside of business hours. AWS Lambda is a serverless compute service, meaning you only pay for the compute time you consume. This is significantly cheaper than running a dedicated EC2 instance. Amazon EventBridge (formerly CloudWatch Events) is a serverless event bus service that enables you to schedule events to trigger actions, such as invoking a Lambda function.

Option D leverages these serverless services to automate the start/stop process without requiring the company to manage any underlying infrastructure. A Lambda function can be written to execute the necessary AWS CLI or SDK commands to start and stop both EC2 instances and RDS DB instances. EventBridge can be configured with a cron expression to trigger the Lambda function at specified times, thereby implementing the desired on/off schedule. This eliminates the need for manual intervention and minimizes operational overhead.

Other options are less suitable:

A: Elastic resize is not a native term; most likely implying Elastic Load Balancing Auto Scaling. Auto Scaling

groups manage capacity, not start/stop functionality directly according to a precise schedule. Scaling RDS DB instances to zero is not a standard or supported operation.
B: AWS Marketplace solutions might exist, but they introduce third-party dependencies and potential costs. The requirement is to minimize cost and infrastructure maintenance. This adds complexity.
C: Launching another EC2 instance to run a cron schedule adds infrastructure management overhead and cost that is not necessary since the solution can be implemented serverlessly.

Therefore, utilizing Lambda with EventBridge fulfills the requirements of cost minimization, reduced infrastructure maintenance, and automation of instance start/stop procedures based on a defined schedule.

Supporting Links:

AWS Lambda: https://aws.amazon.com/lambda/
Amazon EventBridge: https://aws.amazon.com/eventbridge/
Stopping and Starting Your RDS Instance:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_StopInstance.html
Stopping and Starting Your EC2 Instance:
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/Stop_Start.html

Question: 381 **Exam Heist**

A company hosts a three-tier web application that includes a PostgreSQL database. The database stores the metadata from documents. The company searches the metadata for key terms to retrieve documents that the company reviews in a report each month. The documents are stored in Amazon S3. The documents are usually written only once, but they are updated frequently.

The reporting process takes a few hours with the use of relational queries. The reporting process must not prevent any document modifications or the addition of new documents. A solutions architect needs to implement a solution to speed up the reporting process.

Which solution will meet these requirements with the LEAST amount of change to the application code?

A. Set up a new Amazon DocumentDB (with MongoDB compatibility) cluster that includes a read replica. Scale the read replica to generate the reports.

B. Set up a new Amazon Aurora PostgreSQL DB cluster that includes an Aurora Replica. Issue queries to the Aurora Replica to generate the reports.

C. Set up a new Amazon RDS for PostgreSQL Multi-AZ DB instance. Configure the reporting module to query the secondary RDS node so that the reporting module does not affect the primary node.

D. Set up a new Amazon DynamoDB table to store the documents. Use a fixed write capacity to support new document entries. Automatically scale the read capacity to support the reports.

Answer: B

Explanation:

The correct answer is B: Set up a new Amazon Aurora PostgreSQL DB cluster that includes an Aurora Replica. Issue queries to the Aurora Replica to generate the reports.

Here's why: The key requirement is to speed up the reporting process without impacting the primary database's performance related to document modifications and additions. The existing setup uses PostgreSQL.

Option B leverages Amazon Aurora PostgreSQL, which is compatible with PostgreSQL. Aurora provides significantly better performance than standard PostgreSQL, which directly addresses the speed requirement. The critical aspect of this solution is the use of an Aurora Replica. Aurora Replicas are read-only endpoints that share the same underlying storage as the primary instance. This means data is replicated asynchronously with minimal latency, enabling near real-time reporting. The reporting process can query the Aurora Replica

without affecting the performance of the primary instance that handles document modifications and additions. Furthermore, Aurora's architecture allows for easy scaling of read replicas to handle report generation loads. This approach minimizes code changes since Aurora PostgreSQL is compatible with the existing PostgreSQL database.

Option A is incorrect because Amazon DocumentDB is a NoSQL database compatible with MongoDB. Migrating the data to DocumentDB would necessitate significant application code changes, which contradicts the requirement of minimizing such changes. Additionally, while DocumentDB offers read replicas, it's an unnecessary database technology shift when a compatible solution exists within the relational database context.

Option C is incorrect because while RDS for PostgreSQL Multi-AZ provides high availability, it's not optimized for read-heavy workloads like reporting. The secondary RDS node in a Multi-AZ setup is primarily for failover and might not be optimized for reporting queries. Using it for reporting could still potentially impact the performance of the primary node if there's significant load.

Option D is incorrect because DynamoDB is a NoSQL database, and migrating the existing PostgreSQL data and relational queries to DynamoDB would require a complete application rewrite. This is a substantial change that is not in line with the given requirements. While DynamoDB can scale reads, the conversion effort outweighs the benefits compared to leveraging Aurora's PostgreSQL compatibility and Aurora Replica functionality.

In conclusion, Aurora PostgreSQL with an Aurora Replica provides the optimal balance of minimal application changes, improved performance, and isolation of the reporting workload from the primary database.

Relevant Links:

Amazon Aurora: https://aws.amazon.com/rds/aurora/
Aurora Replicas:
https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Managing.Replication.html
Amazon RDS: https://aws.amazon.com/rds/
Amazon DocumentDB: https://aws.amazon.com/documentdb/
Amazon DynamoDB: https://aws.amazon.com/dynamodb/

Question: 382 **Exam Heist**

A company has a three-tier application on AWS that ingests sensor data from its users' devices. The traffic flows through a Network Load Balancer (NLB), then to Amazon EC2 instances for the web tier, and finally to EC2 instances for the application tier. The application tier makes calls to a database.

What should a solutions architect do to improve the security of the data in transit?

A.Configure a TLS listener. Deploy the server certificate on the NLB.

B.Configure AWS Shield Advanced. Enable AWS WAF on the NLB.

C.Change the load balancer to an Application Load Balancer (ALB). Enable AWS WAF on the ALB.

D.Encrypt the Amazon Elastic Block Store (Amazon EBS) volume on the EC2 instances by using AWS Key Management Service (AWS KMS).

Answer: A

Explanation:

The correct answer is **A. Configure a TLS listener. Deploy the server certificate on the NLB.**

Here's a detailed justification:

The primary goal is to improve the security of data in transit. TLS (Transport Layer Security) encrypts data as it travels across the network, protecting it from eavesdropping and tampering. Configuring a TLS listener on the NLB ensures that all traffic between the user's devices and the NLB is encrypted. Deploying the server certificate on the NLB allows the NLB to decrypt the traffic and re-encrypt it before forwarding it to the web tier. This establishes a secure connection from the client to the load balancer.

Option B is incorrect because AWS Shield Advanced primarily protects against DDoS attacks, not general data-in-transit encryption. While AWS WAF (Web Application Firewall) helps protect against web exploits, it doesn't address the fundamental need for encryption across the network.

Option C is incorrect because while an Application Load Balancer (ALB) can terminate TLS, the core requirement is TLS termination at the entry point, which the NLB can handle just as effectively. Switching to an ALB only to add WAF isn't the most direct solution for securing data in transit; TLS configuration on the NLB itself directly addresses this concern. Besides, an NLB is often chosen specifically for its high throughput and low latency, which might be critical for ingesting real-time sensor data, and switching to an ALB might introduce performance trade-offs.

Option D is incorrect because encrypting the EBS volumes only protects data at rest. While data-at-rest encryption is important, it does not address the immediate need to secure the data while it's being transmitted between the users' devices, the NLB, the EC2 instances, and the database.

In summary, configuring a TLS listener on the NLB with a server certificate is the most direct and effective way to secure the sensor data in transit in this three-tier architecture. It addresses the specific problem stated in the question without introducing unnecessary complexity or altering the fundamental architecture if the NLB is performant for the application's demands.

Here are authoritative links for further research:

AWS Load Balancing Documentation:
https://docs.aws.amazon.com/elasticloadbalancing/latest/network/load-balancer-listeners.html
TLS Termination on Elastic Load Balancers:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-listeners.html#https-listeners (While this link refers to ALBs, the principle of TLS termination applies to NLBs as well).
AWS Key Management Service (KMS): https://aws.amazon.com/kms/
AWS Shield: https://aws.amazon.com/shield/
AWS WAF: https://aws.amazon.com/waf/

Question: 383 **Exam Heist**

A company is planning to migrate a commercial off-the-shelf application from its on-premises data center to AWS. The software has a software licensing model using sockets and cores with predictable capacity and uptime requirements. The company wants to use its existing licenses, which were purchased earlier this year.

Which Amazon EC2 pricing option is the MOST cost-effective?

A. Dedicated Reserved Hosts
B. Dedicated On-Demand Hosts
C. Dedicated Reserved Instances
D. Dedicated On-Demand Instances

Answer: A

Explanation:

Here's a detailed justification for why option A, Dedicated Hosts with Reserved Instances, is the most cost-

effective solution in the scenario described:

The key requirement is utilizing existing software licenses based on sockets and cores. Dedicated Hosts provide physical servers dedicated for your use, enabling you to bring your existing server-bound software licenses to AWS. This avoids the need to purchase new licenses, which can be a significant cost savings when migrating commercial off-the-shelf (COTS) applications.

On-Demand options (B and D) for Dedicated Hosts are expensive for predictable capacity and uptime. Since the company has predictable capacity and uptime requirements, a long-term commitment will significantly reduce costs.

Reserved Instances (C) offer a discount compared to On-Demand Instances in exchange for a one- or three-year commitment. However, Reserved Instances do not by themselves ensure you can use your existing licenses tied to specific hardware. Reserved Instances still run on shared tenancy hardware managed by AWS.

Dedicated Hosts with Reserved Instances (A) combine the advantages of both. You get the physical server isolation to use your existing software licenses, and you receive a substantial cost reduction through the reservation. Reserving a Dedicated Host provides the most significant cost savings over time when compared to paying on-demand rates for a dedicated host.

Therefore, Dedicated Reserved Hosts offer the most cost-effective solution by letting the company leverage existing licenses (socket/core-based) on dedicated hardware while benefiting from the cost savings of reserved pricing. This aligns with the business requirements for predictable capacity, uptime, and license utilization. Dedicated Hosts allow for granular control over instance placement, useful for licensing constraints, whereas Dedicated Instances are for compliance reasons.

Authoritative Links:

Amazon EC2 Dedicated Hosts: https://aws.amazon.com/ec2/dedicated-hosts/
Amazon EC2 Pricing: https://aws.amazon.com/ec2/pricing/
Reserved Instances: https://aws.amazon.com/ec2/pricing/reserved-instances/

Question: 384 **Exam Heist**

A company runs an application on Amazon EC2 Linux instances across multiple Availability Zones. The application needs a storage layer that is highly available and Portable Operating System Interface (POSIX)-compliant. The storage layer must provide maximum data durability and must be shareable across the EC2 instances. The data in the storage layer will be accessed frequently for the first 30 days and will be accessed infrequently after that time.

Which solution will meet these requirements MOST cost-effectively?

A.Use the Amazon S3 Standard storage class. Create an S3 Lifecycle policy to move infrequently accessed data to S3 Glacier.

B.Use the Amazon S3 Standard storage class. Create an S3 Lifecycle policy to move infrequently accessed data to S3 Standard-Infrequent Access (S3 Standard-IA).

C.Use the Amazon Elastic File System (Amazon EFS) Standard storage class. Create a lifecycle management policy to move infrequently accessed data to EFS Standard-Infrequent Access (EFS Standard-IA).

D.Use the Amazon Elastic File System (Amazon EFS) One Zone storage class. Create a lifecycle management policy to move infrequently accessed data to EFS One Zone-Infrequent Access (EFS One Zone-IA).

Answer: C

Explanation:

The correct answer is C because it addresses all the stated requirements most cost-effectively. The application requires a highly available and POSIX-compliant storage layer shareable across EC2 instances.

Amazon Elastic File System (EFS) is a network file system specifically designed for use with AWS compute services like EC2. EFS provides a POSIX-compliant file system interface, allowing applications to interact with the storage layer as if it were a local file system. This satisfies the POSIX compliance and shareability requirements. Using the EFS Standard storage class ensures high availability because EFS replicates data across multiple Availability Zones.

The data access pattern also dictates the choice of storage class. Because the data is frequently accessed for the first 30 days and infrequently after that, leveraging EFS lifecycle management is the most cost-effective solution. EFS lifecycle management allows automatic transitioning of files from the EFS Standard storage class to the EFS Standard-Infrequent Access (EFS Standard-IA) storage class based on a defined policy. This reduces storage costs for infrequently accessed data without requiring any application changes.

Option A is incorrect because S3 is an object storage service, not a file system, and does not natively support POSIX compliance. Option B is incorrect for the same reason as A, S3 isn't POSIX compliant. Option D is incorrect because while EFS One Zone is a cost-effective option, it's not highly available, as it only stores data within a single Availability Zone, violating the availability requirement. EFS One Zone is suitable for workloads where durability is less important than cost.

Therefore, EFS Standard with lifecycle management to EFS Standard-IA is the optimal solution balancing cost, availability, POSIX compliance, and data access patterns.

Relevant links for further research:

Amazon EFS: https://aws.amazon.com/efs/
EFS Storage Classes: https://docs.aws.amazon.com/efs/latest/ug/storage-classes.html
EFS Lifecycle Management: https://docs.aws.amazon.com/efs/latest/ug/lifecycle-management-efs.html

Question: 385 **Exam Heist**

A solutions architect is creating a new VPC design. There are two public subnets for the load balancer, two private subnets for web servers, and two private subnets for MySQL. The web servers use only HTTPS. The solutions architect has already created a security group for the load balancer allowing port 443 from 0.0.0.0/0. Company policy requires that each resource has the least access required to still be able to perform its tasks.

Which additional configuration strategy should the solutions architect use to meet these requirements?

A. Create a security group for the web servers and allow port 443 from 0.0.0.0/0. Create a security group for the MySQL servers and allow port 3306 from the web servers security group.

B. Create a network ACL for the web servers and allow port 443 from 0.0.0.0/0. Create a network ACL for the MySQL servers and allow port 3306 from the web servers security group.

C. Create a security group for the web servers and allow port 443 from the load balancer. Create a security group for the MySQL servers and allow port 3306 from the web servers security group.

D. Create a network ACL for the web servers and allow port 443 from the load balancer. Create a network ACL for the MySQL servers and allow port 3306 from the web servers security group.

Answer: C

Explanation:

The correct answer is C because it adheres to the principle of least privilege using security groups, which operate at the instance level, providing granular control.

Option C suggests creating a security group for the web servers that allows inbound traffic on port 443 only from the load balancer's security group. This ensures that only the load balancer can send HTTPS traffic to the web servers, restricting access from other sources. Subsequently, it proposes a security group for the MySQL servers, permitting inbound traffic on port 3306 (MySQL's default port) solely from the web servers'

security group. This setup allows the web servers to communicate with the database while preventing any other resources from directly accessing the MySQL servers.

Option A allows port 443 from 0.0.0.0/0 to the web servers, which violates the least privilege principle by opening access to the entire internet.Options B and D use Network ACLs (NACLs). NACLs operate at the subnet level and are stateless, meaning you need to configure both inbound and outbound rules, making security group management for inter-instance communication easier to administer.

Therefore, option C is the only choice that properly leverages security groups to restrict access to the minimum necessary, fulfilling the company's policy requirements.

AWS Security GroupsAWS Network ACLs

Question: 386 **Exam Heist**

An ecommerce company is running a multi-tier application on AWS. The front-end and backend tiers both run on Amazon EC2, and the database runs on Amazon RDS for MySQL. The backend tier communicates with the RDS instance. There are frequent calls to return identical datasets from the database that are causing performance slowdowns.

Which action should be taken to improve the performance of the backend?

A. Implement Amazon SNS to store the database calls.
B. Implement Amazon ElastiCache to cache the large datasets.
C. Implement an RDS for MySQL read replica to cache database calls.
D. Implement Amazon Kinesis Data Firehose to stream the calls to the database.

Answer: B

Explanation:

The most effective solution to improve the backend performance involves caching frequently requested, identical datasets.

Option B, implementing Amazon ElastiCache, is the correct approach. ElastiCache is a fully managed, in-memory data store and caching service. By caching frequently accessed data from the RDS for MySQL database in ElastiCache, the backend application can retrieve the data much faster than querying the database each time. This reduces the load on the RDS instance and significantly improves the response time for the backend. ElastiCache supports Memcached and Redis, both suitable for caching datasets. Memcached provides a distributed memory object caching system, while Redis offers more advanced data structures and features.

Why other options are incorrect:

Option A (Amazon SNS): Amazon SNS (Simple Notification Service) is a messaging service used for pub/sub architectures. It's not designed for caching data.
Option C (RDS Read Replica): While read replicas can offload read traffic from the primary RDS instance, they still query the database. This doesn't eliminate the latency associated with database queries for identical datasets. Caching avoids database queries altogether for commonly requested information, offering better performance gains in this specific scenario. Read replicas are more beneficial for scaling read operations rather than caching identical data.
Option D (Amazon Kinesis Data Firehose): Amazon Kinesis Data Firehose is a service for streaming data to data lakes, data stores, and analytics services. It is not suitable for caching data or improving the backend application's read performance in the context described. Firehose is mainly used for ingestion of streaming data for analytics and long term storage.

In conclusion, ElastiCache provides a dedicated caching layer that directly addresses the issue of performance slowdowns caused by frequent, redundant database calls. It significantly reduces database load and improves response times for the backend application by serving frequently accessed data directly from memory.

Authoritative Links:

Amazon ElastiCache: https://aws.amazon.com/elasticache/
Caching Strategies: https://aws.amazon.com/caching/

Question: 387 **Exam Heist**

A new employee has joined a company as a deployment engineer. The deployment engineer will be using AWS CloudFormation templates to create multiple AWS resources. A solutions architect wants the deployment engineer to perform job activities while following the principle of least privilege.

Which combination of actions should the solutions architect take to accomplish this goal? (Choose two.)

A.Have the deployment engineer use AWS account root user credentials for performing AWS CloudFormation stack operations.

B.Create a new IAM user for the deployment engineer and add the IAM user to a group that has the PowerUsers IAM policy attached.

C.Create a new IAM user for the deployment engineer and add the IAM user to a group that has the AdministratorAccess IAM policy attached.

D.Create a new IAM user for the deployment engineer and add the IAM user to a group that has an IAM policy that allows AWS CloudFormation actions only.

E.Create an IAM role for the deployment engineer to explicitly define the permissions specific to the AWS CloudFormation stack and launch stacks using that IAM role.

Answer: DE

Explanation:

The principle of least privilege dictates granting only the permissions required to perform a specific task. Options A, B, and C are incorrect because they violate this principle.

Option A is wrong because using the root user grants unrestricted access to all AWS services and resources, posing a significant security risk. Root user credentials should be used extremely sparingly, only for tasks requiring it, such as changing the account settings.

Option B and C are wrong because PowerUser or AdministratorAccess policies grant broad permissions that are not necessary for CloudFormation deployments. This level of access could be misused, creating security vulnerabilities.

Option D, creating an IAM user in a group with an IAM policy allowing only CloudFormation actions, aligns with the least privilege principle. It restricts the deployment engineer to CloudFormation-specific tasks, reducing the risk of accidental or malicious actions on other AWS resources. However, even this can be refined further.

Option E, creating an IAM role with specific permissions for the CloudFormation stack and launching stacks using that role, represents an even more granular application of the least privilege principle. This approach grants the exact permissions needed for that specific stack, limiting any potential blast radius if the credentials are compromised. This provides better security over a blanket CloudFormation only policy as suggested in option D.

Therefore, D and E are the most appropriate actions to follow the principle of least privilege.

Relevant links:

IAM Best Practices: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html
IAM Roles: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html
CloudFormation Security Considerations:
https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/best-practices.html

Question: 388 **Exam Heist**

A company is deploying a two-tier web application in a VPC. The web tier is using an Amazon EC2 Auto Scaling group with public subnets that span multiple Availability Zones. The database tier consists of an Amazon RDS for MySQL DB instance in separate private subnets. The web tier requires access to the database to retrieve product information.

The web application is not working as intended. The web application reports that it cannot connect to the database. The database is confirmed to be up and running. All configurations for the network ACLs, security groups, and route tables are still in their default states.

What should a solutions architect recommend to fix the application?

A.Add an explicit rule to the private subnet's network ACL to allow traffic from the web tier's EC2 instances.

B.Add a route in the VPC route table to allow traffic between the web tier's EC2 instances and the database tier.

C.Deploy the web tier's EC2 instances and the database tier's RDS instance into two separate VPCs, and configure VPC peering.

D.Add an inbound rule to the security group of the database tier's RDS instance to allow traffic from the web tiers security group.

Answer: D

Explanation:

The correct answer is **D. Add an inbound rule to the security group of the database tier's RDS instance to allow traffic from the web tiers security group.**

Here's why:

The problem is that the web tier cannot connect to the RDS database, despite the database being up and running. The question states that network ACLs and route tables are in their default states. Default network ACLs allow all inbound and outbound traffic. The default route table allows all traffic within the VPC. This means the problem likely lies with the security groups, which act as virtual firewalls at the instance level.

RDS DB instances use security groups to control network access. By default, a security group denies all inbound traffic. Therefore, the RDS instance's security group is likely blocking connections from the web tier.

To fix this, we need to add an inbound rule to the RDS instance's security group that allows traffic from the web tier's security group. This allows instances in the web tier's security group to connect to the RDS instance on the appropriate port (typically 3306 for MySQL). Specifying the source as the web tier's security group is best practice, as it allows the RDS instance to only accept traffic from authorized sources within the VPC without specifying IP addresses.

Why the other options are incorrect:

A. Add an explicit rule to the private subnet's network ACL to allow traffic from the web tier's EC2 instances: Network ACLs, by default, allow all traffic. Adding explicit rules is unnecessary in this scenario and won't solve the issue, as the restriction is likely at the security group level.

B. Add a route in the VPC route table to allow traffic between the web tier's EC2 instances and the database tier: The default route table already allows all traffic within the VPC, so adding another route won't change anything. The issue isn't routing; it's access control.

C. Deploy the web tier's EC2 instances and the database tier's RDS instance into two separate VPCs, and configure VPC peering: This is a complex solution that is not necessary to resolve the connection issue within the existing VPC. It introduces more administrative overhead than simply adjusting the security group rules. VPC peering is used to connect separate VPCs, which isn't the problem at hand.

Authoritative Links:

Amazon VPC Security Groups: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html
Amazon RDS Security Groups: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_WorkingWithSecurityGroups.html

Question: 389 **Exam Heist**

A company has a large dataset for its online advertising business stored in an Amazon RDS for MySQL DB instance in a single Availability Zone. The company wants business reporting queries to run without impacting the write operations to the production DB instance.

Which solution meets these requirements?

A. Deploy RDS read replicas to process the business reporting queries.
B. Scale out the DB instance horizontally by placing it behind an Elastic Load Balancer.
C. Scale up the DB instance to a larger instance type to handle write operations and queries.
D. Deploy the DB instance in multiple Availability Zones to process the business reporting queries.

Answer: A

Explanation:

The correct answer is A: Deploy RDS read replicas to process the business reporting queries. This solution effectively isolates the business reporting workload from the production database, preventing any performance impact on write operations.

RDS Read Replicas provide a mechanism for asynchronous replication of data from a source RDS database instance (in this case, MySQL) to one or more read-only copies. These replicas reside in separate DB instances and can handle read-heavy workloads without affecting the primary instance's performance. Business reporting typically involves complex and resource-intensive queries that can consume significant database resources. By directing these queries to the read replicas, the primary instance remains dedicated to handling write operations and transactional workloads, ensuring optimal performance for the online advertising business.

Option B is incorrect. Placing an RDS instance behind an Elastic Load Balancer does not scale the database horizontally in the traditional sense of sharing the data. ELB primarily handles distributing connection requests. While it could distribute read queries, it won't prevent the primary instance from being burdened by the reporting queries. It's mostly useful for read scale when multiple instances host the same data and the instances are identical in terms of data.

Option C is also incorrect. Scaling up the DB instance (vertical scaling) might improve performance temporarily, but it doesn't isolate the reporting workload. The reporting queries will still compete for resources with the write operations, eventually leading to performance bottlenecks as the dataset grows and query complexity increases. Moreover, there is a limit to how much you can vertically scale.

Option D is also incorrect. Deploying the DB instance in multiple Availability Zones (Multi-AZ) primarily provides high availability and failover capabilities. While it creates a standby instance, this standby is for failover purposes and isn't designed for actively handling read queries during normal operations. Using it for read operations is discouraged and inefficient since the primary instance is still handling both reads and writes.In summary, RDS Read Replicas are the ideal solution for offloading read-only workloads like business reporting from the primary RDS instance, ensuring that write performance remains unaffected and maximizing overall database performance and availability.

Further Resources:

Amazon RDS Read Replicas
Working with MySQL Read Replicas

Question: 390 **Exam Heist**

A company hosts a three-tier ecommerce application on a fleet of Amazon EC2 instances. The instances run in an Auto Scaling group behind an Application Load Balancer (ALB). All ecommerce data is stored in an Amazon RDS for MariaDB Multi-AZ DB instance.

The company wants to optimize customer session management during transactions. The application must store session data durably.

Which solutions will meet these requirements? (Choose two.)

A. Turn on the sticky sessions feature (session affinity) on the ALB.
B. Use an Amazon DynamoDB table to store customer session information.
C. Deploy an Amazon Cognito user pool to manage user session information.
D. Deploy an Amazon ElastiCache for Redis cluster to store customer session information.
E. Use AWS Systems Manager Application Manager in the application to manage user session information.

Answer: AD

Explanation:

The requirement is to optimize customer session management for an e-commerce application running on EC2 instances behind an ALB, with durable storage of session data.

Option A, turning on sticky sessions (session affinity) on the ALB, addresses session management directly. Sticky sessions ensure that a user's requests are consistently routed to the same EC2 instance within the Auto Scaling group. This is crucial for maintaining session state and avoiding data loss during transactions. ALB sticky sessions use cookies to track which server the client should be routed to. https://docs.aws.amazon.com/elasticloadbalancing/latest/application/sticky-sessions.html

Option D, deploying an Amazon ElastiCache for Redis cluster, provides durable and in-memory storage for customer session information. Redis, being an in-memory data store, offers fast read and write operations, significantly improving session retrieval speed. The question specified the session data must be stored durably, and Redis can be configured with persistence to disk (RDB or AOF), ensuring that session data survives instance failures. Redis Cluster automatically partitions your data among multiple Redis nodes. It improves availability and performance for large datasets. https://aws.amazon.com/elasticache/redis/

Option B is incorrect because while DynamoDB provides durable storage, it is not the optimal choice for session management due to higher latency compared to an in-memory solution like ElastiCache Redis, especially for frequently accessed session data.

Option C, using Amazon Cognito, primarily handles authentication and authorization (identity management), which is a different aspect of the application than session data persistence. Cognito could be used for user

authentication but doesn't satisfy the durability requirement for session data within the context of active transactions.

Option E, using AWS Systems Manager Application Manager, focuses on managing application operations and infrastructure, rather than directly handling session data. It provides centralized views of applications and insights into their operational status. This does not address the session management or durability requirements.

Question: 391 **Exam Heist**

A company needs a backup strategy for its three-tier stateless web application. The web application runs on Amazon EC2 instances in an Auto Scaling group with a dynamic scaling policy that is configured to respond to scaling events. The database tier runs on Amazon RDS for PostgreSQL. The web application does not require temporary local storage on the EC2 instances. The company's recovery point objective (RPO) is 2 hours.

The backup strategy must maximize scalability and optimize resource utilization for this environment.

Which solution will meet these requirements?

A. Take snapshots of Amazon Elastic Block Store (Amazon EBS) volumes of the EC2 instances and database every 2 hours to meet the RPO.

B. Configure a snapshot lifecycle policy to take Amazon Elastic Block Store (Amazon EBS) snapshots. Enable automated backups in Amazon RDS to meet the RPO.

C. Retain the latest Amazon Machine Images (AMIs) of the web and application tiers. Enable automated backups in Amazon RDS and use point-in-time recovery to meet the RPO.

D. Take snapshots of Amazon Elastic Block Store (Amazon EBS) volumes of the EC2 instances every 2 hours. Enable automated backups in Amazon RDS and use point-in-time recovery to meet the RPO.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best solution and why the others are less suitable:

Why Option C is Correct:

Stateless Web Application & AMIs: The EC2 instances are stateless, meaning they don't store persistent data. Creating and retaining AMIs (Amazon Machine Images) captures the configuration and software setup of the EC2 instances. When scaling events occur, the Auto Scaling group can launch new instances using the latest AMI, ensuring consistency and rapid deployment. Since there's no application data to back up on the instances themselves, EBS snapshots are redundant and unnecessary for the web tier.
RDS Automated Backups & Point-in-Time Recovery: Amazon RDS's automated backup feature creates regular snapshots of the database and transaction logs. Enabling automated backups and using point-in-time recovery (PITR) allows you to restore the database to any point in time within the backup retention period, meeting the 2-hour RPO. PITR uses both the snapshots and transaction logs for granular recovery.
Scalability and Resource Optimization: AMIs are efficient for deploying stateless applications because they capture the pre-configured instance state. RDS automated backups are designed for database durability without requiring manual snapshot management. This approach minimizes manual intervention, optimizes resource utilization, and scales effectively with the application.

Why Other Options Are Incorrect:

Option A & D: EBS Snapshots for Stateless Instances: Taking EBS snapshots of the EC2 instances in a stateless web application is unnecessary. Since the instances are stateless, snapshots offer no benefit for application data recovery. This wastes storage resources and management overhead.

Option B: Snapshot lifecycle for EBS & RDS Automated Backups: It is important to note that option B does not address the EC2 instances.
Reliance on EBS snapshot for RDS: While automated backups are correct for RDS, the other options also recommend EBS snapshots for the stateless EC2 instances, making them redundant and less optimal.

In summary, Option C provides the most efficient and scalable solution for backing up a stateless web application on EC2 and a PostgreSQL database on RDS. It leverages AMIs for the stateless web tier and RDS automated backups with PITR for the database tier, meeting the required RPO while optimizing resource utilization.

Supporting Links:

Amazon Machine Images (AMIs): https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AMIs.html
Amazon RDS Automated Backups:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_WorkingWithAutomatedBackups.html
Amazon RDS Point-in-Time Recovery:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_PIT.html

Question: 392 **Exam Heist**

A company wants to deploy a new public web application on AWS. The application includes a web server tier that uses Amazon EC2 instances. The application also includes a database tier that uses an Amazon RDS for MySQL DB instance.

The application must be secure and accessible for global customers that have dynamic IP addresses.

How should a solutions architect configure the security groups to meet these requirements?

A.Configure the security group for the web servers to allow inbound traffic on port 443 from 0.0.0.0/0. Configure the security group for the DB instance to allow inbound traffic on port 3306 from the security group of the web servers.

B.Configure the security group for the web servers to allow inbound traffic on port 443 from the IP addresses of the customers. Configure the security group for the DB instance to allow inbound traffic on port 3306 from the security group of the web servers.

C.Configure the security group for the web servers to allow inbound traffic on port 443 from the IP addresses of the customers. Configure the security group for the DB instance to allow inbound traffic on port 3306 from the IP addresses of the customers.

D.Configure the security group for the web servers to allow inbound traffic on port 443 from 0.0.0.0/0. Configure the security group for the DB instance to allow inbound traffic on port 3306 from 0.0.0.0/0.

Answer: A

Explanation:

The correct answer is A because it implements the principle of least privilege while ensuring global accessibility.

Here's a detailed justification:

Web Server Security Group: The web servers need to be accessible from the internet for global customers. Since the customers have dynamic IP addresses, specifying individual IP addresses (as suggested in options B and C) is impractical and unmanageable. Opening port 443 (HTTPS) to 0.0.0.0/0 allows traffic from any IP address to reach the web servers securely, enabling global access. This is acceptable because the web servers are designed to handle public-facing traffic.
Database Security Group: Exposing the database directly to the internet (as suggested in option D) is a significant security risk. Instead, the database should only accept connections from the web servers. This is

achieved by configuring the DB instance's security group to allow inbound traffic on port 3306 (MySQL's default port) only from the security group of the web servers. This means only EC2 instances associated with the web server's security group can connect to the database. This approach isolates the database and protects it from unauthorized access from the internet.
Why other options are incorrect: Option B and C are incorrect because specifying individual customer IP addresses is not feasible with dynamic IP addresses. Option D is wrong because it opens the database to the entire internet, which is a major security vulnerability.

In summary, option A balances accessibility and security. It allows global customers to access the web application while restricting database access to only the necessary components (the web servers). This configuration follows AWS best practices for securing multi-tier applications.

Relevant AWS Documentation:

Security Groups: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html
RDS Security:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Overview.RDSSecurityGroups.html

Question: 393 **Exam Heist**

A payment processing company records all voice communication with its customers and stores the audio files in an Amazon S3 bucket. The company needs to capture the text from the audio files. The company must remove from the text any personally identifiable information (PII) that belongs to customers.

What should a solutions architect do to meet these requirements?

A. Process the audio files by using Amazon Kinesis Video Streams. Use an AWS Lambda function to scan for known PII patterns.

B. When an audio file is uploaded to the S3 bucket, invoke an AWS Lambda function to start an Amazon Textract task to analyze the call recordings.

C. Configure an Amazon Transcribe transcription job with PII redaction turned on. When an audio file is uploaded to the S3 bucket, invoke an AWS Lambda function to start the transcription job. Store the output in a separate S3 bucket.

D. Create an Amazon Connect contact flow that ingests the audio files with transcription turned on. Embed an AWS Lambda function to scan for known PII patterns. Use Amazon EventBridge to start the contact flow when an audio file is uploaded to the S3 bucket.

Answer: C

Explanation:

The correct answer is C because it directly addresses the requirements of transcribing audio files and redacting PII, utilizing services specifically designed for these tasks.

Option C utilizes Amazon Transcribe, a service designed for speech-to-text conversion. Transcribe's built-in PII redaction feature automatically identifies and removes sensitive information from the transcribed text, satisfying the PII removal requirement efficiently. Configuring a Lambda function to trigger the transcription job upon S3 upload provides an automated, event-driven workflow. Storing the output in a separate S3 bucket ensures separation of the original audio and the processed text, further enhancing security and organization. This approach minimizes custom code and leverages AWS managed services for core functionality, adhering to best practices for scalability and maintainability.

Option A is less ideal because Kinesis Video Streams is primarily for real-time video and audio ingestion, not for batch processing of existing audio files in S3. Scanning for PII patterns within a Lambda function would require custom development and maintenance of PII detection logic, which is less efficient than using Transcribe's built-in feature.

Option B uses Amazon Textract, which is primarily designed for extracting text from documents and images, not audio. While Textract can process images of spectrograms generated from audio, this is an indirect and inefficient approach compared to using a dedicated transcription service like Transcribe.

Option D suggests Amazon Connect, which is a contact center as a service. While Connect offers transcription capabilities, it's typically used for real-time call center interactions. Using Connect for batch processing S3 audio files is an overkill and not its intended use case. Scanning for PII using Lambda within a contact flow is also less efficient and scalable than utilizing Transcribe's PII redaction feature.

Therefore, option C offers the most direct, efficient, and cost-effective solution by leveraging Amazon Transcribe's built-in PII redaction capabilities and automating the process with Lambda triggers.

Relevant Links:

Amazon Transcribe: https://aws.amazon.com/transcribe/
Amazon Transcribe PII Redaction: https://docs.aws.amazon.com/transcribe/latest/dg/pii-redaction.html
AWS Lambda: https://aws.amazon.com/lambda/
Amazon S3: https://aws.amazon.com/s3/

Question: 394 **Exam Heist**

A company is running a multi-tier ecommerce web application in the AWS Cloud. The application runs on Amazon EC2 instances with an Amazon RDS for MySQL Multi-AZ DB instance. Amazon RDS is configured with the latest generation DB instance with 2,000 GB of storage in a General Purpose SSD (gp3) Amazon Elastic Block Store (Amazon EBS) volume. The database performance affects the application during periods of high demand.

A database administrator analyzes the logs in Amazon CloudWatch Logs and discovers that the application performance always degrades when the number of read and write IOPS is higher than 20,000.

What should a solutions architect do to improve the application performance?

A.Replace the volume with a magnetic volume.
B.Increase the number of IOPS on the gp3 volume.
C.Replace the volume with a Provisioned IOPS SSD (io2) volume.
D.Replace the 2,000 GB gp3 volume with two 1,000 GB gp3 volumes.

Answer: D

Explanation:

The correct answer is **B. Increase the number of IOPS on the gp3 volume.**

Here's why and why the other options are incorrect:

The problem states that performance degrades when IOPS exceed 20,000. The current gp3 volume is the bottleneck. gp3 volumes offer a baseline performance and allow you to provision additional IOPS if needed. The root cause is that the provisioned IOPS are insufficient for the application's peak demand. Therefore, increasing the provisioned IOPS is the most direct and efficient way to address the performance issue.

Option A: Replace the volume with a magnetic volume. Magnetic volumes are the lowest performance EBS volume type. Switching to magnetic volumes would drastically decrease performance and is the opposite of what the scenario requires.

Option C: Replace the volume with a Provisioned IOPS SSD (io2) volume. While io2 volumes offer higher IOPS performance than gp3, it's not the most cost-effective initial approach. The best approach is to maximize IOPS within the gp3 volume before upgrading the volume. In addition, the newer io2 Block Express volume is

significantly higher in performance and would cost significantly more.

Option D: Replace the 2,000 GB gp3 volume with two 1,000 GB gp3 volumes. Splitting the volume doesn't inherently increase the total IOPS available to the database. The limit for IOPS depends on the volume size (up to 16,000 IOPS per GB). This might improve performance in some specific IO patterns, but the problem statement clearly indicates the issue is exceeding the total IOPS limit. Also, it does not give the needed IOPS for the EC2 instance.

Justification for increasing IOPS on the gp3 volume:

1. The problem identifies that the bottleneck is the RDS database when read/write IOPS exceed 20,000.
2. The application is already using gp3 volumes, a general-purpose SSD volume type designed to balance price and performance.
3. gp3 volumes allow provisioning specific IOPS, meaning you can increase the IOPS as needed for a given volume size.
4. The gp3 volume type has a maximum IOPS to size ratio. Increasing the volume size and then increasing IOPS is a viable solution.
5. Increasing IOPS directly addresses the stated problem: the database's inability to handle high IOPS demands.
6. It's the most cost-effective solution for the current information.

Authoritative Links:

Amazon EBS volume types: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html
gp3 volumes: https://aws.amazon.com/ebs/gp3/

Question: 395 **Exam Heist**

An IAM user made several configuration changes to AWS resources in their company's account during a production deployment last week. A solutions architect learned that a couple of security group rules are not configured as desired. The solutions architect wants to confirm which IAM user was responsible for making changes.

Which service should the solutions architect use to find the desired information?

A. Amazon GuardDuty
B. Amazon Inspector
C. AWS CloudTrail
D. AWS Config

Answer: C

Explanation:

The correct answer is **C. AWS CloudTrail.**

AWS CloudTrail is a service that enables governance, compliance, operational auditing, and risk auditing of your AWS account. CloudTrail logs API calls made to AWS services, including the identity of the caller (IAM user, role, or AWS account), the time of the call, the source IP address, the request parameters, and the response elements returned by the AWS service. This allows you to track changes made to your AWS resources and identify the IAM user responsible for specific actions. In this scenario, the solutions architect needs to identify the IAM user who modified the security group rules. CloudTrail provides the necessary audit logs to pinpoint the user and the exact configuration changes they made.

Here's why the other options are incorrect:

A. Amazon GuardDuty: GuardDuty is a threat detection service that continuously monitors your AWS accounts and workloads for malicious activity and unauthorized behavior. It doesn't provide detailed logs of configuration changes made by IAM users.
B. Amazon Inspector: Inspector is a vulnerability management service that automatically assesses AWS workloads for software vulnerabilities and unintended network exposure. It helps improve the security and compliance of applications deployed on AWS but does not track IAM user actions.
D. AWS Config: Config enables you to assess, audit, and evaluate the configurations of your AWS resources. While Config can detect that a security group rule has drifted from its desired state, it is CloudTrail that provides the comprehensive audit trail of who made the specific change. Config works well with Cloudtrail data to provide a detailed audit history.

In summary, CloudTrail is the ideal service for identifying the IAM user responsible for making configuration changes to AWS resources due to its API call logging capabilities.

Further research:

AWS CloudTrail: https://aws.amazon.com/cloudtrail/

Question: 396 **Exam Heist**

A company has implemented a self-managed DNS service on AWS. The solution consists of the following:

• Amazon EC2 instances in different AWS Regions
• Endpoints of a standard accelerator in AWS Global Accelerator

The company wants to protect the solution against DDoS attacks.

What should a solutions architect do to meet this requirement?

A. Subscribe to AWS Shield Advanced. Add the accelerator as a resource to protect.
B. Subscribe to AWS Shield Advanced. Add the EC2 instances as resources to protect.
C. Create an AWS WAF web ACL that includes a rate-based rule. Associate the web ACL with the accelerator.
D. Create an AWS WAF web ACL that includes a rate-based rule. Associate the web ACL with the EC2 instances.

Answer: A

Explanation:

The correct answer is A: Subscribe to AWS Shield Advanced. Add the accelerator as a resource to protect.

Here's why:

DDoS Protection with AWS Shield: AWS Shield is a managed DDoS protection service that safeguards applications running on AWS. It comes in two tiers: Standard and Advanced.

AWS Shield Standard: This tier provides automatic, always-on protection against common, frequently occurring network and transport layer DDoS attacks. It's included at no extra cost for all AWS customers. It protects your AWS resources.

AWS Shield Advanced: This tier provides enhanced DDoS protection for applications running on EC2, Elastic Load Balancing (ELB), CloudFront, Global Accelerator, and Route 53. Key benefits include:

24/7 DDoS response team support: Access to AWS DDoS experts who can help mitigate sophisticated attacks.
Visibility into DDoS attacks: Detailed monitoring and reporting to understand attack patterns.

Cost protection: Credits for usage spikes caused by DDoS attacks.

Global Accelerator and DDoS Protection: Global Accelerator uses a static IP address that acts as a single point of contact for your applications, distributed across multiple AWS Regions. This makes it a prime target for DDoS attacks, but also a strategic point to implement protection. Shield Advanced can protect Global Accelerator endpoints.

Why other options are incorrect:

Option B (Protecting EC2 instances directly with Shield Advanced): While possible, it's less effective in this scenario. The Global Accelerator provides a centralized entry point. Protecting the accelerator directly shields the underlying EC2 instances from the initial onslaught of DDoS traffic.

Options C and D (Using AWS WAF): AWS WAF is a web application firewall that helps protect your web applications from common web exploits. It can mitigate some DDoS attacks, especially those at the application layer (Layer 7). However, it doesn't provide the comprehensive, network-layer protection of AWS Shield Advanced. More specifically, AWS WAF is better suited for filtering out malicious requests based on HTTP headers, cookies, or other application-level data and is insufficient for volumetric DDoS attacks. While associating WAF with the accelerator might offer some defense, the best practice is to use Shield Advanced, which integrates with WAF to provide a layered defense. Furthermore, WAF rate-based rules can help with some types of denial of service attacks, they are not a replacement for Shield Advanced.

In Summary: Since the company wants to protect against DDoS attacks and is already using Global Accelerator, subscribing to AWS Shield Advanced and protecting the accelerator is the most effective and comprehensive solution. This provides network and transport layer protection while simplifying the configuration.

Authoritative Links:

AWS Shield: https://aws.amazon.com/shield/
AWS Global Accelerator: https://aws.amazon.com/global-accelerator/
AWS WAF: https://aws.amazon.com/waf/

Question: 397 **Exam Heist**

An ecommerce company needs to run a scheduled daily job to aggregate and filter sales records for analytics. The company stores the sales records in an Amazon S3 bucket. Each object can be up to 10 GB in size. Based on the number of sales events, the job can take up to an hour to complete. The CPU and memory usage of the job are constant and are known in advance.

A solutions architect needs to minimize the amount of operational effort that is needed for the job to run.

Which solution meets these requirements?

A.Create an AWS Lambda function that has an Amazon EventBridge notification. Schedule the EventBridge event to run once a day.

B.Create an AWS Lambda function. Create an Amazon API Gateway HTTP API, and integrate the API with the function. Create an Amazon EventBridge scheduled event that calls the API and invokes the function.

C.Create an Amazon Elastic Container Service (Amazon ECS) cluster with an AWS Fargate launch type. Create an Amazon EventBridge scheduled event that launches an ECS task on the cluster to run the job.

D.Create an Amazon Elastic Container Service (Amazon ECS) cluster with an Amazon EC2 launch type and an Auto Scaling group with at least one EC2 instance. Create an Amazon EventBridge scheduled event that launches an ECS task on the cluster to run the job.

Answer: C

Explanation:

Here's a detailed justification for why option C is the most suitable solution, along with why the other options are less ideal:

Option C: Create an Amazon Elastic Container Service (Amazon ECS) cluster with an AWS Fargate launch type. Create an Amazon EventBridge scheduled event that launches an ECS task on the cluster to run the job.

Justification: This option provides the best balance of managed services, scalability, and operational simplicity.

ECS with Fargate: Fargate is a serverless compute engine for containers. This eliminates the need to manage underlying EC2 instances, handling patching, scaling, and maintenance for you. This significantly reduces operational overhead. You specify the CPU and memory requirements for your container (as these are known in advance), and Fargate provisions the resources.
Containerization: Using a container (e.g., a Docker image) allows you to package your application and its dependencies consistently. This eliminates the "it works on my machine" problem and ensures the application runs identically across different environments.
Amazon EventBridge: EventBridge provides a serverless event bus that allows you to schedule the job to run daily. It can directly trigger an ECS task execution based on a defined schedule. This creates a fully automated process.
Scalability: ECS with Fargate can easily scale based on the resource requirements of the job. You define resource limits, and Fargate handles the scaling of underlying resources.
Suitable Duration: The job duration is up to one hour, which is generally well within the limits of ECS tasks on Fargate.
Cost-Effective: You only pay for the resources consumed during the task execution. This can be more cost-effective than having a dedicated EC2 instance running.
Operational Efficiency: No EC2 instance management, container orchestration through ECS, scheduled executions by EventBridge all result in minimized operational effort.

Why other options are less ideal:

Option A: Create an AWS Lambda function that has an Amazon EventBridge notification. Schedule the EventBridge event to run once a day.

Limitation: Lambda has a maximum execution time limit of 15 minutes. The job can take up to an hour to complete, making Lambda unsuitable.
Memory Limitation: Lambda functions also have memory limits. While the memory usage is known, complex aggregation and filtering of large sales records (up to 10 GB per object) might exceed these limits, particularly if multiple objects are processed in parallel.

Option B: Create an AWS Lambda function. Create an Amazon API Gateway HTTP API, and integrate the API with the function. Create an Amazon EventBridge scheduled event that calls the API and invokes the function.

Limitation: Similar to Option A, Lambda's execution time limit of 15 minutes renders this solution ineffective for a job that can take up to an hour. Adding API Gateway introduces unnecessary complexity.

Option D: Create an Amazon Elastic Container Service (Amazon ECS) cluster with an Amazon EC2 launch type and an Auto Scaling group with at least one EC2 instance. Create an Amazon EventBridge scheduled event that launches an ECS task on the cluster to run the job.

Increased Operational Overhead: This option requires managing EC2 instances. This includes patching, scaling, and maintenance. While Auto Scaling helps, it still introduces more operational overhead compared to Fargate.
Cost: Running EC2 instances even when the job is not executing can be less cost-effective than Fargate,

which only charges you for the resources used during the job's execution.

Authoritative Links:

AWS ECS: https://aws.amazon.com/ecs/
AWS Fargate: https://aws.amazon.com/fargate/
Amazon EventBridge: https://aws.amazon.com/eventbridge/
AWS Lambda Limits: https://docs.aws.amazon.com/lambda/latest/dg/services-limits.html

In summary, Option C, leveraging ECS with Fargate and EventBridge, offers the most suitable solution by providing a managed, scalable, cost-effective, and operationally efficient approach to running the scheduled daily job.

Question: 398 **Exam Heist**

A company needs to transfer 600 TB of data from its on-premises network-attached storage (NAS) system to the AWS Cloud. The data transfer must be complete within 2 weeks. The data is sensitive and must be encrypted in transit. The company's internet connection can support an upload speed of 100 Mbps.

Which solution meets these requirements MOST cost-effectively?

A.Use Amazon S3 multi-part upload functionality to transfer the files over HTTPS.

B.Create a VPN connection between the on-premises NAS system and the nearest AWS Region. Transfer the data over the VPN connection.

C.Use the AWS Snow Family console to order several AWS Snowball Edge Storage Optimized devices. Use the devices to transfer the data to Amazon S3.

D.Set up a 10 Gbps AWS Direct Connect connection between the company location and the nearest AWS Region. Transfer the data over a VPN connection into the Region to store the data in Amazon S3.

Answer: C

Explanation:

Here's a detailed justification for why option C is the most cost-effective solution, along with supporting concepts and links:

Option C, utilizing AWS Snowball Edge Storage Optimized devices, is the most cost-effective because it addresses the time constraint and security requirements without incurring excessive costs. Let's break it down:

1. **Data Volume and Time Constraint:** Transferring 600 TB of data in 2 weeks (14 days) with a 100 Mbps connection is infeasible. 100 Mbps equates to roughly 12.5 MBps. To transfer 600 TB (600,000 GB) would take approximately (600,000 GB * 1024 MB/GB) / 12.5 MBps = 49,152,000 seconds, or about 569 days. This far exceeds the 14-day requirement.
2. **Snowball Edge for Large Data Transfer:** AWS Snowball Edge is designed for moving large amounts of data into and out of AWS. It provides physical storage devices you can ship to your location, load with data, and then ship back to AWS for upload to S3. This circumvents the limitations of the internet connection. https://aws.amazon.com/snowball/
3. **Security:** Snowball Edge devices encrypt data both in transit and at rest, meeting the sensitive data requirement. AWS provides tamper-evident packaging and secure data erasure services.
4. **Cost-Effectiveness:** While Snowball Edge has upfront costs, it is more cost-effective than the other options:

Option A (S3 Multi-part Upload): Unrealistic due to the internet bandwidth limitation, rendering the large upload impossible within the timeframe. It is also vulnerable to errors and disconnections over such a long time.

Option B (VPN): Still limited by the 100 Mbps internet connection. While a VPN provides secure transport, the data transfer will still be prohibitively slow.

Option D (Direct Connect): Direct Connect provides a dedicated network connection to AWS. However, setting up a 10 Gbps Direct Connect connection is significantly more expensive than using Snowball Edge, especially if the large data transfer is a one-time event. The costs involve installation fees, monthly port fees, and data transfer charges. A VPN on top of Direct Connect, while secure, adds further complexity and costs.

5. **Cost Breakdown (Illustrative):** Snowball Edge Storage Optimized pricing varies by region and contract. It is typically charged on a per-day usage basis plus data transfer costs into S3. This would be considerably cheaper than Direct Connect with monthly port fees and VPN setup.

In summary, AWS Snowball Edge offers the optimal balance of speed, security, and cost-effectiveness for a one-time, large-scale data migration when limited by internet bandwidth. It bypasses the limitations of the internet connection and securely transports the data to AWS within the required timeframe, making it the most suitable choice.

Question: 399 **Exam Heist**

A financial company hosts a web application on AWS. The application uses an Amazon API Gateway Regional API endpoint to give users the ability to retrieve current stock prices. The company's security team has noticed an increase in the number of API requests. The security team is concerned that HTTP flood attacks might take the application offline.

A solutions architect must design a solution to protect the application from this type of attack.

Which solution meets these requirements with the LEAST operational overhead?

A. Create an Amazon CloudFront distribution in front of the API Gateway Regional API endpoint with a maximum TTL of 24 hours.

B. Create a Regional AWS WAF web ACL with a rate-based rule. Associate the web ACL with the API Gateway stage.

C. Use Amazon CloudWatch metrics to monitor the Count metric and alert the security team when the predefined rate is reached.

D. Create an Amazon CloudFront distribution with in front of the API Gateway Regional API endpoint. Create an AWS Lambda function to block requests from IP addresses that exceed the predefined rate.

Answer: B

Explanation:

The correct answer is **B. Create a Regional AWS WAF web ACL with a rate-based rule. Associate the web ACL with the API Gateway stage.**

Here's a detailed justification:

The primary goal is to protect the API Gateway application from HTTP flood attacks (DDoS) with minimal operational overhead. AWS WAF is specifically designed for this purpose. A rate-based rule in WAF allows you to specify the maximum number of requests allowed from a single IP address within a defined period (e.g., 5 minutes). If an IP exceeds this threshold, WAF can block or take other defined actions (like CAPTCHA)

against the offending IP.

Associating the WAF web ACL directly with the API Gateway stage offers the most direct and efficient protection. WAF integrates seamlessly with API Gateway and requires minimal configuration and maintenance after the initial setup. The protection is applied at the regional level, safeguarding the application from attacks originating from within the AWS region.

Option A (CloudFront with a maximum TTL) primarily addresses caching and content delivery. While it can help reduce load on the API Gateway, it doesn't directly mitigate HTTP flood attacks. The high TTL could also lead to stale stock prices.

Option C (CloudWatch metrics and alerts) only provides monitoring and notification, which doesn't automatically prevent the attacks. Manual intervention is needed to respond to alerts, resulting in higher operational overhead and potentially a delayed response to ongoing attacks.

Option D (CloudFront with and Lambda) is more complex and introduces operational overhead. adds latency. While it can provide custom IP blocking, it requires managing and maintaining a Lambda function, which is more complex than using AWS WAF's built-in rate-limiting feature. Also, it is better to use AWS WAF's built-in rate limiting rules.

Therefore, AWS WAF provides the most effective and least operationally intensive solution for mitigating HTTP flood attacks against an API Gateway endpoint. It automatically blocks traffic exceeding defined rates, preventing the application from being overwhelmed.

Relevant links:

AWS WAF: https://aws.amazon.com/waf/
AWS WAF Rate-Based Rule: https://docs.aws.amazon.com/waf/latest/developerguide/waf-rule-statement-type-rate-based.html
API Gateway and WAF: https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-control-access-aws-waf.html

Question: 400 **Exam Heist**

A meteorological startup company has a custom web application to sell weather data to its users online. The company uses Amazon DynamoDB to store its data and wants to build a new service that sends an alert to the managers of four internal teams every time a new weather event is recorded. The company does not want this new service to affect the performance of the current application.

What should a solutions architect do to meet these requirements with the LEAST amount of operational overhead?

A. Use DynamoDB transactions to write new event data to the table. Configure the transactions to notify internal teams.

B. Have the current application publish a message to four Amazon Simple Notification Service (Amazon SNS) topics. Have each team subscribe to one topic.

C. Enable Amazon DynamoDB Streams on the table. Use triggers to write to a single Amazon Simple Notification Service (Amazon SNS) topic to which the teams can subscribe.

D. Add a custom attribute to each record to flag new items. Write a cron job that scans the table every minute for items that are new and notifies an Amazon Simple Queue Service (Amazon SQS) queue to which the teams can subscribe.

Answer: C

Explanation:

Option C is the best solution because it leverages DynamoDB Streams, which provides a near real-time, ordered stream of item-level modifications in a DynamoDB table. This is a managed service that minimizes

operational overhead. By enabling DynamoDB Streams and configuring a trigger (e.g., an AWS Lambda function), the solution can asynchronously process new weather event records without impacting the performance of the existing web application. The Lambda function can then publish to a single SNS topic, simplifying management by having all teams subscribe to one topic, rather than multiple topics.

Option A is not optimal because using DynamoDB transactions might impact the performance of the current application, especially under heavy write loads, as transactions are more resource-intensive than regular writes. Configuring transactions to notify internal teams is also not a built-in feature, requiring significant custom coding and management.

Option B isn't as efficient. Directly publishing to four SNS topics from the current application couples the application tightly with the notification service. This increases complexity within the application and potentially affects its performance. Managing four separate SNS topics adds unnecessary overhead.

Option D is the least efficient. Scanning a DynamoDB table every minute is highly inefficient and expensive, especially as the table grows. This approach is also disruptive to DynamoDB's performance and scalability. Using SQS is unnecessary when SNS can directly notify the teams. Moreover, managing a cron job adds operational overhead.

Therefore, option C offers the best balance of minimal operational overhead, asynchronous processing to avoid performance impact, and a manageable notification strategy using a single SNS topic.

Relevant links:

Amazon DynamoDB Streams
Amazon SNS
AWS Lambda Triggers

Question: 401 **Exam Heist**

A company wants to use the AWS Cloud to make an existing application highly available and resilient. The current version of the application resides in the company's data center. The application recently experienced data loss after a database server crashed because of an unexpected power outage.

The company needs a solution that avoids any single points of failure. The solution must give the application the ability to scale to meet user demand.

Which solution will meet these requirements?

A.Deploy the application servers by using Amazon EC2 instances in an Auto Scaling group across multiple Availability Zones. Use an Amazon RDS DB instance in a Multi-AZ configuration.

B.Deploy the application servers by using Amazon EC2 instances in an Auto Scaling group in a single Availability Zone. Deploy the database on an EC2 instance. Enable EC2 Auto Recovery.

C.Deploy the application servers by using Amazon EC2 instances in an Auto Scaling group across multiple Availability Zones. Use an Amazon RDS DB instance with a read replica in a single Availability Zone. Promote the read replica to replace the primary DB instance if the primary DB instance fails.

D.Deploy the application servers by using Amazon EC2 instances in an Auto Scaling group across multiple Availability Zones. Deploy the primary and secondary database servers on EC2 instances across multiple Availability Zones. Use Amazon Elastic Block Store (Amazon EBS) Multi-Attach to create shared storage between the instances.

Answer: A

Explanation:

The correct answer is A because it directly addresses the requirements for high availability, resilience, and scalability while avoiding single points of failure and preventing data loss.

Here's why:

Application Servers in Auto Scaling Group across multiple AZs: Deploying application servers across multiple Availability Zones (AZs) ensures that if one AZ fails, the application remains available in other AZs. An Auto Scaling group dynamically adjusts the number of EC2 instances based on demand, ensuring scalability and resilience. https://aws.amazon.com/autoscaling/
Amazon RDS DB Instance in Multi-AZ configuration: Using Amazon RDS with a Multi-AZ configuration replicates the database to a standby instance in a different AZ. In case of a failure of the primary database instance, RDS automatically fails over to the standby, minimizing downtime and preventing data loss. This addresses the company's concern regarding database server crashes and data loss due to power outages. https://aws.amazon.com/rds/features/multi-az/

Let's analyze why the other options are not optimal:

B: Using a single AZ creates a single point of failure. EC2 Auto Recovery only restarts an instance on different hardware within the same AZ; it doesn't provide cross-AZ redundancy. Deploying the database on an EC2 instance without proper replication management is also a risk.
C: While using Auto Scaling across multiple AZs is good, relying on manually promoting a read replica to replace the primary database instance introduces manual intervention and longer recovery times, which are not ideal for high availability and resilience. The read replica being in a single AZ also presents a single point of failure for the replicated database.
D: Using EBS Multi-Attach is not recommended for most relational database workloads as it can lead to data corruption if not managed very carefully. Setting up database replication on EC2 instances manually can be complex and error-prone compared to using RDS Multi-AZ, which provides managed failover and replication.

In summary, option A provides a fully managed, highly available, and scalable solution using AWS services designed for these purposes, meeting all the stated requirements.

Question: 402 **Exam Heist**

A company needs to ingest and handle large amounts of streaming data that its application generates. The application runs on Amazon EC2 instances and sends data to Amazon Kinesis Data Streams, which is configured with default settings. Every other day, the application consumes the data and writes the data to an Amazon S3 bucket for business intelligence (BI) processing. The company observes that Amazon S3 is not receiving all the data that the application sends to Kinesis Data Streams.

What should a solutions architect do to resolve this issue?

A. Update the Kinesis Data Streams default settings by modifying the data retention period.

B. Update the application to use the Kinesis Producer Library (KPL) to send the data to Kinesis Data Streams.

C. Update the number of Kinesis shards to handle the throughput of the data that is sent to Kinesis Data Streams.

D. Turn on S3 Versioning within the S3 bucket to preserve every version of every object that is ingested in the S3 bucket.

Answer: A

Explanation:

The problem indicates that data is being lost between Kinesis Data Streams and S3. With default settings, Kinesis Data Streams retains data for 24 hours. Since the data is only consumed and written to S3 every other day, it's highly probable that the data older than 24 hours is being discarded by Kinesis before it can be processed and stored in S3. Increasing the retention period allows Kinesis Data Streams to store the data for a longer duration, ensuring that the BI process can access and write it to S3 even when run every other day.

Option A directly addresses this data retention issue by modifying the Kinesis Data Streams default settings. Increasing the retention period to at least 48 hours would ensure that the data is available when the application consumes and writes it to S3.

Option B, using KPL, optimizes data ingestion into Kinesis Data Streams by aggregating records and improving throughput, but it doesn't directly solve the data loss problem caused by insufficient retention.

Option C, increasing the number of shards, addresses throughput and scaling issues in Kinesis Data Streams but won't prevent data loss if the data retention period is too short. Shards influence the parallelism of the data stream processing, but do not impact data retention.

Option D, enabling S3 Versioning, helps maintain a history of objects in S3 and prevent accidental data loss within S3, but does not retrieve data already lost between Kinesis Data Streams and S3. It is useful after data has been ingested and to recover from overwrite or deletion.

Therefore, modifying the data retention period in Kinesis Data Streams is the most direct and effective solution to prevent data loss in this scenario.

Relevant Link:

Amazon Kinesis Data Streams Data Retention

Question: 403 **Exam Heist**

A developer has an application that uses an AWS Lambda function to upload files to Amazon S3 and needs the required permissions to perform the task. The developer already has an IAM user with valid IAM credentials required for Amazon S3.

What should a solutions architect do to grant the permissions?

A.Add required IAM permissions in the resource policy of the Lambda function.
B.Create a signed request using the existing IAM credentials in the Lambda function.
C.Create a new IAM user and use the existing IAM credentials in the Lambda function.
D.Create an IAM execution role with the required permissions and attach the IAM role to the Lambda function.

Answer: D

Explanation:

The correct approach to granting a Lambda function the necessary permissions to interact with other AWS services, like S3, is to use an IAM execution role. Here's why:

IAM execution roles provide a secure and best-practice method for granting permissions to AWS resources like Lambda functions. An IAM role is an IAM entity that defines a set of permissions for making AWS service requests. Attaching an IAM role to the Lambda function gives the function temporary credentials to assume the role and perform actions permitted by the role's policies. This is superior to using hardcoded IAM credentials for several reasons.

Option A, adding permissions to the Lambda function's resource policy, is typically used for granting permissions to other AWS accounts or services to invoke the Lambda function itself, not for the Lambda function to access other services.

Option B, creating a signed request using existing IAM credentials within the Lambda function, is strongly discouraged due to security risks. Embedding IAM credentials directly in code exposes them, potentially leading to unauthorized access if the code is compromised. Managing credential rotation also becomes difficult and cumbersome. Hardcoding AWS credentials in an application is considered a severe security

vulnerability.

Option C, creating a new IAM user and using its credentials in the Lambda function, suffers from the same security issues as option B. It's poor practice to embed IAM user credentials directly in Lambda code.

Option D, creating an IAM execution role and attaching it to the Lambda function, is the recommended approach because it provides a secure, manageable, and best-practice method for granting the Lambda function the necessary permissions. AWS manages the temporary credentials associated with the role, automatically rotating them and removing the need for manual credential management in the code. This aligns with the principle of least privilege and promotes secure application design. The role-based approach is far more secure and easier to manage. The function assumes the role dynamically when it is executed.

Here are some relevant resources for more information:

IAM Roles for AWS Lambda: https://docs.aws.amazon.com/lambda/latest/dg/lambda-intro-execution-role.html
Security best practices in IAM: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html
AWS IAM documentation: https://docs.aws.amazon.com/IAM/index.html

Question: 404 **Exam Heist**

A company has deployed a serverless application that invokes an AWS Lambda function when new documents are uploaded to an Amazon S3 bucket. The application uses the Lambda function to process the documents. After a recent marketing campaign, the company noticed that the application did not process many of the documents.

What should a solutions architect do to improve the architecture of this application?

A. Set the Lambda function's runtime timeout value to 15 minutes.

B. Configure an S3 bucket replication policy. Stage the documents in the S3 bucket for later processing.

C. Deploy an additional Lambda function. Load balance the processing of the documents across the two Lambda functions.

D. Create an Amazon Simple Queue Service (Amazon SQS) queue. Send the requests to the queue. Configure the queue as an event source for Lambda.

Answer: D

Explanation:

The correct answer is **D. Create an Amazon Simple Queue Service (Amazon SQS) queue. Send the requests to the queue. Configure the queue as an event source for Lambda.**

Here's a detailed justification:

The issue described indicates an event-driven architecture struggling to handle a surge in document uploads to S3, overwhelming the Lambda function. Direct invocation of Lambda by S3 events is susceptible to event loss and throttling, especially during peak loads.

Option D introduces Amazon SQS as an intermediary buffer, effectively decoupling the S3 events from the Lambda function invocations. When an S3 object is created, instead of directly invoking the Lambda function, a message is sent to the SQS queue. The Lambda function is then triggered by the SQS queue. This approach offers several benefits:

1. **Buffering and Scalability:** SQS acts as a buffer, absorbing the surge in requests. SQS is designed to handle high volumes of messages reliably. It scales automatically, preventing event loss during peak loads.
2. **Reliability:** SQS ensures guaranteed message delivery at least once. If the Lambda function fails to

process a message, it remains in the queue and can be retried. This prevents data loss and ensures all documents are eventually processed.

3. **Asynchronous Processing:** Decoupling through SQS allows the S3 bucket to continue accepting uploads without waiting for the Lambda function to process each one immediately. This improves the application's responsiveness.
4. **Error Handling:** If the Lambda function consistently fails to process a specific document, SQS can be configured with a dead-letter queue (DLQ) to store the problematic messages for later investigation and resolution.

Option A (increasing Lambda timeout) might help for individual documents taking longer to process, but it doesn't address the fundamental issue of potential event loss or the overwhelming of Lambda during a surge.

Option B (S3 replication) doesn't address the processing issue at all. Replication is for data redundancy and disaster recovery, not for handling event processing loads.

Option C (adding another Lambda function with load balancing) is a valid approach for scaling Lambda, but directly load balancing from S3 events is complex. SQS offers a more robust and manageable way to achieve horizontal scaling and resilience. SQS's built-in mechanisms for retry and DLQ make it a better solution in this scenario. While increased concurrency in Lambda can help (via Reserved Concurrency or Provisioned Concurrency), SQS as the intermediary provides the crucial buffer for bursts of S3 events.

In summary, using SQS provides a reliable and scalable solution for processing S3 events by buffering requests, ensuring delivery, and enabling asynchronous processing, which are critical for handling surges in document uploads and preventing event loss.

Authoritative Links:

AWS SQS: https://aws.amazon.com/sqs/
Using AWS Lambda with Amazon SQS: https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html
Serverless Architectures with AWS Lambda: https://aws.amazon.com/serverless/architectures/

Question: 405 **Exam Heist**

A solutions architect is designing the architecture for a software demonstration environment. The environment will run on Amazon EC2 instances in an Auto Scaling group behind an Application Load Balancer (ALB). The system will experience significant increases in traffic during working hours but is not required to operate on weekends.

Which combination of actions should the solutions architect take to ensure that the system can scale to meet demand? (Choose two.)

A. Use AWS Auto Scaling to adjust the ALB capacity based on request rate.

B. Use AWS Auto Scaling to scale the capacity of the VPC internet gateway.

C. Launch the EC2 instances in multiple AWS Regions to distribute the load across Regions.

D. Use a target tracking scaling policy to scale the Auto Scaling group based on instance CPU utilization.

E. Use scheduled scaling to change the Auto Scaling group minimum, maximum, and desired capacity to zero for weekends. Revert to the default values at the start of the week.

Answer: DE

Explanation:

Here's a detailed justification for why options D and E are the correct choices, and why the others are not:

D. Use a target tracking scaling policy to scale the Auto Scaling group based on instance CPU utilization.

Justification: Target tracking scaling policies automatically adjust the number of EC2 instances in the Auto

Scaling group to maintain a specified target value for a chosen metric, such as CPU utilization. During working hours when traffic increases, CPU utilization on the existing instances will rise. The scaling policy responds by launching more instances to keep the CPU utilization near the target value, thereby ensuring application performance remains consistent despite increasing load. This dynamic scaling is essential for meeting fluctuating demand within working hours.
Why it works: This approach provides a reactive, automated scaling mechanism. It eliminates the need for manual intervention or constant monitoring. If the instances' CPU utilization increases due to higher request loads, the auto-scaling group launches new instances to distribute that load and reduce the CPU per instance, keeping the performance level consistent.
Relevant Concept: Auto Scaling Groups, Target Tracking Scaling.
Authoritative Link: https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-target-tracking.html

E. Use scheduled scaling to change the Auto Scaling group minimum, maximum, and desired capacity to zero for weekends. Revert to the default values at the start of the week.

Justification: Since the system is not required to operate on weekends, scheduled scaling is a perfect fit. By setting the minimum, maximum, and desired capacity to zero for weekends, all instances will be terminated, eliminating unnecessary costs. At the start of the week, the scheduled scaling will revert the capacity settings to the appropriate values, ensuring the system is ready to handle the weekday traffic. This allows for efficient cost management by only running resources when needed.
Why it works: Scheduled scaling allows you to proactively manage the capacity of your Auto Scaling group based on predictable patterns like workdays versus weekends. This ensures that you don't pay for compute capacity when it's not needed.
Relevant Concept: Auto Scaling Groups, Scheduled Scaling.
Authoritative Link: https://docs.aws.amazon.com/autoscaling/ec2/userguide/schedule_time.html

Why the other options are incorrect:

A. Use AWS Auto Scaling to adjust the ALB capacity based on request rate. The ALB automatically scales to handle varying request rates. Auto Scaling is used for the EC2 instances behind the ALB, not the ALB itself.
B. Use AWS Auto Scaling to scale the capacity of the VPC internet gateway. Internet Gateways are horizontally scaled and redundant, so they don't need to be scaled using Auto Scaling.
C. Launch the EC2 instances in multiple AWS Regions to distribute the load across Regions. While this is useful for high availability and disaster recovery, it's overkill for a demonstration environment that only needs to be shut down on weekends. Moreover, it is not cost-effective. Distributing across regions increases the complexity and cost significantly more than simply stopping instances when they are not needed. The problem statement focuses on scaling to meet demand within working hours and avoiding costs on weekends, making a multi-region deployment unnecessary for the given constraints.

Question: 406 **Exam Heist**

A solutions architect is designing a two-tiered architecture that includes a public subnet and a database subnet. The web servers in the public subnet must be open to the internet on port 443. The Amazon RDS for MySQL DB instance in the database subnet must be accessible only to the web servers on port 3306.

Which combination of steps should the solutions architect take to meet these requirements? (Choose two.)

A.Create a network ACL for the public subnet. Add a rule to deny outbound traffic to 0.0.0.0/0 on port 3306.

B.Create a security group for the DB instance. Add a rule to allow traffic from the public subnet CIDR block on port 3306.

C.Create a security group for the web servers in the public subnet. Add a rule to allow traffic from 0.0.0.0/0 on port 443.

D.Create a security group for the DB instance. Add a rule to allow traffic from the web servers’ security group

on port 3306.

E.Create a security group for the DB instance. Add a rule to deny all traffic except traffic from the web servers' security group on port 3306.

Answer: CD

Explanation:

The correct answer is **CD**. Here's why:

C: Create a security group for the web servers in the public subnet. Add a rule to allow traffic from 0.0.0.0/0 on port 443.

This step is crucial for allowing internet traffic to reach the web servers on port 443 (HTTPS). Security groups act as virtual firewalls at the instance level. Allowing 0.0.0.0/0 (all IP addresses) on port 443 enables external users to connect to the web servers via HTTPS. Without this rule, users wouldn't be able to access the web application.

D: Create a security group for the DB instance. Add a rule to allow traffic from the web servers' security group on port 3306.

This step ensures that only the web servers can access the database on port 3306 (MySQL's default port). By referencing the web servers' security group as the source in the DB instance's security group rule, you're specifically granting access only to instances associated with that security group. This is a more secure approach than using CIDR blocks, as it automatically adjusts if the web server's IP address changes due to scaling or instance replacement. This principle of least privilege restricts access and hardens security.

Why other options are incorrect:

A: Create a network ACL for the public subnet. Add a rule to deny outbound traffic to 0.0.0.0/0 on port 3306. Network ACLs operate at the subnet level and are stateless. While you could theoretically use them, security groups provide a more granular and easier-to-manage solution for this scenario. Denying outbound traffic from the public subnet is not the solution. The database traffic flows from the public subnet (web servers) to the database subnet (RDS instance). Blocking outbound traffic from the public subnet wouldn't prevent the access. Security Groups should be configured on the DB instance.

B: Create a security group for the DB instance. Add a rule to allow traffic from the public subnet CIDR block on port 3306. While this allows communication, it's less secure than referencing the web servers' security group. If the web server's IP changes, the security group rule based on CIDR would need manual updating. The security group referencing provides a dynamic association between the web servers and the DB instance.

E: Create a security group for the DB instance. Add a rule to deny all traffic except traffic from the web servers' security group on port 3306. While the intended outcome is correct (restrict access), security group rules operate as "allow" rules. There are no explicit "deny" rules in security groups (except the implicit deny for everything not explicitly allowed).

Authoritative Links:

Amazon VPC Security Groups
Amazon VPC Network ACLs

Question: 407 **Exam Heist**

A company is implementing a shared storage solution for a gaming application that is hosted in the AWS Cloud. The company needs the ability to use Lustre clients to access data. The solution must be fully managed.

Which solution meets these requirements?

A.Create an AWS DataSync task that shares the data as a mountable file system. Mount the file system to the application server.

B.Create an AWS Storage Gateway file gateway. Create a file share that uses the required client protocol. Connect the application server to the file share.

C.Create an Amazon Elastic File System (Amazon EFS) file system, and configure it to support Lustre. Attach the file system to the origin server. Connect the application server to the file system.

D.Create an Amazon FSx for Lustre file system. Attach the file system to the origin server. Connect the application server to the file system.

Answer: D

Explanation:

The correct answer is D because Amazon FSx for Lustre provides a fully managed, high-performance file system optimized for compute-intensive workloads, including gaming applications. This service inherently supports Lustre clients, fulfilling the requirement to access data using them.

Option A is incorrect because AWS DataSync is primarily a data transfer service, not a file system. It doesn't provide a mountable file system directly.

Option B is incorrect because AWS Storage Gateway file gateway translates between cloud storage (like S3) and on-premises file protocols like NFS or SMB. It does not natively support the Lustre protocol.

Option C is incorrect because Amazon EFS is a fully managed NFS file system suitable for a wide range of workloads, but it does not support the Lustre protocol.

Therefore, only Amazon FSx for Lustre meets the requirements of a fully managed solution with native Lustre client support, making it the most suitable choice for the gaming application's shared storage needs. FSx for Lustre's architecture is specifically designed for high-performance computing and can deliver the low latency and high throughput required by demanding gaming applications.

References:

Amazon FSx for Lustre: https://aws.amazon.com/fsx/lustre/
AWS DataSync: https://aws.amazon.com/datasync/
AWS Storage Gateway: https://aws.amazon.com/storagegateway/
Amazon EFS: https://aws.amazon.com/efs/

Question: 408 **Exam Heist**

A company runs an application that receives data from thousands of geographically dispersed remote devices that use UDP. The application processes the data immediately and sends a message back to the device if necessary. No data is stored.

The company needs a solution that minimizes latency for the data transmission from the devices. The solution also must provide rapid failover to another AWS Region.

Which solution will meet these requirements?

A. Configure an Amazon Route 53 failover routing policy. Create a Network Load Balancer (NLB) in each of the two Regions. Configure the NLB to invoke an AWS Lambda function to process the data.

B. Use AWS Global Accelerator. Create a Network Load Balancer (NLB) in each of the two Regions as an endpoint. Create an Amazon Elastic Container Service (Amazon ECS) cluster with the Fargate launch type. Create an ECS service on the cluster. Set the ECS service as the target for the NLProcess the data in Amazon ECS.

C. Use AWS Global Accelerator. Create an Application Load Balancer (ALB) in each of the two Regions as an endpoint. Create an Amazon Elastic Container Service (Amazon ECS) cluster with the Fargate launch type. Create an ECS service on the cluster. Set the ECS service as the target for the ALB. Process the data in Amazon ECS.

D. Configure an Amazon Route 53 failover routing policy. Create an Application Load Balancer (ALB) in each of the two Regions. Create an Amazon Elastic Container Service (Amazon ECS) cluster with the Fargate launch type. Create an ECS service on the cluster. Set the ECS service as the target for the ALB. Process the data in Amazon ECS.

Answer: B

Explanation:

The correct answer is B because it leverages AWS Global Accelerator for low latency and failover, and Network Load Balancers (NLBs) for UDP traffic.

Here's a detailed justification:

Low Latency with AWS Global Accelerator: AWS Global Accelerator uses the AWS global network to direct traffic to the optimal endpoint based on location, network conditions, and health. This significantly reduces latency for geographically dispersed devices, as traffic is routed through the nearest AWS edge location, improving the UDP data transmission speed. https://aws.amazon.com/global-accelerator/
UDP Support with Network Load Balancer (NLB): NLBs are designed to handle UDP traffic, which is crucial for the application's data reception. Application Load Balancers (ALBs) only support HTTP/HTTPS and cannot handle UDP. https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html
Rapid Failover: Global Accelerator provides rapid failover capabilities. If one AWS Region becomes unavailable, Global Accelerator automatically redirects traffic to the healthy NLB in the other Region, ensuring business continuity.
ECS with Fargate for Processing: ECS with Fargate provides a scalable and managed container orchestration service where the data processing logic can be hosted. This allows the application to process the incoming data immediately, as required.
Eliminating other options reasoning:

A & D: Route 53 failover, although capable of failover, relies on DNS propagation, which is slower than the near-instantaneous failover of Global Accelerator. Furthermore, option D uses ALB, which cannot handle UDP traffic.
C: Option C uses ALB, which cannot handle UDP traffic and therefore is not the optimal choice.

Question: 409 **Exam Heist**

A solutions architect must migrate a Windows Internet Information Services (IIS) web application to AWS. The application currently relies on a file share hosted in the user's on-premises network-attached storage (NAS). The solutions architect has proposed migrating the IIS web servers to Amazon EC2 instances in multiple Availability Zones that are connected to the storage solution, and configuring an Elastic Load Balancer attached to the instances.

Which replacement to the on-premises file share is MOST resilient and durable?

A. Migrate the file share to Amazon RDS.
B. Migrate the file share to AWS Storage Gateway.
C. Migrate the file share to Amazon FSx for Windows File Server.
D. Migrate the file share to Amazon Elastic File System (Amazon EFS).

Answer: C

Explanation:

The most resilient and durable replacement for the on-premises file share in this scenario is **Amazon FSx for Windows File Server (Option C)**.

Here's why:

Windows Compatibility: The application is a Windows IIS web application that currently relies on a file share. FSx for Windows File Server provides native support for the SMB protocol and NTFS file system, ensuring seamless compatibility without requiring application code changes. This compatibility minimizes migration effort and potential issues.

Integration with EC2 and Active Directory: FSx for Windows File Server seamlessly integrates with EC2 instances and existing Active Directory environments. This simplifies user authentication and authorization, access control, and overall management.

High Availability and Durability: FSx for Windows File Server offers multi-AZ deployment options, providing high availability and data redundancy across multiple Availability Zones. This helps ensure the application remains available even in the event of an AZ failure. It uses Windows Server failover clustering for high availability.

Scalability and Performance: FSx for Windows File Server offers scalable storage capacity and performance options, allowing the application to handle increasing workloads without performance bottlenecks. It's designed to support enterprise-grade workloads.

RDS (Option A): Amazon RDS is a relational database service and is not designed to serve as a general-purpose file share. Migrating a file share to a database would be highly complex and inefficient.

AWS Storage Gateway (Option B): Storage Gateway connects on-premises applications to cloud storage. While it can provide access to AWS storage, it still requires an on-premises component, negating the goal of migrating the file share to AWS. It introduces added latency and management overhead.

Amazon EFS (Option D): Amazon EFS is a network file system designed primarily for Linux-based applications. While it can be used with Windows EC2 instances, it requires additional configuration and doesn't natively support the SMB protocol or Active Directory integration as seamlessly as FSx for Windows File Server.

In summary, FSx for Windows File Server is the best option because it offers native Windows compatibility, high availability, durability, scalability, and seamless integration with EC2 and Active Directory, making it the most suitable replacement for the on-premises file share.

Authoritative Links:

Amazon FSx for Windows File Server: https://aws.amazon.com/fsx/windows/

Question: 410 **Exam Heist**

A company is deploying a new application on Amazon EC2 instances. The application writes data to Amazon Elastic Block Store (Amazon EBS) volumes. The company needs to ensure that all data that is written to the EBS volumes is encrypted at rest.

Which solution will meet this requirement?

A.Create an IAM role that specifies EBS encryption. Attach the role to the EC2 instances.

B.Create the EBS volumes as encrypted volumes. Attach the EBS volumes to the EC2 instances.

C.Create an EC2 instance tag that has a key of Encrypt and a value of True. Tag all instances that require encryption at the EBS level.

D.Create an AWS Key Management Service (AWS KMS) key policy that enforces EBS encryption in the account.

Ensure that the key policy is active.

Answer: B

Explanation:

The correct answer is **B. Create the EBS volumes as encrypted volumes. Attach the EBS volumes to the EC2 instances.**

Here's why this is the optimal solution and why the other options are incorrect:

Why Option B is Correct: EBS encryption is configured at the volume level. When you create an EBS volume and specify encryption, all data at rest on the volume, all data moving between the EC2 instance and the volume, and all snapshots created from the volume are encrypted. This fulfills the requirement of encrypting all data written to EBS volumes at rest.
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EBSEncryption.html

Why Option A is Incorrect: IAM roles are used to grant permissions to EC2 instances to access other AWS services. While you can control access to KMS keys used for EBS encryption via IAM, simply attaching an IAM role that "specifies EBS encryption" doesn't inherently encrypt existing or newly created EBS volumes. IAM controls who can use encryption, not if encryption is enforced by default on EBS volumes.

Why Option C is Incorrect: EC2 instance tags are metadata applied to EC2 instances for organization and automation. They do not enforce EBS encryption. Tags are used for cost allocation, automation scripts, etc., but they don't have a direct impact on encryption settings for associated resources.

Why Option D is Incorrect: While an AWS KMS key policy can enforce encryption, it must be paired with EBS volume creation that uses the KMS key. This is only effective when creating new EBS volumes with the enforcement key; existing unencrypted EBS volumes would remain unencrypted. It also wouldn't ensure that newly created EBS volumes are always encrypted by default, unless coupled with other mechanisms. Option B provides direct encryption at the volume creation step.

Therefore, creating encrypted EBS volumes directly at the time of their creation is the most straightforward and reliable method to ensure all data written to those volumes is encrypted at rest, satisfying the company's requirement. It ensures every created EBS volume is encrypted, by default, during the volume creation process.

Question: 411 **Exam Heist**

A company has a web application with sporadic usage patterns. There is heavy usage at the beginning of each month, moderate usage at the start of each week, and unpredictable usage during the week. The application consists of a web server and a MySQL database server running inside the data center. The company would like to move the application to the AWS Cloud, and needs to select a cost-effective database platform that will not require database modifications.

Which solution will meet these requirements?

A.Amazon DynamoDB

B.Amazon RDS for MySQL

C.MySQL-compatible Amazon Aurora Serverless

D.MySQL deployed on Amazon EC2 in an Auto Scaling group

Answer: C

Explanation:

The correct answer is C, MySQL-compatible Amazon Aurora Serverless. Here's a detailed justification:

Aurora Serverless v2 is ideal for applications with sporadic and unpredictable workloads. It automatically scales the database capacity up or down based on the application's needs, eliminating the need to provision and manage database servers. This on-demand scaling aligns perfectly with the company's heavy monthly usage, moderate weekly usage, and unpredictable usage during the week. It provides cost optimization by only charging for the compute and storage resources consumed.

Amazon RDS for MySQL (option B) would require the company to provision a specific instance size, which might lead to over-provisioning during periods of low usage, resulting in unnecessary costs. While RDS can scale, it doesn't automatically scale to zero like Serverless does during periods of inactivity.

Amazon DynamoDB (option A) is a NoSQL database and would require significant application modifications to be compatible, which contradicts the requirement of not needing database modifications. DynamoDB's data model and query language are fundamentally different from MySQL.

MySQL deployed on Amazon EC2 in an Auto Scaling group (option D) would involve significantly more operational overhead for managing the database, including patching, backups, and scaling, compared to Aurora Serverless. Also, while Auto Scaling addresses the compute scaling for the web tier, it doesn't inherently solve the database scaling problem. It requires more configurations and scripts for the database layer.

Aurora Serverless's MySQL compatibility allows for a smooth migration without any code changes. Also, Aurora offers improved performance and availability over standard MySQL.

In summary, Aurora Serverless v2 offers the best combination of cost-effectiveness, automatic scaling, minimal management overhead, and MySQL compatibility, making it the most suitable choice for the given scenario.

Relevant links:

Amazon Aurora Serverless v2
Amazon RDS for MySQL
Amazon DynamoDB

Question: 412 **Exam Heist**

An image-hosting company stores its objects in Amazon S3 buckets. The company wants to avoid accidental exposure of the objects in the S3 buckets to the public. All S3 objects in the entire AWS account need to remain private.

Which solution will meet these requirements?

A. Use Amazon GuardDuty to monitor S3 bucket policies. Create an automatic remediation action rule that uses an AWS Lambda function to remediate any change that makes the objects public.

B. Use AWS Trusted Advisor to find publicly accessible S3 buckets. Configure email notifications in Trusted Advisor when a change is detected. Manually change the S3 bucket policy if it allows public access.

C. Use AWS Resource Access Manager to find publicly accessible S3 buckets. Use Amazon Simple Notification Service (Amazon SNS) to invoke an AWS Lambda function when a change is detected. Deploy a Lambda function that programmatically remediates the change.

D. Use the S3 Block Public Access feature on the account level. Use AWS Organizations to create a service control policy (SCP) that prevents IAM users from changing the setting. Apply the SCP to the account.

Answer: D

Explanation:

The correct answer is D because it provides a comprehensive and proactive approach to preventing accidental public exposure of S3 objects at the account level, with a mechanism to ensure the configuration isn't altered.

Here's a detailed justification:

S3 Block Public Access (BPA) at the Account Level: S3 BPA is designed specifically to prevent public access to S3 buckets and objects. Applying it at the account level ensures that all new and existing buckets within the account adhere to the block public access settings. This is the primary preventative measure. It blocks public access through bucket policies, ACLs, or both.

AWS Organizations and Service Control Policies (SCPs): AWS Organizations enables you to centrally manage multiple AWS accounts. SCPs are policies that you can attach to the root, organizational units (OUs), or individual accounts within your organization. By creating an SCP that prevents IAM users from disabling the S3 BPA setting, you establish a guardrail. This ensures that even IAM users with otherwise sufficient permissions cannot inadvertently (or maliciously) bypass the account-level security measure. This ensures compliance across all accounts managed within the AWS Organizations.

Why other options are less suitable:

Option A (GuardDuty): GuardDuty is a threat detection service. While it can identify publicly accessible buckets, it's a reactive measure. The objects might be publicly accessible for a period before GuardDuty detects it.
Option B (Trusted Advisor): Similar to GuardDuty, Trusted Advisor is more of an advisory tool. It provides recommendations but doesn't actively prevent public access. It also requires manual intervention, increasing the risk of delayed remediation.
Option C (Resource Access Manager and SNS): AWS RAM shares AWS resources with other AWS accounts or within your organization. It's not directly related to preventing public access to S3 buckets. While SNS could trigger a Lambda function, this setup is more complex and less effective than directly using S3 BPA and SCPs.

In summary, option D provides a robust, proactive, and centrally managed solution that directly addresses the problem of accidental public exposure of S3 objects across an entire AWS account. It employs both preventive controls (S3 BPA) and enforcement mechanisms (SCPs) to ensure consistent security.

Relevant links for further research:

S3 Block Public Access: https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html
AWS Organizations SCPs:
https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scp.html

Question: 413 **Exam Heist**

An ecommerce company is experiencing an increase in user traffic. The company's store is deployed on Amazon EC2 instances as a two-tier web application consisting of a web tier and a separate database tier. As traffic increases, the company notices that the architecture is causing significant delays in sending timely marketing and order confirmation email to users. The company wants to reduce the time it spends resolving complex email delivery issues and minimize operational overhead.

What should a solutions architect do to meet these requirements?

A. Create a separate application tier using EC2 instances dedicated to email processing.

B. Configure the web instance to send email through Amazon Simple Email Service (Amazon SES).

C. Configure the web instance to send email through Amazon Simple Notification Service (Amazon SNS).

D. Create a separate application tier using EC2 instances dedicated to email processing. Place the instances in an Auto Scaling group.

Answer: B

Explanation:

The correct answer is **B. Configure the web instance to send email through Amazon Simple Email Service (Amazon SES).**

Here's a detailed justification:

The primary problem is the delay in sending emails (marketing and order confirmations) due to increasing traffic impacting the web application performance. Option B proposes using Amazon SES, which is a fully managed email sending service. This offloads the email sending responsibility from the web application servers. By using SES, the web application can quickly hand off email requests to AWS without blocking threads or consuming resources needed to serve web traffic. This reduces the application's workload and improves responsiveness. SES is designed for high deliverability and scalability, minimizing the operational overhead of managing email infrastructure, including dealing with bounce handling, spam filtering, and reputation management.

Option A and D involve creating a separate EC2-based application tier for email processing. While it might seem like a solution, this introduces operational complexity and overhead, including managing and scaling these instances. It also doesn't address the underlying email delivery issues that a managed service like SES is designed to handle. It adds more servers to manage instead of removing workload from current servers.

Option C suggests using Amazon SNS. While SNS is a messaging service, it is not designed for sending transactional emails like order confirmations or marketing emails. SNS is primarily used for push notifications, application-to-application (A2A) and application-to-person (A2P) messaging. Using SNS for this purpose would be an inappropriate and inefficient approach.

Therefore, using Amazon SES is the most efficient and scalable solution for handling email sending, reducing operational overhead, and improving the overall performance of the e-commerce application. It aligns perfectly with the company's requirements.

Authoritative links:

Amazon SES: https://aws.amazon.com/ses/
Amazon SNS: https://aws.amazon.com/sns/

Question: 414 **Exam Heist**

A company has a business system that generates hundreds of reports each day. The business system saves the reports to a network share in CSV format. The company needs to store this data in the AWS Cloud in near-real time for analysis.

Which solution will meet these requirements with the LEAST administrative overhead?

A. Use AWS DataSync to transfer the files to Amazon S3. Create a scheduled task that runs at the end of each day.

B. Create an Amazon S3 File Gateway. Update the business system to use a new network share from the S3 File Gateway.

C. Use AWS DataSync to transfer the files to Amazon S3. Create an application that uses the DataSync API in the automation workflow.

D. Deploy an AWS Transfer for SFTP endpoint. Create a script that checks for new files on the network share and uploads the new files by using SFTP.

Answer: B

Explanation:

Here's a detailed justification for why option B is the best solution, along with supporting explanations and links:

The core requirement is near-real-time data storage in S3 with minimal administrative overhead. Option B, leveraging Amazon S3 File Gateway, achieves this most effectively. S3 File Gateway provides a local cache for frequently accessed data, ensuring fast access for the business system. It integrates seamlessly with the existing network share setup, minimizing disruption. The business system only needs to be reconfigured to write to the new S3 File Gateway network share, a straightforward change.

The gateway automatically and securely transfers data to S3, eliminating the need for complex scripting or scheduling. Data is transferred asynchronously, minimizing impact on the business system's performance. The automatic transfer mechanism reduces administrative overhead, as the file transfer process is handled by the gateway. This solution avoids building custom data transfer pipelines or managing periodic data transfer jobs, thereby streamlining the entire process.

Option A requires scheduling tasks with AWS DataSync which increases the complexity and overhead. While DataSync is a good option for migrating large amounts of data, in this case we need to consider a continuous data transfer with near-real time constraints. It is not optimal to schedule the task to only run at the end of each day.

Option C also involves DataSync but introduces the added complexity of managing an application interacting with the DataSync API, further increasing administrative burden. This approach demands more development and maintenance effort.

Option D, using AWS Transfer for SFTP, requires managing an SFTP server and writing a script to monitor the network share, increasing complexity and overhead. SFTP is primarily intended for secure file transfer by humans, not automated system processes. This is not suitable for an automated system that generates hundreds of reports daily.

Therefore, S3 File Gateway directly addresses the problem by integrating with the existing network share, providing automatic, near-real-time data transfer to S3 with minimal administrative effort.

Supporting Links:

Amazon S3 File Gateway: https://aws.amazon.com/storagegateway/file/
AWS DataSync: https://aws.amazon.com/datasync/
AWS Transfer Family: https://aws.amazon.com/aws-transfer-family/

Question: 415 **Exam Heist**

A company is storing petabytes of data in Amazon S3 Standard. The data is stored in multiple S3 buckets and is accessed with varying frequency. The company does not know access patterns for all the data. The company needs to implement a solution for each S3 bucket to optimize the cost of S3 usage.

Which solution will meet these requirements with the MOST operational efficiency?

A. Create an S3 Lifecycle configuration with a rule to transition the objects in the S3 bucket to S3 Intelligent-Tiering.

B. Use the S3 storage class analysis tool to determine the correct tier for each object in the S3 bucket. Move each object to the identified storage tier.

C. Create an S3 Lifecycle configuration with a rule to transition the objects in the S3 bucket to S3 Glacier Instant Retrieval.

D. Create an S3 Lifecycle configuration with a rule to transition the objects in the S3 bucket to S3 One Zone-Infrequent Access (S3 One Zone-IA).

Answer: A

Explanation:

The best solution is **A. Create an S3 Lifecycle configuration with a rule to transition the objects in the S3 bucket to S3 Intelligent-Tiering.** Here's why:

S3 Intelligent-Tiering: This storage class is designed to automatically optimize storage costs by moving data between frequent, infrequent, and archive access tiers based on changing access patterns. It incurs a small monthly monitoring and automation fee per object, but it eliminates the operational overhead of manually identifying and moving data.
Unknown Access Patterns: The problem explicitly states the company doesn't know the access patterns for all data. Intelligent-Tiering excels in this scenario because it automatically adapts to unknown and changing access patterns, optimizing costs without requiring manual analysis.
Operational Efficiency: S3 Lifecycle configurations automate the process of transitioning objects between storage classes. Using Intelligent-Tiering with a Lifecycle configuration provides the most hands-off and automated approach, minimizing operational burden.
S3 Storage Class Analysis (Option B): While useful, this requires manual analysis and subsequent action to move objects. This is less operationally efficient than Intelligent-Tiering, which is designed to automate this process. Moreover, the analysis is a one-time snapshot and wouldn't adapt to changing access patterns over time without repeated manual analysis.
S3 Glacier Instant Retrieval (Option C): Glacier is designed for archival data with infrequent access. It's not the most suitable choice when access frequency is unknown, as frequent retrieval incurs higher costs.
S3 One Zone-Infrequent Access (S3 One Zone-IA) (Option D): While cost-effective for infrequent access, it's not ideal when access patterns are unknown. Intelligent-Tiering can also move data to infrequent access tiers, and offers the flexibility to return data to frequent access if needed, which One Zone-IA does not. Furthermore, One Zone-IA has lower availability than Intelligent-Tiering.

In essence, Intelligent-Tiering is the most efficient and appropriate choice because it automates cost optimization based on access patterns without requiring upfront knowledge of those patterns.https://aws.amazon.com/s3/storage-classes/intelligent-tiering/https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-examples.html

Question: 416 **Exam Heist**

A rapidly growing global ecommerce company is hosting its web application on AWS. The web application includes static content and dynamic content. The website stores online transaction processing (OLTP) data in an Amazon RDS database The website's users are experiencing slow page loads.

Which combination of actions should a solutions architect take to resolve this issue? (Choose two.)

A.Configure an Amazon Redshift cluster.
B.Set up an Amazon CloudFront distribution.
C.Host the dynamic web content in Amazon S3.
D.Create a read replica for the RDS DB instance.
E.Configure a Multi-AZ deployment for the RDS DB instance.

Answer: BD

Explanation:

The best combination of actions to resolve slow page loads for the ecommerce web application is to use Amazon CloudFront and create a read replica for the RDS DB instance.

B. Set up an Amazon CloudFront distribution: CloudFront is a content delivery network (CDN) service that caches static content (images, CSS, JavaScript, etc.) closer to users globally. By caching content at edge locations, CloudFront reduces latency and improves page load times. This is especially beneficial for a global ecommerce company with users distributed worldwide. (https://aws.amazon.com/cloudfront/)

D. Create a read replica for the RDS DB instance: Creating a read replica allows read-only traffic to be offloaded from the primary RDS database. This reduces the load on the primary database, allowing it to focus on write operations (transactions). The web application can be configured to read data from the read replica, improving response times for data-intensive read operations and contributing to faster page loads. OLTP databases benefit from read replicas as read queries often overwhelm the primary DB instance. (https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ReadRepl.html)

Option A (Amazon Redshift) is not the best choice because Redshift is designed for data warehousing and analytics, not for serving real-time OLTP data for a web application. Option C (Host the dynamic web content in Amazon S3) is incorrect because S3 is designed for static content. Dynamic content needs a compute environment to execute. Option E (Configure a Multi-AZ deployment for the RDS DB instance) improves the availability and durability of the database, but it doesn't directly address the performance issue of slow page loads. Multi-AZ is for failover, not performance scaling.

Question: 417 **Exam Heist**

A company uses Amazon EC2 instances and AWS Lambda functions to run its application. The company has VPCs with public subnets and private subnets in its AWS account. The EC2 instances run in a private subnet in one of the VPCs. The Lambda functions need direct network access to the EC2 instances for the application to work.

The application will run for at least 1 year. The company expects the number of Lambda functions that the application uses to increase during that time. The company wants to maximize its savings on all application resources and to keep network latency between the services low.

Which solution will meet these requirements?

A.Purchase an EC2 Instance Savings Plan Optimize the Lambda functions' duration and memory usage and the number of invocations. Connect the Lambda functions to the private subnet that contains the EC2 instances.

B.Purchase an EC2 Instance Savings Plan Optimize the Lambda functions' duration and memory usage, the number of invocations, and the amount of data that is transferred. Connect the Lambda functions to a public subnet in the same VPC where the EC2 instances run.

C.Purchase a Compute Savings Plan. Optimize the Lambda functions' duration and memory usage, the number of invocations, and the amount of data that is transferred. Connect the Lambda functions to the private subnet that contains the EC2 instances.

D.Purchase a Compute Savings Plan. Optimize the Lambda functions' duration and memory usage, the number of invocations, and the amount of data that is transferred. Keep the Lambda functions in the Lambda service VPC.

Answer: C

Explanation:

The correct answer is C. Here's why:

The requirements include cost optimization, low latency, application running for at least 1 year with increasing Lambda function count, and direct network access from Lambda to EC2 in a private subnet.

Savings Plan Selection: Compute Savings Plans offer flexibility by applying discounts across EC2, Lambda, and Fargate, unlike EC2 Instance Savings Plans which only apply to EC2. Given the anticipated growth in Lambda usage, a Compute Savings Plan ensures optimal savings across all compute resources. https://aws.amazon.com/savingsplans/

Lambda Optimization: Optimizing Lambda functions' duration, memory, invocations, and data transfer directly impacts cost. Reducing execution time and memory allocation minimizes resource consumption, and optimizing data transfer reduces network charges.

VPC Connection: Connecting Lambda functions to the private subnet provides direct, low-latency network access to the EC2 instances without traversing the public internet. It also ensures secure communication within the VPC. Placing Lambda in a public subnet (option B) is less secure and not necessary given the option for private subnet connectivity. Keeping Lambda in the Lambda service VPC (option D) would require more complex networking (like VPC peering or PrivateLink) to reach the EC2 instances, increasing latency and cost. https://docs.aws.amazon.com/lambda/latest/dg/configuration-vpc.html

Therefore, Option C provides the most comprehensive solution for cost savings, low latency, scalability, and secure network access between Lambda functions and EC2 instances within the private subnet.

Question: 418 **Exam Heist**

A solutions architect needs to allow team members to access Amazon S3 buckets in two different AWS accounts: a development account and a production account. The team currently has access to S3 buckets in the development account by using unique IAM users that are assigned to an IAM group that has appropriate permissions in the account.

The solutions architect has created an IAM role in the production account. The role has a policy that grants access to an S3 bucket in the production account.

Which solution will meet these requirements while complying with the principle of least privilege?

A. Attach the Administrator Access policy to the development account users.
B. Add the development account as a principal in the trust policy of the role in the production account.
C. Turn off the S3 Block Public Access feature on the S3 bucket in the production account.
D. Create a user in the production account with unique credentials for each team member.

Answer: B

Explanation:

The correct answer is **B. Add the development account as a principal in the trust policy of the role in the production account.**

Here's a detailed justification:

The scenario requires granting users in the development account access to S3 resources in the production account while adhering to the principle of least privilege. This means giving only the necessary permissions and avoiding overly broad access.

Option A, attaching the AdministratorAccess policy to the development account users, violates the principle of least privilege. It grants excessive permissions to the development account users, far beyond what's needed to access the specific S3 bucket in the production account. This opens up security risks.

Option B is the correct solution because it utilizes IAM roles for cross-account access, a secure and recommended practice. The IAM role is created in the production account and has a policy defining what actions (e.g., s3:GetObject, s3:PutObject) are allowed on the S3 bucket. Crucially, the role's trust policy is modified to allow the development account to assume the role. This is done by specifying the development account's AWS account ID as a principal in the trust policy.

Users in the development account can then assume this role, temporarily gaining the permissions associated with it in the production account. To assume the role, the IAM users in the development account need explicit

permissions to do so (using sts:AssumeRole in their IAM policy). This approach is secure and follows least privilege, as users only gain production access when they explicitly assume the role, and the role itself only grants specific S3 access.

Option C, turning off S3 Block Public Access, is incorrect and dangerous. It makes the S3 bucket publicly accessible, which is a severe security risk and has nothing to do with granting cross-account access to specific users.

Option D, creating a user in the production account for each team member, creates unnecessary administrative overhead and doesn't scale well. Managing separate user credentials for each team member in multiple accounts is complex and prone to errors. Also, it doesn't leverage the benefits of IAM roles for temporary access.

In summary, using IAM roles with a trust policy that specifies the development account as a principal allows authorized users in the development account to assume the role and access the S3 bucket in the production account in a controlled and secure manner, adhering to the principle of least privilege.

Further reading:

AWS IAM Roles: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html
IAM Roles for Cross-Account Access: https://docs.aws.amazon.com/IAM/latest/UserGuide/tutorial_cross-account-with-roles.html
STS AssumeRole: https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRole.html

Question: 419 **Exam Heist**

A company uses AWS Organizations with all features enabled and runs multiple Amazon EC2 workloads in the ap-southeast-2 Region. The company has a service control policy (SCP) that prevents any resources from being created in any other Region. A security policy requires the company to encrypt all data at rest.

An audit discovers that employees have created Amazon Elastic Block Store (Amazon EBS) volumes for EC2 instances without encrypting the volumes. The company wants any new EC2 instances that any IAM user or root user launches in ap-southeast-2 to use encrypted EBS volumes. The company wants a solution that will have minimal effect on employees who create EBS volumes.

Which combination of steps will meet these requirements? (Choose two.)

A.In the Amazon EC2 console, select the EBS encryption account attribute and define a default encryption key.

B.Create an IAM permission boundary. Attach the permission boundary to the root organizational unit (OU). Define the boundary to deny the ec2:CreateVolume action when the ec2:Encrypted condition equals false.

C.Create an SCP. Attach the SCP to the root organizational unit (OU). Define the SCP to deny the ec2:CreateVolume action whenthe ec2:Encrypted condition equals false.

D.Update the IAM policies for each account to deny the ec2:CreateVolume action when the ec2:Encrypted condition equals false.

E.In the Organizations management account, specify the Default EBS volume encryption setting.

Answer: CE

Explanation:

The correct answer is **CE**. Here's a detailed justification:

C: Create an SCP. Attach the SCP to the root organizational unit (OU). Define the SCP to deny the ec2:CreateVolume action when the ec2:Encrypted condition equals false.

This is the most effective way to enforce encryption at the organizational level using AWS Organizations. SCPs act as guardrails, limiting the actions that can be performed by accounts within an OU. By creating an

SCP that denies the ec2:CreateVolume action when the ec2:Encrypted condition is false, you ensure that no unencrypted EBS volumes can be created within the scope of the OU (in this case, the entire organization because it is attached to the root OU). This enforces the security policy for all new EBS volumes created.

E: In the Organizations management account, specify the Default EBS volume encryption setting.

Specifying default EBS encryption at the organizational level simplifies encryption configuration. If an IAM user/role launches an EC2 instance and doesn't explicitly specify EBS encryption, then the "Default EBS volume encryption" setting takes precedence. This minimizes the impact on users since they can skip explicitly specifying encryption during volume creation, yet the organization remains compliant with the policy. This setting also applies to root users. This adds a layer of encryption regardless of what is done in the EC2 console.

Why the other options are incorrect:

A: In the Amazon EC2 console, select the EBS encryption account attribute and define a default encryption key. While you can set a default encryption key in the EC2 console, this only applies at the account level, not the organizational level. Also, this is just a default configuration, and does not enforce that EBS volumes must be encrypted, which the question requires.
B: Create an IAM permission boundary. Attach the permission boundary to the root organizational unit (OU). Define the boundary to deny the ec2:CreateVolume action when the ec2:Encrypted condition equals false. IAM permission boundaries set the maximum permissions that an IAM principal can have. They do not deny explicit actions like SCPs do. They also aren't attached to OUs.
D: Update the IAM policies for each account to deny the ec2:CreateVolume action when the ec2:Encrypted condition equals false. Updating IAM policies across multiple accounts is complex, time-consuming, and error-prone. It's much less manageable than using SCPs at the organizational level. It is also not as effective as using the default volume encryption.

Supporting Documentation:

Service Control Policies (SCPs):
https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scp.html
Default EBS Encryption by Default:
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EBSEncryption.html
IAM Permission Boundaries:
https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_boundaries.html

In conclusion, using an SCP and default volume encryption ensures that all newly created EBS volumes are encrypted without placing a significant burden on employees, satisfying the requirements stated in the question.

Question: 420 **Exam Heist**

A company wants to use an Amazon RDS for PostgreSQL DB cluster to simplify time-consuming database administrative tasks for production database workloads. The company wants to ensure that its database is highly available and will provide automatic failover support in most scenarios in less than 40 seconds. The company wants to offload reads off of the primary instance and keep costs as low as possible.

Which solution will meet these requirements?

A.Use an Amazon RDS Multi-AZ DB instance deployment. Create one read replica and point the read workload to the read replica.

B.Use an Amazon RDS Multi-AZ DB duster deployment Create two read replicas and point the read workload to the read replicas.

C.Use an Amazon RDS Multi-AZ DB instance deployment. Point the read workload to the secondary instances in

the Multi-AZ pair.

D.Use an Amazon RDS Multi-AZ DB cluster deployment Point the read workload to the reader endpoint.

Answer: D

Explanation:

The correct answer is **D: Use an Amazon RDS Multi-AZ DB cluster deployment. Point the read workload to the reader endpoint.**

Here's a detailed justification:

High Availability and Fast Failover: Multi-AZ DB clusters in RDS provide significantly faster failover times compared to Multi-AZ DB instances. While both offer high availability, clusters are designed to meet the 40-second failover requirement, while Multi-AZ instances are not explicitly guaranteed to that level of speed. Multi-AZ DB instances failover to a standby in same availability zone for better availability.

Read Scaling: RDS clusters, specifically Aurora clusters but also RDS for PostgreSQL clusters in this context, have a reader endpoint. This endpoint automatically distributes read traffic across multiple read replicas within the cluster. This offloads read workload from the primary instance as requested by the customer. Multi-AZ DB instances do not have reader endpoints and is not best practice to direct read queries to the standby instance.

Cost Optimization: Using the reader endpoint to direct read traffic eliminates the need to manually manage connections to individual read replicas. This simplifies architecture and reduces the operational overhead associated with managing multiple read replicas. Moreover, using the cluster's reader endpoint effectively balances the load across available read replicas, optimizing resource utilization and cost.

Multi-AZ DB Instance limitations: Options A and C suggest using Multi-AZ DB instances. While they provide high availability through a standby instance, they don't inherently provide read scalability or fast failover guarantees like a cluster. The standby in a Multi-AZ instance is not intended for direct read traffic.

Two Read Replicas: Option B suggests two read replicas. While it enhances read capacity, it is not the most cost efficient option. The prompt does not require read replicas and is not needed to meet high availability or failover requirements.

Reader Endpoint vs. Instance Endpoint: The reader endpoint is the key feature of a DB cluster that directly addresses the requirement to offload reads and automatically balance the load. Options that do not utilize a reader endpoint fail to address the question.

In summary, an RDS Multi-AZ DB cluster with a reader endpoint delivers the required high availability with fast failover, offloads reads efficiently, and simplifies management, making it the optimal solution.

Supporting Documentation:

Amazon RDS High Availability: Overview of HA capabilities for RDS.
Working with Amazon RDS Read Replicas: Information on using read replicas to offload read traffic.
Amazon RDS for PostgreSQL: General information about RDS for PostgreSQL.

Question: 421 **Exam Heist**

A company runs a highly available SFTP service. The SFTP service uses two Amazon EC2 Linux instances that run with elastic IP addresses to accept traffic from trusted IP sources on the internet. The SFTP service is backed by shared storage that is attached to the instances. User accounts are created and managed as Linux users in the SFTP servers.

The company wants a serverless option that provides high IOPS performance and highly configurable security. The

company also wants to maintain control over user permissions.

Which solution will meet these requirements?

A.Create an encrypted Amazon Elastic Block Store (Amazon EBS) volume. Create an AWS Transfer Family SFTP service with a public endpoint that allows only trusted IP addresses. Attach the EBS volume to the SFTP service endpoint. Grant users access to the SFTP service.

B.Create an encrypted Amazon Elastic File System (Amazon EFS) volume. Create an AWS Transfer Family SFTP service with elastic IP addresses and a VPC endpoint that has internet-facing access. Attach a security group to the endpoint that allows only trusted IP addresses. Attach the EFS volume to the SFTP service endpoint. Grant users access to the SFTP service.

C.Create an Amazon S3 bucket with default encryption enabled. Create an AWS Transfer Family SFTP service with a public endpoint that allows only trusted IP addresses. Attach the S3 bucket to the SFTP service endpoint. Grant users access to the SFTP service.

D.Create an Amazon S3 bucket with default encryption enabled. Create an AWS Transfer Family SFTP service with a VPC endpoint that has internal access in a private subnet. Attach a security group that allows only trusted IP addresses. Attach the S3 bucket to the SFTP service endpoint. Grant users access to the SFTP service.

Answer: B

Explanation:

Here's a detailed justification for why option B is the most appropriate solution, along with explanations of why the other options are less suitable:

Why Option B is Correct:

Option B leverages AWS Transfer Family with Amazon EFS to meet the stated requirements. AWS Transfer Family is the AWS service designed to provide serverless, secure, and scalable file transfer capabilities. Using EFS satisfies the need for high IOPS performance shared storage. Specifically:

Amazon EFS: Provides shared storage with high IOPS. EFS is a scalable, fully managed, NFS file system that can be shared between multiple EC2 instances or accessed by AWS Transfer Family. This addresses the company's need for high IOPS and shared storage.
AWS Transfer Family SFTP Service: Offers a serverless SFTP solution, fulfilling the company's desire to move away from managing EC2 instances.
Elastic IP Addresses & VPC Endpoint with Internet Facing Access: Using an internet-facing VPC endpoint enables the SFTP service to be accessed from trusted IP addresses over the internet, as the company currently does with their EC2 instances. The elastic IP addresses are applied at the endpoint level, allowing for predictable addressing for the trusted sources to connect.
Security Group: Attaching a security group to the endpoint that allows traffic only from trusted IP addresses provides granular security control, mirroring the current security setup with the EC2 instances.
User Access Control: AWS Transfer Family allows for integration with identity providers to manage user authentication and permissions. This ensures that the company retains control over user permissions.

Why Other Options are Incorrect:

Option A (EBS): EBS volumes are block storage, not designed for direct sharing between services like AWS Transfer Family. It is usually attached to compute instances. EBS is not a suitable storage option for this service as it cannot be directly attached as a file system for AWS Transfer Family. Additionally, attaching an EBS volume directly to the SFTP endpoint is not a supported configuration.
Option C and D (S3 with Public/Private Endpoint): While S3 can be used with AWS Transfer Family, it doesn't inherently provide the high IOPS performance that EFS does. Transferring data to/from S3 through SFTP can lead to higher costs and increased latency due to limitations in the underlying SFTP protocol. While the requirement can technically be met with Amazon S3, Amazon EFS aligns more directly with the high IOPS requirement described in the prompt. Additionally, if choosing S3, a private endpoint (Option D) is better for

security, but Option B provides a more balanced solution.

Authoritative Links:

AWS Transfer Family: https://aws.amazon.com/aws-transfer-family/
Amazon EFS: https://aws.amazon.com/efs/

In summary, Option B provides a serverless, secure, and high-performance SFTP solution that leverages AWS Transfer Family with EFS, allowing the company to meet all requirements outlined in the scenario.

Question: 422 **Exam Heist**

A company is developing a new machine learning (ML) model solution on AWS. The models are developed as independent microservices that fetch approximately 1 GB of model data from Amazon S3 at startup and load the data into memory. Users access the models through an asynchronous API. Users can send a request or a batch of requests and specify where the results should be sent.

The company provides models to hundreds of users. The usage patterns for the models are irregular. Some models could be unused for days or weeks. Other models could receive batches of thousands of requests at a time.

Which design should a solutions architect recommend to meet these requirements?

A.Direct the requests from the API to a Network Load Balancer (NLB). Deploy the models as AWS Lambda functions that are invoked by the NLB.

B.Direct the requests from the API to an Application Load Balancer (ALB). Deploy the models as Amazon Elastic Container Service (Amazon ECS) services that read from an Amazon Simple Queue Service (Amazon SQS) queue. Use AWS App Mesh to scale the instances of the ECS cluster based on the SQS queue size.

C.Direct the requests from the API into an Amazon Simple Queue Service (Amazon SQS) queue. Deploy the models as AWS Lambda functions that are invoked by SQS events. Use AWS Auto Scaling to increase the number of vCPUs for the Lambda functions based on the SQS queue size.

D.Direct the requests from the API into an Amazon Simple Queue Service (Amazon SQS) queue. Deploy the models as Amazon Elastic Container Service (Amazon ECS) services that read from the queue. Enable AWS Auto Scaling on Amazon ECS for both the cluster and copies of the service based on the queue size.

Answer: D

Explanation:

Option D provides the most scalable and cost-effective solution for the given requirements, especially considering the irregular usage patterns and the need to load 1 GB of data into memory.

Here's why:

Asynchronous Processing with SQS: Using Amazon SQS decouples the API from the model processing. This allows the API to quickly accept requests and queue them for processing, preventing the API from being overwhelmed by sudden spikes in traffic. https://aws.amazon.com/sqs/
Containerization with ECS: Deploying the models as Amazon ECS services allows for efficient resource utilization and scalability. Containers provide a consistent environment for the models to run, regardless of the underlying infrastructure. https://aws.amazon.com/ecs/
Auto Scaling based on Queue Size: Auto Scaling ECS based on the SQS queue size ensures that the number of model instances scales up or down dynamically based on the incoming request volume. This optimizes costs by only using the necessary resources. When the queue is empty (models unused), ECS can scale down close to zero (depending on chosen configuration).
Efficient Memory Management: While the models need to load 1 GB of data, ECS allows you to specify the necessary memory and CPU resources for each container. This is crucial for efficiently handling large model data sets.

Why other options are less suitable:

Option A (Lambda invoked by NLB): Lambda functions have execution time limits and memory constraints. While Lambda can be a good fit for some ML workloads, loading 1 GB of model data into memory at each invocation would likely exceed Lambda's limitations and lead to timeouts and poor performance. Also, cold starts could introduce latency and Lambda might not be as cost efficient for long lived processes.
Option B (ECS with App Mesh): App Mesh is a service mesh that provides fine-grained control over traffic routing and observability. While useful for complex microservice architectures, it adds unnecessary complexity and cost for this use case.
Option C (Lambda invoked by SQS with Auto Scaling): While Lambda can be invoked by SQS, increasing vCPUs for Lambda functions is not a supported feature. Lambda functions have fixed vCPU allocation based on the memory configured. Additionally, the memory limitations of Lambda would still be a concern with the large model size.

Question: 423 **Exam Heist**

A solutions architect wants to use the following JSON text as an identity-based policy to grant specific permissions:

```
{    "Statement": [
        {
            "Action": [
                "ssm:ListDocuments",
                "ssm:GetDocument"
            ],
            "Effect": "Allow",
            "Resource": "*",
            "Sid": ""
        }
    ],
    "Version": "2012-10-17"
}
```

Which IAM principals can the solutions architect attach this policy to? (Choose two.)

A.Role
B.Group
C.Organization
D.Amazon Elastic Container Service (Amazon ECS) resource
E.Amazon EC2 resource

Answer: AB

Explanation:

identity-based policy used for role and group

Question: 424 **Exam Heist**

A company is running a custom application on Amazon EC2 On-Demand Instances. The application has frontend nodes that need to run 24 hours a day, 7 days a week and backend nodes that need to run only for a short time based on workload. The number of backend nodes varies during the day.

The company needs to scale out and scale in more instances based on workload.

Which solution will meet these requirements MOST cost-effectively?

A.Use Reserved Instances for the frontend nodes. Use AWS Fargate for the backend nodes.
B.Use Reserved Instances for the frontend nodes. Use Spot Instances for the backend nodes.
C.Use Spot Instances for the frontend nodes. Use Reserved Instances for the backend nodes.
D.Use Spot Instances for the frontend nodes. Use AWS Fargate for the backend nodes.

Answer: B

Explanation:

The correct answer is **B. Use Reserved Instances for the frontend nodes. Use Spot Instances for the backend nodes.**

Here's a detailed justification:

The problem states that the frontend nodes must run 24/7. Reserved Instances (RIs) are the most cost-effective option for consistently running EC2 instances over a long period (1 or 3 years). By purchasing RIs for the frontend nodes, the company can significantly reduce the hourly cost compared to On-Demand Instances. RIs offer a substantial discount in exchange for a commitment to a specific instance type and region.

The backend nodes only need to run based on workload and the number of instances varies. Spot Instances offer EC2 capacity at significantly reduced prices (up to 90% off On-Demand prices). Spot Instances are ideal for fault-tolerant, stateless workloads that can be interrupted. Since the backend nodes scale based on workload and the number of backend nodes varies during the day, Spot Instances will automatically fit into the cost-effectiveness requirement.

Option A is incorrect because AWS Fargate is a serverless compute engine for containers. While Fargate can be suitable for certain backend workloads, it's generally more expensive than EC2 Spot Instances for tasks that can tolerate interruptions. The question emphasizes cost-effectiveness and implies the application is already designed to run on EC2. Fargate is a better choice when the application is containerized.

Option C is incorrect because Spot Instances are unsuitable for the frontend nodes that must run 24/7. Spot Instances can be terminated if the Spot price exceeds your bid or if capacity becomes unavailable. The frontend requires constant uptime.

Option D is incorrect for the same reasons as options A and C. It uses Spot Instances for the front-end nodes, which need constant uptime. It also suggests Fargate for the backend nodes, which is not the most cost-effective solution compared to Spot Instances for this specific EC2 scaling scenario.

Therefore, using Reserved Instances for the always-on frontend and Spot Instances for the flexible, scalable backend provides the most cost-effective solution.

Authoritative Links:

Amazon EC2 Reserved Instances: https://aws.amazon.com/ec2/pricing/reserved-instances/
Amazon EC2 Spot Instances: https://aws.amazon.com/ec2/spot/
AWS Fargate: https://aws.amazon.com/fargate/

Question: 425 **Exam Heist**

A company uses high block storage capacity to runs its workloads on premises. The company's daily peak input and output transactions per second are not more than 15,000 IOPS. The company wants to migrate the workloads to Amazon EC2 and to provision disk performance independent of storage capacity.

Which Amazon Elastic Block Store (Amazon EBS) volume type will meet these requirements MOST cost-effectively?

A.GP2 volume type
B.io2 volume type
C.GP3 volume type
D.io1 volume type

Answer: C

Explanation:

Here's a detailed justification for why the correct answer is C (GP3 volume type):

The key requirements are high block storage capacity, a maximum of 15,000 IOPS, and the need for performance independent of storage capacity, all while being cost-effective. Let's analyze each option:

GP2: GP2 (General Purpose SSD) volumes offer a balance of price and performance. However, GP2's IOPS performance is tied to the volume size. To achieve 15,000 IOPS with GP2, you would need to provision a very large volume, which would be more expensive than the other options and is not ideal when performance needs to be provisioned independent of capacity. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html

io1/io2: io1 and io2 (Provisioned IOPS SSD) volumes are designed for high-performance, I/O intensive workloads. They allow you to specify the exact IOPS you need. While io1/io2 could meet the performance requirement, they are typically more expensive than GP3, especially for a relatively low IOPS requirement like 15,000. These are optimal when you need very high, guaranteed IOPS.

GP3: GP3 (General Purpose SSD) volumes offer a balance between cost and performance. Crucially, GP3 allows you to provision IOPS and throughput independent of storage capacity. GP3 volumes provide a baseline performance and allow users to provision additional IOPS and throughput as needed, up to a certain limit. For 15,000 IOPS, GP3 is a cost-effective option that avoids over-provisioning storage just to meet IOPS requirements. GP3 also gives you flexibility on throughput. https://aws.amazon.com/blogs/aws/new-amazon-ebs-gp3-volume-provides-up-to-20-lower-cost-and-higher-performance/

Therefore, GP3 is the most cost-effective solution for a company needing high block storage capacity and 15,000 IOPS, while also demanding performance independent of capacity. It provides a better price-performance ratio compared to io1/io2 for this IOPS range and avoids the capacity-tied limitations of GP2.

Question: 426 **Exam Heist**

A company needs to store data from its healthcare application. The application’s data frequently changes. A new regulation requires audit access at all levels of the stored data.

The company hosts the application on an on-premises infrastructure that is running out of storage capacity. A solutions architect must securely migrate the existing data to AWS while satisfying the new regulation.

Which solution will meet these requirements?

A.Use AWS DataSync to move the existing data to Amazon S3. Use AWS CloudTrail to log data events.
B.Use AWS Snowcone to move the existing data to Amazon S3. Use AWS CloudTrail to log management events.
C.Use Amazon S3 Transfer Acceleration to move the existing data to Amazon S3. Use AWS CloudTrail to log

data events.

D.Use AWS Storage Gateway to move the existing data to Amazon S3. Use AWS CloudTrail to log management events.

Answer: A

Explanation:

The correct answer is A because it provides a mechanism for both data migration and audit logging of data access events, fulfilling all the requirements.

Here's a detailed justification:

The company needs to migrate data to AWS, ensure data is securely stored, and enable audit access. Amazon S3 serves as the central storage location because it's a scalable, durable, and secure object storage service suitable for healthcare application data.

AWS DataSync efficiently and securely transfers large datasets from on-premises storage to Amazon S3, handling data transfer and encryption during transit, making it ideal for migrating the existing data.

The new regulation requires audit access at all levels of stored data. AWS CloudTrail logs API calls made to AWS services, providing a detailed audit trail of who accessed what data and when. By configuring CloudTrail to log **data events** for the S3 bucket where the healthcare application data is stored, the company gains comprehensive visibility into all data access activities. Data events specifically track object-level operations such as GET, PUT, DELETE, and HEAD, providing the necessary level of audit access.

Option B is incorrect because AWS Snowcone is primarily used for data collection, processing, and migration in edge locations or environments where network connectivity is limited, which is not the primary concern in this scenario, where the existing data is hosted on an on-premises infrastructure. Further, logging management events will not satisfy the detailed audit requirement for data access at all levels.

Option C is incorrect because Amazon S3 Transfer Acceleration only speeds up transfers to S3 via the public internet. While it improves transfer speed, it doesn't address the core requirement of audit logging. Similar to Option B, logging data events is what is needed, not just accelerating the transfer.

Option D is incorrect. AWS Storage Gateway connects on-premises software appliances to cloud-based storage, it is primarily designed to create seamless hybrid storage solutions rather than a one-time migration of the existing data. More importantly, auditing management events is insufficient for the required data access audit trail. The focus on data events is critical to meeting the audit requirements.

In summary, using AWS DataSync for migration and AWS CloudTrail configured for data events on the S3 bucket addresses both the data migration and detailed data access auditing requirements effectively.

Authoritative Links:

AWS DataSync: https://aws.amazon.com/datasync/
Amazon S3: https://aws.amazon.com/s3/
AWS CloudTrail: https://aws.amazon.com/cloudtrail/

Question: 427 **Exam Heist**

A solutions architect is implementing a complex Java application with a MySQL database. The Java application must be deployed on Apache Tomcat and must be highly available.

What should the solutions architect do to meet these requirements?

A.Deploy the application in AWS Lambda. Configure an Amazon API Gateway API to connect with the Lambda

functions.

B.Deploy the application by using AWS Elastic Beanstalk. Configure a load-balanced environment and a rolling deployment policy.

C.Migrate the database to Amazon ElastiCache. Configure the ElastiCache security group to allow access from the application.

D.Launch an Amazon EC2 instance. Install a MySQL server on the EC2 instance. Configure the application on the server. Create an AMI. Use the AMI to create a launch template with an Auto Scaling group.

Answer: B

Explanation:

The correct answer is B: Deploy the application by using AWS Elastic Beanstalk. Configure a load-balanced environment and a rolling deployment policy.

Here's a detailed justification:

Elastic Beanstalk is a PaaS (Platform as a Service) that simplifies the deployment and management of web applications and services. It handles the underlying infrastructure, allowing the architect to focus on the application code. It inherently supports Java applications with Tomcat, making it a suitable choice for deploying the application.

To meet the high availability requirement, Elastic Beanstalk provides the capability to create a load-balanced environment. This automatically distributes traffic across multiple EC2 instances running the application, ensuring that if one instance fails, others can handle the load. The load balancer (usually an Application Load Balancer or Classic Load Balancer) monitors the health of the instances and routes traffic accordingly.

A rolling deployment policy minimizes downtime during application updates. Instead of taking down all instances at once, a rolling deployment updates the instances in batches, ensuring that some instances are always available to serve traffic. This reduces the impact of deployments on users and increases overall availability. Rolling deployments with health checks are crucial for maintaining service availability during updates.

Option A is incorrect because Lambda is better suited for event-driven, serverless applications, not for complex, long-running Java applications like this one running on Tomcat. Furthermore, while API Gateway can expose Lambda functions, it doesn't directly address the need for a Tomcat environment.

Option C is incorrect because ElastiCache is a caching service, not a replacement for a MySQL database. Migrating the database to ElastiCache would fundamentally change the application's architecture and likely break its functionality as ElastiCache does not offer the relational features needed.

Option D is incorrect because manually managing EC2 instances, AMIs, and Auto Scaling groups adds considerable operational overhead. While it can achieve the desired result, Elastic Beanstalk provides a higher level of abstraction and automation, simplifying the process and reducing management complexity. Manually managing AMIs and scaling policies is less efficient than using Elastic Beanstalk's managed environment. This approach increases operational overhead.

Therefore, Elastic Beanstalk with a load-balanced environment and a rolling deployment policy is the most efficient and straightforward way to deploy the Java application on Tomcat while meeting the high availability requirements.

Supporting links:

AWS Elastic Beanstalk: https://aws.amazon.com/elasticbeanstalk/
Elastic Beanstalk Rolling Deployments: https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.rolling-updates.html

Question: 428 **Exam Heist**

A serverless application uses Amazon API Gateway, AWS Lambda, and Amazon DynamoDB. The Lambda function needs permissions to read and write to the DynamoDB table.

Which solution will give the Lambda function access to the DynamoDB table MOST securely?

A.Create an IAM user with programmatic access to the Lambda function. Attach a policy to the user that allows read and write access to the DynamoDB table. Store the access_key_id and secret_access_key parameters as part of the Lambda environment variables. Ensure that other AWS users do not have read and write access to the Lambda function configuration.

B.Create an IAM role that includes Lambda as a trusted service. Attach a policy to the role that allows read and write access to the DynamoDB table. Update the configuration of the Lambda function to use the new role as the execution role.

C.Create an IAM user with programmatic access to the Lambda function. Attach a policy to the user that allows read and write access to the DynamoDB table. Store the access_key_id and secret_access_key parameters in AWS Systems Manager Parameter Store as secure string parameters. Update the Lambda function code to retrieve the secure string parameters before connecting to the DynamoDB table.

D.Create an IAM role that includes DynamoDB as a trusted service. Attach a policy to the role that allows read and write access from the Lambda function. Update the code of the Lambda function to attach to the new role as an execution role.

Answer: B

Explanation:

The correct answer is B because it leverages IAM roles, the recommended and most secure way to grant permissions to AWS services. IAM roles avoid the need to store and manage long-term credentials (like access keys) within the Lambda function or its environment, mitigating the risk of credential exposure.

Option B creates an IAM role, which is an identity that Lambda can assume. This role explicitly trusts Lambda (as specified in the trust policy of the role) to use it. The role is then granted permissions to read and write to DynamoDB via an attached policy. Finally, the Lambda function is configured to use this role as its execution role. This means when Lambda executes the function, it automatically assumes the permissions defined in the role's policy. This process is seamless and requires no manual credential management.

Option A is less secure because it involves creating an IAM user and storing its access keys in the Lambda environment variables. This is a poor practice as it increases the risk of credential compromise. If the environment variables are exposed (e.g., through a configuration leak), attackers can use the credentials to access DynamoDB.

Option C, while using Systems Manager Parameter Store to store the access keys, still relies on managing long-term credentials. Although storing secrets in Parameter Store is better than storing them directly in the environment variables, the need to manage and retrieve these credentials introduces complexity and potential security risks.

Option D is incorrect because IAM roles are used to grant permissions to services (like Lambda) to access other AWS resources, not the other way around. DynamoDB wouldn't assume a role to allow Lambda to access it. Lambda assumes the role to access DynamoDB.

Therefore, using an IAM role specifically designed for the Lambda function, as outlined in option B, aligns with AWS security best practices by adhering to the principle of least privilege and eliminating the need to manage long-term credentials directly.

Supporting documentation:

IAM Roles: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html
Lambda Execution Role: https://docs.aws.amazon.com/lambda/latest/dg/lambda-intro-execution-role.html

Security best practices in IAM: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

Question: 429 **Exam Heist**

The following IAM policy is attached to an IAM group. This is the only policy applied to the group.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "1",
            "Effect": "Allow",
            "Action": "ec2:*",
            "Resource": "*",
            "Condition": {
                "StringEquals": {
                    "ec2:Region": "us-east-1"
                }
            }
        },
        {
            "Sid": "2",
            "Effect": "Deny",
            "Action": [
                "ec2:StopInstances",
                "ec2:TerminateInstances"
            ],
            "Resource": "*",
            "Condition": {
                "BoolIfExists": {"aws:MultiFactorAuthPresent": false}
            }
        }
    ]
}
```

What are the effective IAM permissions of this policy for group members?

A.Group members are permitted any Amazon EC2 action within the us-east-1 Region. Statements after the Allow permission are not applied.

B.Group members are denied any Amazon EC2 permissions in the us-east-1 Region unless they are logged in with multi-factor authentication (MFA).

C.Group members are allowed the ec2:StopInstances and ec2:TerminateInstances permissions for all Regions when logged in with multi-factor authentication (MFA). Group members are permitted any other Amazon EC2 action.

D.Group members are allowed the ec2:StopInstances and ec2:TerminateInstances permissions for the us-east-1 Region only when logged in with multi-factor authentication (MFA). Group members are permitted any other Amazon EC2 action within the us-east-1 Region.

Answer: D

Explanation:

Group members are allowed the ec2:StopInstances and ec2:TerminateInstances permissions for the us-east-1 Region only when logged in with multi-factor authentication (MFA). Group members are permitted any other Amazon EC2 action within the us-east-1 Region.

Question: 430 **Exam Heist**

A manufacturing company has machine sensors that upload .csv files to an Amazon S3 bucket. These .csv files must be converted into images and must be made available as soon as possible for the automatic generation of graphical reports.

The images become irrelevant after 1 month, but the .csv files must be kept to train machine learning (ML) models twice a year. The ML trainings and audits are planned weeks in advance.

Which combination of steps will meet these requirements MOST cost-effectively? (Choose two.)

A.Launch an Amazon EC2 Spot Instance that downloads the .csv files every hour, generates the image files, and uploads the images to the S3 bucket.

B.Design an AWS Lambda function that converts the .csv files into images and stores the images in the S3 bucket. Invoke the Lambda function when a .csv file is uploaded.

C.Create S3 Lifecycle rules for .csv files and image files in the S3 bucket. Transition the .csv files from S3 Standard to S3 Glacier 1 day after they are uploaded. Expire the image files after 30 days.

D.Create S3 Lifecycle rules for .csv files and image files in the S3 bucket. Transition the .csv files from S3 Standard to S3 One Zone-Infrequent Access (S3 One Zone-IA) 1 day after they are uploaded. Expire the image files after 30 days.

E.Create S3 Lifecycle rules for .csv files and image files in the S3 bucket. Transition the .csv files from S3 Standard to S3 Standard-Infrequent Access (S3 Standard-IA) 1 day after they are uploaded. Keep the image files in Reduced Redundancy Storage (RRS).

Answer: BC

Explanation:

The question requires converting .csv files uploaded to S3 into images for rapid report generation, maintaining the .csv files for ML training, and managing storage costs. The selected solution, BC, addresses these requirements effectively.

B. Design an AWS Lambda function that converts the .csv files into images and stores the images in the S3 bucket. Invoke the Lambda function when a .csv file is uploaded. This is the most efficient way to immediately convert the .csv files into images. Using S3 event notifications and a Lambda function, the conversion happens automatically upon upload. Lambda is cost-effective for event-driven tasks as you only pay for the compute time used during the conversion. An EC2 instance (option A) would be constantly running, incurring costs even when idle, making Lambda more economical for this task. This ensures the images are available for report generation as quickly as possible.

C. Create S3 Lifecycle rules for .csv files and image files in the S3 bucket. Transition the .csv files from S3 Standard to S3 Glacier 1 day after they are uploaded. Expire the image files after 30 days. This part handles the lifecycle management of both the .csv and image files to optimize storage costs. Since the images are only needed for a month, expiring them after 30 days prevents unnecessary storage charges. Transitioning the .csv files to S3 Glacier reduces the storage costs for the .csv files, as they are only used twice a year. S3 Glacier is a low-cost storage option suitable for infrequently accessed data. S3 Standard-IA (option E) would be more expensive than Glacier for data accessed only twice a year. S3 One Zone-IA (option D) loses data if the availability zone is destroyed, violating best practices. Storing images in RRS (option E) is deprecated and

not recommended.

Therefore, Lambda is the ideal service for immediate file conversion and S3 lifecycle policies efficiently manage storage costs, fulfilling both speed and cost-effectiveness requirements.

Relevant Links:

AWS Lambda: https://aws.amazon.com/lambda/
Amazon S3 Lifecycle: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-management-overview.html
Amazon S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/

Question: 431 **Exam Heist**

A company has developed a new video game as a web application. The application is in a three-tier architecture in a VPC with Amazon RDS for MySQL in the database layer. Several players will compete concurrently online. The game's developers want to display a top-10 scoreboard in near-real time and offer the ability to stop and restore the game while preserving the current scores.

What should a solutions architect do to meet these requirements?

A.Set up an Amazon ElastiCache for Memcached cluster to cache the scores for the web application to display.

B.Set up an Amazon ElastiCache for Redis cluster to compute and cache the scores for the web application to display.

C.Place an Amazon CloudFront distribution in front of the web application to cache the scoreboard in a section of the application.

D.Create a read replica on Amazon RDS for MySQL to run queries to compute the scoreboard and serve the read traffic to the web application.

Answer: B

Explanation:

The correct answer is **B. Set up an Amazon ElastiCache for Redis cluster to compute and cache the scores for the web application to display.**

Here's a detailed justification:

The requirements include near-real-time scoreboard updates and the ability to save/restore game state including scores. Redis is an in-memory data store that excels in these scenarios due to its speed and versatile data structures.

1. **Near Real-Time Updates:** Redis provides extremely low latency read and write operations, crucial for displaying scores in near real-time. Unlike disk-based databases (like RDS MySQL), in-memory operations are significantly faster.
2. **Score Computation and Storage:** Redis offers sorted sets, a data structure perfectly suited for maintaining a ranked leaderboard. The score for each player can be added or updated, and Redis automatically maintains the order. Retrieving the top 10 scores is an efficient operation.
3. **Save/Restore Game State:** Redis supports persistence. Data can be periodically saved to disk (RDB snapshots) or logged to an append-only file (AOF), enabling the restoration of the entire dataset (including scores) in case of failure or for the stop/restore feature.
4. **ElastiCache for Redis:** AWS ElastiCache simplifies the management of a Redis cluster, handling tasks like scaling, patching, and monitoring. It provides a managed service, reducing the operational overhead for the game developers.

Why other options are not ideal:

A. Amazon ElastiCache for Memcached: While Memcached is also an in-memory cache, it primarily focuses on simple key-value caching and does not offer the sorted set data structure or built-in persistence capabilities required for the scoreboard and game state preservation.

C. Amazon CloudFront: CloudFront is a CDN (Content Delivery Network) that caches static content at edge locations. While it's useful for caching static parts of the web application, it is not suitable for caching dynamically changing data like the scoreboard, as CloudFront cache invalidation may not be quick enough for near-real-time updates. Furthermore, it cannot handle the score computation or game state persistence.

D. Amazon RDS for MySQL Read Replica: While a read replica offloads read traffic from the primary database, it is still a disk-based database. Computing the scoreboard on the read replica and serving read traffic would introduce latency and not meet the near-real-time requirement. Also, reading top scores from a relational database is much slower than using a Redis sorted set. Moreover, a read replica doesn't offer the in-memory performance or data structures optimized for real-time ranking and quick restore.

Authoritative Links:

Amazon ElastiCache for Redis: https://aws.amazon.com/elasticache/redis/
Redis Sorted Sets: https://redis.io/docs/data-types/sorted-sets/
Redis Persistence: https://redis.io/docs/data-persistence/

Question: 432 **Exam Heist**

An ecommerce company wants to use machine learning (ML) algorithms to build and train models. The company will use the models to visualize complex scenarios and to detect trends in customer data. The architecture team wants to integrate its ML models with a reporting platform to analyze the augmented data and use the data directly in its business intelligence dashboards.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use AWS Glue to create an ML transform to build and train models. Use Amazon OpenSearch Service to visualize the data.

B.Use Amazon SageMaker to build and train models. Use Amazon QuickSight to visualize the data.

C.Use a pre-built ML Amazon Machine Image (AMI) from the AWS Marketplace to build and train models. Use Amazon OpenSearch Service to visualize the data.

D.Use Amazon QuickSight to build and train models by using calculated fields. Use Amazon QuickSight to visualize the data.

Answer: B

Explanation:

The correct answer is B because it leverages managed AWS services specifically designed for machine learning and business intelligence, minimizing operational overhead.

Here's a detailed justification:

Amazon SageMaker: SageMaker is a fully managed machine learning service that provides a comprehensive environment for building, training, and deploying ML models. It eliminates the need for managing underlying infrastructure, handling dependencies, and configuring training clusters. This significantly reduces operational overhead compared to options A and C.

Amazon QuickSight: QuickSight is a serverless business intelligence service that allows you to easily create and share interactive dashboards and reports. Its direct integration with AWS data sources, including

SageMaker-generated data, simplifies the process of visualizing and analyzing ML-augmented data. Unlike OpenSearch Service (in options A and C), QuickSight is designed specifically for business intelligence and reporting, offering features like calculated fields and interactive visualizations, reducing the need for custom coding and configurations. QuickSight also scales automatically, minimizing operational overhead.

Option A is less suitable because AWS Glue ML Transforms are primarily for data cleaning and transformation, not comprehensive model building and training. While OpenSearch Service can visualize data, it's primarily a search and analytics engine and requires more configuration and management for BI use cases compared to QuickSight.

Option C requires managing a custom AMI, which increases operational overhead related to security patching, software updates, and compatibility issues. Furthermore, using OpenSearch Service for visualization, as mentioned before, is not as optimized as using QuickSight.

Option D is incorrect because QuickSight's calculated fields are limited in their machine learning capabilities. They cannot be used to build and train complex ML models as required by the problem statement.

In summary, Option B using SageMaker for ML model development and QuickSight for visualization provides a fully managed, integrated, and scalable solution with the least operational overhead for the given requirements.

Supporting Links:

Amazon SageMaker: https://aws.amazon.com/sagemaker/
Amazon QuickSight: https://aws.amazon.com/quicksight/
AWS Glue: https://aws.amazon.com/glue/
Amazon OpenSearch Service: https://aws.amazon.com/opensearch-service/

Question: 433 **Exam Heist**

A company is running its production and nonproduction environment workloads in multiple AWS accounts. The accounts are in an organization in AWS Organizations. The company needs to design a solution that will prevent the modification of cost usage tags.

Which solution will meet these requirements?

A.Create a custom AWS Config rule to prevent tag modification except by authorized principals.
B.Create a custom trail in AWS CloudTrail to prevent tag modification.
C.Create a service control policy (SCP) to prevent tag modification except by authorized principals.
D.Create custom Amazon CloudWatch logs to prevent tag modification.

Answer: C

Explanation:

The correct answer is C: Create a service control policy (SCP) to prevent tag modification except by authorized principals. Here's a detailed justification:

SCPs are a powerful feature of AWS Organizations that allow you to centrally control permissions for all accounts within your organization. They act as guardrails, setting the maximum permissions available to member accounts. Critically, SCPs affect all IAM users and roles within the affected accounts, even the root user. This centralized enforcement is precisely what's needed to prevent unauthorized tag modification across multiple accounts.

Option C's "except by authorized principals" is also key. SCPs can include conditions to allow specific IAM roles or users to bypass the restriction. This enables authorized users (like those in the Cost Management

team) to modify tags as needed for legitimate purposes, while still preventing accidental or malicious modification by others. This conditional control makes SCPs ideal for this scenario.

Option A (AWS Config rule) can detect non-compliant tag modifications after they happen, but it can't prevent them directly. Config is for auditing, not enforcement. Option B (CloudTrail) also logs activity, so it's valuable for auditing but not prevention. Option D (CloudWatch logs) similarly focuses on monitoring and alerting based on logs; it doesn't inherently block actions. Only SCPs offer the preventive control necessary.

In summary, SCPs provide a centralized and enforceable mechanism to restrict actions (like tag modification) across multiple AWS accounts within an organization, while allowing exceptions for authorized users. This makes it the best choice for the specified requirements.

Further reading:

AWS Organizations documentation:
https://docs.aws.amazon.com/organizations/latest/userguide/orgs_introduction.html
Service control policies (SCPs):
https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html

Question: 434 **Exam Heist**

A company hosts its application in the AWS Cloud. The application runs on Amazon EC2 instances behind an Elastic Load Balancer in an Auto Scaling group and with an Amazon DynamoDB table. The company wants to ensure the application can be made available in anotherAWS Region with minimal downtime.

What should a solutions architect do to meet these requirements with the LEAST amount of downtime?

A.Create an Auto Scaling group and a load balancer in the disaster recovery Region. Configure the DynamoDB table as a global table. Configure DNS failover to point to the new disaster recovery Region's load balancer.

B.Create an AWS CloudFormation template to create EC2 instances, load balancers, and DynamoDB tables to be launched when needed Configure DNS failover to point to the new disaster recovery Region's load balancer.

C.Create an AWS CloudFormation template to create EC2 instances and a load balancer to be launched when needed. Configure the DynamoDB table as a global table. Configure DNS failover to point to the new disaster recovery Region's load balancer.

D.Create an Auto Scaling group and load balancer in the disaster recovery Region. Configure the DynamoDB table as a global table. Create an Amazon CloudWatch alarm to trigger an AWS Lambda function that updates Amazon Route 53 pointing to the disaster recovery load balancer.

Answer: A

Explanation:

The correct answer is A because it provides the most streamlined approach to minimizing downtime during a failover to a disaster recovery region.

Here's a breakdown of why:

Auto Scaling Group and Load Balancer in DR Region: Creating these resources before a disaster event allows for a faster switchover. The EC2 instances are ready (or can be pre-warmed).
DynamoDB Global Table: DynamoDB global tables provide automatic, multi-region, multi-active replication. This ensures that the data is available in the disaster recovery region and that data consistency is maintained. This removes the need to handle data replication manually, which can be time-consuming and error-prone.
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/globaltables.html
DNS Failover: Using Route 53 DNS failover allows for quick redirection of traffic to the disaster recovery region by simply changing the DNS record when needed. This is a standard and efficient method for handling regional failures. https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-failover.html

Why other options are less ideal:

Option B: Creating resources using CloudFormation only when needed introduces delay. This is not an ideal solution for minimizing downtime, as the application is unavailable while the resources are provisioned. Furthermore, it does not consider the need to have the database layer pre-synced.
Option C: This option is missing the Auto Scaling Group. If EC2 instances are not created using ASG, it might be difficult to scale them up, especially during/after a DR event.
Option D: This option is overly complicated. It uses CloudWatch alarm and Lambda function for Route 53 DNS, which adds unnecessary complexity and increases the potential for failure. Route 53 supports native failover mechanisms and is the best choice.

In conclusion, option A provides the best combination of pre-provisioned resources, data replication via DynamoDB global tables, and efficient failover using Route 53 to minimize downtime during a regional disaster.

Question: 435 **Exam Heist**

A company needs to migrate a MySQL database from its on-premises data center to AWS within 2 weeks. The database is 20 TB in size. The company wants to complete the migration with minimal downtime.

Which solution will migrate the database MOST cost-effectively?

A.Order an AWS Snowball Edge Storage Optimized device. Use AWS Database Migration Service (AWS DMS) with AWS Schema Conversion Tool (AWS SCT) to migrate the database with replication of ongoing changes. Send the Snowball Edge device to AWS to finish the migration and continue the ongoing replication.

B.Order an AWS Snowmobile vehicle. Use AWS Database Migration Service (AWS DMS) with AWS Schema Conversion Tool (AWS SCT) to migrate the database with ongoing changes. Send the Snowmobile vehicle back to AWS to finish the migration and continue the ongoing replication.

C.Order an AWS Snowball Edge Compute Optimized with GPU device. Use AWS Database Migration Service (AWS DMS) with AWS Schema Conversion Tool (AWS SCT) to migrate the database with ongoing changes. Send the Snowball device to AWS to finish the migration and continue the ongoing replication

D.Order a 1 GB dedicated AWS Direct Connect connection to establish a connection with the data center. Use AWS Database Migration Service (AWS DMS) with AWS Schema Conversion Tool (AWS SCT) to migrate the database with replication of ongoing changes.

Answer: A

Explanation:

The most cost-effective solution for migrating a 20 TB MySQL database to AWS within 2 weeks with minimal downtime is option A, using AWS Snowball Edge Storage Optimized and AWS DMS with AWS SCT.

Here's why:

Data Size and Time Constraint: Migrating 20 TB over the internet, even with a dedicated 1 GB AWS Direct Connect connection (Option D), is likely to take longer than 2 weeks and cause more significant downtime. AWS Snowball Edge devices are designed for large-scale data transfers when network bandwidth is a constraint. https://aws.amazon.com/snowball/
Cost-Effectiveness: AWS Snowball Edge Storage Optimized devices are more cost-effective for this amount of data compared to AWS Snowmobile (Option B). AWS Snowmobile is intended for exabyte-scale data transfers.
Minimal Downtime via Replication: AWS Database Migration Service (DMS) allows for continuous replication of changes to the database during the migration process. The AWS Schema Conversion Tool (SCT) helps convert the source database schema to the target AWS database (if needed) reducing manual intervention and potential errors. This keeps downtime minimal. https://aws.amazon.com/dms/

https://aws.amazon.com/sct/
Compute Optimized Snowball: AWS Snowball Edge Compute Optimized (Option C) is not needed since the primary requirement is data transfer and not intensive computation at the edge. Storage Optimized is sufficient and more cost-effective.
Process: The process involves using DMS/SCT to set up the initial database schema and begin replicating data to the Snowball Edge while it's on-premises. Once the initial data is transferred to the Snowball, it's shipped to AWS. After AWS receives the Snowball and uploads the data to AWS, the continuous replication via DMS ensures that only the changes that occurred during shipping need to be applied to the target database, minimizing the final cutover downtime.
Direct Connect Limitations: Though Direct Connect provides dedicated network connectivity, transferring 20 TB of data within two weeks while maintaining minimal downtime can still be challenging and potentially costly due to bandwidth usage charges.

Question: 436 **Exam Heist**

A company moved its on-premises PostgreSQL database to an Amazon RDS for PostgreSQL DB instance. The company successfully launched a new product. The workload on the database has increased. The company wants to accommodate the larger workload without adding infrastructure.

Which solution will meet these requirements MOST cost-effectively?

A.Buy reserved DB instances for the total workload. Make the Amazon RDS for PostgreSQL DB instance larger.
B.Make the Amazon RDS for PostgreSQL DB instance a Multi-AZ DB instance.
C.Buy reserved DB instances for the total workload. Add another Amazon RDS for PostgreSQL DB instance.
D.Make the Amazon RDS for PostgreSQL DB instance an on-demand DB instance.

Answer: A

Explanation:

The question asks for the most cost-effective solution to handle an increased database workload in Amazon RDS for PostgreSQL without adding infrastructure. Option A, buying reserved DB instances for the total workload and making the RDS instance larger, is the most cost-effective.

Here's why:

Reserved Instances (RIs): RIs offer significant cost savings (up to 75%) compared to on-demand instances for predictable workloads. Buying RIs for the expected total workload guarantees these savings.
Vertical Scaling (Increasing Instance Size): RDS allows you to scale vertically (increase the instance size) to provide more CPU, memory, and storage. This addresses the increased workload without requiring code changes or creating new databases. It leverages existing infrastructure.
Cost-Effectiveness: While RIs require upfront commitment, the substantial discount on the hourly rate over the long term makes them far cheaper than continuously running larger on-demand instances.
Alternative B (Multi-AZ): Multi-AZ provides high availability and failover, which is important, but does not directly address the performance scaling issue required by increased workload. It's mainly for disaster recovery, not performance scaling. While Multi-AZ provides a standby replica, it is typically for failover and not for read-heavy operations.
Alternative C (Adding another RDS Instance): This implies creating a read replica (or some other form of sharding/database partitioning), which adds infrastructure and complexity. While read replicas can help with read scaling, they are not necessary if increasing instance size is sufficient and not the most cost effective. The question says 'without adding infrastructure'.
Alternative D (On-Demand): Using on-demand instances is the most expensive option for a continuous workload. It offers no cost savings compared to RIs and is generally used for unpredictable or short-term workloads.

Therefore, purchasing Reserved Instances to cover the projected increased workload, coupled with scaling the existing RDS instance, offers the most cost-effective path to accommodating the larger load without the complexity of new infrastructure or the higher expense of on-demand resources.

Here are authoritative links for further research:

Amazon RDS Pricing: https://aws.amazon.com/rds/pricing/
Amazon RDS Reserved Instances: https://aws.amazon.com/rds/reserved-instances/
Scaling Amazon RDS Instances:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ScalingUp.html

Question: 437 **Exam Heist**

A company operates an ecommerce website on Amazon EC2 instances behind an Application Load Balancer (ALB) in an Auto Scaling group. The site is experiencing performance issues related to a high request rate from illegitimate external systems with changing IP addresses. The security team is worried about potential DDoS attacks against the website. The company must block the illegitimate incoming requests in a way that has a minimal impact on legitimate users.

What should a solutions architect recommend?

A.Deploy Amazon Inspector and associate it with the ALB.
B.Deploy AWS WAF, associate it with the ALB, and configure a rate-limiting rule.
C.Deploy rules to the network ACLs associated with the ALB to block the incomingtraffic.
D.Deploy Amazon GuardDuty and enable rate-limiting protection when configuring GuardDuty.

Answer: B

Explanation:

The correct answer is **B. Deploy AWS WAF, associate it with the ALB, and configure a rate-limiting rule.**

Here's a detailed justification:

The scenario requires blocking illegitimate requests targeting an ecommerce website, mitigating potential DDoS attacks, and minimizing impact on legitimate users. AWS WAF (Web Application Firewall) is specifically designed to protect web applications from common web exploits and bots that could affect availability, compromise security, or consume excessive resources.

AWS WAF's Rate-Based Rules: WAF allows you to define rules based on the rate of requests coming from a specific IP address. By configuring a rate-limiting rule, you can automatically block or count requests exceeding a defined threshold within a specified time period. This directly addresses the problem of high request rates from illegitimate systems with changing IP addresses. Legitimate users are less likely to trigger these rate limits.

Association with ALB: AWS WAF can be directly associated with an Application Load Balancer (ALB), making it an ideal solution for filtering traffic before it reaches the EC2 instances. This integration provides a point of control at the application layer (Layer 7).

Minimal Impact on Legitimate Users: WAF's rate-based rules and other filtering mechanisms are designed to minimize the impact on legitimate users. Instead of a blanket block, only requests exceeding the defined rate are blocked. You can also configure WAF rules to challenge suspicious requests (e.g., with CAPTCHA) instead of outright blocking them.

Let's examine why the other options are less suitable:

A. Deploy Amazon Inspector and associate it with the ALB. Amazon Inspector is a vulnerability management service that primarily focuses on identifying security vulnerabilities within your EC2 instances and network configurations. It is not designed to block incoming traffic based on request rates.

C. Deploy rules to the network ACLs associated with the ALB to block the incoming traffic. Network ACLs operate at the subnet level and are primarily used for network-level filtering (Layer 3 and 4). While they can block traffic based on IP addresses, they are not suitable for dynamic rate limiting or complex web application filtering like AWS WAF. Updating ACLs requires more operational overhead and can be difficult when IP addresses are changing rapidly.

D. Deploy Amazon GuardDuty and enable rate-limiting protection when configuring GuardDuty. Amazon GuardDuty is a threat detection service that analyzes AWS CloudTrail, VPC Flow Logs, and DNS logs to identify malicious activity. While GuardDuty can detect potential DDoS attacks, it does not provide the direct traffic filtering and rate-limiting capabilities required to mitigate the issue in real-time. It mainly sends alerts to the security team.

Therefore, AWS WAF with rate-limiting rules is the most effective solution for blocking illegitimate incoming requests from changing IP addresses while minimizing the impact on legitimate users, addressing the requirement to protect the ecommerce website from potential DDoS attacks.

Authoritative Links:

AWS WAF: https://aws.amazon.com/waf/
AWS WAF Rate-Based Rules: https://docs.aws.amazon.com/waf/latest/developerguide/waf-rate-based-rule.html
Amazon Inspector: https://aws.amazon.com/inspector/
Amazon GuardDuty: https://aws.amazon.com/guardduty/

Question: 438 **Exam Heist**

A company wants to share accounting data with an external auditor. The data is stored in an Amazon RDS DB instance that resides in a private subnet. The auditor has its own AWS account and requires its own copy of the database.

What is the MOST secure way for the company to share the database with the auditor?

A.Create a read replica of the database. Configure IAM standard database authentication to grant the auditor access.

B.Export the database contents to text files. Store the files in an Amazon S3 bucket. Create a new IAM user for the auditor. Grant the user access to the S3 bucket.

C.Copy a snapshot of the database to an Amazon S3 bucket. Create an IAM user. Share the user's keys with the auditor to grant access to the object in the S3 bucket.

D.Create an encrypted snapshot of the database. Share the snapshot with the auditor. Allow access to the AWS Key Management Service (AWS KMS) encryption key.

Answer: D

Explanation:

The most secure way to share the database with the external auditor is to create an encrypted snapshot and share it with the auditor's AWS account along with access to the KMS key. This approach ensures data confidentiality and integrity while minimizing the attack surface.

Option D is superior because it avoids exposing a live, replicated database instance directly to the auditor's account (as in Option A). Sharing a live replica introduces potential security vulnerabilities and risks unauthorized access or modifications. Encrypting the snapshot at rest ensures that the data remains

protected even if the snapshot itself is compromised. Sharing the snapshot rather than the live data allows the auditor to work on a point-in-time copy, without impacting the company's production database. Sharing a snapshot is better than exporting to text files (as in B) which can be cumbersome, difficult to manage, and increase the risk of exposure. Additionally, storing in an S3 bucket and granting bucket access to the auditor is more complex and presents additional risk when compared to sharing a snapshot. Sharing the snapshot is preferred over sharing user keys (as in C) which is insecure.

Sharing an encrypted snapshot through AWS provides proper access controls and accountability, while sharing user keys is considered a security anti-pattern. It also allows for proper key rotation policies and auditing of key usage. By leveraging KMS, the company maintains control over the encryption key and can revoke access at any time. The auditor, in their own AWS account, can then restore the snapshot into a new database instance in their own environment, isolated from the company's infrastructure.

For further information, you can refer to the following AWS documentation:

Sharing AWS KMS keys: https://docs.aws.amazon.com/kms/latest/developerguide/share-keys.html
Sharing snapshots: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-share-snapshot.html

Question: 439 **Exam Heist**

A solutions architect configured a VPC that has a small range of IP addresses. The number of Amazon EC2 instances that are in the VPC is increasing, and there is an insufficient number of IP addresses for future workloads.

Which solution resolves this issue with the LEAST operational overhead?

A.Add an additional IPv4 CIDR block to increase the number of IP addresses and create additional subnets in the VPC. Create new resources in the new subnets by using the new CIDR.

B.Create a second VPC with additional subnets. Use a peering connection to connect the second VPC with the first VPC Update the routes and create new resources in the subnets of the second VPC.

C.Use AWS Transit Gateway to add a transit gateway and connect a second VPC with the first VPUpdate the routes of the transit gateway and VPCs. Create new resources in the subnets of the second VPC.

D.Create a second VPC. Create a Site-to-Site VPN connection between the first VPC and the second VPC by using a VPN-hosted solution on Amazon EC2 and a virtual private gateway. Update the route between VPCs to the traffic through the VPN. Create new resources in the subnets of the second VPC.

Answer: A

Explanation:

The correct answer is A: Add an additional IPv4 CIDR block to the existing VPC. This offers the least operational overhead because it directly addresses the IP address exhaustion issue within the existing infrastructure without requiring the creation and management of new VPCs or complex networking configurations.

Here's a detailed justification:

IP Address Exhaustion: The core problem is the lack of available IP addresses within the current VPC's CIDR block.
Direct Solution: Adding an IPv4 CIDR block expands the address space of the existing VPC, providing more IP addresses for EC2 instances. This avoids the need to create new network infrastructure.
Subnet Creation: Additional subnets can be created using the new CIDR block, allowing for logical grouping and organization of resources.
Operational Overhead: Option A involves modifying the existing VPC rather than creating entirely new ones, minimizing the complexity of routing configurations, security groups, and network ACLs. It also keeps all resources within a single VPC, simplifying management and monitoring.

Peering, Transit Gateway, and VPNs: Options B, C, and D all involve creating additional VPCs and connecting them using peering connections, Transit Gateway, or Site-to-Site VPN. These are valid solutions for network connectivity and scalability but introduce significant operational overhead compared to simply expanding the existing VPC's address space. They require managing inter-VPC routing, security considerations, and potential bandwidth limitations.
Least Effort: Adding a CIDR block is the most straightforward way to increase the number of IP addresses in an existing VPC, requiring minimal configuration changes.

In summary, adding an additional IPv4 CIDR block to the existing VPC addresses the IP address exhaustion issue directly with the least amount of configuration and management overhead compared to creating and connecting multiple VPCs.

For further research, refer to the AWS documentation on VPCs:

Your VPC and subnets
Associate additional IPv4 CIDR blocks with your VPC

Question: 440 **Exam Heist**

A company used an Amazon RDS for MySQL DB instance during application testing. Before terminating the DB instance at the end of the test cycle, a solutions architect created two backups. The solutions architect created the first backup by using the mysqldump utility to create a database dump. The solutions architect created the second backup by enabling the final DB snapshot option on RDS termination.

The company is now planning for a new test cycle and wants to create a new DB instance from the most recent backup. The company has chosen a MySQL-compatible edition ofAmazon Aurora to host the DB instance.

Which solutions will create the new DB instance? (Choose two.)

A.Import the RDS snapshot directly into Aurora.
B.Upload the RDS snapshot to Amazon S3. Then import the RDS snapshot into Aurora.
C.Upload the database dump to Amazon S3. Then import the database dump into Aurora.
D.Use AWS Database Migration Service (AWS DMS) to import the RDS snapshot into Aurora.
E.Upload the database dump to Amazon S3. Then use AWS Database Migration Service (AWS DMS) to import the database dump into Aurora.

Answer: AC

Explanation:

The correct solutions for creating a new MySQL-compatible Amazon Aurora DB instance from the backups are **A** and **C**.

Option A: Import the RDS snapshot directly into Aurora. RDS snapshots are the native backup mechanism provided by AWS for RDS databases. Aurora is designed to be compatible with RDS snapshots of MySQL and PostgreSQL engines. Therefore, you can directly restore an RDS snapshot of a MySQL instance into an Aurora MySQL instance using the AWS Management Console, AWS CLI, or SDKs. This is the most straightforward and efficient method if you have a readily available RDS snapshot.https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/AuroraMySQL.Managing.html

Option C: Upload the database dump to Amazon S3. Then import the database dump into Aurora. The mysqldump utility creates a logical backup of your database. This backup can be imported into a new database instance. The mysqldump file needs to be uploaded to Amazon S3 for Aurora to access it. Aurora can then import the data from S3. This method provides flexibility, especially if you need to perform some data transformations before importing into

Aurora.https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/AuroraMySQL.Migrating.ExtMySQL.html

Why the other options are incorrect:

B: Upload the RDS snapshot to Amazon S3. Then import the RDS snapshot into Aurora. While you can export RDS snapshots to S3, this process is more complex and primarily used for cross-account or cross-region migrations. You can directly restore from an RDS snapshot without uploading to S3 first.
D: Use AWS Database Migration Service (AWS DMS) to import the RDS snapshot into Aurora. While DMS is useful for heterogeneous database migrations or for migrating with minimal downtime, it is not needed to import RDS snapshots of MySQL into Aurora. Native restoration from snapshot is simpler and faster.
E: Upload the database dump to Amazon S3. Then use AWS Database Migration Service (AWS DMS) to import the database dump into Aurora. DMS is not needed for restoring mysqldump files to Aurora. Aurora has built-in functionality to restore from mysqldump files located in S3. DMS would add unnecessary complexity and cost.

Question: 441 **Exam Heist**

A company hosts a multi-tier web application on Amazon Linux Amazon EC2 instances behind an Application Load Balancer. The instances run in an Auto Scaling group across multiple Availability Zones. The company observes that the Auto Scaling group launches more On-Demand Instances when the application's end users access high volumes of static web content. The company wants to optimize cost.

What should a solutions architect do to redesign the application MOST cost-effectively?

A.Update the Auto Scaling group to use Reserved Instances instead of On-Demand Instances.
B.Update the Auto Scaling group to scale by launching Spot Instances instead of On-Demand Instances.
C.Create an Amazon CloudFront distribution to host the static web contents from an Amazon S3 bucket.
D.Create an AWS Lambda function behind an Amazon API Gateway API to host the static website contents.

Answer: C

Explanation:

The most cost-effective solution is to use Amazon CloudFront to serve static web content from an Amazon S3 bucket (Option C). Here's why:

Cost Optimization: The question specifically highlights the need for cost optimization. Serving static content directly from EC2 instances behind an Application Load Balancer is expensive due to the compute resources consumed and the associated networking costs. CloudFront and S3 are designed for efficient and cost-effective static content delivery.

CloudFront Caching: CloudFront is a Content Delivery Network (CDN) that caches content at edge locations globally. This means that when a user requests static content, CloudFront can often serve it from a nearby edge location, reducing latency and offloading traffic from the EC2 instances. This reduces the need for the Auto Scaling group to scale up when static content is frequently accessed.

S3 Cost Efficiency: Amazon S3 is a highly scalable and cost-effective object storage service. Storing static content in S3 is significantly cheaper than storing it on EC2 instance storage.

Reduced EC2 Load: By offloading static content delivery to CloudFront and S3, the EC2 instances behind the Application Load Balancer only need to handle dynamic content and application logic. This reduces the overall load on the EC2 instances and allows the Auto Scaling group to scale based on the actual application workload, rather than static content requests.

Alternatives Considered:

Option A (Reserved Instances): While Reserved Instances can offer cost savings compared to On-Demand Instances, they don't address the underlying problem of using compute instances to serve static content. It is still less efficient than using a CDN.
Option B (Spot Instances): Spot Instances can be cheaper than On-Demand Instances, but they are subject to interruption. Serving essential web content from Spot Instances can lead to application instability if the Spot Instances are terminated, moreover it does not address the inefficiency of serving static content from EC2 instances.
Option D (Lambda and API Gateway): Using Lambda and API Gateway for static content delivery is not generally recommended. While it's possible, it's more complex and potentially more expensive than using CloudFront and S3. Lambda is better suited for dynamic content and serverless functions, and API Gateway adds unnecessary overhead for simply serving static files.

Therefore, offloading the static contents to S3 and distributing it through CloudFront offers optimal cost savings, enhanced performance, and simplified architecture compared to the other options.

Relevant Links:

Amazon CloudFront: https://aws.amazon.com/cloudfront/
Amazon S3: https://aws.amazon.com/s3/

Question: 442 **Exam Heist**

A company stores several petabytes of data across multiple AWS accounts. The company uses AWS Lake Formation to manage its data lake. The company's data science team wants to securely share selective data from its accounts with the company's engineering team for analytical purposes.

Which solution will meet these requirements with the LEAST operational overhead?

A.Copy the required data to a common account. Create an IAM access role in that account. Grant access by specifying a permission policy that includes users from the engineering team accounts as trusted entities.

B.Use the Lake Formation permissions Grant command in each account where the data is stored to allow the required engineering team users to access the data.

C.Use AWS Data Exchange to privately publish the required data to the required engineering team accounts.

D.Use Lake Formation tag-based access control to authorize and grant cross-account permissions for the required data to the engineering team accounts.

Answer: D

Explanation:

The correct answer is D because it provides a centralized and scalable approach to cross-account data sharing with minimal operational overhead using Lake Formation. Lake Formation's tag-based access control allows you to categorize data based on tags and then grant permissions based on these tags. This simplifies management, especially when dealing with petabytes of data across multiple accounts. By tagging the specific data needed by the engineering team and granting the engineering team's accounts cross-account permissions based on these tags, you achieve selective data sharing without needing to copy or move data.

Option A, copying data to a common account, introduces data duplication, increased storage costs, and the need for a separate access management system. Managing data synchronization and ensuring data consistency become added burdens.

Option B, granting permissions directly to individual users, is not scalable and creates a management nightmare as the engineering team grows or data access requirements change. Modifying permissions for each user across multiple accounts is operationally expensive.

Option C, using AWS Data Exchange, is more suitable for sharing data with external parties. It incurs higher

costs than native Lake Formation features and is not optimized for internal data sharing within an organization. The overhead associated with data exchange, even privately, is significantly greater.

Tag-based access control allows you to centrally define who has access to what data based on the applied tags. This significantly reduces the overhead associated with managing permissions for individual users or roles across multiple accounts. Because Lake Formation manages the underlying access controls, the operation is optimized for the AWS environment.

For further research, refer to the AWS documentation on Lake Formation's tag-based access control: https://docs.aws.amazon.com/lake-formation/latest/dg/tag-based-access-control.html and cross-account access: https://docs.aws.amazon.com/lake-formation/latest/dg/cross-account-permissions.html.

Question: 443 **Exam Heist**

A company wants to host a scalable web application on AWS. The application will be accessed by users from different geographic regions of the world. Application users will be able to download and upload unique data up to gigabytes in size. The development team wants a cost-effective solution to minimize upload and download latency and maximize performance.

What should a solutions architect do to accomplish this?

A.Use Amazon S3 with Transfer Acceleration to host the application.
B.Use Amazon S3 with CacheControl headers to host the application.
C.Use Amazon EC2 with Auto Scaling and Amazon CloudFront to host the application.
D.Use Amazon EC2 with Auto Scaling and Amazon ElastiCache to host the application.

Answer: A

Explanation:

The correct answer is **A. Use Amazon S3 with Transfer Acceleration to host the application.**

Here's why:

S3 for Object Storage: Amazon S3 (Simple Storage Service) is designed for scalable, durable, and cost-effective object storage. It's ideal for storing and serving large files like those being uploaded and downloaded by the application's users. https://aws.amazon.com/s3/

Transfer Acceleration for Speed: S3 Transfer Acceleration leverages the globally distributed AWS edge locations (same infrastructure as CloudFront) to accelerate data transfers to and from S3. When a user uploads or downloads data, the connection is routed to the nearest edge location, which then uses optimized network paths to transfer the data to or from the S3 bucket. This significantly reduces latency for users in different geographic regions. https://aws.amazon.com/s3/transfer-acceleration/

Cost-Effectiveness: While Transfer Acceleration incurs some additional cost, it is generally a cost-effective way to improve transfer speeds, especially for geographically dispersed users and large files. The increased performance can outweigh the added expense.

Let's analyze why the other options are less suitable:

B. Use Amazon S3 with CacheControl headers to host the application. While CacheControl headers are useful for controlling how browsers and CDNs cache content, they don't inherently accelerate uploads. They primarily optimize download speeds for content that's already stored in S3 and cached.

C. Use Amazon EC2 with Auto Scaling and Amazon CloudFront to host the application. EC2 with Auto

Scaling would be suitable for the application's compute layer (e.g., web servers, application servers). CloudFront would be excellent for caching and distributing static content, but it doesn't directly accelerate uploads. Also, using EC2 to store multi-gigabyte user data is not the most cost-effective approach. S3 is better suited for large object storage.

D. Use Amazon EC2 with Auto Scaling and Amazon ElastiCache to host the application. ElastiCache is an in-memory data caching service. It is great for accelerating reads of frequently accessed data, but it doesn't help with large file uploads or the global distribution of content in the way that Transfer Acceleration does. Like option C, using EC2 as the primary storage for multi-gigabyte user data is not ideal.

Question: 444 **Exam Heist**

A company has hired a solutions architect to design a reliable architecture for its application. The application consists of one Amazon RDS DB instance and two manually provisioned Amazon EC2 instances that run web servers. The EC2 instances are located in a single Availability Zone.

An employee recently deleted the DB instance, and the application was unavailable for 24 hours as a result. The company is concerned with the overall reliability of its environment.

What should the solutions architect do to maximize reliability of the application's infrastructure?

A.Delete one EC2 instance and enable termination protection on the other EC2 instance. Update the DB instance to be Multi-AZ, and enable deletion protection.

B.Update the DB instance to be Multi-AZ, and enable deletion protection. Place the EC2 instances behind an Application Load Balancer, and run them in an EC2 Auto Scaling group across multiple Availability Zones.

C.Create an additional DB instance along with an Amazon API Gateway and an AWS Lambda function. Configure the application to invoke the Lambda function through API Gateway. Have the Lambda function write the data to the two DB instances.

D.Place the EC2 instances in an EC2 Auto Scaling group that has multiple subnets located in multiple Availability Zones. Use Spot Instances instead of On-Demand Instances. Set up Amazon CloudWatch alarms to monitor the health of the instances Update the DB instance to be Multi-AZ, and enable deletion protection.

Answer: B

Explanation:

Here's a detailed justification for why option B is the best solution to maximize reliability:

The Problem: The original architecture has several single points of failure: a single RDS instance and EC2 instances within a single Availability Zone. A manual deletion of the RDS instance caused a significant outage, highlighting the need for redundancy and automated recovery mechanisms.

Why Option B is the Best Solution:

Multi-AZ RDS: Upgrading the RDS instance to Multi-AZ creates a synchronous standby replica in a different Availability Zone. If the primary instance fails, the service automatically fails over to the standby, minimizing downtime. https://aws.amazon.com/rds/features/multi-az/
Deletion Protection: Enabling deletion protection on the RDS instance prevents accidental deletion, addressing the root cause of the initial outage.
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/UsingWithRDS.Overview.html
Application Load Balancer (ALB): An ALB distributes incoming traffic across multiple EC2 instances, improving availability and scalability. It also performs health checks and automatically routes traffic away from unhealthy instances. https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
EC2 Auto Scaling Group (ASG) across multiple AZs: Placing the EC2 instances in an ASG spanning multiple Availability Zones ensures that if one AZ experiences an outage, the application remains available in other AZs. The ASG automatically replaces unhealthy instances, maintaining the desired capacity.

https://aws.amazon.com/autoscaling/

Why Other Options are Less Suitable:

Option A: Deleting one EC2 instance reduces capacity and does not address Availability Zone failures. Termination protection only protects a single instance from termination, and while multi-AZ RDS helps, this option doesn't scale or balance load.
Option C: Introducing API Gateway and Lambda adds complexity and potential points of failure without necessarily improving the core reliability of the web server tier. Writing to two DB instances manually through Lambda can introduce data consistency issues and is not a standard high-availability configuration for RDS.
Option D: While using an ASG across multiple AZs and multi-AZ RDS are good ideas, using Spot Instances can lead to interruptions if the Spot price increases, potentially reducing reliability. Also, while CloudWatch alarms are helpful for monitoring, they don't automatically provide fault tolerance like a Multi-AZ setup and an ALB. CloudWatch alarms would only notify about a problem after it already occurred.

In summary, option B provides a comprehensive approach to maximize the application's reliability by addressing both data and compute layer redundancy, automating recovery, and preventing accidental data loss, making it the optimal solution.

Question: 445 **Exam Heist**

A company is storing 700 terabytes of data on a large network-attached storage (NAS) system in its corporate data center. The company has a hybrid environment with a 10 Gbps AWS Direct Connect connection.

After an audit from a regulator, the company has 90 days to move the data to the cloud. The company needs to move the data efficiently and without disruption. The company still needs to be able to access and update the data during the transfer window.

Which solution will meet these requirements?

A.Create an AWS DataSync agent in the corporate data center. Create a data transfer task Start the transfer to an Amazon S3 bucket.

B.Back up the data to AWS Snowball Edge Storage Optimized devices. Ship the devices to an AWS data center. Mount a target Amazon S3 bucket on the on-premises file system.

C.Use rsync to copy the data directly from local storage to a designated Amazon S3 bucket over the Direct Connect connection.

D.Back up the data on tapes. Ship the tapes to an AWS data center. Mount a target Amazon S3 bucket on the on-premises file system.

Answer: A

Explanation:

The most suitable solution is A: Using AWS DataSync to transfer the data to Amazon S3. Here's why:

Efficient Transfer: AWS DataSync is specifically designed for online data transfer between on-premises storage and AWS services. It uses a purpose-built protocol and parallel data streams to accelerate the transfer process, maximizing the 10 Gbps Direct Connect link.
No Disruption: DataSync supports incremental transfers. Only the changed data is copied after the initial transfer, minimizing the impact on ongoing operations and allowing continuous access and updates during the 90-day window.
Large Dataset Handling: DataSync is designed to handle large datasets, making it a good fit for 700 TB.
S3 Integration: The data can be directly transferred into an Amazon S3 bucket, making it readily available for cloud-based applications and analysis.

Let's examine why other options are less suitable:

B (Snowball Edge): While Snowball Edge is useful for transferring large datasets, it involves physical shipping, which is not the most efficient approach given the Direct Connect link and the 90-day deadline. More importantly, Option B includes mounting the S3 bucket on-premises, which is not how S3 works; it's not a mountable filesystem.
C (rsync): rsync can transfer data, but it lacks the advanced features of DataSync, like built-in data verification, encryption, bandwidth management, and optimized transfer protocols. It would likely be slower and require more manual configuration.
D (Tapes): Transferring data via tapes is an outdated approach. It is extremely slow and impractical considering the availability of a 10 Gbps Direct Connect link.

Therefore, DataSync provides the optimal balance of speed, efficiency, and minimal disruption for transferring the 700 TB dataset within the given timeframe.

Supporting Documentation:

AWS DataSync: https://aws.amazon.com/datasync/

Question: 446 **Exam Heist**

A company stores data in PDF format in an Amazon S3 bucket. The company must follow a legal requirement to retain all new and existing data in Amazon S3 for 7 years.

Which solution will meet these requirements with the LEAST operational overhead?

A.Turn on the S3 Versioning feature for the S3 bucket. Configure S3 Lifecycle to delete the data after 7 years. Configure multi-factor authentication (MFA) delete for all S3 objects.

B.Turn on S3 Object Lock with governance retention mode for the S3 bucket. Set the retention period to expire after 7 years. Recopy all existing objects to bring the existing data into compliance.

C.Turn on S3 Object Lock with compliance retention mode for the S3 bucket. Set the retention period to expire after 7 years. Recopy all existing objects to bring the existing data into compliance.

D.Turn on S3 Object Lock with compliance retention mode for the S3 bucket. Set the retention period to expire after 7 years. Use S3 Batch Operations to bring the existing data into compliance.

Answer: D

Explanation:

Here's a detailed justification for why option D is the best solution for meeting the company's data retention requirement with the least operational overhead:

The core requirement is to retain all data (new and existing) in Amazon S3 for 7 years while adhering to legal mandates. This necessitates immutability, preventing accidental or intentional deletion or modification during the retention period.

S3 Object Lock: This is the ideal feature for implementing immutability. It prevents objects from being deleted or overwritten for a specified retention period or until a specific date.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html

Compliance Retention Mode: This mode offers the strongest protection, as it prevents even users with administrative privileges from deleting or shortening the retention period. Governance mode allows privileged users to modify the retention settings, which could violate the legal requirement.

7-Year Retention Period: Setting the retention period to 7 years aligns directly with the legal retention requirement.

Addressing Existing Data: Since the retention requirement applies to both new and existing data, all objects in the S3 bucket must be protected by Object Lock. Simply enabling Object Lock won't retroactively protect existing objects.

S3 Batch Operations: This feature provides a highly efficient way to apply Object Lock configurations to the existing data in the S3 bucket. Instead of manually recopying objects (as in options B and C), S3 Batch Operations automates the process, minimizing operational overhead. S3 Batch Operations allows you to perform large-scale actions on S3 objects. https://docs.aws.amazon.com/AmazonS3/latest/userguide/batch-ops-intro.html

Let's examine why the other options are less suitable:

Option A: S3 Versioning is useful for recovering from accidental deletions or overwrites, but it does not prevent deletion if proper permissions are present. Configuring MFA Delete adds an extra layer of security, but an authorized user with MFA can still delete objects. S3 Lifecycle policies would eventually delete all data after 7 years, but it doesn't guarantee immutability during those 7 years. Thus, Option A fails the immutability requirement.

Options B and C: While they correctly use S3 Object Lock and the 7-year retention period, they suggest recopying all existing objects to bring them into compliance. This is a manual and operationally expensive approach compared to using S3 Batch Operations. Furthermore, using governance mode (Option B) does not fully protect from deletion if a privileged user decides to override the settings.

Question: 447 **Exam Heist**

A company has a stateless web application that runs on AWS Lambda functions that are invoked by Amazon API Gateway. The company wants to deploy the application across multiple AWS Regions to provide Regional failover capabilities.

What should a solutions architect do to route traffic to multiple Regions?

A.Create Amazon Route 53 health checks for each Region. Use an active-active failover configuration.

B.Create an Amazon CloudFront distribution with an origin for each Region. Use CloudFront health checks to route traffic.

C.Create a transit gateway. Attach the transit gateway to the API Gateway endpoint in each Region. Configure the transit gateway to route requests.

D.Create an Application Load Balancer in the primary Region. Set the target group to point to the API Gateway endpoint hostnames in each Region.

Answer: B

Explanation:

The correct answer is **B. Create an Amazon CloudFront distribution with an origin for each Region. Use CloudFront health checks to route traffic.**

Here's why:

Regional Failover Requirement: The prompt emphasizes deploying across multiple AWS Regions for Regional failover. This means that if one Region experiences an outage, traffic should automatically be routed to a healthy Region.

CloudFront for Global Content Delivery and Routing: Amazon CloudFront is a content delivery network (CDN) that caches content closer to users. Crucially, it also offers origin failover capabilities based on health checks.

Origin Groups and Health Checks: By creating a CloudFront distribution with an origin group consisting of the

API Gateway endpoints in each Region, you can leverage CloudFront's health checks. If CloudFront detects that an origin (a specific Region's API Gateway) is unhealthy, it automatically routes traffic to a healthy origin in another Region. This provides seamless failover.

Stateless Application Compatibility: The web application is stateless, which means requests can be served from any Region without causing data consistency issues. This perfectly aligns with CloudFront's ability to route requests to different origins based on health.

Why other options are incorrect:

A (Route 53 active-active): While Route 53 can do active-active failover, it's primarily DNS-based. The failure detection and switchover are slower than CloudFront. Also, it would introduce complexity in managing DNS records and may not be as effective for API-driven failover.
C (Transit Gateway): Transit Gateway is for connecting VPCs and on-premises networks. It's overkill for routing traffic to API Gateway endpoints for regional failover. It also doesn't offer the built-in health check and failover capabilities suitable for this scenario.
D (Application Load Balancer): An ALB is regional. While you can create an ALB, the single ALB wouldn't provide multi-region failover. ALB's are designed to distribute traffic within the same region, not across multiple regions. The "target group" should be in one region.

In summary, CloudFront provides the most efficient and reliable solution by natively supporting origin failover with health checks, aligning perfectly with the need for Regional failover and the stateless nature of the web application.

Relevant Links:

Amazon CloudFront Origin Failover
Amazon CloudFront Health Checks

Question: 448 **Exam Heist**

A company has two VPCs named Management and Production. The Management VPC uses VPNs through a customer gateway to connect to a single device in the data center. The Production VPC uses a virtual private gateway with two attached AWS Direct Connect connections. The Management and Production VPCs both use a single VPC peering connection to allow communication between the applications.

What should a solutions architect do to mitigate any single point of failure in this architecture?

A.Add a set of VPNs between the Management and Production VPCs.
B.Add a second virtual private gateway and attach it to the Management VPC.
C.Add a second set of VPNs to the Management VPC from a second customer gateway device.
D.Add a second VPC peering connection between the Management VPC and the Production VPC.

Answer: C

Explanation:

The most critical single point of failure in the described architecture lies in the Management VPC's connectivity to the data center via a single VPN connection and customer gateway. If this single VPN or gateway fails, the Management VPC loses its connection to the data center, impacting critical functions potentially dependent on the data center resources.

Option C, adding a second set of VPNs to the Management VPC from a second customer gateway device, addresses this single point of failure directly. By establishing a redundant VPN connection through a separate customer gateway, the solution ensures that if one VPN or customer gateway fails, the other will maintain

connectivity, preventing disruption to the Management VPC's operations. This establishes redundancy and high availability for the data center connection.

Option A doesn't address the single point of failure in the Management VPC's data center connection. It only adds connectivity between the Management and Production VPCs, not to the data center.

Option B is incorrect. A second virtual private gateway attached to the Management VPC doesn't solve the single point of failure, as both gateways would still rely on the single VPN and customer gateway device for data center connectivity.

Option D, adding a second VPC peering connection, only increases the bandwidth and resilience of communication between the two VPCs but does not address the vulnerability of the single VPN connection from the Management VPC to the data center. VPC peering is irrelevant to the data center connectivity problem.

In summary, option C provides the necessary redundancy at the customer gateway level, mitigating the risk of losing connectivity between the Management VPC and the on-premises data center. This aligns with best practices for high availability in hybrid cloud environments.

Further reading on VPN gateway redundancy and high availability can be found in the AWS documentation:

AWS VPN Connections
AWS Site-to-Site VPN

Question: 449 **Exam Heist**

A company runs its application on an Oracle database. The company plans to quickly migrate to AWS because of limited resources for the database, backup administration, and data center maintenance. The application uses third-party database features that require privileged access.

Which solution will help the company migrate the database to AWS MOST cost-effectively?

A.Migrate the database to Amazon RDS for Oracle. Replace third-party features with cloud services.

B.Migrate the database to Amazon RDS Custom for Oracle. Customize the database settings to support third-party features.

C.Migrate the database to an Amazon EC2 Amazon Machine Image (AMI) for Oracle. Customize the database settings to support third-party features.

D.Migrate the database to Amazon RDS for PostgreSQL by rewriting the application code to remove dependency on Oracle APEX.

Answer: B

Explanation:

The most cost-effective solution for migrating the Oracle database to AWS while maintaining third-party database feature support and addressing resource constraints is Amazon RDS Custom for Oracle.

Here's a detailed justification:

RDS Custom for Oracle: This service offers managed database infrastructure similar to standard RDS, but with the added capability to customize the underlying operating system and database settings. This is critical for supporting third-party database features requiring privileged access that are not available in standard RDS. https://aws.amazon.com/rds/custom/

Cost-Effectiveness: RDS Custom provides a balance between managed services and control. Migrating to RDS Custom avoids the overhead of fully self-managing an Oracle database on EC2, reducing administrative burden and operational costs associated with patching, backups, and infrastructure management.

Privileged Access: The key requirement is that the third-party features require privileged access, which is not available in the normal RDS. RDS Custom provides OS and DB access.

Option A (RDS for Oracle): Standard RDS for Oracle does not allow privileged access or customization needed for third-party features, thus not meeting the requirements. It would require the third-party applications to be re-written.

Option C (EC2 AMI for Oracle): While offering full control, running Oracle on EC2 necessitates significant administrative overhead for database management, patching, backups, and scaling. This contradicts the need for reduced administrative burden and might not be the most cost-effective approach, considering the other managed service offerings.

Option D (RDS for PostgreSQL): Rewriting the application code to remove Oracle APEX dependency would be a time-consuming and costly undertaking and is not the question is asking. Migration should be done quickly.

Therefore, migrating to Amazon RDS Custom for Oracle is the most cost-effective solution that maintains third-party database feature support while benefiting from managed database services, addressing the company's limited resources.

Question: 450 **Exam Heist**

A company has a three-tier web application that is in a single server. The company wants to migrate the application to the AWS Cloud. The company also wants the application to align with the AWS Well-Architected Framework and to be consistent with AWS recommended best practices for security, scalability, and resiliency.

Which combination of solutions will meet these requirements? (Choose three.)

A.Create a VPC across two Availability Zones with the application's existing architecture. Host the application with existing architecture on an Amazon EC2 instance in a private subnet in each Availability Zone with EC2 Auto Scaling groups. Secure the EC2 instance with security groups and network access control lists (network ACLs).

B.Set up security groups and network access control lists (network ACLs) to control access to the database layer. Set up a single Amazon RDS database in a private subnet.

C.Create a VPC across two Availability Zones. Refactor the application to host the web tier, application tier, and database tier. Host each tier on its own private subnet with Auto Scaling groups for the web tier and application tier.

D.Use a single Amazon RDS database. Allow database access only from the application tier security group.

E.Use Elastic Load Balancers in front of the web tier. Control access by using security groups containing references to each layer's security groups.

F.Use an Amazon RDS database Multi-AZ cluster deployment in private subnets. Allow database access only from application tier security groups.

Answer: CEF

Explanation:

Here's a detailed justification for why options C, E, and F are the most suitable for migrating the three-tier web application to AWS, aligning with the Well-Architected Framework, and ensuring security, scalability, and resiliency:

E. Use Elastic Load Balancers in front of the web tier. Control access by using security groups containing references to each layer's security groups.

ELBs are essential for distributing incoming traffic across multiple EC2 instances in the web tier, providing high availability and fault tolerance. This directly addresses scalability and resilience.

Security groups at the web tier should only allow traffic from the ELB, and the ELB acts as the single point of entry, minimizing the attack surface.
Security group references enable a layered security approach, controlling traffic flow between tiers and enhancing security posture.

Authoritative link: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html

F. Use an Amazon RDS database Multi-AZ cluster deployment in private subnets. Allow database access only from application tier security groups.

Multi-AZ RDS deployment ensures high availability and failover capabilities for the database tier. If the primary database instance fails, RDS automatically fails over to a standby instance in another Availability Zone.
Placing the database in private subnets restricts direct internet access and protects sensitive data.
Security group rules should be configured so that only EC2 instances in the application tier can access the database. This minimizes potential access points and mitigates risks.

Authoritative link: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html

C. Create a VPC across two Availability Zones. Refactor the application to host the web tier, application tier, and database tier. Host each tier on its own private subnet with Auto Scaling groups for the web tier and application tier.

A VPC across two Availability Zones provides infrastructure isolation and fault tolerance. If one Availability Zone experiences an issue, the application can continue running in the other.
Refactoring the application into three tiers promotes modularity and easier management. Each tier can be scaled and secured independently.
Hosting each tier in its own private subnet improves security by isolating each tier from direct internet access. The web tier typically resides in a public subnet for internet-facing access.
Auto Scaling groups enable automatic scaling of the web and application tiers based on demand, ensuring application performance and resilience.

Authoritative link: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Introduction.html

Why the other options are incorrect:

A: While creating a VPC and using Auto Scaling is helpful, maintaining the existing monolithic architecture on EC2 doesn't provide the full benefits of the cloud in terms of scalability and modularity. It doesn't fully leverage AWS best practices.

B: Setting up security groups and using a single RDS instance is a basic improvement, but it doesn't address high availability and fault tolerance for the database tier, which is a critical aspect of the Well-Architected Framework.

Question: 451 **Exam Heist**

A company is migrating its applications and databases to the AWS Cloud. The company will use Amazon Elastic Container Service (Amazon ECS), AWS Direct Connect, and Amazon RDS.

Which activities will be managed by the company's operational team? (Choose three.)

A.Management of the Amazon RDS infrastructure layer, operating system, and platforms

B.Creation of an Amazon RDS DB instance and configuring the scheduled maintenance window

C.Configuration of additional software components on Amazon ECS for monitoring, patch management, log management, and host intrusion detection

D.Installation of patches for all minor and major database versions for Amazon RDS

E.Ensure the physical security of the Amazon RDS infrastructure in the data center

F.Encryption of the data that moves in transit through Direct Connect

Answer: BCF

Explanation:

The correct answer is BCF. Here's why:

B. Creation of an Amazon RDS DB instance and configuring the scheduled maintenance window: The company is responsible for the configuration and operation of their RDS DB instances, including tasks such as creating the instance, setting parameters, and scheduling maintenance windows. Amazon handles the underlying infrastructure, but the configurations are managed by the user.
https://docs.aws.amazon.com/rds/latest/userguide/USER_CreateDBInstance.html

C. Configuration of additional software components on Amazon ECS for monitoring, patch management, log management, and host intrusion detection: While ECS manages container orchestration, responsibilities like monitoring the containers and host, patching software within the containers, implementing log management solutions, and setting up host intrusion detection systems fall under the company's operational control. AWS provides tools and services (like CloudWatch, Systems Manager, and security agents), but the user configures and manages them within their ECS environment.
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/monitoring-cloudwatch.html

F. Encryption of the data that moves in transit through Direct Connect: While Direct Connect provides a dedicated network connection, securing data in transit is the customer's responsibility. The company must implement encryption mechanisms (e.g., VPN over Direct Connect, or application-level encryption) to protect sensitive data moving between their on-premises environment and AWS. AWS handles the physical connection, but the security protocols are implemented and managed by the customer.
https://aws.amazon.com/directconnect/security/

Here's why the other options are incorrect:

A. Management of the Amazon RDS infrastructure layer, operating system, and platforms: AWS manages the underlying infrastructure, operating system, and platform for RDS under the shared responsibility model.

D. Installation of patches for all minor and major database versions for Amazon RDS: AWS handles the patching of the RDS database engine itself. While the customer schedules when these updates are applied within the maintenance window they configure, the actual patching is performed by AWS.

E. Ensure the physical security of the Amazon RDS infrastructure in the data center: AWS is responsible for the physical security of the data centers where the RDS infrastructure resides.

Question: 452 **Exam Heist**

A company runs a Java-based job on an Amazon EC2 instance. The job runs every hour and takes 10 seconds to run. The job runs on a scheduled interval and consumes 1 GB of memory. The CPU utilization of the instance is low except for short surges during which the job uses the maximum CPU available. The company wants to optimize the costs to run the job.

Which solution will meet these requirements?

A.Use AWS App2Container (A2C) to containerize the job. Run the job as an Amazon Elastic Container Service (Amazon ECS) task on AWS Fargate with 0.5 virtual CPU (vCPU) and 1 GB of memory.

B.Copy the code into an AWS Lambda function that has 1 GB of memory. Create an Amazon EventBridge

scheduled rule to run the code each hour.

C.Use AWS App2Container (A2C) to containerize the job. Install the container in the existing Amazon Machine Image (AMI). Ensure that the schedule stops the container when the task finishes.

D.Configure the existing schedule to stop the EC2 instance at the completion of the job and restart the EC2 instance when the next job starts.

Answer: B

Explanation:

The best solution is B: Copy the code into an AWS Lambda function with 1 GB of memory and create an Amazon EventBridge scheduled rule to run the code each hour. This is because Lambda is a serverless compute service that allows you to run code without provisioning or managing servers.

Here's why the other options are not as suitable:

A: While containerizing the job and running it on Fargate is a viable option, it is more complex and likely more expensive than using Lambda for such a short, infrequent task. Fargate is better suited for continuous or long-running applications. This adds complexity and cost compared to the event-driven approach.
C: Containerizing the job and installing the container in the existing AMI is overly complex and does not optimize costs. The EC2 instance still needs to be running constantly, even though the job only runs for 10 seconds each hour. It does not fully leverage serverless technologies.
D: Stopping and restarting the EC2 instance will add latency to the job and is not cost-effective because you will be charged for the EC2 instance even when it is stopped. Also, stopping/starting an EC2 instance takes a few minutes, exceeding the 10-second runtime.

Why Lambda and EventBridge are Ideal:

Cost Optimization: Lambda's pricing model is based on the actual compute time your function consumes. Since the job only runs for 10 seconds per hour, the cost will be minimal. You are not paying for idle time.
Scalability: Lambda automatically scales to handle the workload. If the job becomes more frequent or requires more resources in the future, Lambda can handle the increased load.
Simplicity: Lambda functions are easy to deploy and manage. You can easily copy the code into a Lambda function and configure the required memory and timeout.
Event-Driven: Amazon EventBridge is a serverless event bus that allows you to schedule events to trigger Lambda functions. This provides a simple and reliable way to run the job every hour. It is directly integrated with Lambda, making configuration straightforward.
Suitable for short, intermittent workloads: Lambda is specifically designed for tasks that run for a short duration and are triggered periodically, aligning perfectly with the question's requirements.

Lambda's execution model and cost structure are perfect for this use case.

Authoritative Links:

AWS Lambda: https://aws.amazon.com/lambda/
Amazon EventBridge: https://aws.amazon.com/eventbridge/

Question: 453 **Exam Heist**

A company wants to implement a backup strategy for Amazon EC2 data and multiple Amazon S3 buckets. Because of regulatory requirements, the company must retain backup files for a specific time period. The company must not alter the files for the duration of the retention period.

Which solution will meet these requirements?

A.Use AWS Backup to create a backup vault that has a vault lock in governance mode. Create the required

backup plan.

B.Use Amazon Data Lifecycle Manager to create the required automated snapshot policy.

C.Use Amazon S3 File Gateway to create the backup. Configure the appropriate S3 Lifecycle management.

D.Use AWS Backup to create a backup vault that has a vault lock in compliance mode. Create the required backup plan.

Answer: D

Explanation:

The correct answer is D because it leverages AWS Backup with a compliance mode vault lock, fulfilling the requirements for data immutability and retention.

AWS Backup is a centralized backup service that makes it easy to automate and manage backups across various AWS services, including EC2 and S3. The key requirement is immutable backups with a fixed retention period.

AWS Backup Vault Lock is a feature that helps you enforce a write-once-read-many (WORM) model on your backups. This ensures that backups cannot be deleted or altered during a specified retention period, adhering to compliance and regulatory needs. There are two modes for Vault Lock: Governance and Compliance.

Governance mode allows privileged users to delete the lock before the retention period expires under certain conditions. This doesn't fully meet the 'must not alter the files' requirement.

Compliance mode, however, is more restrictive. Once locked in compliance mode, even privileged users cannot reduce the retention period or delete the lock. This ensures immutability and adherence to retention policies.

Therefore, using AWS Backup with a Compliance mode Vault Lock directly fulfills the company's requirements for immutability and retention, across both EC2 and S3, making option D the most suitable solution. A backup plan defines what resources to back up, when to back up, and how long to retain backups, making it an integral component of the solution.

Option A (Governance mode) does not guarantee immutability, failing the 'must not alter the files' criterion. Option B (Amazon Data Lifecycle Manager) is primarily for EBS snapshots and doesn't directly address S3 bucket backups with retention requirements. Option C (S3 File Gateway) is designed for on-premises applications to store data in S3 and doesn't provide the enforced immutability provided by Vault Lock.

Here are some authoritative links:

AWS Backup: https://aws.amazon.com/backup/
AWS Backup Vault Lock: https://docs.aws.amazon.com/aws-backup/latest/devguide/vault-lock.html
AWS Backup Plans: https://docs.aws.amazon.com/aws-backup/latest/devguide/whatis-backup-plans.html

Question: 454 **Exam Heist**

A company has resources across multiple AWS Regions and accounts. A newly hired solutions architect discovers a previous employee did not provide details about the resources inventory. The solutions architect needs to build and map the relationship details of the various workloads across all accounts.

Which solution will meet these requirements in the MOST operationally efficient way?

A.Use AWS Systems Manager Inventory to generate a map view from the detailed view report.

B.Use AWS Step Functions to collect workload details. Build architecture diagrams of the workloads manually.

C.Use Workload Discovery on AWS to generate architecture diagrams of the workloads.

D.Use AWS X-Ray to view the workload details. Build architecture diagrams with relationships.

Answer: C

Explanation:

The best solution is C, using Workload Discovery on AWS. Here's why:

Workload Discovery on AWS (Option C) is specifically designed to automatically discover and map application components and their relationships across AWS accounts and Regions. It generates architecture diagrams, providing a visual representation of the infrastructure. This automation directly addresses the problem of undocumented resources and relationships, minimizing manual effort. It's the most operationally efficient way to map existing workloads and build architecture diagrams in a multi-account, multi-region environment.

Why other options are less suitable:

AWS Systems Manager Inventory (Option A): While Systems Manager Inventory collects metadata about managed instances, it primarily focuses on software, patches, and configuration details. It does not automatically map relationships between different AWS services or generate architecture diagrams. Generating a "map view" from this data is possible but requires significant manual configuration and scripting, making it less efficient.

AWS Step Functions (Option B): Step Functions orchestrates workflows, but it doesn't inherently discover resources or map relationships. Building architecture diagrams manually defeats the purpose of operational efficiency. This approach requires writing custom code to gather workload details and then manually create diagrams, which is time-consuming and error-prone.

AWS X-Ray (Option D): X-Ray traces requests as they move through an application, providing insights into performance bottlenecks and dependencies within an application. It's not designed to discover and map the overall infrastructure landscape or generate architecture diagrams across multiple accounts and Regions. Its focus is on application performance analysis, not infrastructure mapping.

In conclusion, Workload Discovery on AWS offers the most automated and efficient solution for discovering resources, mapping relationships, and generating architecture diagrams, directly addressing the solutions architect's need to understand undocumented workloads across a complex AWS environment.

Authoritative Links:

Workload Discovery on AWS: https://aws.amazon.com/solutions/implementations/workload-discovery-on-aws/
AWS Systems Manager Inventory: https://docs.aws.amazon.com/systems-manager/latest/userguide/inventory.html
AWS Step Functions: https://aws.amazon.com/step-functions/
AWS X-Ray: https://aws.amazon.com/xray/

Question: 455 **Exam Heist**

A company uses AWS Organizations. The company wants to operate some of its AWS accounts with different budgets. The company wants to receive alerts and automatically prevent provisioning of additional resources on AWS accounts when the allocated budget threshold is met during a specific period.

Which combination of solutions will meet these requirements? (Choose three.)

A.Use AWS Budgets to create a budget. Set the budget amount under the Cost and Usage Reports section of the required AWS accounts.

B.Use AWS Budgets to create a budget. Set the budget amount under the Billing dashboards of the required AWS accounts.

C.Create an IAM user for AWS Budgets to run budget actions with the required permissions.

D.Create an IAM role for AWS Budgets to run budget actions with the required permissions.

E.Add an alert to notify the company when each account meets its budget threshold. Add a budget action that selects the IAM identity created with the appropriate config rule to prevent provisioning of additional resources.

F.Add an alert to notify the company when each account meets its budget threshold. Add a budget action that selects the IAM identity created with the appropriate service control policy (SCP) to prevent provisioning of additional resources.

Answer: BDF

Explanation:

The correct answer is BDF because they provide a comprehensive approach to budget control and resource provisioning prevention using AWS Budgets, IAM roles, and budget actions.

B - Use AWS Budgets to create a budget. Set the budget amount under the Billing dashboards of the required AWS accounts: AWS Budgets is a service designed for cost management and budget tracking. It allows setting custom budgets for individual AWS accounts, providing a granular control over spending. The Billing dashboards are the logical location to configure these budgets. https://aws.amazon.com/aws-cost-management/aws-budgets/

D - Create an IAM role for AWS Budgets to run budget actions with the required permissions: AWS Budgets requires specific permissions to take actions when a budget threshold is met. Using an IAM role is the recommended security best practice because it allows granting only the necessary permissions to the AWS Budgets service to perform actions without using long-term credentials. This ensures least privilege access. https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-actions.html

F - Add an alert to notify the company when each account meets its budget threshold. Add a budget action that selects the IAM identity created with the appropriate service control policy (SCP) to prevent provisioning of additional resources: This step involves setting up notifications through AWS Budgets when the threshold is reached. Crucially, it also creates a budget action. SCPs are a powerful feature within AWS Organizations that enable central control over the AWS accounts in an organization. By linking a budget action to an IAM identity, and defining an appropriate SCP, provisioning of additional resources can be blocked once the budget threshold is reached.
https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html

A is incorrect because the Cost and Usage Reports section is primarily for generating detailed reports about your AWS costs and usage, and not for directly setting budget amounts.

C is incorrect because creating an IAM user for AWS Budgets to run budget actions is not a recommended security best practice. IAM roles are preferred over IAM users in this scenario. Using a user requires managing long-term credentials which adds unnecessary risk.

E is incorrect because config rules are not a mechanism to prevent provisioning of additional resources, and they do not directly integrate with budget actions to stop resource provisioning. SCP's, not config rules, are used within Organizations to control what services and actions are allowed within member accounts.

Question: 456 **Exam Heist**

A company runs applications on Amazon EC2 instances in one AWS Region. The company wants to back up the EC2 instances to a second Region. The company also wants to provision EC2 resources in the second Region and manage the EC2 instances centrally from one AWS account.

Which solution will meet these requirements MOST cost-effectively?

A.Create a disaster recovery (DR) plan that has a similar number of EC2 instances in the second Region. Configure data replication.

B.Create point-in-time Amazon Elastic Block Store (Amazon EBS) snapshots of the EC2 instances. Copy the snapshots to the second Region periodically.

C.Create a backup plan by using AWS Backup. Configure cross-Region backup to the second Region for the EC2 instances.

D.Deploy a similar number of EC2 instances in the second Region. Use AWS DataSync to transfer the data from the source Region to the second Region.

Answer: C

Explanation:

The most cost-effective solution is **C. Create a backup plan by using AWS Backup. Configure cross-Region backup to the second Region for the EC2 instances.**

Here's why:

AWS Backup: AWS Backup is a fully managed backup service that centralizes and automates data protection across AWS services, including EC2. This addresses the requirement of central management from one AWS account.
Cost-Effectiveness: AWS Backup provides a cost-optimized solution for creating and managing backups. You only pay for the backup storage and the data transferred during backup and restore operations. Compared to running a separate replication solution or maintaining a full DR environment, this is generally more economical.
Cross-Region Backup: AWS Backup supports cross-Region backup, allowing you to copy your EC2 instance backups (including EBS volumes) to a second AWS Region for disaster recovery purposes. This meets the requirement of backing up to a second Region.
Recovery Point Objective (RPO) and Recovery Time Objective (RTO): You can configure backup frequency (RPO) and restore capabilities to meet your business requirements. The AWS Backup console simplifies the recovery process.

Let's analyze why the other options are less suitable:

A. Create a disaster recovery (DR) plan that has a similar number of EC2 instances in the second Region. Configure data replication. This option involves maintaining a hot or warm DR site with provisioned EC2 instances, which is the most expensive approach. While it provides a low RTO, it incurs significant costs for idle resources.
B. Create point-in-time Amazon Elastic Block Store (Amazon EBS) snapshots of the EC2 instances. Copy the snapshots to the second Region periodically. While EBS snapshots can be used for backup, this approach requires manual scripting and management of snapshot copies across Regions. It lacks the centralized management and automation provided by AWS Backup. It's also more complex to restore and maintain consistency.
D. Deploy a similar number of EC2 instances in the second Region. Use AWS DataSync to transfer the data from the source Region to the second Region. Similar to option A, this approach involves running EC2 instances continuously in the second Region, making it costly. AWS DataSync is primarily for migrating data, not continuous backup.

Authoritative Links:

AWS Backup: https://aws.amazon.com/backup/
Cross-Region Backup with AWS Backup: https://docs.aws.amazon.com/aws-backup/latest/devguide/cross-region-backup.html

Question: 457 **Exam Heist**

A company that uses AWS is building an application to transfer data to a product manufacturer. The company has its own identity provider (IdP). The company wants the IdP to authenticate application users while the users use the application to transfer data. The company must use Applicability Statement 2 (AS2) protocol.

Which solution will meet these requirements?

A.Use AWS DataSync to transfer the data. Create an AWS Lambda function for IdP authentication.

B.Use Amazon AppFlow flows to transfer the data. Create an Amazon Elastic Container Service (Amazon ECS) task for IdP authentication.

C.Use AWS Transfer Family to transfer the data. Create an AWS Lambda function for IdP authentication.

D.Use AWS Storage Gateway to transfer the data. Create an Amazon Cognito identity pool for IdP authentication.

Answer: C

Explanation:

The correct answer is C. Here's why:

Why AWS Transfer Family?

AWS Transfer Family is the correct choice because it is a fully managed service that directly supports the AS2 protocol, which is a core requirement of the question. AS2 is a secure and widely adopted protocol for EDI (Electronic Data Interchange) over the internet, frequently used for B2B data transfer. AWS Transfer Family simplifies the setup and management of AS2 file transfers.

Why AWS Lambda for IdP Authentication?

AWS Lambda is a serverless compute service that allows you to run code without provisioning or managing servers. In this scenario, a Lambda function can be used to integrate with the company's existing identity provider (IdP). The Lambda function would be responsible for authenticating users based on the IdP's authentication mechanism (e.g., SAML, OAuth) before granting them access to transfer data. Lambda functions provide the flexibility needed to connect to any custom or third-party IdP.

Why other options are incorrect:

A (AWS DataSync): AWS DataSync is designed for moving large amounts of data between on-premises storage and AWS storage services. It does not inherently support AS2 or integration with external IdPs for user-level authentication within the application.

B (Amazon AppFlow): Amazon AppFlow is a fully managed integration service that enables you to securely transfer data between SaaS applications and AWS services. While AppFlow can handle data transfer, it doesn't natively support AS2 and the need for custom IdP integration is better served by Lambda.

D (AWS Storage Gateway, Amazon Cognito): AWS Storage Gateway connects on-premises applications to AWS cloud storage. While useful for hybrid cloud setups, it is not directly involved in data transfer to manufacturers or the application's user authentication flow. While Amazon Cognito is for Identity and Access Management, using Cognito identity pools in this scenario does not fit because the company already has an existing IdP and wants to leverage that for authentication. Cognito identity pools are typically used for scenarios where you want to authenticate users directly against AWS or a social identity provider, rather than integrating with an existing corporate IdP and also doesn't solve the AS2 transfer protocol requirement.

In summary, AWS Transfer Family addresses the AS2 protocol requirement directly, and AWS Lambda allows flexible integration with the company's existing IdP for user authentication.

Supporting links:

AWS Transfer Family: https://aws.amazon.com/transfer/
AWS Lambda: https://aws.amazon.com/lambda/

Question: 458 **Exam Heist**

A solutions architect is designing a RESTAPI in Amazon API Gateway for a cash payback service. The application requires 1 GB of memory and 2 GB of storage for its computation resources. The application will require that the data is in a relational format.

Which additional combination ofAWS services will meet these requirements with the LEAST administrative effort? (Choose two.)

A.Amazon EC2
B.AWS Lambda
C.Amazon RDS
D.Amazon DynamoDB
E.Amazon Elastic Kubernetes Services (Amazon EKS)

Answer: BC

Explanation:

Here's a detailed justification for why the correct answer is B (AWS Lambda) and C (Amazon RDS), along with explanations of why the other options are less suitable, and authoritative links for further reading.

The problem states that a REST API needs to be designed for a cash payback service requiring 1 GB of memory, 2 GB of storage, and a relational database. The goal is to choose the combination of AWS services that meets these requirements with the least administrative overhead.

B. AWS Lambda: AWS Lambda is a serverless compute service. It allows you to run code without provisioning or managing servers. Lambda functions can be configured with up to 10 GB of memory and execution times up to 15 minutes. The serverless nature drastically reduces administrative overhead compared to managing EC2 instances. Using Lambda for the API's compute element aligns with the principle of minimizing operational burden. It is a suitable option for the required memory allocation.

C. Amazon RDS: Amazon Relational Database Service (RDS) is a managed database service that simplifies setting up, operating, and scaling a relational database in the cloud. RDS supports various database engines (MySQL, PostgreSQL, MariaDB, Oracle, SQL Server). Since the requirement specifically mentions a relational database format, RDS is the most logical and efficient choice. RDS offers automatic backups, patching, and scaling options, which significantly reduces the administrative overhead compared to self-managing a database on EC2.

Why other options are incorrect:

A. Amazon EC2: While EC2 instances can certainly meet the memory and storage requirements, managing them involves significantly more administrative overhead. You'd need to provision, configure, patch, and scale the instances yourself. Also, you'd need to setup the relational database yourself on this EC2 instance adding more to the administrative overhead, going against the goal of "least administrative effort".
D. Amazon DynamoDB: DynamoDB is a NoSQL database service. The problem states that the data needs to be in a relational format. DynamoDB does not satisfy this fundamental requirement.
E. Amazon Elastic Kubernetes Service (Amazon EKS): EKS is a container orchestration service used for managing and deploying containerized applications. While you could use EKS to run a containerized application meeting the resource requirements, it introduces considerably more complexity and administrative overhead compared to Lambda. EKS requires managing the Kubernetes cluster itself, including nodes, networking, and scaling. It's overkill for this specific scenario focused on minimal administration.

In summary, Lambda handles the compute workload serverlessly, and RDS provides a managed relational database. This combination minimizes administrative tasks and aligns with the problem's objectives.

Authoritative Links:

AWS Lambda: https://aws.amazon.com/lambda/
Amazon RDS: https://aws.amazon.com/rds/

Question: 459 **Exam Heist**

A company uses AWS Organizations to run workloads within multiple AWS accounts. A tagging policy adds department tags to AWS resources when the company creates tags.

An accounting team needs to determine spending on Amazon EC2 consumption. The accounting team must determine which departments are responsible for the costs regardless ofAWS account. The accounting team has access to AWS Cost Explorer for all AWS accounts within the organization and needs to access all reports from Cost Explorer.

Which solution meets these requirements in the MOST operationally efficient way?

A.From the Organizations management account billing console, activate a user-defined cost allocation tag named department. Create one cost report in Cost Explorer grouping by tag name, and filter by EC2.

B.From the Organizations management account billing console, activate an AWS-defined cost allocation tag named department. Create one cost report in Cost Explorer grouping by tag name, and filter by EC2.

C.From the Organizations member account billing console, activate a user-defined cost allocation tag named department. Create one cost report in Cost Explorer grouping by the tag name, and filter by EC2.

D.From the Organizations member account billing console, activate an AWS-defined cost allocation tag named department. Create one cost report in Cost Explorer grouping by tag name, and filter by EC2.

Answer: A

Explanation:

The correct answer is A. Here's a detailed justification:

The scenario requires a centralized way to track EC2 costs across all accounts in an AWS Organization, grouped by department tag. The solution must be operationally efficient and leverage Cost Explorer, which the accounting team already has access to.

Option A is correct because it centralizes the configuration and reporting within the Organizations management account. Activating a user-defined cost allocation tag named "department" in the management account ensures that all accounts in the organization can utilize this tag for cost tracking. User-defined tags are suitable since the tagging policy is already adding the 'department' tag. Creating one cost report in Cost Explorer, grouping by the "department" tag and filtering by EC2, provides a consolidated view of EC2 costs per department across all accounts. This approach fulfills the requirement for a single report and centralized view.

Option B is incorrect because AWS-defined cost allocation tags are specific AWS-managed tags, not ones the user defines like the 'department' tag in the company's tagging policy.Option C is incorrect because activating cost allocation tags from each member account would be extremely inefficient and would require the accounting team to create and manage multiple cost reports, one per account or a combined report requiring more effort to set up. Centralizing the tag activation makes operations significantly easier.Option D suffers from the same problem as C. Using an AWS-defined tag when a custom 'department' tag already exists is inappropriate.

The critical element here is that the cost allocation tags must be activated from the management account to apply to all accounts in the organization, and using the user-defined tag is appropriate since the company

already has a tagging policy for the 'department' tag. This allows for centralized reporting through Cost Explorer as required.

Supporting Links:

AWS Cost Allocation Tags
AWS Organizations Billing and Cost Management
AWS Cost Explorer

Question: 460 **Exam Heist**

A company wants to securely exchange data between its software as a service (SaaS) application Salesforce account and Amazon S3. The company must encrypt the data at rest by using AWS Key Management Service (AWS KMS) customer managed keys (CMKs). The company must also encrypt the data in transit. The company has enabled API access for the Salesforce account.

A.Create AWS Lambda functions to transfer the data securely from Salesforce to Amazon S3.

B.Create an AWS Step Functions workflow. Define the task to transfer the data securely from Salesforce to Amazon S3.

C.Create Amazon AppFlow flows to transfer the data securely from Salesforce to Amazon S3.

D.Create a custom connector for Salesforce to transfer the data securely from Salesforce to Amazon S3.

Answer: C

Explanation:

The correct answer is **C. Create Amazon AppFlow flows to transfer the data securely from Salesforce to Amazon S3.**

Here's a detailed justification:

Amazon AppFlow is a fully managed integration service that enables secure data transfer between SaaS applications like Salesforce and AWS services like S3. AppFlow is designed to address exactly this type of integration scenario. It offers built-in security features and is specifically designed for data transfer between SaaS applications and AWS.

Why AppFlow is ideal:

Native Integration: AppFlow natively integrates with Salesforce and S3, simplifying the configuration and data transfer process. It handles the complexities of API authentication and authorization.
Security: AppFlow supports encryption at rest using AWS KMS customer managed keys (CMKs), fulfilling the requirement for data encryption. It also ensures data is encrypted in transit using TLS. You can configure the flow to use a CMK for encrypting the data stored during the transfer process.
Managed Service: Being a fully managed service, AppFlow handles the underlying infrastructure, scaling, and maintenance, reducing operational overhead. You don't need to manage servers or code.
No-Code/Low-Code: AppFlow provides a visual interface to create flows without writing custom code, accelerating the development process.

Why other options are less suitable:

A. AWS Lambda functions: While Lambda functions can be used for data transfer, it would require writing custom code to handle Salesforce API authentication, data extraction, transformation, and loading into S3. Furthermore, you would need to manage encryption, security, and scaling on your own. This is more complex than using AppFlow.
B. AWS Step Functions workflow: Step Functions can orchestrate multiple Lambda functions or other tasks.

But using it directly for data transfer between Salesforce and S3 is overly complex. The underlying data transfer still requires Lambda function development with the same drawbacks as option A.
D. Create a custom connector for Salesforce: Developing a custom connector is a significant undertaking that requires deep expertise in Salesforce API and AWS services. AppFlow essentially is the pre-built connector that handles this complexity. It's unnecessary to create a connector from scratch.

In summary, Amazon AppFlow provides a secure, efficient, and managed solution for transferring data from Salesforce to S3 with encryption at rest using KMS CMKs and encryption in transit, perfectly aligning with the company's requirements.

Authoritative Links:

Amazon AppFlow: https://aws.amazon.com/appflow/
Amazon AppFlow Security: https://docs.aws.amazon.com/appflow/latest/userguide/security.html
AWS KMS: https://aws.amazon.com/kms/

Question: 461 **Exam Heist**

A company is developing a mobile gaming app in a single AWS Region. The app runs on multiple Amazon EC2 instances in an Auto Scaling group. The company stores the app data in Amazon DynamoDB. The app communicates by using TCP traffic and UDP traffic between the users and the servers. The application will be used globally. The company wants to ensure the lowest possible latency for all users.

Which solution will meet these requirements?

A.Use AWS Global Accelerator to create an accelerator. Create an Application Load Balancer (ALB) behind an accelerator endpoint that uses Global Accelerator integration and listening on the TCP and UDP ports. Update the Auto Scaling group to register instances on the ALB.

B.Use AWS Global Accelerator to create an accelerator. Create a Network Load Balancer (NLB) behind an accelerator endpoint that uses Global Accelerator integration and listening on the TCP and UDP ports. Update the Auto Scaling group to register instances on the NLB.

C.Create an Amazon CloudFront content delivery network (CDN) endpoint. Create a Network Load Balancer (NLB) behind the endpoint and listening on the TCP and UDP ports. Update the Auto Scaling group to register instances on the NLB. Update CloudFront to use the NLB as the origin.

D.Create an Amazon CloudFront content delivery network (CDN) endpoint. Create an Application Load Balancer (ALB) behind the endpoint and listening on the TCP and UDP ports. Update the Auto Scaling group to register instances on the ALB. Update CloudFront to use the ALB as the origin.

Answer: B

Explanation:

The optimal solution for minimizing latency for a globally distributed mobile gaming app utilizing TCP and UDP traffic with EC2 instances, DynamoDB, and an Auto Scaling group is to use AWS Global Accelerator with a Network Load Balancer (NLB).

Here's why:

AWS Global Accelerator: Global Accelerator is specifically designed to direct traffic to optimal endpoints across the AWS global network, reducing latency for users worldwide. It leverages the AWS backbone network, known for its speed and reliability. https://aws.amazon.com/global-accelerator/

Network Load Balancer (NLB): NLBs are ideal for handling TCP and UDP traffic at extremely high throughput and low latency. Unlike Application Load Balancers (ALBs), NLBs operate at Layer 4 (transport layer), making them more efficient for routing non-HTTP traffic. Gaming applications frequently use UDP for real-time data transmission, making NLB a natural fit.

ALB Incompatibility: Application Load Balancers are designed for HTTP/HTTPS traffic. While they are excellent for web applications, they cannot handle raw TCP or UDP traffic directly without modification, making them inappropriate for the stated needs.

CloudFront Limitations: While CloudFront excels at caching static content, it is not designed for real-time bidirectional communication like gaming applications, and does not support UDP. CloudFront is better suited for distributing game assets like textures and updates, not for handling the real-time game communication traffic itself.

Direct Integration: Global Accelerator offers seamless integration with NLBs. It can automatically route traffic to healthy instances behind an NLB in multiple Availability Zones or even Regions.

Lowest Latency: Global Accelerator ensures that users are routed to the nearest healthy endpoint based on their geographic location and network conditions, resulting in the lowest possible latency. The NLB provides the necessary handling for the TCP and UDP traffic from the application.

Question: 462 **Exam Heist**

A company has an application that processes customer orders. The company hosts the application on an Amazon EC2 instance that saves the orders to an Amazon Aurora database. Occasionally when traffic is high the workload does not process orders fast enough.

What should a solutions architect do to write the orders reliably to the database as quickly as possible?

A.Increase the instance size of the EC2 instance when traffic is high. Write orders to Amazon Simple Notification Service (Amazon SNS). Subscribe the database endpoint to the SNS topic.

B.Write orders to an Amazon Simple Queue Service (Amazon SQS) queue. Use EC2 instances in an Auto Scaling group behind an Application Load Balancer to read from the SQS queue and process orders into the database.

C.Write orders to Amazon Simple Notification Service (Amazon SNS). Subscribe the database endpoint to the SNS topic. Use EC2 instances in an Auto Scaling group behind an Application Load Balancer to read from the SNS topic.

D.Write orders to an Amazon Simple Queue Service (Amazon SQS) queue when the EC2 instance reaches CPU threshold limits. Use scheduled scaling of EC2 instances in an Auto Scaling group behind an Application Load Balancer to read from the SQS queue and process orders into the database.

Answer: B

Explanation:

Here's a detailed justification for why option B is the best solution:

The problem describes a scenario where an EC2 instance hosting an application struggles to keep up with processing customer orders and writing them to an Aurora database during peak traffic. The goal is to reliably write orders to the database as quickly as possible, even under heavy load.

Option B proposes using Amazon SQS as a buffer for incoming orders. SQS is a fully managed message queuing service that allows decoupling the application from the database. Instead of writing directly to the database, the EC2 instance quickly writes orders to the SQS queue. This immediately frees up the EC2 instance to handle more incoming requests.

The solution further suggests using an Auto Scaling group of EC2 instances behind an Application Load Balancer (ALB). These EC2 instances are responsible for reading messages from the SQS queue and processing them into the Aurora database. The ALB distributes the workload across these instances, ensuring high availability and scalability. The Auto Scaling group automatically adjusts the number of EC2 instances based on the queue length or other performance metrics, dynamically scaling to meet demand.

This approach effectively decouples the order receiving and order processing components. SQS acts as a buffer, preventing the database from being overwhelmed during peak traffic. The Auto Scaling group allows for horizontal scaling of the processing capacity, ensuring that orders are processed and written to the database as quickly as possible. The ALB ensures that the processing load is evenly distributed among the available EC2 instances.

Option A is incorrect because SNS is a publish/subscribe service, better suited for broadcasting notifications to multiple subscribers. It doesn't provide the buffering and guaranteed delivery that SQS offers. Directly subscribing the database endpoint to an SNS topic is also generally not recommended due to potential issues with high message volume and database overload.

Option C is also incorrect for similar reasons as option A. Using SNS doesn't offer the benefits of queueing and guaranteed delivery that SQS provides.

Option D introduces unnecessary complexity and relies on reactive scaling. Waiting for the EC2 instance to reach CPU threshold limits before queuing orders adds latency and doesn't proactively address the problem. Scheduled scaling can be helpful in anticipating traffic patterns, but Auto Scaling based on queue length offers a more dynamic and responsive approach. Moreover, it ties the queuing decision to the original EC2 instance, defeating the purpose of decoupling.

Therefore, option B provides the most robust and scalable solution for reliably writing orders to the database as quickly as possible under high traffic.

Authoritative Links:

Amazon SQS: https://aws.amazon.com/sqs/
Amazon EC2 Auto Scaling: https://aws.amazon.com/autoscaling/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/

Question: 463 **Exam Heist**

An IoT company is releasing a mattress that has sensors to collect data about a user's sleep. The sensors will send data to an Amazon S3 bucket. The sensors collect approximately 2 MB of data every night for each mattress. The company must process and summarize the data for each mattress. The results need to be available as soon as possible. Data processing will require 1 GB of memory and will finish within 30 seconds.

Which solution will meet these requirements MOST cost-effectively?

A.Use AWS Glue with a Scala job
B.Use Amazon EMR with an Apache Spark script
C.Use AWS Lambda with a Python script
D.Use AWS Glue with a PySpark job

Answer: C

Explanation:

The question requires a cost-effective solution for processing IoT sleep data from mattresses, given constraints on memory, processing time, and data volume. AWS Lambda emerges as the most suitable option due to its serverless nature and cost model.

Lambda functions are billed based on execution time and memory consumed. Given the data processing requirements of 1 GB of memory and a 30-second execution time, a Lambda function can readily handle this workload. The small data size (2MB per mattress per night) is easily accommodated by Lambda.

AWS Glue (using either Scala or PySpark) and Amazon EMR with Spark are designed for large-scale data

processing. These services involve starting and maintaining clusters, which incurs higher costs even for small datasets. While they offer powerful data transformation capabilities, they are overkill for this scenario. The overhead of managing a Glue or EMR cluster, even briefly, significantly increases costs compared to Lambda.

Python is a perfectly acceptable language for Lambda. The prompt stated that the results need to be available as soon as possible. While there is "cold start" latency with Lambda, the ability to automatically trigger processing upon data arrival into S3 and the speed of Python are sufficient in this case.

Therefore, AWS Lambda provides the most cost-effective solution because it avoids the overhead of cluster management, scales automatically, and only charges for actual execution time and memory consumed.

Supporting Links:

AWS Lambda Pricing: https://aws.amazon.com/lambda/pricing/
AWS Glue Pricing: https://aws.amazon.com/glue/pricing/
Amazon EMR Pricing: https://aws.amazon.com/emr/pricing/

Question: 464 **Exam Heist**

A company hosts an online shopping application that stores all orders in an Amazon RDS for PostgreSQL Single-AZ DB instance. Management wants to eliminate single points of failure and has asked a solutions architect to recommend an approach to minimize database downtime without requiring any changes to the application code.

Which solution meets these requirements?

A.Convert the existing database instance to a Multi-AZ deployment by modifying the database instance and specifying the Multi-AZ option.

B.Create a new RDS Multi-AZ deployment. Take a snapshot of the current RDS instance and restore the new Multi-AZ deployment with the snapshot.

C.Create a read-only replica of the PostgreSQL database in another Availability Zone. Use Amazon Route 53 weighted record sets to distribute requests across the databases.

D.Place the RDS for PostgreSQL database in an Amazon EC2 Auto Scaling group with a minimum group size of two. Use Amazon Route 53 weighted record sets to distribute requests across instances.

Answer: A

Explanation:

The correct answer is A. Converting the existing RDS for PostgreSQL Single-AZ instance to a Multi-AZ deployment provides high availability without requiring application code changes. RDS Multi-AZ synchronously replicates data to a standby instance in a different Availability Zone. If the primary instance fails, RDS automatically fails over to the standby instance. The connection endpoint remains the same, eliminating the need for application modifications.

Option B involves creating a new Multi-AZ deployment and restoring a snapshot. While functional, it introduces a period of downtime during the restoration process, which isn't ideal for minimizing downtime.

Option C, using a read replica and Route 53, is unsuitable because read replicas are asynchronous and primarily for read scaling, not high availability. Route 53 weighted records are inappropriate for database failover because they don't guarantee data consistency or transactional integrity. Furthermore, the application needs to be able to handle potential read inconsistencies and direct writes to the appropriate database.

Option D, using EC2 Auto Scaling and Route 53, is not directly applicable to RDS managed databases. RDS is a managed service that abstracts away the underlying infrastructure management. Also, directly managing the database replication across instances in an Auto Scaling group is more complex than leveraging the built-

in Multi-AZ feature of RDS.

Therefore, Multi-AZ deployment in RDS provides the simplest and most efficient approach to achieve high availability for the database without application code changes.

Refer to the AWS documentation for more information on RDS Multi-AZ deployments:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html

Question: 465 **Exam Heist**

A company is developing an application to support customer demands. The company wants to deploy the application on multiple Amazon EC2 Nitro-based instances within the same Availability Zone. The company also wants to give the application the ability to write to multiple block storage volumes in multiple EC2 Nitro-based instances simultaneously to achieve higher application availability.

Which solution will meet these requirements?

A.Use General Purpose SSD (gp3) EBS volumes with Amazon Elastic Block Store (Amazon EBS) Multi-Attach

B.Use Throughput Optimized HDD (st1) EBS volumes with Amazon Elastic Block Store (Amazon EBS) Multi-Attach

C.Use Provisioned IOPS SSD (io2) EBS volumes with Amazon Elastic Block Store (Amazon EBS) Multi-Attach

D.Use General Purpose SSD (gp2) EBS volumes with Amazon Elastic Block Store (Amazon EBS) Multi-Attach

Answer: C

Explanation:

The requirement is to provide simultaneous write access to multiple EBS volumes from multiple EC2 Nitro-based instances for high availability. Amazon EBS Multi-Attach allows attaching one EBS volume to multiple EC2 instances within the same Availability Zone.

Option C, using io2 EBS volumes with EBS Multi-Attach, is the correct solution because io2 volumes are specifically designed to support EBS Multi-Attach. Other volume types like gp2, gp3, and st1 either don't support Multi-Attach at all or have limitations when used with it. io2 volumes also offer the highest IOPS (Input/Output Operations Per Second) performance, which can be important when multiple instances are writing to the same volume concurrently, as this increases the demand on I/O.

Options A, B and D are incorrect. gp2 and gp3 volumes support Multi-Attach, however, it is not advisable for high-availability applications that require concurrent write access because io2 is optimized for this use case. The st1 volume type does not support the EBS Multi-Attach feature. The need for concurrent writes indicates a requirement for high-performance shared storage, and io2 volumes are designed to meet such needs by offering high IOPS and low latency.

Refer to the AWS documentation on EBS Multi-Attach and EBS volume types for more information:

EBS Multi-Attach
EBS Volume Types

Question: 466 **Exam Heist**

A company designed a stateless two-tier application that uses Amazon EC2 in a single Availability Zone and an Amazon RDS Multi-AZ DB instance. New company management wants to ensure the application is highly available.

What should a solutions architect do to meet this requirement?

A.Configure the application to use Multi-AZ EC2 Auto Scaling and create an Application Load Balancer

B.Configure the application to take snapshots of the EC2 instances and send them to a different AWS Region

C.Configure the application to use Amazon Route 53 latency-based routing to feed requests to the application

D.Configure Amazon Route 53 rules to handle incoming requests and create a Multi-AZ Application Load Balancer

Answer: A

Explanation:

The best way to achieve high availability for a stateless two-tier application running on EC2 with an RDS Multi-AZ DB instance is to distribute the EC2 instances across multiple Availability Zones (AZs) and use a load balancer to distribute traffic.

Option A is the correct answer because it implements this strategy. Multi-AZ EC2 Auto Scaling ensures that EC2 instances are launched and maintained in multiple AZs. If an AZ fails, Auto Scaling automatically launches new instances in a healthy AZ. The Application Load Balancer (ALB) distributes traffic across the healthy EC2 instances in all available AZs. This setup ensures that the application remains available even if one or more AZs experience an outage.

Option B, while providing a backup strategy, doesn't provide immediate high availability. Snapshots can be used for disaster recovery, but restoring from snapshots takes time, causing application downtime.

Option C, using Route 53 latency-based routing, improves performance by directing users to the closest region but doesn't inherently guarantee high availability within a single region where the application is currently deployed.

Option D, configuring Route 53 rules combined with a Multi-AZ ALB is incomplete. While the Multi-AZ ALB is a good start, it needs the backing EC2 instances also deployed across multiple AZs using something like Auto Scaling for full high availability within a region. Route 53 is best utilized for global failover between regions.

Therefore, the combination of Multi-AZ EC2 Auto Scaling and an Application Load Balancer offers the most robust solution for high availability in a single AWS Region.

Relevant links for further reading:

AWS Auto Scaling: https://aws.amazon.com/autoscaling/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
AWS Regions and Availability Zones: https://aws.amazon.com/about-aws/global-infrastructure/

Question: 467 **Exam Heist**

A company uses AWS Organizations. A member account has purchased a Compute Savings Plan. Because of changes in the workloads inside the member account, the account no longer receives the full benefit of the Compute Savings Plan commitment. The company uses less than 50% of its purchased compute power.

A.Turn on discount sharing from the Billing Preferences section of the account console in the member account that purchased the Compute Savings Plan.

B.Turn on discount sharing from the Billing Preferences section of the account console in the company's Organizations management account.

C.Migrate additional compute workloads from another AWS account to the account that has the Compute Savings Plan.

D.Sell the excess Savings Plan commitment in the Reserved Instance Marketplace.

Answer: B

Explanation:

The correct answer is **B. Turn on discount sharing from the Billing Preferences section of the account console in the company's Organizations management account.**

Here's why:

AWS Organizations Discount Sharing: AWS Organizations allows for centralized management and billing of multiple AWS accounts. One key benefit is discount sharing, where savings plans (like Compute Savings Plans) purchased by one account can be applied across all accounts within the organization. This maximizes the utilization of the savings and reduces overall costs.

Centralized Management Account: Discount sharing is configured at the management account (formerly known as the master account) level within AWS Organizations, not at the individual member account level. The management account acts as the central point for managing the organization and its billing.

Turning on Discount Sharing: Within the management account's Billing Preferences, there's a setting to enable savings plans and reserved instance sharing. By activating this, the unused portion of the Compute Savings Plan's commitment from the member account can be applied to compute usage in other accounts within the organization.

Why other options are incorrect:

A: Attempting to enable discount sharing within the member account that purchased the savings plan won't work. The setting exists solely in the management account.
C: While migrating additional workloads could utilize more of the compute power, it might not be feasible or strategically aligned with the company's architecture. Also, it does not address the issue of sharing the saving if the loads are in the member account that holds the saving plan. Discount sharing is a more direct and efficient solution.
D: Savings Plans cannot be sold on the Reserved Instance Marketplace. This Marketplace is for selling Reserved Instances, not Savings Plans.

In summary, leveraging the discount sharing feature of AWS Organizations allows the company to maximize the benefits of its Compute Savings Plan by automatically applying unused savings to other accounts within the organization. This is configured in the Organizations management account's billing preferences.

Authoritative Links for further research:

AWS Organizations Billing: https://docs.aws.amazon.com/organizations/latest/userguide/orgs_getting-started_concepts.html
Savings Plans Sharing within Organizations: https://docs.aws.amazon.com/savingsplans/latest/userguide/sp-sharing.html
Compute Savings Plans: https://aws.amazon.com/savingsplans/compute-savings-plans/

Question: 468 **Exam Heist**

A company is developing a microservices application that will provide a search catalog for customers. The company must use REST APIs to present the frontend of the application to users. The REST APIs must access the backend services that the company hosts in containers in private VPC subnets.

Which solution will meet these requirements?

A.Design a WebSocket API by using Amazon API Gateway. Host the application in Amazon Elastic Container Service (Amazon ECS) in a private subnet. Create a private VPC link for API Gateway to access Amazon ECS.

B.Design a REST API by using Amazon API Gateway. Host the application in Amazon Elastic Container Service (Amazon ECS) in a private subnet. Create a private VPC link for API Gateway to access Amazon ECS.

C.Design a WebSocket API by using Amazon API Gateway. Host the application in Amazon Elastic Container Service (Amazon ECS) in a private subnet. Create a security group for API Gateway to access Amazon ECS.

D.Design a REST API by using Amazon API Gateway. Host the application in Amazon Elastic Container Service (Amazon ECS) in a private subnet. Create a security group for API Gateway to access Amazon ECS.

Answer: B

Explanation:

The correct answer is B because it aligns perfectly with the requirements of creating a REST API frontend for accessing backend microservices hosted in private subnets.

Here's a breakdown of the justification:

REST API Requirement: The problem explicitly states that the application must expose REST APIs. Amazon API Gateway can be configured to create RESTful APIs.
Containerization in Private Subnets: The backend services are hosted in containers within private VPC subnets. This means they are not directly accessible from the public internet. Amazon ECS is suitable to manage the containers.
Private VPC Link: To securely access the services in the private subnets from API Gateway, a Private VPC Link is the appropriate solution. It establishes a private connection between API Gateway and the ECS services, without exposing them to the internet. This improves security and reduces attack surface.
Security Groups vs. VPC Link: While security groups are important for controlling traffic within the VPC, they don't create a secure, direct connection between API Gateway and resources within the private subnet. VPC links are designed specifically for this purpose.
WebSocket APIs: WebSocket APIs are used for real-time, bidirectional communication, which is not a requirement in this scenario. REST APIs are appropriate for a simple request-response model that is suitable for catalog search.

Therefore, designing a REST API with API Gateway, hosting the application in Amazon ECS in private subnets, and creating a private VPC link is the solution that will meet all requirements.

Further Reading:

Amazon API Gateway: https://aws.amazon.com/api-gateway/
Amazon ECS: https://aws.amazon.com/ecs/
API Gateway Private Integration:
https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-private-apis.html

Question: 469 **Exam Heist**

A company stores raw collected data in an Amazon S3 bucket. The data is used for several types of analytics on behalf of the company's customers. The type of analytics requested determines the access pattern on the S3 objects.

The company cannot predict or control the access pattern. The company wants to reduce its S3 costs.

Which solution will meet these requirements?

A.Use S3 replication to transition infrequently accessed objects to S3 Standard-Infrequent Access (S3 Standard-IA)

B.Use S3 Lifecycle rules to transition objects from S3 Standard to Standard-Infrequent Access (S3 Standard-IA)

C.Use S3 Lifecycle rules to transition objects from S3 Standard to S3 Intelligent-Tiering

D.Use S3 Inventory to identify and transition objects that have not been accessed from S3 Standard to S3 Intelligent-Tiering

Answer: C

Explanation:

The correct answer is C: Use S3 Lifecycle rules to transition objects from S3 Standard to S3 Intelligent-Tiering. Here's why:

The problem requires optimizing S3 costs for unpredictable access patterns. S3 Intelligent-Tiering is designed for this exact scenario. It automatically moves data between frequent, infrequent, and archive access tiers based on usage patterns, with no operational overhead and no retrieval fees when accessing the data.

S3 Lifecycle rules enable automated transitioning of objects between storage classes based on defined criteria. This allows moving objects from S3 Standard (a higher-cost, frequently accessed tier) to S3 Intelligent-Tiering without manual intervention. The lifecycle rule dictates the transition, ensuring that data is placed in the appropriate tier based on its access pattern.

Option A, S3 Replication, is primarily for creating copies of data in different regions or buckets for disaster recovery or data sovereignty. It doesn't directly address cost optimization based on access patterns within a single bucket.

Option B, transitioning to S3 Standard-IA, is suitable for infrequently accessed data, but if the access pattern is unpredictable and the data sometimes needs to be accessed frequently, using Standard-IA will incur retrieval charges every time the infrequently accessed data is accessed, making it more expensive. S3 Intelligent-Tiering is preferable because it automatically adapts to fluctuating access patterns.

Option D, using S3 Inventory, helps identify infrequently accessed objects, but doesn't automatically transition them. This still requires additional steps to move the data and introduces a manual intervention component, while S3 Lifecycle rules automatically does the transition. S3 Inventory works well with S3 Lifecycle, and allows reporting on the success or failure of applying rules.

In summary, S3 Intelligent-Tiering paired with Lifecycle rules offers the most effective and automated solution for cost optimization when dealing with unpredictable data access patterns on S3, ensuring that frequently accessed data is readily available while minimizing storage costs for infrequently accessed data.

Authoritative Links:

S3 Intelligent-Tiering: https://aws.amazon.com/s3/storage-classes/intelligent-tiering/
S3 Lifecycle: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-management-overview.html

Question: 470 **Exam Heist**

A company has applications hosted on Amazon EC2 instances with IPv6 addresses. The applications must initiate communications with other external applications using the internet. However the company’s security policy states that any external service cannot initiate a connection to the EC2 instances.

What should a solutions architect recommend to resolve this issue?

A.Create a NAT gateway and make it the destination of the subnet's route table
B.Create an internet gateway and make it the destination of the subnet's route table
C.Create a virtual private gateway and make it the destination of the subnet's route table
D.Create an egress-only internet gateway and make it the destination of the subnet's route table

Answer: D

Explanation:

The correct answer is **D. Create an egress-only internet gateway and make it the destination of the subnet's route table.**

Here's why:

The scenario requires EC2 instances with IPv6 addresses to initiate outbound connections to the internet while preventing inbound connections from the internet. A standard Internet Gateway (IGW) facilitates both inbound and outbound traffic, which violates the security policy. A NAT Gateway is designed for IPv4 addresses, providing a mechanism for instances in a private subnet to connect to the internet. This is not suitable for instances with IPv6 addresses. A Virtual Private Gateway (VGW) is used to connect a VPC to a corporate data center using VPN or Direct Connect, which is irrelevant to this scenario.

An egress-only internet gateway (EIGW) is specifically designed for IPv6. It allows instances in a VPC to initiate outbound connections over IPv6 to the internet but prevents the internet from initiating an IPv6 connection with the instances. By associating the EIGW with the subnet's route table as the destination for IPv6 traffic (::/0), outbound traffic from the EC2 instances will be routed through the EIGW, while preventing unsolicited inbound traffic. This approach aligns perfectly with the company's security policy.

Therefore, the EIGW fulfills the requirement of enabling outbound-only IPv6 internet access for the EC2 instances while adhering to the specified security constraints.

Relevant Documentation:

Egress-Only Internet Gateway: https://docs.aws.amazon.com/vpc/latest/userguide/egress-only-internet-gateway.html

Question: 471 **Exam Heist**

A company is creating an application that runs on containers in a VPC. The application stores and accesses data in an Amazon S3 bucket. During the development phase, the application will store and access 1 TB of data in Amazon S3 each day. The company wants to minimize costs and wants to prevent traffic from traversing the internet whenever possible.

Which solution will meet these requirements?

A.Enable S3 Intelligent-Tiering for the S3 bucket

B.Enable S3 Transfer Acceleration for the S3 bucket

C.Create a gateway VPC endpoint for Amazon S3. Associate this endpoint with all route tables in the VPC

D.Create an interface endpoint for Amazon S3 in the VPC. Associate this endpoint with all route tables in the VPC

Answer: C

Explanation:

The correct answer is **C. Create a gateway VPC endpoint for Amazon S3. Associate this endpoint with all route tables in the VPC.**

Here's a detailed justification:

The company needs to minimize costs and ensure that S3 traffic doesn't traverse the internet. A VPC endpoint allows secure and private connectivity between resources within your VPC and supported AWS services, including S3, without requiring an internet gateway, NAT device, VPN connection, or AWS Direct Connect connection. This addresses the requirement of avoiding internet traffic.

Gateway endpoints are specifically designed for S3 and DynamoDB. They are cost-effective because they don't incur data transfer charges within the same AWS Region. The question prioritizes cost minimization, making the gateway endpoint a strong candidate.

Creating a gateway VPC endpoint and associating it with all route tables in the VPC ensures that all containerized applications within the VPC automatically route their S3 traffic through the endpoint. This achieves the desired private connectivity to S3 for all workloads.

Option A (S3 Intelligent-Tiering) optimizes storage costs based on access patterns but doesn't address the requirement of keeping traffic within the AWS network. It's about storage tier optimization, not network routing.

Option B (S3 Transfer Acceleration) is designed to speed up data transfers to S3 using CloudFront's globally distributed edge locations. While it can improve performance, it's not essential for keeping traffic private and can potentially increase costs.

Option D (Interface endpoint for S3) also provides private connectivity to S3, however, it's based on AWS PrivateLink, which can incur hourly and data processing charges. This is less cost-effective than a gateway endpoint, which has no hourly or data processing fees for S3 traffic within the same Region. Since the problem emphasizes cost minimization, the gateway endpoint is the better choice. Interface endpoints are also more complex to manage and are not generally the preferred method for S3 access within a VPC unless very specific security requirements necessitate them. Gateway endpoints automatically scale horizontally to handle network throughput.

Therefore, a gateway VPC endpoint provides a cost-effective and simple solution for ensuring private S3 connectivity within the VPC, fulfilling both the cost and security requirements.

Reference Links:

VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html
Gateway VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-s3.html
Interface VPC Endpoints (AWS PrivateLink): https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-interface.html

Question: 472 **Exam Heist**

A company has a mobile chat application with a data store based in Amazon DynamoDB. Users would like new messages to be read with as little latency as possible. A solutions architect needs to design an optimal solution that requires minimal application changes.

Which method should the solutions architect select?

A.Configure Amazon DynamoDB Accelerator (DAX) for the new messages table. Update the code to use the DAX endpoint.

B.Add DynamoDB read replicas to handle the increased read load. Update the application to point to the read endpoint for the read replicas.

C.Double the number of read capacity units for the new messages table in DynamoDB. Continue to use the existing DynamoDB endpoint.

D.Add an Amazon ElastiCache for Redis cache to the application stack. Update the application to point to the Redis cache endpoint instead of DynamoDB.

Answer: A

Explanation:

The correct answer is A: Configure Amazon DynamoDB Accelerator (DAX) for the new messages table and

update the code to use the DAX endpoint. Here's why:

DynamoDB Accelerator (DAX) is a fully managed, highly available, in-memory cache for DynamoDB. Its primary purpose is to reduce read latency for frequently accessed data, making it ideal for latency-sensitive applications like chat applications. DAX sits in front of DynamoDB and caches the results of read requests. Subsequent read requests for the same data are served directly from the cache, bypassing DynamoDB and resulting in significantly lower latency (often measured in microseconds).

Option A requires minimal application changes because it involves only updating the application to use the DAX endpoint instead of directly connecting to DynamoDB for reads. This minimizes the development effort and disruption to existing code. DAX automatically manages cache invalidation and synchronization with DynamoDB, ensuring data consistency.

Option B, adding DynamoDB read replicas, is not as suitable. While read replicas can help offload read traffic, they introduce eventual consistency. This means that reads from the replica might not immediately reflect the latest writes. For a chat application where users expect to see new messages instantly, eventual consistency is unacceptable. Furthermore, managing read replicas and ensuring failover can be more complex than using DAX.

Option C, doubling read capacity units, may improve read throughput but doesn't address latency as effectively as DAX. Increasing capacity increases the resources available for read operations but still involves network latency and the inherent overhead of querying DynamoDB. DAX, by caching frequently accessed data in memory, provides a much lower latency. Additionally, simply doubling read capacity might lead to increased costs without necessarily achieving the desired low latency.

Option D, adding Amazon ElastiCache for Redis, could provide low latency reads but would require significant application changes. The application would need to implement caching logic, manage cache invalidation, and handle synchronization between DynamoDB and Redis. This is a much more complex solution than using DAX. It also introduces additional operational overhead for managing a separate caching cluster. DAX is purpose-built for DynamoDB, so it offers better integration and requires significantly less development effort for the application.

In summary, DAX offers the optimal balance of low latency, minimal application changes, and ease of management for this use case.

Relevant links:

Amazon DynamoDB Accelerator (DAX)
DynamoDB Read Replicas
Amazon ElastiCache for Redis

Question: 473 **Exam Heist**

A company hosts a website on Amazon EC2 instances behind an Application Load Balancer (ALB). The website serves static content. Website traffic is increasing, and the company is concerned about a potential increase in cost.

A.Create an Amazon CloudFront distribution to cache state files at edge locations

B.Create an Amazon ElastiCache cluster. Connect the ALB to the ElastiCache cluster to serve cached files

C.Create an AWS WAF web ACL and associate it with the ALB. Add a rule to the web ACL to cache static files

D.Create a second ALB in an alternative AWS Region. Route user traffic to the closest Region to minimize data transfer costs

Answer: A

Explanation:

The best solution to reduce costs while handling increasing website traffic for static content served from EC2 instances behind an ALB is option A: creating an Amazon CloudFront distribution.

CloudFront is a content delivery network (CDN) that caches website content at edge locations closer to users. This significantly reduces latency and improves user experience. More importantly, it reduces the load on the origin servers (EC2 instances), as CloudFront serves the cached static content directly to users without hitting the EC2 instances. This reduces the compute resources needed to handle web traffic, resulting in decreased EC2 instance utilization and thus lowers costs. CloudFront's global network and caching capabilities are purpose-built for this scenario.

Option B, using ElastiCache, is not ideal for caching static content. ElastiCache is primarily used for caching dynamic data that requires frequent updates, like database query results or session data. Connecting the ALB to ElastiCache would require significant code changes and may not be as effective for caching static files as CloudFront. Also, ElastiCache sits within the AWS region, so it does not have the benefit of edge locations.

Option C, AWS WAF, is for web application firewalls and not optimized for caching static content. While WAF can provide some caching capabilities, its primary focus is protecting against web exploits, not content delivery optimization. Caching within WAF would be less efficient and more costly than using CloudFront.

Option D, creating a second ALB in another region, primarily addresses reducing data transfer costs through regional proximity but it increases the overall cost as there are additional instances, ALB infrastructure, and more resources required to maintain the other region. It doesn't address the issue of caching static assets directly.

In summary, CloudFront is the most cost-effective and efficient solution because it utilizes edge locations to cache static content, reduces the load on the origin servers, and improves website performance.

Relevant Documentation:

Amazon CloudFront: https://aws.amazon.com/cloudfront/
Amazon ElastiCache: https://aws.amazon.com/elasticache/
AWS WAF: https://aws.amazon.com/waf/

Question: 474 **Exam Heist**

A company has multiple VPCs across AWS Regions to support and run workloads that are isolated from workloads in other Regions. Because of a recent application launch requirement, the company's VPCs must communicate with all other VPCs across all Regions.

Which solution will meet these requirements with the LEAST amount of administrative effort?

A.Use VPC peering to manage VPC communication in a single Region. Use VPC peering across Regions to manage VPC communications.

B.Use AWS Direct Connect gateways across all Regions to connect VPCs across regions and manage VPC communications.

C.Use AWS Transit Gateway to manage VPC communication in a single Region and Transit Gateway peering across Regions to manage VPC communications.

D.Use AWS PrivateLink across all Regions to connect VPCs across Regions and manage VPC communications

Answer: C

Explanation:

The correct answer is C: Use AWS Transit Gateway to manage VPC communication in a single Region and

Transit Gateway peering across Regions to manage VPC communications.

Here's why:

AWS Transit Gateway (TGW): TGW simplifies network architecture by acting as a central hub to connect multiple VPCs within a Region. Instead of creating numerous point-to-point VPC peering connections, you connect each VPC to the TGW, significantly reducing administrative overhead and complexity.https://aws.amazon.com/transit-gateway/

Transit Gateway Peering: To extend connectivity across AWS Regions, you can use Transit Gateway peering. This allows TGWs in different Regions to establish a connection, enabling VPCs in one Region to communicate with VPCs in another Region through the interconnected TGWs. This avoids the need for individual inter-region VPC peering connections.

Least Administrative Effort: TGW and TGW peering together dramatically reduce the administrative burden compared to managing a full mesh of VPC peering connections across multiple Regions. The number of connections you need to manage grows linearly with the number of Regions when using TGW peering, as opposed to exponentially when using VPC peering.

Why other options are incorrect:

A (VPC Peering): While VPC peering establishes connections between VPCs, managing a full mesh of VPC peering connections across multiple Regions becomes extremely complex and administratively intensive as the number of VPCs and Regions increases. It creates a complex web of connections to manage, which can be prone to errors.

B (AWS Direct Connect Gateway): Direct Connect Gateway is used to connect on-premises networks to AWS, not VPCs within AWS. While it can be part of a larger solution to connect on-premises and AWS, it's not suitable for connecting VPCs in different regions directly and adds unnecessary complexity.

D (AWS PrivateLink): PrivateLink is designed to provide private connectivity to AWS services or services hosted by other AWS customers. It's not meant for general VPC-to-VPC communication. Using it for this purpose would be an inefficient and complex workaround. Furthermore, PrivateLink focuses on providing a service, not interconnecting multiple VPCs for general communication.https://aws.amazon.com/privatelink/

Question: 475 **Exam Heist**

A company is designing a containerized application that will use Amazon Elastic Container Service (Amazon ECS). The application needs to access a shared file system that is highly durable and can recover data to another AWS Region with a recovery point objective (RPO) of 8 hours. The file system needs to provide a mount target m each Availability Zone within a Region.

A solutions architect wants to use AWS Backup to manage the replication to another Region.

Which solution will meet these requirements?

A.Amazon FSx for Windows File Server with a Multi-AZ deployment
B.Amazon FSx for NetApp ONTAP with a Multi-AZ deployment
C.Amazon Elastic File System (Amazon EFS) with the Standard storage class
D.Amazon FSx for OpenZFS

Answer: C

Explanation:

The correct answer is C: Amazon Elastic File System (Amazon EFS) with the Standard storage class.

Here's why:

EFS and Mount Targets: EFS is designed to provide a shared file system that can be mounted concurrently from multiple EC2 instances or containers across multiple Availability Zones (AZs) within a Region. This directly addresses the requirement of providing a mount target in each AZ.

Durability and Availability: EFS stores data redundantly across multiple AZs, offering high durability and availability.

AWS Backup Integration: AWS Backup natively supports EFS, enabling automated backups and cross-region replication for disaster recovery. This supports the requirement of using AWS Backup to manage replication to another region and meet the RPO.

RPO: While the exact RPO is dependent on your backup schedule configuration in AWS Backup, it can be configured to achieve an 8-hour RPO for EFS.

FSx for Windows File Server & FSx for NetApp ONTAP Limitations: While FSx for Windows File Server and FSx for NetApp ONTAP are suitable file systems for specific workloads, they are not the best fit for this scenario because they may have higher costs and complexity for simple file sharing across multiple AZs with easy AWS Backup integration. Also, FSx for Windows File Server is typically used for Windows-based applications.

FSx for OpenZFS Limitations: Although FSx for OpenZFS offers high performance, it might be an overkill for a general purpose containerized application needing just shared storage and durability.

Therefore, Amazon EFS offers the simplest and most cost-effective solution for providing a highly durable, shared file system with multi-AZ access and easy integration with AWS Backup for cross-region replication and disaster recovery.

Relevant Links:

Amazon EFS: https://aws.amazon.com/efs/
AWS Backup: https://aws.amazon.com/backup/
Cross-Region Backups with AWS Backup: https://aws.amazon.com/blogs/storage/enabling-cross-region-backup-with-aws-backup/

Question: 476 **Exam Heist**

A company is expecting rapid growth in the near future. A solutions architect needs to configure existing users and grant permissions to new users on AWS. The solutions architect has decided to create IAM groups. The solutions architect will add the new users to IAM groups based on department.

Which additional action is the MOST secure way to grant permissions to the new users?

A.Apply service control policies (SCPs) to manage access permissions
B.Create IAM roles that have least privilege permission. Attach the roles to the IAM groups
C.Create an IAM policy that grants least privilege permission. Attach the policy to the IAM groups
D.Create IAM roles. Associate the roles with a permissions boundary that defines the maximum permissions

Answer: C

Explanation:

The most secure way to grant permissions to new users added to IAM groups, given the scenario, is to create an IAM policy that grants least privilege permission and attach that policy to the IAM groups (Option C).

Here's why:

IAM Policies: IAM policies are JSON documents that define permissions. They specify what actions a user, group, or role can perform on AWS resources. Attaching a policy directly to an IAM group is a straightforward and effective way to define the permissions for all users within that group.
Least Privilege: Least privilege is a security principle that dictates granting users only the permissions they need to perform their job duties. Creating a specific IAM policy tailored to each group and granting only the necessary permissions minimizes the potential impact of compromised credentials.
IAM Groups vs. Roles: IAM groups are collections of IAM users. They are a convenient way to manage permissions for multiple users who require the same level of access. IAM roles, on the other hand, are assumed by entities, not directly assigned to users. While roles are excellent for EC2 instances and other services, they aren't the most direct way to manage access for users within groups.

Why other options are less secure:

Service Control Policies (SCPs) (Option A): SCPs are used at the AWS Organizations level to manage permissions across multiple AWS accounts. While valuable for central governance, they are not the best approach for granting specific permissions to IAM groups within a single account. SCPs define maximum permissions; IAM policies still need to be in place to grant the actual access.
IAM Roles attached to Groups (Option B): You don't attach roles to groups. Roles are assumed by entities (like EC2 instances) or assumed by users (via the console or CLI). While a user can assume a role, it adds unnecessary complexity and is not the standard way to grant basic permissions to a group of users.
Permissions Boundaries (Option D): Permissions boundaries define the maximum permissions that an IAM entity can have. While using them with Roles is a valid setup, a Role isn't the most fitting solution here. Furthermore, attaching a permissions boundary without also defining specific permissions within IAM policies will result in no access.

In summary, attaching IAM policies that grant least privilege directly to IAM groups is the most direct, manageable, and secure approach in this scenario. It follows the principle of least privilege while leveraging the convenience of IAM groups for managing user permissions.

Reference:

IAM Policies
IAM Groups
Least Privilege
IAM Roles

Question: 477 **Exam Heist**

A group requires permissions to list an Amazon S3 bucket and delete objects from that bucket. An administrator has created the following IAM policy to provide access to the bucket and applied that policy to the group. The group is not able to delete objects in the bucket. The company follows least-privilege access rules.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Action": [
                "s3:ListBucket",
                "s3:DeleteObject"
            ],
            "Resource": [
                "arn:aws:s3:::bucket-name"
            ],
            "Effect": "Allow"
        }
    ]
}
```

Which statement should a solutions architect add to the policy to correct bucket access?

A.
```
"Action": [
    "s3:*Object"
],
"Resource": [
    "arn:aws:s3:::bucket-name/*"
],
"Effect": "Allow"
```

B.
```
"Action": [
    "s3:*"
],
"Resource": [
    "arn:aws:s3:::bucket-name/*"
],
"Effect": "Allow"
```

```
"Action": [
    "s3:DeleteObject"
],
"Resource": [
    "arn:aws:s3:::bucket-name*"
],
"Effect": "Allow"
```

C.

```
"Action": [
    "s3:DeleteObject"
],
"Resource": [
    "arn:aws:s3:::bucket-name/*"
],
"Effect": "Allow"
```

D.

Answer: D

Explanation:

D for sure

Question: 478 **Exam Heist**

A law firm needs to share information with the public. The information includes hundreds of files that must be publicly readable. Modifications or deletions of the files by anyone before a designated future date are prohibited.

Which solution will meet these requirements in the MOST secure way?

A.Upload all files to an Amazon S3 bucket that is configured for static website hosting. Grant read-only IAM permissions to any AWS principals that access the S3 bucket until the designated date.

B.Create a new Amazon S3 bucket with S3 Versioning enabled. Use S3 Object Lock with a retention period in accordance with the designated date. Configure the S3 bucket for static website hosting. Set an S3 bucket policy to allow read-only access to the objects.

C.Create a new Amazon S3 bucket with S3 Versioning enabled. Configure an event trigger to run an AWS Lambda function in case of object modification or deletion. Configure the Lambda function to replace the objects with the original versions from a private S3 bucket.

D.Upload all files to an Amazon S3 bucket that is configured for static website hosting. Select the folder that contains the files. Use S3 Object Lock with a retention period in accordance with the designated date. Grant read-only IAM permissions to any AWS principals that access the S3 bucket.

Answer: B

Explanation:

The correct answer is B. Let's dissect why it's the most secure solution for the law firm's requirements.

Option B leverages several AWS security features optimally. First, creating a new S3 bucket ensures a

dedicated space for public-facing data, minimizing the risk of accidental exposure of other sensitive information. Enabling S3 Versioning is crucial because it preserves every version of an object, providing a built-in audit trail and facilitating easy recovery if unintended changes occur. The cornerstone of this solution is S3 Object Lock. By using Object Lock with a retention period extending to the designated future date, the law firm prevents modification or deletion of the files, adhering strictly to the requirement of immutability. Configuring the S3 bucket for static website hosting makes the files readily accessible to the public. Finally, setting an S3 bucket policy to allow read-only access explicitly restricts write operations, further enhancing security by limiting potential avenues for unauthorized modifications.

Option A falls short in terms of true immutability. While read-only IAM permissions restrict who can directly write to the bucket, they don't protect against accidental or malicious deletion by authorized principals.

Option C introduces unnecessary complexity and potential latency. Using Lambda functions triggered by object modification/deletion adds an additional layer of processing that can increase response times and introduces a potential point of failure. While the intention is good, the built-in capabilities of S3 Object Lock provide a more direct and efficient solution. Also, storing original versions in a separate private bucket adds to management overhead.

Option D incorrectly assumes that Object Lock can be applied to folders within S3. Object Lock is applied at the object level. This option also requires IAM policy to set read-only permissions, but it's not the most secure option since it doesn't prevent accidental deletion if user account compromised.

Therefore, Option B is the most secure and efficient solution. It leverages built-in features of S3 to enforce immutability, provide versioning for recovery, and grant read-only access, directly addressing all requirements without unnecessary complexity.

Authoritative links for further research:

S3 Object Lock: https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html
S3 Versioning: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html
S3 Bucket Policies: https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-iam-policies.html
S3 Static Website Hosting: https://docs.aws.amazon.com/AmazonS3/latest/userguide/WebsiteHosting.html

Question: 479 **Exam Heist**

A company is making a prototype of the infrastructure for its new website by manually provisioning the necessary infrastructure. This infrastructure includes an Auto Scaling group, an Application Load Balancer and an Amazon RDS database. After the configuration has been thoroughly validated, the company wants the capability to immediately deploy the infrastructure for development and production use in two Availability Zones in an automated fashion.

What should a solutions architect recommend to meet these requirements?

A.Use AWS Systems Manager to replicate and provision the prototype infrastructure in two Availability Zones

B.Define the infrastructure as a template by using the prototype infrastructure as a guide. Deploy the infrastructure with AWS CloudFormation.

C.Use AWS Config to record the inventory of resources that are used in the prototype infrastructure. Use AWS Config to deploy the prototype infrastructure into two Availability Zones.

D.Use AWS Elastic Beanstalk and configure it to use an automated reference to the prototype infrastructure to automatically deploy new environments in two Availability Zones.

Answer: B

Explanation:

The best approach is to define the infrastructure as code using AWS CloudFormation. CloudFormation allows

you to create and manage AWS infrastructure as code, using templates written in YAML or JSON. This approach ensures consistency, repeatability, and version control.CloudFormation templates describe the desired state of your infrastructure, including resources like Auto Scaling groups, Application Load Balancers, and RDS databases. The template serves as a blueprint for creating identical infrastructure in different environments or regions.Option B directly addresses the requirement to deploy the infrastructure in an automated fashion into two Availability Zones. CloudFormation handles the provisioning of resources across multiple Availability Zones based on the template's configuration.Option A, AWS Systems Manager, is primarily designed for managing existing infrastructure rather than provisioning new infrastructure from scratch. While SSM can automate tasks, it's not the ideal choice for defining and deploying entire infrastructure stacks.Option C, AWS Config, is used for assessing, auditing, and evaluating the configurations of your AWS resources. It records the configuration history of your resources but doesn't directly provision infrastructure like CloudFormation.Option D, AWS Elastic Beanstalk, is a PaaS (Platform as a Service) offering designed for deploying and managing web applications. While it can deploy applications, it's not designed for managing the entire infrastructure stack as comprehensively as CloudFormation, especially custom configurations involving multiple components.CloudFormation is the most suitable solution because it enables infrastructure as code, ensuring consistency, repeatability, and automated deployment across different environments and Availability Zones.

Here are some links for further research:

AWS CloudFormation: https://aws.amazon.com/cloudformation/
CloudFormation Documentation: https://docs.aws.amazon.com/cloudformation/
AWS Systems Manager: https://aws.amazon.com/systems-manager/
AWS Config: https://aws.amazon.com/config/
AWS Elastic Beanstalk: https://aws.amazon.com/elasticbeanstalk/

Question: 480 **Exam Heist**

A business application is hosted on Amazon EC2 and uses Amazon S3 for encrypted object storage. The chief information security officer has directed that no application traffic between the two services should traverse the public internet.

Which capability should the solutions architect use to meet the compliance requirements?

A.AWS Key Management Service (AWS KMS)
B.VPC endpoint
C.Private subnet
D.Virtual private gateway

Answer: B

Explanation:

Here's a detailed justification for why VPC endpoints are the correct solution:

The requirement is to ensure that traffic between an EC2 instance and an S3 bucket does not traverse the public internet.

VPC Endpoints: VPC endpoints enable you to privately connect your VPC to supported AWS services and VPC endpoint services powered by PrivateLink without requiring an internet gateway, NAT device, VPN connection, or AWS Direct Connect connection. Endpoints are horizontally scaled, redundant, and highly available VPC components that allow communication between instances in your VPC and services without imposing availability risks or bandwidth constraints on your network traffic. Specifically, using a Gateway Endpoint for S3 allows traffic to remain within the AWS network, bypassing the internet.

AWS KMS (Option A): AWS KMS is for managing encryption keys. While it is important for securing data at rest and in transit, it doesn't directly prevent traffic from going over the public internet. It addresses encryption, not network routing.

Private Subnet (Option C): A private subnet, by definition, does not have a route to an internet gateway. However, instances in a private subnet still require a NAT gateway or NAT instance to access S3 over the public internet unless a VPC endpoint is configured. Simply placing the EC2 instance in a private subnet doesn't solve the core problem.

Virtual Private Gateway (Option D): A Virtual Private Gateway is used to establish a VPN connection between your VPC and your on-premises network. This is irrelevant to the requirement of keeping traffic within the AWS network for communication between EC2 and S3.

Therefore, the correct answer is VPC endpoints because they provide a private connection to S3, ensuring the traffic stays within the AWS network and does not traverse the public internet, fulfilling the compliance requirement.

Authoritative Links:

VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/privatelink/vpc-endpoints.html
Gateway Endpoints for S3: https://docs.aws.amazon.com/vpc/latest/privatelink/vpc-endpoints-s3.html

Question: 481 **Exam Heist**

A company hosts a three-tier web application in the AWS Cloud. A Multi-AZAmazon RDS for MySQL server forms the database layer Amazon ElastiCache forms the cache layer. The company wants a caching strategy that adds or updates data in the cache when a customer adds an item to the database. The data in the cache must always match the data in the database.

Which solution will meet these requirements?

A.Implement the lazy loading caching strategy
B.Implement the write-through caching strategy
C.Implement the adding TTL caching strategy
D.Implement the AWS AppConfig caching strategy

Answer: B

Explanation:

The question requires a caching strategy that ensures data consistency between the database and the cache layer for a three-tier web application on AWS. The desired behavior is that when data is added to the database (specifically, an item is added), the corresponding cache entry is immediately updated to reflect this change.

Option B, implementing a write-through caching strategy, is the correct answer. A write-through cache updates both the cache and the database simultaneously. When a customer adds an item (data) to the database, the write-through cache mechanism will synchronously update the cache. This ensures that the cache always contains the most up-to-date information, matching the data in the database. This satisfies the requirement of data consistency.

Option A, lazy loading, is incorrect. Lazy loading caches data only when it is first requested. While it saves resources on initial data loading, it doesn't guarantee immediate cache updates after database modifications, leading to potential staleness.

Option C, adding TTL (Time-to-Live), while a common caching optimization, is not a strategy itself but rather a parameter. Even with TTL, there's no guarantee the cache is updated immediately upon database changes.

Stale data could exist until the TTL expires.

Option D, AWS AppConfig, is primarily a service for application configuration management, not a caching strategy. While you could use it to control parameters related to caching, it doesn't inherently provide the synchronous update mechanism required.

Therefore, only the write-through caching strategy provides the guarantee of data consistency by updating the cache immediately when the database is updated.

For more information on caching strategies:

Amazon ElastiCache Strategies: https://aws.amazon.com/elasticache/
Caching Strategies: https://learn.microsoft.com/en-us/azure/architecture/patterns/cache-aside (though Azure-specific, the caching concepts apply generally)

Question: 482 **Exam Heist**

A company wants to migrate 100 GB of historical data from an on-premises location to an Amazon S3 bucket. The company has a 100 megabits per second (Mbps) internet connection on premises. The company needs to encrypt the data in transit to the S3 bucket. The company will store new data directly in Amazon S3.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use the s3 sync command in the AWS CLI to move the data directly to an S3 bucket

B.Use AWS DataSync to migrate the data from the on-premises location to an S3 bucket

C.Use AWS Snowball to move the data to an S3 bucket

D.Set up an IPsec VPN from the on-premises location to AWS. Use the s3 cp command in the AWS CLI to move the data directly to an S3 bucket

Answer: B

Explanation:

Here's a detailed justification of why option B is the best solution for migrating 100 GB of on-premises data to Amazon S3 with encryption in transit and minimal operational overhead, given the specified constraints:

Option B: Use AWS DataSync

DataSync is a fully managed data transfer service that simplifies, automates, and accelerates moving data between on-premises storage and AWS storage services. Crucially, DataSync encrypts data in transit using TLS, meeting the encryption requirement.

Efficiency and Speed: DataSync utilizes a purpose-built transfer protocol designed to optimize data transfer over the internet or AWS Direct Connect. It automatically handles tasks such as data validation, checksumming, and error recovery, reducing operational burden.
Encryption in Transit: DataSync natively encrypts data during transit, fulfilling the requirement to encrypt data moving to the S3 bucket.
Simplicity and Automation: DataSync simplifies the migration process through its agent-based architecture. You deploy an agent in your on-premises environment, configure it to connect to your S3 bucket, and then define a transfer task. It automates the transfer process and provides monitoring capabilities.
No Hardware Logistics: Unlike AWS Snowball, DataSync doesn't involve shipping physical devices, eliminating associated logistical overhead and delays.

Why other options are less suitable:

Option A (s3 sync with AWS CLI): While functional, s3 sync lacks the optimization features of DataSync,

leading to slower transfer speeds and increased operational overhead. It requires more manual intervention to monitor and restart transfers.
Option C (AWS Snowball): Snowball is overkill for only 100 GB of data. The turnaround time for shipping a Snowball device can be significantly longer than using DataSync over a 100 Mbps connection. It also introduces logistical complexity.
Option D (IPsec VPN + s3 cp with AWS CLI): Setting up and maintaining an IPsec VPN introduces significant operational overhead. While it secures the connection, it doesn't provide the optimized transfer capabilities of DataSync. s3 cp shares the same drawbacks as s3 sync in terms of optimization and automation.

Calculations & Network Limitations:

Transferring 100GB (800Gb) over a 100Mbps connection would take approximately 22.2 hours at maximum theoretical speed. DataSync's features to optimize this, it becomes the best option.

Conclusion:

AWS DataSync provides the best balance of speed, security, and simplicity for migrating 100 GB of data to S3 over a 100 Mbps connection. It eliminates the complexity of VPNs and physical devices while ensuring encryption in transit.

Supporting Documentation:

AWS DataSync: https://aws.amazon.com/datasync/
DataSync Encryption: https://docs.aws.amazon.com/datasync/latest/userguide/security.html

Question: 483 **Exam Heist**

A company containerized a Windows job that runs on .NET 6 Framework under a Windows container. The company wants to run this job in the AWS Cloud. The job runs every 10 minutes. The job's runtime varies between 1 minute and 3 minutes.

Which solution will meet these requirements MOST cost-effectively?

A.Create an AWS Lambda function based on the container image of the job. Configure Amazon EventBridge to invoke the function every 10 minutes.

B.Use AWS Batch to create a job that uses AWS Fargate resources. Configure the job scheduling to run every 10 minutes.

C.Use Amazon Elastic Container Service (Amazon ECS) on AWS Fargate to run the job. Create a scheduled task based on the container image of the job to run every 10 minutes.

D.Use Amazon Elastic Container Service (Amazon ECS) on AWS Fargate to run the job. Create a standalone task based on the container image of the job. Use Windows task scheduler to run the job every 10 minutes.

Answer: C

Explanation:

The most cost-effective solution is **C. Use Amazon Elastic Container Service (Amazon ECS) on AWS Fargate to run the job. Create a scheduled task based on the container image of the job to run every 10 minutes.**

Here's why:

Cost-Effectiveness: Fargate allows you to pay only for the compute resources your container uses, without managing the underlying infrastructure. For a short-running job executed every 10 minutes, this is more cost-effective than provisioning and maintaining EC2 instances.
Scheduled Tasks with ECS: ECS provides built-in scheduling capabilities to run tasks on a defined schedule using services like Amazon CloudWatch Events (now EventBridge), making option C a straightforward solution

to the problem requirements.
Containerization Compatibility: ECS Fargate readily supports running containerized applications, including Windows containers, fulfilling the requirement that the job is already containerized.
No need for external schedulers: Option D requires the use of Windows task scheduler, which is not a native cloud scheduler for AWS services and requires an external dependency.
Comparing to Lambda (Option A): AWS Lambda generally has a maximum execution time limit that could be an issue if the job's runtime occasionally approaches or exceeds that limit. Furthermore, while Lambda supports container images, it's generally optimized for event-driven, stateless functions, not periodic tasks that require more persistent processing.

In contrast:

AWS Batch (Option B), while suitable for batch workloads, adds unnecessary complexity for a simple scheduled task that requires consistent intervals.
Lambda (Option A) AWS Lambda's maximum execution time limit and general optimization for event-driven stateless functions makes it less suitable for periodic tasks that require persistent processing.
Windows task scheduler (Option D) requires external dependencies.

Therefore, ECS on Fargate with scheduled tasks provides the most straightforward, cost-effective, and manageable solution.

Supporting Resources:

Amazon ECS Scheduled Tasks
AWS Fargate Pricing
AWS Lambda Function Configuration

Question: 484 **Exam Heist**

A company wants to move from many standalone AWS accounts to a consolidated, multi-account architecture. The company plans to create many new AWS accounts for different business units. The company needs to authenticate access to these AWS accounts by using a centralized corporate directory service.

Which combination of actions should a solutions architect recommend to meet these requirements? (Choose two.)

A.Create a new organization in AWS Organizations with all features turned on. Create the new AWS accounts in the organization.

B.Set up an Amazon Cognito identity pool. Configure AWS IAM Identity Center (AWS Single Sign-On) to accept Amazon Cognito authentication.

C.Configure a service control policy (SCP) to manage the AWS accounts. Add AWS IAM Identity Center (AWS Single Sign-On) to AWS Directory Service.

D.Create a new organization in AWS Organizations. Configure the organization's authentication mechanism to use AWS Directory Service directly.

E.Set up AWS IAM Identity Center (AWS Single Sign-On) in the organization. Configure IAM Identity Center, and integrate it with the company's corporate directory service.

Answer: AE

Explanation:

The correct answer is AE. Here's why:

A: Create a new organization in AWS Organizations with all features turned on. Create the new AWS accounts in the organization. AWS Organizations is designed to centrally manage and govern multiple AWS accounts. Creating an organization provides a hierarchical structure to manage permissions and policies across accounts, which is essential for a consolidated, multi-account architecture. With all features enabled,

you can leverage service control policies (SCPs) for centralized governance. https://aws.amazon.com/organizations/

E: Set up AWS IAM Identity Center (AWS Single Sign-On) in the organization. Configure IAM Identity Center, and integrate it with the company's corporate directory service. AWS IAM Identity Center (successor to AWS SSO) enables you to centrally manage access to multiple AWS accounts and applications. Integrating it with your corporate directory service (like Active Directory) allows users to use their existing credentials to access AWS resources. This centralizes authentication and simplifies user management across the organization. https://aws.amazon.com/iam/identity-center/

Why other options are incorrect:

B: Set up an Amazon Cognito identity pool. Configure AWS IAM Identity Center (AWS Single Sign-On) to accept Amazon Cognito authentication. Cognito Identity Pools are primarily used to grant temporary AWS credentials to users of mobile and web applications. While Cognito can integrate with identity providers, it's not the direct and preferred method for integrating a corporate directory service with multiple AWS accounts in a multi-account architecture, compared to IAM Identity Center.

C: Configure a service control policy (SCP) to manage the AWS accounts. Add AWS IAM Identity Center (AWS Single Sign-On) to AWS Directory Service. While SCPs are important for governance, you don't add IAM Identity Center to AWS Directory Service. Instead, you integrate IAM Identity Center with the directory service, configuring IAM Identity Center to use the directory as its identity source. This integration is managed within IAM Identity Center, not the directory service itself. You cannot simply "add" IAM Identity Center in such a manner.

D: Create a new organization in AWS Organizations. Configure the organization's authentication mechanism to use AWS Directory Service directly. AWS Organizations itself doesn't have a direct authentication mechanism to directly integrate with a directory service. The authentication is facilitated via a service like AWS IAM Identity Center. Organizations provides the framework to manage the accounts, and IAM Identity Center handles the authentication across them.

Question: 485 **Exam Heist**

A company is looking for a solution that can store video archives in AWS from old news footage. The company needs to minimize costs and will rarely need to restore these files. When the files are needed, they must be available in a maximum of five minutes.

What is the MOST cost-effective solution?

A.Store the video archives in Amazon S3 Glacier and use Expedited retrievals.
B.Store the video archives in Amazon S3 Glacier and use Standard retrievals.
C.Store the video archives in Amazon S3 Standard-Infrequent Access (S3 Standard-IA).
D.Store the video archives in Amazon S3 One Zone-Infrequent Access (S3 One Zone-IA).

Answer: A

Explanation:

The correct answer is A because it offers the most cost-effective storage option for infrequently accessed data that needs to be retrieved within five minutes. Amazon S3 Glacier is designed for archiving data at a very low cost.

Here's a detailed breakdown:

Cost-Effectiveness: S3 Glacier is significantly cheaper than S3 Standard-IA and S3 One Zone-IA for storage.

The company's primary requirement is to minimize costs, making Glacier a strong contender.

Retrieval Time: The key differentiator is the retrieval time requirement of "a maximum of five minutes." While Glacier has different retrieval options, "Expedited retrievals" allow for access to data within 1-5 minutes.

Why not B? Standard retrievals in S3 Glacier can take several hours (3-5 hours), which violates the five-minute requirement.

Why not C? S3 Standard-IA is more expensive than S3 Glacier for storage. While retrieval times are faster by default, the cost is not justified for archives rarely needed.

Why not D? S3 One Zone-IA is cheaper than S3 Standard-IA but still more expensive than S3 Glacier. Additionally, S3 One Zone-IA stores data in only one Availability Zone, which makes it less durable than S3 Glacier. Durability is implicitly desired when archiving old, important footage, and using Glacier is a better practice for data longevity.

In summary, Amazon S3 Glacier with Expedited retrievals perfectly balances the need for low-cost storage with the specific requirement of quick access when needed.

Authoritative Links for further research:

Amazon S3 Glacier: https://aws.amazon.com/glacier/
S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
S3 Glacier Retrieval Options: https://docs.aws.amazon.com/AmazonS3/latest/userguide/retrieving-objects-glacier.html

Question: 486 **Exam Heist**

A company is building a three-tier application on AWS. The presentation tier will serve a static website The logic tier is a containerized application. This application will store data in a relational database. The company wants to simplify deployment and to reduce operational costs.

Which solution will meet these requirements?

A.Use Amazon S3 to host static content. Use Amazon Elastic Container Service (Amazon ECS) with AWS Fargate for compute power. Use a managed Amazon RDS cluster for the database.

B.Use Amazon CloudFront to host static content. Use Amazon Elastic Container Service (Amazon ECS) with Amazon EC2 for compute power. Use a managed Amazon RDS cluster for the database.

C.Use Amazon S3 to host static content. Use Amazon Elastic Kubernetes Service (Amazon EKS) with AWS Fargate for compute power. Use a managed Amazon RDS cluster for the database.

D.Use Amazon EC2 Reserved Instances to host static content. Use Amazon Elastic Kubernetes Service (Amazon EKS) with Amazon EC2 for compute power. Use a managed Amazon RDS cluster for the database.

Answer: A

Explanation:

The correct answer is A. Here's why:

Static Website Hosting: Amazon S3 is a cost-effective and scalable solution for hosting static websites. It offers high availability and durability without requiring server management. https://aws.amazon.com/s3/

Containerized Application: AWS Fargate allows you to run containers without managing the underlying EC2 instances. It simplifies deployment and reduces operational overhead, perfectly aligning with the requirement to reduce operational costs. Amazon Elastic Container Service (ECS) is a fully managed container orchestration service that supports Fargate. https://aws.amazon.com/fargate/

Relational Database: Amazon RDS provides managed relational database services, reducing the operational burden of database administration. It automates tasks like patching, backups, and scaling. https://aws.amazon.com/rds/

Let's look at why the other options are less ideal:

Option B: Using Amazon CloudFront solely for static content is overkill. While CloudFront is a great CDN for caching and distributing content globally, it primarily adds value on top of a static storage solution like S3. For a basic static website, S3 alone is usually sufficient and more cost-effective. Also, while ECS with EC2 is valid, Fargate is preferable for reducing operational costs.

Option C: Amazon EKS (Kubernetes) is generally more complex than ECS, especially when deploying it with Fargate. While EKS with Fargate is a perfectly acceptable option for microservice architectures and applications requiring portability, it might be an unnecessary complexity layer for a simple three-tier application. Choosing ECS simplifies setup and management.

Option D: Using Amazon EC2 Reserved Instances for static content is the least optimal. EC2 instances require constant management, and they are not designed to serve static content, while S3 is. Also, EKS with EC2 increases complexity and operational overhead compared to ECS with Fargate.

Question: 487 **Exam Heist**

A company seeks a storage solution for its application. The solution must be highly available and scalable. The solution also must function as a file system be mountable by multiple Linux instances in AWS and on premises through native protocols, and have no minimum size requirements. The company has set up a Site-to-Site VPN for access from its on-premises network to its VPC.

Which storage solution meets these requirements?

A.Amazon FSx Multi-AZ deployments
B.Amazon Elastic Block Store (Amazon EBS) Multi-Attach volumes
C.Amazon Elastic File System (Amazon EFS) with multiple mount targets
D.Amazon Elastic File System (Amazon EFS) with a single mount target and multiple access points

Answer: C

Explanation:

The correct answer is C: Amazon Elastic File System (Amazon EFS) with multiple mount targets. Here's why:

Amazon EFS is designed to provide a scalable, elastic, and highly available network file system. Crucially, EFS supports multiple Linux instances mounting the file system concurrently. This directly addresses the requirement for a file system mountable by multiple Linux instances in AWS and on-premises. EFS seamlessly integrates with a Site-to-Site VPN, enabling access from the on-premises network. EFS is also elastic, meaning its storage capacity automatically grows and shrinks as you add and remove files, thus fulfilling the "no minimum size requirements" criterion.

Multiple mount targets in different Availability Zones (AZs) within a region enhance availability. If one AZ experiences an issue, instances in other AZs can still access the file system.

Option A, Amazon FSx Multi-AZ deployments, although highly available, is designed for Windows file servers or Lustre, not general-purpose file systems mountable by Linux instances using standard file protocols across on-premises and cloud environments.

Option B, Amazon Elastic Block Store (Amazon EBS) Multi-Attach volumes, allows attaching one EBS volume

to multiple instances. However, it's restricted to specific Nitro-based instances and, importantly, doesn't offer direct support for accessing the file system from on-premises environments via Site-to-Site VPN. It's also block storage and doesn't behave as a network file system inherently.

Option D, Amazon Elastic File System (Amazon EFS) with a single mount target and multiple access points, while utilizing EFS, a single mount target would create a single point of failure and not provide the necessary high availability across Availability Zones, which is crucial. While access points enhance security and simplify access management, they do not replace the need for multiple mount targets for availability.

Therefore, only Amazon EFS with multiple mount targets provides the required scalability, availability, file system functionality, and on-premises access via Site-to-Site VPN, satisfying all the conditions outlined in the scenario.

Further reading:

Amazon EFS: https://aws.amazon.com/efs/
EFS Mount Targets: https://docs.aws.amazon.com/efs/latest/ug/mounting-fs-mount-targets.html
EFS Access from On-premises: https://docs.aws.amazon.com/efs/latest/ug/accessing-fs-nfs-vpc.html

Question: 488 **Exam Heist**

A 4-year-old media company is using the AWS Organizations all features feature set to organize its AWS accounts. According to the company's finance team, the billing information on the member accounts must not be accessible to anyone, including the root user of the member accounts.

Which solution will meet these requirements?

A.Add all finance team users to an IAM group. Attach an AWS managed policy named Billing to the group.

B.Attach an identity-based policy to deny access to the billing information to all users, including the root user.

C.Create a service control policy (SCP) to deny access to the billing information. Attach the SCP to the root organizational unit (OU).

D.Convert from the Organizations all features feature set to the Organizations consolidated billing feature set.

Answer: C

Explanation:

The correct answer is **C. Create a service control policy (SCP) to deny access to the billing information. Attach the SCP to the root organizational unit (OU).**

Here's why:

Service Control Policies (SCPs) are the ideal mechanism to enforce organization-wide governance in AWS Organizations. They act as guardrails, defining the maximum permissions that member accounts within an OU can have. Even the root user of a member account cannot bypass restrictions imposed by an SCP applied to the OU they belong to. This makes SCPs the only option to reliably restrict access to billing information, even for root users.

Option A is incorrect because assigning finance team users to an IAM group and attaching an AWS managed policy would grant them access, not restrict it.

Option B is incorrect because IAM policies within a member account cannot override restrictions enforced by SCPs at the organizational level. The root user could potentially modify or detach the policy.

Option D, switching to the consolidated billing feature set, does not, by itself, restrict access to billing information within member accounts. It just consolidates billing for the organization into the management

account. The consolidated billing setup does not offer control over who views billing data within member accounts. The all features set is required to use SCPs, so the idea of converting to the consolidated billing feature set is antithetical to the problem at hand.

Attaching the SCP to the root OU ensures that the restriction applies to all accounts within the organization (unless explicitly overridden in a lower-level OU, which wouldn't meet the requirements). By denying access to billing information through an SCP, the organization ensures compliance with the finance team's requirement that member accounts' billing data remains inaccessible. This is essential for maintaining financial control and preventing unauthorized access within the AWS environment.

Further reading:

AWS Organizations documentation on SCPs:
https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html
AWS Organizations features: https://aws.amazon.com/organizations/features/

Question: 489 **Exam Heist**

An ecommerce company runs an application in the AWS Cloud that is integrated with an on-premises warehouse solution. The company uses Amazon Simple Notification Service (Amazon SNS) to send order messages to an on-premises HTTPS endpoint so the warehouse application can process the orders. The local data center team has detected that some of the order messages were not received.

A solutions architect needs to retain messages that are not delivered and analyze the messages for up to 14 days.

Which solution will meet these requirements with the LEAST development effort?

A.Configure an Amazon SNS dead letter queue that has an Amazon Kinesis Data Stream target with a retention period of 14 days.

B.Add an Amazon Simple Queue Service (Amazon SQS) queue with a retention period of 14 days between the application and Amazon SNS.

C.Configure an Amazon SNS dead letter queue that has an Amazon Simple Queue Service (Amazon SQS) target with a retention period of 14 days.

D.Configure an Amazon SNS dead letter queue that has an Amazon DynamoDB target with a TTL attribute set for a retention period of 14 days.

Answer: C

Explanation:

The best solution to retain undelivered SNS messages for up to 14 days with the least development effort is to use a dead-letter queue (DLQ) backed by Amazon Simple Queue Service (SQS).

Here's why:

Dead-Letter Queues (DLQs): SNS dead-letter queues are specifically designed for handling undeliverable messages. When SNS fails to deliver a message to a subscriber after retries, it can be configured to send the message to a DLQ. This ensures that messages are not lost and can be analyzed later.

Amazon SQS: SQS is a fully managed message queuing service that allows you to decouple and scale microservices, distributed systems, and serverless applications. SQS offers message retention periods of up to 14 days, which directly fulfills the requirement.

Least Development Effort: SNS natively supports configuring DLQs with SQS as the target. This is a configuration-based solution, requiring minimal code development. You simply configure the SNS subscription with the DLQ ARN.

Let's analyze why other options are less suitable:

Option A (Kinesis Data Streams): While Kinesis Data Streams can store data, it is more suited for real-time data processing and analytics. Using it solely for retaining undelivered messages adds unnecessary complexity. The primary purpose of Kinesis is not message queuing with retention for analysis like SQS.

Option B (SQS queue between application and SNS): This option doesn't directly address the SNS dead-letter queue concept. It introduces an intermediary queue, which could complicate the message flow and require changes to the application that sends messages to SNS. It's not a direct solution to the problem of messages failing delivery from SNS to the on-premises endpoint.

Option D (DynamoDB with TTL): DynamoDB with TTL (Time To Live) could technically store the messages, but it involves more development effort. You would need to write code to take the messages from the SNS failure notification, format them, and store them in DynamoDB. Additionally, configuring TTL attributes and ensuring proper data formatting adds overhead. Storing messages intended for queuing in a database seems unnatural.

In summary, option C leverages native AWS features (SNS DLQ and SQS) to provide a simple, configuration-based solution for message retention with minimal development effort. SQS queue retention period and SNS DLQ configurations are straightforward within the AWS console or using Infrastructure as Code (IaC).

Authoritative Links:

Amazon SNS Dead-Letter Queues: https://docs.aws.amazon.com/sns/latest/dg/sns-attribute-delivery-policy.html
Amazon SQS Message Retention: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-configure-message-retention-period.html

Question: 490 **Exam Heist**

A gaming company uses Amazon DynamoDB to store user information such as geographic location, player data, and leaderboards. The company needs to configure continuous backups to an Amazon S3 bucket with a minimal amount of coding. The backups must not affect availability of the application and must not affect the read capacity units (RCUs) that are defined for the table.

Which solution meets these requirements?

A.Use an Amazon EMR cluster. Create an Apache Hive job to back up the data to Amazon S3.

B.Export the data directly from DynamoDB to Amazon S3 with continuous backups. Turn on point-in-time recovery for the table.

C.Configure Amazon DynamoDB Streams. Create an AWS Lambda function to consume the stream and export the data to an Amazon S3 bucket.

D.Create an AWS Lambda function to export the data from the database tables to Amazon S3 on a regular basis. Turn on point-in-time recovery for the table.

Answer: B

Explanation:

The correct answer is B because it directly leverages built-in DynamoDB features for backups with minimal coding and no impact on application availability or RCUs. Let's break down why the other options are not as suitable:

Option A (EMR/Hive): Using EMR and Hive is overkill. It requires significant setup, configuration, and coding to create the Hive job and manage the cluster. This introduces complexity and operational overhead that can

be avoided. https://aws.amazon.com/emr/

Option C (DynamoDB Streams/Lambda): DynamoDB Streams captures item-level changes. While it can be used for data replication, it's not designed for full table backups. Using Lambda to consume the stream and export data to S3 requires custom coding to assemble the full table state and handle potential inconsistencies. Also, data stored in S3 might not be consistent with the table state. https://aws.amazon.com/dynamodb/streams/

Option D (Lambda/Periodic Export): While Lambda can be used for data export, scheduling it on a regular basis might impact the table's performance and RCU consumption, depending on the frequency and size of the data being exported. This method requires custom coding and careful monitoring to avoid impacting application availability. Also, it only provides point-in-time recovery for the time when you scheduled backups and is not truly continuous. https://aws.amazon.com/lambda/

Option B leverages DynamoDB's native export functionality directly to S3 with continuous backups using Point-in-Time Recovery (PITR).

Export to S3: DynamoDB allows you to export a table's data directly to an S3 bucket. This functionality is managed by DynamoDB and avoids impacting the table's RCU.
Point-in-Time Recovery (PITR): PITR allows you to restore your DynamoDB table to any point in time during the preceding 35 days. This provides continuous backup capabilities because it continuously maintains backup data. Turning on PITR provides automatic and continuous backups without affecting application performance. It does not consume RCUs.
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/PointInTimeRecovery.html and https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/S3Export.html
Minimal Coding: Turning on PITR and starting an export to S3 requires minimal configuration through the AWS Management Console, AWS CLI, or SDK. It does not require writing custom code.

In summary, option B offers the simplest, most efficient, and most reliable way to back up DynamoDB data to S3 without impacting application availability or RCUs by utilizing the built-in DynamoDB features for export and PITR.

Question: 491 **Exam Heist**

A solutions architect is designing an asynchronous application to process credit card data validation requests for a bank. The application must be secure and be able to process each request at least once.

Which solution will meet these requirements MOST cost-effectively?

A.Use AWS Lambda event source mapping. Set Amazon Simple Queue Service (Amazon SQS) standard queues as the event source. Use AWS Key Management Service (SSE-KMS) for encryption. Add the kms:Decrypt permission for the Lambda execution role.

B.Use AWS Lambda event source mapping. Use Amazon Simple Queue Service (Amazon SQS) FIFO queues as the event source. Use SQS managed encryption keys (SSE-SQS) for encryption. Add the encryption key invocation permission for the Lambda function.

C.Use the AWS Lambda event source mapping. Set Amazon Simple Queue Service (Amazon SQS) FIFO queues as the event source. Use AWS KMS keys (SSE-KMS). Add the kms:Decrypt permission for the Lambda execution role.

D.Use the AWS Lambda event source mapping. Set Amazon Simple Queue Service (Amazon SQS) standard queues as the event source. Use AWS KMS keys (SSE-KMS) for encryption. Add the encryption key invocation permission for the Lambda function.

Answer: A

Explanation:

Here's a detailed justification for why option A is the most cost-effective solution for processing credit card data validation requests asynchronously, securely, and at least once, using Lambda and SQS:

Key Requirements and Considerations:

Asynchronous Processing: Decoupling the request origin from the processing is essential for scalability and responsiveness. SQS enables this.
Security: Protecting sensitive credit card data requires encryption at rest.
At-Least-Once Delivery: Guarantees that each request is processed, even in failure scenarios.
Cost-Effectiveness: Minimizing operational expenses is a crucial design goal.

Why Option A is the Best Choice:

Option A leverages SQS standard queues with AWS Lambda event source mapping, offering a balance of functionality and cost. Let's break down the components:

1. **SQS Standard Queues:** Standard queues provide high throughput and best-effort ordering, which is suitable for credit card validation as the order of individual validations isn't as critical as the speed and ability to process all requests. Standard queues are generally cheaper than FIFO queues. https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-queue-types.html
2. **AWS Lambda Event Source Mapping:** This feature directly connects the SQS queue to the Lambda function. Lambda automatically polls the queue for messages and invokes the function whenever messages are available. This eliminates the need for custom polling logic or additional services. https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html
3. **SSE-KMS Encryption:** Server-Side Encryption with KMS provides robust encryption at rest using keys managed by AWS Key Management Service (KMS). This ensures that sensitive credit card data is protected while stored in the SQS queue. https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-configure-sse.html
4. **kms:Decrypt Permission:** Granting the Lambda execution role the kms:Decrypt permission allows the Lambda function to decrypt messages encrypted with SSE-KMS. This is essential for the Lambda function to process the encrypted data. https://docs.aws.amazon.com/kms/latest/developerguide/services-sqs.html

Why Other Options are Less Suitable:

Options B and C (FIFO Queues): FIFO (First-In-First-Out) queues guarantee message ordering, but come with higher cost and lower throughput than standard queues. Ordering is not a critical requirement, using FIFO adds unnecessary cost. The at-least-once delivery guarantee can still be maintained with standard queues using retry mechanisms and dead-letter queues (DLQs).

Option B (SSE-SQS): SSE-SQS uses SQS managed encryption keys, which is simpler to set up but offers less control and flexibility compared to SSE-KMS. SSE-KMS is preferred for sensitive data because you can manage key rotation, access control, and auditing policies more granularly. Additionally, the permission in the lambda execution role to allow KMS Decryption.

Option D: Option D describes using KMS but grants "encryption key invocation permission" to the Lambda function. Instead, it must be kms:Decrypt permission for the Lambda execution role, as the Lambda role performs decryption.

Cost Considerations:

Standard queues are generally more cost-effective than FIFO queues due to their higher throughput and less

stringent ordering requirements. While both SSE-SQS and SSE-KMS incur encryption costs, the additional control and auditing capabilities of SSE-KMS are often justified for sensitive data. Lambda event source mapping is a cost-effective way to integrate SQS with Lambda because it eliminates the need for constant polling or a separate polling service.

In summary, Option A provides the necessary security, delivery guarantees, and asynchronous processing capabilities in a cost-optimized manner by leveraging SQS standard queues with SSE-KMS encryption and Lambda event source mapping with the correct KMS decryption permission.

Question: 492 **Exam Heist**

A company has multiple AWS accounts for development work. Some staff consistently use oversized Amazon EC2 instances, which causes the company to exceed the yearly budget for the development accounts. The company wants to centrally restrict the creation of AWS resources in these accounts.

Which solution will meet these requirements with the LEAST development effort?

A.Develop AWS Systems Manager templates that use an approved EC2 creation process. Use the approved Systems Manager templates to provision EC2 instances.

B.Use AWS Organizations to organize the accounts into organizational units (OUs). Define and attach a service control policy (SCP) to control the usage of EC2 instance types.

C.Configure an Amazon EventBridge rule that invokes an AWS Lambda function when an EC2 instance is created. Stop disallowed EC2 instance types.

D.Set up AWS Service Catalog products for the staff to create the allowed EC2 instance types. Ensure that staff can deploy EC2 instances only by using the Service Catalog products.

Answer: B

Explanation:

The most efficient solution to centrally restrict EC2 instance creation in multiple AWS development accounts involves using AWS Organizations and Service Control Policies (SCPs). AWS Organizations allows you to organize multiple AWS accounts into organizational units (OUs). By applying an SCP to an OU, you can restrict the actions that IAM users and roles in the member accounts can perform.

In this case, you can create an SCP that denies the creation of EC2 instances beyond a specific instance type size or family. This centralized control mechanism ensures that developers within the development accounts cannot launch oversized EC2 instances that exceed the budget. This approach minimizes development effort since SCPs are declarative policies that are easily defined and attached to OUs without requiring custom coding or complex infrastructure. Options A, C, and D require more development and maintenance effort compared to leveraging the native capabilities of AWS Organizations and SCPs for centralized governance. While Systems Manager templates (Option A) and Service Catalog (Option D) can enforce instance type choices, they don't provide central control across multiple accounts as effectively as Organizations. Option C, using EventBridge and Lambda, is reactive and requires continuous monitoring and updates, making it less efficient than SCPs. SCPs are proactively applied, preventing non-compliant instances from being created in the first place.

Authoritative links:

AWS Organizations: https://aws.amazon.com/organizations/
Service Control Policies (SCPs):
https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html

Question: 493 **Exam Heist**

A company wants to use artificial intelligence (AI) to determine the quality of its customer service calls. The company currently manages calls in four different languages, including English. The company will offer new languages in the future. The company does not have the resources to regularly maintain machine learning (ML) models.

The company needs to create written sentiment analysis reports from the customer service call recordings. The customer service call recording text must be translated into English.

Which combination of steps will meet these requirements? (Choose three.)

A.Use Amazon Comprehend to translate the audio recordings into English.
B.Use Amazon Lex to create the written sentiment analysis reports.
C.Use Amazon Polly to convert the audio recordings into text.
D.Use Amazon Transcribe to convert the audio recordings in any language into text.
E.Use Amazon Translate to translate text in any language to English.
F.Use Amazon Comprehend to create the sentiment analysis reports.

Answer: DEF

Explanation:

The correct answer is D, E, and F. Here's why:

D: Use Amazon Transcribe to convert the audio recordings in any language into text. Amazon Transcribe is a speech-to-text service that can handle multiple languages. It's designed to transcribe audio from customer service calls into text, a crucial first step for sentiment analysis. This aligns directly with the requirement of converting call recordings into text, regardless of the language. https://aws.amazon.com/transcribe/

E: Use Amazon Translate to translate text in any language to English. Since the requirement is to translate the call recording text into English for sentiment analysis, Amazon Translate is the appropriate service. It offers automated translation from various source languages into the target language (English). This addresses the company's need to handle calls in multiple languages and future expansion into new languages. https://aws.amazon.com/translate/

F: Use Amazon Comprehend to create the sentiment analysis reports. Amazon Comprehend is a natural language processing (NLP) service that can perform sentiment analysis on text. The translated text from the call recordings can be fed into Comprehend, which will analyze the sentiment (positive, negative, neutral) and generate the required reports. Comprehend requires no ML model maintenance, satisfying that constraint. https://aws.amazon.com/comprehend/

Now, let's explain why the other options are incorrect:

A: Use Amazon Comprehend to translate the audio recordings into English. Comprehend is a text-based NLP service, not an audio translation service. It can't directly translate audio; it requires text as input.
B: Use Amazon Lex to create the written sentiment analysis reports. Amazon Lex is a service for building conversational interfaces (chatbots). While Lex can integrate with other services to perform sentiment analysis, it's not directly designed for generating sentiment analysis reports from call recordings.
C: Use Amazon Polly to convert the audio recordings into text. Amazon Polly is a text-to-speech service, which performs the opposite function of what is needed. It converts text into audio, rather than audio into text.

Question: 494 **Exam Heist**

A company uses Amazon EC2 instances to host its internal systems. As part of a deployment operation, an administrator tries to use the AWS CLI to terminate an EC2 instance. However, the administrator receives a 403

(Access Denied) error message.

The administrator is using an IAM role that has the following IAM policy attached:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": ["ec2:TerminateInstances"],
            "Resource": ["*"]
        },
        {
            "Effect": "Deny",
            "Action": ["ec2:TerminateInstances"],
            "Condition": {
                "NotIpAddress": {
                    "aws:SourceIp": [
                        "192.0.2.0/24",
                        "203.0.113.0/24"
                    ]
                }
            },
            "Resource": ["*"]
        }
    ]
}
```

What is the cause of the unsuccessful request?

A.The EC2 instance has a resource-based policy with a Deny statement.

B.The principal has not been specified in the policy statement.

C.The "Action" field does not grant the actions that are required to terminate the EC2 instance.

D.The request to terminate the EC2 instance does not originate from the CIDR blocks 192.0.2.0/24 or 203.0.113.0/24.

Answer: D

Explanation:

The request to terminate the EC2 instance does not originate from the CIDR blocks 192.0.2.0/24 or 203.0.113.0/24.

Question: 495 **Exam Heist**

A company is conducting an internal audit. The company wants to ensure that the data in an Amazon S3 bucket that is associated with the company's AWS Lake Formation data lake does not contain sensitive customer or employee data. The company wants to discover personally identifiable information (PII) or financial information, including passport numbers and credit card numbers.

Which solution will meet these requirements?

A.Configure AWS Audit Manager on the account. Select the Payment Card Industry Data Security Standards (PCI DSS) for auditing.

B.Configure Amazon S3 Inventory on the S3 bucket Configure Amazon Athena to query the inventory.

C.Configure Amazon Macie to run a data discovery job that uses managed identifiers for the required data types.

D.Use Amazon S3 Select to run a report across the S3 bucket.

Answer: C

Explanation:

The best solution for discovering sensitive data within an S3 bucket linked to AWS Lake Formation, specifically PII and financial information like passport and credit card numbers, is **C. Configure Amazon Macie to run a data discovery job that uses managed identifiers for the required data types.**

Here's why:

Amazon Macie is a fully managed data security and data privacy service that uses machine learning and pattern matching to discover and protect sensitive data stored in Amazon S3. It is designed for this specific purpose. https://aws.amazon.com/macie/
Macie provides pre-defined **managed identifiers** for common types of PII and financial data, including credit card numbers, passport numbers, social security numbers, and more. This allows for out-of-the-box detection capabilities tailored to the company's requirements.
Macie's data discovery jobs scan the contents of S3 objects and identify instances of sensitive data based on the configured managed identifiers. It can handle various file formats and storage classes.
Macie integrates with other AWS services, enabling automated remediation actions based on its findings.
Option A is incorrect because AWS Audit Manager is primarily used for compliance auditing against predefined frameworks, not for specific data discovery within an S3 bucket. PCI DSS is relevant if the data is related to payment card information, but Audit Manager does not directly scan the S3 bucket contents for PII like Macie does.
Option B is incorrect because Amazon S3 Inventory provides a list of objects in the bucket with metadata (size, modification date, etc.). Athena can then be used to query the inventory, but the inventory data does not include the contents of the files. Thus, it would not be able to detect PII inside files.
Option D is incorrect because Amazon S3 Select allows querying the contents of S3 objects using SQL. While technically possible, it would require a significant amount of manual SQL query crafting and maintenance to identify all potential PII patterns, compared to Macie's managed approach. It also requires knowledge of the data schemas stored within the S3 bucket.
Lake Formation provides a central repository for data catalog and access controls, but it doesn't offer built-in sensitive data discovery. Macie's discovery integrates well with S3 buckets associated with Lake Formation data lakes. Macie directly analyzes the data at rest to identify patterns that match sensitive data, making it the most effective and efficient choice.

Question: 496 **Exam Heist**

A company uses on-premises servers to host its applications. The company is running out of storage capacity. The applications use both block storage and NFS storage. The company needs a high-performing solution that supports local caching without re-architecting its existing applications.

Which combination of actions should a solutions architect take to meet these requirements? (Choose two.)

A.Mount Amazon S3 as a file system to the on-premises servers.

B.Deploy an AWS Storage Gateway file gateway to replace NFS storage.

C.Deploy AWS Snowball Edge to provision NFS mounts to on-premises servers.

D.Deploy an AWS Storage Gateway volume gateway to replace the block storage.

E.Deploy Amazon Elastic File System (Amazon EFS) volumes and mount them to on-premises servers.

Answer: BD

Explanation:

The company's requirements include addressing storage capacity issues for both block and NFS storage on-premises, achieving high performance with local caching, and minimizing application re-architecting.

Option B, deploying an AWS Storage Gateway file gateway to replace NFS storage, directly addresses the need for NFS storage expansion. A file gateway allows on-premises applications to access data stored in Amazon S3 as NFS file shares. Critically, it provides a local cache, which improves performance by storing frequently accessed data locally, thereby reducing latency. This is a core function of file gateways. [https://aws.amazon.com/storagegateway/file-gateway/]

Option D, deploying an AWS Storage Gateway volume gateway to replace block storage, directly addresses the need for block storage expansion. A volume gateway enables on-premises applications to access cloud-based block storage volumes as iSCSI devices. Like the file gateway, it provides a local cache for frequently accessed data. This ensures that applications experience low-latency access for frequently used data while offloading infrequently used data to the cloud. The volume gateway comes in two flavors - cached volumes which stores all data in S3 and copies a cache of frequently accessed data locally. This ensures local low-latency and Stored Volumes - keeps the entire data copy locally, backed up asynchronously to AWS. Since the question specifies the need for high performance that supports local caching, the Cached Volumes is implicitly implied. [https://aws.amazon.com/storagegateway/volume-gateway/]

Option A is incorrect because directly mounting Amazon S3 as a file system to on-premises servers typically doesn't offer the high-performance local caching required. Although tools exist to achieve this, they often introduce complexity. Option C is inappropriate. AWS Snowball Edge is primarily for large-scale data migrations and edge computing and not generally used for continuous NFS mounts. Option E is incorrect because while Amazon EFS can be mounted on-premises via AWS Direct Connect or VPN, it does not offer the local caching mechanism needed for high performance in this scenario. Also it doesn't address block storage requirements.

Question: 497 **Exam Heist**

A company has a service that reads and writes large amounts of data from an Amazon S3 bucket in the same AWS Region. The service is deployed on Amazon EC2 instances within the private subnet of a VPC. The service communicates with Amazon S3 over a NAT gateway in the public subnet. However, the company wants a solution that will reduce the data output costs.

Which solution will meet these requirements MOST cost-effectively?

A.Provision a dedicated EC2 NAT instance in the public subnet. Configure the route table for the private subnet to use the elastic network interface of this instance as the destination for all S3 traffic.

B.Provision a dedicated EC2 NAT instance in the private subnet. Configure the route table for the public subnet

to use the elastic network interface of this instance as the destination for all S3 traffic.

C.Provision a VPC gateway endpoint. Configure the route table for the private subnet to use the gateway endpoint as the route for all S3 traffic.

D.Provision a second NAT gateway. Configure the route table for the private subnet to use this NAT gateway as the destination for all S3 traffic.

Answer: C

Explanation:

The most cost-effective solution to reduce data output costs for EC2 instances accessing S3 within the same AWS Region is to use a VPC gateway endpoint for S3. VPC gateway endpoints allow EC2 instances in private subnets to access S3 directly, bypassing the NAT gateway and the associated data transfer charges.

Option A is incorrect because while using a dedicated EC2 NAT instance might offer slightly more control, it doesn't eliminate data transfer charges and introduces management overhead.

Option B is incorrect because placing a NAT instance in the private subnet would not solve the problem of data transfer costs and would violate common best practices.

Option D is incorrect because provisioning a second NAT gateway would double the NAT gateway costs and wouldn't address the data transfer charges between EC2 and S3.

Option C, provisioning a VPC gateway endpoint, is the best solution. VPC gateway endpoints are free to use (you only pay for the S3 storage and requests), and they provide a direct, secure connection to S3 without traversing the public internet. This eliminates data transfer charges associated with using a NAT gateway and reduces overall costs. The route table configuration directs S3-bound traffic through the endpoint, ensuring a direct connection.

Here are some authoritative links for further research:

VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html
NAT Gateways: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html

Question: 498 **Exam Heist**

A company uses Amazon S3 to store high-resolution pictures in an S3 bucket. To minimize application changes, the company stores the pictures as the latest version of an S3 object. The company needs to retain only the two most recent versions of the pictures.

The company wants to reduce costs. The company has identified the S3 bucket as a large expense.

Which solution will reduce the S3 costs with the LEAST operational overhead?

A.Use S3 Lifecycle to delete expired object versions and retain the two most recent versions.

B.Use an AWS Lambda function to check for older versions and delete all but the two most recent versions.

C.Use S3 Batch Operations to delete noncurrent object versions and retain only the two most recent versions.

D.Deactivate versioning on the S3 bucket and retain the two most recent versions.

Answer: A

Explanation:

The correct answer is A: Use S3 Lifecycle to delete expired object versions and retain the two most recent versions. Here's why:

S3 Lifecycle policies are designed specifically to automate the management of objects over their lifetime,

including transitioning them to different storage classes or deleting them after a specified period or based on versioning rules. In this scenario, the requirement is to reduce S3 costs by retaining only the two most recent versions of the pictures while minimizing operational overhead. S3 Lifecycle rules can be configured to automatically delete noncurrent versions of objects, allowing you to specify the number of recent versions to keep. This directly addresses the requirement.

Option B (Lambda function) would involve writing and maintaining custom code, which increases operational overhead and complexity. While Lambda is powerful, it's overkill for a task that S3 Lifecycle is designed to handle. Option C (S3 Batch Operations) is suitable for performing large-scale batch operations on S3 objects. While it could technically be used to delete older versions, it involves more setup and is less efficient for continuous version management compared to Lifecycle policies. Option D (deactivating versioning) would completely remove the ability to retain multiple versions, which is contrary to the requirement of keeping the two most recent versions. Therefore, it is not a suitable option.

S3 Lifecycle policies are built into S3 and require minimal configuration. They automatically and continuously manage object versions, resulting in the least operational overhead. Lifecycle rules can be configured from the S3 Management Console or using the AWS CLI/SDKs, making the process simple and straightforward. By using S3 Lifecycle rules to delete older versions, the company can automatically reduce storage costs without continuous manual intervention.

In conclusion, S3 Lifecycle is the most efficient and cost-effective solution for managing object versions in S3 while minimizing operational overhead.

Authoritative Links:

Amazon S3 Lifecycle documentation
Managing your storage lifecycle

Question: 499 **Exam Heist**

A company needs to minimize the cost of its 1 Gbps AWS Direct Connect connection. The company's average connection utilization is less than 10%. A solutions architect must recommend a solution that will reduce the cost without compromising security.

Which solution will meet these requirements?

A.Set up a new 1 Gbps Direct Connect connection. Share the connection with another AWS account.

B.Set up a new 200 Mbps Direct Connect connection in the AWS Management Console.

C.Contact an AWS Direct Connect Partner to order a 1 Gbps connection. Share the connection with another AWS account.

D.Contact an AWS Direct Connect Partner to order a 200 Mbps hosted connection for an existing AWS account.

Answer: D

Explanation:

The correct answer is D because it directly addresses the company's need to minimize cost while maintaining security and connectivity.

Here's why:

Cost Reduction: The company's average utilization is only 10% of the 1 Gbps connection, indicating significant over-provisioning. Reducing the connection speed to 200 Mbps aligns the bandwidth with actual usage, leading to lower costs.

AWS Direct Connect Partner for Hosted Connection: Contacting an AWS Direct Connect Partner is

necessary to obtain a hosted connection. AWS directly provides dedicated connections (1 Gbps and above), but for lower speeds, you must work through a partner.

Hosted Connection Advantage: Hosted connections are generally more cost-effective for lower bandwidth requirements compared to dedicated connections. The partner handles the physical infrastructure and shares the cost among multiple customers.

Security Maintenance: Switching to a 200 Mbps hosted connection doesn't inherently compromise security. Security is managed through Virtual Private Gateways (VGWs), Direct Connect Gateways, and VPC configurations, which remain independent of the physical connection speed.

Existing AWS Account: Using an existing AWS account simplifies the setup and integration process. No new accounts or complex cross-account configurations are needed.

Why other options are incorrect:

A: Setting up another 1 Gbps connection and sharing it doesn't reduce costs; it duplicates them. Sharing also adds complexity and potential security concerns.
B: Setting up a 200 Mbps connection directly in the AWS Management Console isn't possible; AWS only provides higher speed connections directly, lower speeds are available through partners.
C: Similar to A, setting up another 1 Gbps connection doesn't solve the over-provisioning issue. Sharing with another AWS account introduces complexity. While AWS Direct Connect Partners provide connections, a 1 Gbps connection isn't needed when the average utilization is 10%.

In summary: Answer D provides the optimal balance between cost reduction and security by scaling the connection speed down to match utilization through a cost-effective hosted connection obtained via an AWS Direct Connect Partner.

Supporting Links:

AWS Direct Connect Pricing: https://aws.amazon.com/directconnect/pricing/ (Illustrates cost differences based on connection speed)
AWS Direct Connect Partners: https://aws.amazon.com/directconnect/partners/
AWS Direct Connect FAQs: https://aws.amazon.com/directconnect/faqs/ (Clarifies dedicated vs. hosted connections)

Question: 500 **Exam Heist**

A company has multiple Windows file servers on premises. The company wants to migrate and consolidate its files into an Amazon FSx for Windows File Server file system. File permissions must be preserved to ensure that access rights do not change.

Which solutions will meet these requirements? (Choose two.)

A.Deploy AWS DataSync agents on premises. Schedule DataSync tasks to transfer the data to the FSx for Windows File Server file system.

B.Copy the shares on each file server into Amazon S3 buckets by using the AWS CLI. Schedule AWS DataSync tasks to transfer the data to the FSx for Windows File Server file system.

C.Remove the drives from each file server. Ship the drives to AWS for import into Amazon S3. Schedule AWS DataSync tasks to transfer the data to the FSx for Windows File Server file system.

D.Order an AWS Snowcone device. Connect the device to the on-premises network. Launch AWS DataSync agents on the device. Schedule DataSync tasks to transfer the data to the FSx for Windows File Server file system.

E.Order an AWS Snowball Edge Storage Optimized device. Connect the device to the on-premises network. Copy data to the device by using the AWS CLI. Ship the device back to AWS for import into Amazon S3. Schedule AWS DataSync tasks to transfer the data to the FSx for Windows File Server file system.

Answer: AD

Explanation:

The question requires selecting solutions for migrating on-premises Windows file servers to Amazon FSx for Windows File Server while preserving file permissions.

Option A: Deploy AWS DataSync agents on premises. Schedule DataSync tasks to transfer the data to the FSx for Windows File Server file system. This is a valid option because AWS DataSync is specifically designed for online data transfer between on-premises storage and AWS storage services, including FSx for Windows File Server. DataSync preserves file metadata and permissions during the transfer, fulfilling the requirement. Deploying agents on-premises allows DataSync to access the file servers and transfer the data efficiently.

Option D: Order an AWS Snowcone device. Connect the device to the on-premises network. Launch AWS DataSync agents on the device. Schedule DataSync tasks to transfer the data to the FSx for Windows File Server file system. This is also a valid option, especially when dealing with limited network bandwidth or a need to minimize network impact during the migration. Snowcone is a small, ruggedized, and secure edge computing and data transfer device. By launching DataSync agents directly on Snowcone, the data transfer process can be accelerated by minimizing the reliance on the network connectivity. DataSync, as in Option A, maintains permissions. The Snowcone would need to be connected to the network.

Why other options are incorrect:

Option B: Copying shares to S3 using the AWS CLI and then using DataSync is not the correct approach because the native file share structure and NTFS permissions would be lost during the S3 transfer. S3 is an object storage, not a file system. The conversion to an object storage format would discard the metadata required.
Option C: Shipping drives to AWS for import into S3 is not the correct approach for the same reason as option B. Removing the drives from the file servers and shipping them is also likely to be extremely disruptive and impractical. It won't preserve the file permissions and requires physical handling.
Option E: This option is also not appropriate for the same reasons as Option B. The AWS CLI copy to the Snowball and subsequent transfer to S3 results in the loss of file permissions and NTFS metadata. Furthermore, it is an unnecessary step.

Key Concepts:

AWS DataSync: An online data transfer service that simplifies, automates, and accelerates the secure movement of data between on-premises storage and AWS storage services.
Amazon FSx for Windows File Server: A fully managed native Microsoft Windows file system built on Windows Server.
AWS Snowcone: A small, rugged, and secure edge computing and data transfer device.
File Permissions Preservation: Maintaining the original access control settings (NTFS permissions) of files during migration.

Authoritative Links:

AWS DataSync: https://aws.amazon.com/datasync/
Amazon FSx for Windows File Server: https://aws.amazon.com/fsx/windows/
AWS Snowcone: https://aws.amazon.com/snowcone/

Question: 501 **Exam Heist**

A company wants to ingest customer payment data into the company's data lake in Amazon S3. The company receives payment data every minute on average. The company wants to analyze the payment data in real time. Then the company wants to ingest the data into the data lake.

Which solution will meet these requirements with the MOST operational efficiency?

A.Use Amazon Kinesis Data Streams to ingest data. Use AWS Lambda to analyze the data in real time.

B.Use AWS Glue to ingest data. Use Amazon Kinesis Data Analytics to analyze the data in real time.

C.Use Amazon Kinesis Data Firehose to ingest data. Use Amazon Kinesis Data Analytics to analyze the data in real time.

D.Use Amazon API Gateway to ingest data. Use AWS Lambda to analyze the data in real time.

Answer: C

Explanation:

Here's a detailed justification for why option C is the most operationally efficient solution for ingesting and analyzing real-time payment data into an S3 data lake:

Option C: Amazon Kinesis Data Firehose and Amazon Kinesis Data Analytics

Kinesis Data Firehose: Kinesis Data Firehose is designed specifically for loading streaming data into data lakes and data stores like S3. It's fully managed, requiring minimal administration. It automatically scales to handle the data volume and provides built-in features for data transformation, compression, and encryption. It can deliver directly to S3 with configurable buffering, which optimizes costs and performance. https://aws.amazon.com/kinesis/data-firehose/

Kinesis Data Analytics: Kinesis Data Analytics is a powerful service for real-time data stream processing. It allows you to write SQL queries or use Apache Flink to analyze streaming data as it arrives. It provides low-latency results and integrates seamlessly with other AWS services. https://aws.amazon.com/kinesis/data-analytics/

Why other options are less optimal:

Option A (Kinesis Data Streams & Lambda): While Kinesis Data Streams can ingest data, using Lambda for real-time analysis for every payment event might be expensive and less efficient than Kinesis Data Analytics. Lambda functions have execution time limitations, and managing concurrency and scaling can be complex for continuous, high-volume streams. Lambda doesn't inherently integrate with S3 for data lake population. https://aws.amazon.com/lambda/ https://aws.amazon.com/kinesis/data-streams/

Option B (Glue & Kinesis Data Analytics): AWS Glue is primarily for ETL (Extract, Transform, Load) operations and data cataloging, not real-time data ingestion. While Kinesis Data Analytics is suitable for analyzing data once ingested, Glue would not satisfy the requirement to have data ingested on a minute-by-minute basis. Glue is more suited for batch processing. https://aws.amazon.com/glue/

Option D (API Gateway & Lambda): API Gateway is designed to manage APIs, not stream high volumes of data directly into a data lake. Using API Gateway to forward every payment to Lambda and then to S3 would add unnecessary overhead and cost. API Gateway is not meant for ingestion of high velocity data. https://aws.amazon.com/api-gateway/

In summary:

Option C provides the most streamlined and cost-effective approach. Kinesis Data Firehose efficiently handles the ingestion of streaming data into S3, and Kinesis Data Analytics provides a platform for real-time processing, meeting both real-time analysis and data lake population requirements with minimal operational overhead. The managed nature of these services reduces the operational burden on the company.

Question: 502 **Exam Heist**

A company runs a website that uses a content management system (CMS) on Amazon EC2. The CMS runs on a single EC2 instance and uses an Amazon Aurora MySQL Multi-AZ DB instance for the data tier. Website images are stored on an Amazon Elastic Block Store (Amazon EBS) volume that is mounted inside the EC2 instance.

Which combination of actions should a solutions architect take to improve the performance and resilience of the website? (Choose two.)

A.Move the website images into an Amazon S3 bucket that is mounted on every EC2 instance

B.Share the website images by using an NFS share from the primary EC2 instance. Mount this share on the other EC2 instances.

C.Move the website images onto an Amazon Elastic File System (Amazon EFS) file system that is mounted on every EC2 instance.

D.Create an Amazon Machine Image (AMI) from the existing EC2 instance. Use the AMI to provision new instances behind an Application Load Balancer as part of an Auto Scaling group. Configure the Auto Scaling group to maintain a minimum of two instances. Configure an accelerator in AWS Global Accelerator for the website

E.Create an Amazon Machine Image (AMI) from the existing EC2 instance. Use the AMI to provision new instances behind an Application Load Balancer as part of an Auto Scaling group. Configure the Auto Scaling group to maintain a minimum of two instances. Configure an Amazon CloudFront distribution for the website.

Answer: CE

Explanation:

The correct answer is **C and E**. Here's why:

C. Move the website images onto an Amazon Elastic File System (Amazon EFS) file system that is mounted on every EC2 instance.

Performance and Scalability: Storing images on a single EBS volume tied to a single EC2 instance creates a single point of failure and performance bottleneck. EFS provides a shared file system that can be simultaneously accessed by multiple EC2 instances. This allows the website to scale horizontally and improve performance by distributing the image serving load across multiple servers.
Resilience: If the original EC2 instance fails, the images are still available because they are stored on EFS, which is designed for high availability and durability. EFS replicates data across multiple Availability Zones.
Shared Storage: EFS is designed for scenarios where multiple instances need to access the same data.
https://aws.amazon.com/efs/

E. Create an Amazon Machine Image (AMI) from the existing EC2 instance. Use the AMI to provision new instances behind an Application Load Balancer as part of an Auto Scaling group. Configure the Auto Scaling group to maintain a minimum of two instances. Configure an Amazon CloudFront distribution for the website.

Resilience: An Auto Scaling group with a minimum of two instances ensures that the website remains available even if one instance fails. The Application Load Balancer distributes traffic across healthy instances.
Performance and Scalability: The Application Load Balancer distributes incoming traffic across multiple EC2 instances, improving response times and handling increased load. Auto Scaling allows the website to automatically scale up or down based on traffic demands.
Content Delivery: CloudFront caches website content, including images, at edge locations around the world. This reduces latency for users and offloads traffic from the origin server (EC2 instances).
https://aws.amazon.com/cloudfront/
AMI for Consistent Deployment: Creating an AMI from the existing instance ensures that new instances are provisioned with the same configuration and software as the original instance, reducing the risk of inconsistencies. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AMIs.html

Why other options are less suitable:

A: Mounting an S3 bucket directly on EC2 instances is generally not recommended for frequently accessed files like website images due to performance limitations. S3 is object storage, not a file system.
B: NFS shares are complex to manage and can become single points of failure. EFS is a managed service and much simpler.
D: While creating an AMI and using an ALB with an ASG is good, configuring an accelerator does not address content delivery to end users as well as a CDN.

Question: 503 **Exam Heist**

A company runs an infrastructure monitoring service. The company is building a new feature that will enable the service to monitor data in customer AWS accounts. The new feature will call AWS APIs in customer accounts to describe Amazon EC2 instances and read Amazon CloudWatch metrics.

What should the company do to obtain access to customer accounts in the MOST secure way?

A.Ensure that the customers create an IAM role in their account with read-only EC2 and CloudWatch permissions and a trust policy to the company's account.

B.Create a serverless API that implements a token vending machine to provide temporary AWS credentials for a role with read-only EC2 and CloudWatch permissions.

C.Ensure that the customers create an IAM user in their account with read-only EC2 and CloudWatch permissions. Encrypt and store customer access and secret keys in a secrets management system.

D.Ensure that the customers create an Amazon Cognito user in their account to use an IAM role with read-only EC2 and CloudWatch permissions. Encrypt and store the Amazon Cognito user and password in a secrets management system.

Answer: A

Explanation:

The most secure way for the monitoring service to access customer AWS accounts is through IAM roles with trust policies. Option A proposes that customers create an IAM role in their account granting read-only access to EC2 and CloudWatch, with a trust policy that allows the company's AWS account to assume the role. This approach, known as cross-account access using IAM roles, is the best practice for granting permissions between AWS accounts without sharing long-term credentials. The customer retains control over the permissions granted to the monitoring service and can revoke access at any time by modifying or deleting the IAM role. The monitoring service uses the AWS Security Token Service (STS) to assume the role, receiving temporary credentials that are used to access resources in the customer account.

Option B is less suitable because it involves creating and managing a serverless API to vend temporary credentials. While this may provide temporary credentials, it adds unnecessary complexity and introduces a potential point of failure and security risk. Option C is highly insecure because it involves creating IAM users in customer accounts and storing their long-term access keys, which presents a significant security risk if the keys are compromised. Option D introduces Amazon Cognito, which is typically used for authenticating users, not for cross-account access in this scenario. Cognito does not provide a mechanism for allowing access to AWS resources in a different account using credentials.

Therefore, Option A provides the most secure and efficient method to access customer accounts because it adheres to the principle of least privilege and utilizes IAM roles for cross-account access, eliminating the need to share long-term credentials.

Relevant links:

IAM Roles for cross-account access
AWS STS

Question: 504 **Exam Heist**

A company needs to connect several VPCs in the us-east-1 Region that span hundreds of AWS accounts. The company's networking team has its own AWS account to manage the cloud network.

What is the MOST operationally efficient solution to connect the VPCs?

A.Set up VPC peering connections between each VPC. Update each associated subnet's route table

B.Configure a NAT gateway and an internet gateway in each VPC to connect each VPC through the internet

C.Create an AWS Transit Gateway in the networking team's AWS account. Configure static routes from each VPC.

D.Deploy VPN gateways in each VPC. Create a transit VPC in the networking team's AWS account to connect to each VPC.

Answer: C

Explanation:

The most operationally efficient solution for connecting hundreds of VPCs across multiple AWS accounts in the us-east-1 Region, with a centralized networking team managing the cloud network, is to use AWS Transit Gateway (TGW).

Option C is correct because TGW acts as a hub, simplifying VPC connectivity. Instead of creating numerous peer-to-peer connections, each VPC attaches to the TGW. This significantly reduces the management overhead and complexity associated with VPC peering, which would quickly become unmanageable with hundreds of VPCs, and the associated route table maintenance (as stated in option A). TGW is designed for this type of use case. It is a central hub in the networking team's account simplifies network management and routing across the many VPCs that can be connected via the TGW attachment. Route tables within the TGW manage the traffic between the VPCs.

Option A is not operationally efficient. Managing hundreds of VPC peering connections and constantly updating associated route tables introduces considerable overhead and a high risk of misconfiguration. The administrative burden scales quadratically with the number of VPCs.

Option B, using NAT gateways and internet gateways to connect VPCs over the internet, is insecure and inefficient. This exposes traffic to the public internet and does not leverage the private networking capabilities within AWS. Also, egress costs would be significant.

Option D, utilizing VPN gateways and a transit VPC, is an older approach compared to Transit Gateway and adds complexity. While a transit VPC can centralize VPN connections, it requires managing instances and routing within the transit VPC, which increases operational burden, and introduces potential bottlenecks. TGW provides a more scalable and managed solution than a transit VPC.

In summary, Transit Gateway provides the most efficient and scalable solution for connecting numerous VPCs across many AWS accounts because it centralizes network management, simplifies routing, and reduces operational complexity compared to VPC peering, internet-based connections, or transit VPCs.

Authoritative Links:

AWS Transit Gateway: https://aws.amazon.com/transit-gateway/
AWS Transit Gateway Documentation: https://docs.aws.amazon.com/transit-gateway/index.html

Question: 505 **Exam Heist**

A company has Amazon EC2 instances that run nightly batch jobs to process data. The EC2 instances run in an Auto Scaling group that uses On-Demand billing. If a job fails on one instance, another instance will reprocess the job. The batch jobs run between 12:00 AM and 06:00 AM local time every day.

Which solution will provide EC2 instances to meet these requirements MOST cost-effectively?

A.Purchase a 1-year Savings Plan for Amazon EC2 that covers the instance family of the Auto Scaling group that the batch job uses.

B.Purchase a 1-year Reserved Instance for the specific instance type and operating system of the instances in the Auto Scaling group that the batch job uses.

C.Create a new launch template for the Auto Scaling group. Set the instances to Spot Instances. Set a policy to scale out based on CPU usage.

D.Create a new launch template for the Auto Scaling group. Increase the instance size. Set a policy to scale out based on CPU usage.

Answer: C

Explanation:

The most cost-effective solution is **C. Create a new launch template for the Auto Scaling group. Set the instances to Spot Instances. Set a policy to scale out based on CPU usage.**

Here's why:

Spot Instances: Spot Instances offer significant cost savings (up to 90% compared to On-Demand) by utilizing spare EC2 capacity. The nightly batch job processing, which tolerates interruptions (as evidenced by the job reprocessing mechanism), is a perfect use case for Spot Instances. If a Spot Instance is terminated due to price fluctuations, another instance will automatically launch to reprocess the failed job, maintaining reliability.

Launch Template: Using a launch template simplifies instance configuration and allows easy updates to the Auto Scaling group.

Scaling Policy based on CPU: Scaling out based on CPU usage ensures that instances are launched only when needed to handle the batch job's workload, optimizing resource utilization and minimizing costs. Scaling policies linked to CPU utilization offer direct and adaptive responses to workload demands.

Why other options are less ideal:

A. Savings Plan: While Savings Plans offer discounts, they commit you to a consistent usage level over a longer period. Since the batch jobs only run for 6 hours each night, the commitment for the other 18 hours could lead to wasted resources.

B. Reserved Instances: Similar to Savings Plans, Reserved Instances commit you to a specific instance type and operating system for a longer period. The risk of underutilization is also present, although less than the savings plan, given the batch job's limited duration.

D. Increasing Instance Size: Increasing the instance size will lead to higher costs for the entire 6 hour run time. Scaling out based on CPU with the correct sized instances offers a more granular approach to costs.

Authoritative Links:

Spot Instances: https://aws.amazon.com/ec2/spot/
Auto Scaling Groups: https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-groups.html
Launch Templates: https://docs.aws.amazon.com/autoscaling/ec2/userguide/launch-templates.html
Savings Plan: https://aws.amazon.com/savingsplans/
Reserved Instances: https://aws.amazon.com/ec2/pricing/reserved-instances/

In summary, leveraging Spot Instances within an Auto Scaling group, configured through a launch template, and scaling based on CPU usage provides the most cost-effective approach for the given scenario because it aligns with the application's tolerance for interruption and the specific time window of operation.

Question: 506 **Exam Heist**

A social media company is building a feature for its website. The feature will give users the ability to upload photos. The company expects significant increases in demand during large events and must ensure that the website can handle the upload traffic from users.

Which solution meets these requirements with the MOST scalability?

A.Upload files from the user's browser to the application servers. Transfer the files to an Amazon S3 bucket.

B.Provision an AWS Storage Gateway file gateway. Upload files directly from the user's browser to the file gateway.

C.Generate Amazon S3 presigned URLs in the application. Upload files directly from the user's browser into an S3 bucket.

D.Provision an Amazon Elastic File System (Amazon EFS) file system. Upload files directly from the user's browser to the file system.

Answer: C

Explanation:

The most scalable solution for handling user photo uploads in a social media application with expected surges in traffic is option C: generating Amazon S3 presigned URLs. Here's why:

Direct Upload to S3: Presigned URLs allow users to upload files directly to Amazon S3 without passing through the application servers. This offloads the upload traffic from the application tier, freeing up resources and significantly improving scalability.

Scalability of S3: S3 is designed for virtually unlimited scalability and handles massive amounts of data and requests. It automatically scales to meet demand without requiring any manual intervention. https://aws.amazon.com/s3/

Reduced Application Server Load: By bypassing the application servers, the solution minimizes their load, enabling them to focus on other critical tasks like processing and serving user requests. This ensures the application remains responsive during peak upload periods.

Cost Efficiency: S3 is cost-effective for storing large amounts of data. Since users upload directly to S3, you only pay for the storage and data transfer used, avoiding costs associated with application server resources for handling the uploads.

Security: Presigned URLs are time-limited and provide granular control over upload permissions. The application controls the parameters of the presigned URL, defining the allowed actions (e.g., uploading), expiration time, and target bucket/object.

Other options are less scalable:

A (Upload through Application Servers): This bottlenecks the application servers, making it less scalable, particularly during peak upload periods.
B (AWS Storage Gateway): While Storage Gateway can provide on-premises access to cloud storage, it is less suitable for directly handling high-volume browser uploads. It introduces additional complexity and potential latency.
D (Amazon EFS): EFS is designed for shared file storage access across EC2 instances and not for direct client

uploads. EFS also typically has higher costs compared to S3 for simple storage of user uploads.

Therefore, generating S3 presigned URLs offers the optimal combination of scalability, cost-effectiveness, and security for handling large-scale user photo uploads.

Question: 507 **Exam Heist**

A company has a web application for travel ticketing. The application is based on a database that runs in a single data center in North America. The company wants to expand the application to serve a global user base. The company needs to deploy the application to multiple AWS Regions. Average latency must be less than 1 second on updates to the reservation database.

The company wants to have separate deployments of its web platform across multiple Regions. However, the company must maintain a single primary reservation database that is globally consistent.

Which solution should a solutions architect recommend to meet these requirements?

A.Convert the application to use Amazon DynamoDB. Use a global table for the center reservation table. Use the correct Regional endpoint in each Regional deployment.

B.Migrate the database to an Amazon Aurora MySQL database. Deploy Aurora Read Replicas in each Region. Use the correct Regional endpoint in each Regional deployment for access to the database.

C.Migrate the database to an Amazon RDS for MySQL database. Deploy MySQL read replicas in each Region. Use the correct Regional endpoint in each Regional deployment for access to the database.

D.Migrate the application to an Amazon Aurora Serverless database. Deploy instances of the database to each Region. Use the correct Regional endpoint in each Regional deployment to access the database. Use AWS Lambda functions to process event streams in each Region to synchronize the databases.

Answer: A

Explanation:

The best solution is **A. Convert the application to use Amazon DynamoDB. Use a global table for the center reservation table. Use the correct Regional endpoint in each Regional deployment.**

Here's why:

Global Consistency: The key requirement is maintaining a single, globally consistent reservation database. Amazon DynamoDB Global Tables are specifically designed for this purpose. They provide multi-Region, multi-active database replication, ensuring data consistency across multiple Regions with low latency. https://aws.amazon.com/dynamodb/global-tables/
Low Latency Updates: DynamoDB Global Tables are designed for low-latency global writes. Writes to any Region are replicated to other Regions, typically within milliseconds.
Regional Deployments: Using Regional endpoints in each deployment allows the application to access the DynamoDB Global Table from the nearest Region, minimizing latency for both reads and writes.
Alternatives are unsuitable: Options B, C and D use a relational database. While Aurora and RDS offer read replicas for improved read performance in different Regions, they struggle to meet the requirement for a single, globally consistent, writable database with low-latency updates. Aurora Serverless in option D introduces complexity by using Lambda functions for data synchronization. This does not align well with the consistency and latency requirement of this application.

In summary, DynamoDB Global Tables are perfectly suited for applications requiring a single, globally consistent database with low-latency writes across multiple Regions, making option A the correct choice.

Question: 508 **Exam Heist**

A company has migrated multiple Microsoft Windows Server workloads to Amazon EC2 instances that run in the us-west-1 Region. The company manually backs up the workloads to create an image as needed.

In the event of a natural disaster in the us-west-1 Region, the company wants to recover workloads quickly in the us-west-2 Region. The company wants no more than 24 hours of data loss on the EC2 instances. The company also wants to automate any backups of the EC2 instances.

Which solutions will meet these requirements with the LEAST administrative effort? (Choose two.)

A.Create an Amazon EC2-backed Amazon Machine Image (AMI) lifecycle policy to create a backup based on tags. Schedule the backup to run twice daily. Copy the image on demand.

B.Create an Amazon EC2-backed Amazon Machine Image (AMI) lifecycle policy to create a backup based on tags. Schedule the backup to run twice daily. Configure the copy to the us-west-2 Region.

C.Create backup vaults in us-west-1 and in us-west-2 by using AWS Backup. Create a backup plan for the EC2 instances based on tag values. Create an AWS Lambda function to run as a scheduled job to copy the backup data to us-west-2.

D.Create a backup vault by using AWS Backup. Use AWS Backup to create a backup plan for the EC2 instances based on tag values. Define the destination for the copy as us-west-2. Specify the backup schedule to run twice daily.

E.Create a backup vault by using AWS Backup. Use AWS Backup to create a backup plan for the EC2 instances based on tag values. Specify the backup schedule to run twice daily. Copy on demand to us-west-2.

Answer: BD

Explanation:

The correct answer is **BD**. Here's a detailed justification:

Option B: Creating an AMI lifecycle policy with twice-daily backups and automated copying to us-west-2 directly addresses the requirements. AMI lifecycle policies provide a managed way to automate AMI creation and deletion based on tags, minimizing administrative effort. Configuring the copy to us-west-2 ensures a replicated image for disaster recovery. The twice-daily schedule ensures that the Recovery Point Objective (RPO) of 24 hours is met (or exceeded). This option leverages native AWS features for backups and replication, leading to a simple and efficient setup.https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/image-lifecycle.html

Option D: AWS Backup is a centralized backup service that offers automated and managed backups across various AWS services, including EC2. By creating a backup plan based on tags and specifying us-west-2 as the destination for the copy, the company automates the backup and replication process in a single solution. Configuring twice-daily backups directly fulfills the RPO requirement. This provides ease of management and consistent backup policies across AWS resources.https://aws.amazon.com/backup/

Why other options are incorrect:

A: This is not the most efficient. Copying on demand is an administrative overhead.
C: Using a Lambda function to copy data to us-west-2 adds complexity and overhead compared to using AWS Backup's built-in cross-region copy feature. It's extra administrative burden.
E: Copying on demand is an administrative overhead. AWS Backup offers a simpler automated approach for cross-region replication.

Question: 509 **Exam Heist**

A company operates a two-tier application for image processing. The application uses two Availability Zones, each with one public subnet and one private subnet. An Application Load Balancer (ALB) for the web tier uses the public subnets. Amazon EC2 instances for the application tier use the private subnets.

Users report that the application is running more slowly than expected. A security audit of the web server log files shows that the application is receiving millions of illegitimate requests from a small number of IP addresses. A solutions architect needs to resolve the immediate performance problem while the company investigates a more permanent solution.

What should the solutions architect recommend to meet this requirement?

A.Modify the inbound security group for the web tier. Add a deny rule for the IP addresses that are consuming resources.

B.Modify the network ACL for the web tier subnets. Add an inbound deny rule for the IP addresses that are consuming resources.

C.Modify the inbound security group for the application tier. Add a deny rule for the IP addresses that are consuming resources.

D.Modify the network ACL for the application tier subnets. Add an inbound deny rule for the IP addresses that are consuming resources.

Answer: B

Explanation:

The correct answer is **B. Modify the network ACL for the web tier subnets. Add an inbound deny rule for the IP addresses that are consuming resources.**

Here's why:

Network ACLs (NACLs) vs. Security Groups: Both control network traffic, but they operate at different layers and have different characteristics. Security Groups are stateful and operate at the instance level, allowing you to control traffic to and from individual instances. NACLs, on the other hand, are stateless and operate at the subnet level, allowing you to control traffic entering and exiting subnets. Because NACLs work at the subnet level and are stateless, they are a more efficient and immediate way to block traffic from specific IP addresses affecting the entire web tier.

Immediate Performance Problem: The question emphasizes resolving the immediate performance problem. NACLs are processed before Security Groups, so changes to NACLs take effect immediately. Security Group changes might take longer to propagate and have less immediate impact across all instances.

Web Tier Focus: The illegitimate requests are hitting the web tier (ALB in public subnets). The fastest way to mitigate this is to block the traffic at the point of entry to the web tier subnets.

Deny Rule for Problematic IPs: Adding an inbound deny rule to the NACL for the web tier subnets will immediately block the malicious traffic from reaching the ALB and the EC2 instances behind it, freeing up resources and improving application performance.

Why other options are less suitable:

A. Modify the inbound security group for the web tier: Security groups are stateful and operate at the instance level. While this would block traffic, it's less efficient and potentially less immediate than a NACL change, especially if the number of instances is large.

C. Modify the inbound security group for the application tier: The illegitimate requests are targeting the web tier, not the application tier directly. Blocking traffic at the application tier would not prevent the web tier from being overloaded.

D. Modify the network ACL for the application tier subnets: Similar to option C, the application tier is not the primary target. Blocking traffic at the application tier would not resolve the immediate performance problem at the web tier.

Supporting Concepts:

Defense in Depth: This scenario illustrates the concept of defense in depth, where multiple layers of security are implemented to protect resources. NACLs and Security Groups provide complementary security controls.
Network Segmentation: Subnets provide a way to segment your network and control traffic flow.

Authoritative Links:

AWS Documentation on Network ACLs
AWS Documentation on Security Groups

Question: 510 **Exam Heist**

A global marketing company has applications that run in the ap-southeast-2 Region and the eu-west-1 Region. Applications that run in a VPC in eu-west-1 need to communicate securely with databases that run in a VPC in ap-southeast-2.

Which network design will meet these requirements?

A.Create a VPC peering connection between the eu-west-1 VPC and the ap-southeast-2 VPC. Create an inbound rule in the eu-west-1 application security group that allows traffic from the database server IP addresses in the ap-southeast-2 security group.

B.Configure a VPC peering connection between the ap-southeast-2 VPC and the eu-west-1 VPC. Update the subnet route tables. Create an inbound rule in the ap-southeast-2 database security group that references the security group ID of the application servers in eu-west-1.

C.Configure a VPC peering connection between the ap-southeast-2 VPC and the eu-west-1 VPUpdate the subnet route tables. Create an inbound rule in the ap-southeast-2 database security group that allows traffic from the eu-west-1 application server IP addresses.

D.Create a transit gateway with a peering attachment between the eu-west-1 VPC and the ap-southeast-2 VPC. After the transit gateways are properly peered and routing is configured, create an inbound rule in the database security group that references the security group ID of the application servers in eu-west-1.

Answer: C

Explanation:

The correct answer is C. Here's a detailed justification:

VPC Peering creates a direct networking connection between two VPCs, enabling them to route traffic between each other privately. This is a suitable option when you need to connect two VPCs in different regions. For VPC Peering to work, route tables in each VPC must be updated to point to the peer VPC's CIDR block. This allows traffic to be routed correctly. Security groups control inbound and outbound traffic at the instance level. To allow traffic from the eu-west-1 application servers to the ap-southeast-2 database servers, the database security group in ap-southeast-2 needs an inbound rule. This rule should specifically allow traffic from the CIDR block or the specific IP addresses of the application servers in eu-west-1. Using security group IDs in the other region is not supported and won't work due to the regional boundary.

Option A is incorrect because while VPC peering is valid, referencing a security group from another region's security group is invalid. Security Groups can only reference other Security Groups within the same AWS region and VPC.

Option B is incorrect for the same reason as option A; Security Groups can only reference other Security Groups within the same AWS region and VPC.

Option D is incorrect because while Transit Gateway can connect VPCs, it's generally more complex and costly than VPC peering for a simple two-VPC connection. A Transit Gateway introduces unnecessary complexity and costs when a simple peering connection will suffice. Also, just like option A and B, referencing security group IDs in another region does not work.

Here are some helpful links for further research:

VPC Peering: https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html
Security Groups: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html
Transit Gateway: https://docs.aws.amazon.com/vpc/latest/tgw/what-is-transit-gateway.html

Question: 511 **Exam Heist**

A company is developing software that uses a PostgreSQL database schema. The company needs to configure multiple development environments and databases for the company's developers. On average, each development environment is used for half of the 8-hour workday.

Which solution will meet these requirements MOST cost-effectively?

A.Configure each development environment with its own Amazon Aurora PostgreSQL database
B.Configure each development environment with its own Amazon RDS for PostgreSQL Single-AZ DB instances
C.Configure each development environment with its own Amazon Aurora On-Demand PostgreSQL-Compatible database
D.Configure each development environment with its own Amazon S3 bucket by using Amazon S3 Object Select

Answer: C

Explanation:

The most cost-effective solution is **C. Configure each development environment with its own Amazon Aurora On-Demand PostgreSQL-Compatible database.**

Here's why:

Cost Optimization: Aurora On-Demand allows you to pay only for the compute resources your database consumes, per second, with no minimum charges. This aligns perfectly with the requirement that each development environment is used for only half of the workday. During the idle periods, the database cost will be significantly reduced, leading to substantial savings compared to always-on RDS or Aurora instances.

Aurora PostgreSQL Compatibility: Aurora PostgreSQL is wire-compatible with standard PostgreSQL, ensuring the company's software works seamlessly without code modifications.

Development Environment Isolation: Each developer gets their isolated database environment, preventing conflicts and ensuring independent development.

Scalability and Performance: Even in on-demand mode, Aurora PostgreSQL offers excellent performance and scalability compared to Single-AZ RDS instances.

Single-AZ RDS Inefficiency: RDS Single-AZ instances (option B) are always running, regardless of usage, making them less cost-effective when instances are idle for significant portions of the day.

Aurora vs RDS: Aurora is designed for high availability and performance than RDS.

S3 Object Select Inapplicability: Amazon S3 Object Select (option D) is used for querying data within S3 objects and is completely irrelevant for setting up a PostgreSQL database for development environments.

Aurora Provisioned Instances Inefficiency: Option A, using standard Aurora, would be expensive since the instances would be charged regardless of whether they are being used or not.

In Summary: On-demand Aurora PostgreSQL clusters enable users to launch a fully compliant and completely isolated version of Aurora PostgreSQL in minutes. You can immediately reduce the cost of running non-production databases by only paying for the capacity you consume.

Further reading on Aurora On-Demand can be found here:

AWS Aurora On-Demand Pricing

Question: 512 **Exam Heist**

A company uses AWS Organizations with resources tagged by account. The company also uses AWS Backup to back up its AWS infrastructure resources. The company needs to back up all AWS resources.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use AWS Config to identify all untagged resources. Tag the identified resources programmatically. Use tags in the backup plan.

B.Use AWS Config to identify all resources that are not running. Add those resources to the backup vault.

C.Require all AWS account owners to review their resources to identify the resources that need to be backed up.

D.Use Amazon Inspector to identify all noncompliant resources.

Answer: A

Explanation:

The correct answer is A. Here's why:

AWS Organizations and Tagging: The company already leverages AWS Organizations and tags resources by account. This establishes a foundation for organized resource management.
AWS Backup and Tag-Based Backups: AWS Backup natively supports tag-based backups. This allows you to dynamically include resources in backup plans based on their tags, minimizing manual configuration.
Identifying Untagged Resources with AWS Config: The problem states that the company needs to back up all resources. Therefore, any resources not currently tagged will not be backed up using their existing method. AWS Config can continuously assess your AWS resources, making it ideal for finding resources without specific tags.
Programmatic Tagging: Once AWS Config identifies untagged resources, you can use scripts (e.g., AWS CLI, SDKs) to apply the necessary tags programmatically. This ensures consistent tagging and avoids manual errors.
Least Operational Overhead: This approach minimizes manual effort and ongoing maintenance. By automating the tagging process, you avoid the need for constant manual checks and updates to backup plans. Options C and D would add complexity in the form of manual review or focus on security rather than backup coverage.
Incorrect Options:

B: AWS Config can identify many different aspects of the AWS environment, however, this is an over-engineered solution that does not directly address the tagging requirement.
C: This places a burden on individual account owners to remember to identify and tag their respective resources. This is also error prone, time consuming, and adds a large amount of operational overhead.
D: Amazon Inspector is designed for security vulnerability management, not resource inventory or tagging.
Supporting Documentation:

AWS Backup Tag-Based Backups: https://docs.aws.amazon.com/aws-backup/latest/devguide/tags.html
AWS Config: https://aws.amazon.com/config/

Question: 513 **Exam Heist**

A social media company wants to allow its users to upload images in an application that is hosted in the AWS Cloud. The company needs a solution that automatically resizes the images so that the images can be displayed on multiple device types. The application experiences unpredictable traffic patterns throughout the day. The company is seeking a highly available solution that maximizes scalability.

What should a solutions architect do to meet these requirements?

A.Create a static website hosted in Amazon S3 that invokes AWS Lambda functions to resize the images and store the images in an Amazon S3 bucket.

B.Create a static website hosted in Amazon CloudFront that invokes AWS Step Functions to resize the images and store the images in an Amazon RDS database.

C.Create a dynamic website hosted on a web server that runs on an Amazon EC2 instance. Configure a process that runs on the EC2 instance to resize the images and store the images in an Amazon S3 bucket.

D.Create a dynamic website hosted on an automatically scaling Amazon Elastic Container Service (Amazon ECS) cluster that creates a resize job in Amazon Simple Queue Service (Amazon SQS). Set up an image-resizing program that runs on an Amazon EC2 instance to process the resize jobs.

Answer: A

Explanation:

The correct answer is **A**. Here's why:

Scalability and High Availability: AWS Lambda is a serverless compute service, meaning it automatically scales to handle fluctuating workloads without requiring management of underlying infrastructure. Amazon S3 provides highly durable and scalable object storage, ideal for storing images.
Cost-Effectiveness: Lambda functions are billed based on execution time, making it a cost-effective solution for unpredictable traffic patterns. S3 offers various storage classes that can be used to optimize costs based on access frequency.
Suitability for Image Processing: Lambda functions can be easily triggered by S3 events (e.g., object creation), allowing for automated image resizing upon upload.
Static Website Hosting: S3 can host static websites directly, serving the front-end of the application.
Option B is incorrect: AWS Step Functions are for orchestrating complex workflows, which is overkill for simple image resizing. Amazon RDS is a relational database service and not suitable for storing images directly.
Option C is incorrect: Running a process on an EC2 instance requires managing the instance and its scaling, which adds complexity and is less cost-effective compared to Lambda. EC2 might not scale quickly enough for unpredictable traffic.
Option D is incorrect: While ECS can scale, it involves managing container clusters. SQS adds complexity that is unnecessary for a straightforward image resizing task. It's more complex and costly than using Lambda directly.

Authoritative Links:

AWS Lambda: https://aws.amazon.com/lambda/
Amazon S3: https://aws.amazon.com/s3/
Serverless Architectures: https://aws.amazon.com/serverless/

Question: 514 **Exam Heist**

A company is running a microservices application on Amazon EC2 instances. The company wants to migrate the application to an Amazon Elastic Kubernetes Service (Amazon EKS) cluster for scalability. The company must configure the Amazon EKS control plane with endpoint private access set to true and endpoint public access set to false to maintain security compliance. The company must also put the data plane in private subnets. However, the company has received error notifications because the node cannot join the cluster.

Which solution will allow the node to join the cluster?

A.Grant the required permission in AWS Identity and Access Management (IAM) to the AmazonEKSNodeRole IAM role.

B.Create interface VPC endpoints to allow nodes to access the control plane.

C.Recreate nodes in the public subnet. Restrict security groups for EC2 nodes.

D.Allow outbound traffic in the security group of the nodes.

Answer: B

Explanation:

The correct answer is **B. Create interface VPC endpoints to allow nodes to access the control plane.**

Here's a detailed justification:

The problem states that the EKS control plane has private access enabled and public access disabled. This means that the control plane's API server is only accessible from within the VPC where the EKS cluster resides. The worker nodes, which form the data plane, are in private subnets. Therefore, they don't have direct internet access (unless a NAT gateway is configured, which isn't mentioned).

Since the nodes cannot reach the control plane directly via public internet, and public access is disabled on the control plane, they need a private network path. This is achieved using VPC endpoints. Specifically, interface VPC endpoints provide private connectivity to AWS services within your VPC, without requiring internet access or a NAT gateway. By creating an interface VPC endpoint for EKS, the worker nodes in the private subnets can communicate with the EKS control plane over the AWS private network, enabling them to join the cluster.

Let's examine why the other options are less suitable:

A. Grant the required permission in AWS Identity and Access Management (IAM) to the AmazonEKSNodeRole IAM role: IAM permissions control authorization, not network connectivity. While the nodes need the correct IAM permissions to interact with the cluster, this won't solve the underlying problem of the nodes being unable to reach the control plane due to network isolation.

C. Recreate nodes in the public subnet. Restrict security groups for EC2 nodes: Moving the nodes to public subnets and then attempting to lock them down with security groups goes against the initial requirement of maintaining security compliance. Moreover, it creates an unnecessary public exposure surface, increasing the security risk. The initial setup was attempting to avoid public exposure, and this option reverts that.

D. Allow outbound traffic in the security group of the nodes: Allowing outbound traffic (even if restricted) is not enough. With the control plane set to private access only, outbound traffic from the nodes needs to reach the control plane privately. Simply allowing general outbound traffic doesn't establish the necessary private connection. Outbound traffic alone will likely still try to reach the public endpoint, which is disabled. You need a dedicated path.

In summary, VPC endpoints provide a secure and compliant way for nodes in private subnets to communicate with the EKS control plane when public access is disabled.

Supporting Links:

Amazon EKS Cluster Endpoint Access Control
VPC Endpoints

Question: 515 **Exam Heist**

A company is migrating an on-premises application to AWS. The company wants to use Amazon Redshift as a

solution.

Which use cases are suitable for Amazon Redshift in this scenario? (Choose three.)

A.Supporting data APIs to access data with traditional, containerized, and event-driven applications
B.Supporting client-side and server-side encryption
C.Building analytics workloads during specified hours and when the application is not active
D.Caching data to reduce the pressure on the backend database
E.Scaling globally to support petabytes of data and tens of millions of requests per minute
F.Creating a secondary replica of the cluster by using the AWS Management Console

Answer: BCE

Explanation:

The correct answer is BCE. Here's why:

B. Supporting client-side and server-side encryption: Amazon Redshift supports both client-side and server-side encryption to protect data at rest and in transit. This is a crucial security feature for sensitive data being migrated to the cloud. https://docs.aws.amazon.com/redshift/latest/mgmt/working-with-encryption.html

C. Building analytics workloads during specified hours and when the application is not active: Redshift is designed for analytical workloads, such as running complex queries on large datasets. It is suited to batch processing and scheduled analysis, making it ideal for workloads executed during off-peak hours to avoid impacting application performance. Amazon Redshift's concurrency scaling features can automatically add query processing power during periods of high analytical demand, and then scale back down when the workload diminishes.

E. Scaling globally to support petabytes of data and tens of millions of requests per minute: Redshift is a massively parallel processing (MPP) data warehouse that can scale to petabytes of data. Redshift also can handle high query concurrency, making it suitable for handling tens of millions of requests, though the exact performance is dependent on various factors like query complexity and cluster configuration. https://aws.amazon.com/redshift/features/

Here's why the other options are incorrect:

A. Supporting data APIs to access data with traditional, containerized, and event-driven applications: While you can access Redshift via APIs, it's not primarily designed to be a general-purpose data API provider for all types of applications. Redshift focuses on analytical querying. Options like Amazon RDS with the Data API, or DynamoDB are better suited for general data API access.

D. Caching data to reduce the pressure on the backend database: Redshift is a data warehouse, not a caching layer. Caching is typically handled by services like Amazon ElastiCache or Amazon CloudFront.

F. Creating a secondary replica of the cluster by using the AWS Management Console: While you can take snapshots of Redshift clusters for backups and use them to restore to a new cluster, creating a secondary replica in the sense of a real-time failover replica via the console is not the typical way Redshift high availability is managed. Redshift leverages features like automatic snapshots and cross-region restores for disaster recovery, rather than continuously replicating to a hot standby. Redshift also allows for automated backups.

Question: 516 **Exam Heist**

A company provides an API interface to customers so the customers can retrieve their financial information. Ehe

company expects a larger number of requests during peak usage times of the year.

The company requires the API to respond consistently with low latency to ensure customer satisfaction. The company needs to provide a compute host for the API.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use an Application Load Balancer and Amazon Elastic Container Service (Amazon ECS).
B.Use Amazon API Gateway and AWS Lambda functions with provisioned concurrency.
C.Use an Application Load Balancer and an Amazon Elastic Kubernetes Service (Amazon EKS) cluster.
D.Use Amazon API Gateway and AWS Lambda functions with reserved concurrency.

Answer: B

Explanation:

The correct answer is **B. Use Amazon API Gateway and AWS Lambda functions with provisioned concurrency.**

Here's why:

Low Latency & Consistent Performance: Lambda functions with provisioned concurrency ensure that a specified number of Lambda function instances are initialized and ready to respond instantly to requests. This eliminates cold starts, providing consistent low latency, which is crucial during peak usage periods. API Gateway acts as a front door, managing requests and routing them efficiently to Lambda.

Least Operational Overhead: Lambda is a serverless compute service. AWS manages the underlying infrastructure, operating system patching, and scaling, significantly reducing operational overhead compared to container-based solutions like ECS or EKS. API Gateway also simplifies API management, authorization, and monitoring.

Why other options are less suitable:

A & C (ECS/EKS): Managing container orchestration platforms like ECS or EKS requires significant operational overhead. You need to manage the cluster, scaling, and container deployments.
D (Reserved Concurrency): While similar to provisioned concurrency, the option given of reserved concurrency is not the same feature provided by Lambda. Provisioned concurrency actively pre-warms instances, and using the "reserved" setting only prevents other functions from using allocated resources.

Benefits of API Gateway and Lambda:

Scalability: Both services automatically scale to handle increased traffic during peak periods.
Cost-Effectiveness: You only pay for the compute time consumed by Lambda functions.
Security: API Gateway provides features like authentication, authorization, and request validation.

In summary, leveraging Amazon API Gateway and AWS Lambda functions with provisioned concurrency offers the optimal balance of low latency, consistent performance, scalability, and minimal operational overhead for a high-traffic API service.

Authoritative Links:

AWS Lambda Provisioned Concurrency: https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html
Amazon API Gateway: https://aws.amazon.com/api-gateway/

Question: 517 **Exam Heist**

A company wants to send all AWS Systems Manager Session Manager logs to an Amazon S3 bucket for archival purposes.

Which solution will meet this requirement with the MOST operational efficiency?

A.Enable S3 logging in the Systems Manager console. Choose an S3 bucket to send the session data to.

B.Install the Amazon CloudWatch agent. Push all logs to a CloudWatch log group. Export the logs to an S3 bucket from the group for archival purposes.

C.Create a Systems Manager document to upload all server logs to a central S3 bucket. Use Amazon EventBridge to run the Systems Manager document against all servers that are in the account daily.

D.Install an Amazon CloudWatch agent. Push all logs to a CloudWatch log group. Create a CloudWatch logs subscription that pushes any incoming log events to an Amazon Kinesis Data Firehose delivery stream. Set Amazon S3 as the destination.

Answer: A

Explanation:

The most operationally efficient solution for archiving Systems Manager Session Manager logs to an S3 bucket is **A. Enable S3 logging in the Systems Manager console. Choose an S3 bucket to send the session data to.**

Here's why:

Direct Integration: Systems Manager Session Manager has native integration with S3 for logging. This means you can configure logging directly within the Systems Manager console, avoiding the need for additional services or complex configurations.

Simplified Configuration: Enabling S3 logging in the Systems Manager console is a straightforward process involving selecting an S3 bucket as the destination. This requires minimal setup and maintenance.

Operational Overhead: Solution A has the least operational overhead. It leverages the built-in functionality of Systems Manager Session Manager, which is designed to directly stream logs to S3. This eliminates the need for deploying, configuring, and managing additional components, making it operationally efficient.

Cost-Effectiveness: By minimizing the number of services and configurations, Solution A reduces the potential for errors and the need for monitoring and troubleshooting, thereby lowering operational costs.

Alternatives: Solution B (CloudWatch Agent and exporting logs to S3) introduces extra layers of complexity. Installing and configuring agents on instances and managing CloudWatch log groups adds overhead. Furthermore, exporting logs from CloudWatch requires additional configuration and potentially incurs additional costs. Solutions C introduces complexities of SSM documents and eventbridge schedules.

In summary, leveraging the native S3 logging feature of Systems Manager Session Manager provides the simplest, most direct, and operationally efficient way to archive session logs to an S3 bucket. It avoids unnecessary complexity and minimizes the management burden.

Reference:

AWS Documentation on Systems Manager Session Manager Logging

Question: 518 **Exam Heist**

An application uses an Amazon RDS MySQL DB instance. The RDS database is becoming low on disk space. A solutions architect wants to increase the disk space without downtime.

Which solution meets these requirements with the LEAST amount of effort?

A.Enable storage autoscaling in RDS

B.Increase the RDS database instance size

C.Change the RDS database instance storage type to Provisioned IOPS

D.Back up the RDS database, increase the storage capacity, restore the database, and stop the previous instance

Answer: A

Explanation:

The correct answer is A: Enable storage autoscaling in RDS. This is the simplest and most efficient way to address the low disk space issue without downtime. Storage autoscaling automatically increases storage capacity when needed, up to a user-defined maximum, preventing applications from running out of space. This eliminates the need for manual intervention and potential downtime associated with manual scaling operations.

Option B, increasing the RDS database instance size, affects the compute resources (CPU, memory) but doesn't directly address storage capacity. It's an unnecessary change for resolving a storage problem. Option C, changing the storage type to Provisioned IOPS, primarily focuses on improving I/O performance rather than increasing capacity. While it might offer some incidental capacity increase, it's not the primary solution. Option D, backing up, increasing storage, and restoring, involves significant downtime, which is contrary to the question's requirements.

Storage autoscaling is specifically designed for this scenario and is the least intrusive. It seamlessly extends the storage without interrupting the application. Amazon RDS monitors the free storage space and automatically scales the storage when it detects that the database is approaching its capacity limit. This feature is enabled with a few clicks in the AWS Management Console or using the AWS CLI. It offers a cost-effective and hands-off approach to managing storage requirements.

For further research, refer to the official AWS documentation on RDS Storage Autoscaling: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_PIOPS.Autoscaling.html and Managing Storage Capacity: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ConfigureStorage.html. These resources will provide comprehensive information on enabling and managing storage autoscaling, along with best practices.

Question: 519 **Exam Heist**

A consulting company provides professional services to customers worldwide. The company provides solutions and tools for customers to expedite gathering and analyzing data on AWS. The company needs to centrally manage and deploy a common set of solutions and tools for customers to use for self-service purposes.

Which solution will meet these requirements?

A.Create AWS CloudFormation templates for the customers.

B.Create AWS Service Catalog products for the customers.

C.Create AWS Systems Manager templates for the customers.

D.Create AWS Config items for the customers.

Answer: B

Explanation:

The correct answer is **B. Create AWS Service Catalog products for the customers.**

Here's why:

AWS Service Catalog enables organizations to create and manage catalogs of IT services that are approved for use on AWS. These services can include everything from virtual machines and databases to complete multi-tier application architectures. The key benefit here is centralized management and deployment. The consulting company can define a standardized set of solutions and tools as Service Catalog products.

By creating these products, the company provides a self-service portal for customers to deploy these pre-approved, configured solutions. This ensures consistency, reduces errors, and simplifies the process for the customers. Service Catalog products are versioned, allowing for controlled updates and rollbacks. Customers can self-provision these solutions without needing extensive AWS expertise or direct interaction with the consulting company. This aligns with the requirement for self-service and standardized tools.

Options A, C, and D are not as suitable:

A. Create AWS CloudFormation templates for the customers: While CloudFormation is a valid infrastructure-as-code tool, it doesn't provide the same level of abstraction and governance as Service Catalog. CloudFormation templates require customers to have a certain level of AWS understanding, which defeats the purpose of self-service. There's no central management and versioning like Service Catalog.

C. Create AWS Systems Manager templates for the customers: AWS Systems Manager is primarily for operational management and automation tasks on existing AWS resources. Although it includes automation capabilities, it's not designed for creating and managing self-service catalogs of IT services. It's more suitable for operational tasks after the infrastructure is deployed.

D. Create AWS Config items for the customers: AWS Config is for auditing and compliance; it does not provide provisioning capabilities. It's used to assess, audit, and evaluate the configurations of AWS resources. It can't deploy solutions.

In summary, AWS Service Catalog is the most appropriate solution for providing a centrally managed, self-service portal for customers to deploy pre-defined solutions and tools.

Authoritative Links:

AWS Service Catalog Documentation: https://aws.amazon.com/servicecatalog/
AWS Service Catalog Features: https://aws.amazon.com/servicecatalog/features/

Question: 520 **Exam Heist**

A company is designing a new web application that will run on Amazon EC2 Instances. The application will use Amazon DynamoDB for backend data storage. The application traffic will be unpredictable. The company expects that the application read and write throughput to the database will be moderate to high. The company needs to scale in response to application traffic.

Which DynamoDB table configuration will meet these requirements MOST cost-effectively?

A.Configure DynamoDB with provisioned read and write by using the DynamoDB Standard table class. Set DynamoDB auto scaling to a maximum defined capacity.

B.Configure DynamoDB in on-demand mode by using the DynamoDB Standard table class.

C.Configure DynamoDB with provisioned read and write by using the DynamoDB Standard Infrequent Access (DynamoDB Standard-IA) table class. Set DynamoDB auto scaling to a maximum defined capacity.

D.Configure DynamoDB in on-demand mode by using the DynamoDB Standard Infrequent Access (DynamoDB Standard-IA) table class.

Answer: B

Explanation:

The most cost-effective DynamoDB configuration for an application with unpredictable traffic and moderate to high throughput requirements is using on-demand mode with the DynamoDB Standard table class (Option B).

Here's why:

Unpredictable Traffic: On-demand mode eliminates the need to provision read and write capacity units (RCUs/WCUs). DynamoDB automatically scales capacity in response to application traffic. This is ideal for unpredictable workloads because you only pay for the reads and writes your application actually performs.
Cost-Effectiveness: While provisioned mode with auto-scaling can handle traffic fluctuations, it requires you to estimate peak capacity and set scaling parameters. On-demand mode avoids over-provisioning, which is a common issue with auto-scaling when workloads are highly variable, leading to cost savings.
DynamoDB Standard: DynamoDB Standard is the general-purpose table class designed for frequently accessed data. Since the application expects moderate to high throughput, it's assumed the data will be accessed regularly, making Standard the appropriate choice.
DynamoDB Standard-IA is less suitable since it is optimized for infrequently accessed data and has higher read and write costs. It's not the optimal option for the expected workload.

In summary, on-demand mode's pay-per-request pricing model aligns perfectly with unpredictable traffic, ensuring that costs are minimized while dynamically scaling to meet the application's throughput demands.

Refer to the following AWS documentation for more information:

DynamoDB Pricing: https://aws.amazon.com/dynamodb/pricing/
Choosing Between On-Demand and Provisioned Capacity: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadWriteCapacityMode.html
DynamoDB Table Classes: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/table-classes.html

Question: 521 **Exam Heist**

A retail company has several businesses. The IT team for each business manages its own AWS account. Each team account is part of an organization in AWS Organizations. Each team monitors its product inventory levels in an Amazon DynamoDB table in the team's own AWS account.

The company is deploying a central inventory reporting application into a shared AWS account. The application must be able to read items from all the teams' DynamoDB tables.

Which authentication option will meet these requirements MOST securely?

A.Integrate DynamoDB with AWS Secrets Manager in the inventory application account. Configure the application to use the correct secret from Secrets Manager to authenticate and read the DynamoDB table. Schedule secret rotation for every 30 days.

B.In every business account, create an IAM user that has programmatic access. Configure the application to use the correct IAM user access key ID and secret access key to authenticate and read the DynamoDB table. Manually rotate IAM access keys every 30 days.

C.In every business account, create an IAM role named BU_ROLE with a policy that gives the role access to the DynamoDB table and a trust policy to trust a specific role in the inventory application account. In the inventory account, create a role named APP_ROLE that allows access to the STS AssumeRole API operation. Configure the application to use APP_ROLE and assume the crossaccount role BU_ROLE to read the DynamoDB table.

D.Integrate DynamoDB with AWS Certificate Manager (ACM). Generate identity certificates to authenticate DynamoDB. Configure the application to use the correct certificate to authenticate and read the DynamoDB table.

Answer: C

Explanation:

Option C is the most secure solution because it leverages AWS's recommended practice of cross-account IAM roles for accessing resources across different AWS accounts. This approach provides temporary credentials with least privilege, enhancing security posture.

Here's a detailed breakdown:

IAM Roles over IAM Users: IAM roles are preferred over IAM users for cross-account access because roles provide temporary credentials through the AssumeRole API, eliminating the need to manage long-term access keys which are vulnerable to leakage.

Cross-Account Access with AssumeRole: The described setup uses AssumeRole in a secure manner. The application in the central inventory account (APP_ROLE) assumes the role in each business account (BU_ROLE).

Least Privilege: Each BU_ROLE is granted specific permissions to access only the DynamoDB table in that business account. The trust policy of BU_ROLE ensures that only the designated APP_ROLE can assume it, preventing unauthorized access.

Centralized Control: The APP_ROLE in the inventory application account handles the assumption of roles across all business accounts, enabling centralized control and auditing of access.

No Hardcoded Credentials: No long-term credentials (like access keys) need to be stored or managed within the application, reducing the risk of exposure.

Option A is less secure because Secrets Manager would require storing database credentials, which is a valid approach, but not the best in a multi-account scenario. It does not intrinsically facilitate cross-account access as effectively or securely as cross-account IAM roles.

Option B is highly insecure because it involves managing and rotating long-term IAM user access keys, which is a significant security risk. Manual rotation is prone to errors and delays. Exposing access keys opens up a large attack surface if compromised.

Option D is incorrect. DynamoDB does not directly integrate with ACM for authentication in this manner. ACM is primarily for managing SSL/TLS certificates for secure communication. Identity certificates are not the standard way to authenticate DynamoDB access.

In summary, leveraging cross-account IAM roles with AssumeRole ensures secure, auditable, and easily manageable access to DynamoDB tables across multiple AWS accounts, adhering to the principle of least privilege and utilizing temporary credentials.

References:

IAM roles:
Granting access to multiple AWS accounts:
AssumeRole API:

Question: 522 **Exam Heist**

A company runs container applications by using Amazon Elastic Kubernetes Service (Amazon EKS). The company's workload is not consistent throughout the day. The company wants Amazon EKS to scale in and out according to the workload.

Which combination of steps will meet these requirements with the LEAST operational overhead? (Choose two.)

A.Use an AWS Lambda function to resize the EKS cluster.
B.Use the Kubernetes Metrics Server to activate horizontal pod autoscaling.
C.Use the Kubernetes Cluster Autoscaler to manage the number of nodes in the cluster.
D.Use Amazon API Gateway and connect it to Amazon EKS.
E.Use AWS App Mesh to observe network activity.

Answer: BC

Explanation:

The optimal solution for dynamically scaling an Amazon EKS cluster based on workload with minimal operational overhead involves two key components: the Kubernetes Metrics Server and the Kubernetes Cluster Autoscaler.

B. Use the Kubernetes Metrics Server to activate horizontal pod autoscaling.

The Kubernetes Metrics Server collects resource usage data (CPU, memory) from pods and nodes. This data is crucial for Horizontal Pod Autoscaler (HPA). HPA automatically scales the number of pods in a deployment or replication controller based on observed CPU utilization or other select metrics. By using the Metrics Server, the HPA can dynamically increase or decrease the number of pods running to meet demand, thereby efficiently utilizing resources and improving application responsiveness. The Metrics Server is a lightweight, readily available tool for this purpose and is simple to deploy and manage.

Kubernetes Metrics Server
Horizontal Pod Autoscaling

C. Use the Kubernetes Cluster Autoscaler to manage the number of nodes in the cluster.

The Kubernetes Cluster Autoscaler automatically adjusts the size of the EKS cluster (i.e., the number of worker nodes) based on the resource requirements of the pods that are scheduled or unscheduled. If the HPA scales out and new pods cannot be scheduled due to insufficient node resources, the Cluster Autoscaler will automatically provision new nodes to accommodate them. Conversely, if pods are scaled down by the HPA and nodes become underutilized, the Cluster Autoscaler will terminate unnecessary nodes. This dynamic scaling of the worker nodes ensures that the cluster has the right amount of resources to meet the application's needs.

Kubernetes Cluster Autoscaler
Scaling worker nodes

Why other options are incorrect:

A. Use an AWS Lambda function to resize the EKS cluster: While possible, using Lambda for this purpose is significantly more complex and requires custom coding to monitor metrics, determine scaling needs, and interact with the EKS API. This approach introduces substantial operational overhead.
D. Use Amazon API Gateway and connect it to Amazon EKS: API Gateway is used for managing and exposing APIs, not for cluster scaling. It's irrelevant to the workload-based scaling requirement.
E. Use AWS App Mesh to observe network activity: App Mesh is a service mesh that provides visibility and control over microservices communication. While helpful for observability and traffic management, it doesn't directly address cluster scaling based on workload. App Mesh focuses on the application layer, not on the infrastructure capacity needed to support the application.

Question: 523 **Exam Heist**

A company runs a microservice-based serverless web application. The application must be able to retrieve data from multiple Amazon DynamoDB tables A solutions architect needs to give the application the ability to retrieve the data with no impact on the baseline performance of the application.

Which solution will meet these requirements in the MOST operationally efficient way?

A.AWS AppSync pipeline resolvers
B.Amazon CloudFront with functions
C.Edge-optimized Amazon API Gateway with AWS Lambda functions
D.Amazon Athena Federated Query with a DynamoDB connector

Answer: B

Explanation:

The correct answer is **B. Amazon CloudFront with functions.**

Here's why:

Operational Efficiency: Lambda@Edge functions are deployed globally within CloudFront's infrastructure. This close proximity to users reduces latency as requests don't need to travel back to the origin server (in this case, the serverless application) for simple data retrieval logic. This minimizes the performance impact on the baseline application.
Data Aggregation and Transformation: Lambda@Edge allows you to intercept requests before they reach the origin and responses before they reach the user. In this scenario, Lambda@Edge can be configured to fetch data from the multiple DynamoDB tables, aggregate the results, and transform them into the required format before sending the consolidated response to the user. This offloads the data aggregation from the origin server, contributing to better performance.
Caching Benefits: CloudFront's caching capabilities can further reduce the load on the DynamoDB tables and the serverless application. By caching frequently accessed data close to the users, CloudFront eliminates the need to repeatedly fetch the same data from the DynamoDB tables, enhancing performance and reducing operational costs.

Why the other options are less optimal:

A. AWS AppSync pipeline resolvers: While AppSync is suitable for data aggregation from multiple sources, it introduces a separate managed service and requires restructuring the existing microservice-based architecture. The additional hop to AppSync increases latency and complexity compared to Lambda@Edge. The question asks for a solution with no impact on the baseline performance.
C. Edge-optimized Amazon API Gateway with AWS Lambda functions: This option is similar to Lambda@Edge, but API Gateway is optimized for API management and security. It adds extra complexity without solving the performance issue since the Lambda would still need to query the DynamoDB tables.
D. Amazon Athena Federated Query with a DynamoDB connector: Athena is designed for analyzing data stored in DynamoDB using SQL. While it can retrieve data from multiple DynamoDB tables, it's primarily used for batch processing and querying large datasets, not for real-time data retrieval in a web application. It will impact the latency of the responses, failing the no performance impact requirement.

Therefore, Lambda@Edge with CloudFront is the most operationally efficient and performant solution for retrieving data from multiple DynamoDB tables in a serverless web application with minimal impact on the baseline performance.

Authoritative Links:

Lambda@Edge: https://aws.amazon.com/lambda/edge/
Amazon CloudFront: https://aws.amazon.com/cloudfront/

Question: 524 **Exam Heist**

A company wants to analyze and troubleshoot Access Denied errors and Unauthorized errors that are related to IAM permissions. The company has AWS CloudTrail turned on.

Which solution will meet these requirements with the LEAST effort?

A.Use AWS Glue and write custom scripts to query CloudTrail logs for the errors.
B.Use AWS Batch and write custom scripts to query CloudTrail logs for the errors.
C.Search CloudTrail logs with Amazon Athena queries to identify the errors.
D.Search CloudTrail logs with Amazon QuickSight. Create a dashboard to identify the errors.

Answer: C

Explanation:

The optimal solution for analyzing IAM-related Access Denied and Unauthorized errors from CloudTrail logs with minimal effort is option C: using Amazon Athena.

Athena directly queries data stored in Amazon S3, where CloudTrail typically delivers its logs. It leverages standard SQL, making analysis relatively straightforward for individuals familiar with SQL. The key advantage is that Athena eliminates the need for complex ETL (Extract, Transform, Load) processes, unlike AWS Glue (option A), which necessitates creating and managing ETL jobs to prepare the data for querying. While AWS Batch (option B) could be used to run custom scripts against CloudTrail logs, this approach involves more operational overhead and script development, which negates the 'least effort' requirement.

Amazon QuickSight (option D) is primarily a business intelligence tool for visualization and creating dashboards. While QuickSight can connect to Athena, it's an additional layer of complexity for the initial troubleshooting stage. Constructing a complete dashboard before identifying error patterns introduces unnecessary work. Athena provides the capability to query and filter CloudTrail logs directly to identify these errors, after which, insights can be visualized for deeper analysis. This makes Athena a more targeted and efficient first step. It also supports federated queries so you can access multiple data stores, which is irrelevant to the question.

Therefore, Athena's ability to directly query CloudTrail logs using SQL, without requiring ETL or dashboard creation upfront, makes it the most efficient solution to identify the specified IAM errors.

Supporting Links:

AWS CloudTrail Logging IAM API Calls
Analyzing CloudTrail Logs with Amazon Athena

Question: 525 **Exam Heist**

A company wants to add its existing AWS usage cost to its operation cost dashboard. A solutions architect needs to recommend a solution that will give the company access to its usage cost programmatically. The company must be able to access cost data for the current year and forecast costs for the next 12 months.

Which solution will meet these requirements with the LEAST operational overhead?

A.Access usage cost-related data by using the AWS Cost Explorer API with pagination.
B.Access usage cost-related data by using downloadable AWS Cost Explorer report .csv files.
C.Configure AWS Budgets actions to send usage cost data to the company through FTP.
D.Create AWS Budgets reports for usage cost data. Send the data to the company through SMTP.

Answer: A

Explanation:

The correct answer is A: Access usage cost-related data by using the AWS Cost Explorer API with pagination.

Here's why:

Programmatic Access: The requirement specifies programmatic access to cost data. The Cost Explorer API directly addresses this, allowing the company to retrieve data through code. Options B, C, and D require manual intervention or rely on delivery mechanisms less suitable for automated integration with a dashboard.
Current Year and Forecast Data: The AWS Cost Explorer API provides access to historical cost data for the current year and forecasting capabilities for the next 12 months, meeting the data requirements.
Least Operational Overhead: The Cost Explorer API, when used with pagination, minimizes the overhead. Pagination allows the retrieval of data in manageable chunks, preventing large data transfers that could strain resources. Options C and D require configuring and managing budgets and delivery mechanisms like FTP or SMTP, increasing operational complexity. Option B would require manual downloading and processing of CSV files.
Automation and Integration: The API facilitates direct integration with the company's operation cost dashboard, enabling automated data updates and analysis. This aligns with best practices for cloud cost management and operational efficiency.
Scalability: The API-based approach can easily scale to accommodate increasing data volumes and complexity as the company's AWS usage grows. The pagination feature helps maintain performance during scaling.
Cost Optimization: By providing granular cost data, the Cost Explorer API can help the company identify cost optimization opportunities and improve its overall cloud cost efficiency.

Authoritative Links:

AWS Cost Explorer API: https://docs.aws.amazon.com/aws-cost-management/latest/APIReference/
AWS Cost Explorer: https://aws.amazon.com/aws-cost-management/aws-cost-explorer/

Question: 526 **Exam Heist**

A solutions architect is reviewing the resilience of an application. The solutions architect notices that a database administrator recently failed over the application's Amazon Aurora PostgreSQL database writer instance as part of a scaling exercise. The failover resulted in 3 minutes of downtime for the application.

Which solution will reduce the downtime for scaling exercises with the LEAST operational overhead?

A.Create more Aurora PostgreSQL read replicas in the cluster to handle the load during failover.

B.Set up a secondary Aurora PostgreSQL cluster in the same AWS Region. During failover, update the application to use the secondary cluster's writer endpoint.

C.Create an Amazon ElastiCache for Memcached cluster to handle the load during failover.

D.Set up an Amazon RDS proxy for the database. Update the application to use the proxy endpoint.

Answer: D

Explanation:

The best solution to minimize downtime during Aurora PostgreSQL writer instance failovers, with the least operational overhead, is **D. Set up an Amazon RDS proxy for the database. Update the application to use the proxy endpoint.**

Here's why:

RDS Proxy's Primary Benefit: RDS Proxy sits between your application and your database. Its core functionality is to manage database connections efficiently, maintain connection pools, and most importantly, automatically reconnect to the new writer instance during a failover event.
https://aws.amazon.com/rds/proxy/
Reduced Downtime: By automatically handling reconnection, RDS Proxy significantly reduces application downtime during failovers. The application doesn't need to implement complex retry logic or be aware of the database's internal failover process. This typically reduces downtime from minutes to seconds.
Connection Management: RDS Proxy handles connection management, preventing connection storms that can overload the database during failover. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/rds-proxy.html
Minimal Operational Overhead: Setting up RDS Proxy is relatively straightforward and requires minimal changes to the application code. The application only needs to be configured to connect to the RDS Proxy endpoint instead of the database endpoint.
Why other options are less suitable:

A. Create more Aurora PostgreSQL read replicas: Read replicas are for read scaling and don't help with write operations during a writer failover. The problem is the unavailability of the writer instance.
B. Secondary Aurora PostgreSQL cluster: This involves a much more complex setup, including data replication and application failover logic. It has a significantly higher operational overhead. Maintaining a secondary cluster requires continuous synchronization and testing of failover procedures.
C. Amazon ElastiCache for Memcached: ElastiCache is a caching service. While caching can reduce database load, it doesn't address the problem of the application being unable to write to the database during a writer failover. Data loss or inconsistency can also be major concerns.
RDS Proxy is specifically designed for this use case: It provides a managed service for connection pooling and automatic failover, simplifying database management and increasing application availability. Therefore, option D is the most effective and efficient solution.

Question: 527 **Exam Heist**

A company has a regional subscription-based streaming service that runs in a single AWS Region. The architecture consists of web servers and application servers on Amazon EC2 instances. The EC2 instances are in Auto Scaling groups behind Elastic Load Balancers. The architecture includes an Amazon Aurora global database cluster that extends across multiple Availability Zones.

The company wants to expand globally and to ensure that its application has minimal downtime.

Which solution will provide the MOST fault tolerance?

A.Extend the Auto Scaling groups for the web tier and the application tier to deploy instances in Availability Zones in a second Region. Use an Aurora global database to deploy the database in the primary Region and the second Region. Use Amazon Route 53 health checks with a failover routing policy to the second Region.

B.Deploy the web tier and the application tier to a second Region. Add an Aurora PostgreSQL cross-Region Aurora Replica in the second Region. Use Amazon Route 53 health checks with a failover routing policy to the second Region. Promote the secondary to primary as needed.

C.Deploy the web tier and the application tier to a second Region. Create an Aurora PostgreSQL database in the second Region. Use AWS Database Migration Service (AWS DMS) to replicate the primary database to the second Region. Use Amazon Route 53 health checks with a failover routing policy to the second Region.

D.Deploy the web tier and the application tier to a second Region. Use an Amazon Aurora global database to deploy the database in the primary Region and the second Region. Use Amazon Route 53 health checks with a failover routing policy to the second Region. Promote the secondary to primary as needed.

Answer: D

Explanation:

Here's a detailed justification for why option D is the best solution for maximizing fault tolerance while

expanding a regional streaming service globally:

The primary goal is to minimize downtime during a regional failure. Option D achieves this most effectively by replicating the application and database tiers to a second region and using Route 53 for failover.

Application Tier Replication: Deploying the web and application tiers to a second region provides redundancy. If the primary region experiences issues, Route 53 can direct traffic to the healthy second region. This ensures application availability.
Aurora Global Database for Database Replication: Aurora Global Database is crucial. It replicates data across AWS Regions with low latency, typically under a second. This ensures that the second region's database is nearly up-to-date, minimizing data loss during failover.
Route 53 Failover: Amazon Route 53 health checks monitor the health of the application in both regions. A failover routing policy automatically redirects traffic to the second region if the primary region becomes unavailable.
Controlled Failover (Promotion): Promoting the secondary Aurora instance to primary is essential for write operations. The controlled failover approach minimizes downtime and ensures data consistency.

Option A is less desirable because it suggests extending Auto Scaling groups across regions. While technically possible with some configurations, it introduces unnecessary complexity and potential latency issues related to cross-region networking for application components.

Option B uses Aurora PostgreSQL cross-Region Aurora Replica. While an Aurora replica does replicate data, promotion can be more complex and slower than using the built-in global database capabilities. Moreover, it doesn't utilize the automatic failover benefits of Aurora Global Database.

Option C uses AWS DMS for database replication. DMS is suitable for migrations or scenarios requiring data transformation, but it's not the best choice for a constantly replicating, highly available database setup. DMS's replication latency is higher than Aurora Global Database, leading to a greater potential for data loss during failover.

Therefore, option D balances redundancy, performance, and ease of failover, making it the most fault-tolerant solution.

Authoritative Links:

Amazon Aurora Global Database: https://aws.amazon.com/rds/aurora/global-database/
Amazon Route 53 Failover: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-failover.html
AWS Database Migration Service (DMS): https://aws.amazon.com/dms/

Question: 528 **Exam Heist**

A data analytics company wants to migrate its batch processing system to AWS. The company receives thousands of small data files periodically during the day through FTP. An on-premises batch job processes the data files overnight. However, the batch job takes hours to finish running.

The company wants the AWS solution to process incoming data files as soon as possible with minimal changes to the FTP clients that send the files. The solution must delete the incoming data files after the files have been processed successfully. Processing for each file needs to take 3-8 minutes.

Which solution will meet these requirements in the MOST operationally efficient way?

A.Use an Amazon EC2 instance that runs an FTP server to store incoming files as objects in Amazon S3 Glacier Flexible Retrieval. Configure a job queue in AWS Batch. Use Amazon EventBridge rules to invoke the job to process the objects nightly from S3 Glacier Flexible Retrieval. Delete the objects after the job has processed the objects.

B.Use an Amazon EC2 instance that runs an FTP server to store incoming files on an Amazon Elastic Block Store (Amazon EBS) volume. Configure a job queue in AWS Batch. Use Amazon EventBridge rules to invoke the

job to process the files nightly from the EBS volume. Delete the files after the job has processed the files.

C.Use AWS Transfer Family to create an FTP server to store incoming files on an Amazon Elastic Block Store (Amazon EBS) volume. Configure a job queue in AWS Batch. Use an Amazon S3 event notification when each file arrives to invoke the job in AWS Batch. Delete the files after the job has processed the files.

D.Use AWS Transfer Family to create an FTP server to store incoming files in Amazon S3 Standard. Create an AWS Lambda function to process the files and to delete the files after they are processed. Use an S3 event notification to invoke the Lambda function when the files arrive.

Answer: D

Explanation:

Here's a detailed justification for why option D is the most operationally efficient solution:

Option D leverages several AWS services optimally for the given scenario. First, AWS Transfer Family is a managed service specifically designed for secure file transfers to and from AWS storage services, minimizing the operational overhead of managing an EC2-based FTP server. It directly integrates with S3, simplifying the file storage aspect.https://aws.amazon.com/transfer/

Storing the files in Amazon S3 Standard provides immediate availability and scalability. The use of S3 event notifications triggers an AWS Lambda function immediately when a file arrives. AWS Lambda is ideal for processing individual files that take only a few minutes to process, fitting the 3-8 minute requirement. Lambda functions are serverless, automatically scaling based on the number of invocations, thus reducing operational burden.https://aws.amazon.com/lambda/

The Lambda function can process the file and then delete it from S3, automating the cleanup process. This entire solution requires minimal manual intervention and scales automatically based on the workload.

Options A, B, and C are less optimal for several reasons. Options A and B use Amazon EC2 to manage the FTP server. This increases operational complexity because you need to patch, scale, and manage the EC2 instance. They also use AWS Batch nightly, which is not what the customer asked for. Option A moves the data to Glacier Flexible Retrieval, which is not a use-case for immediate processing. The process of moving objects to Glacier is not immediate. Option C unnecessarily uses EBS, which adds persistent storage costs and does not readily integrate with event-driven processing. Option C uses AWS Batch, which may introduce more complexity to the design since Lambda can more easily process incoming files and delete them.

Question: 529 **Exam Heist**

A company is migrating its workloads to AWS. The company has transactional and sensitive data in its databases. The company wants to use AWS Cloud solutions to increase security and reduce operational overhead for the databases.

Which solution will meet these requirements?

A.Migrate the databases to Amazon EC2. Use an AWS Key Management Service (AWS KMS) AWS managed key for encryption.

B.Migrate the databases to Amazon RDS Configure encryption at rest.

C.Migrate the data to Amazon S3 Use Amazon Macie for data security and protection

D.Migrate the database to Amazon RDS. Use Amazon CloudWatch Logs for data security and protection.

Answer: B

Explanation:

The correct answer is B: Migrate the databases to Amazon RDS and configure encryption at rest.

Here's why:

Amazon RDS (Relational Database Service) is a managed database service. It significantly reduces operational overhead by automating tasks like patching, backups, and recovery, which the company desires. This removes the need for the company to manage the underlying infrastructure, improving efficiency.

Encryption at rest in Amazon RDS directly addresses the security requirement for transactional and sensitive data. RDS provides encryption options using AWS Key Management Service (KMS) for managing encryption keys. Data is encrypted while stored on disk (at rest), enhancing data protection.

Option A (Amazon EC2): While EC2 allows you to host databases, it increases operational overhead because you are responsible for managing the entire infrastructure, including backups, patching, and security. This contradicts the requirement to reduce operational overhead.

Option C (Amazon S3 and Macie): Amazon S3 is primarily object storage, not designed for transactional databases. Amazon Macie is a data security and data privacy service that uses machine learning and pattern matching to discover and protect sensitive data in AWS. It is not suitable for hosting a transactional database.

Option D (Amazon RDS and CloudWatch Logs): While RDS provides a managed database solution, CloudWatch Logs is primarily for monitoring and logging. While logs can be used for security analysis, CloudWatch Logs does not encrypt data at rest, therefore not directly addressing data protection for transactional databases. Therefore, it doesn't satisfy the security requirement.

Therefore, migrating to Amazon RDS and enabling encryption at rest provides both reduced operational overhead and increased security for sensitive data.

Here are some authoritative links for further research:

Amazon RDS: https://aws.amazon.com/rds/
Amazon RDS Encryption:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Overview.Encryption.html
AWS Key Management Service (KMS): https://aws.amazon.com/kms/

Question: 530 **Exam Heist**

A company has an online gaming application that has TCP and UDP multiplayer gaming capabilities. The company uses Amazon Route 53 to point the application traffic to multiple Network Load Balancers (NLBs) in different AWS Regions. The company needs to improve application performance and decrease latency for the online game in preparation for user growth.

Which solution will meet these requirements?

A.Add an Amazon CloudFront distribution in front of the NLBs. Increase the Cache-Control max-age parameter.

B.Replace the NLBs with Application Load Balancers (ALBs). Configure Route 53 to use latency-based routing.

C.Add AWS Global Accelerator in front of the NLBs. Configure a Global Accelerator endpoint to use the correct listener ports.

D.Add an Amazon API Gateway endpoint behind the NLBs. Enable API caching. Override method caching for the different stages.

Answer: C

Explanation:

The best solution for improving application performance and decreasing latency for a global online game with TCP and UDP traffic, while using Route 53 and NLBs, is to use AWS Global Accelerator.

Detailed Justification:

AWS Global Accelerator (C): Global Accelerator uses the AWS global network to direct user traffic to the optimal endpoint based on proximity, health, and configuration. It supports both TCP and UDP protocols, which are essential for the game's multiplayer capabilities. It provides static anycast IP addresses that serve as a fixed entry point to the application, improving resilience. Global Accelerator's endpoint groups can be configured to use different ports for the TCP and UDP listeners on the NLBs. By directing traffic based on network proximity, latency is minimized.

CloudFront (A): CloudFront excels at caching static content, which is less relevant for real-time gaming traffic that requires low latency and constant updates. Additionally, CloudFront primarily works with HTTP and HTTPS traffic and isn't designed for general TCP/UDP acceleration. While caching can help, the dynamic nature of game data makes CloudFront less effective than Global Accelerator.

Application Load Balancers (ALBs) with Latency-Based Routing (B): Replacing NLBs with ALBs doesn't directly address the global latency issue as efficiently as Global Accelerator. While latency-based routing in Route 53 helps direct users to the closest region, it does not leverage the optimized AWS global network for low-latency routing between the user and the region, which Global Accelerator provides. ALBs also do not handle UDP traffic natively.

API Gateway (D): API Gateway is designed for managing APIs and doesn't directly improve latency for real-time TCP/UDP gaming traffic. It introduces an additional layer of complexity and isn't suitable for the gaming use case described.

Why Global Accelerator is optimal:

Global Accelerator is specifically designed to optimize performance for applications with a global user base, particularly for use cases requiring consistent, low-latency connectivity over TCP and UDP. It reduces latency by leveraging the highly available and congestion-free AWS global network. The anycast static IPs provide a stable entry point to the application, masking regional failures and changes.

Authoritative Links:

AWS Global Accelerator: https://aws.amazon.com/global-accelerator/
AWS Global Accelerator Features: https://docs.aws.amazon.com/global-accelerator/latest/dg/what-is-global-accelerator.html

Question: 531 **Exam Heist**

A company needs to integrate with a third-party data feed. The data feed sends a webhook to notify an external service when new data is ready for consumption. A developer wrote an AWS Lambda function to retrieve data when the company receives a webhook callback. The developer must make the Lambda function available for the third party to call.

Which solution will meet these requirements with the MOST operational efficiency?

A.Create a function URL for the Lambda function. Provide the Lambda function URL to the third party for the webhook.

B.Deploy an Application Load Balancer (ALB) in front of the Lambda function. Provide the ALB URL to the third party for the webhook.

C.Create an Amazon Simple Notification Service (Amazon SNS) topic. Attach the topic to the Lambda function. Provide the public hostname of the SNS topic to the third party for the webhook.

D.Create an Amazon Simple Queue Service (Amazon SQS) queue. Attach the queue to the Lambda function. Provide the public hostname of the SQS queue to the third party for the webhook.

Answer: A

Explanation:

The correct answer is A: Create a function URL for the Lambda function. Provide the Lambda function URL to the third party for the webhook.

Here's a detailed justification:

Lambda function URLs are a dedicated, built-in feature that allows you to directly invoke your Lambda functions over HTTPS without needing to configure and manage separate API Gateways. This represents the most operationally efficient solution for exposing a Lambda function as a webhook endpoint to a third party.

Option B, deploying an ALB in front of Lambda, adds significant complexity and operational overhead. ALB introduces the need to manage load balancing, listener rules, and target groups, which are unnecessary for a simple webhook integration.

Option C, using SNS, introduces an unnecessary level of indirection. SNS is a pub/sub messaging service, which is suitable for decoupling applications but adds unnecessary complexity for a direct webhook integration. The third party would need to publish to the SNS topic, and the Lambda function would then be triggered by SNS, adding latency and more points of failure.

Option D, using SQS, is also an unnecessarily complex approach. SQS is a queuing service designed for asynchronous message processing. For a simple webhook integration where you expect an immediate action from the Lambda function, SQS introduces unnecessary queuing and latency.

Lambda function URLs offer a direct, simplified way to expose a Lambda function as a publicly accessible HTTPS endpoint. They are cost-effective, require minimal configuration, and are designed for scenarios like webhook integrations.

In summary, Lambda Function URLs provide the simplest, most direct, and most operationally efficient method to expose a Lambda function to a third-party service as a webhook endpoint.

Here are some authoritative links for further research:

AWS Lambda Function URLs: https://docs.aws.amazon.com/lambda/latest/dg/lambda-urls.html
AWS Lambda: https://aws.amazon.com/lambda/

Question: 532 **Exam Heist**

A company has a workload in an AWS Region. Customers connect to and access the workload by using an Amazon API Gateway REST API. The company uses Amazon Route 53 as its DNS provider. The company wants to provide individual and secure URLs for all customers.

Which combination of steps will meet these requirements with the MOST operational efficiency? (Choose three.)

A.Register the required domain in a registrar. Create a wildcard custom domain name in a Route 53 hosted zone and record in the zone that points to the API Gateway endpoint.

B.Request a wildcard certificate that matches the domains in AWS Certificate Manager (ACM) in a different Region.

C.Create hosted zones for each customer as required in Route 53. Create zone records that point to the API Gateway endpoint.

D.Request a wildcard certificate that matches the custom domain name in AWS Certificate Manager (ACM) in the same Region.

E.Create multiple API endpoints for each customer in API Gateway.

F.Create a custom domain name in API Gateway for the REST API. Import the certificate from AWS Certificate Manager (ACM).

Answer: ADF

Explanation:

The correct answer is ADF. Here's why:

A. Register the required domain in a registrar. Create a wildcard custom domain name in a Route 53 hosted zone and record in the zone that points to the API Gateway endpoint. This establishes the foundation for using custom domains and Route 53 for managing DNS records. A wildcard domain (e.g., *.example.com) allows you to handle multiple subdomains without creating individual records for each. This dramatically simplifies DNS management. https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/domain-name-basics.html

D. Request a wildcard certificate that matches the custom domain name in AWS Certificate Manager (ACM) in the same Region. To secure the custom domain with HTTPS, you need a certificate. Using a wildcard certificate (e.g., *.example.com) covers all subdomains under your main domain, streamlining certificate management. The certificate must be in the same region as your API Gateway instance. https://docs.aws.amazon.com/acm/latest/userguide/acm-overview.html

F. Create a custom domain name in API Gateway for the REST API. Import the certificate from AWS Certificate Manager (ACM). This step links the secured custom domain to your API Gateway. You configure API Gateway to use the wildcard certificate from ACM, enabling secure access through the custom domain. This is a critical part of providing personalized URLs. https://docs.aws.amazon.com/apigateway/latest/developerguide/how-to-custom-domains.html

Why other options are incorrect:

B. Request a wildcard certificate that matches the domains in AWS Certificate Manager (ACM) in a different Region. ACM certificates used with API Gateway must be in the same region.
C. Create hosted zones for each customer as required in Route 53. Create zone records that point to the API Gateway endpoint. Creating individual hosted zones for each customer adds unnecessary complexity and management overhead. A wildcard record in a single hosted zone is much more efficient.
E. Create multiple API endpoints for each customer in API Gateway. Creating multiple API endpoints significantly increases management overhead. Using custom domains allows for a single API endpoint to be accessed through different URLs.

Question: 533 **Exam Heist**

A company stores data in Amazon S3. According to regulations, the data must not contain personally identifiable information (PII). The company recently discovered that S3 buckets have some objects that contain PII. The company needs to automatically detect PII in S3 buckets and to notify the company's security team.

Which solution will meet these requirements?

A.Use Amazon Macie. Create an Amazon EventBridge rule to filter the SensitiveData event type from Macie findings and to send an Amazon Simple Notification Service (Amazon SNS) notification to the security team.

B.Use Amazon GuardDuty. Create an Amazon EventBridge rule to filter the CRITICAL event type from GuardDuty findings and to send an Amazon Simple Notification Service (Amazon SNS) notification to the security team.

C.Use Amazon Macie. Create an Amazon EventBridge rule to filter the SensitiveData:S3Object/Personal event type from Macie findings and to send an Amazon Simple Queue Service (Amazon SQS) notification to the security team.

D.Use Amazon GuardDuty. Create an Amazon EventBridge rule to filter the CRITICAL event type from GuardDuty findings and to send an Amazon Simple Queue Service (Amazon SQS) notification to the security team.

Answer: A

Explanation:

The correct answer is A. Here's why:

Amazon Macie is a fully managed data security and data privacy service that uses machine learning and pattern matching to discover and protect sensitive data in AWS. It's designed specifically for identifying PII and other sensitive information within S3 buckets.

Macie for PII Detection: Macie is built to automatically discover PII in S3, which directly addresses the company's requirement.

EventBridge for Event-Driven Architecture: Amazon EventBridge allows you to build event-driven applications by routing events between AWS services. In this case, Macie findings are events.

Filtering Sensitive Data Events: An EventBridge rule can be configured to specifically listen for the SensitiveData event type from Macie findings. This ensures that the security team is only notified when PII is detected, avoiding unnecessary alerts for other types of findings.

SNS for Notification: Amazon Simple Notification Service (SNS) is a fully managed messaging service that can be used to send notifications to various endpoints, including email, SMS, and HTTP endpoints, which is suitable for notifying the security team.

Why other options are incorrect:

GuardDuty: GuardDuty focuses on threat detection and monitoring malicious activity, not specifically identifying PII in S3. It's not designed for data privacy in the same way Macie is.
SQS: While SQS is a useful queuing service, sending notifications directly to SQS wouldn't notify the security team. SNS is a better choice for direct notifications.
CRITICAL event type: GuardDuty's CRITICAL event type is related to security threats and is not designed to find PII.Using Macie, EventBridge, and SNS creates an automated pipeline: Macie scans S3, detects PII, EventBridge filters the alerts for sensitive data, and SNS sends a notification to the security team for review and action.

Authoritative Links:

Amazon Macie: https://aws.amazon.com/macie/
Amazon EventBridge: https://aws.amazon.com/eventbridge/
Amazon SNS: https://aws.amazon.com/sns/

Question: 534 **Exam Heist**

A company wants to build a logging solution for its multiple AWS accounts. The company currently stores the logs from all accounts in a centralized account. The company has created an Amazon S3 bucket in the centralized account to store the VPC flow logs and AWS CloudTrail logs. All logs must be highly available for 30 days for frequent analysis, retained for an additional 60 days for backup purposes, and deleted 90 days after creation.

Which solution will meet these requirements MOST cost-effectively?

A.Transition objects to the S3 Standard storage class 30 days after creation. Write an expiration action that directs Amazon S3 to delete objects after 90 days.

B.Transition objects to the S3 Standard-Infrequent Access (S3 Standard-IA) storage class 30 days after creation. Move all objects to the S3 Glacier Flexible Retrieval storage class after 90 days. Write an expiration action that directs Amazon S3 to delete objects after 90 days.

C.Transition objects to the S3 Glacier Flexible Retrieval storage class 30 days after creation. Write an expiration action that directs Amazon S3 to delete objects after 90 days.

D.Transition objects to the S3 One Zone-Infrequent Access (S3 One Zone-IA) storage class 30 days after creation. Move all objects to the S3 Glacier Flexible Retrieval storage class after 90 days. Write an expiration action that directs Amazon S3 to delete objects after 90 days.

Answer: C

Explanation:

Here's a breakdown of why option C is the most cost-effective solution for the logging requirements:

The core requirement is to store logs for 90 days with frequent analysis for the first 30 and backup retention for the next 60. Then, delete the logs. The most important factor is cost-effectiveness.

Why S3 Glacier Flexible Retrieval (formerly S3 Glacier) after 30 days? S3 Glacier Flexible Retrieval offers the lowest storage cost of the S3 storage classes designed for data archiving and backup. Since the logs are for backup after 30 days, they will rarely be accessed. S3 Glacier Flexible Retrieval fits well.

Why not S3 Standard-IA or S3 One Zone-IA? While these are cheaper than S3 Standard, they are still more expensive than S3 Glacier Flexible Retrieval, and are designed for more frequent access than required after the initial 30 days. Also, S3 One Zone-IA data is lost if the AZ fails, not suitable for backup retention.

Why use an expiration action? Expiration actions are the built-in, automated way within S3 to delete objects after a specified time. This ensures compliance with the 90-day deletion requirement.

Let's analyze why the other options are not the most cost-effective:

Option A (S3 Standard): S3 Standard is the most expensive S3 storage class, making it unsuitable for long-term storage of logs intended for infrequent access after the initial 30 days. This drastically increases the storage cost without adding any value for logs mainly used for backup.

Option B (S3 Standard-IA then Glacier Flexible Retrieval): While it uses Glacier Flexible Retrieval eventually, the initial 30 days in S3 Standard-IA add unnecessary cost. The logs can be moved to Glacier Flexible Retrieval after 30 days to reduce storage costs. Moving to Glacier after 90 days would defeat the purpose of the backup retention requirements, since the intention is deletion at 90.

Option D (S3 One Zone-IA then Glacier Flexible Retrieval): S3 One Zone-IA's cost is lower than S3 Standard-IA, however data is lost if the single availability zone is compromised, which does not align with data retention for audit purposes. There's also the unnecessary transition to S3 One Zone-IA before moving it to Glacier.

Therefore, Option C provides the most cost-effective solution by using the S3 Glacier Flexible Retrieval storage class for the 60-day backup phase and automating deletion with an S3 lifecycle policy.

Supporting Documentation:

S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
S3 Lifecycle Management: https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-management.html

Question: 535 **Exam Heist**

A company is building an Amazon Elastic Kubernetes Service (Amazon EKS) cluster for its workloads. All secrets that are stored in Amazon EKS must be encrypted in the Kubernetes etcd key-value store.

Which solution will meet these requirements?

A.Create a new AWS Key Management Service (AWS KMS) key. Use AWS Secrets Manager to manage, rotate, and store all secrets in Amazon EKS.

B.Create a new AWS Key Management Service (AWS KMS) key. Enable Amazon EKS KMS secrets encryption on the Amazon EKS cluster.

C.Create the Amazon EKS cluster with default options. Use the Amazon Elastic Block Store (Amazon EBS) Container Storage Interface (CSI) driver as an add-on.

D.Create a new AWS Key Management Service (AWS KMS) key with the alias/aws/ebs alias. Enable default Amazon Elastic Block Store (Amazon EBS) volume encryption for the account.

Answer: B

Explanation:

The correct answer is B: Create a new AWS Key Management Service (AWS KMS) key. Enable Amazon EKS KMS secrets encryption on the Amazon EKS cluster.

Here's a detailed justification:

The requirement is to encrypt Kubernetes secrets stored in the etcd key-value store of an Amazon EKS cluster. Kubernetes stores secrets in etcd, and by default, these secrets are only base64 encoded, not encrypted. Therefore, an encryption mechanism is needed.

Option A, using AWS Secrets Manager, is incorrect because while Secrets Manager can store secrets securely, it doesn't directly encrypt the secrets within the Kubernetes etcd datastore. Secrets Manager would require the application code to retrieve secrets from Secrets Manager, which is a different operational model than directly encrypting secrets managed within Kubernetes. This does not address the direct encryption of secrets stored in etcd.

Option B directly addresses the requirement. Amazon EKS offers the capability to encrypt secrets at rest in etcd using a KMS key. When this feature is enabled, all secrets stored in the etcd database are encrypted using envelope encryption. The data encryption key (DEK) used to encrypt the secrets is itself encrypted by the KMS key (KEK) you provide. This ensures that even if someone gains unauthorized access to the etcd store, the secrets remain encrypted and unreadable without the correct KMS key. Creating a new KMS key provides you with control over the encryption key, aligning with security best practices.

Option C is incorrect because using the EBS CSI driver is for managing persistent volumes attached to the EKS cluster nodes, it doesn't encrypt the secrets stored in etcd. It only deals with the storage and encryption of data on EBS volumes.

Option D is partially correct as EBS encryption encrypts the data at rest on EBS volumes, but it is irrelevant for encrypting secrets within the etcd database. The alias/aws/ebs KMS key is used for EBS volume encryption and is separate from the encryption of secrets within EKS.

Therefore, option B provides the solution that directly fulfills the stated requirement of encrypting Kubernetes secrets in etcd within the EKS cluster using a dedicated KMS key.

Supporting documentation:

Amazon EKS Secrets Encryption: https://docs.aws.amazon.com/eks/latest/userguide/managing-secrets.html
Encrypting Secrets at Rest: https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/

Question: 536 **Exam Heist**

A company wants to provide data scientists with near real-time read-only access to the company's production Amazon RDS for PostgreSQL database. The database is currently configured as a Single-AZ database. The data scientists use complex queries that will not affect the production database. The company needs a solution that is highly available.

Which solution will meet these requirements MOST cost-effectively?

A.Scale the existing production database in a maintenance window to provide enough power for the data scientists.

B.Change the setup from a Single-AZ to a Multi-AZ instance deployment with a larger secondary standby instance. Provide the data scientists access to the secondary instance.

C.Change the setup from a Single-AZ to a Multi-AZ instance deployment. Provide two additional read replicas for the data scientists.

D.Change the setup from a Single-AZ to a Multi-AZ cluster deployment with two readable standby instances. Provide read endpoints to the data scientists.

Answer: C

Explanation:

The requirement is to provide near real-time read-only access to an RDS PostgreSQL database for data scientists without impacting the production database, while maintaining high availability and cost-effectiveness.

Option A is incorrect because scaling the production database does not address the need for read-only access and could still impact production performance, especially with complex queries. It also doesn't inherently improve high availability.

Option B is partially correct in enabling high availability by switching to Multi-AZ. However, providing data scientists direct access to the secondary standby instance is not the intended use case and may interfere with failover operations. The secondary is primarily for failover and not optimized for heavy read workloads.

Option C offers the best balance of requirements. Converting to a Multi-AZ deployment provides high availability. Then creating read replicas off the primary instance allows data scientists to run their complex queries without impacting the production workload. Multiple read replicas further improve read scalability and availability.

Option D is similar to option C but utilizes a Multi-AZ cluster deployment with readable standby instances. This is typically associated with Aurora PostgreSQL, not standard RDS PostgreSQL. Furthermore, while readable standby instances are a good solution, providing **two additional read replicas as in option C** offers more flexibility in distributing the read load and ensuring resilience for the data scientist workload within the context of standard RDS PostgreSQL, potentially at a lower cost compared to migrating to Aurora.

Therefore, option C is the most cost-effective solution that meets all the requirements: near real-time read-only access, minimal impact on the production database, and high availability for the data scientists' workload.

Relevant Links:

Amazon RDS Read Replicas
Amazon RDS Multi-AZ Deployments

Question: 537 **Exam Heist**

A company runs a three-tier web application in the AWS Cloud that operates across three Availability Zones. The application architecture has an Application Load Balancer, an Amazon EC2 web server that hosts user session states, and a MySQL database that runs on an EC2 instance. The company expects sudden increases in application traffic. The company wants to be able to scale to meet future application capacity demands and to ensure high availability across all three Availability Zones.

Which solution will meet these requirements?

A.Migrate the MySQL database to Amazon RDS for MySQL with a Multi-AZ DB cluster deployment. Use Amazon ElastiCache for Redis with high availability to store session data and to cache reads. Migrate the web server to an Auto Scaling group that is in three Availability Zones.

B.Migrate the MySQL database to Amazon RDS for MySQL with a Multi-AZ DB cluster deployment. Use Amazon ElastiCache for Memcached with high availability to store session data and to cache reads. Migrate the web server to an Auto Scaling group that is in three Availability Zones.

C.Migrate the MySQL database to Amazon DynamoDB Use DynamoDB Accelerator (DAX) to cache reads. Store the session data in DynamoDB. Migrate the web server to an Auto Scaling group that is in three Availability Zones.

D.Migrate the MySQL database to Amazon RDS for MySQL in a single Availability Zone. Use Amazon ElastiCache for Redis with high availability to store session data and to cache reads. Migrate the web server to an Auto Scaling group that is in three Availability Zones.

Answer: A

Explanation:

Here's a detailed justification for why option A is the best solution:

The primary goal is to achieve high availability and scalability for a three-tier web application in AWS across three Availability Zones while handling sudden traffic increases.

Database: Migrating the on-premises MySQL database to Amazon RDS for MySQL with a Multi-AZ deployment provides high availability. In case of a failure in the primary AZ, RDS automatically fails over to a standby replica in another AZ. This eliminates single points of failure for the database layer.
https://aws.amazon.com/rds/mysql/

Session Management: Using Amazon ElastiCache for Redis with high availability is ideal for storing session data. Redis is an in-memory data store that provides fast read and write operations, crucial for handling user sessions efficiently. The high availability feature of ElastiCache ensures that session data remains accessible even if a node fails. Redis also supports more advanced data structures beneficial for session management.
https://aws.amazon.com/elasticache/redis/

Web Tier Scalability: Migrating the EC2 web server to an Auto Scaling group across three Availability Zones enables the application to scale horizontally based on demand. Auto Scaling automatically launches and terminates EC2 instances based on predefined metrics, ensuring the application can handle sudden traffic spikes. Spreading instances across multiple AZs provides fault tolerance.
https://aws.amazon.com/autoscaling/

Why other options are less suitable:

Option B: While Memcached is a viable caching solution, Redis offers more advanced features for session management, such as persistence and more complex data structures. For session storage, Redis generally provides better functionality and capabilities.

Option C: Migrating the MySQL database to Amazon DynamoDB could be a valid solution if the application's data model and access patterns are suitable for a NoSQL database. However, DynamoDB is a major architectural shift and requires significant application changes. Also, storing session data in DynamoDB may not be as efficient as using a dedicated in-memory cache like Redis. While DynamoDB Accelerator (DAX) improves performance, it's primarily designed for hot data in DynamoDB, and Redis provides a more targeted solution for session data.

Option D: Deploying the MySQL database in a single Availability Zone introduces a single point of failure. If that Availability Zone experiences an outage, the entire application could become unavailable. This violates the requirement for high availability.

Therefore, option A provides the best solution by addressing scalability and high availability requirements across all three tiers of the application using appropriate AWS services.

Question: 538 **Exam Heist**

A global video streaming company uses Amazon CloudFront as a content distribution network (CDN). The company wants to roll out content in a phased manner across multiple countries. The company needs to ensure that viewers who are outside the countries to which the company rolls out content are not able to view the content.

Which solution will meet these requirements?

A.Add geographic restrictions to the content in CloudFront by using an allow list. Set up a custom error message.

B.Set up a new URL tor restricted content. Authorize access by using a signed URL and cookies. Set up a custom error message.

C.Encrypt the data for the content that the company distributes. Set up a custom error message.

D.Create a new URL for restricted content. Set up a time-restricted access policy for signed URLs.

Answer: A

Explanation:

The correct answer is A: "Add geographic restrictions to the content in CloudFront by using an allow list. Set up a custom error message."

Here's a detailed justification:

The scenario requires a phased rollout of video content based on geographic location. Users outside the target countries should not be able to access the content. CloudFront offers built-in geographic restrictions (Geo Restriction) that are ideal for this requirement. An allow list (or a whitelist) approach specifies the countries where the content is allowed. This aligns directly with the need to allow content only in rolled-out countries.

Using geographic restrictions, CloudFront automatically inspects the viewer's IP address and compares it against the configured allow list. If the viewer's location is not in the allowed countries, CloudFront blocks the request and can display a custom error message. This is the functionality exactly asked for in the prompt.

Option B is incorrect because Signed URLs and Cookies primarily control access based on authentication and authorization rather than geographic location. While they can be combined with other methods to achieve geographic restriction, it's a more complex and less efficient solution than using CloudFront's built-in Geo Restriction. Signed URLs are more appropriate for restricting access based on user credentials or time-based access, not location.

Option C is incorrect because encrypting the data doesn't prevent access based on geographic location. Encryption protects the confidentiality of the data but doesn't control who can access it. Encryption addresses data security, not geographic access control. A user outside of the allowed region could theoretically still download the encrypted data even if they cannot decrypt and view it.

Option D is incorrect. Time-restricted access policies are not a solution to the problem.

Therefore, CloudFront's Geo Restriction configured with an allow list offers the simplest, most efficient, and most direct solution for the stated requirement.

Refer to the official AWS documentation on CloudFront Geo Restriction for more information:

https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/georestrictions.html

Question: 539 **Exam Heist**

A company wants to use the AWS Cloud to improve its on-premises disaster recovery (DR) configuration. The company's core production business application uses Microsoft SQL Server Standard, which runs on a virtual machine (VM). The application has a recovery point objective (RPO) of 30 seconds or fewer and a recovery time objective (RTO) of 60 minutes. The DR solution needs to minimize costs wherever possible.

Which solution will meet these requirements?

A.Configure a multi-site active/active setup between the on-premises server and AWS by using Microsoft SQL Server Enterprise with Always On availability groups.

B.Configure a warm standby Amazon RDS for SQL Server database on AWS. Configure AWS Database Migration Service (AWS DMS) to use change data capture (CDC).

C.Use AWS Elastic Disaster Recovery configured to replicate disk changes to AWS as a pilot light.

D.Use third-party backup software to capture backups every night. Store a secondary set of backups in Amazon S3.

Answer: B

Explanation:

The correct answer is **B. Configure a warm standby Amazon RDS for SQL Server database on AWS. Configure AWS Database Migration Service (AWS DMS) to use change data capture (CDC).**

Here's why:

This solution strikes a balance between meeting the RPO and RTO requirements while minimizing costs, which is a primary concern. Using Amazon RDS for SQL Server in a warm standby configuration keeps a SQL Server instance running on AWS but not actively serving production traffic. AWS DMS with change data capture (CDC) continuously replicates changes from the on-premises SQL Server to the RDS instance. This ensures that the standby database is kept relatively up-to-date, meeting the 30-second RPO.

In the event of a disaster, the RDS instance can be promoted to become the primary database. While a warm standby takes longer than an active/active setup, the cost savings are significant. The RDS instance is pre-configured and ready, reducing the time needed for recovery and aiming to meet the 60-minute RTO.

Option A (Always On availability groups with SQL Server Enterprise) is very expensive due to the licensing costs associated with SQL Server Enterprise. This option is also more complex to manage.

Option C (AWS Elastic Disaster Recovery as pilot light) might not consistently meet the 30-second RPO, as it involves spinning up instances after a disaster. While Elastic Disaster Recovery can reduce costs through minimal infrastructure during normal operation, the restore process itself can impact the RTO if the database requires significant recovery time.

Option D (Backup software to S3) will not meet the RTO. Restoring from backups stored in S3 will take significantly longer than 60 minutes, especially for a substantial database.

Here's a breakdown of why each solution is or isn't the best fit for RPO, RTO, and cost:

RPO (Recovery Point Objective): The objective is 30 seconds. AWS DMS CDC keeps the warm standby database reasonably close to real-time changes from the on-premises database. Options C and D fail here.

RTO (Recovery Time Objective): The objective is 60 minutes. An already provisioned RDS instance can be failed over relatively quickly. Options C and D require more setup time, exceeding the RTO. Option A will meet the RTO but it is too expensive.

Cost: RDS warm standby with DMS is significantly cheaper than running SQL Server Enterprise with Always On Availability Groups (Option A). Options C and D may be cheaper in terms of upfront costs, but they fail to

meet the recovery objectives.

Therefore, option B is the most balanced solution considering all constraints.

Relevant links:

AWS DMS: https://aws.amazon.com/dms/
Amazon RDS for SQL Server: https://aws.amazon.com/rds/sqlserver/
AWS Elastic Disaster Recovery: https://aws.amazon.com/elastic-disaster-recovery/

Question: 540 **Exam Heist**

A company has an on-premises server that uses an Oracle database to process and store customer information. The company wants to use an AWS database service to achieve higher availability and to improve application performance. The company also wants to offload reporting from its primary database system.

Which solution will meet these requirements in the MOST operationally efficient way?

A.Use AWS Database Migration Service (AWS DMS) to create an Amazon RDS DB instance in multiple AWS Regions. Point the reporting functions toward a separate DB instance from the primary DB instance.

B.Use Amazon RDS in a Single-AZ deployment to create an Oracle database. Create a read replica in the same zone as the primary DB instance. Direct the reporting functions to the read replica.

C.Use Amazon RDS deployed in a Multi-AZ cluster deployment to create an Oracle database. Direct the reporting functions to use the reader instance in the cluster deployment.

D.Use Amazon RDS deployed in a Multi-AZ instance deployment to create an Amazon Aurora database. Direct the reporting functions to the reader instances.

Answer: C

Explanation:

The most operationally efficient solution is C, utilizing Amazon RDS in a Multi-AZ cluster deployment for Oracle. This addresses the requirements of high availability, improved application performance, and offloading reporting.

Multi-AZ deployments inherently provide high availability by replicating data across multiple availability zones. If the primary instance fails, automatic failover to a standby instance ensures minimal downtime. This meets the "higher availability" requirement efficiently, without needing to manage cross-region replication (option A).

An RDS Multi-AZ cluster provides a reader endpoint, also known as a reader instance, specifically designed for read-heavy workloads such as reporting. By directing reporting functions to this reader instance, the load on the primary database is significantly reduced, improving the performance of the primary database for transaction processing. This fulfills the need to "offload reporting from its primary database system" and "improve application performance."

Option A, using AWS DMS for cross-region replication, adds operational complexity. DMS requires continuous replication, which consumes resources and introduces potential latency. Furthermore, managing databases across multiple regions increases operational overhead.

Option B, using RDS in a Single-AZ deployment, contradicts the high availability requirement. A read replica in the same AZ offers no resilience against AZ failures.

Option D suggests using Amazon Aurora, which is a different database engine, which might require application changes and a more complex migration. The question implied that the existing Oracle database should be maintained as a critical requirement by mentioning "Oracle database to process and store customer information". Thus making it the most disruptive solution.

Therefore, Option C strikes the best balance between meeting the requirements and minimizing operational overhead, as it provides high availability and read replica capabilities within a single, managed RDS cluster using the same Oracle database engine.

Relevant links:

Amazon RDS Multi-AZ Deployments
Amazon RDS Read Replicas

Question: 541 **Exam Heist**

A company wants to build a web application on AWS. Client access requests to the website are not predictable and can be idle for a long time. Only customers who have paid a subscription fee can have the ability to sign in and use the web application.

Which combination of steps will meet these requirements MOST cost-effectively? (Choose three.)

A.Create an AWS Lambda function to retrieve user information from Amazon DynamoDB. Create an Amazon API Gateway endpoint to accept RESTful APIs. Send the API calls to the Lambda function.

B.Create an Amazon Elastic Container Service (Amazon ECS) service behind an Application Load Balancer to retrieve user information from Amazon RDS. Create an Amazon API Gateway endpoint to accept RESTful APIs. Send the API calls to the Lambda function.

C.Create an Amazon Cognito user pool to authenticate users.

D.Create an Amazon Cognito identity pool to authenticate users.

E.Use AWS Amplify to serve the frontend web content with HTML, CSS, and JS. Use an integrated Amazon CloudFront configuration.

F.Use Amazon S3 static web hosting with PHP, CSS, and JS. Use Amazon CloudFront to serve the frontend web content.

Answer: ACE

Explanation:

The correct answer is ACE. Here's why:

A. Create an AWS Lambda function to retrieve user information from Amazon DynamoDB. Create an Amazon API Gateway endpoint to accept RESTful APIs. Send the API calls to the Lambda function. This is a suitable backend architecture. API Gateway provides a managed, scalable entry point for the web application, handling incoming requests and routing them to the Lambda function. Lambda offers a serverless, cost-effective way to process these requests and interact with DynamoDB. DynamoDB's NoSQL nature is excellent for storing and retrieving user information with low latency. Since client access requests are unpredictable and can be idle, the serverless nature of Lambda ensures that you only pay for actual usage, making it cost-effective.

C. Create an Amazon Cognito user pool to authenticate users. Cognito user pools are designed to handle user registration, authentication, and password recovery. This addresses the requirement that only paying subscribers can access the web application by controlling access to the application based on successful authentication against the user pool. Cognito simplifies the authentication process and integrates seamlessly with other AWS services.

E. Use AWS Amplify to serve the frontend web content with HTML, CSS, and JS. Use an integrated Amazon CloudFront configuration. AWS Amplify is an ideal solution for hosting static frontend content. Amplify offers built-in capabilities for hosting, continuous deployment, and integration with other AWS services. Integrating it with Amazon CloudFront provides a content delivery network (CDN) for faster content delivery and improved user experience globally. This is also cost-effective due to pay-as-you-go pricing and CDN

caching minimizing origin requests.

Why other options are less suitable:

B. ECS with an ALB and RDS, while viable, is a less cost-effective option for unpredictable workloads. ECS requires maintaining servers or containers even when idle, and RDS incurs database costs regardless of usage. Lambda scales down to zero when not in use, making it more efficient in terms of cost.

D. Cognito identity pools are used for providing temporary AWS credentials to users who are already authenticated, whether through a user pool or another identity provider. It doesn't directly handle user authentication itself but rather grants access to AWS resources.

F. S3 static web hosting, without Amplify, would necessitate managing complex configurations, and using PHP for static web hosting is generally incorrect. Also, Amplify provides additional features such as CI/CD pipeline to make the whole process easier.

Supporting Links:

AWS Lambda: https://aws.amazon.com/lambda/
Amazon API Gateway: https://aws.amazon.com/api-gateway/
Amazon DynamoDB: https://aws.amazon.com/dynamodb/
Amazon Cognito: https://aws.amazon.com/cognito/
AWS Amplify: https://aws.amazon.com/amplify/
Amazon CloudFront: https://aws.amazon.com/cloudfront/

Question: 542 **Exam Heist**

A media company uses an Amazon CloudFront distribution to deliver content over the internet. The company wants only premium customers to have access to the media streams and file content. The company stores all content in an Amazon S3 bucket. The company also delivers content on demand to customers for a specific purpose, such as movie rentals or music downloads.

Which solution will meet these requirements?

A.Generate and provide S3 signed cookies to premium customers.
B.Generate and provide CloudFront signed URLs to premium customers.
C.Use origin access control (OAC) to limit the access of non-premium customers.
D.Generate and activate field-level encryption to block non-premium customers.

Answer: B

Explanation:

The correct answer is B: Generate and provide CloudFront signed URLs to premium customers.

Here's a detailed justification:

The media company needs a way to restrict access to its content stored in S3, delivered via CloudFront, so that only premium customers can access it. Several options are available, but CloudFront signed URLs are the most appropriate for on-demand content delivery, like movie rentals and music downloads.

Signed URLs are dynamically generated, time-limited URLs that grant access to specific resources on S3 through the CloudFront distribution. The company can generate these URLs when a premium customer requests content (e.g., rents a movie). The URL contains a cryptographic signature which verifies the authenticity and validity of the request. By implementing this approach, the company retains precise control over who can access the content and for how long, facilitating its on-demand content business model.

Option A, using S3 signed cookies, is more suited for controlling access to multiple restricted files, for instance, an entire "premium content" section of a website. While it provides access control, it's less granular for on-demand scenarios.

Option C, using Origin Access Control (OAC), primarily secures the S3 bucket by allowing only CloudFront to access it. While necessary for overall security, OAC doesn't differentiate between premium and non-premium customers. It only controls access to the origin (S3), not who is requesting content from CloudFront. Premium customer differentiation still needs to be handled within CloudFront.

Option D, field-level encryption, focuses on encrypting specific data fields for security purposes during transit and at rest. It is not directly related to access control and wouldn't prevent non-premium users from attempting to access the media files.

Therefore, generating and providing CloudFront signed URLs provides the most fine-grained and suitable access control mechanism for the media company's requirements, specifically tailored to its on-demand content model.

Reference:

Using Signed URLs - Amazon CloudFront

Question: 543 **Exam Heist**

A company runs Amazon EC2 instances in multiple AWS accounts that are individually bled. The company recently purchased a Savings Pian. Because of changes in the company's business requirements, the company has decommissioned a large number of EC2 instances. The company wants to use its Savings Plan discounts on its other AWS accounts.

Which combination of steps will meet these requirements? (Choose two.)

A.From the AWS Account Management Console of the management account, turn on discount sharing from the billing preferences section.

B.From the AWS Account Management Console of the account that purchased the existing Savings Plan, turn on discount sharing from the billing preferences section. Include all accounts.

C.From the AWS Organizations management account, use AWS Resource Access Manager (AWS RAM) to share the Savings Plan with other accounts.

D.Create an organization in AWS Organizations in a new payer account. Invite the other AWS accounts to join the organization from the management account.

E.Create an organization in AWS Organizations in the existing AWS account with the existing EC2 instances and Savings Plan. Invite the other AWS accounts to join the organization from the management account.

Answer: AE

Explanation:

The correct answer is AE because it leverages AWS Organizations to enable Savings Plan discount sharing across accounts. Here's a detailed justification:

A. From the AWS Account Management Console of the management account, turn on discount sharing from the billing preferences section.

This step is correct because it's the mechanism AWS Organizations provides for distributing the benefits of Savings Plans to other accounts within the organization. Without enabling discount sharing, Savings Plan benefits are confined to the account that purchased it. The management account (payer account in AWS Organizations) is the central point for controlling billing and cost management. This is done in the Billing console, specifically under Billing preferences where you configure Savings Plans sharing. This ensures the

discounts aren't wasted and are efficiently applied.

E. Create an organization in AWS Organizations in the existing AWS account with the existing EC2 instances and Savings Plan. Invite the other AWS accounts to join the organization from the management account.

This step is necessary to group the AWS accounts together under a single management account. AWS Organizations provides centralized governance and management across multiple AWS accounts. By creating an organization and inviting other accounts to join, you can leverage features like consolidated billing and, most importantly in this case, Savings Plan sharing. Placing the Savings Plan-owning account as the management account ensures the Savings Plan is readily available to be shared across the newly formed organization. Moving all accounts into an Organization allows centralized visibility and control.

Why the other options are incorrect:

B: While discount sharing is configured from the existing account with the Savings Plan, doing it from the "AWS Account Management Console" and not correctly setting up an AWS Organization would not be the correct mechanism, as it has to be paired with AWS Organizations to work effectively. This option fails to consider the underlying organizational structure required for proper sharing.
C: AWS RAM (Resource Access Manager) is generally used for sharing specific resources within an AWS organization, not for broadly sharing Savings Plan benefits. While you can share specific compute resources with RAM, savings plan discounts need to be organization-wide sharing which is handled directly by the organization settings. The purpose of RAM is different.
D: Creating a new payer account adds unnecessary complexity. The savings plan and the associated EC2 instance utilization already exists in a specific account. Migrating everything to a new organization and payer account is an overcomplicated solution. This would require migrating existing AWS infrastructure.

Authoritative Links:

AWS Organizations: https://aws.amazon.com/organizations/
Savings Plans Sharing in AWS Organizations:
https://docs.aws.amazon.com/savingsplans/latest/userguide/sp-sharing.html
AWS Resource Access Manager (RAM): https://aws.amazon.com/ram/

Question: 544 **Exam Heist**

A retail company uses a regional Amazon API Gateway API for its public REST APIs. The API Gateway endpoint is a custom domain name that points to an Amazon Route 53 alias record. A solutions architect needs to create a solution that has minimal effects on customers and minimal data loss to release the new version of APIs.

Which solution will meet these requirements?

A.Create a canary release deployment stage for API Gateway. Deploy the latest API version. Point an appropriate percentage of traffic to the canary stage. After API verification, promote the canary stage to the production stage.

B.Create a new API Gateway endpoint with a new version of the API in OpenAPI YAML file format. Use the import-to-update operation in merge mode into the API in API Gateway. Deploy the new version of the API to the production stage.

C.Create a new API Gateway endpoint with a new version of the API in OpenAPI JSON file format. Use the import-to-update operation in overwrite mode into the API in API Gateway. Deploy the new version of the API to the production stage.

D.Create a new API Gateway endpoint with new versions of the API definitions. Create a custom domain name for the new API Gateway API. Point the Route 53 alias record to the new API Gateway API custom domain name.

Answer: A

Explanation:

The correct answer is **A. Create a canary release deployment stage for API Gateway. Deploy the latest API version. Point an appropriate percentage of traffic to the canary stage. After API verification, promote the canary stage to the production stage.**

Here's a detailed justification:

Canary Deployment Minimizes Risk: Canary deployment involves releasing a new version of the API to a small subset of users (defined by percentage of traffic). This allows you to test the new version in a real-world environment with minimal impact if issues arise.

Controlled Rollout: By directing a percentage of traffic to the canary stage, you can monitor the performance and stability of the new API version before exposing it to the entire user base. If any issues are detected, the traffic can be quickly reverted to the original production version.

Minimal Data Loss: A canary deployment inherently minimizes data loss as only a fraction of the requests are routed to the new version initially. Any errors or data discrepancies will affect only a small group of users, limiting the potential impact.

Seamless Transition: Once the canary version is validated, promoting it to the production stage effectively replaces the old version without disrupting the entire user base at once. This provides a smoother transition.

API Gateway Support: Amazon API Gateway provides built-in support for canary deployments through its deployment stages and traffic management features.

Alternatives and Why They Aren't Best:

B and C (Import-to-Update): While import-to-update can be used for API updates, it lacks the controlled rollout and risk mitigation of a canary deployment. An overwrite approach (option C) is particularly risky, as it immediately replaces the existing API and could lead to widespread issues. Merging (option B) can be complex to manage and doesn't isolate the rollout for initial validation.
D (New API Gateway Endpoint): Creating a new API Gateway endpoint with a custom domain name and changing the Route 53 record leads to a potentially disruptive switchover. All users would immediately be directed to the new API. While blue/green deployments can be useful, they require more infrastructure and aren't as gradual as canary releases. Furthermore, it needs a DNS change, creating possible propagation issues.

In summary, a canary release in API Gateway provides a controlled, low-risk approach for deploying new API versions, ensuring minimal impact on customers and data loss.

Reference:AWS API Gateway Canary DeploymentsAWS API Gateway Deployment Stages

Question: 545 **Exam Heist**

A company wants to direct its users to a backup static error page if the company's primary website is unavailable. The primary website's DNS records are hosted in Amazon Route 53. The domain is pointing to an Application Load Balancer (ALB). The company needs a solution that minimizes changes and infrastructure overhead.

Which solution will meet these requirements?

A.Update the Route 53 records to use a latency routing policy. Add a static error page that is hosted in an Amazon S3 bucket to the records so that the traffic is sent to the most responsive endpoints.

B.Set up a Route 53 active-passive failover configuration. Direct traffic to a static error page that is hosted in an Amazon S3 bucket when Route 53 health checks determine that the ALB endpoint is unhealthy.

C.Set up a Route 53 active-active configuration with the ALB and an Amazon EC2 instance that hosts a static

error page as endpoints. Configure Route 53 to send requests to the instance only if the health checks fail for the ALB.

D.Update the Route 53 records to use a multivalue answer routing policy. Create a health check. Direct traffic to the website if the health check passes. Direct traffic to a static error page that is hosted in Amazon S3 if the health check does not pass.

Answer: B

Explanation:

The correct answer is B: Set up a Route 53 active-passive failover configuration. Direct traffic to a static error page that is hosted in an Amazon S3 bucket when Route 53 health checks determine that the ALB endpoint is unhealthy.

Here's why this is the best solution and why the others are not ideal:

Active-Passive Failover: Route 53's active-passive failover is designed precisely for scenarios where you have a primary endpoint (the ALB in this case) and a backup endpoint (the static error page). Route 53 continuously monitors the health of the primary endpoint using health checks. If the health check fails (indicating the ALB is down), Route 53 automatically starts routing traffic to the backup endpoint. This provides a seamless failover experience for users.
S3 Static Website Hosting: Hosting the static error page in an S3 bucket configured for static website hosting is a cost-effective and highly available solution. S3 offers high durability and availability, making it ideal for serving static content.
Minimizes Changes and Overhead: This solution minimizes changes because it leverages existing Route 53 functionality and S3, which are common AWS services. It avoids the need for complex configurations or additional infrastructure like EC2 instances specifically for error pages.

Let's examine why the other options are less suitable:

A (Latency Routing Policy): Latency routing routes traffic to the endpoint with the lowest latency. It's not designed for failover scenarios. If the ALB is unhealthy, latency routing won't necessarily direct traffic away from it.
C (Active-Active with EC2): While active-active configurations are suitable for distributing traffic, using an EC2 instance to host a static error page adds unnecessary overhead and management complexity compared to S3. Furthermore, an EC2 instance requires patching, monitoring, and scaling, which are not needed for a simple error page. Also, configuring health checks to only fail over to the EC2 instance after the ALB fails requires custom configuration and is less efficient than Route 53's built-in failover mechanism.
D (Multivalue Answer Routing Policy): While multivalue answer routing can provide redundancy, it's not designed for a true failover scenario. If the ALB is unhealthy, Route 53 might still return its IP address alongside the S3 bucket endpoint, potentially leading to inconsistent behavior for users. It distributes the traffic across multiple healthy resources, not to a single backup resource when the primary is unhealthy.

Authoritative Links:

Route 53 Health Checks: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/health-checks-creating-deleting.html
Route 53 Failover: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-failover.html
S3 Static Website Hosting: https://docs.aws.amazon.com/AmazonS3/latest/userguide/WebsiteHosting.html

Question: 546 **Exam Heist**

A recent analysis of a company's IT expenses highlights the need to reduce backup costs. The company's chief information officer wants to simplify the on-premises backup infrastructure and reduce costs by eliminating the use of physical backup tapes. The company must preserve the existing investment in the on-premises backup

applications and workflows.

What should a solutions architect recommend?

A.Set up AWS Storage Gateway to connect with the backup applications using the NFS interface.

B.Set up an Amazon EFS file system that connects with the backup applications using the NFS interface.

C.Set up an Amazon EFS file system that connects with the backup applications using the iSCSI interface.

D.Set up AWS Storage Gateway to connect with the backup applications using the iSCSI-virtual tape library (VTL) interface.

Answer: D

Explanation:

The best solution is **D. Set up AWS Storage Gateway to connect with the backup applications using the iSCSI-virtual tape library (VTL) interface.**

Here's why:

Preserves Existing Investment: The question explicitly requires maintaining the existing on-premises backup applications and workflows. The VTL interface of AWS Storage Gateway is designed to seamlessly integrate with existing backup software that's traditionally used with physical tape libraries. It presents itself as a virtual tape library to the on-premises application, requiring minimal or no changes to the backup process.

Eliminates Physical Tapes: The VTL interface allows the backup application to write data to virtual tapes stored in AWS. These virtual tapes are then stored in Amazon S3 or Amazon S3 Glacier Deep Archive, effectively replacing the physical tapes and eliminating the need for physical tape management.

Cost Reduction: By moving backups to AWS storage services (S3 or Glacier), the company can leverage the cost-effective nature of cloud storage, particularly S3 Glacier Deep Archive for long-term archival, leading to significant cost savings compared to managing physical tapes.

AWS Storage Gateway's Capabilities: AWS Storage Gateway provides a bridge between on-premises environments and AWS. Its VTL interface is specifically designed for tape backup replacement.

Why Other Options Are Less Suitable:

A. NFS interface with Storage Gateway: While the NFS interface could allow backup applications to write to AWS, it's less tailored to the existing backup workflow using tape libraries. It might require more significant changes to the backup application configuration.
B. & C. Amazon EFS: Amazon EFS is a file storage service ideal for shared file systems but not typically used as a direct replacement for tape-based backups. Integrating EFS with backup software designed for tape drives would require more substantial application modifications and may not be the most efficient approach. The iSCSI interface of EFS is primarily intended for database, web server and development workloads, not large scale backup scenarios.

In conclusion, the VTL interface of AWS Storage Gateway directly addresses the core requirement of replacing physical tapes while preserving the existing on-premises backup applications and workflows with minimal disruption, enabling cost reduction through the use of AWS storage services.

Further Research:

AWS Storage Gateway: https://aws.amazon.com/storagegateway/
AWS Storage Gateway VTL: https://docs.aws.amazon.com/storagegateway/latest/userguide/VTL.html

Question: 547 **Exam Heist**

A company has data collection sensors at different locations. The data collection sensors stream a high volume of data to the company. The company wants to design a platform on AWS to ingest and process high-volume streaming data. The solution must be scalable and support data collection in near real time. The company must store the data in Amazon S3 for future reporting.

Which solution will meet these requirements with the LEAST operational overhead?

A. Use Amazon Kinesis Data Firehose to deliver streaming data to Amazon S3.
B. Use AWS Glue to deliver streaming data to Amazon S3.
C. Use AWS Lambda to deliver streaming data and store the data to Amazon S3.
D. Use AWS Database Migration Service (AWS DMS) to deliver streaming data to Amazon S3.

Answer: A

Explanation:

The correct answer is A: Use Amazon Kinesis Data Firehose to deliver streaming data to Amazon S3. Here's why:

Kinesis Data Firehose is specifically designed for near real-time ingestion, transformation, and loading of streaming data into data lakes, data stores, and analytics services like Amazon S3. It provides a managed service, thus minimizing operational overhead. Firehose automatically scales to handle varying data volumes, buffers data, and can optionally transform it before delivering to S3. It also handles error management and retries, simplifying the data ingestion pipeline.

Option B is incorrect because AWS Glue is primarily an ETL (Extract, Transform, Load) service focused on preparing data for analytics, typically from databases or data warehouses. While Glue can process data, it is not optimized for continuous, high-volume streaming data ingestion like Kinesis Data Firehose is. It involves more setup and operational overhead for streaming data scenarios.

Option C is incorrect because while Lambda can process data, directly using Lambda to handle high-volume streaming data and store it in S3 would lead to increased operational complexity. Managing scaling, error handling, and concurrency for Lambda functions in such a scenario requires significant effort. Furthermore, Lambda functions have execution time limits and payload size restrictions that might hinder processing high-volume streaming data reliably. Lambda is typically used for event-driven processing, often triggered by Firehose or other streaming services.

Option D is incorrect because AWS DMS is designed for database migration, not for general streaming data ingestion. It is used to migrate data between different database platforms, not to collect and deliver streaming data to Amazon S3 for analytical purposes. Using DMS for this purpose would be an inappropriate and highly inefficient solution.

In summary, Kinesis Data Firehose is the best choice because it is a managed service specifically built for ingesting streaming data into S3 with minimal operational overhead, automatic scaling, and built-in buffering and transformation capabilities.

For further reading:

Amazon Kinesis Data Firehose: https://aws.amazon.com/kinesis/data-firehose/
AWS Glue: https://aws.amazon.com/glue/
AWS Lambda: https://aws.amazon.com/lambda/
AWS Database Migration Service: https://aws.amazon.com/dms/

Question: 548 **Exam Heist**

A company has separate AWS accounts for its finance, data analytics, and development departments. Because of costs and security concerns, the company wants to control which services each AWS account can use.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use AWS Systems Manager templates to control which AWS services each department can use.

B.Create organization units (OUs) for each department in AWS Organizations. Attach service control policies (SCPs) to the OUs.

C.Use AWS CloudFormation to automatically provision only the AWS services that each department can use.

D.Set up a list of products in AWS Service Catalog in the AWS accounts to manage and control the usage of specific AWS services.

Answer: B

Explanation:

The correct answer is B: Create organization units (OUs) for each department in AWS Organizations. Attach service control policies (SCPs) to the OUs.

Here's a detailed justification:

AWS Organizations enables centralized management and governance across multiple AWS accounts. By creating separate OUs for each department (finance, data analytics, and development), the company can group the accounts logically based on their functional areas. Service Control Policies (SCPs) are a powerful feature of AWS Organizations that allows you to define policies that specify the maximum permissions available to the accounts within an OU. These policies act as guardrails, preventing accounts from performing actions that are not explicitly allowed, regardless of the IAM permissions granted within individual accounts. SCPs address the company's need to control which services each AWS account can use, aligning with both cost and security concerns. The SCPs restrict service access at the organizational level, providing centralized control. This centralized approach minimizes operational overhead because policies are defined and managed in one place (the AWS Organizations management account), rather than individually across multiple accounts. SCPs inherit down the organizational hierarchy, ensuring consistent policy enforcement.

Option A is incorrect because AWS Systems Manager templates are primarily used for automating infrastructure management tasks within a single account, not for controlling service access across multiple accounts in an organization. It lacks the centralized policy enforcement capability offered by SCPs.

Option C is not the best solution because AWS CloudFormation is designed for infrastructure-as-code and provisioning resources. While you can control which services are used through CloudFormation templates, it doesn't provide a centralized, organization-wide policy enforcement mechanism like SCPs. Managing separate CloudFormation stacks for each department to restrict service usage would increase operational complexity.

Option D is incorrect because AWS Service Catalog focuses on creating and managing catalogs of IT services that users can request. It helps standardize the deployment of applications and resources but does not prevent accounts from directly accessing or using restricted AWS services. It's more about managing product offerings, not controlling access across accounts.

In summary, AWS Organizations with SCPs provides the most efficient and centralized way to control service access across multiple AWS accounts, minimizing operational overhead and meeting the company's requirements.

Relevant links:

AWS Organizations: https://aws.amazon.com/organizations/
Service Control Policies (SCPs):

https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html

Question: 549 **Exam Heist**

A company has created a multi-tier application for its ecommerce website. The website uses an Application Load Balancer that resides in the public subnets, a web tier in the public subnets, and a MySQL cluster hosted on Amazon EC2 instances in the private subnets. The MySQL database needs to retrieve product catalog and pricing information that is hosted on the internet by a third-party provider. A solutions architect must devise a strategy that maximizes security without increasing operational overhead.

What should the solutions architect do to meet these requirements?

A.Deploy a NAT instance in the VPC. Route all the internet-based traffic through the NAT instance.

B.Deploy a NAT gateway in the public subnets. Modify the private subnet route table to direct all internet-bound traffic to the NAT gateway.

C.Configure an internet gateway and attach it to the VPModify the private subnet route table to direct internet-bound traffic to the internet gateway.

D.Configure a virtual private gateway and attach it to the VPC. Modify the private subnet route table to direct internet-bound traffic to the virtual private gateway.

Answer: B

Explanation:

The correct answer is **B. Deploy a NAT gateway in the public subnets. Modify the private subnet route table to direct all internet-bound traffic to the NAT gateway.**

Here's a detailed justification:

The requirement is to allow the MySQL database instances in the private subnets to access the internet to retrieve product catalog information from a third-party provider while maximizing security and minimizing operational overhead. The database instances should not be directly exposed to the internet.

NAT Gateway (Option B): A NAT (Network Address Translation) gateway allows instances in a private subnet to connect to the internet or other AWS services, but prevents the internet from initiating a connection with those instances. This is a key security feature. Deploying a NAT gateway in the public subnets and routing the database tier's internet-bound traffic through it ensures that the database instances can reach the third-party service without being directly exposed. NAT gateway is also managed by AWS, so the operational overhead is minimal. It automatically scales based on demand.

NAT Instance (Option A): While a NAT instance can provide similar functionality to a NAT gateway, it requires more manual configuration and management. You need to manage patching, scaling, and high availability. This increases operational overhead, which contradicts the requirements. Further, the NAT instance can become a bottleneck as it is a single EC2 instance.

Internet Gateway (Option C): An internet gateway allows direct internet access to resources within the VPC. Directly routing the database tier's traffic to the internet gateway would expose the database instances to the internet, which compromises security. This violates the requirement to maximize security. This would essentially make the subnet a public subnet.

Virtual Private Gateway (Option D): A virtual private gateway is used to connect the VPC to an on-premises network using VPN or Direct Connect. It is not used for general internet access from private subnets. It's purpose is to allow access into the VPC, which is the opposite of what is needed.

Therefore, a NAT gateway is the optimal solution because it provides secure internet access from private subnets without direct internet exposure and has minimal operational overhead due to being a managed

service.

Authoritative Links:

NAT Gateway: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html
NAT Instances vs NAT Gateways: https://aws.amazon.com/premiumsupport/knowledge-center/nat-gateway-instance/

Question: 550 **Exam Heist**

A company is using AWS Key Management Service (AWS KMS) keys to encrypt AWS Lambda environment variables. A solutions architect needs to ensure that the required permissions are in place to decrypt and use the environment variables.

Which steps must the solutions architect take to implement the correct permissions? (Choose two.)

A.Add AWS KMS permissions in the Lambda resource policy.
B.Add AWS KMS permissions in the Lambda execution role.
C.Add AWS KMS permissions in the Lambda function policy.
D.Allow the Lambda execution role in the AWS KMS key policy.
E.Allow the Lambda resource policy in the AWS KMS key policy.

Answer: BD

Explanation:

Let's break down why options B and D are the correct choices for managing KMS permissions when Lambda uses KMS-encrypted environment variables.

B. Add AWS KMS permissions in the Lambda execution role: The Lambda function's execution role is the IAM role assumed by Lambda when it executes your code. It needs permission to decrypt the environment variables using the KMS key. Therefore, the execution role must explicitly grant the kms:Decrypt permission (and potentially kms:GenerateDataKey if data keys are being used). This allows the Lambda function, when running, to call the KMS API to decrypt the environment variables before the function code executes.

D. Allow the Lambda execution role in the AWS KMS key policy: The KMS key policy defines who can use the key. It needs to explicitly allow the Lambda execution role to use the key for decryption. This is achieved by adding a statement to the key policy granting the kms:Decrypt permission to the Lambda execution role's ARN. Without this, KMS will deny the Lambda function's request to decrypt the environment variables. Essentially, the execution role says, "I need to decrypt something," and the KMS key policy says, "Okay, I trust this role to decrypt using this key."

Let's look at why the other options are incorrect:

A. Add AWS KMS permissions in the Lambda resource policy: The Lambda resource policy grants permissions to entities outside of Lambda to invoke the function. It's primarily used to allow services like API Gateway or S3 to trigger the Lambda function. It is not used for managing permissions related to decrypting environment variables within the Lambda function's execution environment.

C. Add AWS KMS permissions in the Lambda function policy: The Lambda function policy isn't a typical term. Lambda uses a resource-based policy (the resource policy mentioned above) to manage who can invoke it, and uses an execution role to manage what the Lambda function can access. There isn't a separate "function policy" that would be relevant here.

E. Allow the Lambda resource policy in the AWS KMS key policy: The resource policy controls who can

invoke the Lambda function. The decryption process is related to what the Lambda function does after it's invoked. Therefore, putting the resource policy in the KMS key policy is incorrect. The key policy needs to trust the execution role which is doing the decrypting, not whoever is invoking the Lambda function.

In short, Lambda's execution role assumes permissions to interact with AWS resources and the KMS key policy grants those permissions to decrypt the variables.

Further research links:

AWS KMS Key Policies
Using AWS Lambda environment variables
AWS Lambda Execution Role

Question: 551 **Exam Heist**

A company has a financial application that produces reports. The reports average 50 KB in size and are stored in Amazon S3. The reports are frequently accessed during the first week after production and must be stored for several years. The reports must be retrievable within 6 hours.

Which solution meets these requirements MOST cost-effectively?

A.Use S3 Standard. Use an S3 Lifecycle rule to transition the reports to S3 Glacier after 7 days.

B.Use S3 Standard. Use an S3 Lifecycle rule to transition the reports to S3 Standard-Infrequent Access (S3 Standard-IA) after 7 days.

C.Use S3 Intelligent-Tiering. Configure S3 Intelligent-Tiering to transition the reports to S3 Standard-Infrequent Access (S3 Standard-IA) and S3 Glacier.

D.Use S3 Standard. Use an S3 Lifecycle rule to transition the reports to S3 Glacier Deep Archive after 7 days.

Answer: A

Explanation:

Here's a detailed justification for why option A is the most cost-effective solution, along with supporting concepts and links:

The problem requires frequent access for the first week and long-term archival with a retrieval time of under 6 hours. The cost-effectiveness is paramount.

Option A leverages S3 Standard for the initial frequent access period. S3 Standard offers high availability, durability, and performance suitable for frequently accessed objects. After 7 days, the lifecycle rule automatically transitions the reports to S3 Glacier.

S3 Glacier is designed for low-cost archival storage. It's significantly cheaper than S3 Standard and S3 Standard-IA for long-term storage. The retrieval time for S3 Glacier, even Glacier Flexible Retrieval, meets the requirement of being retrievable within 6 hours.

Option B is less cost-effective because S3 Standard-IA is more expensive than S3 Glacier for long-term storage. While S3 Standard-IA provides faster retrieval than Glacier, this isn't necessary after the first week, so the extra cost is not justified.

Option C, S3 Intelligent-Tiering, automatically moves data between tiers based on access patterns. However, since the access pattern is clearly defined (frequent for 1 week, infrequent after), manually controlled transitions using Lifecycle rules provide better cost optimization in this scenario. Intelligent Tiering also has a monitoring cost that may be unnecessary.

Option D, transitioning to S3 Glacier Deep Archive after 7 days, would not be ideal since the minimum retrieval

time for Glacier Deep Archive is 12 hours, violating the 6-hour requirement. Also, for archival, it would be even more cost-effective than Glacier, the slower retreival time is not useful.

Therefore, the best balance of cost and retrieval time is achieved by storing the data initially in S3 Standard and then transitioning it to S3 Glacier using a lifecycle rule after one week.

Key Concepts:

S3 Storage Classes: Understanding the different S3 storage classes (Standard, Standard-IA, Glacier, Deep Archive) and their cost/performance trade-offs is crucial.
S3 Lifecycle Rules: Lifecycle rules automate the movement of objects between storage classes or their deletion, reducing storage costs.
Cost Optimization: Choosing the most cost-effective storage solution involves analyzing access patterns and selecting the appropriate storage class with the desired balance of availability, durability, and retrieval time.

Authoritative Links:

Amazon S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
S3 Lifecycle Management: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-examples.html
S3 Glacier: https://aws.amazon.com/glacier/

Question: 552 **Exam Heist**

A company needs to optimize the cost of its Amazon EC2 instances. The company also needs to change the type and family of its EC2 instances every 2-3 months.

What should the company do to meet these requirements?

A.Purchase Partial Upfront Reserved Instances for a 3-year term.
B.Purchase a No Upfront Compute Savings Plan for a 1-year term.
C.Purchase All Upfront Reserved Instances for a 1-year term.
D.Purchase an All Upfront EC2 Instance Savings Plan for a 1-year term.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer, along with supporting information:

The company's primary goals are cost optimization for EC2 instances and flexibility to change instance types and families frequently (every 2-3 months). Let's analyze why each option is either suitable or unsuitable:

Option A: Purchase Partial Upfront Reserved Instances for a 3-year term. While Reserved Instances offer discounts, a 3-year term introduces inflexibility. The company needs to change instance types frequently, making a long-term commitment less desirable. If instance types are changed, the Reserved Instance might not apply, wasting money.

Option B: Purchase a No Upfront Compute Savings Plan for a 1-year term. Compute Savings Plans provide significant discounts compared to On-Demand pricing, similar to Reserved Instances. The key advantage is flexibility. Compute Savings Plans apply to EC2 instances regardless of instance family, size, OS, or tenancy, as long as compute usage stays within the committed amount. Because it is a "no upfront" option, the company can benefit from savings while retaining the ability to change instance types as needed. A 1-year term balances cost savings with flexibility better than a 3-year term.

Option C: Purchase All Upfront Reserved Instances for a 1-year term. Similar to option A, Reserved Instances

lack the flexibility the company needs. An "all upfront" payment means the company pays the full cost upfront, which might not be ideal if resources change quickly.

Option D: Purchase an All Upfront EC2 Instance Savings Plan for a 1-year term. Instance Savings Plans offer discounts specific to instance families within a region. While providing savings, these plans are less flexible than Compute Savings Plans because they are tied to specific instance types/families. The all upfront cost is not desirable, and the limited scope compared to compute savings plans makes this option worse.

Why Compute Savings Plans are optimal:

Compute Savings Plans are designed for scenarios where consistent compute usage is expected, but the specific EC2 instance configurations might change. They provide a commitment to a certain amount of compute spending per hour across a region and are suitable for different instance types, sizes, operating systems, and tenancies. This flexibility aligns perfectly with the company's need to change instance types frequently.

In Summary: Option B, purchasing a No Upfront Compute Savings Plan for a 1-year term, provides the best balance of cost savings and flexibility to accommodate the company's evolving EC2 instance requirements.

Authoritative Links:

AWS Savings Plans: https://aws.amazon.com/savingsplans/
Amazon EC2 Pricing: https://aws.amazon.com/ec2/pricing/ (compare Savings Plans to other options)

Question: 553 **Exam Heist**

A solutions architect needs to review a company's Amazon S3 buckets to discover personally identifiable information (PII). The company stores the PII data in the us-east-1 Region and us-west-2 Region.

Which solution will meet these requirements with the LEAST operational overhead?

A.Configure Amazon Macie in each Region. Create a job to analyze the data that is in Amazon S3.

B.Configure AWS Security Hub for all Regions. Create an AWS Config rule to analyze the data that is in Amazon S3.

C.Configure Amazon Inspector to analyze the data that is in Amazon S3.

D.Configure Amazon GuardDuty to analyze the data that is in Amazon S3.

Answer: A

Explanation:

The correct answer is A because Amazon Macie is specifically designed for discovering and protecting sensitive data like PII stored in Amazon S3. Macie automates the process of identifying PII by leveraging machine learning and pattern matching techniques. Configuring Macie in each region (us-east-1 and us-west-2) and creating a job to analyze the S3 data allows the solution architect to pinpoint the exact locations where PII is stored. This requires minimal configuration and ongoing maintenance.

Option B is incorrect because AWS Security Hub, while providing a consolidated view of security alerts and compliance status, doesn't inherently analyze the contents of S3 buckets for PII. Creating an AWS Config rule might identify if the S3 buckets have appropriate encryption or access controls, but it won't perform content analysis to locate PII. Security Hub primarily aggregates findings from other services.

Option C is wrong because Amazon Inspector is a vulnerability management service focused on assessing EC2 instances and container images for software vulnerabilities and unintended network exposure. It doesn't directly analyze data within S3 buckets for PII.

Option D is incorrect because Amazon GuardDuty is a threat detection service that monitors for malicious activity and unauthorized behavior within the AWS environment. It doesn't inspect the contents of S3 buckets for PII. GuardDuty focuses on network traffic, API calls, and log analysis to detect threats, not data content.

Therefore, Macie provides the most efficient and specialized solution for the problem of identifying PII within S3 buckets with the least operational overhead. It directly addresses the requirement to discover sensitive data stored within S3.

Further research:

Amazon Macie: https://aws.amazon.com/macie/

Question: 554 **Exam Heist**

A company's SAP application has a backend SQL Server database in an on-premises environment. The company wants to migrate its on-premises application and database server to AWS. The company needs an instance type that meets the high demands of its SAP database. On-premises performance data shows that both the SAP application and the database have high memory utilization.

Which solution will meet these requirements?

A.Use the compute optimized instance family for the application. Use the memory optimized instance family for the database.

B.Use the storage optimized instance family for both the application and the database.

C.Use the memory optimized instance family for both the application and the database.

D.Use the high performance computing (HPC) optimized instance family for the application. Use the memory optimized instance family for the database.

Answer: C

Explanation:

The correct answer is C because the problem statement explicitly mentions high memory utilization for both the SAP application and the SQL Server database.

Option A is incorrect because while the application server might benefit from compute optimization, the database server requires memory optimization due to the stated high memory usage. Compute-optimized instances focus on CPU performance rather than memory.

Option B is incorrect because storage-optimized instances are designed for applications that require high, sequential read and write access to large datasets on local storage. This does not address the problem's core requirement of high memory utilization for the application and database.

Option D is incorrect because HPC-optimized instances are tailored for computationally intensive workloads like scientific simulations, weather modeling, and deep learning, which are not implied in this scenario of migrating an SAP application and SQL Server database. The primary constraint here is memory, not extreme computational power.

Memory-optimized instances are designed to deliver fast performance for workloads that process large datasets in memory. Instance families like R5, R6i, and X2gd are excellent choices for memory-intensive applications. The description of the scenario clearly indicates the importance of in-memory performance for both the SAP application and the SQL Server database. Therefore, using memory-optimized instances for both ensures that the applications can efficiently access and process data, preventing performance bottlenecks due to memory constraints.

Further research:

AWS EC2 Instance Types: https://aws.amazon.com/ec2/instance-types/ - This page provides detailed information about all EC2 instance types and their intended use cases.
Memory Optimized Instances: https://aws.amazon.com/ec2/instance-types/#Memory_optimized - Specifically reviews the families best for in memory intensive tasks.

Question: 555 **Exam Heist**

A company runs an application in a VPC with public and private subnets. The VPC extends across multiple Availability Zones. The application runs on Amazon EC2 instances in private subnets. The application uses an Amazon Simple Queue Service (Amazon SQS) queue.

A solutions architect needs to design a secure solution to establish a connection between the EC2 instances and the SQS queue.

Which solution will meet these requirements?

A.Implement an interface VPC endpoint for Amazon SQS. Configure the endpoint to use the private subnets. Add to the endpoint a security group that has an inbound access rule that allows traffic from the EC2 instances that are in the private subnets.

B.Implement an interface VPC endpoint for Amazon SQS. Configure the endpoint to use the public subnets. Attach to the interface endpoint a VPC endpoint policy that allows access from the EC2 instances that are in the private subnets.

C.Implement an interface VPC endpoint for Amazon SQS. Configure the endpoint to use the public subnets. Attach an Amazon SQS access policy to the interface VPC endpoint that allows requests from only a specified VPC endpoint.

D.Implement a gateway endpoint for Amazon SQS. Add a NAT gateway to the private subnets. Attach an IAM role to the EC2 instances that allows access to the SQS queue.

Answer: A

Explanation:

The correct solution to securely connect EC2 instances in private subnets to an SQS queue is to use an interface VPC endpoint configured within the private subnets. Here's why:

Interface VPC Endpoints for SQS: Interface endpoints provide private connectivity to AWS services, in this case, SQS, without exposing traffic to the public internet. This enhances security and reduces latency. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html
Placement in Private Subnets: Placing the interface endpoint in the private subnets ensures that EC2 instances can reach SQS privately, as the instances themselves reside within the private subnets and are not exposed to the public internet.
Security Group Rules: Security groups act as virtual firewalls. An inbound rule on the endpoint's security group should allow traffic from the EC2 instances' security group. This controls which instances can access the SQS queue through the endpoint.
VPC Endpoint Policies: While VPC endpoint policies can restrict access, the primary control here is through the security group associated with the interface endpoint, simplifying the configuration and security management.
Why other options are incorrect:

Public subnets: Placing the endpoint in public subnets defeats the purpose of private connectivity.
Gateway Endpoints: Gateway endpoints only support S3 and DynamoDB. SQS requires an interface endpoint. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-s3.html
NAT Gateway + IAM Role: While a NAT gateway enables outbound internet access from private subnets and an IAM role grants permissions, this approach still requires traffic to traverse the public internet to reach SQS, which is less secure. Furthermore, it's an unnecessary and costlier setup when private connectivity using an interface endpoint is available.

SQS Access Policy: While SQS access policies are relevant, in the context of VPC endpoints the endpoint policy and security group configurations together provide a better approach for defining access based on the VPC endpoint itself and the security contexts of the resources making the requests.

Therefore, implementing an interface VPC endpoint in the private subnets with appropriate security group rules provides the most secure and efficient solution.

Question: 556 **Exam Heist**

A solutions architect is using an AWS CloudFormation template to deploy a three-tier web application. The web application consists of a web tier and an application tier that stores and retrieves user data in Amazon DynamoDB tables. The web and application tiers are hosted on Amazon EC2 instances, and the database tier is not publicly accessible. The application EC2 instances need to access the DynamoDB tables without exposing API credentials in the template.

What should the solutions architect do to meet these requirements?

A.Create an IAM role to read the DynamoDB tables. Associate the role with the application instances by referencing an instance profile.

B.Create an IAM role that has the required permissions to read and write from the DynamoDB tables. Add the role to the EC2 instance profile, and associate the instance profile with the application instances.

C.Use the parameter section in the AWS CloudFormation template to have the user input access and secret keys from an already-created IAM user that has the required permissions to read and write from the DynamoDB tables.

D.Create an IAM user in the AWS CloudFormation template that has the required permissions to read and write from the DynamoDB tables. Use the GetAtt function to retrieve the access and secret keys, and pass them to the application instances through the user data.

Answer: B

Explanation:

The correct answer is **B**. Here's a detailed justification:

The core requirement is to grant EC2 instances in the application tier access to DynamoDB tables securely without hardcoding or exposing credentials within the CloudFormation template or instance configuration. IAM roles provide a secure way to grant permissions to AWS resources without managing long-term credentials directly within the application or infrastructure.

Option B utilizes IAM roles and instance profiles, which is the recommended approach for granting permissions to EC2 instances. An IAM role is created with the necessary permissions (read and write access) to DynamoDB tables. This role is then associated with an EC2 instance profile. The instance profile acts as a container for the IAM role and is automatically applied to the EC2 instances when they are launched. When the application running on the EC2 instance makes calls to DynamoDB, the AWS SDK automatically uses the credentials provided by the instance profile to authenticate. This eliminates the need to store or manage access keys directly on the instance or within the CloudFormation template. This approach aligns with the principle of least privilege, granting only the necessary permissions to the EC2 instances.

Option A is partially correct because it suggests using an IAM role. However, it doesn't explicitly mention assigning write permissions to the IAM role. Therefore, it's less complete than Option B.

Option C is incorrect because it involves asking the user to manually input access and secret keys. This is bad practice because it requires manual intervention, poses a security risk if keys are compromised, and is not a scalable or automated solution. It goes against the principle of minimizing human intervention.

Option D is also incorrect and a very bad practice. Creating IAM users within the CloudFormation template and

attempting to retrieve and pass credentials via user data is insecure. Secret and access keys are long-term credentials that should never be passed via user data because user data is easily accessible and can be logged. Furthermore, hardcoding credentials in templates increases the risk of accidental exposure.

In summary, leveraging IAM roles and instance profiles provides a secure, automated, and auditable mechanism for granting EC2 instances access to other AWS services like DynamoDB, fulfilling the requirements without compromising security.

For further research:

IAM Roles for Amazon EC2: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_use_ec2.html
Granting Applications Running on Amazon EC2 Instances Access to AWS Resources: https://docs.aws.amazon.com/IAM/latest/UserGuide/tutorial_delegation.html
AWS CloudFormation: https://aws.amazon.com/cloudformation/

Question: 557 **Exam Heist**

A solutions architect manages an analytics application. The application stores large amounts of semistructured data in an Amazon S3 bucket. The solutions architect wants to use parallel data processing to process the data more quickly. The solutions architect also wants to use information that is stored in an Amazon Redshift database to enrich the data.

Which solution will meet these requirements?

A.Use Amazon Athena to process the S3 data. Use AWS Glue with the Amazon Redshift data to enrich the S3 data.

B.Use Amazon EMR to process the S3 data. Use Amazon EMR with the Amazon Redshift data to enrich the S3 data.

C.Use Amazon EMR to process the S3 data. Use Amazon Kinesis Data Streams to move the S3 data into Amazon Redshift so that the data can be enriched.

D.Use AWS Glue to process the S3 data. Use AWS Lake Formation with the Amazon Redshift data to enrich the S3 data.

Answer: A

Explanation:

The correct answer is **A. Use Amazon Athena to process the S3 data. Use AWS Glue with the Amazon Redshift data to enrich the S3 data.**

Here's why:

Amazon Athena for S3 data processing: Athena is a serverless, interactive query service that makes it easy to analyze data directly in Amazon S3 using standard SQL. It's well-suited for processing large amounts of semi-structured data stored in S3 because it doesn't require setting up or managing any infrastructure. Athena excels at parallel data processing directly on S3 data, allowing for faster query execution. https://aws.amazon.com/athena/

AWS Glue for data enrichment: AWS Glue is a fully managed extract, transform, and load (ETL) service that makes it easy to prepare and load data for analytics. Glue can access data in Amazon Redshift and transform it to enrich the data stored in S3. Glue's capabilities include data cataloging, code generation, and job scheduling, which can be leveraged to orchestrate the data enrichment process. Glue is also capable of performing parallel processing using Spark. https://aws.amazon.com/glue/

Now, let's analyze why the other options are less suitable:

B. Use Amazon EMR to process the S3 data. Use Amazon EMR with the Amazon Redshift data to enrich the S3 data. While EMR can process S3 data in parallel, it's generally an overkill for simple SQL queries. Athena is more cost-effective and easier to manage for such use cases. Using EMR to enrich data from Redshift might involve more complex setup and maintenance than using Glue, which is specifically designed for ETL tasks.

C. Use Amazon EMR to process the S3 data. Use Amazon Kinesis Data Streams to move the S3 data into Amazon Redshift so that the data can be enriched. Kinesis Data Streams is designed for real-time data streaming, which isn't the core requirement here. The problem describes a batch-oriented analytics application. Moving all the S3 data into Redshift simply for enrichment is less efficient and more costly compared to enriching it in place using Glue.

D. Use AWS Glue to process the S3 data. Use AWS Lake Formation with the Amazon Redshift data to enrich the S3 data. Glue is typically used for ETL, not for interactive querying like the solutions architect intended to do. While Lake Formation helps manage and secure data lakes, it's not directly involved in the data enrichment process itself. Glue performs the actual enrichment, potentially making use of Lake Formation's governance features for data access.

Question: 558 **Exam Heist**

A company has two VPCs that are located in the us-west-2 Region within the same AWS account. The company needs to allow network traffic between these VPCs. Approximately 500 GB of data transfer will occur between the VPCs each month.

What is the MOST cost-effective solution to connect these VPCs?

A.Implement AWS Transit Gateway to connect the VPCs. Update the route tables of each VPC to use the transit gateway for inter-VPC communication.

B.Implement an AWS Site-to-Site VPN tunnel between the VPCs. Update the route tables of each VPC to use the VPN tunnel for inter-VPC communication.

C.Set up a VPC peering connection between the VPCs. Update the route tables of each VPC to use the VPC peering connection for inter-VPC communication.

D.Set up a 1 GB AWS Direct Connect connection between the VPCs. Update the route tables of each VPC to use the Direct Connect connection for inter-VPC communication.

Answer: C

Explanation:

The most cost-effective solution for connecting two VPCs within the same region and account, with a moderate data transfer requirement (500 GB/month), is VPC peering. VPC peering allows direct networking between VPCs using AWS's internal network backbone.

Option A, AWS Transit Gateway, is more suitable for connecting many VPCs (hub-and-spoke model) or connecting to on-premises networks. While it offers more features than VPC peering, it incurs higher costs, including hourly charges for the Transit Gateway itself and data processing charges.

Option B, AWS Site-to-Site VPN, introduces overhead related to IPsec encryption and decryption. It's better suited for connecting to on-premises networks or across regions where direct peering isn't possible. VPN tunnels also have bandwidth limitations which could impact performance for larger data transfers.

Option D, AWS Direct Connect, is designed for establishing a dedicated network connection between on-premises infrastructure and AWS. It's overkill for connecting VPCs within the same region and account and is significantly more expensive due to monthly port fees and hourly charges. It's targeted towards hybrid cloud environments where consistent and high-bandwidth connectivity is critical between the on-premises data center and AWS.

VPC peering involves setting up a peering connection request and accepting it. Once established, route tables in each VPC are updated to direct traffic destined for the other VPC's CIDR block through the peering connection. The primary cost associated with VPC peering is data transfer charges between the VPCs, which are generally lower than the costs associated with Transit Gateway, VPN, or Direct Connect for this particular use case. Therefore, VPC peering is the simplest and most cost-effective solution for the given scenario.

Refer to these links for further information:

VPC Peering: https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html
AWS Transit Gateway: https://aws.amazon.com/transit-gateway/
AWS Site-to-Site VPN: https://aws.amazon.com/vpn/
AWS Direct Connect: https://aws.amazon.com/directconnect/

Question: 559 **Exam Heist**

A company hosts multiple applications on AWS for different product lines. The applications use different compute resources, including Amazon EC2 instances and Application Load Balancers. The applications run in different AWS accounts under the same organization in AWS Organizations across multiple AWS Regions. Teams for each product line have tagged each compute resource in the individual accounts.

The company wants more details about the cost for each product line from the consolidated billing feature in Organizations.

Which combination of steps will meet these requirements? (Choose two.)

A.Select a specific AWS generated tag in the AWS Billing console.
B.Select a specific user-defined tag in the AWS Billing console.
C.Select a specific user-defined tag in the AWS Resource Groups console.
D.Activate the selected tag from each AWS account.
E.Activate the selected tag from the Organizations management account.

Answer: BE

Explanation:

The correct answer is BE. Here's a detailed justification:

The problem requires cost allocation based on product lines using tags that are already applied to resources across multiple AWS accounts within an Organization. AWS Organizations provides a consolidated billing feature, which allows for aggregated cost tracking. To leverage this feature effectively with existing tags, certain steps are essential within the AWS Billing console.

Option B is correct because the company has already applied user-defined tags (specific to product lines) to their resources. To utilize these tags for cost allocation, they must be activated in the AWS Billing console. AWS-generated tags typically serve other purposes and don't align with the specific product line breakdown the company requires, making option A incorrect.

Option E is correct because tag activation for cost allocation must occur from the AWS Organizations management account. This centralized activation ensures that the tag is enabled across all member accounts contributing to the consolidated bill, enabling the cost allocation to be displayed correctly in billing reports. Activating the tags in individual accounts (option D) would not centrally manage the cost allocation across the entire Organization. The AWS Resource Groups console (option C) is designed for managing and organizing resources, not for activating tags for cost allocation purposes in billing.

In summary, selecting the appropriate user-defined tag in the AWS Billing console and activating it from the Organizations management account enables the cost allocation based on product lines across all member

accounts within the AWS Organization, thus meeting the company's requirements.

Supporting Documentation:

AWS Organizations Consolidated Billing
Using Cost Allocation Tags

Question: 560 **Exam Heist**

A company's solutions architect is designing an AWS multi-account solution that uses AWS Organizations. The solutions architect has organized the company's accounts into organizational units (OUs).

The solutions architect needs a solution that will identify any changes to the OU hierarchy. The solution also needs to notify the company's operations team of any changes.

Which solution will meet these requirements with the LEAST operational overhead?

A.Provision the AWS accounts by using AWS Control Tower. Use account drift notifications to identify the changes to the OU hierarchy.

B.Provision the AWS accounts by using AWS Control Tower. Use AWS Config aggregated rules to identify the changes to the OU hierarchy.

C.Use AWS Service Catalog to create accounts in Organizations. Use an AWS CloudTrail organization trail to identify the changes to the OU hierarchy.

D.Use AWS CloudFormation templates to create accounts in Organizations. Use the drift detection operation on a stack to identify the changes to the OU hierarchy.

Answer: A

Explanation:

The correct answer is **A: Provision the AWS accounts by using AWS Control Tower. Use account drift notifications to identify the changes to the OU hierarchy.**

Here's why:

AWS Control Tower simplifies multi-account management: AWS Control Tower provides a high-level abstraction for managing multiple AWS accounts through AWS Organizations. It establishes guardrails and automates account provisioning, making it easier to enforce compliance and security best practices across your AWS environment.

Account drift detection directly addresses the requirement: The core problem is detecting changes (drift) from the desired, governed state. Control Tower's account drift notifications are specifically designed to detect configuration changes within provisioned accounts that deviate from the established guardrails. These changes could indicate modifications to the OU hierarchy or other important configurations.

Notifications minimize operational overhead: Control Tower automatically sends notifications when drift is detected. This proactive monitoring eliminates the need for manual checks or custom scripting, significantly reducing the operational overhead for the company's operations team.

Let's examine why the other options are less suitable:

B. AWS Config aggregated rules: While AWS Config can monitor configurations, aggregated rules are best suited for assessing compliance across multiple accounts based on specific configuration rules, not for detecting structural changes in the OU hierarchy. Setting up these rules and maintaining them would require more operational effort than Control Tower's built-in drift detection.

C. AWS Service Catalog and CloudTrail: AWS Service Catalog helps manage and provision services. While

CloudTrail can log API calls, including those related to OU changes, this solution requires the operations team to sift through CloudTrail logs to identify OU changes. This manual process increases the operational burden.

D. CloudFormation and drift detection on stacks: CloudFormation is excellent for infrastructure as code. However, using it to manage accounts directly in Organizations is less common than using Control Tower. Furthermore, CloudFormation drift detection is specific to changes within a stack, not the broader OU hierarchy. This approach would require creating stacks specifically for managing the OU structure and monitoring drift on those stacks, increasing complexity and overhead.

In summary, AWS Control Tower's built-in account drift notifications provide the most straightforward and efficient way to identify and notify the operations team about changes to the OU hierarchy, thereby minimizing operational overhead.

Supporting Documentation:

AWS Control Tower: https://aws.amazon.com/controltower/
AWS Organizations: https://aws.amazon.com/organizations/
AWS Control Tower Account Factory: https://docs.aws.amazon.com/controltower/latest/userguide/account-factory.html

Question: 561 **Exam Heist**

A company's website handles millions of requests each day, and the number of requests continues to increase. A solutions architect needs to improve the response time of the web application. The solutions architect determines that the application needs to decrease latency when retrieving product details from the Amazon DynamoDB table.

Which solution will meet these requirements with the LEAST amount of operational overhead?

A.Set up a DynamoDB Accelerator (DAX) cluster. Route all read requests through DAX.

B.Set up Amazon ElastiCache for Redis between the DynamoDB table and the web application. Route all read requests through Redis.

C.Set up Amazon ElastiCache for Memcached between the DynamoDB table and the web application. Route all read requests through Memcached.

D.Set up Amazon DynamoDB Streams on the table, and have AWS Lambda read from the table and populate Amazon ElastiCache. Route all read requests through ElastiCache.

Answer: A

Explanation:

The question focuses on reducing latency for retrieving product details from DynamoDB with minimal operational overhead. Let's analyze why option A, using DynamoDB Accelerator (DAX), is the best choice.

DAX is purpose-built for DynamoDB: DAX is a fully managed, highly available, in-memory cache specifically designed for DynamoDB. This tight integration eliminates the complexities of managing a general-purpose cache like Redis or Memcached.

Lowest Operational Overhead: DAX requires minimal configuration and management. AWS handles the underlying infrastructure, patching, and scaling. This contrasts with ElastiCache solutions, which require more manual setup and ongoing maintenance.

Seamless Integration: DAX works directly with DynamoDB, requiring minimal application code changes. You simply point your application to the DAX cluster.

Microsecond Latency: DAX is optimized for extremely low latency, providing single-digit millisecond response times, often even in the microsecond range, significantly improving read performance.

No Data Consistency Concerns (for this scenario): The question focuses on product details. This suggests relatively static data that is suitable for caching without strong consistency requirements. DAX provides eventual consistency, which is acceptable in many read-heavy scenarios where stale data is tolerable for a short period.

Why other options are less suitable:

ElastiCache (Redis/Memcached): While ElastiCache can be used for caching DynamoDB data, it involves more operational overhead. You need to manage the ElastiCache cluster, handle data serialization/deserialization, and implement the caching logic within your application. Furthermore, it doesn't natively understand DynamoDB's data model.

DynamoDB Streams + Lambda + ElastiCache: This is a complex solution that introduces significant operational overhead. It requires setting up DynamoDB Streams, configuring a Lambda function to process the stream, populating ElastiCache, and managing the entire pipeline. This is overkill for simply caching read data and increases points of failure. It also introduces latency due to the asynchronous nature of the stream processing.

In summary, DAX provides the simplest, most efficient, and lowest-overhead solution for accelerating DynamoDB reads by providing a managed in-memory cache specifically optimized for DynamoDB, minimizing latency without the need for extensive configuration or application modifications.

Authoritative Links:

DynamoDB Accelerator (DAX):
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/DAX.html
Amazon ElastiCache: https://aws.amazon.com/elasticache/

Question: 562 **Exam Heist**

A solutions architect needs to ensure that API calls to Amazon DynamoDB from Amazon EC2 instances in a VPC do not travel across the internet.

Which combination of steps should the solutions architect take to meet this requirement? (Choose two.)

A.Create a route table entry for the endpoint.
B.Create a gateway endpoint for DynamoDB.
C.Create an interface endpoint for Amazon EC2.
D.Create an elastic network interface for the endpoint in each of the subnets of the VPC.
E.Create a security group entry in the endpoint's security group to provide access.

Answer: AB

Explanation:

The goal is to ensure DynamoDB traffic from EC2 instances within a VPC doesn't traverse the public internet. Option A, creating a route table entry for the endpoint, is essential. A route table directs network traffic within a VPC. By adding a route that directs traffic destined for DynamoDB (identified by the endpoint's prefix list) to the gateway endpoint, we ensure traffic stays within the VPC.

Option B, creating a gateway endpoint for DynamoDB, is also crucial. A gateway endpoint is a VPC resource that enables private connections to supported AWS services (like DynamoDB) without requiring an internet gateway, NAT device, or VPN connection. This prevents traffic from leaving the VPC. Gateway endpoints are highly available and scaled automatically to support high traffic volumes.

Option C, creating an interface endpoint for Amazon EC2, is incorrect. Interface endpoints (powered by PrivateLink) are used for connecting to AWS services or other VPC endpoints privately, but specifically for services that are not supported by gateway endpoints. DynamoDB is supported by gateway endpoints, so interface endpoints are not the correct choice here. Furthermore, an interface endpoint is used to connect to a service, not to be connected from.

Option D, creating an elastic network interface for the endpoint in each subnet, is incorrect. This is not how gateway endpoints work. Gateway endpoints do not require explicit ENIs. They're managed by AWS.

Option E, creating a security group entry in the endpoint's security group to provide access, is incorrect. Gateway endpoints do not use security groups. Instead, access is controlled by the resource policies of the service being accessed (DynamoDB in this case). Resource policies specify which VPCs and accounts are allowed to access the service. While security groups still control traffic to the EC2 instance, they don't directly control the path of traffic that traverses the gateway endpoint.

Therefore, creating a route table entry and creating a gateway endpoint for DynamoDB is the correct approach.

Here are helpful links for more information:

VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html
Gateway VPC Endpoints for DynamoDB:
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/vpc-endpoints-access.html
Route Tables: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html

Question: 563 **Exam Heist**

A company runs its applications on both Amazon Elastic Kubernetes Service (Amazon EKS) clusters and on-premises Kubernetes clusters. The company wants to view all clusters and workloads from a central location.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use Amazon CloudWatch Container Insights to collect and group the cluster information.

B.Use Amazon EKS Connector to register and connect all Kubernetes clusters.

C.Use AWS Systems Manager to collect and view the cluster information.

D.Use Amazon EKS Anywhere as the primary cluster to view the other clusters with native Kubernetes commands.

Answer: B

Explanation:

The question requires a solution that centralizes Kubernetes cluster and workload visibility across Amazon EKS and on-premises environments with minimal operational overhead.

Option B, using Amazon EKS Connector, is the most suitable solution. EKS Connector allows you to connect any conformant Kubernetes cluster (including on-premises) to Amazon EKS. Once connected, you can view these clusters and their associated resources (like deployments, pods, and services) directly from the AWS Management Console. This offers a centralized view without requiring extensive configuration or management of monitoring agents. The integration leverages AWS IAM for authentication and authorization.

Option A, CloudWatch Container Insights, focuses primarily on performance monitoring and metrics collection. While it can provide insights into cluster performance, it doesn't offer a unified console for viewing all clusters and workloads without significant configuration and management of agents on each cluster.

Option C, AWS Systems Manager, is a powerful management service but primarily focuses on managing EC2

instances and on-premises servers. It can execute commands on instances within the clusters, but it doesn't inherently provide a centralized Kubernetes-aware view of all clusters and their workloads like EKS Connector does.

Option D, Amazon EKS Anywhere, creates EKS-compatible clusters on-premises. While it helps standardize the Kubernetes experience, it doesn't directly address the requirement of viewing existing on-premises clusters alongside EKS clusters from a central location without migrating the on-premises workloads to EKS Anywhere. Using EKS Anywhere as a primary viewing cluster would involve significant operational overhead.

Therefore, EKS Connector offers the simplest and most direct way to meet the requirements of centralized visibility with minimal operational overhead by connecting existing clusters to the EKS console.

Amazon EKS Connector DocumentationAmazon EKS Anywhere Documentation

Question: 564 **Exam Heist**

A company is building an ecommerce application and needs to store sensitive customer information. The company needs to give customers the ability to complete purchase transactions on the website. The company also needs to ensure that sensitive customer data is protected, even from database administrators.

Which solution meets these requirements?

A.Store sensitive data in an Amazon Elastic Block Store (Amazon EBS) volume. Use EBS encryption to encrypt the data. Use an IAM instance role to restrict access.

B.Store sensitive data in Amazon RDS for MySQL. Use AWS Key Management Service (AWS KMS) client-side encryption to encrypt the data.

C.Store sensitive data in Amazon S3. Use AWS Key Management Service (AWS KMS) server-side encryption to encrypt the data. Use S3 bucket policies to restrict access.

D.Store sensitive data in Amazon FSx for Windows Server. Mount the file share on application servers. Use Windows file permissions to restrict access.

Answer: B

Explanation:

The correct answer is **B. Store sensitive data in Amazon RDS for MySQL. Use AWS Key Management Service (AWS KMS) client-side encryption to encrypt the data.**

Here's why:

Requirement: Storing sensitive customer information for an e-commerce application: RDS for MySQL is a suitable database solution for structured data like customer information and transaction details.

Requirement: Protecting data even from database administrators: Client-side encryption using AWS KMS ensures that the data is encrypted before it's stored in the database. This means the database administrators (DBAs) only see encrypted data and cannot decrypt it without the proper KMS key.

Client-Side Encryption: With client-side encryption, the application handles the encryption and decryption of the data using the KMS key. The RDS instance never has direct access to the key itself, making it impossible for the database to reveal the plain-text data.

AWS KMS: AWS KMS is a managed service that makes it easy for you to create and control the encryption keys used to encrypt your data. KMS provides a secure and centralized way to manage cryptographic keys.

Why other options are incorrect:

A: Storing sensitive data on EBS volumes with EBS encryption only protects data at rest on the volume itself. If someone with proper IAM permissions gets access to the instance and the mounted volume, they can access the unencrypted data if the application isn't specifically encrypting data at the application level. It does not protect against DBAs if they have instance access to the database EC2 instance.

C: S3 is generally used for unstructured data (objects). While S3 server-side encryption protects data at rest, it doesn't provide the same level of control as client-side encryption for protecting against DBAs. Furthermore, using S3 for transactional data in an e-commerce application is not an efficient or standard design pattern.

D: Amazon FSx is a file system service. While you can store data on FSx and restrict access using Windows file permissions, this option doesn't address the requirement of protecting data from database administrators since file storage is not designed for transactional data.

Authoritative Links:

AWS KMS: https://aws.amazon.com/kms/
Amazon RDS: https://aws.amazon.com/rds/
Client-Side Encryption: https://docs.aws.amazon.com/kms/latest/developerguide/services-rds.html

Question: 565 **Exam Heist**

A company has an on-premises MySQL database that handles transactional data. The company is migrating the database to the AWS Cloud. The migrated database must maintain compatibility with the company's applications that use the database. The migrated database also must scale automatically during periods of increased demand.

Which migration solution will meet these requirements?

A.Use native MySQL tools to migrate the database to Amazon RDS for MySQL. Configure elastic storage scaling.

B.Migrate the database to Amazon Redshift by using the mysqldump utility. Turn on Auto Scaling for the Amazon Redshift cluster.

C.Use AWS Database Migration Service (AWS DMS) to migrate the database to Amazon Aurora. Turn on Aurora Auto Scaling.

D.Use AWS Database Migration Service (AWS DMS) to migrate the database to Amazon DynamoDB. Configure an Auto Scaling policy.

Answer: C

Explanation:

The correct answer is C because it addresses all the requirements outlined: compatibility, managed database service, and auto-scaling.

Compatibility: Amazon Aurora with MySQL compatibility is designed to be a drop-in replacement for MySQL databases. This ensures that the existing applications that rely on MySQL will function correctly with minimal or no code changes. https://aws.amazon.com/rds/aurora/mysql-features/

Migration: AWS Database Migration Service (DMS) is specifically designed to migrate databases from on-premises environments to AWS. It supports heterogeneous database migrations, but for a MySQL-to-Aurora migration, it's a homogeneous migration, which simplifies the process. https://aws.amazon.com/dms/

Auto-Scaling: Aurora Auto Scaling automatically adjusts the number of Aurora DB instances in an Aurora cluster. This is vital to maintain performance during peak demand periods without manual intervention, meeting the scalability requirement.
https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Managing.Scaling.html

Option A is partially correct (RDS for MySQL allows elastic storage scaling), but it doesn't offer the superior performance and scalability of Aurora, especially for demanding workloads. Native MySQL tools for migration can be complex and time-consuming compared to DMS. Elastic storage scaling is not the same as scaling the compute resources/instances, which Aurora Auto Scaling provides.

Option B suggests migrating to Amazon Redshift, which is a data warehouse service optimized for analytical workloads, not transactional data. Redshift is incompatible with MySQL applications and is not suitable for a direct database migration requiring compatibility. Furthermore, Auto Scaling in Redshift refers to scaling the number of compute nodes, not intended for auto-scaling based on transaction load like Aurora.

Option D proposes migrating to Amazon DynamoDB, a NoSQL database. This option fails on the compatibility requirement. DynamoDB uses a different data model and query language than MySQL, meaning significant application changes would be needed, which the problem statement aims to avoid.

Question: 566 **Exam Heist**

A company runs multiple Amazon EC2 Linux instances in a VPC across two Availability Zones. The instances host applications that use a hierarchical directory structure. The applications need to read and write rapidly and concurrently to shared storage.

What should a solutions architect do to meet these requirements?

A.Create an Amazon S3 bucket. Allow access from all the EC2 instances in the VPC.

B.Create an Amazon Elastic File System (Amazon EFS) file system. Mount the EFS file system from each EC2 instance.

C.Create a file system on a Provisioned IOPS SSD (io2) Amazon Elastic Block Store (Amazon EBS) volume. Attach the EBS volume to all the EC2 instances.

D.Create file systems on Amazon Elastic Block Store (Amazon EBS) volumes that are attached to each EC2 instance. Synchronize the EBS volumes across the different EC2 instances.

Answer: B

Explanation:

The best solution for rapidly and concurrently reading and writing to shared storage from multiple EC2 Linux instances across Availability Zones, while maintaining a hierarchical directory structure, is to use Amazon Elastic File System (Amazon EFS).

Option A is incorrect because Amazon S3 is object storage, not file storage. While S3 can store files, it lacks the hierarchical directory structure required and isn't designed for the rapid and concurrent read/write operations needed for applications expecting a traditional file system interface. S3 is better suited for storing large files, backups, or static content.

Option C is incorrect because attaching a single EBS volume to multiple EC2 instances is not supported. EBS volumes are block storage and are designed for single-instance attachment. Attempting to attach the same EBS volume to multiple instances can lead to data corruption. Also, even if possible, io2 EBS would only be in one Availability Zone, impacting availability across the two zones.

Option D is incorrect because synchronizing EBS volumes across multiple instances would be complex to set up and manage, prone to errors, and would not be a native AWS solution. It would also introduce significant latency and overhead, hindering the performance requirements. Furthermore, it wouldn't offer the same level of availability and resilience as a managed service.

EFS, on the other hand, is a fully managed network file system specifically designed for use with AWS compute services like EC2. It provides a simple, scalable, elastic, and highly available file system that can be

simultaneously accessed by multiple EC2 instances across multiple Availability Zones within a VPC. This makes it ideal for shared storage scenarios requiring a traditional file system interface, such as those described in the question. Mounting the EFS file system on each EC2 instance allows the applications to read and write to the shared storage rapidly and concurrently, while automatically handling data replication and availability across Availability Zones. EFS uses a hierarchical directory structure natively.

Amazon EFS Documentation

Question: 567 **Exam Heist**

A solutions architect is designing a workload that will store hourly energy consumption by business tenants in a building. The sensors will feed a database through HTTP requests that will add up usage for each tenant. The solutions architect must use managed services when possible. The workload will receive more features in the future as the solutions architect adds independent components.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use Amazon API Gateway with AWS Lambda functions to receive the data from the sensors, process the data, and store the data in an Amazon DynamoDB table.

B.Use an Elastic Load Balancer that is supported by an Auto Scaling group of Amazon EC2 instances to receive and process the data from the sensors. Use an Amazon S3 bucket to store the processed data.

C.Use Amazon API Gateway with AWS Lambda functions to receive the data from the sensors, process the data, and store the data in a Microsoft SQL Server Express database on an Amazon EC2 instance.

D.Use an Elastic Load Balancer that is supported by an Auto Scaling group of Amazon EC2 instances to receive and process the data from the sensors. Use an Amazon Elastic File System (Amazon EFS) shared file system to store the processed data.

Answer: A

Explanation:

The correct answer is A because it offers the least operational overhead and best aligns with managed services and future scalability requirements. Let's break down why:

Option A leverages fully managed services: Amazon API Gateway, AWS Lambda, and Amazon DynamoDB. API Gateway handles the HTTP requests from the sensors, Lambda provides serverless compute to process the data, and DynamoDB offers a NoSQL database that's highly scalable and requires minimal administration. This architecture is designed for event-driven processing and scales automatically with the workload. DynamoDB is also optimized for key-value data, making it ideal for storing tenant usage data.

Option B involves managing EC2 instances within an Auto Scaling group and an Elastic Load Balancer (ELB). While ELB is managed, the EC2 instances require patching, scaling, and monitoring, increasing operational overhead. Storing processed data in S3 is a good choice for archival purposes, but not ideal for real-time querying and updates.

Option C introduces a Microsoft SQL Server Express database on an EC2 instance. This requires managing the database server, including patching, backups, and ensuring high availability. Although API Gateway and Lambda are beneficial, the SQL Server significantly increases operational overhead compared to DynamoDB.

Option D involves similar operational overhead to option B because it requires managing EC2 instances. While EFS is a managed service, its primary use case is sharing file systems, not storing structured data like tenant consumption. Storing usage data on EFS wouldn't be the most efficient approach.

Therefore, Option A is the most efficient choice due to the combination of managed services, scalability, and event-driven architecture, minimizing the administrative burden and facilitating future feature additions. Using DynamoDB instead of a file system, relational database, or object storage is better suited for quick and

easy updates to tenant usage data based on sensor readings.

Supporting Resources:

Amazon API Gateway: https://aws.amazon.com/api-gateway/
AWS Lambda: https://aws.amazon.com/lambda/
Amazon DynamoDB: https://aws.amazon.com/dynamodb/
Amazon EC2 Auto Scaling: https://aws.amazon.com/autoscaling/

Question: 568 **Exam Heist**

A solutions architect is designing the storage architecture for a new web application used for storing and viewing engineering drawings. All application components will be deployed on the AWS infrastructure.

The application design must support caching to minimize the amount of time that users wait for the engineering drawings to load. The application must be able to store petabytes of data.

Which combination of storage and caching should the solutions architect use?

A.Amazon S3 with Amazon CloudFront
B.Amazon S3 Glacier with Amazon ElastiCache
C.Amazon Elastic Block Store (Amazon EBS) volumes with Amazon CloudFront
D.AWS Storage Gateway with Amazon ElastiCache

Answer: A

Explanation:

The correct solution is Amazon S3 with Amazon CloudFront. Here's why:

Amazon S3 is ideal for storing large amounts of unstructured data like engineering drawings. It offers virtually unlimited storage, scalability, and high durability. S3 is designed for object storage, making it perfect for storing individual files accessible via HTTP/HTTPS.

Amazon CloudFront, a content delivery network (CDN), integrates seamlessly with S3. It caches content (like those engineering drawings) at edge locations globally, reducing latency for users accessing the web application. When a user requests an engineering drawing, CloudFront first checks its cache. If the drawing is cached, it delivers it from the nearest edge location, resulting in faster load times. If the drawing is not cached, CloudFront retrieves it from the S3 origin and caches it for future requests.

Option B is incorrect because Amazon S3 Glacier is designed for archival storage of data that is infrequently accessed, which doesn't align with the need for fast access for viewing engineering drawings. ElastiCache is an in-memory caching service typically used for databases, not for serving large files like engineering drawings.

Option C is incorrect because EBS volumes are block storage devices primarily used for persistent storage for EC2 instances. While EBS can store data, it isn't a scalable and cost-effective solution for storing petabytes of data compared to S3. Also, EBS isn't directly integrated with CloudFront for content caching in the same way that S3 is.

Option D is incorrect because AWS Storage Gateway is a hybrid storage service that connects on-premises software appliances to AWS cloud storage. It isn't appropriate for a fully cloud-native web application storing petabytes of data within AWS. ElastiCache isn't a substitute for a CDN like CloudFront, which optimizes content delivery globally.

Therefore, S3 provides the scalable storage for petabytes of drawings, and CloudFront provides caching for

low latency delivery, fulfilling the application's requirements.

Further research:

Amazon S3: https://aws.amazon.com/s3/
Amazon CloudFront: https://aws.amazon.com/cloudfront/

Question: 569 **Exam Heist**

An Amazon EventBridge rule targets a third-party API. The third-party API has not received any incoming traffic. A solutions architect needs to determine whether the rule conditions are being met and if the rule's target is being invoked.

Which solution will meet these requirements?

A.Check for metrics in Amazon CloudWatch in the namespace for AWS/Events.
B.Review events in the Amazon Simple Queue Service (Amazon SQS) dead-letter queue.
C.Check for the events in Amazon CloudWatch Logs.
D.Check the trails in AWS CloudTrail for the EventBridge events.

Answer: A

Explanation:

The correct answer is **A. Check for metrics in Amazon CloudWatch in the namespace for AWS/Events.**

Here's a detailed justification:

Amazon EventBridge emits metrics to Amazon CloudWatch under the AWS/Events namespace. These metrics provide valuable insights into the performance and behavior of EventBridge rules. Specifically, metrics like Invocations, FailedInvocations, TriggeredRules, and MatchedEvents can help determine if the rule conditions are being met and if the target is being invoked.

Invocations: This metric indicates the number of times the target has been invoked by the EventBridge rule. If the target is a third-party API, a zero value suggests that the rule hasn't invoked it yet, indicating a potential issue with rule conditions or event matching.
FailedInvocations: This metric represents the number of times the target invocation failed. A high value indicates a problem with the target or the EventBridge configuration.
TriggeredRules: This metric indicates how many times a rule was matched to an incoming event. This is useful for confirming that the rule conditions are being met by incoming events.
MatchedEvents: This metric represents the number of events that matched a particular EventBridge rule.

By monitoring these metrics, a solutions architect can quickly ascertain whether the EventBridge rule is triggering based on the defined conditions and if the target API is being successfully invoked. If the Invocations metric remains at zero, it suggests that either the rule conditions are not being met, or the events are not being routed correctly. If FailedInvocations is high, the API endpoint itself might have problems.

Why other options are incorrect:

B. Review events in the Amazon Simple Queue Service (Amazon SQS) dead-letter queue: A dead-letter queue (DLQ) is used to capture events that could not be processed successfully after multiple retry attempts. While a DLQ might contain information about failed invocations, it won't help determine if the rule is being triggered in the first place. DLQs only get populated after a failed target invocation (after retries), not if the rule never fired initially.
C. Check for the events in Amazon CloudWatch Logs: While CloudWatch Logs can capture detailed logs from

various AWS services and applications, EventBridge doesn't automatically log every event it processes or rule it triggers to CloudWatch Logs unless explicitly configured to do so using CloudWatch Logs as a target. Moreover, logging every event can become costly and inefficient for debugging this specific scenario.
D. Check the trails in AWS CloudTrail for the EventBridge events: AWS CloudTrail records API calls made to AWS services, including EventBridge. CloudTrail can show the creation, modification, or deletion of EventBridge rules, but it doesn't directly track the number of times a rule is triggered or the target is invoked. CloudTrail is primarily for auditing purposes, not for real-time monitoring of rule performance.

Authoritative Links:

Monitoring Amazon EventBridge - Amazon EventBridge
AWS/Events Metrics - Amazon CloudWatch

Question: 570 **Exam Heist**

A company has a large workload that runs every Friday evening. The workload runs on Amazon EC2 instances that are in two Availability Zones in the us-east-1 Region. Normally, the company must run no more than two instances at all times. However, the company wants to scale up to six instances each Friday to handle a regularly repeating increased workload.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create a reminder in Amazon EventBridge to scale the instances.
B.Create an Auto Scaling group that has a scheduled action.
C.Create an Auto Scaling group that uses manual scaling.
D.Create an Auto Scaling group that uses automatic scaling.

Answer: B

Explanation:

Here's a detailed justification for why option B is the best solution for scaling EC2 instances for a recurring Friday workload with minimal operational overhead:

The core requirement is to automatically scale the EC2 instance count based on a predictable schedule (every Friday). Auto Scaling groups are designed to automatically manage the number of EC2 instances based on defined parameters. Option B, using a scheduled action within an Auto Scaling group, directly addresses this need. A scheduled action allows you to predefine instance count adjustments (minimum, maximum, desired capacity) that occur at specific times or recurring intervals. This eliminates manual intervention each Friday.

Option A, using Amazon EventBridge (formerly CloudWatch Events) to remind someone to scale the instances, introduces manual steps. While EventBridge can trigger actions, a simple reminder still requires someone to log in and adjust the instance count. This increases operational overhead and the risk of human error.

Option C, manual scaling, defeats the purpose of automation entirely. It would require someone to manually adjust the Auto Scaling group's parameters (desired capacity) every Friday, which is exactly what the question aims to avoid.

Option D, automatic scaling, is usually based on metrics like CPU utilization or network traffic. While it can handle fluctuating workloads, it is less appropriate for predictable, time-based scaling. Automatic scaling would react to the increased workload only after it has already started, potentially delaying the scaling and impacting performance. Setting up and maintaining the metrics for automatic scaling adds complexity.

Therefore, a scheduled action within an Auto Scaling group provides the most streamlined and automated approach to managing the EC2 instance count for a recurring Friday workload with minimal operational overhead, perfectly aligning with the stated requirements.

Relevant links for further research:

AWS Auto Scaling: https://aws.amazon.com/autoscaling/
Auto Scaling Scheduled Actions:
https://docs.aws.amazon.com/autoscaling/ec2/userguide/schedule_time.html
Amazon EventBridge: https://aws.amazon.com/eventbridge/

Question: 571 **Exam Heist**

A company is creating a REST API. The company has strict requirements for the use of TLS. The company requires TLSv1.3 on the API endpoints. The company also requires a specific public third-party certificate authority (CA) to sign the TLS certificate.

Which solution will meet these requirements?

A.Use a local machine to create a certificate that is signed by the third-party CImport the certificate into AWS Certificate Manager (ACM). Create an HTTP API in Amazon API Gateway with a custom domain. Configure the custom domain to use the certificate.

B.Create a certificate in AWS Certificate Manager (ACM) that is signed by the third-party CA. Create an HTTP API in Amazon API Gateway with a custom domain. Configure the custom domain to use the certificate.

C.Use AWS Certificate Manager (ACM) to create a certificate that is signed by the third-party CA. Import the certificate into AWS Certificate Manager (ACM). Create an AWS Lambda function with a Lambda function URL. Configure the Lambda function URL to use the certificate.

D.Create a certificate in AWS Certificate Manager (ACM) that is signed by the third-party CA. Create an AWS Lambda function with a Lambda function URL. Configure the Lambda function URL to use the certificate.

Answer: B

Explanation:

The correct answer is **B**. Here's why:

AWS Certificate Manager (ACM): ACM is the preferred service for provisioning, managing, and deploying SSL/TLS certificates for use with AWS services. ACM simplifies the process of obtaining and renewing certificates. https://aws.amazon.com/certificate-manager/
Importing Certificates: ACM allows you to import certificates issued by a third-party CA, which is a key requirement in this scenario. https://docs.aws.amazon.com/acm/latest/userguide/import-certificate.html
API Gateway Custom Domains: Amazon API Gateway supports custom domain names for your APIs. To use HTTPS with a custom domain, you need to associate a TLS certificate with it. ACM makes this integration seamless. https://docs.aws.amazon.com/apigateway/latest/developerguide/how-to-custom-domains.html
HTTP API and TLS Requirements: HTTP APIs in API Gateway support TLS. The question specifies TLSv1.3. API Gateway manages TLS versions and supports configuring a secure policy.
Why Option A is Incorrect: Option A suggests creating a certificate on a local machine and then importing. While possible, ACM's primary function is certificate management; relying on a local machine introduces complexities and defeats the purpose of using ACM. The certificate creation and signing should be handled by the third party CA.
Why Option C and D are Incorrect: Lambda function URLs don't directly offer the same level of custom certificate management and custom domain integration as API Gateway. While you can handle TLS within the Lambda function itself, this is less efficient and more complex than using API Gateway's built-in features. API Gateway is specifically designed to handle API traffic, including TLS termination.

Therefore, the most efficient and manageable approach is to import the certificate issued by the third-party CA into ACM and then associate it with a custom domain in API Gateway for your HTTP API, satisfying the TLSv1.3 requirement.

Question: 572 **Exam Heist**

A company runs an application on AWS. The application receives inconsistent amounts of usage. The application uses AWS Direct Connect to connect to an on-premises MySQL-compatible database. The on-premises database consistently uses a minimum of 2 GiB of memory.

The company wants to migrate the on-premises database to a managed AWS service. The company wants to use auto scaling capabilities to manage unexpected workload increases.

Which solution will meet these requirements with the LEAST administrative overhead?

A.Provision an Amazon DynamoDB database with default read and write capacity settings.
B.Provision an Amazon Aurora database with a minimum capacity of 1 Aurora capacity unit (ACU).
C.Provision an Amazon Aurora Serverless v2 database with a minimum capacity of 1 Aurora capacity unit (ACU).
D.Provision an Amazon RDS for MySQL database with 2 GiB of memory.

Answer: C

Explanation:

The optimal solution is to provision an Amazon Aurora Serverless v2 database with a minimum capacity of 1 Aurora capacity unit (ACU). This choice best addresses the requirements for a managed database service with auto-scaling capabilities and minimal administrative overhead, suitable for handling inconsistent workloads after migrating from an on-premises MySQL-compatible database.

Option A, using Amazon DynamoDB, is not ideal because it's a NoSQL database. While DynamoDB scales very well, migrating a MySQL-compatible database to DynamoDB would require a significant application rewrite and redesign, increasing complexity and administrative overhead, especially considering the existing application's design.

Option B, provisioning a standard Amazon Aurora database with a minimum capacity of 1 ACU, fulfills the requirements for a managed MySQL-compatible service and provides scalability. However, Aurora's standard provisioned instances are not as efficient at scaling to zero when the application has near-zero or unpredictable usage. Administratively, Aurora requires more effort to right-size compared to Serverless v2.

Option D, provisioning an Amazon RDS for MySQL database with 2 GiB of memory, meets the minimum memory requirement but lacks the automatic scaling capability needed to manage unpredictable workload increases efficiently. While RDS MySQL can be scaled, manual intervention or complex custom automation would be required, increasing administrative burden.

Aurora Serverless v2 (Option C) excels because it combines compatibility with MySQL, automatic scaling, and minimal administrative overhead. It starts with a minimum of 0.5 ACUs (effectively using about 1 GiB memory) and automatically scales up or down based on application demand. This ensures efficient resource utilization and cost optimization during periods of low usage, aligning perfectly with the need to manage inconsistent workloads. Aurora Serverless v2 also manages patching and other maintenance tasks. The cost of the service scales directly with usage, which can be more efficient than a provisioned Aurora DB if the workload is extremely variable. It effectively handles unexpected workload increases without manual intervention.

Therefore, Aurora Serverless v2 provides the best balance between compatibility, scalability, cost-effectiveness, and ease of management for this scenario.

Reference:

Amazon Aurora Serverless v2
Amazon RDS
Amazon DynamoDB

Question: 573 **Exam Heist**

A company wants to use an event-driven programming model with AWS Lambda. The company wants to reduce startup latency for Lambda functions that run on Java 11. The company does not have strict latency requirements for the applications. The company wants to reduce cold starts and outlier latencies when a function scales up.

Which solution will meet these requirements MOST cost-effectively?

A.Configure Lambda provisioned concurrency.
B.Increase the timeout of the Lambda functions.
C.Increase the memory of the Lambda functions.
D.Configure Lambda SnapStart.

Answer: D

Explanation:

The correct answer is D: Configure Lambda SnapStart. Here's why:

The scenario focuses on reducing startup latency ("cold starts") for Java 11 Lambda functions without strict latency requirements for the application as a whole. We also want to reduce outlier latencies during function scale-up and cost-effectiveness.

A. Configure Lambda provisioned concurrency: Provisioned concurrency keeps Lambda functions initialized and ready to respond, directly addressing cold starts. However, it comes at a cost: you pay for the allocated concurrency regardless of whether it's used. While effective, it's not the most cost-effective solution when strict latency requirements are absent, and it does not fundamentally address the underlying cold start process.

B. Increase the timeout of the Lambda functions: Increasing the timeout merely allows longer execution times but does nothing to reduce cold start latency itself. Cold starts occur before the function code begins executing. This choice is irrelevant to the problem.

C. Increase the memory of the Lambda functions: While increasing memory can sometimes improve performance by providing more resources during execution, it doesn't fundamentally address the cold start issue. The JVM initialization and class loading, which are major contributors to cold starts, are still present. It also directly increases costs, as you pay for the provisioned memory.

D. Configure Lambda SnapStart: SnapStart optimizes for Java 11 functions. It captures a snapshot of the initialized execution environment (including the JVM) when the function is first deployed or updated. When the function is invoked for the first time or needs to scale, Lambda resumes from the snapshot instead of starting from scratch. This drastically reduces cold start latency, approaching near-instant startup. This makes it the most cost-effective choice because you only pay for the actual invocation time of the function from the resumed state. It is particularly suited when there aren't strict latency requirements.

Therefore, SnapStart is the best option as it directly tackles the cold start issue for Java 11, reduces outlier latencies during scaling, and is more cost-effective than provisioned concurrency. It is also the most suitable solution when strict latency requirements are not an issue.

Supporting documentation:

AWS Lambda SnapStart: https://aws.amazon.com/blogs/aws/lambda-snapstart-for-java-functions-generally-available-increase-startup-speed-by-up-to-10x/
Optimizing Lambda performance: https://docs.aws.amazon.com/lambda/latest/dg/optimization-tips.html

Question: 574 **Exam Heist**

A financial services company launched a new application that uses an Amazon RDS for MySQL database. The company uses the application to track stock market trends. The company needs to operate the application for only 2 hours at the end of each week. The company needs to optimize the cost of running the database.

Which solution will meet these requirements MOST cost-effectively?

A.Migrate the existing RDS for MySQL database to an Aurora Serverless v2 MySQL database cluster.

B.Migrate the existing RDS for MySQL database to an Aurora MySQL database cluster.

C.Migrate the existing RDS for MySQL database to an Amazon EC2 instance that runs MySQL. Purchase an instance reservation for the EC2 instance.

D.Migrate the existing RDS for MySQL database to an Amazon Elastic Container Service (Amazon ECS) cluster that uses MySQL container images to run tasks.

Answer: B

Explanation:

The optimal solution is to migrate the existing RDS for MySQL database to an Aurora MySQL database cluster. Here's why:

Aurora MySQL is a cost-effective option for intermittent workloads:

Cost Optimization: Aurora MySQL offers cost-effective scaling, especially when the application operates for a short duration each week. It does not have the capability to pause completely, but the cost will be low.
Performance and Scalability: Aurora offers enhanced performance and scalability compared to standard MySQL, which is not directly related to the prompt.
Managed Service: RDS and Aurora handle patching, backups, and maintenance, reducing operational overhead compared to managing MySQL on EC2.
Alternatives: While Aurora Serverless v2 could seem appealing because it scales to zero capacity when idle, the prompt did not specify that the application be completely idle.

Why other options are less suitable:

Option A (Aurora Serverless v2 MySQL): Although Aurora Serverless v2 scales to zero when idle, it may not be the most cost-effective option since the prompt did not specify it needs to scale to zero.
Option C (EC2 with MySQL): Running MySQL on EC2 requires more manual management of the database, backups, patching, and OS, increasing operational overhead. Instance reservations don't address the fundamental issue of wasted resources during the idle period.
Option D (ECS with MySQL): Deploying MySQL in containers on ECS introduces complexity and overhead for managing containers and orchestration, without a significant cost advantage over using Aurora.

Supporting Documentation:

Amazon Aurora: https://aws.amazon.com/rds/aurora/
Amazon RDS Pricing: https://aws.amazon.com/rds/pricing/

Question: 575 **Exam Heist**

A company deploys its applications on Amazon Elastic Kubernetes Service (Amazon EKS) behind an Application Load Balancer in an AWS Region. The application needs to store data in a PostgreSQL database engine. The company wants the data in the database to be highly available. The company also needs increased capacity for read workloads.

Which solution will meet these requirements with the MOST operational efficiency?

A.Create an Amazon DynamoDB database table configured with global tables.

B.Create an Amazon RDS database with Multi-AZ deployments.

C.Create an Amazon RDS database with Multi-AZ DB cluster deployment.

D.Create an Amazon RDS database configured with cross-Region read replicas.

Answer: C

Explanation:

The correct answer is C: Create an Amazon RDS database with Multi-AZ DB cluster deployment. Here's why:

High Availability: Multi-AZ DB cluster deployments in Amazon RDS are designed to provide high availability for PostgreSQL database engines. They involve creating a primary DB instance along with multiple synchronous standby DB instances in different Availability Zones within the same AWS Region. In case of a failure of the primary instance, RDS automatically fails over to one of the standby instances, minimizing downtime. This addresses the requirement for high availability.

Increased Read Capacity: Multi-AZ DB clusters support read workloads through reader endpoints. Applications can direct read traffic to these reader endpoints, which distribute the load across the available standby instances. This increases the read capacity without impacting the performance of the primary instance handling write operations.

Operational Efficiency: Amazon RDS simplifies database management tasks like backups, patching, and recovery. With Multi-AZ DB clusters, these tasks are handled automatically, reducing the operational overhead for the company. This efficiency is crucial for environments where the company desires to minimize administrative burden.

Option A is incorrect because DynamoDB, while highly available, is a NoSQL database and not suitable for applications requiring a PostgreSQL database engine. Option B offers High Availability but it doesn't offer direct support for scaling read workloads as efficiently as multi-az DB clusters. Option D provides read replicas but requires cross-region configuration, adding complexity and increased latency. It is also more complex than a multi-az DB cluster.

Therefore, using an Amazon RDS database with Multi-AZ DB cluster deployment provides the best balance of high availability, increased read capacity, and operational efficiency for the company's PostgreSQL database needs.

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.htmlhttps://aws.amazon.com/rds/fe

Question: 576 **Exam Heist**

A company is building a RESTful serverless web application on AWS by using Amazon API Gateway and AWS Lambda. The users of this web application will be geographically distributed, and the company wants to reduce the latency of API requests to these users.

Which type of endpoint should a solutions architect use to meet these requirements?

A.Private endpoint

B.Regional endpoint

C.Interface VPC endpoint

D.Edge-optimized endpoint

Answer: D

Explanation:

The correct answer is **D. Edge-optimized endpoint.**

Here's why: Edge-optimized endpoints in Amazon API Gateway are designed to minimize latency for geographically distributed users. When a user makes a request to an edge-optimized API, the request is routed to the nearest AWS edge location via Amazon CloudFront. CloudFront is a content delivery network (CDN) that caches content and routes requests through its global network of edge locations, reducing the distance the request needs to travel. This results in lower latency and faster response times for users regardless of their location.

A regional endpoint, on the other hand, serves requests from within a specific AWS Region. While it offers low latency for users in that region, it doesn't provide the global distribution benefits of an edge-optimized endpoint.

A private endpoint is used to access APIs from within a VPC without exposing them to the public internet. It doesn't directly address latency reduction for geographically distributed users.

An Interface VPC endpoint is used to connect to AWS services privately from within a VPC. It also does not provide geographical distribution for latency reduction like an edge-optimized endpoint.

Therefore, for the requirement of reducing latency for geographically distributed users accessing a serverless web application, the **edge-optimized endpoint** is the most suitable choice due to its integration with Amazon CloudFront and global edge location network.

Relevant links for further research:

Amazon API Gateway Endpoints
Amazon CloudFront

Question: 577 **Exam Heist**

A company uses an Amazon CloudFront distribution to serve content pages for its website. The company needs to ensure that clients use a TLS certificate when accessing the company's website. The company wants to automate the creation and renewal of the TLS certificates.

Which solution will meet these requirements with the MOST operational efficiency?

A.Use a CloudFront security policy to create a certificate.
B.Use a CloudFront origin access control (OAC) to create a certificate.
C.Use AWS Certificate Manager (ACM) to create a certificate. Use DNS validation for the domain.
D.Use AWS Certificate Manager (ACM) to create a certificate. Use email validation for the domain.

Answer: C

Explanation:

The correct answer is C: Use AWS Certificate Manager (ACM) to create a certificate and use DNS validation for the domain. Here's why:

The requirement is to automate TLS certificate creation and renewal for a CloudFront distribution with minimal operational overhead. AWS Certificate Manager (ACM) is specifically designed for this purpose. ACM integrates directly with CloudFront and can automatically provision, deploy, and renew SSL/TLS certificates without manual intervention.

DNS validation is the preferred method for ACM certificate validation because it allows automated renewals

without requiring manual email verification each time. When using DNS validation, you add a CNAME record to your DNS configuration, which ACM periodically checks. This removes the need to respond to emails for validation and makes the renewal process fully automated and significantly more operationally efficient.

Option A is incorrect because CloudFront security policies don't create certificates. They configure the security protocols and ciphers that CloudFront uses when communicating with viewers. Option B is incorrect because Origin Access Control (OAC) secures the communication between CloudFront and the origin server (e.g., S3 bucket), not the communication between clients and CloudFront, and OAC doesn't handle certificate creation. Option D, while using ACM, utilizes email validation, which requires manual intervention to validate the certificate upon creation and renewal. This is less efficient than DNS validation, which fully automates the process. ACM handles the entire lifecycle of the certificate automatically when paired with DNS validation.

Therefore, ACM with DNS validation offers the most automated and least operationally burdensome method for managing TLS certificates for CloudFront.

Supporting Links:

AWS Certificate Manager (ACM): https://aws.amazon.com/certificate-manager/
ACM DNS Validation: https://docs.aws.amazon.com/acm/latest/userguide/gs-acm-validate-dns.html
Using SSL/TLS Certificates with CloudFront:
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-https-cloudfront.html

Question: 578 **Exam Heist**

A company deployed a serverless application that uses Amazon DynamoDB as a database layer. The application has experienced a large increase in users. The company wants to improve database response time from milliseconds to microseconds and to cache requests to the database.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use DynamoDB Accelerator (DAX).
B.Migrate the database to Amazon Redshift.
C.Migrate the database to Amazon RDS.
D.Use Amazon ElastiCache for Redis.

Answer: A

Explanation:

The correct answer is A: Use DynamoDB Accelerator (DAX).

Here's why:

The requirement is to improve DynamoDB response time from milliseconds to microseconds and to cache requests with the least operational overhead. DAX is a fully managed, highly available, in-memory cache for DynamoDB. It's designed specifically to accelerate read performance without requiring developers to manage cache invalidation, data population, or cluster management. This minimizes operational overhead. DAX sits between the application and DynamoDB, caching frequently accessed data. Subsequent requests for the same data are served directly from the DAX cache, significantly reducing latency to microseconds.

Option B, migrating to Amazon Redshift, is inappropriate. Redshift is a data warehousing solution optimized for analytical queries on large datasets, not for transactional workloads requiring microsecond latency. It would be a significant operational undertaking and would not directly address the caching requirement.

Option C, migrating to Amazon RDS, is also unsuitable. While RDS offers various database engines, it doesn't inherently solve the caching problem without implementing a separate caching layer (like ElastiCache).

Migrating would involve significant operational effort and wouldn't guarantee microsecond latency for all read operations without additional caching mechanisms. RDS primarily addresses relational database needs, not necessarily the NoSQL caching requirements.

Option D, using Amazon ElastiCache for Redis, while a valid caching solution, introduces more operational overhead compared to DAX. You would need to manage the ElastiCache cluster, configure caching strategies, and handle cache invalidation. DAX is purpose-built for DynamoDB, simplifying these aspects and providing a tighter integration with the database. DAX requires less configuration because it understands DynamoDB access patterns and can handle the underlying data structures automatically.

In summary, DAX provides the simplest and most efficient solution to improve DynamoDB response time to microseconds through caching, with minimal operational overhead due to its tight integration and fully managed nature.

Here are some authoritative links for further research:

DynamoDB Accelerator (DAX): https://aws.amazon.com/dynamodb/dax/
DynamoDB Documentation - DAX:
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/DAX.html

Question: 579 **Exam Heist**

A company runs an application that uses Amazon RDS for PostgreSQL. The application receives traffic only on weekdays during business hours. The company wants to optimize costs and reduce operational overhead based on this usage.

Which solution will meet these requirements?

A.Use the Instance Scheduler on AWS to configure start and stop schedules.
B.Turn off automatic backups. Create weekly manual snapshots of the database.
C.Create a custom AWS Lambda function to start and stop the database based on minimum CPU utilization.
D.Purchase All Upfront reserved DB instances.

Answer: A

Explanation:

The correct answer is A: Use the Instance Scheduler on AWS to configure start and stop schedules.

Here's why:

The primary requirement is cost optimization for an RDS PostgreSQL database that's only needed during weekday business hours. The Instance Scheduler directly addresses this by automatically starting and stopping the RDS instance based on a predefined schedule. This eliminates compute costs during off-peak hours (nights and weekends) when the application isn't in use.

Option B is incorrect because turning off automatic backups creates a significant risk of data loss. Manual snapshots, while possible, increase operational overhead and require careful management to ensure backups are taken consistently and retained appropriately. Automatic backups are a fundamental best practice for database management.

Option C is less efficient than Instance Scheduler. While a Lambda function could technically achieve the same outcome, it requires custom coding, deployment, and maintenance. The Instance Scheduler is a pre-built, managed service specifically designed for this purpose, reducing operational overhead and providing a simpler configuration interface. Relying on CPU utilization as a trigger can be unreliable, as even idle databases consume some CPU resources, potentially leading to the instance staying on longer than

necessary.

Option D is incorrect because All Upfront reserved DB instances are a cost optimization strategy for databases running consistently. They don't address the requirement of turning off the database during off-peak hours, and the company would be paying for compute resources it isn't using. Reserved instances are most effective for consistent, high-utilization workloads.

Therefore, using the Instance Scheduler offers the best balance of cost optimization and operational simplicity, directly aligning with the company's requirement to reduce costs during off-peak hours while minimizing management effort.

Further Research:

AWS Instance Scheduler: https://aws.amazon.com/solutions/implementations/instance-scheduler/
RDS Cost Optimization: https://aws.amazon.com/rds/pricing/

Question: 580 **Exam Heist**

A company uses locally attached storage to run a latency-sensitive application on premises. The company is using a lift and shift method to move the application to the AWS Cloud. The company does not want to change the application architecture.

Which solution will meet these requirements MOST cost-effectively?

A.Configure an Auto Scaling group with an Amazon EC2 instance. Use an Amazon FSx for Lustre file system to run the application.

B.Host the application on an Amazon EC2 instance. Use an Amazon Elastic Block Store (Amazon EBS) GP2 volume to run the application.

C.Configure an Auto Scaling group with an Amazon EC2 instance. Use an Amazon FSx for OpenZFS file system to run the application.

D.Host the application on an Amazon EC2 instance. Use an Amazon Elastic Block Store (Amazon EBS) GP3 volume to run the application.

Answer: D

Explanation:

The best solution is to host the application on an Amazon EC2 instance and use an Amazon Elastic Block Store (Amazon EBS) GP3 volume.

Here's why:

Lift and Shift: The company wants to move the application without significant architectural changes. This implies a direct migration of the existing application stack to AWS.
Latency-Sensitive Application: The application requires low latency storage. EBS is designed for block storage and provides low latency access.
Cost-Effectiveness: GP3 volumes offer a balance of performance and cost, making them generally more cost-effective than other EBS volume types for many workloads.

Let's eliminate the other options:

Option A (FSx for Lustre): FSx for Lustre is a high-performance file system optimized for compute-intensive workloads. While it offers low latency, it's more complex to set up and generally more expensive than EBS, making it less cost-effective for a simple lift and shift.
Option B (EBS GP2): GP2 volumes are older generation EBS volumes. GP3 volumes offer better performance and pricing in many cases.

Option C (FSx for OpenZFS): FSx for OpenZFS is a fully managed file system service built on ZFS. While it provides features like data compression and snapshots, it adds complexity and can be more expensive than using EBS directly. It isn't the most cost-effective for a simple lift and shift of an application using locally attached storage.

In essence, using EBS directly attached to EC2 instance mirrors the on-premises setup of an application running with locally attached storage. Using the newer generation EBS volumes can provide cost and performance advantages.Here are some resources that could be helpful.

Amazon EBS Volume Types
Amazon FSx

Question: 581 **Exam Heist**

A company runs a stateful production application on Amazon EC2 instances. The application requires at least two EC2 instances to always be running.

A solutions architect needs to design a highly available and fault-tolerant architecture for the application. The solutions architect creates an Auto Scaling group of EC2 instances.

Which set of additional steps should the solutions architect take to meet these requirements?

A.Set the Auto Scaling group's minimum capacity to two. Deploy one On-Demand Instance in one Availability Zone and one On-Demand Instance in a second Availability Zone.

B.Set the Auto Scaling group's minimum capacity to four. Deploy two On-Demand Instances in one Availability Zone and two On-Demand Instances in a second Availability Zone.

C.Set the Auto Scaling group's minimum capacity to two. Deploy four Spot Instances in one Availability Zone.

D.Set the Auto Scaling group's minimum capacity to four. Deploy two On-Demand Instances in one Availability Zone and two Spot Instances in a second Availability Zone.

Answer: B

Explanation:

The correct answer is B because it ensures high availability and fault tolerance for the application. Here's why:

Minimum Capacity of Two: The requirement is that at least two EC2 instances must always be running. Setting the minimum capacity of the Auto Scaling group to two guarantees that the group will always attempt to maintain at least this number of instances.
High Availability Through Multiple Availability Zones: Distributing instances across multiple Availability Zones (AZs) is a core principle of high availability in AWS. If one AZ experiences an outage, the instances in the other AZ remain operational, ensuring the application continues to function. Deploying instances into two AZs meets this requirement. The question does not require spot instances.

Therefore, deploying two On-Demand Instances in one Availability Zone and two On-Demand Instances in a second Availability Zone ensures that the application is both fault-tolerant (can withstand the failure of an instance) and highly available (can withstand the failure of an Availability Zone). Setting the minimum capacity to four allows the Auto Scaling Group to function if an instance in either AZ fails.

Option A is incorrect because while it satisfies the minimum capacity requirement of two instances, it doesn't provide enough redundancy within each Availability Zone. A single instance failure in an AZ could impact performance or availability.

Option C and D use spot instances. Spot instances are cheaper than on-demand but come with a risk of interruption that doesn't align with the "always be running" condition of the application.

Further research:https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-groups.htmlhttps://wa.aws.amazon.com/wellarchitected/2020-07-02T19-33-23/best-practices/rel12017_design_for_high_availability.en.html

Question: 582 **Exam Heist**

An ecommerce company uses Amazon Route 53 as its DNS provider. The company hosts its website on premises and in the AWS Cloud. The company's on-premises data center is near the us-west-1 Region. The company uses the eu-central-1 Region to host the website. The company wants to minimize load time for the website as much as possible.

Which solution will meet these requirements?

A.Set up a geolocation routing policy. Send the traffic that is near us-west-1 to the on-premises data center. Send the traffic that is near eu-central-1 to eu-central-1.

B.Set up a simple routing policy that routes all traffic that is near eu-central-1 to eu-central-1 and routes all traffic that is near the on-premises datacenter to the on-premises data center.

C.Set up a latency routing policy. Associate the policy with us-west-1.

D.Set up a weighted routing policy. Split the traffic evenly between eu-central-1 and the on-premises data center.

Answer: A

Explanation:

The correct answer is **A. Set up a geolocation routing policy. Send the traffic that is near us-west-1 to the on-premises data center. Send the traffic that is near eu-central-1 to eu-central-1.**

Here's why:

The primary goal is to minimize website load time for users globally. Geolocation routing in Route 53 allows directing users to specific resources based on their geographic location. By directing users geographically close to the on-premises data center (near us-west-1) to that data center and users geographically close to the eu-central-1 region to the AWS eu-central-1 Region, the solution reduces latency, and thus speeds up load times, for the majority of users. This approach minimizes the distance data travels, leading to faster response times.

Let's analyze why the other options are not optimal:

B. Simple routing policy: Simple routing policies provide no intelligent distribution based on location or latency, making it less effective for minimizing load times across different geographic areas. It will not effectively distribute traffic based on user location proximity.
C. Latency routing policy: While latency routing aims to minimize latency, it only considers the latency between the user's DNS resolver and the AWS regions. It doesn't take into account the on-premises data center's location.
D. Weighted routing policy: Weighted routing balances traffic across multiple resources but doesn't consider the user's location. It may not efficiently minimize latency for users in specific regions.

Therefore, geolocation routing provides the best approach to serve users from the geographically closest location, resulting in the lowest latency and best user experience.

For further research, consider these resources:

AWS Route 53 Routing Policies: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html

Geolocation Routing: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy-geo.html

Question: 583 **Exam Heist**

A company has 5 PB of archived data on physical tapes. The company needs to preserve the data on the tapes for another 10 years for compliance purposes. The company wants to migrate to AWS in the next 6 months. The data center that stores the tapes has a 1 Gbps uplink internet connectivity.

Which solution will meet these requirements MOST cost-effectively?

A.Read the data from the tapes on premises. Stage the data in a local NFS storage. Use AWS DataSync to migrate the data to Amazon S3 Glacier Flexible Retrieval.

B.Use an on-premises backup application to read the data from the tapes and to write directly to Amazon S3 Glacier Deep Archive.

C.Order multiple AWS Snowball devices that have Tape Gateway. Copy the physical tapes to virtual tapes in Snowball. Ship the Snowball devices to AWS. Create a lifecycle policy to move the tapes to Amazon S3 Glacier Deep Archive.

D.Configure an on-premises Tape Gateway. Create virtual tapes in the AWS Cloud. Use backup software to copy the physical tape to the virtual tape.

Answer: C

Explanation:

The most cost-effective solution for migrating 5 PB of tape-archived data to AWS for a 10-year compliance retention period, given a limited 1 Gbps internet uplink, is option C, leveraging AWS Snowball Edge with Tape Gateway.

Here's a detailed justification:

1. **High Data Volume & Low Bandwidth:** Migrating 5 PB over a 1 Gbps link would take an impractically long time (potentially years). Network-based transfer solutions (options A & B) become highly inefficient due to the sheer data volume and bandwidth limitations.
2. **AWS Snowball Edge (Data Transfer Device):** Snowball Edge devices provide a secure, physically transportable way to move massive amounts of data to AWS, bypassing the bandwidth constraints of the internet connection. https://aws.amazon.com/snowball/
3. **Tape Gateway Integration:** Snowball Edge can be configured with Tape Gateway, allowing the company to treat the Snowball device as a virtual tape library on-premises. This enables direct copying of data from physical tapes to virtual tapes stored on the Snowball. https://aws.amazon.com/storagegateway/tape-gateway/
4. **S3 Glacier Deep Archive (Cost-Effective Archival):** Amazon S3 Glacier Deep Archive is the lowest-cost storage class, ideal for long-term data archiving with infrequent access requirements. https://aws.amazon.com/glacier/pricing/
5. **Lifecycle Policy Automation:** A lifecycle policy in S3 can automatically transition the virtual tapes uploaded from Snowball to S3 Glacier Deep Archive after a specified period. This ensures the data is stored in the most cost-effective storage tier for long-term retention.
6. **Cost Efficiency Comparison:** While option B might seem appealing in its simplicity, the prolonged transfer time over the limited internet connection would incur significant operational costs (network usage, staff time). Option A involves staging data in NFS storage, adding an unnecessary intermediate step and infrastructure cost. Option C minimizes infrastructure and network costs by using a physical appliance and leveraging the lowest-cost archive tier.

7. **Compliance Considerations:** S3 Glacier Deep Archive offers features like data encryption and versioning, which support compliance requirements for long-term data retention and integrity. Snowball transfer are also secured by KMS encryption.

Therefore, ordering multiple Snowball devices equipped with Tape Gateway, copying tapes to Snowball, shipping them to AWS, and utilizing a lifecycle policy to archive to Glacier Deep Archive provides the most cost-effective and practical solution for the company's needs, circumventing the bandwidth limitation and meeting the 10-year compliance requirement at minimal cost.

Question: 584 **Exam Heist**

A company is deploying an application that processes large quantities of data in parallel. The company plans to use Amazon EC2 instances for the workload. The network architecture must be configurable to prevent groups of nodes from sharing the same underlying hardware.

Which networking solution meets these requirements?

A.Run the EC2 instances in a spread placement group.
B.Group the EC2 instances in separate accounts.
C.Configure the EC2 instances with dedicated tenancy.
D.Configure the EC2 instances with shared tenancy.

Answer: A

Explanation:

The correct answer is A: Run the EC2 instances in a spread placement group. Here's why:

The requirement is to prevent groups of nodes (EC2 instances) from sharing the same underlying hardware for an application processing large quantities of data in parallel. This isolates instances, minimizing the impact of failures or performance issues on one host affecting others.

Spread placement groups are designed to meet precisely this requirement. They ensure that each instance within the group runs on distinct underlying hardware. This provides high availability and minimizes correlated failures. AWS guarantees that a small number of instances within a spread placement group won't share the same physical hardware.

Option B (Separate accounts) technically isolates instances at a management level, but it doesn't guarantee hardware separation without additional configurations and can significantly increase management overhead. It is not primarily a network configuration choice.

Option C (Dedicated tenancy) ensures EC2 instances run on hardware dedicated solely to the account, avoiding sharing with other AWS customers. While this provides isolation, it is more costly and doesn't necessarily prevent instances within the account from potentially sharing hardware unless combined with placement groups.

Option D (Shared tenancy) is the default tenancy and allows EC2 instances to share hardware with other AWS customers, which is the opposite of what's required.

Therefore, spread placement groups are the most appropriate solution because they focus on distributing instances across distinct hardware while staying within a single AWS account for simplified management and network configuration. The other options either don't directly address the hardware isolation requirement (separate accounts) or contradict it (shared tenancy) or are more expensive for the same result (dedicated tenancy, without necessarily the needed spread).

Authoritative Links:

AWS EC2 Placement Groups: Detailed documentation on placement groups, including spread placement groups.
AWS EC2 Tenancy Options: Information on dedicated tenancy and its use cases.

Question: 585 **Exam Heist**

A solutions architect is designing a disaster recovery (DR) strategy to provide Amazon EC2 capacity in a failover AWS Region. Business requirements state that the DR strategy must meet capacity in the failover Region.

Which solution will meet these requirements?

A.Purchase On-Demand Instances in the failover Region.
B.Purchase an EC2 Savings Plan in the failover Region.
C.Purchase regional Reserved Instances in the failover Region.
D.Purchase a Capacity Reservation in the failover Region.

Answer: D

Explanation:

The correct answer is **D. Purchase a Capacity Reservation in the failover Region.**

Justification:

The primary requirement is to meet capacity in the failover region during a disaster recovery event. Capacity Reservations guarantee that EC2 capacity will be available in a specific Availability Zone when you need it. This directly addresses the requirement of ensuring capacity for EC2 instances in the failover region.

Option A (On-Demand Instances): While On-Demand Instances are readily available, they do not guarantee capacity. During a disaster, demand for resources in the failover Region may surge, potentially leading to insufficient capacity to launch all required instances.

Option B (EC2 Savings Plan): Savings Plans offer cost savings based on committed usage. However, Savings Plans do not guarantee capacity. They simply offer discounted pricing for EC2 usage. The instances still compete for available capacity.

Option C (Regional Reserved Instances): Regional Reserved Instances provide a discount on EC2 usage and provide a capacity reservation at a regional level. Although these instances provide increased availability of capacity, they do not guarantee the availability of resources during a widespread disaster.

By using Capacity Reservations, the organization can ensure that the necessary EC2 resources are always available, even during a DR event where demand for resources spikes. This proactively addresses the capacity constraint and ensures the DR strategy can successfully meet the business requirements.

Authoritative Links:

Capacity Reservations
EC2 Instance Purchasing Options

Question: 586 **Exam Heist**

A company has five organizational units (OUs) as part of its organization in AWS Organizations. Each OU correlates to the five businesses that the company owns. The company's research and development (R&D) business is

separating from the company and will need its own organization. A solutions architect creates a separate new management account for this purpose.

What should the solutions architect do next in the new management account?

A.Have the R&D AWS account be part of both organizations during the transition.

B.Invite the R&D AWS account to be part of the new organization after the R&D AWS account has left the prior organization.

C.Create a new R&D AWS account in the new organization. Migrate resources from the prior R&D AWS account to the new R&D AWS account.

D.Have the R&D AWS account join the new organization. Make the new management account a member of the prior organization.

Answer: B

Explanation:

The correct answer is B: Invite the R&D AWS account to be part of the new organization after the R&D AWS account has left the prior organization.

Here's why:

The goal is to move the R&D business's AWS account from the existing AWS Organization to a completely separate one. AWS Organizations does not allow an account to be part of two organizations simultaneously. Therefore, the R&D account must first leave the original organization before it can join the new one.

Option A is incorrect because an AWS account cannot be a member of two AWS Organizations at the same time. This violates the fundamental structure and control mechanisms of AWS Organizations.

Option C is not the most efficient approach. Creating a new account and migrating resources would involve significant time, effort, and potential disruption. Direct account transfer, after detachment, is a more streamlined method.

Option D is incorrect because the new management account (which represents the new R&D organization) should not be a member of the prior organization. The R&D business is meant to be entirely independent. Making the new management account a member of the prior organization would defeat the purpose of the separation and create a dependency.

The correct procedure involves first removing the R&D account from the initial organization. After this detachment, the new organization's management account can send an invitation to the detached R&D account. Once accepted, the R&D account becomes a member of the new organization. This ensures a clean separation and transfer of the AWS account to its new, independent organizational structure.

https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_accounts_remove.htmlhttps://docs.aw

Question: 587 **Exam Heist**

A company is designing a solution to capture customer activity in different web applications to process analytics and make predictions. Customer activity in the web applications is unpredictable and can increase suddenly. The company requires a solution that integrates with other web applications. The solution must include an authorization step for security purposes.

Which solution will meet these requirements?

A.Configure a Gateway Load Balancer (GWLB) in front of an Amazon Elastic Container Service (Amazon ECS) container instance that stores the information that the company receives in an Amazon Elastic File System (Amazon EFS) file system. Authorization is resolved at the GWLB.

B.Configure an Amazon API Gateway endpoint in front of an Amazon Kinesis data stream that stores the information that the company receives in an Amazon S3 bucket. Use an AWS Lambda function to resolve authorization.

C.Configure an Amazon API Gateway endpoint in front of an Amazon Kinesis Data Firehose that stores the information that the company receives in an Amazon S3 bucket. Use an API Gateway Lambda authorizer to resolve authorization.

D.Configure a Gateway Load Balancer (GWLB) in front of an Amazon Elastic Container Service (Amazon ECS) container instance that stores the information that the company receives on an Amazon Elastic File System (Amazon EFS) file system. Use an AWS Lambda function to resolve authorization.

Answer: C

Explanation:

The correct answer is C. Here's a detailed justification:

The problem requires a solution for capturing unpredictable customer activity from web applications, integrating with existing apps, and implementing authorization.

Option C leverages Amazon API Gateway, which is designed to handle API requests from various sources, including web applications. This satisfies the integration requirement. API Gateway can scale automatically to handle unpredictable surges in traffic, meeting the scalability requirement.

Kinesis Data Firehose is a suitable choice for ingesting high volumes of streaming data directly into destinations like Amazon S3. Storing the data in S3 allows for cost-effective storage and future analysis using services like Athena or Redshift. This fulfills the data capture and storage requirements.

Crucially, Option C uses an API Gateway Lambda authorizer. Lambda authorizers (formerly known as custom authorizers) are powerful tools that allow you to implement custom authorization logic for your API Gateway endpoints. This directly addresses the authorization requirement, providing fine-grained control over who can access your API and the data it processes. The Lambda function can verify identity, check permissions, and return an IAM policy that determines what resources the caller is authorized to access.

Options A and D use Gateway Load Balancers (GWLBs) and ECS containers. While GWLB handles network traffic, it's not the primary service for API management and authorization in the context of web applications. ECS containers storing data on EFS are also not the most efficient or scalable solution for handling streaming data from web applications. The authorization methods in A and D are also less integrated than the API Gateway Lambda Authorizer in C.

Option B uses Kinesis Data Streams. While Streams can handle high-velocity data, Data Firehose is a more suitable choice when you need to reliably load data into data lakes or data warehouses like S3, especially with a large volume of web application event activity. Furthermore, the Lambda function used for authorization in option B is outside of the AWS API Gateway and might be more difficult to maintain for complex API authorizations.

In summary, option C best meets all the requirements by providing a scalable API endpoint with authorization and efficient data ingestion and storage, making it the optimal solution.

Supporting links:

Amazon API Gateway: https://aws.amazon.com/api-gateway/
Amazon Kinesis Data Firehose: https://aws.amazon.com/kinesis/data-firehose/
Amazon S3: https://aws.amazon.com/s3/
API Gateway Lambda Authorizers:
https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-use-lambda-authorizer.html

Question: 588 **Exam Heist**

An ecommerce company wants a disaster recovery solution for its Amazon RDS DB instances that run Microsoft SQL Server Enterprise Edition. The company's current recovery point objective (RPO) and recovery time objective (RTO) are 24 hours.

Which solution will meet these requirements MOST cost-effectively?

A.Create a cross-Region read replica and promote the read replica to the primary instance.

B.Use AWS Database Migration Service (AWS DMS) to create RDS cross-Region replication.

C.Use cross-Region replication every 24 hours to copy native backups to an Amazon S3 bucket.

D.Copy automatic snapshots to another Region every 24 hours.

Answer: D

Explanation:

The question asks for the most cost-effective disaster recovery solution for RDS SQL Server with a 24-hour RPO/RTO. Let's analyze the options:

A. Create a cross-Region read replica and promote the read replica to the primary instance: Read replicas provide faster failover times compared to snapshots, potentially meeting a shorter RTO. However, for a 24-hour RTO, this option is generally more expensive because read replicas continuously replicate data, incurring higher operational costs. For this 24-hour RTO, using Snapshots is enough.

B. Use AWS Database Migration Service (AWS DMS) to create RDS cross-Region replication: AWS DMS is more suitable for heterogeneous database migrations and ongoing replication. For disaster recovery with a 24-hour RPO/RTO for RDS SQL Server, it adds complexity and cost compared to native snapshot replication.

C. Use cross-Region replication every 24 hours to copy native backups to an Amazon S3 bucket: Although a valid option, this manual copy approach may be less efficient than using automated snapshot copy feature offered by AWS. The snapshot copy automates the process of moving backups across regions.

D. Copy automatic snapshots to another Region every 24 hours: RDS SQL Server has a native feature to automate snapshot creation and cross-Region copy. Configuring automatic snapshots with a cross-Region copy every 24 hours meets the RPO/RTO of 24 hours. This approach is cost-effective because you only pay for storage and the occasional snapshot copy operation.

Therefore, copying automated snapshots cross-Region is the most cost-effective approach for a 24-hour RPO/RTO, utilizing the native capabilities of AWS RDS.

Refer to https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_WorkingWithAutomatedBackups.html and https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/SQLServer.Procedural.Importing.html for details on RDS SQL Server backups and disaster recovery. Also read more about RTO/RPO best practices https://aws.amazon.com/blogs/storage/backup-and-restore-strategies-for-amazon-elastic-file-system-amazon-efs/

Question: 589 **Exam Heist**

A company runs a web application on Amazon EC2 instances in an Auto Scaling group behind an Application Load Balancer that has sticky sessions enabled. The web server currently hosts the user session state. The company wants to ensure high availability and avoid user session state loss in the event of a web server outage.

Which solution will meet these requirements?

A.Use an Amazon ElastiCache for Memcached instance to store the session data. Update the application to use ElastiCache for Memcached to store the session state.

B.Use Amazon ElastiCache for Redis to store the session state. Update the application to use ElastiCache for Redis to store the session state.

C.Use an AWS Storage Gateway cached volume to store session data. Update the application to use AWS Storage Gateway cached volume to store the session state.

D.Use Amazon RDS to store the session state. Update the application to use Amazon RDS to store the session state.

Answer: B

Explanation:

The requirement is to ensure high availability and prevent session state loss when EC2 instances running a web application in an Auto Scaling group behind an ALB fail. Since the current setup stores the user session state on the web server itself, a server outage leads to session loss.

Option B, using Amazon ElastiCache for Redis, is the best solution because it provides a highly available and persistent store for user session data. Redis is an in-memory data structure store that can be configured for persistence, which prevents data loss in case of failure. By updating the application to use ElastiCache for Redis, the session state is stored externally to the EC2 instances. Therefore, even if an instance fails, the session data is preserved in the Redis cluster.

Option A, using ElastiCache for Memcached, is less ideal. While Memcached offers fast caching, it is primarily designed for caching data that can be easily recreated and doesn't natively support persistence. This means in case of a Memcached instance failure, the session data is lost.

Option C, utilizing AWS Storage Gateway cached volume, is not suitable for storing session state. Storage Gateway primarily focuses on bridging on-premises storage with AWS cloud storage. It introduces significant latency compared to an in-memory cache, negatively impacting application performance. Session data needs to be rapidly accessible.

Option D, using Amazon RDS, while a persistent storage option, is generally not optimized for the speed and access patterns required for session management. Relational databases like RDS introduce overhead that significantly impacts performance compared to in-memory caching solutions like Redis or Memcached. RDS is much slower at retrieving user session data.Redis offers better performance.

Therefore, ElastiCache for Redis addresses the high availability and session state persistence requirement in the most effective manner.

Amazon ElastiCache for RedisSession Management with ElastiCache

Question: 590 **Exam Heist**

A company migrated a MySQL database from the company's on-premises data center to an Amazon RDS for MySQL DB instance. The company sized the RDS DB instance to meet the company's average daily workload. Once a month, the database performs slowly when the company runs queries for a report. The company wants to have the ability to run reports and maintain the performance of the daily workloads.

Which solution will meet these requirements?

A.Create a read replica of the database. Direct the queries to the read replica.

B.Create a backup of the database. Restore the backup to another DB instance. Direct the queries to the new database.

C.Export the data to Amazon S3. Use Amazon Athena to query the S3 bucket.

D.Resize the DB instance to accommodate the additional workload.

Answer: A

Explanation:

Option A, creating a read replica and directing report queries to it, is the most suitable solution because it directly addresses the problem of performance degradation during report generation without impacting the primary database's daily workload. Read replicas in Amazon RDS provide a near real-time, read-only copy of the primary database. By offloading the resource-intensive report queries to the read replica, the primary database remains available and performs optimally for regular transactions and operations. This avoids performance bottlenecks for daily users while still allowing for reporting functionality. It's also a cost-effective approach, as the read replica can be sized appropriately for the reporting workload, and the data replication is managed by RDS. Option B, while isolating the reporting workload, introduces complexity with creating and restoring backups, which can be time-consuming. Furthermore, restoring a backup to a new DB instance could potentially lead to data staleness, as it represents a point-in-time snapshot of the data, and not the current database state. Option C, while potentially scalable, involves exporting data to S3 and using Athena, which is suited for analyzing large datasets. This approach is overkill for a simple reporting requirement, adds complexity and potential latency. Option D, resizing the DB instance, addresses the symptom but not the root cause. It is cost-inefficient because the larger instance would be underutilized most of the time. Using a read replica provides a more targeted and cost-effective scaling strategy only during report generation periods.

Authoritative Links:

Amazon RDS Read Replicas:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.ReadReplicas.html
Amazon Athena: https://aws.amazon.com/athena/

Question: 591 **Exam Heist**

A company runs a container application by using Amazon Elastic Kubernetes Service (Amazon EKS). The application includes microservices that manage customers and place orders. The company needs to route incoming requests to the appropriate microservices.

Which solution will meet this requirement MOST cost-effectively?

A.Use the AWS Load Balancer Controller to provision a Network Load Balancer.
B.Use the AWS Load Balancer Controller to provision an Application Load Balancer.
C.Use an AWS Lambda function to connect the requests to Amazon EKS.
D.Use Amazon API Gateway to connect the requests to Amazon EKS.

Answer: D

Explanation:

The most cost-effective solution for routing incoming requests to microservices within an Amazon EKS cluster for the given scenario is **D. Use Amazon API Gateway to connect the requests to Amazon EKS.**

Here's a detailed justification:

API Gateway's Role: Amazon API Gateway is designed to handle API requests at scale. It acts as a single entry point for all incoming requests to your application. It can route these requests to different backend services based on the request's path, headers, or other attributes.

Cost-Effectiveness:

Pay-as-you-go Pricing: API Gateway operates on a pay-as-you-go model, charging only for the API calls you receive and the data transferred. This is often more cost-effective than running a dedicated load balancer constantly, especially for applications with variable traffic.
Reduced Infrastructure Overhead: By offloading API management tasks to API Gateway, you reduce the operational burden on your EKS cluster and simplify your infrastructure.

Functionality:

Request Routing: API Gateway offers sophisticated request routing capabilities based on URL paths, HTTP headers, query parameters, and more. This allows precise control over how requests are directed to your microservices.
Security: API Gateway provides security features such as authentication, authorization, and request validation to protect your application.
Traffic Management: API Gateway can manage traffic by throttling requests to prevent overload on the EKS cluster.
Integration with EKS: API Gateway can integrate with EKS through the use of Ingress Controller and Service Discovery.

Why other options are less cost-effective or less suitable:

A & B (Load Balancers): While AWS Load Balancer Controller can provision Application Load Balancer (ALB) or Network Load Balancer (NLB), they are typically more expensive than API Gateway for scenarios where you require complex routing and API management features. Load balancers also require continuous operation, leading to higher costs even when the application is not heavily used. Furthermore, using an NLB would require you to manage TLS termination and additional complexity that ALB and API Gateway can offload.
C (Lambda Function): Using Lambda functions as a gateway to EKS is typically not recommended. Lambda has execution time limits and might not be cost-effective for handling high-volume, long-running requests. It also adds complexity to the application architecture.

In conclusion, Amazon API Gateway provides the most cost-effective and feature-rich solution for routing requests to microservices within an Amazon EKS cluster by offering fine-grained control over request routing, security, and traffic management, while adhering to a pay-as-you-go pricing model.

Authoritative Links:

Amazon API Gateway: https://aws.amazon.com/api-gateway/
AWS Load Balancer Controller: https://github.com/kubernetes-sigs/aws-load-balancer-controller

Question: 592 **Exam Heist**

A company uses AWS and sells access to copyrighted images. The company's global customer base needs to be able to access these images quickly. The company must deny access to users from specific countries. The company wants to minimize costs as much as possible.

Which solution will meet these requirements?

A.Use Amazon S3 to store the images. Turn on multi-factor authentication (MFA) and public bucket access. Provide customers with a link to the S3 bucket.

B.Use Amazon S3 to store the images. Create an IAM user for each customer. Add the users to a group that has permission to access the S3 bucket.

C.Use Amazon EC2 instances that are behind Application Load Balancers (ALBs) to store the images. Deploy the instances only in the countries the company services. Provide customers with links to the ALBs for their specific country's instances.

D.Use Amazon S3 to store the images. Use Amazon CloudFront to distribute the images with geographic

restrictions. Provide a signed URL for each customer to access the data in CloudFront.

Answer: D

Explanation:

The correct answer is D because it leverages both S3 and CloudFront to meet the requirements of fast global access, geographic restrictions, and cost optimization.

Here's a detailed justification:

S3 for Storage: Amazon S3 provides highly durable, scalable, and cost-effective storage for the images. Storing the images in S3 as the origin simplifies the overall architecture and management overhead.
CloudFront for Distribution: CloudFront is a Content Delivery Network (CDN) that caches content at edge locations worldwide. This ensures low latency and fast access to the images for global customers, fulfilling the first requirement.
Geographic Restrictions: CloudFront allows you to configure geographic restrictions, also known as geo-filtering or geo-blocking, to deny access to content based on the viewer's country. This directly addresses the requirement to deny access to users from specific countries.
(https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/georestrictions.html)
Signed URLs: Signed URLs provide controlled access to the images. They allow you to grant temporary access to specific images to authorized users without exposing the underlying S3 bucket directly. This enhances security and control. This approach ensures that users can access the images only when they have a valid, signed URL.
Cost Optimization: Using CloudFront helps optimize costs by caching content closer to users, reducing the load on the S3 bucket and potentially lowering S3 data transfer costs. CloudFront's pay-as-you-go pricing model also aligns with the company's cost minimization goal.
Why other options are not optimal:

A: MFA and public bucket access are contradictory to each other. MFA is to secure bucket while Public Bucket exposes the bucket and is insecure.
B: Creating IAM users for each customer is not scalable and can be difficult to manage as the customer base grows. It also doesn't directly address the geographic restriction requirement.
C: Deploying EC2 instances in specific countries could be complex to manage, and it's not an optimal method to restrict customer access globally. It also might not provide the best performance for users outside those specific countries and cost inefficient compared to CloudFront.

In conclusion, option D is the most suitable solution because it efficiently combines S3 for storage, CloudFront for global content delivery and geo-restrictions, and signed URLs for controlled access, while aligning with the company's cost optimization goals.

Question: 593 **Exam Heist**

A solutions architect is designing a highly available Amazon ElastiCache for Redis based solution. The solutions architect needs to ensure that failures do not result in performance degradation or loss of data locally and within an AWS Region. The solution needs to provide high availability at the node level and at the Region level.

Which solution will meet these requirements?

A.Use Multi-AZ Redis replication groups with shards that contain multiple nodes.
B.Use Redis shards that contain multiple nodes with Redis append only files (AOF) turned on.
C.Use a Multi-AZ Redis cluster with more than one read replica in the replication group.
D.Use Redis shards that contain multiple nodes with Auto Scaling turned on.

Answer: A

Explanation:

The correct answer is **A. Use Multi-AZ Redis replication groups with shards that contain multiple nodes.**

Here's a detailed justification:

High Availability at the Node Level (within a shard): Redis replication groups with multiple nodes in each shard provide node-level high availability. A primary node handles writes, and one or more read replicas asynchronously replicate the data. If the primary node fails, one of the read replicas is automatically promoted to primary, minimizing downtime.

Multi-AZ for Regional High Availability: Deploying the Redis replication group across multiple Availability Zones (Multi-AZ) provides resilience against AZ-level failures. If an entire AZ goes down, the automatic failover mechanism will promote a replica in a different AZ to become the new primary. This ensures that the application continues to function without significant interruption.

Data Durability: Redis replication ensures that data is not lost in case of a node failure within a shard. The asynchronous replication mechanism keeps replicas updated with the changes made to the primary. Even though asynchronous replication may lead to slight data loss in extreme scenarios, it offers the best balance between performance and data safety.

Why other options are incorrect:

B. Use Redis shards that contain multiple nodes with Redis append only files (AOF) turned on: AOF provides data durability in case of server restarts and is not a solution for node-level high availability or region-level high availability. AOF stores every write operation in a log. If a node crashes and is restarted, Redis can reconstruct the dataset by replaying the AOF log. However, it does not automatically failover to another node in case of a primary failure.
C. Use a Multi-AZ Redis cluster with more than one read replica in the replication group: This option is partially correct in its usage of Multi-AZ and read replicas, but the term "cluster" isn't appropriate here. ElastiCache for Redis uses replication groups (for non-cluster mode) or clusters (for cluster mode). Also, the replication group consists of one primary and multiple replicas.
D. Use Redis shards that contain multiple nodes with Auto Scaling turned on: Auto Scaling will not provide high availability since the process of scaling up the number of shards will take time to complete and there would still be data loss in case of node failure. It helps manage the capacity of your ElastiCache cluster based on demand but does not offer an immediate failover mechanism for high availability. Auto Scaling is more suitable for addressing scalability concerns rather than immediate failure recovery.

Authoritative Links:

Amazon ElastiCache for Redis: High Availability
Replication: Redis Documentation

Question: 594 **Exam Heist**

A company plans to migrate to AWS and use Amazon EC2 On-Demand Instances for its application. During the migration testing phase, a technical team observes that the application takes a long time to launch and load memory to become fully productive.

Which solution will reduce the launch time of the application during the next testing phase?

A.Launch two or more EC2 On-Demand Instances. Turn on auto scaling features and make the EC2 On-Demand Instances available during the next testing phase.

B.Launch EC2 Spot Instances to support the application and to scale the application so it is available during the

next testing phase.

C.Launch the EC2 On-Demand Instances with hibernation turned on. Configure EC2 Auto Scaling warm pools during the next testing phase.

D.Launch EC2 On-Demand Instances with Capacity Reservations. Start additional EC2 instances during the next testing phase.

Answer: C

Explanation:

The correct answer is C: Launch the EC2 On-Demand Instances with hibernation turned on. Configure EC2 Auto Scaling warm pools during the next testing phase. This approach effectively reduces application launch time.

Here's why:

Hibernation: Hibernation saves the in-memory state of an EC2 instance to the EBS root volume when the instance is stopped. Upon restart, the instance resumes from this saved state, drastically reducing the time it takes to load the application and its data back into memory. (https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/Hibernate.html)

Warm Pools: EC2 Auto Scaling warm pools allow you to pre-initialize a group of EC2 instances before they are needed. These instances are already launched and configured, waiting in a "ready" state. This eliminates the delay of launching and configuring instances when demand increases. (https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-warm-pools.html)

By combining hibernation and warm pools, instances can be pre-launched, pre-configured, and their state saved. When needed, instances resume quickly from their hibernated state within the warm pool, significantly decreasing the time it takes for the application to become fully productive during the next testing phase.

Option A is less effective because simply launching more instances and using auto-scaling doesn't address the underlying issue of slow application loading. Auto-scaling helps with scaling in response to demand but doesn't inherently speed up individual instance launch and initialization.

Option B, using Spot Instances, introduces the risk of instance interruption if Spot prices rise above the bid price, which is unsuitable for predictable testing. Furthermore, it doesn't solve the problem of the initial slow loading time.

Option D, Capacity Reservations, guarantees EC2 capacity but doesn't inherently reduce application launch time. It ensures resources are available, but the application will still take the same time to load into memory on newly launched instances. The addition of extra instances during the testing phase will have no benefit.

Question: 595 **Exam Heist**

A company's applications run on Amazon EC2 instances in Auto Scaling groups. The company notices that its applications experience sudden traffic increases on random days of the week. The company wants to maintain application performance during sudden traffic increases.

Which solution will meet these requirements MOST cost-effectively?

A.Use manual scaling to change the size of the Auto Scaling group.

B.Use predictive scaling to change the size of the Auto Scaling group.

C.Use dynamic scaling to change the size of the Auto Scaling group.

D.Use schedule scaling to change the size of the Auto Scaling group.

Answer: C

Explanation:

The most cost-effective solution is **C. Use dynamic scaling to change the size of the Auto Scaling group.**

Here's why:

Dynamic Scaling (also known as Target Tracking or Step Scaling): This approach automatically adjusts the number of EC2 instances in response to real-time changes in demand, based on pre-defined metrics like CPU utilization or network traffic. Because the traffic increases are sudden and on random days, a dynamic scaling policy allows the Auto Scaling group to react immediately, maintaining performance.

Why other options are not as suitable:

Manual Scaling: Requires human intervention to adjust the Auto Scaling group, which is not ideal for sudden traffic increases. This approach is time-consuming and may result in performance degradation during the scaling process.
Predictive Scaling: Analyzes historical traffic patterns to anticipate future demand and proactively adjust the Auto Scaling group. While effective for predictable patterns, it is less suitable for random traffic spikes as it relies on predictable patterns.
Scheduled Scaling: Scales the Auto Scaling group based on a pre-defined schedule. Since the traffic increases occur on random days, scheduled scaling is not a viable option.

Dynamic scaling is the optimal choice as it provides real-time responsiveness to fluctuating workloads without manual intervention or reliance on historical data, ensuring application performance is maintained during unexpected traffic increases in a cost-effective manner.

Here are authoritative links for further research:

AWS Auto Scaling Documentation: https://docs.aws.amazon.com/autoscaling/ec2/userguide/
Scaling Your Application on AWS: https://aws.amazon.com/blogs/architecture/scaling-your-application-on-aws/
AWS Well-Architected Framework: https://wa.aws.amazon.com/ (Focus on the Performance Efficiency pillar for scaling strategies).

Question: 596 **Exam Heist**

An ecommerce application uses a PostgreSQL database that runs on an Amazon EC2 instance. During a monthly sales event, database usage increases and causes database connection issues for the application. The traffic is unpredictable for subsequent monthly sales events, which impacts the sales forecast. The company needs to maintain performance when there is an unpredictable increase in traffic.

Which solution resolves this issue in the MOST cost-effective way?

A.Migrate the PostgreSQL database to Amazon Aurora Serverless v2.
B.Enable auto scaling for the PostgreSQL database on the EC2 instance to accommodate increased usage.
C.Migrate the PostgreSQL database to Amazon RDS for PostgreSQL with a larger instance type.
D.Migrate the PostgreSQL database to Amazon Redshift to accommodate increased usage.

Answer: A

Explanation:

The best solution is to migrate the PostgreSQL database to Amazon Aurora Serverless v2. Here's why:

Aurora Serverless v2 Cost Efficiency: Aurora Serverless v2 automatically scales database capacity based on application needs. This is ideal for unpredictable traffic patterns like the monthly sales event. You only pay for the capacity you consume, making it highly cost-effective compared to constantly running a larger, potentially underutilized, instance.

RDS for PostgreSQL Scaling Limitations: While RDS for PostgreSQL with a larger instance type (Option C) can handle increased load, it involves vertical scaling (changing the instance size). This requires downtime and can be more expensive than Aurora Serverless v2 if the increased capacity is only needed intermittently.

EC2 Auto Scaling Complexities: Enabling auto-scaling for PostgreSQL on an EC2 instance (Option B) is more complex and requires significant configuration for replication, failover, and data consistency. It involves creating and managing multiple EC2 instances, which adds operational overhead and can be less cost-effective than Aurora Serverless v2, especially given the unpredictable workload.

Redshift is Inappropriate: Amazon Redshift (Option D) is a data warehouse service designed for analytical workloads (OLAP), not transactional workloads (OLTP) like an ecommerce application's database. Migrating to Redshift would require significant application changes and is not suitable for resolving database connection issues during sales events.

Minimal Downtime Migration: Aurora offers migration tools and strategies that can minimize downtime during the migration from PostgreSQL on EC2.

Simplified Management: Aurora Serverless v2 simplifies database management, reducing operational overhead compared to managing a PostgreSQL database on EC2 or using EC2 auto scaling. It takes care of patching, backups, and other maintenance tasks.

In summary, Aurora Serverless v2 provides the most cost-effective and scalable solution for handling unpredictable traffic spikes in a PostgreSQL database used by an ecommerce application. It eliminates the need for over-provisioning resources, simplifies management, and minimizes downtime during migration.

Here are some authoritative links for further research:

Amazon Aurora Serverless v2: https://aws.amazon.com/rds/aurora/serverless/
Amazon RDS: https://aws.amazon.com/rds/
Amazon Redshift: https://aws.amazon.com/redshift/

Question: 597 **Exam Heist**

A company hosts an internal serverless application on AWS by using Amazon API Gateway and AWS Lambda. The company's employees report issues with high latency when they begin using the application each day. The company wants to reduce latency.

Which solution will meet these requirements?

A.Increase the API Gateway throttling limit.

B.Set up a scheduled scaling to increase Lambda provisioned concurrency before employees begin to use the application each day.

C.Create an Amazon CloudWatch alarm to initiate a Lambda function as a target for the alarm at the beginning of each day.

D.Increase the Lambda function memory.

Answer: B

Explanation:

The correct answer is B: Set up a scheduled scaling to increase Lambda provisioned concurrency before

employees begin to use the application each day. Here's a detailed justification:

The problem describes high latency at the beginning of each day when employees start using the serverless application. This suggests a cold start issue with the Lambda functions. Lambda functions operate on a pay-per-use model, meaning AWS only provisions resources when the function is invoked. When a function hasn't been used for a while, it enters a "cold state," and the first invocation triggers a cold start. A cold start involves provisioning the execution environment, loading the code, and initializing the function, which adds significant latency.

Provisioned Concurrency (PC) addresses this cold start issue by pre-initializing a specified number of Lambda function execution environments. These environments are ready to respond to requests immediately, eliminating the cold start latency. By scheduling an increase in provisioned concurrency before employees begin their workday, the necessary execution environments are already warm and waiting, significantly reducing latency for the initial requests.

Option A, increasing the API Gateway throttling limit, addresses a different problem. Throttling limits are in place to prevent API Gateway from being overwhelmed with requests. High latency caused by throttling would manifest as errors and request rejections, not simply slow response times.

Option C, creating a CloudWatch alarm to initiate a Lambda function, is an overly complex solution. It essentially triggers a "dummy" function to try to keep the function warm. While it might help slightly, it's less effective and more resource-intensive than provisioned concurrency. PC is specifically designed to manage cold starts and handle predictable traffic patterns.

Option D, increasing Lambda function memory, can reduce latency in some cases by providing more resources for code execution. However, it doesn't directly address the cold start problem. While increased memory might slightly improve the performance of subsequent invocations, it won't eliminate the initial cold start penalty.

Therefore, provisioned concurrency offers the most direct and efficient solution to mitigate the latency caused by Lambda cold starts at the beginning of each day. Scheduled scaling automates this process, ensuring consistent performance during peak usage.

Supporting documentation:

AWS Lambda Provisioned Concurrency: https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html
Improving Lambda Cold Start Time: https://aws.amazon.com/blogs/compute/operating-lambda-performance-optimization-part-1/

Question: 598 **Exam Heist**

A research company uses on-premises devices to generate data for analysis. The company wants to use the AWS Cloud to analyze the data. The devices generate .csv files and support writing the data to an SMB file share. Company analysts must be able to use SQL commands to query the data. The analysts will run queries periodically throughout the day.

Which combination of steps will meet these requirements MOST cost-effectively? (Choose three.)

A.Deploy an AWS Storage Gateway on premises in Amazon S3 File Gateway mode.

B.Deploy an AWS Storage Gateway on premises in Amazon FSx File Gateway made.

C.Set up an AWS Glue crawler to create a table based on the data that is in Amazon S3.

D.Set up an Amazon EMR cluster with EMR File System (EMRFS) to query the data that is in Amazon S3. Provide access to analysts.

E.Set up an Amazon Redshift cluster to query the data that is in Amazon S3. Provide access to analysts.

F.Setup Amazon Athena to query the data that is in Amazon S3. Provide access to analysts.

Answer: ACF

Explanation:

The correct answer is ACF. Here's a detailed justification:

A. Deploy an AWS Storage Gateway on premises in Amazon S3 File Gateway mode: AWS Storage Gateway simplifies connecting on-premises environments to AWS storage. The S3 File Gateway mode allows the research company's on-premises devices that write .csv files to SMB file shares to seamlessly write data directly to an S3 bucket. This avoids complex data transfer mechanisms and facilitates data ingestion into AWS for analysis. https://aws.amazon.com/storagegateway/file-gateway/

C. Set up an AWS Glue crawler to create a table based on the data that is in Amazon S3: AWS Glue is a fully managed ETL (extract, transform, and load) service. Its crawler feature can automatically discover the schema of the .csv files in the S3 bucket and create a corresponding table in the AWS Glue Data Catalog. This metadata catalog is essential for querying the data using SQL. https://aws.amazon.com/glue/

F. Set up Amazon Athena to query the data that is in Amazon S3. Provide access to analysts: Amazon Athena is a serverless query service that allows analysts to use standard SQL to analyze data directly in S3. Athena uses the AWS Glue Data Catalog to understand the structure of the data. By pointing Athena to the table created by Glue, the analysts can directly query the .csv files without needing to set up or manage any infrastructure. It's also cost-effective since you pay only for the queries you run. https://aws.amazon.com/athena/

Why other options are not ideal or less cost-effective:

B. Deploy an AWS Storage Gateway on premises in Amazon FSx File Gateway mode: While FSx File Gateway allows on-premises access to Amazon FSx file systems, it's typically used when you need a fully-managed, highly performant file system in the cloud for workloads like video editing or large-scale simulations. For simple .csv files, directly using S3 is more cost-effective.

D. Set up an Amazon EMR cluster with EMR File System (EMRFS) to query the data that is in Amazon S3. Provide access to analysts: While EMR can query data in S3, setting up a full EMR cluster is overkill for simply running SQL queries on .csv files. EMR is best suited for complex data processing tasks that require distributed computing, such as data transformation, machine learning, or running Spark jobs.

E. Set up an Amazon Redshift cluster to query the data that is in Amazon S3. Provide access to analysts: Redshift is a data warehouse designed for complex analytical workloads. While Redshift Spectrum can query S3 data, loading the .csv data into Redshift is less cost-effective than using Athena for periodic queries. Redshift requires a persistent cluster, resulting in higher costs compared to Athena's pay-per-query model.

Question: 599 **Exam Heist**

A company wants to use Amazon Elastic Container Service (Amazon ECS) clusters and Amazon RDS DB instances to build and run a payment processing application. The company will run the application in its on-premises data center for compliance purposes.

A solutions architect wants to use AWS Outposts as part of the solution. The solutions architect is working with the company's operational team to build the application.

Which activities are the responsibility of the company's operational team? (Choose three.)

A.Providing resilient power and network connectivity to the Outposts racks

B.Managing the virtualization hypervisor, storage systems, and the AWS services that run on Outposts

C.Physical security and access controls of the data center environment

D.Availability of the Outposts infrastructure including the power supplies, servers, and networking equipment within the Outposts racks

E.Physical maintenance of Outposts components

F.Providing extra capacity for Amazon ECS clusters to mitigate server failures and maintenance events

Answer: ACE

Explanation:

The correct answer is ACE. Here's why:

A. Providing resilient power and network connectivity to the Outposts racks: AWS Outposts extends AWS infrastructure and services to on-premises environments. The customer is responsible for providing the physical environment including power, cooling, and network connectivity. AWS manages the services once they are deployed on Outposts. This is a shared responsibility model. (https://aws.amazon.com/outposts/)

C. Physical security and access controls of the data center environment: Since the Outposts racks are located within the company's data center, the company retains responsibility for the physical security of that environment. This includes access controls, surveillance, and other security measures. AWS is responsible for the security of the Outposts hardware itself.

E. Physical maintenance of Outposts components: While AWS owns and manages the hardware components within the Outposts racks, the physical maintenance to be performed on-site (under AWS guidance and support) is the responsibility of the customer. AWS handles complex repairs and replacements, but the operational team will generally handle the regular physical maintenance.

Here's why the other options are incorrect:

B. Managing the virtualization hypervisor, storage systems, and the AWS services that run on Outposts: AWS manages the virtualization hypervisor and AWS services running on the Outposts. This aligns with AWS's managed service model.

D. Availability of the Outposts infrastructure including the power supplies, servers, and networking equipment within the Outposts racks: While the customer provides the environment, AWS is responsible for the availability of the Outposts infrastructure itself (power supplies, servers, networking hardware). This is part of the AWS service level agreement.

F. Providing extra capacity for Amazon ECS clusters to mitigate server failures and maintenance events: Scaling the Amazon ECS clusters is primarily an application-level activity to maintain application availability, and AWS is not responsible for managing application scaling. The operational team is responsible for setting up Auto Scaling policies for the ECS clusters to ensure capacity during failures, or for scaling the ECS infrastructure.

Question: 600 **Exam Heist**

A company is planning to migrate a TCP-based application into the company's VPC. The application is publicly accessible on a nonstandard TCP port through a hardware appliance in the company's data center. This public endpoint can process up to 3 million requests per second with low latency. The company requires the same level of performance for the new public endpoint in AWS.

What should a solutions architect recommend to meet this requirement?

A.Deploy a Network Load Balancer (NLB). Configure the NLB to be publicly accessible over the TCP port that the application requires.

B.Deploy an Application Load Balancer (ALB). Configure the ALB to be publicly accessible over the TCP port that the application requires.

C.Deploy an Amazon CloudFront distribution that listens on the TCP port that the application requires. Use an Application Load Balancer as the origin.

D.Deploy an Amazon API Gateway API that is configured with the TCP port that the application requires. Configure AWS Lambda functions with provisioned concurrency to process the requests.

Answer: A

Explanation:

The correct answer is **A. Deploy a Network Load Balancer (NLB). Configure the NLB to be publicly accessible over the TCP port that the application requires.**

Here's why:

The primary requirement is to replicate the existing on-premises application's performance in AWS, specifically handling 3 million requests per second with low latency over a non-standard TCP port. NLBs are designed for high-performance, low-latency traffic routing at the transport layer (Layer 4), making them ideal for TCP-based applications. NLBs can handle millions of requests per second while maintaining low latency, aligning perfectly with the stated performance requirement. They also support listening on non-standard ports.

Option B, using an Application Load Balancer (ALB), is not suitable. ALBs operate at the application layer (Layer 7) and are designed for HTTP/HTTPS traffic. While ALBs can handle a significant load, they are not optimized for the raw throughput and low latency needed for a TCP-based application that requires 3 million requests per second. They are more suited for routing based on content, headers, or hostnames.

Option C, using Amazon CloudFront with an ALB, is also inappropriate. CloudFront is a Content Delivery Network (CDN) primarily for caching static and dynamic content closer to users to improve latency for web content. It adds unnecessary complexity and potential latency for a TCP-based application that requires direct, low-latency connections. CloudFront is best suited for content distribution, not as a direct replacement for a load balancer handling TCP traffic volume.

Option D, using Amazon API Gateway with Lambda functions, is unsuitable due to scalability and latency concerns. API Gateway introduces overhead and might not handle 3 million requests per second with the required low latency, especially when integrated with Lambda functions. Lambda functions, even with provisioned concurrency, might not be able to scale rapidly enough and maintain the needed throughput for this volume of requests. API Gateway is also generally optimized for HTTP APIs, not raw TCP connections.

In summary, the NLB provides the necessary performance, protocol support (TCP), and port flexibility to meet the application's requirements in AWS.

Authoritative Links:

AWS Network Load Balancer:
https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html
AWS Application Load Balancer:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html
Amazon CloudFront: https://docs.aws.amazon.com/cloudfront/latest/developerguide/Introduction.html
Amazon API Gateway: https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html

Question: 601 **Exam Heist**

A company runs its critical database on an Amazon RDS for PostgreSQL DB instance. The company wants to migrate to Amazon Aurora PostgreSQL with minimal downtime and data loss.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create a DB snapshot of the RDS for PostgreSQL DB instance to populate a new Aurora PostgreSQL DB cluster.

B.Create an Aurora read replica of the RDS for PostgreSQL DB instance. Promote the Aurora read replicate to a new Aurora PostgreSQL DB cluster.

C.Use data import from Amazon S3 to migrate the database to an Aurora PostgreSQL DB cluster.

D.Use the pg_dump utility to back up the RDS for PostgreSQL database. Restore the backup to a new Aurora PostgreSQL DB cluster.

Answer: B

Explanation:

The best solution to migrate an RDS for PostgreSQL database to Aurora PostgreSQL with minimal downtime and data loss, and with the least operational overhead, is **B. Create an Aurora read replica of the RDS for PostgreSQL DB instance. Promote the Aurora read replica to a new Aurora PostgreSQL DB cluster.**

Here's why:

Minimal Downtime: Creating an Aurora read replica allows continuous replication from the source RDS database. This means that changes are constantly applied to the Aurora replica. Promoting the replica to be the primary instance involves a cutover, which results in much less downtime compared to other migration strategies.
Minimal Data Loss: Because of the continuous replication, data loss is minimized to the replication lag during the final cutover.
Least Operational Overhead: Aurora read replicas provide a straightforward and automated method for migrating data. It avoids the complexities of managing manual backups, restores, or data import/export processes.
Data import from S3 (Option C) and pg_dump (Option D): require more operational overhead compared to Aurora read replica. They also might involve a longer downtime window for backing up, transferring, and restoring data.
DB snapshot (Option A): A DB snapshot is a point-in-time copy of the data, and will not capture any transactions happening on the RDS database after the snapshot is taken.

Aurora Read Replicas are specifically designed to enable migrations with minimal downtime and data loss. Promoting the read replica avoids manual data transfer, reducing the risk of human error and data inconsistencies.

Supporting Links:

Amazon Aurora Migration: https://aws.amazon.com/rds/aurora/migration/
Creating an Amazon Aurora Read Replica:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Aurora.CreateReplica.html

Question: 602 **Exam Heist**

A company's infrastructure consists of hundreds of Amazon EC2 instances that use Amazon Elastic Block Store (Amazon EBS) storage. A solutions architect must ensure that every EC2 instance can be recovered after a disaster.

What should the solutions architect do to meet this requirement with the LEAST amount of effort?

A.Take a snapshot of the EBS storage that is attached to each EC2 instance. Create an AWS CloudFormation template to launch new EC2 instances from the EBS storage.

B.Take a snapshot of the EBS storage that is attached to each EC2 instance. Use AWS Elastic Beanstalk to set the environment based on the EC2 template and attach the EBS storage.

C.Use AWS Backup to set up a backup plan for the entire group of EC2 instances. Use the AWS Backup API or the AWS CLI to speed up the restore process for multiple EC2 instances.

D.Create an AWS Lambda function to take a snapshot of the EBS storage that is attached to each EC2 instance and copy the Amazon Machine Images (AMIs). Create another Lambda function to perform the restores with the copied AMIs and attach the EBS storage.

Answer: C

Explanation:

The correct answer is C, using AWS Backup. Here's why:

AWS Backup is a fully managed backup service designed to centralize and automate the backup of data across AWS services, including EC2 instances and their associated EBS volumes. This means less manual effort compared to managing snapshots individually. AWS Backup enables you to create backup plans that define backup frequency, retention policies, and target backup vaults. You can easily assign these plans to groups of EC2 instances. This solution inherently simplifies the process of ensuring every EC2 instance can be recovered.

Option A, taking individual snapshots and using CloudFormation, requires you to manually manage snapshots for hundreds of instances and create/maintain complex CloudFormation templates. This increases complexity and management overhead. Moreover, CloudFormation focuses on infrastructure as code, not specifically backup and restore.

Option B, using Elastic Beanstalk, is primarily for deploying and managing web applications, not for backing up and restoring EC2 instances. Attaching EBS storage through Beanstalk can be complex and isn't a native backup solution.

Option D, using Lambda to manage snapshots and AMIs, necessitates custom code and adds operational burden. You would need to write, test, and maintain the Lambda functions, increasing complexity compared to using a managed service like AWS Backup. Additionally, AMIs are instance-specific and don't inherently include the data from EBS volumes, thus not guaranteeing data recovery.

AWS Backup, on the other hand, provides a centralized dashboard for monitoring backup jobs and simplifying the restore process. The AWS Backup API or CLI can then be used to further expedite the restoration of multiple instances, thereby minimizing the time required to recover from a disaster, hence fulfilling the "least amount of effort" criterion.

Therefore, AWS Backup provides the most straightforward, scalable, and manageable solution for backing up hundreds of EC2 instances and ensuring their recoverability.

Supporting links:

AWS Backup: https://aws.amazon.com/backup/
AWS Backup Documentation: https://docs.aws.amazon.com/aws-backup/latest/devguide/whatisawsbackup.html

Question: 603 **Exam Heist**

A company recently migrated to the AWS Cloud. The company wants a serverless solution for large-scale parallel on-demand processing of a semistructured dataset. The data consists of logs, media files, sales transactions, and IoT sensor data that is stored in Amazon S3. The company wants the solution to process thousands of items in the dataset in parallel.

Which solution will meet these requirements with the MOST operational efficiency?

A.Use the AWS Step Functions Map state in Inline mode to process the data in parallel.

B.Use the AWS Step Functions Map state in Distributed mode to process the data in parallel.

C.Use AWS Glue to process the data in parallel.

D.Use several AWS Lambda functions to process the data in parallel.

Answer: B

Explanation:

The company requires a serverless, operationally efficient solution for large-scale parallel processing of semistructured data in S3. The ideal solution should handle thousands of items in parallel.

Option A, using Step Functions Map state in Inline mode, is unsuitable for large-scale parallelism. Inline mode has limitations on the number of iterations and payload sizes, making it less efficient for processing thousands of items.

Option B, using Step Functions Map state in Distributed mode, is the most appropriate choice. Distributed mode is specifically designed for high-throughput parallel processing of large datasets. It can handle thousands or even millions of parallel executions, making it suitable for the company's requirements. This mode leverages AWS resources efficiently, providing operational simplicity and scalability.

Option C, using AWS Glue, while capable of processing data in parallel, is primarily designed for ETL (Extract, Transform, Load) operations. It may be overkill for simple processing tasks, and the operational overhead could be higher compared to Step Functions Distributed Map. Glue is a powerful service, but its ETL focus makes it less suitable for the described scenario.

Option D, using multiple Lambda functions directly, requires manual management of parallelism and scaling. This approach would be operationally complex and less efficient compared to using Step Functions Map state in Distributed mode, which automatically manages the parallel execution and resource allocation.

In summary, Step Functions Distributed Map provides the best balance between serverless architecture, operational efficiency, scalability, and the ability to process thousands of items in parallel. It is specifically tailored for this type of high-throughput data processing.

Therefore, option B is the correct answer.

Supporting Links:

AWS Step Functions Map State: https://docs.aws.amazon.com/step-functions/latest/dg/amazon-states-language-map-state.html
Step Functions Distributed Map vs. Inline Map: https://aws.amazon.com/blogs/compute/introducing-distributed-map-in-aws-step-functions/

Question: 604 **Exam Heist**

A company will migrate 10 PB of data to Amazon S3 in 6 weeks. The current data center has a 500 Mbps uplink to the internet. Other on-premises applications share the uplink. The company can use 80% of the internet bandwidth for this one-time migration task.

Which solution will meet these requirements?

A.Configure AWS DataSync to migrate the data to Amazon S3 and to automatically verify the data.

B.Use rsync to transfer the data directly to Amazon S3.

C.Use the AWS CLI and multiple copy processes to send the data directly to Amazon S3.

D.Order multiple AWS Snowball devices. Copy the data to the devices. Send the devices to AWS to copy the data to Amazon S3.

Answer: D

Explanation:

Here's a detailed justification for why option D, using AWS Snowball devices, is the most suitable solution for migrating 10 PB of data to Amazon S3 within 6 weeks, given the limited network bandwidth:

Option D (AWS Snowball) is the best approach because it bypasses the internet bottleneck. With only a 500 Mbps uplink, and only 80% usable, the effective bandwidth is 400 Mbps (50 MBps). Transferring 10 PB (10,000 TB) of data over this connection in 6 weeks is highly impractical. 10,000 TB 1024 GB/TB 1024 MB/GB = 10,485,760,000 MB. To transfer this data in 6 weeks (6 weeks 7 days/week 24 hours/day * 3600 seconds/hour = 3,628,800 seconds), you would need a throughput of 10,485,760,000 MB / 3,628,800 seconds = ~2890 MBps. This vastly exceeds the available 50 MBps.

AWS Snowball allows for offline data transfer. The data is copied to the Snowball devices on-premises, shipped to AWS, and then uploaded to S3. This eliminates dependence on the internet bandwidth, reducing transfer time.

Options A, B, and C rely on the limited internet connection. While AWS DataSync (A) could be used, it's constrained by the 400 Mbps. Rsync (B) and the AWS CLI (C) are also limited by the available bandwidth. Calculating the total transfer time using the internet connection would significantly exceed the 6-week deadline, making these options infeasible. These methods are more suitable for smaller data volumes or when a high-bandwidth connection is available. Snowball provides a faster solution that is unaffected by the uplink limitations. Using Snowball devices allows for parallel transfer which improves the time taken to migrate the data.https://aws.amazon.com/snowball/

Question: 605 **Exam Heist**

A company has several on-premises Internet Small Computer Systems Interface (ISCSI) network storage servers. The company wants to reduce the number of these servers by moving to the AWS Cloud. A solutions architect must provide low-latency access to frequently used data and reduce the dependency on on-premises servers with a minimal number of infrastructure changes.

Which solution will meet these requirements?

A.Deploy an Amazon S3 File Gateway.
B.Deploy Amazon Elastic Block Store (Amazon EBS) storage with backups to Amazon S3.
C.Deploy an AWS Storage Gateway volume gateway that is configured with stored volumes.
D.Deploy an AWS Storage Gateway volume gateway that is configured with cached volumes.

Answer: D

Explanation:

Here's a detailed justification for why option D is the best solution:

The core requirements are low-latency access to frequently used data, reduced dependency on on-premises servers, and minimal infrastructure changes during migration.

Option D, deploying an AWS Storage Gateway volume gateway configured with cached volumes, directly addresses these requirements. A cached volume gateway stores the entire dataset in Amazon S3, while frequently accessed data is cached locally on the on-premises servers. This provides low-latency access to frequently used data, as it's readily available locally. The bulk of the data resides in S3, fulfilling the requirement of reducing dependency on on-premises servers and decreasing the needed storage on the local

ISCSI infrastructure.

Option A, deploying an Amazon S3 File Gateway, is less suitable because it's primarily designed for file-based access, not block-level access required by ISCSI. Converting ISCSI volumes to files and then using S3 File Gateway would involve significant infrastructure changes.

Option B, deploying Amazon EBS storage with backups to Amazon S3, requires migrating data entirely to AWS. This is a significant infrastructure change and doesn't provide low-latency access to on-premises users without completely migrating the applications.

Option C, deploying an AWS Storage Gateway volume gateway configured with stored volumes, stores the entire dataset locally on the on-premises servers. While this reduces dependency on the original servers, it does not provide low-latency local access to the application server because it would need to access the storage via Storage Gateway.

Therefore, the cached volumes configuration with AWS Storage Gateway strikes the best balance between low latency, reduced on-premises dependency, and minimal infrastructure changes by leveraging the on-premises servers to cache frequently accessed data.

For further research, you can refer to these authoritative links:

AWS Storage Gateway Documentation: https://aws.amazon.com/storagegateway/
Cached Volumes: https://docs.aws.amazon.com/storagegateway/latest/userguide/cached-volumes.html

Question: 606 **Exam Heist**

A solutions architect is designing an application that will allow business users to upload objects to Amazon S3. The solution needs to maximize object durability. Objects also must be readily available at any time and for any length of time. Users will access objects frequently within the first 30 days after the objects are uploaded, but users are much less likely to access objects that are older than 30 days.

Which solution meets these requirements MOST cost-effectively?

A.Store all the objects in S3 Standard with an S3 Lifecycle rule to transition the objects to S3 Glacier after 30 days.

B.Store all the objects in S3 Standard with an S3 Lifecycle rule to transition the objects to S3 Standard-Infrequent Access (S3 Standard-IA) after 30 days.

C.Store all the objects in S3 Standard with an S3 Lifecycle rule to transition the objects to S3 One Zone-Infrequent Access (S3 One Zone-IA) after 30 days.

D.Store all the objects in S3 Intelligent-Tiering with an S3 Lifecycle rule to transition the objects to S3 Standard-Infrequent Access (S3 Standard-IA) after 30 days.

Answer: B

Explanation:

The correct answer is **B. Store all the objects in S3 Standard with an S3 Lifecycle rule to transition the objects to S3 Standard-Infrequent Access (S3 Standard-IA) after 30 days.**

Here's a detailed justification:

Durability: S3 Standard provides 99.999999999% (11 9's) of data durability because it stores data redundantly across multiple devices in multiple Availability Zones (AZs). S3 Standard-IA also offers this high level of durability as it is designed for infrequently accessed data but still needs the same resilience as S3 Standard. This meets the requirement for maximizing object durability.
Availability: S3 Standard offers high availability (99.99%). S3 Standard-IA also offers high availability (99.9%) suitable for data that needs to be accessed when needed, though with a slightly lower availability

guarantee than standard.
Accessibility: Both S3 Standard and S3 Standard-IA provide immediate accessibility. This is important because the objects must be "readily available at any time."
Cost-Effectiveness: Because most accesses occur within the first 30 days, storing the data initially in S3 Standard makes sense because it is optimized for frequent access. After 30 days, transitioning to S3 Standard-IA reduces storage costs for the less frequently accessed data.
Lifecycle Policies: S3 Lifecycle policies allow you to automatically transition objects between storage classes based on predefined rules. This automation simplifies storage management and reduces costs.

Why the other options are less suitable:

A (S3 Glacier): S3 Glacier is designed for long-term archival, not for data that needs to be readily available. Retrievals from Glacier can take several hours, which violates the "readily available at any time" requirement.
C (S3 One Zone-IA): S3 One Zone-IA stores data in only one Availability Zone. While cost-effective, it compromises durability. If the AZ is destroyed, data could be lost. This violates the requirement to maximize object durability.
D (S3 Intelligent-Tiering transitioning to S3 Standard-IA): While S3 Intelligent-Tiering automatically moves data between frequent and infrequent access tiers based on usage patterns, using a lifecycle rule to transition to S3 Standard-IA after 30 days provides a more predictable cost optimization for the specific access pattern described (frequent access for 30 days, then infrequent access). Intelligent-Tiering is more beneficial when access patterns are unknown or vary significantly. Using both Intelligent Tiering with a lifecycle rule transitioning to Standard-IA might introduce unnecessary complexity and potentially higher costs if the 30-day access pattern holds true.

Therefore, storing the objects in S3 Standard initially and then transitioning to S3 Standard-IA after 30 days using a lifecycle rule strikes the best balance between durability, availability, accessibility, and cost-effectiveness for the given requirements.

Authoritative Links for Further Research:

Amazon S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
Amazon S3 Lifecycle: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-examples.html

Question: 607 **Exam Heist**

A company has migrated a two-tier application from its on-premises data center to the AWS Cloud. The data tier is a Multi-AZ deployment of Amazon RDS for Oracle with 12 TB of General Purpose SSD Amazon Elastic Block Store (Amazon EBS) storage. The application is designed to process and store documents in the database as binary large objects (blobs) with an average document size of 6 MB.

The database size has grown over time, reducing the performance and increasing the cost of storage. The company must improve the database performance and needs a solution that is highly available and resilient.

Which solution will meet these requirements MOST cost-effectively?

A.Reduce the RDS DB instance size. Increase the storage capacity to 24 TiB. Change the storage type to Magnetic.

B.Increase the RDS DB instance size. Increase the storage capacity to 24 TiChange the storage type to Provisioned IOPS.

C.Create an Amazon S3 bucket. Update the application to store documents in the S3 bucket. Store the object metadata in the existing database.

D.Create an Amazon DynamoDB table. Update the application to use DynamoDB. Use AWS Database Migration Service (AWS DMS) to migrate data from the Oracle database to DynamoDB.

Answer: C

Explanation:

The most cost-effective and performant solution is to offload the BLOB storage to Amazon S3 (Option C). Here's why:

Cost: EBS storage, especially large volumes and Provisioned IOPS, is significantly more expensive than S3. S3 offers cost-effective object storage, optimized for storing large binary files like documents.
Performance: Storing BLOBs directly in the database bloats its size, impacting query performance and overall database responsiveness. Moving BLOBs to S3 reduces database size and improves performance by allowing it to focus on structured data.
Scalability and Durability: S3 provides virtually unlimited storage and high durability (11 9's), ensuring data availability and resilience.
Database Optimization: The database can then focus on storing metadata (e.g., document name, S3 object key) for each document, which is much smaller and faster to query.
Other Options' Drawbacks:

Options A and B suggest increasing EBS storage, which is counterproductive given the cost and performance issues associated with storing BLOBs in the database. Magnetic storage (Option A) is also unsuitable for performance-sensitive applications. DynamoDB (Option D) is a NoSQL database, which would require a significant application rewrite and data migration effort, adding complexity and cost, and is likely not necessary for storing document data, whose metadata can easily be held within the relational RDS DB.

In summary, moving the BLOBs to S3 frees up valuable database resources, lowers storage costs, improves application performance, and leverages S3's scalability and durability features, while storing metadata within the existing database maintains the integrity and structure of the data.

Authoritative Links:

Amazon S3: https://aws.amazon.com/s3/
Amazon RDS: https://aws.amazon.com/rds/
EBS Storage Types: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html
Amazon DynamoDB: https://aws.amazon.com/dynamodb/

Question: 608 **Exam Heist**

A company has an application that serves clients that are deployed in more than 20.000 retail storefront locations around the world. The application consists of backend web services that are exposed over HTTPS on port 443. The application is hosted on Amazon EC2 instances behind an Application Load Balancer (ALB). The retail locations communicate with the web application over the public internet. The company allows each retail location to register the IP address that the retail location has been allocated by its local ISP.

The company's security team recommends to increase the security of the application endpoint by restricting access to only the IP addresses registered by the retail locations.

What should a solutions architect do to meet these requirements?

A.Associate an AWS WAF web ACL with the ALB. Use IP rule sets on the ALB to filter traffic. Update the IP addresses in the rule to include the registered IP addresses.

B.Deploy AWS Firewall Manager to manage the ALConfigure firewall rules to restrict traffic to the ALModify the firewall rules to include the registered IP addresses.

C.Store the IP addresses in an Amazon DynamoDB table. Configure an AWS Lambda authorization function on the ALB to validate that incoming requests are from the registered IP addresses.

D.Configure the network ACL on the subnet that contains the public interface of the ALB. Update the ingress rules on the network ACL with entries for each of the registered IP addresses.

Answer: A

Explanation:

The correct answer is A because it leverages AWS WAF, a service specifically designed for protecting web applications from common web exploits. AWS WAF allows you to create rules, called web access control lists (web ACLs), to control which traffic is allowed or blocked based on criteria like IP addresses, HTTP headers, HTTP body, URI strings, SQL injection, and cross-site scripting. Associating a web ACL with the ALB enables filtering traffic based on registered IP addresses. Implementing IP rule sets within AWS WAF is a direct and scalable method to allow only traffic originating from the registered retail location IP addresses.

Option B is incorrect because AWS Firewall Manager is designed to centrally manage firewall rules across multiple AWS accounts and resources, not to enforce IP address restrictions at the application level. While Firewall Manager can leverage WAF, the core functionality needed here is within WAF itself.

Option C is less optimal. While Lambda authorization can achieve the desired outcome, it introduces unnecessary complexity and potential latency. Every request would trigger a Lambda function invocation, which can impact application performance and increase operational overhead. WAF offers a more efficient and purpose-built solution for IP address filtering.

Option D is not recommended. Network ACLs operate at the subnet level and are stateless, meaning ingress and egress rules must be configured separately. Managing 20,000+ IP addresses in network ACL rules is cumbersome, error-prone, and can impact network performance due to the sheer number of rules. Furthermore, network ACLs have a limited number of rules per list, making this option infeasible. WAF provides a much more manageable and scalable solution for filtering traffic at the application layer.

Therefore, using AWS WAF with IP rule sets offers the most efficient, scalable, and secure method to restrict access to the application endpoint based on the registered IP addresses of the retail locations.

Relevant links for further research:

AWS WAF: https://aws.amazon.com/waf/
AWS Firewall Manager: https://aws.amazon.com/firewall-manager/
Amazon DynamoDB: https://aws.amazon.com/dynamodb/
AWS Lambda: https://aws.amazon.com/lambda/
Network ACLs: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html

Question: 609 **Exam Heist**

A company is building a data analysis platform on AWS by using AWS Lake Formation. The platform will ingest data from different sources such as Amazon S3 and Amazon RDS. The company needs a secure solution to prevent access to portions of the data that contain sensitive information.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create an IAM role that includes permissions to access Lake Formation tables.

B.Create data filters to implement row-level security and cell-level security.

C.Create an AWS Lambda function that removes sensitive information before Lake Formation ingests the data.

D.Create an AWS Lambda function that periodically queries and removes sensitive information from Lake Formation tables.

Answer: B

Explanation:

The correct answer is B: Create data filters to implement row-level security and cell-level security.

Here's why: AWS Lake Formation allows you to define fine-grained access control policies directly on your data lake. Data filters provide a mechanism to implement row-level and cell-level security without requiring custom code or complex infrastructure.

Option B leverages Lake Formation's built-in capabilities for security. Row-level security limits access to specific rows based on defined criteria (e.g., only allowing certain users to see records for their region). Cell-level security restricts access to specific columns or attributes, masking sensitive data elements within a row. Both are natively supported through Lake Formation's data filter feature. This means you don't have to build and maintain additional ETL pipelines or custom access control mechanisms.

Option A, creating an IAM role with permissions to access Lake Formation tables, provides a basic level of access control but doesn't offer the granularity needed to prevent access to specific rows or cells containing sensitive information. It's a necessary component for Lake Formation overall, but insufficient for the specific security requirements of the question.

Option C and D involve creating AWS Lambda functions to remove or modify sensitive data. These options introduce significant operational overhead. Option C modifies the data before it's even stored, potentially impacting downstream analysis. Option D requires periodic processing and introduces complexity in managing and scheduling these Lambda functions. They also require the creation and maintenance of custom code, increasing the possibility of errors and vulnerabilities. These options add unnecessary complexity because Lake Formation provides a native solution.

Therefore, using Lake Formation's data filters for row-level and cell-level security is the most efficient and secure method for restricting access to sensitive data within the data lake, minimizing operational overhead.

For further research, refer to the AWS documentation on Lake Formation data filtering:

AWS Lake Formation Data Filtering: https://docs.aws.amazon.com/lake-formation/latest/dg/data-filtering.html
Lake Formation Fine-Grained Access Control: https://aws.amazon.com/blogs/big-data/fine-grained-access-control-using-aws-lake-formation/

Question: 610 **Exam Heist**

A company deploys Amazon EC2 instances that run in a VPC. The EC2 instances load source data into Amazon S3 buckets so that the data can be processed in the future. According to compliance laws, the data must not be transmitted over the public internet. Servers in the company's on-premises data center will consume the output from an application that runs on the EC2 instances.

Which solution will meet these requirements?

A.Deploy an interface VPC endpoint for Amazon EC2. Create an AWS Site-to-Site VPN connection between the company and the VPC.

B.Deploy a gateway VPC endpoint for Amazon S3. Set up an AWS Direct Connect connection between the on-premises network and the VPC.

C.Set up an AWS Transit Gateway connection from the VPC to the S3 buckets. Create an AWS Site-to-Site VPN connection between the company and the VPC.

D.Set up proxy EC2 instances that have routes to NAT gateways. Configure the proxy EC2 instances to fetch S3 data and feed the application instances.

Answer: B

Explanation:

The correct answer is B because it provides a solution that ensures data transfer between EC2 instances and S3 buckets, and between the VPC and the on-premises network, without traversing the public internet.

A gateway VPC endpoint for S3 creates a direct, private connection to S3 from within the VPC. It avoids the need to route traffic through the internet gateway, NAT gateway, or VPC peering connections to access S3. This directly addresses the compliance requirement of avoiding public internet transmission for S3 data access. Gateway endpoints are designed specifically for S3 and DynamoDB.

An AWS Direct Connect connection establishes a private network connection between the company's on-premises data center and the VPC. This allows the servers in the on-premises data center to securely consume the output from the EC2 instances without sending data over the internet. Direct Connect offers consistent, low-latency connectivity compared to internet-based connections.

Option A is incorrect because interface VPC endpoints are used to privately connect to AWS services other than S3 and DynamoDB. EC2 itself doesn't require a VPC endpoint for its normal functioning within a VPC. An AWS Site-to-Site VPN connection addresses the on-premises connectivity but not the S3 traffic routing requirements.

Option C is incorrect because Transit Gateway is used to connect multiple VPCs and on-premises networks, but does not provide a private connection to S3. While a VPN addresses on-premises connectivity, it doesn't isolate S3 traffic from the internet.

Option D is overly complex and inefficient. Using proxy EC2 instances with NAT gateways still involves traffic potentially traversing public internet infrastructure at some point. It also adds unnecessary overhead and cost to the architecture. Using gateway VPC endpoints is the recommended, more performant and cost-effective approach for private S3 access.

Relevant Links:

AWS Direct Connect: https://aws.amazon.com/directconnect/
VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html

Question: 611 **Exam Heist**

A company has an application with a REST-based interface that allows data to be received in near-real time from a third-party vendor. Once received, the application processes and stores the data for further analysis. The application is running on Amazon EC2 instances.

The third-party vendor has received many 503 Service Unavailable Errors when sending data to the application. When the data volume spikes, the compute capacity reaches its maximum limit and the application is unable to process all requests.

Which design should a solutions architect recommend to provide a more scalable solution?

A.Use Amazon Kinesis Data Streams to ingest the data. Process the data using AWS Lambda functions.

B.Use Amazon API Gateway on top of the existing application. Create a usage plan with a quota limit for the third-party vendor.

C.Use Amazon Simple Notification Service (Amazon SNS) to ingest the data. Put the EC2 instances in an Auto Scaling group behind an Application Load Balancer.

D.Repackage the application as a container. Deploy the application using Amazon Elastic Container Service (Amazon ECS) using the EC2 launch type with an Auto Scaling group.

Answer: A

Explanation:

The recommended solution is **A: Use Amazon Kinesis Data Streams to ingest the data. Process the data using AWS Lambda functions.**

Here's why:

Scalability and Decoupling: Kinesis Data Streams provides a highly scalable and durable data ingestion service. It can handle large volumes of data in real-time and decouples the data source (third-party vendor) from the processing application. This decoupling is crucial because it allows the application to process data at its own pace, preventing overload. https://aws.amazon.com/kinesis/data-streams/
Elastic Processing: AWS Lambda functions offer a serverless compute environment that can automatically scale in response to the incoming data stream from Kinesis. Each Lambda function can process individual records from the stream, enabling parallel processing and efficient utilization of resources. This eliminates the bottleneck caused by the EC2 instances reaching their maximum capacity.
https://aws.amazon.com/lambda/
Real-time Processing: Kinesis Data Streams and Lambda enable near-real-time processing of data, meeting the application's requirements.
Cost-Effectiveness: Lambda's pay-per-execution model can be more cost-effective than running EC2 instances continuously, especially during periods of low data volume.

Here's why the other options are less suitable:

B: Amazon API Gateway with usage plans: While API Gateway can provide rate limiting, it doesn't address the underlying issue of limited compute capacity on the EC2 instances. It might reduce the frequency of 503 errors but won't enable the application to handle high data volumes.
C: Amazon SNS and Auto Scaling group behind an Application Load Balancer: SNS is a publish/subscribe messaging service and not ideal for ingesting a continuous data stream. While Auto Scaling would help with horizontal scaling of EC2 instances, it doesn't offer the same level of scalability and elasticity as Kinesis and Lambda. Additionally, managing the Auto Scaling group adds operational overhead. SNS has a maximum message size limitation as well, which makes it less suitable for large data payloads.
https://docs.aws.amazon.com/sns/latest/dg/sns-limits.html
D: Amazon ECS with Auto Scaling: ECS provides container orchestration, which can improve resource utilization and deployment. However, it still relies on underlying compute resources (EC2 instances), and the scaling may not be as responsive or cost-effective as Lambda for handling unpredictable data spikes. It requires more operational overhead for maintaining the ECS cluster.

In summary, Kinesis Data Streams and Lambda provide a scalable, cost-effective, and near-real-time solution for ingesting and processing data from the third-party vendor, addressing the issue of 503 errors and compute capacity limitations.

Question: 612 **Exam Heist**

A company has an application that runs on Amazon EC2 instances in a private subnet. The application needs to process sensitive information from an Amazon S3 bucket. The application must not use the internet to connect to the S3 bucket.

Which solution will meet these requirements?

A.Configure an internet gateway. Update the S3 bucket policy to allow access from the internet gateway. Update the application to use the new internet gateway.

B.Configure a VPN connection. Update the S3 bucket policy to allow access from the VPN connection. Update the application to use the new VPN connection.

C.Configure a NAT gateway. Update the S3 bucket policy to allow access from the NAT gateway. Update the application to use the new NAT gateway.

D.Configure a VPC endpoint. Update the S3 bucket policy to allow access from the VPC endpoint. Update the application to use the new VPC endpoint.

Answer: D

Explanation:

The correct answer is D: Configure a VPC endpoint. Update the S3 bucket policy to allow access from the VPC endpoint. Update the application to use the new VPC endpoint.

Here's why:

VPC endpoints provide private connectivity between your VPC and supported AWS services, including S3, without requiring an internet gateway, NAT device, VPN connection, or AWS Direct Connect connection. This means traffic between your EC2 instances in the private subnet and the S3 bucket stays within the AWS network, enhancing security and reducing latency.

Option A is incorrect because an internet gateway provides access to the internet, which violates the requirement of not using the internet to connect to S3.

Option B is incorrect because while a VPN connection does provide a secure connection, it is primarily used to connect your on-premises network to your VPC. It's an unnecessary complexity for internal VPC communication.

Option C is incorrect because a NAT gateway allows instances in a private subnet to connect to the internet (e.g., for software updates), but it doesn't provide private connectivity to S3. It also utilizes the internet, which again goes against the problem's requirements.

Option D, by using a VPC endpoint, specifically a Gateway Endpoint for S3, provides a secure, private connection between the EC2 instances in the private subnet and the S3 bucket. The S3 bucket policy can be updated to only allow access from that specific VPC endpoint, further enhancing security by restricting access to only the EC2 instances. The application is configured to use the VPC endpoint, ensuring traffic flows privately within AWS.

Gateway endpoints are a cost-effective solution when compared to VPNs and do not introduce extra hops like NAT gateways. They are specifically designed for this kind of direct S3 access.

For further reading:

AWS VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html
Gateway VPC Endpoints for S3: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-s3.html

Question: 613 **Exam Heist**

A company uses Amazon Elastic Kubernetes Service (Amazon EKS) to run a container application. The EKS cluster stores sensitive information in the Kubernetes secrets object. The company wants to ensure that the information is encrypted.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use the container application to encrypt the information by using AWS Key Management Service (AWS KMS).

B.Enable secrets encryption in the EKS cluster by using AWS Key Management Service (AWS KMS).

C.Implement an AWS Lambda function to encrypt the information by using AWS Key Management Service (AWS KMS).

D.Use AWS Systems Manager Parameter Store to encrypt the information by using AWS Key Management Service (AWS KMS).

Answer: B

Explanation:

The best solution is **B. Enable secrets encryption in the EKS cluster by using AWS Key Management Service (AWS KMS).**

Here's why:

Native Integration: EKS natively supports secrets encryption at rest using KMS. This is the most straightforward and least operationally heavy approach because it leverages built-in functionality. No custom code or external services are required.
Centralized Key Management: KMS provides a centralized and secure way to manage the encryption keys. AWS handles the key lifecycle, rotation, and access control.
Ease of Implementation: Enabling secrets encryption in EKS involves configuring the control plane to use a KMS key. Once enabled, all newly created or updated secrets are automatically encrypted.
Reduced Development Effort: Option A requires the container application to handle encryption, increasing the complexity of the application code and potentially exposing the key within the application. Option C introduces a Lambda function which creates an additional component to manage and monitor.
Secrets Management: While Systems Manager Parameter Store (Option D) can store sensitive information, it's not designed to replace Kubernetes secrets for application deployment within an EKS cluster. Using Parameter Store would require additional logic within the application to fetch and manage the secrets.

Therefore, enabling secrets encryption in EKS using KMS directly addresses the requirement with the least operational overhead by utilizing native functionality and centralized key management.

Authoritative Links:

Encrypting Secrets at Rest
AWS Key Management Service (KMS)

Question: 614 **Exam Heist**

A company is designing a new multi-tier web application that consists of the following components:

•Web and application servers that run on Amazon EC2 instances as part of Auto Scaling groups
•An Amazon RDS DB instance for data storage

A solutions architect needs to limit access to the application servers so that only the web servers can access them.

Which solution will meet these requirements?

A.Deploy AWS PrivateLink in front of the application servers. Configure the network ACL to allow only the web servers to access the application servers.

B.Deploy a VPC endpoint in front of the application servers. Configure the security group to allow only the web servers to access the application servers.

C.Deploy a Network Load Balancer with a target group that contains the application servers' Auto Scaling group. Configure the network ACL to allow only the web servers to access the application servers.

D.Deploy an Application Load Balancer with a target group that contains the application servers' Auto Scaling group. Configure the security group to allow only the web servers to access the application servers.

Answer: D

Explanation:

The correct solution is D: Deploy an Application Load Balancer (ALB) with a target group containing the application servers' Auto Scaling group, and configure the security group to allow only the web servers to access the application servers. Here's why:

ALB for Application Tier Traffic: ALBs are designed for HTTP/HTTPS traffic, typical for communication between web servers and application servers in a multi-tier architecture. They provide advanced routing capabilities based on content, hostnames, or other request attributes. This makes them suitable for directing traffic from the web tier to the application tier.

Target Group for Dynamic Scaling: Using a target group linked to the application servers' Auto Scaling group allows the ALB to automatically adapt as the number of application servers scales up or down. The ALB dynamically registers and deregisters instances as they are launched or terminated by Auto Scaling, ensuring high availability and scalability.

Security Group for Fine-Grained Access Control: Security groups act as virtual firewalls at the instance level. By configuring the security group of the application servers to allow traffic only from the web servers' security group, we can effectively limit access to the application tier. This ensures that only authorized web servers can communicate with the application servers, enhancing security and preventing unauthorized access.

Why not A: AWS PrivateLink is used to privately connect your VPC to supported AWS services, services hosted by other AWS accounts (called endpoint services), and supported AWS Marketplace partner services. It's overkill for internal communication between tiers within the same VPC. Using network ACLs for this level of granularity can become complex and harder to manage than security groups.

Why not B: VPC endpoints are used to connect to AWS services privately without traversing the public internet. Similar to PrivateLink, it's designed for external AWS service communication, not internal tier communication. Security groups are again, more effective for instance-level access control in this scenario.

Why not C: While Network Load Balancers (NLBs) provide high throughput and low latency, they operate at the transport layer (TCP/UDP). They lack the application-level awareness of ALBs and are less suited for HTTP/HTTPS traffic routing based on request content. Also, security groups are easier to manage for this level of security than network ACLs.

In summary, using an ALB for the application tier along with a well-defined security group rule achieves the goal of restricting access to the application servers specifically to the web servers in a scalable and manageable way.

Authoritative Links:

Application Load Balancer
Security Groups
Target Groups for your Application Load Balancers

Question: 615 **Exam Heist**

A company runs a critical, customer-facing application on Amazon Elastic Kubernetes Service (Amazon EKS). The application has a microservices architecture. The company needs to implement a solution that collects, aggregates, and summarizes metrics and logs from the application in a centralized location.

Which solution meets these requirements?

A.Run the Amazon CloudWatch agent in the existing EKS cluster. View the metrics and logs in the CloudWatch console.

B.Run AWS App Mesh in the existing EKS cluster. View the metrics and logs in the App Mesh console.

C.Configure AWS CloudTrail to capture data events. Query CloudTrail by using Amazon OpenSearch Service.

D.Configure Amazon CloudWatch Container Insights in the existing EKS cluster. View the metrics and logs in the CloudWatch console.

Answer: D

Explanation:

The correct answer is D because it directly addresses the need for centralized metrics and log aggregation from an EKS cluster running a microservices application. Amazon CloudWatch Container Insights is

specifically designed for monitoring containerized applications, including those running on EKS. It automatically discovers container environments and collects performance data such as CPU, memory, network, and disk utilization, as well as application logs. These metrics and logs are then readily available in the CloudWatch console, providing a single pane of glass for monitoring the entire application.

Option A is partially correct, as the CloudWatch agent can collect logs and some metrics. However, it requires more manual configuration and doesn't offer the same level of pre-built insights and dashboards tailored for container environments as Container Insights.

Option B, AWS App Mesh, is a service mesh that focuses on managing and securing communication between microservices. While App Mesh provides metrics related to service-to-service communication (e.g., latency, request rates, error rates), it doesn't directly address the broader requirement of collecting all metrics and logs from the entire application across the EKS cluster. Additionally, viewing logs and metrics primarily focuses on inter-service communication, not the entire cluster.

Option C, using AWS CloudTrail, is designed for auditing API calls made to AWS services. While CloudTrail captures data events, it doesn't collect the granular application metrics and logs needed for performance monitoring within a Kubernetes cluster. Also, querying CloudTrail logs in OpenSearch is not a suitable replacement for metrics dashboards.

Therefore, CloudWatch Container Insights provides the most comprehensive and efficient solution for collecting, aggregating, and summarizing metrics and logs from an EKS application in a centralized location. It is designed to automatically discover and monitor containerized environments.

References:

Amazon CloudWatch Container Insights
Monitoring Amazon EKS with CloudWatch Container Insights

Question: 616 **Exam Heist**

A company has deployed its newest product on AWS. The product runs in an Auto Scaling group behind a Network Load Balancer. The company stores the product's objects in an Amazon S3 bucket.

The company recently experienced malicious attacks against its systems. The company needs a solution that continuously monitors for malicious activity in the AWS account, workloads, and access patterns to the S3 bucket. The solution must also report suspicious activity and display the information on a dashboard.

Which solution will meet these requirements?

A.Configure Amazon Macie to monitor and report findings to AWS Config.
B.Configure Amazon Inspector to monitor and report findings to AWS CloudTrail.
C.Configure Amazon GuardDuty to monitor and report findings to AWS Security Hub.
D.Configure AWS Config to monitor and report findings to Amazon EventBridge.

Answer: C

Explanation:

The correct solution is C: Configure Amazon GuardDuty to monitor and report findings to AWS Security Hub.

Here's why:

Amazon GuardDuty is a threat detection service that continuously monitors for malicious activity and unauthorized behavior to protect your AWS accounts, workloads, and data stored in Amazon S3. It analyzes VPC Flow Logs, CloudTrail logs, and DNS logs to identify threats. This addresses the requirement of

continuously monitoring for malicious activity within the AWS account, workloads, and S3 bucket access patterns.

AWS Security Hub is a security service that provides a comprehensive view of your security state in AWS. It collects security findings from various AWS security services like GuardDuty, Inspector, and Macie, as well as from integrated third-party partner products. It then correlates and prioritizes these findings to help you identify the most important security issues. Security Hub provides a central dashboard to view and manage these findings. This satisfies the need to report suspicious activity and display it on a dashboard.

Option A is incorrect because while **Amazon Macie** can identify sensitive data stored in S3, it doesn't provide comprehensive threat detection across the entire AWS environment like GuardDuty. Also, AWS Config is primarily a configuration management service and doesn't provide threat detection capabilities.

Option B is incorrect because **Amazon Inspector** focuses on identifying software vulnerabilities and unintended network exposure within EC2 instances and container images. While valuable, it doesn't monitor access patterns to S3 buckets or provide a central dashboard. CloudTrail logs API calls, but is not designed to automatically generate security findings or provide a dashboard like Security Hub.

Option D is incorrect because **AWS Config** monitors the configuration of your AWS resources, and **Amazon EventBridge** is an event bus service. While Config can detect configuration changes that might indicate a security issue, it doesn't have built-in threat detection capabilities. EventBridge can trigger actions based on events, but it doesn't provide the threat intelligence needed for identifying malicious activity.

In summary, GuardDuty provides the threat detection and monitoring capabilities, and Security Hub provides the central dashboard for reporting and managing security findings, making option C the best solution.

Relevant Documentation:

Amazon GuardDuty: https://aws.amazon.com/guardduty/
AWS Security Hub: https://aws.amazon.com/security-hub/

Question: 617 **Exam Heist**

A company wants to migrate an on-premises data center to AWS. The data center hosts a storage server that stores data in an NFS-based file system. The storage server holds 200 GB of data. The company needs to migrate the data without interruption to existing services. Multiple resources in AWS must be able to access the data by using the NFS protocol.

Which combination of steps will meet these requirements MOST cost-effectively? (Choose two.)

A.Create an Amazon FSx for Lustre file system.

B.Create an Amazon Elastic File System (Amazon EFS) file system.

C.Create an Amazon S3 bucket to receive the data.

D.Manually use an operating system copy command to push the data into the AWS destination.

E.Install an AWS DataSync agent in the on-premises data center. Use a DataSync task between the on-premises location and AWS.

Answer: BE

Explanation:

The correct answer is BE. Here's why:

B. Create an Amazon Elastic File System (Amazon EFS) file system: Amazon EFS is a fully managed, scalable, elastic, and NFS file system. It's designed for use with AWS Cloud services and on-premises resources. Given the requirement that multiple AWS resources need to access the data via the NFS protocol,

EFS is a natural fit. It supports concurrent access from multiple EC2 instances, containers, and serverless functions.

E. Install an AWS DataSync agent in the on-premises data center. Use a DataSync task between the on-premises location and AWS: AWS DataSync is a data transfer service that simplifies, automates, and accelerates moving data between on-premises storage and AWS storage services. It's designed to handle active datasets and provides features such as incremental transfers and encryption. Deploying a DataSync agent on-premises allows for efficient and secure data transfer from the existing NFS server to the EFS file system in AWS, without interrupting existing services because DataSync handles the data movement in the background.

Here's why the other options are not the best choice:

A. Create an Amazon FSx for Lustre file system: While FSx for Lustre is a high-performance file system, it's typically used for compute-intensive workloads such as machine learning, high-performance computing (HPC), and video processing. It's not the most cost-effective option for general-purpose file storage and sharing via NFS.

C. Create an Amazon S3 bucket to receive the data: S3 is object storage, not a file system. While data could technically be moved to S3, it wouldn't natively support the NFS protocol requirement. Directly copying the data into S3 would mean a change in accessing protocol to S3 API calls, which is not in line with original requirement.

D. Manually use an operating system copy command to push the data into the AWS destination: This is not practical or efficient for a 200GB dataset, especially with the requirement for zero downtime. It lacks built-in features for data integrity, error handling, and automated synchronization.

Authoritative Links:

Amazon EFS: https://aws.amazon.com/efs/
AWS DataSync: https://aws.amazon.com/datasync/

Question: 618 **Exam Heist**

A company wants to use Amazon FSx for Windows File Server for its Amazon EC2 instances that have an SMB file share mounted as a volume in the us-east-1 Region. The company has a recovery point objective (RPO) of 5 minutes for planned system maintenance or unplanned service disruptions. The company needs to replicate the file system to the us-west-2 Region. The replicated data must not be deleted by any user for 5 years.

Which solution will meet these requirements?

A.Create an FSx for Windows File Server file system in us-east-1 that has a Single-AZ 2 deployment type. Use AWS Backup to create a daily backup plan that includes a backup rule that copies the backup to us-west-2. Configure AWS Backup Vault Lock in compliance mode for a target vault in us-west-2. Configure a minimum duration of 5 years.

B.Create an FSx for Windows File Server file system in us-east-1 that has a Multi-AZ deployment type. Use AWS Backup to create a daily backup plan that includes a backup rule that copies the backup to us-west-2. Configure AWS Backup Vault Lock in governance mode for a target vault in us-west-2. Configure a minimum duration of 5 years.

C.Create an FSx for Windows File Server file system in us-east-1 that has a Multi-AZ deployment type. Use AWS Backup to create a daily backup plan that includes a backup rule that copies the backup to us-west-2. Configure AWS Backup Vault Lock in compliance mode for a target vault in us-west-2. Configure a minimum duration of 5 years.

D.Create an FSx for Windows File Server file system in us-east-1 that has a Single-AZ 2 deployment type. Use AWS Backup to create a daily backup plan that includes a backup rule that copies the backup to us-west-2. Configure AWS Backup Vault Lock in governance mode for a target vault in us-west-2. Configure a minimum duration of 5 years.

Answer: C

Explanation:

The correct solution is option C because it addresses the RPO, cross-region replication, and data retention requirements. Here's a breakdown:

1. **Multi-AZ Deployment:** A Multi-AZ deployment type is essential for meeting the 5-minute RPO requirement. Multi-AZ provides high availability, automatically failing over to a standby file server in case of a failure in the primary Availability Zone. Single-AZ deployments do not offer this level of fault tolerance. https://docs.aws.amazon.com/fsx/latest/WindowsGuide/high-availability-multi-AZ.html
2. **AWS Backup for Cross-Region Replication:** AWS Backup is used to create a daily backup of the file system in us-east-1 and copy it to us-west-2. This meets the requirement of replicating the file system to another region for disaster recovery purposes. AWS Backup enables scheduled backups and retention policies, vital for data protection and compliance. https://aws.amazon.com/backup/
3. **AWS Backup Vault Lock in Compliance Mode:** To ensure the replicated data cannot be deleted by any user for 5 years, AWS Backup Vault Lock in compliance mode is used. Compliance mode prevents any modifications to the retention policy once it's configured, ensuring immutability. Governance mode allows some modifications under specific conditions, which doesn't meet the strict requirement of no deletion. https://docs.aws.amazon.com/aws-backup/latest/devguide/vault-lock.html
4. **Retention Period:** Configuring a minimum duration of 5 years in the Vault Lock ensures that the backups cannot be deleted before the specified period, satisfying the data retention policy.

Options A and D are incorrect because they use Single-AZ deployment, which doesn't meet the high availability requirements for the given RPO. Option B uses governance mode for Vault Lock, which doesn't guarantee the data will not be deleted. Therefore, option C is the only option that meets all requirements.

Question: 619 **Exam Heist**

A solutions architect is designing a security solution for a company that wants to provide developers with individual AWS accounts through AWS Organizations, while also maintaining standard security controls. Because the individual developers will have AWS account root user-level access to their own accounts, the solutions architect wants to ensure that the mandatory AWS CloudTrail configuration that is applied to new developer accounts is not modified.

Which action meets these requirements?

A.Create an IAM policy that prohibits changes to CloudTrail. and attach it to the root user.

B.Create a new trail in CloudTrail from within the developer accounts with the organization trails option enabled.

C.Create a service control policy (SCP) that prohibits changes to CloudTrail, and attach it the developer accounts.

D.Create a service-linked role for CloudTrail with a policy condition that allows changes only from an Amazon Resource Name (ARN) in the management account.

Answer: C

Explanation:

The correct answer is C because it utilizes Service Control Policies (SCPs) to centrally enforce mandatory configurations across AWS Organizations accounts, addressing the specific requirement of preventing modifications to the CloudTrail configuration in developer accounts.

Here's a breakdown of the justification:

Service Control Policies (SCPs): SCPs are a feature of AWS Organizations that allow administrators to define guardrails and controls over the AWS services and actions that users can perform within member accounts (in this case, the developer accounts). These policies are applied at the organizational unit (OU) or account level, effectively setting boundaries for permitted actions.
https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scp.html

Preventing CloudTrail Modification: By creating an SCP that explicitly denies actions related to modifying or deleting CloudTrail configurations (e.g., cloudtrail:UpdateTrail, cloudtrail:DeleteTrail), the solutions architect can ensure that even with root user access in the developer accounts, developers cannot alter the mandatory CloudTrail setup defined at the organization level.

Centralized Enforcement: SCPs are managed from the AWS Organizations management account, providing a single point of control for enforcing security policies across all member accounts. This ensures consistency and simplifies governance.

Root User Limitation: While developers have root user access in their individual accounts, SCPs override these permissions. The SCP acts as a "guardrail," preventing actions regardless of the user's IAM permissions within the account. This addresses the core requirement of securing the CloudTrail configuration despite the developers having root user privileges.

Why other options are incorrect:

A: Applying an IAM policy to the root user within each developer account is not scalable and would require manual configuration in each account. Furthermore, if a developer with root user access compromises the account, they could potentially modify or delete the policy itself.
B: Creating a new trail from the developer account does not meet the request. If the initial CloudTrail is changed and the only record is the additional trail, this becomes a misconfiguration.
D: Service-linked roles are used for services to access other services and do not restrict permissions for users within the developer accounts. It will also not prevent developers from modifying or deleting CloudTrail configurations.

In summary, using SCPs is the most effective and scalable way to enforce mandatory CloudTrail configurations across multiple AWS accounts in an organization, ensuring that developers cannot bypass centrally defined security controls even with root user access to their individual accounts.

Question: 620 **Exam Heist**

A company is planning to deploy a business-critical application in the AWS Cloud. The application requires durable storage with consistent, low-latency performance.

Which type of storage should a solutions architect recommend to meet these requirements?

A.Instance store volume
B.Amazon ElastiCache for Memcached cluster
C.Provisioned IOPS SSD Amazon Elastic Block Store (Amazon EBS) volume
D.Throughput Optimized HDD Amazon Elastic Block Store (Amazon EBS) volume

Answer: C

Explanation:

The correct answer is C, Provisioned IOPS SSD Amazon Elastic Block Store (Amazon EBS) volume. Here's why:

The application requires durable storage, consistent, and low-latency performance. Let's evaluate each option:

A. Instance store volume: Instance store provides very high performance but is ephemeral. Data is lost if the instance fails or is stopped. This violates the durability requirement. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/InstanceStorage.html

B. Amazon ElastiCache for Memcached cluster: ElastiCache is a caching service. While it offers low latency, it is primarily for caching frequently accessed data and is not designed for durable storage. The data within Memcached instances is not persistent across failures. It's not suitable as the primary storage for a business-critical application. https://aws.amazon.com/elasticache/

C. Provisioned IOPS SSD Amazon Elastic Block Store (Amazon EBS) volume: Provisioned IOPS (io1 and io2) EBS volumes are designed for I/O-intensive workloads and offer consistent, low-latency performance by allowing you to specify the number of IOPS (Input/Output Operations Per Second) that the volume can support. EBS provides durable, block-level storage that is independent of the EC2 instance lifecycle. This ensures data persistence and meets the requirements. The io2 Block Express volumes provide the highest performance. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html

D. Throughput Optimized HDD Amazon Elastic Block Store (Amazon EBS) volume: Throughput Optimized HDD (st1) volumes are designed for frequently accessed, throughput-intensive workloads, such as big data, data warehouses, and log processing. While they are cost-effective, they don't offer the consistent, low-latency performance required for a business-critical application. HDD drives have slower seek times compared to SSDs.

Therefore, only Provisioned IOPS SSD EBS volumes provide the necessary durability, consistency, and low latency for the application.

Question: 621 **Exam Heist**

An online photo-sharing company stores its photos in an Amazon S3 bucket that exists in the us-west-1 Region. The company needs to store a copy of all new photos in the us-east-1 Region.

Which solution will meet this requirement with the LEAST operational effort?

A.Create a second S3 bucket in us-east-1. Use S3 Cross-Region Replication to copy photos from the existing S3 bucket to the second S3 bucket.

B.Create a cross-origin resource sharing (CORS) configuration of the existing S3 bucket. Specify us-east-1 in the CORS rule's AllowedOrigin element.

C.Create a second S3 bucket in us-east-1 across multiple Availability Zones. Create an S3 Lifecycle rule to save photos into the second S3 bucket.

D.Create a second S3 bucket in us-east-1. Configure S3 event notifications on object creation and update events to invoke an AWS Lambda function to copy photos from the existing S3 bucket to the second S3 bucket.

Answer: A

Explanation:

The correct answer is **A. Create a second S3 bucket in us-east-1. Use S3 Cross-Region Replication to copy photos from the existing S3 bucket to the second S3 bucket.**

Here's why:

S3 Cross-Region Replication (CRR) is specifically designed to automatically and asynchronously replicate objects between S3 buckets in different AWS Regions. It's the most straightforward and efficient way to

achieve the stated requirement of storing copies of new photos in a different Region with minimal operational overhead. CRR handles the replication process seamlessly, ensuring data consistency and minimizing manual intervention.

Option B is incorrect because CORS (Cross-Origin Resource Sharing) is a browser security mechanism that allows web pages from one domain to access resources from a different domain. It doesn't facilitate S3 object replication across regions.

Option C is incorrect because S3 Lifecycle rules are primarily for managing object storage lifecycle, such as transitioning objects to different storage classes (e.g., Standard to Glacier) or deleting them after a certain period. While lifecycle rules can move data within a region, they are not the primary method for replicating data between regions.

Option D is incorrect because while Lambda could be used to copy the objects, it involves significantly more operational effort than CRR. It requires writing, deploying, and maintaining Lambda function code, configuring S3 event notifications, and managing potential scaling and error handling issues. This approach introduces unnecessary complexity compared to the built-in functionality of CRR. CRR handles the replication in a managed way.In terms of 'least operational effort', CRR requires the least hands-on management because the replication is automatically managed by AWS once configured.

For further research, refer to the AWS documentation on S3 Cross-Region Replication:

Amazon S3 Cross-Region Replication

Question: 622 **Exam Heist**

A company is creating a new web application for its subscribers. The application will consist of a static single page and a persistent database layer. The application will have millions of users for 4 hours in the morning, but the application will have only a few thousand users during the rest of the day. The company's data architects have requested the ability to rapidly evolve their schema.

Which solutions will meet these requirements and provide the MOST scalability? (Choose two.)

A.Deploy Amazon DynamoDB as the database solution. Provision on-demand capacity.

B.Deploy Amazon Aurora as the database solution. Choose the serverless DB engine mode.

C.Deploy Amazon DynamoDB as the database solution. Ensure that DynamoDB auto scaling is enabled.

D.Deploy the static content into an Amazon S3 bucket. Provision an Amazon CloudFront distribution with the S3 bucket as the origin.

E.Deploy the web servers for static content across a fleet of Amazon EC2 instances in Auto Scaling groups. Configure the instances to periodically refresh the content from an Amazon Elastic File System (Amazon EFS) volume.

Answer: CD

Explanation:

The correct answer is **CD**. Here's why:

Option C: Deploy Amazon DynamoDB as the database solution. Ensure that DynamoDB auto scaling is enabled.

DynamoDB is a fully managed NoSQL database service. Its key advantage in this scenario is its ability to scale automatically based on demand. Given the huge spike in users during the morning hours and a significant drop-off for the rest of the day, DynamoDB auto-scaling is crucial for handling the fluctuating workload efficiently and cost-effectively. DynamoDB's NoSQL nature aligns with the architects' need to rapidly evolve the schema as it provides more flexibility than a relational database.https://aws.amazon.com/dynamodb/

Option D: Deploy the static content into an Amazon S3 bucket. Provision an Amazon CloudFront distribution with the S3 bucket as the origin.

S3 is highly scalable and provides a durable storage for static content. By pairing it with CloudFront, a content delivery network (CDN), the application can effectively serve millions of users globally. CloudFront caches content at edge locations, reducing latency and offloading the traffic from S3. Since the application will consist of a static single page, serving it directly from S3 through CloudFront is the most cost-effective and scalable solution.https://aws.amazon.com/s3/https://aws.amazon.com/cloudfront/

Why other options are not optimal:

A: Deploy Amazon DynamoDB as the database solution. Provision on-demand capacity. While DynamoDB is a good choice, provisioning on-demand capacity requires estimating the peak load and pre-allocating resources. This approach is less flexible and can be more expensive than auto-scaling, especially with the drastically changing user load. It doesn't adapt dynamically to usage changes as well as autoscaling.
B: Deploy Amazon Aurora as the database solution. Choose the serverless DB engine mode. Aurora Serverless could be a good choice, but it may not be the best option if rapid schema evolution is required. Relational databases generally involve more rigid schemas and schema migrations than NoSQL databases like DynamoDB.
E: Deploy the web servers for static content across a fleet of Amazon EC2 instances in Auto Scaling groups. Configure the instances to periodically refresh the content from an Amazon Elastic File System (Amazon EFS) volume. Deploying a fleet of EC2 instances to serve static content is overkill and more complex. Using S3 and CloudFront is much simpler, more scalable, and cheaper for this purpose. EFS is designed for shared file storage, not for serving static web content to a large number of users. EC2 is best suited for compute-intensive tasks or running dynamic applications, not for directly serving static content.

Question: 623 **Exam Heist**

A company uses Amazon API Gateway to manage its REST APIs that third-party service providers access. The company must protect the REST APIs from SQL injection and cross-site scripting attacks.

What is the MOST operationally efficient solution that meets these requirements?

A.Configure AWS Shield.
B.Configure AWS WAF.
C.Set up API Gateway with an Amazon CloudFront distribution. Configure AWS Shield in CloudFront.
D.Set up API Gateway with an Amazon CloudFront distribution. Configure AWS WAF in CloudFront.

Answer: B

Explanation:

The most operationally efficient solution to protect REST APIs managed by Amazon API Gateway from SQL injection and cross-site scripting (XSS) attacks is to configure AWS WAF (Web Application Firewall).

Here's why:

AWS WAF's Core Functionality: AWS WAF is specifically designed to protect web applications from common web exploits like SQL injection and XSS. It allows you to define customizable rules to filter malicious traffic based on patterns in HTTP requests.

Direct Integration with API Gateway: AWS WAF can be directly associated with API Gateway, providing a seamless and efficient way to protect APIs. This direct integration minimizes operational overhead.

Pre-configured Rules: AWS WAF offers pre-configured rule sets (managed rules) specifically designed to

mitigate common web vulnerabilities, including those associated with SQL injection and XSS. Using managed rules simplifies deployment and maintenance.

Operational Efficiency: Using AWS WAF directly on API Gateway avoids the added complexity and operational overhead of implementing a CDN like CloudFront solely for WAF integration. The other solutions are not as operationally efficient.

AWS Shield's Limitations: AWS Shield primarily protects against DDoS attacks, not SQL injection or XSS. While Shield is valuable for availability, it doesn't address the stated security requirements.

CloudFront and WAF Combination (Suboptimal): While combining CloudFront with AWS WAF is a valid approach for protecting web applications, it's less operationally efficient for API Gateway protection compared to directly associating WAF with API Gateway. The CloudFront layer introduces additional complexity, and it's not strictly necessary when the primary goal is to protect the API layer.

Therefore, AWS WAF provides the most operationally efficient solution, by allowing you to define rules or leverage managed rules to filter traffic and protect against SQL injection and XSS attacks directly at the API Gateway level.

Supporting Links:

AWS WAF: https://aws.amazon.com/waf/
AWS WAF and API Gateway: https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-control-access-aws-waf.html
AWS Shield: https://aws.amazon.com/shield/

Question: 624 **Exam Heist**

A company wants to provide users with access to AWS resources. The company has 1,500 users and manages their access to on-premises resources through Active Directory user groups on the corporate network. However, the company does not want users to have to maintain another identity to access the resources. A solutions architect must manage user access to the AWS resources while preserving access to the on-premises resources.

What should the solutions architect do to meet these requirements?

A.Create an IAM user for each user in the company. Attach the appropriate policies to each user.

B.Use Amazon Cognito with an Active Directory user pool. Create roles with the appropriate policies attached.

C.Define cross-account roles with the appropriate policies attached. Map the roles to the Active Directory groups.

D.Configure Security Assertion Markup Language (SAML) 2 0-based federation. Create roles with the appropriate policies attached Map the roles to the Active Directory groups.

Answer: D

Explanation:

Here's a detailed justification for why option D is the best solution:

The scenario describes a common use case: integrating on-premises Active Directory (AD) with AWS for seamless user authentication and authorization. The core requirement is to allow existing users to access AWS resources without creating separate AWS identities, leveraging their existing AD credentials.

Option D, using SAML 2.0-based federation, directly addresses this need. SAML (Security Assertion Markup Language) is an open standard for exchanging authentication and authorization data between security domains, namely an Identity Provider (IdP) and a Service Provider (SP). In this case, Active Directory acts as the IdP, and AWS acts as the SP.

Here's how it works:

1. A user attempts to access an AWS resource.
2. AWS redirects the user to the AD Federation Services (ADFS) server, which acts as the SAML IdP.
3. The user authenticates with their existing AD credentials.
4. ADFS creates a SAML assertion containing information about the user's identity and group memberships.
5. ADFS sends the SAML assertion to AWS.
6. AWS uses the information in the SAML assertion to determine which IAM role the user should assume. You configure this mapping in IAM.
7. AWS grants the user temporary credentials based on the permissions defined in the assumed role.
8. The user can now access the AWS resource with the permissions granted by the role.

By mapping AD groups to IAM roles, you can control access to AWS resources based on the user's existing group memberships in Active Directory. This approach avoids creating and managing individual IAM users for each of the 1,500 users, simplifying administration and maintaining consistency with on-premises access controls. SAML federation provides a single sign-on (SSO) experience, enhancing user convenience.

The other options are less suitable:

Option A, creating individual IAM users, is unmanageable for a large number of users (1,500) and defeats the purpose of leveraging the existing Active Directory.
Option B, using Amazon Cognito with an Active Directory user pool, is primarily designed for authenticating application users and doesn't directly integrate with IAM roles for resource access in the same way SAML does. Cognito does not directly map AD groups to IAM roles. While cognito can authenticate against AD, it is not the most straightforward solution to the described problem.
Option C, defining cross-account roles, is mainly used for granting access to resources in other AWS accounts, not for federating with an external identity provider like Active Directory. This option also doesn't solve the problem of linking AD identities to AWS access.

Authoritative links:

AWS Documentation on SAML
AWS Whitepaper on Federated Authentication with AWS

Question: 625 **Exam Heist**

A company is hosting a website behind multiple Application Load Balancers. The company has different distribution rights for its content around the world. A solutions architect needs to ensure that users are served the correct content without violating distribution rights.

Which configuration should the solutions architect choose to meet these requirements?

A.Configure Amazon CloudFront with AWS WAF.
B.Configure Application Load Balancers with AWS WAF
C.Configure Amazon Route 53 with a geolocation policy
D.Configure Amazon Route 53 with a geoproximity routing policy

Answer: C

Explanation:

The correct answer is C: Configure Amazon Route 53 with a geolocation policy.

Geolocation routing in Amazon Route 53 allows you to route traffic to different resources based on the

geographic location of your users. This is perfectly suited for scenarios where content distribution rights vary by region. By configuring Route 53 to direct users from specific countries or regions to the appropriate Application Load Balancer and origin server holding the content licensed for that area, the company can ensure compliance with distribution rights. Each record in Route 53 would specify a geographic location (e.g., a country) and the corresponding Application Load Balancer endpoint.

Option A, using CloudFront with AWS WAF, primarily addresses security concerns and content caching for performance improvements but doesn't directly facilitate geographic content distribution. While CloudFront supports geo-restriction, it mainly blocks access rather than routing to different content origins based on location.

Option B, using Application Load Balancers with AWS WAF, focuses on securing the web application layer and doesn't inherently offer geolocation-based routing capabilities. While WAF can identify the origin of a request through IP addresses, it cannot dynamically route traffic to different servers based on that information.

Option D, using Route 53 with geoproximity routing, is designed to route traffic based on the physical proximity of users to your resources. While it uses geographical information, it's primarily for optimizing latency and not for complying with content distribution rights as precisely as geolocation routing. Geoproximity considers distances between resources and users, making it less suitable when adherence to defined regional boundaries for legal reasons is paramount. Geolocation offers precise control over region-based content delivery.

For further research, refer to the following resources:

Amazon Route 53 Geolocation Routing:
https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html#routing-policy-geo
AWS WAF: https://aws.amazon.com/waf/
Amazon CloudFront Geo Restriction:
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/georestrictions.html
Amazon Route 53 Geoproximity Routing:
https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html#routing-policy-geoproximity

Question: 626 **Exam Heist**

A company stores its data on premises. The amount of data is growing beyond the company's available capacity.

The company wants to migrate its data from the on-premises location to an Amazon S3 bucket. The company needs a solution that will automatically validate the integrity of the data after the transfer.

Which solution will meet these requirements?

A.Order an AWS Snowball Edge device. Configure the Snowball Edge device to perform the online data transfer to an S3 bucket

B.Deploy an AWS DataSync agent on premises. Configure the DataSync agent to perform the online data transfer to an S3 bucket.

C.Create an Amazon S3 File Gateway on premises Configure the S3 File Gateway to perform the online data transfer to an S3 bucket

D.Configure an accelerator in Amazon S3 Transfer Acceleration on premises. Configure the accelerator to perform the online data transfer to an S3 bucket.

Answer: B

Explanation:

The correct answer is B: Deploy an AWS DataSync agent on premises. Configure the DataSync agent to

perform the online data transfer to an S3 bucket.

Here's why: AWS DataSync is specifically designed for efficiently and securely transferring large amounts of data between on-premises storage and AWS services like S3. A key feature of DataSync is its built-in data integrity validation. During the transfer process, DataSync performs checksum calculations on both the source and destination data. These checksums are compared to ensure that the data transferred is identical to the original data, thus automatically validating integrity. DataSync also offers features like encryption during transit and at rest, compression, and automatic retries for failed transfers. This ensures reliable and secure data migration.

Option A (AWS Snowball Edge) is suitable for large datasets when network bandwidth is limited or unreliable. While Snowball Edge validates data during import to AWS, it doesn't automatically validate the integrity during an online transfer from on-premises, as the question requires. It's primarily an offline data transfer solution.

Option C (Amazon S3 File Gateway) provides a way to access S3 objects as files on premises, effectively creating a hybrid storage solution. While it can transfer data, its primary purpose isn't bulk data migration with automatic integrity validation like DataSync. It's more focused on providing local file system access to S3.

Option D (Amazon S3 Transfer Acceleration) leverages AWS's globally distributed edge locations to accelerate uploads to S3. While it can improve transfer speeds, it doesn't provide the automatic data integrity validation feature that DataSync offers. It focuses solely on optimizing transfer speeds.

In summary, DataSync's core functionalities are efficient data transfer and automatic data integrity validation, precisely meeting the requirements of the question.

Further Reading:

AWS DataSync: https://aws.amazon.com/datasync/
AWS Snowball Edge: https://aws.amazon.com/snowball/
Amazon S3 File Gateway: https://aws.amazon.com/storagegateway/file/
Amazon S3 Transfer Acceleration: https://aws.amazon.com/s3/transfer-acceleration/

Question: 627 **Exam Heist**

A company wants to migrate two DNS servers to AWS. The servers host a total of approximately 200 zones and receive 1 million requests each day on average. The company wants to maximize availability while minimizing the operational overhead that is related to the management of the two servers.

What should a solutions architect recommend to meet these requirements?

A.Create 200 new hosted zones in the Amazon Route 53 console Import zone files.

B.Launch a single large Amazon EC2 instance Import zone tiles. Configure Amazon CloudWatch alarms and notifications to alert the company about any downtime.

C.Migrate the servers to AWS by using AWS Server Migration Service (AWS SMS). Configure Amazon CloudWatch alarms and notifications to alert the company about any downtime.

D.Launch an Amazon EC2 instance in an Auto Scaling group across two Availability Zones. Import zone files. Set the desired capacity to 1 and the maximum capacity to 3 for the Auto Scaling group. Configure scaling alarms to scale based on CPU utilization.

Answer: A

Explanation:

The best solution to migrate the DNS servers to AWS, maximizing availability and minimizing operational overhead, is option A: creating 200 new hosted zones in Amazon Route 53 and importing zone files.

Here's why:

Managed Service: Route 53 is a highly available and scalable managed DNS service. This means AWS handles the underlying infrastructure, patching, scaling, and maintenance, greatly reducing operational overhead for the company compared to managing EC2 instances. https://aws.amazon.com/route53/
High Availability: Route 53 is designed for high availability, automatically distributing DNS records across multiple authoritative DNS servers. The company benefits from this inherent redundancy without having to configure and manage it themselves.
Scalability: Route 53 can easily handle the 1 million requests per day. Its architecture scales to handle large query volumes without manual intervention.
Simple Migration: Importing zone files is a straightforward process, making the migration relatively simple and less prone to errors. It maintains the existing DNS configurations.
Cost-Effective: While Route 53 has associated costs for queries, these are likely to be lower than the costs and operational overhead associated with running EC2 instances, especially considering the HA requirements.
No Server Management: Unlike options B, C, and D, Option A avoids the need to manage servers, apply patches, and deal with operating system level issues. The managed service model abstracts these operational burdens away.
Auto Scaling is Unnecessary: The auto scaling features in Option D are not pertinent as Route 53 is a highly available service.

Options B, C, and D involve managing EC2 instances, which increases operational overhead. Launching an Amazon EC2 instance and running the DNS server on EC2 will require ongoing server maintenance, security patching, and scaling efforts. While options C and D try to automate the deployment of servers, they introduce added layers of complexity to manage.

Therefore, using Route 53 (Option A) is the most efficient and cost-effective solution for migrating DNS servers to AWS, maximizing availability, and minimizing operational overhead.

Question: 628 **Exam Heist**

A global company runs its applications in multiple AWS accounts in AWS Organizations. The company's applications use multipart uploads to upload data to multiple Amazon S3 buckets across AWS Regions. The company wants to report on incomplete multipart uploads for cost compliance purposes.

Which solution will meet these requirements with the LEAST operational overhead?

A.Configure AWS Config with a rule to report the incomplete multipart upload object count.
B.Create a service control policy (SCP) to report the incomplete multipart upload object count.
C.Configure S3 Storage Lens to report the incomplete multipart upload object count.
D.Create an S3 Multi-Region Access Point to report the incomplete multipart upload object count.

Answer: C

Explanation:

The correct answer is C: Configure S3 Storage Lens to report the incomplete multipart upload object count. Here's a detailed justification:

S3 Storage Lens provides organization-wide visibility into object storage usage, activity trends, and makes actionable recommendations to optimize costs and apply data protection best practices. It aggregates data across all S3 buckets within an AWS organization (or a single account), including metrics related to incomplete multipart uploads. It offers pre-built dashboards that are accessible through the S3 console, making setup and reporting straightforward.

Option A (AWS Config) is not ideal because while Config can track configuration changes to S3 buckets, it doesn't directly report on metrics like incomplete multipart uploads in a cost-effective and aggregated manner across multiple accounts and Regions. Implementing a custom Config rule for this purpose would involve significant operational overhead and custom logic.

Option B (SCP) is inappropriate. SCPs control what IAM principals (users and roles) within the organization can do. They are not designed to collect or report on S3 metrics. SCPs prevent actions; they don't provide visibility or reporting capabilities for storage-related costs.

Option D (S3 Multi-Region Access Point) is irrelevant to the problem. Multi-Region Access Points provide a single global endpoint for S3 data stored in multiple Regions, improving application availability. They do not address the requirement of reporting on incomplete multipart uploads.

S3 Storage Lens directly addresses the reporting requirement with the least operational overhead because it's designed to provide this type of aggregated storage metrics across accounts and Regions in AWS Organizations. It offers a dedicated dashboard to analyze and report incomplete multipart uploads for cost optimization.

Here are some authoritative links for further research:

AWS S3 Storage Lens: https://aws.amazon.com/s3/storage-lens/
Using S3 Storage Lens: https://docs.aws.amazon.com/AmazonS3/latest/userguide/storage-lens.html
Understanding S3 Storage Lens Metrics: https://docs.aws.amazon.com/AmazonS3/latest/userguide/storage-lens-metrics-values.html

Question: 629 **Exam Heist**

A company runs a production database on Amazon RDS for MySQL. The company wants to upgrade the database version for security compliance reasons. Because the database contains critical data, the company wants a quick solution to upgrade and test functionality without losing any data.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create an RDS manual snapshot. Upgrade to the new version of Amazon RDS for MySQL.

B.Use native backup and restore. Restore the data to the upgraded new version of Amazon RDS for MySQL.

C.Use AWS Database Migration Service (AWS DMS) to replicate the data to the upgraded new version of Amazon RDS for MySQL.

D.Use Amazon RDS Blue/Green Deployments to deploy and test production changes.

Answer: D

Explanation:

The correct answer is D: Use Amazon RDS Blue/Green Deployments to deploy and test production changes.

Here's a detailed justification:

Amazon RDS Blue/Green Deployments offer the simplest and fastest method for database engine upgrades while minimizing downtime and data loss risk. They allow creating a fully functional, isolated staging environment (the "green" environment) that mirrors the production environment ("blue"). This makes it possible to test the upgraded MySQL version without affecting the production database.

Using Blue/Green Deployments, the company can create a green environment with the upgraded MySQL version. RDS handles the data replication from the blue to the green environment. The company can then perform thorough testing in the green environment, ensuring all applications function correctly with the new version. If testing is successful, a simple switchover promotes the green environment to production, making it

the new blue environment. This process involves a brief outage, considerably less than other migration methods. Crucially, the entire process is managed by RDS, reducing manual intervention and operational overhead. If issues arise, the switchover can be reversed, falling back to the original blue environment.

Option A is not ideal because taking a manual snapshot and restoring it to a new, upgraded instance involves manual steps and potential downtime for the restore process. Option B, using native backup and restore, also increases operational overhead. Option C, AWS DMS, is more suited for migrating between different database engines or moving databases to AWS and typically incurs more overhead than a simple version upgrade within RDS. Blue/Green deployments are specifically designed for these types of upgrade scenarios within RDS.

Therefore, Amazon RDS Blue/Green Deployments is the most efficient approach for upgrading the MySQL database version with minimal downtime, reduced operational overhead, and a safe testing environment.

Refer to the AWS documentation for more information:

Amazon RDS Blue/Green Deployments: https://aws.amazon.com/rds/blue-green-deployments/
Performing blue/green deployments for Amazon RDS for MySQL:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/blue-green-deployments.MySQL.html

Question: 630 **Exam Heist**

A solutions architect is creating a data processing job that runs once daily and can take up to 2 hours to complete. If the job is interrupted, it has to restart from the beginning.

How should the solutions architect address this issue in the MOST cost-effective manner?

A.Create a script that runs locally on an Amazon EC2 Reserved Instance that is triggered by a cron job.

B.Create an AWS Lambda function triggered by an Amazon EventBridge scheduled event.

C.Use an Amazon Elastic Container Service (Amazon ECS) Fargate task triggered by an Amazon EventBridge scheduled event.

D.Use an Amazon Elastic Container Service (Amazon ECS) task running on Amazon EC2 triggered by an Amazon EventBridge scheduled event.

Answer: C

Explanation:

The most cost-effective solution is to use an Amazon ECS Fargate task triggered by an Amazon EventBridge scheduled event (Option C). Here's why:

Fargate's "Pay-as-you-go" Model: Fargate allows you to run containers without managing servers. You pay only for the compute resources your container uses while it's running, making it efficient for sporadic workloads.

EventBridge Scheduling: EventBridge can reliably trigger the ECS task at the scheduled time (once daily).

No Idle EC2 Costs: Options A and D involve running EC2 instances, which incur costs even when idle. Since the job only runs for up to 2 hours daily, maintaining a dedicated EC2 instance is wasteful. Reserved Instances (Option A) could potentially reduce EC2 costs, but require long-term commitment and might not be the most cost-effective if utilization is low.

Lambda Time Constraints: AWS Lambda (Option B) has execution time limits (currently 15 minutes). Since the data processing job can take up to 2 hours, Lambda is not a suitable option.

Fault Tolerance: ECS Fargate automatically handles underlying infrastructure and availability. It restarts the container if it fails during execution.

Simplified Management: Fargate abstracts away the complexities of EC2 instance management, making the solution simpler to deploy and maintain.

In summary, ECS Fargate combined with EventBridge provides a serverless, scalable, and cost-optimized solution for the scheduled data processing job that can handle interruptions by restarting the task, all while adhering to the budget-conscious constraint.

Supporting Links:

Amazon ECS Fargate: https://aws.amazon.com/fargate/
Amazon EventBridge: https://aws.amazon.com/eventbridge/
AWS Lambda Limits: https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html

Question: 631 **Exam Heist**

A social media company wants to store its database of user profiles, relationships, and interactions in the AWS Cloud. The company needs an application to monitor any changes in the database. The application needs to analyze the relationships between the data entities and to provide recommendations to users.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use Amazon Neptune to store the information. Use Amazon Kinesis Data Streams to process changes in the database.

B.Use Amazon Neptune to store the information. Use Neptune Streams to process changes in the database.

C.Use Amazon Quantum Ledger Database (Amazon QLDB) to store the information. Use Amazon Kinesis Data Streams to process changes in the database.

D.Use Amazon Quantum Ledger Database (Amazon QLDB) to store the information. Use Neptune Streams to process changes in the database.

Answer: B

Explanation:

The optimal solution leverages Amazon Neptune and Neptune Streams.

Amazon Neptune: Neptune is a purpose-built graph database ideal for storing and querying data with complex relationships, such as social media user profiles, connections, and interactions. This directly addresses the company's need to represent relationships between data entities. (https://aws.amazon.com/neptune/)

Neptune Streams: Neptune Streams provide a managed mechanism for capturing changes (insertions, updates, deletions) happening within the Neptune graph database. This allows the company to monitor database changes for analysis and recommendations. This is a feature tightly integrated with Neptune, minimizing operational overhead compared to external streaming solutions. (https://docs.aws.amazon.com/neptune/latest/userguide/streams.html)

Option A is less optimal because Kinesis Data Streams, while a robust streaming service, requires additional configuration and integration to extract change data from Neptune, increasing operational overhead compared to the native Neptune Streams feature.

Options C and D use Amazon QLDB, which is a ledger database designed for maintaining a verifiable and immutable history of data changes. While QLDB ensures data integrity, it is not optimized for analyzing complex relationships or providing recommendations based on graph patterns. It excels at auditing, not the relationship analysis required in the scenario. Further, Neptune Streams can not be used with QLDB, as it is a Neptune specific service. Thus, QLDB is not suitable for the requirement, and the combination of QLDB and Kinesis Data Streams introduces unnecessary complexity compared to Neptune and Neptune Streams.

Question: 632 **Exam Heist**

A company is creating a new application that will store a large amount of data. The data will be analyzed hourly and will be modified by several Amazon EC2 Linux instances that are deployed across multiple Availability Zones. The needed amount of storage space will continue to grow for the next 6 months.

Which storage solution should a solutions architect recommend to meet these requirements?

A.Store the data in Amazon S3 Glacier. Update the S3 Glacier vault policy to allow access to the application instances.

B.Store the data in an Amazon Elastic Block Store (Amazon EBS) volume. Mount the EBS volume on the application instances.

C.Store the data in an Amazon Elastic File System (Amazon EFS) file system. Mount the file system on the application instances.

D.Store the data in an Amazon Elastic Block Store (Amazon EBS) Provisioned IOPS volume shared between the application instances.

Answer: C

Explanation:

The correct answer is **C. Store the data in an Amazon Elastic File System (Amazon EFS) file system. Mount the file system on the application instances.**

Here's why:

Shared Access: The scenario specifies that multiple EC2 instances in different Availability Zones need to modify the data concurrently. Amazon EFS is designed for shared file storage, allowing multiple instances to access the same data simultaneously. EBS, in contrast, can only be attached to a single instance at a time (unless using EBS multi-attach, but this is a more complex solution for few instances).
Scalability: The requirement states that the storage space will grow significantly in the next 6 months. EFS is designed to scale automatically as data is added, without the need for manual intervention or pre-provisioning. This makes it suitable for dynamic storage needs.
Availability Zones: EFS is a regional service, meaning it replicates data across multiple Availability Zones. This provides high availability and durability, ensuring that the data is accessible even if one Availability Zone experiences an outage.
Performance: EFS offers different performance modes and throughput options, including Bursting Throughput and Provisioned Throughput, allowing the company to choose the performance level that best meets its analysis requirements.

Why the other options are incorrect:

A. Amazon S3 Glacier: Glacier is designed for long-term archival storage where infrequent access is expected. Hourly analysis and modification would make Glacier unsuitable due to its retrieval times and cost structure.
B & D. Amazon EBS: EBS is a block storage service that is directly attached to a single EC2 instance (with limited multi-attach use cases). While EBS can be used, sharing a standard or Provisioned IOPS EBS volume among multiple instances is not natively supported (without complex solutions). EBS is also limited to a single AZ per Volume, and while snapshots can be replicated across AZs, that is not a solution for a shared filesystem. Additionally, EBS requires manual resizing to increase the storage capacity.

Therefore, using Amazon EFS offers the most efficient and scalable solution for storing and sharing data among multiple EC2 instances in different Availability Zones, aligning perfectly with the specified requirements.

Authoritative Links:

Amazon EFS: https://aws.amazon.com/efs/
Amazon EBS: https://aws.amazon.com/ebs/
Amazon S3 Glacier: https://aws.amazon.com/glacier/

Question: 633 **Exam Heist**

A company manages an application that stores data on an Amazon RDS for PostgreSQL Multi-AZ DB instance. Increases in traffic are causing performance problems. The company determines that database queries are the primary reason for the slow performance.

What should a solutions architect do to improve the application's performance?

A.Serve read traffic from the Multi-AZ standby replica.

B.Configure the DB instance to use Transfer Acceleration.

C.Create a read replica from the source DB instance. Serve read traffic from the read replica.

D.Use Amazon Kinesis Data Firehose between the application and Amazon RDS to increase the concurrency of database requests.

Answer: C

Explanation:

Here's a detailed justification for why option C is the correct answer:

The problem clearly states that increasing traffic is causing performance problems specifically related to database queries. The RDS PostgreSQL instance is a Multi-AZ deployment, which is primarily for high availability and disaster recovery, not for scaling read operations. The standby replica in a Multi-AZ configuration is not directly accessible for serving read traffic in a standard configuration; it's only activated in case of a failover. Therefore, option A is incorrect.

Option B, configuring the DB instance to use Transfer Acceleration, is irrelevant. Transfer Acceleration is an Amazon S3 feature for accelerating data transfers into and out of S3 buckets. It doesn't apply to RDS database performance or query optimization.

Option D, using Amazon Kinesis Data Firehose between the application and RDS, is also incorrect. Kinesis Data Firehose is designed for streaming data into data lakes, data stores, and analytics services. It's not intended as a solution for optimizing database query performance or increasing concurrency of database requests in a transactional database.

Option C, creating a read replica from the source DB instance and serving read traffic from the read replica, is the correct solution. Read replicas are designed specifically to offload read traffic from the primary database instance. By directing read queries to the read replica, the primary instance is freed up to handle write operations and other critical tasks, significantly improving overall application performance and reducing the load on the primary database. This approach leverages the concept of read scaling, a common pattern for improving the performance of read-heavy applications.

Here are some authoritative links for further research:

Amazon RDS Read Replicas:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Overview.ReadReplicas.html
Amazon RDS Multi-AZ Deployments:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html
Scaling with RDS: https://aws.amazon.com/rds/scaling/

Question: 634 **Exam Heist**

A company collects 10 GB of telemetry data daily from various machines. The company stores the data in an Amazon S3 bucket in a source data account.

The company has hired several consulting agencies to use this data for analysis. Each agency needs read access to the data for its analysts. The company must share the data from the source data account by choosing a solution that maximizes security and operational efficiency.

Which solution will meet these requirements?

A.Configure S3 global tables to replicate data for each agency.

B.Make the S3 bucket public for a limited time. Inform only the agencies.

C.Configure cross-account access for the S3 bucket to the accounts that the agencies own.

D.Set up an IAM user for each analyst in the source data account. Grant each user access to the S3 bucket.

Answer: C

Explanation:

The best solution is **C. Configure cross-account access for the S3 bucket to the accounts that the agencies own.**

Here's why:

Security: Cross-account access using IAM roles is the most secure approach. It avoids sharing credentials or making the bucket public. The agencies' analysts access the data through their own AWS accounts, ensuring that the company retains control over its data while granting specific permissions. This aligns with the principle of least privilege.

Operational Efficiency: Managing access via cross-account roles is operationally efficient. The company doesn't need to manage individual IAM users for each analyst within their account. Changes to analyst access are handled within the agency's AWS account.

Avoids data duplication: Cross-account access avoids replicating the data as S3 global tables would. This saves on storage costs and reduces operational overhead.

Limited access duration and Public access are not secure Making the bucket public even for a limited time (option B) introduces significant security risks. It's vulnerable to unauthorized access and potential data breaches.

IAM for individual analyst management is complex and not efficient: Creating IAM users for each analyst in the company's account (option D) is not scalable or efficient, creating complexity and operational overhead.

S3 Global tables (replication) is not required: Replicating 10GB of data daily to separate buckets for each agency (option A) introduces significant storage and data transfer costs and delays. S3 global tables is meant for low-latency access across regions, not a distribution mechanism for data analysis.

Cross-account access: grants specific and auditable permissions, while using the agencies' existing accounts, avoiding unnecessary IAM user management.

Supporting Documentation:

AWS Documentation on Cross-Account Access:
https://docs.aws.amazon.com/IAM/latest/UserGuide/tutorial_cross-account-with-roles.html
AWS Documentation on S3 Bucket Permissions:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/security-iam.html

Question: 635 **Exam Heist**

A company uses Amazon FSx for NetApp ONTAP in its primary AWS Region for CIFS and NFS file shares. Applications that run on Amazon EC2 instances access the file shares. The company needs a storage disaster recovery (DR) solution in a secondary Region. The data that is replicated in the secondary Region needs to be accessed by using the same protocols as the primary Region.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create an AWS Lambda function to copy the data to an Amazon S3 bucket. Replicate the S3 bucket to the secondary Region.

B.Create a backup of the FSx for ONTAP volumes by using AWS Backup. Copy the volumes to the secondary Region. Create a new FSx for ONTAP instance from the backup.

C.Create an FSx for ONTAP instance in the secondary Region. Use NetApp SnapMirror to replicate data from the primary Region to the secondary Region.

D.Create an Amazon Elastic File System (Amazon EFS) volume. Migrate the current data to the volume. Replicate the volume to the secondary Region.

Answer: C

Explanation:

The correct answer is **C. Create an FSx for ONTAP instance in the secondary Region. Use NetApp SnapMirror to replicate data from the primary Region to the secondary Region.**

Here's why:

Meeting Requirements: The primary requirement is a DR solution for FSx for ONTAP with the least operational overhead, accessible via the same CIFS/NFS protocols in the DR region.
SnapMirror's Efficiency: NetApp SnapMirror is a native feature of FSx for ONTAP specifically designed for block-level, incremental replication between ONTAP systems. This makes it very efficient in replicating only the changes, thus minimizing bandwidth usage and replication time. It directly addresses the need for DR across regions.
Protocol Consistency: SnapMirror replicates the data while preserving the underlying file system structure and protocol support (CIFS/NFS). The DR FSx for ONTAP instance will natively serve the replicated data using the same protocols as the primary.
Least Operational Overhead: SnapMirror setup is relatively straightforward within the AWS console/API, requiring configuration of source and destination file systems. Once configured, replication can be automated with minimal ongoing manual intervention.

Let's analyze why the other options are less suitable:

A (Lambda and S3): This introduces unnecessary complexity and overhead. Data would need to be extracted from FSx for ONTAP, converted to an object format suitable for S3, and then re-hydrated into a file system format in the DR region. This process is slow, complex, and doesn't natively support CIFS/NFS directly from S3.
B (AWS Backup): AWS Backup can be used to create backups of FSx for ONTAP volumes and copy them across regions. While this provides a point-in-time recovery option, it's not the most efficient DR solution, especially for minimizing recovery time objective (RTO). Restoring from backup in DR would be slower than failing over to a replica maintained by SnapMirror. Also, it would require periodic backup schedules which adds to operational overhead compared to SnapMirror.
D (Amazon EFS): Migrating from FSx for ONTAP to EFS is a significant undertaking that introduces complexity and potential compatibility issues. Moreover, it's not a direct replacement for the FSx for ONTAP environment. EFS supports NFS, but not CIFS directly, which means the applications accessing the CIFS shares will require modification. This also fails to meet the core requirements and dramatically increases the

operational overhead.

Authoritative Links:

Amazon FSx for NetApp ONTAP Replication: https://aws.amazon.com/blogs/storage/implement-cross-region-disaster-recovery-with-amazon-fsx-for-netapp-ontap-using-netapp-snapmirror/
NetApp SnapMirror: https://docs.netapp.com/us-en/ontap/data-protection/concept_snapmirror_replication.html

Question: 636 **Exam Heist**

A development team is creating an event-based application that uses AWS Lambda functions. Events will be generated when files are added to an Amazon S3 bucket. The development team currently has Amazon Simple Notification Service (Amazon SNS) configured as the event target from Amazon S3.

What should a solutions architect do to process the events from Amazon S3 in a scalable way?

A.Create an SNS subscription that processes the event in Amazon Elastic Container Service (Amazon ECS) before the event runs in Lambda.

B.Create an SNS subscription that processes the event in Amazon Elastic Kubernetes Service (Amazon EKS) before the event runs in Lambda

C.Create an SNS subscription that sends the event to Amazon Simple Queue Service (Amazon SQS). Configure the SOS queue to trigger a Lambda function.

D.Create an SNS subscription that sends the event to AWS Server Migration Service (AWS SMS). Configure the Lambda function to poll from the SMS event.

Answer: C

Explanation:

The correct answer is **C. Create an SNS subscription that sends the event to Amazon Simple Queue Service (Amazon SQS). Configure the SQS queue to trigger a Lambda function.**

Here's why:

Scalability and Decoupling: Directly invoking Lambda from S3 via SNS can become problematic at scale. If S3 generates a high volume of events, SNS might overwhelm Lambda, leading to throttling and potential event loss. Using SQS as an intermediary decouples S3 event generation from Lambda processing. SQS acts as a buffer, holding events until Lambda can process them.

Reliability: SQS provides guaranteed delivery of messages. Even if Lambda fails to process an event, the message remains in the queue and can be retried, enhancing the application's reliability. This is crucial for event-driven architectures where data loss is unacceptable.

Asynchronous Processing: SQS allows Lambda to process events asynchronously. Lambda doesn't have to wait for S3 to acknowledge receipt, improving the overall performance of the application.

Fanout Pattern: SNS itself can be used to fan out events to multiple SQS queues, if needed, for different processing requirements. However, for a single Lambda function as the processor, a single SQS queue provides sufficient buffering and scalability.

Alternatives are less suitable:

A & B (ECS/EKS): Introducing container orchestration like ECS or EKS adds unnecessary complexity for this scenario. Lambda is designed for event-driven, serverless execution, making it a more suitable choice. Moreover, adding a processing layer before Lambda is counterintuitive to the problem's intent. The goal is to make Lambda event processing scalable.

D (AWS SMS): AWS SMS is for migrating on-premises servers to AWS, completely unrelated to event processing from S3.

In summary, using SNS to deliver S3 events to an SQS queue and then triggering a Lambda function from the queue creates a scalable, reliable, and decoupled event processing architecture.

Supporting Links:

AWS SQS: https://aws.amazon.com/sqs/
AWS Lambda: https://aws.amazon.com/lambda/
Amazon SNS: https://aws.amazon.com/sns/
Using AWS Lambda with Amazon SQS: https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html

Question: 637 **Exam Heist**

A solutions architect is designing a new service behind Amazon API Gateway. The request patterns for the service will be unpredictable and can change suddenly from 0 requests to over 500 per second. The total size of the data that needs to be persisted in a backend database is currently less than 1 GB with unpredictable future growth. Data can be queried using simple key-value requests.

Which combination ofAWS services would meet these requirements? (Choose two.)

A.AWS Fargate
B.AWS Lambda
C.Amazon DynamoDB
D.Amazon EC2 Auto Scaling
E.MySQL-compatible Amazon Aurora

Answer: BC

Explanation:

Here's a detailed justification for why AWS Lambda and Amazon DynamoDB are the best choices for the described scenario:

AWS Lambda: Lambda excels at handling unpredictable and sudden spikes in traffic. It's a serverless compute service, meaning it automatically scales in response to incoming requests without requiring manual intervention or capacity provisioning. This aligns perfectly with the requirement for handling request patterns that can change drastically. Lambda functions are triggered by API Gateway, processing requests as they arrive and scaling up or down as needed. Fargate, while also scalable, requires containerization and more operational overhead. EC2 Auto Scaling requires provisioning and managing virtual machines, adding unnecessary complexity for this use case.

Supporting Concept: Serverless computing and event-driven architecture.
Authoritative Link: https://aws.amazon.com/lambda/

Amazon DynamoDB: DynamoDB is a NoSQL database service designed for key-value and document data models. It's highly scalable and can handle unpredictable growth in data size. DynamoDB scales horizontally and automatically, without requiring manual intervention. Its ability to handle simple key-value queries efficiently matches the data access pattern described. The current data size of less than 1 GB is easily accommodated, and its ability to scale allows it to handle unpredictable future growth. MySQL-compatible Aurora, while a good relational database option, introduces more overhead than needed for simple key-value queries. Additionally, the auto-scaling capabilities of DynamoDB are better suited for unpredictable growth compared to Aurora's scaling mechanisms in this simplified case.

Supporting Concept: NoSQL database, horizontal scaling, key-value data model.
Authoritative Link: https://aws.amazon.com/dynamodb/

Therefore, the combination of AWS Lambda for processing unpredictable request volumes from API Gateway and Amazon DynamoDB for storing and querying data using a simple key-value approach provides the most efficient and scalable solution.

Question: 638 **Exam Heist**

A company collects and shares research data with the company's employees all over the world. The company wants to collect and store the data in an Amazon S3 bucket and process the data in the AWS Cloud. The company will share the data with the company's employees. The company needs a secure solution in the AWS Cloud that minimizes operational overhead.

Which solution will meet these requirements?

A.Use an AWS Lambda function to create an S3 presigned URL. Instruct employees to use the URL.

B.Create an IAM user for each employee. Create an IAM policy for each employee to allow S3 access. Instruct employees to use the AWS Management Console.

C.Create an S3 File Gateway. Create a share for uploading and a share for downloading. Allow employees to mount shares on their local computers to use S3 File Gateway.

D.Configure AWS Transfer Family SFTP endpoints. Select the custom identity provider options. Use AWS Secrets Manager to manage the user credentials Instruct employees to use Transfer Family.

Answer: A

Explanation:

The most suitable solution is **A: Use an AWS Lambda function to create an S3 presigned URL. Instruct employees to use the URL.**

Here's why:

Security: Presigned URLs provide time-limited, temporary access to specific S3 objects. This is much more secure than long-term credentials. Each URL grants access only for a specific object and a defined duration, reducing the risk of unauthorized access if the URL is compromised.
Minimal Operational Overhead: A Lambda function can be triggered to generate these URLs on demand. This automated process reduces the administrative burden of managing individual IAM users and policies. It avoids the complexity of managing infrastructure.
Scalability: Lambda functions scale automatically based on demand, making the solution suitable for a fluctuating number of employees and data access requests.
Data Sharing: Presigned URLs allow the company to share data with employees without granting them permanent access to the S3 bucket.
IAM User Management Avoidance: Option B, creating IAM users for each employee, is cumbersome and introduces significant operational overhead for user management, policy updates, and credential rotation.
S3 File Gateway Limitation: Option C, S3 File Gateway, is designed to integrate on-premises applications with S3 storage. While it provides file-based access, it is more complex to set up and maintain for simple data sharing with employees compared to presigned URLs. File Gateway requires additional infrastructure on the on-premise side to mount the drives.
AWS Transfer Family SFTP Overhead: Option D, Transfer Family, is best suited for secure file transfers, often for business-to-business scenarios. Implementing a custom identity provider with Secrets Manager adds complexity that is unnecessary for this simple internal data sharing use case. Transfer Family also incurs more operational overhead in managing users and SFTP endpoints.

In summary, using Lambda with S3 presigned URLs offers a balanced solution that prioritizes security,

minimizes operational overhead, and efficiently enables data sharing with the company's employees.

Supporting Links:

S3 Presigned URLs: https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-urls.html
AWS Lambda: https://aws.amazon.com/lambda/

Question: 639 **Exam Heist**

A company is building a new furniture inventory application. The company has deployed the application on a fleet ofAmazon EC2 instances across multiple Availability Zones. The EC2 instances run behind an Application Load Balancer (ALB) in their VPC.

A solutions architect has observed that incoming traffic seems to favor one EC2 instance, resulting in latency for some requests.

What should the solutions architect do to resolve this issue?

A.Disable session affinity (sticky sessions) on the ALB
B.Replace the ALB with a Network Load Balancer
C.Increase the number of EC2 instances in each Availability Zone
D.Adjust the frequency of the health checks on the ALB's target group

Answer: A

Explanation:

The issue is that traffic favors one EC2 instance, leading to latency. This suggests an uneven distribution of requests across the available instances.

Option A, disabling session affinity (sticky sessions) on the ALB, is the best solution. Session affinity, when enabled, directs all requests from a specific user session to the same EC2 instance. This can cause imbalances if some users are more active than others, leading to overloaded instances while others remain underutilized. Disabling sticky sessions ensures that the ALB distributes traffic more evenly across all registered instances based on its configured load-balancing algorithm (e.g., round robin). This promotes better resource utilization and reduces latency.

Option B, replacing the ALB with a Network Load Balancer (NLB), is not appropriate. NLBs are designed for high-performance, low-latency traffic such as TCP, UDP, and TLS. They do not provide the application-level features of ALBs, such as content-based routing or host-based routing, and would not directly address the session affinity issue.

Option C, increasing the number of EC2 instances, might alleviate the problem somewhat by increasing the overall capacity. However, it does not address the root cause of uneven distribution due to sticky sessions and could be a less cost-effective solution than simply disabling sticky sessions.

Option D, adjusting health check frequency, is irrelevant to the observed traffic imbalance. Health checks determine the availability of instances, but they do not control how traffic is distributed among healthy instances.

Therefore, disabling sticky sessions (Option A) is the most direct and efficient way to address the load imbalance observed on the EC2 instances behind the ALB, ensuring a more even distribution of traffic and reduced latency.

Further Reading:

Application Load Balancer Concepts:

https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html
ALB Target Group Attributes (Sticky Sessions):
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/target-group-attributes.html

Question: 640 **Exam Heist**

A company has an application workflow that uses an AWS Lambda function to download and decrypt files from Amazon S3. These files are encrypted using AWS Key Management Service (AWS KMS) keys. A solutions architect needs to design a solution that will ensure the required permissions are set correctly.

Which combination of actions accomplish this? (Choose two.)

A.Attach the kms:decrypt permission to the Lambda function’s resource policy
B.Grant the decrypt permission for the Lambda IAM role in the KMS key's policy
C.Grant the decrypt permission for the Lambda resource policy in the KMS key's policy.
D.Create a new IAM policy with the kms:decrypt permission and attach the policy to the Lambda function.
E.Create a new IAM role with the kms:decrypt permission and attach the execution role to the Lambda function.

Answer: BE

Explanation:

The correct answer is BE. Here's a detailed justification:

The Lambda function needs permission to decrypt the files encrypted with KMS. This involves configuring appropriate IAM policies and KMS key policies. Two actions are crucial:

B. Grant the decrypt permission for the Lambda IAM role in the KMS key's policy: The KMS key policy controls who can use the key. The Lambda function needs the kms:Decrypt permission to decrypt the S3 files. This permission is granted by modifying the KMS key's policy to explicitly allow the Lambda function's IAM role to perform the kms:Decrypt action. This grants the Lambda function access to use the KMS key for decryption.

AWS KMS Key Policies

E. Create a new IAM role with the kms:decrypt permission and attach the execution role to the Lambda function: The Lambda function executes with an IAM role. To ensure the Lambda function has the necessary permissions to perform actions on AWS services, like KMS, you create an IAM role that grants these permissions. In this case, a new IAM role with the kms:Decrypt permission would be created. The kms:decrypt permission is granted to the IAM role in an IAM policy. Then, attach this execution role to the Lambda function.

IAM Roles for Lambda

Let's examine why other options are incorrect:

A. Attach the kms:decrypt permission to the Lambda function’s resource policy: Lambda resource policies are primarily used to grant other AWS services permission to invoke the Lambda function. They are not the primary mechanism for granting the Lambda function itself permissions to access other services like KMS. The Lambda function needs an IAM role, not just a resource policy, to execute with permissions.

C. Grant the decrypt permission for the Lambda resource policy in the KMS key's policy: As in A, the Lambda resource policy focuses on invocation by other services, not the Lambda function's permissions to access other AWS services. Adding the Lambda resource policy to the KMS key policy won't give the function the ability to use KMS to decrypt.

D. Create a new IAM policy with the kms:decrypt permission and attach the policy to the Lambda function: While creating an IAM policy with kms:decrypt is correct, attaching the policy to the Lambda function is less common and often confusing. It's best practice to attach the IAM policy to an IAM role, then assign the role to the Lambda function. Lambda functions are typically associated with IAM roles as their execution context rather than attaching policies directly. Option E correctly captures this.

In summary, granting the Lambda function's IAM role the kms:Decrypt permission in both the IAM policy and the KMS key policy ensures that the function is authorized to decrypt the files using the KMS key. This is the standard method for granting permissions between services in AWS, adhering to the principle of least privilege.

Question: 641 **Exam Heist**

A company wants to monitor its AWS costs for financial review. The cloud operations team is designing an architecture in the AWS Organizations management account to query AWS Cost and Usage Reports for all member accounts. The team must run this query once a month and provide a detailed analysis of the bill.

Which solution is the MOST scalable and cost-effective way to meet these requirements?

A.Enable Cost and Usage Reports in the management account. Deliver reports to Amazon Kinesis. Use Amazon EMR for analysis.

B.Enable Cost and Usage Reports in the management account. Deliver the reports to Amazon S3 Use Amazon Athena for analysis.

C.Enable Cost and Usage Reports for member accounts. Deliver the reports to Amazon S3 Use Amazon Redshift for analysis.

D.Enable Cost and Usage Reports for member accounts. Deliver the reports to Amazon Kinesis. Use Amazon QuickSight tor analysis.

Answer: B

Explanation:

The most scalable and cost-effective solution is to use Cost and Usage Reports (CUR) with S3 and Athena.

Here's why:

Cost and Usage Reports (CUR): CUR provides detailed information about AWS costs and usage. Enabling it in the management account consolidates data from all member accounts in AWS Organizations, fulfilling the requirement to analyze costs across the organization.
https://docs.aws.amazon.com/cur/latest/userguide/what-is-cur.html
Amazon S3: Storing the CUR in S3 offers a highly scalable and cost-effective storage solution. S3 provides durable and readily available storage for the monthly reports. https://aws.amazon.com/s3/
Amazon Athena: Athena is a serverless query service that enables analyzing data directly in S3 using standard SQL. It is ideal for ad-hoc querying and monthly analysis of the CUR data without the need to manage any infrastructure. Athena is pay-per-query, making it cost-effective for a monthly analysis.
https://aws.amazon.com/athena/

Let's analyze the other options:

Option A (Kinesis and EMR): Kinesis is designed for real-time data streaming, which isn't necessary for a monthly cost analysis. EMR (Elastic MapReduce) is suitable for large-scale data processing, but it is an overkill for monthly CUR analysis and more costly than Athena for this specific use case.
Option C (Redshift): Redshift is a data warehouse service, which is ideal for complex data warehousing scenarios. Using Redshift would be much more costly and complex than required for the monthly analysis of CUR.

Option D (Kinesis and QuickSight): As with option A, Kinesis is not necessary here. QuickSight is a BI tool suitable for visualizing data, but the raw CUR needs to be queried and prepared for analysis first, which Athena efficiently handles. QuickSight can complement Athena, but it does not replace Athena's role in efficiently querying the raw CUR data stored in S3.

Therefore, Option B delivers the best balance of scalability, cost-effectiveness, and simplicity for monthly cost analysis within an AWS Organizations environment. It allows the cloud operations team to easily analyze the CUR data and generate detailed billing analyses.

Question: 642 **Exam Heist**

A company wants to run a gaming application on Amazon EC2 instances that are part of an Auto Scaling group in the AWS Cloud. The application will transmit data by using UDP packets. The company wants to ensure that the application can scale out and in as traffic increases and decreases.

What should a solutions architect do to meet these requirements?

A.Attach a Network Load Balancer to the Auto Scaling group.
B.Attach an Application Load Balancer to the Auto Scaling group.
C.Deploy an Amazon Route 53 record set with a weighted policy to route traffic appropriately.
D.Deploy a NAT instance that is configured with port forwarding to the EC2 instances in the Auto Scaling group.

Answer: A

Explanation:

The correct answer is **A. Attach a Network Load Balancer to the Auto Scaling group.**

Here's a detailed justification:

UDP Protocol Support: Network Load Balancers (NLBs) are designed to handle UDP traffic efficiently. Application Load Balancers (ALBs), on the other hand, primarily work with HTTP/HTTPS (Layer 7) protocols and do not support UDP. Since the gaming application uses UDP packets, an NLB is the appropriate choice.

Auto Scaling Integration: NLBs seamlessly integrate with Auto Scaling groups. As the Auto Scaling group scales instances in or out, the NLB automatically updates its target group to include the new instances and remove the terminated ones, ensuring traffic is always routed to healthy instances. This is essential for the application to scale effectively based on traffic demand.

High Performance and Low Latency: NLBs offer high throughput and low latency, making them well-suited for performance-sensitive applications like online gaming where responsiveness is critical. They operate at Layer 4 (Transport Layer), forwarding traffic based on IP addresses and ports.

Route 53 Weighted Policy Inadequacy: While Route 53 can distribute traffic across multiple endpoints, it operates at the DNS level. It doesn't provide health checks at the instance level or automatically adjust routing based on the dynamic scaling of instances within an Auto Scaling group. This option does not fulfill the automatic scaling requirement efficiently. It's also not UDP specific and doesn't replace a load balancer for instance health management.

NAT Instance Inefficiency: Using a NAT instance for port forwarding is a complex, single point of failure, and scaling bottleneck. NAT instances are typically used for enabling private instances to access the internet, not for load balancing or traffic distribution in a highly available manner. Also, manually configuring port forwarding on a NAT instance is not a scalable solution and would require continuous adjustments as the Auto Scaling group changes.

In summary, attaching a Network Load Balancer to the Auto Scaling group provides the necessary UDP support, seamless integration with Auto Scaling, high performance, and automatic target registration, making it the most appropriate solution for the gaming application's requirements.

Supporting Documentation:

AWS Network Load Balancer
Auto Scaling Groups

Question: 643 **Exam Heist**

A company runs several websites on AWS for its different brands. Each website generates tens of gigabytes of web traffic logs each day. A solutions architect needs to design a scalable solution to give the company's developers the ability to analyze traffic patterns across all the company's websites. This analysis by the developers will occur on demand once a week over the course of several months. The solution must support queries with standard SQL.

Which solution will meet these requirements MOST cost-effectively?

A.Store the logs in Amazon S3. Use Amazon Athena tor analysis.
B.Store the logs in Amazon RDS. Use a database client for analysis.
C.Store the logs in Amazon OpenSearch Service. Use OpenSearch Service for analysis.
D.Store the logs in an Amazon EMR cluster Use a supported open-source framework for SQL-based analysis.

Answer: A

Explanation:

The most cost-effective solution is to use Amazon S3 and Amazon Athena (Option A) for this scenario because it directly addresses the requirements of scalable storage, on-demand SQL analysis, and cost optimization.

Here's why:

Amazon S3 for Scalable Storage: S3 provides virtually unlimited storage at a low cost. It's designed for storing large amounts of data like web traffic logs. S3's durability and scalability make it ideal for this purpose. https://aws.amazon.com/s3/

Amazon Athena for SQL Analysis: Athena is a serverless query service that allows you to analyze data in S3 using standard SQL. You only pay for the queries you run, which is perfect for infrequent, on-demand analysis. It avoids the costs of maintaining a dedicated database or cluster. https://aws.amazon.com/athena/

Cost-Effectiveness: Since the analysis occurs only once a week for a few months, Athena's pay-per-query model is significantly cheaper than running a persistent database or cluster. S3's storage costs are also very low.

Why other options are less suitable:

Amazon RDS (Option B): RDS is a managed relational database service. Storing tens of gigabytes of logs daily in a database and then using a database client would be far more expensive than using S3 for storage. The RDS instance would need to be sized appropriately to handle the volume of data and SQL queries. This is also less scalable for handling rapidly growing amounts of logs. The cost of keeping the RDS instance active even when not in use would not be cost-effective.

Amazon OpenSearch Service (Option C): OpenSearch is suitable for log analytics, especially when real-time or near-real-time insights are needed. However, for infrequent analysis, the ongoing costs of running an OpenSearch cluster would be higher than using Athena, since you are billed continuously for OpenSearch

even when it's not performing queries.

Amazon EMR (Option D): EMR is suitable for large-scale data processing and analytics using frameworks like Hadoop and Spark. While EMR can support SQL-based analysis, the cost of running an EMR cluster, even on-demand, is generally higher than using Athena for this specific infrequent analysis scenario. The overhead of starting up and managing an EMR cluster also contributes to higher operational complexity.

Question: 644 **Exam Heist**

An international company has a subdomain for each country that the company operates in. The subdomains are formatted as example.com, country1.example.com, and country2.example.com. The company's workloads are behind an Application Load Balancer. The company wants to encrypt the website data that is in transit.

Which combination of steps will meet these requirements? (Choose two.)

A.Use the AWS Certificate Manager (ACM) console to request a public certificate for the apex top domain example com and a wildcard certificate for *.example.com.

B.Use the AWS Certificate Manager (ACM) console to request a private certificate for the apex top domain example.com and a wildcard certificate for *.example.com.

C.Use the AWS Certificate Manager (ACM) console to request a public and private certificate for the apex top domain example.com.

D.Validate domain ownership by email address. Switch to DNS validation by adding the required DNS records to the DNS provider.

E.Validate domain ownership for the domain by adding the required DNS records to the DNS provider.

Answer: AE

Explanation:

The correct answer is AE. Here's a detailed justification:

*A: Use the AWS Certificate Manager (ACM) console to request a public certificate for the apex top domain example.com and a wildcard certificate for .example.com.

This step is essential because the company wants to encrypt website data in transit, requiring SSL/TLS certificates. A public certificate issued by ACM is needed for the Application Load Balancer (ALB) to establish secure HTTPS connections with clients. The apex domain example.com needs a separate certificate because the wildcard certificate (*.example.com*) only covers subdomains directly beneath *example.com*. The wildcard certificate .example.com will cover country1.example.com, country2.example.com, etc. A private certificate is not appropriate in this case because external clients need to trust the certificate authority, and private certificates are not inherently trusted by public browsers.

E: Validate domain ownership for the domain by adding the required DNS records to the DNS provider.

ACM requires validation of domain ownership before issuing a certificate. This proves that the requester has control over the domain they're requesting a certificate for. DNS validation is the recommended method. It involves adding a CNAME record provided by ACM to the DNS configuration of the domain. This is a more reliable and automated method than email validation, especially when dealing with wildcard certificates and multiple subdomains. With email validation, each subdomain will need to be validated separately. DNS validation only needs to be done once.

Why other options are incorrect:

B: Using a private certificate is incorrect because public clients (browsers) won't trust it. Private certificates are suitable for internal services where you control the client environment.

C: Requesting both public and private certificates for the apex domain is unnecessary and doesn't address the need for a wildcard certificate to cover all subdomains.
D: While validating domain ownership is necessary, switching from email validation to DNS validation is not strictly required. DNS validation is simply the recommended approach due to its automation and reliability. Email validation is also an acceptable alternative (though less preferred).

Supporting concepts and links:

SSL/TLS Certificates: Required for encrypting data in transit over HTTPS.https://docs.aws.amazon.com/acm/latest/userguide/acm-overview.html
AWS Certificate Manager (ACM): AWS service for provisioning, managing, and deploying SSL/TLS certificates.https://aws.amazon.com/certificate-manager/
Wildcard Certificates: Cover multiple subdomains with a single certificate.https://docs.aws.amazon.com/acm/latest/userguide/acm-overview.html
DNS Validation: Preferred method for validating domain ownership with ACM.https://docs.aws.amazon.com/acm/latest/userguide/dns-validation.html
Application Load Balancer (ALB): Load balancer that supports HTTPS listeners and uses ACM certificates for SSL/TLS termination.https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-listeners.html

Question: 645 **Exam Heist**

A company is required to use cryptographic keys in its on-premises key manager. The key manager is outside of the AWS Cloud because of regulatory and compliance requirements. The company wants to manage encryption and decryption by using cryptographic keys that are retained outside of the AWS Cloud and that support a variety of external key managers from different vendors.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use AWS CloudHSM key store backed by a CloudHSM cluster.
B.Use an AWS Key Management Service (AWS KMS) external key store backed by an external key manager.
C.Use the default AWS Key Management Service (AWS KMS) managed key store.
D.Use a custom key store backed by an AWS CloudHSM cluster.

Answer: B

Explanation:

The correct answer is B: Use an AWS Key Management Service (AWS KMS) external key store backed by an external key manager.

Here's why:

The core requirement is to use an on-premises key manager (outside AWS) due to regulatory compliance, while minimizing operational overhead. AWS KMS external key store (XKS) is specifically designed for this scenario. It allows you to use KMS to encrypt and decrypt data, but the actual cryptographic operations are performed by your own external key management infrastructure, which satisfies the compliance needs. KMS only holds metadata about the keys.

Option B achieves the required control and compliance with the least amount of custom development and management effort. It's a native integration provided by AWS KMS. You don't need to build and maintain custom software to interface with your on-premises key manager.

Option A (AWS CloudHSM key store) doesn't address the specific requirement of using an existing on-premises key manager. CloudHSM creates HSMs within AWS, not outside of it. While CloudHSM offers high

security, it necessitates managing HSMs within AWS, adding operational overhead.

Option C (default KMS managed key store) is incorrect because the keys are entirely managed by AWS KMS, which contradicts the requirement to retain keys outside of AWS.

Option D (custom key store backed by CloudHSM) requires significantly more configuration and operational overhead. Building a custom key store involves developing the integration between KMS and CloudHSM, and subsequently managing the underlying HSMs, which contradicts the goal of minimizing operational overhead. Furthermore, it does not address the need to use an existing on-premise key manager.

In summary, XKS is the AWS-provided solution that directly addresses the stated requirement of managing keys outside the AWS cloud while leveraging the KMS service, thereby minimizing operational overhead compared to managing CloudHSM or developing custom solutions.

Further Reading:

AWS KMS external key store: https://docs.aws.amazon.com/kms/latest/developerguide/keystore-external.html

Question: 646 **Exam Heist**

A solutions architect needs to host a high performance computing (HPC) workload in the AWS Cloud. The workload will run on hundreds of Amazon EC2 instances and will require parallel access to a shared file system to enable distributed processing of large datasets. Datasets will be accessed across multiple instances simultaneously. The workload requires access latency within 1 ms. After processing has completed, engineers will need access to the dataset for manual postprocessing.

Which solution will meet these requirements?

A.Use Amazon Elastic File System (Amazon EFS) as a shared file system. Access the dataset from Amazon EFS.

B.Mount an Amazon S3 bucket to serve as the shared file system. Perform postprocessing directly from the S3 bucket.

C.Use Amazon FSx for Lustre as a shared file system. Link the file system to an Amazon S3 bucket for postprocessing.

D.Configure AWS Resource Access Manager to share an Amazon S3 bucket so that it can be mounted to all instances for processing and postprocessing.

Answer: C

Explanation:

The correct answer is **C. Use Amazon FSx for Lustre as a shared file system. Link the file system to an Amazon S3 bucket for postprocessing.**

Here's why:

High-Performance Computing (HPC) needs: HPC workloads, involving hundreds of EC2 instances and parallel processing of large datasets, demand a high-performance, low-latency shared file system.
Amazon FSx for Lustre: This service is specifically designed for HPC and machine learning workloads. It provides sub-millisecond latency, which meets the 1 ms requirement, and delivers high throughput for parallel data access. https://aws.amazon.com/fsx/lustre/
Amazon S3 integration: FSx for Lustre can be linked to an S3 bucket. This enables the HPC workload to efficiently process data stored in S3 and provides a seamless way for engineers to access the processed dataset in S3 for postprocessing. Data can be exported easily and efficiently to S3 after the HPC job concludes.
Why not Amazon EFS (A): Amazon EFS provides a scalable, elastic file system, but it is not optimized for the

very low latency requirements (under 1 ms) that are typical of HPC workloads. While EFS is excellent for general-purpose file sharing, it doesn't offer the performance needed here.
Why not Amazon S3 as a file system (B & D): S3 is an object storage service, not a file system. Mounting S3 directly to EC2 instances via S3 fuse will introduce high latency and poor performance for parallel file system access. S3 is great for storage but does not provide the required parallel access speed for HPC workloads. AWS Resource Access Manager (RAM) allows sharing AWS resources, but it doesn't change the fundamental characteristics of S3, which is still not suitable for this purpose.

In summary, FSx for Lustre provides the necessary low latency and high throughput for the HPC workload, while the S3 integration facilitates efficient data access and postprocessing.

Question: 647 **Exam Heist**

A gaming company is building an application with Voice over IP capabilities. The application will serve traffic to users across the world. The application needs to be highly available with an automated failover across AWS Regions. The company wants to minimize the latency of users without relying on IP address caching on user devices.

What should a solutions architect do to meet these requirements?

A.Use AWS Global Accelerator with health checks.
B.Use Amazon Route 53 with a geolocation routing policy.
C.Create an Amazon CloudFront distribution that includes multiple origins.
D.Create an Application Load Balancer that uses path-based routing.

Answer: C

Explanation:

The correct answer is **C. Create an Amazon CloudFront distribution that includes multiple origins.**

Here's a detailed justification:

CloudFront is a content delivery network (CDN) that excels at minimizing latency for users across the globe. By setting up multiple origins (representing application endpoints in different AWS Regions) behind a CloudFront distribution, the company can leverage CloudFront's global network of edge locations. CloudFront automatically routes user requests to the nearest available origin (the one with the lowest latency), thereby improving the user experience. This is crucial for VoIP applications where latency is very sensitive.

CloudFront can be configured to use origin failover. In this approach, CloudFront monitors the health of your origins. If the primary origin becomes unhealthy, CloudFront automatically switches traffic to a secondary origin located in a different region. This achieves the high availability and automated failover requirements.

Option A is incorrect because AWS Global Accelerator is a great option for directing traffic at the regional level. It minimizes latency by routing user traffic to the optimal AWS endpoint, but requires you to configure failover within the regional endpoint (e.g. through Elastic Load Balancing). For simple application servers, this approach is not as well suited to the needs of this application.

Option B is incorrect because while Route 53 with geolocation routing can direct users to specific Regions, it relies on DNS caching at the user's resolver. This caching can lead to users being directed to a failed region if their DNS record has not expired yet (DNS TTL). Since VoIP apps are sensitive to failures, you cannot rely on DNS caching.

Option D is incorrect because Application Load Balancers (ALBs) are regional resources. Although ALBs support routing based on path, host headers etc, they do not natively provide global distribution or automated

cross-region failover in the same way as CloudFront or Global Accelerator. They also won't route traffic to different origins based on latency considerations. You can have multiple ALBs routing to different regions and create a regional fallback with Lambda, but this does not eliminate the need for a global CDN like CloudFront.

In summary, CloudFront with multiple origins and origin failover directly addresses all the requirements (low latency, high availability, automated failover, and global distribution).

Relevant Links:

Amazon CloudFront documentation
Configuring Origin Failover in CloudFront

Question: 648 **Exam Heist**

A weather forecasting company needs to process hundreds of gigabytes of data with sub-millisecond latency. The company has a high performance computing (HPC) environment in its data center and wants to expand its forecasting capabilities.

A solutions architect must identify a highly available cloud storage solution that can handle large amounts of sustained throughput. Files that are stored in the solution should be accessible to thousands of compute instances that will simultaneously access and process the entire dataset.

What should the solutions architect do to meet these requirements?

A.Use Amazon FSx for Lustre scratch file systems.
B.Use Amazon FSx for Lustre persistent file systems.
C.Use Amazon Elastic File System (Amazon EFS) with Bursting Throughput mode.
D.Use Amazon Elastic File System (Amazon EFS) with Provisioned Throughput mode.

Answer: B

Explanation:

The correct answer is **B. Use Amazon FSx for Lustre persistent file systems.**

Here's why:

High Throughput and Low Latency: The core requirement is to handle "large amounts of sustained throughput" with "sub-millisecond latency." FSx for Lustre, particularly its persistent file systems, is designed for high-performance workloads. It excels in providing the necessary throughput and low latency.

Parallel Access: The problem states that "thousands of compute instances" need to "simultaneously access and process the entire dataset." FSx for Lustre is a parallel file system, meaning it's built to handle concurrent access from numerous compute instances without significant performance degradation.

HPC Compatibility: The company already uses an HPC environment. FSx for Lustre is a natural extension of that, as it's commonly used for HPC workloads like weather forecasting, genomic research, and financial modeling. Its architecture and optimizations align with the needs of these kinds of environments.

Persistent vs. Scratch: While FSx for Lustre offers both scratch and persistent file systems, persistent is better for this scenario. Scratch file systems are optimized for short-term storage and rapid creation/deletion, while persistent file systems provide durable storage and are suitable for workloads requiring long-term data access and availability. The problem doesn't mention data being short-lived, rather it emphasizes accessibility. Persistent FSx for Lustre instances can be deployed across multiple availability zones for fault tolerance and recovery.

EFS Limitations: Amazon EFS, even with provisioned throughput, is generally not designed to handle the

same extreme levels of sustained throughput and low latency as FSx for Lustre. While EFS is a good choice for many file-sharing workloads, it's not the best option when sub-millisecond latency and extremely high throughput are critical. It offers lower peak and sustained performance compared to FSx for Lustre. EFS Bursting throughput is also unideal because it would likely require significant capacity and bursts to handle "hundreds of gigabytes of data", and doesn't explicitly support the high performance required in the question.

Authoritative Links:

Amazon FSx for Lustre: https://aws.amazon.com/fsx/lustre/
Amazon EFS: https://aws.amazon.com/efs/

Question: 649 **Exam Heist**

An ecommerce company runs a PostgreSQL database on premises. The database stores data by using high IOPS Amazon Elastic Block Store (Amazon EBS) block storage. The daily peak I/O transactions per second do not exceed 15,000 IOPS. The company wants to migrate the database to Amazon RDS for PostgreSQL and provision disk IOPS performance independent of disk storage capacity.

Which solution will meet these requirements MOST cost-effectively?

A.Configure the General Purpose SSD (gp2) EBS volume storage type and provision 15,000 IOPS.
B.Configure the Provisioned IOPS SSD (io1) EBS volume storage type and provision 15,000 IOPS.
C.Configure the General Purpose SSD (gp3) EBS volume storage type and provision 15,000 IOPS.
D.Configure the EBS magnetic volume type to achieve maximum IOPS.

Answer: C

Explanation:

The company needs a cost-effective solution for migrating their PostgreSQL database to Amazon RDS for PostgreSQL while ensuring they can handle up to 15,000 IOPS independently of storage capacity. Let's analyze why option C is the most appropriate:

Option A: gp2: gp2 volume performance is tied to its size. Reaching 15,000 IOPS with gp2 would require provisioning a large volume, potentially leading to unnecessary storage costs since the requirement is to optimize IOPS performance independent of storage capacity.

Option B: io1: While io1 allows you to provision IOPS independently of storage, it is generally more expensive per IOPS than gp3, especially for moderate IOPS requirements.

Option C: gp3: gp3 volumes provide a baseline performance level (3,000 IOPS) and allow you to provision additional IOPS separately from storage capacity. This means the company can provision the necessary IOPS without having to over-provision storage, resulting in cost savings. In many AWS Regions gp3 supports provisioned IOPS beyond its baseline without increasing the provisioned storage capacity. Also, gp3 provides higher performance than gp2 for a similar price.

Option D: EBS Magnetic: Magnetic volumes offer the lowest cost but have significantly lower IOPS performance capabilities. They would be completely unsuitable for the stated performance requirement of 15,000 IOPS. Magnetic volumes are unsuitable for databases due to their poor and inconsistent performance.

Therefore, gp3 offers the best balance of cost and performance for the requirement. gp3 enables independent provisioning of IOPS and storage and has a lower cost compared to io1.

Supporting Documentation:

Amazon EBS Volume Types: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html
General Purpose SSD (gp3) volumes https://aws.amazon.com/ebs/general-purpose/

Question: 650 **Exam Heist**

A company wants to migrate its on-premises Microsoft SQL Server Enterprise edition database to AWS. The company's online application uses the database to process transactions. The data analysis team uses the same production database to run reports for analytical processing. The company wants to reduce operational overhead by moving to managed services wherever possible.

Which solution will meet these requirements with the LEAST operational overhead?

A.Migrate to Amazon RDS for Microsoft SOL Server. Use read replicas for reporting purposes
B.Migrate to Microsoft SQL Server on Amazon EC2. Use Always On read replicas for reporting purposes
C.Migrate to Amazon DynamoDB. Use DynamoDB on-demand replicas for reporting purposes
D.Migrate to Amazon Aurora MySQL. Use Aurora read replicas for reporting purposes

Answer: A

Explanation:

Here's a detailed justification for why option A is the best solution, along with supporting explanations and links:

The key requirements are migrating an on-premises Microsoft SQL Server Enterprise edition database to AWS with minimal operational overhead, supporting both transactional processing for an online application and analytical reporting, and leveraging managed services where feasible.

Option A, migrating to Amazon RDS for Microsoft SQL Server and using read replicas for reporting, is the optimal choice because RDS is a managed service. This significantly reduces operational overhead as AWS handles patching, backups, and database management tasks. Using read replicas offloads analytical reporting from the primary database, preventing performance impact on the online application's transactional workload. RDS for SQL Server allows you to continue to use familiar SQL Server functionality.

Option B, using SQL Server on EC2, increases operational overhead because you are responsible for managing the operating system, SQL Server installation, patching, and backups. While Always On Availability Groups provide read replicas, the operational burden is significantly higher compared to using RDS.

Option C, migrating to Amazon DynamoDB, would require a significant application rewrite since DynamoDB is a NoSQL database, which is not compatible with the existing SQL Server database schema and SQL queries. DynamoDB on-demand backups are not the intended replica function for reporting. This leads to a much greater effort required to convert the application.

Option D, migrating to Amazon Aurora MySQL, also necessitates a database migration and application rewrite. While Aurora offers high performance and read replicas, the migration process is a substantial undertaking, which contrasts with the need to minimize overhead and maintain the business functionality.

Therefore, RDS provides the least operational overhead by being a managed service. It allows the company to continue to use MS SQL Server, which avoids a costly and complex migration. The use of read replicas maintains application performance.

Supporting Links:

Amazon RDS: https://aws.amazon.com/rds/
Amazon RDS for SQL Server: https://aws.amazon.com/rds/sqlserver/

RDS Read Replicas: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ReadRepl.html

Question: 651 **Exam Heist**

A company stores a large volume of image files in an Amazon S3 bucket. The images need to be readily available for the first 180 days. The images are infrequently accessed for the next 180 days. After 360 days, the images need to be archived but must be available instantly upon request. After 5 years, only auditors can access the images. The auditors must be able to retrieve the images within 12 hours. The images cannot be lost during this process.

A developer will use S3 Standard storage for the first 180 days. The developer needs to configure an S3 Lifecycle rule.

Which solution will meet these requirements MOST cost-effectively?

A.Transition the objects to S3 One Zone-Infrequent Access (S3 One Zone-IA) after 180 days. S3 Glacier Instant Retrieval after 360 days, and S3 Glacier Deep Archive after 5 years.

B.Transition the objects to S3 One Zone-Infrequent Access (S3 One Zone-IA) after 180 days. S3 Glacier Flexible Retrieval after 360 days, and S3 Glacier Deep Archive after 5 years.

C.Transition the objects to S3 Standard-Infrequent Access (S3 Standard-IA) after 180 days, S3 Glacier Instant Retrieval after 360 days, and S3 Glacier Deep Archive after 5 years.

D.Transition the objects to S3 Standard-Infrequent Access (S3 Standard-IA) after 180 days, S3 Glacier Flexible Retrieval after 360 days, and S3 Glacier Deep Archive after 5 years.

Answer: C

Explanation:

The correct answer is **C. Transition the objects to S3 Standard-Infrequent Access (S3 Standard-IA) after 180 days, S3 Glacier Instant Retrieval after 360 days, and S3 Glacier Deep Archive after 5 years.**

Here's a detailed justification:

The problem states that the images need to be readily available for the first 180 days, and S3 Standard fulfills this requirement. The subsequent stages require different access patterns and archival needs, influencing the choice of storage classes for the S3 Lifecycle rule.

180 Days to 360 Days (Infrequent Access): S3 Standard-IA is the most cost-effective choice for data that is accessed infrequently but requires rapid access when needed. S3 One Zone-IA has lower availability (data loss can occur) than S3 Standard-IA because data is stored in a single availability zone. Since data cannot be lost during the process this eliminates options A and B.

360 Days to 5 Years (Instant Retrieval): The problem explicitly states that after 360 days the images must be "available instantly upon request." S3 Glacier Instant Retrieval delivers the lowest-cost storage for long-term data that is accessed once a quarter, with the same low latency performance as S3 Standard and S3 Standard-IA. S3 Glacier Flexible Retrieval does not meet this requirement since retrieval times can vary from minutes to hours.

After 5 Years (Archival with 12-Hour Retrieval): S3 Glacier Deep Archive is the lowest-cost storage class suitable for long-term archival. The 12-hour retrieval window aligns with the retrieval capabilities of S3 Glacier Deep Archive, which has retrieval times ranging from 12 hours.

Therefore, configuring an S3 Lifecycle rule to transition objects as follows:

1. S3 Standard for the first 180 days (already configured).
2. S3 Standard-IA after 180 days.
3. S3 Glacier Instant Retrieval after 360 days.

4. S3 Glacier Deep Archive after 5 years.

This solution adheres to all requirements of the question while minimizing storage costs.

Here are some authoritative links for further research:

Amazon S3 Storage Classes
Managing your storage lifecycle

Question: 652 **Exam Heist**

A company has a large data workload that runs for 6 hours each day. The company cannot lose any data while the process is running. A solutions architect is designing an Amazon EMR cluster configuration to support this critical data workload.

Which solution will meet these requirements MOST cost-effectively?

A.Configure a long-running cluster that runs the primary node and core nodes on On-Demand Instances and the task nodes on Spot Instances.

B.Configure a transient cluster that runs the primary node and core nodes on On-Demand Instances and the task nodes on Spot Instances.

C.Configure a transient cluster that runs the primary node on an On-Demand Instance and the core nodes and task nodes on Spot Instances.

D.Configure a long-running cluster that runs the primary node on an On-Demand Instance, the core nodes on Spot Instances, and the task nodes on Spot Instances.

Answer: B

Explanation:

The correct answer is B. Here's a detailed justification:

The requirement is to run a 6-hour workload daily without data loss and in a cost-effective manner using Amazon EMR. A transient cluster is best suited for this because it automatically provisions, executes the workload, and terminates after completion, minimizing idle time and associated costs.

Option B suggests running the primary and core nodes on On-Demand Instances and the task nodes on Spot Instances. On-Demand Instances for the primary and core nodes provide predictable availability during the 6-hour workload. The primary node is critical for cluster management, and core nodes typically store data (HDFS). Losing these nodes during processing could lead to data loss and job failure. Thus, utilizing On-Demand Instances here reduces that risk.

Task nodes perform compute tasks and do not store persistent data. Therefore, using Spot Instances for task nodes can significantly reduce costs. If a Spot Instance is interrupted, the task will be rescheduled on another available instance without affecting the overall data integrity as data isn't stored persistently on these nodes. EMR is fault-tolerant and can handle task failures due to Spot Instance revocations.

Options A and D propose long-running clusters. Keeping the cluster alive even when it's not processing the workload results in unnecessary expenses. Option C suggests using Spot Instances for core nodes, which store data. Loss of core nodes leads to potential data loss, violating the critical requirement of the workload.

In summary, Option B provides the optimal balance between cost and reliability by leveraging On-Demand Instances for data-critical nodes (primary, core) and Spot Instances for compute-intensive tasks (task nodes), utilizing the benefits of transient clusters for cost optimization and ephemeral workloads.

Further Research:

Amazon EMR Instance Fleets and Instance Groups:
https://docs.aws.amazon.com/emr/latest/ManagementGuide/emr-instance-fleet.html
Amazon EMR Pricing: https://aws.amazon.com/emr/pricing/
AWS Spot Instances: https://aws.amazon.com/ec2/spot/

Question: 653 **Exam Heist**

A company maintains an Amazon RDS database that maps users to cost centers. The company has accounts in an organization in AWS Organizations. The company needs a solution that will tag all resources that are created in a specific AWS account in the organization. The solution must tag each resource with the cost center ID of the user who created the resource.

Which solution will meet these requirements?

A.Move the specific AWS account to a new organizational unit (OU) in Organizations from the management account. Create a service control policy (SCP) that requires all existing resources to have the correct cost center tag before the resources are created. Apply the SCP to the new OU.

B.Create an AWS Lambda function to tag the resources after the Lambda function looks up the appropriate cost center from the RDS database. Configure an Amazon EventBridge rule that reacts to AWS CloudTrail events to invoke the Lambda function.

C.Create an AWS CloudFormation stack to deploy an AWS Lambda function. Configure the Lambda function to look up the appropriate cost center from the RDS database and to tag resources. Create an Amazon EventBridge scheduled rule to invoke the CloudFormation stack.

D.Create an AWS Lambda function to tag the resources with a default value. Configure an Amazon EventBridge rule that reacts to AWS CloudTrail events to invoke the Lambda function when a resource is missing the cost center tag.

Answer: B

Explanation:

The correct answer is B. Here's why:

Option B leverages a real-time, event-driven approach using Lambda and EventBridge, which aligns perfectly with the requirement to tag resources immediately after creation. AWS Lambda is a serverless compute service that allows you to run code without provisioning or managing servers, making it ideal for on-demand tasks like tagging. Amazon EventBridge, a serverless event bus, can be configured to trigger Lambda functions based on events, such as the creation of an AWS resource.

The solution works as follows:

1. **CloudTrail Event:** AWS CloudTrail logs API calls made in the AWS account. When a resource is created, CloudTrail captures this event.
2. **EventBridge Rule:** An EventBridge rule is set up to listen for CloudTrail events related to resource creation (e.g., CreateVolume, CreateInstance). When such an event occurs, the rule is triggered.
3. **Lambda Function:** The EventBridge rule invokes an AWS Lambda function. This function receives the event data, which includes the user's identity (via the AWS IAM user or role that initiated the API call) and the resource ARN (Amazon Resource Name).
4. **Cost Center Lookup:** The Lambda function uses the user's identity to query the RDS database (mapping users to cost centers) to retrieve the correct cost center ID.
5. **Resource Tagging:** The Lambda function then uses the AWS SDK (e.g., boto3 for Python) to tag the created resource with the CostCenter tag, setting its value to the retrieved cost center ID. The function uses the resource ARN from the CloudTrail event data to identify the specific resource to tag.

This approach ensures that resources are tagged immediately after creation with the appropriate cost center

ID based on the user who created them. The solution is fully automated and requires no manual intervention.

Option A is incorrect because Service Control Policies (SCPs) primarily enforce permission boundaries across accounts in an organization and do not directly tag resources based on user information stored in a database. SCPs can require tags to be present, but not dynamically set them. Furthermore, applying an SCP to an existing resource won't retroactively add tags based on user information.

Option C is inefficient. Using a scheduled EventBridge rule to invoke a CloudFormation stack that then runs a Lambda function adds unnecessary complexity. The scheduled invocation is not event-driven or real-time, therefore it is more likely to cause delay in tagging.

Option D is insufficient because it relies on a default value and only attempts to tag resources that are missing the cost center tag. This approach is reactive rather than proactive and does not guarantee that all resources will be correctly tagged with the appropriate cost center from the beginning. It also means that a resource will initially be created with the wrong tag, which is then corrected later.

Authoritative Links:

AWS Lambda: https://aws.amazon.com/lambda/
Amazon EventBridge: https://aws.amazon.com/eventbridge/
AWS CloudTrail: https://aws.amazon.com/cloudtrail/
AWS Organizations: https://aws.amazon.com/organizations/

Question: 654 **Exam Heist**

A company recently migrated its web application to the AWS Cloud. The company uses an Amazon EC2 instance to run multiple processes to host the application. The processes include an Apache web server that serves static content. The Apache web server makes requests to a PHP application that uses a local Redis server for user sessions.

The company wants to redesign the architecture to be highly available and to use AWS managed solutions.

Which solution will meet these requirements?

A.Use AWS Elastic Beanstalk to host the static content and the PHP application. Configure Elastic Beanstalk to deploy its EC2 instance into a public subnet. Assign a public IP address.

B.Use AWS Lambda to host the static content and the PHP application. Use an Amazon API Gateway REST API to proxy requests to the Lambda function. Set the API Gateway CORS configuration to respond to the domain name. Configure Amazon ElastiCache for Redis to handle session information.

C.Keep the backend code on the EC2 instance. Create an Amazon ElastiCache for Redis cluster that has Multi-AZ enabled. Configure the ElastiCache for Redis cluster in cluster mode. Copy the frontend resources to Amazon S3. Configure the backend code to reference the EC2 instance.

D.Configure an Amazon CloudFront distribution with an Amazon S3 endpoint to an S3 bucket that is configured to host the static content. Configure an Application Load Balancer that targets an Amazon Elastic Container Service (Amazon ECS) service that runs AWS Fargate tasks for the PHP application. Configure the PHP application to use an Amazon ElastiCache for Redis cluster that runs in multiple Availability Zones.

Answer: D

Explanation:

Here's a breakdown of why option D is the best solution for achieving high availability and using AWS managed services, along with justifications for excluding the other options:

Why Option D is Correct:

Option D leverages several AWS managed services to achieve high availability, scalability, and resilience.

Amazon CloudFront with Amazon S3: Serving static content from S3 behind CloudFront provides global content delivery, caching, and reduced load on the application servers. S3 offers high availability and durability for static assets. https://aws.amazon.com/cloudfront/
Application Load Balancer (ALB): The ALB distributes traffic across multiple ECS Fargate tasks, ensuring high availability and enabling scaling based on demand. ALBs offer built-in health checks and automatic failover. https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
Amazon ECS with AWS Fargate: ECS Fargate allows running the PHP application in containers without managing the underlying EC2 instances. Fargate provides automatic scaling and resource management. https://aws.amazon.com/fargate/
Amazon ElastiCache for Redis (Multi-AZ): Using ElastiCache for Redis with Multi-AZ ensures high availability of the session data. In case of a failure in one Availability Zone, Redis automatically fails over to another zone. https://aws.amazon.com/elasticache/redis/

Why Other Options Are Incorrect:

Option A (Elastic Beanstalk): While Elastic Beanstalk can simplify deployment, using a single EC2 instance in a public subnet with a public IP address negates high availability.
Option B (Lambda): Hosting static content with Lambda is not cost-effective and it's not suitable for serving static web assets. Lambda functions are designed for stateless, event-driven computing and have execution time limitations.
Option C (EC2 and S3 with ElastiCache): Keeping backend code on a single EC2 instance doesn't address the high availability requirement for the application logic. The backend code must also be HA to benefit the most from HA Redis.

In summary, option D is the best answer because it delivers a highly available, scalable, and managed solution using a combination of CloudFront, S3, ALB, ECS Fargate, and ElastiCache for Redis. The other options do not provide the same level of availability and/or utilize less suitable AWS services.

Question: 655 **Exam Heist**

A company runs a web application on Amazon EC2 instances in an Auto Scaling group that has a target group. The company designed the application to work with session affinity (sticky sessions) for a better user experience.

The application must be available publicly over the internet as an endpoint. A WAF must be applied to the endpoint for additional security. Session affinity (sticky sessions) must be configured on the endpoint.

Which combination of steps will meet these requirements? (Choose two.)

A.Create a public Network Load Balancer. Specify the application target group.
B.Create a Gateway Load Balancer. Specify the application target group.
C.Create a public Application Load Balancer. Specify the application target group.
D.Create a second target group. Add Elastic IP addresses to the EC2 instances.
E.Create a web ACL in AWS WAF. Associate the web ACL with the endpoint

Answer: CE

Explanation:

Let's break down why options C and E are the correct choices to satisfy the requirements:

Why C is correct: Create a public Application Load Balancer. Specify the application target group.

An Application Load Balancer (ALB) is the appropriate choice when routing HTTP/HTTPS traffic and requiring application-level features like session affinity (sticky sessions). ALBs operate at Layer 7 of the OSI model,

allowing them to inspect the content of the traffic and make routing decisions based on it.
ALBs can be public, meaning they have a public IP address and can accept traffic directly from the internet.
ALBs have built-in support for session affinity (sticky sessions), which ensures that a user's requests are consistently routed to the same EC2 instance for the duration of their session.
By specifying the application's target group (which contains the EC2 instances), the ALB knows where to forward the incoming traffic.

Reference: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html

Why E is correct: Create a web ACL in AWS WAF. Associate the web ACL with the endpoint.

AWS WAF (Web Application Firewall) is a service that protects web applications from common web exploits and bots.
A Web ACL (Access Control List) in AWS WAF defines the rules and conditions that determine how WAF should inspect and filter incoming traffic.
Critically, AWS WAF integrates directly with Application Load Balancers. By creating a web ACL and associating it with the ALB, you apply WAF's protection to the web application, fulfilling the security requirement.

Reference: https://docs.aws.amazon.com/waf/latest/developerguide/what-is-aws-waf.html

Why A is incorrect: Create a public Network Load Balancer. Specify the application target group.

Network Load Balancers (NLBs) operate at Layer 4 of the OSI model (TCP/UDP). They are designed for high-performance, low-latency traffic and do not have the same application-level features as ALBs, such as session affinity. While NLBs can be public, they are not the best choice when needing sticky sessions and integrating with WAF effectively.

Why B is incorrect: Create a Gateway Load Balancer. Specify the application target group.

Gateway Load Balancers (GWLBs) are designed to route traffic to virtual appliances, such as firewalls or intrusion detection systems. They are not intended for directly serving web application traffic.
GWLBs are not directly associated with WAF.
GWLBs do not handle sticky sessions.

Why D is incorrect: Create a second target group. Add Elastic IP addresses to the EC2 instances.

Creating a second target group doesn't address the requirements of session affinity or WAF integration.
Target groups are merely containers for registered instances.
While adding Elastic IP addresses to EC2 instances might seem to provide fixed addresses, it is not a scalable or recommended approach for public-facing applications. Load balancers are specifically designed to distribute traffic across multiple instances, making Elastic IPs unnecessary and harder to manage. It also does not provide any WAF protection.

Question: 656 **Exam Heist**

A company runs a website that stores images of historical events. Website users need the ability to search and view images based on the year that the event in the image occurred. On average, users request each image only once or twice a year. The company wants a highly available solution to store and deliver the images to users.

Which solution will meet these requirements MOST cost-effectively?

A.Store images in Amazon Elastic Block Store (Amazon EBS). Use a web server that runs on Amazon EC2.
B.Store images in Amazon Elastic File System (Amazon EFS). Use a web server that runs on Amazon EC2.
C.Store images in Amazon S3 Standard. Use S3 Standard to directly deliver images by using a static website.

D.Store images in Amazon S3 Standard-Infrequent Access (S3 Standard-IA). Use S3 Standard-IA to directly deliver images by using a static website.

Answer: D

Explanation:

The correct answer is **D. Store images in Amazon S3 Standard-Infrequent Access (S3 Standard-IA). Use S3 Standard-IA to directly deliver images by using a static website.**

Here's why:

Cost-Effectiveness: The scenario specifies that images are accessed infrequently (once or twice a year). S3 Standard-IA is designed for data that is accessed less frequently but requires rapid access when needed. It offers lower storage costs compared to S3 Standard, making it more cost-effective for this usage pattern.
High Availability: Amazon S3, including S3 Standard-IA, provides high availability and durability, fulfilling the company's requirement. S3 offers 99.999999999% (11 9's) of data durability because it automatically creates and stores object copies across multiple Availability Zones.
Direct Delivery: Using S3 to directly deliver images via a static website eliminates the need for web servers (like EC2 instances), reducing operational overhead and costs associated with server management and maintenance. S3 can be configured to host static websites, allowing users to access images directly from the S3 bucket.
Alternatives are Less Suitable:

A (EBS on EC2): EBS is block storage, typically used for operating systems or databases on EC2 instances. Hosting images directly on EBS attached to an EC2 instance would be more expensive and require managing the EC2 instance.
B (EFS on EC2): EFS is a file system suitable for shared storage. While highly available, it's more expensive than S3 and requires managing EC2 instances to serve the images.
C (S3 Standard): S3 Standard is more expensive than S3 Standard-IA. Given the infrequent access pattern, S3 Standard-IA is the more cost-effective choice without sacrificing availability.

Therefore, storing images in S3 Standard-IA and directly delivering them as a static website provides a cost-effective, highly available, and easily manageable solution for the company's requirements.

Supporting Links:

Amazon S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
Hosting a Static Website on Amazon S3:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/WebsiteHosting.html

Question: 657 **Exam Heist**

A company has multiple AWS accounts in an organization in AWS Organizations that different business units use. The company has multiple offices around the world. The company needs to update security group rules to allow new office CIDR ranges or to remove old CIDR ranges across the organization. The company wants to centralize the management of security group rules to minimize the administrative overhead that updating CIDR ranges requires.

Which solution will meet these requirements MOST cost-effectively?

A.Create VPC security groups in the organization's management account. Update the security groups when a CIDR range update is necessary.

B.Create a VPC customer managed prefix list that contains the list of CIDRs. Use AWS Resource Access Manager (AWS RAM) to share the prefix list across the organization. Use the prefix list in the security groups across the organization.

C.Create an AWS managed prefix list. Use an AWS Security Hub policy to enforce the security group update across the organization. Use an AWS Lambda function to update the prefix list automatically when the CIDR ranges change.

D.Create security groups in a central administrative AWS account. Create an AWS Firewall Manager common security group policy for the whole organization. Select the previously created security groups as primary groups in the policy.

Answer: B

Explanation:

Here's a detailed justification for why option B is the most cost-effective solution:

Option B leverages AWS-managed services to centralize and streamline security group rule management across multiple AWS accounts within an organization. The key is the customer-managed prefix list. A prefix list is a managed collection of CIDR blocks. By creating a prefix list containing the company's office CIDR ranges, the company can define these ranges in one central location.

AWS Resource Access Manager (RAM) facilitates sharing this prefix list across all accounts within the AWS Organizations setup. This eliminates the need to manually create and maintain the same CIDR range information in each account's security groups. This substantially reduces administrative overhead.

Using the shared prefix list within the security groups means that instead of specifying individual CIDR ranges, the security group rules simply reference the prefix list. When a CIDR range needs to be updated (e.g., a new office opens or an old one closes), the company only needs to modify the prefix list in its central location. The change automatically propagates to all security groups that use the prefix list across the organization.

Cost-effectiveness stems from a few factors. Prefix lists and RAM are relatively inexpensive services. The primary benefit is the massive reduction in administrative effort and potential for errors associated with manually managing security group rules across multiple accounts.

Other options are less efficient. Option A lacks centralized management, increasing administrative burden. Option C relies on AWS-managed prefix list which can't be updated. Option D is expensive since Firewall Manager is not cost-effective for just security group management. Therefore, option B provides the best balance of centralized management, automation, and cost-effectiveness.

Authoritative Links:

AWS Resource Access Manager (RAM): https://aws.amazon.com/ram/
VPC Prefix Lists: https://docs.aws.amazon.com/vpc/latest/userguide/working-with-prefix-lists.html
Security Groups: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html

Question: 658 **Exam Heist**

A company uses an on-premises network-attached storage (NAS) system to provide file shares to its high performance computing (HPC) workloads. The company wants to migrate its latency-sensitive HPC workloads and its storage to the AWS Cloud. The company must be able to provide NFS and SMB multi-protocol access from the file system.

Which solution will meet these requirements with the LEAST latency? (Choose two.)

A.Deploy compute optimized EC2 instances into a cluster placement group.

B.Deploy compute optimized EC2 instances into a partition placement group.

C.Attach the EC2 instances to an Amazon FSx for Lustre file system.

D.Attach the EC2 instances to an Amazon FSx for OpenZFS file system.

E.Attach the EC2 instances to an Amazon FSx for NetApp ONTAP file system.

Answer: AE

Explanation:

The question focuses on migrating latency-sensitive HPC workloads and their storage to AWS while maintaining NFS and SMB multi-protocol access with the least latency.

Option A suggests deploying compute-optimized EC2 instances into a cluster placement group. This is a good starting point as cluster placement groups minimize latency by placing instances within the same Availability Zone on the same high-bandwidth network. This reduces the network distance between the compute instances, crucial for latency-sensitive applications.

Option E proposes attaching the EC2 instances to an Amazon FSx for NetApp ONTAP file system. FSx for NetApp ONTAP provides high performance and supports both NFS and SMB protocols, fulfilling the multi-protocol requirement. Furthermore, it offers low-latency access to data, crucial for HPC workloads. FSx for NetApp ONTAP also offers advanced data management features.

While Option C (FSx for Lustre) is known for high-performance computing, especially with large datasets and parallel workloads, FSx for NetApp ONTAP can also be suitable for HPC needs, especially when the workload requires multi-protocol access.

Option D (FSx for OpenZFS) also supports NFS and SMB, but might not always provide the lowest latency compared to ONTAP and isn't typically the go-to choice when the focus is explicitly on minimizing latency in a multi-protocol environment.

Option B (Partition placement group) is less relevant here. While it enhances fault tolerance, it doesn't inherently minimize latency like a cluster placement group. The primary goal is to reduce latency for the performance sensitive workload.

Therefore, combining cluster placement groups for the EC2 instances (A) with FSx for NetApp ONTAP (E) provides the lowest latency solution that meets the multi-protocol access requirement. The placement group minimizes latency between compute nodes, and FSx for NetApp ONTAP delivers low-latency shared storage.

Supporting links:

Amazon EC2 Placement Groups: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/placement-groups.html
Amazon FSx for NetApp ONTAP: https://aws.amazon.com/fsx/netapp-ontap/

Question: 659 **Exam Heist**

A company is relocating its data center and wants to securely transfer 50 TB of data to AWS within 2 weeks. The existing data center has a Site-to-Site VPN connection to AWS that is 90% utilized.

Which AWS service should a solutions architect use to meet these requirements?

A.AWS DataSync with a VPC endpoint
B.AWS Direct Connect
C.AWS Snowball Edge Storage Optimized
D.AWS Storage Gateway

Answer: C

Explanation:

The correct answer is **C. AWS Snowball Edge Storage Optimized**. Here's a detailed justification:

The primary challenge is transferring a large amount of data (50 TB) quickly and securely to AWS within a limited timeframe (2 weeks) while minimizing the impact on the existing, heavily utilized (90%) Site-to-Site VPN connection.

AWS Snowball Edge Storage Optimized devices are designed for transferring large datasets to AWS when network bandwidth is limited or cost-prohibitive. They provide a secure and ruggedized physical appliance that can be shipped directly to the data center. This bypasses the need to transfer the data over the congested VPN connection, addressing the core constraint. Once the data is loaded onto the Snowball Edge device, it's shipped back to AWS, where the data is loaded into the desired AWS service (e.g., S3). https://aws.amazon.com/snowball/

AWS DataSync with a VPC endpoint is a good option for ongoing data synchronization, but it relies on network bandwidth. Given the existing VPN is 90% utilized, using DataSync would be slow and potentially disrupt other services. It doesn't solve the initial large data migration problem within the timeframe.

AWS Direct Connect establishes a dedicated network connection from the data center to AWS. While ideal for ongoing, high-bandwidth needs, it takes time to provision and implement. It would likely not be feasible within the 2-week timeframe for the initial data transfer. Furthermore, it's overkill for a one-time migration and more suited to a long-term hybrid cloud strategy. https://aws.amazon.com/directconnect/

AWS Storage Gateway connects on-premises applications to AWS storage. While it's useful for hybrid cloud scenarios, it relies on the existing network connection for data transfer. Therefore, it would be constrained by the already saturated VPN connection.

Therefore, AWS Snowball Edge is the best option because it circumvents the network limitations, offers a secure transfer method, and is specifically designed for large data migrations in scenarios where network bandwidth is a constraint. It allows for an offline data transfer.

Question: 660 **Exam Heist**

A company hosts an application on Amazon EC2 On-Demand Instances in an Auto Scaling group. Application peak hours occur at the same time each day. Application users report slow application performance at the start of peak hours. The application performs normally 2-3 hours after peak hours begin. The company wants to ensure that the application works properly at the start of peak hours.

Which solution will meet these requirements?

A.Configure an Application Load Balancer to distribute traffic properly to the instances.

B.Configure a dynamic scaling policy for the Auto Scaling group to launch new instances based on memory utilization.

C.Configure a dynamic scaling policy for the Auto Scaling group to launch new instances based on CPU utilization.

D.Configure a scheduled scaling policy for the Auto Scaling group to launch new instances before peak hours.

Answer: D

Explanation:

Here's a detailed justification for why option D, configuring a scheduled scaling policy, is the best solution and why the other options are less suitable:

The core problem is predictable performance degradation during peak hours. This predictability is key. The goal is to ensure proper application function at the start of these peak hours.

Option A, configuring an Application Load Balancer (ALB), helps distribute traffic evenly across instances. However, an ALB alone won't address the underlying problem of insufficient capacity at the beginning of peak hours. While a properly configured ALB is essential for a well-architected application, it doesn't proactively increase resources to handle the anticipated surge in demand. It's a component of the solution, but not the primary fix.

Options B and C, using dynamic scaling policies based on memory or CPU utilization, react after the performance degradation has already begun. Dynamic scaling waits for the metrics to exceed a certain threshold (high CPU/memory) before launching new instances. This inherent delay means users will still experience slow performance at the start of peak hours while the Auto Scaling group reacts to the increased load. While reactive scaling is valuable for unexpected surges, it doesn't solve a predictable, recurring issue. Dynamic scaling policies are best used in conjunction with scheduled scaling, or for unexpected traffic spikes.

Option D, configuring a scheduled scaling policy, is the most effective solution. Because the peak hours are predictable, a scheduled policy can be configured to proactively launch additional EC2 instances before the peak traffic arrives. This ensures that sufficient capacity is available right from the start, eliminating the performance degradation reported by users. The Auto Scaling group will automatically increase the desired and minimum capacity at a specific time, adding instances before users experience problems. This proactive approach directly addresses the stated requirement.

In summary, scheduled scaling anticipates the demand, proactively adds capacity, and prevents performance degradation at the start of peak hours. ALBs provide traffic distribution, and dynamic scaling reacts to unforeseen changes, but neither effectively handles a consistently predictable surge in demand as well as scheduled scaling.

Authoritative Links:

AWS Auto Scaling – Scheduled Scaling:
https://docs.aws.amazon.com/autoscaling/ec2/userguide/schedule_time.html
AWS Auto Scaling – Dynamic Scaling: https://docs.aws.amazon.com/autoscaling/ec2/userguide/dynamic-scaling.html
Application Load Balancer:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html

Question: 661 **Exam Heist**

A company runs applications on AWS that connect to the company's Amazon RDS database. The applications scale on weekends and at peak times of the year. The company wants to scale the database more effectively for its applications that connect to the database.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use Amazon DynamoDB with connection pooling with a target group configuration for the database. Change the applications to use the DynamoDB endpoint.

B.Use Amazon RDS Proxy with a target group for the database. Change the applications to use the RDS Proxy endpoint.

C.Use a custom proxy that runs on Amazon EC2 as an intermediary to the database. Change the applications to use the custom proxy endpoint.

D.Use an AWS Lambda function to provide connection pooling with a target group configuration for the database. Change the applications to use the Lambda function.

Answer: B

Explanation:

The correct answer is B: Use Amazon RDS Proxy with a target group for the database and change the

applications to use the RDS Proxy endpoint.

RDS Proxy is a fully managed, highly available database proxy that sits between your applications and your RDS databases. It manages database connections efficiently, which is crucial for applications that experience scaling issues due to increased load. By pooling and sharing database connections, RDS Proxy reduces the overhead associated with establishing new connections each time an application needs to interact with the database. This enhances database efficiency and lowers operational costs.

By using RDS Proxy, the company can effectively manage database connections, minimizing connection overhead and improving overall performance during peak times. This solution requires minimal operational overhead because RDS Proxy is a managed service, meaning AWS handles the infrastructure management and patching. The only configuration needed is pointing applications to the RDS Proxy endpoint, making it the least operationally intensive option.

Option A is incorrect because DynamoDB is a NoSQL database, and migrating applications to use DynamoDB would require significant code changes and is not a simple scaling solution for an existing RDS database. Option C requires managing and maintaining a custom proxy on EC2, which introduces significant operational overhead. Option D, using a Lambda function for connection pooling, is not a recommended solution because Lambda functions have execution time limits and are better suited for event-driven architectures, not as continuous database connection proxies. Additionally, it adds complexity compared to a managed RDS Proxy.

In summary, RDS Proxy offers a straightforward, fully managed solution to address the company's database scaling needs with minimal operational overhead.

https://aws.amazon.com/rds/proxy/https://docs.aws.amazon.com/rds/proxy/latest/userguide/what-is.html

Question: 662 **Exam Heist**

A company uses AWS Cost Explorer to monitor its AWS costs. The company notices that Amazon Elastic Block Store (Amazon EBS) storage and snapshot costs increase every month. However, the company does not purchase additional EBS storage every month. The company wants to optimize monthly costs for its current storage usage.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use logs in Amazon CloudWatch Logs to monitor the storage utilization of Amazon EBS. Use Amazon EBS Elastic Volumes to reduce the size of the EBS volumes.

B.Use a custom script to monitor space usage. Use Amazon EBS Elastic Volumes to reduce the size of the EBS volumes.

C.Delete all expired and unused snapshots to reduce snapshot costs.

D.Delete all nonessential snapshots. Use Amazon Data Lifecycle Manager to create and manage the snapshots according to the company's snapshot policy requirements.

Answer: D

Explanation:

Here's a detailed justification for why option D is the best solution, along with supporting concepts and links:

The problem highlights escalating EBS storage and snapshot costs, despite no apparent increase in storage purchases. This strongly indicates inefficiencies in snapshot management. The goal is to optimize costs with minimal operational overhead.

Option A and B, while potentially helpful in identifying over-provisioned volumes, involve significant overhead. Monitoring EBS volume utilization using CloudWatch logs (A) or custom scripts (B) requires configuration, maintenance, and interpretation, and they don't directly address the root cause of increasing snapshot costs.

Elastic Volumes allow you to reduce the size of the EBS volumes but doesn't solve the issue with snapshot costs.

Option C focuses on deleting expired/unused snapshots, which is a good practice, but it's a manual process. Regularly identifying and deleting snapshots is time-consuming and prone to error. It doesn't establish an automated, policy-driven approach. It might be useful if you don't want to implement automation.

Option D is the most effective and efficient approach. By deleting nonessential snapshots and implementing Amazon Data Lifecycle Manager (DLM), the company can automate snapshot creation, retention, and deletion based on defined policies. DLM significantly reduces operational overhead by handling the snapshot lifecycle automatically. This ensures that snapshots are created regularly for data protection but are also purged when they are no longer needed, preventing unnecessary cost accumulation. DLM directly addresses the snapshot cost issue in a scalable and sustainable manner. It also addresses potential compliance requirements for data backup and retention. Nonessential snapshots take up storage and are potentially unassociated.

Here are some authoritative links for further research:

Amazon Data Lifecycle Manager (DLM): https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/snapshot-lifecycle.html
Amazon EBS Snapshots: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EBSSnapshots.html
Amazon EBS Elastic Volumes: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-expand-volume.html

Question: 663 **Exam Heist**

A company is developing a new application on AWS. The application consists of an Amazon Elastic Container Service (Amazon ECS) cluster, an Amazon S3 bucket that contains assets for the application, and an Amazon RDS for MySQL database that contains the dataset for the application. The dataset contains sensitive information. The company wants to ensure that only the ECS cluster can access the data in the RDS for MySQL database and the data in the S3 bucket.

Which solution will meet these requirements?

A.Create a new AWS Key Management Service (AWS KMS) customer managed key to encrypt both the S3 bucket and the RDS for MySQL database. Ensure that the KMS key policy includes encrypt and decrypt permissions for the ECS task execution role.

B.Create an AWS Key Management Service (AWS KMS) AWS managed key to encrypt both the S3 bucket and the RDS for MySQL database. Ensure that the S3 bucket policy specifies the ECS task execution role as a user.

C.Create an S3 bucket policy that restricts bucket access to the ECS task execution role. Create a VPC endpoint for Amazon RDS for MySQL. Update the RDS for MySQL security group to allow access from only the subnets that the ECS cluster will generate tasks in.

D.Create a VPC endpoint for Amazon RDS for MySQL. Update the RDS for MySQL security group to allow access from only the subnets that the ECS cluster will generate tasks in. Create a VPC endpoint for Amazon S3. Update the S3 bucket policy to allow access from only the S3 VPC endpoint.

Answer: D

Explanation:

The correct answer is D because it provides a secure and controlled access mechanism for the ECS cluster to access both the RDS for MySQL database and the S3 bucket without exposing them to the public internet or other unauthorized entities.

Here's a breakdown of why option D is correct:

RDS Access Control: Creating a VPC endpoint for RDS ensures that traffic to RDS remains within the AWS network and doesn't traverse the public internet. Restricting the RDS security group to only allow traffic from

the subnets where the ECS tasks are running isolates the database access solely to the authorized ECS cluster. This follows the principle of least privilege and prevents other resources from accessing the database.

S3 Access Control: Creating a VPC endpoint for S3 also keeps S3 traffic within the AWS network. By updating the S3 bucket policy to only allow access from the S3 VPC endpoint, you restrict access to the bucket solely through the VPC and, indirectly, only from resources authorized to use that VPC endpoint (in this case, the ECS cluster).

Why other options are incorrect:

Option A: While KMS can encrypt data at rest, it doesn't control network access. Even with encryption, anyone with network access and the necessary KMS permissions could still attempt to access the data. Relying solely on KMS for access control is insufficient. Also, encrypting using KMS only solves data at rest.
Option B: AWS managed keys are not customizable in terms of policies. Therefore, one cannot configure the S3 bucket policy to specify the ECS task execution role.
Option C: Using only an S3 bucket policy to restrict access to the ECS task execution role is not as secure as using a VPC endpoint. The bucket policy can be bypassed if someone gains access to the ECS task execution role's credentials outside the controlled network environment.

Key Concepts and Links:

VPC Endpoints: VPC endpoints allow you to privately connect your VPC to supported AWS services without requiring an internet gateway, NAT device, VPN connection, or AWS Direct Connect connection.
https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html
Security Groups: Security groups act as virtual firewalls for your EC2 instances and other AWS resources, controlling inbound and outbound traffic.
https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html
S3 Bucket Policies: S3 bucket policies are access control policies that you can attach to S3 buckets to grant or deny permissions to users or other AWS accounts.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/bucket-policies.html
Principle of Least Privilege: Granting only the minimum necessary permissions to perform a task.

Question: 664 **Exam Heist**

A company has a web application that runs on premises. The application experiences latency issues during peak hours. The latency issues occur twice each month. At the start of a latency issue, the application's CPU utilization immediately increases to 10 times its normal amount.

The company wants to migrate the application to AWS to improve latency. The company also wants to scale the application automatically when application demand increases. The company will use AWS Elastic Beanstalk for application deployment.

Which solution will meet these requirements?

A.Configure an Elastic Beanstalk environment to use burstable performance instances in unlimited mode. Configure the environment to scale based on requests.

B.Configure an Elastic Beanstalk environment to use compute optimized instances. Configure the environment to scale based on requests.

C.Configure an Elastic Beanstalk environment to use compute optimized instances. Configure the environment to scale on a schedule.

D.Configure an Elastic Beanstalk environment to use burstable performance instances in unlimited mode. Configure the environment to scale on predictive metrics.

Answer: D

Explanation:

Here's a detailed justification for why option D is the most suitable solution, considering the scenario's requirements:

The problem describes a web application experiencing significant, but infrequent, latency spikes due to CPU utilization increasing tenfold during peak hours. The company needs to migrate the application to AWS, improve latency, and implement automatic scaling to handle demand surges. Elastic Beanstalk is chosen as the deployment platform.

Option D is the most effective because it combines burstable performance instances in unlimited mode with scaling based on predictive metrics. Burstable performance instances (like those in the T family) are designed for workloads with average CPU utilization interspersed with occasional bursts. Unlimited mode allows these instances to burst beyond their baseline CPU for as long as required, at the cost of accruing CPU credits or charges. This is ideal for handling the sudden, drastic increase in CPU utilization.

Scaling based on predictive metrics utilizes AWS's forecasting services to anticipate demand. Instead of reacting to immediate CPU spikes (reactive scaling), predictive scaling proactively adds or removes instances before the anticipated spike, thereby mitigating latency issues before they occur. This is crucial, as the company knows when to expect the load.

Options A, B, and C are less effective. While using requests as a scaling trigger (Option A & B) is valid, it's reactive. The application will already be experiencing latency before the scaling action takes effect. Compute optimized instances (Option B & C) are designed for sustained high CPU usage, not infrequent bursts. Scheduling scaling (Option C) might work, but it requires precise prediction and is less flexible than predictive scaling if the spikes shift slightly in time. Also, if the spikes are actually unpredictable, scheduled scaling is not a proper solution.

Therefore, using burstable performance instances in unlimited mode combined with scaling based on predictive metrics allows the application to handle the bursty CPU demand efficiently and proactively, minimizing latency and meeting the company's requirements.

Relevant Resources:

Amazon EC2 Instance Types: https://aws.amazon.com/ec2/instance-types/
Elastic Beanstalk Auto Scaling: https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.managing.as.html
Amazon EC2 CPU Credits and Baseline Utilization: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-resize.html
Predictive Scaling for EC2: https://aws.amazon.com/blogs/compute/introducing-predictive-scaling-for-amazon-ec2/

Question: 665 **Exam Heist**

A company has customers located across the world. The company wants to use automation to secure its systems and network infrastructure. The company's security team must be able to track and audit all incremental changes to the infrastructure.

Which solution will meet these requirements?

A. Use AWS Organizations to set up the infrastructure. Use AWS Config to track changes.
B. Use AWS CloudFormation to set up the infrastructure. Use AWS Config to track changes.
C. Use AWS Organizations to set up the infrastructure. Use AWS Service Catalog to track changes.
D. Use AWS CloudFormation to set up the infrastructure. Use AWS Service Catalog to track changes.

Answer: B

Explanation:

The correct answer is B. AWS CloudFormation allows you to define and provision your infrastructure as code, enabling automation and consistency across deployments. By defining your infrastructure in CloudFormation templates, you can version control them, ensuring all incremental changes are tracked. This addresses the requirement for automated infrastructure setup and tracking of modifications.

AWS Config continuously monitors and records the configuration of your AWS resources. It allows you to track changes to resource configurations and evaluate them against desired states, providing an audit trail of all modifications. This meets the requirement for the security team to track and audit all incremental changes to the infrastructure.

Option A is incorrect because while AWS Organizations is useful for managing multiple AWS accounts, it doesn't directly provision infrastructure or track individual configuration changes. Option C is incorrect because AWS Service Catalog allows organizations to create and manage catalogs of IT services that are approved for use, but it doesn't inherently track configuration changes like AWS Config does. Option D is incorrect because, while CloudFormation is useful for defining and provisioning resources, Service Catalog doesn't provide the detailed tracking of incremental infrastructure changes provided by AWS Config.

CloudFormation provisions the infrastructure, providing the automation component, while Config provides continuous monitoring of changes for auditing, fulfilling both requirements. This makes option B the most complete and correct answer.

Further Reading:

AWS CloudFormation: https://aws.amazon.com/cloudformation/
AWS Config: https://aws.amazon.com/config/

Question: 666 **Exam Heist**

A startup company is hosting a website for its customers on an Amazon EC2 instance. The website consists of a stateless Python application and a MySQL database. The website serves only a small amount of traffic. The company is concerned about the reliability of the instance and needs to migrate to a highly available architecture. The company cannot modify the application code.

Which combination of actions should a solutions architect take to achieve high availability for the website? (Choose two.)

A.Provision an internet gateway in each Availability Zone in use.

B.Migrate the database to an Amazon RDS for MySQL Multi-AZ DB instance.

C.Migrate the database to Amazon DynamoDB, and enable DynamoDB auto scaling.

D.Use AWS DataSync to synchronize the database data across multiple EC2 instances.

E.Create an Application Load Balancer to distribute traffic to an Auto Scaling group of EC2 instances that are distributed across two Availability Zones.

Answer: BE

Explanation:

The correct answer is **BE**. Here's a detailed justification:

B - Migrate the database to an Amazon RDS for MySQL Multi-AZ DB instance: Moving the MySQL database to Amazon RDS (Relational Database Service) and enabling Multi-AZ (Multi-Availability Zone) is crucial for high availability. RDS Multi-AZ automatically provisions and maintains a synchronous standby replica of the

database in a different Availability Zone. If the primary database instance fails, RDS automatically fails over to the standby instance, minimizing downtime. This eliminates the single point of failure associated with a database running on a single EC2 instance. This ensures the database component of the website is highly available.

E - Create an Application Load Balancer to distribute traffic to an Auto Scaling group of EC2 instances that are distributed across two Availability Zones: Creating an Application Load Balancer (ALB) and an Auto Scaling group (ASG) for the stateless Python application ensures high availability and scalability. The ALB distributes incoming traffic across multiple EC2 instances running the application in different Availability Zones. This prevents a single point of failure for the web application layer. The Auto Scaling group dynamically adjusts the number of EC2 instances based on traffic demand. If an instance fails in one Availability Zone, Auto Scaling automatically launches a new instance in a healthy Availability Zone.

Why other options are incorrect:

A - Provision an internet gateway in each Availability Zone in use: Internet Gateways (IGWs) are regional resources, not zonal. You only need one IGW per VPC. Availability Zones do not require individual IGWs. This option is incorrect since Internet Gateway is used to provide internet access for VPCs and this does not improve the high availability of the EC2 instance.

C - Migrate the database to Amazon DynamoDB, and enable DynamoDB auto scaling: While DynamoDB offers high availability and scalability, it requires application code changes to interact with a NoSQL database. The question explicitly states that the company cannot modify the application code, making this option unsuitable. The application is currently using MySQL so it is not designed to connect to a NoSQL database

D - Use AWS DataSync to synchronize the database data across multiple EC2 instances: While AWS DataSync is useful for data transfer, it is not a solution for high availability of a MySQL database. DataSync does not provide automatic failover capabilities, which are essential for achieving high availability. RDS with Multi-AZ handles failover automatically, ensuring minimal downtime.

Relevant AWS documentation:

Amazon RDS Multi-AZ: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html
Application Load Balancer:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html
Auto Scaling: https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-groups.html

Question: 667 **Exam Heist**

A company is moving its data and applications to AWS during a multiyear migration project. The company wants to securely access data on Amazon S3 from the company's AWS Region and from the company's on-premises location. The data must not traverse the internet. The company has established an AWS Direct Connect connection between its Region and its on-premises location.

Which solution will meet these requirements?

A.Create gateway endpoints for Amazon S3. Use the gateway endpoints to securely access the data from the Region and the on-premises location.

B.Create a gateway in AWS Transit Gateway to access Amazon S3 securely from the Region and the on-premises location.

C.Create interface endpoints for Amazon S3. Use the interface endpoints to securely access the data from the Region and the on-premises location.

D.Use an AWS Key Management Service (AWS KMS) key to access the data securely from the Region and the on-premises location.

Answer: C

Explanation:

The correct answer is **C. Create interface endpoints for Amazon S3. Use the interface endpoints to securely access the data from the Region and the on-premises location.**

Here's a detailed justification:

Requirement Analysis: The core requirements are secure access to S3 from both an AWS Region and on-premises, and data must not traverse the internet, leveraging the existing AWS Direct Connect.

Interface Endpoints and Private Connectivity: Interface endpoints, powered by AWS PrivateLink, enable private connectivity to AWS services like S3 without exposing traffic to the public internet. They create a network interface within your VPC that acts as an entry point to S3. Traffic between your VPC (and connected on-premises location via Direct Connect) and S3 remains within the AWS network.

Direct Connect Integration: The AWS Direct Connect connection provides a private, dedicated network connection between the on-premises environment and the AWS Region. By using interface endpoints, the on-premises location can access S3 through Direct Connect, maintaining the required security and avoiding the internet.

Why not Gateway Endpoints (Option A)? Gateway endpoints are designed specifically for S3 and DynamoDB, but they only support access from within the same AWS Region's VPC. They do not provide access from on-premises locations via Direct Connect.

Why not Transit Gateway (Option B)? While Transit Gateway facilitates connectivity between multiple VPCs and on-premises, it doesn't directly provide a secure, private path to S3 that avoids the internet. You would still need a mechanism like interface endpoints for the actual S3 access. The transit gateway itself isn't directly involved in the secure S3 access.

Why not AWS KMS (Option D)? AWS KMS is used for encrypting data at rest and in transit. While encryption is essential for data security, it doesn't address the requirement of avoiding the internet for data access. KMS keys protect the data itself but do not change the network path.

In summary: Interface endpoints offer the only solution that satisfies both the security requirement (no internet access) and the connectivity requirement (access from both the AWS Region and on-premises via Direct Connect).

Authoritative Links:

AWS PrivateLink: https://aws.amazon.com/privatelink/
Accessing S3 Through Interface Endpoints: https://docs.aws.amazon.com/vpc/latest/privatelink/vpc-endpoints-s3.html
AWS Direct Connect: https://aws.amazon.com/directconnect/

Question: 668 **Exam Heist**

A company created a new organization in AWS Organizations. The organization has multiple accounts for the company's development teams. The development team members use AWS IAM Identity Center (AWS Single Sign-On) to access the accounts. For each of the company's applications, the development teams must use a predefined application name to tag resources that are created.

A solutions architect needs to design a solution that gives the development team the ability to create resources only if the application name tag has an approved value.

Which solution will meet these requirements?

A.Create an IAM group that has a conditional Allow policy that requires the application name tag to be specified for resources to be created.

B.Create a cross-account role that has a Deny policy for any resource that has the application name tag.

C.Create a resource group in AWS Resource Groups to validate that the tags are applied to all resources in all accounts.

D.Create a tag policy in Organizations that has a list of allowed application names.

Answer: D

Explanation:

The correct answer is **D: Create a tag policy in Organizations that has a list of allowed application names.**

Here's why:

AWS Organizations provides centralized governance and management of multiple AWS accounts. Tag policies, a feature within Organizations, allow administrators to enforce tagging standards across all accounts within an organization or specific organizational units (OUs). By creating a tag policy with a list of allowed application names, you directly address the requirement of ensuring that resources are only created with approved application name tag values.

When a tag policy is in effect, AWS will deny the creation of a resource if it lacks the required tag or if the tag's value doesn't match an approved value defined in the policy. This preventative approach is crucial for maintaining consistent tagging, simplifying resource management, and ensuring compliance. The tag policy applies to all accounts governed by the organization, guaranteeing that the development teams adhere to the defined standards regardless of the account they're working in.

Option A is incorrect because while IAM policies can enforce tagging requirements, applying this directly through IAM groups is less scalable and harder to centrally manage across multiple accounts in an AWS Organization. Each account would need similar, yet possibly diverging, IAM policies. Organizations' tag policies provide a centralized point of control.

Option B is incorrect because a cross-account role with a Deny policy wouldn't prevent the resource creation initially. It would primarily act after resource creation which isn't ideal. It would also require complex implementation across accounts and not be easily managed.

Option C is incorrect because AWS Resource Groups helps in organizing and managing resources based on tags, but it doesn't prevent the creation of resources without the correct tags. Resource Groups are reactive; they identify issues after resource creation, whereas the requirement calls for a proactive solution to prevent non-compliant resource creation.

In summary, tag policies in AWS Organizations are the most suitable solution because they provide a centralized, proactive, and scalable way to enforce mandatory tags with predefined allowed values across multiple accounts, meeting the company's requirements.

Supporting documentation:

AWS Organizations Tag Policies:
https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_tag-policies.html
Tagging Best Practices: https://aws.amazon.com/blogs/aws/aws-tagging-strategies/

Question: 669 **Exam Heist**

A company runs its databases on Amazon RDS for PostgreSQL. The company wants a secure solution to manage the master user password by rotating the password every 30 days.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use Amazon EventBridge to schedule a custom AWS Lambda function to rotate the password every 30 days.

B.Use the modify-db-instance command in the AWS CLI to change the password.

C.Integrate AWS Secrets Manager with Amazon RDS for PostgreSQL to automate password rotation.

D.Integrate AWS Systems Manager Parameter Store with Amazon RDS for PostgreSQL to automate password rotation.

Answer: C

Explanation:

The correct answer is C: Integrate AWS Secrets Manager with Amazon RDS for PostgreSQL to automate password rotation. Here's why:

AWS Secrets Manager is specifically designed for securely storing and managing secrets like database credentials, API keys, and other sensitive information. Its built-in rotation feature allows automatic password rotation for RDS databases, including PostgreSQL, with minimal operational overhead. Secrets Manager handles the entire rotation process: it creates a new password, validates it, updates the database, and then stores the new secret.

Option A is less desirable because it requires developing and maintaining a custom Lambda function and EventBridge rule. This adds operational overhead for development, testing, and maintenance. While possible, it's more complex than using a dedicated secrets management service.

Option B, using the AWS CLI modify-db-instance command, only allows manual password changes. It does not automate the rotation process, requiring manual intervention every 30 days, which defeats the purpose of automation and increases operational overhead.

Option D, using AWS Systems Manager Parameter Store, while capable of storing secrets, is primarily designed for storing configuration data. It lacks the built-in rotation features of Secrets Manager, requiring custom code or scripting for rotation, which again increases operational overhead. Secrets Manager provides a purpose-built solution for secrets management including rotation, auditing and access control. Parameter Store is more appropriate for storing non-sensitive configuration data.

Therefore, integrating AWS Secrets Manager provides the most secure and least operationally intensive solution for automating RDS PostgreSQL password rotation.

References:

AWS Secrets Manager: https://aws.amazon.com/secrets-manager/
Rotating Amazon RDS, Amazon Redshift, and Amazon DocumentDB secrets automatically with AWS Secrets Manager: https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotating-secrets.html

Question: 670 **Exam Heist**

A company performs tests on an application that uses an Amazon DynamoDB table. The tests run for 4 hours once a week. The company knows how many read and write operations the application performs to the table each second during the tests. The company does not currently use DynamoDB for any other use case. A solutions architect needs to optimize the costs for the table.

Which solution will meet these requirements?

A.Choose on-demand mode. Update the read and write capacity units appropriately.

B.Choose provisioned mode. Update the read and write capacity units appropriately.

C.Purchase DynamoDB reserved capacity for a 1-year term.

D.Purchase DynamoDB reserved capacity for a 3-year term.

Answer: B

Explanation:

The correct answer is B: Choose provisioned mode. Update the read and write capacity units appropriately.

Here's why:

The scenario describes predictable, recurring usage patterns. The company knows the read/write operations per second during the 4-hour weekly tests. This predictability is the key indicator that provisioned capacity is a better fit than on-demand.

Provisioned Capacity: Allows you to specify the read and write capacity units (RCUs and WCUs) required for your application. You pay for the capacity you provision. This is cost-effective when your workload is predictable. Since the company knows its read and write needs during the tests, they can accurately provision capacity, avoiding the higher per-request cost of on-demand.

On-Demand Capacity: Charges based on actual read and write requests, without the need to pre-provision capacity. It's suitable for unpredictable workloads. While convenient, it's generally more expensive for consistent workloads.

Reserved Capacity: Offers discounts for committing to provisioned capacity for 1 or 3 years. While attractive for cost savings, it's only beneficial if the capacity utilization is consistently high. Given that the DynamoDB table is only used for 4 hours a week, reserved capacity would likely lead to significant underutilization of the reserved capacity, making it less cost-effective.

Answer A is incorrect because even though you can "update the read and write capacity units appropriately", on-demand capacity is not the ideal option for a predictable workload that occurs every week. Provisioned mode is better.

Answers C and D are incorrect because reserved capacity is beneficial when the capacity utilization is high, and the DynamoDB table is only used for a short amount of time each week. The reserved capacity would be underutilized.

By choosing provisioned mode and setting the RCUs and WCUs based on the known requirements during the 4-hour test, the company can optimize costs by avoiding the on-demand per-request pricing and the underutilization that would occur when purchasing DynamoDB reserved capacity.

Relevant Documentation:

DynamoDB On-Demand vs. Provisioned Capacity: Costs & Use Cases | Chaos Genius
Choosing Between Provisioned and On-Demand Capacity in AWS DynamoDB - Scalegrid

Question: 671 **Exam Heist**

A company runs its applications on Amazon EC2 instances. The company performs periodic financial assessments of its AWS costs. The company recently identified unusual spending.

The company needs a solution to prevent unusual spending. The solution must monitor costs and notify responsible stakeholders in the event of unusual spending.

Which solution will meet these requirements?

A.Use an AWS Budgets template to create a zero spend budget.

B.Create an AWS Cost Anomaly Detection monitor in the AWS Billing and Cost Management console.

C.Create AWS Pricing Calculator estimates for the current running workload pricing details.

D.Use Amazon CloudWatch to monitor costs and to identify unusual spending.

Answer: B

Explanation:

The correct answer is **B. Create an AWS Cost Anomaly Detection monitor in the AWS Billing and Cost Management console.**

Here's why: AWS Cost Anomaly Detection is specifically designed to identify unusual spending patterns in your AWS costs. It uses machine learning algorithms to learn your typical cost behavior and automatically detects anomalies. When an anomaly is detected, it sends notifications to designated stakeholders, enabling prompt action to investigate and mitigate unexpected costs. This directly addresses the requirement of monitoring costs and notifying stakeholders of unusual spending.

Option A, using an AWS Budgets template to create a zero spend budget, while useful for setting hard limits, is not designed for detecting unusual spending. A zero spend budget would simply prevent any spending, which is not the desired outcome (the company wants to monitor and be notified of unusual spending). [https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-managing-costs.html]

Option C, creating AWS Pricing Calculator estimates, is helpful for initial cost planning, but it's a static calculation. It doesn't continuously monitor actual spending or detect anomalies based on historical cost patterns. It only provides a point-in-time estimate. [https://calculator.aws/]

Option D, using Amazon CloudWatch to monitor costs, is not a dedicated cost monitoring tool. While CloudWatch can monitor various AWS metrics, including billing metrics, it requires manual configuration of alarms and thresholds based on cost data. It doesn't automatically learn cost patterns or detect anomalies like Cost Anomaly Detection does. Manually setting thresholds is more cumbersome and less effective at identifying unexpected increases in cost patterns compared to Cost Anomaly Detection. [https://aws.amazon.com/cloudwatch/]

Cost Anomaly Detection provides an automated and proactive approach to identify and address unusual spending, aligning perfectly with the company's requirements. [https://aws.amazon.com/aws-cost-management/aws-cost-anomaly-detection/] It also provides customization for anomaly detection and notification preferences.

Question: 672 **Exam Heist**

A marketing company receives a large amount of new clickstream data in Amazon S3 from a marketing campaign. The company needs to analyze the clickstream data in Amazon S3 quickly. Then the company needs to determine whether to process the data further in the data pipeline.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create external tables in a Spark catalog. Configure jobs in AWS Glue to query the data.

B.Configure an AWS Glue crawler to crawl the data. Configure Amazon Athena to query the data.

C.Create external tables in a Hive metastore. Configure Spark jobs in Amazon EMR to query the data.

D.Configure an AWS Glue crawler to crawl the data. Configure Amazon Kinesis Data Analytics to use SQL to query the data.

Answer: B

Explanation:

The correct answer is B. Here's a detailed justification:

The scenario requires quickly analyzing clickstream data in S3 with minimal operational overhead to determine if further processing is needed. Option B, utilizing AWS Glue and Amazon Athena, offers the most efficient and cost-effective approach. AWS Glue crawler automatically infers the schema from the data stored in S3, catalogs the data, and creates metadata tables in the AWS Glue Data Catalog. This eliminates the need for manual schema definition or table creation, reducing operational overhead. Athena can then directly query the data in S3 using SQL.

Option A, using a Spark catalog, is less optimal. While AWS Glue can be used with Spark, configuring Spark jobs specifically for this quick analysis introduces more complexity. Setting up and managing Spark jobs involves higher operational overhead than using Athena.

Option C, using a Hive metastore with Amazon EMR, is an even heavier approach. EMR clusters require provisioning and management, adding significant operational complexity and cost compared to the serverless Athena. It also requires maintaining and managing the Hive metastore.

Option D, using AWS Glue crawler and Kinesis Data Analytics, is not the right choice because Kinesis Data Analytics is typically used for real-time stream processing. While it supports querying data, it's designed for continuous data streams, not for analyzing a static data set residing in S3. Also, using SQL in Kinesis Data Analytics is more complex and doesn't fit the use case of quickly analyzing data in S3.

Athena is serverless, meaning there are no servers to manage, scale, or provision. You only pay for the queries you run. Coupled with Glue's automatic schema discovery, this provides the lowest operational overhead for quickly querying the clickstream data to determine whether further processing is necessary.

Therefore, AWS Glue Crawler + Amazon Athena strikes the best balance of speed, minimal management, and cost-effectiveness for quickly analyzing S3 data and determining whether it warrants further data pipeline processing.

Relevant links:

AWS Glue: https://aws.amazon.com/glue/
Amazon Athena: https://aws.amazon.com/athena/

Question: 673 **Exam Heist**

A company runs an SMB file server in its data center. The file server stores large files that the company frequently accesses for up to 7 days after the file creation date. After 7 days, the company needs to be able to access the files with a maximum retrieval time of 24 hours.

Which solution will meet these requirements?

A.Use AWS DataSync to copy data that is older than 7 days from the SMB file server to AWS.

B.Create an Amazon S3 File Gateway to increase the company's storage space. Create an S3 Lifecycle policy to transition the data to S3 Glacier Deep Archive after 7 days.

C.Create an Amazon FSx File Gateway to increase the company's storage space. Create an Amazon S3 Lifecycle policy to transition the data after 7 days.

D.Configure access to Amazon S3 for each user. Create an S3 Lifecycle policy to transition the data to S3 Glacier Flexible Retrieval after 7 days.

Answer: B

Explanation:

Here's a detailed justification for why option B is the most suitable solution, along with supporting concepts

and links:

The primary requirement is to provide fast access to files for 7 days and then transition them to a slower, cheaper storage tier with a retrieval time of less than 24 hours. Let's analyze the options:

Option A: Using AWS DataSync alone doesn't address the need for immediate access for the first 7 days. DataSync primarily focuses on data transfer, not tiered storage with lifecycle management.

Option B: This option effectively addresses both requirements. Amazon S3 File Gateway allows on-premises applications (like the SMB server) to seamlessly store and access data in Amazon S3. It acts as a local cache for frequently accessed files, providing low-latency access. Critically, the S3 Lifecycle policy can automatically transition older data (older than 7 days) to S3 Glacier Deep Archive. Glacier Deep Archive offers the lowest storage cost and meets the requirement of retrieval within 24 hours.

Option C: While Amazon FSx File Gateway could provide file system access in AWS, it is not designed to extend on-premise storage capacity in the way S3 File Gateway does. Additionally, an S3 Lifecycle policy is not directly compatible with FSx. It manages objects in S3 buckets.

Option D: Requiring each user to access S3 directly bypasses the existing SMB server infrastructure, introducing complexity and potential permission management issues. While the lifecycle policy to Glacier Flexible Retrieval is valid, it doesn't address the fast access requirement for the first 7 days.

Therefore, S3 File Gateway coupled with a Lifecycle policy to Glacier Deep Archive offers the most integrated and cost-effective solution for this scenario. It extends the SMB server's storage, provides fast local access for recent files, and transitions older files to a cheaper archive tier while meeting the retrieval time requirement.

Key Concepts:

Amazon S3 File Gateway: Provides a local cache for frequently accessed files, enabling low-latency access from on-premises applications.
Amazon S3 Lifecycle policies: Automate the transition of objects between different storage classes (e.g., S3 Standard, S3 Glacier Deep Archive) based on defined rules.
S3 Glacier Deep Archive: Offers the lowest cost storage for long-term archiving with infrequent access.
Storage Tiering: Moving data to different storage classes based on access frequency and cost considerations.

Authoritative Links:

Amazon S3 File Gateway: https://aws.amazon.com/storagegateway/file/
Amazon S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
Amazon S3 Lifecycle: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-concept.html
Amazon S3 Glacier Deep Archive: https://aws.amazon.com/glacier/deep-archive/

Question: 674 **Exam Heist**

A company runs a web application on Amazon EC2 instances in an Auto Scaling group. The application uses a database that runs on an Amazon RDS for PostgreSQL DB instance. The application performs slowly when traffic increases. The database experiences a heavy read load during periods of high traffic.

Which actions should a solutions architect take to resolve these performance issues? (Choose two.)

A.Turn on auto scaling for the DB instance.

B.Create a read replica for the DB instance. Configure the application to send read traffic to the read replica.

C.Convert the DB instance to a Multi-AZ DB instance deployment. Configure the application to send read traffic

to the standby DB instance.

D.Create an Amazon ElastiCache cluster. Configure the application to cache query results in the ElastiCache cluster.

E.Configure the Auto Scaling group subnets to ensure that the EC2 instances are provisioned in the same Availability Zone as the DB instance.

Answer: BD

Explanation:

Here's a detailed justification for the answer BD:

The problem describes a web application experiencing performance issues due to a heavy read load on an RDS for PostgreSQL database during periods of high traffic. The core issue is database read contention, which is slowing down the application.

B. Create a read replica for the DB instance. Configure the application to send read traffic to the read replica. This is a direct and effective solution. Read replicas are designed to offload read traffic from the primary database instance. By directing read queries to the read replica, the primary instance is freed up to handle write operations and other critical tasks. This significantly reduces the load on the primary database, improving overall application performance. RDS makes it easy to create and manage read replicas.

D. Create an Amazon ElastiCache cluster. Configure the application to cache query results in the ElastiCache cluster. Implementing a caching layer using Amazon ElastiCache addresses the read load problem in a different but complementary way. By caching frequently accessed data or query results, the application can retrieve information directly from the cache instead of querying the database every time. This reduces the number of read requests hitting the database, further alleviating the load and improving response times. ElastiCache offers managed in-memory caching services, allowing for fast data retrieval and efficient scaling.

Why other options are incorrect:

A. Turn on auto scaling for the DB instance. While RDS does support autoscaling for storage, it does not autoscaling for compute resources directly like EC2 instances. This will not address the read load problem. Also, autoscaling would not work when traffic decreases as quickly as it goes up.

C. Convert the DB instance to a Multi-AZ DB instance deployment. Configure the application to send read traffic to the standby DB instance. Multi-AZ is primarily for high availability and disaster recovery, not read scaling. The standby instance in a Multi-AZ deployment is not intended for serving read traffic. It's a passive replica that becomes active only in case of a failure of the primary instance. Directly using the standby instance for reads is not supported and can lead to data inconsistency or other issues.

E. Configure the Auto Scaling group subnets to ensure that the EC2 instances are provisioned in the same Availability Zone as the DB instance. This is a best practice for reducing latency and network costs, it does nothing to reduce load on the database instance.

Supporting Links:

Amazon RDS Read Replicas:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ReadRepl.html
Amazon ElastiCache: https://aws.amazon.com/elasticache/
Amazon RDS Multi-AZ Deployments:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html

Question: 675 **Exam Heist**

A company uses Amazon EC2 instances and Amazon Elastic Block Store (Amazon EBS) volumes to run an application. The company creates one snapshot of each EBS volume every day to meet compliance requirements. The company wants to implement an architecture that prevents the accidental deletion of EBS volume snapshots. The solution must not change the administrative rights of the storage administrator user.

Which solution will meet these requirements with the LEAST administrative effort?

A.Create an IAM role that has permission to delete snapshots. Attach the role to a new EC2 instance. Use the AWS CLI from the new EC2 instance to delete snapshots.

B.Create an IAM policy that denies snapshot deletion. Attach the policy to the storage administrator user.

C.Add tags to the snapshots. Create retention rules in Recycle Bin for EBS snapshots that have the tags.

D.Lock the EBS snapshots to prevent deletion.

Answer: D

Explanation:

The correct answer is **D: Lock the EBS snapshots to prevent deletion.**

Here's why:

Recycle Bin (Option C) provides a temporary buffer against accidental deletion, allowing recovery within a configured retention period. However, it doesn't prevent deletion, which is a critical requirement for the company. An administrator could still permanently delete a snapshot from the Recycle Bin.
IAM Policy (Option B) while effective at preventing a specific user (the storage administrator) from deleting snapshots, it directly modifies their administrative rights, which violates the requirement that these rights should not change. This approach also doesn't protect against deletion from other users or services with sufficient permissions.
Using an EC2 instance with a restricted IAM role to delete snapshots (Option A) complicates the process and adds unnecessary overhead. It doesn't prevent accidental deletion by the storage administrator; it simply delegates the deletion task to another entity.

EBS snapshot locking directly addresses the requirement of preventing accidental deletion without altering the storage administrator's fundamental rights. Snapshot locking provides a write-once-read-many (WORM) capability, meaning that once a snapshot is locked, it cannot be deleted until the lock expires, regardless of user permissions. This fulfills the compliance requirement and ensures data retention without modifying the existing administrative structure.

Snapshot locking is the least administrative effort option because it's a built-in feature designed specifically for this purpose. The storage administrator maintains their existing rights, but the snapshots are protected by the lock.

Further research on EBS snapshot locking can be found in the official AWS documentation:

Amazon EBS Snapshots - Preventing Accidental Deletion with Snapshot Locks

Question: 676 **Exam Heist**

A company's application uses Network Load Balancers, Auto Scaling groups, Amazon EC2 instances, and databases that are deployed in an Amazon VPC. The company wants to capture information about traffic to and from the network interfaces in near real time in its Amazon VPC. The company wants to send the information to Amazon OpenSearch Service for analysis.

Which solution will meet these requirements?

A.Create a log group in Amazon CloudWatch Logs. Configure VPC Flow Logs to send the log data to the log group. Use Amazon Kinesis Data Streams to stream the logs from the log group to OpenSearch Service.

B.Create a log group in Amazon CloudWatch Logs. Configure VPC Flow Logs to send the log data to the log group. Use Amazon Kinesis Data Firehose to stream the logs from the log group to OpenSearch Service.

C.Create a trail in AWS CloudTrail. Configure VPC Flow Logs to send the log data to the trail. Use Amazon Kinesis Data Streams to stream the logs from the trail to OpenSearch Service.

D.Create a trail in AWS CloudTrail. Configure VPC Flow Logs to send the log data to the trail. Use Amazon Kinesis Data Firehose to stream the logs from the trail to OpenSearch Service.

Answer: B

Explanation:

The requirement is to capture and analyze VPC traffic information in near real-time using Amazon OpenSearch Service.

Option B is the correct solution because it utilizes VPC Flow Logs to capture the network traffic data, sends it to CloudWatch Logs, and then uses Kinesis Data Firehose to stream the logs to OpenSearch Service. VPC Flow Logs directly capture IP traffic information at the network interface level within the VPC. CloudWatch Logs acts as a central repository for these logs. Kinesis Data Firehose is specifically designed to reliably stream data to destinations like Amazon OpenSearch Service.

Option A is incorrect because while it uses CloudWatch Logs and VPC Flow Logs correctly, Kinesis Data Streams is more suitable for custom processing of data in transit and isn't the most efficient way to deliver directly to OpenSearch in this scenario. Firehose has direct integration and built-in buffering for OpenSearch Service.

Options C and D are incorrect because CloudTrail is used to log API calls made to AWS services, not network traffic. VPC Flow Logs capture the network traffic directly. Therefore, sending data to CloudTrail and then streaming from there is not suitable for network traffic analysis.

In summary, the combination of VPC Flow Logs, CloudWatch Logs, and Kinesis Data Firehose provides the most efficient and appropriate solution for capturing, storing, and analyzing VPC network traffic data in near real-time with Amazon OpenSearch Service.

Supporting Documentation:

VPC Flow Logs: https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html
Amazon Kinesis Data Firehose: https://docs.aws.amazon.com/firehose/latest/dev/what-is-this-service.html
Amazon OpenSearch Service: https://aws.amazon.com/opensearch-service/

Question: 677 **Exam Heist**

A company is developing an application that will run on a production Amazon Elastic Kubernetes Service (Amazon EKS) cluster. The EKS cluster has managed node groups that are provisioned with On-Demand Instances.

The company needs a dedicated EKS cluster for development work. The company will use the development cluster infrequently to test the resiliency of the application. The EKS cluster must manage all the nodes.

Which solution will meet these requirements MOST cost-effectively?

A.Create a managed node group that contains only Spot Instances.

B.Create two managed node groups. Provision one node group with On-Demand Instances. Provision the second node group with Spot Instances.

C.Create an Auto Scaling group that has a launch configuration that uses Spot Instances. Configure the user data to add the nodes to the EKS cluster.

D.Create a managed node group that contains only On-Demand Instances.

Answer: A

Explanation:

The correct answer is **A. Create a managed node group that contains only Spot Instances.**

Here's why:

Cost-Effectiveness: The primary requirement is cost-effectiveness for an infrequently used development cluster. Spot Instances offer significant discounts (up to 90%) compared to On-Demand Instances because you bid on unused EC2 capacity. Since the development cluster will be used sporadically for resiliency testing, interruptions from Spot Instance revocations are acceptable, making them the most economical choice.

Managed Node Group Convenience: Using a managed node group simplifies node management. EKS automatically handles provisioning, updating, and scaling the nodes within the group, reducing operational overhead compared to self-managed nodes (which Auto Scaling groups would entail).

Meeting Requirements: A managed node group with Spot Instances fulfills the requirement of a dedicated EKS cluster managed entirely by EKS itself. The infrequent usage aligns perfectly with the nature of Spot Instances, as interruptions are tolerable in a development/testing environment.

Why other options are not ideal:

B: Provisioning both On-Demand and Spot Instances increases complexity and cost. On-Demand Instances are not needed for infrequent development use.
C: Using an Auto Scaling group with a launch configuration and user data requires more manual configuration and management, which is contrary to the simplicity offered by managed node groups. Also, it moves away from managed nodes by EKS.
D: On-Demand Instances are too expensive for infrequent development use.

Authoritative Links:

Amazon EKS Managed Node Groups: https://docs.aws.amazon.com/eks/latest/userguide/managed-node-groups.html
Amazon EC2 Spot Instances: https://aws.amazon.com/ec2/spot/

Question: 678 **Exam Heist**

A company stores sensitive data in Amazon S3. A solutions architect needs to create an encryption solution. The company needs to fully control the ability of users to create, rotate, and disable encryption keys with minimal effort for any data that must be encrypted.

Which solution will meet these requirements?

A.Use default server-side encryption with Amazon S3 managed encryption keys (SSE-S3) to store the sensitive data.

B.Create a customer managed key by using AWS Key Management Service (AWS KMS). Use the new key to encrypt the S3 objects by using server-side encryption with AWS KMS keys (SSE-KMS).

C.Create an AWS managed key by using AWS Key Management Service (AWS KMS). Use the new key to encrypt the S3 objects by using server-side encryption with AWS KMS keys (SSE-KMS).

D.Download S3 objects to an Amazon EC2 instance. Encrypt the objects by using customer managed keys. Upload the encrypted objects back into Amazon S3.

Answer: B

Explanation:

The correct answer is B because it allows the company to fully control the encryption keys. The question explicitly states the need for the company to "fully control the ability of users to create, rotate, and disable encryption keys."

Let's analyze why the other options are incorrect:

A. Use default server-side encryption with Amazon S3 managed encryption keys (SSE-S3): SSE-S3 encrypts data at rest using keys managed by AWS. While easy to implement, the customer has no control over key rotation or disabling. AWS manages the keys entirely, failing to meet the requirement of full control.

C. Create an AWS managed key by using AWS Key Management Service (AWS KMS): This is incorrect. While KMS keys are used, "AWS managed keys" (formerly known as "AWS managed CMKs") in KMS are not fully controlled by the customer. Although you can choose to use these keys for SSE-KMS encryption in S3, the customer has very limited control over key rotation policies. The customer cannot disable or delete these keys.

D. Download S3 objects to an Amazon EC2 instance. Encrypt the objects by using customer managed keys: This solution is inefficient, complex, and introduces unnecessary overhead. It involves transferring data out of S3, performing encryption on an EC2 instance, and then re-uploading the encrypted data. This approach creates a management burden and doesn't leverage the built-in encryption capabilities of S3. Moreover, this method adds complexity and is not suitable for the requirement of minimal effort.

Why Option B is the Best Choice:

Option B, using a customer managed key in AWS KMS for SSE-KMS, provides the best balance of security, control, and ease of use:

Customer Control: The company creates and manages the KMS key, granting them complete control over the key's lifecycle, including creation, rotation, enabling, disabling, and deletion. This perfectly aligns with the stated requirement.
Security: AWS KMS provides a secure and compliant environment for managing cryptographic keys.
Integration with S3: SSE-KMS is a native feature of S3, simplifying encryption and decryption processes.
Minimal Effort: Using SSE-KMS with a customer managed key involves configuring the S3 bucket to use the key, which is relatively straightforward.

In summary, customer managed keys in AWS KMS, used with SSE-KMS, are designed to give customers complete control over their encryption keys while leveraging the security and scalability of AWS.

Authoritative Links:

AWS KMS: https://aws.amazon.com/kms/
Protecting Data Using Server-Side Encryption:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/serv-side-encryption.html
AWS KMS concepts: https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html

Question: 679 **Exam Heist**

A company wants to back up its on-premises virtual machines (VMs) to AWS. The company's backup solution exports on-premises backups to an Amazon S3 bucket as objects. The S3 backups must be retained for 30 days and must be automatically deleted after 30 days.

Which combination of steps will meet these requirements? (Choose three.)

A.Create an S3 bucket that has S3 Object Lock enabled.

B.Create an S3 bucket that has object versioning enabled.

C.Configure a default retention period of 30 days for the objects.

D.Configure an S3 Lifecycle policy to protect the objects for 30 days.

E.Configure an S3 Lifecycle policy to expire the objects after 30 days.

F.Configure the backup solution to tag the objects with a 30-day retention period

Answer: ACE

Explanation:

Here's a detailed justification for why options A, C, and E are the correct choices for the scenario, along with supporting concepts and links:

The requirement is to back up on-premises VMs to S3, retain the backups for 30 days, and automatically delete them after 30 days.

A. Create an S3 bucket that has S3 Object Lock enabled: S3 Object Lock is crucial to ensure that the backups are protected from accidental or malicious deletion during the retention period. Once a backup is written to S3 with Object Lock enabled, it cannot be deleted until the retention period expires. This is an important protection mechanism for backups.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html

C. Configure a default retention period of 30 days for the objects: This ensures that all objects placed in the S3 bucket are subject to the retention policy enforced by Object Lock. The objects cannot be deleted or overwritten until the 30-day period has elapsed, fulfilling the 30-day retention requirement. A retention period protects data integrity.

E. Configure an S3 Lifecycle policy to expire the objects after 30 days: While Object Lock protects the objects during the 30-day retention period, it doesn't automatically delete them after the retention period. A lifecycle policy configured to expire (delete) the objects after 30 days ensures automatic cleanup. If a retention period of 30 days is enabled using Object Lock, the expiration action of the Lifecycle rule cannot be for a period less than the retention period. This ensures that the backups are automatically purged once they are no longer needed, optimizing storage costs.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-examples.html

Why the other options are incorrect:

B. Create an S3 bucket that has object versioning enabled: While object versioning is a good practice for data protection in general (allowing you to recover previous versions of objects), it doesn't automatically delete objects. Versioning alone doesn't satisfy the requirement of automatic deletion after 30 days. It only keeps all versions of the object, which will increase storage costs without addressing the retention and automatic deletion requirement.

D. Configure an S3 Lifecycle policy to protect the objects for 30 days: While the intention might be correct, the phrase "protect the objects" is vague in the context of S3 Lifecycle policies. Lifecycle policies primarily manage object transitions (e.g., moving to cheaper storage classes) or expirations (deletions). Protection from deletion is achieved via Object Lock, not standard Lifecycle actions.

F. Configure the backup solution to tag the objects with a 30-day retention period: Tagging objects is useful for metadata and categorization, but S3 by itself does not automatically enforce retention based on tags. While the backup solution could be programmed to delete based on these tags, it introduces unnecessary complexity and reliance on the backup solution's custom logic. S3's built-in Object Lock and Lifecycle policies are more reliable and scalable for this purpose.

In summary, the combination of S3 Object Lock (A), a default retention period (C), and a Lifecycle policy for expiration (E) provides a robust and automated solution for backing up on-premises VMs to S3 with the

specified retention and deletion requirements.

Question: 680 **Exam Heist**

A solutions architect needs to copy files from an Amazon S3 bucket to an Amazon Elastic File System (Amazon EFS) file system and another S3 bucket. The files must be copied continuously. New files are added to the original S3 bucket consistently. The copied files should be overwritten only if the source file changes.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create an AWS DataSync location for both the destination S3 bucket and the EFS file system. Create a task for the destination S3 bucket and the EFS file system. Set the transfer mode to transfer only data that has changed.

B.Create an AWS Lambda function. Mount the file system to the function. Set up an S3 event notification to invoke the function when files are created and changed in Amazon S3. Configure the function to copy files to the file system and the destination S3 bucket.

C.Create an AWS DataSync location for both the destination S3 bucket and the EFS file system. Create a task for the destination S3 bucket and the EFS file system. Set the transfer mode to transfer all data.

D.Launch an Amazon EC2 instance in the same VPC as the file system. Mount the file system. Create a script to routinely synchronize all objects that changed in the origin S3 bucket to the destination S3 bucket and the mounted file system.

Answer: A

Explanation:

The correct answer is A. Here's a detailed justification:

A. AWS DataSync with Change Detection (Best Solution)

DataSync Efficiency: AWS DataSync is purpose-built for efficient data transfer between on-premises storage, S3, and EFS. It handles data transfer, synchronization, and metadata management automatically.

Change Detection: DataSync's "transfer only data that has changed" setting avoids unnecessary transfers and reduces operational overhead. It detects changes based on file modification times and sizes. This ensures only new or updated files are copied, satisfying the requirement of overwriting only changed files.

Continuous Operation: DataSync can be scheduled to run periodically or triggered by events, enabling continuous file synchronization.

Least Operational Overhead: DataSync is a managed service, meaning AWS handles the underlying infrastructure, scaling, and maintenance. This significantly reduces the operational burden compared to solutions involving EC2 instances or Lambda functions.

Authoritative Link: https://aws.amazon.com/datasync/

Why other options are less suitable:

B. Lambda Function: While feasible, Lambda requires custom code to handle file copying and change detection. Managing the function, handling potential errors, and ensuring scalability adds operational overhead. Mounting EFS to Lambda functions introduces complexity and can be less performant than a dedicated service like DataSync.

C. DataSync with Full Transfer: Transferring all data on each run is inefficient and costly, especially as the data volume grows. It violates the requirement to overwrite only changed files.

D. EC2 Instance: This approach requires managing an EC2 instance, configuring synchronization scripts

(potentially using rsync or similar tools), and handling scaling and fault tolerance. This is significantly more complex and has more operational overhead than a managed service like DataSync.

Question: 681 **Exam Heist**

A company uses Amazon EC2 instances and stores data on Amazon Elastic Block Store (Amazon EBS) volumes. The company must ensure that all data is encrypted at rest by using AWS Key Management Service (AWS KMS). The company must be able to control rotation of the encryption keys.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create a customer managed key. Use the key to encrypt the EBS volumes.
B.Use an AWS managed key to encrypt the EBS volumes. Use the key to configure automatic key rotation.
C.Create an external KMS key with imported key material. Use the key to encrypt the EBS volumes.
D.Use an AWS owned key to encrypt the EBS volumes.

Answer: A

Explanation:

The correct answer is A because it offers the most control over key rotation while minimizing operational overhead. The requirement is to encrypt EBS volumes at rest using KMS, with the ability to control key rotation.

Option A, using a customer-managed key (CMK), directly addresses this requirement. CMKs allow for full control over the key lifecycle, including defining rotation policies. AWS KMS automatically rotates CMKs created within KMS every year, unless you opt-out. This meets the need for key rotation without any manual intervention.

Option B, using an AWS-managed key, also provides encryption at rest. However, AWS-managed keys do not allow the user to control key rotation, which is a critical requirement. Although AWS will automatically rotate these keys, you cannot manage the rotation schedule or bring your own rotation schedule.

Option C, importing key material into KMS, introduces significant operational overhead. While it technically fulfills the requirement of using KMS and encryption, the management of external keys, including their availability and security, becomes the company's responsibility. This adds unnecessary complexity compared to using CMKs.

Option D, using AWS-owned keys, is not suitable because these keys are managed entirely by AWS and are not accessible or controllable by the user. You have no control over encryption or rotation, and are also not suitable for regulatory compliance in several cases. AWS owned keys are used by default when you encrypt EBS volumes.

Therefore, creating a customer-managed key is the most efficient and controllable solution for encrypting EBS volumes at rest with KMS and managing key rotation policies. It balances security requirements with operational efficiency.

Supporting Links:

AWS KMS Key Types: https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html
Encrypting EBS Volumes: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EBSEncryption.html
Rotating Customer Managed Keys: https://docs.aws.amazon.com/kms/latest/developerguide/rotate-keys.html

Question: 682 **Exam Heist**

A company needs a solution to enforce data encryption at rest on Amazon EC2 instances. The solution must automatically identify noncompliant resources and enforce compliance policies on findings.

Which solution will meet these requirements with the LEAST administrative overhead?

A.Use an IAM policy that allows users to create only encrypted Amazon Elastic Block Store (Amazon EBS) volumes. Use AWS Config and AWS Systems Manager to automate the detection and remediation of unencrypted EBS volumes.

B.Use AWS Key Management Service (AWS KMS) to manage access to encrypted Amazon Elastic Block Store (Amazon EBS) volumes. Use AWS Lambda and Amazon EventBridge to automate the detection and remediation of unencrypted EBS volumes.

C.Use Amazon Macie to detect unencrypted Amazon Elastic Block Store (Amazon EBS) volumes. Use AWS Systems Manager Automation rules to automatically encrypt existing and new EBS volumes.

D.Use Amazon inspector to detect unencrypted Amazon Elastic Block Store (Amazon EBS) volumes. Use AWS Systems Manager Automation rules to automatically encrypt existing and new EBS volumes.

Answer: A

Explanation:

The best solution is A because it combines preventative and reactive measures with minimal administrative overhead.

First, using an IAM policy to restrict users to creating only encrypted EBS volumes enforces encryption at the point of creation, preventing unencrypted volumes from being provisioned in the first place. This is a proactive approach.

Second, AWS Config continuously monitors your AWS resource configurations. It can be configured to evaluate whether EBS volumes are encrypted, identifying non-compliant resources. This provides continuous auditing and drift detection.

Third, integrating AWS Config with AWS Systems Manager (SSM) Automation allows for automated remediation. When AWS Config identifies a non-compliant (unencrypted) EBS volume, it can trigger an SSM Automation runbook to encrypt the volume. This automates the remediation process, minimizing manual intervention.

Option B is less efficient because it requires the creation and maintenance of Lambda functions and EventBridge rules, adding administrative overhead. While possible, it's more complex than using AWS Config and SSM Automation.

Option C, while Amazon Macie is designed for data security and data privacy service and can be used to discover sensitive data stored in Amazon S3, relational databases, and data warehouses, including personally identifiable information (PII), it is primarily designed for detecting and reporting on data breaches. It doesn't directly automate remediation actions on EBS volumes in the same straightforward way as AWS Config.

Option D: Amazon Inspector is designed for vulnerability assessments and security hardening. While it might be able to identify some security issues related to unencrypted volumes, its primary focus isn't on enforcing encryption policies.

Therefore, the combination of preventative IAM policies and automated detection and remediation through AWS Config and SSM Automation in option A provides the least administrative overhead and directly addresses the requirement of enforcing data encryption at rest.

Authoritative links:

AWS Config: https://aws.amazon.com/config/

AWS Systems Manager: https://aws.amazon.com/systems-manager/
Amazon EBS Encryption: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EBSEncryption.html

Question: 683 **Exam Heist**

A company is migrating its multi-tier on-premises application to AWS. The application consists of a single-node MySQL database and a multi-node web tier. The company must minimize changes to the application during the migration. The company wants to improve application resiliency after the migration.

Which combination of steps will meet these requirements? (Choose two.)

A.Migrate the web tier to Amazon EC2 instances in an Auto Scaling group behind an Application Load Balancer.
B.Migrate the database to Amazon EC2 instances in an Auto Scaling group behind a Network Load Balancer.
C.Migrate the database to an Amazon RDS Multi-AZ deployment.
D.Migrate the web tier to an AWS Lambda function.
E.Migrate the database to an Amazon DynamoDB table.

Answer: AC

Explanation:

The correct answer is **A. Migrate the web tier to Amazon EC2 instances in an Auto Scaling group behind an Application Load Balancer and C. Migrate the database to an Amazon RDS Multi-AZ deployment.**

Here's a detailed justification:

Option A: Migrating the web tier to EC2 instances in an Auto Scaling group behind an Application Load Balancer (ALB) provides high availability and scalability for the web tier. The Auto Scaling group ensures that the desired number of EC2 instances is maintained, even if some instances fail. The ALB distributes incoming traffic across the healthy EC2 instances, further enhancing availability and resilience. This addresses the requirement to improve application resiliency after migration while minimizing changes to the application logic itself. https://aws.amazon.com/elasticloadbalancing/application-load-balancer/ and https://aws.amazon.com/autoscaling/

Option C: Migrating the MySQL database to an Amazon RDS Multi-AZ deployment significantly improves database availability and durability. RDS Multi-AZ creates a synchronous, replicated standby instance in a different Availability Zone. In the event of a failure of the primary instance, RDS automatically fails over to the standby instance, minimizing downtime and improving resiliency. This meets the company's objective of enhancing application resiliency post-migration with minimal application changes as RDS largely handles the failover process. https://aws.amazon.com/rds/features/multi-az/

Why other options are incorrect:

Option B: Migrating the database to EC2 instances in an Auto Scaling group behind a Network Load Balancer (NLB) is not the best approach for a MySQL database. While Auto Scaling can help manage EC2 instance failures, setting up and managing database replication and failover on EC2 instances would be more complex than using RDS Multi-AZ, requiring more changes to the application and database setup. Also, NLB is designed for TCP traffic, not necessarily application layer awareness.

Option D: Migrating the web tier to an AWS Lambda function might require significant refactoring of the web tier code, as Lambda functions are typically used for event-driven, stateless workloads. This contradicts the requirement to minimize changes to the application during the migration.

Option E: Migrating the database to an Amazon DynamoDB table would necessitate substantial changes to the application's data access layer because MySQL and DynamoDB are fundamentally different database

types (relational vs. NoSQL). This approach would violate the requirement of minimizing application changes.

Question: 684 **Exam Heist**

A company wants to migrate its web applications from on premises to AWS. The company is located close to the eu-central-1 Region. Because of regulations, the company cannot launch some of its applications in eu-central-1. The company wants to achieve single-digit millisecond latency.

Which solution will meet these requirements?

A.Deploy the applications in eu-central-1. Extend the company's VPC from eu-central-1 to an edge location in Amazon CloudFront.

B.Deploy the applications in AWS Local Zones by extending the company's VPC from eu-central-1 to the chosen Local Zone.

C.Deploy the applications in eu-central-1. Extend the company's VPC from eu-central-1 to the regional edge caches in Amazon CloudFront.

D.Deploy the applications in AWS Wavelength Zones by extending the company's VPC from eu-central-1 to the chosen Wavelength Zone.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer, along with why the other options are less suitable, supported by relevant AWS concepts:

The key requirements are: (1) migration to AWS, (2) compliance preventing some apps in eu-central-1, (3) single-digit millisecond latency, and (4) the company's proximity to eu-central-1.

Option B, deploying applications in AWS Local Zones by extending the VPC from eu-central-1, directly addresses these requirements. AWS Local Zones are designed to place compute, storage, database, and other select AWS services closer to large population, industry, and IT centers. This proximity is crucial for achieving the single-digit millisecond latency target. Because the company cannot use eu-central-1 for all applications, a local zone geographically separate allows for compliance without the need for global infrastructure deployment. Extending the VPC allows for private, low-latency connectivity between resources in the eu-central-1 region and the Local Zone.

Option A is incorrect because CloudFront Edge Locations are for caching content closer to users, not for running applications that require low latency access to the primary region. Extending a VPC to a CloudFront edge location is not possible. CloudFront does not run compute resources.

Option C is incorrect for the same reason as Option A. Regional Edge Caches are still part of the content delivery network (CDN) infrastructure and do not provide the compute resources needed to run the applications. They are for caching content. Extending a VPC into an edge location is incorrect and not possible.

Option D is incorrect because AWS Wavelength Zones are designed for ultra-low latency applications at the edge of the 5G network, which is more pertinent to mobile and IoT scenarios, and typically necessitates integrating with specific telecommunications providers. The scenario doesn't mention 5G or mobile considerations. Furthermore, the question indicates some applications cannot be run within eu-central-1, not all. Local Zones provide a more generalized compute environment near the region.

Therefore, Local Zones are the most appropriate solution for fulfilling all the requirements of the scenario.

Authoritative Links:

AWS Local Zones: https://aws.amazon.com/local-zones/
Amazon CloudFront: https://aws.amazon.com/cloudfront/
AWS Wavelength: https://aws.amazon.com/wavelength/

Question: 685 **Exam Heist**

A company's ecommerce website has unpredictable traffic and uses AWS Lambda functions to directly access a private Amazon RDS for PostgreSQL DB instance. The company wants to maintain predictable database performance and ensure that the Lambda invocations do not overload the database with too many connections.

What should a solutions architect do to meet these requirements?

A.Point the client driver at an RDS custom endpoint. Deploy the Lambda functions inside a VPC.
B.Point the client driver at an RDS proxy endpoint. Deploy the Lambda functions inside a VPC.
C.Point the client driver at an RDS custom endpoint. Deploy the Lambda functions outside a VPC.
D.Point the client driver at an RDS proxy endpoint. Deploy the Lambda functions outside a VPC.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct solution, along with supporting concepts and links:

The core problem is managing unpredictable traffic from Lambda functions directly accessing an RDS database, potentially causing connection exhaustion and impacting database performance. The key to solving this is to decouple the Lambda functions from directly managing database connections.

Why option B is correct: Point the client driver at an RDS proxy endpoint. Deploy the Lambda functions inside a VPC.

RDS Proxy: RDS Proxy acts as a fully managed database proxy service. It sits between your application (Lambda functions) and your database (RDS for PostgreSQL). It pools and shares database connections, reducing the overhead of establishing new connections for each Lambda invocation. This is crucial for handling unpredictable traffic spikes from Lambda, preventing the database from being overloaded. It also automatically manages connection reuse, allowing Lambda to scale efficiently.

Deploying Lambda inside a VPC: This is necessary for Lambda functions to access private resources like an RDS database within your AWS environment. When Lambda functions reside inside a VPC, they can leverage security groups and network ACLs to establish secure, private connections to the RDS instance. If the database is only accessible within the VPC (which is a common security practice), the Lambda functions must also be within the VPC to reach it.

Why other options are incorrect:

Options A & C (RDS Custom Endpoint): RDS Custom endpoints are used for managing database instances with customizations and advanced configurations. These are not appropriate for managing the connection management issues associated with high concurrency from Lambda. Custom endpoints won't solve the database connection overload problem.

Option D (Deploy Lambda functions outside a VPC): If the RDS instance resides within a VPC (which is generally recommended for security), Lambda functions outside the VPC will not be able to access the database unless configured with public accessibility, which introduces security vulnerabilities. This is generally discouraged. Also, using an RDS proxy outside the VPC would require the database to be publicly accessible, which is a bad security practice.

In Summary: RDS Proxy handles connection management, preventing database overload, and deploying Lambda inside the VPC ensures secure, private access to the database. The combination addresses both requirements effectively.

Authoritative Links:

AWS RDS Proxy: https://aws.amazon.com/rds/proxy/
Configuring Lambda to access resources in a VPC: https://docs.aws.amazon.com/lambda/latest/dg/configuration-vpc.html
Best Practices for Lambda functions and RDS: https://aws.amazon.com/blogs/compute/using-amazon-rds-proxy-with-aws-lambda/

Question: 686 **Exam Heist**

A company is creating an application. The company stores data from tests of the application in multiple on-premises locations.

The company needs to connect the on-premises locations to VPCs in an AWS Region in the AWS Cloud. The number of accounts and VPCs will increase during the next year. The network architecture must simplify the administration of new connections and must provide the ability to scale.

Which solution will meet these requirements with the LEAST administrative overhead?

A.Create a peering connection between the VPCs. Create a VPN connection between the VPCs and the on-premises locations.

B.Launch an Amazon EC2 instance. On the instance, include VPN software that uses a VPN connection to connect all VPCs and on-premises locations.

C.Create a transit gateway. Create VPC attachments for the VPC connections. Create VPN attachments for the on-premises connections.

D.Create an AWS Direct Connect connection between the on-premises locations and a central VPC. Connect the central VPC to other VPCs by using peering connections.

Answer: C

Explanation:

The correct answer is C: Create a transit gateway. Create VPC attachments for the VPC connections. Create VPN attachments for the on-premises connections. This solution offers the least administrative overhead and greatest scalability for connecting multiple VPCs and on-premises locations.

Here's why:

Transit Gateway Simplifies Network Management: AWS Transit Gateway acts as a central hub, simplifying the routing between multiple VPCs and on-premises networks. Instead of managing numerous point-to-point connections (like VPC peering), you manage connections to the Transit Gateway.

Scalability: As the number of VPCs and on-premises locations grows, the Transit Gateway scales easily. You simply create new attachments without having to reconfigure existing connections. This eliminates the complexity of managing a mesh network created by VPC peering or VPN connections.

Centralized Routing Policies: Transit Gateway provides a central place to manage routing policies. You can control the traffic flow between different VPCs and on-premises locations using route tables associated with the Transit Gateway.

VPN Attachments: It supports VPN attachments for on-premises connections, integrating seamlessly with existing VPN infrastructure.

Let's analyze why the other options are not optimal:

A: VPC Peering and VPNs: This solution is not scalable and involves high administrative overhead. VPC peering is a one-to-one connection, meaning you'd need numerous peering connections as your VPCs increase. Managing all these connections becomes complex and time-consuming.

B: EC2 VPN Instance: While it's a potential solution, it introduces a single point of failure. The EC2 instance becomes a bottleneck. It also requires more hands-on management for patching, scaling, and high availability, adding to the administrative overhead.

D: Direct Connect and VPC Peering: Direct Connect provides a dedicated network connection from on-premises to AWS, but using a central VPC and peering to others becomes complex as the number of VPCs grows. Similar to option A, it results in a mesh network of peering connections, which is difficult to manage. Direct Connect is also more expensive and is more suitable for continuous large data transfers. For this scenario where a company will increase the number of accounts and VPCs during the next year, transit gateway will provide a scalable solution.

In summary, Transit Gateway centralizes network management, offers scalability, and integrates well with both VPCs and VPN connections, making it the most suitable solution for the company's needs.

Supporting Documentation:

AWS Transit Gateway: https://aws.amazon.com/transit-gateway/
AWS VPN Connections: https://aws.amazon.com/vpn/
AWS Direct Connect: https://aws.amazon.com/directconnect/

Question: 687 **Exam Heist**

A company that uses AWS needs a solution to predict the resources needed for manufacturing processes each month. The solution must use historical values that are currently stored in an Amazon S3 bucket. The company has no machine learning (ML) experience and wants to use a managed service for the training and predictions.

Which combination of steps will meet these requirements? (Choose two.)

A.Deploy an Amazon SageMaker model. Create a SageMaker endpoint for inference.

B.Use Amazon SageMaker to train a model by using the historical data in the S3 bucket.

C.Configure an AWS Lambda function with a function URL that uses Amazon SageMaker endpoints to create predictions based on the inputs.

D.Configure an AWS Lambda function with a function URL that uses an Amazon Forecast predictor to create a prediction based on the inputs.

E.Train an Amazon Forsecast predictor by using the historical data in the S3 bucket.

Answer: BD

Explanation:

The correct answer is BD because it leverages the most suitable managed service for time-series forecasting when the company lacks ML expertise. Here's a detailed breakdown:

B. Use Amazon SageMaker to train a model by using the historical data in the S3 bucket. SageMaker offers managed model training capabilities. It allows users to bring their data stored in S3 and select a suitable algorithm or even implement their own. SageMaker handles the infrastructure provisioning, scaling, and optimization for training the model. This is a valid step to create a prediction model using the historical data.

D. Configure an AWS Lambda function with a function URL that uses an Amazon Forecast predictor to create a prediction based on the inputs. Amazon Forecast is a managed service specifically designed for

time-series forecasting. Once the model is trained in Forecast, you can trigger it using a Lambda function. Function URLs offer a simple way to invoke your Lambda functions over HTTP(S), meaning you can easily integrate Forecast predictions into your applications.

Why other options are incorrect:

A. Deploy an Amazon SageMaker model. Create a SageMaker endpoint for inference. While SageMaker is a valuable tool, this option skips the crucial training stage, which the scenario requires. Deploying a model before training is not possible or makes sense.
C. Configure an AWS Lambda function with a function URL that uses Amazon SageMaker endpoints to create predictions based on the inputs. This option is viable, but it should follow training the model first. In addition, using Amazon Forecast, which the scenario is looking for, would be a much simpler and fully managed approach.
E. Train an Amazon Forecast predictor by using the historical data in the S3 bucket. While this option is valid as the data is used for training a model, it is paired with choice A which isn't valid.

Amazon Forecast is the superior choice compared to SageMaker in this particular scenario because the company explicitly states that it has no ML experience and wants to use a managed service. Forecast abstracts away the complexities of choosing algorithms, feature engineering, and model tuning, allowing the company to focus on their manufacturing predictions.

Supporting Links:

Amazon SageMaker: https://aws.amazon.com/sagemaker/
Amazon Forecast: https://aws.amazon.com/forecast/
AWS Lambda Function URLs: https://aws.amazon.com/blogs/compute/introducing-aws-lambda-function-urls-built-in-https-endpoints-for-single-function-microservices/

Question: 688 **Exam Heist**

A company manages AWS accounts in AWS Organizations. AWS IAM Identity Center (AWS Single Sign-On) and AWS Control Tower are configured for the accounts. The company wants to manage multiple user permissions across all the accounts.

The permissions will be used by multiple IAM users and must be split between the developer and administrator teams. Each team requires different permissions. The company wants a solution that includes new users that are hired on both teams.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create individual users in IAM Identity Center for each account. Create separate developer and administrator groups in IAM Identity Center. Assign the users to the appropriate groups. Create a custom IAM policy for each group to set fine-grained permissions.

B.Create individual users in IAM Identity Center for each account. Create separate developer and administrator groups in IAM Identity Center. Assign the users to the appropriate groups. Attach AWS managed IAM policies to each user as needed for fine-grained permissions.

C.Create individual users in IAM Identity Center. Create new developer and administrator groups in IAM Identity Center. Create new permission sets that include the appropriate IAM policies for each group. Assign the new groups to the appropriate accounts. Assign the new permission sets to the new groups. When new users are hired, add them to the appropriate group.

D.Create individual users in IAM Identity Center. Create new permission sets that include the appropriate IAM policies for each user. Assign the users to the appropriate accounts. Grant additional IAM permissions to the users from within specific accounts. When new users are hired, add them to IAM Identity Center and assign them to the accounts.

Answer: C

Explanation:

The correct answer is C because it leverages AWS IAM Identity Center's permission sets for centralized permission management, minimizing operational overhead. Here's a detailed justification:

Centralized Management with IAM Identity Center: AWS IAM Identity Center (successor to AWS Single Sign-On) is designed for managing user access across multiple AWS accounts within an AWS Organization. This centralizes identity and access management, reducing the complexity of managing individual IAM users in each account.
Permission Sets for Role-Based Access: Permission sets define a collection of IAM policies that grant specific permissions. By creating developer and administrator permission sets, the company can define granular permissions for each role.
Group-Based Assignment: Assigning permission sets to IAM Identity Center groups (developer and administrator) simplifies user management. When a new user joins a team, they are added to the appropriate group, and they automatically inherit the corresponding permissions. This approach avoids managing permissions on a per-user basis.
IAM Policies within Permission Sets: Permission sets contain IAM policies, which are JSON documents that define permissions. This enables administrators to define precise access controls for each team.
Least Operational Overhead: This solution offers the least operational overhead because new users are added to the appropriate group, rather than creating new policies/users for each account individually. This method significantly reduces the manual work required to manage permissions, especially as the number of users and accounts grows.
AWS Control Tower Integration: The question mentions AWS Control Tower, which works well with IAM Identity Center to enforce governance and compliance across AWS accounts. Control Tower helps establish guardrails and policies, while IAM Identity Center manages user access.
Incorrect Alternatives:

A & B: Creating individual users in IAM Identity Center for each account is not scalable or optimal, as it would create an admin overhead, and defeat the goal of a single managed solution.
D: Assigning different permissions to different accounts for the same user is a high-overhead solution and a cumbersome method.
Alternatives B and D are not optimal because they do not correctly apply IAM policies to the appropriate groups.

Supporting Documentation:

AWS IAM Identity Center (successor to AWS Single Sign-On) Documentation
AWS Control Tower Documentation
IAM Policies

Question: 689 **Exam Heist**

A company wants to standardize its Amazon Elastic Block Store (Amazon EBS) volume encryption strategy. The company also wants to minimize the cost and configuration effort required to operate the volume encryption check.

Which solution will meet these requirements?

A.Write API calls to describe the EBS volumes and to confirm the EBS volumes are encrypted. Use Amazon EventBridge to schedule an AWS Lambda function to run the API calls.

B.Write API calls to describe the EBS volumes and to confirm the EBS volumes are encrypted. Run the API calls on an AWS Fargate task.

C.Create an AWS Identity and Access Management (IAM) policy that requires the use of tags on EBS volumes. Use AWS Cost Explorer to display resources that are not properly tagged. Encrypt the untagged resources manually.

D.Create an AWS Config rule for Amazon EBS to evaluate if a volume is encrypted and to flag the volume if it is not encrypted.

Answer: D

Explanation:

The correct answer is D because it leverages AWS Config, a service specifically designed for evaluating the configuration of AWS resources. AWS Config rules can automatically check whether EBS volumes are encrypted and flag those that are not, ensuring continuous compliance with the desired encryption strategy. This automated approach minimizes both configuration effort and ongoing operational costs. Option A and B involve custom scripting and scheduling (Lambda/Fargate), which introduce unnecessary complexity and management overhead compared to using AWS Config's built-in capabilities. While these options are technically feasible, they don't represent the most cost-effective and streamlined solution. Option C, relying on IAM policies and tagging, only prevents unencrypted volumes from being created if the policies are correctly configured. It also relies on Cost Explorer which is not designed to alert on configuration compliance. It doesn't automatically detect or flag existing unencrypted volumes, nor does it provide continuous monitoring of encryption status. Thus, AWS Config provides the most suitable solution for standardizing and enforcing EBS volume encryption with minimal operational overhead.

Supporting documentation:

AWS Config: https://aws.amazon.com/config/
AWS Config managed rules for EBS encryption:
https://docs.aws.amazon.com/config/latest/developerguide/managed-rules-by-aws-config.html

Question: 690 **Exam Heist**

A company regularly uploads GB-sized files to Amazon S3. After the company uploads the files, the company uses a fleet of Amazon EC2 Spot Instances to transcode the file format. The company needs to scale throughput when the company uploads data from the on-premises data center to Amazon S3 and when the company downloads data from Amazon S3 to the EC2 instances.

Which solutions will meet these requirements? (Choose two.)

A.Use the S3 bucket access point instead of accessing the S3 bucket directly.
B.Upload the files into multiple S3 buckets.
C.Use S3 multipart uploads.
D.Fetch multiple byte-ranges of an object in parallel.
E.Add a random prefix to each object when uploading the files.

Answer: CD

Explanation:

The question focuses on improving the throughput of data uploads from on-premises to S3 and downloads from S3 to EC2 instances for large files, especially considering EC2 Spot Instances and transcoding needs.

Option C, using S3 multipart uploads, directly addresses the need for increased throughput when uploading large files to S3. Multipart upload allows you to upload a single object as a set of parts. Each part can be uploaded independently and in parallel, improving upload speed and resilience to network interruptions. [https://docs.aws.amazon.com/AmazonS3/latest/userguide/mpuoverview.html]

Option D, fetching multiple byte-ranges of an object in parallel, improves the download throughput from S3 to the EC2 instances. By requesting different byte ranges concurrently, the EC2 instances can assemble the

complete file much faster than downloading it sequentially. This is particularly beneficial given the transcoding workload on the EC2 Spot Instances.
[https://docs.aws.amazon.com/AmazonS3/latest/userguide/range-get-http.html]

Option A, using S3 bucket access points, primarily controls access and permissions, not throughput. While access points can simplify management, they don't inherently boost upload or download speeds.

Option B, uploading files into multiple S3 buckets, might offer some partitioning benefits, but doesn't directly address the individual large file's upload/download speed like multipart upload and byte-range fetching.

Option E, adding a random prefix to each object, is primarily used to avoid hot partitions or uneven distribution of keys within S3. This helps improve performance and scalability at the S3 level, especially with a high volume of uploads, but it's not the primary solution for optimizing the throughput of individual file transfers. It mainly helps prevent throttling at the partition level within S3.

Therefore, options C and D directly enhance the throughput of large file uploads to S3 and downloads to EC2, meeting the company's requirements. Multipart upload accelerates uploads, and parallel byte-range fetching accelerates downloads.

Question: 691 **Exam Heist**

A solutions architect is designing a shared storage solution for a web application that is deployed across multiple Availability Zones. The web application runs on Amazon EC2 instances that are in an Auto Scaling group. The company plans to make frequent changes to the content. The solution must have strong consistency in returning the new content as soon as the changes occur.

Which solutions meet these requirements? (Choose two.)

A.Use AWS Storage Gateway Volume Gateway Internet Small Computer Systems Interface (iSCSI) block storage that is mounted to the individual EC2 instances.

B.Create an Amazon Elastic File System (Amazon EFS) file system. Mount the EFS file system on the individual EC2 instances.

C.Create a shared Amazon Elastic Block Store (Amazon EBS) volume. Mount the EBS volume on the individual EC2 instances.

D.Use AWS DataSync to perform continuous synchronization of data between EC2 hosts in the Auto Scaling group.

E.Create an Amazon S3 bucket to store the web content. Set the metadata for the Cache-Control header to no-cache. Use Amazon CloudFront to deliver the content.

Answer: BE

Explanation:

The requirement for strong consistency in returning new content immediately after changes rules out eventually consistent solutions like Amazon S3.

B. Create an Amazon Elastic File System (Amazon EFS) file system. Mount the EFS file system on the individual EC2 instances. This is a suitable option because Amazon EFS provides a shared file system that can be mounted concurrently by multiple EC2 instances across different Availability Zones. EFS offers strong consistency for file system metadata operations (like renaming files) and close-to-open consistency for data reads and writes. This ensures that once a write operation completes, subsequent reads from any EC2 instance will reflect the new content, fulfilling the strong consistency requirement.
https://aws.amazon.com/efs/

E. Create an Amazon S3 bucket to store the web content. Set the metadata for the Cache-Control header to no-cache. Use Amazon CloudFront to deliver the content. While S3 is great for storage, it's only eventually

consistent. However, using CloudFront with Cache-Control: no-cache helps mitigate this. Cache-Control: no-cache instructs CloudFront to always validate with the origin (S3) before serving content. This effectively forces CloudFront to retrieve the latest version from S3 frequently, thus satisfying the requirement for returning the new content as soon as the changes occur. Despite the presence of a cache, using no-cache transforms it into a validation proxy rather than a classic cache where stale content could be served. This ensures that updates are swiftly reflected to users.
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Expiration.html

Why other options are not suitable:

A. Use AWS Storage Gateway Volume Gateway Internet Small Computer Systems Interface (iSCSI) block storage that is mounted to the individual EC2 instances. Volume Gateway doesn't inherently provide shared access between multiple EC2 instances across different Availability Zones in a way that guarantees consistency for rapidly changing web content without complex application-level coordination and management.
C. Create a shared Amazon Elastic Block Store (Amazon EBS) volume. Mount the EBS volume on the individual EC2 instances. EBS volumes cannot be simultaneously attached to multiple EC2 instances except in certain limited scenarios with specific Nitro-based instances and clustered applications. This option is therefore fundamentally unsuitable for a web application deployed across multiple Availability Zones and relying on Auto Scaling.
D. Use AWS DataSync to perform continuous synchronization of data between EC2 hosts in the Auto Scaling group. While DataSync excels at data transfer, it does not provide the strong consistency needed. DataSync performs asynchronous transfers, meaning changes will not be immediately reflected across all instances. The application would experience inconsistencies during the synchronization window. Furthermore, using DataSync between multiple EC2 instances in an Auto Scaling group is an anti-pattern and extremely complex to manage reliably.

Question: 692 **Exam Heist**

A company is deploying an application in three AWS Regions using an Application Load Balancer. Amazon Route 53 will be used to distribute traffic between these Regions.

Which Route 53 configuration should a solutions architect use to provide the MOST high-performing experience?

A.Create an A record with a latency policy.
B.Create an A record with a geolocation policy.
C.Create a CNAME record with a failover policy.
D.Create a CNAME record with a geoproximity policy.

Answer: B

Explanation:

The correct answer is B: Create an A record with a geolocation policy.

Here's why:

Geolocation Routing: Geolocation routing in Route 53 directs traffic based on the geographic location of the user making the request. This means users are sent to the AWS Region that is geographically closest to them. This minimizes latency, providing a faster and more responsive experience for the user. This directly addresses the "MOST high-performing experience" requirement.

Latency Routing (Option A): While latency routing also aims to minimize latency, it relies on measuring the latency from Route 53's DNS servers to the application endpoints. This measurement might not perfectly

reflect the actual latency experienced by end users, especially users far from the DNS servers. Geolocation is more accurate.

CNAME Records (Options C & D): CNAME records map a domain name to another domain name, not directly to IP addresses. Using CNAME records would add an extra DNS lookup, which could increase latency. A records map a domain name directly to an IP address.

Failover Policy (Option C): A failover policy is primarily used for high availability and disaster recovery, not for optimizing performance based on location. It directs traffic to a secondary region only when the primary region is unavailable.

Geoproximity Routing (Option D): Geoproximity routing routes traffic based on geographic proximity with the option of biasing traffic to specific locations. While useful, it's more complex to configure and may not provide as consistent a performance improvement as simple geolocation-based routing when the goal is merely to direct users to the nearest region. It also typically requires Route 53 Traffic Flow which adds extra cost.

In summary, geolocation routing provides the most direct and effective way to minimize latency and improve performance for users across multiple AWS Regions, aligning perfectly with the prompt's requirement for the "MOST high-performing experience." It maps user location to the closest AWS region, unlike latency routing that depends on less precise measurements, and avoids the additional DNS lookup associated with CNAME records.

Supporting Documentation:

Route 53 Geolocation Routing: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html#routing-policy-geo
Route 53 Routing Policies: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html

Question: 693 **Exam Heist**

A company has a web application that includes an embedded NoSQL database. The application runs on Amazon EC2 instances behind an Application Load Balancer (ALB). The instances run in an Amazon EC2 Auto Scaling group in a single Availability Zone.

A recent increase in traffic requires the application to be highly available and for the database to be eventually consistent.

Which solution will meet these requirements with the LEAST operational overhead?

A.Replace the ALB with a Network Load Balancer. Maintain the embedded NoSQL database with its replication service on the EC2 instances.

B.Replace the ALB with a Network Load Balancer. Migrate the embedded NoSQL database to Amazon DynamoDB by using AWS Database Migration Service (AWS DMS).

C.Modify the Auto Scaling group to use EC2 instances across three Availability Zones. Maintain the embedded NoSQL database with its replication service on the EC2 instances.

D.Modify the Auto Scaling group to use EC2 instances across three Availability Zones. Migrate the embedded NoSQL database to Amazon DynamoDB by using AWS Database Migration Service (AWS DMS).

Answer: D

Explanation:

Here's a detailed justification for why option D is the most appropriate solution:

The core requirements are high availability and eventual consistency for the application and its database, all while minimizing operational overhead. The existing architecture has several limitations: running in a single Availability Zone makes it vulnerable to AZ failures, and relying on an embedded NoSQL database within EC2

instances complicates replication and scaling, increasing operational burden.

Option D addresses these issues directly. Spanning the EC2 Auto Scaling group across three Availability Zones (AZs) inherently improves high availability. If one AZ experiences an outage, the application continues to run in the remaining AZs. [https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-availability-zones.html]

Migrating the embedded NoSQL database to Amazon DynamoDB fulfills the high availability and eventual consistency requirements while significantly reducing operational overhead. DynamoDB is a fully managed NoSQL database service that automatically handles replication, scaling, and patching. This eliminates the need for manual database administration, patching, and scaling efforts. DynamoDB also offers built-in features for high availability and eventual consistency, aligning perfectly with the requirements. [https://aws.amazon.com/dynamodb/] AWS DMS can efficiently handle the database migration.

Option A is incorrect because Network Load Balancers do not offer significant advantages over Application Load Balancers in this specific scenario for web applications and do not address the database's availability or scalability issues. It maintains the complexities of managing an embedded NoSQL database.

Option B suffers from the same database management issue as Option A, while also utilizing Network Load Balancers which aren't optimal for this specific web application. It solves the DB portion but not the AZ part.

Option C addresses the Availability Zone issue, but keeps the complexity of managing the embedded NoSQL database. This significantly increases the operational overhead. Maintaining replication within the EC2 instances requires manual configuration, monitoring, and troubleshooting.Therefore, only Option D addresses both the availability of the application (through multi-AZ deployment) and the availability and manageability of the database (through DynamoDB), while minimizing operational overhead through a fully managed database service.

Question: 694 **Exam Heist**

A company is building a shopping application on AWS. The application offers a catalog that changes once each month and needs to scale with traffic volume. The company wants the lowest possible latency from the application. Data from each user's shopping cart needs to be highly available. User session data must be available even if the user is disconnected and reconnects.

What should a solutions architect do to ensure that the shopping cart data is preserved at all times?

A.Configure an Application Load Balancer to enable the sticky sessions feature (session affinity) for access to the catalog in Amazon Aurora.

B.Configure Amazon ElastiCache for Redis to cache catalog data from Amazon DynamoDB and shopping cart data from the user's session.

C.Configure Amazon OpenSearch Service to cache catalog data from Amazon DynamoDB and shopping cart data from the user's session.

D.Configure an Amazon EC2 instance with Amazon Elastic Block Store (Amazon EBS) storage for the catalog and shopping cart. Configure automated snapshots.

Answer: B

Explanation:

The correct solution leverages Amazon ElastiCache for Redis to ensure highly available shopping cart data preservation. Here's why:

Option A is incorrect because sticky sessions provided by an Application Load Balancer only ensure that a user's requests are routed to the same backend server. While it can improve performance by keeping session data in memory on that server, it doesn't guarantee data persistence if that server fails. Session affinity is not

a reliable method for highly available data storage.

Option C is incorrect because while OpenSearch Service is suitable for searching and analyzing large volumes of data, it's not the optimal choice for caching frequently accessed data like shopping carts. OpenSearch is not designed for low-latency, key-value retrieval in the same way that a caching service like Redis is. Additionally, using OpenSearch for session state management is not a standard practice.

Option D is not ideal as EC2 instances with EBS volumes lack the high availability and scalability required for shopping cart data. Relying on a single EC2 instance creates a single point of failure. Automated snapshots are a good backup strategy, but they don't provide real-time data availability during instance failures. Recovery from snapshots also involves downtime.

Option B utilizes Amazon ElastiCache for Redis, a highly available, in-memory data store, is ideal for caching shopping cart data and user session data. Redis offers very low latency read and write operations, satisfying the requirement for low latency. Crucially, ElastiCache provides managed Redis clusters with automatic failover, ensuring high availability. If a Redis node fails, ElastiCache automatically promotes a replica to the primary role, minimizing downtime. By caching catalog data from DynamoDB in ElastiCache, the solution also improves the performance of catalog access. Furthermore, Redis's persistence options (RDB snapshots and AOF) can provide durability for the shopping cart data, although, for shopping carts, eventual consistency can usually be tolerated in the event of a loss in the cache.

Relevant documentation:

Amazon ElastiCache for Redis: https://aws.amazon.com/elasticache/redis/
Redis Data Persistence: https://redis.io/docs/management/persistence/
Application Load Balancer sticky sessions:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/sticky-sessions.html

Question: 695 **Exam Heist**

A company is building a microservices-based application that will be deployed on Amazon Elastic Kubernetes Service (Amazon EKS). The microservices will interact with each other. The company wants to ensure that the application is observable to identify performance issues in the future.

Which solution will meet these requirements?

A.Configure the application to use Amazon ElastiCache to reduce the number of requests that are sent to the microservices.

B.Configure Amazon CloudWatch Container Insights to collect metrics from the EKS clusters. Configure AWS X-Ray to trace the requests between the microservices.

C.Configure AWS CloudTrail to review the API calls. Build an Amazon QuickSight dashboard to observe the microservice interactions.

D.Use AWS Trusted Advisor to understand the performance of the application.

Answer: B

Explanation:

Option B is the correct solution because it directly addresses the observability requirements for a microservices application deployed on Amazon EKS.

Amazon CloudWatch Container Insights: This service is specifically designed to collect, aggregate, and summarize container logs and metrics from containerized applications and microservices. It provides insights into the performance of EKS clusters, including CPU utilization, memory usage, network traffic, and disk I/O at the cluster, node, pod, and container levels. This detailed visibility is crucial for identifying performance bottlenecks within the EKS environment.

https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Container-Insights.html

AWS X-Ray: X-Ray is a distributed tracing system that helps developers analyze and debug production, distributed applications, such as those built using a microservices architecture. It tracks requests as they travel through the different microservices, providing a trace map that shows the latency and dependencies between services. This allows developers to pinpoint which microservice is causing performance issues or errors. https://docs.aws.amazon.com/xray/latest/devguide/aws-xray.html

Together, CloudWatch Container Insights and X-Ray offer a comprehensive observability solution. Container Insights monitors the performance of the EKS infrastructure, while X-Ray traces requests across microservices, enabling the company to identify and resolve performance issues in their microservices-based application.

Let's examine why the other options are incorrect:

Option A: ElastiCache is a caching service that can improve application performance by reducing database load, but it does not provide observability into the microservices themselves. Caching can be part of an overall solution, but doesn't fulfill the observability requirement.

Option C: CloudTrail tracks API calls made to AWS services. While useful for auditing and security, it doesn't provide insight into the performance or interactions of the microservices themselves. QuickSight can visualize data, but it relies on data sources; CloudTrail isn't the right source for application performance.

Option D: Trusted Advisor provides recommendations on cost optimization, security, fault tolerance, and performance, but it doesn't offer detailed observability data into the internal workings of the microservices application. It's a high-level check, not a monitoring tool.

Question: 696 **Exam Heist**

A company needs to provide customers with secure access to its data. The company processes customer data and stores the results in an Amazon S3 bucket.

All the data is subject to strong regulations and security requirements. The data must be encrypted at rest. Each customer must be able to access only their data from their AWS account. Company employees must not be able to access the data.

Which solution will meet these requirements?

A.Provision an AWS Certificate Manager (ACM) certificate for each customer. Encrypt the data client-side. In the private certificate policy, deny access to the certificate for all principals except an IAM role that the customer provides.

B.Provision a separate AWS Key Management Service (AWS KMS) key for each customer. Encrypt the data server-side. In the S3 bucket policy, deny decryption of data for all principals except an IAM role that the customer provides.

C.Provision a separate AWS Key Management Service (AWS KMS) key for each customer. Encrypt the data server-side. In each KMS key policy, deny decryption of data for all principals except an IAM role that the customer provides.

D.Provision an AWS Certificate Manager (ACM) certificate for each customer. Encrypt the data client-side. In the public certificate policy, deny access to the certificate for all principals except an IAM role that the customer provides.

Answer: C

Explanation:

The correct solution is C, which involves using separate AWS KMS keys for each customer and controlling access through KMS key policies. Let's break down why:

Data Encryption at Rest: The requirement of data encryption at rest is crucial. Both client-side and server-side encryption address this, but server-side encryption is generally preferred for S3 as it simplifies key management on the client side. AWS KMS (Key Management Service) is the go-to service for managing encryption keys within AWS.
Customer-Specific Keys: Having a dedicated KMS key per customer ensures strong isolation. If one key is compromised (theoretically), it only affects data encrypted with that specific key, limiting the blast radius. This approach aligns well with strong regulatory and security requirements.
Access Control: Each customer should only be able to access their data. This requires fine-grained access control at the key level. This is precisely what KMS key policies are for.
Key Policy vs. Bucket Policy: S3 bucket policies can control access to objects within the bucket, but they cannot control access to the KMS key itself. To prevent unauthorized decryption of the data, you need to control who can use the KMS key to decrypt the data. Key policies are specifically designed to manage the use of the KMS key itself, including controlling who can encrypt or decrypt data with it.
Denying Access to Company Employees: The key policies can explicitly deny decryption permissions to company employees. This is a critical aspect of the solution to meet the requirement that company employees must not be able to access customer data.
Using IAM Roles: The solution mentions using IAM roles that the customers provide. Customers can create an IAM role within their AWS account and grant it permission to decrypt the data using the appropriate KMS key in the company's account. This allows for secure cross-account access.

Why other options are incorrect:

Options A and D (Using ACM Certificates): AWS Certificate Manager (ACM) is used for managing SSL/TLS certificates for securing network traffic (HTTPS). It's not designed for encrypting data at rest. ACM certificates are for encryption in transit, not at rest. Although you could technically perform client-side encryption using the certificate, it's not the intended use case, and KMS is a more appropriate solution. Additionally, ACM policies don't offer the same level of granular control over decryption as KMS key policies.
Option B (Using S3 Bucket Policy): While S3 bucket policies are important for overall bucket access control, they can't directly control access to the KMS key used for server-side encryption. The bucket policy can define who can access the encrypted objects, but the KMS key policy determines who can decrypt those objects.

Authoritative Links:

AWS KMS Key Policies: https://docs.aws.amazon.com/kms/latest/developerguide/key-policies.html
Amazon S3 Encryption: https://docs.aws.amazon.com/AmazonS3/latest/userguide/serv-side-encryption.html
AWS IAM Roles: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html

Question: 697 **Exam Heist**

A solutions architect creates a VPC that includes two public subnets and two private subnets. A corporate security mandate requires the solutions architect to launch all Amazon EC2 instances in a private subnet. However, when the solutions architect launches an EC2 instance that runs a web server on ports 80 and 443 in a private subnet, no external internet traffic can connect to the server.

What should the solutions architect do to resolve this issue?

A.Attach the EC2 instance to an Auto Scaling group in a private subnet. Ensure that the DNS record for the website resolves to the Auto Scaling group identifier.

B.Provision an internet-facing Application Load Balancer (ALB) in a public subnet. Add the EC2 instance to the target group that is associated with the ALEnsure that the DNS record for the website resolves to the ALB.

C.Launch a NAT gateway in a private subnet. Update the route table for the private subnets to add a default route to the NAT gateway. Attach a public Elastic IP address to the NAT gateway.

D.Ensure that the security group that is attached to the EC2 instance allows HTTP traffic on port 80 and HTTPS traffic on port 443. Ensure that the DNS record for the website resolves to the public IP address of the EC2

instance.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer, and why the other options are incorrect:

Justification for Option B (Correct)

Option B, provisioning an internet-facing Application Load Balancer (ALB) in a public subnet, and associating the EC2 instance in the private subnet as a target, is the standard and recommended approach for exposing web applications securely and scalably in AWS.

1. **Public Subnet ALB:** An ALB placed in a public subnet is reachable from the internet. It receives incoming HTTP/HTTPS requests on ports 80 and 443.
2. **Private Subnet EC2:** The EC2 instance running the web server resides securely in a private subnet, shielded from direct internet access. This aligns with the security mandate.
3. **ALB Target Group:** The EC2 instance is registered as a target within a target group associated with the ALB. The ALB forwards requests to the EC2 instance in the private subnet.
4. **DNS Resolution:** The DNS record for the website resolves to the ALB's DNS name. Therefore, all internet traffic directed to the website first goes to the ALB.
5. **Security Benefits:** This architecture provides enhanced security by hiding the EC2 instance behind the ALB. The ALB can handle SSL/TLS termination, protect against common web exploits (e.g., using AWS WAF), and provide health checks.
6. **Scalability and Availability:** ALBs are highly scalable and can distribute traffic across multiple EC2 instances in the private subnet, improving application availability and performance. The ALB can automatically scale resources based on traffic demand.

Why other options are incorrect:

Option A: While Auto Scaling helps with scaling and availability, it doesn't inherently solve the issue of exposing the web server to the internet from a private subnet. DNS resolution to an Auto Scaling group identifier is not standard practice and doesn't provide direct internet access to the instances.

Option C: NAT Gateways allow instances in a private subnet to initiate outbound traffic to the internet (e.g., to download updates or access external services). However, they do not allow unsolicited inbound traffic from the internet to reach the instances. A NAT Gateway is for outbound communication only.

Option D: Even if the security group allows inbound HTTP/HTTPS traffic, the EC2 instance is in a private subnet. Private subnets by definition do not have direct internet connectivity. A DNS record pointing to the EC2 instance's public IP address (if it had one, which it ideally shouldn't in this scenario) would not work because the instance is not directly accessible from the internet.

Authoritative Links:

Application Load Balancers:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html
VPC with Public and Private Subnets (NAT):
https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Scenario2.html
Security Groups: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html

Question: 698 **Exam Heist**

A company is deploying a new application to Amazon Elastic Kubernetes Service (Amazon EKS) with an AWS Fargate cluster. The application needs a storage solution for data persistence. The solution must be highly available and fault tolerant. The solution also must be shared between multiple application containers.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create Amazon Elastic Block Store (Amazon EBS) volumes in the same Availability Zones where EKS worker nodes are placed. Register the volumes in a StorageClass object on an EKS cluster. Use EBS Multi-Attach to share the data between containers.

B.Create an Amazon Elastic File System (Amazon EFS) file system. Register the file system in a StorageClass object on an EKS cluster. Use the same file system for all containers.

C.Create an Amazon Elastic Block Store (Amazon EBS) volume. Register the volume in a StorageClass object on an EKS cluster. Use the same volume for all containers.

D.Create Amazon Elastic File System (Amazon EFS) file systems in the same Availability Zones where EKS worker nodes are placed. Register the file systems in a StorageClass object on an EKS cluster. Create an AWS Lambda function to synchronize the data between file systems.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer and why the other options are not suitable, focusing on minimizing operational overhead, high availability, fault tolerance, and shared access within an EKS cluster using Fargate.

Option B (Create an Amazon Elastic File System (Amazon EFS) file system. Register the file system in a StorageClass object on an EKS cluster. Use the same file system for all containers.) is the best solution because it directly addresses the requirements with the least operational overhead. Amazon EFS is a fully managed, scalable, and highly available network file system ideal for sharing data between multiple containers in an EKS cluster. Registering the EFS file system with a StorageClass object simplifies provisioning and dynamic mounting of the file system to pods within the EKS cluster. Since EFS is a managed service, AWS handles the underlying infrastructure, including backups, replication, and scaling, minimizing operational tasks for the company. EFS is inherently designed for shared access, high availability across multiple Availability Zones, and durability. This eliminates the need for manual configuration of replication or synchronization mechanisms. Fargate doesn't manage underlying EC2 instances; therefore, storage solutions need to be network-based for them to attach.

Option A (Create Amazon Elastic Block Store (Amazon EBS) volumes in the same Availability Zones where EKS worker nodes are placed. Register the volumes in a StorageClass object on an EKS cluster. Use EBS Multi-Attach to share the data between containers.) is incorrect. While EBS Multi-Attach allows a single EBS volume to be attached to multiple instances, its application with Fargate presents challenges. Fargate manages the underlying compute infrastructure abstractly, making AZ placement of EBS volumes based on Fargate nodes less predictable and manageable. Also, EBS Multi-Attach is more complex to set up than EFS and might not perfectly align with the shared filesystem needs of multiple containers. The availability zones of the Fargate compute cannot be chosen.

Option C (Create an Amazon Elastic Block Store (Amazon EBS) volume. Register the volume in a StorageClass object on an EKS cluster. Use the same volume for all containers.) is incorrect. EBS volumes are block storage and are not inherently designed for shared access between multiple instances or containers concurrently. Attempting to share a single EBS volume directly among multiple containers typically leads to data corruption and is not a recommended practice without advanced clustering or locking mechanisms, which increases operational complexity. Moreover, EBS volumes are zonal resources, so they would require additional steps for high availability and fault tolerance. Fargate compute nodes live independently across different Availability Zones, making this solution prone to errors due to zonal affinity.

Option D (Create Amazon Elastic File System (Amazon EFS) file systems in the same Availability Zones where EKS worker nodes are placed. Register the file systems in a StorageClass object on an EKS cluster. Create an AWS Lambda function to synchronize the data between file systems.) is incorrect because it introduces unnecessary complexity. Creating multiple EFS file systems and synchronizing data between them using a Lambda function adds significant operational overhead. The Lambda function requires maintenance, monitoring, and configuration, increasing the management burden. A single EFS file system is already highly available and durable, making data synchronization redundant and less efficient. Fargate doesn't grant you control over underlying Availability Zone placements of compute nodes; therefore, choosing Availability Zones for different EFS instances is problematic.

In summary, EFS provides a native, highly available, and shared filesystem solution that tightly integrates with EKS and Fargate, requiring minimal configuration and management. It aligns perfectly with the requirement for data persistence and shared access with the least operational overhead.

Relevant links:

Amazon EFS: https://aws.amazon.com/efs/
Amazon EKS: https://aws.amazon.com/eks/
AWS Fargate: https://aws.amazon.com/fargate/

Question: 699 **Exam Heist**

A company has an application that uses Docker containers in its local data center. The application runs on a container host that stores persistent data in a volume on the host. The container instances use the stored persistent data.

The company wants to move the application to a fully managed service because the company does not want to manage any servers or storage infrastructure.

Which solution will meet these requirements?

A.Use Amazon Elastic Kubernetes Service (Amazon EKS) with self-managed nodes. Create an Amazon Elastic Block Store (Amazon EBS) volume attached to an Amazon EC2 instance. Use the EBS volume as a persistent volume mounted in the containers.

B.Use Amazon Elastic Container Service (Amazon ECS) with an AWS Fargate launch type. Create an Amazon Elastic File System (Amazon EFS) volume. Add the EFS volume as a persistent storage volume mounted in the containers.

C.Use Amazon Elastic Container Service (Amazon ECS) with an AWS Fargate launch type. Create an Amazon S3 bucket. Map the S3 bucket as a persistent storage volume mounted in the containers.

D.Use Amazon Elastic Container Service (Amazon ECS) with an Amazon EC2 launch type. Create an Amazon Elastic File System (Amazon EFS) volume. Add the EFS volume as a persistent storage volume mounted in the containers.

Answer: B

Explanation:

Here's a breakdown of why option B is the correct solution and why the others are not:

The requirement specifies a fully managed service where the company doesn't want to manage servers or storage infrastructure. This immediately points us toward using a serverless container orchestration solution with a fully managed storage service.

Option B: Use Amazon Elastic Container Service (Amazon ECS) with an AWS Fargate launch type. Create an Amazon Elastic File System (Amazon EFS) volume. Add the EFS volume as a persistent storage volume mounted in the containers.

Amazon ECS with Fargate: Fargate is a serverless compute engine for ECS that eliminates the need to manage EC2 instances. It's a fully managed solution.
Amazon EFS: EFS is a fully managed, scalable, elastic network file system. It can be mounted across multiple EC2 instances and, importantly, with Fargate tasks, providing persistent storage accessible to the containers without requiring server or storage management. This addresses the need for persistent data storage.
Mounting EFS Volume: The containers can be configured to mount the EFS volume, allowing them to read and write persistent data.

Why the other options are incorrect:

Option A: Amazon EKS with self-managed nodes and EBS: EKS can be used for container orchestration, but self-managed nodes involve managing EC2 instances. EBS is also tied to a specific EC2 instance, making it less suitable for shared, persistent storage in a dynamic container environment. This violates the "fully managed" requirement because you must maintain the underlying EC2 instances.
Option C: Amazon ECS with Fargate and S3: While Fargate is suitable, S3 is not designed as a file system for direct mounting into containers and accessing files like a typical file system. S3 is object storage, and you'd need additional layers or custom code to treat it as a persistent volume in this scenario. This adds unnecessary complexity and potentially impacts application performance compared to EFS. S3 is also optimized for storing large amounts of data that is typically not frequently modified within a running application.
Option D: Amazon ECS with EC2 launch type and EFS: The EC2 launch type requires you to manage the EC2 instances, directly contradicting the requirement for a fully managed service.

In summary, the correct answer provides a completely serverless and fully managed approach to container orchestration and persistent storage, meeting all the stated requirements.

Supporting Links:

AWS Fargate: https://aws.amazon.com/fargate/
Amazon EFS: https://aws.amazon.com/efs/
ECS Storage Configuration: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/ecs-storage-configuration.html

Question: 700 **Exam Heist**

A gaming company wants to launch a new internet-facing application in multiple AWS Regions. The application will use the TCP and UDP protocols for communication. The company needs to provide high availability and minimum latency for global users.

Which combination of actions should a solutions architect take to meet these requirements? (Choose two.)

A.Create internal Network Load Balancers in front of the application in each Region.
B.Create external Application Load Balancers in front of the application in each Region.
C.Create an AWS Global Accelerator accelerator to route traffic to the load balancers in each Region.
D.Configure Amazon Route 53 to use a geolocation routing policy to distribute the traffic.
E.Configure Amazon CloudFront to handle the traffic and route requests to the application in each Region

Answer: AC

Explanation:

Here's a breakdown of why options A and C are correct and why the others are not:

A. Create internal Network Load Balancers in front of the application in each Region. This is a good starting point. Network Load Balancers (NLBs) are designed for handling TCP and UDP traffic efficiently and at high

scale. Because the question states the application is internet-facing, it may be confusing to suggest internal NLBs. However, the NLBs are a necessary component of the overall solution because they manage traffic distribution within each region. They distribute traffic across multiple instances of the application for high availability within that region.

C. Create an AWS Global Accelerator accelerator to route traffic to the load balancers in each Region. AWS Global Accelerator is designed to improve the availability and performance of applications for global users. It uses the AWS global network to route user traffic to the optimal endpoint based on geographic location, network health, and application health. Using Global Accelerator in front of the NLBs in each region ensures users are directed to the closest and healthiest endpoint, reducing latency and improving the overall user experience. Global Accelerator works with both TCP and UDP.

Let's analyze why the other options are incorrect:

B. Create external Application Load Balancers in front of the application in each Region. Application Load Balancers (ALBs) primarily handle HTTP/HTTPS traffic. While ALBs could handle some of the traffic if the gaming application also has a web-based component, they do not support UDP, which is a key requirement. Therefore, ALBs alone are insufficient.

D. Configure Amazon Route 53 to use a geolocation routing policy to distribute the traffic. While Route 53's geolocation routing is useful for directing traffic based on user location, it relies on DNS resolution. This makes it slower to react to changes in network conditions or application health than AWS Global Accelerator. It doesn't offer the same performance advantages. Route 53 also involves client-side DNS caching, which can delay failover.

E. Configure Amazon CloudFront to handle the traffic and route requests to the application in each Region. Amazon CloudFront is a content delivery network (CDN) optimized for caching and delivering static and dynamic content. While CloudFront can accelerate web applications, it is not primarily designed for handling real-time, low-latency TCP and UDP traffic like a gaming application. While CloudFront can now support UDP through WebSockets, it doesn't provide the global routing and optimization benefits that Global Accelerator offers. Using CloudFront also might not fit with the architectural design of this application as other aspects of this question point towards Global Accelerator.

In summary, the combination of internal NLBs for regional high availability and AWS Global Accelerator for global routing and low latency is the best solution.

Relevant Documentation:

AWS Global Accelerator: https://aws.amazon.com/global-accelerator/
Network Load Balancer: https://aws.amazon.com/elasticloadbalancing/network-load-balancer/
Amazon Route 53: https://aws.amazon.com/route53/
Amazon CloudFront: https://aws.amazon.com/cloudfront/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/

Question: 701 **Exam Heist**

A city has deployed a web application running on Amazon EC2 instances behind an Application Load Balancer (ALB). The application's users have reported sporadic performance, which appears to be related to DDoS attacks originating from random IP addresses. The city needs a solution that requires minimal configuration changes and provides an audit trail for the DDoS sources.

Which solution meets these requirements?

A.Enable an AWS WAF web ACL on the ALB, and configure rules to block traffic from unknown sources.

B.Subscribe to Amazon Inspector. Engage the AWS DDoS Response Team (DRT) to integrate mitigating controls into the service.

C.Subscribe to AWS Shield Advanced. Engage the AWS DDoS Response Team (DRT) to integrate mitigating controls into the service.

D.Create an Amazon CloudFront distribution for the application, and set the ALB as the origin. Enable an AWS WAF web ACL on the distribution, and configure rules to block traffic from unknown sources

Answer: C

Explanation:

The correct answer is C because it offers a comprehensive solution to the described problem, addressing both mitigation and reporting of DDoS attacks with minimal configuration changes.

Here's a breakdown of why:

AWS Shield Advanced: This service provides enhanced DDoS protection beyond the standard DDoS protections included with AWS Shield Standard (which is automatically enabled for all AWS customers). Specifically, it provides deeper inspection of traffic patterns and offers advanced detection and mitigation techniques tailored to the specific application and attack vectors. It is designed for applications running on services like EC2, ELB, CloudFront, Global Accelerator, and Route 53. https://aws.amazon.com/shield/

AWS DDoS Response Team (DRT): A crucial advantage of Shield Advanced is direct access to the AWS DDoS Response Team. This team of experts can assist in understanding attack patterns and integrating custom mitigation controls tailored to the unique characteristics of the detected DDoS attacks. This hands-on expert support is vital for effectively responding to complex or rapidly evolving attacks.

Audit Trail and Reporting: Shield Advanced offers detailed reports and dashboards that provide visibility into DDoS events, including the source IP addresses and attack vectors used. This is a key requirement for the city, as they need an audit trail to understand the origins of the attacks.

Let's examine why the other options are less suitable:

Option A (AWS WAF on ALB): While AWS WAF can block traffic based on rules, it requires significant configuration and maintenance to identify and block unknown sources effectively, especially with rapidly changing IP addresses in a DDoS attack. It doesn't provide the deep inspection and automated mitigation capabilities of Shield Advanced. Furthermore, manually creating and maintaining rules under a DDoS attack's pressure can be cumbersome.

Option B (Amazon Inspector and AWS DRT): Amazon Inspector is a vulnerability management service, not a DDoS protection service. While DRT can provide mitigation support, subscribing to Inspector is unrelated to DDoS mitigation and won't solve the city's problem efficiently. Inspector focuses on security assessments and vulnerability detection within EC2 instances and container images, whereas the city needs immediate protection against external network attacks.

Option D (CloudFront with WAF): While using CloudFront as a CDN with WAF is a good security practice, it doesn't offer the same level of DDoS protection as Shield Advanced, especially against sophisticated attacks. Additionally, introducing CloudFront involves architectural changes to the application, which goes against the requirement of minimal configuration changes. Furthermore, CloudFront primarily protects the origin (ALB in this case) by caching content and absorbing some of the traffic, but a determined DDoS attack can still overload the ALB.

Therefore, AWS Shield Advanced and engagement of the DRT offer the most effective solution for mitigating DDoS attacks, providing a comprehensive audit trail, and requiring minimal architectural changes to the existing application. This makes it the best answer.

Question: 702 **Exam Heist**

A company copies 200 TB of data from a recent ocean survey onto AWS Snowball Edge Storage Optimized devices. The company has a high performance computing (HPC) cluster that is hosted on AWS to look for oil and gas deposits. A solutions architect must provide the cluster with consistent sub-millisecond latency and high-throughput access to the data on the Snowball Edge Storage Optimized devices. The company is sending the devices back to AWS.

Which solution will meet these requirements?

A.Create an Amazon S3 bucket. Import the data into the S3 bucket. Configure an AWS Storage Gateway file gateway to use the S3 bucket. Access the file gateway from the HPC cluster instances.

B.Create an Amazon S3 bucket. Import the data into the S3 bucket. Configure an Amazon FSx for Lustre file system, and integrate it with the S3 bucket. Access the FSx for Lustre file system from the HPC cluster instances.

C.Create an Amazon S3 bucket and an Amazon Elastic File System (Amazon EFS) file system. Import the data into the S3 bucket. Copy the data from the S3 bucket to the EFS file system. Access the EFS file system from the HPC cluster instances.

D.Create an Amazon FSx for Lustre file system. Import the data directly into the FSx for Lustre file system. Access the FSx for Lustre file system from the HPC cluster instances.

Answer: D

Explanation:

The requirement is to provide the HPC cluster with sub-millisecond latency and high-throughput access to 200 TB of data after importing it from Snowball Edge devices. Option D, creating an Amazon FSx for Lustre file system and directly importing the data into it, best meets these stringent requirements.

FSx for Lustre is specifically designed for high-performance workloads, including HPC. Its architecture provides the necessary low latency and high throughput. The data is directly imported into the Lustre file system, eliminating the need for intermediate storage or gateways, and reducing potential bottlenecks.

Options A, B, and C involve S3, which, while scalable and durable, isn't optimized for the sub-millisecond latency demanded by the HPC cluster. AWS Storage Gateway File Gateway (option A) introduces latency. While FSx for Lustre can be integrated with S3 (option B), directly importing the data circumvents S3 altogether, leading to lower latency and complexity. EFS (option C) is suitable for shared file storage but typically doesn't match the performance of FSx for Lustre for demanding HPC workloads. Furthermore, copying data from S3 to EFS adds unnecessary time and complexity. Importing directly to FSx for Lustre is the streamlined and performant choice, making it the most suitable solution.

https://aws.amazon.com/fsx/lustre/https://aws.amazon.com/hpc/

Question: 703 **Exam Heist**

A company has NFS servers in an on-premises data center that need to periodically back up small amounts of data to Amazon S3.

Which solution meets these requirements and is MOST cost-effective?

A.Set up AWS Glue to copy the data from the on-premises servers to Amazon S3.

B.Set up an AWS DataSync agent on the on-premises servers, and sync the data to Amazon S3.

C.Set up an SFTP sync using AWS Transfer for SFTP to sync data from on premises to Amazon S3.

D.Set up an AWS Direct Connect connection between the on-premises data center and a VPC, and copy the data to Amazon S3.

Answer: B

Explanation:

Here's a detailed justification for why option B (AWS DataSync) is the most cost-effective solution for backing up small amounts of data from on-premises NFS servers to Amazon S3, along with explanations of why the other options are less suitable:

Why AWS DataSync is the Best Choice:

AWS DataSync is specifically designed for efficient and automated data transfer between on-premises storage and AWS services like S3. It leverages a purpose-built agent installed on-premises to handle data transfer optimization, including compression, encryption, and incremental transfers. For small amounts of data, the ease of setup and the efficiency of DataSync's transfer mechanisms offer a distinct cost advantage. DataSync minimizes the amount of data transferred by only transferring changed blocks, which significantly reduces network bandwidth usage and S3 storage costs. The agent's management of the connection helps ensure reliable and secure data transfer without needing constant intervention. DataSync provides detailed monitoring and reporting, giving visibility into the transfer process. The pay-as-you-go pricing model of DataSync, which charges only for the data transferred, is ideal for periodic backups of small datasets, avoiding the fixed costs associated with other solutions.

Why Other Options are Less Suitable:

A. AWS Glue: AWS Glue is a fully managed extract, transform, and load (ETL) service. While it could be used to move data, it's fundamentally designed for data processing and transformation at scale, not simple file replication. The overhead of setting up Glue jobs and the associated compute resources make it overkill and more expensive for this use case. Furthermore, Glue is geared towards structured and semi-structured data, not necessarily unstructured files on an NFS share.

C. AWS Transfer for SFTP: AWS Transfer for SFTP (formerly AWS SFTP) is a managed SFTP service. While it facilitates secure file transfer, it requires setting up and managing SFTP clients and users on the on-premises side. This adds complexity and overhead compared to DataSync's agent-based approach. Additionally, Transfer for SFTP typically involves transferring entire files, potentially leading to higher bandwidth consumption compared to DataSync's incremental transfer capability.

D. AWS Direct Connect: AWS Direct Connect establishes a dedicated network connection between the on-premises data center and AWS. While providing high bandwidth and potentially lower latency, it is significantly more expensive than other options, especially for small data transfers. Direct Connect involves upfront costs (port fees, router configuration) and monthly recurring charges, making it unsuitable for infrequent backups of small datasets. This option is an overkill solution as this connection is not needed for a small amount of data.

Authoritative Links:

AWS DataSync: https://aws.amazon.com/datasync/
AWS Glue: https://aws.amazon.com/glue/
AWS Transfer Family: https://aws.amazon.com/aws-transfer-family/
AWS Direct Connect: https://aws.amazon.com/directconnect/

Question: 704 **Exam Heist**

An online video game company must maintain ultra-low latency for its game servers. The game servers run on Amazon EC2 instances. The company needs a solution that can handle millions of UDP internet traffic requests each second.

Which solution will meet these requirements MOST cost-effectively?

A.Configure an Application Load Balancer with the required protocol and ports for the internet traffic. Specify the EC2 instances as the targets.

B.Configure a Gateway Load Balancer for the internet traffic. Specify the EC2 instances as the targets.

C.Configure a Network Load Balancer with the required protocol and ports for the internet traffic. Specify the EC2 instances as the targets.

D.Launch an identical set of game servers on EC2 instances in separate AWS Regions. Route internet traffic to both sets of EC2 instances.

Answer: C

Explanation:

Here's a detailed justification for why option C, using a Network Load Balancer (NLB), is the most cost-effective solution for the online video game company's requirements:

The primary requirements are ultra-low latency and the ability to handle millions of UDP traffic requests per second for game servers running on EC2 instances.

Network Load Balancer (NLB): NLBs are designed for high performance and low latency, operating at the transport layer (Layer 4) of the OSI model. This makes them well-suited for UDP traffic. They can handle millions of requests per second and are optimized for TCP, UDP, and TLS traffic. NLBs provide static IP addresses per Availability Zone, which can be beneficial for game clients needing consistent endpoints. Crucially, NLBs offer the lowest latency compared to other load balancer types.

Application Load Balancer (ALB): ALBs operate at the application layer (Layer 7), offering features like content-based routing. However, these features add overhead, increasing latency compared to NLBs. While ALBs support UDP, they are not optimized for this protocol as much as NLBs are.

Gateway Load Balancer (GWLB): GWLBs are designed for integrating third-party network virtual appliances like firewalls and intrusion detection systems. They are not the best choice for directly load balancing game server traffic as their main purpose lies in routing traffic through security appliances. They also introduce more latency than NLBs.

Multi-Region Deployment: Launching identical game servers in multiple AWS Regions would increase cost significantly due to the duplication of infrastructure, data replication costs, and the complexity of managing deployments and data consistency across regions. While it improves availability, it is not cost-effective for the stated requirements, especially when a load balancer can handle the traffic within a single region.

The justification emphasizes the NLB's low latency and ability to handle UDP traffic efficiently, fitting the requirements for online games. Furthermore, compared to launching a separate set of EC2 instances in different AWS Regions, the NLB provides a cost-effective solution for load balancing the incoming UDP requests to the game servers. By comparison, the Gateway Load Balancer is for routing network traffic and the Application Load Balancer operates at a higher layer, resulting in higher latency, making them less ideal for the gaming use case.

Authoritative Links:

AWS Load Balancing Documentation: https://docs.aws.amazon.com/elasticloadbalancing/index.html
Network Load Balancer: https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html
Application Load Balancer:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html
Gateway Load Balancer:
https://docs.aws.amazon.com/elasticloadbalancing/latest/gateway/introduction.html

Question: 705 **Exam Heist**

A company runs a three-tier application in a VPC. The database tier uses an Amazon RDS for MySQL DB instance.

The company plans to migrate the RDS for MySQL DB instance to an Amazon Aurora PostgreSQL DB cluster. The company needs a solution that replicates the data changes that happen during the migration to the new database.

Which combination of steps will meet these requirements? (Choose two.)

A.Use AWS Database Migration Service (AWS DMS) Schema Conversion to transform the database objects.

B.Use AWS Database Migration Service (AWS DMS) Schema Conversion to create an Aurora PostgreSQL read replica on the RDS for MySQL DB instance.

C.Configure an Aurora MySQL read replica for the RDS for MySQL DB instance.

D.Define an AWS Database Migration Service (AWS DMS) task with change data capture (CDC) to migrate the data.

E.Promote the Aurora PostgreSQL read replica to a standalone Aurora PostgreSQL DB cluster when the replica lag is zero.

Answer: AD

Explanation:

The correct answer is A and D because these choices offer the most effective approach for migrating data from RDS for MySQL to Aurora PostgreSQL with minimal downtime and data loss.

Option A is correct because AWS DMS Schema Conversion is specifically designed to translate database schemas from one engine to another. MySQL and PostgreSQL have different syntaxes and data types, so converting the schema is a crucial first step before migrating the data. This ensures that the Aurora PostgreSQL database is ready to receive the data in the correct format.

Option D is correct because AWS DMS with Change Data Capture (CDC) is the recommended way to continuously replicate data changes from the source database to the target database. CDC captures all the updates, inserts, and deletes happening on the RDS for MySQL database and applies them to the Aurora PostgreSQL DB cluster. This ensures that the Aurora PostgreSQL database stays synchronized with the RDS for MySQL database during the migration process, minimizing downtime when switching over.

Option B is incorrect because AWS DMS Schema Conversion doesn't create read replicas. It focuses on schema transformation only.

Option C is incorrect because Aurora MySQL read replicas cannot be created for RDS for MySQL databases. Read replicas must be of the same database engine.

Option E is incorrect because Aurora PostgreSQL read replicas cannot be created directly from RDS for MySQL DB instance.

In summary, the combination of AWS DMS Schema Conversion and AWS DMS with CDC provides a robust solution for migrating data from RDS for MySQL to Aurora PostgreSQL with minimal downtime and data loss. DMS Schema Conversion handles the necessary schema translations, while DMS with CDC ensures continuous data replication, keeping the target database in sync with the source database.

Relevant links:

AWS Database Migration Service: https://aws.amazon.com/dms/
AWS DMS Schema Conversion:
https://docs.aws.amazon.com/dms/latest/userguide/CHAP_SchemaConversion.html
AWS DMS Change Data Capture (CDC):
https://docs.aws.amazon.com/dms/latest/userguide/CHAP_Task.CDC.html

Question: 706 **Exam Heist**

A company hosts a database that runs on an Amazon RDS instance that is deployed to multiple Availability Zones. The company periodically runs a script against the database to report new entries that are added to the database. The script that runs against the database negatively affects the performance of a critical application. The company needs to improve application performance with minimal costs.

Which solution will meet these requirements with the LEAST operational overhead?

A.Add functionality to the script to identify the instance that has the fewest active connections. Configure the script to read from that instance to report the total new entries.

B.Create a read replica of the database. Configure the script to query only the read replica to report the total new entries.

C.Instruct the development team to manually export the new entries for the day in the database at the end of each day.

D.Use Amazon ElastiCache to cache the common queries that the script runs against the database.

Answer: B

Explanation:

The correct answer is B: Create a read replica of the database. Configure the script to query only the read replica to report the total new entries.

Here's why this is the best solution:

Offloading Read Workload: Read replicas are designed specifically to offload read-heavy workloads from the primary database instance. This is exactly the problem the company is facing with its reporting script impacting application performance.
Minimal Operational Overhead: Creating a read replica is a straightforward process in RDS. Once created, the script can be reconfigured to point to the read replica. RDS handles replication automatically, minimizing operational overhead. There are no extra steps needed to create this solution like in option C.
Cost-Effective: Read replicas generally have lower costs than scaling the primary instance itself or implementing more complex solutions. This aligns with the requirement of minimizing costs.
No Application Code Changes (Mostly): The primary application should not need any modification, as it continues to read/write to the primary instance. Only the reporting script needs to be reconfigured to target the read replica.
Improved Application Performance: By directing the reporting script's queries to the read replica, the primary database instance is freed from the performance impact, allowing the critical application to perform optimally.
Replication Latency Consideration: While generally low, replication latency should be considered. Depending on the acceptable delay for reporting data, this might influence the choice of read replica (e.g., same Availability Zone for minimal latency).

Here's why the other options are not as suitable:

A: Adding connection-checking logic to the script is unnecessarily complex and doesn't fundamentally solve the problem of the script impacting the primary database. It only attempts to mitigate the impact.
C: Manual exporting is a tedious and error-prone process. It does not scale well and has high operational overhead. Also, it's not automated, contradicting the need for efficiency.
D: ElastiCache is suitable for caching frequently accessed data to improve read performance, but it doesn't directly address the problem of offloading the reporting workload. Furthermore, setting up and managing caching invalidation can introduce unnecessary complexity for this particular scenario.

Authoritative Links:

Amazon RDS Read Replicas:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Aurora.Managing.Replication.html

Question: 707 **Exam Heist**

A company is using an Application Load Balancer (ALB) to present its application to the internet. The company finds abnormal traffic access patterns across the application. A solutions architect needs to improve visibility into the infrastructure to help the company understand these abnormalities better.

What is the MOST operationally efficient solution that meets these requirements?

A.Create a table in Amazon Athena for AWS CloudTrail logs. Create a query for the relevant information.

B.Enable ALB access logging to Amazon S3. Create a table in Amazon Athena, and query the logs.

C.Enable ALB access logging to Amazon S3. Open each file in a text editor, and search each line for the relevant information.

D.Use Amazon EMR on a dedicated Amazon EC2 instance to directly query the ALB to acquire traffic access log information.

Answer: B

Explanation:

The most operationally efficient solution to analyze abnormal traffic patterns accessing an application via an Application Load Balancer (ALB) is to enable ALB access logging to Amazon S3 and then query the logs using Amazon Athena. This approach combines the benefits of comprehensive logging and cost-effective, serverless querying.

Option B is the best because ALB access logs provide detailed information about requests received by the ALB, including the source IP address, request path, latency, and response codes. Storing these logs in Amazon S3 provides a durable and scalable storage solution. Amazon Athena allows you to analyze the S3-based log data using standard SQL queries without the need for infrastructure management. This combination enables efficient analysis of traffic patterns and identification of anomalies.

Option A is less ideal because AWS CloudTrail logs are primarily for auditing API calls made to AWS services, not for capturing detailed request-level data for the application itself. While CloudTrail provides valuable insights, it does not provide the granularity needed to analyze abnormal traffic patterns.

Option C is inefficient because manually opening and searching through individual log files in a text editor is time-consuming, error-prone, and not scalable for large volumes of log data. This method doesn't leverage the advantages of automated analysis.

Option D is unnecessarily complex and expensive. Amazon EMR is designed for large-scale data processing and analysis, but using it on a dedicated EC2 instance solely to query the ALB directly is overkill for this use case. The overhead of managing and configuring EMR is significant compared to the simplicity of Athena. Additionally, querying the ALB directly is not a standard practice for retrieving access logs.

In conclusion, Option B offers a balance of comprehensive logging, scalable storage, and cost-effective analysis. This allows for better visibility into the infrastructure, thus empowering the company to efficiently understand traffic abnormalities.

Relevant Links:

Application Load Balancer Access Logs:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-access-logs.html
Amazon Athena: https://aws.amazon.com/athena/

Question: 708 **Exam Heist**

A company wants to use NAT gateways in its AWS environment. The company's Amazon EC2 instances in private subnets must be able to connect to the public internet through the NAT gateways.

Which solution will meet these requirements?

A.Create public NAT gateways in the same private subnets as the EC2 instances.
B.Create private NAT gateways in the same private subnets as the EC2 instances.
C.Create public NAT gateways in public subnets in the same VPCs as the EC2 instances.
D.Create private NAT gateways in public subnets in the same VPCs as the EC2 instances.

Answer: C

Explanation:

The correct solution is to create public NAT gateways in public subnets within the same VPC as the EC2 instances residing in private subnets. NAT gateways enable instances in private subnets to initiate outbound connections to the internet without allowing inbound connections from the internet.

Option C is correct because public NAT gateways require placement in public subnets. These subnets have a route to an internet gateway, allowing the NAT gateway to communicate with the internet. EC2 instances in private subnets then route their internet-bound traffic to the NAT gateway, which performs network address translation, replacing the private IP address of the instance with its own public IP address. The internet sees the traffic originating from the NAT gateway's public IP. The NAT gateway remembers the source and destination and forwards return traffic to the appropriate EC2 instance.

Option A is incorrect because NAT gateways should not be created in the same private subnets as the EC2 instances. While this technically could work in a custom setup, it does not utilize the intended functionality of a NAT gateway and introduces complexities. More importantly, NAT Gateways in private subnets are not designed to directly facilitate internet access.

Option B is incorrect because "private NAT gateways" are not a recognized AWS service or feature. NAT gateways are designed for outbound internet access, a function that inherently requires a route to the internet.

Option D is incorrect. While placing something within a public subnet is the first step, creating a "private NAT gateway" implies that it is internal to the VPC only. A NAT Gateway must be "public" to give instances a route to the internet.

In summary, creating a public NAT gateway in a public subnet provides a secure and managed way for instances in private subnets to connect to the internet.

Relevant resources:

AWS NAT Gateway Documentation: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html

Question: 709 **Exam Heist**

A company has an organization in AWS Organizations. The company runs Amazon EC2 instances across four AWS accounts in the root organizational unit (OU). There are three nonproduction accounts and one production account. The company wants to prohibit users from launching EC2 instances of a certain size in the nonproduction accounts. The company has created a service control policy (SCP) to deny access to launch instances that use the prohibited types.

Which solutions to deploy the SCP will meet these requirements? (Choose two.)

A.Attach the SCP to the root OU for the organization.

B.Attach the SCP to the three nonproduction Organizations member accounts.

C.Attach the SCP to the Organizations management account.

D.Create an OU for the production account. Attach the SCP to the OU. Move the production member account into the new OU.

E.Create an OU for the required accounts. Attach the SCP to the OU. Move the nonproduction member accounts into the new OU.

Answer: BE

Explanation:

The correct answer is B and E. Let's break down why:

Why B is correct: Attaching the SCP directly to the three non-production member accounts is a straightforward way to achieve the requirement. It directly targets the accounts where you want to restrict EC2 instance types. SCPs, when applied to an account, limit what identities (users, roles) within that account can do. This is a precise application of the policy, ensuring that only the desired non-production accounts are affected.

Why E is correct: Creating a dedicated OU for the non-production accounts and then attaching the SCP to that OU provides a more organized and scalable solution, especially if the number of non-production accounts is expected to grow. When an SCP is attached to an OU, it applies to all accounts contained within that OU. This approach simplifies management because you only need to maintain the policy at the OU level, rather than individually on each account.

Why A is incorrect: Attaching the SCP to the root OU would restrict instance types in all accounts within the organization, including the production account, which violates the requirement. SCPs applied at the root level act as the outermost limit.

Why C is incorrect: Attaching the SCP to the Organizations management account has no effect on the member accounts' permissions. The management account manages the organization but doesn't inherently restrict activities within its member accounts through SCPs.

Why D is incorrect: Creating an OU for the production account and attaching the SCP to that OU would essentially have no effect as the policy limits instance types, and it's being applied only to the production account which they want to exclude from the restriction.

Therefore, options B and E directly address the stated requirements by applying the restriction to the specific non-production environments, either individually or as a grouped OU.

Here are some authoritative links for further research:

AWS Organizations documentation on Service Control Policies (SCPs)
AWS Organizations concepts

Question: 710 **Exam Heist**

A company’s website hosted on Amazon EC2 instances processes classified data stored in Amazon S3. Due to security concerns, the company requires a private and secure connection between its EC2 resources and Amazon S3.

Which solution meets these requirements?

A.Set up S3 bucket policies to allow access from a VPC endpoint.

B.Set up an IAM policy to grant read-write access to the S3 bucket.

C.Set up a NAT gateway to access resources outside the private subnet.

D.Set up an access key ID and a secret access key to access the S3 bucket.

Answer: A

Explanation:

The correct solution is to set up S3 bucket policies to allow access from a VPC endpoint. Here's why:

The core requirement is a private and secure connection between EC2 instances and S3, without traversing the public internet. VPC endpoints for S3 address this directly by creating a secure, private connection within the AWS network.

Option A leverages VPC endpoints, specifically Gateway Endpoints for S3. These endpoints allow EC2 instances in a VPC to access S3 without requiring internet access or NAT gateways. Traffic destined for S3 from the EC2 instances remains within the AWS network, enhancing security and reducing data transfer costs. The S3 bucket policy is then configured to only allow access from the specified VPC endpoint, further restricting access and enforcing the private connection requirement. This creates a secure channel, as traffic never leaves the AWS network and is isolated from external access.

Option B, setting up an IAM policy, while necessary for granting permissions, does not guarantee a private connection. IAM policies control what actions an EC2 instance can perform on S3, but the traffic might still traverse the internet if there isn't a mechanism to keep it within AWS.

Option C, setting up a NAT gateway, is the opposite of what is required. A NAT gateway allows instances in a private subnet to initiate outbound traffic to the internet, which violates the security requirement of a private connection to S3. It is used when the connection is internet-based.

Option D, setting up an access key ID and a secret access key, also doesn't address the private connection requirement. These keys are used for authentication, but the traffic to S3 can still go over the internet. Moreover, hardcoding access keys in EC2 instances is a security risk. Using IAM roles is a best practice.

In summary, VPC endpoints in conjunction with restrictive S3 bucket policies provide the private and secure connection required, fulfilling the security concerns outlined in the problem. The traffic from EC2 to S3 will not go through the internet, and the bucket policy will restrict access to the specific VPC endpoint.

For further research, refer to these AWS documentation links:

VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html
S3 Bucket Policies: https://docs.aws.amazon.com/AmazonS3/latest/userguide/bucket-policies.html

Question: 711 **Exam Heist**

An ecommerce company runs its application on AWS. The application uses an Amazon Aurora PostgreSQL cluster in Multi-AZ mode for the underlying database. During a recent promotional campaign, the application experienced heavy read load and write load. Users experienced timeout issues when they attempted to access the application.

A solutions architect needs to make the application architecture more scalable and highly available.

Which solution will meet these requirements with the LEAST downtime?

A.Create an Amazon EventBridge rule that has the Aurora cluster as a source. Create an AWS Lambda function to log the state change events of the Aurora cluster. Add the Lambda function as a target for the EventBridge rule. Add additional reader nodes to fail over to.

B.Modify the Aurora cluster and activate the zero-downtime restart (ZDR) feature. Use Database Activity Streams on the cluster to track the cluster status.

C.Add additional reader instances to the Aurora cluster. Create an Amazon RDS Proxy target group for the

Aurora cluster.

D.Create an Amazon ElastiCache for Redis cache. Replicate data from the Aurora cluster to Redis by using AWS Database Migration Service (AWS DMS) with a write-around approach.

Answer: C

Explanation:

The best solution to improve scalability and availability of the Aurora PostgreSQL cluster with the least downtime is option C: adding read replicas and implementing RDS Proxy.

Adding Aurora read replicas (reader instances) allows offloading read traffic from the primary Aurora instance. This immediately alleviates the read load on the primary, improving application performance and reducing timeouts. Aurora automatically replicates data to these read replicas, ensuring data consistency. [https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.ReadReplicas.html]

RDS Proxy acts as a connection pooler in front of the database. By pooling database connections, it reduces the overhead of establishing new connections for each request. This improves application response times and protects the database from being overwhelmed by a large number of concurrent connections, especially during peak periods. It enhances availability by allowing the application to continue operating even if the primary database instance experiences temporary issues. [https://aws.amazon.com/rds/proxy/]

Option A is incorrect because EventBridge and Lambda are not directly involved in scaling or improving the availability of the database itself. They are used for monitoring cluster state changes, not for addressing performance issues. Adding read nodes to fail over to does not solve the load issue as read queries still go to the primary node.

Option B is incorrect because Zero-Downtime Restart (ZDR) primarily addresses planned maintenance events and does not help with scaling for heavy read/write loads. Database Activity Streams is for auditing purposes, not performance optimization.

Option D involves setting up ElastiCache with DMS. This requires significant time to set up and synchronize data. Implementing a write-around cache can also introduce complexities with data consistency and is generally more suitable for scenarios with very high read-to-write ratios. It's also more complex and involves more downtime compared to simply adding read replicas. It would be better if the company had the cache implemented and tested before the promotional campaign. Therefore, this option takes time and has a downtime risk.

Question: 712 **Exam Heist**

A company is designing a web application on AWS. The application will use a VPN connection between the company’s existing data centers and the company's VPCs.

The company uses Amazon Route 53 as its DNS service. The application must use private DNS records to communicate with the on-premises services from a VPC.

Which solution will meet these requirements in the MOST secure manner?

A.Create a Route 53 Resolver outbound endpoint. Create a resolver rule. Associate the resolver rule with the VPC.

B.Create a Route 53 Resolver inbound endpoint. Create a resolver rule. Associate the resolver rule with the VPC.

C.Create a Route 53 private hosted zone. Associate the private hosted zone with the VPC.

D.Create a Route 53 public hosted zone. Create a record for each service to allow service communication

Answer: A

Explanation:

The correct answer is **A. Create a Route 53 Resolver outbound endpoint. Create a resolver rule. Associate the resolver rule with the VPC.**

Here's a detailed justification:

The scenario requires resolving private DNS records of on-premises services from within an AWS VPC. This means that DNS queries originating from the VPC need to be forwarded to the on-premises DNS servers for resolution.

Route 53 Resolver Outbound Endpoint: An outbound endpoint allows DNS queries from the VPC to be forwarded to on-premises DNS servers. This is essential for resolving private DNS records hosted on-premises. It acts as a forwarder.
Resolver Rule: A resolver rule specifies the conditions under which DNS queries should be forwarded to the on-premises DNS servers via the outbound endpoint. For instance, a rule can specify that any query for a domain like "internal.example.com" should be forwarded to the on-premises DNS servers' IP addresses.
Associating the Rule with the VPC: This ensures that the resolver rule applies to all DNS queries originating from the specified VPC.

This approach is the most secure because it allows fine-grained control over which DNS queries are forwarded to the on-premises environment. You only forward queries that match your specific internal domain, minimizing the exposure of your VPC's DNS traffic.

Option B is incorrect because an inbound endpoint is used to resolve DNS records hosted in Route 53 from on-premises networks.

Option C is incorrect because a private hosted zone is used to host private DNS records within AWS, not for resolving on-premises records from AWS. While useful for internal AWS services, it does not facilitate communication with on-premises resources in this scenario.

Option D is incorrect because a public hosted zone is for publically accessible DNS records and is not applicable for resolving private on-premises records.

Supporting Links:

AWS Documentation on Route 53 Resolver:
https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html
AWS Documentation on Resolver Endpoints:
https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver-endpoints.html
AWS Knowledge Center: How do I configure Route 53 to resolve domain names in my private network from my Amazon VPC?: https://aws.amazon.com/premiumsupport/knowledge-center/route-53-resolve-private-domains/

Question: 713 **Exam Heist**

A company is running a photo hosting service in the us-east-1 Region. The service enables users across multiple countries to upload and view photos. Some photos are heavily viewed for months, and others are viewed for less than a week. The application allows uploads of up to 20 MB for each photo. The service uses the photo metadata to determine which photos to display to each user.

Which solution provides the appropriate user access MOST cost-effectively?

A.Store the photos in Amazon DynamoDB. Turn on DynamoDB Accelerator (DAX) to cache frequently viewed items.

B.Store the photos in the Amazon S3 Intelligent-Tiering storage class. Store the photo metadata and its S3 location in DynamoDB.

C.Store the photos in the Amazon S3 Standard storage class. Set up an S3 Lifecycle policy to move photos older than 30 days to the S3 Standard-Infrequent Access (S3 Standard-IA) storage class. Use the object tags to keep track of metadata.

D.Store the photos in the Amazon S3 Glacier storage class. Set up an S3 Lifecycle policy to move photos older than 30 days to the S3 Glacier Deep Archive storage class. Store the photo metadata and its S3 location in Amazon OpenSearch Service.

Answer: B

Explanation:

Here's a detailed justification for why option B is the most cost-effective solution for the described photo hosting service:

The core requirement is cost-effective storage and retrieval of photos with varying access patterns. Option B proposes storing photos in Amazon S3 Intelligent-Tiering. This is ideal because Intelligent-Tiering automatically moves data between frequent and infrequent access tiers based on usage patterns, optimizing cost without operational overhead. Heavily viewed photos remain in the more expensive, readily accessible tier, while rarely viewed photos are automatically moved to a cheaper tier. This automated tiering directly addresses the variable access pattern described in the question.

Furthermore, storing photo metadata and S3 location in DynamoDB is a sound strategy. DynamoDB provides fast, low-latency access to this metadata, which is essential for determining which photos to display to users. Unlike storing the entire photo in DynamoDB (option A), this approach avoids the high cost and performance limitations of storing large binary objects in a NoSQL database.

Option A is inefficient because storing photos directly in DynamoDB is generally not recommended due to the cost associated with storing large binary objects in DynamoDB. DAX can help with caching but doesn't address the fundamental inefficiency.

Option C is less optimal because it manually transitions photos to S3 Standard-IA after a fixed period (30 days). This might not be the most cost-effective strategy as it doesn't react dynamically to actual access patterns. Some photos viewed heavily for longer than 30 days will be prematurely moved to S3 Standard-IA, incurring retrieval costs if still accessed. Object tags are also less efficient than a dedicated database like DynamoDB for complex querying and filtering of metadata.

Option D utilizes Amazon S3 Glacier, which is primarily for archiving data that is infrequently accessed. Retrieval from Glacier is slow and costly, making it unsuitable for a photo hosting service where users need relatively quick access to photos, even if they aren't accessed frequently. OpenSearch Service is overkill for simply storing metadata and object locations and adds unnecessary complexity and cost.

In summary, using S3 Intelligent-Tiering for the photos and DynamoDB for the metadata offers the best balance of cost-effectiveness, performance, and manageability for the given scenario.

Supporting Links:

Amazon S3 Intelligent-Tiering: https://aws.amazon.com/s3/storage-classes/intelligent-tiering/
Amazon DynamoDB: https://aws.amazon.com/dynamodb/
S3 Lifecycle: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-management-overview.html

Question: 714 **Exam Heist**

A company runs a highly available web application on Amazon EC2 instances behind an Application Load Balancer.

The company uses Amazon CloudWatch metrics.

As the traffic to the web application increases, some EC2 instances become overloaded with many outstanding requests. The CloudWatch metrics show that the number of requests processed and the time to receive the responses from some EC2 instances are both higher compared to other EC2 instances. The company does not want new requests to be forwarded to the EC2 instances that are already overloaded.

Which solution will meet these requirements?

A.Use the round robin routing algorithm based on the RequestCountPerTarget and ActiveConnectionCount CloudWatch metrics.

B.Use the least outstanding requests algorithm based on the RequestCountPerTarget and ActiveConnectionCount CloudWatch metrics.

C.Use the round robin routing algorithm based on the RequestCount and TargetResponseTime CloudWatch metrics.

D.Use the least outstanding requests algorithm based on the RequestCount and TargetResponseTime CloudWatch metrics.

Answer: B

Explanation:

The problem describes a scenario where some EC2 instances behind an Application Load Balancer (ALB) are overloaded, leading to higher processing times and request counts. The goal is to prevent new requests from being routed to these overloaded instances. The correct solution utilizes the "least outstanding requests" algorithm, which is not natively supported by ALB. However, the best option is **B. Use the least outstanding requests algorithm based on the RequestCountPerTarget and ActiveConnectionCount CloudWatch metrics.**

Here's why:

Understanding the Need: Overloaded instances have a higher number of requests in flight. The ALB should ideally route new requests to instances that are less burdened, improving overall application performance and response times.

Least Outstanding Requests: The core principle is to direct traffic to instances with fewer pending requests. While the ALB doesn't directly offer a "least outstanding requests" routing algorithm, these values can be calculated for a custom solution.

RequestCountPerTarget and ActiveConnectionCount Metrics: These metrics provide insights into the current load on each instance:

RequestCountPerTarget: Represents the number of requests that were sent to each target.
ActiveConnectionCount: Represents the number of established connections from clients to the target.

Why other options are incorrect:

Round Robin: Round robin distributes requests evenly across all instances, regardless of their current load. This is unsuitable for handling overloaded instances because it will continue to send requests to them, exacerbating the problem.
TargetResponseTime: The TargetResponseTime metric is primarily focused on measuring the time it takes for the ALB to receive a response from the target. While helpful for monitoring, it's not directly used by the ALB for its native routing algorithms. It can be used for creating custom algorithms.

Implementation with CloudWatch metrics: You cannot directly configure ALB to use a least outstanding request algorithm based on CloudWatch metrics. The described solution would necessitate implementing a custom solution which leverages CloudWatch metrics to monitor instances and dynamically modify the target group weights associated with your instances. AWS services like Lambda could automate this.

Authoritative Links:

Application Load Balancer Listener Rules:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-listeners.html
CloudWatch Metrics for ALB: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-cloudwatch-metrics.html

In conclusion, while option B isn't a built-in ALB feature, it presents the most suitable approach by leveraging metrics that reflect instance load and creating a custom routing mechanism or auto-scaling policy based on those metrics.

Question: 715 **Exam Heist**

A company uses Amazon EC2, AWS Fargate, and AWS Lambda to run multiple workloads in the company's AWS account. The company wants to fully make use of its Compute Savings Plans. The company wants to receive notification when coverage of the Compute Savings Plans drops.

Which solution will meet these requirements with the MOST operational efficiency?

A.Create a daily budget for the Savings Plans by using AWS Budgets. Configure the budget with a coverage threshold to send notifications to the appropriate email message recipients.

B.Create a Lambda function that runs a coverage report against the Savings Plans. Use Amazon Simple Email Service (Amazon SES) to email the report to the appropriate email message recipients.

C.Create an AWS Budgets report for the Savings Plans budget. Set the frequency to daily.

D.Create a Savings Plans alert subscription. Enable all notification options. Enter an email address to receive notifications.

Answer: A

Explanation:

Here's a detailed justification for why option A is the best solution, focusing on operational efficiency and AWS best practices:

Option A leverages AWS Budgets' built-in capabilities for monitoring Savings Plans coverage. AWS Budgets allows defining a budget for Savings Plans and setting coverage thresholds. When the actual coverage falls below the defined threshold, AWS Budgets automatically sends notifications to specified email addresses. This approach provides a proactive and automated way to track coverage without requiring custom coding or infrastructure management. The daily budget frequency ensures timely alerts, allowing the company to address any coverage drops quickly.

Option B involves creating a Lambda function and using Amazon SES, requiring custom code development, deployment, and maintenance. While it provides flexibility, it increases operational overhead compared to using the readily available AWS Budgets feature.

Option C creates a budget report but lacks the critical component of notification. A report alone won't actively alert the company when coverage drops below the desired level, necessitating manual monitoring.

Option D, while seemingly simple, oversimplifies the requirement. Savings Plans alert subscriptions are often geared towards payment failures or other account-related issues, not specifically coverage thresholds against your compute usage. While it might provide some alerts, it won't directly and efficiently address the specific need to monitor coverage percentage against their actual EC2, Fargate, and Lambda spend. This is the primary goal outlined in the prompt.

Therefore, option A offers the most operationally efficient solution by utilizing AWS Budgets' integrated

capabilities for Savings Plans monitoring and automated notifications, eliminating the need for custom code or manual intervention.

Here are some authoritative links for further research:

AWS Budgets: https://aws.amazon.com/aws-cost-management/aws-budgets/
Creating a Savings Plans Coverage Budget: https://docs.aws.amazon.com/cost-management/latest/userguide/sp-coverage.html
Compute Savings Plans: https://aws.amazon.com/savingsplans/compute-savings-plans/

Question: 716 **Exam Heist**

A company runs a real-time data ingestion solution on AWS. The solution consists of the most recent version of Amazon Managed Streaming for Apache Kafka (Amazon MSK). The solution is deployed in a VPC in private subnets across three Availability Zones.

A solutions architect needs to redesign the data ingestion solution to be publicly available over the internet. The data in transit must also be encrypted.

Which solution will meet these requirements with the MOST operational efficiency?

A.Configure public subnets in the existing VPC. Deploy an MSK cluster in the public subnets. Update the MSK cluster security settings to enable mutual TLS authentication.

B.Create a new VPC that has public subnets. Deploy an MSK cluster in the public subnets. Update the MSK cluster security settings to enable mutual TLS authentication.

C.Deploy an Application Load Balancer (ALB) that uses private subnets. Configure an ALB security group inbound rule to allow inbound traffic from the VPC CIDR block for HTTPS protocol.

D.Deploy a Network Load Balancer (NLB) that uses private subnets. Configure an NLB listener for HTTPS communication over the internet.

Answer: A

Explanation:

The most operationally efficient solution for making an Amazon MSK cluster publicly available over the internet with encryption is option A: configure public subnets in the existing VPC, deploy an MSK cluster in those subnets, and enable mutual TLS authentication.

Here's why:

Operational Efficiency: Reusing the existing VPC avoids the complexity of creating and managing a new VPC. Creating new infrastructure introduces operational overhead. Using the existing VPC leverages existing security groups and network configurations.
Public Subnets: Deploying the MSK cluster in public subnets allows the instances to have public IP addresses (or be associated with a NAT Gateway for outbound access), making them reachable from the internet.
Mutual TLS Authentication: Mutual TLS (mTLS) provides robust encryption and authentication. It ensures that both the client and the server (MSK cluster) authenticate each other before establishing a connection. This provides a secure, encrypted channel for data ingestion.

Why the other options are less optimal:

Option B (New VPC): Creating a new VPC adds unnecessary complexity. It requires configuring new networking (subnets, route tables, security groups) and potentially peering it with the existing VPC if communication between internal and external applications is needed.
Option C (ALB): An ALB is designed for HTTP/HTTPS traffic and operates at Layer 7 of the OSI model. MSK uses a binary protocol, making ALB unsuitable.

Option D (NLB): While an NLB can handle TCP traffic, it doesn't provide the application-level security and mutual authentication that mTLS does. NLBs operate at Layer 4 and do not offer the same deep inspection or security features as mTLS. NLB HTTPS listeners do not operate in the same manner as with standard web applications.

Therefore, placing the MSK cluster in public subnets within the existing VPC and enabling mTLS offers the best balance of security, functionality, and operational simplicity.

Relevant Links:

Amazon MSK Security: https://docs.aws.amazon.com/msk/latest/developerguide/security.html
Amazon VPC: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Subnets.html
Mutual TLS Authentication: https://aws.amazon.com/blogs/security/how-to-enable-mutual-tls-authentication-on-your-applications-using-aws-certificate-manager-private-ca/

Question: 717 **Exam Heist**

A company wants to migrate an on-premises legacy application to AWS. The application ingests customer order files from an on-premises enterprise resource planning (ERP) system. The application then uploads the files to an SFTP server. The application uses a scheduled job that checks for order files every hour.

The company already has an AWS account that has connectivity to the on-premises network. The new application on AWS must support integration with the existing ERP system. The new application must be secure and resilient and must use the SFTP protocol to process orders from the ERP system immediately.

Which solution will meet these requirements?

A.Create an AWS Transfer Family SFTP internet-facing server in two Availability Zones. Use Amazon S3 storage. Create an AWS Lambda function to process order files. Use S3 Event Notifications to send s3:ObjectCreated:* events to the Lambda function.

B.Create an AWS Transfer Family SFTP internet-facing server in one Availability Zone. Use Amazon Elastic File System (Amazon EFS) storage. Create an AWS Lambda function to process order files. Use a Transfer Family managed workflow to invoke the Lambda function.

C.Create an AWS Transfer Family SFTP internal server in two Availability Zones. Use Amazon Elastic File System (Amazon EFS) storage. Create an AWS Step Functions state machine to process order files. Use Amazon EventBridge Scheduler to invoke the state machine to periodically check Amazon EFS for order files.

D.Create an AWS Transfer Family SFTP internal server in two Availability Zones. Use Amazon S3 storage. Create an AWS Lambda function to process order files. Use a Transfer Family managed workflow to invoke the Lambda function.

Answer: D

Explanation:

The correct answer is D. Here's a detailed justification:

Requirement for SFTP: The application must use the SFTP protocol for processing orders. AWS Transfer Family directly addresses this requirement by providing managed SFTP servers.

Internal Access: The application integrates with an on-premises ERP system through existing network connectivity to AWS. Therefore, the SFTP server should be internal, not internet-facing, to maintain security and utilize existing private network connections.

Resilience: Deploying the SFTP server in two Availability Zones (AZs) enhances resilience by ensuring that the application remains available even if one AZ experiences an outage.

Storage: Amazon S3 is a suitable storage option for order files due to its scalability, durability, and cost-effectiveness.

Immediate Processing: Using AWS Transfer Family managed workflows allows for immediate processing of uploaded files. When a file is uploaded to the S3 bucket via SFTP, the Transfer Family workflow can automatically trigger a Lambda function.

Lambda Function: The Lambda function processes the order files. Because it's triggered by the Transfer Family workflow after each upload, the processing happens immediately, meeting the "process orders from the ERP system immediately" requirement.

EFS vs. S3: While Amazon EFS could be used for storage, S3 is generally preferred for file storage due to its cost-effectiveness, scalability, and integration with serverless architectures. EFS is more suitable for applications requiring shared file system access across multiple EC2 instances. Also, Transfer Family workflows are natively integrated with S3.

EventBridge Scheduler vs. Transfer Family Workflows: Using EventBridge Scheduler to periodically check EFS for order files would introduce latency and miss the requirement of immediate processing. Transfer Family workflows directly integrate with the file transfer process.

Step Functions: Step Functions manage complex workflows. While it could be used in conjunction with Lambda, it isn't necessary for this scenario, which is a simple file processing trigger.

Authoritative Links:

AWS Transfer Family: https://aws.amazon.com/transfer/
AWS Lambda: https://aws.amazon.com/lambda/
Amazon S3: https://aws.amazon.com/s3/
Transfer Family Managed Workflows:
https://docs.aws.amazon.com/transfer/latest/userguide/workflows.html

Question: 718 **Exam Heist**

A company's applications use Apache Hadoop and Apache Spark to process data on premises. The existing infrastructure is not scalable and is complex to manage.

A solutions architect must design a scalable solution that reduces operational complexity. The solution must keep the data processing on premises.

Which solution will meet these requirements?

A.Use AWS Site-to-Site VPN to access the on-premises Hadoop Distributed File System (HDFS) data and application. Use an Amazon EMR cluster to process the data.

B.Use AWS DataSync to connect to the on-premises Hadoop Distributed File System (HDFS) cluster. Create an Amazon EMR cluster to process the data.

C.Migrate the Apache Hadoop application and the Apache Spark application to Amazon EMR clusters on AWS Outposts. Use the EMR clusters to process the data.

D.Use an AWS Snowball device to migrate the data to an Amazon S3 bucket. Create an Amazon EMR cluster to process the data.

Answer: C

Explanation:

The correct answer is C because it directly addresses the requirements of scalability, reduced operational complexity, and on-premises data processing. AWS Outposts brings AWS services, including Amazon EMR, directly to the on-premises environment. By running Amazon EMR on AWS Outposts, the company can leverage the scalability and managed services of EMR for their Hadoop and Spark workloads without migrating the data off-premises. This fulfills the requirement of keeping data processing on premises while

simplifying management and improving scalability.

Option A is incorrect because it involves using AWS Site-to-Site VPN and Amazon EMR in the AWS cloud. This requires moving the data across the VPN, which doesn't satisfy the on-premises processing requirement.

Option B is also incorrect because it involves moving data to the AWS cloud using AWS DataSync and processing it there with Amazon EMR, which again violates the requirement of keeping data processing on premises.

Option D is incorrect because it involves moving the data to Amazon S3 in the AWS cloud using AWS Snowball, and then processing the data there, violating the on-premises processing requirement. Snowball is for data migration to the cloud, not for on-premises processing.

In summary, only option C maintains data processing on-premises using Amazon EMR on AWS Outposts, directly addressing the stated requirements of scalability, reduced operational complexity, and data residency.AWS Outposts: https://aws.amazon.com/outposts/Amazon EMR: https://aws.amazon.com/emr/

Question: 719 **Exam Heist**

A company is migrating a large amount of data from on-premises storage to AWS. Windows, Mac, and Linux based Amazon EC2 instances in the same AWS Region will access the data by using SMB and NFS storage protocols. The company will access a portion of the data routinely. The company will access the remaining data infrequently.

The company needs to design a solution to host the data.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create an Amazon Elastic File System (Amazon EFS) volume that uses EFS Intelligent-Tiering. Use AWS DataSync to migrate the data to the EFS volume.

B.Create an Amazon FSx for ONTAP instance. Create an FSx for ONTAP file system with a root volume that uses the auto tiering policy. Migrate the data to the FSx for ONTAP volume.

C.Create an Amazon S3 bucket that uses S3 Intelligent-Tiering. Migrate the data to the S3 bucket by using an AWS Storage Gateway Amazon S3 File Gateway.

D.Create an Amazon FSx for OpenZFS file system. Migrate the data to the new volume.

Answer: C

Explanation:

The correct answer is **C. Create an Amazon S3 bucket that uses S3 Intelligent-Tiering. Migrate the data to the S3 bucket by using an AWS Storage Gateway Amazon S3 File Gateway.**

Here's a detailed justification:

The scenario requires a solution to host a large amount of data accessible via SMB and NFS protocols, with both frequently and infrequently accessed portions, and aims for minimal operational overhead.

Option C provides the most efficient and cost-effective solution. S3 Intelligent-Tiering automatically moves data between frequent and infrequent access tiers based on access patterns, optimizing costs without manual intervention. AWS Storage Gateway's File Gateway presents the S3 bucket as a network file share accessible via SMB and NFS, fulfilling the protocol requirements. This eliminates the need for managing file systems directly, greatly reducing operational overhead.

Options A and B, involving Amazon EFS and Amazon FSx for ONTAP, respectively, require managing file systems and involve more administrative effort compared to S3. While EFS Intelligent-Tiering offers cost optimization, EFS doesn't natively support SMB. FSx for ONTAP, while supporting both protocols, is more

complex and expensive than S3 for this specific use case.

Option D, utilizing Amazon FSx for OpenZFS, is suitable for high-performance file systems. It doesn't have built-in intelligent tiering, leading to less cost optimization for infrequently accessed data and might have a higher operational burden than the S3 solution.

S3's scalability and durability are well-suited for storing large datasets. Using Storage Gateway's File Gateway simplifies the integration with existing Windows, Mac, and Linux environments, providing a seamless user experience without requiring significant application changes. By utilizing S3 Intelligent-Tiering, the solution minimizes storage costs by automatically transitioning data to the most cost-effective tier based on access patterns, without any operational overhead. Therefore, option C strikes the best balance between functionality, cost, and ease of management.

Supporting links:

Amazon S3 Intelligent-Tiering: https://aws.amazon.com/s3/storage-classes/intelligent-tiering/
AWS Storage Gateway: https://aws.amazon.com/storagegateway/
AWS Storage Gateway File Gateway: https://aws.amazon.com/storagegateway/file-gateway/

Question: 720 **Exam Heist**

A manufacturing company runs its report generation application on AWS. The application generates each report in about 20 minutes. The application is built as a monolith that runs on a single Amazon EC2 instance. The application requires frequent updates to its tightly coupled modules. The application becomes complex to maintain as the company adds new features.

Each time the company patches a software module, the application experiences downtime. Report generation must restart from the beginning after any interruptions. The company wants to redesign the application so that the application can be flexible, scalable, and gradually improved. The company wants to minimize application downtime.

Which solution will meet these requirements?

A.Run the application on AWS Lambda as a single function with maximum provisioned concurrency.

B.Run the application on Amazon EC2 Spot Instances as microservices with a Spot Fleet default allocation strategy.

C.Run the application on Amazon Elastic Container Service (Amazon ECS) as microservices with service auto scaling.

D.Run the application on AWS Elastic Beanstalk as a single application environment with an all-at-once deployment strategy.

Answer: C

Explanation:

The best solution for the manufacturing company is to run the application on Amazon Elastic Container Service (Amazon ECS) as microservices with service auto scaling. This approach addresses the issues of monolithic architecture, frequent updates causing downtime, and the need for scalability and flexibility.

Here's why:

Microservices Architecture: Breaking down the monolithic application into smaller, independent microservices allows for independent deployment and scaling of individual components. This means updates to one microservice don't require redeploying the entire application, reducing downtime and complexity.
Amazon ECS: ECS is a fully managed container orchestration service. It allows you to run, scale, and manage containerized applications, simplifying deployment and management of microservices.
Service Auto Scaling: ECS service auto scaling automatically adjusts the number of running tasks

(containers) based on demand. This ensures the application can handle peak loads efficiently and cost-effectively, providing scalability.
Flexibility and Gradual Improvement: Microservices enable the company to introduce new features and improvements incrementally without disrupting the entire application. Each microservice can be updated and deployed independently.
Minimized Downtime: ECS facilitates rolling deployments, where new versions of microservices are deployed gradually while old versions remain active. This significantly reduces or eliminates downtime during updates.
Resilience: In case a container fails, ECS can automatically restart it or launch a new one, enhancing the application's resilience and availability.

Options A, B, and D are less suitable:

A (Lambda as a single function): While Lambda provides scalability, running the entire monolithic application as a single Lambda function is not recommended. Lambda functions have execution time limits and are better suited for smaller, event-driven tasks.
B (EC2 Spot Instances): Using Spot Instances can be cost-effective, but relying solely on them for a critical application can lead to unpredictable interruptions if Spot Instances are reclaimed. Although Spot Fleets allow you to specify fallback on-demand instances, this choice still isn't as robust or scalable as ECS for the scenario described. Furthermore, running the application as microservices on Spot Instances without a container orchestrator introduces additional complexity in managing deployments, scaling, and service discovery.
D (Elastic Beanstalk with all-at-once deployment): Elastic Beanstalk provides a platform for deploying and managing web applications, but an all-at-once deployment strategy causes significant downtime. The all-at-once deployment strategy isn't appropriate for minimizing downtime. Furthermore, deploying a monolith into Beanstalk does not resolve the complexity of deployments, scaling, and service discovery.

Authoritative Links:

Amazon ECS: https://aws.amazon.com/ecs/
Microservices Architecture on AWS: https://aws.amazon.com/microservices/
Service Auto Scaling: https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-integrations.html

Question: 721 **Exam Heist**

A company wants to rearchitect a large-scale web application to a serverless microservices architecture. The application uses Amazon EC2 instances and is written in Python.

The company selected one component of the web application to test as a microservice. The component supports hundreds of requests each second. The company wants to create and test the microservice on an AWS solution that supports Python. The solution must also scale automatically and require minimal infrastructure and minimal operational support.

Which solution will meet these requirements?

A.Use a Spot Fleet with auto scaling of EC2 instances that run the most recent Amazon Linux operating system.
B.Use an AWS Elastic Beanstalk web server environment that has high availability configured.
C.Use Amazon Elastic Kubernetes Service (Amazon EKS). Launch Auto Scaling groups of self-managed EC2 instances.
D.Use an AWS Lambda function that runs custom developed code.

Answer: C

Explanation:

The incorrect answer is **D. Use an AWS Lambda function that runs custom developed code.**

Here's why the correct answer is a better choice and why option D is not:

The primary reason option D (AWS Lambda) is unsuitable is the statement that the component supports "hundreds of requests each second." While Lambda can handle concurrent invocations, managing the scaling and dependencies for a component processing hundreds of requests per second can quickly become complex and lead to cold start issues or function timeouts if not appropriately provisioned and optimized. Lambda is best suited for event-driven, short-lived, and more granular microservices. The sheer volume of requests makes orchestrating multiple Lambda functions and managing their state challenging.

Now, let's address why the other options are not suitable and expand on why Amazon Elastic Kubernetes Service (Amazon EKS) with Auto Scaling is a good option:

A. Spot Fleet with Auto Scaling: Although Spot Fleets offer cost savings, they are not ideal for a highly available production environment due to the possibility of instances being terminated with short notice. Managing the disruption caused by Spot instance terminations would add operational overhead and complexity, conflicting with the requirement for minimal operational support.

B. AWS Elastic Beanstalk: While Elastic Beanstalk simplifies deployment and management, it's still managing instances. Transitioning to microservices generally involves containerization. Although Beanstalk supports Docker, it does not provide the native orchestration capabilities of Kubernetes.

C. Amazon Elastic Kubernetes Service (Amazon EKS): EKS offers a managed Kubernetes service, allowing the company to deploy their microservice as a containerized application. Kubernetes excels at managing containerized workloads, providing automatic scaling, self-healing, and simplified deployment and management. Auto Scaling groups for the underlying EC2 worker nodes further ensure the cluster can handle the required load. With Python support available through containers, this approach meets all the requirements.

In Summary:

EKS with Auto Scaling provides the best solution for managing a high-volume microservice written in Python. It offers automated scaling, minimal operational support through a managed Kubernetes service, and a flexible containerized environment suitable for microservices. The high volume of requests is more naturally handled by container orchestration than individual Lambda functions.

Supporting Links:

Amazon EKS: https://aws.amazon.com/eks/
Kubernetes Auto Scaling: https://kubernetes.io/docs/tasks/configure-pod-container/horizontal-pod-autoscale/
AWS Lambda Limitations: https://docs.aws.amazon.com/lambda/latest/dg/limits.html

Question: 722 **Exam Heist**

A company has an AWS Direct Connect connection from its on-premises location to an AWS account. The AWS account has 30 different VPCs in the same AWS Region. The VPCs use private virtual interfaces (VIFs). Each VPC has a CIDR block that does not overlap with other networks under the company's control.

The company wants to centrally manage the networking architecture while still allowing each VPC to communicate with all other VPCs and on-premises networks.

Which solution will meet these requirements with the LEAST amount of operational overhead?

A.Create a transit gateway, and associate the Direct Connect connection with a new transit VIF. Turn on the transit gateway's route propagation feature.

B.Create a Direct Connect gateway. Recreate the private VIFs to use the new gateway. Associate each VPC by creating new virtual private gateways.

C.Create a transit VPConnect the Direct Connect connection to the transit VPCreate a peering connection between all other VPCs in the Region. Update the route tables.

D.Create AWS Site-to-Site VPN connections from on premises to each VPC. Ensure that both VPN tunnels are UP for each connection. Turn on the route propagation feature.

Answer: A

Explanation:

The correct answer is A because it leverages AWS Transit Gateway, the recommended solution for interconnecting multiple VPCs and on-premises networks with minimal operational overhead. Transit Gateway acts as a central hub, simplifying the routing configuration between numerous VPCs and Direct Connect.

Here's a breakdown:

Option A: Transit Gateway: Creates a transit gateway and associates the Direct Connect connection using a new transit VIF. Enables route propagation, automatically learning and distributing routes between connected VPCs and the Direct Connect connection. This eliminates the need for manual route table updates, significantly reducing operational burden. Transit Gateway's route propagation simplifies routing management by dynamically adjusting routes as networks change. https://aws.amazon.com/transit-gateway/
Option B: Direct Connect Gateway: Direct Connect Gateway provides global access for resources connected through Direct Connect. However, integrating it with numerous VPCs still requires managing separate virtual private gateways (VGWs) for each VPC, adding complexity compared to Transit Gateway. This option requires recreating existing VIFs, increasing disruption and effort.
Option C: Transit VPC and Peering: Transit VPCs, while a viable older solution, are more complex to manage at scale than Transit Gateways, especially with 30 VPCs. Peering all VPCs together creates a full mesh network, leading to complex route table management and scaling issues. The number of peering connections grows quadratically with the number of VPCs, becoming unwieldy and difficult to manage. This approach doesn't effectively address the need for centralized management and low operational overhead. https://docs.aws.amazon.com/whitepapers/latest/aws-vpc-connectivity-options/transit-vpc.html
Option D: Site-to-Site VPN: Creating VPN connections to each VPC is highly inefficient and costly. Maintaining 30 VPN connections would introduce significant operational overhead, including managing individual VPN configurations, certificates, and monitoring. VPN connections don't provide the centralized management and efficient routing capabilities of Transit Gateway.

In summary, Transit Gateway offers the most streamlined and scalable solution for connecting multiple VPCs and on-premises networks via Direct Connect, minimizing operational overhead through centralized routing and automatic route propagation.

Question: 723 **Exam Heist**

A company has applications that run on Amazon EC2 instances. The EC2 instances connect to Amazon RDS databases by using an IAM role that has associated policies. The company wants to use AWS Systems Manager to patch the EC2 instances without disrupting the running applications.

Which solution will meet these requirements?

A.Create a new IAM role. Attach the AmazonSSMManagedInstanceCore policy to the new IAM role. Attach the new IAM role to the EC2 instances and the existing IAM role.

B.Create an IAM user. Attach the AmazonSSMManagedInstanceCore policy to the IAM user. Configure Systems Manager to use the IAM user to manage the EC2 instances.

C.Enable Default Host Configuration Management in Systems Manager to manage the EC2 instances.

D.Remove the existing policies from the existing IAM role. Add the AmazonSSMManagedInstanceCore policy to the existing IAM role.

Answer: A

Explanation:

The correct answer is A: Create a new IAM role. Attach the AmazonSSMManagedInstanceCore policy to the new IAM role. Attach the new IAM role to the EC2 instances and the existing IAM role.

Here's a detailed justification:

The requirement is to use AWS Systems Manager to patch EC2 instances without disrupting the running applications. The applications are already using an IAM role with specific policies to access RDS. If we modify this existing role, it could potentially interrupt the application's ability to access RDS.

Option A addresses this perfectly by creating a separate IAM role specifically for Systems Manager. The AmazonSSMManagedInstanceCore policy grants Systems Manager the necessary permissions to manage the EC2 instance (e.g., install updates, run commands). By attaching both the existing application IAM role and the new Systems Manager IAM role to the EC2 instances, we ensure that:

1. The applications retain their original permissions for RDS access, preventing disruption.
2. Systems Manager gains the necessary permissions to patch the instances.

Option B is incorrect because IAM users are not directly associated with EC2 instances for granting permissions to the instance itself. IAM users are used for programmatic access (e.g., via the CLI or SDK), not for granting instances permissions to access AWS resources. Systems Manager needs instance-level permissions.

Option C, enabling Default Host Configuration Management in Systems Manager, doesn't address the IAM permissions needed by Systems Manager to manage the instances. While it simplifies initial configuration in some cases, it doesn't automatically grant the necessary permissions. Systems Manager still needs an IAM role on the instance to act on it.

Option D is problematic because removing existing policies from the original IAM role will definitely disrupt the running applications' access to RDS. The current IAM role provides the applications with the necessary permissions to access the database. Modifying it directly is explicitly against the requirement to avoid disruption.

Therefore, Option A is the only solution that provides the necessary permissions for Systems Manager without interfering with the existing application functionality. It follows the principle of least privilege by granting Systems Manager only the permissions it needs and avoids modifying a role that is already in use by a critical application.

Here are some authoritative links for further research:

IAM Roles for Amazon EC2: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_use_switch-role-ec2.html
AmazonSSMManagedInstanceCore Policy: https://docs.aws.amazon.com/systems-manager/latest/userguide/security-iam-awsmanpol.html
AWS Systems Manager Patch Manager: https://docs.aws.amazon.com/systems-manager/latest/userguide/patch-manager.html

Question: 724 **Exam Heist**

A company runs container applications by using Amazon Elastic Kubernetes Service (Amazon EKS) and the Kubernetes Horizontal Pod Autoscaler. The workload is not consistent throughout the day. A solutions architect notices that the number of nodes does not automatically scale out when the existing nodes have reached maximum capacity in the cluster, which causes performance issues.

Which solution will resolve this issue with the LEAST administrative overhead?

A.Scale out the nodes by tracking the memory usage.
B.Use the Kubernetes Cluster Autoscaler to manage the number of nodes in the cluster.
C.Use an AWS Lambda function to resize the EKS cluster automatically.
D.Use an Amazon EC2 Auto Scaling group to distribute the workload.

Answer: B

Explanation:

The correct answer is **B. Use the Kubernetes Cluster Autoscaler to manage the number of nodes in the cluster.**

The problem describes a scenario where the Kubernetes Horizontal Pod Autoscaler (HPA) is successfully scaling the pods within the cluster, but the underlying compute capacity (the nodes themselves) is not scaling to accommodate the increased pod count, leading to performance bottlenecks. This indicates a need for cluster-level autoscaling, not just pod-level autoscaling.

The Kubernetes Cluster Autoscaler is specifically designed to automatically adjust the size of the Kubernetes cluster by adding or removing nodes based on resource demands. It monitors the Kubernetes scheduler for pending pods that cannot be scheduled due to insufficient resources. When it finds such pods, it triggers the underlying infrastructure (in this case, the EC2 Auto Scaling group behind the EKS cluster) to add more nodes. Conversely, if nodes are underutilized, the Cluster Autoscaler can remove nodes.

Option A is incorrect because scaling nodes based on memory usage alone might not be the most effective strategy. CPU utilization and pod scheduling constraints are equally important factors. The Cluster Autoscaler considers all these factors.

Option C, using a Lambda function, would require custom logic to monitor the cluster state, make scaling decisions, and interact with the EC2 Auto Scaling group. This introduces significant administrative overhead and complexity compared to using the purpose-built Cluster Autoscaler.

Option D, using an EC2 Auto Scaling group to distribute the workload, doesn't directly address the Kubernetes cluster's scaling needs. While an Auto Scaling group is essential for the underlying infrastructure, it needs to be integrated with Kubernetes awareness to function correctly in this scenario. The Cluster Autoscaler utilizes an EC2 Auto Scaling group but provides the necessary Kubernetes integration.

The Cluster Autoscaler provides a native, integrated, and automated solution for scaling the EKS cluster, minimizing administrative overhead and ensuring that the cluster capacity matches the application's resource requirements. It's the simplest and most efficient way to address the described problem.

Supporting documentation:

Kubernetes Cluster Autoscaler: https://github.com/kubernetes/autoscaler/tree/master/cluster-autoscaler
AWS EKS Cluster Autoscaler: https://docs.aws.amazon.com/eks/latest/userguide/cluster-autoscaler.html

Question: 725 **Exam Heist**

A company maintains about 300 TB in Amazon S3 Standard storage month after month. The S3 objects are each typically around 50 GB in size and are frequently replaced with multipart uploads by their global application. The number and size of S3 objects remain constant, but the company's S3 storage costs are increasing each month.

How should a solutions architect reduce costs in this situation?

A.Switch from multipart uploads to Amazon S3 Transfer Acceleration.

B.Enable an S3 Lifecycle policy that deletes incomplete multipart uploads.

C.Configure S3 inventory to prevent objects from being archived too quickly.

D.Configure Amazon CloudFront to reduce the number of objects stored in Amazon S3.

Answer: B

Explanation:

The correct answer is **B. Enable an S3 Lifecycle policy that deletes incomplete multipart uploads.**

Here's why:

The problem statement indicates the company's S3 storage costs are increasing despite the number and size of objects remaining relatively constant. This suggests a buildup of something that's not meant to be there and is consuming storage space. Multipart uploads are a prime suspect.

Multipart uploads are used to upload large objects to S3 in parts. If an upload is interrupted or fails before all parts are committed, these incomplete parts remain in S3 storage, consuming space and incurring costs. Since the company frequently uses multipart uploads, especially for large 50GB objects, the likelihood of incomplete uploads is relatively high. Over time, these fragments accumulate, leading to increased storage costs.

An S3 Lifecycle policy can be configured to automatically delete incomplete multipart uploads after a specified number of days. This action reclaims storage space occupied by the orphaned parts and reduces overall storage costs. It addresses the root cause of the increasing costs by cleaning up unfinished processes.

Let's analyze the other options:

A. Switch from multipart uploads to Amazon S3 Transfer Acceleration: S3 Transfer Acceleration optimizes data transfer speeds over long distances using Amazon's globally distributed edge locations. This improves upload performance but doesn't directly address the issue of increasing storage costs due to incomplete uploads. While Transfer Acceleration can be beneficial, it won't fix the underlying storage waste problem.

C. Configure S3 inventory to prevent objects from being archived too quickly: S3 Inventory provides a listing of your objects and their metadata. While useful for auditing and reporting, it doesn't prevent incomplete multipart uploads or address the cost issue directly. Furthermore, the scenario states they use S3 Standard which isn't an archive storage class.

D. Configure Amazon CloudFront to reduce the number of objects stored in Amazon S3: CloudFront is a content delivery network (CDN) that caches content at edge locations to improve content delivery performance. CloudFront caches objects, so this would reduce the number of requests to S3, not the objects stored in S3. It might indirectly reduce egress costs but doesn't address the primary problem of increasing storage costs. Objects remain in S3.

Therefore, configuring an S3 Lifecycle policy to delete incomplete multipart uploads is the most effective solution to reduce the company's increasing S3 storage costs in this scenario.

Further Research:

Amazon S3 Lifecycle: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-examples.html
Multipart Upload Overview: https://docs.aws.amazon.com/AmazonS3/latest/userguide/mpuoverview.html
S3 Transfer Acceleration: https://docs.aws.amazon.com/AmazonS3/latest/userguide/transfer-acceleration.html

Question: 726 **Exam Heist**

A company has deployed a multiplayer game for mobile devices. The game requires live location tracking of players based on latitude and longitude. The data store for the game must support rapid updates and retrieval of locations.

The game uses an Amazon RDS for PostgreSQL DB instance with read replicas to store the location data. During peak usage periods, the database is unable to maintain the performance that is needed for reading and writing updates. The game's user base is increasing rapidly.

What should a solutions architect do to improve the performance of the data tier?

A.Take a snapshot of the existing DB instance. Restore the snapshot with Multi-AZ enabled.

B.Migrate from Amazon RDS to Amazon OpenSearch Service with OpenSearch Dashboards.

C.Deploy Amazon DynamoDB Accelerator (DAX) in front of the existing DB instance. Modify the game to use DAX.

D.Deploy an Amazon ElastiCache for Redis cluster in front of the existing DB instance. Modify the game to use Redis.

Answer: D

Explanation:

Here's a detailed justification for why option D is the best solution, along with why the other options are less suitable:

Why Option D (Amazon ElastiCache for Redis) is the Best Choice:

Option D, deploying an Amazon ElastiCache for Redis cluster in front of the RDS instance and modifying the game to utilize Redis, addresses the core performance issues effectively. The question highlights rapid updates and retrieval of location data as crucial requirements. Redis is an in-memory data store, known for its extremely low latency and high throughput, which is perfect for caching frequently accessed data. By placing Redis in front of the RDS PostgreSQL database, the game application can read and write location data to Redis first. Most read requests can then be served directly from the Redis cache, drastically reducing the load on the RDS instance and improving read performance. Furthermore, Redis's fast write capabilities allow for quick updates to the location data, and these updates can be asynchronously persisted to the RDS instance in the background, further offloading the database. This caching strategy is especially useful given the read-heavy nature of location tracking in a multiplayer game. ElastiCache simplifies the management and scaling of the Redis cluster. Redis is well-suited for geospatial data as well.

Why the other options are less suitable:

Option A (Multi-AZ for RDS): Enabling Multi-AZ on the RDS instance primarily provides high availability and fault tolerance, not a significant performance boost for read and write operations. While Multi-AZ can help with failover, it doesn't address the issue of database overload during peak usage.

Option B (Amazon OpenSearch Service): While OpenSearch is suitable for search and analytics, it's not the ideal solution for the described use case. OpenSearch is designed for indexing and searching large volumes of data, but it's not optimized for the rapid, low-latency updates required for real-time location tracking. It's also more complex to set up and maintain than Redis in this scenario.

Option C (DynamoDB Accelerator (DAX)): DAX is a caching service specifically designed for Amazon DynamoDB. Since the application currently uses RDS PostgreSQL, DAX is not applicable. Moreover, migrating to DynamoDB would necessitate major application code changes which makes this approach less efficient.

Authoritative Links:

Amazon ElastiCache: https://aws.amazon.com/elasticache/

Redis: https://redis.io/
Caching Strategies: https://aws.amazon.com/caching/

Question: 727 **Exam Heist**

A company stores critical data in Amazon DynamoDB tables in the company's AWS account. An IT administrator accidentally deleted a DynamoDB table. The deletion caused a significant loss of data and disrupted the company's operations. The company wants to prevent this type of disruption in the future.

Which solution will meet this requirement with the LEAST operational overhead?

A.Configure a trail in AWS CloudTrail. Create an Amazon EventBridge rule for delete actions. Create an AWS Lambda function to automatically restore deleted DynamoDB tables.

B.Create a backup and restore plan for the DynamoDB tables. Recover the DynamoDB tables manually.

C.Configure deletion protection on the DynamoDB tables.

D.Enable point-in-time recovery on the DynamoDB tables.

Answer: C

Explanation:

The correct answer is **C. Configure deletion protection on the DynamoDB tables.** This option directly addresses the problem of accidental table deletion with the least operational overhead.

Here's why:

Deletion protection is a feature specifically designed to prevent accidental deletion of DynamoDB tables. Once enabled, a table cannot be deleted without first disabling the protection. This introduces a necessary step that mitigates unintended deletions.

Operational Overhead: This solution requires only enabling a setting on each DynamoDB table. This minimal configuration translates to the least amount of operational overhead compared to the other options.

Let's analyze why the other options are less suitable:

A. CloudTrail, EventBridge, and Lambda: While this approach allows for automated restoration, it's far more complex. It requires configuring multiple services, writing code, and testing the automated restoration process. The operational overhead is significant. Furthermore, restoration isn't prevention; the table is still momentarily deleted, and the recovery process might have its own issues.

B. Backup and Restore Plan: While creating backups is a good practice for disaster recovery, it doesn't prevent accidental deletion. Restoring from a backup is a manual process that takes time and introduces potential data loss (data written after the backup). This option addresses data recovery, not prevention.

D. Point-in-Time Recovery (PITR): PITR allows you to restore a table to any point in time within the past 35 days. While useful for recovering from data corruption or unintended updates, it doesn't prevent deletion. Like backups, it addresses data recovery, not prevention. The table is still deleted initially, which causes downtime until the point-in-time recovery completes. Also, enabling PITR introduces some cost overhead.

In summary, deletion protection is the most straightforward, least operationally intensive, and most effective way to prevent accidental DynamoDB table deletions. It's a preventative measure, unlike the other options that focus on recovery after the event has already occurred.Here are some authoritative links for further research:

DynamoDB Deletion Protection:
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/DAXDeletionProtection.html

DynamoDB Point-in-Time Recovery:
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/PointInTimeRecovery.html

Question: 728 **Exam Heist**

A company has an on-premises data center that is running out of storage capacity. The company wants to migrate its storage infrastructure to AWS while minimizing bandwidth costs. The solution must allow for immediate retrieval of data at no additional cost.

How can these requirements be met?

A.Deploy Amazon S3 Glacier Vault and enable expedited retrieval. Enable provisioned retrieval capacity for the workload.

B.Deploy AWS Storage Gateway using cached volumes. Use Storage Gateway to store data in Amazon S3 while retaining copies of frequently accessed data subsets locally.

C.Deploy AWS Storage Gateway using stored volumes to store data locally. Use Storage Gateway to asynchronously back up point-in-time snapshots of the data to Amazon S3.

D.Deploy AWS Direct Connect to connect with the on-premises data center. Configure AWS Storage Gateway to store data locally. Use Storage Gateway to asynchronously back up point-in-time snapshots of the data to Amazon S3.

Answer: C

Explanation:

The correct solution is **C. Deploy AWS Storage Gateway using stored volumes to store data locally. Use Storage Gateway to asynchronously back up point-in-time snapshots of the data to Amazon S3.**

Here's a detailed justification:

The primary goal is to migrate on-premises storage to AWS while minimizing bandwidth costs and ensuring immediate data retrieval at no extra cost. AWS Storage Gateway helps bridge the gap between on-premises environments and AWS storage services. Different Storage Gateway types offer varying strategies for data management.

Stored Volumes: In this configuration, the entire dataset is stored locally on the on-premises storage appliance. Storage Gateway then asynchronously backs up point-in-time snapshots of this local data to Amazon S3. This directly addresses the requirements:

Immediate Retrieval: Since the complete dataset is stored locally, data retrieval is fast and doesn't incur bandwidth costs associated with pulling data from AWS every time.
Minimized Bandwidth Costs: Only snapshots are transferred to S3, reducing the overall bandwidth usage compared to constantly synchronizing the entire dataset.
Data Migration: Provides a path for migrating data by storing it in the cloud while retaining a local copy.

Let's analyze why the other options are less suitable:

A. Amazon S3 Glacier Vault and expedited retrieval: S3 Glacier is designed for long-term archiving and infrequent access. Expedited retrievals come at a cost, which violates the "no additional cost" requirement for immediate retrievals. Also, Glacier is not a storage gateway.
B. AWS Storage Gateway using cached volumes: Cached volumes store frequently accessed data locally and store the entire dataset in S3. While this reduces latency for frequently accessed data, it still requires transferring the entire dataset to S3 initially, increasing bandwidth costs. Furthermore, infrequently accessed data still requires retrieval from S3.
D. AWS Direct Connect and Storage Gateway with stored volumes: While AWS Direct Connect provides a dedicated network connection between the on-premises environment and AWS, it is primarily focused on the

network link. It reduces bandwidth costs compared to internet transit, but still represents a cost. The Storage Gateway part is the same as option C, but the Direct Connect component adds unnecessary complexity and cost without directly addressing the core requirement of immediate retrieval at no additional cost. Option C already provides the method of data backup through snapshots. Direct Connect is only needed when there is a higher bandwidth needed for the backup and the network connection is critical.

In summary, Stored Volumes provide the optimal balance between local accessibility, reduced bandwidth consumption, and gradual migration to the cloud.**Authoritative Links:**

AWS Storage Gateway: https://aws.amazon.com/storagegateway/
Understanding Storage Gateway Volume Types:
https://docs.aws.amazon.com/storagegateway/latest/userguide/HowStorageGatewayWorks.html

Question: 729 **Exam Heist**

A company runs a three-tier web application in a VPC across multiple Availability Zones. Amazon EC2 instances run in an Auto Scaling group for the application tier.

The company needs to make an automated scaling plan that will analyze each resource's daily and weekly historical workload trends. The configuration must scale resources appropriately according to both the forecast and live changes in utilization.

Which scaling strategy should a solutions architect recommend to meet these requirements?

A.Implement dynamic scaling with step scaling based on average CPU utilization from the EC2 instances.
B.Enable predictive scaling to forecast and scale. Configure dynamic scaling with target tracking
C.Create an automated scheduled scaling action based on the traffic patterns of the web application.
D.Set up a simple scaling policy. Increase the cooldown period based on the EC2 instance startup time.

Answer: B

Explanation:

Here's a detailed justification for why option B is the best choice, addressing the requirements of analyzing historical workload trends, forecasting, and reacting to live utilization changes:

Option B, "Enable predictive scaling to forecast and scale. Configure dynamic scaling with target tracking," is the most comprehensive solution for the stated problem. It leverages the strengths of both predictive and dynamic scaling approaches.

Predictive scaling uses machine learning to analyze historical workload patterns (both daily and weekly trends, as required) and forecasts future demand. This proactive approach allows the system to pre-emptively scale resources before the actual increase in traffic occurs. This is superior to purely reactive approaches as it avoids potential performance bottlenecks during the initial surge.

Dynamic scaling with target tracking provides real-time responsiveness. It continuously monitors a chosen metric (like CPU utilization or request latency) and adjusts resources to maintain that metric at a defined target. This complements predictive scaling by handling unexpected traffic spikes or deviations from the forecast that predictive scaling might miss. This allows the system to adapt to live changes in utilization.

The combination ensures the application is scaled both proactively based on trends and reactively based on current demand.

Option A (step scaling based on CPU utilization) only addresses dynamic scaling, not historical trend analysis or predictive scaling. It’s reactive and doesn't prepare for anticipated load increases.

Option C (scheduled scaling) might work for predictable traffic patterns, but it doesn't adapt to live changes or leverage historical trend analysis effectively. It's also less flexible than a predictive and dynamic scaling combination. It would require manual updating of the schedule.

Option D (simple scaling with cooldown) is a basic scaling strategy, not suitable for complex workloads. It lacks trend analysis, forecasting, and the precision of target tracking. Cooldown periods are intended to prevent excessive scaling actions, but don't help with the underlying problem of matching resources to demand.

Therefore, option B, combining predictive scaling with dynamic target tracking, provides the best solution to meet the requirements of historical analysis, forecasting, and real-time responsiveness in the three-tier web application.

Authoritative Links for further research:

AWS Auto Scaling Predictive Scaling: https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-predictive-scaling.html
AWS Auto Scaling Dynamic Scaling: https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-scaling-policies.html
AWS Auto Scaling - Choose a Scaling Strategy: https://aws.amazon.com/autoscaling/scaling-strategies/

Question: 730 **Exam Heist**

A package delivery company has an application that uses Amazon EC2 instances and an Amazon Aurora MySQL DB cluster. As the application becomes more popular, EC2 instance usage increases only slightly. DB cluster usage increases at a much faster rate.

The company adds a read replica, which reduces the DB cluster usage for a short period of time. However, the load continues to increase. The operations that cause the increase in DB cluster usage are all repeated read statements that are related to delivery details. The company needs to alleviate the effect of repeated reads on the DB cluster.

Which solution will meet these requirements MOST cost-effectively?

A.Implement an Amazon ElastiCache for Redis cluster between the application and the DB cluster.
B.Add an additional read replica to the DB cluster.
C.Configure Aurora Auto Scaling for the Aurora read replicas.
D.Modify the DB cluster to have multiple writer instances.

Answer: A

Explanation:

The best solution for alleviating the effect of repeated read operations on the Aurora MySQL DB cluster cost-effectively is to implement an Amazon ElastiCache for Redis cluster between the application and the DB cluster.

Here's why:

The problem focuses on repeated read operations causing a bottleneck in the DB cluster. While read replicas offload some read traffic, the load continues to grow. Adding more read replicas (option B) might offer temporary relief but doesn't address the underlying issue of repeatedly fetching the same data. Aurora Auto Scaling (option C) scales read replicas, but if the root cause is repetitive queries, it will just scale more servers performing the same inefficient operation, incurring higher costs. Option D, multiple writer instances, is not applicable for read scaling and is used for handling write-heavy workloads and is not related to the scenario.

ElastiCache for Redis acts as an in-memory data store (cache) situated between the application and the

database. This caching mechanism stores frequently accessed delivery details. When the application requests this data, ElastiCache retrieves it from memory, drastically reducing the load on the DB cluster. Redis's in-memory nature makes it significantly faster than accessing the database directly for read operations. Because it avoids querying the DB cluster for repetitive data, it's the most cost-effective solution. It avoids scaling the DB cluster while directly addressing the performance bottleneck.

Here are some resources for further research:

Amazon ElastiCache: https://aws.amazon.com/elasticache/
Caching Strategies: https://aws.amazon.com/caching/
Aurora Read Replicas: https://aws.amazon.com/rds/aurora/features/

Question: 731 **Exam Heist**

A company has an application that uses an Amazon DynamoDB table for storage. A solutions architect discovers that many requests to the table are not returning the latest data. The company's users have not reported any other issues with database performance. Latency is in an acceptable range.

Which design change should the solutions architect recommend?

A.Add read replicas to the table.
B.Use a global secondary index (GSI).
C.Request strongly consistent reads for the table.
D.Request eventually consistent reads for the table.

Answer: C

Explanation:

The problem indicates that the application is retrieving stale data from DynamoDB, even though overall latency is acceptable. This points directly to the read consistency level being used. DynamoDB offers both eventually consistent reads and strongly consistent reads.

Option A, adding read replicas, doesn't apply to DynamoDB's internal architecture. DynamoDB automatically replicates data across multiple Availability Zones for high availability and durability; users do not manually create read replicas.

Option B, using a Global Secondary Index (GSI), improves query performance based on different attributes but does not directly address data consistency issues. While a GSI could improve query speed, it wouldn't guarantee the latest data is returned unless it uses strongly consistent reads.

Option D, requesting eventually consistent reads, is actually the default behavior in DynamoDB and explains why the application isn't getting the latest data. Eventually consistent reads provide the best read performance (lowest latency) because they may read from any replica, but the trade-off is a small delay before the latest updates become visible across all replicas. This is what is causing the stale data issue.

Option C, requesting strongly consistent reads, will directly solve the problem. Strongly consistent reads guarantee that the read operation will return the most up-to-date version of an item, reflecting all previous write operations that completed successfully. While strongly consistent reads may have slightly higher latency than eventually consistent reads, it's crucial in this scenario where data accuracy is paramount. The question states that latency is within an acceptable range, so the slightly higher latency associated with strongly consistent reads is permissible in the context of solving the consistency problem. Therefore, requesting strongly consistent reads is the most appropriate design change.

Here are links for further research:

DynamoDB Read Consistency:
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html
DynamoDB Global Secondary Indexes:
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GSI.html

Question: 732 **Exam Heist**

A company has deployed its application on Amazon EC2 instances with an Amazon RDS database. The company used the principle of least privilege to configure the database access credentials. The company's security team wants to protect the application and the database from SQL injection and other web-based attacks.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use security groups and network ACLs to secure the database and application servers.

B.Use AWS WAF to protect the application. Use RDS parameter groups to configure the security settings.

C.Use AWS Network Firewall to protect the application and the database.

D.Use different database accounts in the application code for different functions. Avoid granting excessive privileges to the database users.

Answer: B

Explanation:

Here's a detailed justification for why option B is the best solution for protecting an application and RDS database from SQL injection and web-based attacks with the least operational overhead, adhering to the principle of least privilege:

The problem requires mitigating SQL injection and web-based attacks with minimal operational effort, leveraging the principle of least privilege. AWS WAF (Web Application Firewall) directly addresses this requirement by inspecting HTTP(S) traffic and blocking malicious requests before they reach the application. It offers pre-configured rules for common web exploits, including SQL injection, cross-site scripting (XSS), and others, reducing the need for custom security rules and maintenance.

Using RDS parameter groups to configure security settings helps in hardening the database. Although parameter groups primarily manage database configurations, they can indirectly contribute to security. For example, you can use parameter groups to enforce stricter password policies or disable certain features that might be susceptible to vulnerabilities.

Option A (Security Groups and Network ACLs) focuses on network-level security, controlling traffic based on IP addresses and ports. While crucial, they don't provide deep inspection of application layer traffic necessary to prevent SQL injection and other web-based attacks. They are more concerned with allowing or denying connections, not analyzing the content within those connections.

Option C (AWS Network Firewall) is a powerful network security service, but it is generally more complex and operationally heavy than AWS WAF for protecting web applications. While it offers broader protection at the network layer, its configuration and maintenance would be higher than that of WAF, adding unnecessary complexity.

Option D (Different database accounts with limited privileges) aligns with the principle of least privilege. However, by itself, this doesn't protect against SQL injection; it only limits the damage if an attack succeeds. It also requires more careful management of database credentials within the application code. While a good practice, it doesn't replace the need for web application firewall protection.

Therefore, using AWS WAF to filter malicious requests and employing RDS parameter groups to indirectly enhance database security is the most effective and operationally efficient solution. It provides specific

protection against web-based attacks while keeping the operational overhead manageable, and complements the least privilege principle. WAF's managed rulesets and easy deployment make it ideal for addressing the core security concerns with minimal manual intervention.

Supporting links:

AWS WAF: https://aws.amazon.com/waf/
Amazon RDS Parameter Groups:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_WorkingWithParamGroups.html

Question: 733 **Exam Heist**

An ecommerce company runs applications in AWS accounts that are part of an organization in AWS Organizations. The applications run on Amazon Aurora PostgreSQL databases across all the accounts. The company needs to prevent malicious activity and must identify abnormal failed and incomplete login attempts to the databases.

Which solution will meet these requirements in the MOST operationally efficient way?

A.Attach service control policies (SCPs) to the root of the organization to identity the failed login attempts.

B.Enable the Amazon RDS Protection feature in Amazon GuardDuty for the member accounts of the organization.

C.Publish the Aurora general logs to a log group in Amazon CloudWatch Logs. Export the log data to a central Amazon S3 bucket.

D.Publish all the Aurora PostgreSQL database events in AWS CloudTrail to a central Amazon S3 bucket.

Answer: B

Explanation:

The best approach to detect abnormal database login attempts across an AWS Organization involves leveraging specialized security services designed for threat detection. Option B, enabling Amazon RDS Protection in Amazon GuardDuty for member accounts, directly addresses this requirement efficiently.

GuardDuty RDS Protection monitors Aurora PostgreSQL logs and activity to identify suspicious behavior related to login attempts, password failures, and potential brute-force attacks. This solution is operationally efficient because GuardDuty is a managed threat detection service that automates log analysis, threat intelligence correlation, and anomaly detection, removing the need to manually parse logs. GuardDuty automatically integrates with AWS Organizations and can be enabled centrally, applying to all member accounts. This reduces administrative overhead compared to setting up and maintaining custom logging solutions.

Option A, using SCPs, can restrict actions but isn't designed to analyze and identify failed login attempts. Option C, which involves publishing logs to CloudWatch and exporting them to S3, requires a custom analysis pipeline to detect anomalies, adding significant operational complexity. Option D, publishing database events to CloudTrail, while helpful for auditing API calls, doesn't directly monitor database login activity.

GuardDuty's RDS Protection offers a purpose-built, managed service that provides pre-built detectors for common database security threats and integrates seamlessly with AWS Organizations, providing the most operationally efficient
solution.https://aws.amazon.com/guardduty/features/https://docs.aws.amazon.com/guardduty/latest/ug/guardduty

Question: 734 **Exam Heist**

A company has an AWS Direct Connect connection from its corporate data center to its VPC in the us-east-1 Region. The company recently acquired a corporation that has several VPCs and a Direct Connect connection between its on-premises data center and the eu-west-2 Region. The CIDR blocks for the VPCs of the company and the corporation do not overlap. The company requires connectivity between two Regions and the data centers. The company needs a solution that is scalable while reducing operational overhead.

What should a solutions architect do to meet these requirements?

A.Set up inter-Region VPC peering between the VPC in us-east-1 and the VPCs in eu-west-2.

B.Create private virtual interfaces from the Direct Connect connection in us-east-1 to the VPCs in eu-west-2.

C.Establish VPN appliances in a fully meshed VPN network hosted by Amazon EC2. Use AWS VPN CloudHub to send and receive data between the data centers and each VPC.

D.Connect the existing Direct Connect connection to a Direct Connect gateway. Route traffic from the virtual private gateways of the VPCs in each Region to the Direct Connect gateway.

Answer: D

Explanation:

The best solution is **D. Connect the existing Direct Connect connection to a Direct Connect gateway. Route traffic from the virtual private gateways of the VPCs in each Region to the Direct Connect gateway.**

Here's why:

Scalability and Reduced Overhead: Direct Connect Gateway (DXGW) is designed for connecting multiple VPCs across different AWS Regions. It simplifies the management of multiple connections and routing configurations, reducing operational overhead as the company's AWS footprint grows.

Centralized Management: DXGW acts as a central point for routing traffic between on-premises networks (via Direct Connect) and multiple VPCs across different Regions.

No Overlapping CIDR Blocks: Since the CIDR blocks of the VPCs don't overlap, DXGW can seamlessly route traffic between them without any address translation issues.

Inter-Region Connectivity: DXGW supports inter-Region connectivity, fulfilling the requirement to connect VPCs in us-east-1 and eu-west-2.

Direct Connect Utilization: This solution leverages the existing Direct Connect connections, ensuring cost-effectiveness and minimizing the need for new infrastructure.

Alternatives Considered & Why They Are Less Suitable:

A (Inter-Region VPC Peering): While possible, VPC peering is point-to-point. Connecting multiple VPCs across Regions requires establishing and managing numerous peering connections, leading to increased complexity and administrative burden. It doesn't scale well.
B (Direct Connect Private VIFs to Each VPC): This is not recommended because creating multiple private VIFs from one Direct Connect connection to VPCs in different Regions is complex and less efficient than using DXGW. Also, some organizations may have Direct Connect port limits.
C (VPN Appliances in EC2 with AWS VPN CloudHub): VPN appliances in EC2 incur higher operational overhead compared to DXGW. Managing VPN appliances, routing, and security can become complex, and the performance may be less reliable than using a Direct Connect Gateway. Also, it doesn't fully utilize the existing Direct Connect connections. EC2 based VPNs will also be significantly more costly at high throughputs.

In summary, Direct Connect Gateway provides a scalable, manageable, and cost-effective solution for connecting multiple VPCs across different AWS Regions using existing Direct Connect connections, fulfilling all the company's requirements.

Authoritative Links:

AWS Direct Connect Gateway: https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-gateways-intro.html
AWS VPN CloudHub: https://docs.aws.amazon.com/vpn/latest/s2svpn/VPN_CloudHub.html
Inter-Region VPC Peering: https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html

Question: 735 **Exam Heist**

A company is developing a mobile game that streams score updates to a backend processor and then posts results on a leaderboard. A solutions architect needs to design a solution that can handle large traffic spikes, process the mobile game updates in order of receipt, and store the processed updates in a highly available database. The company also wants to minimize the management overhead required to maintain the solution.

What should the solutions architect do to meet these requirements?

A.Push score updates to Amazon Kinesis Data Streams. Process the updates in Kinesis Data Streams with AWS Lambda. Store the processed updates in Amazon DynamoDB.

B.Push score updates to Amazon Kinesis Data Streams. Process the updates with a fleet of Amazon EC2 instances set up for Auto Scaling. Store the processed updates in Amazon Redshift.

C.Push score updates to an Amazon Simple Notification Service (Amazon SNS) topic. Subscribe an AWS Lambda function to the SNS topic to process the updates. Store the processed updates in a SQL database running on Amazon EC2.

D.Push score updates to an Amazon Simple Queue Service (Amazon SQS) queue. Use a fleet of Amazon EC2 instances with Auto Scaling to process the updates in the SQS queue. Store the processed updates in an Amazon RDS Multi-AZ DB instance.

Answer: A

Explanation:

Here's a detailed justification for why option A is the most suitable solution, along with supporting cloud computing concepts and links for further research:

Option A leverages several AWS services optimized for high-throughput data streaming, processing, and storage, aligning perfectly with the problem's requirements. Amazon Kinesis Data Streams is designed for real-time streaming of large volumes of data, making it ideal for handling score updates from the mobile game. Crucially, Kinesis Data Streams guarantees data ordering within each shard, fulfilling the requirement to process updates in the order they are received. AWS Lambda provides a serverless compute environment, minimizing management overhead, as it automatically scales in response to the volume of updates from Kinesis. Lambda's function will consume the Kinesis stream and process the updates. Finally, Amazon DynamoDB is a highly available and scalable NoSQL database, well-suited for storing the processed score updates. Its managed nature further reduces management overhead. DynamoDB also offers fast read and write performance.

Why other options are less suitable:

Option B: While Kinesis Data Streams is appropriate, using EC2 instances with Auto Scaling for processing introduces more management overhead compared to the serverless Lambda option. Also, Amazon Redshift is designed for analytical workloads and isn't the best choice for transactional storage of individual score updates.
Option C: Amazon SNS is designed for publish/subscribe messaging and doesn't guarantee message ordering. The mobile game needs to receive updates in order. Storing the processed updates in a SQL database running on EC2 increases management overhead and doesn't offer the same scalability as DynamoDB.
Option D: Amazon SQS provides a message queueing service, which isn't designed for preserving the order of

message within queue (unless configured using FIFO). It's ideal for decoupling components but might not be the best choice when order is important. Also, using EC2 instances with Auto Scaling for processing is good, but adds overhead. Although Amazon RDS Multi-AZ DB instance offers high availability, managing the database and EC2 instances leads to increased management overhead compared to serverless alternatives.

In summary: Option A provides the best balance of scalability, managed services, guaranteed data ordering, and minimal management overhead, making it the optimal solution for handling the mobile game's score updates.

Authoritative Links for Further Research:

Amazon Kinesis Data Streams: https://aws.amazon.com/kinesis/data-streams/
AWS Lambda: https://aws.amazon.com/lambda/
Amazon DynamoDB: https://aws.amazon.com/dynamodb/
Amazon SNS: https://aws.amazon.com/sns/
Amazon SQS: https://aws.amazon.com/sqs/
Amazon Redshift: https://aws.amazon.com/redshift/
Amazon RDS: https://aws.amazon.com/rds/

Question: 736 **Exam Heist**

A company has multiple AWS accounts with applications deployed in the us-west-2 Region. Application logs are stored within Amazon S3 buckets in each account. The company wants to build a centralized log analysis solution that uses a single S3 bucket. Logs must not leave us-west-2, and the company wants to incur minimal operational overhead.

Which solution meets these requirements and is MOST cost-effective?

A.Create an S3 Lifecycle policy that copies the objects from one of the application S3 buckets to the centralized S3 bucket.

B.Use S3 Same-Region Replication to replicate logs from the S3 buckets to another S3 bucket in us-west-2. Use this S3 bucket for log analysis.

C.Write a script that uses the PutObject API operation every day to copy the entire contents of the buckets to another S3 bucket in us-west-2. Use this S3 bucket for log analysis.

D.Write AWS Lambda functions in these accounts that are triggered every time logs are delivered to the S3 buckets (s3:ObjectCreated:* event). Copy the logs to another S3 bucket in us-west-2. Use this S3 bucket for log analysis.

Answer: B

Explanation:

Here's a detailed justification for why option B is the most suitable solution:

Option B, using S3 Same-Region Replication (SRR), is the most cost-effective and operationally efficient solution for centralizing logs from multiple AWS accounts into a single S3 bucket within the us-west-2 region. S3 SRR automatically and asynchronously replicates objects between S3 buckets in the same AWS Region. This aligns perfectly with the requirement of keeping the logs within us-west-2.

The main advantage of SRR is its ease of configuration and management. Once enabled between source and destination buckets, the replication process is fully managed by AWS S3, requiring minimal operational overhead. There's no need to write custom scripts or functions. This simplifies the log centralization process significantly.

Option A, using an S3 Lifecycle policy for copying, is less suitable because Lifecycle policies are primarily designed for object transition to cheaper storage classes (like Glacier) or deletion, not general cross-bucket

copying for analysis. While it could be configured, SRR is the more direct and intended mechanism for replication.

Option C, using a script with PutObject API calls, is the least efficient. It involves significant operational overhead in developing, deploying, and maintaining the script. Additionally, manually copying data is prone to errors and can lead to data inconsistencies if not implemented correctly. It also lacks the real-time aspect that is usually expected with log centralization.

Option D, employing Lambda functions triggered by s3:ObjectCreated:* events, is a viable option but more complex and costly than SRR. While it offers near real-time log transfer, it introduces the overhead of managing Lambda functions, including code development, deployment, monitoring, and potential scaling issues, along with associated Lambda execution costs. The cost of invoking Lambda per log write is likely to outweigh the S3 replication costs.

Compared to the other options, SRR leverages a built-in S3 feature designed specifically for data replication, reducing operational complexity and potential for errors. It's cost-effective due to its managed nature, and avoids the overhead associated with custom scripting or Lambda function management. This makes it the most suitable approach for the company's centralized log analysis needs.

For further research, consult these AWS documentation pages:

S3 Same-Region Replication (SRR):
https://docs.aws.amazon.com/AmazonS3/latest/userguide/replication.html
S3 Lifecycle Policies: https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-management.html

Question: 737 **Exam Heist**

A company has an application that delivers on-demand training videos to students around the world. The application also allows authorized content developers to upload videos. The data is stored in an Amazon S3 bucket in the us-east-2 Region.

The company has created an S3 bucket in the eu-west-2 Region and an S3 bucket in the ap-southeast-1 Region. The company wants to replicate the data to the new S3 buckets. The company needs to minimize latency for developers who upload videos and students who stream videos near eu-west-2 and ap-southeast-1.

Which combination of steps will meet these requirements with the FEWEST changes to the application? (Choose two.)

A.Configure one-way replication from the us-east-2 S3 bucket to the eu-west-2 S3 bucket. Configure one-way replication from the us-east-2 S3 bucket to the ap-southeast-1 S3 bucket.

B.Configure one-way replication from the us-east-2 S3 bucket to the eu-west-2 S3 bucket. Configure one-way replication from the eu-west-2 S3 bucket to the ap-southeast-1 S3 bucket.

C.Configure two-way (bidirectional) replication among the S3 buckets that are in all three Regions.

D.Create an S3 Multi-Region Access Point. Modify the application to use the Amazon Resource Name (ARN) of the Multi-Region Access Point for video streaming. Do not modify the application for video uploads.

E.Create an S3 Multi-Region Access Point. Modify the application to use the Amazon Resource Name (ARN) of the Multi-Region Access Point for video streaming and uploads.

Answer: CE

Explanation:

Here's a detailed justification for the answer choices C and E, and why the other options are incorrect:

Why C and E are correct:

E: Create an S3 Multi-Region Access Point. Modify the application to use the Amazon Resource Name (ARN) of the Multi-Region Access Point for video streaming and uploads. This is a core element of the solution because S3 Multi-Region Access Points (MRAP) are designed to simplify accessing data stored across multiple S3 buckets in different AWS Regions. By using a MRAP, the application interacts with a single endpoint (the ARN), and S3 intelligently routes requests to the appropriate bucket based on proximity, availability, and configured routing policies. This minimizes latency for both uploads and downloads because users are directed to the nearest S3 bucket. Modifying the application to use the MRAP ARN for both streaming and uploading is crucial for ensuring all requests benefit from the multi-region setup.

S3 Multi-Region Access Points documentation

C: Configure two-way (bidirectional) replication among the S3 buckets that are in all three Regions. Bidirectional replication is essential when content developers are uploading videos not just to us-east-2, but also to eu-west-2 and ap-southeast-1, or if there are developers in all these regions who upload content. Replicating data bi-directionally between all three regions ensures that any changes made in one region are automatically propagated to the other two regions. This keeps the data consistent and up-to-date across all locations, which is vital for content delivery and user experience. If uploads only occurred in the us-east-2 bucket, then bi-directional replication would not be required and would not be cost effective.

S3 Replication documentation

Why other options are incorrect:

A: Configure one-way replication from the us-east-2 S3 bucket to the eu-west-2 S3 bucket. Configure one-way replication from the us-east-2 S3 bucket to the ap-southeast-1 S3 bucket. This option addresses the replication requirement but doesn't minimize latency for developers uploading videos in the eu-west-2 and ap-southeast-1 regions. They would still be uploading to the us-east-2 bucket, which defeats the purpose of minimizing latency.
B: Configure one-way replication from the us-east-2 S3 bucket to the eu-west-2 S3 bucket. Configure one-way replication from the eu-west-2 S3 bucket to the ap-southeast-1 S3 bucket. Similar to option A, this setup doesn't address the latency issue for developers uploading videos in the ap-southeast-1 region. Uploads will be required from eu-west-2 to ap-southeast-1, which will not improve latency for uploads coming from the ap-southeast-1 region.
D: Create an S3 Multi-Region Access Point. Modify the application to use the Amazon Resource Name (ARN) of the Multi-Region Access Point for video streaming. Do not modify the application for video uploads. While using a Multi-Region Access Point for streaming is a good step to minimize latency for students, this option doesn't address the latency issue for developers uploading videos. They would still be uploading to the original us-east-2 bucket, negating the benefits of multi-region access.

In summary, the combination of creating a Multi-Region Access Point and configuring bidirectional replication ensures both low-latency video delivery for students and low-latency video uploads for content developers, regardless of their location.

Question: 738 **Exam Heist**

A company has a new mobile app. Anywhere in the world, users can see local news on topics they choose. Users also can post photos and videos from inside the app.

Users access content often in the first minutes after the content is posted. New content quickly replaces older content, and then the older content disappears. The local nature of the news means that users consume 90% of the content within the AWS Region where it is uploaded.

Which solution will optimize the user experience by providing the LOWEST latency for content uploads?

A.Upload and store content in Amazon S3. Use Amazon CloudFront for the uploads.

B.Upload and store content in Amazon S3. Use S3 Transfer Acceleration for the uploads.

C.Upload content to Amazon EC2 instances in the Region that is closest to the user. Copy the data to Amazon S3.

D.Upload and store content in Amazon S3 in the Region that is closest to the user. Use multiple distributions of Amazon CloudFront.

Answer: B

Explanation:

The optimal solution for minimizing content upload latency for a globally distributed mobile app with geographically localized content consumption is **B. Upload and store content in Amazon S3. Use S3 Transfer Acceleration for the uploads.**

Here's why:

S3 as the core storage: Amazon S3 provides highly durable, scalable, and available storage for the media assets (photos and videos). It's a natural fit for this use case because the content is largely consumed soon after upload, fitting the object storage paradigm well.
S3 Transfer Acceleration: This feature leverages the globally distributed AWS edge locations (same network used by CloudFront) to accelerate uploads to S3. When a user uploads content, it's routed to the closest edge location. Then, data is transferred to the destination S3 bucket over an optimized network path, potentially reducing latency. The edge location handles the TCP handshake and optimizes the transfer protocol.

Let's analyze why other options are not ideal:

A. Upload and store content in Amazon S3. Use Amazon CloudFront for the uploads. While CloudFront excels at content delivery, it primarily caches content that is already stored in an origin (like S3). CloudFront can accept uploads, but this functionality (using CloudFront for PUT requests) is generally less optimized for upload performance than S3 Transfer Acceleration, especially considering the use case involves a large volume of uploads. The primary function of CloudFront is CDN, and it is optimal for content distribution rather than ingestion.
C. Upload content to Amazon EC2 instances in the Region that is closest to the user. Copy the data to Amazon S3. Introducing EC2 instances adds unnecessary complexity and operational overhead. You'd need to manage these instances, ensure their availability, and handle scaling. The transfer from EC2 to S3 would still incur latency. Moreover, the data transfer out costs from EC2 could be higher.
D. Upload and store content in Amazon S3 in the Region that is closest to the user. Use multiple distributions of Amazon CloudFront. This would be a valid solution for content delivery but does not directly address the upload latency concern. You'd still need a fast mechanism to get the content into S3. The complexity of managing multiple distributions is also unnecessary for simply improving upload speeds. It doesn't help to resolve the initial upload latency to S3.

In summary, S3 Transfer Acceleration is specifically designed to speed up uploads to S3, making it the best option for this scenario where low latency uploads are crucial for user experience.

Authoritative Links:

Amazon S3 Transfer Acceleration: https://aws.amazon.com/s3/transfer-acceleration/
CloudFront Documentation: https://aws.amazon.com/cloudfront/

Question: 739 **Exam Heist**

A company is building a new application that uses serverless architecture. The architecture will consist of an Amazon API Gateway REST API and AWS Lambda functions to manage incoming requests.

The company wants to add a service that can send messages received from the API Gateway REST API to multiple target Lambda functions for processing. The service must offer message filtering that gives the target Lambda functions the ability to receive only the messages the functions need.

Which solution will meet these requirements with the LEAST operational overhead?

A.Send the requests from the API Gateway REST API to an Amazon Simple Notification Service (Amazon SNS) topic. Subscribe Amazon Simple Queue Service (Amazon SQS) queues to the SNS topic. Configure the target Lambda functions to poll the different SQS queues.

B.Send the requests from the API Gateway REST API to Amazon EventBridge. Configure EventBridge to invoke the target Lambda functions.

C.Send the requests from the API Gateway REST API to Amazon Managed Streaming for Apache Kafka (Amazon MSK). Configure Amazon MSK to publish the messages to the target Lambda functions.

D.Send the requests from the API Gateway REST API to multiple Amazon Simple Queue Service (Amazon SQS) queues. Configure the target Lambda functions to poll the different SQS queues.

Answer: A

Explanation:

The best solution is **A. Send the requests from the API Gateway REST API to an Amazon Simple Notification Service (Amazon SNS) topic. Subscribe Amazon Simple Queue Service (Amazon SQS) queues to the SNS topic. Configure the target Lambda functions to poll the different SQS queues.** This approach provides message filtering and fan-out capability with the least operational overhead.

Here's why:

SNS for Fan-out: Amazon SNS excels at distributing messages to multiple subscribers. It natively supports a publish-subscribe (pub/sub) pattern, which is perfect for sending the same message to multiple Lambda functions.

SQS for Decoupling and Reliability: By placing SQS queues between SNS and Lambda functions, you decouple the services. This adds resilience, allowing Lambda functions to process messages even if they are temporarily unavailable or experiencing errors. SQS also provides buffering, preventing message loss during traffic spikes.

SQS Filtering through SNS Message Filtering: SQS queues can subscribe to the SNS topic with filter policies. These policies allow each queue (and thus the corresponding Lambda function) to receive only messages with specific attributes. This satisfies the requirement for message filtering.

Reduced Operational Overhead: This solution minimizes the amount of custom code and infrastructure management. SNS and SQS are managed services, meaning AWS handles the underlying infrastructure, scaling, and maintenance.

Alternatives Considered:

B (EventBridge): While EventBridge can invoke Lambda functions, it is generally better suited for event-driven architectures that integrate various AWS services. Using it solely for API Gateway to Lambda fan-out adds unnecessary complexity compared to SNS+SQS.

C (Amazon MSK): MSK is designed for high-throughput, real-time streaming data, which is overkill for this use case. Managing an MSK cluster requires significantly more operational effort than SNS and SQS. It is also not directly integrated with API Gateway for message pushing.

D (Multiple SQS Queues): Sending the request directly from API Gateway to multiple SQS queues would require the API Gateway configuration to know about each target Lambda function. This creates tight coupling and doesn't support filtering without custom code within API Gateway to evaluate the messages and direct them to the appropriate queues. This approach violates the "least operational overhead" requirement.

In summary, SNS with SQS offers a cost-effective, scalable, and easily manageable solution for distributing messages to multiple Lambda functions with filtering capabilities, making it the best choice for this serverless architecture.

Relevant Documentation:

Amazon SNS: https://aws.amazon.com/sns/
Amazon SQS: https://aws.amazon.com/sqs/
SNS Message Filtering: https://docs.aws.amazon.com/sns/latest/dg/sns-message-filtering.html
API Gateway Lambda Integration: https://docs.aws.amazon.com/apigateway/latest/developerguide/services-lambda-integrations.html

Question: 740 **Exam Heist**

A company migrated millions of archival files to Amazon S3. A solutions architect needs to implement a solution that will encrypt all the archival data by using a customer-provided key. The solution must encrypt existing unencrypted objects and future objects.

Which solution will meet these requirements?

A.Create a list of unencrypted objects by filtering an Amazon S3 Inventory report. Configure an S3 Batch Operations job to encrypt the objects from the list with a server-side encryption with a customer-provided key (SSE-C). Configure the S3 default encryption feature to use a server-side encryption with a customer-provided key (SSE-C).

B.Use S3 Storage Lens metrics to identify unencrypted S3 buckets. Configure the S3 default encryption feature to use a server-side encryption with AWS KMS keys (SSE-KMS).

C.Create a list of unencrypted objects by filtering the AWS usage report for Amazon S3. Configure an AWS Batch job to encrypt the objects from the list with a server-side encryption with AWS KMS keys (SSE-KMS). Configure the S3 default encryption feature to use a server-side encryption with AWS KMS keys (SSE-KMS).

D.Create a list of unencrypted objects by filtering the AWS usage report for Amazon S3. Configure the S3 default encryption feature to use a server-side encryption with a customer-provided key (SSE-C).

Answer: A

Explanation:

The correct answer is A because it directly addresses both requirements: encrypting existing unencrypted objects and ensuring future objects are encrypted with SSE-C.

Encrypting existing objects: S3 Inventory is the efficient way to identify unencrypted objects within an S3 bucket. Filtering the Inventory report provides a list of unencrypted objects. S3 Batch Operations is then used to process this list and encrypt each object using SSE-C, ensuring the archival data is encrypted with the customer-provided key.
Encrypting future objects: Configuring the S3 default encryption feature to use SSE-C guarantees that all new objects uploaded to the bucket will be automatically encrypted with the customer-provided key. This ensures all future archival data is encrypted as required.

Other options are incorrect because:

B: S3 Storage Lens is focused on bucket-level metrics and doesn't identify individual unencrypted objects. SSE-KMS is not SSE-C as required.
C: The AWS usage report is not designed to list unencrypted objects and AWS Batch can't be directly used to encrypt S3 objects. SSE-KMS is not SSE-C as required.
D: The AWS usage report is not designed to list unencrypted objects. Setting the S3 default encryption handles future objects but doesn't encrypt existing ones.

Authoritative Links:

Amazon S3 Inventory: https://docs.aws.amazon.com/AmazonS3/latest/userguide/storage-inventory.html
Amazon S3 Batch Operations: https://docs.aws.amazon.com/AmazonS3/latest/userguide/batch-ops-manage.html
Amazon S3 Default Encryption: https://docs.aws.amazon.com/AmazonS3/latest/userguide/default-bucket-encryption.html
Server-Side Encryption with Customer-Provided Keys (SSE-C): https://docs.aws.amazon.com/AmazonS3/latest/userguide/ServerSideEncryptionCustomerKeys.html

Question: 741 **Exam Heist**

The DNS provider that hosts a company's domain name records is experiencing outages that cause service disruption for a website running on AWS. The company needs to migrate to a more resilient managed DNS service and wants the service to run on AWS.

What should a solutions architect do to rapidly migrate the DNS hosting service?

A.Create an Amazon Route 53 public hosted zone for the domain name. Import the zone file containing the domain records hosted by the previous provider.

B.Create an Amazon Route 53 private hosted zone for the domain name. Import the zone file containing the domain records hosted by the previous provider.

C.Create a Simple AD directory in AWS. Enable zone transfer between the DNS provider and AWS Directory Service for Microsoft Active Directory for the domain records.

D.Create an Amazon Route 53 Resolver inbound endpoint in the VPC. Specify the IP addresses that the provider's DNS will forward DNS queries to. Configure the provider's DNS to forward DNS queries for the domain to the IP addresses that are specified in the inbound endpoint.

Answer: A

Explanation:

Here's a detailed justification for why option A is the correct answer:

The primary goal is to migrate a publicly accessible website's DNS hosting to a more resilient AWS service, specifically in response to current outages. Amazon Route 53 is AWS's highly available and scalable DNS service. To host a public website's DNS records, a **public hosted zone** in Route 53 is required. This is because the public hosted zone contains records that are accessible to the internet, allowing users to resolve the website's domain name to the correct IP address.

The fastest way to migrate the existing DNS configuration is to **import the zone file** from the previous provider. A zone file is a standard text file that contains all the DNS records (e.g., A, CNAME, MX) for a domain. Route 53 provides a mechanism to import zone files, significantly reducing the manual effort required to recreate the records. This ensures minimal disruption during the migration.

Option B is incorrect because a **private hosted zone** is used for internal DNS resolution within a VPC, not for public websites. Public users won't be able to resolve the website's domain through a private hosted zone.

Option C is incorrect. **Simple AD** is used to create a directory service integrated with AWS. While it can be used for DNS, it is more complicated than Route 53 for public DNS migration. Also, the question is more about rapidly migrating a DNS service, not setting up an entirely new directory service solution. While AWS Directory Service for Microsoft Active Directory can handle DNS forwarding, it is an overkill to use it to simply migrate to a more resilient DNS service.

Option D is incorrect because **Route 53 Resolver inbound endpoints** are used for hybrid cloud scenarios where you want to forward DNS queries from your on-premises network to AWS. This is the reverse of what's

needed – the question is about moving to AWS for DNS hosting. Route 53 resolver can handle the DNS queries if they are hosted there; the scenario is to use a third party and forward those queries to AWS.Therefore, creating a public hosted zone and importing the existing records is the most efficient and appropriate solution for rapidly migrating a public website's DNS hosting to AWS.

Here are authoritative links for further research:

Amazon Route 53: https://aws.amazon.com/route53/
Working with Public Hosted Zones: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/hosted-zones-working-with.html
Importing a Zone File: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/migrate-dns-zone-file.html

Question: 742 **Exam Heist**

A company is building an application on AWS that connects to an Amazon RDS database. The company wants to manage the application configuration and to securely store and retrieve credentials for the database and other services.

Which solution will meet these requirements with the LEAST administrative overhead?

A.Use AWS AppConfig to store and manage the application configuration. Use AWS Secrets Manager to store and retrieve the credentials.

B.Use AWS Lambda to store and manage the application configuration. Use AWS Systems Manager Parameter Store to store and retrieve the credentials.

C.Use an encrypted application configuration file. Store the file in Amazon S3 for the application configuration. Create another S3 file to store and retrieve the credentials.

D.Use AWS AppConfig to store and manage the application configuration. Use Amazon RDS to store and retrieve the credentials.

Answer: A

Explanation:

The correct answer is A because it offers the most suitable and streamlined approach for managing application configurations and securely handling credentials with minimal administrative burden.

AWS AppConfig is specifically designed for managing application configurations. It allows you to create, manage, and deploy application configuration updates in a controlled and validated manner. AppConfig integrates with other AWS services, enabling you to validate configuration data before deploying it to your applications, thereby reducing the risk of configuration errors and application downtime. It offers features like validation schemas, controlled rollouts, and rollback capabilities. https://aws.amazon.com/appconfig/

AWS Secrets Manager is the AWS service dedicated to securely storing and retrieving sensitive information like database credentials, API keys, and other secrets. It enables you to rotate, manage, and retrieve secrets throughout their lifecycle. Secrets Manager also offers automatic rotation of database credentials, enhancing security without requiring manual intervention. https://aws.amazon.com/secrets-manager/

Option B is less suitable. While AWS Systems Manager Parameter Store can store credentials, it's primarily intended for configuration data and operational parameters rather than sensitive secrets. Lambda functions are not designed to manage application configuration.

Option C is not recommended due to security vulnerabilities. Storing credentials in an encrypted file within S3 introduces risks of unauthorized access if the file is compromised, or the encryption key is leaked. This method requires manual management and rotation of encryption keys and credentials.

Option D is incorrect because using Amazon RDS to store application configuration is not its intended purpose, and it complicates the database schema and overall architecture. Furthermore, it does not provide the same level of features and benefits that AppConfig provides for managing configuration changes. RDS is designed for storing structured data within a relational database, not for storing configurations or secrets. Using RDS to store secrets exposes security vulnerabilities.

Therefore, the combination of AWS AppConfig for configuration management and AWS Secrets Manager for secure credential storage offers the most secure, scalable, and manageable solution with the least administrative overhead.

Question: 743 **Exam Heist**

To meet security requirements, a company needs to encrypt all of its application data in transit while communicating with an Amazon RDS MySQL DB instance. A recent security audit revealed that encryption at rest is enabled using AWS Key Management Service (AWS KMS), but data in transit is not enabled.

What should a solutions architect do to satisfy the security requirements?

A.Enable IAM database authentication on the database.
B.Provide self-signed certificates. Use the certificates in all connections to the RDS instance.
C.Take a snapshot of the RDS instance. Restore the snapshot to a new instance with encryption enabled.
D.Download AWS-provided root certificates. Provide the certificates in all connections to the RDS instance.

Answer: D

Explanation:

The correct solution is to download AWS-provided root certificates and use them in all connections to the RDS instance because it enables SSL/TLS encryption for data in transit, securing communication between the application and the RDS MySQL database.

Option A, enabling IAM database authentication, primarily manages authentication and authorization, not encryption of data in transit. While it enhances security by controlling access, it doesn't address the requirement of encrypting data while it's being transferred.

Option B, providing self-signed certificates, is not a best practice in production environments due to security concerns. Self-signed certificates are not trusted by default and can lead to man-in-the-middle attacks. Using AWS-provided certificates ensures trust and validity.

Option C, taking a snapshot and restoring it to a new instance with encryption enabled, addresses encryption at rest, which is already configured. The security audit revealed that data in transit encryption is the missing piece, so this option doesn't solve the core issue.

By using AWS-provided root certificates, the application can establish an SSL/TLS connection with the RDS instance. SSL/TLS encrypts the data stream between the application and the database, ensuring that sensitive information is protected during transmission. These certificates are pre-trusted and automatically updated by AWS, simplifying management and improving security posture. This approach directly fulfills the security requirement of encrypting all application data in transit while communicating with the RDS MySQL DB instance.

Reference:

Using SSL/TLS to encrypt a connection to a DB instance

Question: 744 **Exam Heist**

A company is designing a new web service that will run on Amazon EC2 instances behind an Elastic Load Balancing (ELB) load balancer. However, many of the web service clients can only reach IP addresses authorized on their firewalls.

What should a solutions architect recommend to meet the clients' needs?

A. A Network Load Balancer with an associated Elastic IP address.
B. An Application Load Balancer with an associated Elastic IP address.
C. An A record in an Amazon Route 53 hosted zone pointing to an Elastic IP address.
D. An EC2 instance with a public IP address running as a proxy in front of the load balancer.

Answer: C

Explanation:

The correct answer is **C. An A record in an Amazon Route 53 hosted zone pointing to an Elastic IP address.**

Here's why:

The primary requirement is that clients, with firewall restrictions allowing only specific IP addresses, need to access the web service. Using Elastic IPs directly tied to the load balancer (options A and B) is generally problematic and doesn't align with best practices. Elastic Load Balancers (both Application and Network Load Balancers) are dynamic. While you can technically associate Elastic IPs with Network Load Balancers, it's generally discouraged because the whole point of a load balancer is to distribute traffic across multiple instances which may change, and fixing a static IP to it defeats this purpose and can introduce availability risks. Application Load Balancers cannot directly associate with Elastic IPs. This makes options A and B technically impractical, and certainly not the best solution.

Option D, using a proxy EC2 instance with a public IP, is also suboptimal. It introduces a single point of failure and adds unnecessary operational overhead for managing the proxy server. It also complicates the architecture when a load balancer is already in place.

Option C provides a cleaner and more scalable solution. By creating an A record in Route 53 that resolves to an Elastic IP address, clients can use the IP address in their firewall rules. Here's how it works:

1. **Elastic IP Address:** You create and associate an Elastic IP address. This IP address remains constant and can be moved between resources (although not while it's assigned to the load balancer as we will resolve it to the load balancer's DNS name).
2. **Route 53 A Record:** In Route 53, you create an A record for a domain name (e.g., api.example.com) and point it to the Elastic IP address. **Important:** Instead of directly assigning the Elastic IP to the ELB, the DNS record initially points to the Elastic IP, and the Elastic IP is configured to point to the ELB's DNS name (CNAME) internally within Route 53 or via a custom resource. This means that even if the underlying EC2 instances behind the ELB change (due to scaling or failures), the clients' firewalls remain unaffected because they only allow traffic to the static Elastic IP. However, we never expose the ELB's IP addresses directly to the internet. This is a critical difference.

This approach offers several advantages:

Static IP for Clients: Clients can use the Elastic IP address in their firewall rules, ensuring connectivity.
Dynamic Backend: The load balancer can still dynamically distribute traffic across multiple EC2 instances. The Elastic IP acts as a consistent entry point, while the load balancer handles the backend infrastructure changes.
Scalability and High Availability: The load balancer provides scalability and high availability for the web service.

Managed DNS: Route 53 is a highly available and scalable DNS service.
Cost-Effective: Compared to managing a proxy server, using Route 53 is generally more cost-effective.

In summary, using an A record in Route 53 pointing to an Elastic IP address provides a stable and manageable entry point for clients with firewall restrictions while leveraging the benefits of a load balancer for scalability and high availability.

Authoritative Links:

Elastic IP Addresses: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/elastic-ip-addresses-eip.html
Amazon Route 53: https://aws.amazon.com/route53/
Elastic Load Balancing: https://aws.amazon.com/elasticloadbalancing/

Clarification: To be completely clear, the A record (api.example.com) points to the Elastic IP, and then some mechanism, likely a custom AWS Lambda, or manual intervention would be required to ensure the Elastic IP continues to point to the load balancer's DNS name. This indirection makes it possible to use the ELB's load balancing functionality without exposing the ELB's dynamic IP addresses to the internet. An alternative strategy would be to utilize AWS Global Accelerator with its static IP addresses and direct traffic through it to the ALB. This would also accomplish the task but can introduce additional cost and complexity.

Question: 745 **Exam Heist**

A company has established a new AWS account. The account is newly provisioned and no changes have been made to the default settings. The company is concerned about the security of the AWS account root user.

What should be done to secure the root user?

A.Create IAM users for daily administrative tasks. Disable the root user.

B.Create IAM users for daily administrative tasks. Enable multi-factor authentication on the root user.

C.Generate an access key for the root user. Use the access key for daily administration tasks instead of the AWS Management Console.

D.Provide the root user credentials to the most senior solutions architect. Have the solutions architect use the root user for daily administration tasks.

Answer: B

Explanation:

The correct answer is B: Create IAM users for daily administrative tasks. Enable multi-factor authentication on the root user. Here's why:

The AWS account root user has unrestricted access to all AWS resources in the account. It should only be used for initial setup tasks or when root access is explicitly required, such as changing account settings. Daily administrative tasks should never be performed with the root user due to the high risk of accidental or malicious misuse. Option A suggests disabling the root user, which is not recommended. The root user needs to be available for essential account management activities like changing the AWS support plan or closing the account.

Creating IAM users (option B) follows the principle of least privilege. IAM users can be granted specific permissions needed for their roles, limiting the potential impact of a security breach or accidental misconfiguration. This greatly reduces the attack surface.

Furthermore, enabling multi-factor authentication (MFA) on the root user account adds a crucial layer of security. Even if the root user's password is compromised, an attacker would still need the MFA device to gain access. MFA makes it significantly harder to compromise the root user.

Option C suggests generating an access key for the root user. This is a dangerous practice. Access keys should be used carefully, and not for day-to-day use. Using an access key on the root account creates a significant security risk. If the key is compromised, the entire account is compromised. IAM users should always be utilized with appropriate policies attached.

Option D is fundamentally incorrect. Giving the root user credentials to a single individual for daily tasks completely defeats the purpose of least privilege and shared responsibility. It centralizes a critical security risk and violates best practices.

In summary, using IAM users for daily tasks with MFA on the root account is the most secure approach. It protects the root account, promotes least privilege, and is a fundamental security practice in AWS. Disabling the root account is discouraged because it's needed for specific administrative tasks. Giving the root user key or credentials for daily tasks makes the account vulnerable.

Relevant links:

AWS Identity and Access Management (IAM): https://aws.amazon.com/iam/
Securing Your AWS Account Root User: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_root-user.html
IAM Best Practices: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

Question: 746 **Exam Heist**

A company is deploying an application that processes streaming data in near-real time. The company plans to use Amazon EC2 instances for the workload. The network architecture must be configurable to provide the lowest possible latency between nodes.

Which combination of network solutions will meet these requirements? (Choose two.)

A.Enable and configure enhanced networking on each EC2 instance.
B.Group the EC2 instances in separate accounts.
C.Run the EC2 instances in a cluster placement group.
D.Attach multiple elastic network interfaces to each EC2 instance.
E.Use Amazon Elastic Block Store (Amazon EBS) optimized instance types.

Answer: AC

Explanation:

The optimal solution for minimizing latency between EC2 instances processing streaming data in near real-time necessitates reducing network hops and maximizing network throughput.

Option A, enabling and configuring enhanced networking, is crucial. Enhanced networking utilizes Single Root I/O Virtualization (SR-IOV) to provide high performance networking capabilities on supported EC2 instance types. This bypasses the virtual switch layer, enabling lower latency, lower jitter, and higher packets per second (PPS) performance. By leveraging SR-IOV, data transmission between instances benefits from a more direct path, minimizing delays. [https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/enhanced-networking.html]

Option C, running the EC2 instances in a cluster placement group, is also vital. A cluster placement group places instances within a single Availability Zone, grouping them as close as possible to each other. This proximity reduces network latency because traffic doesn't have to traverse across different zones, thereby providing high network throughput. This minimizes the physical distance and network hops, crucial for low-latency communication. [https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/placement-groups.html]

Option B is incorrect as separating instances into different accounts introduces greater network overhead and likely increases latency due to cross-account communication complexities. Option D, attaching multiple ENIs, doesn't inherently lower latency; it primarily increases network bandwidth and provides redundancy, and the increased complexity can sometimes introduce latency. Option E, using EBS-optimized instance types, optimizes the throughput between the EC2 instance and EBS volumes, impacting storage I/O performance, not network latency between instances. Therefore, only A and C directly address the low latency requirement.

Question: 747 **Exam Heist**

A financial services company wants to shut down two data centers and migrate more than 100 TB of data to AWS. The data has an intricate directory structure with millions of small files stored in deep hierarchies of subfolders. Most of the data is unstructured, and the company's file storage consists of SMB-based storage types from multiple vendors. The company does not want to change its applications to access the data after migration.

What should a solutions architect do to meet these requirements with the LEAST operational overhead?

A. Use AWS Direct Connect to migrate the data to Amazon S3.

B. Use AWS DataSync to migrate the data to Amazon FSx for Lustre.

C. Use AWS DataSync to migrate the data to Amazon FSx for Windows File Server.

D. Use AWS Direct Connect to migrate the data on-premises file storage to an AWS Storage Gateway volume gateway.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best solution, considering the requirements of a financial services company migrating a large amount of unstructured data with complex directory structure from SMB-based file storage to AWS, while minimizing operational overhead and application changes:

Option C, using AWS DataSync to migrate the data to Amazon FSx for Windows File Server, is the most suitable choice for several key reasons. The company's reliance on SMB-based storage types points towards Windows-centric applications and workflows. Amazon FSx for Windows File Server provides a fully managed, native Windows file system compatible with the SMB protocol. This compatibility allows the company to migrate its data without needing to refactor existing applications, as they can continue to access files using the same SMB paths. AWS DataSync is designed to efficiently move large datasets between on-premises storage and AWS storage services. It optimizes data transfer using features like parallel data streams, in-transit encryption, and automatic handling of file metadata. This helps to accelerate the migration process and ensures data integrity.

Option A, using AWS Direct Connect to migrate data to Amazon S3, requires application changes. S3 is an object storage service, which doesn't directly support the hierarchical file system structure that the company currently utilizes. Adapting applications to use the S3 API would involve significant development effort and operational overhead.

Option B, using AWS DataSync to migrate the data to Amazon FSx for Lustre, isn't the best fit because FSx for Lustre is optimized for high-performance computing and doesn't offer native SMB protocol support. While it could handle large datasets, it would likely require application changes to interact with Lustre's parallel file system.

Option D, using AWS Direct Connect to migrate the on-premises file storage to an AWS Storage Gateway volume gateway, introduces operational overhead related to managing the Storage Gateway appliance or VM and the underlying storage it fronts. Furthermore, it doesn't inherently solve the application compatibility issue as it mostly provides block-level storage.

In summary, option C provides the best balance between ease of migration (DataSync), application compatibility (FSx for Windows File Server's SMB support), and minimizing operational overhead through a fully managed AWS service. FSx for Windows File Server addresses the key requirements of maintaining the existing file structure and application compatibility.

Authoritative links:

AWS DataSync: https://aws.amazon.com/datasync/
Amazon FSx for Windows File Server: https://aws.amazon.com/fsx/windows/

Question: 748 **Exam Heist**

A company uses an organization in AWS Organizations to manage AWS accounts that contain applications. The company sets up a dedicated monitoring member account in the organization. The company wants to query and visualize observability data across the accounts by using Amazon CloudWatch.

Which solution will meet these requirements?

A.Enable CloudWatch cross-account observability for the monitoring account. Deploy an AWS CloudFormation template provided by the monitoring account in each AWS account to share the data with the monitoring account.

B.Set up service control policies (SCPs) to provide access to CloudWatch in the monitoring account under the Organizations root organizational unit (OU).

C.Configure a new IAM user in the monitoring account. In each AWS account, configure an IAM policy to have access to query and visualize the CloudWatch data in the account. Attach the new IAM policy to the new IAM user.

D.Create a new IAM user in the monitoring account. Create cross-account IAM policies in each AWS account. Attach the IAM policies to the new IAM user.

Answer: A

Explanation:

The correct answer is **A** because it leverages the built-in, purpose-built features of CloudWatch cross-account observability, designed specifically for centralizing monitoring data across multiple AWS accounts within an organization.

Here's a detailed justification:

1. **CloudWatch Cross-Account Observability:** AWS provides a feature called "CloudWatch cross-account observability" that allows you to monitor and troubleshoot applications across multiple AWS accounts from a central monitoring account. This feature eliminates the need to manually configure complex IAM roles and policies in each account. https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch-Unified-Cross-Account.html
2. **Monitoring Account as Central Hub:** The question explicitly states the desire for a dedicated monitoring member account. CloudWatch cross-account observability is designed with this use case in mind, allowing you to designate one account as the monitoring hub.
3. **Simplified Data Sharing:** To share data with the monitoring account, a CloudFormation template provided by the monitoring account needs to be deployed in each source account. This template sets up the necessary permissions and configurations for securely transmitting observability data to the central monitoring account. This automation is essential for managing many accounts within an organization.
4. **Avoiding Complex IAM Management:** Option C and D suggest creating IAM users and cross-account

roles/policies manually. This approach becomes increasingly complex and difficult to manage as the number of accounts grows, contradicting the need for a scalable solution in a multi-account AWS Organizations setup. Option A simplifies this by using pre-built CloudFormation templates.

5. **SCPs Are Not Intended for Data Access:** Service Control Policies (SCPs), as suggested in option B, are primarily used to set guardrails and enforce policies at the organization level. While SCPs can restrict actions, they are not designed for granting granular access to CloudWatch data within the monitoring account. They operate at a higher level of abstraction, controlling which services and actions are allowed, rather than facilitating data sharing. https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scp.html

6. **Scalability and Manageability:** CloudWatch cross-account observability provides a scalable and manageable solution for monitoring across multiple accounts. By utilizing CloudFormation templates, the deployment and configuration process can be automated and easily replicated across all accounts in the organization.

In conclusion, option A provides the most efficient and scalable solution for querying and visualizing observability data across multiple AWS accounts by using Amazon CloudWatch, aligned with the design principles of AWS Organizations and the intended use of CloudWatch cross-account observability. The other options involve more manual configuration and less efficient use of AWS features.

Question: 749 **Exam Heist**

A company's website is used to sell products to the public. The site runs on Amazon EC2 instances in an Auto Scaling group behind an Application Load Balancer (ALB). There is also an Amazon CloudFront distribution, and AWS WAF is being used to protect against SQL injection attacks. The ALB is the origin for the CloudFront distribution. A recent review of security logs revealed an external malicious IP that needs to be blocked from accessing the website.

What should a solutions architect do to protect the application?

A.Modify the network ACL on the CloudFront distribution to add a deny rule for the malicious IP address.

B.Modify the configuration of AWS WAF to add an IP match condition to block the malicious IP address.

C.Modify the network ACL for the EC2 instances in the target groups behind the ALB to deny the malicious IP address.

D.Modify the security groups for the EC2 instances in the target groups behind the ALB to deny the malicious IP address.

Answer: B

Explanation:

The correct answer is **B. Modify the configuration of AWS WAF to add an IP match condition to block the malicious IP address.** Here's a detailed justification:

AWS WAF (Web Application Firewall) is specifically designed to protect web applications from common web exploits and bots. It operates at Layer 7 of the OSI model, allowing it to inspect HTTP/HTTPS traffic and apply rules based on various criteria, including IP addresses.

Since the requirement is to block a specific malicious IP address, WAF provides the most suitable and direct method. An IP match condition within WAF can be configured to block requests originating from that IP address. This rule will be evaluated for every incoming request, preventing the malicious IP from reaching the ALB and the EC2 instances. WAF is integrated directly with CloudFront, and protects CloudFront traffic.

Option A is incorrect because Network ACLs (NACLs) operate at Layer 3 and 4 of the OSI model and are

primarily designed for controlling network traffic at the subnet level. While NACLs can block IP addresses, using them at the CloudFront distribution would be less efficient than using WAF, because WAF integrates directly with CloudFront.

Option C is not optimal because modifying the NACL for the EC2 instances would require the traffic to traverse CloudFront, the ALB, and then be blocked by the NACL. This approach wastes resources and does not prevent the malicious traffic from reaching the ALB. This is also less efficient than blocking traffic at the WAF level before it reaches those resources.

Option D is not the ideal approach, as security groups are instance-level firewalls that primarily control traffic based on source and destination IP addresses, ports, and protocols. Modifying the security group would require the malicious traffic to traverse CloudFront and the ALB. Again, this is also less efficient than blocking traffic at the WAF level before it reaches those resources.

Here are some authoritative links for further research:

AWS WAF: https://aws.amazon.com/waf/
AWS WAF IP Address Condition: https://docs.aws.amazon.com/waf/latest/developerguide/waf-ip-condition.html
AWS Network ACLs: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html
AWS Security Groups: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html
Amazon CloudFront: https://aws.amazon.com/cloudfront/

Question: 750 **Exam Heist**

A company sets up an organization in AWS Organizations that contains 10 AWS accounts. A solutions architect must design a solution to provide access to the accounts for several thousand employees. The company has an existing identity provider (IdP). The company wants to use the existing IdP for authentication to AWS.

Which solution will meet these requirements?

A.Create IAM users for the employees in the required AWS accounts. Connect IAM users to the existing IdP. Configure federated authentication for the IAM users.

B.Set up AWS account root users with user email addresses and passwords that are synchronized from the existing IdP.

C.Configure AWS IAM Identity Center (AWS Single Sign-On). Connect IAM Identity Center to the existing IdP. Provision users and groups from the existing IdP.

D.Use AWS Resource Access Manager (AWS RAM) to share access to the AWS accounts with the users in the existing IdP.

Answer: C

Explanation:

The correct answer is C because it leverages AWS IAM Identity Center (successor to AWS Single Sign-On), which is specifically designed for centralized management of access to multiple AWS accounts using an existing identity provider (IdP). IAM Identity Center directly integrates with popular IdPs like Okta, Azure AD, and Ping Identity, enabling users to authenticate using their existing credentials. After successful authentication with the IdP, IAM Identity Center provides users with temporary AWS credentials, allowing them to access the AWS accounts and resources assigned to them based on predefined permissions.

Option A is incorrect because creating individual IAM users in each of the 10 AWS accounts for thousands of employees would be incredibly cumbersome and difficult to manage. While IAM federation is possible, it doesn't provide the centralized management and single sign-on capabilities needed for this scenario. Manually synchronizing thousands of users across multiple accounts is also prone to errors.

Option B is incorrect and a highly discouraged practice. AWS account root users should never be used for everyday access. Sharing root user credentials or synchronizing them with an IdP is a significant security risk, as the root user has unrestricted access to the entire AWS account.

Option D is incorrect because AWS Resource Access Manager (RAM) is designed for sharing AWS resources (like VPCs, subnets, and transit gateways) between AWS accounts, not for managing user access across multiple accounts. RAM doesn't handle authentication or integration with external identity providers. It only allows you to grant resource-level permissions to accounts.

Therefore, using IAM Identity Center to connect to the existing IdP is the most efficient, secure, and scalable solution for managing access to multiple AWS accounts for a large number of employees using their existing credentials. It simplifies user management, improves security, and provides a better user experience.

Supporting links:

AWS IAM Identity Center (successor to AWS Single Sign-On)
AWS Organizations
Identity Providers and Federation

Question: 751 **Exam Heist**

A solutions architect is designing an AWS Identity and Access Management (IAM) authorization model for a company's AWS account. The company has designated five specific employees to have full access to AWS services and resources in the AWS account.

The solutions architect has created an IAM user for each of the five designated employees and has created an IAM user group.

Which solution will meet these requirements?

A.Attach the AdministratorAccess resource-based policy to the IAM user group. Place each of the five designated employee IAM users in the IAM user group.

B.Attach the SystemAdministrator identity-based policy to the IAM user group. Place each of the five designated employee IAM users in the IAM user group.

C.Attach the AdministratorAccess identity-based policy to the IAM user group. Place each of the five designated employee IAM users in the IAM user group.

D.Attach the SystemAdministrator resource-based policy to the IAM user group. Place each of the five designated employee IAM users in the IAM user group.

Answer: C

Explanation:

The correct answer is C, attaching the AdministratorAccess identity-based policy to the IAM user group and placing the five employee IAM users in the group. Here's why:

IAM Policies: IAM policies define permissions that determine what actions an identity (user, group, or role) can perform on AWS resources.
Identity-Based Policies: These policies are attached directly to IAM users, groups, or roles. They grant permissions to the identity to perform actions on resources. This is the standard and recommended way to grant permissions to users.
Resource-Based Policies: These policies are attached to AWS resources (e.g., S3 buckets, KMS keys). They define who can access the resource and what actions they can perform. Resource-based policies are typically used to grant permissions to other AWS accounts or services to access the resource. Applying a resource-based policy to a user group wouldn't achieve the goal of giving the users themselves admin access across the account.

AdministratorAccess Policy: This is a pre-defined AWS managed policy that grants full access to all AWS services and resources within an account.
SystemAdministrator Policy: There is no AWS managed policy named "SystemAdministrator". This is likely a custom policy, but it is not a standard AWS policy.

Therefore, to provide five employees with full administrative access, attaching the AdministratorAccess identity-based policy to an IAM user group and adding those five users to that group is the correct approach. This grants the group (and thus its members) the necessary permissions efficiently. Using a user group also simplifies permission management as future admin users can simply be added to the group.

Why other options are incorrect:

A & D: Resource-based policies are not applicable here. They control access to a resource, not permissions of a user or group.
B: The policy SystemAdministrator is not a standard AWS managed policy, which could grant insufficient or incorrect permissions.

Authoritative Links:

IAM Policies Overview: https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies.html
AWS Managed Policies: https://docs.aws.amazon.com/IAM/latest/UserGuide/aws-managed-policies.html
Identity-Based Policies vs. Resource-Based Policies:
https://docs.aws.amazon.com/IAM/latest/UserGuide/security-identity-policy-evaluations.html

Question: 752 **Exam Heist**

A company has a multi-tier payment processing application that is based on virtual machines (VMs). The communication between the tiers occurs asynchronously through a third-party middleware solution that guarantees exactly-once delivery.

The company needs a solution that requires the least amount of infrastructure management. The solution must guarantee exactly-once delivery for application messaging.

Which combination of actions will meet these requirements? (Choose two.)

A.Use AWS Lambda for the compute layers in the architecture.

B.Use Amazon EC2 instances for the compute layers in the architecture.

C.Use Amazon Simple Notification Service (Amazon SNS) as the messaging component between the compute layers.

D.Use Amazon Simple Queue Service (Amazon SQS) FIFO queues as the messaging component between the compute layers.

E.Use containers that are based on Amazon Elastic Kubernetes Service (Amazon EKS) for the compute layers in the architecture.

Answer: AD

Explanation:

The correct answer is AD. Here's a detailed justification:

A. Use AWS Lambda for the compute layers in the architecture: AWS Lambda is a serverless compute service. This aligns with the requirement of minimizing infrastructure management because Lambda abstracts away the need to provision and manage servers. The company can focus solely on writing the code for the payment processing tiers. This reduces operational overhead.

D. Use Amazon Simple Queue Service (Amazon SQS) FIFO queues as the messaging component between

the compute layers: SQS FIFO (First-In, First-Out) queues provide exactly-once processing and preserve the order of messages. The application requirement mandates exactly-once delivery, which SQS FIFO directly addresses. Standard SQS queues offer at-least-once delivery, which could result in duplicate processing, so using FIFO is crucial.

Let's examine why the other options are not ideal:

B. Use Amazon EC2 instances for the compute layers in the architecture: While EC2 provides flexibility, it contradicts the requirement of minimizing infrastructure management. EC2 instances require patching, scaling, and other management tasks, adding operational complexity.

C. Use Amazon Simple Notification Service (Amazon SNS) as the messaging component between the compute layers: SNS is a publish-subscribe service, not a queue. It doesn't guarantee exactly-once delivery or message ordering. SNS typically provides at-least-once delivery. Therefore, it doesn't satisfy the application's requirements.

E. Use containers that are based on Amazon Elastic Kubernetes Service (Amazon EKS) for the compute layers in the architecture: EKS offers container orchestration, which can be beneficial for managing complex applications. However, it requires significant infrastructure management related to the Kubernetes control plane, worker nodes, and associated networking and security configurations. This contradicts the "least amount of infrastructure management" requirement. Serverless compute is more appropriate here.

In summary, Lambda eliminates server management for compute, and SQS FIFO queues guarantee exactly-once delivery for messaging, making them the most suitable combination to fulfill the specified requirements.

Authoritative Links:

AWS Lambda: https://aws.amazon.com/lambda/
Amazon SQS FIFO queues:
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-fifo-queues.html

Question: 753 **Exam Heist**

A company has a nightly batch processing routine that analyzes report files that an on-premises file system receives daily through SFTP. The company wants to move the solution to the AWS Cloud. The solution must be highly available and resilient. The solution also must minimize operational effort.

Which solution meets these requirements?

A.Deploy AWS Transfer for SFTP and an Amazon Elastic File System (Amazon EFS) file system for storage. Use an Amazon EC2 instance in an Auto Scaling group with a scheduled scaling policy to run the batch operation.

B.Deploy an Amazon EC2 instance that runs Linux and an SFTP service. Use an Amazon Elastic Block Store (Amazon EBS) volume for storage. Use an Auto Scaling group with the minimum number of instances and desired number of instances set to 1.

C.Deploy an Amazon EC2 instance that runs Linux and an SFTP service. Use an Amazon Elastic File System (Amazon EFS) file system for storage. Use an Auto Scaling group with the minimum number of instances and desired number of instances set to 1.

D.Deploy AWS Transfer for SFTP and an Amazon S3 bucket for storage. Modify the application to pull the batch files from Amazon S3 to an Amazon EC2 instance for processing. Use an EC2 instance in an Auto Scaling group with a scheduled scaling policy to run the batch operation.

Answer: D

Explanation:

The best solution is D because it leverages fully managed services like AWS Transfer for SFTP and Amazon S3, minimizing operational overhead. AWS Transfer for SFTP handles the secure file transfer aspects,

removing the need to manage SFTP servers on EC2 instances as suggested in B and C. Amazon S3 offers excellent durability, scalability, and availability for storing the report files. Instead of relying on EFS as the primary storage, S3 presents a better choice due to its cost-effectiveness, stronger data protection, and simpler management compared to EFS in this scenario.

The EC2 instance in an Auto Scaling group with a scheduled scaling policy provides the compute power to run the batch operation. Scheduling ensures the EC2 instance is only running when needed, which optimizes costs. This approach also improves resilience because Auto Scaling can replace unhealthy instances automatically. Modifying the application to pull files from S3 gives it the flexibility to adapt to the cloud environment, aligning with best practices for decoupling application components. Options A, B, and C require managing infrastructure elements like file systems or SFTP servers on EC2, increasing operational burden. The direct S3 integration in option D simplifies the overall architecture and minimizes manual intervention.

Further Reading:

AWS Transfer Family: https://aws.amazon.com/aws-transfer-family/
Amazon S3: https://aws.amazon.com/s3/
Amazon EC2 Auto Scaling: https://aws.amazon.com/autoscaling/

Question: 754 **Exam Heist**

A company has users all around the world accessing its HTTP-based application deployed on Amazon EC2 instances in multiple AWS Regions. The company wants to improve the availability and performance of the application. The company also wants to protect the application against common web exploits that may affect availability, compromise security, or consume excessive resources. Static IP addresses are required.

What should a solutions architect recommend to accomplish this?

A.Put the EC2 instances behind Network Load Balancers (NLBs) in each Region. Deploy AWS WAF on the NLBs. Create an accelerator using AWS Global Accelerator and register the NLBs as endpoints.

B.Put the EC2 instances behind Application Load Balancers (ALBs) in each Region. Deploy AWS WAF on the ALBs. Create an accelerator using AWS Global Accelerator and register the ALBs as endpoints.

C.Put the EC2 instances behind Network Load Balancers (NLBs) in each Region. Deploy AWS WAF on the NLBs. Create an Amazon CloudFront distribution with an origin that uses Amazon Route 53 latency-based routing to route requests to the NLBs.

D.Put the EC2 instances behind Application Load Balancers (ALBs) in each Region. Create an Amazon CloudFront distribution with an origin that uses Amazon Route 53 latency-based routing to route requests to the ALBs. Deploy AWS WAF on the CloudFront distribution.

Answer: D

Explanation:

The correct answer is **D** because it provides a comprehensive solution addressing the requirements for global availability, performance, security, and static IP addresses.

Here's a breakdown of why option D is superior:

Global Availability and Performance: Amazon CloudFront, a content delivery network (CDN), caches content at edge locations worldwide, reducing latency for users accessing the application from different geographical locations. Route 53 latency-based routing directs users to the Application Load Balancer (ALB) in the region with the lowest network latency, ensuring optimal performance. This also contributes to high availability as traffic is automatically routed away from unhealthy regions.
Security: AWS WAF, deployed on the CloudFront distribution, protects the application against common web exploits like SQL injection and cross-site scripting (XSS). Placing WAF at the edge (CloudFront) provides protection closest to the source of malicious traffic.

Static IP Addresses: CloudFront provides static IP addresses. This fulfils the given requirement in the question.

Let's examine why the other options are less suitable:

Option A & B: AWS Global Accelerator provides static IP addresses and improves performance, but it does not inherently provide protection against web exploits. Using it without WAF leaves the application vulnerable. Also, Global Accelerator is suited more to TCP or UDP traffic and isn't as efficient at caching content as CloudFront.
Option C: While this option uses NLBs, WAF at the NLB level is not ideal. WAF is better suited at the CDN level (CloudFront) for protection against web exploits. Additionally, NLBs are designed for high-performance, low-latency traffic and don't offer the content caching benefits of CloudFront.

In summary, deploying ALBs behind CloudFront with WAF ensures optimal performance through caching and latency-based routing, provides static IP addresses, and protects the application against web exploits at the edge, fulfilling all requirements in the most efficient and secure manner.

Here are some helpful links for further research:

Amazon CloudFront: https://aws.amazon.com/cloudfront/
AWS WAF: https://aws.amazon.com/waf/
Amazon Route 53: https://aws.amazon.com/route53/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
AWS Global Accelerator: https://aws.amazon.com/global-accelerator/

Question: 755 **Exam Heist**

A company's data platform uses an Amazon Aurora MySQL database. The database has multiple read replicas and multiple DB instances across different Availability Zones. Users have recently reported errors from the database that indicate that there are too many connections. The company wants to reduce the failover time by 20% when a read replica is promoted to primary writer.

Which solution will meet this requirement?

A.Switch from Aurora to Amazon RDS with Multi-AZ cluster deployment.
B.Use Amazon RDS Proxy in front of the Aurora database.
C.Switch to Amazon DynamoDB with DynamoDB Accelerator (DAX) for read connections.
D.Switch to Amazon Redshift with relocation capability.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer, along with supporting concepts and resources:

The problem presents a scenario with an Aurora MySQL database experiencing "too many connections" errors and a need to reduce failover time when a read replica becomes the primary writer. The goal is to address the connection management issue while also improving failover performance.

Option B, using Amazon RDS Proxy in front of the Aurora database, is the most appropriate solution for several key reasons:

1. **Connection Pooling and Multiplexing:** RDS Proxy sits between your application and the database. It pools database connections and reuses them efficiently. Instead of each application connection holding a direct database connection open, RDS Proxy manages a smaller pool of connections to the

Aurora database. When an application needs to execute a query, RDS Proxy checks out a connection from the pool, executes the query, and then returns the connection to the pool. This reduces the number of active connections to the database and alleviates the "too many connections" errors.

2. **Connection Management During Failover:** During a failover, RDS Proxy automatically reconnects to the new primary instance. It maintains application connections without interruption, minimizing downtime. It accomplishes this by detecting the failover event and rerouting traffic to the new primary. It can reduce the failover time because the application does not need to establish new connections from scratch. This is how the 20% failover reduction is achieved.

3. **Improved Application Performance:** By reducing connection overhead, RDS Proxy can also improve application performance, especially for applications that make frequent short-lived connections to the database.

Option A (Switching to Amazon RDS with Multi-AZ) addresses high availability but does not inherently solve the "too many connections" issue. Multi-AZ provides failover capabilities, but it doesn't manage connection pooling. It's a foundational HA strategy but not the specific fix required here.

Option C (Switching to DynamoDB with DAX) represents a significant architectural change. DynamoDB is a NoSQL database, and Aurora MySQL is relational. A complete migration would be required, and it would not be suitable unless the application is significantly refactored. While DAX caches DynamoDB reads, it addresses latency but isn't related to reducing failover time of a Relational Database, nor is it related to connection management in relational databases.

Option D (Switching to Amazon Redshift) is designed for data warehousing and analytics, not transactional workloads like the one described in the problem. Relocation capability does not address the connection management issues nor improve the read replica promotion speed significantly enough to meet the 20% reduction requirement. The switch would also imply significant application rewriting.

In summary, RDS Proxy directly addresses the connection management bottleneck, reducing the number of connections to the database and providing connection persistence during failover. This directly leads to a faster failover time because the application connections are maintained automatically.

Authoritative Links:

Amazon RDS Proxy: https://aws.amazon.com/rds/proxy/
AWS Documentation - Using Amazon RDS Proxy for High Availability: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/rds-proxy.html (Specifically look at sections related to Failover)

Question: 756 **Exam Heist**

A company stores text files in Amazon S3. The text files include customer chat messages, date and time information, and customer personally identifiable information (PII).

The company needs a solution to provide samples of the conversations to an external service provider for quality control. The external service provider needs to randomly pick sample conversations up to the most recent conversation. The company must not share the customer PII with the external service provider. The solution must scale when the number of customer conversations increases.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create an Object Lambda Access Point. Create an AWS Lambda function that redacts the PII when the function reads the file. Instruct the external service provider to access the Object Lambda Access Point.

B.Create a batch process on an Amazon EC2 instance that regularly reads all new files, redacts the PII from the files, and writes the redacted files to a different S3 bucket. Instruct the external service provider to access the bucket that does not contain the PII.

B. Create a web application on an Amazon EC2 instance that presents a list of the files, redacts the PII from the files, and allows the external service provider to download new versions of the files that have the PII redacted.

D.Create an Amazon DynamoDB table. Create an AWS Lambda function that reads only the data in the files that does not contain PII. Configure the Lambda function to store the non-PII data in the DynamoDB table when a new file is written to Amazon S3. Grant the external service provider access to the DynamoDB table.

Answer: A

Explanation:

Option A, using S3 Object Lambda, is the most efficient and scalable solution for redacting PII from S3 objects on-the-fly for an external service provider with minimal operational overhead. Object Lambda lets you add custom code to S3 to process data as it's being retrieved, transforming the content before returning it to the requesting application. This eliminates the need for separate data processing pipelines or copies of the data.

Here's why Option A is superior to the other options:

Least Operational Overhead: Object Lambda automates the redaction process upon data access. No need to manage EC2 instances, batch jobs, or DynamoDB.

Scalability: Object Lambda scales automatically with S3 requests, handling increases in data volume and access frequency seamlessly.

Real-time Redaction: PII is redacted at the time of retrieval, ensuring the external service provider never sees the raw, unredacted data.

Security: Redaction logic is contained within a Lambda function, minimizing the risk of accidental PII exposure. Access is controlled via the Object Lambda Access Point.

Cost-Effectiveness: Pay only for the Lambda execution time and the data processed by S3. Eliminates costs associated with maintaining infrastructure.

In contrast, Option B involves a batch process on EC2, which requires infrastructure management, scheduling, and can introduce latency between data arrival and redaction. It also requires duplicate storage. Option C requires you to maintain a custom web application, involving significant operational complexity. Option D, which stores data into DynamoDB, is also an inferior choice, because it duplicates data into a DynamoDB database unnecessarily, adding complexity and cost. S3 Object Lambda is the leanest, most scalable, most cost effective, and most secure solution.

Authoritative Links:

AWS S3 Object Lambda: https://aws.amazon.com/s3/features/object-lambda/
Transforming objects with S3 Object Lambda: https://aws.amazon.com/blogs/aws/introducing-amazon-s3-object-lambda-transform-objects-as-they-are-being-retrieved/

Question: 757 **Exam Heist**

A company is running a legacy system on an Amazon EC2 instance. The application code cannot be modified, and the system cannot run on more than one instance. A solutions architect must design a resilient solution that can improve the recovery time for the system.

What should the solutions architect recommend to meet these requirements?

A.Enable termination protection for the EC2 instance.

B.Configure the EC2 instance for Multi-AZ deployment.

C.Create an Amazon CloudWatch alarm to recover the EC2 instance in case of failure.

D.Launch the EC2 instance with two Amazon Elastic Block Store (Amazon EBS) volumes that use RAID configurations for storage redundancy.

Answer: C

Explanation:

The correct answer is **C. Create an Amazon CloudWatch alarm to recover the EC2 instance in case of failure.**

Here's why this is the best solution and why the other options are not suitable:

Why C is correct: The goal is to improve recovery time for a single, non-modifiable legacy system running on EC2. A CloudWatch alarm with a "Recover" action provides an automated mechanism to reboot the EC2 instance if it becomes impaired due to underlying hardware issues. This is faster than manual intervention and requires no application changes, aligning with the requirements. The "Recover" action is designed for scenarios where the instance itself is unhealthy, rather than an issue with the application running on it. CloudWatch continuously monitors the instance's health and triggers the recovery action based on pre-defined thresholds (e.g., system status checks failing). This automated recovery minimizes downtime, improving resilience without application modification.

Why A is incorrect: Enabling termination protection prevents accidental termination of the EC2 instance, but it doesn't help in recovering from underlying hardware or system issues that cause the instance to become impaired. Termination protection ensures the instance isn't deleted, but doesn't ensure its availability.

Why B is incorrect: Multi-AZ deployment is typically used for applications that can be scaled across multiple instances. The requirement states the application cannot run on more than one instance, making Multi-AZ deployment unsuitable. Multi-AZ relies on application-level clustering and data replication, which are not possible with the specified constraints.

Why D is incorrect: RAID configurations on EBS volumes primarily provide data redundancy within the EC2 instance. While data loss is mitigated, RAID does not automatically recover the instance if it becomes impaired. The EC2 instance would still need to be replaced or restarted, and RAID is not a substitute for instance-level recovery. Moreover, RAID configurations can add complexity and might not be suitable for the legacy application without thorough testing. The focus should be on recovering the instance, not just the data.

Authoritative Links:

Amazon CloudWatch Recover an EC2 Instance:
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-recover.html
Amazon EC2 Termination Protection: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/terminating-instances.html#Using_ChangingDisableAPITermination

Question: 758 **Exam Heist**

A company wants to deploy its containerized application workloads to a VPC across three Availability Zones. The company needs a solution that is highly available across Availability Zones. The solution must require minimal changes to the application.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use Amazon Elastic Container Service (Amazon ECS). Configure Amazon ECS Service Auto Scaling to use target tracking scaling. Set the minimum capacity to 3. Set the task placement strategy type to spread with an Availability Zone attribute.

B.Use Amazon Elastic Kubernetes Service (Amazon EKS) self-managed nodes. Configure Application Auto Scaling to use target tracking scaling. Set the minimum capacity to 3.

C.Use Amazon EC2 Reserved Instances. Launch three EC2 instances in a spread placement group. Configure an

Auto Scaling group to use target tracking scaling. Set the minimum capacity to 3.

D.Use an AWS Lambda function. Configure the Lambda function to connect to a VPC. Configure Application Auto Scaling to use Lambda as a scalable target. Set the minimum capacity to 3.

Answer: A

Explanation:

The best solution for deploying containerized applications across three Availability Zones with high availability and minimal operational overhead is using Amazon ECS with Service Auto Scaling.

Option A is correct because ECS inherently simplifies container management and orchestration. Service Auto Scaling with target tracking scaling automatically adjusts the number of ECS tasks (containers) based on a metric like CPU utilization or memory consumption. Setting the minimum capacity to 3 ensures that at least one task runs in each of the three AZs, providing high availability. The 'spread' task placement strategy with an Availability Zone attribute ensures ECS distributes tasks evenly across AZs. This approach requires minimal application changes, as the application remains containerized and is deployed via ECS's standard mechanisms.

Option B, using EKS with self-managed nodes, introduces significantly higher operational overhead. Managing the Kubernetes cluster and nodes (EC2 instances) requires substantial effort, including patching, scaling, and security configurations. While EKS provides container orchestration, self-managing nodes undermines the goal of minimal operational overhead.

Option C, utilizing EC2 Reserved Instances, lacks the built-in container orchestration capabilities that are crucial for managing containerized applications effectively. Configuring an Auto Scaling group to manage EC2 instances only addresses the scaling of VMs and does not inherently handle container deployment, health checks, or networking as effectively as container orchestration platforms. This approach requires manual configuration of container deployment and management within the EC2 instances, which is more complex. A spread placement group offers distribution across AZs, but it doesn't offer the container focus desired.

Option D, using AWS Lambda, is unsuitable for running typical containerized application workloads. Lambda functions are designed for event-driven, short-lived computations. While Lambda can be configured within a VPC, it's not meant to handle persistent, complex container applications. Additionally, the container limitations of Lambda make it unrealistic for this scenario.

Therefore, ECS with Service Auto Scaling and spread placement strategy delivers the desired high availability across AZs with the least operational burden and requires minimal modifications to the containerized application.

Supporting Links:

Amazon ECS: https://aws.amazon.com/ecs/
Amazon ECS Service Auto Scaling:
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/service-auto-scaling.html
Amazon EKS: https://aws.amazon.com/eks/

Question: 759 **Exam Heist**

A media company stores movies in Amazon S3. Each movie is stored in a single video file that ranges from 1 GB to 10 GB in size.

The company must be able to provide the streaming content of a movie within 5 minutes of a user purchase. There is higher demand for movies that are less than 20 years old than for movies that are more than 20 years old. The company wants to minimize hosting service costs based on demand.

Which solution will meet these requirements?

A.Store all media content in Amazon S3. Use S3 Lifecycle policies to move media data into the Infrequent Access tier when the demand for a movie decreases.

B.Store newer movie video files in S3 Standard. Store older movie video files in S3 Standard-infrequent Access (S3 Standard-IA). When a user orders an older movie, retrieve the video file by using standard retrieval.

C.Store newer movie video files in S3 Intelligent-Tiering. Store older movie video files in S3 Glacier Flexible Retrieval. When a user orders an older movie, retrieve the video file by using expedited retrieval.

D.Store newer movie video files in S3 Standard. Store older movie video files in S3 Glacier Flexible Retrieval. When a user orders an older movie, retrieve the video file by using bulk retrieval.

Answer: C

Explanation:

Here's a detailed justification for why option C is the most suitable solution, along with supporting concepts and links:

The primary requirements are fast content delivery (within 5 minutes) after purchase and cost minimization based on demand, considering varying demand for newer vs. older movies.

Option C, storing newer movies in S3 Intelligent-Tiering and older movies in S3 Glacier Flexible Retrieval, balances performance and cost effectively.

S3 Intelligent-Tiering automatically moves data between frequent, infrequent, and archive access tiers based on changing access patterns. This is ideal for newer movies with potentially fluctuating demand, ensuring low latency while optimizing storage costs. https://aws.amazon.com/s3/storage-classes/intelligent-tiering/

S3 Glacier Flexible Retrieval (formerly Glacier) is designed for archiving data that is infrequently accessed. The key is using "expedited retrieval" which guarantees access within 1-5 minutes. This meets the 5-minute requirement even for older, rarely accessed movies. Although expedited retrieval has a higher cost, it's acceptable because these are infrequent requests. https://aws.amazon.com/s3/storage-classes/glacier/

Option A is less efficient because S3 Lifecycle policies react more slowly than Intelligent-Tiering to shifting access patterns. Using only S3 Lifecycle policies may also require custom scripting to manage tiers, adding complexity. Moreover, S3 Infrequent Access (S3 IA) doesn’t necessarily deliver within the guaranteed 5 minutes.

Option B, storing older movies in S3 Standard-IA, is also suboptimal. S3 Standard-IA provides a lower storage cost than S3 Standard but is still more expensive than Glacier Flexible Retrieval. It doesn't optimize cost as effectively as Glacier Flexible Retrieval for infrequently accessed data while lacking a guaranteed retrieval time.

Option D uses S3 Glacier Flexible Retrieval, but suggests "bulk retrieval." Bulk retrieval can take 5-12 hours, violating the 5-minute requirement. Therefore, despite the lower cost, it is inappropriate for time-sensitive post-purchase streaming.

Therefore, the combination of S3 Intelligent-Tiering for recent movies (automatically handling variable access patterns with low latency) and S3 Glacier Flexible Retrieval with expedited retrieval for older movies (meeting the retrieval time requirement at a reasonable price for infrequent access) makes option C the best solution. It is a robust and cost-effective solution that addresses both performance and cost optimization needs.

Question: 760 **Exam Heist**

A solutions architect needs to design the architecture for an application that a vendor provides as a Docker

container image. The container needs 50 GB of storage available for temporary files. The infrastructure must be serverless.

Which solution meets these requirements with the LEAST operational overhead?

A.Create an AWS Lambda function that uses the Docker container image with an Amazon S3 mounted volume that has more than 50 GB of space.

B.Create an AWS Lambda function that uses the Docker container image with an Amazon Elastic Block Store (Amazon EBS) volume that has more than 50 GB of space.

C.Create an Amazon Elastic Container Service (Amazon ECS) cluster that uses the AWS Fargate launch type. Create a task definition for the container image with an Amazon Elastic File System (Amazon EFS) volume. Create a service with that task definition.

D.Create an Amazon Elastic Container Service (Amazon ECS) cluster that uses the Amazon EC2 launch type with an Amazon Elastic Block Store (Amazon EBS) volume that has more than 50 GB of space. Create a task definition for the container image. Create a service with that task definition.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best answer, along with supporting information:

The question prioritizes a serverless architecture with minimal operational overhead for a Docker container requiring 50 GB of temporary storage.

Option C (Amazon ECS with Fargate and EFS): This solution leverages the strengths of both Amazon ECS with Fargate and Amazon EFS. Fargate provides a serverless compute environment, eliminating the need to manage EC2 instances, thus reducing operational overhead. Amazon EFS offers a fully managed, scalable, and elastic file system. It integrates seamlessly with ECS and can provide the necessary 50 GB of storage for temporary files. Mounting an EFS volume to an ECS task using Fargate is straightforward and requires minimal configuration.

Option A (Lambda with S3 mounted volume): While Lambda functions can run Docker images, Lambda does not support directly mounting S3 buckets as volumes within the container. S3 is object storage and not designed to be used as a file system for temporary file operations, violating the original requirement. This option also involves significant complexity in managing data transfer between Lambda and S3 for temporary file usage.

Option B (Lambda with EBS volume): Lambda functions do not support direct attachment of EBS volumes. EBS volumes are designed to be attached to EC2 instances and cannot be used with the serverless Lambda environment. Thus it violates the "serverless" and "LEAST operational overhead" requirements.

Option D (ECS with EC2 and EBS): While this option provides the required storage, it does not meet the requirement for a serverless architecture. Using EC2 instances introduces significant operational overhead, including patching, scaling, and monitoring the EC2 instances themselves. This directly contradicts the principle of minimizing operational effort.

Therefore, option C is the best choice as it combines serverless compute with managed file storage, minimizing operational overhead while fulfilling the storage requirements of the Docker container.

Supporting Documentation:

Amazon ECS with Fargate: https://aws.amazon.com/fargate/
Amazon EFS: https://aws.amazon.com/efs/
Using Amazon EFS with Amazon ECS: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/efs-volumes.html
Lambda Storage Options: https://docs.aws.amazon.com/lambda/latest/dg/configuration-layers.html

Question: 761 **Exam Heist**

A company needs to use its on-premises LDAP directory service to authenticate its users to the AWS Management Console. The directory service is not compatible with Security Assertion Markup Language (SAML).

Which solution meets these requirements?

A.Enable AWS IAM Identity Center (AWS Single Sign-On) between AWS and the on-premises LDAP.

B.Create an IAM policy that uses AWS credentials, and integrate the policy into LDAP.

C.Set up a process that rotates the IAM credentials whenever LDAP credentials are updated.

D.Develop an on-premises custom identity broker application or process that uses AWS Security Token Service (AWS STS) to get short-lived credentials.

Answer: D

Explanation:

Here's a detailed justification for why option D is the best solution for authenticating users from an on-premises LDAP directory to the AWS Management Console when SAML isn't an option:

The core challenge is to bridge the on-premises LDAP directory (incompatible with SAML) with AWS authentication. Options A, B, and C have significant drawbacks:

Option A (IAM Identity Center): AWS IAM Identity Center primarily relies on SAML for federation. Since the LDAP is not SAML compatible, this option is not viable.

Option B (IAM Policy Integration): Directly embedding AWS credentials into LDAP or linking IAM policies to LDAP authentication is a major security risk. AWS credentials should never be stored or managed outside of AWS Identity and Access Management (IAM) to prevent credential leakage and unauthorized access.

Option C (Rotating IAM Credentials): Rotating IAM credentials based on LDAP updates is complex, error-prone, and still requires storing AWS credentials somewhere outside of AWS, which is a security vulnerability.

Option D (Custom Identity Broker): A custom identity broker is the most suitable approach for LDAP integration when SAML is unavailable.

Here's why it works:

1. **Authentication Proxy:** The custom application acts as an intermediary between the LDAP directory and AWS. Users authenticate with their existing LDAP credentials.
2. **STS Integration:** After successful LDAP authentication, the broker application uses the AWS Security Token Service (STS) to request temporary, short-lived AWS credentials. The custom application will assume an IAM role configured with permissions to use STS.
3. **Secure Credential Management:** The application securely communicates with AWS STS using an AWS IAM user with only permissions to access the STS service, retrieves the temporary credentials.
4. **Console Access:** These temporary credentials allow the user to access the AWS Management Console or AWS resources based on the permissions defined in the IAM role that the broker assumes.
5. **Security Best Practices:** This approach keeps AWS credentials secure within AWS and avoids exposing them to the on-premises environment. The temporary credentials expire automatically, further limiting the potential impact of any compromise.
6. **Flexibility:** Provides fine-grained control over access based on the user's LDAP group memberships.

In summary, the identity broker model using STS provides a secure and manageable solution for integrating

non-SAML LDAP directories with AWS, addressing the company's authentication needs without compromising security.

Authoritative Links:

AWS Security Token Service (STS): https://aws.amazon.com/iam/features/security-token-service/
IAM Roles: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html

Question: 762 **Exam Heist**

A company stores multiple Amazon Machine Images (AMIs) in an AWS account to launch its Amazon EC2 instances. The AMIs contain critical data and configurations that are necessary for the company's operations. The company wants to implement a solution that will recover accidentally deleted AMIs quickly and efficiently.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create Amazon Elastic Block Store (Amazon EBS) snapshots of the AMIs. Store the snapshots in a separate AWS account.

B.Copy all AMIs to another AWS account periodically.

C.Create a retention rule in Recycle Bin.

D.Upload the AMIs to an Amazon S3 bucket that has Cross-Region Replication.

Answer: C

Explanation:

The optimal solution is to utilize Recycle Bin for AMI retention. Here's why:

Option C, using Recycle Bin, offers the lowest operational overhead because it directly addresses accidental deletion with a built-in AWS feature. Recycle Bin allows you to create retention rules that specify a period during which deleted AMIs (and other resources) are retained before permanent deletion. This provides a buffer for recovery without requiring manual intervention or scripting.

Option A, EBS snapshots, while useful for backup, adds complexity. Managing snapshots across accounts requires cross-account IAM roles and potentially scripts for automating the snapshot creation and replication. Snapshots alone don't inherently prevent deletion; they just provide a point-in-time copy.

Option B, copying AMIs, creates operational overhead in terms of management, storage costs, and transfer bandwidth. Periodic copying requires automation scripts and introduces complexity in synchronization and version control.

Option D, uploading AMIs to S3, is not the standard way to back up AMIs. AMI backup methods usually utilize snapshots or cross-account AMI copy, which are designed specifically for the purpose. Additionally, restoring AMIs from S3 wouldn't be as direct or manageable as the original AMIs.

Recycle Bin is designed specifically to prevent accidental deletion and enable easy recovery within a defined retention period, making it the most efficient solution.

For more information on Recycle Bin, refer to the AWS documentation:
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/recycle-bin.html

Question: 763 **Exam Heist**

A company has 150 TB of archived image data stored on-premises that needs to be moved to the AWS Cloud within

the next month. The company's current network connection allows up to 100 Mbps uploads for this purpose during the night only.

What is the MOST cost-effective mechanism to move this data and meet the migration deadline?

A.Use AWS Snowmobile to ship the data to AWS.
B.Order multiple AWS Snowball devices to ship the data to AWS.
C.Enable Amazon S3 Transfer Acceleration and securely upload the data.
D.Create an Amazon S3 VPC endpoint and establish a VPN to upload the data.

Answer: B

Explanation:

Here's a detailed justification for why option B, "Order multiple AWS Snowball devices to ship the data to AWS," is the most cost-effective solution for migrating 150 TB of data within a month, given a limited 100 Mbps upload speed available only at night:

Let's analyze the limitations first. A 100 Mbps connection, even if fully utilized, is extremely slow for transferring 150 TB. To calculate the transfer time:

150 TB = 150 1024 1024 1024 8 bits = 1,288,490,188,800 bits
Upload speed = 100 Mbps = 100,000,000 bits per second
Total time = 1,288,490,188,800 bits / 100,000,000 bits/second = 12,884,901.888 seconds
Total time = 12,884,901.888 seconds / (60 60 24) days = 148.8 days

Since the data can only be transferred at night, the actual transfer time will be substantially longer than 148.8 days, making it infeasible within the one-month deadline.

Now, let's evaluate the options:

A. AWS Snowmobile: Snowmobile is for exabyte-scale data transfer. It is overkill and significantly more expensive than Snowball for 150 TB.
B. AWS Snowball: Snowball is specifically designed for transferring large amounts of data in and out of AWS when network bandwidth is limited. Ordering multiple Snowball devices allows for parallel data transfer, significantly reducing the overall migration time. The cost of Snowball is based on the device usage and the job performed, and is typically far more cost-effective than upgrading network infrastructure or dealing with prolonged transfer times.
C. Amazon S3 Transfer Acceleration: Transfer Acceleration utilizes CloudFront edge locations to improve transfer speeds to S3. However, the improvement is highly dependent on the distance between the client and the S3 bucket, and the network conditions. Given the already slow 100 Mbps connection, Transfer Acceleration is unlikely to provide a sufficient speed increase to meet the deadline and comes with additional costs. Furthermore, the bottleneck is the on-premises network speed, which Transfer Acceleration cannot overcome.
D. Amazon S3 VPC endpoint and VPN: Creating an S3 VPC endpoint provides secure access to S3 from within a VPC. Establishing a VPN connection provides a secure tunnel between the on-premises network and the VPC. However, this solution still relies on the limited 100 Mbps network connection, making it infeasible for transferring 150 TB within the one-month deadline. This setup also adds complexity and cost without addressing the core problem: slow network speed.

Snowball offers the most practical combination of speed and cost-effectiveness. By using multiple devices concurrently, the company can significantly accelerate the data transfer without incurring the higher costs associated with Snowmobile or the limitations of the existing network connection.

Therefore, the answer is B.

Authoritative Links:

AWS Snowball: https://aws.amazon.com/snowball/
AWS Snowmobile: https://aws.amazon.com/snowmobile/
Amazon S3 Transfer Acceleration: https://aws.amazon.com/s3/transfer-acceleration/

Question: 764 **Exam Heist**

A company wants to migrate its three-tier application from on premises to AWS. The web tier and the application tier are running on third-party virtual machines (VMs). The database tier is running on MySQL.

The company needs to migrate the application by making the fewest possible changes to the architecture. The company also needs a database solution that can restore data to a specific point in time.

Which solution will meet these requirements with the LEAST operational overhead?

A.Migrate the web tier and the application tier to Amazon EC2 instances in private subnets. Migrate the database tier to Amazon RDS for MySQL in private subnets.

B.Migrate the web tier to Amazon EC2 instances in public subnets. Migrate the application tier to EC2 instances in private subnets. Migrate the database tier to Amazon Aurora MySQL in private subnets.

C.Migrate the web tier to Amazon EC2 instances in public subnets. Migrate the application tier to EC2 instances in private subnets. Migrate the database tier to Amazon RDS for MySQL in private subnets.

D.Migrate the web tier and the application tier to Amazon EC2 instances in public subnets. Migrate the database tier to Amazon Aurora MySQL in public subnets.

Answer: A

Explanation:

The best solution is A because it minimizes changes and operational overhead while fulfilling all requirements.

Minimizing Changes: Migrating the web and application tiers to EC2 instances is a straightforward "lift and shift" approach, requiring minimal code modification. Keeping both tiers in private subnets enhances security. Similarly, migrating the MySQL database to Amazon RDS for MySQL requires minimal changes compared to other database options.

Database Point-in-Time Recovery: Both Amazon RDS for MySQL and Amazon Aurora MySQL offer point-in-time recovery.

Least Operational Overhead: Amazon RDS manages database administration tasks like patching, backups, and recovery, significantly reducing operational overhead compared to managing a MySQL instance on EC2. Using the same MySQL engine reduces the risk of compatibility issues during the database migration. Choosing RDS over Aurora also minimizes changes to database configuration.

Security: Placing the web and application tiers, as well as the database, in private subnets adds a layer of security, preventing direct internet access. This is essential for protecting the application and data.

Let's analyze why the other options are less suitable:

Option B & D: Migrating to Aurora MySQL introduces a change in the database engine, potentially requiring code modifications and increased testing, thus more changes and overhead. Option D is also insecure by placing the database in a public subnet.

Option C: Migrating to RDS for MySQL is a good move. However, the answer states the web-tier needs to be in a public subnet which might not be needed if the application is using a load balancer.

Therefore, option A provides the most efficient and secure migration path with the least operational overhead and minimal changes to the existing architecture.

Supporting Links:

Amazon EC2: https://aws.amazon.com/ec2/
Amazon RDS: https://aws.amazon.com/rds/
Amazon Aurora: https://aws.amazon.com/rds/aurora/
AWS VPC: https://aws.amazon.com/vpc/

Question: 765 **Exam Heist**

A development team is collaborating with another company to create an integrated product. The other company needs to access an Amazon Simple Queue Service (Amazon SQS) queue that is contained in the development team's account. The other company wants to poll the queue without giving up its own account permissions to do so.

How should a solutions architect provide access to the SQS queue?

A.Create an instance profile that provides the other company access to the SQS queue.
B.Create an IAM policy that provides the other company access to the SQS queue.
C.Create an SQS access policy that provides the other company access to the SQS queue.
D.Create an Amazon Simple Notification Service (Amazon SNS) access policy that provides the other company access to the SQS queue.

Answer: C

Explanation:

The correct answer is **C. Create an SQS access policy that provides the other company access to the SQS queue.**

Here's why:

Resource-Based Policies: SQS queues support resource-based policies (also known as access policies). These policies directly attach to the SQS queue itself and specify who (which AWS accounts or IAM users) can access the queue and what actions they can perform.
Cross-Account Access: SQS access policies enable cross-account access. This means you can grant permissions to principals (users, roles, or accounts) in another AWS account to access your SQS queue. The other company retains control of its own account and identities.
Granular Permissions: SQS access policies allow you to specify precisely which SQS actions the other company can perform on the queue (e.g., sqs:ReceiveMessage, sqs:SendMessage, sqs:DeleteMessage).

Let's examine why the other options are incorrect:

A. Create an instance profile that provides the other company access to the SQS queue. Instance profiles are used to grant permissions to EC2 instances. The other company does not need an EC2 instance in the dev team account.
B. Create an IAM policy that provides the other company access to the SQS queue. While you could create an IAM policy, the other company would still need to assume a role in the development team's account, negating their request to not use their account.
D. Create an Amazon Simple Notification Service (Amazon SNS) access policy that provides the other company access to the SQS queue. SNS is a different service for pub/sub messaging. An SNS policy would not grant access to an SQS queue.

By using an SQS access policy, the development team maintains ownership and control of the SQS queue, and the other company can access it using its own AWS account credentials and identities, without needing to share or assume roles within the development team's account.

For more information, refer to these AWS documentation links:

Amazon SQS Access Control:
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-security-overview.html
Granting Cross-Account Permissions:
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-grant-cross-account-permissions-to-role.html

Question: 766 **Exam Heist**

A company's developers want a secure way to gain SSH access on the company's Amazon EC2 instances that run the latest version of Amazon Linux. The developers work remotely and in the corporate office.

The company wants to use AWS services as a part of the solution. The EC2 instances are hosted in a VPC private subnet and access the internet through a NAT gateway that is deployed in a public subnet.

What should a solutions architect do to meet these requirements MOST cost-effectively?

A.Create a bastion host in the same subnet as the EC2 instances. Grant the ec2:CreateVpnConnection IAM permission to the developers. Install EC2 Instance Connect so that the developers can connect to the EC2 instances.

B.Create an AWS Site-to-Site VPN connection between the corporate network and the VPC. Instruct the developers to use the Site-to-Site VPN connection to access the EC2 instances when the developers are on the corporate network. Instruct the developers to set up another VPN connection for access when they work remotely.

C.Create a bastion host in the public subnet of the VPConfigure the security groups and SSH keys of the bastion host to only allow connections and SSH authentication from the developers' corporate and remote networks. Instruct the developers to connect through the bastion host by using SSH to reach the EC2 instances.

D.Attach the AmazonSSMManagedInstanceCore IAM policy to an IAM role that is associated with the EC2 instances. Instruct the developers to use AWS Systems Manager Session Manager to access the EC2 instances.

Answer: D

Explanation:

The most cost-effective and secure solution is to use AWS Systems Manager (SSM) Session Manager. Here's why:

Security: Session Manager provides secure and auditable instance management without needing to open inbound SSH ports or manage SSH keys, reducing the attack surface. It uses IAM policies to control access to instances, ensuring proper authorization.
Cost-Effectiveness: Session Manager is included in the AWS Free Tier and has minimal costs beyond that, primarily related to CloudTrail logging (if enabled), making it more cost-effective than setting up and maintaining VPN connections or bastion hosts.
Ease of Use: Developers can connect to EC2 instances directly from the AWS Management Console, AWS CLI, or AWS SDKs, without needing to manage SSH keys or configure SSH clients.
Auditing: Session Manager integrates with CloudTrail to log session activity for auditing and compliance purposes.
No Public IPs: Since the EC2 instances are in a private subnet and can access the internet through a NAT Gateway, Session Manager can function effectively, as the instances need outbound access to SSM endpoints, not inbound access.
IAM Role: Attaching the AmazonSSMManagedInstanceCore IAM policy to the EC2 instances' IAM role grants SSM the necessary permissions to manage the instances.

Why other options are less suitable:

A: Bastion Host in Private Subnet: Placing a bastion host in the same private subnet defeats the purpose of a bastion host, as it should be in a public subnet. Additionally, granting ec2:CreateVpnConnection permission is unnecessary and overly permissive.
B: Site-to-Site VPN: Setting up a Site-to-Site VPN for access from the corporate network is a viable option but less cost-effective than SSM, especially if the company only needs SSH access. Furthermore, requiring developers to set up another VPN for remote access adds complexity.
C: Bastion Host in Public Subnet: While a valid approach, maintaining a bastion host involves operational overhead (patching, scaling, monitoring) and costs associated with the instance and its network traffic. SSM offers a managed and more cost-effective alternative.

Supporting Links:

AWS Systems Manager Session Manager: https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html
AmazonSSMManagedInstanceCore IAM Policy: https://docs.aws.amazon.com/systems-manager/latest/userguide/security-iam-id-based-policy-examples.html

Question: 767 **Exam Heist**

A pharmaceutical company is developing a new drug. The volume of data that the company generates has grown exponentially over the past few months. The company's researchers regularly require a subset of the entire dataset to be immediately available with minimal lag. However, the entire dataset does not need to be accessed on a daily basis. All the data currently resides in on-premises storage arrays, and the company wants to reduce ongoing capital expenses.

Which storage solution should a solutions architect recommend to meet these requirements?

A.Run AWS DataSync as a scheduled cron job to migrate the data to an Amazon S3 bucket on an ongoing basis.

B.Deploy an AWS Storage Gateway file gateway with an Amazon S3 bucket as the target storage. Migrate the data to the Storage Gateway appliance.

C.Deploy an AWS Storage Gateway volume gateway with cached volumes with an Amazon S3 bucket as the target storage. Migrate the data to the Storage Gateway appliance.

D.Configure an AWS Site-to-Site VPN connection from the on-premises environment to AWS. Migrate data to an Amazon Elastic File System (Amazon EFS) file system.

Answer: C

Explanation:

The best storage solution is option C, deploying an AWS Storage Gateway volume gateway with cached volumes backed by Amazon S3. Here's why:

Reduced Capital Expenses: Migrating data to S3 significantly reduces the need for expensive on-premises storage arrays.
Immediate Availability of Subset: The cached volumes feature ensures that frequently accessed subsets of data are stored locally on the Storage Gateway appliance. This provides low-latency access for the researchers.
Infrequent Access of Full Dataset: The entire dataset resides in S3, providing cost-effective storage for data that is not accessed daily.
Volume Gateway and Cached Volumes: Volume Gateway provides block-based access, which can be more suitable for certain types of data or applications. The cached volumes mode keeps frequently accessed data on-premises, while storing the full dataset in S3. This balances cost and performance.

Why other options are less ideal:

Option A (AWS DataSync to S3): While DataSync migrates data to S3, it doesn't address the immediate availability requirement. Researchers would still have to download data from S3, which introduces lag.
Option B (Storage Gateway file gateway): File Gateway stores the entire dataset in S3 and caches recently used files locally. It might not be optimized for block-level access if the researchers are working with applications requiring it.
Option D (Site-to-Site VPN and EFS): EFS is suitable for shared file storage but generally more expensive than S3 for large, infrequently accessed datasets. Maintaining a VPN connection and transferring large amounts of data over the VPN may also be less performant and more complex.

AWS Storage Gateway allows you to seamlessly integrate on-premises application environments with AWS cloud storage. The Cached Volumes mode is ideal for organizations that want to reduce on-premises storage footprint while maintaining performance for frequently accessed data.

Authoritative Links:

AWS Storage Gateway: https://aws.amazon.com/storagegateway/
AWS Storage Gateway Cached Volumes: https://docs.aws.amazon.com/storagegateway/latest/userguide/HowStorageGatewayWorks.html
Amazon S3: https://aws.amazon.com/s3/

Question: 768 **Exam Heist**

A company has a business-critical application that runs on Amazon EC2 instances. The application stores data in an Amazon DynamoDB table. The company must be able to revert the table to any point within the last 24 hours.

Which solution meets these requirements with the LEAST operational overhead?

A.Configure point-in-time recovery for the table.

B.Use AWS Backup for the table.

C.Use an AWS Lambda function to make an on-demand backup of the table every hour.

D.Turn on streams on the table to capture a log of all changes to the table in the last 24 hours. Store a copy of the stream in an Amazon S3 bucket.

Answer: A

Explanation:

The correct answer is A: Configure point-in-time recovery (PITR) for the table. Here's why:

Option A directly addresses the requirement of reverting the DynamoDB table to any point within the last 24 hours with minimal operational overhead. DynamoDB PITR provides automated, continuous backups that allow you to restore your table to any point in time during the specified recovery window. This is the simplest and most efficient way to meet the requirement. You enable it once, and it handles the rest.

Option B, using AWS Backup, is a viable solution but has a slightly higher operational overhead than PITR. While AWS Backup simplifies managing backups across multiple AWS services, it typically involves configuring backup plans with specific schedules. Although effective, this requires more initial configuration compared to simply enabling PITR on the table. While AWS Backup is excellent for long-term retention, the requirement specifically asks for restoration within 24 hours.

Option C, creating hourly backups with a Lambda function, is not recommended due to its significant operational overhead and potential for inconsistencies. Developing, deploying, and maintaining a Lambda function to perform backups introduces complexity. Furthermore, relying on a Lambda function for regular

backups can be less reliable than DynamoDB's built-in PITR feature, as the Lambda function could fail, leading to missed backups.

Option D, using DynamoDB Streams and storing them in S3, is not designed for full table restoration and would be complex to implement. Streams capture individual changes to the table, not the entire table state. Reconstructing the table from the stream data would be computationally expensive and time-consuming, potentially leading to inconsistencies and significantly more operational effort. Also, the question is to revert the table within the last 24 hours, meaning the stream would have to be processed every time a restore is needed, which could be problematic.

Therefore, enabling point-in-time recovery (PITR) for the DynamoDB table is the best solution as it offers a managed, automated, and efficient way to restore the table to any point within the last 24 hours with the least operational overhead.

Here are links for more research:

DynamoDB Point-in-Time Recovery:
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/PointInTimeRecovery.html
AWS Backup: https://aws.amazon.com/backup/
DynamoDB Streams: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html

Question: 769 **Exam Heist**

A company hosts an application used to upload files to an Amazon S3 bucket. Once uploaded, the files are processed to extract metadata, which takes less than 5 seconds. The volume and frequency of the uploads varies from a few files each hour to hundreds of concurrent uploads. The company has asked a solutions architect to design a cost-effective architecture that will meet these requirements.

What should the solutions architect recommend?

A.Configure AWS CloudTrail trails to log S3 API calls. Use AWS AppSync to process the files.

B.Configure an object-created event notification within the S3 bucket to invoke an AWS Lambda function to process the files.

C.Configure Amazon Kinesis Data Streams to process and send data to Amazon S3. Invoke an AWS Lambda function to process the files.

D.Configure an Amazon Simple Notification Service (Amazon SNS) topic to process the files uploaded to Amazon S3. Invoke an AWS Lambda function to process the files.

Answer: B

Explanation:

The correct solution is B: "Configure an object-created event notification within the S3 bucket to invoke an AWS Lambda function to process the files." This approach directly addresses the requirement of processing files after they are uploaded to S3 in a cost-effective manner.

Here's why this solution is optimal:

Event-driven architecture: S3 event notifications allow for real-time responses to object creation, ensuring immediate processing.
Lambda for processing: AWS Lambda is a serverless compute service, ideal for short-duration, event-triggered tasks. The metadata extraction process, taking less than 5 seconds, perfectly fits Lambda's use case. Lambda's pricing model (pay-per-execution) ensures cost-effectiveness, especially with variable upload frequency.
Scalability: Both S3 event notifications and Lambda scale automatically to handle varying upload volumes, from a few files per hour to hundreds of concurrent uploads.

Why other options are less suitable:

A: CloudTrail and AppSync: CloudTrail is for auditing API calls, not for triggering file processing. AWS AppSync is for building GraphQL APIs, which is unnecessary overhead for simple metadata extraction.
C: Kinesis Data Streams and Lambda: Kinesis Data Streams are designed for real-time streaming data, not for processing individual files uploaded to S3. Using Kinesis would be an overkill, add unnecessary complexity, and increase costs.
D: SNS and Lambda: While SNS can be used to trigger Lambda functions, it adds an unnecessary intermediary. S3 can directly trigger Lambda functions through event notifications, simplifying the architecture.

In summary, S3 event notifications coupled with AWS Lambda offer a direct, cost-effective, and scalable solution for processing files uploaded to S3. This combination leverages the strengths of each service to meet the given requirements efficiently.

Relevant links:

Amazon S3 Event Notifications:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventNotifications.html
AWS Lambda: https://aws.amazon.com/lambda/

Question: 770 **Exam Heist**

A company's application is deployed on Amazon EC2 instances and uses AWS Lambda functions for an event-driven architecture. The company uses nonproduction development environments in a different AWS account to test new features before the company deploys the features to production.

The production instances show constant usage because of customers in different time zones. The company uses nonproduction instances only during business hours on weekdays. The company does not use the nonproduction instances on the weekends. The company wants to optimize the costs to run its application on AWS.

Which solution will meet these requirements MOST cost-effectively?

A.Use On-Demand Instances for the production instances. Use Dedicated Hosts for the nonproduction instances on weekends only.

B.Use Reserved Instances for the production instances and the nonproduction instances. Shut down the nonproduction instances when not in use.

C.Use Compute Savings Plans for the production instances. Use On-Demand Instances for the nonproduction instances. Shut down the nonproduction instances when not in use.

D.Use Dedicated Hosts for the production instances. Use EC2 Instance Savings Plans for the nonproduction instances.

Answer: C

Explanation:

The correct answer is **C**. Here's a detailed justification:

Production Instances: Production instances require consistent uptime due to global users across time zones. Compute Savings Plans offer significant cost savings for consistent, long-term EC2 usage compared to On-Demand, Reserved Instances, or Dedicated Hosts. They provide discounts on EC2 usage regardless of instance type, size, or operating system within a region.

Reference: https://aws.amazon.com/savingsplans/compute-savings-plans/

Non-Production Instances: Non-production instances are only needed during business hours on weekdays, and not on weekends. Therefore, shutting them down when not in use is crucial for cost optimization. Using

On-Demand Instances for non-production ensures you only pay for the compute time actually consumed.

Reference: https://aws.amazon.com/ec2/pricing/on-demand/

Why other options are less optimal:

A: Dedicated Hosts are expensive and only provide cost benefits with specific licensing constraints which aren't stated in the prompt. They do not provide cost benefits for instances that are only needed for limited timeframes. Using On-Demand for production does not allow you to take advantage of discounts offered by sustained usage.
B: Reserved Instances require commitment for a specific instance type and region, and while they save money, they are less flexible than Savings Plans. Also, RI for non-production instances is still less cost-effective if they are not being used for large periods of time. Shutting them down reduces the cost even more when used with On-Demand instances.
D: Dedicated Hosts are expensive as mentioned before. While EC2 Instance Savings Plans are cost-effective, shutting down the non-production instances when unused provides more savings for a variable workload that only operates during business hours.

Question: 771 **Exam Heist**

A company stores data in an on-premises Oracle relational database. The company needs to make the data available in Amazon Aurora PostgreSQL for analysis. The company uses an AWS Site-to-Site VPN connection to connect its on-premises network to AWS.

The company must capture the changes that occur to the source database during the migration to Aurora PostgreSQL.

Which solution will meet these requirements?

A.Use the AWS Schema Conversion Tool (AWS SCT) to convert the Oracle schema to Aurora PostgreSQL schema. Use the AWS Database Migration Service (AWS DMS) full-load migration task to migrate the data.

B.Use AWS DataSync to migrate the data to an Amazon S3 bucket. Import the S3 data to Aurora PostgreSQL by using the Aurora PostgreSQL aws_s3 extension.

C.Use the AWS Schema Conversion Tool (AWS SCT) to convert the Oracle schema to Aurora PostgreSQL schema. Use AWS Database Migration Service (AWS DMS) to migrate the existing data and replicate the ongoing changes.

D.Use an AWS Snowball device to migrate the data to an Amazon S3 bucket. Import the S3 data to Aurora PostgreSQL by using the Aurora PostgreSQL aws_s3 extension.

Answer: C

Explanation:

The correct answer is C because it provides a comprehensive solution for migrating an on-premises Oracle database to Aurora PostgreSQL while capturing ongoing changes during the migration.

Here's a breakdown of the justification:

AWS Schema Conversion Tool (AWS SCT): SCT is specifically designed to convert database schemas from one database engine to another. In this case, it efficiently converts the Oracle schema to an Aurora PostgreSQL-compatible schema, addressing the initial schema incompatibility.

https://aws.amazon.com/dms/schema-conversion-tool/

AWS Database Migration Service (AWS DMS): DMS is a managed service that facilitates database migrations. It supports both one-time full-load migrations and continuous data replication. DMS can handle the initial data load and, critically, capture ongoing changes (change data capture or CDC) from the Oracle

database and apply them to the Aurora PostgreSQL database in near real-time. This ensures minimal downtime and data loss during the migration. The Site-to-Site VPN provides the necessary connectivity for DMS to access the on-premises Oracle database.

https://aws.amazon.com/dms/

Option A is incomplete because it only addresses the initial data load and misses the crucial requirement of capturing ongoing changes during the migration.

Options B and D involve using S3 as an intermediary. While S3 is a good storage option, it doesn't directly address the schema conversion or the need to capture real-time changes. Using S3 for the initial data load is less efficient than DMS. The aws_s3 extension can import data from S3 but doesn't handle the initial schema conversion or CDC. Also, using Snowball (option D) for initial data transfer is only relevant if network bandwidth is severely limited, which is not stated in the problem; the company has a Site-to-Site VPN.

Question: 772 **Exam Heist**

A company built an application with Docker containers and needs to run the application in the AWS Cloud. The company wants to use a managed service to host the application.

The solution must scale in and out appropriately according to demand on the individual container services. The solution also must not result in additional operational overhead or infrastructure to manage.

Which solutions will meet these requirements? (Choose two.)

A.Use Amazon Elastic Container Service (Amazon ECS) with AWS Fargate.
B.Use Amazon Elastic Kubernetes Service (Amazon EKS) with AWS Fargate.
C.Provision an Amazon API Gateway API. Connect the API to AWS Lambda to run the containers.
D.Use Amazon Elastic Container Service (Amazon ECS) with Amazon EC2 worker nodes.
E.Use Amazon Elastic Kubernetes Service (Amazon EKS) with Amazon EC2 worker nodes.

Answer: AB

Explanation:

The question asks for container orchestration solutions on AWS that scale automatically and minimize operational overhead.

A. Use Amazon Elastic Container Service (Amazon ECS) with AWS Fargate. ECS is a managed container orchestration service. Fargate is a serverless compute engine for ECS that allows you to run containers without managing EC2 instances. This combination directly addresses the requirements by automatically scaling containers based on demand and eliminating the need to manage underlying infrastructure. https://aws.amazon.com/ecs/fargate/

B. Use Amazon Elastic Kubernetes Service (Amazon EKS) with AWS Fargate. EKS is a managed Kubernetes service, and Kubernetes is a powerful container orchestration platform. Using EKS with Fargate provides similar benefits to ECS with Fargate: automatic scaling and reduced operational overhead, as Fargate manages the underlying compute resources. EKS offers greater flexibility and richer features for complex container deployments compared to ECS, which might be relevant depending on the application's complexity, though ECS will still generally be a better solution. https://aws.amazon.com/eks/fargate/

C. Provision an Amazon API Gateway API. Connect the API to AWS Lambda to run the containers. API Gateway and Lambda are not directly designed for running persistent containerized applications at scale. While you could conceivably run containers inside Lambda (though this is not their intended use, and would involve a lot of orchestration code and limitations like cold starts), it is an inefficient and unnecessarily

complex approach compared to ECS or EKS. Lambda is intended for event-driven, short-lived functions.

D. Use Amazon Elastic Container Service (Amazon ECS) with Amazon EC2 worker nodes. While ECS itself is a managed service, using EC2 worker nodes negates the requirement of minimizing operational overhead. You are then responsible for provisioning, managing, patching, and scaling the EC2 instances hosting the containers.

E. Use Amazon Elastic Kubernetes Service (Amazon EKS) with Amazon EC2 worker nodes. Similar to option D, using EC2 worker nodes with EKS introduces significant operational overhead that the question aims to avoid. You have to manage the EC2 instances backing the Kubernetes cluster.

Question: 773 **Exam Heist**

An ecommerce company is running a seasonal online sale. The company hosts its website on Amazon EC2 instances spanning multiple Availability Zones. The company wants its website to manage sudden traffic increases during the sale.

Which solution will meet these requirements MOST cost-effectively?

A.Create an Auto Scaling group that is large enough to handle peak traffic load. Stop half of the Amazon EC2 instances. Configure the Auto Scaling group to use the stopped instances to scale out when traffic increases.

B.Create an Auto Scaling group for the website. Set the minimum size of the Auto Scaling group so that it can handle high traffic volumes without the need to scale out.

C.Use Amazon CloudFront and Amazon ElastiCache to cache dynamic content with an Auto Scaling group set as the origin. Configure the Auto Scaling group with the instances necessary to populate CloudFront and ElastiCache. Scale in after the cache is fully populated.

D.Configure an Auto Scaling group to scale out as traffic increases. Create a launch template to start new instances from a preconfigured Amazon Machine Image (AMI).

Answer: D

Explanation:

Here's a detailed justification for why option D is the most cost-effective solution for handling sudden traffic increases during a seasonal sale for an e-commerce website hosted on EC2 instances across multiple Availability Zones, along with why the other options are less ideal:

Justification for Option D:

Option D, configuring an Auto Scaling group (ASG) to scale out as traffic increases and using a launch template with a preconfigured Amazon Machine Image (AMI), is the most cost-effective approach because it adheres to the principle of elasticity inherent in cloud computing. An ASG dynamically adjusts the number of EC2 instances based on real-time demand.

1. **Cost Optimization:** You only pay for the EC2 instances you're actively using to serve traffic. When traffic is low, the ASG scales in, reducing costs. This aligns with the principle of paying for what you use.
2. **Scalability:** An ASG can rapidly provision new instances to meet sudden spikes in demand. The launch template simplifies and speeds up the instance provisioning process because the AMI already contains the application code and configurations.
3. **High Availability:** By spanning multiple Availability Zones (AZs), the ASG ensures that the application remains available even if one AZ experiences an outage.
4. **Automated Response:** The ASG automatically detects and replaces unhealthy instances, further

enhancing the application's reliability.

Why other options are less suitable:

Option A: Stopping instances and then starting them to scale is much slower than launching new instances from an AMI. Additionally, keeping half of the instances stopped still incurs costs for the storage associated with the stopped instances. It doesn't fully leverage the "pay-as-you-go" model as efficiently as dynamically scaling.

Option B: Setting a fixed, high minimum size for the ASG wastes resources during periods of low traffic. You're essentially paying for unused capacity. This is not cost-effective.

Option C: While using CloudFront and ElastiCache is good practice for improving performance and reducing load on the origin (EC2 instances), it doesn't eliminate the need for scaling. Also, scaling in the ASG after populating the cache might be premature; the cache hit ratio will determine the actual load on the origin. Furthermore, treating CloudFront and ElastiCache as the sole scalability solution without properly configured origin scaling leaves the website vulnerable to overload if the cache is insufficient.

In summary, option D provides the best balance between cost efficiency, scalability, and high availability by dynamically adjusting EC2 instance capacity based on actual demand using Auto Scaling and preconfigured AMIs.

Authoritative Links:

AWS Auto Scaling: https://aws.amazon.com/autoscaling/
Amazon Machine Images (AMIs): https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AMIs.html
AWS Launch Templates: https://docs.aws.amazon.com/autoscaling/ec2/userguide/launch-templates.html

Question: 774 **Exam Heist**

A solutions architect must provide an automated solution for a company's compliance policy that states security groups cannot include a rule that allows SSH from 0.0.0.0/0. The company needs to be notified if there is any breach in the policy. A solution is needed as soon as possible.

What should the solutions architect do to meet these requirements with the LEAST operational overhead?

A.Write an AWS Lambda script that monitors security groups for SSH being open to 0.0.0.0/0 addresses and creates a notification every time it finds one.

B.Enable the restricted-ssh AWS Config managed rule and generate an Amazon Simple Notification Service (Amazon SNS) notification when a noncompliant rule is created.

C.Create an IAM role with permissions to globally open security groups and network ACLs. Create an Amazon Simple Notification Service (Amazon SNS) topic to generate a notification every time the role is assumed by a user.

D.Configure a service control policy (SCP) that prevents non-administrative users from creating or editing security groups. Create a notification in the ticketing system when a user requests a rule that needs administrator permissions.

Answer: B

Explanation:

The correct answer is **B** because it leverages AWS Config, a fully managed service designed for configuration management and compliance. AWS Config's restricted-ssh managed rule directly addresses the requirement of monitoring security groups for SSH access from 0.0.0.0/0. This managed rule automates the detection of policy violations, minimizing operational overhead compared to a custom Lambda function.

Option A, writing a custom Lambda function, requires development, deployment, and ongoing maintenance,

which increases operational overhead. Although effective, it's not the least overhead option due to the manual effort involved.

Option C introduces an unnecessary and risky security practice. Creating an IAM role that globally opens security groups and network ACLs elevates privilege escalation and potential misuse. Monitoring role assumption doesn't directly address the specific security group rule violation, and the operational overhead of managing and monitoring this role is high.

Option D, using SCPs, prevents non-administrative users from creating or editing security groups but doesn't detect existing violations or notify when non-compliant rules are already in place. This approach requires administrators to manually review and remediate existing security groups, increasing operational overhead. SCPs also don't directly integrate with notification mechanisms.

Therefore, using the AWS Config restricted-ssh managed rule and its built-in integration with Amazon SNS offers the fastest and least operationally intensive solution. AWS Config handles the continuous monitoring, evaluation, and notification of compliance status automatically, fulfilling the company's requirements with minimal effort.

Authoritative Links:

AWS Config Managed Rules: https://docs.aws.amazon.com/config/latest/developerguide/evaluate-config.html
AWS Config restricted-ssh Managed Rule:
https://docs.aws.amazon.com/config/latest/developerguide/restricted-ssh.html
Amazon SNS: https://aws.amazon.com/sns/

Question: 775 **Exam Heist**

Use Amazon Elastic Kubernetes Service (Amazon EKS) with Amazon EC2 worker nodes.

A company has deployed an application in an AWS account. The application consists of microservices that run on AWS Lambda and Amazon Elastic Kubernetes Service (Amazon EKS). A separate team supports each microservice. The company has multiple AWS accounts and wants to give each team its own account for its microservices.

A solutions architect needs to design a solution that will provide service-to-service communication over HTTPS (port 443). The solution also must provide a service registry for service discovery.

Which solution will meet these requirements with the LEAST administrative overhead?

A.Create an inspection VPC. Deploy an AWS Network Firewall firewall to the inspection VPC. Attach the inspection VPC to a new transit gateway. Route VPC-to-VPC traffic to the inspection VPC. Apply firewall rules to allow only HTTPS communication.

B.Create a VPC Lattice service network. Associate the microservices with the service network. Define HTTPS listeners for each service. Register microservice compute resources as targets. Identify VPCs that need to communicate with the services. Associate those VPCs with the service network.

C.Create a Network Load Balancer (NLB) with an HTTPS listener and target groups for each microservice. Create an AWS PrivateLink endpoint service for each microservice. Create an interface VPC endpoint in each VPC that needs to consume that microservice.

D.Create peering connections between VPCs that contain microservices. Create a prefix list for each service that requires a connection to a client. Create route tables to route traffic to the appropriate VPC. Create security groups to allow only HTTPS communication.

Answer: B

Explanation:

The correct solution is **B. Create a VPC Lattice service network. Associate the microservices with the**

service network. Define HTTPS listeners for each service. Register microservice compute resources as targets. Identify VPCs that need to communicate with the services. Associate those VPCs with the service network.

Here's a detailed justification:

VPC Lattice is designed specifically for service-to-service communication in a multi-account and multi-VPC environment, making it the most suitable and least administratively overhead solution. It provides service discovery, traffic management (including HTTPS routing), and security policies at the application layer (Layer 7). By associating the Lambda functions and EKS-based microservices with the VPC Lattice service network, you create a unified platform for inter-service communication. Defining HTTPS listeners for each service ensures all communication is encrypted and occurs over port 443. Registering the microservices (Lambda and EKS) as targets enables VPC Lattice to route traffic to the appropriate endpoints. Associating VPCs that need communication with the lattice provides connectivity without complex routing or peering configurations.

Options A, C, and D have significant drawbacks in terms of administrative complexity and suitability for the given requirements. Option A involves creating and managing a central inspection VPC with Network Firewall, which is more suited for network-level security rather than application-level routing and service discovery. It does not directly address the service registry requirement. Option C, using NLBs and PrivateLink, is a valid solution for service exposure, but it requires managing individual NLBs for each service and configuring PrivateLink endpoints in multiple VPCs, leading to higher administrative overhead than VPC Lattice. Furthermore, PrivateLink doesn't inherently provide service discovery capabilities. Option D, VPC peering, prefix lists, route tables, and security groups is the most complex and least scalable solution, as it requires managing a large number of peering connections and security rules across multiple accounts and VPCs, leading to significant administrative overhead and potential security risks. VPC peering does not provide built-in service discovery, which is a critical requirement.

Therefore, VPC Lattice provides the most streamlined and efficient solution for service-to-service communication with service discovery, HTTPS support, and minimal administrative overhead in a multi-account AWS environment.

Authoritative links:

VPC Lattice: https://aws.amazon.com/vpc/lattice/
AWS PrivateLink: https://aws.amazon.com/privatelink/
Amazon EKS: https://aws.amazon.com/eks/

Question: 776 **Exam Heist**

A company has a mobile game that reads most of its metadata from an Amazon RDS DB instance. As the game increased in popularity, developers noticed slowdowns related to the game's metadata load times. Performance metrics indicate that simply scaling the database will not help. A solutions architect must explore all options that include capabilities for snapshots, replication, and sub-millisecond response times.

What should the solutions architect recommend to solve these issues?

A.Migrate the database to Amazon Aurora with Aurora Replicas.
B.Migrate the database to Amazon DynamoDB with global tables.
C.Add an Amazon ElastiCache for Redis layer in front of the database.
D.Add an Amazon ElastiCache for Memcached layer in front of the database.

Answer: C

Explanation:

The optimal solution is adding an Amazon ElastiCache for Redis layer in front of the database.

Here's why: The problem highlights read performance bottlenecks when retrieving metadata from RDS. Scaling the database doesn't solve this, implying the issue is high read load rather than database capacity. The requirements are snapshots, replication, and sub-millisecond response times. ElastiCache for Redis excels at caching frequently accessed data, reducing the load on the RDS database. This significantly speeds up metadata retrieval, providing sub-millisecond response times.

Redis supports snapshotting (RDB) for point-in-time recovery and replication (master-slave or cluster mode) for high availability and read scaling. Aurora Replicas (Option A) are beneficial but primarily improve read scalability within the database layer, not providing the ultra-fast caching ElastiCache offers. DynamoDB (Option B) with global tables is a NoSQL solution. Migrating the entire RDS database to DynamoDB for metadata, while viable, would be a significant architectural change with associated complexities and may not fully leverage the relational nature of the metadata. ElastiCache for Memcached (Option D) is also a caching solution, but it does not offer native snapshotting. Redis is therefore the most suitable choice considering all the requirements (snapshots, replication, and sub-millisecond response times). Adding a cache layer (ElastiCache) is a common and efficient strategy for handling read-heavy workloads.

Authoritative Links:

Amazon ElastiCache: https://aws.amazon.com/elasticache/
ElastiCache for Redis: https://aws.amazon.com/elasticache/redis/

Question: 777 **Exam Heist**

A company uses AWS Organizations for its multi-account AWS setup. The security organizational unit (OU) of the company needs to share approved Amazon Machine Images (AMIs) with the development OU. The AMIs are created by using AWS Key Management Service (AWS KMS) encrypted snapshots.

Which solution will meet these requirements? (Choose two.)

A.Add the development team's OU Amazon Resource Name (ARN) to the launch permission list for the AMIs.

B.Add the Organizations root Amazon Resource Name (ARN) to the launch permission list for the AMIs.

C.Update the key policy to allow the development team's OU to use the AWS KMS keys that are used to decrypt the snapshots.

D.Add the development team’s account Amazon Resource Name (ARN) to the launch permission list for the AMIs.

E.Recreate the AWS KMS key. Add a key policy to allow the Organizations root Amazon Resource Name (ARN) to use the AWS KMS key.

Answer: AC

Explanation:

Here's a breakdown of why options A and C are the correct choices for sharing KMS-encrypted AMI snapshots across OUs in AWS Organizations:

Option A: Add the development team's OU Amazon Resource Name (ARN) to the launch permission list for the AMIs. This allows members within the Development OU to launch instances using those specific AMIs. AMIs have launch permissions which control who can use them to create instances. By adding the Development OU ARN, you're granting launch access to anyone in that OU without needing to individually manage permissions for each account within. This is a key aspect of leveraging AWS Organizations for centralized management.

Option C: Update the key policy to allow the development team's OU to use the AWS KMS keys that are

used to decrypt the snapshots. Since the AMI's snapshots are KMS-encrypted, the Development OU needs permissions to use those keys. KMS key policies control who can use a KMS key. Without the necessary KMS permissions, the Development OU won't be able to decrypt the snapshots and, consequently, won't be able to launch instances from the AMIs. Adding the Development OU ARN to the KMS key policy grants the OU the ability to decrypt the snapshots.

Why the other options are incorrect:

Option B: Add the Organizations root Amazon Resource Name (ARN) to the launch permission list for the AMIs. While this would technically work, it grants access to every account within the entire AWS Organizations hierarchy, which is a violation of the principle of least privilege. It's overly broad and not secure.

Option D: Add the development team's account Amazon Resource Name (ARN) to the launch permission list for the AMIs. This is not scalable. OUs can contain multiple accounts. Adding individual account ARNs means you need to update the AMI permissions every time an account is added/removed from the Development OU, increasing administrative overhead.

Option E: Recreate the AWS KMS key. Add a key policy to allow the Organizations root Amazon Resource Name (ARN) to use the AWS KMS key. This is unnecessary and disruptive. Recreating a KMS key requires re-encrypting all data protected by the original key. Also, like Option B, using the Organizations root ARN grants excessively broad access to the KMS key.

Supporting Concepts:

AWS Organizations: Enables you to centrally manage and govern multiple AWS accounts. https://aws.amazon.com/organizations/
Amazon Machine Images (AMIs): Provide the information required to launch an instance, which is a virtual server in the cloud. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AMIs.html
AWS Key Management Service (AWS KMS): Enables you to create and manage cryptographic keys and control their use across a wide range of AWS services. https://aws.amazon.com/kms/
KMS Key Policies: Control who can use a KMS key and the actions they can perform. https://docs.aws.amazon.com/kms/latest/developerguide/key-policies.html
Principle of Least Privilege: Granting only the minimum necessary permissions to perform a task.

By combining launch permissions on the AMI itself (Option A) with KMS key access permissions (Option C), the Development OU can launch instances from the secure, KMS-encrypted AMIs while adhering to the security principle of least privilege.

Question: 778 **Exam Heist**

A data analytics company has 80 offices that are distributed globally. Each office hosts 1 PB of data and has between 1 and 2 Gbps of internet bandwidth.

The company needs to perform a one-time migration of a large amount of data from its offices to Amazon S3. The company must complete the migration within 4 weeks.

Which solution will meet these requirements MOST cost-effectively?

A. Establish a new 10 Gbps AWS Direct Connect connection to each office. Transfer the data to Amazon S3.
B. Use multiple AWS Snowball Edge storage-optimized devices to store and transfer the data to Amazon S3.
C. Use an AWS Snowmobile to store and transfer the data to Amazon S3.
D. Set up an AWS Storage Gateway Volume Gateway to transfer the data to Amazon S3.

Answer: B

Explanation:

Here's a detailed justification for why option B is the most cost-effective solution for migrating the data to Amazon S3 within the given constraints:

Data Volume: The company has a substantial amount of data to migrate (80 offices * 1 PB/office = 80 PB). This volume necessitates a solution designed for large-scale data transfers.

Time Constraint: The 4-week deadline is a critical factor. The chosen method must be able to move a significant amount of data within this timeframe.

Bandwidth Limitation: While each office has 1-2 Gbps internet, it's still a limiting factor for transferring 1PB per office over the internet within the 4-week deadline. Relying solely on the internet may be too slow.

Cost-Effectiveness: This is the primary concern. Solutions like AWS Direct Connect involve significant upfront and recurring costs. Storage Gateway relies on existing internet bandwidth and incurs data transfer charges. AWS Snowmobile, designed for exabyte-scale migrations, may be overkill (and more expensive) for 80 PB.

Snowball Edge Storage Optimized: Snowball Edge Storage Optimized devices are specifically designed for large-scale data transfers where network bandwidth is limited. They offer substantial storage capacity (around 80 TB per device, but can vary based on the AWS region), allowing for physical data shipping to AWS.

Estimating number of devices: With 80PB, and approximately 80TB (0.08PB) per device, you'd need around 1000 Snowball Edge devices.
Snowball Edge devices support local processing, but for this use case the value is in the transfer.
Parallel Transfers: Multiple devices can operate in parallel, speeding up the overall migration process.
AWS handles the security and integrity of the data during transport.

Why other options are less suitable:

Direct Connect: While providing dedicated bandwidth, establishing Direct Connect at 80 global offices would be prohibitively expensive and time-consuming. It would be more beneficial for ongoing operations.

Snowmobile: Intended for exabyte scale migrations, meaning Snowmobile has a higher cost, and it's likely an overkill for 80 PB of data.

Storage Gateway: Suitable for hybrid cloud scenarios and continuous data replication, not ideal for a one-time, large-scale migration when bandwidth is constrained. The transfer would take very long over a 1-2 Gbps link.

Conclusion: AWS Snowball Edge offers the best balance of speed, cost-effectiveness, and suitability for the network constraints. It avoids the high costs of Direct Connect and the potential overkill of Snowmobile, while overcoming the bandwidth limitations hindering Storage Gateway.

Authoritative Links:

AWS Snowball Edge: https://aws.amazon.com/snowball/
AWS Direct Connect: https://aws.amazon.com/directconnect/
AWS Storage Gateway: https://aws.amazon.com/storagegateway/
AWS Snowmobile: https://aws.amazon.com/snowmobile/

Question: 779 **Exam Heist**

A company has an Amazon Elastic File System (Amazon EFS) file system that contains a reference dataset. The company has applications on Amazon EC2 instances that need to read the dataset. However, the applications must not be able to change the dataset. The company wants to use IAM access control to prevent the applications from

being able to modify or delete the dataset.

Which solution will meet these requirements?

A.Mount the EFS file system in read-only mode from within the EC2 instances.

B.Create a resource policy for the EFS file system that denies the elasticfilesystem:ClientWrite action to the IAM roles that are attached to the EC2 instances.

C.Create an identity policy for the EFS file system that denies the elasticfilesystem:ClientWrite action on the EFS file system.

D.Create an EFS access point for each application. Use Portable Operating System Interface (POSIX) file permissions to allow read-only access to files in the root directory.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct solution for providing read-only access to an Amazon EFS file system for EC2 instances, while preventing modifications or deletion via IAM access control:

The core requirement is to ensure that EC2 instances can read the EFS dataset but cannot modify it. This necessitates a mechanism to restrict write access at the EFS level. IAM policies offer granular control over access to AWS resources. Specifically, resource-based policies attach directly to the EFS file system and define permissions for who can access the resource and what actions they can perform.

Option B leverages a resource policy attached to the EFS file system. By explicitly denying the elasticfilesystem:ClientWrite action (which encompasses all actions that would modify the data) to the IAM roles associated with the EC2 instances, the company effectively prevents those instances from writing to or deleting anything on the EFS volume. This directly addresses the security requirement of ensuring that the dataset remains unchanged.

Here's why the other options are less suitable:

A. Mount the EFS file system in read-only mode from within the EC2 instances: This is insufficient because mounting in read-only mode on the client (EC2 instance) side doesn't prevent a malicious actor who has gained access to the EC2 instance with the correct permissions from remounting it with write access or modifying the mount configuration. This approach relies on the EC2 instance configuration which could be changed. IAM policies provide centralized, authoritative enforcement, making it more secure.

C. Create an identity policy for the EFS file system that denies the elasticfilesystem:ClientWrite action on the EFS file system: While technically possible, identity policies are attached to IAM roles or users, not directly to the EFS resource. This means you would have to apply this policy to every IAM entity that should not have write access. It's much easier and less error-prone to control access from the EFS resource itself using resource-based policies. Resource based policies give you a centralized location to manage these controls.

D. Create an EFS access point for each application. Use Portable Operating System Interface (POSIX) file permissions to allow read-only access to files in the root directory: EFS access points primarily focus on controlling the user identity and file system path for access. While POSIX permissions can restrict access to specific files and directories within the EFS file system, they don't fundamentally prevent an authorized user (authorized via IAM) from modifying the permissions themselves or other files in the file system. This adds complexity without a direct security benefit in this scenario. Access points are better suited for situations where you need different applications to access the same data under different user contexts or paths. They don't replace the need for IAM-based authorization. Access points on their own do not prevent against authorized users from modifying or deleting objects in the FS.

In summary, option B directly implements the desired access control by denying write access to the EC2 instances via the resource policy attached to the EFS. This method enforces centralized, declarative access

control, which is more robust and maintainable.

Authoritative Links:

Amazon EFS Identity and Access Management: https://docs.aws.amazon.com/efs/latest/ug/iam.html
Using IAM to Control EFS Access: https://docs.aws.amazon.com/efs/latest/ug/using-iam-efs.html
Resource-Based Policies: https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_identity-vs-resource.html

Question: 780 **Exam Heist**

A company has hired an external vendor to perform work in the company's AWS account. The vendor uses an automated tool that is hosted in an AWS account that the vendor owns. The vendor does not have IAM access to the company's AWS account. The company needs to grant the vendor access to the company's AWS account.

Which solution will meet these requirements MOST securely?

A.Create an IAM role in the company's account to delegate access to the vendor's IAM role. Attach the appropriate IAM policies to the role for the permissions that the vendor requires.

B.Create an IAM user in the company's account with a password that meets the password complexity requirements. Attach the appropriate IAM policies to the user for the permissions that the vendor requires.

C.Create an IAM group in the company's account. Add the automated tool's IAM user from the vendor account to the group. Attach the appropriate IAM policies to the group for the permissions that the vendor requires.

D.Create an IAM user in the company's account that has a permission boundary that allows the vendor's account. Attach the appropriate IAM policies to the user for the permissions that the vendor requires.

Answer: A

Explanation:

The most secure solution is to use IAM roles for cross-account access. Option A proposes creating an IAM role in the company's AWS account that the vendor's IAM role can assume. This is the best approach because it avoids sharing long-term credentials like passwords or access keys.

IAM roles enable you to delegate access to users or services that don't normally have access to your AWS resources. In this case, the vendor's tool, authenticated by its own IAM role in the vendor's account, assumes the role in the company's account. This temporary access is governed by the policies attached to the assumed role.

Option B is insecure because creating an IAM user with a password shares long-term credentials, violating security best practices. Option C is also insecure because adding the vendor's IAM user directly to a group requires managing individual user permissions in the company's account. Option D's permission boundary approach is also suboptimal as it involves creating an IAM user and managing permissions using a boundary when role assumption is the intended workflow.

Cross-account IAM roles using the principle of least privilege are the gold standard for this scenario. The policies attached to the company's role precisely define what the vendor's tool can do. By using a trust policy on the role, you specify which AWS accounts (in this case, the vendor's) are allowed to assume it. This ensures that only authorized accounts and users can access resources.https://docs.aws.amazon.com/IAM/latest/UserGuide/tutorial_cross-account-with-roles.htmlhttps://aws.amazon.com/blogs/security/how-to-delegate-access-across-aws-accounts-using-iam-roles/

Question: 781 **Exam Heist**

A company wants to run its experimental workloads in the AWS Cloud. The company has a budget for cloud spending. The company's CFO is concerned about cloud spending accountability for each department. The CFO wants to receive notification when the spending threshold reaches 60% of the budget.

Which solution will meet these requirements?

A.Use cost allocation tags on AWS resources to label owners. Create usage budgets in AWS Budgets. Add an alert threshold to receive notification when spending exceeds 60% of the budget.

B.Use AWS Cost Explorer forecasts to determine resource owners. Use AWS Cost Anomaly Detection to create alert threshold notifications when spending exceeds 60% of the budget.

C.Use cost allocation tags on AWS resources to label owners. Use AWS Support API on AWS Trusted Advisor to create alert threshold notifications when spending exceeds 60% of the budget.

D.Use AWS Cost Explorer forecasts to determine resource owners. Create usage budgets in AWS Budgets. Add an alert threshold to receive notification when spending exceeds 60% of the budget.

Answer: A

Explanation:

Option A is the correct solution because it leverages AWS Budgets, which is specifically designed for setting spending thresholds and receiving alerts when those thresholds are approached or exceeded.

Here's why:

1. **Cost Allocation Tags:** Cost allocation tags (See https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/cost-alloc-tags.html) enable you to categorize and track your AWS resource costs. By tagging resources with department-specific tags, you can attribute spending to the correct department, which addresses the CFO's concern for accountability.
2. **AWS Budgets:** AWS Budgets (See https://aws.amazon.com/aws/tools/aws-budgets/) allows you to set custom budgets and track your AWS usage and costs against those budgets. You can define budget periods (e.g., monthly, quarterly) and set thresholds for alerts.
3. **Alert Thresholds:** The key requirement is to receive a notification when spending reaches 60% of the budget. AWS Budgets lets you define custom alert thresholds (e.g., 60%, 80%, 100%) and configure email notifications when these thresholds are breached. This directly satisfies the CFO's notification requirement.

Why the other options are incorrect:

Option B: While AWS Cost Explorer forecasts can help with anticipating future costs, it does not automatically assign resource ownership. AWS Cost Anomaly Detection is designed to identify unusual spending patterns, but is less effective than the straight forward AWS budget approach for simple budget monitoring.
Option C: While cost allocation tags are useful, using the AWS Support API on AWS Trusted Advisor for budget alerting is not the intended function of these services. AWS Trusted Advisor mainly focuses on best practices, cost optimization, security, and performance, not detailed budget threshold notifications.
Option D: Similar to option B, AWS Cost Explorer forecasts cannot determine resource owners, and using it to set alert thresholds is not it's intended function, use AWS Budgets instead.

Question: 782 **Exam Heist**

A company wants to deploy an internal web application on AWS. The web application must be accessible only from

the company's office. The company needs to download security patches for the web application from the internet.

The company has created a VPC and has configured an AWS Site-to-Site VPN connection to the company's office. A solutions architect must design a secure architecture for the web application.

Which solution will meet these requirements?

A.Deploy the web application on Amazon EC2 instances in public subnets behind a public Application Load Balancer (ALB). Attach an internet gateway to the VPC. Set the inbound source of the ALB's security group to 0.0.0.0/0.

B.Deploy the web application on Amazon EC2 instances in private subnets behind an internal Application Load Balancer (ALB). Deploy NAT gateways in public subnets. Attach an internet gateway to the VPC. Set the inbound source of the ALB's security group to the company's office network CIDR block.

C.Deploy the web application on Amazon EC2 instances in public subnets behind an internal Application Load Balancer (ALB). Deploy NAT gateways in private subnets. Attach an internet gateway to the VPSet the outbound destination of the ALB’s security group to the company's office network CIDR block.

D.Deploy the web application on Amazon EC2 instances in private subnets behind a public Application Load Balancer (ALB). Attach an internet gateway to the VPC. Set the outbound destination of the ALB’s security group to 0.0.0.0/0.

Answer: B

Explanation:

The correct solution is B because it aligns with the requirement of internal accessibility and outbound internet access for patching.

Here's why:

Private Subnets for Security: Placing the EC2 instances hosting the web application in private subnets isolates them from direct internet exposure, fulfilling the requirement that the application be accessible only from the company's office. https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Subnets.html
Internal ALB for Load Balancing: An internal Application Load Balancer (ALB) ensures traffic is distributed efficiently across the EC2 instances within the private subnets. It is accessible only within the VPC. https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html
NAT Gateway for Outbound Internet Access: NAT gateways in public subnets allow the EC2 instances in the private subnets to initiate outbound connections to the internet (for downloading security patches) without exposing them to inbound traffic from the internet. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html
Internet Gateway for NAT Gateway: The internet gateway is necessary for the NAT gateway to function. The NAT gateway resides in the public subnet and uses the internet gateway to access the internet.
Security Group Inbound Rule: Restricting the ALB's security group inbound traffic to the company's office network CIDR block, via the Site-to-Site VPN connection, ensures only authorized traffic from the company's network can access the web application. Security groups act as virtual firewalls, controlling traffic at the instance level. https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html

Option A is incorrect because deploying instances in public subnets and setting the ALB's inbound source to 0.0.0.0/0 makes the application accessible to the entire internet, violating the security requirement.

Option C is wrong because the placement of NAT gateways and the intended direction of outbound security group rules are incorrect. NAT Gateways need to reside in public subnets to forward requests to the internet, and security group rules filter inbound traffic, not outbound traffic.

Option D is incorrect because it uses a public ALB with EC2 instances in private subnets. While the EC2 instances are secured, the ALB would be exposed to the public internet, contrary to the requirement. Furthermore, outbound destination rules are uncommon in security groups.

Question: 783 **Exam Heist**

A company maintains its accounting records in a custom application that runs on Amazon EC2 instances. The company needs to migrate the data to an AWS managed service for development and maintenance of the application data. The solution must require minimal operational support and provide immutable, cryptographically verifiable logs of data changes.

Which solution will meet these requirements MOST cost-effectively?

A.Copy the records from the application into an Amazon Redshift cluster.
B.Copy the records from the application into an Amazon Neptune cluster.
C.Copy the records from the application into an Amazon Timestream database.
D.Copy the records from the application into an Amazon Quantum Ledger Database (Amazon QLDB) ledger.

Answer: D

Explanation:

The most cost-effective solution for migrating accounting records to an AWS managed service that requires minimal operational support and provides immutable, cryptographically verifiable logs is Amazon Quantum Ledger Database (QLDB).

Here's why:

Immutability and Cryptographic Verification: QLDB is designed specifically for systems of record that require a high degree of trust and transparency. It provides an immutable transaction log that is cryptographically verifiable. Each transaction is chained together using a cryptographic hash, ensuring that any tampering is detectable. https://aws.amazon.com/qldb/

Minimal Operational Support: As a fully managed database, QLDB handles patching, backups, and other operational tasks, reducing the operational burden on the company.

Cost-Effectiveness: For accounting records, QLDB offers a pay-per-use pricing model based on storage, I/O, and journal storage, which can be more cost-effective than other options for this specific use case.

Now let's analyze why the other options are less suitable:

Amazon Redshift: While Redshift is a powerful data warehouse, it's not designed for maintaining immutable ledgers or providing cryptographic verification of data changes. It's more suitable for analytical workloads, which is not the primary need here. Also, managing a Redshift cluster requires more operational overhead than QLDB.

Amazon Neptune: Neptune is a graph database service and is designed for storing and querying relationships between data. It does not inherently provide immutability or cryptographic verification of changes, nor is it the best choice for structured accounting data.

Amazon Timestream: Timestream is a time-series database service optimized for storing and querying time-series data. While it can store data over time, it does not provide the immutability and cryptographic verification features required for accounting records that are natively present in QLDB.

In conclusion, QLDB is the best option because it inherently provides the required immutability, cryptographic verification, and minimal operational overhead at a potentially lower cost for this particular use case.

Question: 784 **Exam Heist**

A company's marketing data is uploaded from multiple sources to an Amazon S3 bucket. A series of data

preparation jobs aggregate the data for reporting. The data preparation jobs need to run at regular intervals in parallel. A few jobs need to run in a specific order later.

The company wants to remove the operational overhead of job error handling, retry logic, and state management.

Which solution will meet these requirements?

A.Use an AWS Lambda function to process the data as soon as the data is uploaded to the S3 bucket. Invoke other Lambda functions at regularly scheduled intervals.

B.Use Amazon Athena to process the data. Use Amazon EventBridge Scheduler to invoke Athena on a regular internal.

C.Use AWS Glue DataBrew to process the data. Use an AWS Step Functions state machine to run the DataBrew data preparation jobs.

D.Use AWS Data Pipeline to process the data. Schedule Data Pipeline to process the data once at midnight.

Answer: C

Explanation:

The correct answer is C: Use AWS Glue DataBrew to process the data. Use an AWS Step Functions state machine to run the DataBrew data preparation jobs.

Here's why:

AWS Glue DataBrew excels at data preparation tasks like cleaning, normalizing, and enriching data without requiring you to write code. It provides a visual interface for data transformation. This aligns with the requirement of aggregating data for reporting.
AWS Step Functions is a serverless orchestration service that lets you define workflows (state machines) to coordinate multiple AWS services. It addresses the needs for running data preparation jobs at regular intervals, in parallel where needed, and in a specific order later. The visual workflow simplifies orchestration, error handling, retry logic, and state management.
Step Functions' built-in error handling and retry logic significantly reduce the operational overhead by automating these critical aspects.
Step Functions can orchestrate DataBrew jobs, ensuring they run in the specified order with the needed parallel executions at the desired intervals.

Let's examine why the other options are less suitable:

A. AWS Lambda: While Lambda can process data upon S3 upload, managing complex dependencies, parallelism, and a series of jobs in a specific order becomes unwieldy. Regularly scheduled intervals with other Lambda invocations would require custom scheduling and intricate error handling, which increases operational overhead. Lambda has execution time limits that might hinder lengthy data preparation jobs.
B. Amazon Athena: Athena is a query service to analyze data in S3, not a data preparation tool. It focuses on querying and analysis after the data is prepared. While EventBridge Scheduler can schedule Athena queries, Athena itself doesn't handle data preparation tasks or orchestration of multiple jobs.
D. AWS Data Pipeline: Data Pipeline is an older service, largely superseded by more modern alternatives like Glue and Step Functions. While it can schedule and orchestrate data processing tasks, it's more complex to manage, less flexible than Step Functions, and requires more manual configuration for error handling and state management. Choosing a legacy service when better options exist is not ideal.

Therefore, the combination of AWS Glue DataBrew for data preparation and AWS Step Functions for orchestration delivers the best solution by simplifying the data preparation process and reducing operational overhead through serverless orchestration, built-in error handling, and retry logic.

Supporting Documentation:

AWS Glue DataBrew: https://aws.amazon.com/glue/databrew/

AWS Step Functions: https://aws.amazon.com/step-functions/

Question: 785 **Exam Heist**

A solutions architect is designing a payment processing application that runs on AWS Lambda in private subnets across multiple Availability Zones. The application uses multiple Lambda functions and processes millions of transactions each day.

The architecture must ensure that the application does not process duplicate payments.

Which solution will meet these requirements?

A.Use Lambda to retrieve all due payments. Publish the due payments to an Amazon S3 bucket. Configure the S3 bucket with an event notification to invoke another Lambda function to process the due payments.

B.Use Lambda to retrieve all due payments. Publish the due payments to an Amazon Simple Queue Service (Amazon SQS) queue. Configure another Lambda function to poll the SQS queue and to process the due payments.

C.Use Lambda to retrieve all due payments. Publish the due payments to an Amazon Simple Queue Service (Amazon SQS) FIFO queue. Configure another Lambda function to poll the FIFO queue and to process the due payments.

D.Use Lambda to retrieve all due payments. Store the due payments in an Amazon DynamoDB table. Configure streams on the DynamoDB table to invoke another Lambda function to process the due payments.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best solution for ensuring no duplicate payment processing in the described architecture, along with supporting information:

The primary requirement is to prevent duplicate payment processing. In a distributed system like this, transient errors or retries can inadvertently cause the same payment to be processed multiple times. The key to solving this problem is to enforce ordered processing and deduplication.

Option C utilizes Amazon SQS FIFO (First-In, First-Out) queues. FIFO queues guarantee that messages are processed in the exact order they are sent and provide exactly-once processing semantics when combined with deduplication. This is crucial for payment processing, where the order of transactions may be significant, and preventing duplicates is paramount. The first Lambda function retrieves payments and publishes them to the FIFO queue. The second Lambda function polls the queue and processes the payments in the order they were received. The FIFO queue's built-in deduplication feature prevents duplicate messages from being processed, even if they are sent multiple times.

Option A is incorrect because using Amazon S3 for queuing introduces complexities and doesn't guarantee order or prevent duplicate processing without significant custom coding. S3 event notifications are not designed for reliable message queuing.

Option B is incorrect because standard SQS queues do not guarantee message order or prevent duplicates. While you could implement deduplication in the second Lambda function, the FIFO queue approach in Option C is a cleaner, more reliable, and AWS-managed solution for preventing duplicates.

Option D is incorrect because while DynamoDB streams can provide near real-time data propagation, they do not inherently guarantee ordered delivery or deduplication. It also introduces an additional dependency on DynamoDB, which isn't necessary given the capabilities of SQS FIFO queues. Implementing deduplication with DynamoDB streams would require complex logic and potentially introduce latency. The scaling characteristics of SQS are also better suited for a high-volume, asynchronous payment processing system.

In summary, using an Amazon SQS FIFO queue with a Lambda function consumer provides a robust and

scalable solution for ordered, exactly-once processing, which directly addresses the requirement of preventing duplicate payment processing in a multi-AZ, Lambda-based payment application. The other options lack the built-in capabilities for ensuring both message order and deduplication.

Supporting Documentation:

Amazon SQS FIFO Queues:
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues.html
Lambda Best Practices: https://docs.aws.amazon.com/lambda/latest/dg/best-practices.html

Question: 786 **Exam Heist**

A company runs multiple workloads in its on-premises data center. The company's data center cannot scale fast enough to meet the company's expanding business needs. The company wants to collect usage and configuration data about the on-premises servers and workloads to plan a migration to AWS.

Which solution will meet these requirements?

A.Set the home AWS Region in AWS Migration Hub. Use AWS Systems Manager to collect data about the on-premises servers.

B.Set the home AWS Region in AWS Migration Hub. Use AWS Application Discovery Service to collect data about the on-premises servers.

C.Use the AWS Schema Conversion Tool (AWS SCT) to create the relevant templates. Use AWS Trusted Advisor to collect data about the on-premises servers.

D.Use the AWS Schema Conversion Tool (AWS SCT) to create the relevant templates. Use AWS Database Migration Service (AWS DMS) to collect data about the on-premises servers.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer, and why the other options are incorrect:

The core requirement is to collect usage and configuration data from on-premises servers to facilitate migration planning. This necessitates a discovery and assessment process.

Option B: Setting the home AWS Region in AWS Migration Hub and utilizing AWS Application Discovery Service (ADS) accurately addresses this need. Migration Hub acts as a central location to track migrations and ADS is specifically designed to gather information about on-premises servers, including their configuration, utilization, and dependencies. https://aws.amazon.com/application-discovery/ and https://aws.amazon.com/migration-hub/

ADS provides a comprehensive view of the on-premises environment by discovering servers, their operating systems, applications, and network dependencies. This data helps organizations plan the migration by identifying compatible AWS services and estimating the required resources. Setting the home AWS Region within Migration Hub allows for centralized tracking of migration progress.

Option A: While AWS Systems Manager can collect data from servers, it typically requires the Systems Manager agent to be installed on the servers. This might present challenges in an on-premises environment where automated agent deployment and management could be difficult. Furthermore, Systems Manager is not primarily designed for in-depth migration discovery and assessment. While useful for managed instances, its strength isn't the initial discovery of servers in an unknown environment.

Option C & D: AWS Schema Conversion Tool (SCT) and AWS Database Migration Service (DMS) are focused on database migrations, not general server discovery and assessment. SCT helps convert database schemas between different database engines. DMS is used to migrate databases. Trusted Advisor provides cost

optimization, security, fault tolerance, and performance recommendations for already running AWS resources, not for on-premises environments during a migration assessment phase. https://aws.amazon.com/dms/ and https://aws.amazon.com/schema-conversion-tool/ and https://aws.amazon.com/premiumsupport/technology/trusted-advisor/

In summary, ADS provides the required discovery capabilities for understanding the on-premises environment prior to migration, and Migration Hub helps organize and track the overall migration process. Therefore, option B is the most appropriate solution for the scenario.

Question: 787 **Exam Heist**

A company has an organization in AWS Organizations that has all features enabled. The company requires that all API calls and logins in any existing or new AWS account must be audited. The company needs a managed solution to prevent additional work and to minimize costs. The company also needs to know when any AWS account is not compliant with the AWS Foundational Security Best Practices (FSBP) standard.

Which solution will meet these requirements with the LEAST operational overhead?

A.Deploy an AWS Control Tower environment in the Organizations management account. Enable AWS Security Hub and AWS Control Tower Account Factory in the environment.

B.Deploy an AWS Control Tower environment in a dedicated Organizations member account. Enable AWS Security Hub and AWS Control Tower Account Factory in the environment.

C.Use AWS Managed Services (AMS) Accelerate to build a multi-account landing zone (MALZ). Submit an RFC to self-service provision Amazon GuardDuty in the MALZ.

D.Use AWS Managed Services (AMS) Accelerate to build a multi-account landing zone (MALZ). Submit an RFC to self-service provision AWS Security Hub in the MALZ.

Answer: A

Explanation:

The best solution is to deploy AWS Control Tower in the organization's management account and enable AWS Security Hub and Account Factory. Control Tower simplifies multi-account management within AWS Organizations by automating the setup of a well-architected, secure, and compliant landing zone. Deploying in the management account leverages Control Tower's organizational-wide governance capabilities.

Enabling AWS Security Hub provides a centralized view of security alerts and compliance status across all AWS accounts in the organization. It automatically assesses resource configurations against security standards like the AWS Foundational Security Best Practices (FSBP) and generates findings. Security Hub also aggregates findings from other AWS security services like GuardDuty and Inspector.

AWS Control Tower Account Factory automates the provisioning of new accounts pre-configured with baseline security and compliance settings. This ensures that all new accounts adhere to the organization's security policies from the start. By enabling Security Hub at the organization level, all API calls and logins are automatically audited, and compliance with FSBP is continuously monitored, minimizing operational overhead.

Option B is incorrect because deploying Control Tower in a member account limits its scope and requires additional configuration to extend governance across the entire organization. Options C and D using AWS Managed Services (AMS) Accelerate involve more complexity and operational overhead compared to Control Tower. AMS requires submitting RFCs for provisioning services and involves a more hands-on approach, making it less managed than Control Tower for the stated requirements. Furthermore, focusing solely on GuardDuty (option C) doesn't address the FSBP compliance requirement directly as Security Hub does.

Here are some authoritative links for further research:

AWS Control Tower: https://aws.amazon.com/controltower/
AWS Security Hub: https://aws.amazon.com/security-hub/
AWS Organizations: https://aws.amazon.com/organizations/
AWS Foundational Security Best Practices (FSBP):
https://docs.aws.amazon.com/securityhub/latest/userguide/fsbp-standards-cis.html

Question: 788 **Exam Heist**

A company has stored 10 TB of log files in Apache Parquet format in an Amazon S3 bucket. The company occasionally needs to use SQL to analyze the log files.

Which solution will meet these requirements MOST cost-effectively?

A.Create an Amazon Aurora MySQL database. Migrate the data from the S3 bucket into Aurora by using AWS Database Migration Service (AWS DMS). Issue SQL statements to the Aurora database.

B.Create an Amazon Redshift cluster. Use Redshift Spectrum to run SQL statements directly on the data in the S3 bucket.

C.Create an AWS Glue crawler to store and retrieve table metadata from the S3 bucket. Use Amazon Athena to run SQL statements directly on the data in the S3 bucket.

D.Create an Amazon EMR cluster. Use Apache Spark SQL to run SQL statements directly on the data in the S3 bucket.

Answer: C

Explanation:

The most cost-effective solution for analyzing Parquet-formatted log files in S3 with occasional SQL queries is using AWS Glue and Amazon Athena.

Explanation:

Cost Considerations: Athena and Glue follow a pay-per-query model. This is significantly more cost-effective for infrequent analysis compared to running and maintaining a persistent cluster like Redshift or EMR. Aurora would incur costs for storage, compute, and data migration, making it less suitable for occasional queries against log files.

Functionality: Athena directly queries data in S3 using standard SQL. It requires metadata about the data's structure. AWS Glue crawler automatically discovers the schema of the Parquet files and creates a metadata catalog that Athena can use.

Alternatives Breakdown:

Aurora MySQL with DMS: Overkill and expensive. Requires database management, data loading, and ongoing Aurora costs, even when not actively querying.
Redshift Spectrum: While Redshift Spectrum allows querying S3 data, Redshift itself is a data warehouse designed for more intensive analytical workloads and would be more expensive for occasional use.
EMR with Spark SQL: EMR involves spinning up and managing a cluster, which is not cost-effective for infrequent queries. It also adds operational overhead. Spark SQL is suitable for more complex data processing and transformations. Athena offers a simpler SQL interface directly on S3.

Workflow:

1. **AWS Glue Crawler:** Configured to crawl the S3 bucket containing the Parquet log files.
2. **AWS Glue Data Catalog:** The crawler creates a table definition in the AWS Glue Data Catalog, defining the schema of the Parquet data.
3. **Amazon Athena:** Users can then use the Athena console or API to run SQL queries against the table

defined in the Glue Data Catalog. Athena uses the table metadata to understand the structure of the Parquet data in S3 and execute the queries.

Authoritative Links:

Amazon Athena: https://aws.amazon.com/athena/
AWS Glue: https://aws.amazon.com/glue/
Redshift Spectrum: https://aws.amazon.com/redshift/spectrum/

Question: 789 **Exam Heist**

A company needs a solution to prevent AWS CloudFormation stacks from deploying AWS Identity and Access Management (IAM) resources that include an inline policy or "*" in the statement. The solution must also prohibit deployment of Amazon EC2 instances with public IP addresses. The company has AWS Control Tower enabled in its organization in AWS Organizations.

Which solution will meet these requirements?

A.Use AWS Control Tower proactive controls to block deployment of EC2 instances with public IP addresses and inline policies with elevated access or "*".

B.Use AWS Control Tower detective controls to block deployment of EC2 instances with public IP addresses and inline policies with elevated access or "*".

C.Use AWS Config to create rules for EC2 and IAM compliance. Configure the rules to run an AWS Systems Manager Session Manager automation to delete a resource when it is not compliant.

D.Use a service control policy (SCP) to block actions for the EC2 instances and IAM resources if the actions lead to noncompliance.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct answer, along with explanations of why the other options are incorrect, and supporting documentation:

The requirement is to prevent the deployment of specific non-compliant resources within an AWS Organization governed by AWS Control Tower. The solution needs to block the deployment of EC2 instances with public IPs and IAM resources with overly permissive inline policies.

Option D: Use a service control policy (SCP) to block actions for the EC2 instances and IAM resources if the actions lead to noncompliance. This is the most suitable solution. SCPs are designed for preventative governance within AWS Organizations. They allow you to define guardrails at the organizational unit (OU) or account level, preventing actions that violate the policy. In this scenario, an SCP can be written to deny the creation of EC2 instances with public IP addresses and deny the creation of IAM roles or users that include inline policies containing "*". SCPs evaluate before the request is made, preventing the action from ever occurring. This fulfills the requirement of preventing the deployment of these resources.

Option A: Use AWS Control Tower proactive controls to block deployment of EC2 instances with public IP addresses and inline policies with elevated access or "*". While AWS Control Tower proactively guides resource provisioning, it primarily relies on enforcing guardrails using SCPs and AWS Config rules behind the scenes. Proactive controls, particularly with AWS Control Tower, often initiate guardrails based on best practices to prevent deployment of non-compliant resources. Although, these controls also end up being an SCP under the hood, so the answer to this question is D because it is the actual service being utilized.

Option B: Use AWS Control Tower detective controls to block deployment of EC2 instances with public IP addresses and inline policies with elevated access or "*". Detective controls, like AWS Config rules, identify and report non-compliant resources after they have been deployed. They do not prevent the initial

deployment. The requirement is to prevent deployment, so this option fails. Detective controls provide auditing and reporting, not preventative measures.

Option C: Use AWS Config to create rules for EC2 and IAM compliance. Configure the rules to run an AWS Systems Manager Session Manager automation to delete a resource when it is not compliant. AWS Config detects non-compliant resources. While you can use remediation actions (like a Systems Manager automation to delete a resource), this reacts to the deployment after it happens. The requirement is to prevent deployment in the first place. This option does not provide a preventative solution, and attempting to automatically delete newly created non-compliant resources can lead to operational complexities and potential data loss.

In summary: SCPs are the only option that provides the preventative control required to block the deployment of non-compliant EC2 instances and IAM resources. AWS Config provides detective controls. AWS Control Tower simplifies the management of governance at the OU level but typically relies on underlying services such as SCPs and Config.

Authoritative Links:

AWS Organizations Service Control Policies (SCPs):
https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scp.html
AWS Control Tower: https://aws.amazon.com/controltower/
AWS Config: https://aws.amazon.com/config/

Question: 790 **Exam Heist**

A company's web application that is hosted in the AWS Cloud recently increased in popularity. The web application currently exists on a single Amazon EC2 instance in a single public subnet. The web application has not been able to meet the demand of the increased web traffic.

The company needs a solution that will provide high availability and scalability to meet the increased user demand without rewriting the web application.

Which combination of steps will meet these requirements? (Choose two.)

A.Replace the EC2 instance with a larger compute optimized instance.
B.Configure Amazon EC2 Auto Scaling with multiple Availability Zones in private subnets.
C.Configure a NAT gateway in a public subnet to handle web requests.
D.Replace the EC2 instance with a larger memory optimized instance.
E.Configure an Application Load Balancer in a public subnet to distribute web traffic.

Answer: BE

Explanation:

The correct answer is **B. Configure Amazon EC2 Auto Scaling with multiple Availability Zones in private subnets** and **E. Configure an Application Load Balancer in a public subnet to distribute web traffic**.

Justification:

The problem requires a solution that provides high availability and scalability without rewriting the web application. Let's analyze why each option is or isn't suitable:

A. Replace the EC2 instance with a larger compute optimized instance: While this may provide some temporary relief, it doesn't address high availability (single point of failure) or automatic scalability as demand fluctuates. It's a vertical scaling approach, which has limitations.

B. Configure Amazon EC2 Auto Scaling with multiple Availability Zones in private subnets: This is a crucial component of the solution. Auto Scaling groups automatically launch and terminate EC2 instances based on demand, ensuring scalability. Deploying instances across multiple Availability Zones (AZs) provides high availability; if one AZ fails, the application continues to run in others. Placing the instances in private subnets is important for security, shielding them from direct internet access.

C. Configure a NAT gateway in a public subnet to handle web requests: A NAT Gateway is used to allow instances in private subnets to connect to the internet, but not to handle incoming web traffic. This option is related to outbound connectivity, not the inbound traffic the application needs to handle.

D. Replace the EC2 instance with a larger memory optimized instance: Similar to option A, this is vertical scaling, which doesn't provide high availability or dynamic scalability. It might not even be the correct optimization if the application is CPU-bound rather than memory-bound.

E. Configure an Application Load Balancer in a public subnet to distribute web traffic: This is the second crucial component. An Application Load Balancer (ALB) distributes incoming web traffic across multiple EC2 instances within the Auto Scaling group. It provides a single point of entry for users and intelligently routes requests to healthy instances. Placing the ALB in a public subnet makes it accessible to internet users. The ALB will be able to reach the private instances through the VPC network.

Therefore, the combination of Auto Scaling (B) and an Application Load Balancer (E) delivers the required high availability and scalability to meet increased user demand without rewriting the application. The ALB distributes traffic, while Auto Scaling ensures enough instances are running across multiple Availability Zones.

Authoritative Links:

Amazon EC2 Auto Scaling: https://aws.amazon.com/autoscaling/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/

Question: 791 **Exam Heist**

A company has AWS Lambda functions that use environment variables. The company does not want its developers to see environment variables in plaintext.

Which solution will meet these requirements?

A.Deploy code to Amazon EC2 instances instead of using Lambda functions.

B.Configure SSL encryption on the Lambda functions to use AWS CloudHSM to store and encrypt the environment variables.

C.Create a certificate in AWS Certificate Manager (ACM). Configure the Lambda functions to use the certificate to encrypt the environment variables.

D.Create an AWS Key Management Service (AWS KMS) key. Enable encryption helpers on the Lambda functions to use the KMS key to store and encrypt the environment variables.

Answer: D

Explanation:

The correct answer is **D. Create an AWS Key Management Service (AWS KMS) key. Enable encryption helpers on the Lambda functions to use the KMS key to store and encrypt the environment variables.**

Here's why:

The core requirement is to protect Lambda environment variables from being seen in plaintext by developers. AWS KMS provides a managed service for creating and controlling the encryption keys used to encrypt data.

By using a KMS key, the environment variables can be encrypted at rest, ensuring that developers only see the encrypted values, not the actual secrets.

AWS Lambda integrates with AWS KMS to natively support encryption of environment variables. Lambda provides "encryption helpers," which are libraries or utilities that simplify the process of encrypting and decrypting environment variables using KMS. This allows you to encrypt sensitive information before storing it as environment variables. The Lambda function can then decrypt the variables at runtime.

Option A is incorrect because deploying to EC2 instances doesn't solve the problem of securing environment variables. You'd still need a mechanism to protect the secrets stored on the EC2 instances. It also moves away from serverless, which may be an undesired architectural change.

Option B is incorrect because configuring SSL encryption on the Lambda functions relates to securing communication to the function, not encrypting the environment variables themselves. While CloudHSM could store keys, integrating it directly for encrypting environment variables within Lambda isn't the standard or most efficient approach compared to KMS. CloudHSM is more geared towards applications requiring FIPS 140-2 Level 3 compliance and dedicated hardware security modules.

Option C is incorrect because ACM certificates are primarily used for securing network traffic (TLS/SSL). They are not designed for encrypting data at rest, such as environment variables. ACM manages SSL/TLS certificates, not general-purpose encryption keys.

Using KMS offers several advantages:

Centralized key management: KMS simplifies the management and rotation of encryption keys.
Access control: KMS allows fine-grained control over who can use the key, ensuring only authorized users and services can access the encrypted environment variables.
Auditing: KMS provides audit trails of key usage, allowing you to track who accessed the encrypted environment variables.
Integration with Lambda: Direct integration and readily available encryption helpers streamline the implementation.

Here are some relevant links for further research:

AWS Lambda Environment Variables: https://docs.aws.amazon.com/lambda/latest/dg/configuration-envvars.html
AWS KMS: https://aws.amazon.com/kms/
Encrypting Lambda Environment Variables: https://aws.amazon.com/blogs/security/encrypting-aws-lambda-environment-variables-using-aws-kms/

Question: 792 **Exam Heist**

An analytics company uses Amazon VPC to run its multi-tier services. The company wants to use RESTful APIs to offer a web analytics service to millions of users. Users must be verified by using an authentication service to access the APIs.

Which solution will meet these requirements with the MOST operational efficiency?

A.Configure an Amazon Cognito user pool for user authentication. Implement Amazon API Gateway REST APIs with a Cognito authorizer.

B.Configure an Amazon Cognito identity pool for user authentication. Implement Amazon API Gateway HTTP APIs with a Cognito authorizer.

C.Configure an AWS Lambda function to handle user authentication. Implement Amazon API Gateway REST APIs with a Lambda authorizer.

D.Configure an IAM user to handle user authentication. Implement Amazon API Gateway HTTP APIs with an IAM authorizer.

Answer: A

Explanation:

The correct answer is A because it offers the most operationally efficient solution for user authentication and API access.

Here's a detailed justification:

Amazon Cognito User Pools: Cognito User Pools are designed specifically for managing user directories and handling authentication. They provide a managed and scalable solution for user registration, login, and password management. They handle the complexities of user management, including security best practices, reducing operational overhead.

https://aws.amazon.com/cognito/user-pools/

Amazon API Gateway REST APIs: REST APIs offer more features and flexibility, suitable for complex API requirements, including request validation, transformation, and caching, if required for the analytics service. Although HTTP APIs are cheaper and have lower latency, REST APIs provide more control for future extensions and complex data processing.

Cognito Authorizer: API Gateway seamlessly integrates with Cognito User Pools via Cognito authorizers. This allows API Gateway to validate the JSON Web Tokens (JWTs) issued by Cognito upon successful user authentication. This removes the need for custom authentication logic within your backend services, simplifying the architecture.

https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-integrate-with-cognito.html

Why other options are less efficient:

B: While HTTP APIs are cost-effective, Cognito Identity Pools are used to grant users access to AWS resources directly (like S3) and are not the primary mechanism for API authentication like User Pools.
C: Using a Lambda function for authentication requires managing custom authentication logic, which is less efficient than leveraging Cognito. This adds operational burden regarding security and maintenance.
D: IAM users should not be used directly for end-user authentication. It's difficult to manage and scale IAM users for millions of users, and exposes AWS account keys.

Therefore, using Amazon Cognito User Pools for authentication and integrating with API Gateway REST APIs through a Cognito authorizer is the most operationally efficient and secure solution for offering a web analytics service to millions of users. It leverages managed services to handle user authentication, offloading this responsibility from the backend and reducing operational overhead.

Question: 793 **Exam Heist**

A company has a mobile app for customers. The app's data is sensitive and must be encrypted at rest. The company uses AWS Key Management Service (AWS KMS).

The company needs a solution that prevents the accidental deletion of KMS keys. The solution must use Amazon Simple Notification Service (Amazon SNS) to send an email notification to administrators when a user attempts to delete a KMS key.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create an Amazon EventBridge rule that reacts when a user tries to delete a KMS key. Configure an AWS Config rule that cancels any deletion of a KMS key. Add the AWS Config rule as a target of the EventBridge rule. Create an SNS topic that notifies the administrators.

B.Create an AWS Lambda function that has custom logic to prevent KMS key deletion. Create an Amazon CloudWatch alarm that is activated when a user tries to delete a KMS key. Create an Amazon EventBridge rule that invokes the Lambda function when the DeleteKey operation is performed. Create an SNS topic. Configure the EventBridge rule to publish an SNS message that notifies the administrators.

C.Create an Amazon EventBridge rule that reacts when the KMS DeleteKey operation is performed. Configure the rule to initiate an AWS Systems Manager Automation runbook. Configure the runbook to cancel the deletion of the KMS key. Create an SNS topic. Configure the EventBridge rule to publish an SNS message that notifies the administrators.

D.Create an AWS CloudTrail trail. Configure the trail to deliver logs to a new Amazon CloudWatch log group. Create a CloudWatch alarm based on the metric filter for the CloudWatch log group. Configure the alarm to use Amazon SNS to notify the administrators when the KMS DeleteKey operation is performed.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best solution for preventing accidental KMS key deletion with the least operational overhead, incorporating relevant cloud computing concepts and links to AWS documentation:

Option C leverages the power of EventBridge to detect the DeleteKey operation. When EventBridge identifies this event, it triggers an AWS Systems Manager Automation runbook. This runbook is designed to immediately cancel the KMS key deletion process, acting as a preventative measure. Concurrently, EventBridge publishes a message to an SNS topic, which then notifies administrators via email about the attempted deletion. This provides immediate awareness and allows for further investigation if needed.

The reason this is the preferred solution lies in its minimal operational burden. EventBridge simplifies event-driven architectures, allowing for streamlined event detection and routing. AWS Systems Manager Automation offers pre-built runbooks or the ability to create custom ones, enabling quick implementation of automated responses like stopping a deletion. The SNS integration provides a simple, scalable, and cost-effective way to deliver notifications.

Option A, while similar, utilizes AWS Config. Config's primary role is for compliance and auditing, providing a historical view of resource configurations. While Config can enforce rules, its strength isn't immediate real-time action. It's better suited for detecting configuration drift after it occurs. Trying to use Config to cancel a deletion is less straightforward and potentially less responsive than a System Manager Automation runbook directly triggered by EventBridge.

Option B introduces a Lambda function and a CloudWatch alarm. While effective, this approach adds complexity. You would need to write, deploy, and maintain the custom Lambda function, increasing operational overhead. The CloudWatch alarm also creates an additional dependency, as the Lambda would need to be triggered by the Alarm to prevent the KMS deletion

Option D uses CloudTrail logs and CloudWatch alarms. CloudTrail captures API activity, so this setup relies on detecting the DeleteKey operation after it has been logged. This means there's a delay between the attempted deletion and the notification, and the solution doesn't actively prevent the deletion from occurring. This makes it less effective than option C, which prevents the deletion directly.

In summary, Option C offers a robust and streamlined solution that effectively prevents KMS key deletion with minimal operational overhead by using EventBridge to trigger an AWS Systems Manager Automation runbook that cancels the deletion process and delivers immediate notifications via SNS. The focus on prevention rather than simple detection is the key differentiator.

Authoritative Links:

Amazon EventBridge: https://aws.amazon.com/eventbridge/
AWS Systems Manager Automation: https://aws.amazon.com/systems-manager/automation/

Amazon SNS: https://aws.amazon.com/sns/
AWS KMS: https://aws.amazon.com/kms/

Question: 794 **Exam Heist**

A company wants to analyze and generate reports to track the usage of its mobile app. The app is popular and has a global user base. The company uses a custom report building program to analyze application usage.

The program generates multiple reports during the last week of each month. The program takes less than 10 minutes to produce each report. The company rarely uses the program to generate reports outside of the last week of each month The company wants to generate reports in the least amount of time when the reports are requested.

Which solution will meet these requirements MOST cost-effectively?

A.Run the program by using Amazon EC2 On-Demand Instances. Create an Amazon EventBridge rule to start the EC2 instances when reports are requested. Run the EC2 instances continuously during the last week of each month.

B.Run the program in AWS Lambda. Create an Amazon EventBridge rule to run a Lambda function when reports are requested.

C.Run the program in Amazon Elastic Container Service (Amazon ECS). Schedule Amazon ECS to run the program when reports are requested.

D.Run the program by using Amazon EC2 Spot Instances. Create an Amazon EventBndge rule to start the EC2 instances when reports are requested. Run the EC2 instances continuously during the last week of each month.

Answer: B

Explanation:

The correct answer is B, running the program in AWS Lambda and triggering it with an Amazon EventBridge rule. Here's why:

Cost-Effectiveness: Lambda functions are cost-effective for infrequent and short-running tasks. You only pay for the compute time consumed when the Lambda function is executing. Since the report generation program runs mostly during the last week of the month and each report takes less than 10 minutes to produce, Lambda's pay-per-use model aligns perfectly with the usage pattern, minimizing costs.
Event-Driven Architecture: EventBridge is ideal for creating event-driven architectures. Using an EventBridge rule to trigger the Lambda function ensures that the report generation process is initiated only when a report is requested, eliminating the need for continuous resource provisioning and reducing costs.
Scalability and Management: Lambda automatically scales to handle concurrent requests. AWS takes care of the underlying infrastructure management, allowing the company to focus on the report generation logic rather than server maintenance.
EC2 On-Demand (A): While On-Demand EC2 instances provide predictable performance, running them continuously for a week each month would incur significant costs, even if the program is not actively generating reports.
ECS (C): ECS offers more control over the underlying infrastructure, but it also involves more operational overhead. It might be suitable for more complex or long-running tasks, which is not needed in the use-case.
EC2 Spot Instances (D): Spot instances offer cost savings, but they are subject to interruption if the spot price exceeds the bid price. Running the instances continuously during the last week of the month is also inefficient.

In contrast, Lambda provides a serverless, cost-optimized, and scalable solution for this specific report generation use case. It offers the lowest total cost of ownership (TCO) due to its pay-per-use model, minimal operational overhead, and seamless integration with EventBridge for event-driven execution.

Supporting Links:

AWS Lambda: https://aws.amazon.com/lambda/
Amazon EventBridge: https://aws.amazon.com/eventbridge/
AWS Pricing: https://aws.amazon.com/pricing/

Question: 795 **Exam Heist**

A company is designing a tightly coupled high performance computing (HPC) environment in the AWS Cloud. The company needs to include features that will optimize the HPC environment for networking and storage.

Which combination of solutions will meet these requirements? (Choose two.)

A.Create an accelerator in AWS Global Accelerator. Configure custom routing for the accelerator.
B.Create an Amazon FSx for Lustre file system. Configure the file system with scratch storage.
C.Create an Amazon CloudFront distribution. Configure the viewer protocol policy to be HTTP and HTTPS.
D.Launch Amazon EC2 instances. Attach an Elastic Fabric Adapter (EFA) to the instances.
E.Create an AWS Elastic Beanstalk deployment to manage the environment.

Answer: BD

Explanation:

Here's a detailed justification for why options B and D are the correct solutions for optimizing a tightly coupled HPC environment on AWS for networking and storage:

Option B: Create an Amazon FSx for Lustre file system. Configure the file system with scratch storage. Amazon FSx for Lustre is specifically designed for high-performance workloads like HPC. Lustre is a parallel distributed file system that provides sub-millisecond latencies and high throughput, essential for HPC applications that require rapid access to large datasets. Configuring it with scratch storage is appropriate because HPC environments often generate large amounts of temporary data during computations. Scratch storage on FSx for Lustre is ideal for this type of transient data, offering performance without the cost of long-term storage.

Option D: Launch Amazon EC2 instances. Attach an Elastic Fabric Adapter (EFA) to the instances. Elastic Fabric Adapter (EFA) is a network interface designed to accelerate HPC and machine learning applications. It enables EC2 instances to achieve lower latency and higher throughput network communication, which is critical for tightly coupled HPC workloads. EFA utilizes a custom transport protocol that bypasses the operating system kernel to directly communicate with other EFAs, significantly reducing latency and improving scalability for inter-node communication. EFA supports Scalable Reliable Datagram (SRD) protocol to improve performance of inter-instance communication.

Now, let's address why the other options are incorrect:

Option A: Create an accelerator in AWS Global Accelerator. Configure custom routing for the accelerator. AWS Global Accelerator is designed to improve the performance of applications for a global user base by routing traffic to the optimal endpoint. While it improves latency and availability, it doesn't directly address the specific networking requirements within a tightly coupled HPC cluster itself, which needs low-latency communication between compute nodes.

Option C: Create an Amazon CloudFront distribution. Configure the viewer protocol policy to be HTTP and HTTPS. Amazon CloudFront is a content delivery network (CDN) service. It is used for caching and distributing content to users worldwide. While it's valuable for making data available to users after HPC processing, it doesn't contribute to the performance of the HPC computations themselves.

Option E: Create an AWS Elastic Beanstalk deployment to manage the environment. AWS Elastic Beanstalk is a platform-as-a-service (PaaS) designed for deploying and managing web applications. While it simplifies

deployment, it doesn't offer specific networking or storage optimizations tailored for HPC environments. It is not designed to handle the specific needs of tightly coupled applications or data-intensive HPC workloads.

Authoritative Links for Further Research:

Amazon FSx for Lustre: https://aws.amazon.com/fsx/lustre/
Elastic Fabric Adapter (EFA): https://aws.amazon.com/hpc/efa/
AWS Global Accelerator: https://aws.amazon.com/global-accelerator/
Amazon CloudFront: https://aws.amazon.com/cloudfront/
AWS Elastic Beanstalk: https://aws.amazon.com/elasticbeanstalk/

Question: 796 **Exam Heist**

A company needs a solution to prevent photos with unwanted content from being uploaded to the company's web application. The solution must not involve training a machine learning (ML) model.

Which solution will meet these requirements?

A.Create and deploy a model by using Amazon SageMaker Autopilot. Create a real-time endpoint that the web application invokes when new photos are uploaded.

B.Create an AWS Lambda function that uses Amazon Rekognition to detect unwanted content. Create a Lambda function URL that the web application invokes when new photos are uploaded.

C.Create an Amazon CloudFront function that uses Amazon Comprehend to detect unwanted content. Associate the function with the web application.

D.Create an AWS Lambda function that uses Amazon Rekognition Video to detect unwanted content. Create a Lambda function URL that the web application invokes when new photos are uploaded.

Answer: B

Explanation:

The correct answer is B because it leverages Amazon Rekognition, a pre-trained AWS service, for image analysis without requiring custom ML model training. This aligns directly with the requirement of avoiding ML model training. Here's a detailed breakdown:

Option B proposes an AWS Lambda function that uses Amazon Rekognition to detect unwanted content in uploaded photos. Rekognition offers pre-built models for tasks like object and scene detection, facial analysis, and content moderation. The Lambda function acts as an intermediary, receiving the image, sending it to Rekognition for analysis, and then acting upon the results (e.g., rejecting the upload if unwanted content is detected). Creating a Lambda function URL enables a simple and direct way for the web application to invoke the function whenever a new photo is uploaded. This architecture promotes a serverless and event-driven design.

Option A is incorrect because Amazon SageMaker Autopilot specifically automates the process of building, training, and deploying machine learning models. This directly contradicts the requirement to avoid training an ML model. While SageMaker is a powerful ML tool, it's not suitable when pre-trained services can suffice.

Option C is incorrect because Amazon Comprehend is a natural language processing (NLP) service used for analyzing text, not images. It can't detect unwanted content in photos. CloudFront functions are used for lightweight processing of HTTP requests at the edge, such as modifying headers or redirecting requests.

Option D is incorrect because Amazon Rekognition Video is designed for analyzing video streams, not individual images. While it could technically process a single-frame video, it's overkill and more expensive than using Rekognition for image analysis.

In summary, option B is the most efficient and cost-effective solution because it uses a pre-trained AWS

service (Rekognition) to fulfill the requirement of preventing unwanted content in uploaded photos without requiring any ML model training. It is also a more efficient implementation utilizing a Lambda function URL that the web application can easily invoke, ensuring real-time analysis and filtering of images.

Further Reading:

Amazon Rekognition: https://aws.amazon.com/rekognition/
AWS Lambda: https://aws.amazon.com/lambda/
Lambda Function URLs: https://docs.aws.amazon.com/lambda/latest/dg/lambda-urls.html

Question: 797 **Exam Heist**

A company uses AWS to run its ecommerce platform. The platform is critical to the company's operations and has a high volume of traffic and transactions. The company configures a multi-factor authentication (MFA) device to secure its AWS account root user credentials. The company wants to ensure that it will not lose access to the root user account if the MFA device is lost.

Which solution will meet these requirements?

A.Set up a backup administrator account that the company can use to log in if the company loses the MFA device.

B.Add multiple MFA devices for the root user account to handle the disaster scenario.

C.Create a new administrator account when the company cannot access the root account.

D.Attach the administrator policy to another IAM user when the company cannot access the root account.

Answer: B

Explanation:

The correct answer is B. Adding multiple MFA devices for the root user account provides redundancy and ensures continued access even if one MFA device is lost or unavailable. Here's why:

The AWS root user has complete, unrestricted access to all AWS resources and services in an account. Securing it with MFA is a best practice. However, losing the MFA device presents a significant risk of being locked out of the entire AWS account.

Option B directly addresses this risk by implementing a contingency plan. By configuring multiple MFA devices (such as a hardware token and a virtual MFA app on a different phone), the company creates a backup access method. If the primary MFA device is lost, the company can still authenticate using the secondary device, maintaining uninterrupted access to the root user account and all its privileges. This provides resilience against the failure or loss of a single MFA device.

Option A, while seemingly a good idea, doesn't completely address the root user problem. While a backup administrator account could be useful, losing access to the root account is still a critical vulnerability. You wouldn't be able to perform all root-level tasks. Options C and D are both reactive and inadequate. Creating a new administrator account (C) or attaching admin policies to another user (D) after losing access to the root account is not a valid solution because you would need the existing root user to perform these actions. They also involve a period of downtime, which is unacceptable for a critical ecommerce platform.

By using multiple MFA devices, the company adheres to security best practices while ensuring business continuity. This proactive approach avoids a single point of failure for root user access.

Supporting Links:

AWS Documentation on Multi-Factor Authentication (MFA):
AWS Security Best Practices - IAM: Specifically, review the sections about securing the root user.

Question: 798 **Exam Heist**

A social media company is creating a rewards program website for its users. The company gives users points when users create and upload videos to the website. Users redeem their points for gifts or discounts from the company's affiliated partners. A unique ID identifies users. The partners refer to this ID to verify user eligibility for rewards.

The partners want to receive notification of user IDs through an HTTP endpoint when the company gives users points. Hundreds of vendors are interested in becoming affiliated partners every day. The company wants to design an architecture that gives the website the ability to add partners rapidly in a scalable way.

Which solution will meet these requirements with the LEAST implementation effort?

A.Create an Amazon Timestream database to keep a list of affiliated partners. Implement an AWS Lambda function to read the list. Configure the Lambda function to send user IDs to each partner when the company gives users points.

B.Create an Amazon Simple Notification Service (Amazon SNS) topic. Choose an endpoint protocol. Subscribe the partners to the topic. Publish user IDs to the topic when the company gives users points.

C.Create an AWS Step Functions state machine. Create a task for every affiliated partner. Invoke the state machine with user IDs as input when the company gives users points.

D.Create a data stream in Amazon Kinesis Data Streams. Implement producer and consumer applications. Store a list of affiliated partners in the data stream. Send user IDs when the company gives users points.

Answer: B

Explanation:

The best solution to meet the requirements of rapid partner onboarding, scalability, and minimal implementation effort is **B. Create an Amazon Simple Notification Service (Amazon SNS) topic. Choose an endpoint protocol. Subscribe the partners to the topic. Publish user IDs to the topic when the company gives users points.**

Here's why:

Scalability and Ease of Onboarding: Amazon SNS is designed for high-throughput, scalable message delivery. New partners can simply subscribe to the topic without requiring code changes on the company's side. This makes onboarding hundreds of vendors daily manageable.
Loose Coupling: SNS facilitates loose coupling between the rewards website and the partners. The website simply publishes user IDs to the SNS topic, and SNS handles the delivery to subscribed partners. This decoupling simplifies maintenance and allows partners to consume notifications independently.
Endpoint Flexibility: SNS supports various endpoint protocols (HTTP, HTTPS, email, SMS, AWS Lambda, SQS) which provides partners with flexibility in how they receive notifications. The company does not have to implement individual mechanisms for each partner's preferred method.
Minimal Implementation Effort: Setting up an SNS topic and publishing messages to it requires minimal code. The company simply needs to integrate the SNS API into their rewards system. The partners handle their subscription and consumption logic.
Alternatives are Less Efficient:

A (Timestream and Lambda): This approach requires the company to maintain a list of partners and iterate through it for each user ID, which adds complexity and overhead. Lambda has invocation limits and scaling concerns if the list of partners becomes very large.
C (Step Functions): Step Functions are designed for orchestrating complex workflows. Sending notifications to partners does not require a complex workflow. Implementing a task for each partner is not efficient and adds unnecessary complexity.
D (Kinesis Data Streams): Kinesis Data Streams is designed for real-time data processing and analytics, not for notification delivery. It requires implementing producer and consumer applications, adding significant

complexity.

Authoritative Links:

Amazon SNS: https://aws.amazon.com/sns/
SNS FAQs: https://aws.amazon.com/sns/faqs/

Question: 799 **Exam Heist**

A company needs to extract the names of ingredients from recipe records that are stored as text files in an Amazon S3 bucket. A web application will use the ingredient names to query an Amazon DynamoDB table and determine a nutrition score.

The application can handle non-food records and errors. The company does not have any employees who have machine learning knowledge to develop this solution.

Which solution will meet these requirements MOST cost-effectively?

A.Use S3 Event Notifications to invoke an AWS Lambda function when PutObject requests occur. Program the Lambda function to analyze the object and extract the ingredient names by using Amazon Comprehend. Store the Amazon Comprehend output in the DynamoDB table.

B.Use an Amazon EventBridge rule to invoke an AWS Lambda function when PutObject requests occur. Program the Lambda function to analyze the object by using Amazon Forecast to extract the ingredient names. Store the Forecast output in the DynamoDB table.

C.Use S3 Event Notifications to invoke an AWS Lambda function when PutObject requests occur. Use Amazon Polly to create audio recordings of the recipe records. Save the audio files in the S3 bucket. Use Amazon Simple Notification Service (Amazon SNS) to send a URL as a message to employees. Instruct the employees to listen to the audio files and calculate the nutrition score. Store the ingredient names in the DynamoDB table.

D.Use an Amazon EventBridge rule to invoke an AWS Lambda function when a PutObject request occurs. Program the Lambda function to analyze the object and extract the ingredient names by using Amazon SageMaker. Store the inference output from the SageMaker endpoint in the DynamoDB table.

Answer: A

Explanation:

The correct answer is A. Here's a detailed justification:

Option A: This solution leverages Amazon S3 Event Notifications, AWS Lambda, and Amazon Comprehend. S3 Event Notifications trigger the Lambda function whenever a new recipe file is uploaded (PutObject). The Lambda function then uses Amazon Comprehend, a natural language processing (NLP) service, to extract the ingredient names from the text file. Amazon Comprehend is designed for text analysis, including entity recognition, which makes it suitable for identifying ingredients. The extracted ingredient names are then stored in the DynamoDB table. This approach minimizes manual effort, cost-effectively automates the process, and requires no machine learning expertise from the company.

Why other options are incorrect:

Option B (Amazon Forecast): Amazon Forecast is for time-series forecasting, not text analysis or ingredient extraction. It is completely unsuitable for the task and would incur unnecessary costs.

Option C (Amazon Polly, Amazon SNS, Manual Calculation): This solution involves converting text to speech using Amazon Polly, notifying employees via Amazon SNS, and manually calculating the nutrition score. This is labor-intensive, error-prone, and expensive due to the high human involvement. It completely defeats the purpose of automation.

Option D (Amazon SageMaker): Amazon SageMaker is a powerful machine learning platform, but using it for a simple ingredient extraction task is overkill. It requires machine learning expertise to train and deploy a

model, which the company lacks. Furthermore, it would be significantly more expensive than using Amazon Comprehend.

Cost-Effectiveness and Automation:

Option A provides a balance between automation and cost. Amazon Comprehend offers pay-as-you-go pricing, only charging for the amount of text processed. Lambda is also cost-effective, as it only charges for the compute time used when the function is executed. This makes the solution scalable and cost-efficient for processing varying numbers of recipe records. EventBridge introduces overhead, for no increase in solution accuracy.

Relevant Cloud Computing Concepts:

Event-driven architecture: S3 Event Notifications trigger the Lambda function, enabling an event-driven architecture where services react to changes in the S3 bucket.
Serverless computing: Lambda allows the company to run code without provisioning or managing servers.
Managed services: Amazon Comprehend and DynamoDB are managed services, meaning AWS handles the underlying infrastructure and maintenance, reducing the operational overhead for the company.
Natural Language Processing (NLP): Amazon Comprehend provides NLP capabilities to extract meaning from text, which is crucial for identifying ingredients.

Authoritative Links:

Amazon S3 Event Notifications:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventNotifications.html
AWS Lambda: https://aws.amazon.com/lambda/
Amazon Comprehend: https://aws.amazon.com/comprehend/
Amazon DynamoDB: https://aws.amazon.com/dynamodb/

Question: 800 **Exam Heist**

A company needs to create an AWS Lambda function that will run in a VPC in the company's primary AWS account. The Lambda function needs to access files that the company stores in an Amazon Elastic File System (Amazon EFS) file system. The EFS file system is located in a secondary AWS account. As the company adds files to the file system, the solution must scale to meet the demand.

Which solution will meet these requirements MOST cost-effectively?

A.Create a new EFS file system in the primary account. Use AWS DataSync to copy the contents of the original EFS file system to the new EFS file system.

B.Create a VPC peering connection between the VPCs that are in the primary account and the secondary account.

C.Create a second Lambda function in the secondary account that has a mount that is configured for the file system. Use the primary account's Lambda function to invoke the secondary account's Lambda function.

D.Move the contents of the file system to a Lambda layer. Configure the Lambda layer's permissions to allow the company's secondary account to use the Lambda layer.

Answer: B

Explanation:

The correct answer is **B. Create a VPC peering connection between the VPCs that are in the primary account and the secondary account.**

Here's why:

Cost-Effectiveness: VPC peering is generally a cost-effective solution for connecting VPCs in different AWS accounts. It avoids data transfer charges that might be incurred with other methods (like copying data to a new EFS).
Direct Network Connectivity: VPC peering establishes a direct network connection between the two VPCs, enabling the Lambda function in the primary account to access the EFS file system in the secondary account as if they were in the same network.
Scalability: EFS is designed to scale automatically to meet demand. Connecting via VPC peering allows the Lambda function to leverage the scalability of the existing EFS file system.
Security: VPC peering allows you to control the traffic flow between the two VPCs using security group rules and network ACLs.

Why other options are not the most cost-effective or efficient:

A. Creating a new EFS file system and using AWS DataSync: This involves significant data transfer costs and ongoing synchronization overhead, making it less cost-effective. It also duplicates the data, increasing storage costs.
C. Creating a second Lambda function in the secondary account: This adds complexity to the architecture and introduces latency due to inter-function calls. It is not the most straightforward or efficient solution for accessing the file system.
D. Moving the contents of the file system to a Lambda layer: Lambda layers have size limitations. An entire file system is likely to exceed this limit. Furthermore, layers are read-only, preventing the Lambda function from modifying the files.

Supporting Documentation:

VPC Peering: https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html
EFS: https://aws.amazon.com/efs/
Lambda VPC: https://docs.aws.amazon.com/lambda/latest/dg/services-vpc.html

Question: 801 **Exam Heist**

A financial company needs to handle highly sensitive data. The company will store the data in an Amazon S3 bucket. The company needs to ensure that the data is encrypted in transit and at rest. The company must manage the encryption keys outside the AWS Cloud.

Which solution will meet these requirements?

A.Encrypt the data in the S3 bucket with server-side encryption (SSE) that uses an AWS Key Management Service (AWS KMS) customer managed key.

B.Encrypt the data in the S3 bucket with server-side encryption (SSE) that uses an AWS Key Management Service (AWS KMS) AWS managed key.

C.Encrypt the data in the S3 bucket with the default server-side encryption (SSE).

D.Encrypt the data at the company's data center before storing the data in the S3 bucket.

Answer: D

Explanation:

The correct solution is to encrypt the data at the company's data center before uploading it to the S3 bucket. This approach meets the requirement of managing encryption keys outside the AWS Cloud.

Options A, B, and C all involve using AWS KMS for key management. While using KMS for encryption provides robust security, it contradicts the explicit requirement of managing the encryption keys outside of AWS. These options place the key management responsibility within the AWS ecosystem, violating the stated constraint.

Option D, encrypting the data at the company's data center before storing it in S3, aligns perfectly with the requirement. By performing encryption on-premises, the company retains full control and management of the encryption keys. This is known as client-side encryption. The data is encrypted before it is transmitted to S3, satisfying the encryption in transit requirement. Additionally, the data remains encrypted at rest in the S3 bucket, fulfilling the at-rest encryption requirement. While S3 offers its own server-side encryption options, client-side encryption provides the desired key management control. The encrypted data is stored in S3, offering durability and availability without AWS ever accessing the unencrypted data or the encryption keys. This addresses the high sensitivity of the data.

Therefore, the company can ensure data encryption both in transit and at rest while maintaining complete control over the encryption keys outside the AWS environment using option D.

Further reading on client-side encryption with S3:

AWS Documentation - Protecting Data Using Client-Side Encryption:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingEncryption.html

Question: 802 **Exam Heist**

A company wants to run its payment application on AWS. The application receives payment notifications from mobile devices. Payment notifications require a basic validation before they are sent for further processing.

The backend processing application is long running and requires compute and memory to be adjusted. The company does not want to manage the infrastructure.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create an Amazon Simple Queue Service (Amazon SQS) queue. Integrate the queue with an Amazon EventBridge rule to receive payment notifications from mobile devices. Configure the rule to validate payment notifications and send the notifications to the backend application. Deploy the backend application on Amazon Elastic Kubernetes Service (Amazon EKS) Anywhere. Create a standalone cluster.

B.Create an Amazon API Gateway API. Integrate the API with an AWS Step Functions state machine to receive payment notifications from mobile devices. Invoke the state machine to validate payment notifications and send the notifications to the backend application. Deploy the backend application on Amazon Elastic Kubernetes Service (Amazon EKS). Configure an EKS cluster with self-managed nodes.

C.Create an Amazon Simple Queue Service (Amazon SQS) queue. Integrate the queue with an Amazon EventBridge rule to receive payment notifications from mobile devices. Configure the rule to validate payment notifications and send the notifications to the backend application. Deploy the backend application on Amazon EC2 Spot Instances. Configure a Spot Fleet with a default allocation strategy.

D.Create an Amazon API Gateway API. Integrate the API with AWS Lambda to receive payment notifications from mobile devices. Invoke a Lambda function to validate payment notifications and send the notifications to the backend application. Deploy the backend application on Amazon Elastic Container Service (Amazon ECS). Configure Amazon ECS with an AWS Fargate launch type.

Answer: D

Explanation:

The optimal solution leverages serverless components to minimize operational overhead. API Gateway provides a managed API endpoint for receiving payment notifications. Integrating API Gateway with Lambda allows for serverless validation of these notifications, offloading infrastructure management. Lambda functions are ideal for short-lived tasks like validation. The validated notifications can then be sent to the backend processing application.

For the backend, ECS with Fargate provides a fully managed container orchestration service. Fargate abstracts away the underlying infrastructure, eliminating the need to manage EC2 instances or Kubernetes nodes. ECS allows you to easily deploy, manage, and scale your containerized backend application without

operational burdens. It's particularly suitable for long-running applications that require compute and memory adjustments, as ECS can handle scaling based on resource utilization.

Option A introduces unnecessary complexity with EKS Anywhere and a standalone cluster, requiring significant infrastructure management. Option B, while using EKS, uses self-managed nodes, increasing operational overhead. Step Functions are better suited for orchestrating complex workflows, and are overkill for basic validation. Option C utilizes EC2 Spot Instances, which can be interrupted and require handling instance terminations, leading to increased operational overhead. Further, SQS and EventBridge are primarily for asynchronous communication, while API Gateway enables synchronous request/response interaction, which fits the requirement of validating requests before proceeding.

In summary, API Gateway, Lambda, ECS with Fargate offer the most serverless and managed approach, minimizing operational overhead while meeting the requirements of validating notifications and running a scalable backend application.

Relevant links:

API Gateway: https://aws.amazon.com/api-gateway/
Lambda: https://aws.amazon.com/lambda/
ECS: https://aws.amazon.com/ecs/
Fargate: https://aws.amazon.com/fargate/

Question: 803 **Exam Heist**

A solutions architect is designing a user authentication solution for a company. The solution must invoke two-factor authentication for users that log in from inconsistent geographical locations, IP addresses, or devices. The solution must also be able to scale up to accommodate millions of users.

Which solution will meet these requirements?

A.Configure Amazon Cognito user pools for user authentication. Enable the risk-based adaptive authentication feature with multifactor authentication (MFA).

B.Configure Amazon Cognito identity pools for user authentication. Enable multi-factor authentication (MFA).

C.Configure AWS Identity and Access Management (IAM) users for user authentication. Attach an IAM policy that allows the AllowManageOwnUserMFA action.

D.Configure AWS IAM Identity Center (AWS Single Sign-On) authentication for user authentication. Configure the permission sets to require multi-factor authentication (MFA).

Answer: A

Explanation:

The correct answer is **A. Configure Amazon Cognito user pools for user authentication. Enable the risk-based adaptive authentication feature with multifactor authentication (MFA).**

Here's a detailed justification:

Amazon Cognito User Pools are a fully managed service specifically designed for user authentication. They can easily scale to accommodate millions of users, fulfilling one of the core requirements. Cognito offers features like self-service registration, sign-in, password recovery, and user profile management, reducing the operational overhead.

Critically, Cognito user pools support risk-based adaptive authentication, a key component of the question's requirements. This feature leverages machine learning to detect unusual sign-in activity based on factors like location, IP address, and device. When suspicious activity is detected, the system can automatically trigger multi-factor authentication (MFA) to verify the user's identity. This ensures that MFA is only invoked when

needed, improving user experience while enhancing security.

Option B is incorrect because Cognito Identity Pools (Federated Identities) provide temporary AWS credentials to users who are already authenticated via an external identity provider (like Facebook, Google, or an existing corporate directory). They don't handle user authentication directly. While they support MFA, they don't offer the risk-based adaptive authentication required.

Option C is incorrect because IAM users are designed for AWS service management, not end-user authentication for applications. Managing millions of IAM users for this purpose is impractical and doesn't provide the necessary features for a scalable user authentication solution. While IAM supports MFA, it doesn't provide adaptive authentication.

Option D is incorrect because AWS IAM Identity Center (successor to AWS Single Sign-On) is designed for providing single sign-on access to multiple AWS accounts and applications for users managed within IAM Identity Center or through connected identity providers. While it supports MFA, it's not primarily designed for the direct user authentication of millions of end-users like Cognito is, and doesn't offer risk-based adaptive authentication natively. It is better suited for authenticating organization employees and granting them AWS access.

In summary, only Amazon Cognito user pools with risk-based adaptive authentication provide the necessary scalability, user management features, and the ability to selectively invoke MFA based on user risk factors, making it the best solution for the stated requirements.

Reference links:

Amazon Cognito: https://aws.amazon.com/cognito/
Amazon Cognito User Pools: https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-identity-pools.html
Risk-based adaptive authentication in Cognito: https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools-advanced-security.html

Question: 804 **Exam Heist**

A company has an Amazon S3 data lake. The company needs a solution that transforms the data from the data lake and loads the data into a data warehouse every day. The data warehouse must have massively parallel processing (MPP) capabilities.

Data analysts then need to create and train machine learning (ML) models by using SQL commands on the data. The solution must use serverless AWS services wherever possible.

Which solution will meet these requirements?

A.Run a daily Amazon EMR job to transform the data and load the data into Amazon Redshift. Use Amazon Redshift ML to create and train the ML models.

B.Run a daily Amazon EMR job to transform the data and load the data into Amazon Aurora Serverless. Use Amazon Aurora ML to create and train the ML models.

C.Run a daily AWS Glue job to transform the data and load the data into Amazon Redshift Serverless. Use Amazon Redshift ML to create and train the ML models.

D.Run a daily AWS Glue job to transform the data and load the data into Amazon Athena tables. Use Amazon Athena ML to create and train the ML models.

Answer: C

Explanation:

The correct solution is C because it optimally leverages serverless AWS services to meet the requirements.

AWS Glue is a fully managed, serverless ETL (extract, transform, load) service, perfectly suited for transforming data from the S3 data lake. It can handle various data formats and complexities, scaling automatically as needed. Amazon Redshift Serverless provides a data warehouse with MPP capabilities without the need to provision and manage infrastructure. This aligns with the serverless requirement and enables efficient querying and analysis of large datasets. Amazon Redshift ML allows data analysts to create, train, and deploy machine learning models directly from the data warehouse using SQL, simplifying the ML workflow and removing the need to move data to separate ML platforms.

Option A is less suitable because while Amazon EMR can transform data, it's not inherently serverless. Managing an EMR cluster involves more operational overhead than AWS Glue. Option B uses Amazon Aurora Serverless, which is a relational database, not a data warehouse. Aurora doesn't inherently have MPP capabilities needed for large-scale data analysis. Option D uses Amazon Athena, which is a serverless query service for S3. While serverless, Athena's ML capabilities aren't as integrated and performant as Redshift ML for data warehouse-based ML model training. Also, Athena doesn't serve as a data warehouse in this scenario.

Therefore, only option C fulfills all the requirements of a serverless architecture, data transformation, MPP data warehouse, and SQL-based ML model creation.

Supporting links:

AWS Glue: https://aws.amazon.com/glue/
Amazon Redshift Serverless: https://aws.amazon.com/redshift/serverless/
Amazon Redshift ML: https://aws.amazon.com/redshift/features/ml/

Question: 805 **Exam Heist**

A company runs containers in a Kubernetes environment in the company's local data center. The company wants to use Amazon Elastic Kubernetes Service (Amazon EKS) and other AWS managed services. Data must remain locally in the company's data center and cannot be stored in any remote site or cloud to maintain compliance.

Which solution will meet these requirements?

A.Deploy AWS Local Zones in the company's data center.
B.Use an AWS Snowmobile in the company's data center.
C.Install an AWS Outposts rack in the company's data center.
D.Install an AWS Snowball Edge Storage Optimized node in the data center.

Answer: C

Explanation:

The correct answer is **C. Install an AWS Outposts rack in the company's data center.**

Justification:

The company's primary requirement is to run containerized applications using Amazon EKS while keeping all data within their on-premises data center due to compliance restrictions. AWS Outposts directly addresses this need by extending AWS infrastructure and services to the company's physical location.

AWS Outposts: Outposts provides a fully managed and configurable compute and storage racks built with AWS-designed hardware. They are specifically designed to run AWS services locally while connecting to AWS cloud for management and control plane operations. With Outposts, the company can run Amazon EKS clusters within their data center, ensuring that Kubernetes workloads and their associated data remain on-premises.

Let's analyze why the other options are not the best fit:

A. Deploy AWS Local Zones in the company's data center: Local Zones are extensions of AWS Regions but are geographically closer to users. However, Local Zones are AWS-owned and operated. You cannot deploy them within a company's private data center. Local Zones do not allow customer-controlled data residency within the company's own infrastructure.

B. Use an AWS Snowmobile in the company's data center: AWS Snowmobile is a petabyte-scale data transfer service. It is used for migrating large datasets into or out of AWS, which doesn't fulfill the requirement of running Kubernetes workloads permanently on-premises and does not help in using EKS locally.

D. Install an AWS Snowball Edge Storage Optimized node in the data center: Snowball Edge is an edge computing, data migration, and data transport device. While Snowball Edge offers compute capabilities, it is primarily intended for temporary or intermittent workloads and data transfer. It's not a permanent solution for running EKS clusters with strict data residency requirements. Also, while it offers edge compute, it doesn't run a fully fledged EKS service.

In summary, AWS Outposts is the only solution that allows the company to use Amazon EKS locally within their data center, keeping data on-premises and adhering to their compliance requirements, by essentially extending the AWS cloud to their physical infrastructure.

Supporting Links:

AWS Outposts: https://aws.amazon.com/outposts/
Amazon EKS: https://aws.amazon.com/eks/

Question: 806 **Exam Heist**

A social media company has workloads that collect and process data. The workloads store the data in on-premises NFS storage. The data store cannot scale fast enough to meet the company's expanding business needs. The company wants to migrate the current data store to AWS.

Which solution will meet these requirements MOST cost-effectively?

A.Set up an AWS Storage Gateway Volume Gateway. Use an Amazon S3 Lifecycle policy to transition the data to the appropriate storage class.

B.Set up an AWS Storage Gateway Amazon S3 File Gateway. Use an Amazon S3 Lifecycle policy to transition the data to the appropriate storage class.

C.Use the Amazon Elastic File System (Amazon EFS) Standard-Infrequent Access (Standard-IA) storage class. Activate the infrequent access lifecycle policy.

D.Use the Amazon Elastic File System (Amazon EFS) One Zone-Infrequent Access (One Zone-IA) storage class. Activate the infrequent access lifecycle policy.

Answer: B

Explanation:

Here's a detailed justification for why option B is the most cost-effective solution, along with supporting concepts and links:

The core requirement is to migrate on-premises NFS data to AWS cost-effectively while addressing scalability issues. Amazon S3 is significantly more scalable and generally cheaper for large data storage than Amazon EFS. AWS Storage Gateway facilitates the transition from on-premises storage to AWS storage services. There are different types of Storage Gateway that each serve different purposes.

Option A (Volume Gateway): Volume Gateway provides block-based storage, mimicking a SAN in AWS. It isn't optimized for file-based data like the NFS system the company currently uses, making it unsuitable.

Option B (S3 File Gateway): S3 File Gateway presents a file system interface that connects to S3. It is a good fit for migrating the data to S3 which is cost-effective, scalable object storage. The NFS data on-premises can be easily copied to the file gateway and then to S3. S3 Lifecycle policies can then transition the data to even cheaper storage tiers such as S3 Glacier or S3 Intelligent-Tiering based on access patterns.

Option C (EFS Standard-IA): While EFS is a fully managed NFS file system in the cloud, using EFS Standard-IA is generally more expensive than S3, especially for large amounts of data where infrequent access is expected. EFS is also not ideal for large scale data storage and analysis workloads.

Option D (EFS One Zone-IA): Similar to option C, EFS is more expensive and not a perfect fit. Additionally, EFS One Zone has less availability and redundancy than EFS Standard, which might not be ideal for the company's data.

Therefore, option B provides the most cost-effective solution. S3 is a cheaper storage option than EFS, and the S3 File Gateway enables seamless data migration from NFS. Lifecycle policies further optimize costs by moving less frequently accessed data to lower-cost S3 storage tiers.

Authoritative Links:

AWS Storage Gateway: https://aws.amazon.com/storagegateway/
Amazon S3: https://aws.amazon.com/s3/
Amazon S3 Lifecycle: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-examples.html
Amazon EFS: https://aws.amazon.com/efs/

Question: 807 **Exam Heist**

A company uses high concurrency AWS Lambda functions to process a constantly increasing number of messages in a message queue during marketing events. The Lambda functions use CPU intensive code to process the messages. The company wants to reduce the compute costs and to maintain service latency for its customers.

Which solution will meet these requirements?

A.Configure reserved concurrency for the Lambda functions. Decrease the memory allocated to the Lambda functions.

B.Configure reserved concurrency for the Lambda functions. Increase the memory according to AWS Compute Optimizer recommendations.

C.Configure provisioned concurrency for the Lambda functions. Decrease the memory allocated to the Lambda functions.

D.Configure provisioned concurrency for the Lambda functions. Increase the memory according to AWS Compute Optimizer recommendations.

Answer: D

Explanation:

The correct answer is D: Configure provisioned concurrency for the Lambda functions and increase the memory according to AWS Compute Optimizer recommendations. Here's why:

Problem Analysis: The company faces high compute costs and latency issues due to CPU-intensive Lambda functions processing a constantly increasing message queue. Optimizing for cost and latency is the goal.

Provisioned Concurrency: Provisioned concurrency pre-initializes Lambda function instances, drastically reducing cold starts and improving latency, which is critical for handling constantly increasing messages and maintaining service levels during peak marketing events. Reserved concurrency, on the other hand, only limits the number of concurrent executions, not eliminating cold starts, making it a less effective solution for

latency issues.

Memory Allocation and CPU Performance: Increasing memory allocation for Lambda functions often translates to more CPU power. For CPU-intensive workloads, this can significantly improve performance and reduce execution time, thus lowering overall compute costs as the functions complete faster.

AWS Compute Optimizer: This service analyzes your AWS resource utilization and provides recommendations for optimal instance types and memory settings. Using its recommendations ensures the Lambda functions are adequately sized for the workload, maximizing performance and cost efficiency. Without this optimization, simply increasing or decreasing memory could lead to either under-utilization (wasted resources) or insufficient resources, negatively impacting performance.

Why other options are incorrect:

A: Decreasing memory decreases CPU allocation, negatively impacting CPU-intensive tasks and potentially increasing execution time and cost.
B: While increasing memory is good, reserved concurrency does not mitigate cold starts like provisioned concurrency does.
C: Decreasing memory decreases CPU allocation, negatively impacting CPU-intensive tasks and potentially increasing execution time and cost.

In summary, provisioned concurrency addresses the latency issue stemming from cold starts, while increasing memory based on Compute Optimizer recommendations ensures adequate CPU power, improving performance and reducing cost.

Authoritative Links:

AWS Lambda Provisioned Concurrency: https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html
AWS Lambda Function Configuration: https://docs.aws.amazon.com/lambda/latest/dg/configuration-function-common.html
AWS Compute Optimizer: https://aws.amazon.com/compute-optimizer/

Question: 808 **Exam Heist**

A company runs its workloads on Amazon Elastic Container Service (Amazon ECS). The container images that the ECS task definition uses need to be scanned for Common Vulnerabilities and Exposures (CVEs). New container images that are created also need to be scanned.

Which solution will meet these requirements with the FEWEST changes to the workloads?

A.Use Amazon Elastic Container Registry (Amazon ECR) as a private image repository to store the container images. Specify scan on push filters for the ECR basic scan.

B.Store the container images in an Amazon S3 bucket. Use Amazon Macie to scan the images. Use an S3 Event Notification to initiate a Macie scan for every event with an s3:ObjectCreated:Put event type.

C.Deploy the workloads to Amazon Elastic Kubernetes Service (Amazon EKS). Use Amazon Elastic Container Registry (Amazon ECR) as a private image repository. Specify scan on push filters for the ECR enhanced scan.

D.Store the container images in an Amazon S3 bucket that has versioning enabled. Configure an S3 Event Notification for s3:ObjectCreated:* events to invoke an AWS Lambda function. Configure the Lambda function to initiate an Amazon Inspector scan.

Answer: A

Explanation:

The most efficient solution to scan container images for CVEs in an ECS environment with minimal workload

changes is to leverage Amazon ECR's built-in scanning capabilities.

Option A proposes using Amazon ECR and configuring scan-on-push. ECR provides native container image storage and integrated security scanning. Setting "scan on push" automatically triggers a scan when a new image is pushed to the repository. The "basic scan" offers a cost-effective solution by providing vulnerability information based on publicly available databases. This requires minimal configuration changes to the existing ECS setup, as it primarily involves altering the image repository location to ECR and configuring scan settings. It integrates seamlessly with the existing container deployment pipeline.

Option B is incorrect because Amazon Macie is primarily designed for discovering and protecting sensitive data within S3 buckets, not for scanning container images for vulnerabilities. While technically feasible to store images in S3 and trigger scans, it is not Macie's intended use case and would require substantial custom scripting to interpret the image data and perform vulnerability assessments.

Option C is incorrect because migrating the workload to Amazon EKS introduces unnecessary complexity. While EKS is a valid container orchestration platform, switching from ECS solely for vulnerability scanning is an over-engineered solution. Furthermore, the "enhanced scan" of ECR requires AWS Inspector, which introduces further complexity for configuration, compared to the basic scan that ECR natively supports. The question requires the FEWEST changes to the workloads.

Option D is also incorrect because using S3, Lambda, and Amazon Inspector is significantly more complex than using ECR's native scanning features. This approach necessitates writing and maintaining custom Lambda code to interact with Inspector, managing S3 event triggers, and handling the scan results. This introduces more overhead and potential points of failure compared to ECR's integrated solution.

In summary, Amazon ECR with scan on push filters offers the simplest, most integrated, and least disruptive approach for scanning container images for CVEs within an ECS environment, fulfilling the requirements of the prompt. The other options involve more complex configurations and integrations.

Relevant links for further research:

Amazon ECR Image Scanning: https://docs.aws.amazon.com/AmazonECR/latest/userguide/image-scanning.html
Amazon ECS: https://aws.amazon.com/ecs/
Amazon Macie: https://aws.amazon.com/macie/
AWS Lambda: https://aws.amazon.com/lambda/
Amazon Inspector: https://aws.amazon.com/inspector/

Question: 809 **Exam Heist**

A company uses an AWS Batch job to run its end-of-day sales process. The company needs a serverless solution that will invoke a third-party reporting application when the AWS Batch job is successful. The reporting application has an HTTP API interface that uses username and password authentication.

Which solution will meet these requirements?

A.Configure an Amazon EventBridge rule to match incoming AWS Batch job SUCCEEDED events. Configure the third-party API as an EventBridge API destination with a username and password. Set the API destination as the EventBridge rule target.

B.Configure Amazon EventBridge Scheduler to match incoming AWS Batch job SUCCEEDED events. Configure an AWS Lambda function to invoke the third-party API by using a username and password. Set the Lambda function as the EventBridge rule target.

C.Configure an AWS Batch job to publish job SUCCEEDED events to an Amazon API Gateway REST API. Configure an HTTP proxy integration on the API Gateway REST API to invoke the third-party API by using a username and password.

D.Configure an AWS Batch job to publish job SUCCEEDED events to an Amazon API Gateway REST API.

Configure a proxy integration on the API Gateway REST API to an AWS Lambda function. Configure the Lambda function to invoke the third-party API by using a username and password.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct solution and why the other options are less suitable for the given scenario:

Justification for Option B:

Option B is the most suitable choice because it offers a serverless, event-driven approach to invoke the third-party reporting application upon the successful completion of the AWS Batch job. Amazon EventBridge Scheduler is designed to trigger actions based on schedules or events. In this case, it detects the SUCCEEDED event from the AWS Batch job. A Lambda function serves as the intermediary, triggered by EventBridge, handling the authentication and invocation of the third-party reporting application's HTTP API using the provided username and password. This ensures that the credentials are secure and managed within the Lambda function. This is a clean, decoupled architecture. AWS Lambda offers a serverless compute environment that scales automatically and eliminates the need for managing servers. The combination of EventBridge and Lambda allows for a flexible and cost-effective integration with the third-party application. The Lambda function is triggered only when the AWS Batch job is successful, optimizing resource utilization and costs.

Why other options are incorrect:

Option A: While EventBridge API destinations offer a way to invoke HTTP endpoints directly, using EventBridge Scheduler in this context is not accurate. EventBridge Scheduler is used for scheduling based on a cron expression and is not triggered by events directly like normal eventBridge rules. This would require configuring a complex scheduled polling mechanism which is less efficient.

Option C: While API Gateway can be used as an intermediary, having the Batch job directly publishing to an API Gateway is less ideal. The Batch job would need additional code to handle HTTP requests and responses, adding complexity to the batch process. Additionally, API Gateway's HTTP proxy integration generally doesn't handle authentication as cleanly as the Lambda approach. The credentials would either need to be embedded in the API Gateway configuration or passed along with the Batch job. It is also not serverless in the truest sense since API gateway require some settings configurations and you have to configure the Batch Job to point to it

Option D: This option introduces unnecessary complexity. While it's functional, adding API Gateway as an intermediary between the Batch job and the Lambda function doesn't provide a significant advantage. It adds overhead and complexity without a clear benefit. Directly triggering the Lambda function from EventBridge is simpler and more efficient. Similar to Option C, it introduces extra complexity.

Authoritative Links:

Amazon EventBridge: https://aws.amazon.com/eventbridge/
AWS Lambda: https://aws.amazon.com/lambda/
AWS Batch: https://aws.amazon.com/batch/
Amazon EventBridge Scheduler: https://aws.amazon.com/scheduler/

Question: 810 **Exam Heist**

A company collects and processes data from a vendor. The vendor stores its data in an Amazon RDS for MySQL database in the vendor's own AWS account. The company's VPC does not have an internet gateway, an AWS Direct Connect connection, or an AWS Site-to-Site VPN connection. The company needs to access the data that is in the

vendor database.

Which solution will meet this requirement?

A.Instruct the vendor to sign up for the AWS Hosted Connection Direct Connect Program. Use VPC peering to connect the company's VPC and the vendor's VPC.

B.Configure a client VPN connection between the company's VPC and the vendor's VPC. Use VPC peering to connect the company's VPC and the vendor's VPC.

C.Instruct the vendor to create a Network Load Balancer (NLB). Place the NLB in front of the Amazon RDS for MySQL database. Use AWS PrivateLink to integrate the company's VPC and the vendor's VPC.

D.Use AWS Transit Gateway to integrate the company's VPC and the vendor's VPC. Use VPC peering to connect the company’s VPC and the vendor's VPC.

Answer: C

Explanation:

The correct answer is C because it leverages AWS PrivateLink to securely and privately access the vendor's RDS database without exposing it to the public internet.

Here's a detailed justification:

Requirement: The company needs to access a vendor's RDS database, but the company's VPC lacks direct internet access and private connectivity options like Direct Connect or Site-to-Site VPN.
AWS PrivateLink: This service provides private connectivity between VPCs, AWS services, and on-premises networks, without exposing traffic to the public internet. It creates a private endpoint in the consumer VPC (the company's VPC) that connects to a service provider's service (the vendor's RDS database via an NLB).
Network Load Balancer (NLB): NLBs are suitable for TCP traffic and can front databases, providing high availability and scalability. Placing an NLB in front of the RDS database allows PrivateLink to access the database in a controlled manner. The vendor uses NLB to expose their RDS database to the VPC endpoint service (PrivateLink).
VPC Peering Inadequacy: VPC peering alone isn't enough because the company's VPC lacks an internet gateway, Direct Connect, or VPN connection. Without one of these, inter-VPC routing won't work. Also, even if VPC peering was set up with a route, it isn't the most secure way to expose an RDS database as the entire VPC network would be connected.
Direct Connect Overhead: Suggesting the vendor sign up for AWS Direct Connect (option A) is overly complex and expensive for a single database access requirement. Direct Connect is typically used for high-bandwidth, low-latency, dedicated connections, which isn't necessary here.
Client VPN & VPC Peering: Option B also uses VPC Peering with the added complexity of configuring client VPN. Setting up a client VPN is a more complex solution and does not align with the requirements of using a private and secure connection.
Transit Gateway & VPC Peering: Option D is less efficient than PrivateLink for this specific use case. Although Transit Gateway can connect VPCs, using PrivateLink offers a more direct and secure path for database access without unnecessary routing complexity.
Security: PrivateLink provides enhanced security by limiting the attack surface and ensuring data remains within the AWS network. The vendor controls who can access their database by accepting or rejecting connection requests to their VPC endpoint service.

In summary, option C provides the most efficient, secure, and cost-effective solution by utilizing AWS PrivateLink and NLB to create a private connection for accessing the vendor's RDS database without exposing it to the public internet or requiring complex network configurations.

Authoritative Links:

AWS PrivateLink: https://aws.amazon.com/privatelink/
Network Load Balancer: https://aws.amazon.com/elasticloadbalancing/network-load-balancer/

Question: 811 **Exam Heist**

A company wants to set up Amazon Managed Grafana as its visualization tool. The company wants to visualize data from its Amazon RDS database as one data source. The company needs a secure solution that will not expose the data over the internet.

Which solution will meet these requirements?

A.Create an Amazon Managed Grafana workspace without a VPC. Create a public endpoint for the RDS database. Configure the public endpoint as a data source in Amazon Managed Grafana.

B.Create an Amazon Managed Grafana workspace in a VPC. Create a private endpoint for the RDS database. Configure the private endpoint as a data source in Amazon Managed Grafana.

C.Create an Amazon Managed Grafana workspace without a VPCreate an AWS PrivateLink endpoint to establish a connection between Amazon Managed Grafana and Amazon RDS. Set up Amazon RDS as a data source in Amazon Managed Grafana.

D.Create an Amazon Managed Grafana workspace in a VPC. Create a public endpoint for the RDS database. Configure the public endpoint as a data source in Amazon Managed Grafana.

Answer: C

Explanation:

The correct solution is C: Create an Amazon Managed Grafana workspace without a VPC, create an AWS PrivateLink endpoint to establish a connection between Amazon Managed Grafana and Amazon RDS, and set up Amazon RDS as a data source in Amazon Managed Grafana.

Here's why:

Security and Avoiding Public Exposure: The question explicitly requires a secure solution that does not expose data over the internet. Options A and D involve creating public endpoints for the RDS database, which directly contradicts this requirement. Public endpoints are accessible from the internet, making the database vulnerable to potential security threats.

AWS PrivateLink for Private Connectivity: AWS PrivateLink provides private connectivity between VPCs and supported AWS services (including Amazon RDS) without exposing traffic to the public internet. It establishes a secure, private connection using private IP addresses. This aligns perfectly with the requirement for secure access and avoiding public exposure.

AWS PrivateLink Documentation

Amazon Managed Grafana and Data Sources: Amazon Managed Grafana needs a way to access the RDS database to visualize its data. AWS PrivateLink provides this secure channel. After establishing the PrivateLink connection, Amazon RDS can be configured as a data source within the Grafana workspace.

VPC Consideration for Grafana: While Option B suggests a Grafana workspace within a VPC, it creates a private endpoint for the RDS database, which is not necessary when using AWS PrivateLink. Option C leverages PrivateLink directly, which is more efficient for secure connectivity between Grafana and RDS, irrespective of whether the Grafana workspace is within a VPC. The key is to avoid public endpoints, and PrivateLink achieves this regardless of Grafana being inside a VPC.

In summary, Option C uses AWS PrivateLink to provide a secure, private connection between Amazon Managed Grafana and Amazon RDS, avoiding the need for public endpoints and ensuring data is not exposed over the internet. This directly addresses the security requirements of the scenario.

Question: 812 **Exam Heist**

A company hosts a data lake on Amazon S3. The data lake ingests data in Apache Parquet format from various data sources. The company uses multiple transformation steps to prepare the ingested data. The steps include filtering of anomalies, normalizing of data to standard date and time values, and generation of aggregates for analyses.

The company must store the transformed data in S3 buckets that data analysts access. The company needs a prebuilt solution for data transformation that does not require code. The solution must provide data lineage and data profiling. The company needs to share the data transformation steps with employees throughout the company.

Which solution will meet these requirements?

A.Configure an AWS Glue Studio visual canvas to transform the data. Share the transformation steps with employees by using AWS Glue jobs.

B.Configure Amazon EMR Serverless to transform the data. Share the transformation steps with employees by using EMR Serverless jobs.

C.Configure AWS Glue DataBrew to transform the data. Share the transformation steps with employees by using DataBrew recipes.

D.Create Amazon Athena tables for the data. Write Athena SQL queries to transform the data. Share the Athena SQL queries with employees.

Answer: C

Explanation:

The correct answer is C: Configure AWS Glue DataBrew to transform the data. Share the transformation steps with employees by using DataBrew recipes.

Here's a detailed justification:

AWS Glue DataBrew is a visual data preparation tool that allows users to clean and normalize data without writing code. This directly addresses the requirement of a "prebuilt solution for data transformation that does not require code." DataBrew provides a visual interface to perform tasks like filtering anomalies, normalizing data, and creating aggregates.

DataBrew inherently offers data lineage and data profiling features. Data lineage is provided by tracking the transformations applied to the data, showing how it was modified at each step. Data profiling is available within DataBrew, allowing users to understand the data's characteristics (e.g., data types, distributions, missing values) before and after transformations.

DataBrew uses "recipes" to encapsulate the transformation steps. These recipes can be easily shared with other employees, fulfilling the requirement to share transformation steps throughout the company. Sharing is facilitated by DataBrew's inherent sharing and collaboration features.

Option A, AWS Glue Studio, while visually oriented, primarily helps build ETL pipelines for data integration and requires more coding knowledge than DataBrew for complex transformations. Also, it's not focused on providing data profiling to end users.

Option B, Amazon EMR Serverless, is an on-demand data analytics runtime that provides a serverless environment for running big data frameworks such as Spark and Hive. It focuses on heavy lifting analytics and not on the no-code data profiling and transformation requested in the question. It is also not focused on allowing end-users to develop their own data cleaning routines.

Option D, Amazon Athena, allows you to query data in S3 using SQL. While Athena can be used for data transformation, it requires writing SQL queries, which violates the requirement of a code-free solution. Sharing the queries could work for sharing transformation logic, but this is not ideal for users who lack strong SQL skills. Athena doesn't natively provide robust data profiling in the same way that Glue DataBrew does.

Therefore, Glue DataBrew is the most appropriate solution because it provides a no-code environment for data transformation, offers built-in data lineage and profiling, and allows easy sharing of transformation steps via recipes.

Supporting Documentation:

AWS Glue DataBrew: https://aws.amazon.com/glue/databrew/
DataBrew Features: https://docs.aws.amazon.com/databrew/latest/dg/what-is.html

Question: 813 **Exam Heist**

A solutions architect runs a web application on multiple Amazon EC2 instances that are in individual target groups behind an Application Load Balancer (ALB). Users can reach the application through a public website.

The solutions architect wants to allow engineers to use a development version of the website to access one specific development EC2 instance to test new features for the application. The solutions architect wants to use an Amazon Route 53 hosted zone to give the engineers access to the development instance. The solution must automatically route to the development instance even if the development instance is replaced.

Which solution will meet these requirements?

A.Create an A Record for the development website that has the value set to the ALB. Create a listener rule on the ALB that forwards requests for the development website to the target group that contains the development instance.

B.Recreate the development instance with a public IP address. Create an A Record for the development website that has the value set to the public IP address of the development instance.

C.Create an A Record for the development website that has the value set to the ALB. Create a listener rule on the ALB to redirect requests for the development website to the public IP address of the development instance.

D.Place all the instances in the same target group. Create an A Record for the development website. Set the value to the ALB. Create a listener rule on the ALB that forwards requests for the development website to the target group.

Answer: A

Explanation:

Here's a detailed justification for why option A is the correct solution and why the others are not, along with supporting concepts and links:

Justification for Option A:

Option A is the most suitable because it leverages the Application Load Balancer's (ALB) capabilities for content-based routing and ensures automatic routing to the development instance even if it's replaced. It maintains a consistent DNS endpoint through Route 53 while providing a flexible routing mechanism. The A record in Route 53 pointing to the ALB ensures that all traffic destined for the development website is directed to the ALB. The ALB listener rule examines the incoming requests and, based on the hostname or path (defining the "development website"), forwards those specific requests only to the target group that contains the development EC2 instance. This configuration allows engineers to access the development instance using a dedicated subdomain or path without impacting the production application. If the development instance is replaced, only the target group membership needs updating, without changing the DNS records. This approach provides a robust, scalable, and easily maintainable solution. The other instances remain untouched, and the development instance's availability doesn't directly depend on a specific IP address.

Why other options are incorrect:

Option B: Relying on a public IP address for the development instance is not a best practice for several

reasons. Public IPs can change upon instance replacement. Directly exposing an instance with a public IP increases the attack surface and violates the principle of least privilege. It bypasses the load balancer, eliminating its benefits for health checks, scalability, and traffic distribution.
Option C: Redirecting requests to a public IP address after they hit the ALB is inefficient and unnecessary. The ALB is perfectly capable of routing directly to the appropriate instance. A redirect adds latency and complexity without providing any additional value. Like option B, using a public IP directly is less secure and harder to manage.
Option D: Placing all instances in the same target group defeats the purpose of having separate development and production environments. All instances would receive traffic indiscriminately. This removes the isolation between environments, potentially disrupting the production application during development testing.

Supporting Concepts:

Application Load Balancer (ALB): An ALB operates at the application layer (layer 7) of the OSI model, allowing for content-based routing, host-based routing, and path-based routing. It can inspect the contents of the HTTP requests (headers, URLs, etc.) and route traffic accordingly.
Target Groups: A target group is a collection of targets (e.g., EC2 instances) that receive traffic from a load balancer. You can define health checks for the targets in a target group.
Listeners and Rules: ALB listeners listen for incoming connection requests. Listener rules define how the ALB routes requests to different target groups based on conditions such as hostname, path, or HTTP headers.
Amazon Route 53: A scalable and highly available DNS web service. It translates domain names into IP addresses, enabling users to access applications using human-readable names.
A Record: A DNS record that maps a hostname to an IPv4 address.

Authoritative Links:

Application Load Balancers:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html
Target Groups for ALB: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/tutorial-application-load-balancer-target-groups.html
Listener Rules: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-listeners.html
Amazon Route 53: https://aws.amazon.com/route53/

Question: 814 **Exam Heist**

A company runs a container application on a Kubernetes cluster in the company's data center. The application uses Advanced Message Queuing Protocol (AMQP) to communicate with a message queue. The data center cannot scale fast enough to meet the company's expanding business needs. The company wants to migrate the workloads to AWS.

Which solution will meet these requirements with the LEAST operational overhead?

A.Migrate the container application to Amazon Elastic Container Service (Amazon ECS). Use Amazon Simple Queue Service (Amazon SQS) to retrieve the messages.

B.Migrate the container application to Amazon Elastic Kubernetes Service (Amazon EKS). Use Amazon MQ to retrieve the messages.

C.Use highly available Amazon EC2 instances to run the application. Use Amazon MQ to retrieve the messages.

D.Use AWS Lambda functions to run the application. Use Amazon Simple Queue Service (Amazon SQS) to retrieve the messages.

Answer: B

Explanation:

The correct answer is **B. Migrate the container application to Amazon Elastic Kubernetes Service (Amazon EKS). Use Amazon MQ to retrieve the messages.**

Here's a detailed justification:

The question emphasizes migrating a containerized application running on Kubernetes and the need to maintain AMQP compatibility with minimal operational overhead.

Amazon EKS is a managed Kubernetes service that allows the company to migrate their existing Kubernetes workloads to AWS without significant changes to their application architecture or operational practices. This aligns with the requirement of minimal operational overhead because AWS manages the Kubernetes control plane, relieving the company of this responsibility.

Amazon MQ is a managed message broker service that supports various message brokers, including RabbitMQ, which uses the AMQP protocol. By using Amazon MQ, the company can continue to use AMQP for message queuing without needing to manage the underlying infrastructure or make significant code changes. This direct AMQP compatibility is a crucial factor because the application currently relies on it.

Let's analyze why the other options are less suitable:

Option A (Amazon ECS and Amazon SQS): ECS is a container orchestration service, but Amazon SQS does not support the AMQP protocol. The application would require significant code changes to switch from AMQP to SQS's message format, increasing operational overhead and complexity.

Option C (Amazon EC2 and Amazon MQ): While EC2 instances can run the application and Amazon MQ provides AMQP support, managing EC2 instances for container applications is more complex and involves more operational overhead than using a managed Kubernetes service like EKS. The company has an existing Kubernetes setup, and replicating that manually on EC2 would be an inefficient approach.

Option D (AWS Lambda and Amazon SQS): Lambda functions might not be suitable for directly running a container application. Also, as stated before, Amazon SQS doesn't natively support AMQP.

Therefore, option B offers the best solution by leveraging a managed Kubernetes service (EKS) to minimize operational overhead while maintaining AMQP compatibility through Amazon MQ, requiring the fewest changes to the existing application architecture.

Relevant Links:

Amazon EKS: https://aws.amazon.com/eks/
Amazon MQ: https://aws.amazon.com/mq/

Question: 815 **Exam Heist**

An online gaming company hosts its platform on Amazon EC2 instances behind Network Load Balancers (NLBs) across multiple AWS Regions. The NLBs can route requests to targets over the internet. The company wants to improve the customer playing experience by reducing end-to-end load time for its global customer base.

Which solution will meet these requirements?

A.Create Application Load Balancers (ALBs) in each Region to replace the existing NLBs. Register the existing EC2 instances as targets for the ALBs in each Region.

B.Configure Amazon Route 53 to route equally weighted traffic to the NLBs in each Region.

C.Create additional NLBs and EC2 instances in other Regions where the company has large customer bases.

D.Create a standard accelerator in AWS Global Accelerator. Configure the existing NLBs as target endpoints.

Answer: D

Explanation:

The correct answer is D: Create a standard accelerator in AWS Global Accelerator. Configure the existing NLBs as target endpoints.

Here's why this solution is optimal and why the others are not:

AWS Global Accelerator (AGA) and Performance: AGA is designed to improve the availability and performance of applications for global users. It achieves this by utilizing the AWS global network infrastructure and strategically located edge locations. It routes user traffic to the closest healthy endpoint (NLB in this case) using Anycast static IP addresses. This reduces latency and improves end-to-end load time by minimizing the distance data travels over the public internet. A standard accelerator will provide the performance benefits needed.

Why other options are incorrect:

A (ALBs instead of NLBs): While Application Load Balancers (ALBs) offer advanced features like content-based routing, they don't inherently reduce latency for a global user base in the same way as AGA. Replacing NLBs with ALBs alone would not directly address the issue of minimizing distance over the public internet.
B (Route 53 weighted routing): Amazon Route 53 with weighted routing balances traffic across Regions, but it doesn't optimize network paths to minimize latency in the same way as AGA. It simply distributes traffic, not necessarily to the closest or best-performing endpoint for each user. Therefore, it won't significantly improve end-to-end load time. Also, Route 53 relies on DNS resolution which adds overhead and is not as dynamic as Global Accelerator.
C (Additional NLBs and EC2 in more Regions): This would require a significant infrastructure investment and ongoing operational overhead. While it could potentially improve latency for some users closer to the new Regions, it doesn't offer the same level of global network optimization as AGA. It's also not as cost-effective as leveraging the existing NLBs with AGA.

Global Accelerator Benefits: AGA offers features like:

Static Anycast IP Addresses: Provide a fixed entry point to your application, simplifying DNS management and improving reliability.
Traffic Dial: Allows you to control the percentage of traffic directed to each endpoint.
Failover: Automatically detects unhealthy endpoints and reroutes traffic to healthy ones.
Global network usage: Utilizes AWS global network infrastructure providing cost-effective, quick, and performant routing.

In summary, AWS Global Accelerator provides the most efficient and scalable solution to improve the customer playing experience by reducing end-to-end load time for a global customer base, leveraging the existing NLB infrastructure. It provides faster performance, lower costs, easier management, and better scalability than the other options.

Supporting Links:

AWS Global Accelerator
Network Load Balancer
Application Load Balancer
Amazon Route 53

Question: 816 **Exam Heist**

A company has an on-premises application that uses SFTP to collect financial data from multiple vendors. The company is migrating to the AWS Cloud. The company has created an application that uses Amazon S3 APIs to upload files from vendors.

Some vendors run their systems on legacy applications that do not support S3 APIs. The vendors want to continue to use SFTP-based applications to upload data. The company wants to use managed services for the needs of the vendors that use legacy applications.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create an AWS Database Migration Service (AWS DMS) instance to replicate data from the storage of the vendors that use legacy applications to Amazon S3. Provide the vendors with the credentials to access the AWS DMS instance.

B.Create an AWS Transfer Family endpoint for vendors that use legacy applications.

C.Configure an Amazon EC2 instance to run an SFTP server. Instruct the vendors that use legacy applications to use the SFTP server to upload data.

D.Configure an Amazon S3 File Gateway for vendors that use legacy applications to upload files to an SMB file share.

Answer: B

Explanation:

The correct answer is **B. Create an AWS Transfer Family endpoint for vendors that use legacy applications.**

Here's why:

AWS Transfer Family is a fully managed service specifically designed to facilitate secure file transfers into and out of Amazon S3, Amazon EFS, and Amazon FSx for Windows File Server using protocols like SFTP, FTPS, and FTP. This aligns perfectly with the requirement of supporting legacy SFTP-based applications while integrating with S3 for the company's cloud-native application. The key benefit is that it's a managed service, relieving the company of the operational burden of managing servers, patching, and scaling the SFTP infrastructure. Vendors can continue using their existing SFTP clients to transfer files directly to S3, simplifying their workflow.

Option A, using AWS DMS, is incorrect because AWS DMS is primarily designed for database migrations, not file transfers. It's not the right tool for this use case, and providing vendors access to a DMS instance would be inappropriate and create security risks.

Option C, configuring an EC2 instance to run an SFTP server, introduces significant operational overhead. The company would be responsible for managing the EC2 instance, ensuring its security, scaling it to handle traffic, and maintaining the SFTP server software. This negates the benefit of using managed services.

Option D, using Amazon S3 File Gateway, involves creating an SMB file share. While File Gateway can integrate on-premises applications with S3, it requires the vendors to adapt to a new protocol (SMB) when they currently use SFTP. This doesn't align with the requirement of supporting their existing SFTP-based applications. Furthermore, it's designed more for on-premises access to cloud storage, rather than directly facilitating vendor uploads. The operational overhead is also higher than using AWS Transfer Family.

Therefore, AWS Transfer Family provides the most straightforward, secure, and managed solution to bridge the gap between legacy SFTP systems and the cloud-native S3 environment with minimal operational effort.

Supporting Links:

AWS Transfer Family: https://aws.amazon.com/transfer/

Question: 817 **Exam Heist**

A marketing team wants to build a campaign for an upcoming multi-sport event. The team has news reports from the past five years in PDF format. The team needs a solution to extract insights about the content and the

sentiment of the news reports. The solution must use Amazon Textract to process the news reports.

Which solution will meet these requirements with the LEAST operational overhead?

A.Provide the extracted insights to Amazon Athena for analysis. Store the extracted insights and analysis in an Amazon S3 bucket.

B.Store the extracted insights in an Amazon DynamoDB table. Use Amazon SageMaker to build a sentiment model.

C.Provide the extracted insights to Amazon Comprehend for analysis. Save the analysis to an Amazon S3 bucket.

D.Store the extracted insights in an Amazon S3 bucket. Use Amazon QuickSight to visualize and analyze the data.

Answer: C

Explanation:

The correct answer is **C. Provide the extracted insights to Amazon Comprehend for analysis. Save the analysis to an Amazon S3 bucket.**

Here's why:

Amazon Comprehend for Sentiment Analysis: Amazon Comprehend is a natural language processing (NLP) service specifically designed for tasks like sentiment analysis and key phrase extraction. It's purpose-built for analyzing text and identifying the overall sentiment (positive, negative, or neutral) as well as extracting relevant entities and topics. This directly fulfills the requirement to extract insights and sentiments.
Least Operational Overhead: Comprehend is a managed service, meaning AWS handles the underlying infrastructure, scaling, and maintenance. You simply provide the text, and Comprehend returns the analysis results. This minimizes operational overhead.
S3 for Storage: Storing the analysis results in an S3 bucket is a cost-effective and scalable way to persist the data. It's also readily accessible for further reporting or integration with other AWS services.

Let's examine why the other options are less suitable:

A. Amazon Athena and S3: While Athena can query data in S3, it's primarily a query service for structured data. The extracted insights from Textract might not be readily structured for Athena without additional processing. It also doesn't directly address the sentiment analysis requirement.
B. Amazon DynamoDB and SageMaker: Storing extracted insights in DynamoDB is a viable option. However, building a sentiment model with SageMaker is an overkill. Comprehend is already providing managed sentiment analysis and entails more development and maintenance.
D. Amazon S3 and QuickSight: Storing extracted insights in S3 is reasonable. However, QuickSight is primarily a visualization tool and doesn't provide built-in sentiment analysis capabilities. It would require additional processing steps to perform sentiment analysis on the extracted text before visualization.

In summary, Option C leverages Amazon Comprehend, a managed service designed specifically for sentiment analysis, thereby minimizing operational overhead and directly addressing the requirements of the question. The S3 bucket serves as a simple and scalable storage solution for the analysis results.

Authoritative Links:

Amazon Comprehend: https://aws.amazon.com/comprehend/
Amazon Textract: https://aws.amazon.com/textract/
Amazon S3: https://aws.amazon.com/s3/

Question: 818 **Exam Heist**

A company's application runs on Amazon EC2 instances that are in multiple Availability Zones. The application needs to ingest real-time data from third-party applications.

The company needs a data ingestion solution that places the ingested raw data in an Amazon S3 bucket.

Which solution will meet these requirements?

A.Create Amazon Kinesis data streams for data ingestion. Create Amazon Kinesis Data Firehose delivery streams to consume the Kinesis data streams. Specify the S3 bucket as the destination of the delivery streams.

B.Create database migration tasks in AWS Database Migration Service (AWS DMS). Specify replication instances of the EC2 instances as the source endpoints. Specify the S3 bucket as the target endpoint. Set the migration type to migrate existing data and replicate ongoing changes.

C.Create and configure AWS DataSync agents on the EC2 instances. Configure DataSync tasks to transfer data from the EC2 instances to the S3 bucket.

D.Create an AWS Direct Connect connection to the application for data ingestion. Create Amazon Kinesis Data Firehose delivery streams to consume direct PUT operations from the application. Specify the S3 bucket as the destination of the delivery streams.

Answer: A

Explanation:

The correct answer is A. Here's why:

Option A utilizes Amazon Kinesis Data Streams for real-time data ingestion. Kinesis Data Streams are designed for high-throughput, real-time data streaming from multiple sources. Each EC2 instance can feed its data into the Kinesis Data Stream. Then, Amazon Kinesis Data Firehose is employed to consume the data from the Kinesis Data Stream and reliably deliver it to an S3 bucket. Kinesis Data Firehose offers seamless integration with S3, automatically handling data buffering, compression, and encryption. This setup efficiently and reliably captures the real-time data and stores it in S3 for further processing or analysis.

Option B, AWS DMS, is generally used for migrating databases, not for ingesting real-time streams of raw data from applications. Using replication instances as the source endpoints doesn't align with the requirement of handling real-time data from multiple EC2 instances.https://aws.amazon.com/dms/

Option C, AWS DataSync, focuses on transferring large amounts of data between on-premises storage and AWS storage services. It's not designed for real-time data ingestion. It's also not the most efficient solution for numerous, continuous updates from multiple instances.https://aws.amazon.com/datasync/

Option D, AWS Direct Connect, establishes a dedicated network connection from your on-premises environment to AWS. While it improves network performance, it is not necessary or cost-effective for ingesting data from EC2 instances already running within AWS. Direct PUT operations might be possible but would require significant custom implementation compared to a managed streaming service like Kinesis.https://aws.amazon.com/directconnect/

Therefore, the Kinesis Data Streams and Kinesis Data Firehose combination provides the most efficient, scalable, and reliable solution for real-time data ingestion from multiple sources (EC2 instances) into an S3 bucket, making Option A the best choice. Kinesis is built precisely for real-time data processing and seamless integration with S3.https://aws.amazon.com/kinesis/data-streams/https://aws.amazon.com/kinesis/data-firehose/

Question: 819 **Exam Heist**

A company’s application is receiving data from multiple data sources. The size of the data varies and is expected to increase over time. The current maximum size is 700 KB. The data volume and data size continue to grow as more data sources are added.

The company decides to use Amazon DynamoDB as the primary database for the application. A solutions architect needs to identify a solution that handles the large data sizes.

Which solution will meet these requirements in the MOST operationally efficient way?

A.Create an AWS Lambda function to filter the data that exceeds DynamoDB item size limits. Store the larger data in an Amazon DocumentDB (with MongoDB compatibility) database.

B.Store the large data as objects in an Amazon S3 bucket. In a DynamoDB table, create an item that has an attribute that points to the S3 URL of the data.

C.Split all incoming large data into a collection of items that have the same partition key. Write the data to a DynamoDB table in a single operation by using the BatchWriteItem API operation.

D.Create an AWS Lambda function that uses gzip compression to compress the large objects as they are written to a DynamoDB table.

Answer: B

Explanation:

The correct answer is **B. Store the large data as objects in an Amazon S3 bucket. In a DynamoDB table, create an item that has an attribute that points to the S3 URL of the data.**

Here's why this solution is the most operationally efficient:

DynamoDB Item Size Limit: DynamoDB has a hard limit of 400 KB per item. Directly storing data exceeding this limit will fail.
Offloading Large Objects to S3: Amazon S3 is designed for storing large, unstructured data. S3 offers virtually unlimited storage capacity and excellent durability and availability. By storing the large data in S3, we bypass DynamoDB's size limitations for individual items.
Reference in DynamoDB: Storing a pointer (S3 URL) in DynamoDB allows you to maintain metadata or identifiers related to the large data. This keeps DynamoDB items within the size limits while providing a link to the actual data.
Operational Efficiency: This approach minimizes the complexity of data manipulation. No complex splitting, filtering, or compression is required. S3 handles storage and retrieval efficiently.
Cost Optimization: S3 storage is generally cheaper than DynamoDB storage, especially for large data volumes.
Scalability and Performance: S3 is highly scalable and offers excellent performance for data retrieval. DynamoDB provides fast access to the metadata.

Why other options are less suitable:

A: Using DocumentDB introduces a separate database technology, increasing operational overhead (managing two databases). Filtering data to fit DynamoDB limits might result in data loss.
C: Splitting data into multiple DynamoDB items requires careful management of the split data, potentially impacting performance. This approach is complex and adds significant overhead. BatchWriteItem also has its limitations.
D: Compression can help, but it might not always reduce the data size to below the 400 KB limit. It also adds computational overhead to both write and read operations. It is possible to use compression, but a solution to the issue would require combining it with S3 to store the data.

Authoritative Links:

Amazon DynamoDB Limits:
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/ServiceQuotas.html
Amazon S3: https://aws.amazon.com/s3/

Question: 820 **Exam Heist**

A company is migrating a legacy application from an on-premises data center to AWS. The application relies on hundreds of cron jobs that run between 1 and 20 minutes on different recurring schedules throughout the day.

The company wants a solution to schedule and run the cron jobs on AWS with minimal refactoring. The solution must support running the cron jobs in response to an event in the future.

Which solution will meet these requirements?

A.Create a container image for the cron jobs. Use Amazon EventBridge Scheduler to create a recurring schedule. Run the cron job tasks as AWS Lambda functions.

B.Create a container image for the cron jobs. Use AWS Batch on Amazon Elastic Container Service (Amazon ECS) with a scheduling policy to run the cron jobs.

C.Create a container image for the cron jobs. Use Amazon EventBridge Scheduler to create a recurring schedule. Run the cron job tasks on AWS Fargate.

D.Create a container image for the cron jobs. Create a workflow in AWS Step Functions that uses a Wait state to run the cron jobs at a specified time. Use the RunTask action to run the cron job tasks on AWS Fargate.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best solution for migrating cron jobs to AWS, along with supporting concepts and links:

The core requirement is to migrate existing cron jobs to AWS with minimal refactoring, supporting future event-driven triggers, and accommodating runtimes from 1 to 20 minutes. Let's analyze each option:

Option A (EventBridge Scheduler + Lambda): Lambda functions have execution time limits. While Lambda functions are suitable for short tasks, they are typically limited to a maximum execution duration (currently 15 minutes). Many cron jobs exceed this limit, making Lambda unsuitable.https://docs.aws.amazon.com/lambda/latest/dg/services-cloudwatchevents.html

Option B (AWS Batch + ECS): AWS Batch is designed for batch processing jobs, particularly those with dependencies or complex workflows. While it could handle cron jobs, it introduces unnecessary complexity and overhead for simple scheduled tasks. Event-driven scheduling isn't inherent, requiring additional configuration. The core problem is its inherent design for background jobs rather than scheduled, time-sensitive tasks.https://docs.aws.amazon.com/batch/latest/userguide/what-is-batch.html

Option C (EventBridge Scheduler + Fargate): Amazon EventBridge Scheduler allows you to create schedules that trigger targets on a recurring basis. AWS Fargate is a serverless compute engine for containers that lets you run containers without managing servers or clusters. It's an ideal compute platform for running containerized cron jobs of various durations. Packaging the cron jobs into containers offers flexibility and reduces refactoring. EventBridge Scheduler supports triggering jobs based on schedules or in response to other events, fulfilling the "event in the future" requirement. This approach provides a simple, scalable, and cost-effective solution for running cron jobs in the cloud.https://aws.amazon.com/fargate/https://aws.amazon.com/eventbridge/scheduler/

Option D (Step Functions + Wait State + Fargate): Step Functions could be used, but introducing a Step Function workflow with Wait states for simple scheduling adds significant complexity. It's an overkill solution when a simpler approach like EventBridge Scheduler can directly trigger tasks. The Wait state isn't ideal for reliable scheduling.https://aws.amazon.com/step-functions/

Therefore, option C provides the most straightforward and appropriate solution by leveraging EventBridge Scheduler for cron job scheduling and AWS Fargate for running the containerized jobs, which accommodates the specified runtime, minimal refactoring, and future event-driven needs, making it the best choice.

Question: 821 **Exam Heist**

A company uses Salesforce. The company needs to load existing data and ongoing data changes from Salesforce to Amazon Redshift for analysis. The company does not want the data to travel over the public internet.

Which solution will meet these requirements with the LEAST development effort?

A.Establish a VPN connection from the VPC to Salesforce. Use AWS Glue DataBrew to transfer data.

B.Establish an AWS Direct Connect connection from the VPC to Salesforce. Use AWS Glue DataBrew to transfer data.

C.Create an AWS PrivateLink connection in the VPC to Salesforce. Use Amazon AppFlow to transfer data.

D.Create a VPC peering connection to Salesforce. Use Amazon AppFlow to transfer data.

Answer: C

Explanation:

The correct answer is C because it provides the most secure and efficient solution for transferring data from Salesforce to Amazon Redshift without traversing the public internet, with minimal development effort.

Here's a detailed justification:

AWS PrivateLink: This service allows you to access services hosted on the AWS network or by other AWS customers in a secure and private manner. It establishes a private connection between your VPC and the service without exposing your traffic to the public internet. Salesforce supports AWS PrivateLink.

Amazon AppFlow: This is a fully managed integration service that enables you to securely transfer data between SaaS applications (like Salesforce) and AWS services (like Amazon Redshift). It's designed for ease of use and requires minimal coding. AppFlow integrates seamlessly with PrivateLink for secure data transfer.

https://aws.amazon.com/privatelink/
https://aws.amazon.com/appflow/

Now, let's examine why the other options are less suitable:

A. VPN connection: While a VPN provides a secure connection, it's more complex to set up and manage than PrivateLink. It also involves more overhead in terms of performance and configuration. Furthermore, AWS Glue DataBrew is primarily a data preparation tool, not the optimal choice for transferring data from Salesforce to Redshift. DataBrew does not natively support direct connections to SaaS applications through private connections.

B. AWS Direct Connect: Direct Connect provides a dedicated network connection between your on-premises environment and AWS. While it avoids the public internet, it's more expensive and requires a physical connection to AWS, which might be overkill for this use case involving a SaaS application like Salesforce. Using DataBrew here still has the same limitation as in option A.

D. VPC peering connection: VPC peering connections are designed to connect VPCs within AWS. You cannot establish a direct VPC peering connection with Salesforce.

Therefore, option C offers the ideal combination of security (using PrivateLink), ease of use (using AppFlow), and cost-effectiveness for the specified requirements. It requires the least amount of development effort by leveraging managed services specifically designed for these types of integration scenarios.

Question: 822 **Exam Heist**

A company recently migrated its application to AWS. The application runs on Amazon EC2 Linux instances in an Auto Scaling group across multiple Availability Zones. The application stores data in an Amazon Elastic File System (Amazon EFS) file system that uses EFS Standard-Infrequent Access storage. The application indexes the company's files. The index is stored in an Amazon RDS database.

The company needs to optimize storage costs with some application and services changes.

Which solution will meet these requirements MOST cost-effectively?

A.Create an Amazon S3 bucket that uses an Intelligent-Tiering lifecycle policy. Copy all files to the S3 bucket. Update the application to use Amazon S3 API to store and retrieve files.

B.Deploy Amazon FSx for Windows File Server file shares. Update the application to use CIFS protocol to store and retrieve files.

C.Deploy Amazon FSx for OpenZFS file system shares. Update the application to use the new mount point to store and retrieve files.

D.Create an Amazon S3 bucket that uses S3 Glacier Flexible Retrieval. Copy all files to the S3 bucket. Update the application to use Amazon S3 API to store and retrieve files as standard retrievals.

Answer: A

Explanation:

Here's a detailed justification for why option A is the most cost-effective solution for the given scenario:

The primary goal is to reduce storage costs while maintaining application functionality after a migration to AWS. The current setup uses EFS Standard-Infrequent Access, which, while cheaper than EFS Standard, can still be relatively expensive compared to Amazon S3, particularly for infrequently accessed data. The key to cost optimization lies in identifying a storage solution that automatically tiers data based on access patterns and integrates well with the application.

Option A leverages Amazon S3 with Intelligent-Tiering. Intelligent-Tiering automatically moves data between frequent, infrequent, and archive access tiers based on usage patterns, without any operational overhead or impact on performance. [https://aws.amazon.com/s3/storage-classes/intelligent-tiering/]. By copying the files to S3 and updating the application to use the Amazon S3 API, the company benefits from the lower storage costs of S3 and the automated tiering capabilities of Intelligent-Tiering. This ensures that frequently accessed files are readily available while infrequently accessed files are stored at a lower cost.

Option B, deploying Amazon FSx for Windows File Server, is less cost-effective because FSx for Windows File Server is generally more expensive than S3. It's more suited for Windows-based applications requiring native Windows file system compatibility. The migration effort is also significant.

Option C, using Amazon FSx for OpenZFS, offers high performance but is also generally more expensive than S3. The complexity of setting up and managing an FSx for OpenZFS file system adds operational overhead.

Option D, using S3 Glacier Flexible Retrieval, is not the most optimal solution despite its very low cost. S3 Glacier Flexible Retrieval (formerly known as S3 Glacier) is suitable for archival data with infrequent access and retrieval times can be several hours. This is likely to impact the application's performance and may not meet the company's requirements for readily accessible files, even if accessed infrequently. Furthermore, "standard retrievals" from Glacier are designed for infrequent, bulk access, not the type of on-demand access the application may still require for some files. [https://aws.amazon.com/s3/storage-classes/glacier/].

Therefore, migrating to S3 with Intelligent-Tiering is the most cost-effective solution because it provides a balance between storage cost, performance, and ease of integration with the existing application after a simple API update. It directly addresses the company's need to optimize storage costs without significant application rework or performance degradation.

Question: 823 **Exam Heist**

A robotics company is designing a solution for medical surgery. The robots will use advanced sensors, cameras, and AI algorithms to perceive their environment and to complete surgeries.

The company needs a public load balancer in the AWS Cloud that will ensure seamless communication with backend services. The load balancer must be capable of routing traffic based on the query strings to different target groups. The traffic must also be encrypted.

Which solution will meet these requirements?

A.Use a Network Load Balancer with a certificate attached from AWS Certificate Manager (ACM). Use query parameter-based routing.

B.Use a Gateway Load Balancer. Import a generated certificate in AWS Identity and Access Management (IAM). Attach the certificate to the load balancer. Use HTTP path-based routing.

C.Use an Application Load Balancer with a certificate attached from AWS Certificate Manager (ACM). Use query parameter-based routing.

D.Use a Network Load Balancer. Import a generated certificate in AWS Identity and Access Management (IAM). Attach the certificate to the load balancer. Use query parameter-based routing.

Answer: C

Explanation:

The correct answer is C: Use an Application Load Balancer (ALB) with a certificate attached from AWS Certificate Manager (ACM). Use query parameter-based routing.

Here's why:

Application Load Balancer (ALB): ALBs operate at the application layer (Layer 7) of the OSI model. This allows them to make routing decisions based on the content of the HTTP requests, including query strings. Network Load Balancers (NLBs) operate at Layer 4 (transport layer) and are not capable of query string-based routing. Gateway Load Balancers are designed for virtual appliances and are not appropriate for this scenario.

Query Parameter-Based Routing: The requirement specifies that the load balancer must route traffic based on query strings. ALBs are explicitly designed to support this type of routing, allowing you to direct requests to different target groups based on the values in the query parameters.

Encryption (HTTPS): The requirement states that traffic must be encrypted. ALBs support HTTPS listeners, allowing you to encrypt traffic between clients and the load balancer. AWS Certificate Manager (ACM) is the recommended service for provisioning, managing, and deploying SSL/TLS certificates for use with AWS services, including ALBs. Importing certificates into IAM is less secure than ACM.

ACM Integration: ACM simplifies the process of obtaining and managing SSL/TLS certificates for use with AWS services. The ALB can directly integrate with ACM to automatically handle certificate renewal and deployment, reducing operational overhead.

Option A and D are incorrect because NLBs do not support query parameter-based routing.

Option B is incorrect as it is not designed to be used with Application load balancing.

Supporting Documentation:

Application Load Balancers:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html
ALB Listener Rules: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-listeners.html

AWS Certificate Manager: https://aws.amazon.com/certificate-manager/

Question: 824 **Exam Heist**

A company has an application that runs on a single Amazon EC2 instance. The application uses a MySQL database that runs on the same EC2 instance. The company needs a highly available and automatically scalable solution to handle increased traffic.

Which solution will meet these requirements?

A.Deploy the application to EC2 instances that run in an Auto Scaling group behind an Application Load Balancer. Create an Amazon Redshift cluster that has multiple MySQL-compatible nodes.

B.Deploy the application to EC2 instances that are configured as a target group behind an Application Load Balancer. Create an Amazon RDS for MySQL cluster that has multiple instances.

C.Deploy the application to EC2 instances that run in an Auto Scaling group behind an Application Load Balancer. Create an Amazon Aurora Serverless MySQL cluster for the database layer.

D.Deploy the application to EC2 instances that are configured as a target group behind an Application Load Balancer. Create an Amazon ElastiCache for Redis cluster that uses the MySQL connector.

Answer: C

Explanation:

The optimal solution involves distributing the application and database across multiple, scalable resources for high availability and automatic scaling. Option C achieves this effectively.

Here's a detailed justification:

1. **Application Scalability:** Deploying the application on EC2 instances within an Auto Scaling group ensures automatic scaling. As traffic increases, the Auto Scaling group automatically launches more EC2 instances to handle the load, and when traffic decreases, it scales down, optimizing costs. An Application Load Balancer (ALB) distributes incoming traffic across these EC2 instances, providing a single point of entry and improving application availability.https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.htmlhttps://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html

2. **Database Scalability and High Availability:** Moving the MySQL database to Amazon Aurora Serverless MySQL provides automatic scaling and high availability. Aurora Serverless automatically scales the database capacity based on application demand, so you only pay for what you use. It also offers built-in fault tolerance and automatic recovery, ensuring high availability.

   https://aws.amazon.com/rds/aurora/serverless/

3. **Why other options are incorrect:**

   A: While using Auto Scaling group and ALB is correct for the application tier, Amazon Redshift is a data warehouse service and not suitable for transactional workloads of a MySQL database.
   B: Target groups configured behind an ALB are acceptable, but the description is less precise. Moreover, while Amazon RDS for MySQL cluster offers HA, it may require more manual intervention for scaling than Aurora Serverless.
   D: ElastiCache is a caching service and not a replacement for a relational database like MySQL. It is not designed to be used with MySQL connector for primary data storage.

Therefore, Option C is the most appropriate solution because it combines automatic scaling and high availability for both the application and the database layers, meeting the company's requirements effectively.

Question: 825 **Exam Heist**

A company is planning to migrate data to an Amazon S3 bucket. The data must be encrypted at rest within the S3 bucket. The encryption key must be rotated automatically every year.

Which solution will meet these requirements with the LEAST operational overhead?

A.Migrate the data to the S3 bucket. Use server-side encryption with Amazon S3 managed keys (SSE-S3). Use the built-in key rotation behavior of SSE-S3 encryption keys.

B.Create an AWS Key Management Service (AWS KMS) customer managed key. Enable automatic key rotation. Set the S3 bucket's default encryption behavior to use the customer managed KMS key. Migrate the data to the S3 bucket.

C.Create an AWS Key Management Service (AWS KMS) customer managed key. Set the S3 bucket's default encryption behavior to use the customer managed KMS key. Migrate the data to the S3 bucket. Manually rotate the KMS key every year.

D.Use customer key material to encrypt the data. Migrate the data to the S3 bucket. Create an AWS Key Management Service (AWS KMS) key without key material. Import the customer key material into the KMS key. Enable automatic key rotation.

Answer: A

Explanation:

The correct answer is A because it offers the least operational overhead while meeting the encryption and key rotation requirements. Server-Side Encryption with Amazon S3-Managed Keys (SSE-S3) automatically handles key rotation without any manual intervention. Amazon S3 rotates these keys regularly, ensuring data is protected by new encryption keys periodically. This aligns directly with the requirement for annual key rotation.

Option B involves AWS KMS customer-managed keys with automatic key rotation, which although viable, introduces more complexity. Managing KMS keys, even with automatic rotation, requires more overhead than simply relying on S3's built-in encryption.

Option C requires manual key rotation, which contradicts the requirement for minimal operational overhead. Manual rotation is prone to human error and necessitates scheduled tasks and monitoring.

Option D is overly complex. Importing customer key material into KMS keys adds a significant layer of operational overhead. While it allows for key rotation, it's much more difficult to manage than S3-managed keys and is not necessary given the scenario's requirements.

SSE-S3 simplifies the process by abstracting the key management away from the user, adhering to security best practices and reducing the operational burden. It ensures that data at rest in the S3 bucket is encrypted and that key rotation occurs automatically without any administrator intervention.

Therefore, leveraging SSE-S3 is the most straightforward and efficient approach, aligning with the principle of least privilege and minimal operational complexity in cloud solutions.

Further reading:

Protecting Data Using Server-Side Encryption
AWS Key Management Service (KMS)
SSE-S3

Question: 826 **Exam Heist**

A company is migrating applications from an on-premises Microsoft Active Directory that the company manages to

AWS. The company deploys the applications in multiple AWS accounts. The company uses AWS Organizations to manage the accounts centrally.

The company's security team needs a single sign-on solution across all the company's AWS accounts. The company must continue to manage users and groups that are in the on-premises Active Directory.

Which solution will meet these requirements?

A.Create an Enterprise Edition Active Directory in AWS Directory Service for Microsoft Active Directory. Configure the Active Directory to be the identity source for AWS IAM Identity Center.

B.Enable AWS IAM Identity Center. Configure a two-way forest trust relationship to connect the company's self-managed Active Directory with IAM Identity Center by using AWS Directory Service for Microsoft Active Directory.

C.Use AWS Directory Service and create a two-way trust relationship with the company's self-managed Active Directory.

D.Deploy an identity provider (IdP) on Amazon EC2. Link the IdP as an identity source within AWS IAM Identity Center.

Answer: B

Explanation:

Option B is the correct solution because it leverages AWS IAM Identity Center (successor to AWS SSO) for centralized single sign-on and integrates seamlessly with the existing on-premises Active Directory. IAM Identity Center enables users to access multiple AWS accounts with a single set of credentials. Establishing a two-way forest trust using AWS Directory Service for Microsoft Active Directory (AD Connector or a managed AD) allows IAM Identity Center to authenticate users directly against the on-premises AD. This avoids replicating user data to AWS and maintains the on-premises AD as the source of truth.

Option A is incorrect because creating an Enterprise Edition Active Directory in AWS Directory Service and making it the identity source for IAM Identity Center would require migrating users and groups to the AWS-managed AD, which contradicts the requirement to continue managing users and groups in the on-premises AD.

Option C is insufficient. While creating a two-way trust relationship with AWS Directory Service is necessary for integration, it doesn't provide the single sign-on functionality across multiple accounts. This option lacks the IAM Identity Center component for centralized access management.

Option D is more complex and less efficient. Deploying an IdP on EC2 introduces unnecessary management overhead and complexity. IAM Identity Center is designed for native integration with identity providers, and using AWS Directory Service in conjunction with IAM Identity Center offers a managed and simplified solution. Furthermore, the AWS-managed solutions can ensure higher availability and scalability compared to an EC2-based IdP. IAM Identity Center directly supports AD integration through Directory Service, avoiding the need to build a custom integration.

Therefore, enabling IAM Identity Center and configuring a two-way forest trust between the on-premises Active Directory and IAM Identity Center using AWS Directory Service is the optimal solution to meet the company's requirements.

Relevant Documentation:

What is AWS IAM Identity Center (successor to AWS Single Sign-On)?
AWS Directory Service
How AWS IAM Identity Center (successor to AWS Single Sign-On) works with Active Directory

Question: 827 **Exam Heist**

A company is planning to deploy its application on an Amazon Aurora PostgreSQL Serverless v2 cluster. The application will receive large amounts of traffic. The company wants to optimize the storage performance of the cluster as the load on the application increases.

Which solution will meet these requirements MOST cost-effectively?

A.Configure the cluster to use the Aurora Standard storage configuration.

B.Configure the cluster storage type as Provisioned IOPS.

C.Configure the cluster storage type as General Purpose.

D.Configure the cluster to use the Aurora I/O-Optimized storage configuration.

Answer: C

Explanation:

The optimal solution for cost-effectively optimizing storage performance for an Aurora PostgreSQL Serverless v2 cluster under increasing load is option C, configuring the cluster storage type as General Purpose.

Here's why:

Aurora Serverless v2's Storage Scalability: Aurora Serverless v2 is designed to automatically scale both compute and storage resources based on application needs. It automatically increases storage capacity as data grows, negating the need for manual intervention in most cases.
General Purpose (gp2/gp3) SSDs: General Purpose SSDs are a cost-effective storage option that provides a balance of performance and cost for a wide variety of workloads. They are well-suited for databases that experience variable I/O patterns, which is typical during workload spikes.
Cost-Effectiveness Comparison: Provisioned IOPS (option B) is significantly more expensive. It allows you to specify the number of IOPS you need, but this comes at a higher cost than General Purpose storage. Since the application load will only experience periodic spikes and Aurora Serverless v2 already handles storage scaling, the excess capacity provisioned will likely be unused, costing unnecessarily.
Aurora Standard vs. Aurora I/O-Optimized: Aurora I/O-Optimized storage configuration (option D) comes at additional cost. While Aurora Standard storage configuration (option A) would still technically work, General Purpose is the optimal cost-effective configuration. General Purpose storage offers the best balance of cost and performance.
Aurora's Storage Management: Aurora manages storage internally, and General Purpose storage is appropriate for most workloads. It abstracts away the complexities of managing storage performance, allowing developers to focus on the application logic.

In summary, choosing General Purpose storage leverages Aurora Serverless v2's built-in scalability and provides a good balance of performance and cost for handling large amounts of traffic and optimizing storage performance as the load increases.

Relevant Links:

Aurora Storage: https://aws.amazon.com/rds/aurora/storage-optimization/
Aurora Serverless v2: https://aws.amazon.com/rds/aurora/serverless/

Question: 828 **Exam Heist**

A financial services company that runs on AWS has designed its security controls to meet industry standards. The industry standards include the National Institute of Standards and Technology (NIST) and the Payment Card Industry Data Security Standard (PCI DSS).

The company's third-party auditors need proof that the designed controls have been implemented and are functioning correctly. The company has hundreds of AWS accounts in a single organization in AWS Organizations.

The company needs to monitor the current state of the controls across accounts.

Which solution will meet these requirements?

A.Designate one account as the Amazon Inspector delegated administrator account from the Organizations management account. Integrate Inspector with Organizations to discover and scan resources across all AWS accounts. Enable Inspector industry standards for NIST and PCI DSS.

B.Designate one account as the Amazon GuardDuty delegated administrator account from the Organizations management account. In the designated GuardDuty administrator account, enable GuardDuty to protect all member accounts. Enable GuardDuty industry standards for NIST and PCI DSS.

C.Configure an AWS CloudTrail organization trail in the Organizations management account. Designate one account as the compliance account. Enable CloudTrail security standards for NIST and PCI DSS in the compliance account.

D.Designate one account as the AWS Security Hub delegated administrator account from the Organizations management account. In the designated Security Hub administrator account, enable Security Hub for all member accounts. Enable Security Hub standards for NIST and PCI DSS.

Answer: D

Explanation:

The correct answer is **D**. Here's why:

The company requires continuous monitoring of security control implementation and effectiveness across hundreds of AWS accounts in an AWS Organization, with a need to demonstrate compliance with NIST and PCI DSS standards to auditors.

AWS Security Hub is designed to provide a comprehensive view of the security state of your AWS resources. It aggregates, organizes, and prioritizes security alerts and findings from various AWS services (like GuardDuty, Inspector, Macie) and supported third-party solutions. It centralizes security management across accounts. (https://aws.amazon.com/security-hub/)
By designating a Security Hub delegated administrator account from the Organizations management account, the company can centrally manage Security Hub settings and view findings across all member accounts. Security Hub has built-in support for security standards like NIST and PCI DSS. By enabling these standards, Security Hub automatically checks your environment against the controls defined in those standards, providing a compliance score and detailed findings on non-compliant resources. This is crucial for audit purposes.
Amazon Inspector (Option A) focuses on automated security vulnerability assessments of EC2 instances and container images. While valuable for finding vulnerabilities, it doesn't provide a comprehensive view of compliance against industry standards across the entire AWS environment in the same way as Security Hub.
Amazon GuardDuty (Option B) is a threat detection service that monitors for malicious activity and unauthorized behavior. It doesn't specifically address compliance monitoring against standards like NIST and PCI DSS, though its findings can contribute to overall security posture.
AWS CloudTrail (Option C) records API calls made within your AWS environment. While it's essential for audit trails and security analysis, it doesn't directly provide compliance assessments or reporting against specific industry standards. CloudTrail provides data for compliance, not compliance itself.
Security Hub's Standards enable easy compliance tracking and reporting against desired benchmarks, allowing the company to demonstrably prove to third-party auditors that controls are implemented and functional.

Therefore, Security Hub is the most appropriate solution for centrally monitoring compliance with NIST and PCI DSS across multiple AWS accounts, fulfilling the company's requirements for security monitoring and auditability.

Question: 829 **Exam Heist**

A company uses an Amazon S3 bucket as its data lake storage platform. The S3 bucket contains a massive amount of data that is accessed randomly by multiple teams and hundreds of applications. The company wants to reduce the S3 storage costs and provide immediate availability for frequently accessed objects.

What is the MOST operationally efficient solution that meets these requirements?

A.Create an S3 Lifecycle rule to transition objects to the S3 Intelligent-Tiering storage class.

B.Store objects in Amazon S3 Glacier. Use S3 Select to provide applications with access to the data.

C.Use data from S3 storage class analysis to create S3 Lifecycle rules to automatically transition objects to the S3 Standard-Infrequent Access (S3 Standard-IA) storage class.

D.Transition objects to the S3 Standard-Infrequent Access (S3 Standard-IA) storage class. Create an AWS Lambda function to transition objects to the S3 Standard storage class when they are accessed by an application.

Answer: A

Explanation:

The optimal solution is **A. Create an S3 Lifecycle rule to transition objects to the S3 Intelligent-Tiering storage class.**

Here's a detailed justification:

The problem requires cost reduction and immediate availability for frequently accessed S3 objects in a data lake environment with random access patterns.

S3 Intelligent-Tiering is designed for scenarios where access patterns are unknown or change over time. It automatically moves data between frequent, infrequent, and archive access tiers based on usage patterns, optimizing storage costs without performance impact. This aligns perfectly with the requirement for both cost reduction and immediate availability of frequently accessed objects.

Operationally Efficient: Implementing an S3 Lifecycle rule to transition objects to S3 Intelligent-Tiering is a straightforward and automated process. It requires minimal operational overhead as S3 handles the tiering automatically based on access patterns. No manual intervention or custom code is necessary.

Why other options are less suitable:

B. Store objects in Amazon S3 Glacier. Use S3 Select to provide applications with access to the data. S3 Glacier is designed for archival storage with infrequent access. While cost-effective, retrieving data from Glacier can take hours, violating the requirement for immediate availability. S3 Select can query data in Glacier without retrieving the entire object, but it still doesn't provide instant access.

C. Use data from S3 storage class analysis to create S3 Lifecycle rules to automatically transition objects to the S3 Standard-Infrequent Access (S3 Standard-IA) storage class. This option requires analyzing storage access patterns first and then creating lifecycle rules. S3 Standard-IA is suitable for infrequently accessed data, but objects accessed frequently won't benefit and will incur retrieval charges if moved there. Intelligent-Tiering is more efficient at doing this without requiring analysis and with multiple tiers available.

D. Transition objects to the S3 Standard-Infrequent Access (S3 Standard-IA) storage class. Create an AWS Lambda function to transition objects to the S3 Standard storage class when they are accessed by an application. This approach is complex and introduces unnecessary operational overhead. It requires writing and maintaining a Lambda function to monitor object access and transition them back to S3 Standard. S3 Intelligent-Tiering handles this automatically and more efficiently. Introducing a Lambda function also adds potential points of failure and increased cost for Lambda invocations.

Authoritative Links for Further Research:

S3 Intelligent-Tiering: https://aws.amazon.com/s3/storage-classes/intelligent-tiering/
S3 Lifecycle: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-examples.html
S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/

Question: 830 **Exam Heist**

A company has 5 TB of datasets. The datasets consist of 1 million user profiles and 10 million connections. The user profiles have connections as many-to-many relationships. The company needs a performance efficient way to find mutual connections up to five levels.

Which solution will meet these requirements?

A.Use an Amazon S3 bucket to store the datasets. Use Amazon Athena to perform SQL JOIN queries to find connections.

B.Use Amazon Neptune to store the datasets with edges and vertices. Query the data to find connections.

C.Use an Amazon S3 bucket to store the datasets. Use Amazon QuickSight to visualize connections.

D.Use Amazon RDS to store the datasets with multiple tables. Perform SQL JOIN queries to find connections.

Answer: B

Explanation:

The most appropriate solution is B: Use Amazon Neptune to store the datasets with edges and vertices, and query the data to find connections. Here's why:

Amazon Neptune is a fully managed graph database service. Graph databases are specifically designed for handling highly connected data and performing relationship-based queries, such as finding mutual connections. The dataset described — user profiles and their connections — perfectly aligns with a graph database model where user profiles are vertices (nodes) and connections are edges. Neptune's architecture is optimized for traversing relationships, enabling highly efficient searches for mutual connections up to five levels deep. This traversal is significantly faster than using relational databases or data lakes for the same task.

Options A and D, which propose using Amazon S3 with Athena or Amazon RDS, are less suitable. While both can store the data, they rely on SQL JOIN operations for finding connections. SQL JOINs become computationally expensive and slow as the number of JOINs increases, especially when traversing multiple levels of connections. Athena, while useful for querying data in S3, is not optimized for complex relationship traversals like Neptune is. RDS, as a relational database, can also struggle with the performance demands of multi-level relationship queries.

Option C, using Amazon QuickSight, is primarily a visualization tool. While QuickSight can visualize data from various sources, it doesn't provide the underlying data storage or efficient querying capabilities needed to find mutual connections. It would rely on a different service to perform the actual connection finding.

In summary, Neptune's graph database architecture is inherently designed for the type of relationship-based queries needed in this scenario. The efficiency of graph traversal in Neptune makes it the optimal choice for finding mutual connections up to five levels in a large dataset.

Further research:

Amazon Neptune: https://aws.amazon.com/neptune/
Graph Databases: https://aws.amazon.com/products/databases/graph/

Question: 831 Exam Heist

A company needs a secure connection between its on-premises environment and AWS. This connection does not need high bandwidth and will handle a small amount of traffic. The connection should be set up quickly.

What is the MOST cost-effective method to establish this type of connection?

A.Implement a client VPN.
B.Implement AWS Direct Connect.
C.Implement a bastion host on Amazon EC2.
D.Implement an AWS Site-to-Site VPN connection.

Answer: D

Explanation:

The most cost-effective solution for a secure, low-bandwidth connection between an on-premises environment and AWS, which needs to be set up quickly, is an AWS Site-to-Site VPN connection. Here's why:

AWS Site-to-Site VPN: Establishes an encrypted tunnel over the public internet between your on-premises network and your Amazon VPC. It's relatively quick to configure and doesn't require dedicated hardware, making it suitable for low-bandwidth needs. Pricing is based on VPN connection hours and data transfer out, which aligns well with infrequent or small traffic volumes.
AWS Direct Connect: Designed for high-bandwidth, low-latency, and dedicated network connections. While secure, it involves higher costs due to dedicated port fees, cross-connect charges, and longer setup times. Direct Connect is overkill for the stated requirements.
Client VPN: Allows individual users to securely connect to AWS resources. It's designed for remote access and not a persistent site-to-site connection, making it unsuitable for connecting an entire on-premises network.
Bastion Host on Amazon EC2: Requires launching and managing an EC2 instance, incurring ongoing compute costs. While a bastion host provides secure access, it is primarily designed for accessing resources within the VPC and does not establish a secure, site-to-site connection between the on-premises network and AWS. It also adds operational overhead. Site-to-Site VPN inherently establishes an encrypted tunnel without needing manual setup of protocols.
The key factor is **cost-effectiveness** for low bandwidth. Site-to-Site VPN strikes a balance between security and cost, utilizing existing internet infrastructure.

Therefore, the AWS Site-to-Site VPN offers the best combination of security, speed of deployment, and cost-effectiveness for the given scenario.

Authoritative Links:

AWS Site-to-Site VPN: https://aws.amazon.com/vpn/site-to-site-vpn/
AWS Direct Connect: https://aws.amazon.com/directconnect/
AWS Client VPN: https://aws.amazon.com/vpn/client-vpn/

Question: 832 Exam Heist

A company has an on-premises SFTP file transfer solution. The company is migrating to the AWS Cloud to scale the file transfer solution and to optimize costs by using Amazon S3. The company's employees will use their credentials for the on-premises Microsoft Active Directory (AD) to access the new solution. The company wants to keep the current authentication and file access mechanisms.

Which solution will meet these requirements with the LEAST operational overhead?

A.Configure an S3 File Gateway. Create SMB file shares on the file gateway that use the existing Active

Directory to authenticate.

B.Configure an Auto Scaling group with Amazon EC2 instances to run an SFTP solution. Configure the group to scale up at 60% CPU utilization.

C.Create an AWS Transfer Family server with SFTP endpoints. Choose the AWS Directory Service option as the identity provider. Use AD Connector to connect the on-premises Active Directory.

D.Create an AWS Transfer Family SFTP endpoint. Configure the endpoint to use the AWS Directory Service option as the identity provider to connect to the existing Active Directory.

Answer: C

Explanation:

The correct answer is C because it provides a managed, scalable, and secure SFTP solution integrated with the existing on-premises Active Directory (AD) using AWS Transfer Family and AD Connector, minimizing operational overhead. AWS Transfer Family is a fully managed service that simplifies secure file transfers into and out of Amazon S3. It directly supports SFTP, FTP, and FTPS protocols. Choosing the AWS Directory Service option as the identity provider within Transfer Family allows for seamless integration with AD. AD Connector establishes a secure connection between AWS and the on-premises AD, enabling users to authenticate with their existing AD credentials. This avoids the need to create new user accounts or modify existing ones.

Option A, using S3 File Gateway, introduces more operational overhead because it requires managing an S3 File Gateway appliance and configuring SMB shares. While it supports AD authentication, it's primarily designed for hybrid cloud file sharing, not as a direct replacement for an SFTP solution.

Option B involves setting up and managing an Auto Scaling group with EC2 instances running an SFTP server, which is far more complex and requires significant operational effort for patching, scaling, and security. CPU utilization scaling is reactive and might not be the most efficient for handling file transfer workloads.

Option D is similar to C but lacks the crucial detail of using AD Connector. Simply choosing AWS Directory Service might not be sufficient to connect to the existing on-premises AD without the necessary connector to bridge the gap. AD Connector is specifically designed for this purpose.

Therefore, option C provides the most straightforward and least operationally intensive method to migrate the on-premises SFTP solution to AWS while preserving the existing authentication mechanism via AD Connector and utilizing the scalable and managed AWS Transfer Family.

References:

AWS Transfer Family: https://aws.amazon.com/transfer/
AD Connector: https://aws.amazon.com/directoryservice/ad-connector/
S3 File Gateway: https://aws.amazon.com/storagegateway/file/

Question: 833 **Exam Heist**

A company is designing an event-driven order processing system. Each order requires multiple validation steps after the order is created. An idempotent AWS Lambda function performs each validation step. Each validation step is independent from the other validation steps. Individual validation steps need only a subset of the order event information.

The company wants to ensure that each validation step Lambda function has access to only the information from the order event that the function requires. The components of the order processing system should be loosely coupled to accommodate future business changes.

Which solution will meet these requirements?

A.Create an Amazon Simple Queue Service (Amazon SQS) queue for each validation step. Create a new Lambda

function to transform the order data to the format that each validation step requires and to publish the messages to the appropriate SQS queues. Subscribe each validation step Lambda function to its corresponding SQS queue.

B.Create an Amazon Simple Notification Service (Amazon SNS) topic. Subscribe the validation step Lambda functions to the SNS topic. Use message body filtering to send only the required data to each subscribed Lambda function.

C.Create an Amazon EventBridge event bus. Create an event rule for each validation step. Configure the input transformer to send only the required data to each target validation step Lambda function.

D.Create an Amazon Simple Queue Service (Amazon SQS) queue. Create a new Lambda function to subscribe to the SQS queue and to transform the order data to the format that each validation step requires. Use the new Lambda function to perform synchronous invocations of the validation step Lambda functions in parallel on separate threads.

Answer: C

Explanation:

The correct answer is C because it leverages Amazon EventBridge's event routing and transformation capabilities to meet the specific requirements of the order processing system.

Here's why:

Loose Coupling: EventBridge promotes loose coupling by decoupling the order creation process from the validation steps. The order creation system publishes events to EventBridge without needing to know about the validation functions. Each validation function subscribes to specific events based on rules.
Targeted Data Delivery: EventBridge's input transformer is the key. It allows you to define how the original event data is transformed before it's passed to the target Lambda function. This ensures that each validation function receives only the required subset of order information, enhancing security and reducing processing overhead.
Event-Driven Architecture: The solution aligns with an event-driven architecture, where validation steps are triggered by the occurrence of an order event, making the system responsive and scalable.
Idempotency: While all options could potentially be implemented to accommodate idempotent Lambda functions, EventBridge provides the cleanest separation of concerns and transformation capabilities that align with the other requirements.

Why other options are less suitable:

A (SQS with transformation Lambda): This introduces an extra Lambda function to transform data, which adds complexity and potential points of failure. While SQS provides reliable message queuing, it doesn't offer the same level of filtering and transformation as EventBridge. The tight coupling between the transformation Lambda and SQS queues makes it less flexible.
B (SNS with message filtering): While SNS can fan out messages to multiple subscribers, its message filtering capabilities are more limited compared to EventBridge's input transformer. SNS filtering primarily works on message attributes, not on the message body content, making it less suitable for extracting specific data subsets.
D (SQS with parallel invocation Lambda): This approach creates significant complexity. Managing parallel Lambda invocations introduces challenges around error handling, concurrency, and resource management. The synchronous invocation also reduces the benefits of the event-driven nature of the system.

Authoritative Links:

Amazon EventBridge: https://aws.amazon.com/eventbridge/
EventBridge Input Transformer:
https://docs.aws.amazon.com/eventbridge/latest/userguide/event_transformer.html
Event-Driven Architecture: https://aws.amazon.com/event-driven-architecture/

Question: 834 **Exam Heist**

A company is migrating a three-tier application to AWS. The application requires a MySQL database. In the past, the application users reported poor application performance when creating new entries. These performance issues were caused by users generating different real-time reports from the application during working hours.

Which solution will improve the performance of the application when it is moved to AWS?

A.Import the data into an Amazon DynamoDB table with provisioned capacity. Refactor the application to use DynamoDB for reports.

B.Create the database on a compute optimized Amazon EC2 instance. Ensure compute resources exceed the on-premises database.

C.Create an Amazon Aurora MySQL Multi-AZ DB cluster with multiple read replicas. Configure the application to use the reader endpoint for reports.

D.Create an Amazon Aurora MySQL Multi-AZ DB cluster. Configure the application to use the backup instance of the cluster as an endpoint for the reports.

Answer: C

Explanation:

The correct answer is C because it addresses the root cause of the performance issue: report generation impacting the main database's performance.

Here's a detailed justification:

Understanding the Problem: The users experienced poor application performance due to real-time report generation during business hours, overloading the database.

Why Option A is Incorrect: DynamoDB, while suitable for many use cases, would require significant application refactoring to support the existing MySQL database structure and reporting requirements. This isn't an optimal solution for a simple migration.

Why Option B is Incorrect: Simply increasing the compute resources on an EC2 instance hosting the MySQL database may alleviate the problem slightly, but it doesn't isolate report queries from affecting transactional performance. The reports will still contend for resources with the main application workload on the same database instance.

Why Option C is Correct: Amazon Aurora MySQL Multi-AZ provides high availability and performance. More importantly, read replicas allow you to offload read-intensive workloads, such as report generation, from the primary database instance. This isolation prevents report queries from impacting the performance of the primary database used for transactions. Configuring the application to use the reader endpoint for report generation ensures these queries are directed to the read replicas, not the primary instance. This is a standard practice for optimizing read-heavy workloads in relational databases.

Multi-AZ: Ensures high availability.
Read Replicas: Offload read traffic (reports), isolating the primary database for transactional workloads.
Reader Endpoint: Provides a single point of access for all read replicas.

Why Option D is Incorrect: The backup instance of an Aurora cluster is not intended to be used for serving read traffic. It's primarily for failover and point-in-time recovery. Using it for report generation could interfere with its primary purpose and potentially impact recovery time if a failover is needed. Furthermore, backup instances might not be consistently up-to-date and ready to serve read requests in the way a dedicated read replica is.

In summary: Option C provides a cost-effective and efficient solution by using Aurora's read replicas to offload report generation, thereby isolating the read workload from the primary database and improving overall application performance during peak hours.

Authoritative Links:

Amazon Aurora Read Replicas:
https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraMySQL.Replication.ReadReplicas.html
Amazon Aurora Multi-AZ Deployments:
https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Concepts.MultiAZ.html

Question: 835 **Exam Heist**

A company is expanding a secure on-premises network to the AWS Cloud by using an AWS Direct Connect connection. The on-premises network has no direct internet access. An application that runs on the on-premises network needs to use an Amazon S3 bucket.

Which solution will meet these requirements MOST cost-effectively?

A.Create a public virtual interface (VIF). Route the AWS traffic over the public VIF.

B.Create a VPC and a NAT gateway. Route the AWS traffic from the on-premises network to the NAT gateway.

C.Create a VPC and an Amazon S3 interface endpoint. Route the AWS traffic from the on-premises network to the S3 interface endpoint.

D.Create a VPC peering connection between the on-premises network and Direct Connect. Route the AWS traffic over the peering connection.

Answer: C

Explanation:

The most cost-effective solution for accessing an S3 bucket from an on-premises network with no direct internet access via AWS Direct Connect is creating a VPC and an Amazon S3 interface endpoint.

Here's why:

Interface Endpoints (AWS PrivateLink): Interface endpoints, powered by AWS PrivateLink, provide private connectivity to AWS services, including S3, without exposing traffic to the public internet. This ensures secure communication over your Direct Connect connection.
https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html

Cost-Effectiveness: Interface endpoints are generally more cost-effective than NAT gateways for accessing S3, especially when considering the data transfer charges.

Security: Interface endpoints keep traffic within the AWS network and the Direct Connect connection, avoiding exposure to the public internet, which aligns with the requirement for a secure connection.

Let's analyze why the other options are less suitable:

Public VIF: Using a public VIF would route traffic over the public internet, which contradicts the requirement to avoid direct internet access.

NAT Gateway: NAT gateways provide internet access to instances within a private subnet. While you could route traffic through a NAT gateway in a VPC connected via Direct Connect, this is intended for instances requiring broader internet access, not solely for accessing S3. It also incurs data processing charges for the NAT gateway itself, adding to the cost.

VPC Peering: VPC peering allows you to connect two VPCs, but it doesn't directly solve the problem of accessing S3 from on-premises without internet access. You would still need a way to reach the S3 service from the peered VPC, potentially using a NAT gateway, making it less optimal than an interface endpoint.

In summary, an S3 interface endpoint provides a secure, private, and cost-effective solution by allowing direct

access to S3 from the on-premises network through the Direct Connect connection, without traversing the public internet.

Question: 836 **Exam Heist**

A company serves its website by using an Auto Scaling group of Amazon EC2 instances in a single AWS Region. The website does not require a database.

The company is expanding, and the company's engineering team deploys the website to a second Region. The company wants to distribute traffic across both Regions to accommodate growth and for disaster recovery purposes. The solution should not serve traffic from a Region in which the website is unhealthy.

Which policy or resource should the company use to meet these requirements?

A.An Amazon Route 53 simple routing policy

B.An Amazon Route 53 multivalue answer routing policy

C.An Application Load Balancer in one Region with a target group that specifies the EC2 instance IDs from both Regions

D.An Application Load Balancer in one Region with a target group that specifies the IP addresses of the EC2 instances from both Regions

Answer: B

Explanation:

The correct answer is **B. An Amazon Route 53 multivalue answer routing policy.** Here's why:

Disaster Recovery and High Availability: The company requires a disaster recovery solution that automatically avoids unhealthy Regions. Route 53 multivalue answer routing is designed to distribute traffic to multiple healthy endpoints (in this case, the different Regions hosting the website).

Health Checks: Route 53 can be configured with health checks for each endpoint. If a Region's health check fails, Route 53 will automatically stop routing traffic to that Region, ensuring users are only served by healthy website instances.

Multi-Region Distribution: Multivalue answer routing allows Route 53 to return multiple IP addresses in response to a DNS query. Clients will randomly choose one of these addresses to connect to, effectively distributing traffic across the healthy Regions.

Simplicity: Compared to more complex solutions, Route 53 multivalue answer routing provides a relatively straightforward setup for multi-Region traffic distribution and failover.

Let's examine why the other options are less suitable:

A. An Amazon Route 53 simple routing policy: Simple routing policy doesn't inherently support health checks or automatic failover. It only returns a single IP address, making it unsuitable for distributing traffic across Regions and handling failures.

C. An Application Load Balancer in one Region with a target group that specifies the EC2 instance IDs from both Regions: ALBs are regional resources. While cross-zone load balancing is possible within a Region, ALBs cannot directly manage EC2 instances in another Region using instance IDs within a single target group. Furthermore, if the entire Region where the ALB is located fails, the entire solution becomes unavailable.

D. An Application Load Balancer in one Region with a target group that specifies the IP addresses of the EC2 instances from both Regions: While technically possible to add IPs from a different Region, this is not the intended use case for ALB target groups and introduces complexities with managing and updating the IPs,

especially with dynamic scaling. More importantly, a single ALB in one region isn't ideal for cross-region DR and high availability as failure of that region will impact the entire solution. This also violates network boundaries.**In conclusion, Route 53 multivalue answer routing, combined with health checks, provides the simplest and most effective way to distribute traffic across multiple Regions, ensure high availability, and automatically fail over from unhealthy Regions, fulfilling the company's requirements.**

Supporting Links:

Amazon Route 53 Routing Policies: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html
Route 53 Health Checks: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/health-checks-creating-deleting.html

Question: 837 **Exam Heist**

A company runs its applications on Amazon EC2 instances that are backed by Amazon Elastic Block Store (Amazon EBS). The EC2 instances run the most recent Amazon Linux release. The applications are experiencing availability issues when the company's employees store and retrieve files that are 25 GB or larger. The company needs a solution that does not require the company to transfer files between EC2 instances. The files must be available across many EC2 instances and across multiple Availability Zones.

Which solution will meet these requirements?

A.Migrate all the files to an Amazon S3 bucket. Instruct the employees to access the files from the S3 bucket.

B.Take a snapshot of the existing EBS volume. Mount the snapshot as an EBS volume across the EC2 instances. Instruct the employees to access the files from the EC2 instances.

C.Mount an Amazon Elastic File System (Amazon EFS) file system across all the EC2 instances. Instruct the employees to access the files from the EC2 instances.

D.Create an Amazon Machine Image (AMI) from the EC2 instances. Configure new EC2 instances from the AMI that use an instance store volume. Instruct the employees to access the files from the EC2 instances.

Answer: C

Explanation:

The correct answer is **C. Mount an Amazon Elastic File System (Amazon EFS) file system across all the EC2 instances. Instruct the employees to access the files from the EC2 instances.**

Here's why:

Requirement for Availability Across EC2 Instances and AZs: The problem specifies that files need to be available across many EC2 instances and multiple Availability Zones. Amazon EFS is designed specifically for this purpose. It provides a scalable, elastic, and fully managed file system service. EFS file systems can be mounted concurrently by multiple EC2 instances in multiple AZs within an AWS Region. This satisfies the availability requirement effectively.
No File Transfer Between EC2 Instances: Using EFS eliminates the need to transfer files between EC2 instances. All instances access the same central file system. When an employee saves a file to the EFS mount point on one instance, the file is immediately accessible from all other instances with the EFS file system mounted.
Handling Large Files: EFS is designed to handle large files efficiently. It scales automatically as data is added or removed, so it can easily accommodate 25 GB files without performance degradation.
Why Other Options Are Incorrect:

A. Amazon S3: While S3 is excellent for object storage, it might not be the ideal solution if the employees require a traditional file system interface. Using S3 would require code or applications to be aware of S3 APIs

and interact with the storage through HTTP, which is not a natural file system interaction. Though compatible with large files, it introduces more complexity than a file system.
B. EBS Snapshot: EBS snapshots are point-in-time copies of EBS volumes. Mounting a snapshot as a new EBS volume would create a separate, isolated copy of the data. This does not provide a shared file system that updates across all EC2 instances, meaning the instances aren't working with one singular filesystem.
D. AMI with Instance Store: Instance store volumes are ephemeral, meaning they lose their data when the instance is stopped or terminated. This contradicts the need for persistent storage and availability. Furthermore, the AMI does not create a system where the files are available to all running instances after the AMI is launched.

In summary, EFS offers the best balance of features, satisfying the requirements of shared access across EC2 instances and Availability Zones, eliminating the need for file transfers, and supporting the large file sizes mentioned in the problem.

Authoritative Links:

Amazon EFS: https://aws.amazon.com/efs/
Amazon EC2 Instance Store: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/InstanceStorage.html
Amazon EBS Snapshots: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EBSSnapshots.html
Amazon S3: https://aws.amazon.com/s3/

Question: 838 **Exam Heist**

A company is running a highly sensitive application on Amazon EC2 backed by an Amazon RDS database. Compliance regulations mandate that all personally identifiable information (PII) be encrypted at rest.

Which solution should a solutions architect recommend to meet this requirement with the LEAST amount of changes to the infrastructure?

A.Deploy AWS Certificate Manager to generate certificates. Use the certificates to encrypt the database volume.

B.Deploy AWS CloudHSM, generate encryption keys, and use the keys to encrypt database volumes.

C.Configure SSL encryption using AWS Key Management Service (AWS KMS) keys to encrypt database volumes.

D.Configure Amazon Elastic Block Store (Amazon EBS) encryption and Amazon RDS encryption with AWS Key Management Service (AWS KMS) keys to encrypt instance and database volumes.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best solution, adhering to the prompt's constraints:

The requirement is to encrypt PII at rest for both the EC2 instance and RDS database, minimizing infrastructure changes. Option C, configuring SSL encryption using AWS Key Management Service (AWS KMS) keys, directly addresses this. RDS already offers encryption at rest using KMS keys. Enabling this feature is straightforward and requires minimal modification to the existing RDS instance. Furthermore, the EC2 instance's connection to RDS can be secured using SSL/TLS encryption, protecting data in transit as well.

Option A is incorrect because AWS Certificate Manager (ACM) primarily manages SSL/TLS certificates for securing network connections. While ACM certificates are crucial for encrypting data in transit, they don't directly encrypt the underlying database or EC2 volumes at rest.

Option B, deploying AWS CloudHSM, is overly complex for this scenario. CloudHSM provides dedicated hardware security modules for key management, offering higher levels of security compliance but at a significantly increased cost and operational overhead. It's generally reserved for organizations with stringent

regulatory requirements beyond basic encryption at rest. CloudHSM introduces more infrastructure changes and administrative burden than necessary.

Option D is also valid, but less ideal than C. It mentions EBS encryption and RDS encryption with KMS keys. While EBS encryption is crucial for the EC2 instance itself, the focus of the question is encrypting the RDS database containing the PII. Option C encapsulates the core requirement for RDS encryption while also providing a mechanism (SSL/TLS) to protect the EC2 instance's data in transit to RDS. Option D implies additional EBS volume management, making it more complex compared to configuring SSL and RDS encryption at rest.

Therefore, Option C offers the most efficient and least disruptive path to encrypting sensitive data at rest for the RDS database and securing the connection from the EC2 instance, all while leveraging the existing AWS KMS service. It balances security with ease of implementation, fulfilling the core requirement with minimal alterations to the infrastructure.

Supporting links:

RDS Encryption: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Overview.Encryption.html
AWS KMS: https://aws.amazon.com/kms/
SSL/TLS with RDS: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/UsingWithRDS.SSL.html

Question: 839 **Exam Heist**

A company runs an AWS Lambda function in private subnets in a VPC. The subnets have a default route to the internet through an Amazon EC2 NAT instance. The Lambda function processes input data and saves its output as an object to Amazon S3.

Intermittently, the Lambda function times out while trying to upload the object because of saturated traffic on the NAT instance's network. The company wants to access Amazon S3 without traversing the internet.

Which solution will meet these requirements?

A.Replace the EC2 NAT instance with an AWS managed NAT gateway.

B.Increase the size of the EC2 NAT instance in the VPC to a network optimized instance type.

C.Provision a gateway endpoint for Amazon S3 in the VPUpdate the route tables of the subnets accordingly.

D.Provision a transit gateway. Place transit gateway attachments in the private subnets where the Lambda function is running.

Answer: C

Explanation:

The correct answer is **C. Provision a gateway endpoint for Amazon S3 in the VPC. Update the route tables of the subnets accordingly.**

Here's why:

Problem: The Lambda function in private subnets is experiencing timeouts due to network congestion on the EC2 NAT instance when uploading data to S3. The goal is to access S3 without going over the internet, thereby alleviating the strain on the NAT instance.

Solution C: Gateway Endpoint

Mechanism: A gateway endpoint for S3 provides a direct, private path from your VPC to S3, completely bypassing the internet. It is highly available and scales with the traffic demand.

Implementation: Creating a gateway endpoint involves selecting the S3 service and the route tables you wish

to associate with it. Once the endpoint is created, you need to update the route tables for the subnets where the Lambda function resides to route traffic destined for S3 through the gateway endpoint.

Benefits:

Avoids Internet: Keeps traffic within the AWS network, improving security and reducing latency.
Scalability: AWS handles the scalability and availability of the endpoint.
Cost-Effective: Gateway endpoints are free to use; you are only charged for the data stored in and transferred out of S3.

Why other options are incorrect:

A. Replace the EC2 NAT instance with an AWS managed NAT gateway: While a NAT Gateway is a better solution than a NAT Instance, because it is managed and highly available, it does not meet the requirement of avoiding the internet. The Lambda function would still need to route all traffic through the NAT Gateway to access S3.
B. Increase the size of the EC2 NAT instance in the VPC to a network optimized instance type: While upgrading the EC2 NAT instance may reduce congestion temporarily, it doesn't eliminate the internet dependency or the bottleneck of the NAT instance itself. This is not as effective as providing a direct connection using a gateway endpoint.
D. Provision a transit gateway. Place transit gateway attachments in the private subnets where the Lambda function is running: A transit gateway is an option for connecting multiple VPCs and on-premises networks but is overkill and more expensive for the simple use case of providing access to S3 from private subnets. It does not solve the primary issue of reducing internet traffic because routing traffic to S3 through a transit gateway still involves traffic going through the transit gateway's route table to the S3 service, which isn't necessarily a private path without additional configurations like VPC endpoints.

Relevant concepts and resources:

VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html
Gateway Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-s3.html
NAT Gateway vs NAT Instance: https://aws.amazon.com/premiumsupport/knowledge-center/vpc-nat-gateway-instance/

In summary, a gateway endpoint offers the most efficient and cost-effective solution to provide private, scalable, and reliable access to S3 for the Lambda function without traversing the internet.

Question: 840 **Exam Heist**

A news company that has reporters all over the world is hosting its broadcast system on AWS. The reporters send live broadcasts to the broadcast system. The reporters use software on their phones to send live streams through the Real Time Messaging Protocol (RTMP).

A solutions architect must design a solution that gives the reporters the ability to send the highest quality streams. The solution must provide accelerated TCP connections back to the broadcast system.

What should the solutions architect use to meet these requirements?

A.Amazon CloudFront
B.AWS Global Accelerator
C.AWS Client VPN
D.Amazon EC2 instances and AWS Elastic IP addresses

Answer: B

Explanation:

AWS Global Accelerator is the optimal solution because it's designed to optimize TCP and UDP traffic, making it ideal for real-time media streaming using RTMP. It uses the AWS global network to route traffic to the nearest healthy application endpoint, reducing latency and improving connection reliability, crucial for live broadcasts from geographically dispersed reporters. CloudFront, while a CDN, is primarily for caching static and dynamic content, and doesn't optimize live streaming ingest in the same way. AWS Client VPN is for secure access to AWS resources and isn't designed for real-time streaming optimization. EC2 instances with Elastic IP addresses lack the global network optimization capabilities needed for accelerated TCP connections. Global Accelerator's Anycast static IPs provide a single entry point for reporters, simplifying the connection process and facilitating seamless failover. By leveraging the AWS global network backbone, Global Accelerator minimizes jitter and packet loss, leading to higher quality streams. Therefore, option B directly addresses the need for accelerated TCP connections and highest quality streams for live broadcasts from a global network of reporters.

Here are authoritative links for further research:

AWS Global Accelerator: https://aws.amazon.com/global-accelerator/
AWS Global Accelerator FAQs: https://aws.amazon.com/global-accelerator/faqs/

Question: 841 **Exam Heist**

A company uses Amazon EC2 instances and Amazon Elastic Block Store (Amazon EBS) to run its self-managed database. The company has 350 TB of data spread across all EBS volumes. The company takes daily EBS snapshots and keeps the snapshots for 1 month. The daily change rate is 5% of the EBS volumes.

Because of new regulations, the company needs to keep the monthly snapshots for 7 years. The company needs to change its backup strategy to comply with the new regulations and to ensure that data is available with minimal administrative effort.

Which solution will meet these requirements MOST cost-effectively?

A.Keep the daily snapshot in the EBS snapshot standard tier for 1 month. Copy the monthly snapshot to Amazon S3 Glacier Deep Archive with a 7-year retention period.

B.Continue with the current EBS snapshot policy. Add a new policy to move the monthly snapshot to Amazon EBS Snapshots Archive with a 7-year retention period.

C.Keep the daily snapshot in the EBS snapshot standard tier for 1 month. Keep the monthly snapshot in the standard tier for 7 years. Use incremental snapshots.

D.Keep the daily snapshot in the EBS snapshot standard tier. Use EBS direct APIs to take snapshots of all the EBS volumes every month. Store the snapshots in an Amazon S3 bucket in the Infrequent Access tier for 7 years.

Answer: B

Explanation:

Here's a detailed justification for why option B is the most cost-effective solution, along with supporting information and links:

The core requirement is to retain monthly snapshots of EBS volumes for 7 years with minimal administrative overhead and cost-effectiveness.

Option B is the most suitable because it leverages the EBS Snapshots Archive tier, which is specifically designed for long-term storage of snapshots that are infrequently accessed. This archive tier offers significantly lower storage costs compared to the standard EBS snapshot tier, making it the most cost-effective option for the 7-year retention requirement. The solution continues the existing daily snapshot policy for one month, ensuring fast recovery if needed, then moves the monthly snapshot to the archive tier

for long-term compliance. This provides a balance between operational restore needs and regulatory demands.

Option A involves moving the monthly snapshot to Amazon S3 Glacier Deep Archive. While Glacier Deep Archive offers very low storage costs, restoring data from Glacier is time-consuming and could impact recovery time objectives (RTO). EBS Snapshots Archive is designed for EBS data, allowing for potentially faster and easier restores than Glacier when necessary. Furthermore, using S3 would require additional configuration for managing the backup process, increasing administrative overhead.

Option C keeps the monthly snapshots in the standard EBS snapshot tier for 7 years. This option is the most expensive because the standard tier is designed for frequently accessed snapshots, not long-term archival. Storing snapshots in the standard tier for 7 years would incur significantly higher storage costs compared to using the archive tier.

Option D uses EBS Direct APIs and stores the snapshots in Amazon S3 Infrequent Access (IA). While S3 IA is cheaper than standard S3, it is still more expensive than EBS Snapshots Archive. Moreover, using EBS Direct APIs would require developing and maintaining custom code for managing snapshots, increasing administrative overhead. EBS Direct APIs are useful in niche cases, but here, it is an unnecessary complication.

By utilizing the EBS Snapshots Archive tier, option B achieves the required 7-year retention period at the lowest possible cost while minimizing administrative effort. It also integrates seamlessly with the existing EBS snapshot policy, minimizing disruption to the current backup process.

Key Concepts & Links:

EBS Snapshots Archive: A low-cost storage tier within EBS Snapshots, ideal for long-term retention and compliance. https://aws.amazon.com/ebs/snapshots/archive/
EBS Snapshots: Point-in-time backups of EBS volumes. https://aws.amazon.com/ebs/snapshots/
Cost Optimization: Selecting the right storage tier based on access frequency to minimize costs.
Glacier Deep Archive: A low-cost storage service for archiving data that is rarely accessed. https://aws.amazon.com/glacier/deep-archive/

Question: 842 **Exam Heist**

A company runs an application on several Amazon EC2 instances that store persistent data on an Amazon Elastic File System (Amazon EFS) file system. The company needs to replicate the data to another AWS Region by using an AWS managed service solution.

Which solution will meet these requirements MOST cost-effectively?

A.Use the EFS-to-EFS backup solution to replicate the data to an EFS file system in another Region.

B.Run a nightly script to copy data from the EFS file system to an Amazon S3 bucket. Enable S3 Cross-Region Replication on the S3 bucket.

C.Create a VPC in another Region. Establish a cross-Region VPC peer. Run a nightly rsync to copy data from the original Region to the new Region.

D.Use AWS Backup to create a backup plan with a rule that takes a daily backup and replicates it to another Region. Assign the EFS file system resource to the backup plan.

Answer: A

Explanation:

The question asks for the most cost-effective AWS-managed solution to replicate data from an EFS file system to another Region.

Option A, using the EFS-to-EFS backup solution, is the most cost-effective because it leverages a managed service specifically designed for replicating EFS data across regions. This eliminates the need for custom scripting or infrastructure management. EFS Replication (part of EFS Backup and Recovery) automates the process, handling incremental updates and data consistency.

Option B involves copying data to S3 and then using S3 Cross-Region Replication. This requires additional S3 storage costs and the initial overhead of transferring data to S3. While S3 replication is efficient, the initial step of copying to S3 adds unnecessary cost and complexity compared to the direct EFS-to-EFS replication.

Option C, creating a VPC peer and using rsync, necessitates managing the rsync process and the cross-region VPC peering connection. It requires manual configuration, monitoring, and potential troubleshooting. This adds operational overhead and costs associated with maintaining the infrastructure and the script. It is less managed and more prone to failure than a dedicated EFS replication service.

Option D, using AWS Backup, is also a viable solution, but it primarily focuses on data backup and recovery, not continuous replication. AWS Backup creates point-in-time snapshots, which is valuable for disaster recovery, but may not offer the same level of continuous replication as EFS Replication. Moreover, frequent backups can lead to increased storage costs. EFS-to-EFS replication is optimized for replicating data, making it more cost-effective in this scenario.

Therefore, EFS-to-EFS replication is the most cost-effective and managed solution for replicating data between EFS file systems across regions.

Refer to these links for more information:

Amazon EFS Replication: Backup and recovery

Question: 843 **Exam Heist**

An ecommerce company is migrating its on-premises workload to the AWS Cloud. The workload currently consists of a web application and a backend Microsoft SQL database for storage.

The company expects a high volume of customers during a promotional event. The new infrastructure in the AWS Cloud must be highly available and scalable.

Which solution will meet these requirements with the LEAST administrative overhead?

A.Migrate the web application to two Amazon EC2 instances across two Availability Zones behind an Application Load Balancer. Migrate the database to Amazon RDS for Microsoft SQL Server with read replicas in both Availability Zones.

B.Migrate the web application to an Amazon EC2 instance that runs in an Auto Scaling group across two Availability Zones behind an Application Load Balancer. Migrate the database to two EC2 instances across separate AWS Regions with database replication.

C.Migrate the web application to Amazon EC2 instances that run in an Auto Scaling group across two Availability Zones behind an Application Load Balancer. Migrate the database to Amazon RDS with Multi-AZ deployment.

D.Migrate the web application to three Amazon EC2 instances across three Availability Zones behind an Application Load Balancer. Migrate the database to three EC2 instances across three Availability Zones.

Answer: C

Explanation:

The best solution for a highly available and scalable e-commerce application with minimal administrative overhead involves leveraging managed services like Amazon RDS with Multi-AZ deployment and EC2 instances behind an Auto Scaling group and Application Load Balancer.

Option C is correct because it suggests using EC2 instances in an Auto Scaling group across multiple Availability Zones (AZs) behind an Application Load Balancer (ALB). The Auto Scaling group ensures the web application can handle increased traffic by automatically scaling the number of instances based on demand. The ALB distributes traffic across these instances, providing high availability.

For the database, it proposes migrating to Amazon RDS with Multi-AZ deployment. RDS Multi-AZ provides high availability by synchronously replicating data to a standby instance in a different AZ. In case of a failure in the primary AZ, RDS automatically fails over to the standby instance, minimizing downtime. RDS handles much of the administrative overhead associated with managing a database, such as patching, backups, and failover.

Option A is less ideal because using read replicas for the database is not the most straightforward way to achieve high availability for the write operations required by the e-commerce application. Read replicas primarily improve read performance and are not designed for automatic failover in the same way as Multi-AZ.

Option B is flawed because using EC2 instances across separate AWS Regions for the database adds significant complexity to database replication and management, increasing administrative overhead. Region-based failover is more complex than AZ-based failover.

Option D involves managing SQL Server on EC2 instances across multiple AZs which requires manual setup and management of database replication, backups, patching, and failover. This introduces significant administrative overhead, making it unsuitable for the "least administrative overhead" requirement.

In summary, option C achieves high availability and scalability with minimal administrative overhead by utilizing managed services like Auto Scaling, ALB, and RDS Multi-AZ.

Relevant Links:

Amazon RDS Multi-AZ
Amazon EC2 Auto Scaling
Application Load Balancer

Question: 844 **Exam Heist**

A company has an on-premises business application that generates hundreds of files each day. These files are stored on an SMB file share and require a low-latency connection to the application servers. A new company policy states all application-generated files must be copied to AWS. There is already a VPN connection to AWS.

The application development team does not have time to make the necessary code modifications to move the application to AWS.

Which service should a solutions architect recommend to allow the application to copy files to AWS?

A.Amazon Elastic File System (Amazon EFS)
B.Amazon FSx for Windows File Server
C.AWS Snowball
D.AWS Storage Gateway

Answer: D

Explanation:

The correct answer is **D. AWS Storage Gateway**. Here's why:

AWS Storage Gateway facilitates connecting on-premises applications to AWS storage services. Specifically, the File Gateway configuration allows you to store files as objects in Amazon S3 while maintaining SMB

access from the on-premises application. Because the question states that the application development team has no time to make modifications to the code, a solution that requires them to implement the copy/move files (such as EFS) isn't a suitable solution.

Storage Gateway's File Gateway addresses the key requirement of copying files to AWS without modifying the application code. The application continues to write files to the SMB file share, and the File Gateway seamlessly uploads them to S3 in the background.

Amazon EFS (A) is a fully managed NFS file system for use with AWS cloud services and on-premises resources. While EFS can be mounted on-premises with Direct Connect or VPN, it would require application code changes to write directly to the EFS mount point instead of the existing SMB share. This option is ruled out by the question's constraints.

Amazon FSx for Windows File Server (B) provides fully managed, native Microsoft Windows file systems. It could be used, but it necessitates a more complex migration strategy, which is not needed for simply copying the files to AWS for archival. It's typically used for migrating entire Windows file servers to the cloud.

AWS Snowball (C) is suitable for large-scale data migration, but it's a one-time transfer solution. The question states the application generates hundreds of files daily, indicating an ongoing need for replication, rendering Snowball impractical for this use case.

Therefore, AWS Storage Gateway - File Gateway is the best option because it provides the needed bridge between the on-premises SMB file share and S3, automatically copies files, and requires no application code changes.

For further research:

AWS Storage Gateway: https://aws.amazon.com/storagegateway/
File Gateway documentation:
https://docs.aws.amazon.com/storagegateway/latest/userguide/WhatIsStorageGateway.html

Question: 845 **Exam Heist**

A company has 15 employees. The company stores employee start dates in an Amazon DynamoDB table. The company wants to send an email message to each employee on the day of the employee's work anniversary.

Which solution will meet these requirements with the MOST operational efficiency?

A.Create a script that scans the DynamoDB table and uses Amazon Simple Notification Service (Amazon SNS) to send email messages to employees when necessary. Use a cron job to run this script every day on an Amazon EC2 instance.

B.Create a script that scans the DynamoDB table and uses Amazon Simple Queue Service (Amazon SQS) to send email messages to employees when necessary. Use a cron job to run this script every day on an Amazon EC2 instance.

C.Create an AWS Lambda function that scans the DynamoDB table and uses Amazon Simple Notification Service (Amazon SNS) to send email messages to employees when necessary. Schedule this Lambda function to run every day.

D.Create an AWS Lambda function that scans the DynamoDB table and uses Amazon Simple Queue Service (Amazon SQS) to send email messages to employees when necessary. Schedule this Lambda function to run every day.

Answer: C

Explanation:

The most operationally efficient solution is to use AWS Lambda and Amazon SNS (Option C). Here's why:

Serverless Architecture: Lambda offers a serverless environment, eliminating the need to manage EC2 instances (as in Options A and B). This reduces operational overhead related to patching, scaling, and infrastructure maintenance.

Scheduled Execution: Lambda functions can be scheduled to run using Amazon CloudWatch Events (now Amazon EventBridge), providing a built-in mechanism for daily execution without relying on cron jobs on EC2 instances.

Scalability and Cost-Effectiveness: Lambda automatically scales based on demand. For only 15 employees, the cost of running Lambda will likely be significantly lower than maintaining a dedicated EC2 instance. You only pay for the compute time you consume.

Direct Email Integration: SNS can directly send email messages, simplifying the workflow. SQS (Options B and D) introduces an unnecessary queue when a direct notification mechanism (SNS) is available. SQS is beneficial when decoupling components and managing message delivery, but that's not a clear requirement here.

DynamoDB Scan Efficiency: Although scanning a DynamoDB table isn't ideal for larger datasets, with only 15 records, the scan operation will be relatively quick and cost-effective. Consider optimizing DynamoDB query if the employee count significantly increases in the future.

Reduced Complexity: Combining Lambda, CloudWatch Events, and SNS results in a simpler and easier-to-manage architecture compared to solutions involving EC2 instances and cron jobs.

In Summary: Option C leverages serverless technologies to minimize operational overhead, automate the anniversary notification process, and optimize costs for a small number of employees.

Authoritative Links:

AWS Lambda: https://aws.amazon.com/lambda/
Amazon SNS: https://aws.amazon.com/sns/
Amazon EventBridge: https://aws.amazon.com/eventbridge/
DynamoDB Scan: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Scan.html

Question: 846 **Exam Heist**

A company's application is running on Amazon EC2 instances within an Auto Scaling group behind an Elastic Load Balancing (ELB) load balancer. Based on the application's history, the company anticipates a spike in traffic during a holiday each year. A solutions architect must design a strategy to ensure that the Auto Scaling group proactively increases capacity to minimize any performance impact on application users.

Which solution will meet these requirements?

A.Create an Amazon CloudWatch alarm to scale up the EC2 instances when CPU utilization exceeds 90%.

B.Create a recurring scheduled action to scale up the Auto Scaling group before the expected period of peak demand.

C.Increase the minimum and maximum number of EC2 instances in the Auto Scaling group during the peak demand period.

D.Configure an Amazon Simple Notification Service (Amazon SNS) notification to send alerts when there are autoscaling:EC2_INSTANCE_LAUNCH events.

Answer: B

Explanation:

The correct answer is **B. Create a recurring scheduled action to scale up the Auto Scaling group before the expected period of peak demand.**

Here's a detailed justification:

The requirement is to proactively increase capacity before the expected traffic spike to minimize performance impact. Scheduled scaling is the ideal solution for this because the company knows when the traffic increase will occur (during the holiday). This allows them to schedule an Auto Scaling group size increase ahead of time.

Option B directly addresses proactive scaling. Scheduled actions allow you to configure the Auto Scaling group to increase its desired capacity at a specific time and date, or on a recurring schedule. This means EC2 instances will be launched and ready to serve traffic before the actual spike begins.

Option A (CloudWatch alarm based on CPU utilization) is a reactive scaling method. While useful, it only scales up after the CPU utilization reaches 90%. This means users may experience performance issues before the scaling happens. This is not proactive, and it doesn't address the need to prepare before the traffic spike.

Option C (Increasing minimum and maximum instances) only sets the boundaries for scaling, it doesn't actually trigger the scaling event. It is possible the group will not scale up to the maximum specified and stay between the old limits. Also, changing the minimum number of instances alone does not guarantee instances will be ready to handle peak load proactively. You would still need a mechanism (like scheduled actions or alarms) to trigger scaling.

Option D (SNS notification of instance launch events) is a monitoring mechanism, not a scaling strategy. It doesn't trigger scaling; it just provides notification about events that have already occurred. This is useful for auditing and troubleshooting but doesn't solve the proactive scaling requirement.

Therefore, scheduled scaling (Option B) is the only solution that proactively increases the capacity of the Auto Scaling group before the holiday traffic spike, ensuring minimal performance impact on users.

For further research, refer to the AWS documentation on scheduled scaling for Auto Scaling groups:

AWS Documentation - Scheduled Scaling

Question: 847 **Exam Heist**

A company uses Amazon RDS for PostgreSQL databases for its data tier. The company must implement password rotation for the databases.

Which solution meets this requirement with the LEAST operational overhead?

A.Store the password in AWS Secrets Manager. Enable automatic rotation on the secret.

B.Store the password in AWS Systems Manager Parameter Store. Enable automatic rotation on the parameter.

C.Store the password in AWS Systems Manager Parameter Store. Write an AWS Lambda function that rotates the password.

D.Store the password in AWS Key Management Service (AWS KMS). Enable automatic rotation on the AWS KMS key.

Answer: A

Explanation:

The most efficient solution for password rotation in Amazon RDS for PostgreSQL, minimizing operational overhead, is to utilize AWS Secrets Manager with automatic rotation enabled.

Here's why:

AWS Secrets Manager is specifically designed for managing secrets like database credentials. It offers built-

in functionality for rotating these secrets automatically. This eliminates the need for custom scripting or complex configurations.

Automatic Rotation: Secrets Manager's automatic rotation feature directly integrates with RDS for PostgreSQL. You configure a rotation schedule, and Secrets Manager handles the process of generating a new password, updating it in the database, and updating the stored secret, all without manual intervention.

Reduced Operational Overhead: This approach drastically reduces the operational burden. You avoid writing and maintaining custom Lambda functions (as in option C) or manually rotating passwords.

Security Best Practices: Secrets Manager adheres to security best practices by encrypting secrets at rest and in transit.

Parameter Store (Option B and C): While Systems Manager Parameter Store can store secrets, it lacks native automatic rotation capabilities for RDS databases. Option C requires additional coding and maintenance of a Lambda function.

AWS KMS (Option D): KMS is for encrypting data, not managing secrets and rotating them automatically. It's not designed for password management.

Cost-Effectiveness: Secrets Manager's pricing is based on the number of secrets stored and API calls. The minimal operational effort often translates to lower overall costs compared to building and maintaining custom solutions.

In contrast, options B, C, and D involve manual configuration, scripting, or using services not directly intended for secret rotation, leading to higher operational overhead and potential for errors. By leveraging the built-in features of AWS Secrets Manager, the company can achieve secure and automated password rotation for its RDS for PostgreSQL databases with minimal administrative effort.

Authoritative Links:

AWS Secrets Manager: https://aws.amazon.com/secrets-manager/
Rotating Secrets: https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotating-secrets.html
Rotating Secrets for Amazon RDS for PostgreSQL databases
https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotating-secrets-rds.html

Question: 848 **Exam Heist**

A company runs its application on Oracle Database Enterprise Edition. The company needs to migrate the application and the database to AWS. The company can use the Bring Your Own License (BYOL) model while migrating to AWS. The application uses third-party database features that require privileged access.

A solutions architect must design a solution for the database migration.

Which solution will meet these requirements MOST cost-effectively?

A.Migrate the database to Amazon RDS for Oracle by using native tools. Replace the third-party features with AWS Lambda.

B.Migrate the database to Amazon RDS Custom for Oracle by using native tools. Customize the new database settings to support the third-party features.

C.Migrate the database to Amazon DynamoDB by using AWS Database Migration Service (AWS DMS). Customize the new database settings to support the third-party features.

D.Migrate the database to Amazon RDS for PostgreSQL by using AWS Database Migration Service (AWS DMS). Rewrite the application code to remove the dependency on third-party features.

Answer: B

Explanation:

The correct answer is B because it directly addresses all the requirements in the most cost-effective manner. The company wants to use the BYOL model for Oracle, and both RDS for Oracle and RDS Custom for Oracle support this. However, the key differentiator is the need for privileged access to support third-party database features.

Amazon RDS for Oracle (option A) is a managed service, which means AWS handles the underlying infrastructure and operating system. While convenient, this limits the level of control and access a user has. Specifically, privileged access required for the third-party features is not available in standard RDS Oracle instances. Replacing these features with AWS Lambda would involve significant application refactoring, increasing cost and complexity.

Amazon DynamoDB (option C) is a NoSQL database, and migrating an Oracle database using DMS would necessitate a complete application rewrite, making it a complex and costly solution. It doesn't support BYOL for Oracle licenses.

Amazon RDS for PostgreSQL (option D) would also require a database migration using DMS and extensive code refactoring to remove the Oracle-specific third-party features. This is a significant effort and cost. It doesn't support BYOL for Oracle licenses.

Amazon RDS Custom for Oracle (option B) is designed to provide database administrators with operating system and database access. It allows for customized settings, including the ability to install and manage third-party features that require privileged access. Because it also supports BYOL, this option allows the company to maintain its licensing model and meets its technical requirements without costly refactoring or a complete migration to a different database engine. This makes it the most cost-effective solution.

Supporting documentation:

Amazon RDS Custom: https://aws.amazon.com/rds/custom/ - Explains the benefits of customization for meeting application requirements.
AWS Database Migration Service: https://aws.amazon.com/dms/ - Discusses the service used to migrate databases to AWS.
Amazon RDS for Oracle: https://aws.amazon.com/rds/oracle/ - Describes the managed Oracle database service.

Question: 849 **Exam Heist**

A large international university has deployed all of its compute services in the AWS Cloud. These services include Amazon EC2, Amazon RDS, and Amazon DynamoDB. The university currently relies on many custom scripts to back up its infrastructure. However, the university wants to centralize management and automate data backups as much as possible by using AWS native options.

Which solution will meet these requirements?

A. Use third-party backup software with an AWS Storage Gateway tape gateway virtual tape library.
B. Use AWS Backup to configure and monitor all backups for the services in use.
C. Use AWS Config to set lifecycle management to take snapshots of all data sources on a schedule.
D. Use AWS Systems Manager State Manager to manage the configuration and monitoring of backup tasks.

Answer: B

Explanation:

The correct answer is **B. Use AWS Backup to configure and monitor all backups for the services in use.**

Here's why:

AWS Backup is a fully managed backup service that centralizes and automates data protection across various AWS services. It provides a single pane of glass for configuring, managing, and monitoring backups of EC2 instances, EBS volumes, RDS databases, DynamoDB tables, EFS file systems, and other AWS resources. This directly addresses the university's need to centralize management and automate backups using AWS native options. https://aws.amazon.com/backup/

Option A is incorrect because using third-party software and AWS Storage Gateway introduces unnecessary complexity and goes against the requirement to use AWS native options. Also, a tape gateway is more suitable for archiving data rather than for regular backup and restore operations.

Option C is incorrect because AWS Config primarily focuses on configuration management and compliance, not backup. While it can detect configuration changes, it doesn't provide comprehensive backup and recovery capabilities. Moreover, directly triggering snapshots with Config is not its primary purpose.

Option D is incorrect because AWS Systems Manager State Manager is designed for configuration management and automation of tasks across EC2 instances, not specifically for managing backups of diverse data sources like RDS and DynamoDB. While it can execute backup scripts, it lacks the centralized backup management and monitoring capabilities of AWS Backup. It requires writing and maintaining custom scripts, which the university wants to avoid. AWS Backup offers a managed and centralized solution.

Question: 850 **Exam Heist**

A company wants to build a map of its IT infrastructure to identify and enforce policies on resources that pose security risks. The company's security team must be able to query data in the IT infrastructure map and quickly identify security risks.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use Amazon RDS to store the data. Use SQL to query the data to identify security risks.
B.Use Amazon Neptune to store the data. Use SPARQL to query the data to identify security risks.
C.Use Amazon Redshift to store the data. Use SQL to query the data to identify security risks.
D.Use Amazon DynamoDB to store the data. Use PartiQL to query the data to identify security risks.

Answer: B

Explanation:

The correct answer is B: Use Amazon Neptune to store the data. Use SPARQL to query the data to identify security risks. Here's why:

The scenario calls for mapping IT infrastructure to identify and enforce security policies. This fundamentally describes a graph database use case. A graph database excels at representing relationships between entities, making it ideal for IT infrastructure mapping where resources are interconnected and dependencies are important for identifying security risks.

Amazon Neptune is a fully managed graph database service. It's designed for storing and querying highly connected data. SPARQL (SPARQL Protocol and RDF Query Language) is a query language specifically designed for querying graph databases. It enables efficient traversal and analysis of relationships within the graph. This allows the security team to quickly identify resources with vulnerabilities or policy violations based on their relationships with other elements in the infrastructure.

Options A, C, and D are less suitable:

Amazon RDS (Relational Database Service) and Amazon Redshift (Data Warehouse): While you could model the data in a relational database, querying relationships would involve complex and inefficient JOIN operations, leading to performance bottlenecks and increased operational overhead. Neither service is inherently designed for graph-like data or fast relationship traversal.

Amazon DynamoDB (NoSQL Database): DynamoDB is a key-value and document database. While it offers flexibility, modelling and querying complex relationships is less intuitive and requires more application logic compared to a graph database. PartiQL, DynamoDB's query language, can perform queries, but it won't be as efficient as SPARQL for traversing relationships in a highly interconnected IT infrastructure map.

Neptune, with its graph-specific design and SPARQL query capabilities, provides the most efficient and performant solution with the least operational overhead for mapping and querying IT infrastructure relationships to identify security risks. The other options require more complex modelling, custom querying, and result in poorer performance.

Authoritative Links:

Amazon Neptune: https://aws.amazon.com/neptune/
SPARQL: https://www.w3.org/TR/rdf-sparql-query/

Question: 851 **Exam Heist**

A large company wants to provide its globally located developers separate, limited size, managed PostgreSQL databases for development purposes. The databases will be low volume. The developers need the databases only when they are actively working.

Which solution will meet these requirements MOST cost-effectively?

A.Give the developers the ability to launch separate Amazon Aurora instances. Set up a process to shut down Aurora instances at the end of the workday and to start Aurora instances at the beginning of the next workday.

B.Develop an AWS Service Catalog product that enforces size restrictions for launching Amazon Aurora instances. Give the developers access to launch the product when they need a development database.

C.Create an Amazon Aurora Serverless cluster. Develop an AWS Service Catalog product to launch databases in the cluster with the default capacity settings. Grant the developers access to the product.

D.Monitor AWS Trusted Advisor checks for idle Amazon RDS databases. Create a process to terminate identified idle RDS databases.

Answer: B

Explanation:

The most cost-effective solution is B. Let's break down why:

Aurora Serverless v1 is the most cost-effective option for intermittent, low-volume workloads. It automatically scales capacity based on application demand and shuts down completely when not in use, eliminating charges during idle periods. https://aws.amazon.com/rds/aurora/serverless/
AWS Service Catalog allows administrators to create and manage catalogs of IT services that are approved for use on AWS. This ensures developers only launch pre-approved, correctly sized databases. https://aws.amazon.com/servicecatalog/
Option A is not ideal because standard Aurora instances are not designed for frequent start/stop cycles. This can be time-consuming and potentially impact availability. The costs associated with manual start/stop procedures may also exceed the advantages of Aurora Serverless v1.
Option C uses a single Aurora Serverless cluster, but it doesn't adequately address the requirement for separate databases for each developer. Sharing a cluster could lead to contention and data security concerns. Creating databases within a single serverless cluster also doesn't enforce size limitations.

Option D focuses on monitoring and terminating idle RDS databases, but it lacks the proactive enforcement of size restrictions. Also, this is a reactive approach.
Combining Aurora Serverless with AWS Service Catalog provides a robust solution: Service Catalog enables the provisioning of Aurora Serverless databases with size restrictions, while Aurora Serverless ensures cost optimization by scaling and pausing based on usage patterns.

Question: 852 **Exam Heist**

A company is building a web application that serves a content management system. The content management system runs on Amazon EC2 instances behind an Application Load Balancer (ALB). The EC2 instances run in an Auto Scaling group across multiple Availability Zones. Users are constantly adding and updating files, blogs, and other website assets in the content management system.

A solutions architect must implement a solution in which all the EC2 instances share up-to-date website content with the least possible lag time.

Which solution meets these requirements?

A.Update the EC2 user data in the Auto Scaling group lifecycle policy to copy the website assets from the EC2 instance that was launched most recently. Configure the ALB to make changes to the website assets only in the newest EC2 instance.

B.Copy the website assets to an Amazon Elastic File System (Amazon EFS) file system. Configure each EC2 instance to mount the EFS file system locally. Configure the website hosting application to reference the website assets that are stored in the EFS file system.

C.Copy the website assets to an Amazon S3 bucket. Ensure that each EC2 instance downloads the website assets from the S3 bucket to the attached Amazon Elastic Block Store (Amazon EBS) volume. Run the S3 sync command once each hour to keep files up to date.

D.Restore an Amazon Elastic Block Store (Amazon EBS) snapshot with the website assets. Attach the EBS snapshot as a secondary EBS volume when a new EC2 instance is launched. Configure the website hosting application to reference the website assets that are stored in the secondary EBS volume.

Answer: B

Explanation:

The correct answer is B: Copying the website assets to Amazon EFS and mounting it on each EC2 instance provides a shared, consistently updated, and low-latency file system accessible to all instances. This approach directly addresses the requirement for all EC2 instances to share up-to-date website content with minimal lag.

Here's why:

Shared File System: Amazon EFS provides a shared file system that can be simultaneously mounted by multiple EC2 instances. This eliminates the need to copy or synchronize data between instances.
Consistency: EFS offers strong consistency, ensuring that all instances see the latest changes to the website assets.
Low Latency: EFS is designed for low-latency access, making it suitable for serving website content. Changes are immediately available to all instances.
Scalability: EFS automatically scales its storage capacity to accommodate growing website assets.
Integration: EFS integrates seamlessly with EC2 and Auto Scaling groups.

Let's examine why the other options are less suitable:

A: Relying on copying assets from the newest EC2 instance is complex and error-prone. It introduces inconsistency during updates and requires intricate configuration changes. ALBs do not make changes to website assets.
C: Using Amazon S3 for website assets requires regular synchronization to EBS volumes. Hourly

synchronization introduces significant lag, which conflicts with the requirement for minimal lag time and is not efficient. S3 is generally best for static object storage and delivery, not a constantly changing file system.
D: EBS snapshots are point-in-time copies, and restoring them as secondary volumes would not provide a mechanism for real-time updates. The instances would have a stale version of the website assets.

In summary, Amazon EFS provides the most effective, scalable, and consistent solution for sharing website assets across multiple EC2 instances with minimal latency.

Supporting Links:

Amazon EFS Documentation
Amazon EC2 Documentation

Question: 853 **Exam Heist**

A company's web application consists of multiple Amazon EC2 instances that run behind an Application Load Balancer in a VPC. An Amazon RDS for MySQL DB instance contains the data. The company needs the ability to automatically detect and respond to suspicious or unexpected behavior in its AWS environment. The company already has added AWS WAF to its architecture.

What should a solutions architect do next to protect against threats?

A.Use Amazon GuardDuty to perform threat detection. Configure Amazon EventBridge to filter for GuardDuty findings and to invoke an AWS Lambda function to adjust the AWS WAF rules.

B.Use AWS Firewall Manager to perform threat detection. Configure Amazon EventBridge to filter for Firewall Manager findings and to invoke an AWS Lambda function to adjust the AWS WAF web ACL.

C.Use Amazon Inspector to perform threat detection and to update the AWS WAF rules. Create a VPC network ACL to limit access to the web application.

D.Use Amazon Macie to perform threat detection and to update the AWS WAF rules. Create a VPC network ACL to limit access to the web application.

Answer: A

Explanation:

The correct answer is A because it provides a comprehensive threat detection and response mechanism tailored to the scenario. Here's a detailed justification:

Amazon GuardDuty is a threat detection service that continuously monitors your AWS accounts and workloads for malicious activity and unauthorized behavior. It uses machine learning, anomaly detection, and integrated threat intelligence to identify threats. https://aws.amazon.com/guardduty/

Amazon EventBridge (formerly CloudWatch Events) allows you to build event-driven applications at scale. It can filter events from various AWS services, including GuardDuty, and trigger actions based on these events. https://aws.amazon.com/eventbridge/

AWS Lambda lets you run code without provisioning or managing servers. In this solution, a Lambda function is used to dynamically adjust AWS WAF rules based on GuardDuty findings, providing automated remediation. https://aws.amazon.com/lambda/

AWS WAF (Web Application Firewall) protects web applications from common web exploits and bots. By dynamically adjusting WAF rules based on GuardDuty findings, the application can mitigate emerging threats. https://aws.amazon.com/waf/

This combination provides a closed-loop threat detection and response system. GuardDuty detects threats, EventBridge triggers a Lambda function based on specific GuardDuty findings, and Lambda updates WAF rules to block the identified malicious activity.

Option B is incorrect because AWS Firewall Manager primarily manages WAF rules and other security policies across multiple accounts and applications. While it provides centralized management, it doesn't inherently perform threat detection in the same way as GuardDuty.

Option C is incorrect because Amazon Inspector primarily assesses EC2 instances and container images for vulnerabilities. While valuable, it doesn't focus on runtime threat detection like GuardDuty. Updating WAF rules based on Inspector findings would be less effective than reacting to actual malicious activity detected by GuardDuty. Adding a VPC Network ACL is a good security practice, but it won't dynamically adapt to new threats based on monitoring, thus is secondary to continuous threat detection.

Option D is incorrect because Amazon Macie discovers and protects sensitive data stored in Amazon S3. While important for data security, it's not designed for threat detection in the same way as GuardDuty. Adding a VPC Network ACL is a good security practice, but it won't dynamically adapt to new threats based on monitoring, thus is secondary to continuous threat detection.

The most effective approach is to use GuardDuty for continuous threat detection and automate the response by adjusting WAF rules based on its findings using EventBridge and Lambda. This offers a dynamic and automated security posture against suspicious activities.

Question: 854 **Exam Heist**

A company is planning to run a group of Amazon EC2 instances that connect to an Amazon Aurora database. The company has built an AWS CloudFormation template to deploy the EC2 instances and the Aurora DB cluster. The company wants to allow the instances to authenticate to the database in a secure way. The company does not want to maintain static database credentials.

Which solution meets these requirements with the LEAST operational effort?

A.Create a database user with a user name and password. Add parameters for the database user name and password to the CloudFormation template. Pass the parameters to the EC2 instances when the instances are launched.

B.Create a database user with a user name and password. Store the user name and password in AWS Systems Manager Parameter Store. Configure the EC2 instances to retrieve the database credentials from Parameter Store.

C.Configure the DB cluster to use IAM database authentication. Create a database user to use with IAM authentication. Associate a role with the EC2 instances to allow applications on the instances to access the database.

D.Configure the DB cluster to use IAM database authentication with an IAM user. Create a database user that has a name that matches the IAM user. Associate the IAM user with the EC2 instances to allow applications on the instances to access the database.

Answer: C

Explanation:

Here's a detailed justification for why option C is the correct answer, along with supporting information:

The problem requires a secure and operationally efficient method for EC2 instances to authenticate to an Aurora database without using static credentials. The key is to avoid managing and rotating database usernames and passwords within the EC2 instances or storing them in Parameter Store.

Option C, utilizing IAM database authentication for Aurora, provides the most secure and streamlined approach. IAM database authentication enables authentication using IAM roles and policies instead of database usernames and passwords. This aligns with the principle of least privilege and reduces the risk of credential compromise.

Here's why option C is superior to the others:

Option A (Static Credentials in CloudFormation): Embedding credentials directly in the CloudFormation template or passing them as parameters is a security risk. These credentials could be exposed if the template is compromised or if instance metadata is accessed improperly. It also creates an ongoing maintenance burden.

Option B (Static Credentials in Parameter Store): While better than Option A, storing static credentials in Parameter Store still requires managing and rotating those credentials. If these credentials are ever compromised, you would need to manually rotate them, which could disrupt your application. It doesn't completely eliminate the risk or the operational overhead.

Option D (IAM Authentication with IAM User): While using IAM is correct, associating an IAM user directly with the EC2 instance is less flexible and scalable than using an IAM role. Roles are designed for EC2 instances and other AWS services, as they can be assumed dynamically. IAM users are usually associated with human identities.

Option C directly addresses the problem by:

1. Enabling IAM database authentication on the Aurora DB cluster, avoiding traditional database credentials.
2. Creating a database user authorized for IAM authentication.
3. Associating an IAM role with the EC2 instances. This role grants the necessary permissions to access the database. The application running on the EC2 instances can then assume this role and authenticate to the database using temporary AWS credentials obtained through the instance metadata service.
4. This approach eliminates the need to manage and rotate database credentials. Permissions are centrally managed through IAM policies, simplifying security management and reducing the attack surface. The short-lived, automatically rotated AWS credentials are far more secure.

In short, IAM database authentication provides a seamless, secure, and scalable solution for authenticating EC2 instances to Aurora DB clusters without the burden of managing static database credentials, making it the best fit for the problem's requirements and operational efficiency.

Supporting documentation:

IAM Database Authentication for MySQL and PostgreSQL:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/UsingWithRDS.IAMDBAuth.html
IAM Roles for Amazon EC2: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/iam-roles-for-amazon-ec2.html

Question: 855 **Exam Heist**

A company wants to configure its Amazon CloudFront distribution to use SSL/TLS certificates. The company does not want to use the default domain name for the distribution. Instead, the company wants to use a different domain name for the distribution.

Which solution will deploy the certificate without incurring any additional costs?

A. Request an Amazon issued private certificate from AWS Certificate Manager (ACM) in the us-east-1 Region.
B. Request an Amazon issued private certificate from AWS Certificate Manager (ACM) in the us-west-1 Region.
C. Request an Amazon issued public certificate from AWS Certificate Manager (ACM) in the us-east-1 Region.
D. Request an Amazon issued public certificate from AWS Certificate Manager (ACM) in the us-west-1 Region.

Answer: C

Explanation:

The correct answer is C, requesting an Amazon-issued public certificate from AWS Certificate Manager (ACM) in the us-east-1 Region. Here's why:

Custom Domain Names and SSL/TLS: To use a custom domain name (e.g., www.example.com) with a CloudFront distribution, you need an SSL/TLS certificate that validates your ownership of that domain. CloudFront uses this certificate to encrypt communication between users and CloudFront edge locations.

AWS Certificate Manager (ACM): ACM is the preferred way to provision, manage, and deploy SSL/TLS certificates for use with AWS services. ACM handles the complexities of obtaining and renewing certificates.

ACM Pricing: ACM provides **free** public SSL/TLS certificates for use with AWS services like CloudFront, Elastic Load Balancing, and API Gateway. This fulfills the requirement of avoiding additional costs. Private ACM certificates do incur costs.

ACM Region for CloudFront: Crucially, ACM certificates used with CloudFront must be requested or imported into the **us-east-1 (N. Virginia)** Region, regardless of where your other AWS resources are located. CloudFront is a global service, and its certificate management is centralized in this region. The certificate is then distributed globally to CloudFront edge locations.

Public vs. Private Certificates: For public-facing websites and applications using CloudFront, you need a public certificate trusted by major browsers and operating systems. These are the certificates ACM issues for free. Private certificates, while they provide encryption, are typically used for internal communication within an organization and would not be trusted by external clients without additional configuration.

Therefore, only requesting a public certificate from ACM in the us-east-1 region provides both SSL/TLS support for your custom domain in CloudFront and avoids any additional costs.

Relevant Documentation:

Using SSL/TLS Certificates with CloudFront:
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-https-cloudfront.html
AWS Certificate Manager Pricing: https://aws.amazon.com/certificate-manager/pricing/
Requesting a Public Certificate: https://docs.aws.amazon.com/acm/latest/userguide/gs-acm-request-public.html

Question: 856 **Exam Heist**

A company creates operations data and stores the data in an Amazon S3 bucket. For the company's annual audit, an external consultant needs to access an annual report that is stored in the S3 bucket. The external consultant needs to access the report for 7 days.

The company must implement a solution to allow the external consultant access to only the report.

Which solution will meet these requirements with the MOST operational efficiency?

A.Create a new S3 bucket that is configured to host a public static website. Migrate the operations data to the new S3 bucket. Share the S3 website URL with the external consultant.

B.Enable public access to the S3 bucket for 7 days. Remove access to the S3 bucket when the external consultant completes the audit.

C.Create a new IAM user that has access to the report in the S3 bucket. Provide the access keys to the external consultant. Revoke the access keys after 7 days.

D.Generate a presigned URL that has the required access to the location of the report on the S3 bucket. Share the presigned URL with the external consultant.

Answer: D

Explanation:

The correct answer is D: Generate a presigned URL that has the required access to the location of the report on the S3 bucket. Share the presigned URL with the external consultant.

Here's why: Presigned URLs offer the most operationally efficient and secure method for granting temporary, restricted access to specific S3 objects. A presigned URL is a URL generated by someone with valid AWS credentials that grants access to a specific S3 object for a limited time. The URL includes embedded credentials, so the user doesn't need their own AWS account to access the object.

Option A is incorrect because creating a public static website and moving the data there exposes all operations data publicly, which violates the requirement for limited access to only the annual report. It also involves unnecessary data migration.

Option B is incorrect because enabling public access to the entire S3 bucket for 7 days is a significant security risk. It exposes all data in the bucket, not just the report, and contradicts the principle of least privilege.

Option C is incorrect because creating a new IAM user and sharing access keys introduces the overhead of managing IAM users and rotating credentials. It's also less secure, as the consultant could potentially use the credentials for longer than the intended 7 days or access other resources.

Presigned URLs are operationally efficient because they don't require the management of IAM users or bucket policies. They are also secure because they provide temporary, object-level access. After the specified expiry time, the URL becomes invalid, preventing further access. This aligns precisely with the requirement of providing access only to the report for 7 days, making option D the most suitable choice.

Relevant Documentation:

AWS S3 Presigned URLs:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/ShareObjectPreSignedURL.html

Question: 857 **Exam Heist**

A company plans to run a high performance computing (HPC) workload on Amazon EC2 Instances. The workload requires low-latency network performance and high network throughput with tightly coupled node-to-node communication.

Which solution will meet these requirements?

A.Configure the EC2 instances to be part of a cluster placement group.
B.Launch the EC2 instances with Dedicated Instance tenancy.
C.Launch the EC2 instances as Spot Instances.
D.Configure an On-Demand Capacity Reservation when the EC2 instances are launched.

Answer: A

Explanation:

The correct answer is **A. Configure the EC2 instances to be part of a cluster placement group.**

Here's a detailed justification:

Cluster placement groups are specifically designed for applications that require low-latency, high-throughput network performance, and tight node-to-node communication, as is typical for HPC workloads. When EC2

instances are launched within a cluster placement group, they are placed in close proximity to each other within a single Availability Zone. This proximity minimizes network latency between the instances.

Placement groups, and especially cluster placement groups, leverage specialized networking infrastructure within AWS that minimizes latency and maximizes bandwidth between instances. This is crucial for tightly coupled HPC applications where communication overhead can significantly impact overall performance. By grouping the instances, AWS can provide network performance superior to what would be possible if the instances were scattered across different racks or Availability Zones.

Option B, Dedicated Instance tenancy, provides dedicated hardware for your instances, but it doesn't guarantee low-latency network communication. While it might offer performance benefits in terms of resource isolation, it doesn't directly address the node-to-node communication requirements of HPC workloads.

Option C, Spot Instances, are cost-effective but can be interrupted with little notice. This unpredictability makes them unsuitable for HPC workloads that require continuous operation to maintain performance. Interruptions can disrupt ongoing calculations and negatively impact the overall time to completion.

Option D, On-Demand Capacity Reservation, guarantees that capacity will be available for your instances when you need them. However, it doesn't directly influence network latency or throughput. While important for ensuring resources are available, it doesn't solve the critical network performance issues of HPC applications.

In summary, cluster placement groups are the most appropriate solution because they directly address the need for low-latency, high-throughput network performance required for tightly coupled node-to-node communication in HPC workloads.

Further Reading:

AWS Documentation on Placement Groups
AWS HPC Documentation

Question: 858 **Exam Heist**

A company has primary and secondary data centers that are 500 miles (804.7 km) apart and interconnected with high-speed fiber-optic cable. The company needs a highly available and secure network connection between its data centers and a VPC on AWS for a mission-critical workload. A solutions architect must choose a connection solution that provides maximum resiliency.

Which solution meets these requirements?

A.Two AWS Direct Connect connections from the primary data center terminating at two Direct Connect locations on two separate devices

B.A single AWS Direct Connect connection from each of the primary and secondary data centers terminating at one Direct Connect location on the same device

C.Two AWS Direct Connect connections from each of the primary and secondary data centers terminating at two Direct Connect locations on two separate devices

D.A single AWS Direct Connect connection from each of the primary and secondary data centers terminating at one Direct Connect location on two separate devices

Answer: C

Explanation:

The correct answer is **C. Two AWS Direct Connect connections from each of the primary and secondary data centers terminating at two Direct Connect locations on two separate devices.**

Here's a detailed justification:

The requirement is for a highly available and secure network connection between the on-premises data centers (primary and secondary) and an AWS VPC for a mission-critical workload. Resiliency is the key concern. AWS Direct Connect provides a dedicated network connection from on-premises to AWS, bypassing the public internet for enhanced security and consistent performance.

Option C offers the highest level of redundancy and resilience. Having two Direct Connect connections from each data center ensures that if one connection fails, the other will maintain connectivity. Terminating these connections at two separate Direct Connect locations on separate devices further mitigates the risk of a single point of failure. For instance, a failure at one Direct Connect location (e.g., power outage, hardware issue) will not completely disrupt connectivity, as the other location will still be operational. Using multiple Direct Connect locations makes it resilient to AWS infrastructure maintenance or regional events. This approach is ideal for mission-critical workloads.

Option A only connects from the primary data center, leaving the secondary data center unconnected, thereby missing redundancy from the secondary site.

Option B has single points of failure. A single device at the Direct Connect location could fail, resulting in complete disconnection.

Option D has a single point of failure at each data center (single Direct Connect connection). A single location could fail, resulting in complete disconnection.

Therefore, option C offers the maximum resiliency and high availability required by the scenario.

Reference:

AWS Direct Connect Resiliency Recommendations: https://aws.amazon.com/blogs/networking-and-content-delivery/aws-direct-connect-resiliency-recommendations/
AWS Direct Connect FAQs: https://aws.amazon.com/directconnect/faqs/

Question: 859 **Exam Heist**

A company runs several Amazon RDS for Oracle On-Demand DB instances that have high utilization. The RDS DB instances run in member accounts that are in an organization in AWS Organizations.

The company's finance team has access to the organization's management account and member accounts. The finance team wants to find ways to optimize costs by using AWS Trusted Advisor.

Which combination of steps will meet these requirements? (Choose two.)

A.Use the Trusted Advisor recommendations in the management account.
B.Use the Trusted Advisor recommendations in the member accounts where the RDS DB instances are running.
C.Review the Trusted Advisor checks for Amazon RDS Reserved Instance Optimization.
D.Review the Trusted Advisor checks for Amazon RDS Idle DB Instances.
E.Review the Trusted Advisor checks for compute optimization. Crosscheck the results by using AWS Compute Optimizer.

Answer: AC

Explanation:

Here's a detailed justification for the correct answer, along with supporting concepts and links:

The goal is to optimize costs for highly utilized RDS for Oracle instances. AWS Trusted Advisor provides recommendations for cost optimization.

A is correct: Using Trusted Advisor in the management account provides an aggregated view of all member accounts within the AWS Organization. This centralizes cost optimization recommendations and allows the finance team to have a comprehensive overview of potential savings across all RDS instances.

C is correct: The "Amazon RDS Reserved Instance Optimization" Trusted Advisor check specifically focuses on identifying opportunities to save money by purchasing Reserved Instances (RIs) for RDS. Since the RDS instances have high utilization, they are good candidates for Reserved Instances, which offer significant cost savings compared to On-Demand pricing for consistent, predictable workloads. This directly addresses the company's objective of cost optimization. The Trusted Advisor RI optimization check analyzes your RDS usage patterns and suggests potential RI purchases.

Why other options are incorrect:

B: While checking Trusted Advisor in member accounts is helpful, it doesn't provide the consolidated view the finance team in the management account likely requires for overall cost management.
D: "Amazon RDS Idle DB Instances" checks are relevant for cost optimization but target instances that aren't being used. The problem states the instances are highly utilized, making this check less relevant in this scenario.
E: Compute optimization is more relevant to EC2 instances. While related, directly addressing RDS Reserved Instance optimization is more focused on the specifics of the question. AWS Compute Optimizer is a separate service for analyzing and recommending optimal AWS compute resources. It does not directly address RDS RI purchasing recommendations.

Key Concepts:

AWS Organizations: Enables centralized management and governance across multiple AWS accounts.
AWS Trusted Advisor: Provides recommendations to optimize AWS infrastructure for cost, security, performance, reliability, and service limits.
Amazon RDS Reserved Instances (RIs): Offer significant cost savings compared to On-Demand pricing in exchange for a commitment to use the DB instance for a specific period.

Supporting Links:

AWS Organizations: https://aws.amazon.com/organizations/
AWS Trusted Advisor: https://aws.amazon.com/premiumsupport/technology/trusted-advisor/
Amazon RDS Reserved Instances: https://aws.amazon.com/rds/reserved-instances/
AWS Compute Optimizer: https://aws.amazon.com/compute-optimizer/

Question: 860 **Exam Heist**

A solutions architect is creating an application. The application will run on Amazon EC2 instances in private subnets across multiple Availability Zones in a VPC. The EC2 instances will frequently access large files that contain confidential information. These files are stored in Amazon S3 buckets for processing. The solutions architect must optimize the network architecture to minimize data transfer costs.

What should the solutions architect do to meet these requirements?

A.Create a gateway endpoint for Amazon S3 in the VPC. In the route tables for the private subnets, add an entry for the gateway endpoint.

B.Create a single NAT gateway in a public subnet. In the route tables for the private subnets, add a default route that points to the NAT gateway.

C.Create an AWS PrivateLink interface endpoint for Amazon S3 in the VPIn the route tables for the private subnets, add an entry for the interface endpoint.

D.Create one NAT gateway for each Availability Zone in public subnets. In each of the route tables for the private subnets, add a default route that points to the NAT gateway in the same Availability Zone.

Answer: A

Explanation:

The correct answer is **A. Create a gateway endpoint for Amazon S3 in the VPC. In the route tables for the private subnets, add an entry for the gateway endpoint.**

Here's a detailed justification:

The requirement is to minimize data transfer costs when EC2 instances in private subnets frequently access S3 buckets containing confidential information. A gateway endpoint for S3 is the most cost-effective and secure solution for this scenario.

Gateway Endpoints for S3: Gateway endpoints provide connectivity to S3 without requiring traffic to traverse the internet. This is crucial because data transfer within an AWS Region using gateway endpoints is free. This drastically reduces data transfer costs compared to using NAT gateways or internet gateways, which incur data transfer charges. Additionally, gateway endpoints operate at the VPC route table level, directing traffic destined for S3 to the endpoint directly.

Why other options are less suitable:

B. NAT Gateway: While a NAT gateway allows instances in private subnets to access the internet (and potentially S3 if configured), all traffic traverses the NAT gateway, incurring data transfer charges for all data moved between EC2 and S3. Also, using NAT gateways increases latency.

C. AWS PrivateLink Interface Endpoint: While PrivateLink is a secure and private way to connect to S3 (or other AWS services), it primarily benefits third-party services or connecting between VPCs. For resources within the same VPC, a gateway endpoint for S3 offers the same level of security at no additional cost. Also, PrivateLink Interface Endpoints, while offering enhanced security, are more expensive compared to Gateway Endpoints.

D. NAT Gateway per AZ: This approach attempts to improve availability by using a NAT gateway in each Availability Zone. However, it doesn't address the core issue of minimizing data transfer costs, as the NAT gateways still incur charges for data transferred to S3. It is also costly.

Security: A gateway endpoint allows you to control access to S3 buckets using VPC endpoint policies, ensuring only authorized instances can access specific S3 resources. Because all traffic is contained within the AWS network, security is enhanced compared to routing traffic via the public internet.In summary, a gateway endpoint for S3 optimizes network architecture by providing free, secure, and direct connectivity between EC2 instances in private subnets and S3 buckets, thereby minimizing data transfer costs while maintaining a high level of security.

Authoritative Links:

VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html
Gateway VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-s3.html
AWS PrivateLink: https://aws.amazon.com/privatelink/

Question: 861 **Exam Heist**

A company wants to relocate its on-premises MySQL database to AWS. The database accepts regular imports from a client-facing application, which causes a high volume of write operations. The company is concerned that the amount of traffic might be causing performance issues within the application.

How should a solutions architect design the architecture on AWS?

A.Provision an Amazon RDS for MySQL DB instance with Provisioned IOPS SSD storage. Monitor write operation metrics by using Amazon CloudWatch. Adjust the provisioned IOPS if necessary.

B.Provision an Amazon RDS for MySQL DB instance with General Purpose SSD storage. Place an Amazon ElastiCache cluster in front of the DB instance. Configure the application to query ElastiCache instead.

C.Provision an Amazon DocumentDB (with MongoDB compatibility) instance with a memory optimized instance type. Monitor Amazon CloudWatch for performance-related issues. Change the instance class if necessary.

D.Provision an Amazon Elastic File System (Amazon EFS) file system in General Purpose performance mode. Monitor Amazon CloudWatch for IOPS bottlenecks. Change to Provisioned Throughput performance mode if necessary.

Answer: A

Explanation:

The correct answer is A: Provision an Amazon RDS for MySQL DB instance with Provisioned IOPS SSD storage. Monitor write operation metrics by using Amazon CloudWatch. Adjust the provisioned IOPS if necessary.

Here's why:

RDS for MySQL suitability: The company is using MySQL on-premises and wants to migrate it to AWS. RDS for MySQL provides a managed MySQL database service, simplifying administration tasks like backups, patching, and scaling. This makes it a suitable option for a lift-and-shift migration.

Provisioned IOPS for write-heavy workloads: The database experiences high write operations due to regular imports. Provisioned IOPS (PIOPS) SSD storage is designed for I/O-intensive workloads. By provisioning a specific number of IOPS, the company can guarantee a consistent level of performance for write operations. General Purpose SSD (gp2/gp3) storage, while cost-effective, might not provide the required consistent performance for heavy write loads.

Monitoring with CloudWatch: Amazon CloudWatch provides metrics for monitoring the database's performance, including write operations. By monitoring these metrics, the company can identify any performance bottlenecks related to I/O and adjust the provisioned IOPS accordingly. This proactive approach ensures optimal performance and prevents the application from experiencing issues.

Why other options are less suitable:

B: ElastiCache is generally used for caching read operations to reduce load on the database. In this scenario, the concern is high write operations, so ElastiCache is not directly helpful.
C: Amazon DocumentDB is a NoSQL database compatible with MongoDB. Migrating a MySQL database to DocumentDB would require significant application changes and might not be feasible or desirable if the company wants to maintain its existing relational data model and SQL-based queries.
D: Amazon EFS is a network file system and is not suitable for hosting a relational database like MySQL. EFS is designed for shared file storage across multiple EC2 instances, not for database storage.

In summary, using RDS for MySQL with Provisioned IOPS and actively monitoring performance metrics using CloudWatch is the most appropriate way to design the architecture on AWS to address the company's concerns about high write operations.

Relevant Links:

Amazon RDS: https://aws.amazon.com/rds/
Amazon RDS Storage Types:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_Storage.html
Amazon CloudWatch: https://aws.amazon.com/cloudwatch/

Question: 862 **Exam Heist**

A company runs an application in the AWS Cloud that generates sensitive archival data files. The company wants to rearchitect the application's data storage. The company wants to encrypt the data files and to ensure that third parties do not have access to the data before the data is encrypted and sent to AWS. The company has already created an Amazon S3 bucket.

Which solution will meet these requirements?

A.Configure the S3 bucket to use client-side encryption with an Amazon S3 managed encryption key. Configure the application to use the S3 bucket to store the archival files.

B.Configure the S3 bucket to use server-side encryption with AWS KMS keys (SSE-KMS). Configure the application to use the S3 bucket to store the archival files.

C.Configure the S3 bucket to use dual-layer server-side encryption with AWS KMS keys (SSE-KMS). Configure the application to use the S3 bucket to store the archival files.

D.Configure the application to use client-side encryption with a key stored in AWS Key Management Service (AWS KMS). Configure the application to store the archival files in the S3 bucket.

Answer: D

Explanation:

The requirement states that the company wants to ensure third parties cannot access the data before it is encrypted and sent to AWS. This necessitates client-side encryption. Server-side encryption, regardless of the method (SSE-S3, SSE-KMS, or SSE-C), encrypts the data after it has been received by AWS. Therefore, options A, B, and C are incorrect because they all use server-side encryption.

Option D, client-side encryption with a key stored in AWS KMS, addresses the core requirement. With client-side encryption, the application encrypts the data before sending it to S3. Storing the encryption key in AWS KMS provides secure key management, enabling the application to retrieve and use the key for encryption operations. This ensures that the data is protected even during transit to S3. Furthermore, because the encryption happens before the data reaches AWS, third parties cannot intercept and read the data. Using KMS ensures proper key rotation and access control.

Client-side encryption allows the company to maintain full control over the encryption process. The application handles encryption using the AWS SDK and the encryption key retrieved from KMS. After encrypting the data locally, the application then uploads the encrypted file to S3. Because the data is already encrypted, unauthorized access during transit or at rest on S3 is prevented.

Therefore, the correct solution is to configure the application to use client-side encryption with a key stored in AWS KMS and then store the encrypted archival files in the S3 bucket.

Relevant Documentation:

AWS KMS: https://aws.amazon.com/kms/
Protecting Data Using Server-Side Encryption:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/serv-side-encryption.html
Protecting Data Using Client-Side Encryption:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingEncryption.html

Question: 863 **Exam Heist**

A company uses Amazon RDS with default backup settings for its database tier. The company needs to make a daily backup of the database to meet regulatory requirements. The company must retain the backups for 30 days.

Which solution will meet these requirements with the LEAST operational overhead?

A.Write an AWS Lambda function to create an RDS snapshot every day.

B.Modify the RDS database to have a retention period of 30 days for automated backups.

C.Use AWS Systems Manager Maintenance Windows to modify the RDS backup retention period.

D.Create a manual snapshot every day by using the AWS CLI. Modify the RDS backup retention period.

Answer: B

Explanation:

The correct answer is **B. Modify the RDS database to have a retention period of 30 days for automated backups.**

Here's why:

Automated Backups are the Simplest: RDS automated backups are designed for exactly this purpose - creating and retaining backups for a specified duration. The default settings might not meet the 30-day retention requirement, so adjusting this setting is the most straightforward approach.

Reduced Operational Overhead: Modifying the backup retention period through the RDS console or AWS CLI is a one-time configuration change. This drastically reduces the operational overhead compared to solutions involving custom scripting or manual snapshots.

Compliance with Requirements: This solution directly addresses the company's need for daily backups retained for 30 days to meet regulatory requirements.

A is Incorrect: While a Lambda function could create snapshots, it adds significant operational overhead for development, deployment, monitoring, and maintenance. It's also unnecessary given the built-in RDS automated backup functionality.

C is Incorrect: Systems Manager Maintenance Windows are primarily for scheduling maintenance tasks like patching, not for managing RDS backup retention. While you could theoretically use it to initiate a backup retention change, it's overkill and less direct than simply configuring the retention period within RDS itself.

D is Incorrect: Manual snapshots are not automated. They require intervention every day, which introduces significant operational overhead. Also, manually managing retention policies for these snapshots would add more complexity.

In summary, utilizing RDS's built-in automated backup feature and adjusting the retention period to 30 days is the simplest, most efficient, and least operationally intensive way to meet the company's requirements for daily backups with a 30-day retention policy. This aligns with cloud best practices of leveraging managed services and minimizing custom code.

Authoritative Links:

Amazon RDS Backups:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_WorkingWithAutomatedBackups.html
Amazon RDS Backup Retention:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_WorkingWithAutomatedBackups.html#USER_V

Question: 864 **Exam Heist**

A company that runs its application on AWS uses an Amazon Aurora DB cluster as its database. During peak usage hours when multiple users access and read the data, the monitoring system shows degradation of database performance for the write queries. The company wants to increase the scalability of the application to meet peak usage demands.

Which solution will meet these requirements MOST cost-effectively?

A.Create a second Aurora DB cluster. Configure a copy job to replicate the users' data to the new database. Update the application to use the second database to read the data.

B.Create an Amazon DynamoDB Accelerator (DAX) cluster in front of the existing Aurora DB cluster. Update the application to use the DAX cluster for read-only queries. Write data directly to the Aurora DB cluster.

C.Create an Aurora read replica in the existing Aurora DB cluster. Update the application to use the replica endpoint for read-only queries and to use the cluster endpoint for write queries.

D.Create an Amazon Redshift cluster. Copy the users' data to the Redshift cluster. Update the application to connect to the Redshift cluster and to perform read-only queries on the Redshift cluster.

Answer: C

Explanation:

The optimal and most cost-effective solution is to create an Aurora read replica. Read replicas provide increased read capacity by offloading read workloads from the primary Aurora instance. This directly addresses the performance degradation issue experienced during peak read-heavy periods. Aurora read replicas share the same underlying storage as the primary instance, leading to minimal latency and data consistency. The application can be updated to direct read queries to the read replica's endpoint, leaving the primary instance to handle write operations without being burdened by read traffic. This separation of read and write operations allows the primary instance to efficiently manage write queries and maintain database performance. Options A and D are less cost-effective, as they involve maintaining an entirely separate database or data warehouse. Option B, using DAX, is more suitable for DynamoDB and not as efficient and cost-effective for Aurora as leveraging Aurora's native read replica functionality. Using Aurora's read replica offers the advantage of seamless integration within the Aurora ecosystem.

Reference:

Amazon Aurora Read Replicas

Question: 865 **Exam Heist**

A company's near-real-time streaming application is running on AWS. As the data is ingested, a job runs on the data and takes 30 minutes to complete. The workload frequently experiences high latency due to large amounts of incoming data. A solutions architect needs to design a scalable and serverless solution to enhance performance.

Which combination of steps should the solutions architect take? (Choose two.)

A.Use Amazon Kinesis Data Firehose to ingest the data.

B.Use AWS Lambda with AWS Step Functions to process the data.

C.Use AWS Database Migration Service (AWS DMS) to ingest the data.

D.Use Amazon EC2 instances in an Auto Scaling group to process the data.

E.Use AWS Fargate with Amazon Elastic Container Service (Amazon ECS) to process the data.

Answer: AE

Explanation:

The combination of using Amazon Kinesis Data Firehose for ingestion and AWS Fargate with Amazon ECS for processing is the most suitable serverless and scalable solution for this near-real-time streaming application.

A. Use Amazon Kinesis Data Firehose to ingest the data: Kinesis Data Firehose is designed for real-time data streaming into data lakes, data stores, and analytics services. It automatically scales to match the throughput of incoming data, handling large volumes with ease. This eliminates the need to manage the underlying

infrastructure for ingestion, making it a serverless choice. https://aws.amazon.com/kinesis/data-firehose/

E. Use AWS Fargate with Amazon Elastic Container Service (Amazon ECS) to process the data: AWS Fargate provides serverless compute for containers. Instead of managing EC2 instances, Fargate handles the underlying infrastructure. ECS provides the container orchestration layer, enabling you to deploy, manage, and scale containerized applications. Using Fargate with ECS allows the processing job to scale dynamically based on the incoming data volume from Kinesis Data Firehose. Containerization helps ensure consistent execution of the 30-minute processing job. https://aws.amazon.com/fargate/, https://aws.amazon.com/ecs/

Why other options are less suitable:

B. Use AWS Lambda with AWS Step Functions to process the data: While Lambda is serverless, its execution time is limited (currently up to 15 minutes). The 30-minute processing time exceeds this limit, making Lambda unsuitable.
C. Use AWS Database Migration Service (AWS DMS) to ingest the data: AWS DMS is designed for database migrations, not real-time data streaming ingestion. It's not appropriate for this use case.
D. Use Amazon EC2 instances in an Auto Scaling group to process the data: While EC2 Auto Scaling provides scalability, it introduces infrastructure management overhead. The requirement is for a serverless solution; thus Fargate is preferred.

Question: 866 **Exam Heist**

A company runs a web application on multiple Amazon EC2 instances in a VPC. The application needs to write sensitive data to an Amazon S3 bucket. The data cannot be sent over the public internet.

Which solution will meet these requirements?

A.Create a gateway VPC endpoint for Amazon S3. Create a route in the VPC route table to the endpoint.
B.Create an internal Network Load Balancer that has the S3 bucket as the target.
C.Deploy the S3 bucket inside the VPCreate a route in the VPC route table to the bucket.
D.Create an AWS Direct Connect connection between the VPC and an S3 regional endpoint.

Answer: A

Explanation:

The correct solution is A: Create a gateway VPC endpoint for Amazon S3 and a route in the VPC route table to the endpoint. This is because gateway VPC endpoints for S3 provide a direct, private connection to S3 within the AWS network, bypassing the public internet. EC2 instances within the VPC can then access S3 without requiring public IP addresses or traversing the internet. The route table entry directs traffic destined for S3 to the gateway endpoint. This approach ensures that sensitive data transfer remains secure and compliant with the requirement of not being sent over the public internet.

Option B is incorrect because an internal Network Load Balancer cannot target an S3 bucket directly. Load balancers distribute traffic to instances, IP addresses, or Lambda functions, but S3 is object storage and not a service that can be directly targeted by an NLB.

Option C is incorrect because S3 buckets cannot be deployed inside a VPC. S3 is a regional service managed by AWS and exists outside the VPC.

Option D is incorrect because AWS Direct Connect is used to establish a private connection between an on-premises environment and AWS. While it provides a private connection, it is more complex and costly than using a gateway VPC endpoint for the purpose of privately accessing S3 from within a VPC. A gateway endpoint is specifically designed for this scenario and is the simpler, more cost-effective solution.

For further reading, refer to the AWS documentation on VPC endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html and gateway endpoints specifically: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-s3.html.

Question: 867 **Exam Heist**

A company runs its production workload on Amazon EC2 instances with Amazon Elastic Block Store (Amazon EBS) volumes. A solutions architect needs to analyze the current EBS volume cost and to recommend optimizations. The recommendations need to include estimated monthly saving opportunities.

Which solution will meet these requirements?

A.Use Amazon Inspector reporting to generate EBS volume recommendations for optimization.
B.Use AWS Systems Manager reporting to determine EBS volume recommendations for optimization.
C.Use Amazon CloudWatch metrics reporting to determine EBS volume recommendations for optimization.
D.Use AWS Compute Optimizer to generate EBS volume recommendations for optimization.

Answer: D

Explanation:

The correct answer is D: Use AWS Compute Optimizer to generate EBS volume recommendations for optimization.

AWS Compute Optimizer directly addresses the requirements of analyzing EBS volume costs and providing optimization recommendations with estimated savings. Compute Optimizer analyzes the configuration and utilization metrics of your EC2 instances and EBS volumes. It uses machine learning to identify under-utilized or over-provisioned EBS volumes and suggests right-sizing options. These suggestions often involve migrating to different EBS volume types or reducing the size of existing volumes. Crucially, Compute Optimizer estimates the potential cost savings associated with each recommendation, fulfilling the need for quantifiable monthly savings estimates.

Option A is incorrect because Amazon Inspector focuses on security vulnerabilities within your infrastructure, not performance optimization or cost analysis of EBS volumes. Option B, AWS Systems Manager, is a management service that helps automate operational tasks across your AWS resources but doesn't provide EBS volume optimization recommendations or cost savings estimates. Option C, Amazon CloudWatch, provides monitoring and observability data, but it does not directly offer recommendations for EBS volume optimization or calculate associated cost savings. While you can derive insights from CloudWatch metrics, it requires manual analysis and doesn't provide the automated analysis and recommendations offered by Compute Optimizer.

Therefore, AWS Compute Optimizer is the most suitable tool for identifying EBS volume optimization opportunities and estimating monthly savings, aligning directly with the problem statement's requirements.

Relevant Link:

AWS Compute Optimizer: https://aws.amazon.com/compute-optimizer/

Question: 868 **Exam Heist**

A global company runs its workloads on AWS. The company's application uses Amazon S3 buckets across AWS Regions for sensitive data storage and analysis. The company stores millions of objects in multiple S3 buckets daily. The company wants to identify all S3 buckets that are not versioning-enabled.

Which solution will meet these requirements?

B.Use Amazon S3 Storage Lens to identify all S3 buckets that are not versioning-enabled across Regions.

C.Enable IAM Access Analyzer for S3 to identify all S3 buckets that are not versioning-enabled across Regions.

D.Create an S3 Multi-Region Access Point to identify all S3 buckets that are not versioning-enabled across Regions.

Answer: B

Explanation:

The correct answer is **B. Use Amazon S3 Storage Lens to identify all S3 buckets that are not versioning-enabled across Regions.**

Here's why:

Amazon S3 Storage Lens is a cloud storage analytics feature that provides organization-wide visibility into object storage usage and activity. It offers a single view across all your buckets to identify trends and outliers, flag data protection gaps, and optimize storage costs. A core capability is its ability to identify buckets that do not have versioning enabled, directly addressing the company's requirement to identify all S3 buckets across multiple AWS Regions that lack versioning. The dashboards and metrics provided by S3 Storage Lens are designed for this type of analysis, providing a comprehensive view that can scale to millions of objects and multiple regions. https://aws.amazon.com/s3/storage-lens/

Option C is incorrect because IAM Access Analyzer for S3 is primarily focused on identifying buckets with access configurations that might expose data unintentionally to external entities. While it deals with bucket security, it doesn't directly report on versioning status. Its focus is on external access, not internal configuration settings like versioning. https://aws.amazon.com/iam/features/access-analyzer/

Option D is incorrect because an S3 Multi-Region Access Point is designed to provide a single endpoint for accessing data stored in multiple S3 buckets across different AWS Regions. Its primary function is to simplify access and improve performance in multi-region deployments, not to analyze bucket configurations such as versioning status. It doesn't inherently provide information on which buckets have versioning enabled. https://aws.amazon.com/s3/features/multi-region-access-points/

Question: 869 **Exam Heist**

A company wants to enhance its ecommerce order-processing application that is deployed on AWS. The application must process each order exactly once without affecting the customer experience during unpredictable traffic surges.

Which solution will meet these requirements?

A.Create an Amazon Simple Queue Service (Amazon SQS) FIFO queue. Put all the orders in the SQS queue. Configure an AWS Lambda function as the target to process the orders.

B.Create an Amazon Simple Notification Service (Amazon SNS) standard topic. Publish all the orders to the SNS standard topic. Configure the application as a notification target.

C.Create a flow by using Amazon AppFlow. Send the orders to the flow. Configure an AWS Lambda function as the target to process the orders.

D.Configure AWS X-Ray in the application to track the order requests. Configure the application to process the orders by pulling the orders from Amazon CloudWatch.

Answer: A

Explanation:

The requirement is to process e-commerce orders exactly once, even during traffic surges, without affecting the customer experience. Option A, using Amazon SQS FIFO queues and a Lambda function, is the best solution for this scenario.

Here's why:

Exactly-Once Processing: SQS FIFO (First-In, First-Out) queues are designed to guarantee that messages are processed exactly once, in the order they are sent, eliminating duplicates and ensuring consistent order processing. This is crucial for e-commerce where duplicate order processing can lead to customer dissatisfaction and financial losses.
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues.html

Handling Traffic Surges: SQS is a fully managed queue service that automatically scales to handle unpredictable traffic surges. It decouples the order placement process from the actual order processing, ensuring that the application remains responsive to customers even during peak loads.

Asynchronous Processing: Using SQS allows for asynchronous order processing. The customer doesn't have to wait for the order to be fully processed before receiving confirmation. This improves the customer experience by providing immediate feedback and freeing up resources on the application servers.

Lambda Integration: AWS Lambda functions can be directly triggered by SQS messages. This allows for serverless and event-driven order processing. When a new order is placed in the queue, Lambda automatically invokes the processing function. Lambda also scales automatically and integrates seamlessly with SQS.
https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html

The other options are less suitable:

Option B (SNS): SNS standard topics do not guarantee message ordering or exactly-once delivery. They are designed for high-throughput, but at the expense of potential message duplication and out-of-order delivery.

Option C (AppFlow): AppFlow is a data integration service designed for transferring data between AWS services and SaaS applications. It's not the right tool for real-time order processing with exactly-once guarantees.

Option D (X-Ray and CloudWatch): X-Ray is a tracing service for debugging distributed applications. CloudWatch is a monitoring service. Neither provides the queueing and processing capabilities required for reliable order processing.

Question: 870 **Exam Heist**

A company has two AWS accounts: Production and Development. The company needs to push code changes in the Development account to the Production account. In the alpha phase, only two senior developers on the development team need access to the Production account. In the beta phase, more developers will need access to perform testing.

Which solution will meet these requirements?

A.Create two policy documents by using the AWS Management Console in each account. Assign the policy to developers who need access.

B.Create an IAM role in the Development account. Grant the IAM role access to the Production account. Allow developers to assume the role.

C.Create an IAM role in the Production account. Define a trust policy that specifies the Development account. Allow developers to assume the role.

D.Create an IAM group in the Production account. Add the group as a principal in a trust policy that specifies the Production account. Add developers to the group.

Answer: D

Explanation:

The proposed solution (D) suggests creating an IAM group in the Production account, adding it as a principal in a trust policy that specifies the Production account, and then adding developers to the group. This is incorrect because the goal is to allow developers from the Development account to access resources in the Production account, not developers already in the Production account to access Production account resources. A trust policy would normally specify the Development account as the trusted entity to allow cross-account access.

The correct approach leverages IAM roles for cross-account access (Option C). Here's a detailed explanation:

1. **IAM Role in Production Account:** Create an IAM role within the Production account. This role will define the permissions developers will have when accessing Production resources. The role's policy document specifies what actions the role can perform on which resources.
2. **Trust Policy:** Crucially, the role's trust policy dictates who is allowed to assume this role. The trust policy should specify the Development account's account ID as a trusted principal. This means only entities (users or roles) from the Development account can assume this Production role. This is how cross-account access is established.
3. **Developer Access:** In the Development account, grant the necessary developers (initially the two senior developers, and later more developers during the beta phase) permission to assume the IAM role in the Production account. This can be done by attaching an IAM policy to their individual IAM users or to an IAM group they belong to in the Development account. This policy will use the sts:AssumeRole action, specifying the ARN (Amazon Resource Name) of the Production account role.

This setup ensures that:

Least Privilege: Developers in the Development account only gain access to the Production account resources explicitly permitted by the Production account's role.
Auditing: All actions taken in the Production account by developers are auditable under the assumed role, linking their actions back to the Development account users who assumed the role.
Scalability: As more developers require access during the beta phase, you can simply add them to the Development account's IAM group with the sts:AssumeRole permission, avoiding the need to modify the Production account's role configuration frequently.
Centralized Control: The Production account maintains control over who can access its resources via the trust policy and the permissions attached to the role.

Option A is not scalable or centrally managed. Option B has the role in the wrong account. Developers need to assume a role in the Production account to access production resources.

Authoritative Links:

IAM Roles: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html
IAM Trust Policies: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_create_policytrust.html
AssumeRole API: https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRole.html

Question: 871 **Exam Heist**

A company wants to restrict access to the content of its web application. The company needs to protect the content by using authorization techniques that are available on AWS. The company also wants to implement a serverless architecture for authorization and authentication that has low login latency.

The solution must integrate with the web application and serve web content globally. The application currently has

a small user base, but the company expects the application's user base to increase.

Which solution will meet these requirements?

A.Configure Amazon Cognito for authentication. Implement Lambda@Edge for authorization. Configure Amazon CloudFront to serve the web application globally.

B.Configure AWS Directory Service for Microsoft Active Directory for authentication. Implement AWS Lambda for authorization. Use an Application Load Balancer to serve the web application globally.

C.Configure Amazon Cognito for authentication. Implement AWS Lambda for authorization. Use Amazon S3 Transfer Acceleration to serve the web application globally.

D.Configure AWS Directory Service for Microsoft Active Directory for authentication. Implement Lambda@Edge for authorization. Use AWS Elastic Beanstalk to serve the web application globally.

Answer: A

Explanation:

The correct answer is A because it provides a serverless, scalable, and globally distributed solution for authentication and authorization with low latency.

Let's break down why the other options are less suitable:

Option B: AWS Directory Service for Microsoft Active Directory is primarily designed for enterprises already using Active Directory and wanting to extend it to AWS. It's not ideal for a new, cloud-native application, and it's certainly not a serverless authentication solution. Moreover, Lambda alone might struggle to provide the low-latency global authorization needed without a globally distributed caching mechanism. An ALB does not serve web content directly; it requires underlying instances.

Option C: While Amazon Cognito provides authentication and Lambda can handle authorization, S3 Transfer Acceleration only accelerates uploads to S3, not content delivery. It doesn't address the requirement of serving web content globally. The web application needs more than just faster uploads.

Option D: AWS Directory Service has the same problem as in Option B. While Lambda@Edge provides global authorization capabilities, Elastic Beanstalk, while useful for deployment, adds operational overhead and doesn't inherently provide the global content delivery capabilities that CloudFront does. It also doesn't directly support serving web content globally without additional configurations for CDN.

Option A leverages the following key AWS services:

Amazon Cognito: Provides secure and scalable authentication. It handles user sign-up, sign-in, and access control for web and mobile applications. Cognito is serverless and can scale to millions of users. https://aws.amazon.com/cognito/
Lambda@Edge: Allows you to run Lambda functions at CloudFront edge locations. This enables low-latency authorization checks as requests are routed to the nearest edge location. It is a serverless compute service. https://aws.amazon.com/lambda/edge/
Amazon CloudFront: A content delivery network (CDN) that distributes web content globally, improving performance and reducing latency for users worldwide. It integrates seamlessly with Lambda@Edge for edge-based authorization. https://aws.amazon.com/cloudfront/

By combining these services, option A delivers a serverless, highly scalable, and globally distributed authorization solution with low latency, perfectly addressing the problem statement's requirements. Lambda@Edge allows for custom authorization logic to be executed close to the user, decreasing latency. CloudFront enables global content delivery, ensuring a good user experience irrespective of user location. Cognito is the standard AWS service for user authentication.

Question: 872 **Exam Heist**

A development team uses multiple AWS accounts for its development, staging, and production environments. Team members have been launching large Amazon EC2 instances that are underutilized. A solutions architect must prevent large instances from being launched in all accounts.

How can the solutions architect meet this requirement with the LEAST operational overhead?

A.Update the IAM policies to deny the launch of large EC2 instances. Apply the policies to all users.

B.Define a resource in AWS Resource Access Manager that prevents the launch of large EC2 instances.

C.Create an IAM role in each account that denies the launch of large EC2 instances. Grant the developers IAM group access to the role.

D.Create an organization in AWS Organizations in the management account with the default policy. Create a service control policy (SCP) that denies the launch of large EC2 instances, and apply it to the AWS accounts.

Answer: D

Explanation:

The correct answer is **D: Create an organization in AWS Organizations in the management account with the default policy. Create a service control policy (SCP) that denies the launch of large EC2 instances, and apply it to the AWS accounts.**

Here's a detailed justification:

AWS Organizations and SCPs for Centralized Control: AWS Organizations allows you to centrally manage and govern multiple AWS accounts. Service Control Policies (SCPs) are a powerful feature within Organizations that enable you to define guardrails and restrictions on the AWS services and actions that users and roles can perform within member accounts.

Least Operational Overhead: SCPs offer the least operational overhead because they are centrally managed from the Organizations management account. You define the policy once and apply it to multiple accounts, eliminating the need to configure individual IAM policies in each account. This significantly reduces administrative effort and ensures consistent enforcement across the environment.

Preventing Large Instance Launches: The SCP can be written to explicitly deny the ec2:RunInstances action when the instance type specified in the request matches a large instance size (e.g., m5.2xlarge, c5.4xlarge). This effectively prevents developers from launching such instances in any account where the SCP is in effect.

IAM Policies vs. SCPs: While IAM policies (option A) can also be used to restrict instance launches, they would need to be updated and applied to every user and account, resulting in significant administrative overhead. IAM roles (option C) also require per-account configuration.

Resource Access Manager (RAM) Inapplicability: AWS Resource Access Manager (RAM) is used for sharing AWS resources between accounts, not for restricting actions within accounts. Option B is therefore not a suitable solution for this problem.

Centralized Enforcement and Auditability: SCPs provide centralized enforcement, ensuring that the restrictions are always in place, even if users attempt to bypass them. They also improve auditability, allowing administrators to easily track which policies are applied to which accounts.

Therefore, using AWS Organizations and SCPs to deny the launch of large EC2 instances is the most efficient and scalable solution that meets the requirement with the least operational overhead.

Relevant Links:

AWS Organizations documentation
Service Control Policies (SCPs)

Question: 873 **Exam Heist**

A company has migrated a fleet of hundreds of on-premises virtual machines (VMs) to Amazon EC2 instances. The instances run a diverse fleet of Windows Server versions along with several Linux distributions. The company wants a solution that will automate inventory and updates of the operating systems. The company also needs a summary of common vulnerabilities of each instance for regular monthly reviews.

What should a solutions architect recommend to meet these requirements?

A.Set up AWS Systems Manager Patch Manager to manage all the EC2 instances. Configure AWS Security Hub to produce monthly reports.

B.Set up AWS Systems Manager Patch Manager to manage all the EC2 instances. Deploy Amazon Inspector, and configure monthly reports.

C.Set up AWS Shield Advanced, and configure monthly reports. Deploy AWS Config to automate patch installations on the EC2 instances.

D.Set up Amazon GuardDuty in the account to monitor all EC2 instances. Deploy AWS Config to automate patch installations on the EC2 instances.

Answer: B

Explanation:

Option B is the most appropriate solution to automate inventory, updates, and vulnerability assessments for the company's EC2 instances. AWS Systems Manager Patch Manager effectively addresses the need for operating system inventory and automated patching on both Windows and Linux instances. Patch Manager centralizes the management of patching across the diverse fleet, ensuring consistent and up-to-date systems.

Amazon Inspector complements Patch Manager by providing vulnerability assessments. Inspector automatically assesses EC2 instances for software vulnerabilities and unintended network exposure. By configuring monthly reports in Inspector, the company gains a summary of common vulnerabilities of each instance, fulfilling the requirement for regular monthly reviews.

Option A is incorrect because AWS Security Hub primarily focuses on security posture management by aggregating findings from other AWS security services like Inspector and GuardDuty. While Security Hub provides a consolidated view of security alerts, it doesn't directly perform vulnerability scanning and assessment at the instance level like Inspector does.Option C is incorrect because AWS Shield Advanced protects against DDoS attacks and doesn't manage EC2 instance patching or vulnerability assessments. AWS Config focuses on configuration management and compliance and can't automate patching.Option D is incorrect because Amazon GuardDuty provides threat detection by analyzing logs and network activity and doesn't automate patching or vulnerability assessments. AWS Config focuses on configuration management and compliance and can't automate patching.

Therefore, the combination of Systems Manager Patch Manager for patching and Amazon Inspector for vulnerability assessment, coupled with monthly reporting, comprehensively addresses the company's requirements.

Relevant links for further research:

AWS Systems Manager Patch Manager: https://docs.aws.amazon.com/systems-manager/latest/userguide/patch-manager.html
Amazon Inspector: https://docs.aws.amazon.com/inspector/latest/userguide/whatis.html

Question: 874 **Exam Heist**

A company hosts its application in the AWS Cloud. The application runs on Amazon EC2 instances in an Auto Scaling group behind an Elastic Load Balancing (ELB) load balancer. The application connects to an Amazon DynamoDB table.

For disaster recovery (DR) purposes, the company wants to ensure that the application is available from another AWS Region with minimal downtime.

Which solution will meet these requirements with the LEAST downtime?

A.Create an Auto Scaling group and an ELB in the DR Region. Configure the DynamoDB table as a global table. Configure DNS failover to point to the new DR Region's ELB.

B.Create an AWS CloudFormation template to create EC2 instances, ELBs, and DynamoDB tables to be launched when necessary. Configure DNS failover to point to the new DR Region's ELB.

C.Create an AWS CloudFormation template to create EC2 instances and an ELB to be launched when necessary. Configure the DynamoDB table as a global table. Configure DNS failover to point to the new DR Region's ELB.

D.Create an Auto Scaling group and an ELB in the DR Region. Configure the DynamoDB table as a global table. Create an Amazon CloudWatch alarm with an evaluation period of 10 minutes to invoke an AWS Lambda function that updates Amazon Route 53 to point to the DR Region's ELB.

Answer: C

Explanation:

The goal is to achieve minimal downtime DR for an application using EC2, ELB, and DynamoDB. Option C offers the best approach by pre-configuring most resources and leveraging DynamoDB's global table feature.

Here's why C is superior:

1. **DynamoDB Global Tables:** DynamoDB global tables provide near real-time replication of data across AWS Regions. This ensures that the DR Region has the latest data, minimizing data loss during a failover. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/globaltables.html
2. **CloudFormation Template:** Using a CloudFormation template allows you to define your EC2 and ELB infrastructure as code. When a DR event occurs, you can quickly launch the stack in the DR Region. This automates the deployment process and reduces the time needed to bring the application back online.
3. **DNS Failover:** Configuring DNS failover (using services like Amazon Route 53) allows you to redirect traffic from the primary Region to the DR Region in the event of a failure. This is crucial for ensuring that users can still access the application, albeit from the DR Region. https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-failover.html

Why other options are less ideal:

Option A: While it creates Auto Scaling group and ELB beforehand, without a CloudFormation template, the configuration might be inconsistent and difficult to manage.
Option B: Similar to Option A, lacks the pre-configured infrastructure management benefit of Auto Scaling.
Option D: Using CloudWatch alarms and Lambda for Route 53 updates adds complexity and potential latency to the failover process. Direct DNS failover is faster and simpler. Furthermore, a 10-minute evaluation period is too long for achieving minimal downtime.

In summary, option C minimizes downtime by replicating data in near real-time with DynamoDB global tables and utilizing infrastructure as code (CloudFormation) to rapidly deploy the application in the DR region and employing DNS failover to seamlessly switch traffic to the DR Region.

Question: 875 **Exam Heist**

A company runs an application on Amazon EC2 instances in a private subnet. The application needs to store and retrieve data in Amazon S3 buckets. According to regulatory requirements, the data must not travel across the public internet.

What should a solutions architect do to meet these requirements MOST cost-effectively?

A.Deploy a NAT gateway to access the S3 buckets.
B.Deploy AWS Storage Gateway to access the S3 buckets.
C.Deploy an S3 interface endpoint to access the S3 buckets.
D.Deploy an S3 gateway endpoint to access the S3 buckets.

Answer: D

Explanation:

The correct answer is **D. Deploy an S3 gateway endpoint to access the S3 buckets.**

Here's a detailed justification:

The primary requirement is that data transfers between the EC2 instances and S3 buckets must not traverse the public internet while also aiming for a cost-effective solution.

Gateway Endpoints: Gateway endpoints are designed specifically for accessing S3 and DynamoDB from within a VPC without using public IPs or NAT gateways. They operate at Layer 3 (Network Layer) and are highly cost-effective because there are no data processing or hourly charges associated with their use. Traffic to S3 via a gateway endpoint remains within the AWS network.
Interface Endpoints: Interface endpoints use AWS PrivateLink, which provides private connectivity to AWS services using private IP addresses within your VPC. While they also avoid public internet traffic, they incur hourly charges and data processing charges, making them less cost-effective than gateway endpoints for S3 access when only S3 is involved. Interface endpoints operate at Layer 4 (Transport Layer).
NAT Gateway: A NAT gateway allows instances in a private subnet to connect to the internet or other AWS services. However, traffic routed through a NAT gateway traverses the public internet to reach S3. This violates the stated requirement of keeping data off the public internet.
AWS Storage Gateway: AWS Storage Gateway is used to integrate on-premises environments with AWS storage services. It is not the correct solution for EC2 instances already running within AWS that need private access to S3. It also is more complex and costly than the endpoint solutions.

Therefore, deploying an S3 gateway endpoint is the most cost-effective way to meet the requirement of accessing S3 from a private subnet without traversing the public internet. Gateway endpoints are free of usage charges beyond the standard S3 storage and data transfer costs, and they are designed specifically for S3 and DynamoDB access.It leverages the AWS backbone and is the most direct way for the given scenario.

Further Research:

VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html
Gateway Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-s3.html
Interface Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-interface.html

Question: 876 **Exam Heist**

A company hosts an application on Amazon EC2 instances that run in a single Availability Zone. The application is accessible by using the transport layer of the Open Systems Interconnection (OSI) model. The company needs the application architecture to have high availability.

Which combination of steps will meet these requirements MOST cost-effectively? (Choose two.)

A.Configure new EC2 instances in a different Availability Zone. Use Amazon Route 53 to route traffic to all instances.

B.Configure a Network Load Balancer in front of the EC2 instances.

C.Configure a Network Load Balancer for TCP traffic to the instances. Configure an Application Load Balancer for HTTP and HTTPS traffic to the instances.

D.Create an Auto Scaling group for the EC2 instances. Configure the Auto Scaling group to use multiple Availability Zones. Configure the Auto Scaling group to run application health checks on the instances.

E.Create an Amazon CloudWatch alarm. Configure the alarm to restart EC2 instances that transition to a stopped state.

Answer: BD

Explanation:

Here's a breakdown of why options B and D are the most cost-effective solutions for achieving high availability for the described application:

B. Configure a Network Load Balancer in front of the EC2 instances.

High Availability: A Network Load Balancer (NLB) is designed for handling TCP traffic at high throughput and low latency. Critically, NLBs operate at layer 4 of the OSI model (transport layer), aligning with the application's described usage. They automatically distribute incoming traffic across multiple healthy EC2 instances.
Availability Zone Awareness: NLBs can distribute traffic across multiple Availability Zones within a region. This means that if one Availability Zone experiences an outage, the NLB can automatically route traffic to instances in other healthy Availability Zones, ensuring continuous application availability.
Health Checks: NLBs continuously monitor the health of registered EC2 instances. If an instance fails a health check, the NLB stops sending traffic to that instance, preventing users from experiencing disruptions.
Cost-Effectiveness: NLBs are highly scalable and cost-efficient. You pay for what you use, based on the number of connections and the amount of data processed. NLBs are generally more cost-effective than solutions that require complex custom configurations or larger infrastructure footprints.

D. Create an Auto Scaling group for the EC2 instances. Configure the Auto Scaling group to use multiple Availability Zones. Configure the Auto Scaling group to run application health checks on the instances.

High Availability: Auto Scaling groups (ASGs) allow you to maintain a desired number of EC2 instances automatically. By configuring the ASG to span multiple Availability Zones, you ensure that if one AZ fails, instances in other AZs can continue to serve traffic.
Automatic Recovery: ASGs can automatically replace unhealthy instances. When an EC2 instance fails a health check, the ASG terminates the unhealthy instance and launches a new instance to maintain the desired capacity. This self-healing capability enhances application availability.
Health Checks: ASGs integrate with Elastic Load Balancing (ELB) health checks. This allows the ASG to monitor the health of instances based on the application's responsiveness.
Cost-Effectiveness: ASGs enable you to scale your infrastructure up or down based on demand. This helps you optimize your costs by only paying for the resources you need. Also, the automated nature of ASGs reduces the need for manual intervention, which can save time and resources.

Why other options are less optimal:

A. Configure new EC2 instances in a different Availability Zone. Use Amazon Route 53 to route traffic to all instances. While this improves availability, Route 53 alone lacks the dynamic health-checking capabilities of NLB or ASG health checks. It's slower to react to instance failures and less efficient in distributing load.
C. Configure a Network Load Balancer for TCP traffic to the instances. Configure an Application Load Balancer for HTTP and HTTPS traffic to the instances. This is unnecessarily complex and expensive if the

application primarily uses TCP, as stated. An ALB is designed for HTTP/HTTPS traffic, and using both would introduce extra overhead.
E. Create an Amazon CloudWatch alarm. Configure the alarm to restart EC2 instances that transition to a stopped state. Restarting instances is not an effective high-availability strategy because it can cause downtime. This approach does not prevent disruptions as well as an NLB with an ASG.

Authoritative Links:

Network Load Balancer: https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html
Auto Scaling Groups: https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-groups.html
High Availability: https://aws.amazon.com/reliability/high-availability/

Question: 877 **Exam Heist**

A company uses Amazon S3 to host its static website. The company wants to add a contact form to the webpage. The contact form will have dynamic server-side components for users to input their name, email address, phone number, and user message.

The company expects fewer than 100 site visits each month. The contact form must notify the company by email when a customer fills out the form.

Which solution will meet these requirements MOST cost-effectively?

A.Host the dynamic contact form in Amazon Elastic Container Service (Amazon ECS). Set up Amazon Simple Email Service (Amazon SES) to connect to a third-party email provider.

B.Create an Amazon API Gateway endpoint that returns the contact form from an AWS Lambda function. Configure another Lambda function on the API Gateway to publish a message to an Amazon Simple Notification Service (Amazon SNS) topic.

C.Host the website by using AWS Amplify Hosting for static content and dynamic content. Use server-side scripting to build the contact form. Configure Amazon Simple Queue Service (Amazon SQS) to deliver the message to the company.

D.Migrate the website from Amazon S3 to Amazon EC2 instances that run Windows Server. Use Internet Information Services (IIS) for Windows Server to host the webpage. Use client-side scripting to build the contact form. Integrate the form with Amazon WorkMail.

Answer: B

Explanation:

The most cost-effective solution for a low-traffic contact form on an S3-hosted website is option B. Here's why:

Scalability and Cost: Lambda and API Gateway offer a serverless architecture. You only pay for the actual invocations, making it incredibly cheap for under 100 monthly visits. ECS (A) would involve running containers constantly, incurring higher costs even when idle. EC2 (D) involves substantial fixed costs for the instance. Amplify Hosting (C) can be suitable, but the chosen components make it less economical compared to a fully serverless approach.

Dynamic Content Handling: API Gateway and Lambda easily handle dynamic content. The Lambda function can generate and serve the initial contact form. The other Lambda function is triggered by the API Gateway upon form submission.

Email Notification: SNS provides a straightforward way to send email notifications. When the contact form is submitted, the Lambda function publishes a message to the SNS topic, which in turn triggers an email to the company. SES is needed to send the mail using AWS.

Simplicity: API Gateway, Lambda, and SNS are straightforward services to configure and integrate. SQS (C) is

not directly used to send mails, and it is less appropriate for notification of the contact forms.

Option D is highly inefficient and costly. Migrating to EC2 for a simple contact form overcomplicates the setup, resulting in higher maintenance overhead and costs. Client-side scripting might work for basic form validation, but it can't handle server-side logic or send emails directly. Option A, while functional, introduces more complexity than needed and may incur higher costs due to ECS cluster maintenance. Option C is reasonable, but SQS is not specifically suited to send email notifications.

Authoritative Links:

AWS Lambda: https://aws.amazon.com/lambda/
Amazon API Gateway: https://aws.amazon.com/api-gateway/
Amazon SNS: https://aws.amazon.com/sns/
Amazon SES: https://aws.amazon.com/ses/

Question: 878 **Exam Heist**

A company creates dedicated AWS accounts in AWS Organizations for its business units. Recently, an important notification was sent to the root user email address of a business unit account instead of the assigned account owner. The company wants to ensure that all future notifications can be sent to different employees based on the notification categories of billing, operations, or security.

Which solution will meet these requirements MOST securely?

A.Configure each AWS account to use a single email address that the company manages. Ensure that all account owners can access the email account to receive notifications. Configure alternate contacts for each AWS account with corresponding distribution lists for the billing team, the security team, and the operations team for each business unit.

B.Configure each AWS account to use a different email distribution list for each business unit that the company manages. Configure each distribution list with administrator email addresses that can respond to alerts. Configure alternate contacts for each AWS account with corresponding distribution lists for the billing team, the security team, and the operations team for each business unit.

C.Configure each AWS account root user email address to be the individual company managed email address of one person from each business unit. Configure alternate contacts for each AWS account with corresponding distribution lists for the billing team, the security team, and the operations team for each business unit.

D.Configure each AWS account root user to use email aliases that go to a centralized mailbox. Configure alternate contacts for each account by using a single business managed email distribution list each for the billing team, the security team, and the operations team.

Answer: A

Explanation:

Here's a detailed justification for why option A is the most secure and appropriate solution, along with supporting concepts and links:

Option A is the most secure because it utilizes alternate contacts and a centrally managed email strategy. By configuring each AWS account to use a single, company-managed email address (that authorized personnel can access) for the root user, the company retains greater control and auditing capabilities. This avoids tying critical notifications to individual employee email addresses which can pose a security risk if an employee leaves the organization. Furthermore, configuring alternate contacts with distribution lists for billing, security, and operations teams ensures that the right stakeholders receive the right notifications without granting unnecessary root user access. This approach follows the principle of least privilege.

Option B is less secure. Distributing root user email to distribution lists raises security concerns. It increases the attack surface because more individuals potentially have access to highly sensitive information and actions associated with the root user.

Option C is less secure. While using individual company-managed emails seems better, it still ties root user access to specific individuals. If that person leaves, there's a risk the email isn't properly transitioned. Also, relying on individuals rather than distribution lists for functional roles (billing, security) is operationally brittle.

Option D is less secure. Using aliases and centralized mailboxes for root user emails can be acceptable, but it needs careful management. The primary issue here is it only creates ONE distribution list for each category (billing, security, operations) across all accounts. This isn't ideal; each AWS account should ideally have its own specific distribution lists for alternate contacts, improving isolation and reducing the risk of cross-account information leakage.

In summary, Option A provides the best balance of security, operational efficiency, and adherence to the principle of least privilege by centralizing root user access management and leveraging alternate contacts with dedicated distribution lists.

Relevant links for further research:

AWS Organizations best practices: https://docs.aws.amazon.com/organizations/latest/userguide/orgs_best-practices.html
AWS Account Root User: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_root-user.html
Alternate Contacts: https://docs.aws.amazon.com/accounts/latest/reference/using-orgs-managing-accounts.html

Question: 879 **Exam Heist**

A company runs an ecommerce application on AWS. Amazon EC2 instances process purchases and store the purchase details in an Amazon Aurora PostgreSQL DB cluster.

Customers are experiencing application timeouts during times of peak usage. A solutions architect needs to rearchitect the application so that the application can scale to meet peak usage demands.

Which combination of actions will meet these requirements MOST cost-effectively? (Choose two.)

A.Configure an Auto Scaling group of new EC2 instances to retry the purchases until the processing is complete. Update the applications to connect to the DB cluster by using Amazon RDS Proxy.

B.Configure the application to use an Amazon ElastiCache cluster in front of the Aurora PostgreSQL DB cluster.

C.Update the application to send the purchase requests to an Amazon Simple Queue Service (Amazon SQS) queue. Configure an Auto Scaling group of new EC2 instances that read from the SQS queue.

D.Configure an AWS Lambda function to retry the ticket purchases until the processing is complete.

E.Configure an Amazon AP! Gateway REST API with a usage plan.

Answer: BC

Explanation:

The best combination of actions to address the timeouts and scale the ecommerce application cost-effectively is B and C.

B. Configure the application to use an Amazon ElastiCache cluster in front of the Aurora PostgreSQL DB cluster:

ElastiCache acts as an in-memory cache, storing frequently accessed data. This significantly reduces the load on the Aurora PostgreSQL database by serving read requests from the cache instead of the database. This directly addresses the performance bottlenecks caused by heavy read operations during peak usage. Using ElastiCache improves response times and reduces database load, enabling the application to handle more concurrent requests. This also avoids costly resizing of database instance to address peak read demands.https://aws.amazon.com/elasticache/

C. Update the application to send the purchase requests to an Amazon Simple Queue Service (Amazon SQS) queue. Configure an Auto Scaling group of new EC2 instances that read from the SQS queue:

Introducing an SQS queue decouples the purchase request processing from the immediate user interaction. When a customer initiates a purchase, the request is placed in the SQS queue. An Auto Scaling group of EC2 instances then retrieves these requests from the queue and processes them asynchronously. This ensures that the application remains responsive to user requests, even during peak loads. The Auto Scaling group automatically scales up or down based on the queue depth, providing elasticity and cost optimization. By offloading processing to a queue, the application can handle a much higher volume of requests without experiencing timeouts. This distributed queueing is a classic pattern for handling spiky workloads.https://aws.amazon.com/sqs/

Why other options are not as suitable:

A: While RDS Proxy can help manage database connections, simply retrying purchases on the same overloaded EC2 instances will likely perpetuate the problem and not effectively address the fundamental scaling bottleneck.
D: Lambda functions have execution time limits and might not be suitable for long-running purchase processing tasks. Furthermore, retrying long running tasks from Lambda may lead to throttling issues.
E: API Gateway usage plans primarily manage API request rates and quotas. While useful for controlling access, they do not directly address the underlying scaling issues related to database load or processing capacity.

Therefore, using ElastiCache for caching and SQS with Auto Scaling for asynchronous processing is the most cost-effective and efficient approach to scaling the ecommerce application to handle peak usage demands.

Question: 880 **Exam Heist**

A company that uses AWS Organizations runs 150 applications across 30 different AWS accounts. The company used AWS Cost and Usage Report to create a new report in the management account. The report is delivered to an Amazon S3 bucket that is replicated to a bucket in the data collection account.

The company's senior leadership wants to view a custom dashboard that provides NAT gateway costs each day starting at the beginning of the current month.

Which solution will meet these requirements?

A.Share an Amazon QuickSight dashboard that includes the requested table visual. Configure QuickSight to use AWS DataSync to query the new report.

B.Share an Amazon QuickSight dashboard that includes the requested table visual. Configure QuickSight to use Amazon Athena to query the new report.

C.Share an Amazon CloudWatch dashboard that includes the requested table visual. Configure CloudWatch to use AWS DataSync to query the new report.

D.Share an Amazon CloudWatch dashboard that includes the requested table visual. Configure CloudWatch to use Amazon Athena to query the new report.

Answer: B

Explanation:

The requirement is to create a custom dashboard showing daily NAT gateway costs from the beginning of the month, sourced from the AWS Cost and Usage Report (CUR) across 30 AWS accounts managed by AWS Organizations. QuickSight and Athena are the appropriate services to achieve this goal.

Option B suggests using QuickSight for dashboarding and Athena to query the CUR data. QuickSight is a

business intelligence service that allows you to create interactive dashboards and visualizations. Athena is a serverless query service that allows you to analyze data stored in Amazon S3 using standard SQL. The CUR is delivered to an S3 bucket, making it easily accessible to Athena. Athena can then query the CUR data to extract the daily NAT gateway costs. Finally, QuickSight can connect to Athena to visualize the data in a custom dashboard.

Option A suggests using QuickSight for dashboarding and AWS DataSync to query the CUR report. While DataSync efficiently moves large amounts of data between on-premises storage and AWS, it does not provide a mechanism for querying and analyzing the report data directly. DataSync is primarily used for data migration and replication, not SQL-based querying. This approach wouldn't readily allow for extraction of the specific daily NAT Gateway costs.

Option C suggests using CloudWatch for dashboarding and AWS DataSync to query the CUR report. CloudWatch is primarily a monitoring service for AWS resources and applications. It is suitable for visualizing metrics, logs, and events, but not for querying complex structured data like the CUR. Like option A, DataSync is not designed for querying the CUR report.

Option D suggests using CloudWatch for dashboarding and Athena to query the CUR report. While Athena can query the CUR data, CloudWatch is not the best tool for creating interactive dashboards. It's mainly intended for monitoring operational metrics, logs, and events. QuickSight is purpose-built for data visualization and creating custom dashboards.

Therefore, using QuickSight for the dashboard and Athena for querying the CUR data is the most suitable and cost-effective solution. Athena offers the flexibility to query and filter the CUR data according to the specified requirements (daily NAT gateway costs from the beginning of the month), and QuickSight enables visualizing that data in a customized dashboard.

Relevant links for further research:

AWS QuickSight: https://aws.amazon.com/quicksight/
Amazon Athena: https://aws.amazon.com/athena/
AWS Cost and Usage Report: https://docs.aws.amazon.com/cur/latest/userguide/what-is-cur.html
AWS CloudWatch: https://aws.amazon.com/cloudwatch/
AWS DataSync: https://aws.amazon.com/datasync/

Question: 881 **Exam Heist**

A company is hosting a high-traffic static website on Amazon S3 with an Amazon CloudFront distribution that has a default TTL of 0 seconds. The company wants to implement caching to improve performance for the website. However, the company also wants to ensure that stale content is not served for more than a few minutes after a deployment.

Which combination of caching methods should a solutions architect implement to meet these requirements? (Choose two.)

A.Set the CloudFront default TTL to 2 minutes.

B.Set a default TTL of 2 minutes on the S3 bucket.

C.Add a Cache-Control private directive to the objects in Amazon S3.

D.Create an AWS Lambda@Edge function to add an Expires header to HTTP responses. Configure the function to run on viewer response.

E.Add a Cache-Control max-age directive of 24 hours to the objects in Amazon S3. On deployment, create a CloudFront invalidation to clear any changed files from edge caches.

Answer: AC

Explanation:

Let's break down why options A and C are the best choices for implementing caching in this scenario while minimizing stale content.

Option A: Set the CloudFront default TTL to 2 minutes.

Setting a default TTL (Time To Live) on the CloudFront distribution is a fundamental step for enabling caching. TTL dictates how long CloudFront edge locations will store a copy of the content before checking back with the origin (S3 in this case) for a fresh version. A TTL of 2 minutes directly addresses the requirement to avoid serving stale content for more than a few minutes. With a 0-second TTL, CloudFront currently fetches every request from S3, negating the benefits of a CDN. A short TTL allows for more frequent updates while still reducing the load on S3 and improving latency for users.

Option C: Add a Cache-Control private directive to the objects in Amazon S3.

The Cache-Control: private directive is crucial because it specifies that the content should only be cached by the browser (viewer) making the request and not by intermediate caches such as proxies or, importantly, CloudFront. However, in this case the object is stored in S3, so this option will not achieve the goal of caching the object on the CloudFront side. A Cache-Control: public on the S3 object, along with a CloudFront TTL, allows CloudFront to cache the object. But this setup is not enough to fulfill the requirements of the use case, it needs the option A to be enabled. So option C is not suitable to be implemented and it is not part of the right answer.

The correct answer is **AE**.

Option A: Set the CloudFront default TTL to 2 minutes.

Setting a default TTL on the CloudFront distribution is fundamental for enabling caching. TTL dictates how long CloudFront edge locations will store content before checking back with the origin (S3) for a fresh version. A TTL of 2 minutes directly addresses the requirement to avoid serving stale content for more than a few minutes. A zero-second TTL causes CloudFront to fetch every request from S3, negating CDN benefits.

Option E: Add a Cache-Control max-age directive of 24 hours to the objects in Amazon S3. On deployment, create a CloudFront invalidation to clear any changed files from edge caches.

Adding Cache-Control: max-age=86400 (24 hours) to S3 objects instructs CloudFront to cache the objects for up to 24 hours. This maximizes caching effectiveness and reduces S3 origin requests. The key here is the subsequent CloudFront invalidation. After a deployment with content changes, an invalidation tells CloudFront to remove existing cached versions of specific files. This forces CloudFront to fetch the latest versions from S3, ensuring users always receive the most up-to-date content. This avoids the problem of the potentially stale objects.

Let's examine why the other options are incorrect:

Option B: Setting a default TTL on the S3 bucket itself is not applicable. S3 buckets don't have TTL settings in the way that CloudFront distributions do. S3 focuses on storage and object management, not content delivery and caching.
Option D: While Lambda@Edge can be used for adding or modifying headers, it's unnecessary and adds complexity to this scenario. Setting the Cache-Control header directly on the S3 objects is a simpler and more efficient approach.

Authoritative Links:

CloudFront TTL: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Expiration.html
CloudFront Invalidation:
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Invalidation.html

S3 Cache-Control: https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingMetadata.html

Question: 882 **Exam Heist**

A company runs its application by using Amazon EC2 instances and AWS Lambda functions. The EC2 instances run in private subnets of a VPC. The Lambda functions need direct network access to the EC2 instances for the application to work.

The application will run for 1 year. The number of Lambda functions that the application uses will increase during the 1-year period. The company must minimize costs on all application resources.

Which solution will meet these requirements?

A.Purchase an EC2 Instance Savings Plan. Connect the Lambda functions to the private subnets that contain the EC2 instances.

B.Purchase an EC2 Instance Savings Plan. Connect the Lambda functions to new public subnets in the same VPC where the EC2 instances run.

C.Purchase a Compute Savings Plan. Connect the Lambda functions to the private subnets that contain the EC2 instances.

D.Purchase a Compute Savings Plan. Keep the Lambda functions in the Lambda service VPC.

Answer: C

Explanation:

The correct answer is C: Purchase a Compute Savings Plan. Connect the Lambda functions to the private subnets that contain the EC2 instances.

Here's a breakdown of why this is the most cost-effective and functional solution:

Lambda and VPC Access: Lambda functions, by default, don't have access to resources inside a VPC. To grant this access, you need to connect them to the VPC's subnets. The question explicitly states that the Lambda functions require direct network access to the EC2 instances. Therefore, the Lambda function must be inside of the VPC.

Private Subnets: Since the EC2 instances reside in private subnets, connecting the Lambda functions to those same private subnets allows them to communicate directly with the EC2 instances without exposing them to the public internet. Option B's proposal of using public subnets introduces unnecessary security risks and is not aligned with best practices when EC2 instances are designed to be private.

Savings Plans (Compute vs. EC2 Instance): Savings Plans offer discounted pricing in exchange for a commitment to a consistent amount of compute usage over a one- or three-year term. There are two types relevant here: Compute Savings Plans and EC2 Instance Savings Plans. A Compute Savings Plan is a better option than an EC2 Instance Savings Plan in this situation. This is because a Compute Savings Plan gives you flexibility across EC2, AWS Lambda, and AWS Fargate. Since the question stated that the application will use both EC2 instances and Lambda functions (and that the number of Lambda functions will increase), a Compute Savings Plan better addresses the overall compute needs.

Cost Optimization: A Compute Savings Plan provides cost savings on both the EC2 instances and Lambda functions. This is essential to meet the "minimize costs" requirement. Since both EC2 and Lambda resources are in the compute footprint, a Compute Savings Plan is preferred over an EC2 Instance Savings Plan.

Lambda in Lambda Service VPC (Option D): The solution specifies that the Lambda functions need direct network access to the EC2 instances. Lambda functions kept in the Lambda service VPC do not have direct network access to a customer's VPC. The solution would therefore fail to fulfill a critical requirement.

In summary, connecting the Lambda functions to the existing private subnets and purchasing a Compute

Savings Plan provides the required connectivity and cost-effectiveness across both EC2 and Lambda, making option C the best solution.

Supporting Documentation:

AWS Lambda VPC Networking: https://docs.aws.amazon.com/lambda/latest/dg/configuration-vpc.html
AWS Savings Plans: https://aws.amazon.com/savingsplans/
Choosing the right Savings Plan: https://aws.amazon.com/savingsplans/faq/

Question: 883 **Exam Heist**

A company has deployed a multi-account strategy on AWS by using AWS Control Tower. The company has provided individual AWS accounts to each of its developers. The company wants to implement controls to limit AWS resource costs that the developers incur.

Which solution will meet these requirements with the LEAST operational overhead?

A.Instruct each developer to tag all their resources with a tag that has a key of CostCenter and a value of the developer's name. Use the required-tags AWS Config managed rule to check for the tag. Create an AWS Lambda function to terminate resources that do not have the tag. Configure AWS Cost Explorer to send a daily report to each developer to monitor their spending.

B.Use AWS Budgets to establish budgets for each developer account. Set up budget alerts for actual and forecast values to notify developers when they exceed or expect to exceed their assigned budget. Use AWS Budgets actions to apply a DenyAll policy to the developer's IAM role to prevent additional resources from being launched when the assigned budget is reached.

C.Use AWS Cost Explorer to monitor and report on costs for each developer account. Configure Cost Explorer to send a daily report to each developer to monitor their spending. Use AWS Cost Anomaly Detection to detect anomalous spending and provide alerts.

D.Use AWS Service Catalog to allow developers to launch resources within a limited cost range. Create AWS Lambda functions in each AWS account to stop running resources at the end of each work day. Configure the Lambda functions to resume the resources at the start of each work day.

Answer: B

Explanation:

The most effective and least operationally intensive solution to control AWS resource costs for developers in a multi-account AWS Control Tower environment is to use AWS Budgets. Here's why:

AWS Budgets: This service is designed specifically for cost management. You can create budgets at the account level, perfectly aligning with individual developer accounts. (https://aws.amazon.com/aws-cost-management/aws-budgets/)
Budget Alerts: AWS Budgets allow setting alerts based on actual and forecasted costs. Developers receive timely notifications when they are approaching or exceeding their allocated budget, promoting cost awareness.
Budget Actions: AWS Budgets Actions offer automated responses to budget breaches. Applying a DenyAll policy to a developer's IAM role upon exceeding the budget prevents further resource provisioning, effectively capping costs.
Least Operational Overhead: AWS Budgets integrates seamlessly with AWS accounts and IAM. Configuration requires minimal custom coding or infrastructure, resulting in lower operational overhead compared to other options.

Let's analyze why the other options are less ideal:

Option A (Tagging and Lambda): While tagging is a good practice for cost allocation, relying on developers to consistently tag resources and using Lambda to enforce it is prone to errors and requires ongoing

maintenance. required-tags checks compliance but doesn't actively limit costs.
Option C (Cost Explorer and Anomaly Detection): Cost Explorer and Anomaly Detection are valuable for monitoring but don't prevent developers from exceeding budgets. They provide reactive alerts, not proactive cost control.
Option D (Service Catalog and Lambda): Service Catalog can limit resource types, but controlling costs within that limit is challenging. Using Lambda to start/stop resources introduces complexity and might disrupt development workflows. It also limits developer flexibility.

In summary, AWS Budgets, with its alerting and action capabilities, offers the most direct and automated way to limit AWS resource costs for developers while minimizing operational burden in an AWS Control Tower multi-account setup.

Question: 884 **Exam Heist**

A solutions architect is designing a three-tier web application. The architecture consists of an internet-facing Application Load Balancer (ALB) and a web tier that is hosted on Amazon EC2 instances in private subnets. The application tier with the business logic runs on EC2 instances in private subnets. The database tier consists of Microsoft SQL Server that runs on EC2 instances in private subnets. Security is a high priority for the company.

Which combination of security group configurations should the solutions architect use? (Choose three.)

A.Configure the security group for the web tier to allow inbound HTTPS traffic from the security group for the ALB.

B.Configure the security group for the web tier to allow outbound HTTPS traffic to 0.0.0.0/0.

C.Configure the security group for the database tier to allow inbound Microsoft SQL Server traffic from the security group for the application tier.

D.Configure the security group for the database tier to allow outbound HTTPS traffic and Microsoft SQL Server traffic to the security group for the web tier.

E.Configure the security group for the application tier to allow inbound HTTPS traffic from the security group for the web tier.

F.Configure the security group for the application tier to allow outbound HTTPS traffic and Microsoft SQL Server traffic to the security group for the web tier.

Answer: ACE

Explanation:

Let's break down why options A, C, and E are the correct security group configurations for this three-tier architecture, focusing on the principle of least privilege and defense in depth.

A: Configure the security group for the web tier to allow inbound HTTPS traffic from the security group for the ALB. This is essential because the ALB acts as the entry point for all external HTTPS requests. By allowing inbound traffic only from the ALB's security group, you restrict access to the web tier to only legitimate requests routed through the load balancer, preventing direct access from the internet. This adheres to the principle of least privilege.

C: Configure the security group for the database tier to allow inbound Microsoft SQL Server traffic from the security group for the application tier. The application tier needs to communicate with the database tier to read and write data. Restricting inbound SQL Server (typically port 1433) traffic only from the application tier's security group is crucial. This prevents unauthorized access to the database from any other source, which is a fundamental security best practice.

E: Configure the security group for the application tier to allow inbound HTTPS traffic from the security group for the web tier. The application tier processes the business logic based on requests it receives from the web tier. Allowing inbound HTTPS (or, ideally, internal HTTP on port 80 if using TLS termination at the

web tier) traffic only from the web tier's security group ensures that only the web tier can initiate requests to the application tier.

Now, let's discuss why the other options are incorrect:

B: Configure the security group for the web tier to allow outbound HTTPS traffic to 0.0.0.0/0. Allowing unrestricted outbound traffic (0.0.0.0/0) from the web tier is a security risk. While outbound internet access may be necessary for some operations (e.g., updates, logging), it should be limited to specific, known destinations and protocols. In many setups, the EC2 instance in the private subnet can leverage a NAT gateway in the public subnet. If the instance only has to connect to AWS services, VPC endpoints are a more secure and preferred solution.

D: Configure the security group for the database tier to allow outbound HTTPS traffic and Microsoft SQL Server traffic to the security group for the web tier. The database tier should not be initiating connections back to the web tier. The communication flow is typically from the web tier to the application tier to the database tier. Allowing the database tier to initiate connections to other tiers is a security violation.

F: Configure the security group for the application tier to allow outbound HTTPS traffic and Microsoft SQL Server traffic to the security group for the web tier. Similar to option D, the application tier needs to communicate only with the database tier, not the web tier. The standard pattern has the web tier fronting the application tier.

In summary, the correct security group configuration focuses on restricting traffic based on the principle of least privilege, allowing only necessary communication between tiers and blocking all other access.

Further Reading:

AWS Security Groups
Network Access Control Lists (NACLs)
AWS Security Best Practices

Question: 885 **Exam Heist**

A company has released a new version of its production application. The company's workload uses Amazon EC2, AWS Lambda, AWS Fargate, and Amazon SageMaker.

The company wants to cost optimize the workload now that usage is at a steady state. The company wants to cover the most services with the fewest savings plans.

Which combination of savings plans will meet these requirements? (Choose two.)

A.Purchase an EC2 Instance Savings Plan for Amazon EC2 and SageMaker.
B.Purchase a Compute Savings Plan for Amazon EC2, Lambda, and SageMaker.
C.Purchase a SageMaker Savings Plan.
D.Purchase a Compute Savings Plan for Lambda, Fargate, and Amazon EC2.
E.Purchase an EC2 Instance Savings Plan for Amazon EC2 and Fargate.

Answer: CD

Explanation:

Here's a breakdown of why the correct answer is C and D, and why the other options are less suitable:

Why C (Purchase a SageMaker Savings Plan) is correct: SageMaker Savings Plans are specifically designed to cover the costs associated with using Amazon SageMaker. Since the company's workload includes SageMaker, purchasing this savings plan directly addresses a key service contributing to their overall

expenses. This is the most direct and cost-effective way to optimize SageMaker costs.

Why D (Purchase a Compute Savings Plan for Lambda, Fargate, and Amazon EC2) is correct: Compute Savings Plans offer flexibility and apply to EC2, Lambda, and Fargate usage. This single savings plan covers a substantial portion of the company's workload (Lambda, Fargate, and EC2), providing significant cost optimization across these services with a single commitment. Compute Savings Plan provides the flexibility to change instance types, operating systems, tenancies, and AWS Regions while still benefiting from the savings plan price.

Why A (Purchase an EC2 Instance Savings Plan for Amazon EC2 and SageMaker) is incorrect: EC2 Instance Savings Plans are designed for EC2 instance usage within a specific instance family and AWS Region. While it covers EC2, it **does not** apply to SageMaker directly.

Why B (Purchase a Compute Savings Plan for Amazon EC2, Lambda, and SageMaker) is incorrect: While Compute Savings Plans do apply to EC2 and Lambda, they **do not** cover SageMaker costs directly. Using a SageMaker Savings Plan to cost optimize SageMaker will be more effective.

Why E (Purchase an EC2 Instance Savings Plan for Amazon EC2 and Fargate) is incorrect: EC2 Instance Savings Plans are specific to EC2 instance families within a region and cannot be directly used for Fargate. Fargate is covered by compute Savings Plans.

By selecting a SageMaker Savings Plan (C) and a Compute Savings Plan (D), the company efficiently covers all their core services (EC2, Lambda, Fargate, and SageMaker) with minimal complexity, achieving cost optimization across their entire workload.

Here are authoritative links for further research:

AWS Savings Plans: https://aws.amazon.com/savingsplans/
SageMaker Savings Plans: https://aws.amazon.com/sagemaker/pricing/
EC2 Instance Savings Plans: https://aws.amazon.com/savingsplans/compute-savings-plans/
Compute Savings Plans: https://aws.amazon.com/savingsplans/compute-savings-plans/

Question: 886 **Exam Heist**

A company uses a Microsoft SQL Server database. The company's applications are connected to the database. The company wants to migrate to an Amazon Aurora PostgreSQL database with minimal changes to the application code.

Which combination of steps will meet these requirements? (Choose two.)

A.Use the AWS Schema Conversion Tool (AWS SCT) to rewrite the SQL queries in the applications.

B.Enable Babelfish on Aurora PostgreSQL to run the SQL queries from the applications.

C.Migrate the database schema and data by using the AWS Schema Conversion Tool (AWS SCT) and AWS Database Migration Service (AWS DMS).

D.Use Amazon RDS Proxy to connect the applications to Aurora PostgreSQL.

E.Use AWS Database Migration Service (AWS DMS) to rewrite the SQL queries in the applications.

Answer: BC

Explanation:

The correct answer is BC. Here's why:

B. Enable Babelfish on Aurora PostgreSQL to run the SQL queries from the applications.

Babelfish for Aurora PostgreSQL is specifically designed to allow Aurora PostgreSQL to understand and

process SQL Server Transact-SQL (T-SQL) commands directly. This drastically minimizes the need for application code changes during migration from Microsoft SQL Server. By enabling Babelfish, the applications can continue to send T-SQL queries, and Babelfish translates them into a format Aurora PostgreSQL can understand, thus fulfilling the requirement of minimal application changes. This approach avoids rewriting queries in the application code, making it the quickest method to get compatible behavior.

https://aws.amazon.com/rds/aurora/babelfish/

C. Migrate the database schema and data by using the AWS Schema Conversion Tool (AWS SCT) and AWS Database Migration Service (AWS DMS).

AWS SCT helps convert the database schema from Microsoft SQL Server to Aurora PostgreSQL. It identifies potential compatibility issues and provides recommendations for resolving them. AWS DMS then migrates the actual data from the source SQL Server database to the Aurora PostgreSQL database. This is the standard approach for migrating database schemas and data to AWS, especially when dealing with different database engines. It ensures that the schema is properly converted and the data is transferred accurately and efficiently.

https://aws.amazon.com/dms/https://aws.amazon.com/sct/

Why other options are incorrect:

A. Use the AWS Schema Conversion Tool (AWS SCT) to rewrite the SQL queries in the applications: AWS SCT primarily focuses on schema conversion and reporting, not directly rewriting application queries. While SCT can provide guidance for adapting queries, automating this in application code is beyond its main function and would involve significant manual effort, conflicting with the "minimal changes" requirement.

D. Use Amazon RDS Proxy to connect the applications to Aurora PostgreSQL: RDS Proxy is used for managing database connections and improving application scalability, resilience, and security. It does not handle query translation or schema conversion, so it wouldn't address the core requirement of application compatibility with Aurora PostgreSQL.

E. Use AWS Database Migration Service (AWS DMS) to rewrite the SQL queries in the applications: AWS DMS is designed for data migration, not for rewriting SQL queries in the application layer. Its primary function is to replicate data changes between database systems.

Question: 887 **Exam Heist**

A company plans to rehost an application to Amazon EC2 instances that use Amazon Elastic Block Store (Amazon EBS) as the attached storage.

A solutions architect must design a solution to ensure that all newly created Amazon EBS volumes are encrypted by default. The solution must also prevent the creation of unencrypted EBS volumes.

Which solution will meet these requirements?

A.Configure the EC2 account attributes to always encrypt new EBS volumes.

B.Use AWS Config. Configure the encrypted-volumes identifier. Apply the default AWS Key Management Service (AWS KMS) key.

C.Configure AWS Systems Manager to create encrypted copies of the EBS volumes. Reconfigure the EC2 instances to use the encrypted volumes.

D.Create a customer managed key in AWS Key Management Service (AWS KMS). Configure AWS Migration Hub to use the key when the company migrates workloads.

Answer: B

Explanation:

The correct answer is **B: Use AWS Config. Configure the encrypted-volumes identifier. Apply the default AWS Key Management Service (AWS KMS) key.**

Here's why:

AWS Config allows you to assess, audit, and evaluate the configurations of your AWS resources. By using AWS Config, you can create rules that check whether your EBS volumes are encrypted. The encrypted-volumes managed rule specifically checks for EBS volume encryption. You can configure this rule to automatically remediate non-compliant resources, specifically unencrypted EBS volumes. By applying a default AWS KMS key to this rule, any newly created EBS volumes that are not explicitly encrypted using a KMS key during creation will be automatically encrypted using the specified key.

Option A (Configure the EC2 account attributes) while useful for default encryption, does not inherently prevent the creation of unencrypted volumes. Users could still explicitly disable encryption during volume creation. AWS Config provides the crucial enforcement aspect.

Option C (AWS Systems Manager) involves creating copies of EBS volumes. This is unnecessary overhead and doesn't directly prevent unencrypted volumes from being created initially. It's a reactive, rather than proactive, approach.

Option D (AWS Migration Hub) is primarily for tracking application migrations and does not directly control EBS volume encryption policies. It's not relevant to the problem of enforcing encryption on newly created EBS volumes.

Therefore, AWS Config with the encrypted-volumes rule and a default KMS key is the most suitable solution to ensure default encryption and prevent unencrypted volume creation. It proactively enforces the required configuration.

Relevant Documentation:

AWS Config managed rules for EBS: https://docs.aws.amazon.com/config/latest/developerguide/encrypted-volumes.html
AWS Config overview: https://aws.amazon.com/config/

Question: 888 **Exam Heist**

An ecommerce company wants to collect user clickstream data from the company's website for real-time analysis. The website experiences fluctuating traffic patterns throughout the day. The company needs a scalable solution that can adapt to varying levels of traffic.

Which solution will meet these requirements?

A.Use a data stream in Amazon Kinesis Data Streams in on-demand mode to capture the clickstream data. Use AWS Lambda to process the data in real time.

B.Use Amazon Kinesis Data Firehose to capture the clickstream data. Use AWS Glue to process the data in real time.

C.Use Amazon Kinesis Video Streams to capture the clickstream data. Use AWS Glue to process the data in real time.

D.Use Amazon Managed Service for Apache Flink (previously known as Amazon Kinesis Data Analytics) to capture the clickstream data. Use AWS Lambda to process the data in real time.

Answer: A

Explanation:

The correct answer is A: "Use a data stream in Amazon Kinesis Data Streams in on-demand mode to capture the clickstream data. Use AWS Lambda to process the data in real time."

Here's a detailed justification:

The scenario requires capturing and processing clickstream data in real-time with fluctuating traffic. Amazon Kinesis Data Streams is designed for high-throughput, real-time data streaming. On-demand mode within Kinesis Data Streams provides a scalable and cost-effective solution for varying traffic patterns. With on-demand capacity mode, Kinesis Data Streams automatically scales in response to your application's throughput. This eliminates the need for capacity planning and manual scaling.

AWS Lambda, being a serverless compute service, can process data in real-time, triggered by events from Kinesis Data Streams. Lambda functions scale automatically with the incoming traffic, ensuring that data is processed efficiently even during peak periods. This combination offers a fully managed, scalable, and cost-effective solution for real-time clickstream analysis.

Option B is incorrect because Amazon Kinesis Data Firehose is best suited for loading data into data lakes or data warehouses like Amazon S3, Amazon Redshift, or Splunk. While it handles scaling well, it's optimized for batching and delivering data rather than real-time, per-record processing. AWS Glue is an ETL service, ideal for data transformation and cataloging, but it's not directly integrated for real-time processing in the same way as Lambda with Kinesis Data Streams.

Option C is incorrect because Amazon Kinesis Video Streams is designed for streaming video and audio data, not general clickstream data. Although it can handle real-time data, it is not the appropriate tool for handling clickstream events. Using AWS Glue in combination is also unsuitable for real-time processing needs.

Option D is incorrect because Amazon Managed Service for Apache Flink (formerly Amazon Kinesis Data Analytics) is used for processing and analyzing streaming data using SQL or Java/Scala code. While it is capable of real-time processing, it introduces more complexity than necessary for simply transforming and forwarding clickstream data. It might be an over-engineered solution if basic transformations are sufficient. Kinesis Data Analytics captures data, and that is not what it primarily does.

Therefore, the combination of Kinesis Data Streams (on-demand) for scalable data ingestion and AWS Lambda for real-time processing best addresses the scenario's requirements.

Relevant Links:

Amazon Kinesis Data Streams: https://aws.amazon.com/kinesis/data-streams/
AWS Lambda: https://aws.amazon.com/lambda/
Kinesis Data Streams On-Demand: https://aws.amazon.com/blogs/aws/amazon-kinesis-data-streams-on-demand-auto-scale-throughput-at-no-extra-cost/

Question: 889 **Exam Heist**

A global company runs its workloads on AWS. The company's application uses Amazon S3 buckets across AWS Regions for sensitive data storage and analysis. The company stores millions of objects in multiple S3 buckets daily. The company wants to identify all S3 buckets that are not versioning-enabled.

Which solution will meet these requirements?

A.Set up an AWS CloudTrail event that has a rule to identify all S3 buckets that are not versioning-enabled across Regions.

B.Use Amazon S3 Storage Lens to identify all S3 buckets that are not versioning-enabled across Regions.

C.Enable IAM Access Analyzer for S3 to identify all S3 buckets that are not versioning-enabled across Regions.

D.Create an S3 Multi-Region Access Point to identify all S3 buckets that are not versioning-enabled across Regions.

Answer: B

Explanation:

The correct answer is B: Use Amazon S3 Storage Lens to identify all S3 buckets that are not versioning-enabled across Regions.

Here's why:

Amazon S3 Storage Lens is designed to provide organization-wide visibility into object storage, identify cost optimization opportunities, and apply data protection best practices. A key feature of S3 Storage Lens is its ability to aggregate metrics across multiple accounts and regions. It provides a central dashboard to view S3 storage usage, activity trends, and identify anomalies. Among the many metrics it tracks, S3 Storage Lens provides the capability to report on buckets that do not have versioning enabled. This makes it the ideal tool for the company's requirement to identify such buckets across all regions.

Option A is incorrect because AWS CloudTrail primarily captures API calls made to AWS services. While CloudTrail can record calls related to S3 versioning, it doesn't natively provide a consolidated view of all S3 buckets and their versioning status across regions. It would require significant post-processing and analysis of CloudTrail logs to achieve the desired outcome, making it inefficient.

Option C is incorrect because IAM Access Analyzer for S3 focuses on identifying buckets with access control list (ACL) settings that allow public access. It's designed to help you discover unintentionally shared buckets and resources. While Access Analyzer provides insights into access permissions, it doesn't directly assess or report on the versioning status of S3 buckets.

Option D is incorrect because S3 Multi-Region Access Points are used to simplify access to data stored in S3 buckets across multiple AWS regions. They provide a single endpoint for applications to access data, regardless of the region where the data is stored. They do not have the built-in functionality to assess or report on the versioning status of S3 buckets.

In summary, S3 Storage Lens is specifically built to provide the required visibility and reporting functionality for the company's use case, making it the most suitable solution.

Reference links:

Amazon S3 Storage Lens
AWS CloudTrail
IAM Access Analyzer for S3
Amazon S3 Multi-Region Access Points

Question: 890 **Exam Heist**

A company needs to optimize its Amazon S3 storage costs for an application that generates many files that cannot be recreated. Each file is approximately 5 MB and is stored in Amazon S3 Standard storage.

The company must store the files for 4 years before the files can be deleted. The files must be immediately accessible. The files are frequently accessed in the first 30 days of object creation, but they are rarely accessed after the first 30 days.

Which solution will meet these requirements MOST cost-effectively?

A.Create an S3 Lifecycle policy to move the files to S3 Glacier Instant Retrieval 30 days after object creation. Delete the files 4 years after object creation.

B.Create an S3 Lifecycle policy to move the files to S3 One Zone-Infrequent Access (S3 One Zone-IA) 30 days after object creation. Delete the files 4 years after object creation.

C.Create an S3 Lifecycle policy to move the files to S3 Standard-Infrequent Access (S3 Standard-IA) 30 days

after object creation. Delete the files 4 years after object creation.

D.Create an S3 Lifecycle policy to move the files to S3 Standard-Infrequent Access (S3 Standard-IA) 30 days after object creation. Move the files to S3 Glacier Flexible Retrieval 4 years after object creation.

Answer: A

Explanation:

The question asks for the most cost-effective solution for storing infrequently accessed files in S3 while ensuring immediate accessibility and long-term retention.

Option A proposes using S3 Glacier Instant Retrieval after 30 days. This class is designed for infrequently accessed data that requires immediate retrieval, making it suitable given the requirement for immediate accessibility. The files are deleted after 4 years, fulfilling the retention requirement.

Option B suggests S3 One Zone-IA. While cheaper than S3 Standard, it only stores data in a single availability zone, posing a risk if that zone becomes unavailable. The question states the data cannot be recreated, therefore availability must be prioritised, making this a risky option.

Option C recommends S3 Standard-IA. This is cheaper than S3 Standard but more expensive than Glacier Instant Retrieval for infrequently accessed data requiring immediate access.

Option D suggests using S3 Standard-IA for a portion of the time and then moving to S3 Glacier Flexible Retrieval (formerly Glacier) after 4 years. Glacier Flexible Retrieval is unsuitable as the files will need to be deleted, not accessed.

Considering the infrequent access, requirement for immediate retrieval, and long-term retention, S3 Glacier Instant Retrieval provides the best balance of cost and accessibility. Using S3 Glacier Instant Retrieval will reduce storage costs compared to S3 Standard-IA or S3 Standard whilst retaining immediate accessibility.

Therefore, option A is the most cost-effective and appropriate choice.

Further research can be done here:https://aws.amazon.com/s3/storage-classes/https://aws.amazon.com/glacier/

Question: 891 **Exam Heist**

A company runs its critical storage application in the AWS Cloud. The application uses Amazon S3 in two AWS Regions. The company wants the application to send remote user data to the nearest S3 bucket with no public network congestion. The company also wants the application to fail over with the least amount of management of Amazon S3.

Which solution will meet these requirements?

A.Implement an active-active design between the two Regions. Configure the application to use the regional S3 endpoints closest to the user.

B.Use an active-passive configuration with S3 Multi-Region Access Points. Create a global endpoint for each of the Regions.

C.Send user data to the regional S3 endpoints closest to the user. Configure an S3 cross-account replication rule to keep the S3 buckets synchronized.

D.Set up Amazon S3 to use Multi-Region Access Points in an active-active configuration with a single global endpoint. Configure S3 Cross-Region Replication.

Answer: D

Explanation:

The correct solution is D: "Set up Amazon S3 to use Multi-Region Access Points in an active-active configuration with a single global endpoint. Configure S3 Cross-Region Replication." This solution directly addresses all the requirements of the question.

Here's a detailed justification:

1. **Nearest S3 Bucket Selection and Reduced Network Congestion:** S3 Multi-Region Access Points (MRAP) provide a single global endpoint that intelligently routes requests to the geographically closest S3 bucket within the MRAP configuration. This ensures user data is sent to the nearest bucket, minimizing latency and avoiding public network congestion.

   https://aws.amazon.com/s3/features/multi-region-access-points/

2. **Active-Active Configuration:** Setting up MRAP in an active-active configuration allows both S3 buckets in different regions to handle requests simultaneously. This maximizes availability and performance by distributing the load across multiple regions. Option D specifies an active-active configuration, fulfilling this requirement.

3. **Failover with Minimal Management:** S3 MRAP handles failover automatically. If one region becomes unavailable, MRAP automatically routes requests to the healthy region. This minimizes the need for manual intervention during failover events, reducing operational overhead.

4. **Data Synchronization:** S3 Cross-Region Replication (CRR) keeps the S3 buckets in different regions synchronized. This ensures data consistency in the event of a failover, guaranteeing that the application can continue operating without data loss.

5. **Single Global Endpoint:** MRAP provides a single global endpoint for accessing data across all configured regions. This simplifies the application's configuration and eliminates the need to manage multiple regional endpoints.Option A is incorrect as simply configuring the application to use regional S3 endpoints doesn't handle failover. Option B employs active-passive, which doesn't take advantage of distributing user load over the regions. Option C would require more operational management to maintain consistent data.

Question: 892 **Exam Heist**

A company is migrating a data center from its on-premises location to AWS. The company has several legacy applications that are hosted on individual virtual servers. Changes to the application designs cannot be made.

Each individual virtual server currently runs as its own EC2 instance. A solutions architect needs to ensure that the applications are reliable and fault tolerant after migration to AWS. The applications will run on Amazon EC2 instances.

Which solution will meet these requirements?

A.Create an Auto Scaling group that has a minimum of one and a maximum of one. Create an Amazon Machine Image (AMI) of each application instance. Use the AMI to create EC2 instances in the Auto Scaling group Configure an Application Load Balancer in front of the Auto Scaling group.

B.Use AWS Backup to create an hourly backup of the EC2 instance that hosts each application. Store the backup in Amazon S3 in a separate Availability Zone. Configure a disaster recovery process to restore the EC2 instance for each application from its most recent backup.

C.Create an Amazon Machine Image (AMI) of each application instance. Launch two new EC2 instances from the AMI. Place each EC2 instance in a separate Availability Zone. Configure a Network Load Balancer that has the EC2 instances as targets.

D.Use AWS Mitigation Hub Refactor Spaces to migrate each application off the EC2 instance. Break down functionality from each application into individual components. Host each application on Amazon Elastic Container Service (Amazon ECS) with an AWS Fargate launch type.

Answer: C

Explanation:

The correct solution is C, which creates an AMI of each application instance and launches two new EC2 instances from the AMI in separate Availability Zones, fronted by a Network Load Balancer (NLB). This approach addresses the requirements of reliability and fault tolerance without requiring application design changes.

Option C achieves fault tolerance by distributing the application instances across multiple Availability Zones. If one Availability Zone experiences an outage, the other instance will continue to serve traffic. The NLB ensures that traffic is routed only to healthy instances. AMIs ensure a consistent and reproducible instance configuration for easy deployment. The NLB is suitable because it can handle TCP traffic and maintains source IP addresses, critical for some legacy applications.

Option A is not ideal because an Auto Scaling group with a minimum and maximum of one instance does not provide fault tolerance. While it can recreate the instance if it fails, there will still be downtime during the recovery. Furthermore, an Application Load Balancer (ALB) might not be suitable for all legacy applications, as ALBs work best with HTTP/HTTPS traffic.

Option B relies on backups, which introduce recovery time objective (RTO) and recovery point objective (RPO) issues. Restoring from backup is not a seamless failover. The backup process does not provide immediate fault tolerance.

Option D involves refactoring applications for containerization. The question specifically states that application design changes cannot be made, so this is invalid. While using ECS and Fargate offers benefits, it's outside the constraint of no application code changes.

In summary, Option C provides the best balance of fault tolerance, minimal changes (only infrastructure), and utilizes standard AWS services.

Relevant links:

Amazon Machine Images (AMI)
Network Load Balancer (NLB)
Availability Zones

Question: 893 **Exam Heist**

A company wants to isolate its workloads by creating an AWS account for each workload. The company needs a solution that centrally manages networking components for the workloads. The solution also must create accounts with automatic security controls (guardrails).

Which solution will meet these requirements with the LEAST operational overhead?

A.Use AWS Control Tower to deploy accounts. Create a networking account that has a VPC with private subnets and public subnets. Use AWS Resource Access Manager (AWS RAM) to share the subnets with the workload accounts.

B.Use AWS Organizations to deploy accounts. Create a networking account that has a VPC with private subnets and public subnets. Use AWS Resource Access Manager (AWS RAM) to share the subnets with the workload accounts.

C.Use AWS Control Tower to deploy accounts. Deploy a VPC in each workload account. Configure each VPC to route through an inspection VPC by using a transit gateway attachment.

D.Use AWS Organizations to deploy accounts. Deploy a VPC in each workload account. Configure each VPC to route through an inspection VPC by using a transit gateway attachment.

Answer: B

Explanation:

The correct answer is **B**. Here's why:

Rationale:

The question focuses on centralizing network management and automating security guardrails while minimizing operational overhead for isolating workloads across AWS accounts.

AWS Organizations: AWS Organizations is designed for centralized management of multiple AWS accounts. It allows you to create and manage accounts programmatically and apply policies across them. https://aws.amazon.com/organizations/
Networking Account: Creating a dedicated networking account centralizes network resources like VPCs, subnets, route tables, and other network services.
AWS Resource Access Manager (RAM): AWS RAM enables you to share AWS resources, such as subnets, with other accounts within your AWS organization. This avoids replicating network infrastructure in each workload account, reducing operational overhead. https://aws.amazon.com/ram/

Option B leverages these AWS services effectively: AWS Organizations for account creation and centralized control, a dedicated networking account for network resource management, and AWS RAM for sharing these network resources with workload accounts. It's a less operationally intensive approach compared to deploying VPCs in each workload account as this centralizes the effort.

Why other options are less suitable:

Option A (AWS Control Tower + AWS RAM): While Control Tower excels at setting up and governing multi-account environments with pre-defined guardrails, using it solely with RAM for subnet sharing doesn't fully exploit its capabilities. Control Tower often comes with more opinionated and prescriptive network configurations.
Options C and D (Transit Gateway): Deploying a VPC in each workload account and routing traffic through a transit gateway introduces unnecessary complexity and operational overhead. Managing individual VPCs, route tables, and transit gateway attachments for each workload account becomes cumbersome. Although Transit Gateway supports central inspection, it necessitates more configuration per account.

Question: 894 **Exam Heist**

A company hosts a website on Amazon EC2 instances behind an Application Load Balancer (ALB). The website serves static content. Website traffic is increasing. The company wants to minimize the website hosting costs.

Which solution will meet these requirements?

A. Move the website to an Amazon S3 bucket. Configure an Amazon CloudFront distribution for the S3 bucket.
B. Move the website to an Amazon S3 bucket. Configure an Amazon ElastiCache cluster for the S3 bucket.
C. Move the website to AWS Amplify. Configure an ALB to resolve to the Amplify website.
D. Move the website to AWS Amplify. Configure EC2 instances to cache the website.

Answer: A

Explanation:

The best solution for minimizing website hosting costs for static content served from EC2 instances behind an ALB, given increasing traffic, is to move the website to Amazon S3 and configure an Amazon CloudFront distribution.

Here's why:

Amazon S3 for Static Content: S3 is a cost-effective, highly scalable, and durable object storage service ideal for storing static website content like HTML, CSS, JavaScript, images, and videos. S3's pay-as-you-go pricing model makes it much more affordable than running EC2 instances for static content delivery.
Amazon CloudFront for Content Delivery: CloudFront is a content delivery network (CDN) that caches website content at edge locations globally. By caching content closer to users, CloudFront reduces latency, improves website performance, and, most importantly, significantly offloads traffic from the origin (in this case, S3).
Cost Optimization: Offloading static content to S3 and serving it through CloudFront dramatically reduces the load on EC2 instances, potentially allowing you to downsize or even eliminate some instances, leading to significant cost savings. You avoid paying for EC2 compute time, bandwidth, and storage for serving static assets. CloudFront also provides features like compression and caching configuration to further optimize delivery and reduce data transfer costs.
Scalability and Reliability: S3 and CloudFront are designed for high scalability and availability. They automatically scale to handle traffic spikes without requiring manual intervention.

Let's look at why the other options are not ideal:

Option B: Amazon ElastiCache with S3: ElastiCache is an in-memory caching service, but it's designed for dynamic content or frequently accessed data that benefits from low-latency access. It is not typically used in conjunction with S3 for serving static content as the primary purpose. CloudFront already provides caching capabilities suited for that purpose. ElastiCache also incurs its own costs.
Option C: AWS Amplify with ALB: AWS Amplify is a good choice for deploying and hosting full-stack web applications, including static content, but introducing an ALB in front of it is unnecessary and adds complexity and cost. Amplify itself is a hosting platform and can serve the content directly without an ALB.
Option D: AWS Amplify with EC2 Instances: This option defeats the purpose of moving away from EC2 instances to minimize costs. EC2 instances still handle the caching, incurring the same cost concerns. AWS Amplify is a hosting platform and does not need to configure EC2 instances.

Authoritative Links:

Amazon S3: https://aws.amazon.com/s3/
Amazon CloudFront: https://aws.amazon.com/cloudfront/
AWS Amplify: https://aws.amazon.com/amplify/

Question: 895 **Exam Heist**

A company is implementing a shared storage solution for a media application that the company hosts on AWS. The company needs the ability to use SMB clients to access stored data.

Which solution will meet these requirements with the LEAST administrative overhead?

A.Create an AWS Storage Gateway Volume Gateway. Create a file share that uses the required client protocol. Connect the application server to the file share.

B.Create an AWS Storage Gateway Tape Gateway. Configure tapes to use Amazon S3. Connect the application server to the Tape Gateway.

C.Create an Amazon EC2 Windows instance. Install and configure a Windows file share role on the instance. Connect the application server to the file share.

D.Create an Amazon FSx for Windows File Server file system. Connect the application server to the file system.

Answer: D

Explanation:

The correct answer is **D. Create an Amazon FSx for Windows File Server file system. Connect the application server to the file system.**

Here's a detailed justification:

Amazon FSx for Windows File Server is a fully managed Windows file system built on Windows Server. It provides native Windows file system capabilities and features like SMB (Server Message Block) protocol support. This directly addresses the requirement of using SMB clients to access stored data. Furthermore, being a fully managed service, FSx for Windows File Server offloads administrative overhead related to hardware provisioning, patching, backups, and other maintenance tasks.

Option A, using AWS Storage Gateway Volume Gateway, requires managing volumes on-premises or in EC2, adding complexity to the solution. While it can present storage to applications via iSCSI, it doesn't inherently fulfill the SMB requirement without additional configuration.

Option B, using AWS Storage Gateway Tape Gateway, is designed for archival purposes and not suitable for active media application storage as it deals with virtual tapes.

Option C, creating an EC2 Windows instance and configuring a file share, involves significant administrative overhead for managing the EC2 instance, operating system, file server role, backups, and security. This contradicts the requirement of minimizing administrative overhead.

Therefore, Amazon FSx for Windows File Server offers the least administrative overhead because it is a fully managed, purpose-built service for Windows file shares that natively supports SMB, simplifying deployment and ongoing management for the media application's shared storage needs.

Relevant Links:

Amazon FSx for Windows File Server: https://aws.amazon.com/fsx/windows/
SMB Protocol: https://en.wikipedia.org/wiki/Server_Message_Block

Question: 896 **Exam Heist**

A company is designing its production application's disaster recovery (DR) strategy. The application is backed by a MySQL database on an Amazon Aurora cluster in the us-east-1 Region. The company has chosen the us-west-1 Region as its DR Region.

The company's target recovery point objective (RPO) is 5 minutes and the target recovery time objective (RTO) is 20 minutes. The company wants to minimize configuration changes.

Which solution will meet these requirements with the MOST operational efficiency?

A.Create an Aurora read replica in us-west-1 similar in size to the production application's Aurora MySQL cluster writer instance.

B.Convert the Aurora cluster to an Aurora global database. Configure managed failover.

C.Create a new Aurora cluster in us-west-1 that has Cross-Region Replication.

D.Create a new Aurora cluster in us-west-1. Use AWS Database Migration Service (AWS DMS) to sync both clusters.

Answer: B

Explanation:

The correct answer is **B. Convert the Aurora cluster to an Aurora global database. Configure managed failover.**

Here's why this solution is the most operationally efficient and meets the RPO/RTO requirements:

Aurora Global Database is specifically designed for disaster recovery scenarios with low RPO and RTO across different AWS Regions. It uses storage-based replication to replicate data with minimal lag, typically under a second, which easily satisfies the 5-minute RPO. This is accomplished by replicating data at the storage layer using dedicated infrastructure.

The managed failover feature of Aurora Global Database automates the failover process in case of a disaster. This reduces the recovery time significantly, aligning with the 20-minute RTO goal. It handles promoting a read replica to a writer instance in the DR region.

Option A, creating an Aurora read replica, is a viable DR strategy. However, it involves manual intervention for failover, which increases the RTO. It will also likely take some time to promote the read replica to a writer instance.

Option C, using Cross-Region Replication, requires creating a new Aurora cluster, which might involve more configuration and setup than simply converting to Aurora Global Database. Also, this is deprecated by Amazon. The preferred mechanism for DR is Aurora Global Database.

Option D, using AWS DMS, introduces complexity and potential latency, making it less suitable for the stringent RPO of 5 minutes. DMS is designed more for database migrations or scenarios where schema transformation is required, not for continuous, low-latency replication for DR. Also DMS does not handle the failover, the user is responsible for it.

Therefore, Aurora Global Database with managed failover provides the most operationally efficient, automated, and optimized solution for meeting the RPO/RTO requirements in a DR scenario. It minimizes configuration changes because it leverages the existing Aurora cluster with just the upgrade to global database, and it handles the complex failover process automatically.

Supporting Documentation:

Aurora Global Database: https://aws.amazon.com/rds/aurora/features/global-database/
Achieving Fast Failover with Amazon Aurora Global Database:
https://aws.amazon.com/blogs/database/achieving-fast-failover-with-amazon-aurora-global-database/

Question: 897 **Exam Heist**

A company runs a critical data analysis job each week before the first day of the work week. The job requires at least 1 hour to complete the analysis. The job is stateful and cannot tolerate interruptions. The company needs a solution to run the job on AWS.

Which solution will meet these requirements?

A.Create a container for the job. Schedule the job to run as an AWS Fargate task on an Amazon Elastic Container Service (Amazon ECS) cluster by using Amazon EventBridge Scheduler.

B.Configure the job to run in an AWS Lambda function. Create a scheduled rule in Amazon EventBridge to invoke the Lambda function.

C.Configure an Auto Scaling group of Amazon EC2 Spot Instances that run Amazon Linux. Configure a crontab entry on the instances to run the analysis.

D.Configure an AWS DataSync task to run the job. Configure a cron expression to run the task on a schedule.

Answer: A

Explanation:

The correct answer is A. Here's why:

Requirement of at least 1-hour execution time: Lambda (Option B) has a maximum execution time limit of 15

minutes. This makes it unsuitable for a job that requires at least an hour.https://docs.aws.amazon.com/lambda/latest/dg/configuration-function-common.html

Stateful and Interruptions Intolerant: The job's stateful nature and intolerance to interruptions require a stable and persistent environment during execution.

Fargate and ECS: Option A, using Fargate and ECS, provides a suitable environment. Fargate allows you to run containers without managing the underlying EC2 instances, simplifying operations. ECS manages the container orchestration.

EventBridge Scheduler: EventBridge Scheduler enables precise scheduling, ensuring the job runs weekly before the first day of the work week. This is more robust than relying on crontab (Option C) on EC2 instances, which can be susceptible to time drift or instance issues.https://aws.amazon.com/eventbridge/scheduler/

Spot Instances: Option C uses Spot Instances, which are cost-effective but can be interrupted if the Spot price exceeds your bid. This violates the "interruptions intolerant" requirement. Also, relying on crontab isn't ideal for managed scheduling.

DataSync: Option D, DataSync, is designed for data transfer and synchronization, not for general-purpose data analysis jobs. It doesn't provide a compute environment to execute the analysis logic.https://aws.amazon.com/datasync/

Containerization: Containerizing the job (Option A) makes it portable and reproducible, ensuring consistent execution across environments. ECS and Fargate are designed for running containerized applications.

In summary, ECS with Fargate gives a stable environment, and EventBridge Scheduler offers reliable scheduled execution to comply with the requirements that the job is stateful and cannot tolerate interruptions. Lambda cannot run for an hour. Spot Instances might be interrupted. DataSync serves a different purpose.

Question: 898 **Exam Heist**

A company runs workloads in the AWS Cloud. The company wants to centrally collect security data to assess security across the entire company and to improve workload protection.

Which solution will meet these requirements with the LEAST development effort?

A.Configure a data lake in AWS Lake Formation. Use AWS Glue crawlers to ingest the security data into the data lake.

B.Configure an AWS Lambda function to collect the security data in .csv format. Upload the data to an Amazon S3 bucket.

C.Configure a data lake in Amazon Security Lake to collect the security data. Upload the data to an Amazon S3 bucket.

D.Configure an AWS Database Migration Service (AWS DMS) replication instance to load the security data into an Amazon RDS cluster.

Answer: C

Explanation:

The correct answer is C, configuring a data lake in Amazon Security Lake to collect the security data and uploading it to an Amazon S3 bucket. Here's why:

Amazon Security Lake is specifically designed to aggregate, transform, and manage security data from various AWS services and third-party sources into a centralized data lake. This minimizes development effort because the service is built for this exact purpose. It automatically collects security-relevant logs and events,

normalizes them into the Open Cybersecurity Schema Framework (OCSF) format, and stores them in a data lake you own.

Option A, using AWS Lake Formation and Glue, would require significant configuration and development effort to define crawlers, schemas, and data transformation logic to ingest and normalize diverse security data sources. While Lake Formation provides governance, Security Lake provides out-of-the-box security data ingestion and normalization.

Option B, a Lambda function collecting CSV data and uploading to S3, lacks the built-in capabilities for large-scale data management, normalization (into OCSF), and centralized security data analysis. It would require substantial coding and maintenance.

Option D, using AWS DMS to load data into an RDS cluster, is not suited for collecting and analyzing the diverse, semi-structured security data from various AWS sources. RDS is designed for structured relational data, not the varied log and event data. Moreover, DMS primarily focuses on database migrations, not security data aggregation.

Security Lake addresses the prompt's requirement for central collection and assessment of security data with minimal development effort by providing a purpose-built service that automates the collection, normalization, and storage processes. The OCSF format ensures interoperability and standardized analysis across different security tools. Uploading to S3, configured as the data lake, is the standard method of inputting the data to Security Lake.

References:

Amazon Security Lake
Open Cybersecurity Schema Framework (OCSF)

Question: 899 **Exam Heist**

A company is migrating five on-premises applications to VPCs in the AWS Cloud. Each application is currently deployed in isolated virtual networks on premises and should be deployed similarly in the AWS Cloud. The applications need to reach a shared services VPC. All the applications must be able to communicate with each other.

If the migration is successful, the company will repeat the migration process for more than 100 applications.

Which solution will meet these requirements with the LEAST administrative overhead?

A.Deploy software VPN tunnels between the application VPCs and the shared services VPC. Add routes between the application VPCs in their subnets to the shared services VPC.

B.Deploy VPC peering connections between the application VPCs and the shared services VPC. Add routes between the application VPCs in their subnets to the shared services VPC through the peering connection.

C.Deploy an AWS Direct Connect connection between the application VPCs and the shared services VPAdd routes from the application VPCs in their subnets to the shared services VPC and the applications VPCs. Add routes from the shared services VPC subnets to the applications VPCs.

D.Deploy a transit gateway with associations between the transit gateway and the application VPCs and the shared services VPC. Add routes between the application VPCs in their subnets and the application VPCs to the shared services VPC through the transit gateway.

Answer: D

Explanation:

The best solution for connecting multiple VPCs with minimal administrative overhead, especially when considering future scalability for over 100 applications, is deploying an AWS Transit Gateway.

Option D is the most efficient because a Transit Gateway acts as a central hub, simplifying network management. Each application VPC and the shared services VPC connect to the Transit Gateway. Routes within each VPC subnet point to the Transit Gateway. The Transit Gateway then handles routing traffic between the connected VPCs based on route tables associated with each VPC attachment.

Option A is less scalable as it involves manually configuring and maintaining multiple VPN tunnels. VPNs add complexity and can become a management burden.

Option B, VPC peering, has a limit on the number of peerings a VPC can have and can quickly become unmanageable with a large number of VPCs. It also requires establishing a peering connection between every pair of VPCs that need to communicate, which is O(n^2) complexity. Additionally, overlapping CIDR blocks are not supported with VPC peering, which could become a constraint.

Option C, Direct Connect, is meant for creating a dedicated network connection from on-premises to AWS and is not relevant for connecting VPCs within AWS. It is also far more expensive and complex than a software-defined networking solution like Transit Gateway.

Transit Gateway's centralized routing policy and simplified management make it ideal for this scenario. As the company migrates more applications, new VPCs can simply be attached to the Transit Gateway, and routes can be updated centrally.

For further research, refer to the AWS documentation on Transit Gateway: https://aws.amazon.com/transit-gateway/ and https://docs.aws.amazon.com/transit-gateway/latest/tgw/what-is-transit-gateway.html

Question: 900 **Exam Heist**

A company wants to use Amazon Elastic Container Service (Amazon ECS) to run its on-premises application in a hybrid environment. The application currently runs on containers on premises.

The company needs a single container solution that can scale in an on-premises, hybrid, or cloud environment. The company must run new application containers in the AWS Cloud and must use a load balancer for HTTP traffic.

Which combination of actions will meet these requirements? (Choose two.)

A.Set up an ECS cluster that uses the AWS Fargate launch type for the cloud application containers. Use an Amazon ECS Anywhere external launch type for the on-premises application containers.

B.Set up an Application Load Balancer for cloud ECS services.

C.Set up a Network Load Balancer for cloud ECS services.

D.Set up an ECS cluster that uses the AWS Fargate launch type. Use Fargate for the cloud application containers and the on-premises application containers.

E.Set up an ECS cluster that uses the Amazon EC2 launch type for the cloud application containers. Use Amazon ECS Anywhere with an AWS Fargate launch type for the on-premises application containers.

Answer: AB

Explanation:

The correct answer is A and B. Here's why:

A. Set up an ECS cluster that uses the AWS Fargate launch type for the cloud application containers. Use an Amazon ECS Anywhere external launch type for the on-premises application containers.

AWS Fargate for Cloud Containers: Fargate is a serverless compute engine for containers, ideal for running ECS tasks without managing the underlying EC2 instances. This aligns perfectly with the requirement to run new application containers in the AWS Cloud in a managed and scalable fashion. It simplifies operational overhead.

ECS Anywhere for On-premises Containers: ECS Anywhere allows you to register your on-premises servers or virtual machines as external instances within your ECS cluster. This fulfills the requirement to run the existing on-premises application containers within a hybrid environment.

B. Set up an Application Load Balancer for cloud ECS services.

ALB for HTTP Traffic: The problem specifically mentions the need for a load balancer for HTTP traffic. Application Load Balancers (ALBs) are specifically designed for HTTP and HTTPS traffic. They provide advanced routing capabilities based on the content of the request. This is necessary for the company to manage and distribute HTTP traffic to the containers running on ECS in the cloud.

Why other options are incorrect:

C. Set up a Network Load Balancer for cloud ECS services: NLBs are suited for TCP, UDP, and TLS traffic and not for HTTP-aware routing.
D. Set up an ECS cluster that uses the AWS Fargate launch type. Use Fargate for the cloud application containers and the on-premises application containers: Fargate cannot be used directly for on-premises containers without ECS Anywhere's external instance capability.
E. Set up an ECS cluster that uses the Amazon EC2 launch type for the cloud application containers. Use Amazon ECS Anywhere with an AWS Fargate launch type for the on-premises application containers: While EC2 launch type for cloud is a valid alternative, it does not represent the best option for the problem statement that prefers easier managed scalability of cloud. And AWS Fargate launch type with ECS Anywhere is incorrect. ECS Anywhere uses external instances, not Fargate.

Supporting Links:

AWS Fargate: https://aws.amazon.com/fargate/
Amazon ECS Anywhere: https://aws.amazon.com/ecs/anywhere/
Application Load Balancer (ALB): https://aws.amazon.com/elasticloadbalancing/application-load-balancer/

Question: 901 **Exam Heist**

A company is migrating its workloads to AWS. The company has sensitive and critical data in on-premises relational databases that run on SQL Server instances.

The company wants to use the AWS Cloud to increase security and reduce operational overhead for the databases.

Which solution will meet these requirements?

A.Migrate the databases to Amazon EC2 instances. Use an AWS Key Management Service (AWS KMS) AWS managed key for encryption.

B.Migrate the databases to a Multi-AZ Amazon RDS for SQL Server DB instance. Use an AWS Key Management Service (AWS KMS) AWS managed key for encryption.

C.Migrate the data to an Amazon S3 bucket. Use Amazon Macie to ensure data security.

D.Migrate the databases to an Amazon DynamoDB table. Use Amazon CloudWatch Logs to ensure data security.

Answer: B

Explanation:

The best solution for migrating on-premises SQL Server databases to AWS with increased security and reduced operational overhead is to use Amazon RDS for SQL Server with Multi-AZ and AWS KMS encryption. Here's why:

Reduced Operational Overhead: Amazon RDS is a managed database service, which handles tasks like

patching, backups, and infrastructure management, significantly reducing the operational burden compared to managing databases on EC2 instances.
Security: Using AWS KMS with RDS allows for encryption at rest, protecting sensitive data from unauthorized access. RDS manages the encryption process, simplifying key management.
High Availability and Disaster Recovery: The Multi-AZ deployment option in RDS provides high availability by synchronously replicating data to a standby instance in a different Availability Zone. In case of a failure, RDS automatically fails over to the standby instance, minimizing downtime.
Suitability for Relational Databases: Amazon RDS for SQL Server is designed to support relational database workloads, ensuring compatibility and performance.

Option A, migrating to EC2, increases operational overhead because the company would have to manage the operating system, database software, backups, patching, and scaling. EC2 doesn't offer the same level of managed services and automation as RDS. Option C is not suitable because S3 is an object storage service, not a relational database, and Macie is a data discovery and classification service, not a database solution. Option D, migrating to DynamoDB, would require significant application rework, as it is a NoSQL database and not a relational database. Relational data structures don't translate to NoSQL structures automatically. CloudWatch Logs, while useful for logging, doesn't provide data security in the context of database storage.

Supporting Links:

Amazon RDS: https://aws.amazon.com/rds/
Amazon RDS for SQL Server: https://aws.amazon.com/rds/sqlserver/
AWS KMS: https://aws.amazon.com/kms/
Amazon RDS Multi-AZ: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html

Question: 902 **Exam Heist**

A company wants to migrate an application to AWS. The company wants to increase the application's current availability. The company wants to use AWS WAF in the application's architecture.

Which solution will meet these requirements?

A.Create an Auto Scaling group that contains multiple Amazon EC2 instances that host the application across two Availability Zones. Configure an Application Load Balancer (ALB) and set the Auto Scaling group as the target. Connect a WAF to the ALB.

B.Create a cluster placement group that contains multiple Amazon EC2 instances that hosts the application. Configure an Application Load Balancer and set the EC2 instances as the targets. Connect a WAF to the placement group.

C.Create two Amazon EC2 instances that host the application across two Availability Zones. Configure the EC2 instances as the targets of an Application Load Balancer (ALB). Connect a WAF to the ALB.

D.Create an Auto Scaling group that contains multiple Amazon EC2 instances that host the application across two Availability Zones. Configure an Application Load Balancer (ALB) and set the Auto Scaling group as the target. Connect a WAF to the Auto Scaling group.

Answer: A

Explanation:

The correct answer is A. Here's why:

Availability: Auto Scaling groups across multiple Availability Zones (AZs) ensure high availability. If one AZ fails, the application continues to run in the other AZ. This is a fundamental principle of fault tolerance in cloud architecture.
Load Balancing: An Application Load Balancer (ALB) distributes incoming traffic across multiple EC2 instances, improving performance and availability. The ALB acts as a single point of contact for the

application, routing requests to healthy instances.
WAF Protection: AWS WAF (Web Application Firewall) protects web applications from common web exploits and bots. It can be connected to an ALB to filter malicious traffic before it reaches the EC2 instances.
Auto Scaling Group as Target: Setting the Auto Scaling group as the target for the ALB dynamically registers and deregisters instances as they are launched or terminated by the Auto Scaling group, automating the load balancing configuration.

Let's analyze why the other options are not ideal:

B: Placement groups are primarily for low-latency network performance between instances, not specifically for high availability or integrating with WAF. WAF cannot be directly connected to a placement group.
C: Using only two EC2 instances provides less resilience compared to an Auto Scaling group. If one instance fails, the application's capacity is halved. Auto Scaling provides automatic scaling and self-healing capabilities.
D: You cannot directly attach a WAF to an Auto Scaling group. The WAF must be attached to a supported resource like an Application Load Balancer or API Gateway.

In summary, option A provides the most robust solution for achieving high availability, load balancing, and WAF protection by utilizing Auto Scaling groups across multiple Availability Zones, an Application Load Balancer, and attaching the WAF to the ALB.

Here are some authoritative links for further research:

AWS Auto Scaling: https://aws.amazon.com/autoscaling/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
AWS WAF: https://aws.amazon.com/waf/
High Availability Architecture: https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/design-for-high-availability.html

Question: 903 **Exam Heist**

A company manages a data lake in an Amazon S3 bucket that numerous applications access. The S3 bucket contains a unique prefix for each application. The company wants to restrict each application to its specific prefix and to have granular control of the objects under each prefix.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create dedicated S3 access points and access point policies for each application.

B.Create an S3 Batch Operations job to set the ACL permissions for each object in the S3 bucket.

C.Replicate the objects in the S3 bucket to new S3 buckets for each application. Create replication rules by prefix.

D.Replicate the objects in the S3 bucket to new S3 buckets for each application. Create dedicated S3 access points for each application.

Answer: B

Explanation:

The provided answer "B" (Create an S3 Batch Operations job to set the ACL permissions for each object in the S3 bucket) is incorrect and involves significant operational overhead.

Here's why the other option (A) is the better solution, and a detailed explanation:

Correct Answer: A. Create dedicated S3 access points and access point policies for each application.

Justification:

1. **S3 Access Points:** S3 Access Points are named network endpoints that are attached to a bucket and can be used to manage data access for shared datasets. They simplify the management of data access at scale for applications using shared S3 buckets. Each access point has a unique hostname and associated access policy. https://aws.amazon.com/s3/features/access-points/
2. **Granular Control:** Access Point policies are written using IAM policy language. You can use them to grant specific permissions to applications accessing the bucket through the access point. In this scenario, each application gets its own access point, and the access point policy restricts access to its designated prefix.
3. **Least Operational Overhead:** Creating access points and configuring policies is a straightforward process. It centralizes access management and avoids the need to modify permissions on individual objects, which is a much more complex and time-consuming task, especially as the data lake grows.
4. **Scalability:** Access points scale well as the number of applications increases. Adding a new application involves creating a new access point and defining its policy, which is a relatively simple operation.
5. **Security Best Practices:** Access Points promote the principle of least privilege by granting applications only the necessary permissions to access the specific data they need.

Why the other options are less suitable:

B. S3 Batch Operations and ACLs: While S3 Batch Operations can modify ACLs, it's a much more cumbersome and less scalable solution than using Access Points. It requires creating and running a Batch Operations job every time you need to update permissions, which introduces unnecessary operational overhead. ACLs are also a legacy access control mechanism and are less flexible and harder to manage than IAM policies.
C & D. Replicating to new buckets: Replicating data to multiple buckets for each application introduces significant operational overhead and storage costs. Data replication can also lead to data consistency issues and adds complexity to the overall architecture. Creating access points on top of replicated buckets (Option D) doesn't reduce the replication overhead.

In summary, S3 Access Points provide the most efficient and scalable solution for restricting applications to specific prefixes in an S3 bucket with granular access control. They minimize operational overhead by centralizing access management and leveraging IAM policies for fine-grained permissions.

Question: 904 **Exam Heist**

A company has an application that customers use to upload images to an Amazon S3 bucket. Each night, the company launches an Amazon EC2 Spot Fleet that processes all the images that the company received that day. The processing for each image takes 2 minutes and requires 512 MB of memory.

A solutions architect needs to change the application to process the images when the images are uploaded.

Which change will meet these requirements MOST cost-effectively?

A.Use S3 Event Notifications to write a message with image details to an Amazon Simple Queue Service (Amazon SQS) queue. Configure an AWS Lambda function to read the messages from the queue and to process the images.

B.Use S3 Event Notifications to write a message with image details to an Amazon Simple Queue Service (Amazon SQS) queue. Configure an EC2 Reserved Instance to read the messages from the queue and to process the images.

C.Use S3 Event Notifications to publish a message with image details to an Amazon Simple Notification Service (Amazon SNS) topic. Configure a container instance in Amazon Elastic Container Service (Amazon ECS) to subscribe to the topic and to process the images.

D.Use S3 Event Notifications to publish a message with image details to an Amazon Simple Notification Service (Amazon SNS) topic. Configure an AWS Elastic Beanstalk application to subscribe to the topic and to process the images.

Answer: A

Explanation:

The correct answer is **A**. Here's why:

A. Use S3 Event Notifications to write a message with image details to an Amazon Simple Queue Service (Amazon SQS) queue. Configure an AWS Lambda function to read the messages from the queue and to process the images.

Event-Driven Architecture: S3 Event Notifications trigger the image processing pipeline immediately upon upload, enabling real-time processing as required. This avoids the nightly batch processing approach.
AWS Lambda for Scalable Processing: Lambda functions offer a serverless compute environment perfectly suited for handling individual image processing tasks. They scale automatically based on the number of messages in the SQS queue, ensuring efficient resource utilization. You only pay for the compute time consumed while processing the image, which is cost-effective for event-driven workloads.
Amazon SQS for Decoupling: SQS acts as a buffer between S3 and Lambda, decoupling the image upload process from the processing logic. This prevents the application from being affected if the image processing fails. SQS ensures that each message is delivered at least once, offering reliability.
Cost Optimization: Lambda's pay-per-use model is highly cost-effective, especially when the processing time per image is short (2 minutes) and the memory requirement is relatively low (512 MB). You avoid the overhead of maintaining and paying for EC2 instances that might sit idle for periods of time.
Suitability: With 2 minutes of processing time, lambda is suitable to handle the task.

Why the other options are less suitable:

B. EC2 Reserved Instance: While Reserved Instances can be cost-effective for steady workloads, they are less efficient than Lambda for event-driven processing, especially with fluctuating workloads. EC2 instances need to be running even if no images are available, costing more.
C & D. Amazon SNS, ECS, and Elastic Beanstalk: While SNS can trigger events, it is better suited for fan-out scenarios (broadcasting messages to multiple subscribers). In this case, we require each image to be processed exactly once, which SQS guarantees. ECS and Elastic Beanstalk are more complex to configure and manage than Lambda for this specific use case, increasing operational overhead. ECS and Elastic Beanstalk can be cost effective where the processing lasts more than 15 minutes, in such a case lambda won't be the best option.

Supporting Links:

S3 Event Notifications
AWS Lambda
Amazon SQS

Question: 905 **Exam Heist**

A company wants to improve the availability and performance of its hybrid application. The application consists of a stateful TCP-based workload hosted on Amazon EC2 instances in different AWS Regions and a stateless UDP-based workload hosted on premises.

Which combination of actions should a solutions architect take to improve availability and performance? (Choose two.)

A.Create an accelerator using AWS Global Accelerator. Add the load balancers as endpoints.

B.Create an Amazon CloudFront distribution with an origin that uses Amazon Route 53 latency-based routing to route requests to the load balancers.

C.Configure two Application Load Balancers in each Region. The first will route to the EC2 endpoints, and the second will route to the on-premises endpoints.

D.Configure a Network Load Balancer in each Region to address the EC2 endpoints. Configure a Network Load Balancer in each Region that routes to the on-premises endpoints.

E.Configure a Network Load Balancer in each Region to address the EC2 endpoints. Configure an Application Load Balancer in each Region that routes to the on-premises endpoints.

Answer: AD

Explanation:

The correct answer is AD because it addresses both the stateful TCP and stateless UDP workload requirements while enhancing availability and performance across the hybrid environment.

A. Create an accelerator using AWS Global Accelerator. Add the load balancers as endpoints. AWS Global Accelerator improves application availability and performance by directing traffic to healthy endpoints based on endpoint health, geographic proximity, and configured weights. It leverages the AWS global network to route traffic to the nearest healthy endpoint, reducing latency and improving user experience, especially beneficial for a hybrid application spread across different regions. Since the EC2 instances are in different AWS Regions, Global Accelerator is perfect for TCP traffic. https://aws.amazon.com/global-accelerator/

D. Configure a Network Load Balancer in each Region to address the EC2 endpoints. Configure a Network Load Balancer in each Region that routes to the on-premises endpoints. Network Load Balancers (NLBs) are suitable for TCP-based workloads that require high performance and low latency. NLBs can handle millions of requests per second while maintaining ultra-low latencies. Using NLBs in each region provides redundancy and handles local traffic efficiently. Also, using NLBs for on-premises UDP traffic ensures high throughput and the ability to forward traffic directly to the on-premises resources without application-layer processing, which would be required by an ALB. This is optimal for the stateless UDP traffic. The combined regional NLBs for on-prem are used as endpoints with the Global Accelerator, providing performance and increased availability for both TCP and UDP workloads. https://aws.amazon.com/elasticloadbalancing/network-load-balancer/

Why other options are incorrect:

B: CloudFront is primarily for caching static content and improving website performance, not for routing traffic to dynamic application endpoints in a hybrid setup with varying latency requirements. It does not efficiently handle stateful TCP connections or stateless UDP connections.
C: Configuring Application Load Balancers (ALBs) for on-premises endpoints isn't the most efficient design for UDP traffic. ALBs work best with HTTP/HTTPS traffic and perform application-layer processing which introduces overhead for UDP. Using two ALBs per region also adds unnecessary complexity.
E: ALBs aren't ideal for on-prem UDP traffic because they are designed for HTTP(S) traffic and add unnecessary overhead with application layer processing. NLB is much more suitable for UDP traffic due to its low latency and direct forwarding capabilities. Using two different types of load balancers is less consistent than just using NLBs.

Question: 906 **Exam Heist**

A company runs a self-managed Microsoft SQL Server on Amazon EC2 instances and Amazon Elastic Block Store (Amazon EBS). Daily snapshots are taken of the EBS volumes.

Recently, all the company's EBS snapshots were accidentally deleted while running a snapshot cleaning script that deletes all expired EBS snapshots. A solutions architect needs to update the architecture to prevent data loss without retaining EBS snapshots indefinitely.

Which solution will meet these requirements with the LEAST development effort?

A.Change the IAM policy of the user to deny EBS snapshot deletion.
B.Copy the EBS snapshots to another AWS Region after completing the snapshots daily.
C.Create a 7-day EBS snapshot retention rule in Recycle Bin and apply the rule for all snapshots.
D.Copy EBS snapshots to Amazon S3 Standard-Infrequent Access (S3 Standard-IA).

Answer: C

Explanation:

The correct answer is **C. Create a 7-day EBS snapshot retention rule in Recycle Bin and apply the rule for all snapshots.**

Here's a detailed justification:

The scenario describes a company accidentally deleting all EBS snapshots due to a faulty script. The goal is to prevent data loss without indefinitely retaining snapshots and minimizing development effort.

Option C, using the Recycle Bin, directly addresses the problem. The Recycle Bin for EBS Snapshots allows you to recover accidentally deleted EBS snapshots within a retention period you configure (in this case, 7 days). This feature is designed precisely for scenarios where accidental deletion occurs. It acts as a safety net, giving a grace period to restore accidentally deleted snapshots. It avoids the need for complex coding, scripting, or significant architectural changes. It also avoids indefinite retention by allowing snapshots to be permanently deleted after the retention period (7 days). This option has minimal development effort, as it involves configuring a retention rule in the Recycle Bin and applying it to the relevant snapshots.

Option A, changing the IAM policy to deny EBS snapshot deletion, is too restrictive. While it would prevent accidental deletion, it also prevents legitimate deletions when they are truly required. This approach lacks flexibility and could hinder legitimate operations. IAM policy should be about least privileges, allowing only what is neccessary for functionality.

Option B, copying EBS snapshots to another AWS Region, provides data redundancy but does not directly address accidental deletion within the primary region. It adds unnecessary complexity and cost compared to the Recycle Bin. Moreover, it only saves snapshots in the secondary region, which could violate regulatory requirements.

Option D, copying EBS snapshots to Amazon S3 Standard-IA, is also a form of data backup but is not directly suited for recovering from accidental deletion. It requires more development effort to implement the copy process and manage the snapshots in S3. Furthermore, it can be costly to retrieve snapshots from S3 Standard-IA, especially when you need to quickly restore a volume from a snapshot, increasing RTO (recovery time objective).

Therefore, the Recycle Bin provides the simplest, most cost-effective, and most direct solution for preventing data loss due to accidental EBS snapshot deletion.The recycle bin feature in AWS is a built in, native feature for addressing accidental deletions, minimizing the need for complex custom solutions.

Relevant AWS Documentation:

Recycle Bin: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/snapshot-recycle-bin.html

Question: 907 **Exam Heist**

A company wants to use an AWS CloudFormation stack for its application in a test environment. The company stores the CloudFormation template in an Amazon S3 bucket that blocks public access. The company wants to grant CloudFormation access to the template in the S3 bucket based on specific user requests to create the test

environment. The solution must follow security best practices.

Which solution will meet these requirements?

A.Create a gateway VPC endpoint for Amazon S3. Configure the CloudFormation stack to use the S3 object URL.

B.Create an Amazon API Gateway REST API that has the S3 bucket as the target. Configure the CloudFormation stack to use the API Gateway URL.

C.Create a presigned URL for the template object. Configure the CloudFormation stack to use the presigned URL.

D.Allow public access to the template object in the S3 bucket. Block the public access after the test environment is created.

Answer: C

Explanation:

The requirement is to securely grant CloudFormation access to an S3 template without exposing the bucket publicly and only upon explicit user requests. Option C, creating a presigned URL, is the most secure and appropriate solution.

A presigned URL grants temporary access to a specific object in S3. This URL is generated with specific permissions and an expiration time. When a user requests to create the test environment, the presigned URL is generated and provided to the CloudFormation stack. This allows CloudFormation to access the template for the duration specified in the presigned URL, after which the URL becomes invalid. This adheres to the principle of least privilege and provides a time-bound, auditable access mechanism.

Option A, creating a gateway VPC endpoint for S3, would only provide access to S3 from within the VPC. While this adds a layer of network security, it doesn't address the requirement of granting access based on specific user requests and doesn't prevent potential misuse from within the VPC if the service role isn't correctly scoped.

Option B, creating an API Gateway REST API, would introduce unnecessary complexity and cost. It would add an additional layer between CloudFormation and S3 without providing significant security benefits over a presigned URL. The API would also need to be secured and managed.

Option D, allowing public access temporarily, is a significant security risk and violates the requirement to follow security best practices. It exposes the template object to anyone on the internet during the time it's publicly accessible, potentially leaking sensitive information and making the system vulnerable to attacks.

Therefore, Option C is the best solution because it directly addresses the requirements for secure, user-triggered access using a temporary and limited scope permission.

Reference:

Using Presigned URLs - Amazon Simple Storage Service

Question: 908 **Exam Heist**

A company has applications that run in an organization in AWS Organizations. The company outsources operational support of the applications. The company needs to provide access for the external support engineers without compromising security.

The external support engineers need access to the AWS Management Console. The external support engineers also need operating system access to the company’s fleet ofAmazon EC2 instances that run Amazon Linux in private subnets.

Which solution will meet these requirements MOST securely?

A.Confirm that AWS Systems Manager Agent (SSM Agent) is installed on all instances. Assign an instance profile with the necessary policy to connect to Systems Manager. Use AWS IAM Identity Center to provide the external support engineers console access. Use Systems Manager Session Manager to assign the required permissions.

B.Confirm that AWS Systems Manager Agent (SSM Agent) is installed on all instances. Assign an instance profile with the necessary policy to connect to Systems Manager. Use Systems Manager Session Manager to provide local IAM user credentials in each AWS account to the external support engineers for console access.

C.Confirm that all instances have a security group that allows SSH access only from the external support engineers' source IP address ranges. Provide local IAM user credentials in each AWS account to the external support engineers for console access. Provide each external support engineer an SSH key pair to log in to the application instances.

D.Create a bastion host in a public subnet. Set up the bastion host security group to allow access from only the external engineers' IP address ranges. Ensure that all instances have a security group that allows SSH access from the bastion host. Provide each external support engineer an SSH key pair to log in to the application instances. Provide local account IAM user credentials to the engineers for console access.

Answer: A

Explanation:

Here's why option A is the most secure solution:

IAM Identity Center for Console Access: IAM Identity Center (successor to AWS SSO) is the recommended way to provide federated access to the AWS Management Console. It allows the external support engineers to use their existing corporate credentials or a directory managed by AWS, rather than creating individual IAM users. This centralizes access control and improves security posture. https://aws.amazon.com/iam/identity-center/

SSM Session Manager for Instance Access: Session Manager, a feature of AWS Systems Manager, enables secure and auditable instance access without opening inbound SSH ports or managing SSH keys. This eliminates the attack surface associated with SSH, making it much more secure. https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html

Principle of Least Privilege: By using IAM Identity Center and Session Manager, you can grant the external support engineers only the necessary permissions to perform their tasks. This follows the principle of least privilege, minimizing the potential impact of a compromised account.

Auditing and Logging: Session Manager provides detailed auditing and logging of all session activity, including commands executed. This is crucial for security monitoring and compliance.

Centralized Access Control: IAM Identity Center provides centralized access control for multiple AWS accounts. Since the applications run across an organization using AWS Organizations, it centralizes authentication for external support engineers.

Options B, C, and D are less secure because:

Option B: Sharing local IAM user credentials is a security risk. If one account is compromised, the attacker gains access to all resources associated with that account.
Option C: Opening SSH ports to specific IP addresses is an improvement over opening them to the world, but it still introduces a security risk. IP addresses can be spoofed, and SSH is a common attack vector. Also, managing SSH keys is an overhead.
Option D: Bastion hosts provide a single point of entry to your infrastructure, which can be a security risk if the bastion host is compromised. It also requires managing another EC2 instance. Sharing IAM credentials will reduce security.

Therefore, Option A is the most secure solution because it uses IAM Identity Center for console access and Session Manager for instance access, avoiding the risks associated with SSH and shared IAM credentials.

Question: 909 **Exam Heist**

A company uses Amazon RDS for PostgreSQL to run its applications in the us-east-1 Region. The company also uses machine learning (ML) models to forecast annual revenue based on near real-time reports. The reports are generated by using the same RDS for PostgreSQL database. The database performance slows during business hours. The company needs to improve database performance.

Which solution will meet these requirements MOST cost-effectively?

A.Create a cross-Region read replica. Configure the reports to be generated from the read replica.

B.Activate Multi-AZ DB instance deployment for RDS for PostgreSQL. Configure the reports to be generated from the standby database.

C.Use AWS Data Migration Service (AWS DMS) to logically replicate data to a new database. Configure the reports to be generated from the new database.

D.Create a read replica in us-east-1. Configure the reports to be generated from the read replica.

Answer: D

Explanation:

The correct answer is **D. Create a read replica in us-east-1. Configure the reports to be generated from the read replica.**

Justification:

The primary issue is database performance degradation due to report generation competing with application workloads on the same RDS instance. A read replica allows read-only traffic to be offloaded from the primary database, thus freeing up resources for the primary application workload and improving performance during business hours.

Option D is the most cost-effective solution because it utilizes a read replica within the same region (us-east-1). Creating a read replica in the same region avoids cross-region data transfer costs, which can be significant, especially for near real-time reporting. Also, network latency between the primary and replica is minimized when they are located within the same region.

Option A is incorrect as cross-region read replicas introduce higher latency due to geographical distance, which can impact the timeliness of the near real-time reports. Additionally, cross-region data transfer incurs higher costs compared to intra-region replication.

Option B, activating Multi-AZ, primarily focuses on high availability and failover capabilities. While Multi-AZ improves resilience, it does not offload read traffic in the same way a read replica does. The standby database in a Multi-AZ setup is primarily for failover purposes and is not designed for consistent read operations. Therefore, using the standby database for report generation isn't generally supported, and the secondary is not designed for read operations.

Option C, using AWS DMS, is an overkill solution for this scenario. AWS DMS is typically used for migrating databases to different platforms or for creating data warehouses. Using DMS to replicate data for reporting adds unnecessary complexity and cost compared to the simplicity and efficiency of a read replica, where changes are automatically replicated. Read replicas are specifically designed for this purpose and are much easier to maintain in this case.Because the application is for near real-time reports, logical replication is not a good approach. The read replica works at the storage layer, but DMS works at the logical layer and can introduce latency.

Authoritative Links:

Amazon RDS Read Replicas:

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.ReadReplicas.html
Amazon RDS Multi-AZ Deployments:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html
AWS Data Migration Service: https://aws.amazon.com/dms/

Question: 910 **Exam Heist**

A company hosts its multi-tier, public web application in the AWS Cloud. The web application runs on Amazon EC2 instances, and its database runs on Amazon RDS. The company is anticipating a large increase in sales during an upcoming holiday weekend. A solutions architect needs to build a solution to analyze the performance of the web application with a granularity of no more than 2 minutes.

What should the solutions architect do to meet this requirement?

A.Send Amazon CloudWatch logs to Amazon Redshift. Use Amazon QuickS ght to perform further analysis.

B.Enable detailed monitoring on all EC2 instances. Use Amazon CloudWatch metrics to perform further analysis.

C.Create an AWS Lambda function to fetch EC2 logs from Amazon CloudWatch Logs. Use Amazon CloudWatch metrics to perform further analysis.

D.Send EC2 logs to Amazon S3. Use Amazon Redshift to fetch logs from the S3 bucket to process raw data for further analysis with Amazon QuickSight.

Answer: B

Explanation:

The requirement is to analyze the web application's performance with a granularity of no more than 2 minutes.

Option A: Sending CloudWatch logs to Amazon Redshift and using QuickSight is more suitable for long-term trend analysis and complex querying across large datasets of log data. It's not the most efficient approach for real-time or near-real-time monitoring at a 2-minute granularity.

Option B: Enabling detailed monitoring on EC2 instances provides metrics at a 1-minute interval. These metrics are readily available in CloudWatch. This allows for a granular view of EC2 performance, making it suitable for identifying performance bottlenecks quickly. Using CloudWatch metrics directly fulfills the 2-minute granularity requirement without complex data transformations.

Option C: Using Lambda to fetch EC2 logs from CloudWatch Logs introduces unnecessary complexity and latency. While CloudWatch Logs contains detailed information, pulling and processing them via Lambda adds overhead and doesn't directly leverage readily available metrics. Furthermore, CloudWatch metrics directly provide aggregated data.

Option D: Sending EC2 logs to S3 and then using Redshift and QuickSight for analysis is an overly complex solution for near real-time monitoring. Similar to option A, this is better suited for large-scale, historical log analysis and reporting, not the immediate performance monitoring required.

Therefore, the most appropriate solution is to enable detailed monitoring on the EC2 instances and use the available CloudWatch metrics. This approach directly satisfies the need for a 2-minute granularity and avoids unnecessary complexity.

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-cloudwatch-metrics.htmlhttps://aws.amazon.com/cloudwatch/features/

Question: 911 **Exam Heist**

A company runs an application that stores and shares photos. Users upload the photos to an Amazon S3 bucket. Every day, users upload approximately 150 photos. The company wants to design a solution that creates a thumbnail of each new photo and stores the thumbnail in a second S3 bucket.

Which solution will meet these requirements MOST cost-effectively?

A.Configure an Amazon EventBridge scheduled rule to invoke a script every minute on a long-running Amazon EMR cluster. Configure the script to generate thumbnails for the photos that do not have thumbnails. Configure the script to upload the thumbnails to the second S3 bucket.

B.Configure an Amazon EventBridge scheduled rule to invoke a script every minute on a memory-optimized Amazon EC2 instance that is always on. Configure the script to generate thumbnails for the photos that do not have thumbnails. Configure the script to upload the thumbnails to the second S3 bucket.

C.Configure an S3 event notification to invoke an AWS Lambda function each time a user uploads a new photo to the application. Configure the Lambda function to generate a thumbnail and to upload the thumbnail to the second S3 bucket.

D.Configure S3 Storage Lens to invoke an AWS Lambda function each time a user uploads a new photo to the application. Configure the Lambda function to generate a thumbnail and to upload the thumbnail to a second S3 bucket.

Answer: C

Explanation:

The most cost-effective solution is to use S3 event notifications and Lambda functions. This approach utilizes a serverless architecture, paying only for the compute time used when thumbnails are actually generated.

Option C is superior because S3 event notifications directly trigger a Lambda function upon each photo upload. This event-driven approach ensures that thumbnails are generated almost immediately after an upload. The Lambda function performs the image processing and stores the thumbnail in the destination S3 bucket. Since it's triggered by events, it scales automatically with the number of uploads, and there are no idle costs.

Option A, using a long-running EMR cluster, is highly inefficient and expensive. EMR is designed for large-scale data processing and analysis, not for real-time image manipulation. Keeping an EMR cluster running continuously incurs significant costs, even when not actively processing images. The scheduled script adds further overhead and delay.

Option B, using an always-on EC2 instance, is also more costly than Lambda. While potentially faster than EMR, an always-on EC2 instance incurs costs even when idle. The scheduled script and instance management add operational overhead.

Option D, using S3 Storage Lens, is incorrect because S3 Storage Lens is designed for storage analytics and visibility, not for triggering actions upon object creation. It does not invoke Lambda functions based on individual file uploads.

Therefore, option C is the most cost-effective and efficient solution because it leverages the serverless capabilities of Lambda, ensuring minimal costs and immediate thumbnail generation through S3 event triggers.

Relevant links:

AWS Lambda Pricing
Amazon S3 Event Notifications

Question: 912 **Exam Heist**

A company has stored millions of objects across multiple prefixes in an Amazon S3 bucket by using the Amazon S3 Glacier Deep Archive storage class. The company needs to delete all data older than 3 years except for a subset of data that must be retained. The company has identified the data that must be retained and wants to implement a serverless solution.

Which solution will meet these requirements?

A.Use S3 Inventory to list all objects. Use the AWS CLI to create a script that runs on an Amazon EC2 instance that deletes objects from the inventory list.

B.Use AWS Batch to delete objects older than 3 years except for the data that must be retained.

C.Provision an AWS Glue crawler to query objects older than 3 years. Save the manifest file of old objects. Create a script to delete objects in the manifest.

D.Enable S3 Inventory. Create an AWS Lambda function to filter and delete objects. Invoke the Lambda function with S3 Batch Operations to delete objects by using the inventory reports.

Answer: D

Explanation:

The correct answer is **D** because it offers the most efficient and serverless approach to meet the complex requirements of filtering and deleting objects within Amazon S3 Glacier Deep Archive. Let's break down why:

Why Option D is Correct:

S3 Inventory: This is crucial for generating a comprehensive list of all objects in the S3 bucket, along with their metadata, including storage class and last modified date. This inventory serves as the foundation for filtering the data. https://docs.aws.amazon.com/AmazonS3/latest/userguide/storage-inventory.html

AWS Lambda: Lambda functions allow for serverless, event-driven code execution. In this scenario, the Lambda function can be programmed to filter the S3 Inventory report based on age (older than 3 years) and any other criteria that identify the data to be retained. It provides the custom logic needed to decide which objects to delete. https://docs.aws.amazon.com/lambda/latest/dg/welcome.html

S3 Batch Operations: This service enables you to perform large-scale batch operations on S3 objects. It can be used to invoke the Lambda function on each object identified in the S3 Inventory report. This avoids the need to manage infrastructure or write complex parallel processing code. S3 Batch Operations seamlessly integrates with Lambda for object manipulation.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/batch-ops-basics.html

Glacier Deep Archive Consideration: The Glacier Deep Archive storage class necessitates restoration of objects before they can be deleted. S3 Batch Operations handles that by automatically initiating the necessary restore operation.

Why Other Options Are Incorrect:

Option A (EC2 Script): This is not serverless, requires managing an EC2 instance, and is less scalable than S3 Batch Operations. The CLI-based approach is also slower and less efficient for such a large dataset.

Option B (AWS Batch): While AWS Batch can handle large-scale processing, it's generally not designed for directly managing S3 objects and object deletion. It would require more complex setup and integration compared to S3 Batch Operations.

Option C (AWS Glue Crawler): AWS Glue crawlers are primarily for discovering and cataloging data schemas in data lakes. It's not the appropriate tool for directly managing and deleting S3 objects based on age. Using it to generate a manifest file would also add unnecessary complexity. Glue is better suited for cataloging the data for later analysis or querying, not for deletion purposes.

In summary, Option D leverages the power of S3 Inventory, Lambda, and S3 Batch Operations to provide a fully serverless, scalable, and efficient solution for filtering and deleting objects from S3 Glacier Deep Archive based on specific criteria. It minimizes operational overhead and maximizes performance, making it the most suitable answer.

Question: 913 **Exam Heist**

A company is building an application on AWS. The application uses multiple AWS Lambda functions to retrieve sensitive data from a single Amazon S3 bucket for processing. The company must ensure that only authorized Lambda functions can access the data. The solution must comply with the principle of least privilege.

Which solution will meet these requirements?

A.Grant full S3 bucket access to all Lambda functions through a shared IAM role.

B.Configure the Lambda functions to run within a VPC. Configure a bucket policy to grant access based on the Lambda functions' VPC endpoint IP addresses.

C.Create individual IAM roles for each Lambda function. Grant the IAM roles access to the S3 bucket. Assign each IAM role as the Lambda execution role for its corresponding Lambda function.

D.Configure a bucket policy granting access to the Lambda functions based on their function ARNs.

Answer: C

Explanation:

The correct answer is **C: Create individual IAM roles for each Lambda function. Grant the IAM roles access to the S3 bucket. Assign each IAM role as the Lambda execution role for its corresponding Lambda function.**

Here's why:

Principle of Least Privilege: This principle dictates granting only the minimum necessary permissions to perform a task. Option C directly adheres to this by providing each Lambda function with only the permissions it needs to access the S3 bucket through its unique IAM role.
IAM Roles for Lambda: Lambda functions require an IAM role (execution role) that defines the permissions the function has when accessing AWS resources. This role acts as a security identity for the function.
Granular Access Control: Using individual IAM roles allows fine-grained control. If one Lambda function is compromised, the attacker only gains access to the resources that function's role is permitted to access, limiting the blast radius.
S3 Bucket Policies: While S3 bucket policies can grant access, directly associating roles to Lambda functions is considered best practice because it scales well with many Lambdas.

Why other options are incorrect:

A: Grant full S3 bucket access to all Lambda functions through a shared IAM role. This violates the principle of least privilege. All Lambda functions have excessive permissions, increasing the risk if one function is compromised.
B: Configure the Lambda functions to run within a VPC. Configure a bucket policy to grant access based on the Lambda functions' VPC endpoint IP addresses. Lambda functions run in an AWS managed network. VPC Endpoints can be used, but this adds unnecessary complexity and cost for the described scenario. Moreover, Lambda function IP addresses can change, making IP-based policies unreliable.
This approach is more complex than using individual IAM roles.
Maintaining VPC endpoint IP addresses in the bucket policy would be an ongoing operational burden.

Authoritative Links:

IAM Roles for Lambda: https://docs.aws.amazon.com/lambda/latest/dg/lambda-intro-execution-role.html
IAM Best Practices: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html
S3 Bucket Policies: https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-iam-policies.html

Question: 914 **Exam Heist**

A company has developed a non-production application that is composed of multiple microservices for each of the company's business units. A single development team maintains all the microservices.

The current architecture uses a static web frontend and a Java-based backend that contains the application logic. The architecture also uses a MySQL database that the company hosts on an Amazon EC2 instance.

The company needs to ensure that the application is secure and available globally.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use Amazon CloudFront and AWS Amplify to host the static web frontend. Refactor the microservices to use AWS Lambda functions that the microservices access by using Amazon API Gateway. Migrate the MySQL database to an Amazon EC2 Reserved Instance.

B.Use Amazon CloudFront and Amazon S3 to host the static web frontend. Refactor the microservices to use AWS Lambda functions that the microservices access by using Amazon API Gateway. Migrate the MySQL database to Amazon RDS for MySQL.

C.Use Amazon CloudFront and Amazon S3 to host the static web frontend. Refactor the microservices to use AWS Lambda functions that are in a target group behind a Network Load Balancer. Migrate the MySQL database to Amazon RDS for MySQL.

D.Use Amazon S3 to host the static web frontend. Refactor the microservices to use AWS Lambda functions that are in a target group behind an Application Load Balancer. Migrate the MySQL database to an Amazon EC2 Reserved Instance.

Answer: B

Explanation:

The correct answer is B. Here's a detailed justification:

Static Web Frontend: CloudFront combined with S3 offers a highly scalable, globally distributed, and cost-effective solution for hosting static content. S3 provides durable object storage, and CloudFront caches the content at edge locations for low latency access worldwide. This approach minimizes operational overhead compared to managing servers.

Microservices Refactoring: Refactoring the Java-based backend microservices into Lambda functions offers several benefits:

Serverless: Lambda eliminates the need to manage servers, patching, and scaling. AWS handles the underlying infrastructure.
Scalability: Lambda automatically scales based on demand, ensuring high availability.
Cost Optimization: You only pay for the compute time consumed by the functions.
API Gateway: API Gateway acts as a front door for the Lambda functions, enabling secure and managed access to the microservices. It handles authentication, authorization, rate limiting, and API versioning.

Database Migration: Migrating the MySQL database from an EC2 instance to Amazon RDS for MySQL provides a managed database service. This shift offloads database administration tasks such as patching, backups, and scaling to AWS. RDS offers high availability options (Multi-AZ) and simplifies the database management process, significantly reducing operational overhead.

Alternatives A and D are incorrect because they suggest using EC2 Reserved Instances for the database. While Reserved Instances can provide cost savings, managing the database on EC2 still requires significant

operational overhead. Option C is less ideal for routing Lambda functions using a Network Load Balancer. API Gateway provides more fine-grained control over API access and management for Lambda functions.

Authoritative Links:

Amazon S3: https://aws.amazon.com/s3/
Amazon CloudFront: https://aws.amazon.com/cloudfront/
AWS Lambda: https://aws.amazon.com/lambda/
Amazon API Gateway: https://aws.amazon.com/api-gateway/
Amazon RDS: https://aws.amazon.com/rds/

Question: 915 **Exam Heist**

A video game company is deploying a new gaming application to its global users. The company requires a solution that will provide near real-time reviews and rankings of the players.

A solutions architect must design a solution to provide fast access to the data. The solution must also ensure the data persists on disks in the event that the company restarts the application.

Which solution will meet these requirements with the LEAST operational overhead?

A.Configure an Amazon CloudFront distribution with an Amazon S3 bucket as the origin. Store the player data in the S3 bucket.

B.Create Amazon EC2 instances in multiple AWS Regions. Store the player data on the EC2 instances. Configure Amazon Route 53 with geolocation records to direct users to the closest EC2 instance.

C.Deploy an Amazon ElastiCache for Redis duster. Store the player data in the ElastiCache cluster.

D.Deploy an Amazon ElastiCache for Memcached duster. Store the player data in the ElastiCache cluster.

Answer: C

Explanation:

The correct answer is **C. Deploy an Amazon ElastiCache for Redis cluster. Store the player data in the ElastiCache cluster.**

Here's a detailed justification:

The core requirement is fast access to near real-time data (reviews and rankings) for a global user base, with data persistence. ElastiCache for Redis perfectly fits this scenario because it's an in-memory data store. This provides extremely low latency reads and writes, which is crucial for real-time applications. Redis also supports data persistence by periodically writing data to disk (RDB snapshots) or by appending every write operation to a log file (AOF). This ensures data survival even if the ElastiCache cluster restarts. The cluster setup distributes the load and improves availability.

Option A is not suitable because S3 is object storage, not designed for near real-time data access. The latency associated with S3 reads/writes is significantly higher than in-memory solutions. CloudFront caching improves read performance, but the initial request still goes to S3.

Option B involves managing EC2 instances, which entails significant operational overhead for patching, scaling, and ensuring high availability. Furthermore, manually setting up data replication across regions adds complexity. While Route 53 geolocation improves latency, the underlying EC2 instance setup is complex and more operationally intensive than a managed service.

Option D, ElastiCache for Memcached, is also an in-memory data store, providing low latency. However, Memcached does not offer built-in data persistence. Since data persistence is a strict requirement, Memcached is unsuitable. Redis provides both speed and the necessary persistence using RDB and AOF

features.

Therefore, ElastiCache for Redis offers the best combination of low latency, scalability, persistence, and reduced operational overhead compared to the other options. It's a managed service, simplifying management tasks.

Further Reading:

Amazon ElastiCache for Redis: https://aws.amazon.com/elasticache/redis/
Redis Persistence: https://redis.io/docs/management/persistence/

Question: 916 **Exam Heist**

A company is designing an application on AWS that processes sensitive data. The application stores and processes financial data for multiple customers.

To meet compliance requirements, the data for each customer must be encrypted separately at rest by using a secure, centralized key management solution. The company wants to use AWS Key Management Service (AWS KMS) to implement encryption.

Which solution will meet these requirements with the LEAST operational overhead?

A.Generate a unique encryption key for each customer. Store the keys in an Amazon S3 bucket. Enable server-side encryption.

B.Deploy a hardware security appliance in the AWS environment that securely stores customer-provided encryption keys. Integrate the security appliance with AWS KMS to encrypt the sensitive data in the application.

C.Create a single AWS KMS key to encrypt all sensitive data across the application.

D.Create separate AWS KMS keys for each customer's data that have granular access control and logging enabled.

Answer: D

Explanation:

The correct solution is D: "Create separate AWS KMS keys for each customer's data that have granular access control and logging enabled."

Here's a detailed justification:

AWS KMS is designed for managing encryption keys securely and centrally. The requirement is to encrypt data for each customer separately at rest using KMS with minimal operational overhead.

Option A is incorrect because storing keys in S3, even with server-side encryption, introduces key management complexity outside of KMS, increasing operational overhead and potentially compromising security. It goes against the centralized key management requirement. S3 server-side encryption won't provide the level of granular access control needed.

Option B introduces a hardware security appliance (HSM). While HSMs can be used with KMS (AWS CloudHSM), they significantly increase operational overhead because they require managing and maintaining physical hardware within the AWS environment. This contradicts the requirement for minimal operational overhead.

Option C is inadequate for compliance. Using a single KMS key for all customer data violates the requirement that data for each customer must be encrypted separately. If the single key were compromised, all customer data would be at risk. It also hinders granular access control as all customers would technically be using the same key.

Option D provides the best solution. Creating a separate KMS key for each customer satisfies the requirement for individual encryption. KMS automatically handles key generation, storage, and rotation securely. Granular access control can be implemented using IAM policies to restrict access to each key to only authorized personnel or services for that specific customer. Enabling logging provides audit trails for key usage, which is essential for compliance. This approach centralizes key management within AWS KMS, minimizing operational overhead while meeting security and compliance needs. KMS is designed to scale and manage many keys effectively. This solution fulfills all given requirements.

Relevant links for further research:

AWS KMS Best Practices: https://docs.aws.amazon.com/kms/latest/developerguide/best-practices.html
AWS KMS Key Management: https://aws.amazon.com/kms/key-management/
AWS IAM Policies: https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies.html

Question: 917 **Exam Heist**

A company needs to design a resilient web application to process customer orders. The web application must automatically handle increases in web traffic and application usage without affecting the customer experience or losing customer orders.

Which solution will meet these requirements?

A.Use a NAT gateway to manage web traffic. Use Amazon EC2 Auto Scaling groups to receive, process, and store processed customer orders. Use an AWS Lambda function to capture and store unprocessed orders.

B.Use a Network Load Balancer (NLB) to manage web traffic. Use an Application Load Balancer to receive customer orders from the NLUse Amazon Redshift with a Multi-AZ deployment to store unprocessed and processed customer orders.

C.Use a Gateway Load Balancer (GWLB) to manage web traffic. Use Amazon Elastic Container Service (Amazon ECS) to receive and process customer orders. Use the GWLB to capture and store unprocessed orders. Use Amazon DynamoDB to store processed customer orders.

D.Use an Application Load Balancer to manage web traffic. Use Amazon EC2 Auto Scaling groups to receive and process customer orders. Use Amazon Simple Queue Service (Amazon SQS) to store unprocessed orders. Use Amazon RDS with a Multi-AZ deployment to store processed customer orders.

Answer: D

Explanation:

The correct solution is **D**, leveraging an Application Load Balancer, EC2 Auto Scaling groups, Amazon SQS, and Amazon RDS with Multi-AZ deployment for resilience and scalability in a web application processing customer orders.

Here's why:

1. **Application Load Balancer (ALB):** ALBs are designed for HTTP/HTTPS traffic and excel at routing requests based on content, making them ideal for managing web traffic and distributing incoming customer orders across the application instances. https://aws.amazon.com/elasticloadbalancing/application-load-balancer/

2. **EC2 Auto Scaling Groups:** Auto Scaling automatically adjusts the number of EC2 instances based on demand. By placing the application servers within Auto Scaling groups, the solution can automatically scale up during traffic surges and scale down during periods of low activity, ensuring consistent performance and availability. https://aws.amazon.com/autoscaling/

3. **Amazon SQS (Simple Queue Service):** SQS provides a reliable message queue for asynchronous communication. Storing unprocessed orders in SQS acts as a buffer during peak loads. This

decoupling allows the web application to quickly acknowledge receipt of the order without needing to process it immediately. If the processing servers are temporarily unavailable or overloaded, the messages remain in the queue until they can be successfully processed, preventing data loss and ensuring eventual consistency. https://aws.amazon.com/sqs/

4. **Amazon RDS with Multi-AZ deployment:** RDS with Multi-AZ ensures high availability and durability for the processed customer order data. Multi-AZ automatically provisions and maintains a synchronous standby database in a different Availability Zone. In the event of a primary database failure, RDS automatically fails over to the standby, minimizing downtime and data loss. This is crucial for maintaining the integrity of processed order information.
https://aws.amazon.com/rds/features/multi-az/

Why other options are less suitable:

A: NAT Gateway is for outbound internet access, not for managing incoming web traffic. Lambda is useful, but SQS is better at queuing unprocessed orders for resilience. Storing orders in Lambda directly is not scalable or durable.
B: While NLB is for TCP/UDP, ALB is more suited for web applications, providing advanced routing features. Redshift is an analytical data warehouse, not an operational database suited for real-time order processing.
C: GWLB is for virtual appliances like firewalls, not general web traffic. ECS is fine, but using GWLB to capture unprocessed orders is an atypical and inefficient usage. DynamoDB is a good choice but the overall architecture is flawed.

Therefore, solution **D** offers the most appropriate and robust approach to building a resilient and scalable web application for customer order processing.

Question: 918 **Exam Heist**

A company is using AWS DataSync to migrate millions of files from an on-premises system to AWS. The files are 10 KB in size on average.

The company wants to use Amazon S3 for file storage. For the first year after the migration, the files will be accessed once or twice and must be immediately available. After 1 year, the files must be archived for at least 7 years.

Which solution will meet these requirements MOST cost-effectively?

A.Use an archive tool to group the files into large objects. Use DataSync to migrate the objects. Store the objects in S3 Glacier Instant Retrieval for the first year. Use a lifecycle configuration to transition the files to S3 Glacier Deep Archive after 1 year with a retention period of 7 years.

B.Use an archive tool to group the files into large objects. Use DataSync to copy the objects to S3 Standard-Infrequent Access (S3 Standard-IA). Use a lifecycle configuration to transition the files to S3 Glacier Instant Retrieval after 1 year with a retention period of 7 years.

C.Configure the destination storage class for the files as S3 Glacier Instant Retrieval. Use a lifecycle policy to transition the files to S3 Glacier Flexible Retrieval after 1 year with a retention period of 7 years.

D.Configure a DataSync task to transfer the files to S3 Standard-Infrequent Access (S3 Standard-IA). Use a lifecycle configuration to transition the files to S3 Deep Archive after 1 year with a retention period of 7 years.

Answer: A

Explanation:

The best approach for cost-effectively migrating and archiving millions of small files to S3, with the specified access patterns and retention, is to archive them into larger objects first and leverage S3 Glacier tiers.

Here's why option A is the most cost-effective:

Grouping Files: Archiving tools (like tar or zip) bundle small files into larger objects. This drastically reduces the overhead associated with storing millions of individual objects in S3. S3 charges are partly based on the number of objects stored, so reducing the object count saves money.

Initial Storage Tier (S3 Glacier Instant Retrieval): For the first year, files need to be immediately accessible (accessed once or twice). S3 Glacier Instant Retrieval is the lowest-cost archive storage class that offers the lowest latency and millisecond retrieval, making it suitable for occasional, immediate access.

Lifecycle Transition to S3 Glacier Deep Archive: After one year, the files can be moved to S3 Glacier Deep Archive for long-term storage. Glacier Deep Archive offers the lowest storage cost for data that is rarely accessed.

Lifecycle Configuration: S3 Lifecycle rules automate the transition between storage classes based on time or other criteria. This automates the move from Glacier Instant Retrieval to Glacier Deep Archive after the initial year.

Why other options are less optimal:

Option B (S3 Standard-IA initially): S3 Standard-IA is designed for infrequently accessed data but is more expensive than Glacier Instant Retrieval for archiving. Also, it does not provide cost benefits in the given access patterns.
Option C (S3 Glacier Instant Retrieval to Glacier Flexible Retrieval): Glacier Flexible Retrieval is typically used for backup and disaster recovery and has retrieval costs. Deep Archive is more cost-effective for the archival timeframe of 7 years. Storing millions of small files directly without archiving would also be more expensive.
Option D (S3 Standard-IA to Deep Archive): S3 Standard-IA is more expensive than Glacier Instant Retrieval for archiving. Storing millions of small files directly without archiving would also be more expensive.

Authoritative Links:

S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
S3 Lifecycle Management: https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-examples.html
AWS DataSync: https://aws.amazon.com/datasync/

Question: 919 **Exam Heist**

A company recently performed a lift and shift migration of its on-premises Oracle database workload to run on an Amazon EC2 memory optimized Linux instance. The EC2 Linux instance uses a 1 TB Provisioned IOPS SSD (io1) EBS volume with 64,000 IOPS.

The database storage performance after the migration is slower than the performance of the on-premises database.

Which solution will improve storage performance?

A.Add more Provisioned IOPS SSD (io1) EBS volumes. Use OS commands to create a Logical Volume Management (LVM) stripe.

B.Increase the Provisioned IOPS SSD (io1) EBS volume to more than 64,000 IOPS.

C.Increase the size of the Provisioned IOPS SSD (io1) EBS volume to 2 TB.

D.Change the EC2 Linux instance to a storage optimized instance type. Do not change the Provisioned IOPS SSD (io1) EBS volume.

Answer: A

Explanation:

Here's a detailed justification for why option A is the correct solution to improve the Oracle database storage performance after the lift-and-shift migration:

The core problem is that the database's I/O requirements exceed the current EBS volume's capabilities, resulting in performance degradation compared to the on-premises setup. While the volume has 64,000 IOPS, this might not be sufficient for the workload, or the underlying EC2 instance is not fully utilizing it.

Option A, adding more io1 EBS volumes and using LVM striping, is the best approach because it provides increased aggregate IOPS and throughput. LVM striping distributes I/O requests across multiple physical volumes (EBS volumes in this case), effectively multiplying the available IOPS and throughput. This parallelization can significantly reduce latency and improve overall database performance. Each io1 volume has its own independent IOPS capability, and striping combines them.

Option B is incorrect because a single io1 volume is limited to a maximum of 64,000 IOPS (as of the SAA-C03 exam time). You can't exceed this limit on a single volume, regardless of the request. AWS documentation confirms this IOPS limit per volume.

Option C, increasing the size of the io1 volume to 2TB, would increase throughput (MB/s). However, since the problem is the need for more IOPS, simply increasing the volume size won't directly address the bottleneck. While larger volumes can have higher throughput, they do not circumvent the 64,000 IOPS limit on a single io1 volume.

Option D, changing the EC2 instance type to a storage optimized instance, might help to some extent by providing better networking and EBS optimization. However, it doesn't directly address the primary issue of insufficient IOPS for the database workload. Storage optimized instances generally provide better performance for workloads that heavily rely on EBS, but if the EBS volume itself is the bottleneck, changing the instance type alone won't solve the problem. Moreover, a memory optimized instance is probably already well suited for an Oracle database. The storage-optimized instance might introduce other constraints relative to the memory needs of the database.

In summary, Option A effectively addresses the performance bottleneck by increasing the overall IOPS available to the database through LVM striping across multiple io1 volumes. This allows the database to handle its I/O requirements more efficiently, improving performance to match or exceed the on-premises environment.

Authoritative Links:

EBS Volume Types: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html
LVM Overview: Linux documentation and tutorials on Logical Volume Management can provide in-depth information on striping.

Question: 920 **Exam Heist**

A company is migrating from a monolithic architecture for a web application that is hosted on Amazon EC2 to a serverless microservices architecture. The company wants to use AWS services that support an event-driven, loosely coupled architecture. The company wants to use the publish/subscribe (pub/sub) pattern.

Which solution will meet these requirements MOST cost-effectively?

A.Configure an Amazon API Gateway REST API to invoke an AWS Lambda function that publishes events to an Amazon Simple Queue Service (Amazon SQS) queue. Configure one or more subscribers to read events from the SQS queue.

B.Configure an Amazon API Gateway REST API to invoke an AWS Lambda function that publishes events to an Amazon Simple Notification Service (Amazon SNS) topic. Configure one or more subscribers to receive events from the SNS topic.

C.Configure an Amazon API Gateway WebSocket API to write to a data stream in Amazon Kinesis Data Streams

with enhanced fan-out. Configure one or more subscribers to receive events from the data stream.

D.Configure an Amazon API Gateway HTTP API to invoke an AWS Lambda function that publishes events to an Amazon Simple Notification Service (Amazon SNS) topic. Configure one or more subscribers to receive events from the topic.

Answer: D

Explanation:

The correct answer is **D**. Here's why:

The problem statement emphasizes a serverless, event-driven, loosely coupled microservices architecture using the publish/subscribe pattern and prioritizes cost-effectiveness.

Option D best fits these criteria:

Amazon API Gateway HTTP API: HTTP APIs are designed for low-cost, low-latency API calls, which is ideal for a microservices architecture that requires efficient communication. Compared to REST APIs, they offer a simpler and more cost-effective solution when the full features of REST API are not needed.

AWS Lambda: Lambda provides a serverless compute environment, which aligns with the architectural requirements. The Lambda function serves as the intermediary between API Gateway and the SNS topic, enabling event publication.

Amazon SNS: SNS directly implements the publish/subscribe pattern. A Lambda function can publish events to an SNS topic. Subscribers (e.g., Lambda functions, SQS queues, HTTP endpoints) receive these events asynchronously, fostering loose coupling.

Cost-Effectiveness: The combination of HTTP API, Lambda, and SNS provides a cost-effective solution due to pay-per-use pricing models. You only pay for what you use.

Why other options are less ideal:

A: Amazon SQS is a queuing service (point-to-point), not a publish/subscribe service. While SQS supports fan-out patterns, it's not its primary function and SNS is more optimized for that use case. Using SQS for pub/sub introduces complexity, management overhead, and potential cost inefficiencies.

B: While using a REST API and SNS would work, REST APIs offer more features than are needed here and the HTTP API offers a lower cost for the same functionality in this case.

C: Kinesis Data Streams with enhanced fan-out is designed for high-throughput, real-time data streaming, like video or IoT data. It's overkill and significantly more expensive for this application. WebSocket APIs are also specialized for real-time bi-directional communication which is not needed here.

In summary, Option D provides the most cost-effective solution using serverless services that naturally implement the publish/subscribe pattern, promoting loose coupling and efficient communication in a microservices environment.

Supporting Documentation:

Amazon SNS: https://aws.amazon.com/sns/
AWS Lambda: https://aws.amazon.com/lambda/
Amazon API Gateway: https://aws.amazon.com/api-gateway/
Choosing between HTTP APIs and REST APIs: https://aws.amazon.com/blogs/compute/choosing-between-apigateway-rest-apis-and-http-apis/

Question: 921 **Exam Heist**

A company recently migrated a monolithic application to an Amazon EC2 instance and Amazon RDS. The application has tightly coupled modules. The existing design of the application gives the application the ability to run on only a single EC2 instance.

The company has noticed high CPU utilization on the EC2 instance during peak usage times. The high CPU utilization corresponds to degraded performance on Amazon RDS for read requests. The company wants to reduce the high CPU utilization and improve read request performance.

Which solution will meet these requirements?

A.Resize the EC2 instance to an EC2 instance type that has more CPU capacity. Configure an Auto Scaling group with a minimum and maximum size of 1. Configure an RDS read replica for read requests.

B.Resize the EC2 instance to an EC2 instance type that has more CPU capacity. Configure an Auto Scaling group with a minimum and maximum size of 1. Add an RDS read replica and redirect all read/write traffic to the replica.

C.Configure an Auto Scaling group with a minimum size of 1 and maximum size of 2. Resize the RDS DB instance to an instance type that has more CPU capacity.

D.Resize the EC2 instance to an EC2 instance type that has more CPU capacity. Configure an Auto Scaling group with a minimum and maximum size of 1. Resize the RDS DB instance to an instance type that has more CPU capacity.

Answer: A

Explanation:

The correct answer is **A**. Here's why:

Addressing CPU Utilization on EC2: The primary issue is the high CPU utilization on the EC2 instance. Increasing the instance size to one with more CPU capacity directly addresses this problem. This provides the application with more processing power to handle the workload.

Auto Scaling Group Configuration: Configuring an Auto Scaling group with a minimum and maximum size of 1 ensures that exactly one EC2 instance is running at all times. This is crucial because the application is designed to run on a single EC2 instance, so scaling beyond one instance would break it. The Auto Scaling group provides the benefit of automatic instance replacement in case of failure, improving availability.

Improving Read Request Performance on RDS: The high CPU utilization on the EC2 instance impacts the performance of read requests on the RDS database. Creating an RDS read replica and redirecting read requests to it offloads the read workload from the primary RDS instance. This reduces the load on the primary database, improving its performance and overall application performance.

Why other options are incorrect:

B: Redirecting all read/write traffic to the replica is not correct. RDS Read Replicas are READ ONLY. Write traffic must go to the primary database instance.
C: Resizing the RDS DB instance to an instance type that has more CPU capacity is NOT the best solution. The RDS performance problems are caused by the read load and can be fixed with an RDS Read Replica. It is not recommended to modify the minimum and maximum size for the autoscaling group as the application is tightly coupled and cannot be scaled to more than one instance.
D: Resizing the RDS DB instance to an instance type that has more CPU capacity is NOT the best solution as read replicas are best for read-heavy applications.

Supporting Resources:

Amazon EC2 Instance Types: https://aws.amazon.com/ec2/instance-types/
Amazon EC2 Auto Scaling: https://aws.amazon.com/autoscaling/

Amazon RDS Read Replicas:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ReadRepl.html

Question: 922 **Exam Heist**

A company needs to grant a team of developers access to the company's AWS resources. The company must maintain a high level of security for the resources.

The company requires an access control solution that will prevent unauthorized access to the sensitive data.

Which solution will meet these requirements?

A.Share the IAM user credentials for each development team member with the rest of the team to simplify access management and to streamline development workflows.

B.Define IAM roles that have fine-grained permissions based on the principle of least privilege. Assign an IAM role to each developer.

C.Create IAM access keys to grant programmatic access to AWS resources. Allow only developers to interact with AWS resources through API calls by using the access keys.

D.Create an AWS Cognito user pool. Grant developers access to AWS resources by using the user pool.

Answer: B

Explanation:

The correct answer is B: Define IAM roles that have fine-grained permissions based on the principle of least privilege. Assign an IAM role to each developer. This solution adheres to security best practices by employing the principle of least privilege and using IAM roles, which are integral for secure access control in AWS.

Here's why other options are incorrect:

A: Sharing IAM user credentials is a severe security risk. It violates the principle of least privilege and makes auditing impossible since actions cannot be attributed to a specific user.
C: While IAM access keys grant programmatic access, they are long-term credentials that, if compromised, can lead to unauthorized access. It's preferable to use IAM roles which are short-term credentials assumed by entities. Furthermore, managing access at a user-by-user access key level is less scalable and maintainable than using IAM roles.
D: AWS Cognito user pools are primarily designed for managing user authentication and authorization for web and mobile applications. While Cognito can integrate with IAM to grant access to AWS resources, it introduces unnecessary complexity for internal team access, which is better handled directly by IAM.

IAM roles provide temporary credentials and can be assigned fine-grained permissions, ensuring that developers only have access to the specific resources they need to perform their tasks. By following the principle of least privilege, you minimize the potential impact of any security breaches or accidental misconfigurations. This approach improves security posture and simplifies access management by centralizing permissions in IAM.

For further reading, refer to the AWS IAM documentation:

IAM Roles
Security best practices in IAM

Question: 923 **Exam Heist**

A company hosts a monolithic web application on an Amazon EC2 instance. Application users have recently reported poor performance at specific times. Analysis of Amazon CloudWatch metrics shows that CPU utilization is

100% during the periods of poor performance.

The company wants to resolve this performance issue and improve application availability.

Which combination of steps will meet these requirements MOST cost-effectively? (Choose two.)

A.Use AWS Compute Optimizer to obtain a recommendation for an instance type to scale vertically.
B.Create an Amazon Machine Image (AMI) from the web server. Reference the AMI in a new launch template.
C.Create an Auto Scaling group and an Application Load Balancer to scale vertically.
D.Use AWS Compute Optimizer to obtain a recommendation for an instance type to scale horizontally.
E.Create an Auto Scaling group and an Application Load Balancer to scale horizontally.

Answer: BE

Explanation:

The question asks for the most cost-effective solution to address performance issues (100% CPU utilization) and improve availability for a monolithic web application on a single EC2 instance. The key is to scale effectively and cost-efficiently.

Option B is crucial because creating an AMI from the current server captures the application's state and configuration. This AMI serves as the base for launching new instances in an Auto Scaling group. Without an AMI, recreating the application environment on each new instance would be complex and time-consuming.

Option E directly addresses both performance and availability. An Auto Scaling group automatically scales the number of EC2 instances based on demand, reducing CPU utilization by distributing the workload across multiple servers (horizontal scaling). The Application Load Balancer (ALB) distributes incoming traffic evenly across these instances, ensuring no single instance is overwhelmed and improving fault tolerance. If one instance fails, the ALB directs traffic to healthy instances, maintaining application availability. This is more cost-effective than vertical scaling as it only uses resources when needed.

Options A and D are not ideal because AWS Compute Optimizer's recommendations would suggest instance types to scale the EC2 instance vertically. While vertical scaling can address CPU utilization, it represents a single point of failure and does not improve availability. Also, vertical scaling has limitations, reaching a point where further upgrades become extremely expensive and may not fully resolve the problem.

Option C suggests creating an Auto Scaling group and an ALB to scale vertically. This is incorrect, as Auto Scaling groups and ALBs are inherently used for horizontal scaling. Vertical scaling refers to increasing the resources (CPU, memory) of a single instance.

Therefore, the combination of creating an AMI (B) and using an Auto Scaling group with an ALB for horizontal scaling (E) provides the most cost-effective and reliable solution for improving application performance and availability in this scenario.

Relevant links:

Amazon EC2 Auto Scaling: https://aws.amazon.com/autoscaling/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
Amazon Machine Images (AMIs): https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AMIs.html

Question: 924 **Exam Heist**

A company runs all its business applications in the AWS Cloud. The company uses AWS Organizations to manage multiple AWS accounts.

A solutions architect needs to review all permissions that are granted to IAM users to determine which IAM users

have more permissions than required.

Which solution will meet these requirements with the LEAST administrative overhead?

A.Use Network Access Analyzer to review all access permissions in the company's AWS accounts.

B.Create an AWS CloudWatch alarm that activates when an IAM user creates or modifies resources in an AWS account.

C.Use AWS Identity and Access Management (IAM) Access Analyzer to review all the company’s resources and accounts.

D.Use Amazon Inspector to find vulnerabilities in existing IAM policies.

Answer: C

Explanation:

The correct answer is **C. Use AWS Identity and Access Management (IAM) Access Analyzer to review all the company’s resources and accounts.**

Here's a detailed justification:

IAM Access Analyzer is a feature of AWS IAM designed to identify resource access granted to principals outside of your AWS account or organization. It helps you refine your permissions and implement the principle of least privilege by analyzing resource-based policies and IAM policies. Access Analyzer continuously monitors your AWS environment and generates findings when it detects resource access that you may not have intended.

Here's why the other options are less suitable:

A. Use Network Access Analyzer to review all access permissions in the company's AWS accounts: Network Access Analyzer focuses on network reachability between resources, not on IAM permissions granted to users. It identifies unintended network access to your AWS resources.
B. Create an AWS CloudWatch alarm that activates when an IAM user creates or modifies resources in an AWS account: While CloudWatch can monitor actions taken by IAM users, it wouldn't provide a comprehensive review of granted permissions. It would only alert you to specific actions, requiring significant manual effort to identify over-permissioned users. Moreover, setting up and maintaining such alarms for every potential resource creation/modification across multiple accounts adds significant administrative overhead.
D. Use Amazon Inspector to find vulnerabilities in existing IAM policies: Amazon Inspector is primarily a vulnerability management service that assesses EC2 instances and container images for security vulnerabilities. While it might provide some insights related to security best practices, it's not designed to analyze IAM permissions or identify users with excessive privileges.

IAM Access Analyzer directly addresses the requirement of reviewing permissions and identifying users with more permissions than required with the least administrative overhead because it automates the analysis of policies and identifies potential risks based on external access. This makes it a far more efficient and effective solution compared to the other options.

Authoritative Links:

AWS IAM Access Analyzer: https://docs.aws.amazon.com/IAM/latest/UserGuide/access-analyzer.html
Principle of Least Privilege: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html#grant-least-privilege

Question: 925 **Exam Heist**

A company needs to implement a new data retention policy for regulatory compliance. As part of this policy, sensitive documents that are stored in an Amazon S3 bucket must be protected from deletion or modification for a

fixed period of time.

Which solution will meet these requirements?

A.Activate S3 Object Lock on the required objects and enable governance mode.

B.Activate S3 Object Lock on the required objects and enable compliance mode.

C.Enable versioning on the S3 bucket. Set a lifecycle policy to delete the objects after a specified period.

D.Configure an S3 Lifecycle policy to transition objects to S3 Glacier Flexible Retrieval for the retention duration.

Answer: B

Explanation:

The correct answer is **B. Activate S3 Object Lock on the required objects and enable compliance mode.**

Here's a detailed justification:

The requirement is to protect sensitive documents in an S3 bucket from deletion or modification for a fixed period to meet regulatory compliance. S3 Object Lock directly addresses this need by making objects immutable. This means that once an object is locked, it cannot be deleted or overwritten until the lock expires.

S3 Object Lock has two modes: governance mode and compliance mode. Governance mode allows users with specific IAM permissions to override the retention settings, offering flexibility but potentially compromising strict compliance. Compliance mode, on the other hand, provides the strongest level of protection. Once an object is locked in compliance mode, its retention settings cannot be altered by any user, including the root user. This ensures that the objects remain protected for the entire retention period, satisfying the regulatory requirement of preventing deletion or modification.

Option A, using governance mode, is less secure as authorized users could potentially bypass the retention policy. Option C, enabling versioning and a lifecycle policy, is not suitable because versioning doesn't prevent deletion of all versions. A user with the necessary permissions could delete all versions, effectively removing the object. Also, lifecycle policies automate the deletion of objects. Option D, transitioning objects to S3 Glacier Flexible Retrieval, focuses on cost optimization and long-term archiving rather than preventing modification or deletion during a specific retention period. Glacier storage doesn't inherently offer the immutability provided by Object Lock.

Therefore, activating S3 Object Lock in compliance mode is the only option that fully guarantees the immutability of objects to meet regulatory compliance, preventing unauthorized deletion or modification for the required duration.

Here are authoritative links for further research:

AWS S3 Object Lock: https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html
AWS S3 Versioning: https://docs.aws.amazon.com/AmazonS3/latest/userguide/versioning-workflows.html
AWS S3 Lifecycle: https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-mgmt.html
AWS S3 Glacier: https://aws.amazon.com/glacier/

Question: 926 **Exam Heist**

A company runs its customer-facing web application on containers. The workload uses Amazon Elastic Container Service (Amazon ECS) on AWS Fargate. The web application is resource intensive.

The web application needs to be available 24 hours a day, 7 days a week for customers. The company expects the application to experience short bursts of high traffic. The workload must be highly available.

Which solution will meet these requirements MOST cost-effectively?

A.Configure an ECS capacity provider with Fargate. Conduct load testing by using a third-party tool. Rightsize the Fargate tasks in Amazon CloudWatch.

B.Configure an ECS capacity provider with Fargate for steady state and Fargate Spot for burst traffic.

C.Configure an ECS capacity provider with Fargate Spot for steady state and Fargate for burst traffic.

D.Configure an ECS capacity provider with Fargate. Use AWS Compute Optimizer to rightsize the Fargate task.

Answer: B

Explanation:

The most cost-effective and highly available solution is **B. Configure an ECS capacity provider with Fargate for steady state and Fargate Spot for burst traffic.**

Here's why:

Fargate for Steady State: Fargate provides serverless compute for containers, eliminating the need to manage EC2 instances. Using Fargate for the consistent, base load ensures the application is always available (24/7) without the overhead of instance management.

Fargate Spot for Burst Traffic: Fargate Spot provides significant cost savings (up to 70%) compared to Fargate, by utilizing spare compute capacity. It's ideal for handling short bursts of high traffic, allowing you to scale affordably during peak demand. While Fargate Spot instances can be interrupted, the short duration of the expected traffic bursts minimizes the risk of significant disruption and the application's container orchestration can handle interruptions by quickly rescheduling containers on available capacity.

Capacity Provider: ECS Capacity Providers allow you to define a strategy for using both Fargate and Fargate Spot. You can configure a weighted strategy where Fargate handles the base load and Fargate Spot is used for scaling up during traffic spikes.

Cost Optimization: This approach optimizes cost by using the cheaper Fargate Spot for the fluctuating demand while relying on the reliability of Fargate for consistent performance. Options A and D lack cost optimization because they rely solely on Fargate, which is more expensive than Fargate Spot. Option C is less reliable, as it relies on spot instances for the steady state.

Load Testing (Included implicitly): Option A mentions load testing, which is a best practice for any containerized application, and this should be done regardless of the chosen deployment strategy. But, doing load testing, and manually right sizing the Fargate tasks isn't a complete solution for cost optimisation or reliable burst management.

Key Concepts:

Amazon ECS: A fully managed container orchestration service.
AWS Fargate: A serverless compute engine for containers.
Fargate Spot: A pricing option for Fargate that provides significant cost savings for interruptible workloads.
ECS Capacity Providers: Allows you to define a strategy for using different compute resources (e.g., Fargate, Fargate Spot).

Authoritative Links:

AWS Fargate: https://aws.amazon.com/fargate/
ECS Capacity Providers:
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/capacity_providers.html
Fargate Spot: https://aws.amazon.com/blogs/containers/fargate-spot-ecs-ec2/

Question: 927 **Exam Heist**

A company is building an application in the AWS Cloud. The application is hosted on Amazon EC2 instances behind an Application Load Balancer (ALB). The company uses Amazon Route 53 for the DNS.

The company needs a managed solution with proactive engagement to detect against DDoS attacks.

Which solution will meet these requirements?

A.Enable AWS Config. Configure an AWS Config managed rule that detects DDoS attacks.

B.Enable AWS WAF on the ALCreate an AWS WAF web ACL with rules to detect and prevent DDoS attacks. Associate the web ACL with the ALB.

C.Store the ALB access logs in an Amazon S3 bucket. Configure Amazon GuardDuty to detect and take automated preventative actions for DDoS attacks.

D.Subscribe to AWS Shield Advanced. Configure hosted zones in Route 53. Add ALB resources as protected resources.

Answer: D

Explanation:

The correct answer is D, subscribing to AWS Shield Advanced and configuring hosted zones in Route 53 and adding ALB resources as protected resources. Here's why:

AWS Shield Advanced is a managed Distributed Denial of Service (DDoS) protection service that provides enhanced detection and mitigation capabilities, including 24/7 access to the AWS DDoS Response Team (DRT). The DRT offers proactive engagement, custom mitigation strategies, and expert guidance during DDoS events, aligning perfectly with the requirement for proactive engagement.

Option A (AWS Config) focuses on configuration management and compliance. While AWS Config can detect configuration changes that might increase vulnerability to DDoS, it doesn't directly detect DDoS attacks or provide proactive engagement like AWS Shield Advanced does. AWS Config's rules are reactive, not proactive in the sense required for active DDoS mitigation support.

Option B (AWS WAF) offers protection against web application attacks, including some types of DDoS attacks targeting the application layer (Layer 7). However, it doesn't provide the comprehensive protection against all types of DDoS attacks (including volumetric attacks that target network infrastructure) or the proactive engagement offered by AWS Shield Advanced. Also, while WAF helps, it requires manual configuration of rules, and the proactive element is missing.

Option C (Amazon GuardDuty) is a threat detection service that analyzes logs for malicious activity. While it can potentially detect some indicators of a DDoS attack based on log analysis, it is not designed primarily for DDoS protection and lacks the specialized mitigation and DRT support that AWS Shield Advanced provides. GuardDuty focuses on detecting intrusions and suspicious behavior after they've occurred, not proactively preventing and mitigating DDoS attacks.

AWS Shield Advanced is specifically designed for DDoS protection, handles a broader range of attack types, and offers the crucial proactive engagement through the DRT. By associating the ALB (Application Load Balancer) as a protected resource and using Route 53 hosted zones, Shield Advanced can effectively protect the application's infrastructure and DNS from DDoS attacks, fulfilling all the requirements.

Further Resources:

AWS Shield: https://aws.amazon.com/shield/
AWS Shield Advanced Features: https://aws.amazon.com/shield/features/
AWS DDoS Response Team (DRT): https://aws.amazon.com/premiumsupport/knowledge-center/ddos-response-team-shield/

Question: 928 **Exam Heist**

A company hosts a video streaming web application in a VPC. The company uses a Network Load Balancer (NLB) to handle TCP traffic for real-time data processing. There have been unauthorized attempts to access the application.

The company wants to improve application security with minimal architectural change to prevent unauthorized attempts to access the application.

Which solution will meet these requirements?

A.Implement a series of AWS WAF rules directly on the NLB to filter out unauthorized traffic.

B.Recreate the NLB with a security group to allow only trusted IP addresses.

C.Deploy a second NLB in parallel with the existing NLB configured with a strict IP address allow list.

D.Use AWS Shield Advanced to provide enhanced DDoS protection and prevent unauthorized access attempts.

Answer: B

Explanation:

The correct answer is **B. Recreate the NLB with a security group to allow only trusted IP addresses.**

Here's a detailed justification:

The primary goal is to improve application security against unauthorized access attempts with minimal architectural changes. Security groups act as virtual firewalls that control inbound and outbound traffic at the instance level and, importantly, can be associated with Network Load Balancers (NLBs). By recreating the NLB and associating a security group with it, the company can explicitly define rules that allow only trusted IP addresses to connect to the NLB. This effectively whitelists the allowed traffic sources, preventing unauthorized access from any other IP addresses.

Option A is incorrect because AWS WAF (Web Application Firewall) is designed for HTTP/HTTPS traffic and cannot be directly associated with an NLB, which handles TCP/UDP traffic at Layer 4. NLBs operate at a lower layer and are not aware of the application-layer content that WAF needs to inspect.

Option C, deploying a second NLB in parallel, adds unnecessary complexity and cost. It also doesn't directly address the root cause of the problem, which is unauthorized IP addresses attempting to access the application. Managing two NLBs would also introduce operational overhead.

Option D, using AWS Shield Advanced, provides enhanced DDoS protection, but DDoS protection focuses on mitigating large-scale distributed attacks that aim to overwhelm the application. While Shield Advanced can offer some protection against malicious traffic, it's not primarily designed for restricting access based on specific IP addresses or creating a whitelist, like a security group does. Shield Advanced also incurs a higher cost compared to using security groups.

Therefore, using a security group on the NLB provides a simple, cost-effective, and targeted solution that directly addresses the requirement of allowing only trusted IP addresses to access the application with minimal architectural change. Security groups are natively integrated with AWS and are a fundamental security component.

Supporting links:

AWS Security Groups: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html
Network Load Balancer: https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html
AWS WAF: https://aws.amazon.com/waf/
AWS Shield: https://aws.amazon.com/shield/

Question: 929 **Exam Heist**

A healthcare company is developing an AWS Lambda function that publishes notifications to an encrypted Amazon Simple Notification Service (Amazon SNS) topic. The notifications contain protected health information (PHI).

The SNS topic uses AWS Key Management Service (AWS KMS) customer managed keys for encryption. The company must ensure that the application has the necessary permissions to publish messages securely to the SNS topic.

Which combination of steps will meet these requirements? (Choose three.)

A.Create a resource policy for the SNS topic that allows the Lambda function to publish messages to the topic.

B.Use server-side encryption with AWS KMS keys (SSE-KMS) for the SNS topic instead of customer managed keys.

C.Create a resource policy for the encryption key that the SNS topic uses that has the necessary AWS KMS permissions.

D.Specify the Lambda function's Amazon Resource Name (ARN) in the SNS topic's resource policy.

E.Associate an Amazon API Gateway HTTP API with the SNS topic to control access to the topic by using API Gateway resource policies.

F.Configure a Lambda execution role that has the necessary IAM permissions to use a customer managed key in AWS KMS.

Answer: ACF

Explanation:

The correct answer is ACF. Let's break down why each choice is either right or wrong:

A. Create a resource policy for the SNS topic that allows the Lambda function to publish messages to the topic. This is essential. SNS topics require explicit permissions for entities to publish to them. The resource policy on the SNS topic must include a statement granting the Lambda function's IAM role or account the sns:Publish permission. This allows the Lambda function to send messages to the SNS topic.

B. Use server-side encryption with AWS KMS keys (SSE-KMS) for the SNS topic instead of customer managed keys. This is incorrect. While SSE-KMS provides encryption, the question specifically states customer-managed keys are being used. Switching to default KMS keys doesn't address the core requirement of granting permissions to access the existing customer-managed key for encryption/decryption.

C. Create a resource policy for the encryption key that the SNS topic uses that has the necessary AWS KMS permissions. This is crucial because the SNS topic is encrypted with a customer-managed KMS key. The Lambda function needs permission to use this key for encryption. The KMS key's resource policy must grant the Lambda function's IAM role permissions to perform KMS actions like kms:GenerateDataKey, kms:Decrypt, and kms:Encrypt (depending on how SNS uses the key). Without these permissions, the Lambda function won't be able to publish messages to the encrypted SNS topic.

D. Specify the Lambda function's Amazon Resource Name (ARN) in the SNS topic's resource policy. While including the ARN in the SNS topic policy is good practice, it's not sufficient on its own. You need to specify what the ARN is allowed to do by specifying the appropriate action (sns:Publish). A is the better choice for the action as it is more descriptive.

E. Associate an Amazon API Gateway HTTP API with the SNS topic to control access to the topic by using API Gateway resource policies. This is incorrect. API Gateway is not required to enable a Lambda to publish directly to an SNS topic. API Gateway is typically used to expose SNS functionality as a REST API endpoint which is not described in this scenario. This also adds unnecessary complexity.

F. Configure a Lambda execution role that has the necessary IAM permissions to use a customer managed key in AWS KMS. This is a vital part of the solution. The Lambda function's IAM role (the execution role)

defines what AWS resources the Lambda function is allowed to access. The role must have IAM policy statements granting it permissions to perform KMS actions. This allows the Lambda to call the KMS service to encrypt data for SNS.

In summary, the Lambda function needs permission to publish to the SNS topic (A) and permission to use the KMS key that the SNS topic uses for encryption (C & F). Therefore, ACF is the correct answer.

Supporting Links:

SNS Encryption: https://docs.aws.amazon.com/sns/latest/dg/sns-server-side-encryption.html
KMS Permissions: https://docs.aws.amazon.com/kms/latest/developerguide/key-policies.html
SNS Resource Based Policies: https://docs.aws.amazon.com/sns/latest/dg/sns-using-identity-based-policies.html

Question: 930 **Exam Heist**

A company has an employee web portal. Employees log in to the portal to view payroll details. The company is developing a new system to give employees the ability to upload scanned documents for reimbursement. The company runs a program to extract text-based data from the documents and attach the extracted information to each employee's reimbursement IDs for processing.

The employee web portal requires 100% uptime. The document extract program runs infrequently throughout the day on an on-demand basis. The company wants to build a scalable and cost-effective new system that will require minimal changes to the existing web portal. The company does not want to make any code changes.

Which solution will meet these requirements with the LEAST implementation effort?

A.Run Amazon EC2 On-Demand Instances in an Auto Scaling group for the web portal. Use an AWS Lambda function to run the document extract program. Invoke the Lambda function when an employee uploads a new reimbursement document.

B.Run Amazon EC2 Spot Instances in an Auto Scaling group for the web portal. Run the document extract program on EC2 Spot Instances. Start document extract program instances when an employee uploads a new reimbursement document.

C.Purchase a Savings Plan to run the web portal and the document extract program. Run the web portal and the document extract program in an Auto Scaling group.

D.Create an Amazon S3 bucket to host the web portal. Use Amazon API Gateway and an AWS Lambda function for the existing functionalities. Use the Lambda function to run the document extract program. Invoke the Lambda function when the API that is associated with a new document upload is called.

Answer: D

Explanation:

Here's a detailed justification for why option D is the best solution, along with supporting concepts and links:

Option D offers the most cost-effective, scalable, and minimally invasive approach, aligning with the company's requirements of 100% uptime for the portal, infrequent document extraction, minimal code changes, and minimal implementation effort.

By hosting the web portal on Amazon S3, the company benefits from S3's inherent high availability and durability. S3 is designed for 99.999999999% (11 9's) of data durability and 99.99% availability. https://aws.amazon.com/s3/

Using API Gateway in conjunction with Lambda functions allows existing functionalities to be exposed and scaled without directly modifying the portal's codebase. API Gateway acts as a front door, routing requests to the appropriate Lambda function. https://aws.amazon.com/api-gateway/

Invoking a Lambda function when a new document is uploaded via the API (associated with upload) ensures

on-demand execution of the document extraction process only when needed. Lambda offers serverless compute, meaning the company doesn't need to manage servers for this infrequent task, contributing to cost savings. https://aws.amazon.com/lambda/

This serverless approach is significantly more cost-effective than running EC2 instances (options A, B, and C) constantly for the document extraction program. The company only pays when the Lambda function executes.

Options A, B, and C require managing EC2 instances, increasing operational overhead and cost, particularly for the infrequent document extraction process.

Option A using on-demand EC2 for the web portal would ensure high availability but is more expensive compared to serving static content from S3. Using Lambda for document extraction is good but the web portal component is not optimal.

Option B relies on EC2 Spot Instances, which can be interrupted, potentially disrupting the document extraction process. Spot Instances are also unsuitable for the web portal which requires 100% uptime.

Option C suggests using a Savings Plan for both the web portal and the document extraction program running in an Auto Scaling group. This means keeping instances running even when the document extraction program isn't actively processing documents, resulting in wasted resources.

Therefore, the serverless architecture using S3, API Gateway, and Lambda is the most efficient solution in terms of cost, scalability, and minimal operational overhead, fulfilling all the given requirements.

Question: 931 **Exam Heist**

A media company has a multi-account AWS environment in the us-east-1 Region. The company has an Amazon Simple Notification Service (Amazon SNS) topic in a production account that publishes performance metrics. The company has an AWS Lambda function in an administrator account to process and analyze log data.

The Lambda function that is in the administrator account must be invoked by messages from the SNS topic that is in the production account when significant metrics are reported.

Which combination of steps will meet these requirements? (Choose two.)

A.Create an IAM resource policy for the Lambda function that allows Amazon SNS to invoke the function.

B.Implement an Amazon Simple Queue Service (Amazon SQS) queue in the administrator account to buffer messages from the SNS topic that is in the production account. Configure the SQS queue to invoke the Lambda function.

C.Create an IAM policy for the SNS topic that allows the Lambda function to subscribe to the topic.

D.Use an Amazon EventBridge rule in the production account to capture the SNS topic notifications. Configure the EventBridge rule to forward notifications to the Lambda function that is in the administrator account.

E.Store performance metrics in an Amazon S3 bucket in the production account. Use Amazon Athena to analyze the metrics from the administrator account.

Answer: AB

Explanation:

The correct answer is AB. Here's a detailed justification:

A. Create an IAM resource policy for the Lambda function that allows Amazon SNS to invoke the function.

This step is crucial for enabling cross-account invocation. By default, a Lambda function can only be invoked by resources within the same account. To allow the SNS topic in the production account to trigger the Lambda

function in the administrator account, we need to grant SNS permission to do so. This is achieved by adding a resource-based IAM policy to the Lambda function. This policy explicitly states that the SNS topic (identified by its ARN) is allowed to invoke the lambda:InvokeFunction action on the function. Without this permission, SNS would be denied access.

Authoritative Link: https://docs.aws.amazon.com/lambda/latest/dg/services-sns.html (See section on "Granting SNS Permissions")

B. Implement an Amazon Simple Queue Service (Amazon SQS) queue in the administrator account to buffer messages from the SNS topic that is in the production account. Configure the SQS queue to invoke the Lambda function.

This step addresses a common best practice and architectural pattern in event-driven systems. Using an SQS queue as an intermediary buffer between the SNS topic and the Lambda function provides several benefits:

1. **Decoupling:** It decouples the publisher (SNS topic) from the subscriber (Lambda function). This means the SNS topic doesn't need to know about the Lambda function's availability or processing speed.
2. **Buffering/Asynchronous Processing:** The SQS queue acts as a buffer, storing messages if the Lambda function is temporarily unavailable or experiencing processing delays. This prevents message loss and ensures eventual delivery. This is especially useful for performance metrics, as some degree of latency is tolerable, but the data should not be lost.
3. **Scalability:** The SQS queue can handle spikes in traffic by accumulating messages until the Lambda function can process them.
4. **Error Handling/Retries:** SQS Dead Letter Queues (DLQs) can be configured to store messages that the Lambda function fails to process after a certain number of attempts. This provides a mechanism for debugging and handling errors.

To implement this, the SNS topic in the production account is configured to send messages to the SQS queue in the administrator account. Then, the SQS queue is configured as an event source for the Lambda function, meaning the Lambda function is automatically invoked whenever a new message arrives in the queue. An SQS queue policy will also be necessary to allow the SNS topic to send messages to the queue.

Authoritative Link: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-configuring-lambda-triggers.html (See section on "Using AWS Lambda with Amazon SQS")

Why other options are incorrect:

C. Create an IAM policy for the SNS topic that allows the Lambda function to subscribe to the topic. This is incorrect because the Lambda function resides in a different account and needs permission to be invoked by SNS, not to subscribe to the topic. IAM policies attached to the SNS topic control who can publish to the topic, not who can receive messages.

D. Use an Amazon EventBridge rule in the production account to capture the SNS topic notifications. Configure the EventBridge rule to forward notifications to the Lambda function that is in the administrator account. While EventBridge could be used to route events cross-account, it adds unnecessary complexity in this case, as SNS to SQS to Lambda is a simpler, more common, and often more cost-effective pattern. EventBridge is more appropriate when you need complex routing rules and transformation capabilities.

E. Store performance metrics in an Amazon S3 bucket in the production account. Use Amazon Athena to analyze the metrics from the administrator account. This option describes a completely different architecture for analyzing data. It doesn't address the requirement of triggering a Lambda function when significant metrics are reported via SNS. Athena is for batch processing and analysis of data at rest, not for real-time triggering based on SNS messages.

Question: 932 **Exam Heist**

A company is migrating an application from an on-premises location to Amazon Elastic Kubernetes Service (Amazon EKS). The company must use a custom subnet for pods that are in the company's VPC to comply with requirements. The company also needs to ensure that the pods can communicate securely within the pods' VPC.

Which solution will meet these requirements?

A.Configure AWS Transit Gateway to directly manage custom subnet configurations for the pods in Amazon EKS.

B.Create an AWS Direct Connect connection from the company's on-premises IP address ranges to the EKS pods.

C.Use the Amazon VPC CNI plugin for Kubernetes. Define custom subnets in the VPC cluster for the pods to use.

D.Implement a Kubernetes network policy that has pod anti-affinity rules to restrict pod placement to specific nodes that are within custom subnets.

Answer: C

Explanation:

The correct answer is C: Use the Amazon VPC CNI plugin for Kubernetes. Define custom subnets in the VPC cluster for the pods to use.

Here's a detailed justification:

The primary requirement is to utilize custom subnets for EKS pods within the company's VPC. The Amazon VPC CNI (Container Network Interface) plugin directly addresses this. It's designed to integrate Kubernetes pods seamlessly with the AWS VPC networking environment. This integration allows pods to receive IP addresses from the VPC's address space, enabling direct communication within the VPC and with other AWS resources. By defining custom subnets specifically for EKS pods, the company gains precise control over the pod network and adheres to its compliance requirements.

The VPC CNI plugin assigns IP addresses to pods from the subnet(s) you specify during cluster creation or update. You can dedicate particular subnets specifically for pod IP address assignment, ensuring compliance and isolation as required. This approach avoids the need for complex workarounds or external networking solutions. It utilizes native AWS networking capabilities for a streamlined and performant solution.

Option A is incorrect because AWS Transit Gateway manages connectivity between VPCs and on-premises networks, but it doesn't directly configure subnets for individual pods within an EKS cluster. Transit Gateway is not a pod networking solution.

Option B is incorrect because AWS Direct Connect is used to establish a dedicated network connection from on-premises to AWS, but it does not handle pod networking within EKS. Direct Connect manages connectivity at a higher level. Furthermore, you wouldn't connect to individual EKS pods via Direct Connect.

Option D is incorrect because Kubernetes network policies control traffic between pods, but they do not dictate which subnets pods reside in. Pod anti-affinity rules can influence pod placement but they don't force the use of custom subnets; they only influence which nodes pods land on, and only indirectly impact subnet usage if those nodes are configured to use specific subnets. The underlying need to configure the EKS cluster to use the desired subnets remains.

Therefore, the Amazon VPC CNI plugin, configured with the desired custom subnets, directly and efficiently fulfills the stated requirements.

Further Reading:

Amazon VPC CNI plugin for Kubernetes:
https://docs.aws.amazon.com/eks/latest/userguide/network_cni_plugin.html
Amazon EKS Networking: https://aws.amazon.com/eks/networking/

Question: 933 **Exam Heist**

A company hosts an ecommerce application that stores all data in a single Amazon RDS for MySQL DB instance that is fully managed by AWS. The company needs to mitigate the risk of a single point of failure.

Which solution will meet these requirements with the LEAST implementation effort?

A.Modify the RDS DB instance to use a Multi-AZ deployment. Apply the changes during the next maintenance window.

B.Migrate the current database to a new Amazon DynamoDB Multi-AZ deployment. Use AWS Database Migration Service (AWS DMS) with a heterogeneous migration strategy to migrate the current RDS DB instance to DynamoDB tables.

C.Create a new RDS DB instance in a Multi-AZ deployment. Manually restore the data from the existing RDS DB instance from the most recent snapshot.

D.Configure the DB instance in an Amazon EC2 Auto Scaling group with a minimum group size of three. Use Amazon Route 53 simple routing to distribute requests to all DB instances.

Answer: A

Explanation:

The correct answer is A: Modify the RDS DB instance to use a Multi-AZ deployment. Apply the changes during the next maintenance window.

Here's why:

The primary requirement is to mitigate the risk of a single point of failure for the RDS database with the least implementation effort. Multi-AZ deployments in RDS are designed precisely for this purpose. Enabling Multi-AZ creates a synchronous, standby replica of your database in a different Availability Zone. AWS automatically handles failover to the standby replica in case of an infrastructure failure, minimizing downtime and human intervention.

Option A is the most straightforward approach. It involves modifying the existing RDS instance settings to enable Multi-AZ. The changes can be applied during a maintenance window to minimize disruption. RDS handles the replication and failover automatically.

Option B, migrating to DynamoDB, is an entirely different database technology (NoSQL vs. relational). This requires significant application code changes and a complex data migration strategy using AWS DMS. It's not the least effort. A heterogeneous migration introduces considerable complexity compared to simply enabling Multi-AZ in RDS.

Option C, creating a new Multi-AZ RDS instance and restoring from a snapshot, is more involved than simply modifying the existing instance. Restoring from a snapshot means downtime while the data is being copied, and it does not provide automatic failover.

Option D, using EC2 Auto Scaling groups for databases, is not a typical or recommended architecture for RDS. RDS is a managed service that handles replication, patching, and backups. Deploying a database on EC2 within an Auto Scaling group requires manual configuration and management of these aspects, increasing operational overhead. Using Route 53 simple routing doesn't guarantee data consistency or automatic failover in the event of a database instance failure, which is the primary goal.

Enabling Multi-AZ directly addresses the high availability requirement with the least amount of administrative and development overhead.

Authoritative Links:

Amazon RDS Multi-AZ Deployments:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html
AWS Database Migration Service (DMS): https://aws.amazon.com/dms/

Question: 934 **Exam Heist**

A company has multiple Microsoft Windows SMB file servers and Linux NFS file servers for file sharing in an on-premises environment. As part of the company's AWS migration plan, the company wants to consolidate the file servers in the AWS Cloud.

The company needs a managed AWS storage service that supports both NFS and SMB access. The solution must be able to share between protocols. The solution must have redundancy at the Availability Zone level.

Which solution will meet these requirements?

A.Use Amazon FSx for NetApp ONTAP for storage. Configure multi-protocol access.

B.Create two Amazon EC2 instances. Use one EC2 instance for Windows SMB file server access and one EC2 instance for Linux NFS file server access.

C.Use Amazon FSx for NetApp ONTAP for SMB access. Use Amazon FSx for Lustre for NFS access.

D.Use Amazon S3 storage. Access Amazon S3 through an Amazon S3 File Gateway.

Answer: A

Explanation:

The best solution is Amazon FSx for NetApp ONTAP with multi-protocol access because it directly addresses the company's need for a managed file storage service supporting both SMB and NFS protocols with inter-protocol sharing. FSx for NetApp ONTAP is built on NetApp's ONTAP file system, which inherently supports both SMB and NFS access. Configuring multi-protocol access within FSx for NetApp ONTAP allows Windows and Linux clients to simultaneously access the same data through their respective preferred protocols. This eliminates the need for separate file servers or data replication between different services.

The requirement for Availability Zone-level redundancy is also met by FSx for NetApp ONTAP. It can be deployed in a Multi-AZ configuration, providing high availability and fault tolerance. In the event of an Availability Zone failure, FSx for NetApp ONTAP automatically fails over to the other Availability Zone, ensuring minimal downtime.

Option B is less desirable as it involves managing EC2 instances, handling patching, backups, and scaling, increasing operational overhead compared to a managed service. Option C involves two different FSx services, lacking a unified storage platform, hence no direct sharing is possible and management complexity is added. Option D utilizes S3, an object storage service, requiring a file gateway. The File Gateway could add complexity and potentially impact performance.

In summary, Amazon FSx for NetApp ONTAP with multi-protocol access offers a fully managed, highly available solution that directly meets the company's requirements for consolidating file servers, supporting both SMB and NFS protocols, and enabling inter-protocol sharing while providing the desired level of redundancy.

Further research can be conducted at:

Amazon FSx for NetApp ONTAP

Multi-protocol Access with Amazon FSx for NetApp ONTAP

Question: 935 **Exam Heist**

A software company needs to upgrade a critical web application. The application currently runs on a single Amazon EC2 instance that the company hosts in a public subnet. The EC2 instance runs a MySQL database. The application's DNS records are published in an Amazon Route 53 zone.

A solutions architect must reconfigure the application to be scalable and highly available. The solutions architect must also reduce MySQL read latency.

Which combination of solutions will meet these requirements? (Choose two.)

A.Launch a second EC2 instance in a second AWS Region. Use a Route 53 failover routing policy to redirect the traffic to the second EC2 instance.

B.Create and configure an Auto Scaling group to launch private EC2 instances in multiple Availability Zones. Add the instances to a target group behind a new Application Load Balancer.

C.Migrate the database to an Amazon Aurora MySQL cluster. Create the primary DB instance and reader DB instance in separate Availability Zones.

D.Create and configure an Auto Scaling group to launch private EC2 instances in multiple AWS Regions. Add the instances to a target group behind a new Application Load Balancer.

E.Migrate the database to an Amazon Aurora MySQL cluster with cross-Region read replicas.

Answer: BC

Explanation:

The correct answer is BC. Here's why:

B. Create and configure an Auto Scaling group to launch private EC2 instances in multiple Availability Zones. Add the instances to a target group behind a new Application Load Balancer: This addresses scalability and high availability for the web application. Auto Scaling Groups (ASGs) automatically adjust the number of EC2 instances based on demand, ensuring the application can handle traffic spikes. Spreading these instances across multiple Availability Zones (AZs) protects against failures in a single AZ. An Application Load Balancer (ALB) distributes incoming traffic evenly across the healthy EC2 instances, improving performance and availability. ALBs offer advanced features like content-based routing, further enhancing the application's capabilities. Instances within a private subnet ensures security.

Auto Scaling Groups Documentation
Application Load Balancer Documentation

C. Migrate the database to an Amazon Aurora MySQL cluster. Create the primary DB instance and reader DB instance in separate Availability Zones: This addresses scalability, high availability, and read latency for the database component. Aurora MySQL is a fully managed, MySQL-compatible relational database engine that offers improved performance and availability compared to traditional MySQL. Creating a primary instance and a read replica in different AZs provides automatic failover in case the primary instance fails. Read replicas can handle read requests, offloading the primary instance and reducing read latency, significantly improving application performance.

Amazon Aurora Documentation
Aurora Read Replicas Documentation

Why other options are incorrect:

A. Launch a second EC2 instance in a second AWS Region. Use a Route 53 failover routing policy to redirect the traffic to the second EC2 instance: While this provides disaster recovery across Regions, it doesn't

address scalability within the primary Region or database read latency. Also, failover mechanisms are slower than multi-AZ setups, leading to potential downtime during failover.
D. Create and configure an Auto Scaling group to launch private EC2 instances in multiple AWS Regions. Add the instances to a target group behind a new Application Load Balancer: Deploying the web application across multiple Regions adds significant complexity and cost. Cross-region deployments are more suitable for disaster recovery scenarios.
E. Migrate the database to an Amazon Aurora MySQL cluster with cross-Region read replicas: Cross-Region read replicas are best suited for disaster recovery or serving globally distributed users, not for addressing read latency within a single Region. Intra-region read replicas as described in Option C provide the necessary low latency read scale.

Question: 936 **Exam Heist**

A company runs thousands of AWS Lambda functions. The company needs a solution to securely store sensitive information that all the Lambda functions use. The solution must also manage the automatic rotation of the sensitive information.

Which combination of steps will meet these requirements with the LEAST operational overhead? (Choose two.)

A.Create HTTP security headers by using Lambda@Edge to retrieve and create sensitive information

B.Create a Lambda layer that retrieves sensitive information

C.Store sensitive information in AWS Secrets Manager

D.Store sensitive information in AWS Systems Manager Parameter Store

E.Create a Lambda consumer with dedicated throughput to retrieve sensitive information and create environmental variables

Answer: CD

Explanation:

The correct answer is C and D. Here's why:

AWS Secrets Manager (C): This is the ideal service for securely storing and automatically rotating secrets (like database credentials, API keys) used by Lambda functions. It's designed specifically for this purpose. https://aws.amazon.com/secrets-manager/
AWS Systems Manager Parameter Store (D): While Secrets Manager is preferable for actual secrets requiring rotation, Parameter Store (especially Secure String parameters) can also store sensitive information, albeit with a bit less automation for rotation compared to Secrets Manager. It's suitable for configuration data and smaller secrets where automated rotation isn't as critical. https://aws.amazon.com/systems-manager/features/parameter-store/

Let's analyze the incorrect options:

A. Create HTTP security headers by using Lambda@Edge to retrieve and create sensitive information: Lambda@Edge is intended for customizing content delivered by CloudFront, not for managing secrets used by internal Lambda functions. It's a misapplication of the service.
B. Create a Lambda layer that retrieves sensitive information: A Lambda layer can help with code reuse, but it doesn't inherently provide security or automatic rotation of secrets. The Lambda layer would still need to fetch the secrets from a secure location.
E. Create a Lambda consumer with dedicated throughput to retrieve sensitive information and create environmental variables: This is an unnecessarily complex and inefficient architecture. It introduces a dedicated Lambda function solely for managing secrets, adding operational overhead and potential performance bottlenecks. Environmental variables are also not the most secure way to store highly sensitive information.

Therefore, storing sensitive information in either Secrets Manager or Parameter Store provides a more manageable and secure solution for thousands of Lambda functions, with Secrets Manager being the more suitable for scenarios needing automatic rotation.

Question: 937 **Exam Heist**

A company has an internal application that runs on Amazon EC2 instances in an Auto Scaling group. The EC2 instances are compute optimized and use Amazon Elastic Block Store (Amazon EBS) volumes.

The company wants to identify cost optimizations across the EC2 instances, the Auto Scaling group, and the EBS volumes.

Which solution will meet these requirements with the MOST operational efficiency?

A.Create a new AWS Cost and Usage Report. Search the report for cost recommendations for the EC2 instances the Auto Scaling group, and the EBS volumes.

B.Create new Amazon CloudWatch billing alerts. Check the alert statuses for cost recommendations for the EC2 instances, the Auto Scaling group, and the EBS volumes.

C.Configure AWS Compute Optimizer for cost recommendations for the EC2 instances, the Auto Scaling group and the EBS volumes.

D.Configure AWS Compute Optimizer for cost recommendations for the EC2 instances. Create a new AWS Cost and Usage Report. Search the report for cost recommendations for the Auto Scaling group and the EBS volumes.

Answer: C

Explanation:

The correct answer is C because AWS Compute Optimizer is specifically designed to analyze the utilization metrics of AWS resources like EC2 instances and EBS volumes, and then provide recommendations for cost optimization and performance improvement. For Auto Scaling groups, Compute Optimizer analyzes the underlying EC2 instances to provide optimization advice. This makes it the most efficient single tool to meet the company's requirements.

Option A is less efficient because parsing through the AWS Cost and Usage Report requires more manual effort to identify cost recommendations for each resource. While the report provides detailed cost information, it does not automatically provide optimization suggestions.Option B is not suitable as CloudWatch billing alerts are used to notify about cost thresholds, not to provide specific resource optimization recommendations.Option D is also less efficient because it combines Compute Optimizer for EC2 instances with manual analysis of the Cost and Usage Report for Auto Scaling groups and EBS volumes. Compute Optimizer can cover all three resource types directly, making the combination redundant and requiring more effort.

In summary, AWS Compute Optimizer offers a consolidated and automated approach to cost optimization recommendations for EC2 instances, EBS volumes, and indirectly, Auto Scaling groups, ensuring the greatest operational efficiency.

Reference:

AWS Compute Optimizer

Question: 938 **Exam Heist**

A company is running a media store across multiple Amazon EC2 instances distributed across multiple Availability

Zones in a single VPC. The company wants a high-performing solution to share data between all the EC2 instances, and prefers to keep the data within the VPC only.

What should a solutions architect recommend?

A.Create an Amazon S3 bucket and call the service APIs from each instance's application
B.Create an Amazon S3 bucket and configure all instances to access it as a mounted volume
C.Configure an Amazon Elastic Block Store (Amazon EBS) volume and mount it across all instances
D.Configure an Amazon Elastic File System (Amazon EFS) file system and mount it across all instances

Answer: D

Explanation:

The correct answer is D, configuring an Amazon Elastic File System (Amazon EFS) file system and mounting it across all instances. Here's why:

Requirement for High-Performance Data Sharing: The company needs a high-performing solution for sharing data.
Data Within the VPC: They want to keep the data within the VPC.
Amazon EFS for Shared Storage: Amazon EFS is designed to provide scalable, elastic, cloud-native NFS file systems for use with AWS Cloud services and on-premises resources. It allows multiple EC2 instances to concurrently access a shared file system.
EFS Performance: EFS offers various performance modes, including General Purpose and Max I/O, allowing the company to optimize for their specific media store workload. It also scales automatically as data is added or removed.
EFS Integration with VPC: EFS file systems are mounted to EC2 instances via mount targets created within the VPC, ensuring data remains within the VPC.

Why other options are incorrect:

A. Amazon S3 Bucket with API Calls: While S3 is excellent for object storage, it's accessed via API calls, which are not ideal for high-performance, file-system-like data sharing. S3 is better suited for storing and retrieving individual files, not for continuous file system access.
B. Amazon S3 Bucket as a Mounted Volume: S3 is not designed to be mounted as a traditional file system volume. While solutions like S3FS exist, they often introduce performance overhead and aren't meant for high-performance scenarios within a VPC where a native shared file system solution is desired.
C. Amazon EBS Volume Mounted Across Instances: EBS volumes are block storage designed to be attached to a single EC2 instance at a time. They are not designed for shared access by multiple instances simultaneously. Attempting to share an EBS volume would lead to data corruption. There's a multi-attach option for EBS but it has many restrictions and is not designed for generalized shared file storage like EFS.

In summary, EFS provides the necessary shared file system capability with high performance, VPC integration, and scalability needed for the media store application.

Authoritative Links:

Amazon EFS: https://aws.amazon.com/efs/
Amazon EFS Documentation: https://docs.aws.amazon.com/efs/latest/ug/whatisefs.html

Question: 939 **Exam Heist**

A company uses an Amazon RDS for MySQL instance. To prepare for end-of-year processing, the company added a read replica to accommodate extra read-only queries from the company's reporting tool. The read replica CPU usage was 60% and the primary instance CPU usage was 60%.

After end-of-year activities are complete, the read replica has a constant 25% CPU usage. The primary instance still has a constant 60% CPU usage. The company wants to rightsize the database and still provide enough performance for future growth.

Which solution will meet these requirements?

A.Delete the read replica Do not make changes to the primary instance

B.Resize the read replica to a smaller instance size Do not make changes to the primary instance

C.Resize the read replica to a larger instance size Resize the primary instance to a smaller instance size

D.Delete the read replica Resize the primary instance to a larger instance

Answer: B

Explanation:

Here's a detailed justification for why option B is the most appropriate solution, along with supporting concepts and links:

The scenario highlights a common situation: a temporary performance need (end-of-year processing) fulfilled by adding a read replica. Now that the temporary need has passed, the read replica is underutilized, consuming resources without providing commensurate value. The primary instance, however, continues to operate at a moderate 60% CPU utilization.

Option A, deleting the read replica entirely, would eliminate the ability to offload any read traffic from the primary instance. While it reduces cost, it also reduces the overall read scalability and could potentially increase the load on the primary instance if reporting queries increase in the future.

Option B, resizing the read replica to a smaller instance size, strikes a balance. Since the read replica's CPU usage is consistently low (25%), reducing its instance size will lower costs without significantly impacting performance. The primary instance remains unchanged because it's already at a manageable 60% CPU utilization. This approach allows the company to retain a read replica for potential future read-heavy workloads or reporting needs, providing flexibility and headroom without unnecessary resource consumption. Resizing is a non-destructive operation and can be easily reversed if performance degrades after the change.

Option C is incorrect. The primary instance is already at a manageable 60% CPU utilization, so there's no need to resize it to a smaller instance size. Resizing the read replica to a larger instance size is also unnecessary since it's underutilized.

Option D, deleting the read replica and resizing the primary instance to a larger size, is also incorrect. Deleting the read replica reduces read scalability, and increasing the primary instance size when it's at 60% utilization is not the optimal cost-effective solution.

Therefore, the most efficient and cost-effective solution is to resize the read replica to a smaller instance size while leaving the primary instance unchanged. This aligns with the principle of rightsizing cloud resources to match actual workload requirements.

Key Concepts:

Read Replicas: Replicas of a database instance that serve read-only traffic, offloading the primary instance and improving read scalability.
(https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ReadRepl.html)
Rightsizing: Optimizing cloud resource allocation to match actual workload needs, minimizing costs and maximizing efficiency.
Cost Optimization: Selecting the most cost-effective solution while meeting performance requirements.
CPU Utilization: A metric that reflects the amount of processing power being used by an instance.

Authoritative Links:

Amazon RDS Read Replicas: https://aws.amazon.com/rds/features/read-replicas/
Amazon RDS Pricing: https://aws.amazon.com/rds/pricing/

Question: 940 **Exam Heist**

A company is migrating its databases to Amazon RDS for PostgreSQL. The company is migrating its applications to Amazon EC2 instances. The company wants to optimize costs for long-running workloads.

Which solution will meet this requirement MOST cost-effectively?

A.Use On-Demand Instances for the Amazon RDS for PostgreSQL workloads. Purchase a 1 year Compute Savings Plan with the No Upfront option for the EC2 instances.

B.Purchase Reserved Instances for a 1 year term with the No Upfront option for the Amazon RDS for PostgreSQL workloads. Purchase a 1 year EC2 Instance Savings Plan with the No Upfront option for the EC2 instances.

C.Purchase Reserved Instances for a 1 year term with the Partial Upfront option for the Amazon RDS for PostgreSQL workloads. Purchase a 1 year EC2 Instance Savings Plan with the Partial Upfront option for the EC2 instances.

D.Purchase Reserved Instances for a 3 year term with the All Upfront option for the Amazon RDS for PostgreSQL workloads. Purchase a 3 year EC2 Instance Savings Plan with the All Upfront option for the EC2 instances.

Answer: D

Explanation:

The correct answer is **D** because it provides the most cost-effective solution for long-running workloads on both Amazon RDS for PostgreSQL and EC2 instances. Here's a detailed breakdown:

Reserved Instances (RIs) and Savings Plans are Cost Optimization Tools: Both RIs and Savings Plans offer significant discounts compared to On-Demand pricing for consistent usage. They are designed for workloads with predictable, long-term resource requirements.

Longer Term = Greater Savings: A longer commitment (3 years vs. 1 year) typically yields a larger discount. AWS rewards customers who make longer-term reservations with greater cost savings.

All Upfront Option for Maximum Discount: Paying upfront (All Upfront) provides the largest discount compared to No Upfront or Partial Upfront options. While it requires a larger initial investment, it results in the lowest overall cost over the commitment period.

EC2 Instance Savings Plan: EC2 Instance Savings Plans provide savings on compute usage, regardless of instance family, size, or AZ, as long as the usage matches the plan's commitment. This offers flexibility.

Reserved Instances for RDS: RDS Reserved Instances work in a similar manner, offering discounted pricing for reserved database capacity.

Why other options are less optimal:

Option A: On-Demand Instances are the most expensive option for long-running workloads. Savings Plans are better, but only cover EC2, not RDS.
Option B: 1-year term offers less savings compared to 3-year.
Option C: Partial Upfront offers less savings compared to All Upfront.

Therefore, **Option D**, which combines a 3-year term with the All Upfront payment option for both Reserved Instances (for RDS) and Savings Plans (for EC2), maximizes cost savings for the company's long-running

database and application workloads.

Authoritative Links:

AWS Reserved Instances
AWS Savings Plans
AWS RDS Pricing
AWS EC2 Pricing

Question: 941 **Exam Heist**

A company is using an Amazon Elastic Kubernetes Service (Amazon EKS) cluster. The company must ensure that Kubernetes service accounts in the EKS cluster have secure and granular access to specific AWS resources by using IAM roles for service accounts (IRSA).

Which combination of solutions will meet these requirements? (Choose two.)

A.Create an IAM policy that defines the required permissions Attach the policy directly to the IAM role of the EKS nodes.

B.Implement network policies within the EKS cluster to prevent Kubernetes service accounts from accessing specific AWS services.

C.Modify the EKS cluster's IAM role to include permissions for each Kubernetes service account. Ensure a one-to-one mapping between IAM roles and Kubernetes roles.

D.Define an IAM role that includes the necessary permissions. Annotate the Kubernetes service accounts with the Amazon ResourceName (ARN) of the IAM role.

E.Set up a trust relationship between the IAM roles for the service accounts and an OpenID Connect (OIDC) identity provider.

Answer: DE

Explanation:

Here's a detailed justification for why options D and E are the correct solutions for implementing IAM roles for service accounts (IRSA) in an Amazon EKS cluster to grant secure and granular access to AWS resources, along with explanations of why the other options are incorrect:

Why D and E are correct:

D. Define an IAM role that includes the necessary permissions. Annotate the Kubernetes service accounts with the Amazon Resource Name (ARN) of the IAM role. This is a core component of IRSA. You create an IAM role that specifies precisely which AWS resources a Kubernetes service account is allowed to access. The key here is to grant least privilege – only the necessary permissions. Annotating the service account with the IAM role's ARN informs Kubernetes that pods using this service account should assume that role.
E. Set up a trust relationship between the IAM roles for the service accounts and an OpenID Connect (OIDC) identity provider. This is another crucial aspect of IRSA. EKS integrates with AWS IAM via an OIDC provider. The IAM role created in option D needs a trust policy. This trust policy specifies that the IAM role can be assumed by entities authenticated by the EKS cluster's OIDC provider. This trust policy verifies that only workloads from the correct EKS cluster and, specifically, the correct Kubernetes service account, can assume the role. This is how AWS verifies the identity of the Kubernetes service account making the AWS API calls.

In summary: Options D and E together establish the IRSA mechanism. An IAM role with specific permissions is created and trusted by the EKS cluster's OIDC provider. By annotating the service account with the IAM role's ARN, pods running under the service account can securely assume that IAM role and access allowed AWS resources.

Why the other options are incorrect:

A. Create an IAM policy that defines the required permissions. Attach the policy directly to the IAM role of the EKS nodes. Attaching permissions directly to the EKS node's IAM role is overly permissive and does not provide granular access control. It means any pod running on the node (even those not requiring access to those AWS resources) would inherit those permissions, violating the principle of least privilege. Node roles are intended for EKS internal operations, not for general application access to AWS services.
B. Implement network policies within the EKS cluster to prevent Kubernetes service accounts from accessing specific AWS services. Network policies control network traffic within the Kubernetes cluster. They do not directly control access to AWS services. While you can restrict egress traffic from pods, this is not the primary mechanism for controlling AWS API access via IAM roles.
C. Modify the EKS cluster's IAM role to include permissions for each Kubernetes service account. Ensure a one-to-one mapping between IAM roles and Kubernetes roles. Directly modifying the EKS cluster's IAM role for individual service accounts is not the correct approach. The cluster role is for EKS management plane operations, not for workloads running within the cluster. Moreover, attempting a one-to-one mapping between Kubernetes roles (RBAC) and AWS IAM roles isn't how IRSA works. IRSA maps Kubernetes service accounts to IAM roles, not RBAC roles.

Authoritative Links:

IAM roles for service accounts: https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html
Configuring a cluster to use IAM roles for service accounts:
https://docs.aws.amazon.com/eks/latest/userguide/enable-iam-roles-for-service-accounts.html

Question: 942 **Exam Heist**

A company regularly uploads confidential data to Amazon S3 buckets for analysis.

The company's security policies mandate that the objects must be encrypted at rest. The company must automatically rotate the encryption key every year. The company must be able to track key rotation by using AWS CloudTrail. The company also must minimize costs for the encryption key.

Which solution will meet these requirements?

A.Use server-side encryption with customer-provided keys (SSE-C)
B.Use server-side encryption with Amazon S3 managed keys (SSE-S3)
C.Use server-side encryption with AWS KMS keys (SSE-KMS)
D.Use server-side encryption with customer managed AWS KMS keys

Answer: D

Explanation:

The correct answer is **D. Use server-side encryption with customer managed AWS KMS keys (SSE-KMS)**. Here's why:

Encryption at Rest: All options provide encryption at rest, but the key is how the keys are managed.
Automatic Key Rotation: Customer managed KMS keys support automatic key rotation on a schedule that you configure, such as annually, meeting the yearly rotation requirement.
https://docs.aws.amazon.com/kms/latest/developerguide/rotate-keys.html
CloudTrail Tracking: AWS KMS integrates with AWS CloudTrail, allowing you to track the usage and rotation of your keys. This satisfies the auditing requirement.
https://docs.aws.amazon.com/kms/latest/developerguide/logging-using-cloudtrail.html
Cost Optimization: While KMS keys have a cost associated with them, customer managed KMS keys offer the flexibility to optimize costs by managing usage and access control effectively.

SSE-C Incompatibility: SSE-C requires you to manage the encryption keys, which contradicts the requirement for automatic key rotation.
SSE-S3 Limitations: SSE-S3 uses S3-managed keys, offering simplicity but no control over key rotation or CloudTrail logging of key usage.
SSE-KMS (AWS Managed Keys) Limitations: While SSE-KMS provides encryption and integration with CloudTrail, AWS manages the key rotation schedule, limiting the company's control.

Therefore, customer managed KMS keys are the ideal choice because they fulfill all the requirements: encryption at rest, automatic yearly key rotation, CloudTrail tracking, and cost optimization through usage control. They give the company the necessary control and visibility over its encryption keys to meet its security policies.

Question: 943 **Exam Heist**

A company has migrated several applications to AWS in the past 3 months. The company wants to know the breakdown of costs for each of these applications. The company wants to receive a regular report that includes this information.

Which solution will meet these requirements MOST cost-effectively?

A.Use AWS Budgets to download data for the past 3 months into a .csv file. Look up the desired information.

B.Load AWS Cost and Usage Reports into an Amazon RDS DB instance. Run SQL queries to get the desired information.

C.Tag all the AWS resources with a key for cost and a value of the application's name. Activate cost allocation tags. Use Cost Explorerto get the desired information.

D.Tag all the AWS resources with a key for cost and a value of the application's name. Use the AWS Billing and Cost Management console todownload bills for the past 3 months. Look up the desired information.

Answer: C

Explanation:

The most cost-effective solution is C, tagging resources and using Cost Explorer. Here's why:

Resource Tagging: Applying tags (key-value pairs) to AWS resources is a fundamental practice for organization and cost management. Tags like cost: application-name allow you to categorize and track expenses associated with each application. https://aws.amazon.com/aws-cost-management/aws-resource-tagging/

Cost Allocation Tags: Activating these tags ensures that AWS considers them when generating cost reports and performing cost analysis. This means your tagged resources will be grouped and summarized accordingly. https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/cost-alloc-tags.html

AWS Cost Explorer: Cost Explorer is a free tool within the AWS Billing and Cost Management console. It provides a user-friendly interface to visualize, understand, and manage AWS costs over time. You can filter and group costs by the tags you've defined, making it easy to see the breakdown of costs for each application. Cost Explorer can also generate reports and forecasts. https://aws.amazon.com/aws-cost-management/aws-cost-explorer/

Option A (AWS Budgets and CSV) is less efficient. AWS Budgets are primarily for setting spending limits and receiving alerts, not for detailed cost breakdown analysis of past expenses. Downloading CSV data and manually analyzing it is time-consuming and prone to errors.

Option B (Cost and Usage Reports to RDS) is more complex and expensive than necessary. While CUR is powerful for detailed analysis, loading it into an RDS instance and running SQL queries is overkill for simply

understanding the cost breakdown of different applications. This option incurs RDS costs and requires database management expertise. Also, it is difficult to get insights into cost allocation without resource tags in place.

Option D (Tagging and Downloading Bills) requires manual searching within potentially very large bill files. It is not as easy and interactive as using Cost Explorer and reporting.

In summary, tagging resources, enabling cost allocation tags, and using Cost Explorer provides the most cost-effective and user-friendly way to get a regular cost breakdown report for each application.

Question: 944 **Exam Heist**

An ecommerce company is preparing to deploy a web application on AWS to ensure continuous service for customers. The architecture includes a web application that the company hosts on Amazon EC2 instances, a relational database in Amazon RDS, and static assets that the company stores in Amazon S3.

The company wants to design a robust and resilient architecture for the application.

Which solution will meet these requirements?

A.Deploy Amazon EC2 instances in a single Availability Zone. Deploy an RDS DB instance in the same Availability Zone. Use Amazon S3 with versioning enabled to store static assets.

B.Deploy Amazon EC2 instances in an Auto Scaling group across multiple Availability Zones. Deploy a Multi-AZ RDS DB instance. Use Amazon CloudFront to distribute static assets.

C.Deploy Amazon EC2 instances in a single Availability Zone. Deploy an RDS DB instance in a second Availability Zone for cross-AZ redundancy. Serve static assets directly from the EC2 instances.

D.Use AWS Lambda functions to serve the web application. Use Amazon Aurora Serverless v2 for the database. Store static assets in Amazon Elastic File System (Amazon EFS) One Zone-Infrequent Access (One Zone-IA).

Answer: B

Explanation:

The optimal solution for a highly available and resilient e-commerce web application on AWS, as described in the question, is option B. This solution incorporates several key AWS services to ensure continuous service.

Option B suggests deploying Amazon EC2 instances within an Auto Scaling group spanning multiple Availability Zones (AZs). This is crucial for fault tolerance; if one AZ experiences an issue, the Auto Scaling group automatically launches instances in other healthy AZs, maintaining application availability. Distributing instances across multiple AZs eliminates a single point of failure for the web application tier.

Similarly, deploying a Multi-AZ RDS DB instance is vital for database resilience. In a Multi-AZ configuration, AWS automatically provisions and maintains a synchronous standby replica of the database in a different AZ. If the primary DB instance fails, RDS automatically fails over to the standby, minimizing downtime.

Finally, using Amazon CloudFront to distribute static assets enhances performance and availability. CloudFront caches static content at edge locations worldwide, reducing latency for users and offloading traffic from the EC2 instances. The edge locations also provide inherent redundancy for serving static content.

Option A fails because it relies on a single Availability Zone for both EC2 and RDS, which introduces a single point of failure. Option C has the same issue with the single AZ EC2 instance. Storing assets directly on EC2 instances also creates a scalability bottleneck and doesn't leverage content delivery networks. Option D, while utilizing serverless technologies, might not be the most cost-effective or performant solution for a general web application, and One Zone EFS offers reduced availability compared to multi-AZ options. Furthermore, while Lambda and Aurora Serverless v2 could work, the scenario specified explicitly mentions

hosting the application on EC2 instances.

Therefore, by leveraging Auto Scaling, Multi-AZ RDS, and CloudFront, Option B establishes a robust and resilient architecture that addresses the e-commerce company's requirements for continuous service and high availability.

Authoritative links:

Auto Scaling: https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-benefits.html
Multi-AZ RDS: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html
Amazon CloudFront: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Introduction.html

Question: 945 **Exam Heist**

An ecommerce company runs several internal applications in multiple AWS accounts. The company uses AWS Organizations to manage its AWS accounts.

A security appliance in the company's networking account must inspect interactions between applications across AWS accounts.

Which solution will meet these requirements?

A.Deploy a Network Load Balancer (NLB) in the networking account to send traffic to the security appliance. Configure the application accounts to send traffic to the NLB by using an interface VPC endpoint in the application accounts.

B.Deploy an Application Load Balancer (ALB) in the application accounts to send traffic directly to the security appliance.

C.Deploy a Gateway Load Balancer (GWLB) in the networking account to send traffic to the security appliance. Configure the application accounts to send traffic to the GWLB by using an interface GWLB endpoint in the application accounts.

D.Deploy an interface VPC endpoint in the application accounts to send traffic directly to the security appliance.

Answer: C

Explanation:

The correct answer is C because it leverages AWS Gateway Load Balancer (GWLB), specifically designed for inspecting, filtering, and steering traffic to virtual appliances. The scenario requires inspecting inter-application traffic across multiple AWS accounts, and GWLB is the optimal solution for this purpose.

Here's a breakdown:

GWLB's Purpose: GWLB simplifies the deployment, scaling, and management of virtual appliances like firewalls, intrusion detection systems (IDS), and deep packet inspection (DPI) tools. It integrates seamlessly with VPCs and other AWS services.

Cross-Account Traffic Inspection: The networking account houses the security appliance and GWLB. Application accounts send traffic to the GWLB endpoint, ensuring all inter-application communication is routed through the security appliance for inspection.

GWLB Endpoint: The GWLB endpoint in each application account provides a private, reliable connection to the GWLB in the networking account. Traffic destined for other applications flows through this endpoint and gets inspected.

Why other options are incorrect:

A (NLB with interface VPC endpoint): Network Load Balancers are designed for TCP, UDP, and TLS traffic. While NLBs can direct traffic, they are not purpose-built for appliance insertion and inspection.
B (ALB): Application Load Balancers are HTTP/HTTPS load balancers. They are not suitable for inspecting general network traffic across multiple accounts.
D (Interface VPC endpoint directly to the appliance): This approach requires managing multiple connections and scaling the security appliance independently in each account, making it a less centralized and scalable solution.

GWLB provides a centralized and managed solution for inspecting network traffic, offering scalability, high availability, and simplified management of security appliances. Its design allows for easy insertion and inspection of traffic between VPCs and AWS accounts.

Reference Link:

AWS Gateway Load Balancer

Question: 946 **Exam Heist**

A company runs its production workload on an Amazon Aurora MySQL DB cluster that includes six Aurora Replicas. The company wants near-real-time reporting queries from one of its departments to be automatically distributed across three of the Aurora Replicas. Those three replicas have a different compute and memory specification from the rest of the DB cluster.

Which solution meets these requirements?

A.Create and use a custom endpoint for the workload
B.Create a three-node cluster clone and use the reader endpoint
C.Use any of the instance endpoints for the selected three nodes
D.Use the reader endpoint to automatically distribute the read-only workload

Answer: A

Explanation:

The correct answer is **A. Create and use a custom endpoint for the workload.** Here's why:

Custom endpoints in Aurora allow you to direct specific read workloads to a defined subset of Aurora Replicas within your DB cluster. This directly addresses the requirement to send the near-real-time reporting queries only to the three specifically configured replicas. By creating a custom endpoint, the company can specify the three desired Aurora Replicas as members of that endpoint. When the reporting department connects to the custom endpoint, Aurora ensures the queries are routed to those designated replicas.

Option B, creating a three-node cluster clone, is overkill. Cloning the entire cluster just for reporting adds unnecessary cost and management overhead when a custom endpoint can achieve the same result with much less complexity. Furthermore, data replication delays between the primary cluster and the clone would also be a concern.

Option C, using instance endpoints, doesn't provide automatic load balancing or distribution of the workload. The reporting department would need to manually manage which instance endpoint to connect to, which is not ideal for an automatically distributed workload. There's also no guarantee queries won't be directed to the primary instance, potentially impacting production performance.

Option D, using the reader endpoint, distributes read traffic across all available Aurora Replicas, not just the specific three replicas with different compute and memory specifications. This fails to meet the requirement of targeting a particular subset of replicas.

Therefore, custom endpoints provide the most efficient and targeted way to direct the reporting workload to the desired Aurora Replicas, enabling near-real-time reporting without impacting the primary production workload or other replicas.

Further Research:

Aurora Custom Endpoints: - AWS Documentation on Custom Endpoints.

Question: 947 **Exam Heist**

A company runs a Node js function on a server in its on-premises data center. The data center stores data in a PostgreSQL database. The company stores the credentials in a connection string in an environment variable on the server. The company wants to migrate its application to AWS and to replace the Node.js application server with AWS Lambda. The company also wants to migrate to Amazon RDS for PostgreSQL and to ensure that the database credentials are securely managed.

Which solution will meet these requirements with the LEAST operational overhead?

A.Store the database credentials as a parameter in AWS Systems Manager Parameter Store Configure Parameter Store to automatically rotate the secrets every 30 days. Update the Lambda function to retrieve the credentials from the parameter.

B.Store the database credentials as a secret in AWS Secrets Manager. Configure Secrets Manager to automatically rotate the credentials every 30 days. Update the Lambda function to retrieve the credentials from the secret.

C.Store the database credentials as an encrypted Lambda environment variable. Write a custom Lambda function to rotate the credentials. Schedule the Lambda function to run every 30 days.

D.Store the database credentials as a key in AWS Key Management Service (AWS KMS). Configure automatic rotation for the key. Update the Lambda function to retneve the credentials from the KMS key.

Answer: B

Explanation:

The best solution is **B. Store the database credentials as a secret in AWS Secrets Manager. Configure Secrets Manager to automatically rotate the credentials every 30 days. Update the Lambda function to retrieve the credentials from the secret.**

Here's why:

Secrets Manager is designed for secret management: AWS Secrets Manager is specifically built to store and manage sensitive information like database credentials, API keys, and other secrets. It provides a secure and centralized location for these secrets. https://aws.amazon.com/secrets-manager/
Automatic Rotation: Secrets Manager supports automatic rotation of credentials, which greatly reduces the operational overhead of manually rotating them. This is crucial for security as regularly rotating credentials limits the potential damage from compromised credentials.
Integration with Lambda: Lambda functions can easily retrieve secrets from Secrets Manager using the AWS SDK or the Secrets Manager Lambda extension. This provides a secure and straightforward way to access credentials within the Lambda function.
Least Operational Overhead: Using Secrets Manager requires minimal configuration and integration effort compared to other options, reducing operational overhead. You don't have to write custom rotation logic or manage encryption/decryption manually.

Here's why the other options are less suitable:

A. Systems Manager Parameter Store: While Parameter Store can store sensitive data, it is generally better suited for configuration data rather than secrets. Secrets Manager provides more robust secret management

capabilities, including automatic rotation. Moreover, rotating secrets using Parameter Store is less straightforward.
C. Lambda Environment Variables: Storing credentials in Lambda environment variables, even if encrypted, is not a best practice. It's difficult to rotate the credentials, and the environment variables are less secure than a dedicated secret management service. Creating and scheduling a Lambda function for rotation would require significant overhead.
D. AWS KMS: AWS KMS is designed for managing encryption keys, not for storing secrets directly. While you can encrypt secrets with KMS, managing the secret data and its rotation would require significantly more custom code and operational effort compared to Secrets Manager. KMS would encrypt the credentials but not handle the key rotation for the credentials themselves; it would rotate the key used to encrypt them.

In summary, Secrets Manager is the optimal solution due to its dedicated secret management capabilities, automatic rotation feature, seamless integration with Lambda, and minimal operational overhead, aligning perfectly with the company's requirements for secure credential management and a serverless architecture.

Question: 948 **Exam Heist**

A company wants to replicate existing and ongoing data changes from an on-premises Oracle database to Amazon RDS for Oracle. The amount of data to replicate varies throughout each day. The company wants to use AWS Database Migration Service (AWS DMS) for data replication. The solution must allocate only the capacity that the replication instance requires.

Which solution will meet these requirements?

A.Configure the AWS DMS replication instance with a Multi-AZ deployment to provision instances across multiple Availability Zones.

B.Create an AWS DMS Serverless replication task to analyze and replicate the data while provisioning the required capacity.

C.Use Amazon EC2 Auto Scaling to scale the size of the AWS DMS replication instance up or down based on the amount of data toreplicate.

D.Provision AWS DMS replication capacity by using Amazon Elastic Container Service (Amazon ECS) with an AWS Fargate launch type to analyze and replicate the data while provisioning the required capacity.

Answer: B

Explanation:

The correct answer is **B. Create an AWS DMS Serverless replication task to analyze and replicate the data while provisioning the required capacity.**

Here's why:

AWS DMS Serverless: AWS DMS Serverless is designed to automatically scale the replication capacity based on the workload. It eliminates the need to manually provision and manage the replication instance, making it ideal for scenarios with varying data replication needs. This aligns perfectly with the company's requirement to allocate only the capacity needed at any given time.
Capacity Provisioning: DMS Serverless automatically provisions the necessary compute and memory resources as the data replication workload changes, ensuring efficient resource utilization and cost optimization. It handles the underlying infrastructure, allowing the company to focus on the data migration itself.
Alternatives Analysis:

A (Multi-AZ): Multi-AZ deployments provide high availability and fault tolerance but do not address the dynamic capacity requirements. It simply replicates the same instance across AZs, which may be over-provisioned during periods of low data replication volume.

C (EC2 Auto Scaling): While EC2 Auto Scaling can scale EC2 instances, it requires significant configuration and management overhead to integrate with DMS. It also involves a delay in scaling up/down, potentially leading to performance bottlenecks or underutilization.
D (ECS with Fargate): Using ECS with Fargate for DMS replication is not a supported or recommended approach. DMS is designed to run on its own replication instance infrastructure, and using ECS would add unnecessary complexity and overhead.

DMS Serverless directly addresses the need for dynamic capacity allocation, simplifying the process and optimizing resource utilization.

Supporting Documentation:

AWS Database Migration Service (DMS) Serverless: https://aws.amazon.com/dms/serverless/
AWS DMS Documentation: https://docs.aws.amazon.com/dms/index.html

Question: 949 **Exam Heist**

A company has a multi-tier web application. The application's internal service components are deployed on Amazon EC2 instances. The internal service components need to access third-party software as a service (SaaS) APIs that are hosted on AWS.

The company needs to provide secure and private connectivity from the application's internal services to the third-party SaaS application. The company needs to ensure that there is minimal public internet exposure.

Which solution will meet these requirements?

A.Implement an AWS Site-to-Site VPN to establish a secure connection with the third-party SaaS provider.

B.Deploy AWS Transit Gateway to manage and route traffic between the application's VPC and the third-party SaaS provider.

C.Configure AWS PrivateLink to allow only outbound traffic from the VPC without enabling the third-party SaaS provider to establish.

D.Use AWS PrivateLink to create a private connection between the application's VPC and the third-party SaaS provider.

Answer: D

Explanation:

The correct answer is D: Use AWS PrivateLink to create a private connection between the application's VPC and the third-party SaaS provider.

Here's a detailed justification:

AWS PrivateLink provides private connectivity between VPCs, AWS services, and on-premises networks, without exposing your traffic to the public internet. This perfectly aligns with the requirement of minimal public internet exposure for the internal service components accessing the third-party SaaS APIs. With PrivateLink, traffic between the application's VPC and the SaaS provider remains within the AWS network.

Option A, AWS Site-to-Site VPN, is more suitable for connecting an on-premises network to AWS. While it provides secure connectivity, it's not the best solution for connecting to a SaaS provider already hosted on AWS.

Option B, AWS Transit Gateway, is excellent for managing connectivity between multiple VPCs and on-premises networks, but it doesn't inherently provide the private connection characteristic of PrivateLink. You could use Transit Gateway, but it would likely be combined with other solutions like VPC peering which introduces more complexity than PrivateLink directly. Also, it is not intended to be used as a direct replacement of AWS PrivateLink.

Option C suggests configuring PrivateLink to allow only outbound traffic. AWS PrivateLink, by design, establishes a private connection in both directions. While you control security groups to limit traffic, it inherently provides bidirectional connectivity for requests and responses. Hence, this is not the primary use case of PrivateLink

Therefore, using AWS PrivateLink is the most secure and efficient method, establishing a private connection directly between the company's VPC and the third-party SaaS provider's service, keeping traffic within the AWS network and minimizing public internet exposure.

Relevant Links:

AWS PrivateLink: https://aws.amazon.com/privatelink/
AWS Site-to-Site VPN: https://aws.amazon.com/vpn/
AWS Transit Gateway: https://aws.amazon.com/transit-gateway/

Question: 950 **Exam Heist**

A solutions architect needs to connect a company's corporate network to its VPC to allow on-premises access to its AWS resources. The solution must provide encryption of all traffic between the corporate network and the VPC at the network layer and the session layer. The solution also must provide security controls to prevent unrestricted access between AWS and the on-premises systems.

Which solution meets these requirements?

A.Configure AWS Direct Connect to connect to the VPC. Configure the VPC route tables to allow and deny traffic between AWS and on premises as required.

B.Create an IAM policy to allow access to the AWS Management Console only from a defined set of corporate IP addresses. Restrict user access based on job responsibility by using an IAM policy and roles.

C.Configure AWS Site-to-Site VPN to connect to the VPConfigure route table entries to direct traffic from on premises to the VPConfigure instance security groups and network ACLs to allow only required traffic from on premises.

D.Configure AWS Transit Gateway to connect to the VPC. Configure route table entries to direct traffic from on premises to the VPC. Configure instance security groups and network ACLs to allow only required traffic from on premises.

Answer: C

Explanation:

Let's break down why option C is the best solution.

Requirements Breakdown:

Encryption: Network layer (IPsec) and session layer (SSL/TLS).
Connectivity: Securely connect the corporate network to the VPC.
Security Controls: Restrict traffic between on-premises and AWS.

Why Option C (AWS Site-to-Site VPN) is Correct:

AWS Site-to-Site VPN provides a secure, encrypted tunnel between your on-premises network and your AWS VPC. It uses IPsec encryption at the network layer, fulfilling the first part of the encryption requirement. By default, VPN connections ensure all data in transit is protected using encryption.

Route table entries, alongside network ACLs and Security groups, are the core of managing ingress and egress traffic in both VPC and subnets. Route tables control traffic flow based on destination IPs while NACLs act as a firewall for associated subnets. Security groups add another layer of defense by acting as a virtual firewall for individual instances in a subnet.

Configuring security groups and Network ACLs allows for granular control over the traffic permitted between the on-premises network and the VPC. This addresses the requirement for restricting unrestricted access. They can be set to allow only specific ports and protocols necessary for communication, minimizing the attack surface.

Why Other Options Are Less Suitable:

A (Direct Connect): Direct Connect provides a dedicated network connection, but it doesn't inherently provide encryption. Encryption must be added via other methods. Furthermore, while route tables handle basic routing, they don't provide the granular access control of security groups and network ACLs.
B (IAM Policies): IAM policies control access to AWS resources based on identity. This approach secures the AWS Management Console but does not address the connectivity or traffic encryption requirements for on-premises resources communicating directly with AWS resources in the VPC.
D (Transit Gateway): AWS Transit Gateway acts as a network transit hub. It provides a central point of connectivity for multiple VPCs and on-premises networks. While it can be used for connectivity, it does not automatically provide network layer encryption for all communication with an on-premise network. The other options such as route tables, NACLs and Security groups are suitable. However, the primary issue is that TGW is more complex and usually introduced when multiple VPCs and networks need to be connected. It may be more costly and complex than a simple site-to-site VPN for a single connection.

Authoritative Links:

AWS Site-to-Site VPN: https://aws.amazon.com/vpn/
Security Groups: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html
Network ACLs: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html

Question: 951 **Exam Heist**

A company has a custom application with embedded credentials that retrieves information from a database in an Amazon RDS for MySQL DB cluster. The company needs to make the application more secure with minimal programming effort. The company has created credentials on the RDS for MySQL database for the application user.

Which solution will meet these requirements?

A.Store the credentials in AWS Key Management Service (AWS KMS). Create keys in AWS KMS. Configure the application to load the database credentials from AWS KMS. Enable automatic key rotation

B.Store the credentials in encrypted local storage. Configure the application to load the database credentials from the local storage. Set up a credentials rotation schedule by creating a cron job.

C.Store the credentials in AWS Secrets Manager. Configure the application to load the database credentials from Secrets Manager. Set up a credentials rotation schedule by creating an AWS Lambda function for Secrets Manager.

D.Store the credentials in AWS Systems Manager Parameter Store. Configure the application to load the database credentials from Parameter Store. Set up a credentials rotation schedule in the RDS for MySQL database by using Parameter Store.

Answer: C

Explanation:

The correct answer is C because it leverages AWS Secrets Manager, which is specifically designed for managing database credentials and other secrets securely. Secrets Manager provides automatic rotation capabilities, reducing the operational overhead and minimizing the risk of using stale or compromised credentials.

Here's a detailed justification:

AWS Secrets Manager is purpose-built for secrets management: It centralizes the storage and management of secrets like database credentials, API keys, and OAuth tokens. This provides a secure and auditable solution compared to storing secrets in code or local storage.

Automatic Rotation: Secrets Manager enables automatic rotation of database credentials. This is a crucial security best practice because it significantly reduces the window of opportunity for attackers to exploit compromised credentials. With automatic rotation, new credentials are automatically generated and updated in both the application and the database.

Lambda Integration: Secrets Manager seamlessly integrates with AWS Lambda to handle the actual credential rotation process. Lambda functions can connect to the database, create new credentials, and update the secret in Secrets Manager.

Minimal Programming Effort: Using Secrets Manager reduces the programming effort required. The application only needs to retrieve the credentials from Secrets Manager. The complexities of key management and rotation are handled by the service and the Lambda function.

Security Best Practices: By storing credentials in a dedicated secrets management service, you adhere to security best practices by ensuring the application does not store them in plain text in config files or code. Secrets are encrypted in transit and at rest.

Why other options are incorrect:

A. AWS KMS: AWS KMS is primarily for encryption key management, not secrets management. While you could encrypt the credentials, KMS doesn't offer built-in rotation capabilities for database credentials, requiring significant custom development.

B. Encrypted local storage: Storing credentials in encrypted local storage is better than plain text, but it is not a secure solution for enterprise applications. It is not centralized, difficult to manage, and lacks built-in rotation capabilities. Cron job based rotation is also prone to errors and difficult to manage at scale.

D. Systems Manager Parameter Store: While Parameter Store can store secrets, it lacks the purpose-built features of Secrets Manager, particularly automatic rotation, and database integration making it a more complex solution.

In conclusion, AWS Secrets Manager offers the best solution for secure, automated, and easily managed database credentials, aligning perfectly with the requirements of minimal programming effort and improved security.

Supporting links:

AWS Secrets Manager
Rotating AWS Secrets Manager secrets

Question: 952 **Exam Heist**

A company wants to move its application to a serverless solution. The serverless solution needs to analyze existing data and new data by using SQL. The company stores the data in an Amazon S3 bucket. The data must be encrypted at rest and replicated to a different AWS Region.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create a new S3 bucket that uses server-side encryption with AWS KMS multi-Region keys (SSE-KMS). Configure Cross-Region Replication (CRR). Load the data into the new S3 bucket. Use Amazon Athena to query the data.

B.Create a new S3 bucket that uses server-side encryption with Amazon S3 managed keys (SSE-S3). Configure Cross-Region Replication (CRR). Load the data into the new S3 bucket. Use Amazon RDS to query the data.

C.Configure Cross-Region Replication (CRR) on the existing S3 bucket. Use server-side encryption with Amazon S3 managed keys (SSE-S3). Use Amazon Athena to query the data.

D.Configure S3 Cross-Region Replication (CRR) on the existing S3 bucket. Use server-side encryption with AWS KMS multi-Region keys (SSE-KMS). Use Amazon RDS to query the data.

Answer: A

Explanation:

Here's a detailed justification for why option A is the best solution, focusing on meeting requirements with the least operational overhead:

The problem requires a serverless solution for analyzing data stored in S3 using SQL, with encryption at rest and cross-region replication. Let's analyze each aspect:

Serverless SQL Querying: Amazon Athena is a serverless query service that allows you to analyze data in S3 using standard SQL. This perfectly aligns with the serverless requirement and the need for SQL querying. Amazon RDS, while supporting SQL, necessitates managing database instances, contradicting the serverless principle and adding operational overhead.

Encryption at Rest: SSE-KMS (Server-Side Encryption with KMS managed keys) and SSE-S3 (Server-Side Encryption with S3 managed keys) both provide encryption at rest. However, SSE-KMS with multi-Region keys provides added flexibility in key management, enabling simpler key rotation across regions and stronger compliance posture.

Cross-Region Replication (CRR): CRR automatically replicates objects from one S3 bucket to another in a different AWS Region. This fulfills the cross-region replication requirement for data durability and disaster recovery.

Least Operational Overhead: Option A offers the least operational overhead because it leverages Athena's serverless nature, eliminating the need to manage a database server like RDS. Utilizing SSE-KMS with multi-region keys provides robust encryption with potentially simpler key management compared to custom solutions. Option B, while providing replication and encryption with SSE-S3, opts for RDS instead of Athena. Option C, while attempting to use Athena, uses SSE-S3, which may be less flexible in key management compared to KMS and might require more overhead if keys need to be centrally managed across multiple regions. Finally, Option D combines RDS's overhead with the potential complexity of SSE-KMS multi-region keys, making it least efficient among the choices.

Therefore, creating a new S3 bucket with SSE-KMS multi-Region keys, configuring CRR, loading the data, and querying with Athena provides the most serverless, secure, and manageable solution.

Authoritative Links:

Amazon Athena: https://aws.amazon.com/athena/
Amazon S3 Encryption: https://docs.aws.amazon.com/AmazonS3/latest/userguide/serv-side-encryption.html
Amazon S3 Cross-Region Replication:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/replication.html
AWS KMS multi-region keys: https://docs.aws.amazon.com/kms/latest/developerguide/multi-region-keys-overview.html

Question: 953 **Exam Heist**

A company has a web application that has thousands of users. The application uses 8-10 user-uploaded images to generate AI images. Users can download the generated AI images once every 6 hours. The company also has a premium user option that gives users the ability to download the generated AI images anytime.

The company uses the user-uploaded images to run AI model training twice a year. The company needs a storage solution to store the images.

Which storage solution meets these requirements MOST cost-effectively?

A.Move uploaded images to Amazon S3 Glacier Deep Archive. Move premium user-generated AI images to S3 Standard. Move non-premium user-generated AI images to S3 Standard-Infrequent Access (S3 Standard-IA).

B.Move uploaded images to Amazon S3 Glacier Deep Archive Move all generated AI images to S3 Glacier Flexible Retrieval.

C.Move uploaded images to Amazon S3 One Zone-Infrequent Access (S3 One Zone-IA). Move premium user-generated AI images to S3 Standard. Move non-premium user-generated AI images to S3 Standard-Infrequent Access (S3 Standard-IA).

D.Move uploaded images to Amazon S3 One Zone-Infrequent Access (S3 One Zone-IA). Move all generated AI images to S3 Glacier Flexible Retrieval.

Answer: A

Explanation:

Here's a detailed justification for why option A is the most cost-effective solution for storing images, considering the usage patterns described, and why other options are less suitable:

Uploaded User Images (Long-Term Archive): The user-uploaded images are only used twice a year for AI model training. This indicates extremely infrequent access. Amazon S3 Glacier Deep Archive is designed for long-term data archiving with the lowest storage cost. While retrieval is slower (hours), this aligns perfectly with the twice-a-year access requirement, making it highly cost-effective for this data.

https://aws.amazon.com/s3/storage-classes/glacier/

Generated AI Images (Premium Users): Premium users require immediate and frequent access to their generated AI images. S3 Standard provides high availability and performance, ideal for frequently accessed data.

https://aws.amazon.com/s3/storage-classes/

Generated AI Images (Non-Premium Users): Non-premium users can only download their images once every 6 hours. S3 Standard-Infrequent Access (S3 Standard-IA) is suitable for data accessed less frequently but requires rapid access when needed. It offers lower storage costs than S3 Standard, making it more cost-effective for this usage pattern.

https://aws.amazon.com/s3/storage-classes/ia/

Why other options are less ideal:

Option B: Moving all generated AI images (including premium) to Glacier Flexible Retrieval is not optimal. Premium users need immediate access, and Glacier Flexible Retrieval has retrieval times from minutes to hours, degrading their experience. Glacier Flexible Retrieval is a good alternative, but not if immediate access is needed.
Option C & D: Storing uploaded images in S3 One Zone-IA introduces risk. It stores data in a single Availability Zone. If that AZ becomes unavailable, the data is lost. While cheaper than Standard, it is not suitable for data that needs to be archived as training data since it is an infrequent process. S3 Glacier Deep Archive is cheaper and suitable for such infrequent use cases.

https://aws.amazon.com/s3/storage-classes/onezone-ia/

In summary, option A aligns storage class with access frequency, providing the best cost optimization without sacrificing the required performance for each category of images.

Question: 954 **Exam Heist**

A company is developing machine learning (ML) models on AWS. The company is developing the ML models as independent microservices. The microservices fetch approximately 1 GB of model data from Amazon S3 at startup and load the data into memory. Users access the ML models through an asynchronous API. Users can send a request or a batch of requests.

The company provides the ML models to hundreds of users. The usage patterns for the models are irregular. Some models are not used for days or weeks. Other models receive batches of thousands of requests at a time.

Which solution will meet these requirements?

A.Direct the requests from the API to a Network Load Balancer (NLB). Deploy the ML models as AWS Lambda functions that the NLB will invoke. Use auto scaling to scale the Lambda functions based on the traffic that the NLB receives.

B.Direct the requests from the API to an Application Load Balancer (ALB). Deploy the ML models as Amazon Elastic Container Service (Amazon ECS) services that the ALB will invoke. Use auto scaling to scale the ECS cluster instances based on the traffic that the ALB receives.

C.Direct the requests from the API into an Amazon Simple Queue Service (Amazon SQS) queue. Deploy the ML models as AWS Lambda functions that SQS events will invoke. Use auto scaling to increase the number of vCPUs for the Lambda functions based on the size of the SQS queue.

D.Direct the requests from the API into an Amazon Simple Queue Service (Amazon SQS) queue. Deploy the ML models as Amazon Elastic Container Service (Amazon ECS) services that read from the queue. Use auto scaling for Amazon ECS to scale both the cluster capacity and number of the services based on the size of the SQS queue.

Answer: D

Explanation:

The correct answer is D because it leverages asynchronous processing with SQS and containerization with ECS to handle irregular traffic patterns and large model sizes efficiently. Here's a detailed breakdown:

Asynchronous API and SQS: Using SQS decouples the API from the ML model processing. The API pushes requests into the SQS queue, providing immediate acknowledgment to the user. This is crucial for handling batches of requests and irregular usage patterns without overwhelming the ML model services. SQS acts as a buffer, smoothing out traffic spikes. https://aws.amazon.com/sqs/

ECS and Containerization: Deploying the ML models as ECS services allows for containerization, which ensures consistent environments and facilitates easy scaling. Each ECS service instance can load the 1 GB model data into memory upon startup, as required. https://aws.amazon.com/ecs/

Auto Scaling based on Queue Size: Auto scaling the ECS cluster and the number of ECS services based on the size of the SQS queue is key to cost optimization. As the queue grows, ECS automatically provisions more resources to process the messages, and as the queue shrinks, it scales down to reduce costs when models are idle. This addresses the irregular usage patterns effectively. ECS supports scaling based on custom metrics, including SQS queue length.

Why other options are not ideal:

A (Lambda with NLB): Lambda functions have execution time limits and cold start issues which is not ideal for ML models which load 1 GB of model data at startup. Also, NLB is designed for TCP traffic not HTTP which is a limitation.
B (ECS with ALB): While ECS is a viable option, using an ALB directly would not be ideal because the API would need to wait for the ML processing to complete before responding, causing potential timeouts. ECS can handle the model sizes.
C (Lambda with SQS): Similar to option A, Lambda functions have execution time limits and cold start issues

which is not ideal for ML models which load 1 GB of model data at startup. Lambda is better suited for smaller stateless tasks. Auto-scaling on vCPUs is not how Lambda functions are scaled; instead, they scale by the number of concurrent executions.

In conclusion, the combination of SQS for asynchronous message queuing and ECS for containerized ML model deployment with auto-scaling based on queue size provides the most scalable, cost-effective, and reliable solution for handling irregular usage patterns and large model data in this scenario.

Question: 955 **Exam Heist**

A company runs a web application on Amazon EC2 instances in an Auto Scaling group behind an Application Load Balancer (ALB). The application stores data in an Amazon Aurora MySQL DB cluster.

The company needs to create a disaster recovery (DR) solution. The acceptable recovery time for the DR solution is up to 30 minutes. The DR solution does not need to support customer usage when the primary infrastructure is healthy.

Which solution will meet these requirements?

A.Deploy the DR infrastructure in a second AWS Region with an ALB and an Auto Scaling group. Set the desired capacity and maximum capacity of the Auto Scaling group to a minimum value. Convert the Aurora MySQL DB cluster to an Aurora global database. Configure Amazon Route 53 for an active-passive failover with ALB endpoints.

B.Deploy the DR infrastructure in a second AWS Region with an ALUpdate the Auto Scaling group to include EC2 instances from the second Region. Use Amazon Route 53 to configure active-active failover. Convert the Aurora MySQL DB cluster to an Aurora global database.

C.Back up the Aurora MySQL DB cluster data by using AWS Backup. Deploy the DR infrastructure in a second AWS Region with an ALB. Update the Auto Scaling group to include EC2 instances from the second Region. Use Amazon Route 53 to configure active-active failover. Create an Aurora MySQL DB cluster in the second Region Restore the data from the backup.

D.Back up the infrastructure configuration by using AWS Backup. Use the backup to create the required infrastructure in a second AWS Region. Set the Auto Scaling group desired capacity to zero. Use Amazon Route 53 to configure active-passive failover. Convert the Aurora MySQL DB cluster to an Aurora global database.

Answer: A

Explanation:

The correct answer is A. Here's why:

Recovery Time Objective (RTO): The requirement is an RTO of up to 30 minutes.

Aurora Global Database: Aurora Global Database is designed for disaster recovery. It replicates data with minimal latency to a secondary region, enabling a very fast failover. This is crucial for meeting the 30-minute RTO. (https://aws.amazon.com/rds/aurora/global-database/)

Active-Passive Failover: Configuring Route 53 for active-passive failover directs traffic to the primary region's ALB under normal conditions. In a disaster, it automatically reroutes traffic to the DR region's ALB. This provides a controlled failover mechanism. (https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-failover.html)

DR Infrastructure on Standby: Deploying a DR infrastructure (ALB, Auto Scaling group, EC2 instances) in a second region ensures that the application can be quickly brought online in case of a primary region failure. Setting the desired capacity of the Auto Scaling group to a minimum value (e.g., zero or a small number of instances) minimizes costs during normal operation while allowing for rapid scaling upon failover.

Other options analysis:

B: Active-active Route 53 failover is not suitable in this scenario since the DR solution is only needed in a failover scenario, so no traffic should be sent to the DR region in normal situations.
C: Restoring the data from backups could violate the 30 minute recovery time objective.
D: Restoring the infrastructure from backups could violate the 30 minute recovery time objective.

Question: 956 **Exam Heist**

A company is migrating its data processing application to the AWS Cloud. The application processes several short-lived batch jobs that cannot be disrupted. Data is generated after each batch job is completed. The data is accessed for 30 days and retained for 2 years.

The company wants to keep the cost of running the application in the AWS Cloud as low as possible.

Which solution will meet these requirements?

A.Migrate the data processing application to Amazon EC2 Spot Instances. Store the data in Amazon S3 Standard. Move the data to Amazon S3 Glacier Instant. Retrieval after 30 days. Set an expiration to delete the data after 2 years.

B.Migrate the data processing application to Amazon EC2 On-Demand Instances. Store the data in Amazon S3 Glacier Instant Retrieval. Move the data to S3 Glacier Deep Archive after 30 days. Set an expiration to delete the data after 2 years.

C.Deploy Amazon EC2 Spot Instances to run the batch jobs. Store the data in Amazon S3 Standard. Move the data to Amazon S3 Glacier Flexible Retrieval after 30 days. Set an expiration to delete the data after 2 years.

D.Deploy Amazon EC2 On-Demand Instances to run the batch jobs. Store the data in Amazon S3 Standard. Move the data to Amazon S3 Glacier Deep Archive after 30 days. Set an expiration to delete the data after 2 years.

Answer: D

Explanation:

The correct solution is D. Here's why:

EC2 Instance Type: The problem states that batch jobs "cannot be disrupted." Spot Instances can be terminated with little warning, which is not suitable for jobs that must complete without interruption. On-Demand Instances are a better fit because they provide a guaranteed compute capacity.
https://aws.amazon.com/ec2/pricing/on-demand/

Initial Storage: Storing the data in Amazon S3 Standard for the first 30 days is optimal because it offers low latency access, which is needed given that data is accessed within this period.
https://aws.amazon.com/s3/storage-classes/

Long-Term Archival: After 30 days, the data is no longer actively accessed. Amazon S3 Glacier Deep Archive is the most cost-effective storage class for data that is infrequently accessed but must be retained for long periods. https://aws.amazon.com/s3/storage-classes/

Data Retention: S3 Lifecycle policies, using expiration, automatically delete data after a specified period (in this case, 2 years). This reduces storage costs and ensures compliance with retention policies.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-examples.html

Why other options are not suitable:

Option A and C (Spot Instances): Not suitable because the workload cannot be interrupted.
Option A and B (S3 Glacier Instant Retrieval): Glacier Instant Retrieval is more expensive than Glacier Deep Archive. Given the data is accessed less frequently after 30 days, using Deep Archive will be more cost-effective.
Option C (S3 Glacier Flexible Retrieval): Glacier Flexible Retrieval is less cost-effective than Glacier Deep

Archive when infrequent access is acceptable.

In summary, Option D balances cost efficiency with the application's requirements for uninterrupted batch job processing, data accessibility, and long-term data retention.

Question: 957 **Exam Heist**

A company needs to design a hybrid network architecture. The company's workloads are currently stored in the AWS Cloud and in on-premises data centers. The workloads require single-digit latencies to communicate. The company uses an AWS Transit Gateway transit gateway to connect multiple VPCs.

Which combination of steps will meet these requirements MOST cost-effectively? (Choose two.)

A.Establish an AWS Site-to-Site VPN connection to each VPC.

B.Associate an AWS Direct Connect gateway with the transit gateway that is attached to the VPCs.

C.Establish an AWS Site-to-Site VPN connection to an AWS Direct Connect gateway.

D.Establish an AWS Direct Connect connection. Create a transit virtual interface (VIF) to a Direct Connect gateway.

E.Associate AWS Site-to-Site VPN connections with the transit gateway that is attached to the VPCs.

Answer: BD

Explanation:

The correct answer is **BD**. Here's a detailed justification:

B. Associate an AWS Direct Connect gateway with the transit gateway that is attached to the VPCs.

Direct Connect for Low Latency: AWS Direct Connect establishes a dedicated network connection from your on-premises environment to AWS. This bypasses the public internet, providing more consistent network performance and lower latency, which is critical for the single-digit millisecond latency requirement. (https://aws.amazon.com/directconnect/)
Transit Gateway Integration: A Direct Connect gateway can be associated with a Transit Gateway. Transit Gateway acts as a central hub, enabling connectivity between multiple VPCs. By associating the Direct Connect gateway with the Transit Gateway, on-premises workloads can communicate with all VPCs connected to the Transit Gateway over the dedicated Direct Connect link, ensuring low-latency communication between on-premises and cloud resources. This is more efficient than managing individual connections to each VPC.

D. Establish an AWS Direct Connect connection. Create a transit virtual interface (VIF) to a Direct Connect gateway.

Transit VIF for Transit Gateway Connectivity: To connect your Direct Connect connection to the Transit Gateway, you need to create a transit virtual interface (VIF). A transit VIF allows you to reach the Direct Connect gateway and, through it, the associated Transit Gateway. (https://docs.aws.amazon.com/directconnect/latest/UserGuide/multi-account-tgw.html)
End-to-End Dedicated Path: Establishing the Direct Connect connection and creating the transit VIF creates a dedicated, low-latency path from your on-premises environment, through the Direct Connect connection, to the Direct Connect gateway, then through the Transit Gateway, and finally to the VPCs.

Why other options are less suitable or incorrect:

A. Establish an AWS Site-to-Site VPN connection to each VPC: While Site-to-Site VPN can provide a connection to AWS, it uses the public internet, which is not suitable for single-digit millisecond latency requirements. The internet introduces variability and is not a dedicated connection. Furthermore, establishing

multiple VPN connections becomes cumbersome and costly to manage for multiple VPCs.
C. Establish an AWS Site-to-Site VPN connection to an AWS Direct Connect gateway: This option doesn't make sense. Direct Connect already provides a dedicated connection. Adding a VPN on top of it is redundant and defeats the purpose of using Direct Connect for low latency. It would also add unnecessary complexity and cost.
E. Associate AWS Site-to-Site VPN connections with the transit gateway that is attached to the VPCs: Similar to option A, using VPN connections will not achieve single-digit millisecond latency due to its reliance on the public internet. While Transit Gateway can route VPN traffic, the inherent limitations of VPNs for latency remain.

Therefore, establishing a Direct Connect connection with a transit VIF to a Direct Connect gateway, and associating that gateway with the Transit Gateway provides the dedicated, low-latency connectivity required, and the transit gateway facilitates the connectivity with multiple VPCs, which meets the requirements of the scenario most cost-effectively.

Question: 958 **Exam Heist**

A global ecommerce company runs its critical workloads on AWS. The workloads use an Amazon RDS for PostgreSQL DB instance that is configured for a Multi-AZ deployment.

Customers have reported application timeouts when the company undergoes database failovers. The company needs a resilient solution to reduce failover time.

Which solution will meet these requirements?

A.Create an Amazon RDS Proxy. Assign the proxy to the DB instance.
B.Create a read replica for the DB instance. Move the read traffic to the read replica.
C.Enable Performance Insights. Monitor the CPU load to identify the timeouts.
D.Take regular automatic snapshots. Copy the automatic snapshots to multiple AWS Regions.

Answer: A

Explanation:

The correct answer is A: Create an Amazon RDS Proxy and assign the proxy to the DB instance.

Justification:

The primary problem is application timeouts during RDS failovers. RDS Multi-AZ deployments provide high availability by automatically failing over to a standby instance in case of an issue. However, the failover process, even though automatic, takes time. During this failover, the application loses its database connection, leading to timeouts.

RDS Proxy is a fully managed, highly available database proxy that sits between your application and your RDS database. Its key feature in this scenario is connection management and failover handling. RDS Proxy maintains a pool of database connections and automatically reconnects to the new primary instance after a failover. It masks the failover event from the application, thus drastically reducing or eliminating application timeouts. The application simply continues using the same connection string via the proxy, and the proxy handles the underlying switchover.

Here's why other options are not as suitable:

B: Create a read replica for the DB instance. Move the read traffic to the read replica. Read replicas are helpful for offloading read traffic from the primary instance, improving performance. However, they do not address the connection timeout issue during the failover of the primary instance. Read replicas also need time

to be promoted to primary after the primary instance fails.
C: Enable Performance Insights. Monitor the CPU load to identify the timeouts. Performance Insights helps diagnose performance bottlenecks but doesn't prevent or mitigate failover-related timeouts. It can help understand why performance is degraded, but it doesn't solve the core availability problem.
D: Take regular automatic snapshots. Copy the automatic snapshots to multiple AWS Regions. Snapshots are useful for disaster recovery and point-in-time recovery but do not improve failover time. Restoring from a snapshot is a time-consuming process and unsuitable for minimizing application timeouts during failover.

Authoritative Links:

Amazon RDS Proxy: Provides a complete overview of RDS Proxy features and benefits.
Using Amazon RDS Multi-AZ deployments for high availability: Explains how Multi-AZ deployments work and their limitations regarding failover time.

Question: 959 **Exam Heist**

A company has multiple Amazon RDS DB instances that run in a development AWS account. All the instances have tags to identify them as development resources. The company needs the development DB instances to run on a schedule only during business hours.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create an Amazon CloudWatch alarm to identify RDS instances that need to be stopped. Create an AWS Lambda function to start and stop the RDS instances.

B.Create an AWS Trusted Advisor report to identify RDS instances to be started and stopped. Create an AWS Lambda function to start and stop the RDS instances.

C.Create AWS Systems Manager State Manager associations to start and stop the RDS instances.

D.Create an Amazon EventBridge rule that invokes AWS Lambda functions to start and stop the RDS instances.

Answer: C

Explanation:

The correct answer is **C. Create AWS Systems Manager State Manager associations to start and stop the RDS instances.**

Here's why:

AWS Systems Manager (SSM) State Manager allows you to automate tasks on a schedule using SSM documents. In this case, you can use SSM documents to start and stop RDS instances based on tags. This provides a centralized and auditable way to manage the scheduling. State Manager operates directly within AWS and integrates seamlessly with RDS, removing the need for custom code.SSM offers built-in scheduling capabilities and provides visibility into the execution status of the automation. This includes monitoring of RDS resources.State Manager is designed for infrastructure management and automation, making it a natural fit for this type of task, and requires minimal operational overhead. Using tags for RDS will also create an easy filter to only include resources that need to be turned off.

Why other options are less suitable:

A (CloudWatch alarm & Lambda): This is feasible, but more complex. You'd need to create alarms to identify instances and then use Lambda to start/stop them. SSM provides a cleaner, simpler way without requiring coding and creating alarms.
B (Trusted Advisor & Lambda): Trusted Advisor focuses on cost optimization, security, and performance best practices, not scheduling resources. It wouldn't directly support starting and stopping RDS instances based on a schedule.

D (EventBridge & Lambda): While EventBridge can schedule events, it's more suitable for triggering actions based on events. Using it to start/stop RDS instances is overkill compared to SSM's built-in scheduling capabilities.

In summary:

SSM State Manager directly addresses the requirement with a straightforward, centralized, and less operationally intensive solution. It allows you to define schedules for starting and stopping RDS instances based on tags, minimizing manual intervention and custom coding.

Authoritative Links:

AWS Systems Manager State Manager: https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-state.html
AWS Systems Manager Automation: https://docs.aws.amazon.com/systems-manager/latest/userguide/automation-workflows.html

Question: 960 **Exam Heist**

A consumer survey company has gathered data for several years from a specific geographic region. The company stores this data in an Amazon S3 bucket in an AWS Region.

The company has started to share this data with a marketing firm in a new geographic region. The company has granted the firm's AWS account access to the S3 bucket. The company wants to minimize the data transfer costs when the marketing firm requests data from the S3 bucket.

Which solution will meet these requirements?

A.Configure the Requester Pays feature on the company’s S3 bucket.

B.Configure S3 Cross-Region Replication (CRR) from the company’s S3 bucket to one of the marketing firm’s S3 buckets.

C.Configure AWS Resource Access Manager to share the S3 bucket with the marketing firm AWS account.

D.Configure the company’s S3 bucket to use S3 Intelligent-Tiering Sync the S3 bucket to one of the marketing firm’s S3 buckets.

Answer: A

Explanation:

The correct answer is **A. Configure the Requester Pays feature on the company’s S3 bucket.**

Here's a detailed justification:

The primary goal is to minimize data transfer costs when the marketing firm (the requester) accesses data from the company's S3 bucket. The key is that the requester is incurring data transfer costs to their region.

Option A (Requester Pays): This configuration makes the requester responsible for the data transfer costs. This is directly aligned with minimizing the company's costs. When the marketing firm accesses the data, they will pay for the data egress from the company's bucket to their region. This fulfills the requirement of minimizing the company's data transfer costs.

Option B (CRR): Cross-Region Replication (CRR) replicates data from one S3 bucket to another in a different AWS Region. While it places the data closer to the marketing firm, the company initially pays for the replication costs. Furthermore, the question prioritizes minimizing data transfer costs specifically incurred when the marketing firm requests data. CRR would incur costs upfront regardless of actual data access.

Option C (AWS RAM): AWS Resource Access Manager (RAM) allows you to share AWS resources across AWS

accounts within your organization or organizational units. Sharing the bucket through RAM doesn't inherently minimize data transfer costs. The company would still be responsible for the egress costs when the marketing firm retrieves data.

Option D (S3 Intelligent-Tiering and Sync): S3 Intelligent-Tiering automatically moves data between access tiers (frequent, infrequent, archive) based on access patterns. This doesn't address the data transfer costs associated with the marketing firm downloading the data. Syncing the bucket to one of the marketing firm's buckets isn't a native S3 feature and would require a custom solution, and further, it wouldn't address the cost issue directly compared to Requester Pays.

Therefore, configuring Requester Pays ensures the marketing firm, as the requester, bears the cost of transferring data out of the company's S3 bucket, effectively minimizing the company's costs.

Authoritative Links:

Amazon S3 Requester Pays: https://docs.aws.amazon.com/AmazonS3/latest/userguide/requester-pays.html

Question: 961 **Exam Heist**

A company uses AWS to host its public ecommerce website. The website uses an AWS Global Accelerator accelerator for traffic from the internet. The Global Accelerator accelerator forwards the traffic to an Application Load Balancer (ALB) that is the entry point for an Auto Scaling group.

The company recently identified a DDoS attack on the website. The company needs a solution to mitigate future attacks.

Which solution will meet these requirements with the LEAST implementation effort?

A.Configure an AWS WAF web ACL for the Global Accelerator accelerator to block traffic by using rate-based rules

B.Configure an AWS Lambda function to read the ALB metrics to block attacks by updating a VPC network ACL

C.Configure an AWS WAF web ACL on the ALB to block traffic by using rate-based rules

D.Configure an Amazon CloudFront distribution in front of the Global Accelerator accelerator

Answer: A

Explanation:

The correct answer is **A. Configure an AWS WAF web ACL for the Global Accelerator accelerator to block traffic by using rate-based rules.**

Here's a detailed justification:

1. **DDoS Mitigation at the Edge:** The goal is to mitigate DDoS attacks. AWS recommends handling these attacks as close to the source as possible. Global Accelerator sits at the edge of the AWS network.
2. **WAF Integration with Global Accelerator:** AWS WAF integrates directly with Global Accelerator. This enables you to create rules to filter malicious traffic before it even reaches your ALB and Auto Scaling group.
3. **Rate-Based Rules:** Rate-based rules within AWS WAF are specifically designed to mitigate DDoS attacks. They monitor the rate of requests from each IP address and block those exceeding a defined threshold. This provides protection against volumetric attacks.
4. **Least Implementation Effort:** Implementing WAF at the Global Accelerator level requires attaching a WAF web ACL to the accelerator. Configuring rate-based rules within the WAF web ACL is relatively

straightforward and requires less configuration than other options.

5. **ALB WAF (Option C):** While you can use WAF with an ALB, it's less effective for DDoS mitigation. The traffic has already traversed the internet and reached the ALB, consuming resources. Also, Global Accelerator provides static IP addresses which makes it easier to track source IPs for rate limiting.
6. **Lambda and VPC Network ACLs (Option B):** This option involves a more complex and potentially less responsive implementation. Using Lambda to analyze ALB metrics and update VPC network ACLs would require custom coding, monitoring, and more operational overhead. It's also slower to react to attacks compared to WAF rate-based rules. VPC Network ACLs operate at the subnet level and are not as granular or flexible as WAF rules.
7. **CloudFront in Front of Global Accelerator (Option D):** This option introduces unnecessary complexity. Global Accelerator already provides benefits similar to a CDN, such as improved performance through static IP addresses and traffic steering. Adding CloudFront would increase costs and management overhead without providing a significant improvement in DDoS mitigation capabilities that WAF on Global Accelerator wouldn't offer.

In summary, configuring an AWS WAF web ACL on the Global Accelerator accelerator with rate-based rules provides an effective and efficient solution for mitigating DDoS attacks by filtering malicious traffic at the edge of the AWS network with the least amount of operational burden.

Authoritative Links:

AWS WAF: https://aws.amazon.com/waf/
AWS Global Accelerator: https://aws.amazon.com/global-accelerator/
AWS WAF Rate-Based Rules: https://docs.aws.amazon.com/waf/latest/developerguide/waf-rate-based-rule.html
AWS Best Practices for DDoS Mitigation: https://d1.awsstatic.com/whitepapers/DDoS_White_Paper.pdf

Question: 962 **Exam Heist**

A company uses an Amazon DynamoDB table to store data that the company receives from devices. The DynamoDB table supports a customer-facing website to display recent activity on customer devices. The company configured the table with provisioned throughput for writes and reads.

The company wants to calculate performance metrics for customer device data on a daily basis. The solution must have minimal effect on the table's provisioned read and write capacity.

Which solution will meet these requirements?

A.Use an Amazon Athena SQL query with the Amazon Athena DynamoDB connector to calculate performance metrics on a recurring schedule.

B.Use an AWS Glue job with the AWS Glue DynamoDB export connector to calculate performance metrics on a recurring schedule.

C.Use an Amazon Redshift COPY command to calculate performance metrics on a recurring schedule.

D.Use an Amazon EMR job with an Apache Hive external table to calculate performance metrics on a recurring schedule.

Answer: B

Explanation:

The best solution is B: Use an AWS Glue job with the AWS Glue DynamoDB export connector to calculate performance metrics on a recurring schedule. Here's why:

Minimizing Impact on DynamoDB: The core requirement is to minimize impact on the provisioned read and write capacity of the DynamoDB table, which serves a customer-facing website. Running queries directly against the DynamoDB table, as Athena (A) and EMR (D) would do, could consume significant read capacity, potentially impacting the website's performance. Similarly, Redshift (C) using the COPY command would require reading data directly from the DynamoDB table, again consuming read capacity.

AWS Glue and DynamoDB Export Connector: The AWS Glue DynamoDB export connector is designed specifically for efficiently extracting data from DynamoDB tables without significantly impacting their provisioned throughput. It utilizes DynamoDB's consistent read capabilities in a throttled manner or, ideally, leverages DynamoDB backups (snapshots) for data extraction. This ensures the production table remains responsive for customer requests.

AWS Glue allows you to schedule recurring jobs, making it suitable for calculating daily performance metrics. AWS Glue provides powerful data transformation capabilities using Spark or Python, enabling you to easily calculate the required performance metrics.

Athena Limitations: While Athena can query DynamoDB, it directly consumes read capacity, making it less desirable when minimizing impact is a priority.

Redshift Limitations: Redshift COPY command reads directly from the DynamoDB table. This directly impacts the read capacity. Moreover, Redshift is not designed for real-time or near-real-time analytics directly on DynamoDB data.

EMR Limitations: EMR using Apache Hive could query DynamoDB, but again, this puts a load on the DynamoDB table. Also, setting up and managing an EMR cluster for this specific task is more complex than using AWS Glue, especially for recurring scheduled jobs.

Cost Considerations: AWS Glue is generally more cost-effective for this purpose than Redshift or EMR because it is a serverless, pay-as-you-go service. You only pay for the time the Glue job runs.

Authoritative Links:

AWS Glue: https://aws.amazon.com/glue/
AWS Glue DynamoDB Connector: https://docs.aws.amazon.com/glue/latest/dg/aws-glue-programming-etl-connect.html
DynamoDB on-demand backup and restore: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/BackupRestore.html

Question: 963 **Exam Heist**

A solutions architect is designing the cloud architecture for a new stateless application that will be deployed on AWS. The solutions architect created an Amazon Machine Image (AMI) and launch template for the application.

Based on the number of jobs that need to be processed, the processing must run in parallel while adding and removing application Amazon EC2 instances as needed. The application must be loosely coupled. The job items must be durably stored.

Which solution will meet these requirements?

A.Create an Amazon Simple Notification Service (Amazon SNS) topic to send the jobs that need to be processed. Create an Auto Scaling group by using the launch template with the scaling policy set to add and remove EC2 instances based on CPU usage.

B.Create an Amazon Simple Queue Service (Amazon SQS) queue to hold the jobs that need to be processed. Create an Auto Scaling group by using the launch template with the scaling policy set to add and remove EC2 instances based on network usage.

C.Create an Amazon Simple Queue Service (Amazon SQS) queue to hold the jobs that need to be processed. Create an Auto Scaling group by using the launch template with the scaling policy set to add and remove EC2

instances based on the number of items in the SQS queue.

D.Create an Amazon Simple Notification Service (Amazon SNS) topic to send the jobs that need to be processed. Create an Auto Scaling group by using the launch template with the scaling policy set to add and remove EC2 instances based on the number of messages published to the SNS topic.

Answer: C

Explanation:

The requirement of durable storage of job items and loose coupling between application components points towards a queuing system. Amazon SQS (Simple Queue Service) is designed for this purpose, providing reliable message queuing. SNS (Simple Notification Service), on the other hand, is for pub/sub messaging, not durable storage of individual job items. Therefore, options A and D, which propose using SNS, are incorrect.

Auto Scaling groups are suitable for dynamically scaling EC2 instances based on demand. Scaling based on the number of items in the SQS queue directly reflects the job backlog and the need for processing power. This allows the Auto Scaling group to add instances when the queue is growing and remove instances when the queue is shrinking, ensuring efficient resource utilization and responsiveness to varying job loads. Scaling based on CPU or network usage, as suggested in option B, may not accurately reflect the volume of jobs needing processing, potentially leading to under- or over-provisioning of EC2 instances.

Therefore, option C, using SQS for queuing jobs and scaling the Auto Scaling group based on the number of items in the queue, correctly addresses the requirements for durable storage, loose coupling, and dynamic scaling based on job volume. This approach ensures that all jobs are reliably stored until processed and that the application scales appropriately to handle the workload.

Here are some links for further research:

Amazon SQS: https://aws.amazon.com/sqs/
Amazon SNS: https://aws.amazon.com/sns/
Auto Scaling groups: https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-groups.html
Launch Templates: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-launch-templates.html

Question: 964 **Exam Heist**

A global ecommerce company uses a monolithic architecture. The company needs a solution to manage the increasing volume of product data. The solution must be scalable and have a modular service architecture. The company needs to maintain its structured database schemas. The company also needs a storage solution to store product data and product images.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use an Amazon EC2 instance in an Auto Scaling group to deploy a containerized application. Use an Application Load Balancer to distribute web traffic. Use an Amazon RDS DB instance to store product data and product images.

B.Use AWS Lambda functions to manage the existing monolithic application. Use Amazon DynamoDB to store product data and product images. Use Amazon Simple Notification Service (Amazon SNS) for event-driven communication between the Lambda functions.

C.Use Amazon Elastic Kubernetes Service (Amazon EKS) with an Amazon EC2 deployment to deploy a containerized application. Use an Amazon Aurora cluster to store the product data. Use AWS Step Functions to manage workflows. Store the product images in Amazon S3 Glacier Deep Archive.

D.Use Amazon Elastic Container Service (Amazon ECS) with AWS Fargate to deploy a containerized application. Use Amazon RDS with a Multi-AZ deployment to store the product data. Store the product images in an Amazon S3 bucket.

Answer: D

Explanation:

The correct answer is D because it offers the most scalable, modular, and operationally efficient solution for the given requirements.

Why D is the Best Choice:

Scalable and Modular Architecture: Amazon ECS with Fargate allows the company to break down the monolithic application into microservices using containers, achieving a modular service architecture. Fargate eliminates the operational overhead of managing EC2 instances for the containers, as AWS handles the underlying infrastructure. https://aws.amazon.com/ecs/fargate/

Structured Database Schemas: Amazon RDS is suitable for maintaining structured database schemas as the company requested. Using a Multi-AZ deployment in RDS ensures high availability and disaster recovery for the product data, increasing the system's resilience. https://aws.amazon.com/rds/features/multi-az/

Product Images Storage: Amazon S3 is the ideal choice for storing product images because it provides scalable, durable, and cost-effective object storage. S3 allows for easy retrieval and integration with the application. https://aws.amazon.com/s3/

Least Operational Overhead: Fargate for container management and S3 for object storage minimize operational overhead compared to managing EC2 instances or complex orchestration tools like EKS. RDS managed services reduce operational overhead.

Why other options are less suitable:

A: While EC2 with Auto Scaling and RDS are good, it requires more operational overhead in managing EC2 instances.
B: DynamoDB is a NoSQL database and does not align with the requirement to maintain structured database schemas. Also, migrating an existing monolithic application to Lambda functions and event-driven architecture might be an extensive effort.
C: Amazon EKS is more complex to manage than ECS with Fargate and introduces higher operational overhead. Glacier Deep Archive is more suitable for long-term archival data and less suitable for product images that may need frequent access. Step Functions introduces complexity that isn't necessary for this scenario.

Therefore, option D offers the best balance of scalability, modularity, and minimal operational overhead while fulfilling the requirements of maintaining structured schemas and storing product images.

Question: 965 **Exam Heist**

A company is migrating an application from an on-premises environment to AWS. The application will store sensitive data in Amazon S3. The company must encrypt the data before storing the data in Amazon S3.

Which solution will meet these requirements?

A.Encrypt the data by using client-side encryption with customer managed keys.
B.Encrypt the data by using server-side encryption with AWS KMS keys (SSE-KMS).
C.Encrypt the data by using server-side encryption with customer-provided keys (SSE-C).
D.Encrypt the data by using client-side encryption with Amazon S3 managed keys.

Answer: A

Explanation:

The correct answer is A: Encrypt the data by using client-side encryption with customer-managed keys.

Here's a detailed justification:

The requirement is to encrypt sensitive data before it's stored in Amazon S3. This indicates the need for client-side encryption. Client-side encryption involves encrypting the data on the client-side (i.e., before uploading it to S3). This provides end-to-end encryption, ensuring data confidentiality even during transit and at rest in S3.

Option A utilizes client-side encryption, which fulfills the core requirement of pre-storage encryption. Importantly, it specifies using customer-managed keys. This means the customer has full control over the encryption keys, managing their lifecycle and access policies through AWS Key Management Service (KMS). This addresses security best practices and compliance needs often associated with sensitive data. Customer-managed keys offer greater control and auditability compared to S3-managed keys.

Option B, Server-Side Encryption with AWS KMS keys (SSE-KMS), encrypts the data after it's received by S3. While secure, it doesn't meet the specific requirement of encrypting the data before storing it. S3 handles the encryption process upon receiving the unencrypted data.

Option C, Server-Side Encryption with Customer-Provided Keys (SSE-C), also encrypts data on the server side (after receipt by S3). While the customer provides the key, they are responsible for managing the key's lifecycle and securely providing it to S3 for each request. This adds operational complexity without necessarily enhancing security compared to KMS.

Option D, Client-Side Encryption with Amazon S3 Managed Keys, is incorrect because S3-managed keys grant Amazon more control over the key lifecycle. The company needs more control given the sensitivity of the data as stated in the problem.

Therefore, client-side encryption with customer-managed keys is the most suitable option to meet the specific requirement of pre-storage encryption with maximum customer control over the encryption keys. This approach aligns with security best practices, compliance requirements, and ensures end-to-end data protection.

Supporting documentation:

AWS KMS: https://aws.amazon.com/kms/
Protecting Data Using Server-Side Encryption:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/serv-side-encryption.html
Protecting Data Using Client-Side Encryption:
https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingEncryption.html

Question: 966 **Exam Heist**

A company wants to create an Amazon EMR cluster that multiple teams will use. The company wants to ensure that each team’s big data workloads can access only the AWS services that each team needs to interact with. The company does not want the workloads to have access to Instance Metadata Service Version 2 (IMDSv2) on the cluster’s underlying EC2 instances.

Which solution will meet these requirements?

A.Configure interface VPC endpoints for each AWS service that the teams need. Use the required interface VPC endpoints to submit the big data workloads.

B.Create EMR runtime roles. Configure the cluster to use the runtime roles. Use the runtime roles to submit the big data workloads.

C.Create an EC2 IAM instance profile that has the required permissions for each team. Use the instance profile to submit the big data workloads.

D.Create an EMR security configuration that has the EnableApplicationScopedIAMRole option set to false. Use the security configuration to submit the big data workloads.

Answer: B

Explanation:

The correct answer is **B. Create EMR runtime roles. Configure the cluster to use the runtime roles. Use the runtime roles to submit the big data workloads.**

Here's why:

EMR Runtime Roles: EMR runtime roles, introduced with EMR 6.1.0 and later, provide a fine-grained access control mechanism for big data applications running on EMR clusters. They allow you to assign different IAM roles to different applications or jobs within the same cluster. This addresses the requirement of allowing each team's workloads to access only the AWS services they need.
Granular Permissions: Runtime roles enable granular control over permissions at the application level, moving away from the broad permissions granted by the EC2 instance profile. This is crucial for implementing the principle of least privilege.
IMDSv2 Control: Runtime roles inherently bypass the need to directly interact with the EC2 instance metadata. This is because applications assume the runtime role credentials which are separate from the instance profile, thus effectively addressing the requirement of not using IMDSv2 for workloads.
VPC Endpoints (Option A - Incorrect): While interface VPC endpoints enhance security by keeping traffic within the AWS network, they don't address the requirement of assigning different permissions to different teams' workloads. All workloads using the VPC endpoints would still operate under the permissions associated with the instance profile or some other global mechanism.
EC2 Instance Profile (Option C - Incorrect): Using EC2 instance profiles would grant the same set of permissions to all workloads running on the cluster's EC2 instances. This contradicts the requirement that each team's workloads should access only the AWS services they need.
EMR Security Configuration (Option D - Incorrect): The EnableApplicationScopedIAMRole option, typically used in older EMR versions, is not the ideal way to manage permissions. It would provide less granular control than runtime roles. Furthermore, setting it to 'false' would revert to using the EC2 instance profile, which is against the requirement.

In summary, EMR runtime roles offer the most suitable solution because they allow for granular, application-level permission management, fulfilling the requirement of controlling AWS service access for each team's workloads and avoiding direct use of IMDSv2.

References:

Use IAM roles for EMR steps
EMR Runtime Roles

Question: 967 **Exam Heist**

A solutions architect is designing an application that helps users fill out and submit registration forms. The solutions architect plans to use a two-tier architecture that includes a web application server tier and a worker tier.

The application needs to process submitted forms quickly. The application needs to process each form exactly once. The solution must ensure that no data is lost.

Which solution will meet these requirements?

A.Use an Amazon Simple Queue Service (Amazon SQS) FIFO queue between the web application server tier and the worker tier to store and forward form data.

B.Use an Amazon API Gateway HTTP API between the web application server tier and the worker tier to store

and forward form data.

C.Use an Amazon Simple Queue Service (Amazon SQS) standard queue between the web application server tier and the worker tier to store and forward form data.

D.Use an AWS Step Functions workflow. Create a synchronous workflow between the web application server tier and the worker tier that stores and forwards form data.

Answer: A

Explanation:

The correct answer is A, using an Amazon SQS FIFO queue. Let's dissect why:

The core requirement is to process each form exactly once and ensure no data loss, guaranteeing data integrity in the application's workflow. This necessitates a queuing system with strict ordering and delivery guarantees.

Amazon SQS FIFO (First-In, First-Out) queues directly address these requirements. FIFO queues maintain the order in which messages are sent and received, ensuring that forms are processed in the order they were submitted. More importantly, they provide exactly-once processing semantics when used with message deduplication, meaning each form will be processed only one time even in the event of failures or retries. Standard SQS queues (option C) offer best-effort ordering, which doesn't guarantee the precise order of form processing and allows for duplicate processing, making them unsuitable.

Amazon API Gateway (option B) is primarily designed for handling API requests and routing traffic. While it can interact with backend services, it doesn't inherently offer the queuing, ordering, and exactly-once processing capabilities required for this scenario. API Gateway is a request/response service, not a durable queue.

AWS Step Functions (option D) manages workflows, but a synchronous workflow doesn't inherently provide the queuing and exactly-once processing guarantees required. A synchronous Step Functions workflow directly invokes the worker, and if the worker fails midway through processing, the workflow will need to be manually retried or include complex error handling logic. While Step Functions can integrate with SQS, using SQS FIFO directly simplifies the solution for this specific requirement of ordered and guaranteed delivery. The problem specifically asks to process forms "quickly." Introducing Step Functions adds orchestration overhead to the overall process.

Therefore, Amazon SQS FIFO is the most appropriate solution because it guarantees ordered delivery and exactly-once processing of form data between the web application server tier and the worker tier, fulfilling the application's requirements for data integrity and processing accuracy. It eliminates the possibility of duplicate form processing and ensures no data loss.

Supporting Documentation:

Amazon SQS FIFO queues:
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues.html
Amazon SQS Message Deduplication:
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queue-recommendations.html
AWS Step Functions: https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html
Amazon API Gateway: https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html

Question: 968 **Exam Heist**

A finance company uses an on-premises search application to collect streaming data from various producers. The application provides real-time updates to search and visualization features.

The company is planning to migrate to AWS and wants to use an AWS native solution.

Which solution will meet these requirements?

A.Use Amazon EC2 instances to ingest and process the data streams to Amazon S3 buckets tor storage. Use Amazon Athena to search the data. Use Amazon Managed Grafana to create visualizations.

B.Use Amazon EMR to ingest and process the data streams to Amazon Redshift for storage. Use Amazon Redshift Spectrum to search the data. Use Amazon QuickSight to create visualizations.

C.Use Amazon Elastic Kubernetes Service (Amazon EKS) to ingest and process the data streams to Amazon DynamoDB for storage. Use Amazon CloudWatch to create graphical dashboards to search and visualize the data.

D.Use Amazon Kinesis Data Streams to ingest and process the data streams to Amazon OpenSearch Service. Use OpenSearch Service to search the data. Use Amazon QuickSight to create visualizations.

Answer: D

Explanation:

The correct answer is D because it leverages AWS services specifically designed for real-time streaming data ingestion, processing, search, and visualization, aligning perfectly with the finance company's requirements.

Here's a detailed justification:

Amazon Kinesis Data Streams: This service is designed for real-time ingestion of streaming data. It provides a scalable and durable way to collect data from multiple producers, addressing the company's need to gather data from various sources. (https://aws.amazon.com/kinesis/data-streams/)

Amazon OpenSearch Service (successor to Elasticsearch): OpenSearch is a powerful search and analytics engine that excels at indexing and querying large volumes of data in near real-time. It is well-suited for the company's need for real-time search functionality. Kinesis Data Streams can directly integrate with OpenSearch, making the pipeline seamless. (https://aws.amazon.com/opensearch-service/)

Amazon QuickSight: QuickSight is a business intelligence (BI) service that enables the creation of interactive dashboards and visualizations. It can connect to OpenSearch Service to provide insights and real-time updates to the company's data, fulfilling the visualization requirement. (https://aws.amazon.com/quicksight/)

Options A, B, and C are less ideal for the following reasons:

Option A (EC2, S3, Athena, Grafana): Using EC2 for ingestion requires custom development and management. S3 is primarily for storage, not real-time search. Athena is suitable for querying data at rest but isn't optimized for the real-time updates required.

Option B (EMR, Redshift, Redshift Spectrum, QuickSight): Amazon EMR is better suited for batch processing and is generally not ideal for constant streaming ingestion. Amazon Redshift is a data warehouse designed for analytical workloads, while Kinesis Data Streams with OpenSearch is better for real-time search applications. Redshift Spectrum could query data in S3 but is not the best choice for Real-time.

Option C (EKS, DynamoDB, CloudWatch Dashboards): While EKS can host streaming applications, it adds operational complexity. DynamoDB is a NoSQL database, but not primarily designed for real-time search. CloudWatch dashboards, while useful for monitoring, are not as robust or feature-rich as QuickSight for complex data visualization and exploration. They are not optimized for this kind of search visualization.

Therefore, Kinesis Data Streams, OpenSearch Service, and QuickSight provide a fully managed, scalable, and AWS-native solution that best satisfies the finance company's need for real-time data ingestion, processing, search, and visualization capabilities.

Question: 969 **Exam Heist**

A company currently runs an on-premises application that usesASP.NET on Linux machines. The application is resource-intensive and serves customers directly.

The company wants to modernize the application to .NET. The company wants to run the application on containers and to scale based on Amazon CloudWatch metrics. The company also wants to reduce the time spent on operational maintenance activities.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use AWS App2Container to containerize the application. Use an AWS CloudFormation template to deploy the application to Amazon Elastic Container Service (Amazon ECS) on AWS Fargate.

B.Use AWS App2Container to containerize the application. Use an AWS CloudFormation template to deploy the application to Amazon Elastic Container Service (Amazon ECS) on Amazon EC2 instances.

C.Use AWS App Runner to containerize the application. Use App Runner to deploy the application to Amazon Elastic Container Service (Amazon ECS) on AWS Fargate.

D.Use AWS App Runner to containerize the application. Use App Runner to deploy the application to Amazon Elastic Kubernetes Service (Amazon EKS) on Amazon EC2 instances.

Answer: A

Explanation:

The correct answer is **A. Use AWS App2Container to containerize the application. Use an AWS CloudFormation template to deploy the application to Amazon Elastic Container Service (Amazon ECS) on AWS Fargate.**

Here's why:

App2Container: This service simplifies the process of containerizing existing applications, especially .NET applications running on Linux, fitting the company's modernization goal. It automates the creation of container images, Dockerfiles, and ECS task definitions. https://aws.amazon.com/app2container/

Amazon ECS on Fargate: Fargate is a serverless compute engine for ECS. This means the company doesn't need to manage the underlying EC2 instances, reducing operational overhead significantly. Fargate handles patching, scaling, and availability automatically. https://aws.amazon.com/fargate/

CloudFormation: Using a CloudFormation template allows for infrastructure as code (IaC), ensuring consistent and repeatable deployments. This is crucial for managing infrastructure at scale and reducing manual configuration errors. It allows for scaling based on CloudWatch metrics through auto-scaling groups configured within the template. https://aws.amazon.com/cloudformation/

Why other options are less suitable:

B. Amazon ECS on EC2 instances: While ECS on EC2 is a valid option, it increases operational overhead. The company would be responsible for managing the EC2 instances, including patching, scaling, and ensuring availability. This contradicts the requirement to reduce operational maintenance.

C. AWS App Runner to containerize the application. Use App Runner to deploy the application to Amazon Elastic Container Service (Amazon ECS) on AWS Fargate: AWS App Runner is a service that directly deploys containerized web applications and APIs from source code or a container image. While App Runner simplifies deployment, it is less flexible than ECS and CloudFormation for scaling and infrastructure management. Also, App Runner directly deploys the application, there isn't deployment to ECS on Fargate.

D. AWS App Runner to containerize the application. Use App Runner to deploy the application to Amazon Elastic Kubernetes Service (Amazon EKS) on Amazon EC2 instances: This option has both increased operational overhead, using EC2 instances for EKS, and is also trying to use AWS App Runner to deploy to

EKS. App Runner doesn't deploy to EKS, and using EKS on EC2 increases the operational overhead. EKS manages Kubernetes, requiring more expertise.

In summary, option A offers the best balance between modernization, scalability based on CloudWatch metrics, and minimal operational overhead by leveraging App2Container for containerization, ECS on Fargate for serverless compute, and CloudFormation for infrastructure as code.

Question: 970 **Exam Heist**

A company is designing a new internal web application in the AWS Cloud. The new application must securely retrieve and store multiple employee usernames and passwords from an AWS managed service.

Which solution will meet these requirements with the LEAST operational overhead?

A.Store the employee credentials in AWS Systems Manager Parameter Store. Use AWS CloudFormation and the BatchGetSecretValue API to retrieve usernames and passwords from Parameter Store.

B.Store the employee credentials in AWS Secrets Manager. Use AWS CloudFormation and AWS Batch with the BatchGetSecretValue API to retrieve the usernames and passwords from Secrets Manager.

C.Store the employee credentials in AWS Systems Manager Parameter Store. Use AWS CloudFormation and AWS Batch with the BatchGetSecretValue API to retrieve the usernames and passwords from Parameter Store.

D.Store the employee credentials in AWS Secrets Manager. Use AWS CloudFormation and the BatchGetSecretValue API to retrieve the usernames and passwords from Secrets Manager.

Answer: D

Explanation:

The most suitable solution for securely storing and retrieving employee credentials with the least operational overhead is to utilize AWS Secrets Manager in conjunction with AWS CloudFormation and the BatchGetSecretValue API.

Secrets Manager is designed explicitly for managing secrets like passwords, API keys, and database credentials. It offers built-in features for encryption, automatic rotation, and access control, minimizing the operational burden associated with managing secrets manually. Storing the credentials here inherently provides a higher level of security compared to Parameter Store (especially for sensitive data like passwords) because Secrets Manager is built specifically for that purpose.

CloudFormation enables infrastructure-as-code, which helps you define and provision resources in a predictable and repeatable manner. By integrating Secrets Manager with CloudFormation, you can automatically retrieve and configure your application with the stored employee credentials during deployment.

The BatchGetSecretValue API (or GetSecretValue if batch retrieval isn't necessary) allows CloudFormation to retrieve the usernames and passwords from Secrets Manager directly during the infrastructure deployment process. This streamlines the deployment and configuration, reduces manual intervention, and ensures consistency across environments.

Options A and C use Parameter Store, which while capable of storing secrets, is primarily designed for configuration data. Secrets Manager offers better security features specifically designed for secrets management, making it the preferred choice for this scenario. Furthermore, incorporating AWS Batch (as suggested in options B and C) introduces unnecessary complexity and overhead. AWS Batch is typically utilized for running batch computing workloads and does not directly contribute to secure retrieval of secrets during deployment. Using CloudFormation with the API call is sufficient for the needs of the application.

Therefore, storing the employee credentials in AWS Secrets Manager and retrieving them during deployment

using CloudFormation and the appropriate Secrets Manager API call provides the most secure and operationally efficient approach.

Supporting Links:

AWS Secrets Manager: https://aws.amazon.com/secrets-manager/
AWS CloudFormation: https://aws.amazon.com/cloudformation/
Secrets Manager GetSecretValue API: https://docs.aws.amazon.com/secretsmanager/latest/dev/reference_awscli.html#cli_reference_getsecretvalue
AWS Systems Manager Parameter Store: https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html

Question: 971 **Exam Heist**

A company that is in the ap-northeast-1 Region has a fleet of thousands of AWS Outposts servers. The company has deployed the servers at remote locations around the world. All the servers regularly download new software versions that consist of 100 files. There is significant latency before all servers run the new software versions.

The company must reduce the deployment latency for new software versions.

Which solution will meet this requirement with the LEAST operational overhead?

A.Create an Amazon S3 bucket in ap-northeast-1. Set up an Amazon CloudFront distribution in ap-northeast-1 that includes a CachingDisabled cache policy. Configure the S3 bucket as the origin. Download the software by using signed URLs.

B.Create an Amazon S3 bucket in ap-northeast-1. Create a second S3 bucket in the us-east-1 Region. Configure replication between the buckets. Set up an Amazon CloudFront distribution that uses ap-northeast-1 as the primary origin and us-east-1 as the secondary origin. Download the software by using signed URLs.

C.Create an Amazon S3 bucket in ap-northeast-1. Configure Amazon S3 Transfer Acceleration. Download the software by using the S3 Transfer Acceleration endpoint.

D.Create an Amazon S3 bucket in ap-northeast-1. Set up an Amazon CloudFront distribution. Configure the S3 bucket as the origin. Download the software by using signed URLs.

Answer: C

Explanation:

The correct answer is C. Here's why:

The primary goal is to reduce deployment latency for software updates across globally distributed AWS Outposts servers with minimal operational overhead.

Option C: S3 Transfer Acceleration is the most effective and least complex solution. S3 Transfer Acceleration utilizes globally distributed AWS edge locations to accelerate data transfer to an S3 bucket. When an Outpost server initiates a download using the Transfer Acceleration endpoint, the data is routed through the nearest edge location, which optimizes the network path and protocol for faster uploads to the S3 bucket in ap-northeast-1. This inherently accelerates the subsequent downloads by the Outposts. The operational overhead is relatively low; it simply involves enabling S3 Transfer Acceleration on the bucket and using the specific endpoint for data transfer.

Why other options are suboptimal:

Option A: Disabling caching in CloudFront defeats the purpose of using a CDN for distributing content closer to the Outposts, thus negating the benefit of reduced latency for repeated downloads.
Option B: Adding S3 replication to a second region introduces unnecessary complexity. While it provides redundancy, it does not inherently accelerate the initial download process for the globally distributed

Outposts, which is the key bottleneck to address. Also, failing over to us-east-1 when the primary origin is ap-northeast-1 will negatively impact the performance.

Option D: While CloudFront can help with content distribution, it still relies on the initial upload to the origin (S3 bucket) in ap-northeast-1. Without accelerating this initial upload, latency will remain a problem, especially considering the geographical distribution of Outposts. S3 Transfer Acceleration directly addresses the initial upload bottleneck. Also the CloudFront needs time to distribute the files to its edge locations globally.

In summary, S3 Transfer Acceleration optimizes the data transfer to the S3 bucket, reducing latency with minimal setup and management. This is the most efficient approach compared to alternatives that add complexity without directly addressing the core issue of initial data transfer speed.

Authoritative Links:

Amazon S3 Transfer Acceleration: https://aws.amazon.com/s3/transfer-acceleration/

Question: 972 **Exam Heist**

A company currently runs an on-premises stock trading application by using Microsoft Windows Server. The company wants to migrate the application to the AWS Cloud.

The company needs to design a highly available solution that provides low-latency access to block storage across multiple Availability Zones.

Which solution will meet these requirements with the LEAST implementation effort?

A.Configure a Windows Server cluster that spans two Availability Zones on Amazon EC2 instances. Install the application on both cluster nodes. Use Amazon FSx for Windows File Server as shared storage between the two cluster nodes.

B.Configure a Windows Server cluster that spans two Availability Zones on Amazon EC2 instances. Install the application on both cluster nodes. Use Amazon Elastic Block Store (Amazon EBS) General Purpose SSD (gp3) volumes as storage attached to the EC2 instances. Set up application-level replication to sync data from one EBS volume in one Availability Zone to another EBS volume in the second Availability Zone.

C.Deploy the application on Amazon EC2 instances in two Availability Zones. Configure one EC2 instance as active and the second EC2 instance in standby mode. Use an Amazon FSx for NetApp ONTAP Multi-AZ file system to access the data by using Internet Small Computer Systems Interface (iSCSI) protocol.

D.Deploy the application on Amazon EC2 instances in two Availability Zones. Configure one EC2 instance as active and the second EC2 instance in standby mode. Use Amazon Elastic Block Store (Amazon EBS) Provisioned IOPS SSD (io2) volumes as storage attached to the EC2 instances. Set up Amazon EBS level replication to sync data from one io2 volume in one Availability Zone to another io2 volume in the second Availability Zone.

Answer: A

Explanation:

The correct answer is **A**. Here's why:

High Availability: The requirement for high availability is best met by a Windows Server cluster spanning multiple Availability Zones (AZs). This ensures that if one AZ fails, the application can failover to the other node in the other AZ.

Low Latency: Amazon FSx for Windows File Server, especially when deployed in a Multi-AZ configuration, provides low-latency, shared file storage accessible from multiple EC2 instances. This meets the low-latency access requirement to block storage. FSx for Windows File Server also offers native SMB support, which is compatible with Windows Server.

Least Implementation Effort: FSx for Windows File Server handles replication and failover automatically,

reducing the implementation effort compared to manual data synchronization or application-level replication. Setting up a Windows Server cluster is a common practice, and using FSx simplifies the shared storage aspect.

Now let's examine why the other options are less suitable:

B: While using EBS is an option, setting up application-level replication introduces complexity and might not be as reliable or performant as a managed file system like FSx. EBS volumes themselves are not inherently shared across Availability Zones without replication solutions.

C: Using FSx for NetApp ONTAP with iSCSI adds unnecessary complexity. The native SMB protocol of FSx for Windows File Server is more aligned with a Windows Server environment, making it simpler to implement. Also, the active/standby configuration is not as highly available as a true cluster.

D: Using EBS io2 volumes and EBS-level replication is more complex than using a shared file system like FSx for Windows File Server. Manually managing EBS replication adds operational overhead. The active/standby setup also limits the application's ability to automatically recover from failures.

In summary, option A provides a combination of high availability, low latency, and least implementation effort by leveraging a Windows Server cluster with shared storage provided by Amazon FSx for Windows File Server.

Supporting Links:

Amazon FSx for Windows File Server: https://aws.amazon.com/fsx/windows/
Windows Server Failover Clustering: https://learn.microsoft.com/en-us/windows-server/failover-clustering/failover-clustering-overview

Question: 973 **Exam Heist**

A company is designing a web application with an internet-facing Application Load Balancer (ALB).

The company needs the ALB to receive HTTPS web traffic from the public internet. The ALB must send only HTTPS traffic to the web application servers hosted on the Amazon EC2 instances on port 443. The ALB must perform a health check of the web application servers over HTTPS on port 8443.

Which combination of configurations of the security group that is associated with the ALB will meet these requirements? (Choose three.)

A. Allow HTTPS inbound traffic from 0.0.0.0/0 for port 443.
B. Allow all outbound traffic to 0.0.0.0/0 for port 443.
C. Allow HTTPS outbound traffic to the web application instances for port 443.
D. Allow HTTPS inbound traffic from the web application instances for port 443.
E. Allow HTTPS outbound traffic to the web application instances for the health check on port 8443.
F. Allow HTTPS inbound traffic from the web application instances for the health check on port 8443.

Answer: ACE

Explanation:

The correct answer is ACE. Here's why:

A. Allow HTTPS inbound traffic from 0.0.0.0/0 for port 443: The ALB needs to accept HTTPS traffic from the public internet (0.0.0.0/0 represents all IP addresses). This is the entry point for users accessing the web application. Without this, the ALB would be inaccessible from the internet.

C. Allow HTTPS outbound traffic to the web application instances for port 443: The ALB needs to forward HTTPS traffic to the web application servers on port 443. This outbound rule allows the ALB to communicate with the backend instances and pass on the decrypted traffic.

E. Allow HTTPS outbound traffic to the web application instances for the health check on port 8443: The ALB performs health checks on the EC2 instances to ensure they are healthy and can receive traffic. These health checks occur over HTTPS on port 8443, as stated in the requirement. This outbound rule is essential for the ALB to monitor the health of the application servers.

Why other options are incorrect:

B. Allow all outbound traffic to 0.0.0.0/0 for port 443: This is overly permissive. The ALB only needs to communicate with the specific web application instances. Allowing outbound traffic to the entire internet on port 443 is a security risk.

D. Allow HTTPS inbound traffic from the web application instances for port 443: This is unnecessary for the ALB. The ALB initiates communication with the web application instances, not the other way around. The web application instances don't need to send HTTPS traffic back to the ALB on port 443.

F. Allow HTTPS inbound traffic from the web application instances for the health check on port 8443: As with option D, The ALB initiates health checks, not the instances. The ALB doesn't need to receive inbound traffic from the web application instances on port 8443.

Supporting Documentation:

Application Load Balancers:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html
Security Groups for Your VPC: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html

Question: 974 **Exam Heist**

A company hosts an application on AWS. The application gives users the ability to upload photos and store the photos in an Amazon S3 bucket. The company wants to use Amazon CloudFront and a custom domain name to upload the photo files to the S3 bucket in the eu-west-1 Region.

Which solution will meet these requirements? (Choose two.)

A.Use AWS Certificate Manager (ACM) to create a public certificate in the us-east-1 Region. Use the certificate in CloudFront.

B.Use AWS Certificate Manager (ACM) to create a public certificate in eu-west-1. Use the certificate in CloudFront.

C.Configure Amazon S3 to allow uploads from CloudFront. Configure S3 Transfer Acceleration.

D.Configure Amazon S3 to allow uploads from CloudFront origin access control (OAC).

E.Configure Amazon S3 to allow uploads from CloudFront. Configure an Amazon S3 website endpoint.

Answer: BD

Explanation:

Here's a detailed justification for why options B and D are the correct choices and why the others are incorrect.

Option B: Use AWS Certificate Manager (ACM) to create a public certificate in eu-west-1. Use the certificate in CloudFront.

This option is correct because to use a custom domain name with CloudFront, you need an SSL/TLS

certificate. ACM certificates are used to provide this encryption for secure HTTPS connections. While ACM certificates for CloudFront distributions can be in us-east-1, which is the global region for certain AWS services, this is only for certificates directly attached to CloudFront for viewer connections. In this scenario, where the application is hosted in eu-west-1 and you want to use a custom domain for accessing resources in that region via CloudFront, the certificate associated with the origin (S3 bucket in this case) should ideally be in the same region. The user uploads photos to the S3 bucket within the eu-west-1 region, and a certificate in the same region is the recommended approach.

Option D: Configure Amazon S3 to allow uploads from CloudFront origin access control (OAC).

This option is correct because CloudFront needs permission to upload objects to the S3 bucket. OAC is the recommended method for controlling access to S3 buckets from CloudFront. It enhances security compared to origin access identity (OAI). OAC allows you to restrict S3 bucket access solely to your CloudFront distribution. This ensures that users can only upload photos to S3 via CloudFront and not directly, which protects against bypassing security measures you may have put in place (like content moderation on the CloudFront distribution).

Why other options are incorrect:

A. Use AWS Certificate Manager (ACM) to create a public certificate in the us-east-1 Region. Use the certificate in CloudFront. While us-east-1 is used for certificates associated with the CloudFront distribution itself for viewer connections, the requirement here is to securely upload to S3 in eu-west-1 through CloudFront. Having the S3 origin's certificate in the same region, eu-west-1 (as in option B), is a more standard and arguably clearer architectural approach.

C. Configure Amazon S3 to allow uploads from CloudFront. Configure S3 Transfer Acceleration. S3 Transfer Acceleration speeds up data transfers between your users and your S3 bucket. While it can improve upload speeds, it doesn't directly address the requirement of using a custom domain name with CloudFront and doesn't secure the upload process itself. The key requirement is to allow only CloudFront to upload, and just enabling Transfer Acceleration doesn't achieve that.

E. Configure Amazon S3 to allow uploads from CloudFront. Configure an Amazon S3 website endpoint. Configuring an S3 website endpoint is usually for serving static content directly from S3. While you can technically upload to S3 through the S3 website endpoint using specific forms and methods, this isn't the standard recommended approach when you have CloudFront in the architecture, especially for direct user uploads. Moreover, the default S3 website endpoint doesn't support HTTPS with a custom domain without significant extra configuration (which the other options handle more directly).

In summary:

Options B and D together provide a solution that uses a certificate to securely upload data (HTTPS with a custom domain via CloudFront) and uses OAC to ensure that only the CloudFront distribution can upload the photos to the S3 bucket.

Authoritative Links:

AWS Certificate Manager: https://aws.amazon.com/certificate-manager/
CloudFront Origin Access Control (OAC):
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html
S3 Transfer Acceleration: https://aws.amazon.com/s3/transfer-acceleration/
Using HTTPS with CloudFront:
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-https.html

Question: 975 **Exam Heist**

A weather forecasting company collects temperature readings from various sensors on a continuous basis. An existing data ingestion process collects the readings and aggregates the readings into larger Apache Parquet files. Then the process encrypts the files by using client-side encryption with KMS managed keys (CSE-KMS). Finally, the process writes the files to an Amazon S3 bucket with separate prefixes for each calendar day.

The company wants to run occasional SQL queries on the data to take sample moving averages for a specific calendar day.

Which solution will meet these requirements MOST cost-effectively?

A.Configure Amazon Athena to read the encrypted files. Run SQL queries on the data directly in Amazon S3.

B.Use Amazon S3 Select to run SQL queries on the data directly in Amazon S3.

C.Configure Amazon Redshift to read the encrypted files. Use Redshift Spectrum and Redshift query editor v2 to run SQL queries on the data directly in Amazon S3.

D.Configure Amazon EMR Serverless to read the encrypted files. Use Apache SparkSQL to run SQL queries on the data directly in Amazon S3.

Answer: A

Explanation:

The most cost-effective solution for running occasional SQL queries on encrypted Parquet data in S3 is to use Amazon Athena.

Here's why:

Athena's Cost Efficiency: Athena is a serverless query service. You only pay for the queries you run, making it ideal for infrequent queries. Options C and D (Redshift and EMR Serverless) involve more significant overhead, as you're paying for a cluster or compute environment to be running, even when not actively querying. S3 Select (option B) has query cost and throughput limitations.

Direct Data Access: Athena directly queries data stored in S3, eliminating the need to load data into a database like Redshift.

Encryption Handling: Athena integrates with KMS to handle CSE-KMS encrypted data. You can configure Athena to use the same KMS key used to encrypt the files.

SQL Compatibility: Athena uses Presto, which is a distributed SQL query engine. It has great SQL compatibility.

Parquet Support: Athena natively supports Parquet files, allowing for efficient data processing due to Parquet's columnar storage format.

Simplicity: Compared to EMR Serverless, setting up and managing Athena is simpler. EMR Serverless is overkill for simple querying.

Therefore, by using Athena, the company can directly query its encrypted Parquet files in S3 for specific calendar days, calculate moving averages, and only pay for the queries executed, ensuring cost-effectiveness.

Authoritative Links:

Amazon Athena: https://aws.amazon.com/athena/
Amazon S3 Select: https://aws.amazon.com/s3/select/
Amazon Redshift Spectrum: https://aws.amazon.com/redshift/spectrum/
Amazon EMR Serverless: https://aws.amazon.com/emr/serverless/

Question: 976 **Exam Heist**

A company is implementing a new application on AWS. The company will run the application on multiple Amazon EC2 instances across multiple Availability Zones within multiple AWS Regions. The application will be available through the internet. Users will access the application from around the world.

The company wants to ensure that each user who accesses the application is sent to the EC2 instances that are closest to the user's location.

Which solution will meet these requirements?

A.Implement an Amazon Route 53 geolocation routing policy. Use an internet-facing Application Load Balancer to distribute the traffic across all Availability Zones within the same Region.

B.Implement an Amazon Route 53 geoproximity routing policy. Use an internet-facing Network Load Balancer to distribute the traffic across all Availability Zones within the same Region.

C.Implement an Amazon Route 53 multivalue answer routing policy. Use an internet-facing Application Load Balancer to distribute the traffic across all Availability Zones within the same Region.

D.Implement an Amazon Route 53 weighted routing policy. Use an internet-facing Network Load Balancer to distribute the traffic across all Availability Zones within the same Region.

Answer: B

Explanation:

The correct answer is B because it leverages Route 53 geoproximity routing and a Network Load Balancer (NLB) to achieve proximity-based routing to application instances across multiple AWS Regions.

Here's why:

Route 53 Geoproximity Routing: This policy routes traffic to your resources based on the geographic location of your users and your resources. It considers both latitude and longitude, allowing you to direct users to the nearest EC2 instances, thereby minimizing latency and improving user experience. This perfectly aligns with the requirement of sending users to the closest instances.
Network Load Balancer (NLB): NLBs are designed for high performance and low latency, making them ideal for applications where minimizing response time is crucial. NLBs operate at Layer 4 (TCP/UDP) and can handle millions of requests per second while maintaining ultra-low latencies. Furthermore, NLBs support static IP addresses per Availability Zone, essential for geoproximity routing where you need to advertise specific IPs for different regions.
Why other options are incorrect:

A (Geolocation Routing): Geolocation routing directs traffic based on pre-defined geographic regions (countries or continents). While it directs traffic based on location, it doesn't consider the precise distance like geoproximity. In the question's context, more precise routing based on distance is needed.
C (Multivalue Answer Routing): Multivalue answer routing returns multiple IP addresses for each request, allowing the client to choose one. While it provides high availability, it doesn't consider proximity.
D (Weighted Routing): Weighted routing distributes traffic based on weights you assign to each resource. This is useful for A/B testing or migrating traffic but doesn't consider the user's location.

In conclusion, Route 53 geoproximity routing in conjunction with an NLB provides the most effective solution for directing users to the nearest EC2 instances across multiple AWS Regions, ensuring low latency and optimal performance, especially when considering worldwide users.

Authoritative Links:

Route 53 Routing Policies
Network Load Balancer

Question: 977 **Exam Heist**

A financial services company plans to launch a new application on AWS to handle sensitive financial transactions. The company will deploy the application on Amazon EC2 instances. The company will use Amazon RDS for MySQL as the database. The company's security policies mandate that data must be encrypted at rest and in transit.

Which solution will meet these requirements with the LEAST operational overhead?

A.Configure encryption at rest for Amazon RDS for MySQL by using AWS KMS managed keys. Configure AWS Certificate Manager (ACM) SSL/TLS certificates for encryption in transit.

B.Configure encryption at rest for Amazon RDS for MySQL by using AWS KMS managed keys. Configure IPsec tunnels for encryption in transit.

C.Implement third-party application-level data encryption before storing data in Amazon RDS for MySQL. Configure AWS Certificate Manager (ACM) SSL/TLS certificates for encryption in transit.

D.Configure encryption at rest for Amazon RDS for MySQL by using AWS KMS managed keys. Configure a VPN connection to enable private connectivity to encrypt data in transit.

Answer: A

Explanation:

Here's a detailed justification for why option A is the best solution, focusing on minimizing operational overhead:

Option A: Using AWS KMS managed keys for RDS encryption at rest and ACM SSL/TLS certificates for encryption in transit is the correct answer.

Encryption at Rest: Amazon RDS natively supports encryption at rest using AWS Key Management Service (KMS). By using KMS managed keys, the company can avoid the operational burden of managing encryption keys themselves. AWS handles key generation, rotation, and storage, reducing administrative overhead.
Encryption in Transit: AWS Certificate Manager (ACM) simplifies the process of obtaining, managing, and deploying SSL/TLS certificates for use with AWS services and internal connected servers. Using ACM to generate and manage SSL/TLS certificates is the easiest way to encrypt connections to RDS, and requires no extra configuration in your EC2 instances.
Least Operational Overhead: Option A requires minimal configuration and management, reducing operational overhead. RDS encryption and ACM certificates are AWS managed services, simplifying the implementation and maintenance.

Why other options are less optimal:

Option B (IPsec tunnels): IPsec tunnels are complex to set up and maintain, adding significant operational overhead. While they provide encryption, they are typically used for site-to-site VPNs, which are not necessary for encrypting connections between EC2 instances and RDS within the same AWS region.
Option C (Third-party application-level encryption): Implementing application-level encryption adds significant development and operational overhead. The application needs to handle encryption and decryption, key management, and potential performance impacts. RDS already provides built-in encryption capabilities, making this approach redundant and more complex.
Option D (VPN Connection): VPN connection and tunnels are used to connect to on-prem or other external network and not needed for encryption between EC2 and RDS services within the same AWS region.

In conclusion, option A balances security with simplicity and minimizes operational overhead by leveraging AWS managed services for encryption at rest and in transit, making it the most efficient solution for the financial services company's requirements.

Authoritative Links:

RDS Encryption at Rest:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Overview.Encryption.html
AWS Certificate Manager (ACM): https://aws.amazon.com/certificate-manager/
Encrypting Connections to DB Instances:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/UsingWithRDS.SSL.html

Question: 978 **Exam Heist**

A company is migrating its on-premises Oracle database to an Amazon RDS for Oracle database. The company needs to retain data for 90 days to meet regulatory requirements. The company must also be able to restore the database to a specific point in time for up to 14 days.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create Amazon RDS automated backups. Set the retention period to 90 days.

B.Create an Amazon RDS manual snapshot every day. Delete manual snapshots that are older than 90 days.

C.Use the Amazon Aurora Clone feature for Oracle to create a point-in-time restore. Delete clones that are older than 90 days.

D.Create a backup plan that has a retention period of 90 days by using AWS Backup for Amazon RDS.

Answer: D

Explanation:

The correct answer is D because AWS Backup provides a centralized and automated way to manage backups and retention policies across AWS services, including RDS.

Here's a breakdown:

Requirement 1: 90-day data retention: AWS Backup allows defining a backup plan with a retention period of 90 days. This ensures that backups are stored for the required duration to meet regulatory requirements.
Requirement 2: Point-in-time restore for 14 days: RDS automated backups, when integrated with AWS Backup, inherently provide point-in-time recovery (PITR) capabilities within the defined retention period. Even though the total retention is 90 days, you can restore to any point within the last 14 days.
Least operational overhead: AWS Backup automates the backup process according to the defined schedule and retention policy. This eliminates the need for manual snapshot creation and deletion, reducing operational overhead.

Let's analyze why the other options are less suitable:

A: While RDS automated backups do offer PITR, extending the retention period to 90 days for automated backups might be more expensive than using AWS Backup. AWS Backup provides more granular control over backup schedules and storage tiers.
B: Creating manual snapshots every day and manually deleting them introduces significant operational overhead. This requires scripting and monitoring to ensure snapshots are created and deleted correctly, which is error-prone.
C: Amazon Aurora Clone is not applicable to RDS for Oracle, and it is for Aurora only, meaning it will not work in this case.

In summary, AWS Backup simplifies the backup management process and ensures compliance with retention requirements with the least operational overhead.

Relevant documentation:

AWS Backup: https://aws.amazon.com/backup/

Backing Up and Restoring Amazon RDS Databases:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_WorkingWithAutomatedBackups.html

Question: 979 **Exam Heist**

A company is developing a new application that uses a relational database to store user data and application configurations. The company expects the application to have steady user growth. The company expects the database usage to be variable and read-heavy, with occasional writes.

The company wants to cost-optimize the database solution. The company wants to use an AWS managed database solution that will provide the necessary performance.

Which solution will meet these requirements MOST cost-effectively?

A.Deploy the database on Amazon RDS. Use Provisioned IOPS SSD storage to ensure consistent performance for read and write operations.

B.Deploy the database on Amazon Aurora Serverless to automatically scale the database capacity based on actual usage to accommodate the workload.

C.Deploy the database on Amazon DynamoDB. Use on-demand capacity mode to automatically scale throughput to accommodate the workload.

D.Deploy the database on Amazon RDS. Use magnetic storage and use read replicas to accommodate the workload.

Answer: B

Explanation:

The most cost-effective solution for a read-heavy, variable workload relational database, with occasional writes, and steady user growth is **B. Deploy the database on Amazon Aurora Serverless.**

Here's why:

Aurora Serverless Cost Optimization: Aurora Serverless is designed to automatically scale compute resources up or down based on application needs. You pay only for the resources consumed. For a variable workload with occasional writes, this eliminates the cost of over-provisioning resources when the database isn't being heavily utilized. This is critical for cost optimization. https://aws.amazon.com/rds/aurora/serverless/

Read-Heavy Workload Suitability: Aurora, in general, is optimized for read performance. Aurora Serverless inherits these benefits.

Relational Database Requirement: The application requires a relational database. Aurora is a MySQL and PostgreSQL-compatible relational database.

Not A: Provisioned IOPS (PIOPS) storage is expensive and best suited for consistent, high-performance workloads, not variable ones. It wouldn't be cost-effective for a workload that often idles.

Not C: DynamoDB is a NoSQL database and therefore not suitable for an application that requires a relational database. While DynamoDB on-demand can scale, the database type is a mismatch.

Not D: Magnetic storage is the slowest and least performant storage option on RDS. While read replicas can help with read performance, magnetic storage would significantly bottleneck performance and is not ideal given the need for "necessary performance". While read replicas do help with read scaling, the base instance with Magnetic storage would not handle the workload effectively, and the magnetic storage itself is not performance-optimized. Aurora is more efficient at scaling and cost savings for variable workloads.

In summary, Aurora Serverless provides the necessary relational database compatibility, automatically scales to handle the variable workload, and optimizes costs by charging only for consumed resources, making it the

most cost-effective choice.

Question: 980 **Exam Heist**

A company hosts its application on several Amazon EC2 instances inside a VPC. The company creates a dedicated Amazon S3 bucket for each customer to store their relevant information in Amazon S3.

The company wants to ensure that the application running on EC2 instances can securely access only the S3 buckets that belong to the company's AWS account.

Which solution will meet these requirements with the LEAST operational overhead?

A.Create a gateway endpoint for Amazon S3 that is attached to the VPC. Update the IAM instance profile policy to provide access to only the specific buckets that the application needs.

B.Create a NAT gateway in a public subnet with a security group that allows access to only Amazon S3. Update the route tables to use the NAT Gateway.

C.Create a gateway endpoint for Amazon S3 that is attached to the VPUpdate the IAM instance profile policy with a Deny action and the following condition key:

```
{
    "StringNotEquals" : {
    "s3:ResourceAccount" : [ "CompanyAWSAcctNumber"]
}
```

D.Create a NAT Gateway in a public subnet. Update route tables to use the NAT Gateway. Assign bucket policies for all buckets with a Deny action and the following condition key:

```
{
    "StringNotEquals" : {
    "s3:ResourceAccount" : [ "CompanyAWSAcctNumber"]
}
```

Answer: C

Explanation:

Create a gateway endpoint for Amazon S3 that is attached to the VPUpdate the IAM instance profile policy with a Deny action and the following condition key.

Question: 981 **Exam Heist**

A company is building a cloud-based application on AWS that will handle sensitive customer data. The application uses Amazon RDS for the database, Amazon S3 for object storage, and S3 Event Notifications that invoke AWS Lambda for serverless processing.

The company uses AWS IAM Identity Center to manage user credentials. The development, testing, and operations teams need secure access to Amazon RDS and Amazon S3 while ensuring the confidentiality of sensitive customer data. The solution must comply with the principle of least privilege.

Which solution meets these requirements with the LEAST operational overhead?

A.Use IAM roles with least privilege to grant all the teams access. Assign IAM roles to each team with customized IAM policies defining specific permission for Amazon RDS and S3 object access based on team

responsibilities.

B.Enable IAM Identity Center with an Identity Center directory. Create and configure permission sets with granular access to Amazon RDS and Amazon S3. Assign all the teams to groups that have specific access with the permission sets.

C.Create individual IAM users for each member in all the teams with role-based permissions. Assign the IAM roles with predefined policies for RDS and S3 access to each user based on user needs. Implement IAM Access Analyzer for periodic credential evaluation.

D.Use AWS Organizations to create separate accounts for each team. Implement cross-account IAM roles with least privilege. Grant specific permission for RDS and S3 access based on team roles and responsibilities.

Answer: B

Explanation:

The correct answer is B. Here's why:

Option B leverages AWS IAM Identity Center (successor to AWS SSO), which is the most efficient and secure method for managing user access across multiple AWS accounts and applications, especially when combined with a central directory. Creating permission sets allows administrators to define granular, role-based access to AWS resources like RDS and S3, adhering to the principle of least privilege. Assigning teams to groups and then associating those groups with specific permission sets streamlines access management and reduces operational overhead significantly. IAM Identity Center also integrates well with existing identity providers (like Active Directory), further simplifying user management.

Option A, while using IAM roles with least privilege, involves managing roles and policies directly for each team. This approach becomes cumbersome and difficult to scale as the number of teams and their access requirements grow. IAM Identity Center provides centralized management, simplifying policy updates and access reviews.

Option C, creating individual IAM users, is against best practices. It introduces significant operational overhead for managing individual credentials and permissions. Moreover, it's harder to track and audit access when dealing with individual users instead of groups or roles. IAM Access Analyzer helps, but it's a reactive measure rather than a proactive, centralized solution.

Option D, using AWS Organizations with separate accounts, is overkill for simply managing access within a single account. While Organizations is useful for multi-account environments, it adds unnecessary complexity and operational overhead when the primary goal is to control access to RDS and S3 within a single AWS account. Cross-account IAM roles also introduce added complexity that IAM Identity Center avoids.

IAM Identity Center's centralized management capabilities, coupled with the ability to define granular permission sets, makes it the most scalable, secure, and least operationally intensive solution for managing team access to RDS and S3 while adhering to the principle of least privilege. It's specifically designed for scenarios where you have multiple users and groups requiring different levels of access to AWS resources.

Relevant links for further research:

AWS IAM Identity Center: https://aws.amazon.com/iam/identity-center/
IAM Best Practices: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html
Principle of Least Privilege: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html#grant-least-privilege

Question: 982 **Exam Heist**

A company has an Amazon S3 bucket that contains sensitive data files. The company has an application that runs on virtual machines in an on-premises data center. The company currently uses AWS IAM Identity Center.

The application requires temporary access to files in the S3 bucket. The company wants to grant the application secure access to the files in the S3 bucket.

Which solution will meet these requirements?

A.Create an S3 bucket policy that permits access to the bucket from the public IP address range of the company's on-premises data center.

B.Use IAM Roles Anywhere to obtain security credentials in IAM Identity Center that grant access to the S3 bucket. Configure the virtual machines to assume the role by using the AWS CLI.

C.Install the AWS CLI on the virtual machine. Configure the AWS CLI with access keys from an IAM user that has access to the bucket.

D.Create an IAM user and policy that grants access to the bucket. Store the access key and secret key for the IAM user in AWS Secrets Manager. Configure the application to retrieve the access key and secret key at startup.

Answer: B

Explanation:

The best solution is **B. Use IAM Roles Anywhere to obtain security credentials in IAM Identity Center that grant access to the S3 bucket. Configure the virtual machines to assume the role by using the AWS CLI.**

Here's why:

IAM Roles Anywhere: This feature allows on-premises applications to use IAM roles for temporary credentials. This eliminates the need to embed long-term credentials directly into the application or virtual machines. https://docs.aws.amazon.com/rolesanywhere/latest/userguide/

Security Best Practices: Storing IAM user credentials (access keys and secret keys) directly on virtual machines or within application code (as in options C and D) is a major security risk. If compromised, these credentials grant potentially broad, persistent access to AWS resources. IAM Roles Anywhere mitigates this risk by providing temporary, short-lived credentials.

IAM Identity Center Integration: The solution leverages the existing IAM Identity Center (formerly AWS SSO) which is a centralized identity management service. This ensures consistent and manageable access control across the AWS environment and on-premises resources.

Dynamic Access: The application uses the AWS CLI to assume the IAM role. The AWS CLI is designed to handle temporary credentials acquired through IAM roles or other authentication mechanisms.

Least Privilege: By using IAM roles, you can grant the application only the necessary permissions to access the specific S3 bucket and files it requires (principle of least privilege). A bucket policy can further restrict access based on the IAM role assumed.

Why other options are not ideal:

A: An S3 bucket policy based on the on-premises IP address range is less secure as the IP address can be spoofed or changed. It is not the recommended approach for secure access, especially for sensitive data.

C & D: Storing or embedding IAM user credentials on the instances directly or storing them in Secrets Manager and providing them to the VMs is not recommended for security reasons. If the instance is compromised, the long-term IAM user credentials are also compromised. This is not the best way to use Secrets Manager.

In conclusion, using IAM Roles Anywhere provides the most secure, scalable, and manageable solution to grant the on-premises application temporary access to the S3 bucket, while integrating seamlessly with the existing IAM Identity Center setup. It avoids the risks associated with managing long-term credentials.

Question: 983 **Exam Heist**

A company hosts its core network services, including directory services and DNS, in its on-premises data center. The data center is connected to the AWS Cloud using AWS Direct Connect (DX). Additional AWS accounts are planned that will require quick, cost-effective, and consistent access to these network services.

What should a solutions architect implement to meet these requirements with the LEAST amount of operational overhead?

A.Create a DX connection in each new account. Route the network traffic to the on-premises servers.

B.Configure VPC endpoints in the DX VPC for all required services. Route the network traffic to the on-premises servers.

C.Create a VPN connection between each new account and the DX VPRoute the network traffic to the on-premises servers.

D.Configure AWS Transit Gateway between the accounts. Assign DX to the transit gateway and route network traffic to the on-premises servers.

Answer: D

Explanation:

The correct answer is **D. Configure AWS Transit Gateway between the accounts. Assign DX to the transit gateway and route network traffic to the on-premises servers.**

Here's a detailed justification:

AWS Transit Gateway simplifies network architecture by acting as a central hub for routing traffic between VPCs and on-premises networks. Using Transit Gateway, you can create a single connection point to your on-premises network via Direct Connect (DX) and then share that connection with multiple AWS accounts and VPCs. This eliminates the need for separate DX connections or VPNs for each account, significantly reducing operational overhead and cost.

Option A is incorrect because creating a DX connection in each new account is expensive and complex to manage. Each connection would require individual configuration and management, adding significant operational overhead.

Option B is incorrect. VPC endpoints are used to securely access AWS services without traversing the public internet, but they are not relevant for routing traffic to on-premises resources. They don't facilitate communication between different AWS accounts and the on-premises data center.

Option C is incorrect because setting up individual VPN connections for each account is more operationally intensive compared to using Transit Gateway. VPN connections, while functional, don't scale as efficiently or provide the centralized management capabilities of Transit Gateway, resulting in higher overhead. Furthermore, VPNs might introduce more latency than Transit Gateway utilizing a DX connection.

Transit Gateway provides a scalable, centralized, and cost-effective solution. It integrates seamlessly with Direct Connect, making it easy to extend your on-premises network connectivity to multiple AWS accounts. Routing policies can be centrally managed within the Transit Gateway, ensuring consistent network access control across all connected VPCs. By associating the Direct Connect gateway to the Transit Gateway, all accounts can leverage the established DX connection to access on-premises resources, satisfying the requirements of quick, cost-effective, and consistent access with minimal operational effort.

Here are some authoritative links for further research:

AWS Transit Gateway: https://aws.amazon.com/transit-gateway/
AWS Direct Connect: https://aws.amazon.com/directconnect/
Transit Gateway Routing: https://docs.aws.amazon.com/vpc/latest/tgw/tgw-routing.html

Question: 984 **Exam Heist**

A company hosts its main public web application in one AWS Region across multiple Availability Zones. The application uses an Amazon EC2 Auto Scaling group and an Application Load Balancer (ALB).

A web development team needs a cost-optimized compute solution to improve the company's ability to serve dynamic content globally to millions of customers.

Which solution will meet these requirements?

A.Create an Amazon CloudFront distribution. Configure the existing ALB as the origin.

B.Use Amazon Route 53 to serve traffic to the ALB and EC2 instances based on the geographic location of each customer.

C.Create an Amazon S3 bucket with public read access enabled. Migrate the web application to the S3 bucket. Configure the S3 bucket for website hosting.

D.Use AWS Direct Connect to directly serve content from the web application to the location of each customer.

Answer: A

Explanation:

The best solution is A, creating an Amazon CloudFront distribution with the existing ALB as the origin. Here's why:

CloudFront is a Content Delivery Network (CDN) that caches content closer to users, reducing latency and improving performance for globally distributed users. This directly addresses the requirement of serving dynamic content to millions of customers globally. By using the existing ALB as the origin, CloudFront will pull dynamic content from the ALB when it's not already cached in its edge locations.

Option B (Route 53 geolocation routing) is less effective because it only routes users to the closest region, not the closest edge location. Route 53 doesn't cache content, so users will still experience latency related to reaching the regional ALB.

Option C (S3 bucket with website hosting) is not suitable for dynamic content. S3 is primarily for serving static content and while it can host a website, it doesn't handle the backend processing of dynamic elements. Migrating the entire web application to S3 would require significant re-architecting and won't inherently improve global performance for dynamic requests.

Option D (AWS Direct Connect) is for establishing a dedicated network connection between on-premises infrastructure and AWS, not for content delivery to millions of customers. It's more suited for hybrid cloud scenarios and is not relevant to serving dynamic content globally. It is also significantly more expensive than a CDN.

CloudFront's caching capabilities are key for cost optimization. By caching content at edge locations, it reduces the load on the origin server (ALB), potentially allowing for a smaller EC2 Auto Scaling group behind the ALB, leading to cost savings.

In summary, CloudFront offers the best balance of improved global performance, cost optimization, and minimal disruption to the existing application architecture by utilizing the current ALB as the origin, making it the ideal solution.

Authoritative links:

Amazon CloudFront: https://aws.amazon.com/cloudfront/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/

Question: 985 **Exam Heist**

A company stores user data in AWS. The data is used continuously with peak usage during business hours. Access patterns vary, with some data not being used for months at a time. A solutions architect must choose a cost-effective solution that maintains the highest level of durability while maintaining high availability.

Which storage solution meets these requirements?

A.Amazon S3 Standard
B.Amazon S3 Intelligent-Tiering
C.Amazon S3 Glacier Deep Archive
D.Amazon S3 One Zone-Infrequent Access (S3 One Zone-IA)

Answer: B

Explanation:

The optimal storage solution is Amazon S3 Intelligent-Tiering. Here's why:

The requirement is a cost-effective solution with high durability and high availability for data with variable access patterns, including infrequently accessed data. S3 Standard provides high durability and availability but isn't cost-effective for data accessed infrequently.

S3 Glacier Deep Archive is the cheapest option for long-term archival, but access times are slower (measured in hours), contradicting the requirement for continuous use and peak usage during business hours. It's intended for archival, not frequent retrieval.

S3 One Zone-IA is cheaper than S3 Standard-IA but stores data in a single Availability Zone, which compromises availability, especially when the goal is to maintain high availability. Its durability is also lower than other S3 storage classes.

S3 Intelligent-Tiering automatically moves data between frequent, infrequent, and archive access tiers based on access patterns, optimizing costs. When data is accessed frequently, it resides in a tier comparable to S3 Standard. When not accessed for a period, it transitions to infrequent access tiers like S3 Standard-IA or S3 Glacier Instant Retrieval, and ultimately, archive tiers, reducing storage costs significantly without retrieval fees. Because Amazon S3 automatically manages the tiers, it removes the manual effort typically involved in data lifecycle management. This achieves both cost-effectiveness and maintains high durability and availability due to S3's inherent design principles. It caters to the peak usage scenario without inflating costs during periods of infrequent access.

Authoritative Links:

Amazon S3 Storage Classes: https://aws.amazon.com/s3/storage-classes/
Amazon S3 Intelligent-Tiering: https://aws.amazon.com/s3/intelligent-tiering/

Question: 986 **Exam Heist**

A company is testing an application that runs on an Amazon EC2 Linux instance. A single 500 GB Amazon Elastic Block Store (Amazon EBS) General Purpose SSO (gp2) volume is attached to the EC2 instance.

The company will deploy the application on multiple EC2 instances in an Auto Scaling group. All instances require access to the data that is stored in the EBS volume. The company needs a highly available and resilient solution that does not introduce significant changes to the application's code.

Which solution will meet these requirements?

A.Provision an EC2 instance that uses NFS server software. Attach a single 500 GB gp2 EBS volume to the instance.

B.Provision an Amazon FSx for Windows File Server file system. Configure the file system as an SMB file store within a single Availability Zone.

C.Provision an EC2 instance with two 250 GB Provisioned IOPS SSD EBS volumes.

D.Provision an Amazon Elastic File System (Amazon EFS) file system. Configure the file system to use General Purpose performance mode.

Answer: D

Explanation:

The correct answer is D, provisioning an Amazon Elastic File System (EFS) file system configured for General Purpose performance mode. Here's why:

The core requirement is to provide a highly available and resilient shared storage solution that can be accessed by multiple EC2 instances within an Auto Scaling group without significant code changes.

Option A (NFS server on a single EC2 instance) presents a single point of failure. If the NFS server instance fails, the entire application's data access is disrupted. This violates the high availability requirement. Also, managing an NFS server adds operational overhead.

Option B (Amazon FSx for Windows File Server) is designed for Windows-based applications that require SMB protocol support. The question specifies a Linux EC2 instance, making FSx for Windows inappropriate.

Option C (Two Provisioned IOPS EBS volumes on a single EC2 instance) doesn't address the core requirement of shared storage accessible by multiple instances. EBS volumes are typically attached to a single instance at a time. While you could potentially configure some software-based RAID and network sharing, it adds complexity and negates the "no significant code changes" requirement. Moreover, managing the redundancy and replication across these volumes increases operational overhead.

Option D (Amazon EFS) is the optimal solution. EFS provides a fully managed, scalable, and highly available network file system. It can be concurrently accessed by multiple EC2 instances across multiple Availability Zones. Since the existing EBS volume is General Purpose (gp2) based, using EFS's General Purpose performance mode is a suitable replacement, minimizing performance disruption. EFS's distributed architecture inherently provides resilience and avoids a single point of failure. The instances within the Auto Scaling group can mount the EFS file system using the NFS protocol, requiring minimal configuration and no significant application code changes. EFS automatically scales capacity as needed, simplifying storage management.

Therefore, EFS directly addresses the shared storage, high availability, and minimal code change requirements.

Authoritative links:

Amazon EFS: https://aws.amazon.com/efs/
EBS vs. EFS: https://aws.amazon.com/premiumsupport/knowledge-center/ebs-vs-efs/

Question: 987 **Exam Heist**

A company recently launched a new application for its customers. The application runs on multiple Amazon EC2 instances across two Availability Zones. End users use TCP to communicate with the application.

The application must be highly available and must automatically scale as the number of users increases.

Which combination of steps will meet these requirements MOST cost-effectively? (Choose two.)

A.Add a Network Load Balancer in front of the EC2 instances.

B.Configure an Auto Scaling group for the EC2 instances.

C.Add an Application Load Balancer in front of the EC2 instances.

D.Manually add more EC2 instances for the application.

E.Add a Gateway Load Balancer in front of the EC2 instances.

Answer: AB

Explanation:

The most cost-effective solution for achieving high availability and automatic scaling for the application is a combination of a Network Load Balancer (NLB) and an Auto Scaling group.

A. Add a Network Load Balancer (NLB) in front of the EC2 instances:

An NLB is designed for high-performance and low-latency applications that use TCP or UDP protocols. Because the application communicates using TCP, an NLB is a suitable choice compared to an Application Load Balancer (ALB), which is intended for HTTP/HTTPS traffic and adds overhead. The NLB distributes traffic across multiple EC2 instances in different Availability Zones, enhancing availability and fault tolerance. If one instance fails, the NLB automatically redirects traffic to the remaining healthy instances. NLBs are also very efficient at handling fluctuating traffic volumes.
https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html

B. Configure an Auto Scaling group for the EC2 instances:

An Auto Scaling group automatically adjusts the number of EC2 instances based on demand. This ensures that the application can handle increased traffic without manual intervention. It scales out (adds more instances) when the load increases and scales in (removes instances) when the load decreases. This dynamic scaling ensures optimal resource utilization and cost efficiency. Moreover, Auto Scaling groups automatically replace unhealthy instances, contributing to high availability. It integrates seamlessly with the NLB, registering and deregistering instances as they are launched or terminated.
https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-group.html

Why other options are less suitable:

C. Add an Application Load Balancer (ALB) in front of the EC2 instances: While ALBs can also provide high availability and scaling, they are optimized for HTTP/HTTPS traffic. Using an ALB for a TCP-based application introduces unnecessary overhead and cost.

D. Manually add more EC2 instances for the application: Manual scaling is time-consuming, error-prone, and not cost-effective. It doesn't provide automatic fault tolerance and requires constant monitoring.

E. Add a Gateway Load Balancer (GWLB) in front of the EC2 instances: GWLB is designed for deploying and managing virtual appliances (like firewalls and intrusion detection systems), not for general application load balancing. It's not relevant for the described scenario and would be more complex and expensive than needed.

Therefore, the combination of an NLB and an Auto Scaling group is the most cost-effective approach to meet the application's requirements for high availability and automatic scaling for TCP traffic.

Question: 988 **Exam Heist**

A company is designing the architecture for a new mobile app that uses the AWS Cloud. The company uses organizational units (OUs) in AWS Organizations to manage its accounts. The company wants to tag Amazon EC2 instances with data sensitivity by using values of sensitive and nonsensitive. IAM identities must not be able to

delete a tag or create instances without a tag.

Which combination of steps will meet these requirements? (Choose two.)

A.In Organizations, create a new tag policy that specifies the data sensitivity tag key and the required values. Enforce the tag values for the EC2 instances. Attach the tag policy to the appropriate OU.

B.In Organizations, create a new service control policy (SCP) that specifies the data sensitivity tag key and the required tag values. Enforce the tag values for the EC2 instances. Attach the SCP to the appropriate OU.

C.Create a tag policy to deny running instances when a tag key is not specified. Create another tag policy that prevents identities from deleting tags. Attach the tag policies to the appropriate OU.

D.Create a service control policy (SCP) to deny creating instances when a tag key is not specified. Create another SCP that prevents identities from deleting tags. Attach the SCPs to the appropriate OU.

E.Create an AWS Config rule to check if EC2 instances use the data sensitivity tag and the specified values. Configure an AWS Lambda function to delete the resource if a noncompliant resource is found.

Answer: AD

Explanation:

Here's a detailed justification for why options A and D are the correct choices, and why the others are incorrect, including supporting information and authoritative links:

Why A is correct:

Tag policies within AWS Organizations allow you to standardize tags across all accounts within your organization. This approach centralizes tag governance and ensures consistency. By defining a tag policy that specifies the data sensitivity tag key and the allowed values (sensitive and nonsensitive), you enforce these values for EC2 instances. Attaching the tag policy to the relevant OU ensures that all accounts within that OU adhere to the tagging requirements. Tag policies can enforce required values. This addresses the requirement of instances having the correct tag values.

Why D is correct:

Service Control Policies (SCPs) provide centralized control over the AWS accounts in your organization. They define guardrails or boundaries for what IAM users and roles within member accounts can do. You can create an SCP that denies the creation of EC2 instances if the data sensitivity tag key is not specified. You can create another SCP to prevent IAM identities from deleting tags. By attaching these SCPs to the appropriate OU, you ensure that no one within the OU can bypass the tagging requirements. SCPs are powerful tools to enforce compliance at the organization level.

Why B is incorrect:

SCPs don't "enforce tag values" in the way tag policies do. They can prevent actions if a tag key is missing, but they can't directly force a tag to have a specific value. Tag policies are designed for value validation.

Why C is incorrect:

While tag policies can deny resources if a tag key is missing, creating two separate tag policies isn't as efficient as using a single tag policy to define the key and required values (option A). Also, tag policies are less effective than SCPs for preventing identities from deleting tags. SCPs provide a stronger, organization-wide preventative control.

Why E is incorrect:

AWS Config rules and Lambda functions for remediation are reactive controls. They detect and fix non-compliant resources after they have been created. The question specifies that IAM identities "must not be able to create instances without a tag." This requires a preventative control, which Config + Lambda does not provide. Also, while this approach can correct the problem, it is more complex than using SCPs and tag

policies.

In summary:

The best solution involves a combination of preventative controls. Tag policies enforce the presence and valid values of the tag (option A), while SCPs prevent the creation of resources if the required tag is missing and prevent tag deletion (option D). This layered approach provides comprehensive tag governance and ensures compliance across the organization.

Authoritative Links:

AWS Organizations Tag Policies:
https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_tag-policies.html
AWS Organizations Service Control Policies (SCPs):
https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scp.html
AWS Config: https://aws.amazon.com/config/

Question: 989 **Exam Heist**

A company runs database workloads on AWS that are the backend for the company's customer portals. The company runs a Multi-AZ database cluster on Amazon RDS for PostgreSQL.

The company needs to implement a 30-day backup retention policy. The company currently has both automated RDS backups and manual RDS backups. The company wants to maintain both types of existing RDS backups that are less than 30 days old.

Which solution will meet these requirements MOST cost-effectively?

A.Configure the RDS backup retention policy to 30 days for automated backups by using AWS Backup. Manually delete manual backups that are older than 30 days.

B.Disable RDS automated backups. Delete automated backups and manual backups that are older than 30 days. Configure the RDS backup retention policy to 30 days for automated backups.

C.Configure the RDS backup retention policy to 30 days for automated backups. Manually delete manual backups that are older than 30 days.

D.Disable RDS automated backups. Delete automated backups and manual backups that are older than 30 days automatically by using AWS CloudFormation. Configure the RDS backup retention policy to 30 days for automated backups.

Answer: C

Explanation:

The correct answer is **C. Configure the RDS backup retention policy to 30 days for automated backups. Manually delete manual backups that are older than 30 days.**

Here's a detailed justification:

Automated Backups: RDS provides automated backups that create a point-in-time recovery of your DB instance. Configuring the retention policy for automated backups directly within RDS is the most cost-effective and straightforward method. Setting this to 30 days ensures that RDS automatically manages these backups, deleting those older than 30 days.

Manual Backups: Manual backups, on the other hand, are retained indefinitely until manually deleted. Therefore, a separate process is needed to manage their retention. Manually deleting backups older than 30 days is the simplest and most direct way to handle them, particularly if the number of manual backups is manageable.

Cost-Effectiveness: AWS Backup is a service for centralizing and automating data protection across AWS services. While it can manage RDS backups, it introduces additional cost and complexity compared to using RDS's built-in retention policy for automated backups.

Why other options are incorrect:

Option A: It is same as option C and makes no sense.
Option B: Disabling RDS automated backups is not a good practice as they are essential for disaster recovery. The customer needs to maintain automated backups as mentioned in the prompt. The use of AWS CloudFormation for deleting backups (as in Option D) is overkill and adds unnecessary complexity when a simple manual deletion process is sufficient.

In summary, leveraging RDS's native backup retention policy for automated backups and manually deleting older manual backups provides the most cost-effective and efficient solution for maintaining a 30-day retention policy while preserving both types of existing RDS backups.

Supporting Links:

Amazon RDS Backups
AWS Backup

Question: 990 **Exam Heist**

A company is planning to migrate a legacy application to AWS. The application currently uses NFS to communicate to an on-premises storage solution to store application data. The application cannot be modified to use any other communication protocols other than NFS for this purpose.

Which storage solution should a solutions architect recommend for use after the migration?

A.AWS DataSync
B.Amazon Elastic Block Store (Amazon EBS)
C.Amazon Elastic File System (Amazon EFS)
D.Amazon EMR File System (Amazon EMRFS)

Answer: C

Explanation:

The correct answer is C, Amazon Elastic File System (Amazon EFS). Here's a detailed justification:

The legacy application relies on NFS (Network File System) for accessing its storage. This means that it requires a shared file system accessible over a network. Among the options, only Amazon EFS provides a fully managed NFS file system service in AWS.

Amazon EFS is designed to provide scalable, elastic, and serverless file storage that can be mounted on multiple EC2 instances or on-premises servers (via Direct Connect or VPN) simultaneously using the NFS protocol. This makes it an ideal solution for applications that require a shared file system and rely on NFS, fulfilling the application's requirement without code modification.

AWS DataSync (A) is a data transfer service used to move large amounts of data between on-premises storage and AWS storage services. It's not a storage solution itself and doesn't provide NFS access to applications after the migration.

Amazon Elastic Block Store (Amazon EBS) (B) provides block-level storage volumes for use with Amazon EC2 instances. While EBS volumes can be formatted with a file system, they are typically attached to a single EC2 instance at a time, making them unsuitable for applications requiring shared storage accessed through NFS.

Setting up NFS server on an EC2 instance backed by EBS is possible, but adds operational overhead and negates the fully-managed benefits of EFS.

Amazon EMR File System (Amazon EMRFS) (D) is a file system implementation used with Amazon EMR (Elastic MapReduce) for processing big data. It's optimized for Hadoop-based workloads and is not generally used as a general-purpose file system for other applications. EMRFS is also often used with Amazon S3 as its underlying storage, and not suited for direct NFS mounts.

Therefore, because the legacy application requires NFS and needs a fully-managed solution for shared storage, Amazon EFS is the most appropriate storage service for the migration.

Further Reading:

Amazon EFS: https://aws.amazon.com/efs/
NFS Protocol: https://en.wikipedia.org/wiki/Network_File_System

Question: 991 **Exam Heist**

A company uses GPS trackers to document the migration patterns of thousands of sea turtles. The trackers check every 5 minutes to see if a turtle has moved more than 100 yards (91.4 meters). If a turtle has moved, its tracker sends the new coordinates to a web application running on three Amazon EC2 instances that are in multiple Availability Zones in one AWS Region.

Recently, the web application was overwhelmed while processing an unexpected volume of tracker data. Data was lost with no way to replay the events. A solutions architect must prevent this problem from happening again and needs a solution with the least operational overhead.

What should the solutions architect do to meet these requirements?

A.Create an Amazon S3 bucket to store the data. Configure the application to scan for new data in the bucket for processing.

B.Create an Amazon API Gateway endpoint to handle transmitted location coordinates. Use an AWS Lambda function to process each item concurrently.

C.Create an Amazon Simple Queue Service (Amazon SQS) queue to store the incoming data. Configure the application to poll for new messages for processing.

D.Create an Amazon DynamoDB table to store transmitted location coordinates. Configure the application to query the table for new data for processing. Use TTL to remove data that has been processed.

Answer: C

Explanation:

The best solution is **C. Create an Amazon Simple Queue Service (Amazon SQS) queue to store the incoming data. Configure the application to poll for new messages for processing.**

Here's why:

Decoupling: SQS decouples the GPS trackers (data producers) from the EC2 instances (data consumers). This means the trackers can send data to the queue regardless of the EC2 instance's availability or processing capacity. This decoupling prevents the web application from being overwhelmed by surges in data volume.

Buffering: SQS acts as a buffer. When there's a sudden spike in tracker data, SQS will queue the messages, ensuring no data loss. The EC2 instances can then process the messages at their own pace. The queue provides a holding place when the processing system cannot keep up.

Reliability: SQS provides reliable message delivery. Messages are stored redundantly across multiple Availability Zones, ensuring durability and availability even in the event of hardware failures. This guarantees

that data isn't lost, addressing the critical requirement of avoiding data loss.

Scalability: SQS is highly scalable and can handle a massive volume of messages, adapting to the fluctuating needs of GPS tracker data input. It dynamically scales to accommodate the workload without requiring manual intervention.

Least Operational Overhead: SQS is a managed service, which means AWS handles the underlying infrastructure, patching, and scaling. This minimizes the operational overhead for the solutions architect, as they don't need to manage the queue infrastructure themselves.

Why other options are less suitable:

A. Amazon S3: S3 is primarily for object storage, not message queuing. Constantly scanning S3 for new data is inefficient and introduces unnecessary complexity. The web application would need to implement polling logic, which isn't ideal for real-time or near-real-time data processing.

B. Amazon API Gateway and Lambda: While API Gateway and Lambda can handle incoming requests, Lambda functions have execution time limits. Processing complex GPS data transformations within a single Lambda execution might exceed those limits. Furthermore, direct Lambda invocation might still overwhelm downstream systems if there is a spike. Lambda is also stateless, making handling complex processing workflows harder than using a dedicated queuing mechanism.

D. Amazon DynamoDB: DynamoDB is a NoSQL database, which can store location data. However, using it as a queue would be complex and inefficient. The application would need to implement its own queuing mechanism on top of DynamoDB, managing read/write consistency, polling, and deletion. Although TTL could be used, this is not the primary purpose of DynamoDB. This also puts a significant load on DynamoDB, especially if the application uses it for other purposes.

Authoritative links for further research:

Amazon SQS: https://aws.amazon.com/sqs/
Decoupling Applications with SQS: https://aws.amazon.com/blogs/architecture/queue-decoupling-pattern-for-serverless-architectures/

Question: 992 **Exam Heist**

A company's software development team needs an Amazon RDS Multi-AZ cluster. The RDS cluster will serve as a backend for a desktop client that is deployed on premises. The desktop client requires direct connectivity to the RDS cluster.

The company must give the development team the ability to connect to the cluster by using the client when the team is in the office.

Which solution provides the required connectivity MOST securely?

A.Create a VPC and two public subnets. Create the RDS cluster in the public subnets. Use AWS Site-to-Site VPN with a customer gateway in the company's office.

B.Create a VPC and two private subnets. Create the RDS cluster in the private subnets. Use AWS Site-to-Site VPN with a customer gateway in the company's office.

C.Create a VPC and two private subnets. Create the RDS cluster in the private subnets. Use RDS security groups to allow the company's office IP ranges to access the cluster.

D.Create a VPC and two public subnets. Create the RDS cluster in the public subnets. Create a cluster user for each developer. Use RDS security groups to allow the users to access the cluster.

Answer: B

Explanation:

The correct answer is B. Here's why:

Security Posture: Exposing an RDS database directly to the internet, as implied by options A and D using public subnets, is a significant security risk. Databases should reside in private subnets, isolated from direct internet access.

Connectivity Requirement: The desktop clients require direct connectivity from the company's office to the RDS cluster. This implies a need for a private, secure connection between the on-premises network and the AWS VPC.

AWS Site-to-Site VPN: This service establishes an encrypted tunnel between the on-premises network (company office) and the AWS VPC. This ensures all traffic between the desktop clients and the RDS cluster is secure. The customer gateway is configured on the company's side of the VPN connection, providing the necessary endpoint for the tunnel.

Private Subnets: Placing the RDS cluster within private subnets reinforces security. Resources in private subnets have no direct route to the internet. They can only be accessed from within the VPC or through services like NAT gateways or, as in this case, a VPN connection.

Option C's Flaw: While using private subnets is good, relying solely on RDS security groups with IP ranges is less secure than a VPN. IP ranges can be spoofed, and the traffic is not encrypted in transit without a VPN.

Option D's Flaw: Creating separate database users is a good practice for access control, but it does nothing to solve the core problem of insecure internet exposure using public subnets.

In summary, option B provides the most secure solution by isolating the RDS cluster in private subnets and establishing a secure, encrypted connection via AWS Site-to-Site VPN between the company's office and the VPC.

Supporting Links:

AWS Site-to-Site VPN: https://aws.amazon.com/vpn/site-to-site-vpn/
Amazon VPC: https://aws.amazon.com/vpc/
Amazon RDS Security: https://aws.amazon.com/rds/security/

Question: 993 **Exam Heist**

A solutions architect is creating an application that will handle batch processing of large amounts of data. The input data will be held in Amazon S3 and the output data will be stored in a different S3 bucket. For processing, the application will transfer the data over the network between multiple Amazon EC2 instances.

What should the solutions architect do to reduce the overall data transfer costs?

A.Place all the EC2 instances in an Auto Scaling group.
B.Place all the EC2 instances in the same AWS Region.
C.Place all the EC2 instances in the same Availability Zone.
D.Place all the EC2 instances in private subnets in multiple Availability Zones.

Answer: C

Explanation:

The most cost-effective approach to minimize data transfer costs when processing data between EC2 instances for batch processing, where data resides in S3, is to place all EC2 instances within the same Availability Zone (AZ).

Here's why:

Data Transfer Costs: AWS charges for data transferred between Availability Zones and Regions. Data transfer within the same AZ is typically free or significantly less expensive. Since the application involves substantial data transfer between EC2 instances during processing, minimizing cross-AZ data movement directly reduces costs.

Option A (Auto Scaling Group): While Auto Scaling is useful for managing EC2 instances and ensuring availability, it doesn't inherently reduce data transfer costs. The instances within the ASG could still be spread across multiple AZs, negating any potential savings.

Option B (Same Region): Keeping EC2 instances within the same Region is good practice for performance and reducing latency but doesn't eliminate data transfer costs. Inter-AZ data transfer charges still apply within a Region.

Option D (Private Subnets in Multiple AZs): While using private subnets adds a layer of security and deploying across multiple AZs enhances availability, it actively increases data transfer costs due to cross-AZ data movement. This option directly contradicts the goal of reducing data transfer expenses.

Therefore, keeping all EC2 instances in the same Availability Zone ensures that data transfer occurs within the AZ, thus avoiding costly cross-AZ charges. This is the most direct and effective method to reduce overall data transfer expenses for the given scenario.

Relevant AWS documentation:

Amazon EC2 Pricing: https://aws.amazon.com/ec2/pricing/ (Review the "Data Transfer" section for pricing details)
AWS Data Transfer Costs: https://aws.amazon.com/premiumsupport/knowledge-center/data-transfer-between-ec2-instances/

Question: 994 **Exam Heist**

A company hosts a multi-tier web application that uses an Amazon Aurora MySQL DB cluster for storage. The application tier is hosted on Amazon EC2 instances. The company's IT security guidelines mandate that the database credentials be encrypted and rotated every 14 days.

What should a solutions architect do to meet this requirement with the LEAST operational effort?

A.Create a new AWS Key Management Service (AWS KMS) encryption key. Use AWS Secrets Manager to create a new secret that uses the KMS key with the appropriate credentials. Associate the secret with the Aurora DB cluster. Configure a custom rotation period of 14 days.

B.Create two parameters in AWS Systems Manager Parameter Store: one for the user name as a string parameter and one that uses the SecureString type for the password. Select AWS Key Management Service (AWS KMS) encryption for the password parameter, and load these parameters in the application tier. Implement an AWS Lambda function that rotates the password every 14 days.

C.Store a file that contains the credentials in an AWS Key Management Service (AWS KMS) encrypted Amazon Elastic File System (Amazon EFS) file system. Mount the EFS file system in all EC2 instances of the application tier. Restrict the access to the file on the file system so that the application can read the file and that only super users can modify the file. Implement an AWS Lambda function that rotates the key in Aurora every 14 days and writes new credentials into the file.

D.Store a file that contains the credentials in an AWS Key Management Service (AWS KMS) encrypted Amazon S3 bucket that the application uses to load the credentials. Download the file to the application regularly to ensure that the correct credentials are used. Implement an AWS Lambda function that rotates the Aurora credentials every 14 days and uploads these credentials to the file in the S3 bucket.

Answer: A

Explanation:

The best solution is A because it leverages AWS Secrets Manager, designed specifically for managing, rotating, and retrieving secrets like database credentials. Secrets Manager integrates directly with Aurora and simplifies the rotation process.

Here's a detailed justification:

AWS Secrets Manager for Secret Management: Secrets Manager centralizes secrets, reducing the risk of hardcoding them in application code or configuration files, which is a security best practice. https://aws.amazon.com/secrets-manager/
KMS Encryption: Using KMS to encrypt the secrets adds an extra layer of security. The secrets are encrypted at rest and in transit when retrieved. https://aws.amazon.com/kms/
Aurora Integration: Secrets Manager integrates directly with Aurora, simplifying credential management and rotation. You can associate the secret with the Aurora cluster, and Aurora will automatically use the credentials from the secret. https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotating-secrets-aurora.html
Automated Rotation: Secrets Manager supports automated secret rotation. This means that you can configure Secrets Manager to automatically rotate the database credentials according to your schedule (in this case, every 14 days). This reduces the operational burden and ensures that the credentials are always up-to-date.
Least Operational Effort: By using Secrets Manager's built-in rotation capabilities, you avoid the need to write and maintain custom Lambda functions for secret rotation. This simplifies the solution and reduces the risk of errors.

Why other options are less suitable:

Option B (SSM Parameter Store): While SSM Parameter Store can store secrets, Secrets Manager is a better choice because it has built-in secret rotation capabilities. Implementing a Lambda function for rotation adds complexity.
Option C (EFS with KMS): Storing credentials in a file on EFS and mounting it on EC2 instances is less secure and more complex than using Secrets Manager. Managing file access and ensuring proper encryption adds overhead.
Option D (S3 with KMS): Similar to Option C, storing credentials in an S3 bucket and regularly downloading them to the application is less secure and more complex than using Secrets Manager. It introduces latency in credential updates and poses a risk of stale credentials.

Question: 995 **Exam Heist**

A streaming media company is rebuilding its infrastructure to accommodate increasing demand for video content that users consume daily.

The company needs to process terabyte-sized videos to block some content in the videos. Video processing can take up to 20 minutes.

The company needs a solution that will scale with demand and remain cost-effective.

Which solution will meet these requirements?

A.Use AWS Lambda functions to process videos. Store video metadata in Amazon DynamoDB. Store video content in Amazon S3 Intelligent-Tiering.

B.Use Amazon Elastic Container Service (Amazon ECS) and AWS Fargate to implement microservices to process videos. Store video metadata in Amazon Aurora. Store video content in Amazon S3 Intelligent-Tiering.

C.Use Amazon EC2 instances in an Auto Scaling group behind an Application Load Balancer (ALB) to process videos. Store video content in Amazon S3 Standard. Use Amazon Simple Queue Service (Amazon SQS) for queuing and to decouple processing tasks.

D.Deploy a containerized video processing application on Amazon Elastic Kubernetes Service (Amazon EKS) on Amazon EC2. Store video metadata in Amazon RDS in a single Availability Zone. Store video content in Amazon S3 Glacier Deep Archive.

Answer: B

Explanation:

The best solution is B because it offers a scalable, cost-effective, and robust approach for processing large video files.

Here's a breakdown:

Amazon ECS with Fargate: ECS, particularly with Fargate, provides a managed container orchestration service. Fargate eliminates the need to manage underlying EC2 instances, simplifying operations and auto-scaling the resources required for processing videos based on demand. This aligns with the requirement for scalability.

https://aws.amazon.com/ecs/fargate/
Microservices Architecture: Breaking down the video processing workflow into microservices allows for independent scaling of different parts of the processing pipeline. This offers better resource utilization and fault isolation compared to a monolithic application.
Amazon Aurora: Aurora is a MySQL and PostgreSQL-compatible relational database engine that combines the speed and availability of high-end commercial databases with the simplicity and cost-effectiveness of open-source databases. It's well-suited for storing video metadata due to its scalability and reliability.

https://aws.amazon.com/rds/aurora/
Amazon S3 Intelligent-Tiering: S3 Intelligent-Tiering automatically moves data to the most cost-effective access tier based on access patterns without performance impact or operational overhead. Given the large size of the videos and the likely variability in access frequency, this is a cost-effective choice.

https://aws.amazon.com/s3/storage-classes/intelligent-tiering/

Why other options are less suitable:

A (Lambda): AWS Lambda functions have execution time limits (maximum 15 minutes). Processing videos that can take up to 20 minutes exceeds this limit, making Lambda unsuitable.
C (EC2 Auto Scaling with SQS): While EC2 Auto Scaling provides scalability, it involves more operational overhead compared to Fargate. Managing EC2 instances (patching, scaling policies, etc.) increases the management burden. Also, S3 Standard might not be as cost-effective as S3 Intelligent-Tiering for infrequently accessed video content.
D (EKS): EKS is a powerful container orchestration service, but it adds significant operational complexity, and it is probably overkill for this situation. It is not cost effective compared to using ECS with Fargate. Storing video metadata in Amazon RDS in a single Availability Zone provides limited availability and is not best practice. S3 Glacier Deep Archive is designed for very long-term archival and retrieval of data, which may not be the most efficient storage class for this use case.

In summary, the combination of ECS with Fargate, Aurora for metadata, and S3 Intelligent-Tiering creates a scalable, cost-effective, and manageable solution that addresses all the requirements, particularly the video processing time and increasing demand.

Question: 996 **Exam Heist**

A company runs an on-premises application on a Kubernetes cluster. The company recently added millions of new customers. The company's existing on-premises infrastructure is unable to handle the large number of new

customers. The company needs to migrate the on-premises application to the AWS Cloud.

The company will migrate to an Amazon Elastic Kubernetes Service (Amazon EKS) cluster. The company does not want to manage the underlying compute infrastructure for the new architecture on AWS.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use a self-managed node to supply compute capacity. Deploy the application to the new EKS cluster.

B.Use managed node groups to supply compute capacity. Deploy the application to the new EKS cluster.

C.Use AWS Fargate to supply compute capacity. Create a Fargate profile. Use the Fargate profile to deploy the application.

D.Use managed node groups with Karpenter to supply compute capacity. Deploy the application to the new EKS cluster.

Answer: C

Explanation:

The correct answer is C because it offers the least operational overhead while fulfilling the requirements of migrating the Kubernetes application to AWS without managing the underlying compute infrastructure. AWS Fargate is a serverless compute engine for containers that works with EKS. This means the company doesn't need to provision, scale, or manage EC2 instances for the Kubernetes worker nodes.

Option A is incorrect as self-managed nodes require manual provisioning, scaling, patching, and management of EC2 instances, increasing operational overhead.

Option B, using managed node groups, simplifies node management compared to self-managed nodes, but still involves some degree of responsibility for the underlying EC2 instances, such as scaling and patching the operating system.

Option D, combining managed node groups with Karpenter, automates node provisioning and scaling more dynamically than managed node groups alone. However, it still relies on EC2 instances and introduces the complexity of managing Karpenter itself, increasing operational overhead compared to Fargate.

Fargate eliminates the need to manage any EC2 instances, aligning with the requirement of minimal operational overhead. The company creates a Fargate profile, which specifies which pods should run on Fargate based on namespace and selectors. EKS then automatically provisions and manages the necessary compute resources when pods matching the profile are deployed. Thus, Fargate presents the most serverless, hands-off approach.

For further reading:

AWS Fargate: https://aws.amazon.com/fargate/
Amazon EKS: https://aws.amazon.com/eks/
EKS Fargate Profiles: https://docs.aws.amazon.com/eks/latest/userguide/fargate-profile.html

Question: 997 **Exam Heist**

A company is launching a new application that requires a structured database to store user profiles, application settings, and transactional data. The database must be scalable with application traffic and must offer backups.

Which solution will meet these requirements MOST cost-effectively?

A.Deploy a self-managed database on Amazon EC2 instances by using open source software. Use Spot Instances for cost optimization. Configure automated backups to Amazon S3.

B.Use Amazon RDS. Use on-demand capacity mode for the database with General Purpose SSD storage. Configure automatic backups with a retention period of 7 days.

C.Use Amazon Aurora Serverless for the database. Use serverless capacity scaling. Configure automated backups to Amazon S3.

D.Deploy a self-managed NoSQL database on Amazon EC2 instances. Use Reserved Instances for cost optimization. Configure automated backups directly to Amazon S3 Glacier Flexible Retrieval.

Answer: C

Explanation:

The most cost-effective solution is C, using Amazon Aurora Serverless. Aurora Serverless automatically scales database capacity based on application needs, eliminating the need to provision and manage database instances. This eliminates the cost of idle capacity during periods of low activity. The automated backup feature of Aurora provides data protection without extra effort.

Option A involves managing a database on EC2, which requires manual patching, scaling, and backup configuration, increasing operational overhead and costs. While Spot Instances can be cheaper, their availability is unpredictable, risking application downtime.

Option B, Amazon RDS with on-demand capacity, although easier to manage than self-managed EC2 instances, doesn't scale to zero like Aurora Serverless. It incurs costs even during periods of inactivity.

Option D suggests using a NoSQL database, which doesn't fit the requirement for a structured database for user profiles and transactional data. Using Reserved Instances provides cost savings only if database utilization is consistently high. Backing up to S3 Glacier Flexible Retrieval would be a more expensive option than just using Aurora Serverless for this use case.

Therefore, Aurora Serverless offers the best balance of scalability, cost optimization, automated backups, and suitability for structured data, making it the most cost-effective solution.

Relevant Links:

Amazon Aurora Serverless v2
Amazon RDS
Amazon EC2 Spot Instances

Question: 998 **Exam Heist**

A company runs its legacy web application on AWS. The web application server runs on an Amazon EC2 instance in the public subnet of a VPC. The web application server collects images from customers and stores the image files in a locally attached Amazon Elastic Block Store (Amazon EBS) volume. The image files are uploaded every night to an Amazon S3 bucket for backup.

A solutions architect discovers that the image files are being uploaded to Amazon S3 through the public endpoint. The solutions architect needs to ensure that traffic to Amazon S3 does not use the public endpoint.

Which solution will meet these requirements?

A.Create a gateway VPC endpoint for the S3 bucket that has the necessary permissions for the VPC. Configure the subnet route table to use the gateway VPC endpoint.

B.Move the S3 bucket inside the VPC. Configure the subnet route table to access the S3 bucket through private IP addresses.

C.Create an Amazon S3 access point for the Amazon EC2 instance inside the VPConfigure the web application to upload by using the Amazon S3 access point.

D.Configure an AWS Direct Connect connection between the VPC that has the Amazon EC2 instance and Amazon S3 to provide a dedicated network path.

Answer: A

Explanation:

The correct answer is A: Create a gateway VPC endpoint for the S3 bucket that has the necessary permissions for the VPC. Configure the subnet route table to use the gateway VPC endpoint.

Here's why:

A gateway VPC endpoint allows resources within a VPC to privately access S3 without traversing the public internet. This aligns directly with the requirement to avoid using the public endpoint for S3 traffic. By creating a gateway endpoint, traffic destined for S3 from the EC2 instance within the VPC will be routed through the AWS network, remaining private and secure. The route table configuration ensures that any traffic destined for S3 from the specified subnet utilizes this endpoint. This approach is cost-effective and straightforward for enabling private connectivity to S3.

Option B is incorrect because S3 buckets are not placed "inside" VPCs. S3 is a global service. While you can restrict access to an S3 bucket from a specific VPC, you can't physically relocate it.

Option C is incorrect because an S3 access point manages access to data in S3 buckets. It doesn't inherently provide private connectivity. While useful for other access control scenarios, it doesn't address the primary requirement of avoiding the public internet.

Option D is incorrect because AWS Direct Connect is for establishing a dedicated network connection between your on-premises infrastructure and AWS. This is overkill for the requirement of simply keeping S3 traffic within the AWS network. It's far more expensive and complex than using a VPC endpoint. Additionally, the EC2 instance is already within AWS, so Direct Connect does not apply.

Therefore, option A offers the most efficient and cost-effective solution to meet the requirements.

Refer to the AWS documentation for more information on VPC endpoints:

VPC Endpoints
Gateway VPC Endpoints

Question: 999 **Exam Heist**

A company is creating a prototype of an ecommerce website on AWS. The website consists of an Application Load Balancer, an Auto Scaling group of Amazon EC2 instances for web servers, and an Amazon RDS for MySQL DB instance that runs with the Single-AZ configuration.

The website is slow to respond during searches of the product catalog. The product catalog is a group of tables in the MySQL database that the company does not update frequently. A solutions architect has determined that the CPU utilization on the DB instance is high when product catalog searches occur.

What should the solutions architect recommend to improve the performance of the website during searches of the product catalog?

A.Migrate the product catalog to an Amazon Redshift database. Use the COPY command to load the product catalog tables.

B.Implement an Amazon ElastiCache for Redis cluster to cache the product catalog. Use lazy loading to populate the cache.

C.Add an additional scaling policy to the Auto Scaling group to launch additional EC2 instances when database response is slow.

D.Turn on the Multi-AZ configuration for the DB instance. Configure the EC2 instances to throttle the product catalog queries that are sent to the database.

Answer: B

Explanation:

The correct answer is B: Implement an Amazon ElastiCache for Redis cluster to cache the product catalog. Use lazy loading to populate the cache.

Here's why:

The problem is high CPU utilization on the RDS for MySQL DB instance during product catalog searches, indicating the database is the bottleneck. Caching the catalog data is the most effective way to alleviate this.

Option B suggests using Amazon ElastiCache for Redis, which is an in-memory data store ideal for caching frequently accessed, relatively static data like a product catalog. Redis offers extremely fast read times compared to querying a database.

Lazy loading ensures the cache is populated only when data is requested, reducing the initial load time and cache misses. Instead of loading everything at once, each item is loaded into the cache only when it's first accessed. This approach optimizes resource utilization and provides an immediate improvement in performance.

Option A (Migrate to Redshift) is not ideal for this use case. Amazon Redshift is designed for large-scale data warehousing and analytics, not for serving frequent, low-latency requests like product catalog searches. The overhead of data transfer using the COPY command is also not ideal for this scenario.

Option C (Adding EC2 instances) addresses the wrong bottleneck. The issue is database CPU, not web server capacity. Adding more EC2 instances will only increase the load on the already overloaded database.

Option D (Multi-AZ and throttling) improves availability, not performance. Multi-AZ provides failover capability but doesn't reduce the load on the primary database instance during normal operation. Throttling queries will worsen the user experience by slowing down responses further.

Caching with ElastiCache and lazy loading directly addresses the database bottleneck and provides a significant performance boost for product catalog searches.

Further Reading:

Amazon ElastiCache: https://aws.amazon.com/elasticache/
Caching Strategies: https://aws.amazon.com/caching/
Redis: https://redis.io/

Question: 1000 **Exam Heist**

A company currently stores 5 TB of data in on-premises block storage systems. The company's current storage solution provides limited space for additional data. The company runs applications on premises that must be able to retrieve frequently accessed data with low latency. The company requires a cloud-based storage solution.

Which solution will meet these requirements with the MOST operational efficiency?

A.Use Amazon S3 File Gateway. Integrate S3 File Gateway with the on-premises applications to store and directly retrieve files by using the SMB file system.

B.Use an AWS Storage Gateway Volume Gateway with cached volumes as iSCSI targets.

C.Use an AWS Storage Gateway Volume Gateway with stored volumes as iSCSI targets.

D.Use an AWS Storage Gateway Tape Gateway. Integrate Tape Gateway with the on-premises applications to store virtual tapes in Amazon S3.

Answer: B

Explanation:

Here's a detailed justification for why option B is the best solution, along with supporting explanations and resources:

The company needs a cloud-based storage solution for 5 TB of data, requires low latency access to frequently accessed data, and aims for operational efficiency.

Option B (AWS Storage Gateway Volume Gateway with cached volumes): This option provides a local cache of frequently accessed data on the on-premises environment while storing the entire dataset in Amazon S3. When an application requests data, Volume Gateway first checks the local cache. If the data is present (cache hit), it's served with low latency. If not (cache miss), it's retrieved from S3, written to the cache, and then served to the application. This approach perfectly balances local performance with cloud storage capacity. Cached volumes only store a subset of your data locally.

Option A (Amazon S3 File Gateway): File Gateway is designed for storing files as objects in S3 using protocols like NFS and SMB. While it's suitable for file-based workloads, it might not provide the low-latency access required for block storage applications that expect consistent sub-millisecond response times. Although it caches recently used data locally, it's generally optimized for throughput over immediate latency for random read/write operations, thus not optimal.

Option C (AWS Storage Gateway Volume Gateway with stored volumes): Stored volumes store the entire dataset on-premises and asynchronously back it up to S3. While it offers local performance, it doesn't directly leverage cloud storage for the majority of the data. This approach negates the need for additional space, since the company already has a on-premise solution with limited storage space. Moreover, this solution does not help free up on-premise storage.

Option D (AWS Storage Gateway Tape Gateway): Tape Gateway is designed for archival and backup purposes, emulating a tape library. It is unsuitable for frequently accessed data that requires low latency.

Why Option B is the most operationally efficient:

Reduced on-premises storage footprint: Only frequently accessed data is stored locally, minimizing the need for large on-premises storage arrays.

Automated data tiering: Volume Gateway automatically manages the cache, moving frequently accessed data to local storage and less frequently accessed data to S3, reducing manual intervention.

Centralized management: AWS Storage Gateway is a managed service that simplifies storage management, monitoring, and backup tasks.

Cost optimization: Cloud storage (S3) is generally more cost-effective for long-term storage compared to on-premises block storage.

In summary, AWS Storage Gateway Volume Gateway with cached volumes provides the best combination of low-latency access, cloud-based storage capacity, and operational efficiency, addressing the company's specific requirements.

Authoritative Links:

AWS Storage Gateway: https://aws.amazon.com/storagegateway/
Cached Volumes: https://docs.aws.amazon.com/storagegateway/latest/userguide/cached-volumes.html
Volume Gateway:
https://docs.aws.amazon.com/storagegateway/latest/userguide/WhatIsStorageGateway.html

Question: 1001 **Exam Heist**

A company operates a food delivery service. Because of recent growth, the company's order processing system is

experiencing scaling problems during peak traffic hours. The current architecture includes Amazon EC2 instances in an Auto Scaling group that collect orders from an application. A second group of EC2 instances in an Auto Scaling group fulfills the orders.

The order collection process occurs quickly, but the order fulfillment process can take longer. Data must not be lost because of a scaling event.

A solutions architect must ensure that the order collection process and the order fulfillment process can both scale adequately during peak traffic hours.

Which solution will meet these requirements?

A.Use Amazon CloudWatch to monitor the CPUUtilization metric for each instance in both Auto Scaling groups. Configure each Auto Scaling group's minimum capacity to meet its peak workload value.

B.Use Amazon CloudWatch to monitor the CPUUtilization metric for each instance in both Auto Scaling groups. Configure a CloudWatch alarm to invoke an Amazon Simple Notification Service (Amazon SNS) topic to create additional Auto Scaling groups on demand.

C.Provision two Amazon Simple Queue Service (Amazon SQS) queues. Use one SQS queue for order collection. Use the second SQS queue for order fulfillment. Configure the EC2 instances to poll their respective queues. Scale the Auto Scaling groups based on notifications that the queues send.

D.Provision two Amazon Simple Queue Service (Amazon SQS) queues. Use one SQS queue for order collection. Use the second SQS queue for order fulfillment. Configure the EC2 instances to poll their respective queues. Scale the Auto Scaling groups based on the number of messages in each queue.

Answer: D

Explanation:

The correct solution is **D**, which involves using Amazon SQS queues for both order collection and fulfillment, and scaling Auto Scaling groups based on the number of messages in each queue.

Here's why:

Decoupling: SQS decouples the order collection process from the order fulfillment process. This means the order collection EC2 instances can quickly add orders to the queue without waiting for fulfillment, preventing bottlenecks during peak hours. https://aws.amazon.com/sqs/

Asynchronous Processing: SQS allows for asynchronous processing. Orders are placed in the queue and fulfilled later by the fulfillment EC2 instances. This is crucial since order fulfillment takes longer than order collection.

Scalability: Both Auto Scaling groups can scale independently based on the number of messages in their respective SQS queues. If the order collection queue has a large backlog, the order collection Auto Scaling group can scale up to handle the load. Similarly, the fulfillment Auto Scaling group can scale based on the number of messages in the fulfillment queue. https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-target.html

Data Durability: SQS is a fully managed message queuing service. It provides durable storage of messages until they are processed. This guarantees that no data is lost during scaling events. https://aws.amazon.com/sqs/features/

Reliable Scaling Trigger: Scaling based on the number of messages in the queue is a direct indicator of backlog and processing needs. It's a more relevant scaling trigger than CPU utilization, which might not accurately reflect the queue's workload.

Why other options are incorrect:

A and B: Scaling based only on CPU utilization might not be effective because the order collection process is quick and may not necessarily increase CPU load significantly, even during peak times. Option B suggests

creating new ASGs via SNS which is an unconventional and unnecessary approach.
C: Scaling based on notifications that the queues send is not standard. Auto Scaling scales most effectively by monitoring metrics and the queue depth is a more accurate metric than general notifications.

In summary, using SQS and scaling based on queue depth ensures that the order collection and fulfillment processes are decoupled, scalable, and data is not lost during peak hours.

Question: 1002 **Exam Heist**

An online gaming company is transitioning user data storage to Amazon DynamoDB to support the company's growing user base. The current architecture includes DynamoDB tables that contain user profiles, achievements, and in-game transactions.

The company needs to design a robust, continuously available, and resilient DynamoDB architecture to maintain a seamless gaming experience for users.

Which solution will meet these requirements MOST cost-effectively?

A.Create DynamoDB tables in a single AWS Region. Use on-demand capacity mode. Use global tables to replicate data across multiple Regions.

B.Use DynamoDB Accelerator (DAX) to cache frequently accessed data. Deploy tables in a single AWS Region and enable auto scaling. Configure Cross-Region Replication manually to additional Regions.

C.Create DynamoDB tables in multiple AWS Regions. Use on-demand capacity mode. Use DynamoDB Streams for Cross-Region Replication between Regions.

D.Use DynamoDB global tables for automatic multi-Region replication. Deploy tables in multiple AWS Regions. Use provisioned capacity mode. Enable auto scaling.

Answer: D

Explanation:

The correct answer is D. Here's why:

High Availability and Resilience: DynamoDB Global Tables inherently provide high availability and resilience by replicating data across multiple AWS Regions. This means that if one region experiences an outage, the application can failover to another region with minimal disruption.

Seamless Gaming Experience: Automatic multi-Region replication ensures that user data is synchronized across all regions, providing a consistent and seamless gaming experience regardless of the user's location.

Cost-Effectiveness: While on-demand capacity mode (options A and C) might seem appealing for unpredictable workloads, provisioned capacity mode with auto-scaling (option D) can be more cost-effective in the long run if the workload has predictable patterns or if you are diligent about setting up proper auto-scaling rules. DynamoDB auto-scaling dynamically adjusts provisioned capacity based on actual workload, optimizing cost without sacrificing performance.

Global Tables Superiority: Although option B uses DAX for caching and auto scaling, and option C attempts to replicate DynamoDB using DynamoDB Streams, these approaches require more manual configuration and are not as robust or efficient as using DynamoDB Global Tables. DynamoDB streams would require custom code to handle conflict resolution and data consistency, while Global Tables are designed to handle this out-of-the-box. Manual cross-region replication is complex and error-prone.

DynamoDB Global Tables Efficiency: Global tables provide very low latency access to data for a global user base by allowing them to connect to the nearest AWS region with their replicated data.

Multi-Region Deployment Necessity: The prompt states that the company needs to design a robust,

continuously available, and resilient DynamoDB architecture, which inherently implies the use of multiple regions for fault tolerance. Therefore, options that only use a single region are not suitable.

In summary, using DynamoDB Global Tables with provisioned capacity mode and auto-scaling offers the best combination of high availability, resilience, automatic replication, and cost-effectiveness for a global online gaming company.

Further Research:

Amazon DynamoDB Global Tables
Amazon DynamoDB Auto Scaling
Amazon DynamoDB pricing

Question: 1003 **Exam Heist**

A company runs its media rendering application on premises. The company wants to reduce storage costs and has moved all data to Amazon S3. The on-premises rendering application needs low-latency access to storage.

The company needs to design a storage solution for the application. The storage solution must maintain the desired application performance.

Which storage solution will meet these requirements in the MOST cost-effective way?

A.Use Mountpoint for Amazon S3 to access the data in Amazon S3 for the on-premises application.

B.Configure an Amazon S3 File Gateway to provide storage for the on-premises application.

C.Copy the data from Amazon S3 to Amazon FSx for Windows File Server. Configure an Amazon FSx File Gateway to provide storage for the on-premises application.

D.Configure an on-premises file server. Use the Amazon S3 API to connect to S3 storage. Configure the application to access the storage from the on-premises file server.

Answer: B

Explanation:

The optimal solution is **B. Configure an Amazon S3 File Gateway to provide storage for the on-premises application.**

Here's why:

S3 File Gateway: This service provides a local cache for frequently accessed data stored in S3. This local cache allows the on-premises rendering application to access data with low latency, meeting the application performance requirement. The File Gateway acts as a bridge, allowing on-premises applications to access S3 data as if it were a local file system.

Cost-effectiveness: S3 File Gateway is cost-effective because it only caches frequently accessed data locally. This minimizes the need for large, expensive local storage. The bulk of the data remains in the cost-optimized S3 storage tier.

Mountpoint for Amazon S3 (Option A): While Mountpoint allows direct access to S3, it does not provide local caching. This can lead to higher latency for frequently accessed data compared to S3 File Gateway. This makes it less suitable for applications requiring low latency.

Amazon FSx for Windows File Server (Option C): This solution involves copying data from S3 to FSx, which incurs data transfer costs and increases storage costs due to FSx's higher storage prices. Adding an FSx File Gateway adds unnecessary complexity and cost. It's a more expensive option than S3 File Gateway for simply providing low-latency access to data stored in S3.

On-premises file server with S3 API (Option D): This approach requires managing and maintaining a local file server, adding operational overhead. Furthermore, direct S3 API calls from the application may not provide the low-latency access the application needs.

In summary, the S3 File Gateway provides a cost-effective balance between low-latency access for on-premises applications and leveraging the scalability and cost-effectiveness of S3. It minimizes the need for expensive local storage by caching frequently accessed data.

Authoritative Links:

AWS Storage Gateway - S3 File Gateway: https://aws.amazon.com/storagegateway/file-gateway/
Mountpoint for Amazon S3: https://aws.amazon.com/blogs/aws/mountpoint-for-amazon-s3-a-high-throughput-file-client/

Question: 1004 **Exam Heist**

A company hosts its enterprise resource planning (ERP) system in the us-east-1 Region. The system runs on Amazon EC2 instances. Customers use a public API that is hosted on the EC2 instances to exchange information with the ERP system. International customers report slow API response times from their data centers.

Which solution will improve response times for the international customers MOST cost-effectively?

A.Create an AWS Direct Connect connection that has a public virtual interface (VIF) to provide connectivity from each customer's data center to us-east-1. Route customer API requests by using a Direct Connect gateway to the ERP system API.

B.Set up an Amazon CloudFront distribution in front of the API. Configure the CachingOptimized managed cache policy to provide improved cache efficiency.

C.Set up AWS Global Accelerator. Configure listeners for the necessary ports. Configure endpoint groups for the appropriate Regions to distribute traffic. Create an endpoint in the group for the API.

D.Use AWS Site-to-Site VPN to establish dedicated VPN tunnels between Regions and customer networks. Route traffic to the API over the VPN connections.

Answer: B

Explanation:

The correct answer is **B. Set up an Amazon CloudFront distribution in front of the API. Configure the CachingOptimized managed cache policy to provide improved cache efficiency.**

Here's why:

CloudFront's Global Reach: CloudFront is a content delivery network (CDN) with edge locations distributed globally. By caching API responses closer to international customers, it significantly reduces latency and improves response times. This is a core function of CDNs.
Cost-Effectiveness: CloudFront is generally more cost-effective than options like Direct Connect or Global Accelerator for this specific use case. It leverages caching to reduce the load on the origin EC2 instances, potentially lowering compute costs.
CachingOptimized Policy: The CachingOptimized managed cache policy is designed to improve cache efficiency, balancing cache hit ratio and time-to-live (TTL) to optimize content delivery. This is suitable because API responses, at least some, can often be cached.

Why other options are less suitable:

A. AWS Direct Connect: Direct Connect provides dedicated network connections, which are expensive and primarily used for hybrid cloud scenarios where high bandwidth and consistent network performance are crucial between a customer's infrastructure and AWS, not for delivering API responses to multiple customers.

Setting up Direct Connect for each customer is neither scalable nor cost-effective.
C. AWS Global Accelerator: Global Accelerator improves availability and performance for applications with users worldwide by routing traffic to the optimal endpoint (Region). While it can improve performance, it mainly optimizes TCP/UDP traffic and doesn't inherently cache content like CloudFront. It is less cost-effective than CloudFront for this specific caching scenario, and its primary benefit lies in failover and traffic optimization across Regions rather than caching closer to users.
D. AWS Site-to-Site VPN: Setting up VPN tunnels to each customer's network is complex, expensive, and creates unnecessary overhead. VPNs are designed for secure connectivity between networks, not for delivering API responses. It does not provide caching benefits.

In summary, CloudFront leverages caching and a global network to improve API response times for international customers most cost-effectively.

Supporting Documentation:

Amazon CloudFront: https://aws.amazon.com/cloudfront/
CloudFront Managed Cache Policies:
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-managed-cache-policies.html
AWS Global Accelerator: https://aws.amazon.com/global-accelerator/
AWS Direct Connect: https://aws.amazon.com/directconnect/

Question: 1005 **Exam Heist**

A company tracks customer satisfaction by using surveys that the company hosts on its website. The surveys sometimes reach thousands of customers every hour. Survey results are currently sent in email messages to the company so company employees can manually review results and assess customer sentiment.

The company wants to automate the customer survey process. Survey results must be available for the previous 12 months.

Which solution will meet these requirements in the MOST scalable way?

A.Send the survey results data to an Amazon API Gateway endpoint that is connected to an Amazon Simple Queue Service (Amazon SQS) queue. Create an AWS Lambda function to poll the SQS queue, call Amazon Comprehend for sentiment analysis, and save the results to an Amazon DynamoDB table. Set the TTL for all records to 365 days in the future.

B.Send the survey results data to an API that is running on an Amazon EC2 instance. Configure the API to store the survey results as a new record in an Amazon DynamoDB table, call Amazon Comprehend for sentiment analysis, and save the results in a second DynamoDB table. Set the TTL for all records to 365 days in the future.

C.Write the survey results data to an Amazon S3 bucket. Use S3 Event Notifications to invoke an AWS Lambda function to read the data and call Amazon Rekognition for sentiment analysis. Store the sentiment analysis results in a second S3 bucket. Use S3 lifecycle policies on each bucket to expire objects after 365 days.

D.Send the survey results data to an Amazon API Gateway endpoint that is connected to an Amazon Simple Queue Service (Amazon SQS) queue. Configure the SQS queue to invoke an AWS Lambda function that calls Amazon Lex for sentiment analysis and saves the results to an Amazon DynamoDB table. Set the TTL for all records to 365 days in the future.

Answer: A

Explanation:

Option A provides the most scalable and cost-effective solution for automating the customer survey process while meeting the given requirements.

Here's why:

Scalability: Amazon API Gateway and Amazon SQS are highly scalable services that can handle thousands of

requests per hour without manual intervention. API Gateway handles the initial influx of requests, while SQS acts as a buffer, decoupling the survey submission from the processing of results. This prevents any potential overload on the backend systems.
Sentiment Analysis: Amazon Comprehend is specifically designed for natural language processing tasks, including sentiment analysis. It offers accurate and efficient analysis of the survey results. Amazon Rekognition (option C) is for image and video analysis, and Amazon Lex (option D) is for building conversational interfaces, neither of which are appropriate for this scenario.
Data Storage: Amazon DynamoDB is a NoSQL database that offers high performance and scalability. It's well-suited for storing the survey results and sentiment analysis data. The TTL (Time To Live) attribute in DynamoDB is an efficient and automatic way to ensure data is only stored for the required 12 months (365 days).
Lambda Integration: AWS Lambda allows for serverless execution of code. Using Lambda to poll the SQS queue, process survey results, and save data to DynamoDB ensures efficient resource utilization and automatic scaling based on demand.
Decoupling: Using SQS decouples the survey submission (via API Gateway) from the sentiment analysis and data storage (performed by Lambda and DynamoDB). This makes the solution more resilient and able to handle traffic spikes.

Option B is less scalable as it relies on a single EC2 instance to handle API requests, potentially creating a bottleneck. It also introduces the overhead of managing an EC2 instance.

Option C uses S3 for sentiment analysis, which is incorrect. Rekognition is for image and video analysis, not text-based sentiment analysis.

Option D uses Amazon Lex, which is for building conversational interfaces, not sentiment analysis of survey data. Lex is not appropriate for analyzing text data.

Authoritative Links:

Amazon API Gateway: https://aws.amazon.com/api-gateway/
Amazon SQS: https://aws.amazon.com/sqs/
AWS Lambda: https://aws.amazon.com/lambda/
Amazon Comprehend: https://aws.amazon.com/comprehend/
Amazon DynamoDB: https://aws.amazon.com/dynamodb/
DynamoDB TTL: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html

Question: 1006 **Exam Heist**

A company uses AWS Systems Manager for routine management and patching of Amazon EC2 instances. The EC2 instances are in an IP address type target group behind an Application Load Balancer (ALB).

New security protocols require the company to remove EC2 instances from service during a patch. When the company attempts to follow the security protocol during the next patch, the company receives errors during the patching window.

Which combination of solutions will resolve the errors? (Choose two.)

A.Change the target type of the target group from IP address type to instance type.

B.Continue to use the existing Systems Manager document without changes because it is already optimized to handle instances that are in an IP address type target group behind an ALB.

C.Implement the AWSEC2-PatchLoadBalanacerInstance Systems Manager Automation document to manage the patching process.

D.Use Systems Manager Maintenance Windows to automatically remove the instances from service to patch the instances.

E.Configure Systems Manager State Manager to remove the instances from service and manage the patching

schedule. Use ALB health checks to re-route traffic.

Answer: CD

Explanation:

Let's break down why options C and D are the correct choices, and why the others are not.

The core issue is the company needs to patch EC2 instances behind an ALB while adhering to a new security protocol that requires instances to be removed from service during the patching process. The errors they are experiencing likely stem from interrupting the ALB's health checks when the instance is taken offline for patching. The ALB sees an unhealthy instance and might try to send traffic to it, or declare it out of service prematurely if the drain process isn't handled gracefully.

Why C is correct:

AWSEC2-PatchLoadBalancerInstance is a pre-built AWS Systems Manager Automation document specifically designed to handle patching EC2 instances behind an ALB. It orchestrates the process of taking an instance out of service (by deregistering it from the target group), patching it, and then putting it back in service (by re-registering it). This ensures minimal disruption to traffic flow.

Why D is correct:

Systems Manager Maintenance Windows provide a scheduled time frame for performing tasks like patching. Crucially, you can configure tasks within a Maintenance Window to execute the AWSEC2-PatchLoadBalancerInstance Automation document described above, thus automating the entire process of removing instances, patching them, and returning them to service within a defined timeframe. Also, Maintenance Windows allow you to configure pre- and post-tasks, so we can easily orchestrate the deregistration and registration of our instance.

Why A is incorrect:

Changing the target type of the target group doesn't address the problem of patching. Whether the target type is IP address or instance ID, the underlying issue of ALB health checks failing during patching remains. It's more about how the patching process interacts with the load balancer.

Why B is incorrect:

The existing Systems Manager document isn't optimized to handle instances behind an ALB if it doesn't include steps to deregister and reregister instances. The prompt states that the company is experiencing errors when trying to patch based on the security protocol.

Why E is incorrect:

State Manager is for maintaining a desired configuration state and isn't the appropriate tool for orchestrating patching behind an ALB. While State Manager could periodically run a patching document, it doesn't inherently handle the graceful removal and re-addition of instances to the ALB target group during the patching process. Furthermore, the state manager is not suited for temporary task.

In summary, the correct solution combines the automation provided by AWSEC2-PatchLoadBalancerInstance and the scheduling and orchestration capabilities of Maintenance Windows to ensure patching is performed gracefully with minimal impact on application availability.

Authoritative Links:

AWS Systems Manager Automation: https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-automation.html

AWSEC2-PatchLoadBalancerInstance Document: Look this document up within the AWS Management Console under Systems Manager > Automation > Owned By Amazon
AWS Systems Manager Maintenance Windows: https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-maintenance.html

Question: 1007 **Exam Heist**

A medical company wants to perform transformations on a large amount of clinical trial data that comes from several customers. The company must extract the data from a relational database that contains the customer data. Then the company will transform the data by using a series of complex rules. The company will load the data to Amazon S3 when the transformations are complete.

All data must be encrypted where it is processed before the company stores the data in Amazon S3. All data must be encrypted by using customer-specific keys.

Which solution will meet these requirements with the LEAST amount of operational effort?

A.Create one AWS Glue job for each customer. Attach a security configuration to each job that uses server-side encryption with Amazon S3 managed keys (SSE-S3) to encrypt the data.

B.Create one Amazon EMR cluster for each customer. Attach a security configuration to each cluster that uses client-side encryption with a custom client-side root key (CSE-Custom) to encrypt the data.

C.Create one AWS Glue job for each customer. Attach a security configuration to each job that uses client-side encryption with AWS KMS managed keys (CSE-KMS) to encrypt the data.

D.Create one Amazon EMR cluster for each customer. Attach a security configuration to each cluster that uses server-side encryption with AWS KMS keys (SSE-KMS) to encrypt the data.

Answer: C

Explanation:

The correct answer is C because it offers the least operational overhead while meeting all security and functional requirements. Here's why:

Data Extraction and Transformation: AWS Glue is designed for ETL (Extract, Transform, Load) operations. It provides a serverless, fully managed environment for running data transformation jobs, which perfectly aligns with the requirement to transform clinical trial data. Amazon EMR, while capable of ETL, is more suited for big data processing and analytics, making it more complex and operationally heavier for simple data transformations.

Customer-Specific Encryption: The requirement of customer-specific keys necessitates the use of KMS (Key Management Service). CSE (Client-Side Encryption) allows the company to manage the encryption process within their environment, using KMS keys they control. This allows for granular control over encryption for each customer. Server-Side Encryption (SSE) options, like SSE-S3, use a single key managed by AWS S3, which is insufficient for the customer-specific encryption mandate. CSE-Custom requires managing the encryption process completely outside AWS KMS, adding operational burden.

Encryption in Processing: CSE-KMS allows data to be encrypted before it's written to S3, ensuring encryption throughout the processing phase. Server-side encryption only protects data at rest in S3. The question specifically states data must be encrypted where it is processed, so options A and D are not suitable.

Least Operational Effort: Creating individual Glue jobs for each customer is operationally simpler than managing individual EMR clusters. Glue is serverless and managed, while EMR requires cluster provisioning, management, and scaling. Using AWS KMS also reduces the burden of managing encryption keys outside AWS infrastructure.

Security Configuration in Glue: Glue's security configurations enable associating KMS keys to each job for

customer-specific encryption. By attaching a different security configuration (and hence KMS key) to each Glue job, the company can ensure each customer's data is encrypted with their dedicated key.

In summary, option C using AWS Glue with client-side encryption using KMS keys provides a managed, secure, and scalable solution to meet all the requirements with the least operational effort.

Further research:

AWS Glue: https://aws.amazon.com/glue/
AWS KMS: https://aws.amazon.com/kms/
Encryption in Amazon S3: https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingEncryption.html
AWS Glue Security Configurations: https://docs.aws.amazon.com/glue/latest/dg/encryption-security-configuration.html

Question: 1008 **Exam Heist**

A company hosts a website analytics application on a single Amazon EC2 On-Demand Instance. The analytics application is highly resilient and is designed to run in stateless mode.

The company notices that the application is showing signs of performance degradation during busy times and is presenting 5xx errors. The company needs to make the application scale seamlessly.

Which solution will meet these requirements MOST cost-effectively?

A.Create an Amazon Machine Image (AMI) of the web application. Use the AMI to launch a second EC2 On-Demand Instance. Use an Application Load Balancer to distribute the load across the two EC2 instances.

B.Create an Amazon Machine Image (AMI) of the web application. Use the AMI to launch a second EC2 On-Demand Instance. Use Amazon Route 53 weighted routing to distribute the load across the two EC2 instances.

C.Create an AWS Lambda function to stop the EC2 instance and change the instance type. Create an Amazon CloudWatch alarm to invoke the Lambda function when CPU utilization is more than 75%.

D.Create an Amazon Machine Image (AMI) of the web application. Apply the AMI to a launch template. Create an Auto Scaling group that includes the launch template. Configure the launch template to use a Spot Fleet. Attach an Application Load Balancer to the Auto Scaling group.

Answer: D

Explanation:

The correct answer is **D** because it provides the most cost-effective and scalable solution for the described scenario.

Here's a breakdown:

Requirement of Seamless Scaling: The company needs the application to scale automatically based on demand. Auto Scaling groups (ASGs) are designed precisely for this purpose. They automatically launch or terminate EC2 instances based on defined metrics (like CPU utilization).

Cost-Effectiveness: Spot Instances offer significant cost savings compared to On-Demand Instances. By configuring the launch template with Spot Fleet, the ASG can leverage these cheaper instances.

Stateless Application: The fact that the application is stateless is crucial. This allows instances to be launched and terminated without data loss or impact on application functionality.

Load Balancing: The Application Load Balancer (ALB) distributes incoming traffic evenly across the healthy EC2 instances within the ASG, preventing any single instance from becoming a bottleneck.

AMIs and Launch Templates: Creating an AMI allows for consistent deployments. The launch template

defines the configuration of each instance launched by the ASG (AMI ID, instance type, security groups, etc.).

Why other options are less suitable:

A & B: Manually creating a second EC2 instance and using either an ALB or Route 53 weighted routing provides redundancy but doesn't scale automatically. It requires manual intervention to add or remove instances as demand fluctuates, which isn't seamless.
C: While this option attempts to address performance by changing the instance type, it's not ideal for several reasons. First, stopping and resizing an instance is disruptive. Second, it doesn't provide true scaling – it only makes the existing instance larger. Third, it doesn't address the need for distributing the load across multiple instances. The stopping of an instance is also a bad practice.

In summary: Option D utilizes Auto Scaling with Spot Instances and an ALB to achieve automatic scaling, cost optimization, and load distribution, effectively addressing the company's requirements in the most cost-effective manner.

Authoritative Links:

Amazon EC2 Auto Scaling: https://aws.amazon.com/autoscaling/
Amazon EC2 Spot Instances: https://aws.amazon.com/ec2/spot/
Application Load Balancer: https://aws.amazon.com/elasticloadbalancing/application-load-balancer/
Launch Templates: https://docs.aws.amazon.com/autoscaling/ec2/userguide/launch-templates.html

Question: 1009 **Exam Heist**

A company runs an environment where data is stored in an Amazon S3 bucket. The objects are accessed frequently throughout the day. The company has strict da ta encryption requirements for data that is stored in the S3 bucket. The company currently uses AWS Key Management Service (AWS KMS) for encryption.

The company wants to optimize costs associated with encrypting S3 objects without making additional calls to AWS KMS.

Which solution will meet these requirements?

A.Use server-side encryption with Amazon S3 managed keys (SSE-S3).
B.Use an S3 Bucket Key for server-side encryption with AWS KMS keys (SSE-KMS) on the new objects.
C.Use client-side encryption with AWS KMS customer managed keys.
D.Use server-side encryption with customer-provided keys (SSE-C) stored in AWS KMS.

Answer: B

Explanation:

The company needs to encrypt data at rest in S3 using KMS, but wants to optimize costs. The core issue is reducing the number of KMS calls since these calls incur charges.

Option A (SSE-S3) doesn't satisfy the requirement of using AWS KMS for encryption. While it offers server-side encryption, it uses S3-managed keys, not KMS keys. Therefore, it is not the correct solution.

Option B (SSE-KMS with S3 Bucket Keys) is the correct solution. S3 Bucket Keys are designed to reduce the cost of SSE-KMS. By enabling a bucket key, S3 reduces the request traffic to AWS KMS by generating a bucket-level key that is used to encrypt objects in the bucket. The bucket key reduces the calls to KMS to unwrap the data encryption key, which is a costly operation. It still uses KMS for encryption, fulfilling the encryption requirement, but optimizes cost by reducing KMS API calls.

Option C (Client-side encryption with KMS) moves the encryption process to the client side. While it uses KMS,

the client is responsible for encrypting the data before uploading it to S3. This does not optimize costs because the client must still make KMS calls to encrypt the data, and might increase complexity. It also does not leverage native S3 features for encryption at rest.

Option D (SSE-C with keys in KMS) is incorrect because SSE-C requires the user to provide their own encryption keys. Although the user suggests storing it in KMS, the solution requires the user to manage the keys. Amazon S3 does not store the encryption key you provide. Instead, Amazon S3 stores a keyed-hash message authentication code (HMAC) of the encryption key to validate future requests. This method will not reduce costs.

Therefore, SSE-KMS with S3 Bucket Keys is the best solution because it offers the required encryption with KMS and optimizes costs by reducing KMS calls.

Relevant documentation:

Using server-side encryption with AWS KMS keys (SSE-KMS)
Reducing the cost of SSE-KMS with S3 Bucket Keys

Question: 1010 **Exam Heist**

A company runs multiple workloads on virtual machines (VMs) in an on-premises data center. The company is expanding rapidly. The on-premises data center is not able to scale fast enough to meet business needs. The company wants to migrate the workloads to AWS.

The migration is time sensitive. The company wants to use a lift-and-shift strategy for non-critical workloads.

Which combination of steps will meet these requirements? (Choose three.)

A.Use the AWS Schema Conversion Tool (AWS SCT) to collect data about the VMs.
B.Use AWS Application Migration Service. Install the AWS Replication Agent on the VMs.
C.Complete the initial replication of the VMs. Launch test instances to perform acceptance tests on the VMs.
D.Stop all operations on the VMs. Launch a cutover instance.
E.Use AWS App2Container (A2C) to collect data about the VMs.
F.Use AWS Database Migration Service (AWS DMS) to migrate the VMs.

Answer: BCD

Explanation:

The chosen solution (BCD) effectively outlines a lift-and-shift migration strategy using AWS Application Migration Service (MGN) to meet the company's needs.

Here's why:

B. Use AWS Application Migration Service. Install the AWS Replication Agent on the VMs. AWS MGN is specifically designed for lift-and-shift migrations, and it replicates on-premises servers to AWS. Installing the agent on the VMs is the first step to initiating replication. https://aws.amazon.com/application-migration-service/

C. Complete the initial replication of the VMs. Launch test instances to perform acceptance tests on the VMs. After initial replication, it's crucial to test the migrated instances in AWS to ensure they function as expected. This verifies the migration process and allows for adjustments before the final cutover.

D. Stop all operations on the VMs. Launch a cutover instance. This step signifies the final migration stage. By stopping operations on the original VMs, the company minimizes data loss and ensures consistency when launching the cutover instances in AWS. This minimizes downtime.

Here's why the other options are incorrect:

A. Use the AWS Schema Conversion Tool (AWS SCT) to collect data about the VMs. AWS SCT is designed for database migrations, not VM migrations. Since the scenario describes a lift-and-shift of workloads on VMs, SCT is not applicable here.

E. Use AWS App2Container (A2C) to collect data about the VMs. AWS App2Container is suitable for modernizing applications by containerizing them. The company wants a lift-and-shift strategy, not containerization.

F. Use AWS Database Migration Service (AWS DMS) to migrate the VMs. AWS DMS is for database migration. The scenario involves the migration of whole VMs, rendering DMS an unsuitable tool for this purpose.

The chosen combination leverages the purpose-built AWS MGN service for efficient and rapid VM migration, aligned with a lift-and-shift approach. Testing and a controlled cutover ensures a smooth transition.

Question: 1011 **Exam Heist**

A company hosts an application in a private subnet. The company has already integrated the application with Amazon Cognito. The company uses an Amazon Cognito user pool to authenticate users.

The company needs to modify the application so the application can securely store user documents in an Amazon S3 bucket.

Which combination of steps will securely integrate Amazon S3 with the application? (Choose two.)

A.Create an Amazon Cognito identity pool to generate secure Amazon S3 access tokens for users when they successfully log in.

B.Use the existing Amazon Cognito user pool to generate Amazon S3 access tokens for users when they successfully log in.

C.Create an Amazon S3 VPC endpoint in the same VPC where the company hosts the application.

D.Create a NAT gateway in the VPC where the company hosts the application. Assign a policy to the S3 bucket to deny any request that is not initiated from Amazon Cognito.

E.Attach a policy to the S3 bucket that allows access only from the users' IP addresses.

Answer: AC

Explanation:

Here's a detailed justification for why options A and C are the correct choices, and why the others are incorrect:

Why A is correct: Create an Amazon Cognito identity pool to generate secure Amazon S3 access tokens for users when they successfully log in.

Amazon Cognito identity pools (now part of Cognito Federated Identities) are specifically designed to grant temporary AWS credentials to users, including those authenticated through Cognito user pools or other identity providers. When a user successfully authenticates, the application can use the Cognito identity pool to obtain temporary credentials with specific permissions defined in IAM roles. These credentials can then be used to access the S3 bucket securely. This approach avoids embedding long-term credentials in the application or directly using the user pool for S3 access, which is a security best practice.

Reference: https://docs.aws.amazon.com/cognito/latest/developerguide/identity-pools.html

Why C is correct: Create an Amazon S3 VPC endpoint in the same VPC where the company hosts the

application.

Since the application is in a private subnet with no direct internet access, it needs a way to communicate with S3 without traversing the public internet. An S3 VPC endpoint provides a private connection between the VPC and S3. Traffic to S3 from within the VPC will then travel over the AWS network and remain within the AWS infrastructure, enhancing security and reducing latency. Using a VPC endpoint ensures that the application can access S3 even without a NAT gateway or internet gateway.

Reference: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html

Why B is incorrect: Use the existing Amazon Cognito user pool to generate Amazon S3 access tokens for users when they successfully log in.

Cognito user pools are primarily for authentication and user management. While user pools can be integrated with identity pools to provide credentials for AWS resources, user pools themselves do not directly generate S3 access tokens. Identity pools are the AWS service component designed to fulfill this role.

Why D is incorrect: Create a NAT gateway in the VPC where the company hosts the application. Assign a policy to the S3 bucket to deny any request that is not initiated from Amazon Cognito.

While a NAT gateway allows instances in a private subnet to initiate outbound traffic to the internet (and therefore S3), it doesn't inherently provide a secure, role-based access control mechanism to S3 based on Cognito user identity. Option A, using an identity pool, is the correct way to leverage Cognito for generating secure S3 access credentials. The second part of option D (deny any request that is not initiated from Cognito) is impractical. S3 bucket policies cannot directly filter based on the originator of a request being "Amazon Cognito" itself. They grant or deny permissions based on the temporary credentials provided via the IAM roles associated with the Cognito identity pool.

Why E is incorrect: Attach a policy to the S3 bucket that allows access only from the users' IP addresses.

This approach is highly impractical and insecure for several reasons. User IP addresses can change frequently, making the policy difficult to maintain. More importantly, it's not reliable because the application is in a private subnet, so the source IP of the request seen by S3 would be the NAT gateway's public IP (if a NAT gateway were used) or the VPC endpoint's IP address. Finally, tying S3 access directly to user IP addresses doesn't provide a granular, role-based access control solution, which is what Cognito identity pools offer.

Question: 1012 **Exam Heist**

A company has a three-tier web application that processes orders from customers. The web tier consists of Amazon EC2 instances behind an Application Load Balancer. The processing tier consists of EC2 instances. The company decoupled the web tier and processing tier by using Amazon Simple Queue Service (Amazon SQS). The storage layer uses Amazon DynamoDB.

At peak times, some users report order processing delays and halls. The company has noticed that during these delays, the EC2 instances are running at 100% CPU usage, and the SQS queue fills up. The peak times are variable and unpredictable.

The company needs to improve the performance of the application.

Which solution will meet these requirements?

A.Use scheduled scaling for Amazon EC2 Auto Scaling to scale out the processing tier instances for the duration of peak usage times. Use the CPU Utilization metric to determine when to scale.

B.Use Amazon ElastiCache for Redis in front of the DynamoDB backend tier. Use target utilization as a metric to determine when to scale.

C.Add an Amazon CloudFront distribution to cache the responses for the web tier. Use HTTP latency as a metric to determine when to scale.

D.Use an Amazon EC2 Auto Scaling target tracking policy to scale out the processing tier instances. Use the ApproximateNumberOfMessages attribute to determine when to scale.

Answer: D

Explanation:

Here's a detailed justification for why option D is the best solution, along with supporting concepts and links:

The problem indicates the processing tier EC2 instances are overloaded during peak times, causing order processing delays and queue buildup in SQS. The key is to automatically scale the processing tier based on the workload queued in SQS.

Option D suggests using an EC2 Auto Scaling target tracking policy driven by the ApproximateNumberOfMessages attribute of the SQS queue. This is the most effective solution because it directly addresses the bottleneck. The ApproximateNumberOfMessages attribute reflects the number of messages waiting to be processed in the SQS queue. By configuring Auto Scaling to scale out the processing tier when this number exceeds a target value, the system proactively adds more processing capacity to handle the backlog. This ensures timely processing of orders and prevents the queue from filling up excessively. Target tracking policies automatically adjust the number of instances to maintain the desired metric value, simplifying scaling management.

Option A suggests scheduled scaling based on CPU utilization. While CPU utilization is a good indicator of load, scheduled scaling is reactive and relies on predictable patterns, which the problem states are unpredictable. Also scaling based on CPU after utilization reaches 100% will cause delay.

Option B introduces ElastiCache for Redis in front of DynamoDB. While caching can improve performance for read-heavy workloads, the bottleneck is in the processing tier, not the data retrieval from DynamoDB. Caching won't solve the CPU overload issue.

Option C adds CloudFront to the web tier. CloudFront is a Content Delivery Network (CDN) that caches static content closer to users. Since the problem describes order processing delays, caching static web assets won't address the root cause of the problem.

In summary, option D provides the most direct and adaptive solution by scaling the processing tier based on the actual workload pending in the SQS queue, mitigating the processing delays and preventing excessive queue buildup. It dynamically adjusts capacity to meet fluctuating demand.**Supporting Links:**

Amazon EC2 Auto Scaling Target Tracking Scaling Policies:
https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-target-tracking.html
Amazon SQS ApproximateNumberOfMessages Metric:
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-available-cloudwatch-metrics.html

Question: 1013 **Exam Heist**

A company's production environment consists of Amazon EC2 On-Demand Instances that run constantly between Monday and Saturday. The instances must run for only 12 hours on Sunday and cannot tolerate interruptions. The company wants to cost-optimize the production environment.

Which solution will meet these requirements MOST cost-effectively?

A.Purchase Scheduled Reserved Instances for the EC2 instances that run for only 12 hours on Sunday. Purchase Standard Reserved Instances for the EC2 instances that run constantly between Monday and

Saturday.

B.Purchase Convertible Reserved Instances for the EC2 instances that run for only 12 hours on Sunday. Purchase Standard Reserved Instances for the EC2 instances that run constantly between Monday and Saturday.

C.Use Spot Instances for the EC2 instances that run for only 12 hours on Sunday. Purchase Standard Reserved Instances for the EC2 instances that run constantly between Monday and Saturday.

D.Use Spot Instances for the EC2 instances that run for only 12 hours on Sunday. Purchase Convertible Reserved Instances for the EC2 instances that run constantly between Monday and Saturday.

Answer: A

Explanation:

Here's a detailed justification for why option A is the most cost-effective solution:

The core goal is to minimize costs while ensuring the production environment runs as required. The instances have two distinct usage patterns: consistent operation Monday through Saturday, and limited operation on Sunday.

For the Monday-Saturday usage, **Standard Reserved Instances** offer the best cost savings. They provide a significant discount compared to On-Demand pricing in exchange for a one- or three-year commitment. Because the workload runs constantly during these days, committing to Reserved Instances is beneficial.

For the 12-hour Sunday usage, **Scheduled Reserved Instances** are the most appropriate. These instances allow you to reserve capacity on a recurring schedule, perfectly matching the need for specific time windows each week. They guarantee capacity availability and come with a discount compared to On-Demand, but without requiring you to pay for the instance for the entire day.

Options C and D are less desirable because **Spot Instances** are prone to interruption. The question explicitly states that instances cannot tolerate interruptions. Therefore, relying on Spot Instances would violate a key requirement of the problem. While Spot Instances are generally the cheapest option for compute, the potential for interruption makes them unsuitable for this use case.

Option B is less ideal because **Convertible Reserved Instances**, while offering flexibility to change instance types and families, are more expensive than Standard Reserved Instances. For workloads with predictable resource requirements, as is the case for Monday-Saturday, the additional flexibility of Convertible Reserved Instances is unnecessary and introduces extra cost.

In summary, Option A strikes the best balance between cost optimization and reliability by using the most cost-effective reserved instance type for each distinct usage pattern. Scheduled Reserved Instances are used where availability is needed for only a defined time window and Standard Reserved Instances are used where there is continuous usage.

Here are some helpful links for further research:

Reserved Instances: https://aws.amazon.com/ec2/pricing/reserved-instances/
Scheduled Instances: https://aws.amazon.com/ec2/purchasing-options/scheduled-instances/
Spot Instances: https://aws.amazon.com/ec2/spot/

Question: 1014 **Exam Heist**

A digital image processing company wants to migrate its on-premises monolithic application to the AWS Cloud. The company processes thousands of images and generates large files as part of the processing workflow.

The company needs a solution to manage the growing number of image processing jobs. The solution must also reduce the manual tasks in the image processing workflow. The company does not want to manage the underlying infrastructure of the solution.

Which solution will meet these requirements with the LEAST operational overhead?

A.Use Amazon Elastic Container Service (Amazon ECS) with Amazon EC2 Spot Instances to process the images. Configure Amazon Simple Queue Service (Amazon SQS) to orchestrate the workflow. Store the processed files in Amazon Elastic File System (Amazon EFS).

B.Use AWS Batch jobs to process the images. Use AWS Step Functions to orchestrate the workflow. Store the processed files in an Amazon S3 bucket.

C.Use AWS Lambda functions and Amazon EC2 Spot Instances to process the images. Store the processed files in Amazon FSx.

D.Deploy a group of Amazon EC2 instances to process the images. Use AWS Step Functions to orchestrate the workflow. Store the processed files in an Amazon Elastic Block Store (Amazon EBS) volume.

Answer: B

Explanation:

The correct answer is B because it offers the least operational overhead and best aligns with the described requirements. Here's why:

AWS Batch eliminates the need to manage underlying compute infrastructure for image processing. Batch automatically provisions and scales compute resources based on job requirements. This is crucial as the company doesn't want to manage infrastructure. (https://aws.amazon.com/batch/)
AWS Step Functions provides a serverless orchestration service to manage the complex image processing workflow. Step Functions allows for defining workflows as state machines, simplifying coordination and monitoring of image processing steps without manual intervention. Step Functions supports error handling and retries. (https://aws.amazon.com/step-functions/)
Amazon S3 offers scalable, durable, and cost-effective object storage for storing the processed files. S3 is well-suited for large files generated by image processing. It eliminates the need for managing file servers or complex file systems. (https://aws.amazon.com/s3/)

Option A is less suitable because managing ECS clusters, even with Spot Instances, still incurs operational overhead related to container management and instance configuration. SQS provides messaging, but Step Functions provides orchestration capabilities specifically designed for defining and managing complex workflows. EFS, while a shared filesystem, isn't as cost-effective or scalable for storing a large number of files as S3.

Option C is less suitable because while Lambda is serverless, it may have limitations in terms of execution time and memory for large image processing tasks. It mentions EC2 Spot Instances together with Lambda functions which does not make sense. FSx introduces complexities in managing file systems.

Option D is least suitable because it requires managing EC2 instances, which contradicts the requirement of minimizing operational overhead. EBS is attached to a single EC2 instance so it is not ideal for storing a very large number of images.

Question: 1015 **Exam Heist**

A company's image-hosting website gives users around the world the ability to up load, view, and download images from their mobile devices. The company currently hosts the static website in an Amazon S3 bucket.

Because of the website's growing popularity, the website's performance has decreased. Users have reported latency issues when they upload and download images.

The company must improve the performance of the website.

Which solution will meet these requirements with the LEAST implementation effort?

A.Configure an Amazon CloudFront distribution for the S3 bucket to improve the download performance. Enable S3 Transfer Acceleration to improve the upload performance.

B.Configure Amazon EC2 instances of the right sizes in multiple AWS Regions. Migrate the application to the EC2 instances. Use an Application Load Balancer to distribute the website traffic equally among the EC2 instances. Configure AWS Global Accelerator to address global demand with low latency.

C.Configure an Amazon CloudFront distribution that uses the S3 bucket as an origin to improve the download performance. Configure the application to use CloudFront to upload images to improve the upload performance. Create S3 buckets in multiple AWS Regions. Configure replication rules for the buckets to replicate users' data based on the users' location. Redirect downloads to the S3 bucket that is closest to each user's location.

D.Configure AWS Global Accelerator for the S3 bucket to improve network performance. Create an endpoint for the application to use Global Accelerator instead of the S3 bucket.

Answer: A

Explanation:

Option A is the most efficient solution because it leverages existing AWS services optimized for content delivery and transfer acceleration with minimal configuration. Amazon CloudFront is a content delivery network (CDN) that caches static content like images in edge locations worldwide, significantly reducing latency for downloads. S3 Transfer Acceleration utilizes optimized network paths and the AWS backbone network to accelerate uploads to S3 buckets, improving performance for users regardless of their location.

Option B is overly complex and requires significant infrastructure management. Migrating the static website to EC2 instances and load balancing it across multiple regions involves setting up and maintaining servers, configuring application deployment, and managing networking, which adds operational overhead. AWS Global Accelerator, while beneficial, is an overkill for a simple static website hosted in S3.

Option C introduces unnecessary complexity and cost. Creating S3 buckets in multiple regions and configuring replication rules adds administrative overhead and cost. The manual redirection of downloads to the closest S3 bucket is complex to implement and maintain. CloudFront already provides global content delivery and caching automatically.

Option D's configuration is not the intended use case for AWS Global Accelerator. Global Accelerator is designed for dynamic applications, like gaming applications or streaming media, that need high availability and consistent performance. S3 Transfer Acceleration addresses the upload performance improvement needed more effectively.

Therefore, Option A provides the best balance of performance improvement, ease of implementation, and cost-effectiveness by using existing AWS services specialized for content delivery and transfer acceleration.

Relevant Documentation:

Amazon CloudFront: https://aws.amazon.com/cloudfront/
Amazon S3 Transfer Acceleration: https://aws.amazon.com/s3/transfer-acceleration/
AWS Global Accelerator: https://aws.amazon.com/global-accelerator/

Question: 1016 **Exam Heist**

A company runs an application in a private subnet behind an Application Load Balancer (ALB) in a VPC. The VPC has a NAT gateway and an internet gateway. The application calls the Amazon S3 API to store objects.

According to the company's security policy, traffic from the application must not travel across the internet.

Which solution will meet these requirements MOST cost-effectively?

A.Configure an S3 interface endpoint. Create a security group that allows outbound traffic to Amazon S3.

B.Configure an S3 gateway endpoint. Update the VPC route table to use the endpoint.

C.Configure an S3 bucket policy to allow traffic from the Elastic IP address that is assigned to the NAT gateway.

D.Create a second NAT gateway in the same subnet where the legacy application is deployed. Update the VPC route table to use the second NAT gateway.

Answer: B

Explanation:

The most cost-effective solution to ensure traffic from the application to Amazon S3 doesn't traverse the internet while adhering to the security policy is option B: configuring an S3 gateway endpoint and updating the VPC route table.

Here's why:

Gateway Endpoints: S3 gateway endpoints are designed to provide private connectivity to S3 within your VPC. They are free to use and do not require an internet gateway, NAT device, or VPN connection to access S3. This aligns directly with the requirement of not traversing the internet.

Cost-Effectiveness: Gateway endpoints do not incur data processing or hourly charges, making them the most cost-effective option for private S3 access. This is because the data transfer within the AWS network is already paid for and gateway endpoints are free of charge.

Route Table Integration: Gateway endpoints work by adding a route to your VPC route table. This route directs traffic destined for S3 to the gateway endpoint, ensuring traffic stays within the AWS network.

Other Options:

Option A (Interface Endpoint): While interface endpoints also provide private connectivity, they use AWS PrivateLink and incur hourly and data processing charges, making them less cost-effective than gateway endpoints for S3.
Option C (Bucket Policy with NAT Gateway IP): This approach involves allowing traffic from the NAT gateway's Elastic IP to the S3 bucket. While it avoids internet traversal, it's less secure and manageable. If the NAT gateway is replaced, the IP changes, potentially disrupting access. Also, bucket policies can become complex to manage at scale.
Option D (Second NAT Gateway): Creating a second NAT gateway increases costs and doesn't address the core requirement of keeping S3 traffic within the AWS network. It's primarily for redundancy, not for private S3 access.

In summary, using an S3 gateway endpoint offers the optimal balance of security, cost-effectiveness, and ease of management for accessing S3 privately from within a VPC.

Authoritative Links:

AWS VPC Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html
S3 Gateway Endpoints: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-s3.html

Question: 1017 **Exam Heist**

A company has an application that runs on an Amazon Elastic Kubernetes Service (Amazon EKS) cluster on Amazon EC2 instances. The application has a UI that uses Amazon DynamoDB and data services that use Amazon S3 as part of the application deployment.

The company must ensure that the EKS Pods for the UI can access only Amazon DynamoDB and that the EKS Pods for the data services can access only Amazon S3. The company uses AWS Identity and Access Management (IAM).

Which solution meals these requirements?

A.Create separate IAM policies for Amazon S3 and DynamoDB access with the required permissions. Attach both IAM policies to the EC2 instance profile. Use role-based access control (RBAC) to control access to Amazon S3 or DynamoDB for the respective EKS Pods.

B.Create separate IAM policies for Amazon S3 and DynamoDB access with the required permissions. Attach the Amazon S3 IAM policy directly to the EKS Pods for the data services and the DynamoDB policy to the EKS Pods for the UI.

C.Create separate Kubernetes service accounts for the UI and data services to assume an IAM role. Attach the AmazonS3FullAccess policy to the data services account and the AmazonDynamoDBFullAccess policy to the UI service account.

D.Create separate Kubernetes service accounts for the UI and data services to assume an IAM role. Use IAM Role for Service Accounts (IRSA) to provide access to the EKS Pods for the UI to Amazon S3 and the EKS Pods for the data services to DynamoDB.

Answer: D

Explanation:

The correct answer is **D**. Here's why:

The requirement is to provide fine-grained access control for EKS pods to AWS services (DynamoDB and S3). Pods running the UI should only access DynamoDB, and pods running data services should only access S3. This necessitates a mechanism that allows granting IAM permissions to pods specifically, not just the underlying EC2 instances.

Option A is incorrect because attaching IAM policies to the EC2 instance profile grants access to all pods running on that instance. RBAC is for Kubernetes API access, not AWS resource access. Therefore, RBAC can't be used to restrict a pod's access to DynamoDB or S3 at the IAM level.

Option B is incorrect because it is not possible to attach IAM policies directly to Kubernetes pods. IAM policies are associated with IAM roles, which are assumed by entities like EC2 instances or, crucially, Kubernetes service accounts.

Option C is incorrect because it attaches AmazonS3FullAccess and AmazonDynamoDBFullAccess to the service accounts directly. However, the key problem is that it attempts to grant S3 access to the UI service account and DynamoDB access to the data services account, the opposite of what's needed. More importantly, without IRSA, the service accounts will not have the necessary permissions to assume an IAM role, hence access won't be effective.

Option D, using IAM Roles for Service Accounts (IRSA), is the correct approach. IRSA allows you to associate IAM roles with Kubernetes service accounts. You create separate service accounts for the UI and data services pods. Each service account is then configured to assume a specific IAM role. The IAM role for the UI service account has a policy allowing access only to DynamoDB. The IAM role for the data services service account has a policy allowing access only to S3. When the pods are launched and associated with their respective service accounts, they automatically inherit the permissions defined in the associated IAM roles.

Here's a breakdown of the IRSA process:

1. **Create IAM roles:** Two roles are created, one for UI pods (DynamoDB access) and one for data services pods (S3 access). Each role includes a trust policy that allows the Kubernetes service account to assume the role.
2. **Create Kubernetes service accounts:** Two service accounts are created, one for UI pods and one for data services pods.
3. **Associate IAM roles with service accounts:** The service accounts are annotated to specify which IAM role they can assume. This annotation securely links the Kubernetes service account to the AWS IAM role.

4. **Deploy pods:** The UI pods are configured to use the UI service account, and the data services pods are configured to use the data services service account.

This setup ensures that the UI pods can only access DynamoDB and the data services pods can only access S3, fulfilling the company's requirements.

References:

IAM roles for service accounts - Amazon EKS
Use IAM roles for service accounts

Question: 1018 **Exam Heist**

A company needs to give a globally distributed development team secure access to the company's AWS resources in a way that complies with security policies.

The company currently uses an on-premises Active Directory for internal authentication. The company uses AWS Organizations to manage multiple AWS accounts that support multiple projects.

The company needs a solution to integrate with the existing infrastructure to provide centralized identity management and access control.

Which solution will meet these requirements with the LEAST operational overhead?

A.Set up AWS Directory Service to create an AWS managed Microsoft Active Directory on AWS. Establish a trust relationship with the on-premises Active Directory. Use IAM rotes that are assigned to Active Directory groups to access AWS resources within the company's AWS accounts.

B.Create an IAM user for each developer. Manually manage permissions for each IAM user based on each user's involvement with each project. Enforce multi-factor authentication (MFA) as an additional layer of security.

C.Use AD Connector in AWS Directory Service to connect to the on-premises Active Directory. Integrate AD Connector with AWS IAM Identity Center. Configure permissions sets to give each AD group access to specific AWS accounts and resources.

D.Use Amazon Cognito to deploy an identity federation solution. Integrate the identity federation solution with the on-premises Active Directory. Use Amazon Cognito to provide access tokens for developers to access AWS accounts and resources.

Answer: C

Explanation:

The best solution is C, using AD Connector with IAM Identity Center (successor to AWS SSO). Here's why:

Centralized Identity Management: AD Connector allows you to leverage your existing on-premises Active Directory as the identity provider, achieving centralized identity management without migrating your directory to AWS.

Integration with AWS Accounts: IAM Identity Center simplifies managing access across multiple AWS accounts within your organization. This aligns with the company's use of AWS Organizations.

Permissions Sets: IAM Identity Center allows you to define permission sets, which are collections of IAM policies, and assign them to AD groups. This provides granular access control to AWS resources based on group membership in Active Directory.

Least Operational Overhead: Compared to alternatives, this solution minimizes the need to create and manage individual IAM users. It leverages existing AD groups, significantly reducing the operational burden. Creating a managed AD (option A) involves a significant operational overhead for managing and maintaining an entire directory service. IAM users (option B) are impractical for a large, dynamic team requiring frequent permission updates, increasing overhead. Cognito (option D) introduces complexity with identity federation

and access token management, adding overhead.

Security Compliance: AD Connector uses secure tunnels to connect to your on-premises Active Directory, while IAM Identity Center uses secure protocols for authentication and authorization. This helps meet security policies.

Therefore, AD Connector with IAM Identity Center streamlines authentication and authorization, centrally manages access across AWS accounts, minimizes operational overhead, and complies with security policies.

Here are some links for further research:

AWS IAM Identity Center
AWS Directory Service
AD Connector

Question: 1019 **Exam Heist**

A company is developing an application in the AWS Cloud. The application's HTTP API contains critical information that is published in Amazon API Gateway. The critical information must be accessible from only a limited set of trusted IP addresses that belong to the company's internal network.

Which solution will meet these requirements?

A.Set up an API Gateway private integration to restrict access to a predefined set of IP addresses.

B.Create a resource policy for the API that denies access to any IP address that is not specifically allowed.

C.Directly deploy the API in a private subnet. Create a network ACL. Set up rules to allow the traffic from specific IP addresses.

D.Modify the security group that is attached to API Gateway to allow inbound traffic from only the trusted IP addresses.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer and why the others are not, along with supporting concepts and links:

Justification for Option B (Create a resource policy for the API that denies access to any IP address that is not specifically allowed):

API Gateway resource policies are specifically designed to control access to your APIs based on various factors, including source IP addresses. This approach provides fine-grained control at the API level, making it ideal for restricting access to a defined set of trusted IP addresses. By creating a resource policy that explicitly denies access to all IP addresses except those on the allowed list, you create a robust security measure directly tied to the API itself. This mechanism works independently of your VPC configuration or network ACLs. This means even if your VPC configuration is somehow misconfigured, the API Gateway policy will still enforce the IP-based restriction.

Why other options are incorrect:

A. Set up an API Gateway private integration to restrict access to a predefined set of IP addresses: Private integrations primarily focus on routing requests within your VPC. While you can control access within your VPC, it doesn't directly address restricting public internet access based on source IP addresses. A private integration routes requests to a VPC endpoint, but the endpoint itself needs further restrictions to ensure that only the trusted IP addresses can access the API Gateway, and that's done through other mechanisms, making this option less direct than option B.

C. Directly deploy the API in a private subnet. Create a network ACL. Set up rules to allow the traffic from specific IP addresses: API Gateway is a managed service that doesn't directly "deploy" into a private subnet in the way that an EC2 instance would. Even if it did, network ACLs (NACLs) offer basic stateless traffic filtering at the subnet level. While NACLs can filter based on IP address, they are not as granular or easily manageable as API Gateway resource policies for controlling API access. Furthermore, deploying API in a private subnet alone doesn't inherently restrict access from the internet. You would require additional networking configuration (like NAT Gateway, VPC Endpoints) to make it work, increasing complexity.

D. Modify the security group that is attached to API Gateway to allow inbound traffic from only the trusted IP addresses: API Gateway doesn't have a security group that you directly modify. API Gateway is a fully managed service, and its underlying infrastructure is managed by AWS. You don't have direct access to its security groups.

Authoritative Links for further research:

API Gateway Resource Policies:
https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-control-access-using-iam-policies-to-invoke-api.html
API Gateway Private Integrations:
https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-private-apis.html
Network ACLs: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

If you have any feedback or thoughts on the bumps, I would love to hear them.
Your insights can help me improve our writing and better understand our readers.

Best of Luck

You have worked hard to get to this point, and you are well-prepared for the exam
Keep your head up, stay positive, and go show that exam what you're made of!

Feedback

More Papers

Future is Secured

100% Pass Guarantee

24/7 Customer Support

Mail us - support@examheist.com

Verified Updates

Lifetime Updates!

Total: **1019 Questions**

Link: https://examheist.com/papers/amazon/aws-certified-solutions-architect-associate-saa-c03